GNU Emacs 手册

GNU Emacs Manual · 中文版

Emacs 31.1.50 · 2026 年 11 月

关于本译本

本译本对应 GNU Emacs Manual（Emacs 31.1.50，2026 年），原手册以 GNU Free Documentation License 发布；中文译本完成于 2026 年 11 月。

- **翻译原则**：逐章对应原手册，完整翻译、不增删技术内容；命令、函数、变量、按键、文件名与包名保留英文，便于直接查阅 `C-h f` / `C-h v` / `C-h k`。
- **交叉引用**：正文中的「参见“标题”」指原手册的对应小节，可在本 PDF 中检索该标题；PDF 书签按章、节、小节分级。
- **代码块**：`elisp` 代码块可直接复制进初始化文件。
- **索引**：原手册末尾的按键索引、命令/函数索引、变量索引与概念索引是按页码编排的指针，未收录；请用阅读器检索替代。
- **许可**：见附录 A、B；译文仅供学习参考，如有歧义以英文原版为准。

按键约定

写法	含义
`C-x`	按住 Ctrl 再按 x
`M-x`	按住 Meta（Alt 或 Option）再按 x
`S-x`	按住 Shift 再按 x
`C-M-x`	Ctrl 与 Meta 同时按住再按 x
`<RET>`	回车键
`<TAB>`	Tab 键
`<ESC>`	Esc 键
`<F1>`	功能键 F1
`C-x C-f`	依次按 `C-x`、`C-f`

如何阅读本手册

按任务查阅即可，不必通读：

1. 第一次配置：第 51 章《自定义》、第 50 章《Emacs Lisp 软件包》、附录 J《常用配置速查》。
2. 键位与模式：第 3、5 章，第 25 章《主模式与次模式》。
3. 文件与缓冲区：第 20、21、22 章。
4. 显示与界面：第 16、23 章。
5. 搜索替换：第 17 章；Dired：第 32 章；Shell：第 40 章。
6. 出问题：第 52 章《退出与中止》、第 53 章《故障处理》。

目录

GNU Emacs 手册

关于本译本

- 按键约定
- 如何阅读本手册

发行

简介

1 屏幕布局

- 1.1 光标点
- 1.2 回显区
- 1.3 模式行
- 1.4 菜单栏

2 输入类型

3 按键

4 鼠标输入

5 按键与命令

6 触摸屏输入与虚拟键盘

- 6.1 在触摸屏上使用 Emacs
- 6.2 配合虚拟键盘使用 Emacs

7 启动 Emacs

8 退出 Emacs

9 基本编辑命令

- 9.1 插入文本
- 9.2 移动光标点的位置
- 9.3 删除文本
- 9.4 撤销修改
- 9.5 文件
- 9.6 帮助
- 9.7 空行
- 9.8 续行
- 9.9 光标位置信息
- 9.10 数字参数

9.11 重复执行命令

10 迷你缓冲区

10.1 使用迷你缓冲区

10.2 用于文件名的迷你缓冲区

10.3 在迷你缓冲区中编辑

10.4 补全

10.4.1 补全示例

10.4.2 补全命令

10.4.3 补全退出

10.4.4 补全候选项如何选定

10.4.5 补全选项

10.5 迷你缓冲区历史

10.6 重复迷你缓冲区命令

10.7 输入密码

10.8 是或否提示

11 按名称执行命令

12 帮助

12.1 帮助摘要

12.2 按键的文档

12.3 按命令或变量名获取帮助

12.4 Apropos

12.5 帮助模式命令

12.6 包的关键字搜索

12.7 国际语言支持的帮助

12.8 其他帮助命令

12.9 帮助文件

12.10 活动文本与工具提示的帮助

13 标记与区域

13.1 设置标记

13.2 标记文本对象的命令

13.3 对区域进行操作

13.4 标记环

13.5 全局标记环

13.6 Shift 选择

13.7 禁用临时标记模式

14 删除与移动文本

14.1 删除与删除环
14.1.1 基本删除
14.1.2 按行删除
14.1.3 其他删除命令
14.1.4 删除选项
14.2 粘贴
14.2.1 删除环
14.2.2 粘贴更早删除的内容
14.2.3 追加删除
14.3 图形显示环境中的“剪切与粘贴”操作
14.3.1 使用剪贴板
14.3.2 与其他窗口应用剪贴
14.3.3 辅助选区
14.4 累积文本
14.5 矩形
14.6 CUA 绑定

15 寄存器

15.1 在寄存器中保存位置
15.2 在寄存器中保存文本
15.3 在寄存器中保存矩形
15.4 在寄存器中保存窗口和框架配置
15.5 在寄存器中保存数字
15.6 在寄存器中保存文件名和缓冲区名
15.7 键盘宏寄存器
15.8 书签

16 控制显示

16.1 滚动
16.2 重新居中
16.3 自动滚动
16.4 水平滚动
16.5 窄化
16.6 视图模式
16.7 跟随模式
16.8 文本字体
16.9 字体的颜色
16.9.1 颜色名称
16.9.2 RGB 三元组
16.10 标准字体

16.11 图标
16.12 文本缩放
16.13 Font Lock 模式
16.13.1 传统 Font Lock
16.13.2 基于解析器的 Font Lock
16.13.3 语义 Font Lock
16.14 交互式高亮
16.15 窗口边缘
16.16 显示边界
16.17 无用的空白
16.18 选择性显示
16.19 可选的模式行功能
16.20 文本的显示
16.21 光标的显示
16.22 行截断
16.23 可视行模式
16.24 显示的自定义

17 搜索与替换

17.1 增量搜索
17.1.1 增量搜索基础
17.1.2 重复增量搜索
17.1.3 Isearch 的粘贴
17.1.4 增量搜索中的错误
17.1.5 增量搜索的特殊输入
17.1.6 不退出增量搜索
17.1.7 搜索迷你缓冲区
17.2 非增量搜索
17.3 单词搜索
17.4 符号搜索
17.5 正则表达式搜索
17.6 正则表达式的语法
17.7 正则表达式中的反斜杠
17.8 正则表达式示例
17.9 搜索时的宽松匹配
17.10 替换命令
17.10.1 无条件替换
17.10.2 正则表达式替换
17.10.3 替换命令与宽松匹配

17.10.4 查询替换
17.11 其他搜索与循环命令
17.12 定制搜索以满足需要

18 纠错命令

18.1 撤销
18.2 调换文本
18.3 大小写转换
18.4 拼写检查与纠正

19 键盘宏

19.1 基本用法
19.2 键盘宏环
19.3 键盘宏计数器
19.4 带变化地执行宏
19.5 命名并保存键盘宏
19.6 编辑键盘宏
19.7 逐步编辑键盘宏
19.8 列出与编辑键盘宏

20 文件处理

20.1 文件名
20.2 访问文件
20.3 保存文件
20.3.1 保存文件的命令
20.3.2 备份文件
20.3.3 自定义文件的保存
20.3.4 防止同时编辑
20.3.5 影子文件
20.3.6 自动更新时间戳
20.4 还原缓冲区
20.5 自动还原：自动保持缓冲区最新
20.5.1 自动还原非文件缓冲区
20.6 自动保存：灾难防护
20.6.1 自动保存文件
20.6.2 控制自动保存
20.6.3 从自动保存恢复数据
20.7 文件名别名
20.8 文件目录
20.9 比较文件

20.10 Diff 模式
20.11 复制、命名和重命名文件
20.12 其他文件操作
20.13 访问压缩文件
20.14 文件归档
20.15 远程文件
20.16 引用的文件名
20.17 文件名缓存
20.18 查找文件的便利功能
20.19 查看图像文件
20.20 文件集

21 多缓冲区

21.1 创建与选择缓冲区
21.2 列出已有缓冲区
21.3 其他缓冲区操作
21.4 删除缓冲区
21.5 对多个缓冲区操作
21.6 间接缓冲区
21.7 缓冲区处理的便利功能与定制
21.7.1 使缓冲区名称唯一
21.7.2 快速迷你缓冲区选择
21.7.3 定制缓冲区菜单

22 多窗口

22.1 Emacs 窗口的概念
22.2 拆分窗口
22.3 使用其他窗口
22.4 在另一个窗口中显示
22.5 删除与调整窗口大小
22.6 在窗口中显示缓冲区
22.6.1 'display-buffer' 的工作方式
22.6.2 显示不可编辑的缓冲区
22.7 窗口处理的便利功能
22.8 窗口标签行
22.9 窗口工具栏

23 框架与图形显示

23.1 鼠标编辑命令
23.2 面向单词和行的鼠标命令

23.3 用鼠标跟随引用
23.4 打开菜单的鼠标点击
23.5 模式行鼠标命令
23.6 创建框架
23.7 框架命令
23.8 字体
23.9 Speedbar 框架
23.10 多个 X 显示
23.11 框架参数
23.12 滚动条
23.13 窗口分隔条
23.14 拖放
23.15 菜单栏
23.16 工具栏
23.17 标签栏
23.18 使用系统 GUI 任务栏功能
23.19 使用对话框
23.20 工具提示
23.21 鼠标回避
23.22 文本终端
23.23 在文本终端中使用鼠标

24 国际字符集支持

24.1 国际字符集简介
24.2 语言环境
24.3 输入法
24.4 选择输入法
24.5 编码系统
24.6 识别编码系统
24.7 指定文件的编码系统
24.8 为输出选择编码系统
24.9 为文件文本指定编码系统
24.10 用于进程间通信的编码系统
24.11 文件名的编码系统
24.12 X 键盘输入的编码系统
24.13 终端 I/O 的编码系统
24.14 字体集
24.15 定义字体集

24.16 修改字体集
24.17 无法显示的字符
24.18 单字节编辑模式
24.19 字符集
24.20 双向编辑

25 主模式与次模式

25.1 主模式
25.2 次模式
25.3 选择文件模式

26 缩进

26.1 缩进命令
26.2 制表位
26.3 制表符与空格
26.4 缩进的便利功能
26.5 代码对齐

27 人类语言命令

27.1 单词
27.2 句子
27.3 段落
27.4 页
27.5 引号
27.6 填充文本
27.6.1 自动填充模式
27.6.2 显式填充命令
27.6.3 填充前缀
27.6.4 自适应填充
27.7 大小写转换命令
27.8 文本模式
27.9 大纲模式
27.9.1 大纲次模式
27.9.2 大纲的格式
27.9.3 大纲移动命令
27.9.4 大纲可见性命令
27.9.5 在多个视图中查看单个大纲
27.9.6 折叠编辑
27.10 Org 模式
27.10.1 用 Org 做组织工具

27.10.2 用 Org 做写作系统
27.11 TeX 模式
27.11.1 TeX 编辑命令
27.11.2 LaTeX 编辑命令
27.11.3 TeX 打印命令
27.11.4 TeX 模式杂项
27.12 SGML 与 HTML 模式
27.13 Nroff 模式
27.14 富文本
27.14.1 富文本模式
27.14.2 硬换行与软换行
27.14.3 编辑格式信息
27.14.4 富文本中的字体
27.14.5 富文本中的缩进
27.14.6 富文本中的对齐
27.14.7 设置其他文本属性
27.15 编辑基于文本的表格
27.15.1 什么是基于文本的表格?
27.15.2 创建表格
27.15.3 表格识别
27.15.4 表格单元格命令
27.15.5 单元格对齐
27.15.6 表格的行与列
27.15.7 纯文本与表格之间的转换
27.15.8 表格杂项
27.16 双栏编辑
28 编辑程序
28.1 编程语言的主模式
28.2 顶层定义（defun）
28.2.1 左边界约定
28.2.2 按 defun 移动
28.2.3 按句子移动
28.2.4 Imenu
28.2.5 Which Function 模式
28.3 程序的缩进
28.3.1 基本的程序缩进命令
28.3.2 缩进多行
28.3.3 定制 Lisp 缩进
28.3.4 C 缩进命令

28.3.5 自定义 C 缩进
28.4 编辑括号的命令
28.4.1 平衡括号表达式
28.4.2 在括号结构中移动
28.4.3 匹配括号
28.5 操作注释
28.5.1 注释命令
28.5.2 多行注释
28.5.3 控制注释的选项
28.6 文档查询
28.6.1 Info 文档查询
28.6.2 man 手册页查询
28.6.3 编程语言文档查询
28.7 Hideshow 次模式
28.8 符号名补全
28.9 混合大小写单词
28.10 Semantic
28.11 编辑程序的其他有用功能
28.12 C 及相关模式
28.12.1 C 模式的移动命令
28.12.2 C 模式中的电击式字符
28.12.3 C 模式中的饥饿删除功能
28.12.4 C 模式的其他命令
28.13 Asm 模式
28.14 Fortran 模式
28.14.1 移动命令
28.14.2 Fortran 缩进
28.14.3 Fortran 注释
28.14.4 Fortran 模式下的自动填充
28.14.5 检查 Fortran 中的列
28.14.6 Fortran 关键字缩写
29 编译与测试程序
29.1 在 Emacs 中运行编译
29.2 编译模式
29.3 供编译使用的子 shell
29.4 在 Emacs 中用 Grep 搜索
29.5 即时查找语法错误
29.6 在 Emacs 中运行调试器

29.6.1 启动 GUD
29.6.2 调试器操作
29.6.3 GUD 的命令
29.6.4 GUD 定制
29.6.5 GDB 图形界面
29.7 执行 Lisp 表达式
29.8 Emacs 的 Lisp 代码库
29.9 求值 Emacs Lisp 表达式
29.10 Lisp 交互缓冲区
29.11 运行外部 Lisp

30 维护大型项目

30.1 版本控制
30.1.1 版本控制简介
30.1.2 版本控制与模式行
30.1.3 版本控制下的基本编辑
30.1.8 撤销版本控制操作
30.1.9 忽略版本控制文件
30.1.10 VC 目录模式
30.1.11 版本控制分支
30.1.12 VC 的其他命令与特性
30.1.13 定制 VC
30.1.13.3 CVS 专有选项
30.2 使用项目
30.2.1 操作文件的项目命令
30.2.2 操作缓冲区的项目命令
30.2.3 切换项目
30.2.4 管理项目列表文件
30.2.5 感知 VC 的项目后端
30.3 变更日志
30.3.1 变更日志命令
30.3.2 ChangeLog 的格式
30.4 查找标识符引用
30.4.1 查找标识符
30.4.1.1 查找标识符定义
30.4.1.2 '*xref*' 缓冲区中可用的命令
30.4.1.3 用标识符搜索与替换
30.4.1.4 标识符查询
30.4.2 标签表
30.4.2.1 源文件标签语法

30.4.3 选择标签表
30.5 Emacs 开发环境
30.6 用 Emerge 合并文件
30.6.1 Emerge 概述
30.6.2 Emerge 的子模式
30.6.3 差异的状态
30.6.4 合并命令
30.6.5 退出 Emerge
30.6.6 组合两个版本
30.6.7 Emerge 的细节
30.7 缺陷引用
自动设置
为第三方包添加支持
与 debbugs 包的集成

31 缩写

31.1 缩写概念
31.2 定义缩写
31.3 控制缩写展开
31.4 缩写建议
31.5 查看与编辑缩写
31.6 保存缩写
31.7 动态缩写展开
31.8 定制动态缩写

32 Dired 目录编辑器

32.1 进入 Dired
32.2 在 Dired 缓冲区中导航
32.3 用 Dired 删除文件
32.4 一次为多个文件加旗标
32.5 在 Dired 中访问文件
32.6 Dired 标记与旗标之别
32.7 对文件进行操作
32.8 Dired 中的 shell 命令
32.9 猜测 shell 命令
32.10 在 Dired 中转换文件名
32.11 用 Dired 比较文件
32.12 Dired 中的子目录
32.13 Dired 中的子目录开关
32.14 在子目录间移动

32.15 隐藏子目录
32.16 更新 Dired 缓冲区
32.17 Dired 与 'find'
32.18 编辑 Dired 缓冲区
32.19 在 Dired 中查看图像缩略图
32.20 Dired 的其他功能

33 日历与日记

33.1 日历中的移动
33.1.1 按标准时间长度移动
33.1.2 周、月或年的开始与结束
33.1.3 指定日期
33.2 日历中的滚动
33.3 计算天数
33.4 其他日历命令
33.5 写出日历文件
33.6 节假日
33.7 日出与日落时间
33.8 月相
33.9 与其他历法的相互转换
33.9.1 支持的历法系统
33.9.2 转换到其他历法
33.9.3 从其他历法转换
33.10 日记
33.10.1 日记文件
33.10.2 显示日记
33.10.3 日期格式
33.10.4 向日记添加条目的命令
33.10.5 特殊日记条目
33.10.6 约会提醒
33.10.7 日记条目的导入与导出
33.11 夏令时
33.12 时间区间累计
33.13 日历与日记的更多高级特性
33.13.1 定制日历
33.13.2 定制节日
33.13.3 从玛雅历转换
33.13.4 日期显示格式
33.13.5 时间显示格式

33.13.6 定制日记
33.13.7 使用非公历的日记条目
33.13.8 日记显示
33.13.9 花式日记显示
33.13.10 S 表达式条目与花式日记显示

34 发送邮件

34.1 邮件缓冲区的格式
34.2 邮件头字段
34.3 邮件别名
34.4 邮件命令
34.4.1 发送邮件
34.4.2 编辑邮件头
34.4.3 引用邮件
34.4.4 邮件杂项
34.5 邮件签名
34.6 邮件娱乐
34.7 邮件撰写方法

35 用 Rmail 读信

35.1 Rmail 基本概念
35.2 在消息内滚动
35.3 在消息之间移动
35.4 删除消息
35.5 Rmail 文件与收件箱
35.6 多个 Rmail 文件
35.7 把消息复制到文件
35.8 标签
35.9 Rmail 属性
35.10 发送回复
35.11 摘要
35.11.1 创建摘要
35.11.2 在摘要中编辑
35.12 排序 Rmail 文件
35.13 邮件的显示
35.14 Rmail 与编码系统
35.15 在邮件内部编辑
35.16 摘要邮件
35.17 阅读 Rot13 邮件
35.18 ‘movemail’ 程序

35.19 从远程邮箱收取邮件

35.20 从各种格式的本地邮箱收取邮件

36 用 Gnus 收发邮件与新闻

36.1 Gnus 缓冲区

36.2 Gnus 启动时

36.3 使用 Gnus 组缓冲区

36.4 使用 Gnus 摘要缓冲区

37 主机安全

38 网络安全

39 文档查看

39.1 DocView 导航

39.2 DocView 搜索

39.3 DocView 切片

39.4 DocView 转换

40 在 Emacs 中运行 Shell 命令

40.1 单条 Shell 命令

40.2 交互式子 shell

40.3 Shell 模式

40.4 Shell 提示符

40.5 Shell 命令历史

40.5.1 Shell 历史环

40.5.2 Shell 历史复制

40.5.3 Shell 历史引用

40.6 目录跟踪

40.7 Shell 书签

40.8 Shell 模式选项

40.9 Emacs 终端模拟器

40.10 Term 模式

40.11 远程主机 Shell

40.12 串口终端

41 把 Emacs 用作服务器

41.1 TCP Emacs 服务器

41.2 调用 'emacsclient'

41.3 'emacsclient' 选项

42 打印

42.1 PostScript 硬拷贝
42.2 PostScript 硬拷贝的变量
42.3 Printing 包

43 文本排序

44 编辑图片

44.1 图片模式中的基本编辑
44.2 控制插入后的移动
44.3 图片模式中的制表符
44.4 图片模式矩形命令

45 编辑二进制文件

46 保存 Emacs 会话

47 递归编辑层级

48 超链接与网页浏览

48.1 使用 EWW 浏览网页
48.2 内嵌的 WebKit 部件
48.3 跟随 URL
48.4 激活 URL
48.5 在光标点处查找文件和 URL

49 游戏与娱乐

50 Emacs Lisp 软件包

50.1 包菜单缓冲区
50.2 包状态
50.3 包的安装
50.4 包文件与目录布局
50.5 获取包源码
50.5.1 指定包源

51 自定义

51.1 简易自定义界面
51.1.1 自定义组
51.1.2 浏览与搜索设置
51.1.3 修改变量
51.1.4 保存自定义
51.1.5 自定义字体
51.1.6 自定义特定项目
51.1.7 自定义主题

51.1.8 创建自定义主题
51.2 变量
51.2.1 查看与设置变量
51.2.2 钩子
51.2.3 局部变量
51.2.4 文件中的局部变量
51.2.5 按目录的局部变量
51.2.6 按连接的局部变量
51.3 定制键绑定
51.3.1 键位图
51.3.2 前缀键位图
51.3.3 局部键位图
51.3.4 迷你缓冲区键位图
51.3.5 交互式更改键绑定
51.3.6 在初始化文件中重绑定按键
51.3.7 修饰键
51.3.8 重绑定功能键
51.3.9 命名的 ASCII 控制字符
51.3.10 重绑定鼠标按钮
51.3.11 禁用命令
51.4 Emacs 初始化文件
51.4.1 初始化文件语法
51.4.2 初始化文件示例
51.4.3 终端专用的初始化
51.4.4 Emacs 如何查找你的初始化文件
51.4.5 初始化文件中的非 ASCII 字符
51.4.6 早期初始化文件
51.4.7 用户 Lisp 目录
51.5 保存持久认证信息
51.6 新手主题

52 退出与中止

53 故障处理

53.1 递归编辑层级
53.2 屏幕乱码
53.3 文本中出现乱码
53.4 内存耗尽
53.5 Emacs 崩溃时
53.6 崩溃后的恢复

53.7 紧急退出

53.8 若 无法删除

54 报告 Bug

54.1 阅读已有的 bug 报告和已知问题

54.2 何时算有 bug

54.3 理解 bug 报告

54.4 bug 报告核对清单

54.5 为 GNU Emacs 发送补丁

55 参与 Emacs 开发

55.1 编码规范

55.2 版权转让

56 获取 GNU Emacs 帮助

附录 A GNU 通用公共许可证

附录 B GNU 自由文档许可证

附录 C 启动 Emacs 的命令行参数

C.1 动作参数

C.2 初始选项

C.3 命令参数示例

C.4 环境变量

C.4.1 通用变量

C.4.2 其他变量

C.4.3 MS-Windows 系统注册表

C.5 指定显示名称

C.6 字体指定选项

C.7 窗口颜色选项

C.8 窗口大小和位置选项

C.9 内部边框和外部边框

C.10 框架标题

C.11 图标

C.12 其他显示选项

附录 D X 选项与资源

D.1 X 资源

D.2 Emacs 的 X 资源表

D.3 Lucid 菜单和对话框 X 资源

D.4 Motif 菜单 X 资源

D.5 GTK+ 资源

D.5.1 GTK+ 资源基础
D.5.2 GTK+ 部件名称
D.5.3 Emacs 中的 GTK+ 控件名
D.5.4 GTK+ 样式

附录 E Emacs 30 反新闻

附录 F Emacs 与 macOS / GNUstep

F.1 macOS 与 GNUstep 下使用 Emacs 的基本方式
F.1.1 获取环境变量
F.2 Mac / GNUstep 自定义
F.2.1 修饰键
F.2.2 框架变量
F.2.3 macOS 触控板/鼠标滚轮变量
F.3 macOS / GNUstep 下的窗口系统事件
F.4 GNUstep 支持

附录 G Emacs 与 Haiku

G.1 Haiku 安装与启动
G.2 Haiku 下的字体后端与选区

附录 H Emacs 与 Android

H.1 Android 历史
H.2 在 Android 上启动 Emacs
H.3 Emacs 在 Android 上能访问哪些文件
H.4 在 Android 上访问其他程序的文件
H.5 在 Android 下运行 Emacs
H.6 Android 窗口系统
H.7 Android 下的字体后端与选区
H.8 Android 启动问题排查
H.9 在 Android 上安装额外软件

附录 I Emacs 与 Microsoft Windows/MS-DOS

I.1 如何在 MS-Windows 上启动 Emacs
I.2 文本文件与二进制文件
I.3 MS-Windows 上的文件名
I.4 在 MS-Windows 上模拟 'ls'
I.5 MS-Windows 上的 HOME 与启动目录
I.6 MS-Windows 上的键盘用法
I.7 MS-Windows 上的鼠标使用
I.8 Windows 9X/ME 和 Windows NT/2K/XP/Vista/7/8/10 上的子进程

I.9 打印与 MS-Windows
I.10 在 MS-Windows 上指定字体
I.11 其他 Windows 特有功能
I.12 Emacs 与 MS-DOS
I.12.1 MS-DOS 上的键盘使用
I.12.2 MS-DOS 上的鼠标使用
I.12.3 MS-DOS 上的显示
I.12.4 MS-DOS 上的文件名
I.12.5 打印与 MS-DOS
I.12.6 MS-DOS 上的国际化支持
I.12.7 MS-DOS 上的子进程

GNU 宣言

GNU 是什么？ Gnu's Not Unix！
我为什么必须编写 GNU
GNU 为何与 Unix 兼容
GNU 将如何发布
为何许多其他程序员愿意帮忙
你可以怎样贡献
为何所有计算机用户都将受益
对 GNU 目标的一些容易反驳的异议

术语表

致谢

附录 J 常用配置速查

初始化文件
最小可用模板
常用按键绑定
钩子
常用变量速查
主题与字体
启动与调试

发行

GNU Emacs 是“自由软件”；这意味着任何人都可以自由使用它，并在特定条件下自由再分发它。GNU Emacs 不属于公有领域；它受版权保护，其分发也受到限制，但这些限制的用意是允许一个好公民所想做的一切事情。所不允许的，是试图阻止他人从你这里取得 GNU Emacs 之后继续分享它。准确的条件见随 Emacs 提供的 GNU General Public License，该许可证也出现在本手册中(1)。参见“附录 A GNU 通用公共许可证”。

获得 GNU Emacs 副本的一种途径是从已经拥有它的人那里取得。你无需为此征求我们的许可，也不必告知其他人，直接复制即可。如果你能访问互联网，可以通过匿名 FTP 获取 GNU Emacs 的最新发行版本；更多信息见我们网站上的 <https://www.gnu.org/software/emacs>。

购买计算机时，你也可能得到 GNU Emacs。计算机制造商可以按照适用于其他所有人的相同条款自由分发副本。这些条款要求他们向你提供完整的源码，包括他们所做的任何修改，并允许你按照 General Public License 的通常条款再分发从他们那里得到的 GNU Emacs。换言之，该程序在你获得它时就必须对你是自由的，而不只是对制造商自由。

如果你觉得 GNU Emacs 有用，**请向自由软件基金会捐款**，以支持我们的工作。在美国，向自由软件基金会的捐款可以抵税。如果你在工作中使用 GNU Emacs，请建议公司捐款。捐款方式见 <https://my.fsf.org/donate/>。其他可以帮助我们的方式见 <https://www.gnu.org/help/help.html>。

我们也出售本手册以及 Robert J. Chassell 所著 ‘An Introduction to Programming in Emacs Lisp’ 的纸质版。你可以访问我们的网上商店 <https://shop.fsf.org/>。销售收入用于支持基金会的宗旨：开发新的自由软件，并改进我们现有的程序（包括 GNU Emacs）。

如果需要联系自由软件基金会，见 <https://www.fsf.org/about/contact/>，或写信至：

```
Free Software Foundation
31 Milk Street, # 960789
Boston, MA 02196
USA
```

> 注：本手册本身受 GNU Free Documentation License 保护。该许可证在精神上类似于 General Public License，但更适合文档。参见“附录 B GNU 自由文档许可证”。

简介

你正在阅读的是关于 GNU Emacs 的内容，它是高级、自文档化、可定制、可扩展的编辑器 Emacs 的 GNU 版本。（GNU（GNU's Not Unix）中的‘G’并非不发音。）

我们称 Emacs 为“高级”，是因为它能做的远不止简单地插入和删除文本。它可以控制子进程、自动缩进程序、一次显示多个文件、像编辑本地文件那样编辑远程文件，等等。Emacs 的编辑命令以字符、单词、行、句子、段落和页为单位操作，也处理各种编程语言中的表达式和注释。

“自文档化”是指：你随时可以使用称为“帮助命令”的特殊命令，查明自己有哪些选项、某条命令做什么，或者找出与给定主题相关的全部命令。参见“帮助”。

“可定制”是指：你可以用简单的办法轻松改变 Emacs 命令的行为。例如，如果你所用的编程语言中注释以‘<’开头、以‘>’结尾，你就可以让 Emacs 的注释操作命令使用这两个字符串（参见“操作注释”）。再举一例，你可以把基本的光标移动命令（上、下、左、右）重新绑定到键盘上任何你觉得顺手的键。参见“自定义”。

“可扩展”是指：你可以超越简单的定制，创建全新的命令。新命令不过是用 Lisp 语言编写的程序，由 Emacs 自带的 Lisp 解释器运行。甚至可以在编辑会话中途重新定义已有的命令，而不必重启 Emacs。Emacs 中的大多数编辑命令都是用 Lisp 编写的；少数例外本可以用 Lisp 编写，但为了效率而改用 C。编写扩展属于编程，但非程序员事后也能使用它。参见《《eintr》手册》的“Top, if you want to learn Emacs Lisp programming”

1 屏幕布局

在图形显示环境（例如使用 X Window System 的 GNU/Linux）中，Emacs 占据一个图形窗口。在文本终端上，Emacs 占据整个终端屏幕。我们用“框架”（frame）一词指代 Emacs 所占用的图形窗口或终端屏幕。Emacs 在这两种框架上的行为非常相似。它通常只启动一个框架，不过你也可以按需创建更多框架（参见“框架与图形显示”）。

每个框架都由若干个不同的区域组成。框架顶部是“菜单栏”（menu bar），它让你通过一系列菜单访问命令。在图形显示环境中，菜单栏正下方是“工具栏”（tool bar），即一排图标，点击它们即可执行编辑命令。框架最底部是“回显区”（echo area），信息性消息在此显示，Emacs 索要信息时也在此输入。

框架的主要区域，即工具栏（如果有的话）下方、回显区上方的部分，称为“窗口”。本手册此后都在这个意义上使用“窗口”一词。图形显示系统通常以另一种含义使用“窗口”一词；但如上所述，我们把那些图形窗口称为“框架”。

Emacs 窗口是用来显示“缓冲区”（buffer）的地方——缓冲区就是你正在编辑或查看的文本或其他图形内容。在图形显示环境中，窗口一侧带有“滚动条”（scroll bar），可用于在缓冲区中滚动。窗口的最后一行是“模式行”（mode line）。它显示关于缓冲区当前状态的各种信息，例如是否有未保存的修改、正在使用哪些编辑模式、当前行号等等。

启动 Emacs 时，框架中通常只有一个窗口。不过，你可以水平或垂直地拆分这个窗口，从而创建多个窗口，每个窗口都能独立显示一个缓冲区（参见“多窗口”）。

任何时候都有一个窗口是“被选中的窗口”（selected window）。在图形显示环境中，被选中的窗口显示一个更醒目的光标（通常是实心且闪烁的），其他窗口则显示不太醒目的光标（通常是空心方框）。在文本终端上只有一个光标，它显示在被选中的窗口中。在被选中窗口中显示的缓冲区称为“当前缓冲区”（current buffer），编辑就发生在这里。多数 Emacs 命令都隐含地作用于当前缓冲区；未被选中的窗口中显示的文本主要供参考。如果在图形显示环境中使用多个框架，选中某个框架就会选中该框架中的一个窗口。

1.1 光标点

被选中窗口中的光标指示多数编辑命令生效的位置，这个位置称为“光标点”（point）(1)。许多 Emacs 命令会把光标点移到缓冲区中的不同位置；例如，你可以在目标位置点击鼠标键 1（通常是左键）来放置光标点。

默认情况下，被选中窗口中的光标被绘制成一个实心方块，看起来似乎压在某个字符*之上*，但你应该把光标点理解为位于两个字符*之间*；它位于光标下方那个字符的*前面*。例如，若你的文本形如 ‘frob’，且光标压在 ‘b’ 上，那么光标点就在 ‘o’ 与 ‘b’ 之间。若你在该位置插入字符 ‘!’，结果就是 ‘fro!b’，光标点位于 ‘!’ 与 ‘b’ 之间。于是光标仍和先前一样压在 ‘b’ 上。

如果在 Emacs 中编辑多个文件，每个文件各自位于一个缓冲区中，那么每个缓冲区都有自己的光标点值。当前没有显示的缓冲区会记住它的光标点值，以便你日后再次显示它。此外，若某个缓冲区显示在多个窗口中，这些窗口各自拥有自己的光标点值。

关于控制 Emacs 如何显示光标的选项，参见“光标的显示”。

> 注：“point”一词来自字符‘.’，它是 TECO（最初编写 Emacs 所用的语言）中用于访问编辑位置的命令。

1.2 回显区

框架最底部的那一行就是“回显区”（echo area）。它用于因各种目的而显示少量文本。

回显区之所以得名，是因为它的用途之一是“回显”（echoing），即在你键入多字符命令时显示该命令的各个字符。单字符命令不回显。若你在输入命令的过程中停顿超过一秒，多字符命令（参见“按键”）就会被回显。此时 Emacs 会把该命令到目前为止的所有字符回显出来，以提示你输入其余部分。一旦回显开始，命令余下的部分就会在你键入时立即回显。这种设计既让有把握的用户得到快速响应，又让迟疑的用户得到最大程度的反馈。

当命令无法完成其工作时，回显区还用于显示“错误消息”（error message）。错误消息可能伴有鸣响或屏幕闪烁。

有些命令会在回显区显示信息性消息，告诉你该命令做了什么，或向你提供某些特定信息。这类“信息性”（informative）消息与错误消息不同，不伴有鸣响或闪烁。例如，‘C-x =’（按住 <Ctrl> 并键入‘x’，然后松开 <Ctrl> 并键入‘=’）会显示一条消息，描述光标点处的字符、它在缓冲区中的位置，以及它在窗口中的当前列。耗时较长的命令在工作过程中常常显示以‘...’结尾的消息（有时还以百分比形式指示已完成多少），并在结束时补上‘done’。

回显区中的信息性消息会保存在名为‘*Messages*’的特殊缓冲区里。（我们还没有解释缓冲区；关于它们的更多信息，参见“多缓冲区”。）如果你错过了屏幕上短暂出现的某条消息，可以切换到‘*Messages*’缓冲区再看一遍。‘*Messages*’缓冲区限定为若干行，具体数目由变量‘message-log-max’指定。（我们也还没有解释变量；关于它们的更多信息，参见“变量”。）超过这个限制后，每当末尾新增一行消息，开头就删除一行。

关于控制 Emacs 如何使用回显区的选项，参见“显示的自定义”。

回显区还用于显示“迷你缓冲区”（minibuffer），这是一个特殊的窗口，你可以在其中输入命令的参数，例如要编辑的文件名。使用迷你缓冲区时，回显区中显示的文本以“提示字符串”（prompt string）开头，活动光标出现在迷你缓冲区中，此时迷你缓冲区被临时视为被选中的窗口。你随时可以键入‘C-g’退出迷你缓冲区。参见“迷你缓冲区”。

1.3 模式行

每个窗口的底部都有一条“模式行”（mode line），它描述当前缓冲区的状况。只有一个窗口时，模式行正好位于回显区上方；它是框架中的倒数第二行。在图形显示环境中，模式行以 3D 方框的外观绘制。Emacs 通常还把被选中窗口的模式行绘成与未选中窗口不同的颜色，以便让它突出显示。

模式行中显示的文本采用以下格式：

```
CS:CH-DFR  BUF      POS LINE   (MAJOR MINOR)
```

在文本终端上，这段文本之后会跟随一串横线，一直延伸到窗口右边缘。在图形显示环境中则省略这些横线。

CS 字符串及其后的冒号字符描述当前缓冲区所使用的字符集和换行约定。通常 Emacs 会自动为你处理这些设置，但有时了解这一信息很有用。

CS 描述缓冲区中文本的字符集（参见“编码系统”）。如果它是一个短横线（‘-’），表示不做特殊的字符集处理（可能的例外是下一段所述的行尾约定）。‘=’ 表示完全不进行任何转换，通常用于包含非文本数据的文件。其他字符表示各种“编码系统”（coding system）——例如，‘1’ 表示 ISO Latin-1。

在文本终端上，CS 之前还有两个字符，用来描述键盘输入和终端输出的编码系统。此外，如果你正在使用输入法，CS 之前还有一个标识该输入法的字符串（参见“输入法”）。

CS 之后的字符通常是冒号。如果显示的是别的字符串，则表示编码文件时采用了某种非普通的行尾约定。通常，文件中的文本行以“换行字符”（newline character）分隔，但有时也会使用另外两种约定。MS-DOS 约定使用回车字符后跟换行字符；编辑这类文件时，冒号会变成反斜杠（‘\’）或 ‘(DOS)’（视操作系统而定）。另一种由较旧的 Macintosh 系统采用的约定使用回车字符代替换行；编辑这类文件时，冒号会变成正斜杠（‘/’）或 ‘(Mac)’。在某些系统上，对于以换行作为行分隔符的文件，Emacs 会显示 ‘(Unix)’ 而不是冒号。

在为 ‘emacsclient’ 创建的框架上（参见“调用 ‘emacsclient’”），下一个字符是 ‘@’。这一标志是作为守护进程运行的 Emacs 进程的框架的典型特征（参见“把 Emacs 用作服务器”）。

模式行上的下一个元素是 CH 所指示的字符串。如果窗口中显示的缓冲区内容与磁盘上对应的文件相同，即缓冲区未被修改，它就显示两个短横线（‘--’）。如果缓冲区已被修改，它就显示两个星号（‘**’）。对于只读缓冲区，若缓冲区已被修改，它显示 ‘%*’，否则显示 ‘%%’。

CH 之后的字符通常是短横线（‘-’）。不过，如果当前缓冲区的 ‘default-directory’（参见“文件名”）位于远程机器上，则改为显示 ‘@’。

如果窗口专用于其当前缓冲区，就会出现 D。强烈专用时显示为 ‘D’，其他形式的专用显示为 ‘d’。如果窗口并非专用，D 就不出现。参见《Emacs Lisp 参考手册》的“Dedicated Windows”。

FR 给出被选中框架的名称（参见“框架与图形显示”）。它只出现在文本终端上。初始框架的名称为 ‘F1’。

BUF 是窗口中显示的缓冲区的名称。通常，这与你正在编辑的文件的名称相同。参见“多缓冲区”。

POS 告诉你窗口顶部之上或底部之下是否还有额外文本。如果缓冲区很小，全部内容都能在窗口中看到，POS 就是 ‘All’。否则，若你看到的是缓冲区的开头，它就是 ‘Top’；若你看到的是缓冲区的结尾，它就是 ‘Bot’；再否则就是 ‘NN%’，其中 NN 是窗口顶部之上部分占缓冲区的百分比。启用 Size Indication 模式后，你还可以显示缓冲区的大小。参见“可选的模式行功能”。

LINE 是字符 ‘L’ 后跟光标点所在的行号。（你还可以打开 Column Number 模式来显示当前列号。参见“可选的模式行功能”。）

MAJOR 是缓冲区所用“主模式”（major mode）的名称。主模式是缓冲区的主要编辑模式，如 Text 模式、Lisp 模式、C 模式等等。参见“主模式”。有些主模式在主模式名称之后显示附加信息。例如，Compilation 缓冲区和 Shell 缓冲区会显示子进程的状态。

MINOR 是所启用“次模式”（minor mode）中一部分模式的列表，这些模式是在主模式之上提供附加功能的可选编辑模式。参见“次模式”。

有些特性在开启时会与次模式一起列出，尽管它们并非真正的次模式。‘Narrow’ 表示所显示的缓冲区被限制为只能编辑其文本的一部分（参见“窄化”）。‘Def’ 表示当前正在定义键盘宏（参见“键盘宏”）。

此外，如果 Emacs 处于某一层递归编辑层级之中，那么在包围各模式名称的圆括号两侧还会出现方括号（‘[...]’）。如果 Emacs 处于一层递归编辑层级中的又一层递归编辑层级，就会出现双方括号，依此类推。由于递归编辑层级对 Emacs 全局生效，这类方括号会出现在每个窗口的模式行中。参见“递归编辑层级”。

你可以改变模式行的外观，也可以改变其内容的格式。参见“可选的模式行功能”。此外，模式行对鼠标敏感；点击模式行的不同部位会执行各种命令。参见“模式行鼠标命令”。而且，将鼠标指针悬停在模式行中对鼠标敏感的部分上时，会显示工具提示（参见“工具提示”），其中包含关于你点击模式行即可调用的命令的信息。

1.4 菜单栏

每个 Emacs 框架通常都在顶部带有一个“菜单栏”（menu bar），你可以用它执行常用的操作。此处无需逐一列举，因为你自己查看这些菜单更为便当。

在支持鼠标的显示环境中，你可以用鼠标从菜单栏中选择命令。菜单项右边缘的箭头表示它会进入一个从属菜单，即“子菜单”（submenu）。菜单项末尾的 ‘...’ 表示该命令在真正执行任何操作之前会提示你进一步输入。

菜单栏中的某些命令也有普通的键绑定；若有，键绑定会显示在菜单项本身之后。要查看某个菜单项的完整命令名称和文档，请键入 ‘C-h k’，然后照常使用鼠标选择菜单栏（参见“按键的文档”）。

除了使用鼠标，你还可以按 <F10>（运行命令 ‘menu-bar-open’）来调用菜单栏的第一项。随后你可以用方向键，或用 ‘C-b’、‘C-f’（左/右）、‘C-p’ 和 ‘C-n’（上/下）在菜单中导航。要激活选中的菜单项，按 <RET>；要取消菜单导航，按 ‘C-g’ 或 ‘<ESC> <ESC> <ESC>’。（不过请注意，当 Emacs 使用 GUI 工具包构建时，菜单由该工具包绘制和控制，取消菜单导航的键序列可能与上述描述不同。）

在文本终端上，你还可以选择在回显区访问菜单栏的菜单。为此，请把变量 ‘tty-menu-open-use-tmm’ 定制为非 ‘nil’ 值。这样键入 <F10> 就会运行命令 ‘tmm-menubar’，而不是下拉菜单。（你也可以键入 ‘M-`’，它总是调用 ‘tmm-menubar’。）‘tmm-menubar’ 让你用键盘选择菜单项。一个临时选择会显示在回显区中。你可以用上下方向键在菜单中移动到不同的菜单项，然后键入 <RET> 选择该项。每个菜单项还由一个字母或数字指定（通常是该项名称中某个单词的首字母）。这个字母或数字与菜单项名称之间用 ‘==>’ 分隔。你可以键入该项的字母或数字来选择它。

2 输入类型

GNU Emacs 主要是为使用键盘而设计的。虽然通过菜单栏和工具栏也可以用鼠标发出编辑命令，但那样通常不如使用键盘高效。

Emacs 的键盘输入基于大幅扩展过的 ASCII 字符集。像 'a'、'B'、'3'、'=' 这样的简单字符，以及空格字符（记作 <SPC>），都通过按相应的键输入。"控制字符"，例如 <RET>、<TAB>、<DEL>、<ESC>、<F1>、<Home> 和 <LEFT>，也是这样输入的；非英语键盘上的一些字符同样如此（参见"国际字符集支持"）。

Emacs 还能识别用"修饰键"输入的控制字符。两种常用的修饰键是 <Control>（通常标注为 <Ctrl>）和 <Meta>（通常标注为 <Alt>）(1)。例如，按住 <Ctrl> 键的同时按 'a'，就输入了 'Control-a'；下文将其简称为 'C-a'。类似地，'<Meta>-a'（简称 'M-a'）的输入方式是按住 <Alt> 键再按 'a'。修饰键也可以作用于非字母数字字符，例如 'C-<F1>' 或 'M-<LEFT>'。

你也可以用 <ESC> 开头的两字符序列来输入 Meta 字符。因此，输入 '<ESC> a' 就能得到 'M-a'。输入 '<ESC> C-a' 则能得到 'C-M-a'（即同时按住 <Ctrl> 和 <Alt>，再按 'a'）。与 <Meta> 不同，<ESC> 是作为一个单独的字符输入的。你不需要在输入下一个字符时一直按住 <ESC>，而是按下 <ESC> 再松开，然后输入下一个字符。在某些 <Meta> 键工作不可靠的文本终端上，这一功能很有用。

Emacs 还支持另外 3 个修饰键，参见"修饰键"。

Emacs 对鼠标按键、鼠标滚轮以及触摸板、触摸屏之类的其他指点设备提供了广泛支持。详情参见"鼠标输入"。

在图形显示环境中，窗口管理器可能会拦截某些键盘输入，包括 'M-<TAB>'、'M-<SPC>'、'C-M-d' 和 'C-M-l'。如果你遇到这个问题，可以调整窗口管理器的设置使其不再拦截这些键，也可以重新绑定受影响的 Emacs 命令（参见"自定义"）。

如果某个 Emacs 框架正在迷你缓冲区中进行提示，而其他 Emacs 框架似乎忽略了输入，你可能希望禁用单一键盘模式。参见"多个 X 显示"。

简单字符和控制字符，以及鼠标点击之类的某些非键盘输入，统称为"输入事件"。关于 Emacs 内部如何处理输入事件的细节，参见《Emacs Lisp 参考手册》的"Input Events"。

> 注：出于历史原因，我们把 <Alt> 称为 <Meta>。

3 按键

有些 Emacs 命令只需一个输入事件即可调用；例如 ‘C-f’ 在缓冲区中向前移动一个字符。另一些命令则需要两个或更多输入事件才能调用，比如 ‘C-x C-f’ 和 ‘C-x 4 C-f’。

“键序列”（key sequence，简称“键”）指作为一个整体有意义的一个或多个输入事件序列。若某键序列能调用命令，就称它为“完整键”（complete key）；例如 ‘C-f’、‘C-x C-f’ 和 ‘C-x 4 C-f’ 都是完整键。若某键序列的长度不足以调用命令，就称它为“前缀键”（prefix key）；从前面的例子可见，‘C-x’ 和 ‘C-x 4’ 都是前缀键。每个键序列要么是完整键，要么是前缀键。

前缀键与随后的输入事件结合，构成更长的键序列。例如 ‘C-x’ 是前缀键，因此单独键入 ‘C-x’ 并不会调用命令；此时 Emacs 会等待后续输入（若你停顿超过一秒，它会回显 ‘C-x’ 键以提示该输入；参见“回显区”）。‘C-x’ 与下一个输入事件结合，构成一个两事件的键序列，这个序列本身可能是前缀键（如 ‘C-x 4’），也可能是完整键（如 ‘C-x C-f’）。键序列的长度没有限制，但实践中很少超过三四个输入事件。

你不能把输入事件追加到完整键上。例如，由于 ‘C-f’ 是完整键，两事件的序列 ‘C-f C-k’ 是两个键序列，而不是一个。

默认情况下，Emacs 中的前缀键有 ‘C-c’、‘C-h’、‘C-x’、‘C-x <RET>’、‘C-x @’、‘C-x a’、‘C-x n’、‘C-x r’、‘C-x t’、‘C-x v’、‘C-x 4’、‘C-x 5’、‘C-x 6’、<ESC> 和 ‘M-g’。（<F1> 和 <F2> 分别是 ‘C-h’ 和 ‘C-x 6’ 的别名。）这份列表并非一成不变；如果你自定义 Emacs，可以设置新的前缀键。你甚至可以去掉一些标准前缀键，不过这对大多数用户并不推荐；例如，若你删除了 ‘C-x 4’ 的前缀定义，那么 ‘C-x 4 C-f’ 就成了无效的键序列。参见“定制键绑定”。

在前缀键之后键入帮助字符（‘C-h’ 或 <F1>），会显示以该前缀开头的命令列表。此规则唯一的例外是 <ESC>：‘<ESC> C-h’ 等价于 ‘C-M-h’，而后者的作用完全不同。不过，你可以用 <F1> 显示以 <ESC> 开头的命令列表。

4 鼠标输入

默认情况下，Emacs 支持所有常见的鼠标操作，例如用左键单击来设置光标点位置，以及拖动鼠标指针来选定区域。所有鼠标动作都能像键盘事件那样用来绑定命令（参见“按键”）。本节概述在 Emacs 中使用鼠标的一般情况；关于 Emacs 中鼠标命令的更多细节，请参见“鼠标编辑命令”及其后续各节。

单击鼠标左键时，Emacs 收到的是 ‘mouse-1’ 事件。要查看该事件绑定了哪个命令，可以键入 ‘C-h c’，然后按下鼠标左键。类似地，鼠标中键是 ‘mouse-2’，鼠标右键是 ‘mouse-3’。如果鼠标带有滚轮，滚轮事件通常绑定到 ‘wheel-down’ 或 ‘wheel-up’，或者 ‘mouse-4’ 与 ‘mouse-5’，但这取决于操作系统的配置。

一般来说，旧的 X 系统和终端（参见“在文本终端中使用鼠标”）会报告 ‘mouse-4’ 和 ‘mouse-5’，而其他所有系统则报告 ‘wheel-down’ 和 ‘wheel-up’。

有些鼠标还有水平滚轮，触摸板通常也支持水平滚动。除终端和旧的 X 系统外，这些事件在所有系统上报告为 ‘wheel-left’ 和 ‘wheel-right’，而在终端和旧的 X 系统上则为 ‘mouse-6’ 和 ‘mouse-7’。

你也可以把键盘修饰键与鼠标事件组合起来，从而绑定一个特殊命令，例如在你按住 Meta 键的同时使用鼠标中键时触发。这种情况下，事件名将是 ‘M-mouse-2’。

在某些系统上，你还可以为处理触摸屏事件绑定命令。此时，这些事件称为 ‘touchscreen-update’ 和 ‘touchscreen-end’。

5 按键与命令

本手册中有大量段落会告诉你特定的键能做什么。但 Emacs 并不直接为键赋予含义。相反，Emacs 为有名字的“命令”赋予含义，然后通过把键“绑定”到命令，来赋予键含义。

每个命令都有一个由程序员选定的名字。名字通常由几个英文单词组成，中间用连字符分隔；例如 ‘next-line’ 或 ‘forward-word’。在内部，每个命令是一种特殊的 Lisp“函数”，与命令相关的动作通过运行该函数来执行。参见《Emacs Lisp 参考手册》的“What Is a Function”

键与命令之间的绑定记录在称为“键位图”（keymap）的表中。参见“键位图”。

当我们说“‘C-n’ 垂直向下移动一行”时，我们忽略了一个微妙的区别——它在日常使用中无关紧要，但对于 Emacs 定制却至关重要。命令 ‘next-line’ 执行向下垂直移动。‘C-n’ 之所以有这个效果，是因为它绑定到了 ‘next-line’。如果你把 ‘C-n’ 重新绑定到命令 ‘forward-word’，‘C-n’ 就会改为向前移动一个单词。

在本手册中，我们常常会把像 ‘C-n’ 这样的键说成命令，尽管严格说来键是绑定到命令的。通常，在提到执行该动作的键之后，我们会用括号给出真正完成工作的命令名。例如，我们会说“命令 ‘C-n’（‘next-line’）把光标点垂直向下移动”，意思是命令 ‘next-line’ 垂直向下移动，而键 ‘C-n’ 通常绑定到它。

既然我们在讨论定制，就应该告诉你“变量”是什么。命令的描述中常常会说：“要改变这一点，请设置变量 ‘mumble-foo’。”变量是用于存储值的名字。本手册中记录的大多数变量都用于定制：某个命令或 Emacs 的其他部分会检查该变量，并根据你设置的值而表现不同。在你想定制变量之前，可以忽略有关变量的信息。之后，读一读关于变量的基本信息（参见“变量”），关于具体变量的信息就会变得有意义。

6 触摸屏输入与虚拟键盘

Emacs 最初编写时假定用户会从台式计算机或计算机终端上使用它，这些设备配有键盘，可能还配有合适的指针设备，例如鼠标（参见“鼠标输入”）。

Emacs 也能接收来自其他输入源的输入，即使用户所用的计算机以这类输入源取代了惯常的键盘与鼠标组合，用户仍能与之交互。

6.1 在触摸屏上使用 Emacs

触摸屏输入（touchscreen input）指通过在显示框架内容的显示器或计算机终端上放置并移动工具（例如手指，以及触控笔之类的指针设备）来操作框架的内容。

这类工具放置的顺序与位置这两个因素，会与称为“手势”（gestures）的预定义模式相比较；随后，与这两个因素相吻合的手势就指定了一系列要施加于工具下方文本之上操作。目前可识别的手势有：

- “点按”（tapping），即短暂地把工具放到显示器上再抬起，会选中被点按的窗口，并执行绑定到该窗口中该位置‘mouse-1’上的命令。若该处存在链接（参见“用鼠标跟随引用”），则 Emacs 会改为访问该链接（前提是该操作与模拟‘mouse-1’事件所执行的操作不同）。

- “滚动”（scrolling），即屏幕上连续的垂直或水平移动，会沿移动方向滚动工具初始位置下方窗口的内容。用户选项‘touch-screen-enable-hscroll’控制是否响应该手势执行水平滚动（参见“水平滚动”）。

- “拖拽”（dragging），即通过把工具放在显示器上并停留一会儿再移到另一位置来执行“长按”（long-press），会把光标点移到工具的初始位置，并在工具继续移动时开始选择其下方的文本，就如同按住‘mouse-1’并相应地移动鼠标一样。参见“鼠标编辑命令”。

有损于文本选择的是，在触摸屏显示器上精确地放置工具可能颇为困难。用户选项‘touch-screen-word-select’在启用后，会促使拖拽选中工具下方的整个单词。（通常，选择范围只扩展到涵盖工具下方的那个字符。）

同样，可能难以在单次手势中选中所有想要的文本。若启用用户选项‘touch-screen-extend-selection’，则在窗口中光标点或标记的位置上点按会开始一个新的“拖拽”手势，区域会沿随后任何移动的方向扩展。

光标被工具物理遮挡而难以对区域做出精确调整，这可以通过在回显区中指示光标点的位置来缓解。若‘touch-screen-preview-select’为非‘nil’，则在工具移动期间会在回显区（参见“回显区”）显示光标点周围的那一行，其下方另有一行指示光标点相对于该行的位置。

- “捏合”（pinching），即在屏幕上将两个工具分开放置，然后调整其位置以增大或减小它们之间的距离，会按该距离变化的比例修改文本缩放（参见“文本缩放”）。

当工具放置在屏幕上的时间超过 0.7 秒后，Emacs 会将其登记为长按。该延迟可通过定制变量‘touch-screen-delay’来调整。

6.2 配合虚拟键盘使用 Emacs

当系统没有连接物理键盘时，其窗口系统可能提供屏幕键盘，即通常所说的“虚拟键盘”（virtual keyboard），其中包含成排的可点击按钮，能像真实键盘一样向应用程序发送键盘输入。

由于虚拟键盘会占用显示器上稀缺的空间，当获得焦点的程序不请求文本输入时它会被隐藏，因此程序被要求在准备好接受键盘输入后再显示它。运行 X 的系统会检测何时有必要出现虚拟键盘，但在 Android 等其他系统上，Emacs 负责在需要时显示它，通常是响应触摸屏的“点按”手势（参见“在触摸屏上使用 Emacs”）或迷你缓冲区被启用（参见“迷你缓冲区”）。

当“点按”手势导致某个命令被执行时，Emacs 会在 ‘touch-screen-set-point-commands’ 列表中查找该命令，以检查它是否意在设置光标点。若是，且新光标点下方的文本不是只读的，就会激活虚拟键盘，预期用户将在该处输入。

‘touch-screen-set-point-commands’ 的默认值只包含命令 ‘mouse-set-point’（参见“鼠标编辑命令”），它是 ‘mouse-1’ 的默认绑定，因而也是触摸屏点按手势的默认绑定。

用户选项 ‘touch-screen-display-keyboard’ 强制 Emacs 在所有点按手势下都显示虚拟键盘，即使文本是只读的；它也可以按缓冲区局部设置，此时只要在显示其所设缓冲区的窗口上点按，Emacs 就总会显示键盘。

此外还有若干函数用于显示或隐藏屏幕键盘。更多细节见 参见《Emacs Lisp 参考手册》的“On-Screen Keyboards”。

由于 Emacs 在执行命令时可能无法显示虚拟键盘，Emacs 在常配无物理键盘的窗口系统上实现了一项特性：快速连按两次设备上始终存在的硬件按钮即可引发退出。参见“退出与中止”。

X 上未启用此类按钮，但可通过变量 ‘x-quit-keysym’ 配置一个；而在 Android 上默认按键是音量减键，它同样可通过变量 ‘android-quit-keycode’ 配置。

为虚拟键盘设计的大多数输入法，编辑文本的方式与桌面输入法不同。

在传统的桌面窗口系统上，输入法只会在屏幕上显示任何正在进行的字符组合的内容，并在用户确认之后把反映其内容的按键事件发送给 Emacs。

相比之下，虚拟键盘输入法直接对每个框架的选中窗口执行编辑；这称为“文本转换”（text conversion），在 X Window System 下称为“字符串转换”（string conversion）。

只要 ‘text-conversion-style’ 的缓冲区局部值非 ‘nil’，Emacs 就会启用这些输入法，也就是说，通常是在 ‘text-mode’ 和 ‘prog-mode’ 的派生模式中。

每当 Emacs 收到输入法要求执行转换的请求，且当前没有正在读取已读过某个前缀键的键序列时（参见“按键”），文本转换就会异步执行。转换完成后，会发送一个 ‘text-conversion’ 事件。参见《Emacs Lisp 参考手册》的“Misc Events”。

若输入法需要作用于缓冲区的某个区域，则该区域被称为“组合区域”（composing region，或“预转换区域”，preconversion region）。变量 ‘text-conversion-face’ 控制是否以独特的字体（face）显示组合区域，以及若显示则使用哪种字体。

7 启动 Emacs

调用 Emacs 的常用方式是使用 shell 命令 'emacs'。在运行 Unix shell 的 GUI 终端窗口中，可以用 'emacs &' 让 Emacs 在后台运行；这样 Emacs 就不会占用该终端窗口，你可以继续用它执行其他 shell 命令。（在 MS-Windows 上启动 Emacs 的类似方法，参见"如何在 MS-Windows 上启动 Emacs"。）

Emacs 启动时，初始框架会显示一个名为 '*GNU Emacs*' 的特殊缓冲区。这个"启动屏幕"包含关于 Emacs 的信息，以及指向初学者常用任务的"链接"。例如，激活 'Emacs Tutorial' 链接会打开 Emacs 教程；其作用与命令 'C-h t'（'help-with-tutorial'）相同。要激活链接，可以把光标点移到该链接上并键入 '<RET>'，或者用 'mouse-1'（鼠标左键）单击它。

通过命令行参数，你可以让 Emacs 一启动就访问一个或多个文件。例如，'emacs foo.txt' 会启动 Emacs，并用一个缓冲区显示文件 'foo.txt' 的内容。这一特性主要是为了与其他编辑器兼容，那些编辑器是为从 shell 启动、进行简短编辑会话而设计的。若以这种方式调用 Emacs，初始框架会分成两个窗口——一个显示指定的文件，另一个显示启动屏幕。参见"多窗口"。

通常，每次编辑文件都重新启动一次 Emacs 既不必要也很浪费。推荐的使用方式是：登录之后只启动一次 Emacs，然后所有编辑都在同一个 Emacs 会话中完成。关于访问多个文件的信息，参见"文件处理"。若这样使用 Emacs，该会话会积累有价值的上下文，例如删除环、寄存器、撤销历史以及标记环数据，这些合在一起能让编辑更方便。这些特性将在手册后面介绍。

若想在 Emacs 运行期间从其他程序编辑文件，可以使用辅助程序 'emacsclient' 在已有的 Emacs 会话中打开文件。参见"把 Emacs 用作服务器"。

Emacs 还接受其他命令行参数，用来告诉它加载某些 Lisp 文件、把初始框架放在何处等等。参见"附录 C 启动 Emacs 的命令行参数"。

若变量 'inhibit-startup-screen' 的值非 'nil'，Emacs 就不显示启动屏幕。在这种情况下，如果命令行上指定了一个或多个文件，Emacs 就只显示这些文件；否则，它显示一个名为 '*scratch*' 的缓冲区，可以用它交互式地求值 Emacs Lisp 表达式。参见"Lisp 交互缓冲区"。你可以使用 Customize 功能（参见"简易自定义界面"）设置变量 'inhibit-startup-screen'，也可以编辑初始化文件（参见"Emacs 初始化文件"）来设置它。

你也可以把变量 'initial-buffer-choice' 设为一个命名某文件或目录的字符串，从而强制 Emacs 在启动时显示该文件或目录。'initial-buffer-choice' 的值还可以是一个（不带参数的）函数，该函数应返回一个缓冲区，随后 Emacs 会显示它。若 'initial-buffer-choice' 非 'nil'，那么即使你在命令行上指定了文件，Emacs 仍会访问它们，但初始时不会显示它们。

> 注：在 'site-start.el' 中设置 'inhibit-startup-screen' 不起作用，因为启动屏幕是在读取 'site-start.el' 之前建立的。关于 'site-start.el' 的信息，参见"Emacs 初始化文件"。

8 退出 Emacs

‘C-x C-c’ 退出 Emacs（‘save-buffers-kill-terminal’）。‘C-z’ 在文本终端上挂起 Emacs；在图形显示上，则将选中的框架图标化（或“最小化”）（‘suspend-frame’）。

“退出”（kill）Emacs 意味着终止 Emacs 程序。为此，请键入 ‘C-x C-c’（‘save-buffers-kill-terminal’）。使用双字符键序列是为了降低误按的可能性。当你键入 ‘C-x C-c’ 时，如果有任何已修改的访问文件的缓冲区，Emacs 会先提示你保存这些缓冲区。如果没有把它们全部保存，它会再次请求确认，因为未保存的改动将会丢失。如果有任何子进程仍在运行，Emacs 也会请求确认，因为退出 Emacs 同时也会杀死这些子进程（参见“在 Emacs 中运行 Shell 命令”）。

如果你把 Emacs 用作服务器，‘C-x C-c’ 的行为会有所不同。若在客户端框架中键入它，它会关闭该客户端连接。参见“把 Emacs 用作服务器”。

Emacs 还可以选择在退出时记录某些会话信息，例如你当时正在访问的文件。这些信息在下次启动 Emacs 时即可使用。参见“保存 Emacs 会话”。

如果变量 ‘confirm-kill-emacs’ 的值非 ‘nil’，‘C-x C-c’ 就把它的值当作一个谓词函数并调用该函数。如果函数调用的结果非 ‘nil’，则结束会话，否则 Emacs 继续运行。可用作 ‘confirm-kill-emacs’ 值的一个方便函数是 ‘yes-or-no-p’。‘confirm-kill-emacs’ 的默认值为 ‘nil’。

如果变量 ‘confirm-kill-processes’ 的值为 ‘nil’，‘C-x C-c’ 在杀死由 Emacs 启动的子进程之前不会请求确认。该值默认为 ‘t’。

要进一步自定义 Emacs 退出时的行为，参见《Emacs Lisp 参考手册》的“Killing Emacs”。

若要在不被提示保存的情况下退出 Emacs，请键入 ‘M-x kill-emacs’。

‘C-z’ 运行命令 ‘suspend-frame’。在图形显示上，该命令会“最小化”（或“图标化”）选中的 Emacs 框架，以一种稍后可将其恢复的方式隐藏它（具体如何隐藏取决于窗口系统）。在文本终端上，‘C-z’ 命令会“挂起” Emacs，暂时停止该程序并把控制权交还给父进程（通常是 shell）；在大多数 shell 中，挂起后可以用 shell 命令 ‘%emacs’ 恢复 Emacs。

文本终端通常会监听某些特殊字符，其含义是杀死或挂起你正在运行的程序。当你处于 Emacs 中时，这一终端特性是关闭的。在 Emacs 中，‘C-z’ 和 ‘C-x C-c’ 作为按键的含义，是受到若干操作系统上用 ‘C-z’ 和 ‘C-c’ 来停止或杀死程序的用法的启发，但这是它们与操作系统唯一的关联。你可以自定义这些按键来运行你选择的任何命令（参见“键位图”）。

9 基本编辑命令

本节介绍输入文本、修改文本以及把文本存入文件的基础知识。如果你对这些内容还不熟悉，我们建议你先运行 Emacs 的边做边学教程，输入‘C-h t’（‘help-with-tutorial’）即可。

- 移动光标点：把光标移到你要改动的位置。
- 删除：删除（delete）与 kill 文本。
- 基本撤销：撤销文本中近来的修改。
- 文件（基本文件操作）：访问、创建和保存文件。
- 帮助（基本帮助）：查询某个字符有什么作用。
- 空行：生成和删除空行。
- 续行：Emacs 如何显示对屏幕来说过宽的行。
- 位置信息：光标点位于哪一行、哪一屏幕行或哪一列?
- 参数：让命令重复 N 次的数值参数。
- 重复：快速重复上一个命令。

9.1 插入文本

只要输入字符对应的键，就能插入普通的“图形字符”（graphic character），例如‘a’、‘B’、‘3’和‘=’。该字符会被加到缓冲区中光标点处。插入操作会使光标点前移，因此光标点停在新插入的文本之后。参见“光标点”。

要结束一行并开始新的一行，输入 <RET>（‘newline’）。（<RET> 键在你的键盘上可能标为 <Return> 或 <Enter>，也可能画成一个样子古怪的向左箭头，但本手册中一律称之为 <RET>。）该命令先向缓冲区插入一个换行字符，然后按照主模式进行缩进（参见“缩进”）。若光标点在行尾，效果就是在该行后面新建一个空行并对新行缩进；若光标点在行中间，则该行在此处被拆开。要关闭自动缩进，你可以禁用 Electric Indent 模式（参见“缩进的便利功能”），也可以改按‘C-j’，后者只插入换行，不做任何自动缩进。

正如本手册后文所述，你可以通过开启“次模式”来改变 Emacs 处理文本插入的方式。例如，名为 Auto Fill 模式的次模式会在行变得过长时自动断行（参见“填充文本”）。名为 Overwrite 模式的次模式则让插入的字符替换（覆盖）已有文本，而不是把已有文本向右挤。参见“次模式”。

只有图形字符才能通过输入其对应的键来插入；其他键都充当编辑命令，不会插入自身。例如，<DEL> 默认运行命令‘delete-backward-char’（某些模式把它绑定到别的命令）；它不会插入一个字面意义上的‘DEL’字符（ASCII 码 127）。

要插入非图形字符，或键盘不支持的字符，先对它进行“引用”：输入‘C-q’（‘quoted-insert’）。使用‘C-q’有两种方式：

- ‘C-q’后跟任意非图形字符（即使是‘C-g’）会插入该字符。例如，‘C-q <DEL>’插入一个字面的‘DEL’字符。
- ‘C-q’后跟一串八进制数字，会插入八进制字符码所指定的字符。八进制数字的个数不限；任何非数字字符都终止该序列。如果终止字符是 <RET>，则该 <RET> 只用于终止序列。其他非数字字符会终止序列，然

后照常作为输入起作用——因此，‘C-q 1 0 1 B’ 会插入 ‘AB’。

在普通的非二进制 Overwrite 模式下，八进制序列的用法被禁用，以便给你提供一种便捷的方式来插入数字，而不是用该数字覆盖原有文本。

要改用十进制或十六进制而非八进制，请把变量 ‘read-quoted-char-radix’ 设为 10 或 16。若基数为 16，则字母 ‘a’ 到 ‘f’ 与数字一样，可作为字符码的一部分。大小写不限。

少数常用 Unicode 字符可通过以 ‘C-x 8’ 开头的命令插入。例如，‘C-x 8 [’ 插入 ‘，即 Unicode 码位 U+2018 LEFT SINGLE QUOTATION MARK，有时称为左单“弯引号”（curved quote）或“花引号”（curly quote）。类似地，‘C-x 8]’、‘C-x 8 {’ 和 ‘C-x 8 }’ 分别插入弯引号 ’、“ 和 ”。此外，一个可用的 <Alt> 键的作用等同于 ‘C-x 8’（除非它后面跟 <RET>）；例如 ‘A-[’ 的作用等同于 ‘C-x 8 [’，会插入 ‘。要查看哪些字符有 ‘C-x 8’ 简写，输入 ‘C-x 8 C-h’。

另一种办法是使用命令 ‘C-x 8 <RET>’（‘insert-char’）。它通过迷你缓冲区提示输入字符的 Unicode 名称或码位。若输入名称，该命令会提供补全（参见“补全”）。若输入码位，应采用十六进制数（Unicode 的惯例），或带基数前缀的数，例如 ‘#o23072’（八进制）；参见《Emacs Lisp 参考手册》的“Integer Basics”。随后该命令把相应的字符插入缓冲区。

例如，下面几种做法插入的都是同一个字符：

```
C-x 8 <RET> left single quotation mark <RET>
C-x 8 <RET> left sin <TAB> <RET>
C-x 8 <RET> 2018 <RET>
C-x 8 [
A-[  (if the Alt key works)
`    (in Electric Quote mode)
```

给 ‘C-q’ 或 ‘C-x 8 …’ 一个数值参数，可以指定要插入该字符的多少份副本（参见“数字参数”）。

除了 ‘C-x 8’，你还可以输入 ‘C-u C-x \ iso-transl <RET>’ 选定对应的临时输入法，然后输入 ‘C-x \ [’ 临时激活该临时输入法，这也会插入同一个字符 ‘（参见“transient input method”）。

此外，在某些场合，如果你用重音符和撇号输入引语 ‘`like this''，即使不用 ‘C-x 8’ 命令，它也会被转换成使用单引号的 ‘like this’ 形式。类似地，用双重重音符和撇号输入的引语 ‘ `like this"’ 会被转换成使用双引号的 “like this” 形式。参见“引号”。

9.2 移动光标点的位置

要做的不只是插入字符，你就必须知道如何移动光标点（参见“光标点”）。键盘命令 ‘C-f’、‘C-b’、‘C-n’ 和 ‘C-p’ 分别把光标点向右、向左、向下和向上移动。你也可以用大多数键盘上有的“箭头键”移动光标点：<RIGHT>、<LEFT>、<DOWN> 和 <UP>；不过许多 Emacs 用户发现，用箭头键比用控制键慢，因为手得移到键盘上那些键所在的区域。

你还可以按一下鼠标左键，把光标点移到所点的位置。Emacs 另外还提供了多种更精巧的移动光标点的键盘命令。

- ‘C-f’：向前移动一个字符（‘forward-char’）。

- ‘<RIGHT>’：该命令（‘right-char’）的行为与‘C-f’相同，但当光标点位于从右到左的段落中时除外（参见“双向编辑”）。
- ‘C-b’：向后移动一个字符（‘backward-char’）。
- ‘<LEFT>’：该命令（‘left-char’）的行为与‘C-b’相同，但当前段落为从右到左时除外（参见“双向编辑”）。
- ‘C-n’‘<DOWN>’：向下移动一个屏幕行（‘next-line’）。该命令会尽量保持水平位置不变，所以若你从一行的中间出发，就会移到下一行的中间。
- ‘C-p’‘<UP>’：向上移动一个屏幕行（‘previous-line’）。该命令像‘C-n’一样保持行内的位置。
- ‘C-a’‘<Home>’：移到行首（‘move-beginning-of-line’）。
- ‘C-e’‘<End>’：移到行尾（‘move-end-of-line’）。
- ‘M-f’：向前移动一个单词（‘forward-word’）。参见“单词”。
- ‘C-<RIGHT>’‘M-<RIGHT>’：该命令（‘right-word’）的行为与‘M-f’相同，但若当前段落为从右到左，则改为向*后*移动一个单词。参见“双向编辑”。
- ‘M-b’：向后移动一个单词（‘backward-word’）。参见“单词”。
- ‘C-<LEFT>’‘M-<LEFT>’：该命令（‘left-word’）的行为与‘M-b’相同，但若当前段落为从右到左，则改为向*前*移动一个单词。参见“双向编辑”。
- ‘M-r’：不移动屏幕上的文本，把光标点重新放到窗口中最靠中间那行文本的左边界上；此后连续调用时，按循环次序依次把光标点移到最上面一行的左边界、最下面一行的左边界，如此继续（‘move-to-window-line-top-bottom’）。

数值参数指定把光标点放到哪个屏幕行，从窗口顶部向下数（零表示第一行）。负参数则从底部向上数（−1表示最后一行）。关于数值参数的更多信息，参见“数字参数”。

- ‘M-<’：移到缓冲区顶部（‘beginning-of-buffer’）。带数值参数 N 时，移到距顶部 N/10 处。在图形显示下，‘C-<HOME>’的作用相同。
- ‘M->’：移到缓冲区末尾（‘end-of-buffer’）。在图形显示下，‘C-<END>’的作用相同。
- ‘C-v’‘<PageDown>’‘<next>’：把显示向前滚动一屏，必要时把光标点移到屏幕内（‘scroll-up-command’）。参见“滚动”。
- ‘M-v’‘<PageUp>’‘<prior>’：向后滚动一屏，必要时把光标点移到屏幕内（‘scroll-down-command’）。参见“滚动”。
- ‘M-g c’：读取一个数 N，把光标点移到缓冲区位置 N。位置 1 是缓冲区开头。若光标点位于缓冲区中某个数字之上或紧接其后，该数字就是 N 的默认值，在迷你缓冲区中直接输入 <RET> 即可采用。你也可以给‘M-g c’一个数值前缀参数来指定 N。

- ‘M-g M-g’ ‘M-g g’：读取一个数 N，把光标点移到第 N 行的行首（‘goto-line’）。第 1 行是缓冲区开头。若光标点位于缓冲区中某个数字之上或紧接其后，该数字就是 N 的默认值，在迷你缓冲区中直接输入 <RET> 即可采用。你也可以给 ‘M-g M-g’ 一个数值前缀参数来指定 N。关于在给出无值前缀参数时 ‘M-g M-g’ 的行为，参见“创建与选择缓冲区”。另一种办法是使用命令 ‘goto-line-relative’，把光标点移到相对于窄化后缓冲区可访问部分的某一行。

‘goto-line’ 有自己的历史列表（*note Minibuffer History::）。通过自定义用户选项 ‘goto-line-history-local’，你可以让所有缓冲区共用一个列表（默认），也可以让每个缓冲区各有一个单独的列表。

- ‘M-g <TAB>’：读取一个数 N，移到当前行的第 N 列。第 0 列是最左一列。若带前缀参数调用，则移到该参数数值所指定的列号。

- ‘C-x C-n’：把光标点当前的列作为当前缓冲区中的“半永久目标列”（‘set-goal-column’）。半永久目标列生效时，‘C-n’、‘C-p’、‘<prior>’ 和 ‘<next>’ 在垂直移动之后总是尽量移到该列，或移到最接近该列的位置。目标列会一直有效，直到被取消。

- ‘C-u C-x C-n’：取消目标列。此后，‘C-n’ 和 ‘C-p’ 像往常一样尽量保持水平位置。

当缓冲区中的一行文本长于窗口宽度时，Emacs 通常把它显示为两个或更多“屏幕行”，也叫“视觉行”。为方便起见，‘C-n’ 和 ‘C-p’ 按屏幕行移动光标点，等效的 <down> 和 <up> 键也是如此。你可以通过把变量 ‘line-move-visual’ 设为 ‘nil’，强制这些命令按“逻辑行”（即按缓冲区中的文本行）移动；若一个逻辑行占据多个屏幕行，光标随后会跳过那些额外的屏幕行。详情参见“续行”。关于如何设置 ‘line-move-visual’ 这类变量，参见“变量”。

与 ‘C-n’ 和 ‘C-p’ 不同，Emacs 中大多数按行操作的命令都作用于*逻辑*行。例如，‘C-a’（‘move-beginning-of-line’）和 ‘C-e’（‘move-end-of-line’）分别移到逻辑行的行首和行尾。每当我们遇到 ‘C-n’ 和 ‘C-p’ 这类按屏幕行操作的命令时，都会特别指出这一点。

当 ‘line-move-visual’ 为 ‘nil’ 时，你还可以把变量 ‘track-eol’ 设为非 ‘nil’ 值。这样，若 ‘C-n’ 和 ‘C-p’ 从逻辑行行尾出发，就会移到下一个逻辑行的行尾。通常 ‘track-eol’ 为 ‘nil’。

当你在缓冲区最后一行上使用 ‘C-n’ 时，它通常停在缓冲区末尾。但是，若你把变量 ‘next-line-add-newlines’ 设为非 ‘nil’ 值，在缓冲区最后一行按 ‘C-n’ 会在末尾多造出一行，并向下移进去。

9.3 删除文本

- ‘<DEL>’ ‘<BACKSPACE>’：删除光标点之前的字符；若区域处于活动状态，则删除区域（‘delete-backward-char’）。

- ‘<Delete>’：删除光标点之后的字符或字素簇；若区域处于活动状态，则删除区域（‘delete-forward-char’）。

- ‘C-d’：删除光标点之后的字符（‘delete-char’）。

- ‘C-k’：删除（kill）到行尾（‘kill-line’）。

- ‘M-d’：向前删除（kill）到下一个单词的末尾（‘kill-word’）。

- ‘M-<DEL>’ ‘M-<BACKSPACE>’：向后删除（kill）到上一个单词的开头（‘backward-kill-word’）。

‘<DEL>’（‘delete-backward-char’）命令删除光标点之前的字符，光标及其后的字符随之向后移。若光标点位于行首，则删除前面的换行符，把本行接到上一行上。

但是，若区域处于活动状态，‘<DEL>’改为删除区域中的文本。关于区域的说明，参见“标记与区域”。

在大多数键盘上，<DEL> 标为 <BACKSPACE>，但本手册中一律称之为 <DEL>。（不要把 <DEL> 与 <Delete> 键混淆；我们马上就会讨论 <Delete>。）在某些文本终端上，Emacs 可能无法正确识别 <DEL> 键。如果你遇到这个问题，参见“若 <DEL> 无法删除”。

<Delete>（‘delete-forward-char’）命令朝相反方向删除：它删除光标点之后的字符，也就是光标底下的字符。若光标点位于行尾，这会把下一行接到本行上。与‘<DEL>’一样，若区域处于活动状态，它删除区域中的文本（参见“标记与区域”）。若光标点之后的字符与其后的字符组合在一起，显示为单个显示单元，即所谓代表整个序列的“字素簇”（grapheme cluster），则 <Delete> 一次性删除整个序列。这与 <DEL> 形成对照：<DEL> 总是删除单个字符，即使该字符是组合而成的。

‘C-d’（‘delete-char’）删除光标点之后的字符，与 <Delete> 类似，但不管区域是否处于活动状态。

关于上述删除命令的更详细信息，参见“基本删除”。

‘C-k’（‘kill-line’）一次擦除（删除）一行。若在行首或行中间输入 ‘C-k’，它会删除该行到行尾的全部文本。若在行尾输入 ‘C-k’，它会把该行与下一行连接起来。

关于 ‘C-k’ 及相关命令的更多信息，参见“删除与移动文本”。

9.4 撤销修改

- ‘C-/’ ‘C-x u’ ‘C-_’：撤销撤销记录中的一项——通常相当于一条命令的改动（‘undo’）。（第一个键在文本模式显示下可能不可用。）

Emacs 会记录对缓冲区文本所做的一系列修改，因此你可以撤销近来的改动。这通过 ‘undo’ 命令完成，它绑定到 ‘C-/’（以及 ‘C-x u’ 和 ‘C-_’）。通常，该命令撤销最后一次改动，并把光标点移回改动前所在的位置。撤销命令只作用于缓冲区中的改动；不能用它来撤销光标的移动。

在支持所有其他键上的 <Control> 修饰键的终端上，调用 ‘undo’ 最简便的方式是 ‘C-/’，因为它不需要 <Shift> 修饰键。在只允许 ASCII 控制字符的终端上，‘C-/’ 并不存在，但对其中许多终端来说 ‘C-/’ 仍然有效，因为它实际上向 Emacs 发送的是 *‘C-’；而另外许多终端允许你在输入 ‘C-’* 时省略 <Shift> 修饰键（实际按的是 ‘C--’），从而使这成为调用 ‘undo’ 最方便的方式。

尽管每条编辑命令通常都会在撤销记录中单独占一项，非常简单的命令也可能被合并在一起。有时，一项可能只覆盖一条复杂命令的一部分。

若重复按 ‘C-/’（或其别名），每重复一次就撤销更早的另一次改动，直到可用撤销信息的限度。若所有已记录的改动都已被撤销，撤销命令会显示一条错误消息，什么也不做。

要进一步了解 ‘undo’ 命令，参见“撤销”。

9.5 文件

你在 Emacs 缓冲区中插入的文本只在本次 Emacs 会话期间存在。要把任何文本永久保存，必须把它放进“文件”。

假设你的主目录中有一个名为 ‘test.emacs’ 的文件。要在 Emacs 中开始编辑该文件，输入：

```
C-x C-f test.emacs <RET>
```

这里文件名作为命令 ‘C-x C-f’（‘find-file’）的“参数”给出。该命令用“迷你缓冲区”读取参数，你输入 <RET> 来结束参数（参见“迷你缓冲区”）。

Emacs 通过“访问”该文件来执行这条命令：它创建一个缓冲区，把文件内容复制到缓冲区中，然后显示该缓冲区供你编辑。若你改动了文本，可以输入 ‘C-x C-s’（‘save-buffer’）把新文本“保存”到文件中。这会把改动后的缓冲区内容复制回文件 ‘test.emacs’，使其成为永久的。在保存之前，改动过的文本只存在于 Emacs 内部，文件 ‘test.emacs’ 并未改变。

要创建文件，只需用 ‘C-x C-f’ 访问它，就像它已经存在一样。这会创建一个空缓冲区，你可以在其中插入想要放进该文件的文本。Emacs 实际上是在你第一次用 ‘C-x C-s’ 保存该缓冲区时才创建这个文件。

要进一步了解在 Emacs 中使用文件，参见“文件处理”。

9.6 帮助

若你忘了某个键做什么，可以输入 ‘C-h k’（‘describe-key’），然后输入你关心的那个键来查明；例如 ‘C-h k C-n’ 会告诉你 ‘C-n’ 做什么。

前缀键 ‘C-h’ 代表“帮助”（help）。键 <F1> 是 ‘C-h’ 的别名。除了 ‘C-h k’，还有许多其他帮助命令，提供各种不同的帮助。

详情参见“帮助”。

9.7 空行

下面是插入和删除空行的一些专用命令和技巧。

- ‘C-o’：在光标之后插入一个空行（‘open-line’）。
- ‘C-x C-o’：把多个连续空行删到只剩一个（‘delete-blank-lines’）。

我们已经看到 ‘<RET>’（‘newline’）如何开始新的一行文本。不过，先造出一个空行再把想要的文本插进去，可能更容易看清自己在做什么。用 ‘C-o’（‘open-line’）键做这件事很方便：它在光标点之后插入一个换行，但把光标点留在换行之前。按过 ‘C-o’ 之后，再输入新行的文本。

输入几次 ‘C-o’，或者给它一个指定要造几个空行的数值参数，就能造出几个空行。具体做法参见“数字参数”。若你有填充前缀，且在行首输入，‘C-o’ 命令会在新行上插入该填充前缀。参见“填充前缀”。

去掉多余空行的简便方法是用命令 ‘C-x C-o’（‘delete-blank-lines’）。若光标点位于一连串空行之中，‘C-x C-o’ 会把它们删到只剩一个。若光标点单独位于一个空行上，‘C-x C-o’ 会删掉它。若光标点位于非空行上，‘C-x C-

o’ 会删除其后所有空行（如果有的话）。

9.8 续行

有时，缓冲区中的一行文本——一个“逻辑行”——太长，窗口装不下，Emacs 就把它显示为两个或更多的“屏幕行”，或称“视觉行”。这称为“行折绕”（line wrapping）或“续接”（continuation），而那行过长的逻辑行称为“续行”（continued line）。在图形显示下，Emacs 在窗口左右两侧的边缘（fringe）中用小的弯箭头表示折行。在文本终端上，Emacs 通过在最右侧显示一个 ‘\’ 字符来表示折行。

大多数按行操作的命令作用于逻辑行，而非屏幕行。例如，‘C-k’ 删除的是一个逻辑行。如前所述，‘C-n’（‘next-line’）和 ‘C-p’（‘previous-line’）是特殊的例外：它们分别向下和向上把光标点移动一个屏幕行（参见“移动光标点的位置”）。

Emacs 可以选择“截断”过长的逻辑行，而不是续接它们。这意味着每个逻辑行只占一个屏幕行；若它长于窗口宽度，该行的其余部分就不显示。在图形显示下，被截断的行由右侧边缘中的一个小直箭头标示；在文本终端上，由最右侧的一个 ‘$’ 字符标示。参见“行截断”。

默认情况下，续行在窗口右边缘处折绕。由于折绕可能发生在单词中间，续行可能不易阅读。通常的解决办法是在行变得过长之前插入换行符，把行断开。如果你愿意，可以使用 Auto Fill 模式，让 Emacs 在行变得过长时自动插入换行。参见“填充文本”。

通常，每个续行的第一个字符都位于其所在屏幕行的开头。次模式 ‘visual-wrap-prefix-mode’ 及其全局版本（参见“次模式”）‘global-visual-wrap-prefix-mode’ 会根据每一行周围的上下文自动计算出一个填充前缀（参见“填充前缀”），让续行在显示时依此缩进。这些前缀只是显示上的特性，不会以任何方式改变缓冲区文本。

有时，你可能需要编辑含有很多长逻辑行的文件，而用插入换行的方法把它们全部断开并不现实。这种情况下，你可以使用 Visual Line 模式，它启用“按词折行”：Emacs 不在窗口右边缘处精确折行，而是在最靠近窗口右边缘的单词边界（即空格或制表字符）处折行。Visual Line 模式还重新定义了 ‘C-a’、‘C-n’ 和 ‘C-k’ 等编辑命令，使它们作用于屏幕行而非逻辑行。参见“可视行模式”。

9.9 光标位置信息

下面是一些获取缓冲区各部分大小和位置信息、以及统计单词和行数的命令。

- ‘M-x what-line’：显示光标点所在的行号。
- ‘M-x line-number-mode’ ‘M-x column-number-mode’：开关当前行号或列号的自动显示。参见“可选的模式行功能”。若你想在每一行前面显示行号，参见“显示的自定义”。

‘M-=’显示区域中存在的行数、句子数、单词数和字符数（‘count-words-region’）。关于区域的信息，参见“标记与区域”。

‘M-x count-words’显示缓冲区中存在的行数、句子数、单词数和字符数。若区域处于活动状态（参见“标记与区域”），则改为显示该区域的这些数字。

‘C-x =’显示光标点之后字符的字符编码、光标点的字符位置以及光标点所在的列（‘what-cursor-position’）。‘M-x hl-line-mode’启用或禁用当前行的高亮（参见“光标的显示”）。‘M-x size-indication-mode’切换

是否自动显示缓冲区的大小（参见“可选的模式行功能”）。

‘M-x what-line’在回显区显示当前行号。此命令通常是多余的，因为模式行中已经显示了当前行号（参见“模式行”）。不过，若你窄化了缓冲区，模式行显示的是相对于可访问部分的行号（参见“窄化”）。相比之下，‘what-line’既显示相对于窄化区域的行号，也显示相对于整个缓冲区的行号。

‘M-=’（‘count-words-region’）显示一条消息，报告区域中的行数、句子数、单词数和字符数（区域的含义参见“标记与区域”）。带前缀参数时，‘C-u M-=’，该命令显示整个缓冲区的计数。

命令‘M-x count-words’完成同样的工作，但调用约定不同。若区域处于活动状态，它显示区域的计数，否则显示缓冲区的计数。

命令‘C-x =’（‘what-cursor-position’）显示关于当前光标位置以及该位置处缓冲区内容的信息。它在回显区显示一行，形如：

```
Char: c (99, #o143, #x63) point=28062 of 36168 (78%) column=53
```

在‘Char:’之后，显示的是光标点处缓冲区中的字符。括号内的文本给出相应的十进制、八进制和十六进制字符编码；关于‘C-x =’如何显示字符信息的更多信息，参见“国际字符集简介”。‘point=’之后是光标点的位置，以字符计数表示（缓冲区中的第一个字符位置为 1，第二个字符位置为 2，依此类推）。其后的数字是缓冲区中的总字符数，括号中的数字以占总数的百分比表示该位置。‘column=’之后是光标点的水平位置，以从窗口左边缘起算的列数表示。

若用户选项‘what-cursor-show-names’为非‘nil’，则还会显示字符的名称，该名称由 Unicode Character Database 定义。此时括号中的部分将变为：

```
(99, #o143, #x63, LATIN SMALL LETTER C)
```

若缓冲区已被窄化，使得开头和结尾的某些文本暂时不可访问，则‘C-x =’会显示额外的文本，描述当前可访问的范围。例如，它可能显示：

```
Char: C (67, #o103, #x43) point=252 of 889 (28%) <231-599> column=0
```

其中多出的两个数字给出光标点允许取的最小和最大字符位置。这两个位置之间的字符就是可访问的字符。参见“窄化”。

一个相关但不同的特性是‘display-line-numbers-mode’（参见“显示的自定义”）。

9.10 数字参数

在数学和计算的术语中，“参数”（argument）指“提供给函数或操作的数据”。你可以给任何 Emacs 命令一个“数字参数”（numeric argument，也称为“前缀参数”，prefix argument）。有些命令把该参数解释为重复次数。例如，给‘C-f’一个参数 10，会使它将光标点向前移动 10 个字符而不是 1 个。对于这些命令，不给参数等同于参数为 1，而负参数使它们向相反方向移动或动作。

指定数字参数最容易的方法是在按住 <Meta> 键的同时输入数字和/或减号。例如，

```
M-5 C-n
```

向下移动五行。‘M-1’、‘M-2’ 等键以及 ‘M--’ 都绑定到命令（‘digit-argument’ 和 ‘negative-argument’），这些命令为下一个命令设置参数。不带数字的‘M--’通常表示 −1。

若你输入不止一位数字，第二位及以后的数字不必按住 <Meta> 键。因此，要向下移动五十行，输入

```
M-5 0 C-n
```

注意这*不会*插入五个‘0’并向下移动一行，如你可能预期的那样——‘0’被当作前缀参数的一部分。

（如果你确实想插入五个‘0’呢？输入‘M-5 C-u 0’。这里‘C-u’终止了前缀参数，因此下一个按键开始你想要执行的命令。注意‘C-u’的这种含义仅适用于此情形。关于‘C-u’的通常作用，见下文。）

除了输入‘M-1’、‘M-2’ 等之外，指定数字参数的另一种方法是输入‘C-u’（‘universal-argument’），后跟一些数字，或者（对于负参数）一个减号后跟数字。不带数字的减号通常表示 −1。

单独的‘C-u’有“四次”的特殊含义：它把下一个命令的参数乘以四。‘C-u C-u’将其乘以十六。因此，‘C-u C-u C-f’向前移动十六个字符。其他有用的组合有‘C-u C-n’、‘C-u C-u C-n’（向下移动大半个屏幕）、‘C-u C-u C-o’（生成十六个空行）和‘C-u C-k’（删除四行）。

你可以在自插入字符之前使用数字参数，以插入该字符的多个副本。当该字符不是数字时，这很直接；例如，‘C-u 6 4 a’插入 64 个字符‘a’。但对插入数字则不起作用；‘C-u 6 4 1’指定的是参数 641。你可以用另一个‘C-u’把参数与要插入的数字分开；例如，‘C-u 6 4 C-u 1’确实会插入 64 个字符‘1’。

有些命令关心是否有参数，但忽略其值。例如，命令‘M-q’（‘fill-paragraph’）填充文本；带参数时，它还会对文本进行两端对齐。（关于‘M-q’的更多信息，参见“填充文本”。）对于这些命令，只需用单个‘C-u’指定参数即可。

有些命令把参数值用作重复次数，但在没有参数时做些特殊的事。例如，命令‘C-k’（‘kill-line’）带参数 N 时删除 N 行，包括行尾的换行符。但不带参数的‘C-k’是特殊的：它删除直到下一个换行符之前的文本，或者，若光标点正好位于行尾，则删除换行符本身。因此，两次不带参数的‘C-k’命令可以删除一个非空行，就像带参数 1 的‘C-k’一样。（关于‘C-k’的更多信息，参见“删除与移动文本”。）

少数命令对单纯的‘C-u’与对普通参数的处理不同。还有少数命令可能对仅为一个减号的参数与对 −1 的处理不同。这些不寻常的情况会在出现时说明；它们的存在是为了让单个命令更方便，并记录在该命令的文档字符串中。

我们用“前缀参数”这一术语，是为了强调你在命令*之前*输入这类参数，并为了把它们与迷你缓冲区参数（参见“迷你缓冲区”）区分开来，后者是在调用命令*之后*输入的。

在图形显示上，‘C-0’、‘C-1’ 等的作用与 ‘M-0’、‘M-1’ 等相同。

9.11 重复执行命令

许多简单命令，例如用单个键或用‘M-x COMMAND-NAME <RET>’调用的命令，可以通过带一个用作重复次数的数字参数调用它们来重复执行（参见“数字参数”）。然而，如果你要重复的命令会提示输入，或以其他方式使用数字参数，那么该方法就不起作用。

命令‘C-x z’（‘repeat’）提供了另一种多次重复执行 Emacs 命令的方法。该命令重复上一个 Emacs 命令，无论它是什么。重复命令使用之前所用的相同参数；它不会每次读取新的参数。

要多次重复该命令，请输入额外的‘z’：每个‘z’再重复该命令一次。当你输入‘z’以外的字符或按下鼠标按钮时，重复结束。

例如，假设你输入‘C-u 2 0 C-d’以删除 20 个字符。你可以通过输入‘C-x z z z’把该命令（包括其参数）再重复三次，共删除 80 个字符。第一个‘C-x z’重复该命令一次，随后每个‘z’再重复一次。

你还可以启用‘repeat-mode’，它允许通过输入单个字符来重复绑定到两个或更多键序列的命令。例如，在输入‘C-x u’（‘undo’，参见“撤销”）撤销最近的编辑之后，你可以通过输入‘u u u...’撤销更多编辑。类似地，输入‘C-x o o o...’而不是‘C-x o C-x o C-x o...’，以切换到相隔几个窗口的窗口。其工作方式是：在你输入调用该命令的完整键序列之后，进入一个临时的重复模式；单键快捷方式会显示在回显区中。

只有部分命令在‘repeat-mode’中支持重复；输入‘M-x describe-repeat-maps <RET>’可查看是哪些。

由临时重复模式启用的单字符快捷方式不必完全相同：例如，在输入‘C-x {’之后，‘{’或‘}’或‘^’或‘v’，或以任意顺序混合这些字符的任意序列，都会以各自的方式调整所选窗口的大小。类似地，在‘M-g n’或‘M-g p’之后，以任意混合方式输入由‘n’和/或‘p’组成的任意序列，都会重复‘next-error’和‘previous-error’，以在‘*compilation*’或‘*grep*’缓冲区中导航（参见“编译模式”）。

输入除定义为重复上一命令的键之外的任何键都会退出临时重复模式，然后你输入的键会正常执行。你还可以定义一个键，它退出临时重复模式而*不*执行导致退出操作的那个键。为此，定制用户选项‘repeat-exit-key’以指定一个键；一个自然的值是 <RET>。最后，可以在空闲一段时间后自动打断重复链：定制用户选项‘repeat-exit-timeout’以指定空闲时间（秒），超过该时间后此临时重复模式将自动关闭。

10 迷你缓冲区

“迷你缓冲区”（minibuffer）是 Emacs 命令读取复杂参数的地方，例如文件名、缓冲区名、Emacs 命令名或 Lisp 表达式。我们称它为“迷你缓冲区”，因为它是只占用少量屏幕空间的专用缓冲区。你可以在迷你缓冲区中使用通常的 Emacs 编辑命令来编辑参数文本。

10.1 使用迷你缓冲区

当迷你缓冲区在使用中时，它显示在回显区里，并带有一个光标。迷你缓冲区以一个“提示符”（prompt）开头，通常以冒号结尾。提示符说明期望何种输入，以及它将如何被使用。提示符用 ‘minibuffer-prompt’ 字体（face）高亮显示（参见“文本字体”）。

输入迷你缓冲区参数的最简单方法是键入文本，然后按 <RET> 提交参数并退出迷你缓冲区。或者，你可以键入 ‘C-g’ 取消索要参数的命令，从而退出迷你缓冲区（参见“退出与中止”）。

有时，提示符会在冒号前的括号中显示“默认参数”（default argument）。如果你直接键入 <RET>，就会把这个默认值用作参数。例如，读取缓冲区名的命令通常显示一个缓冲区名作为默认值；你可以键入 <RET> 对该默认缓冲区进行操作。你可以用用户选项 ‘minibuffer-default-prompt-format’ 自定义默认参数的显示方式。

如果你启用 Minibuffer Electric Default 模式（一个全局次模式），那么一旦你修改迷你缓冲区的内容，Emacs 就会隐藏默认参数（因为此时键入 <RET> 已不会再提交该默认值）。如果你又恢复原来的迷你缓冲区文本，提示符会再次显示默认值。要启用这个次模式，请键入 ‘M-x minibuffer-electric-default-mode’。

由于迷你缓冲区显示在回显区中，它可能与其他使用回显区的场合冲突。如果在迷你缓冲区活动时产生错误消息或提示性消息，该消息会以方括号的形式显示在迷你缓冲区文本之后几秒钟，或直到你键入内容为止；随后消息消失。迷你缓冲区在使用期间，Emacs 不回显按键。

使用迷你缓冲区时，你可以切换到另一个框架，比如去记下需要输入的文本（参见“框架命令”）。默认情况下，活动的迷你缓冲区会移到这个新框架中。如果你把用户选项 ‘minibuffer-follows-selected-frame’ 设为 ‘nil’，那么迷你缓冲区会留在你打开它的那个框架中，你必须切换回那个框架才能完成（或中止）当前命令。如果你把该选项设为既不是 ‘nil’ 也不是 ‘t’ 的值，那么只有当前命令中已经打开过递归迷你缓冲区之后，迷你缓冲区才会移动框架（参见《Emacs Lisp 参考手册》的“Recursive Mini”）。这个选项主要是为了（大致）保留 Emacs 28.1 之前的行为。请注意，当你最终结束使用迷你缓冲区时，命令的效果总是在你最初打开它的框架中生效。唯一的例外是：当那个框架已不存在时，效果在当前选中的框架中生效。

10.2 用于文件名的迷你缓冲区

像 ‘C-x C-f’（‘find-file’）这样的命令用迷你缓冲区读取文件名参数（参见“文件”）。当用迷你缓冲区读取文件名时，它通常以一些以斜杠结尾的初始文本开头。这就是“默认目录”（default directory）。例如，它可能这样开头：

```
Find file: /u2/emacs/src/
```

这里，‘Find file: ’ 是提示符，‘/u2/emacs/src/’ 是默认目录。如果你现在键入 ‘buffer.c’ 作为输入，那就指定了文件 ‘/u2/emacs/src/buffer.c’。关于默认目录的信息，参见“文件名”。

键入 ‘M-n’ 可以得到你可能想要的文件名的其他默认值，参见“迷你缓冲区历史”。

你可以用 ‘..’ 指定上级目录中的文件：‘/a/b/../foo.el’ 等价于 ‘/a/foo.el’。或者，你可以用 ‘M-<DEL>’ 向前删除目录名（参见“单词”）。

要指定位于完全不同目录中的文件，你可以用 ‘C-a C-k’ 删除整个默认目录（参见“在迷你缓冲区中编辑”）。或者，你可以忽略默认目录，在默认目录之后输入一个以斜杠或波浪号开头的绝对文件名。例如，你可以如下指定 ‘/etc/termcap’：

```
Find file: /u2/emacs/src//etc/termcap
```

双斜杠使 Emacs 忽略这一对斜杠中第二个斜杠之前的所有内容。在上面的例子中，‘/u2/emacs/src/’ 被忽略，因此你提供的参数是 ‘/etc/termcap’。如果终端允许，文件名中被忽略的部分会变暗显示。（要禁用这种变暗效果，请用命令 ‘M-x file-name-shadow-mode’ 关闭 File Name Shadow 模式。）

在补全远程文件名时（参见“远程文件”），双斜杠的行为略有不同：它只让 Emacs 忽略文件名部分，其余部分（方法、主机和用户名等）保持不变。连续键入三个斜杠则会忽略远程文件名中的所有内容。参见《Tramp 手册》的“File name completion”。

Emacs 把 ‘~/’ 解释为你的主目录。因此，‘~/foo/bar.txt’ 指定名为 ‘bar.txt’ 的文件，它位于名为 ‘foo’ 的目录中，而该目录又位于你的主目录中。此外，‘~USER-ID/’ 表示登录名为 USER-ID 的用户的主目录。‘~’ 前面的任何目录名都被忽略：因此，‘/u2/emacs/~/foo/bar.txt’ 等价于 ‘~/foo/bar.txt’。

在 MS-Windows 和 MS-DOS 系统上，用户并不总是有主目录，Emacs 会使用若干替代方案。对于 MS-Windows，参见“MS-Windows 上的 HOME 与启动目录”；对于 MS-DOS，参见“MS-DOS 上的文件名”。在这些系统上，‘~USER-ID/’ 这种写法只对当前用户支持，也就是说，只有当 USER-ID 是当前用户的登录名时才支持。

要阻止 Emacs 在读取文件名时插入默认目录，请把变量 ‘insert-default-directory’ 改为 ‘nil’。在这种情况下，迷你缓冲区一开始是空的。尽管如此，相对文件名参数仍按同一个默认目录来解释。

你也可以在迷你缓冲区中输入远程文件名。参见“远程文件”。

10.3 在迷你缓冲区中编辑

迷你缓冲区是一个 Emacs 缓冲区，尽管它比较特殊，通常的 Emacs 命令都可以用来编辑参数文本。（不过提示符是“只读”的，不能修改。）

由于迷你缓冲区中的 <RET> 用于提交参数，你不能用它插入换行符。你可以用 ‘C-q C-j’ 做到这一点，它会插入一个 ‘C-j’ 控制字符，该字符形式上等价于换行字符（参见“插入文本”）。或者，你可以使用 ‘C-o’（‘open-line’）命令（参见“空行”）。

在迷你缓冲区内部，<TAB>、<SPC> 和 ‘?’ 这些键常常绑定到“补全命令”，它们让你不必键入全部内容就能轻松填入所需文本。参见“补全”。与 <RET> 一样，你可以用 ‘C-q’ 插入 <TAB>、<SPC> 或 ‘?’ 字符。如果你想让 <SPC> 和 <?> 正常插入而不启动补全，可以在初始化文件中加入以下内容：

```
(keymap-unset minibuffer-local-completion-map "SPC")
(keymap-unset minibuffer-local-completion-map "?")
```

为方便起见，‘C-a’（‘move-beginning-of-line’）在迷你缓冲区中把光标点移到参数文本的开头，而不是提示符的开头。例如，这让你可以用 ‘C-a C-k’ 删除整个参数。

当迷你缓冲区活动时，回显区的处理方式很像普通的 Emacs 窗口。例如，你可以切换到另一个窗口（用 ‘C-x o’），在那里编辑文本，然后回到迷你缓冲区窗口完成参数。你甚至可以在另一个窗口中删除文本，回到迷你缓冲区窗口，再把文本粘贴到参数中。不过，对迷你缓冲区窗口有一些限制：例如，你不能拆分它。参见“多窗口”。

通常，迷你缓冲区窗口只占一行屏幕。然而，如果你向迷你缓冲区中加入两行或更多行文本，它会自动扩展以容纳这些文本。变量 ‘resize-mini-windows’ 控制迷你缓冲区的尺寸调整。默认值是 ‘grow-only’，即我们刚才描述的行为。如果值为 ‘t’，那么当你从迷你缓冲区中删掉一些行时，迷你缓冲区窗口也会自动缩小，最小到一行屏幕。如果值为 ‘nil’，迷你缓冲区窗口从不自动改变大小，但你可以对它使用通常的窗口缩放命令（参见“多窗口”）。

变量 ‘max-mini-window-height’ 控制迷你缓冲区窗口调整大小的最大高度。浮点数指定框架高度的一个比例；整数指定最大行数；‘nil’ 表示不自动调整迷你缓冲区窗口大小。默认值是 0.25。

迷你缓冲区中的 ‘C-M-v’ 命令滚动显示帮助文本，这些帮助文本来自在另一个窗口中显示某种帮助文本的命令。你也可以用 ‘M-<PageUp>’ 和 ‘M-<PageDown>’（或者等价的 ‘M-<prior>’ 和 ‘M-<next>’）滚动帮助文本。这在处理很长的可能补全项列表时特别有用。参见“使用其他窗口”。

Emacs 通常不允许在迷你缓冲区活动时使用大多数会用到迷你缓冲区的命令。要允许在迷你缓冲区中使用这类命令，请把变量 ‘enable-recursive-minibuffers’ 设为 ‘t’。你可能还需要启用 ‘minibuffer-depth-indicate-mode’，以便在递归使用迷你缓冲区时在迷你缓冲区提示符中显示当前递归深度。

活动时，迷你缓冲区通常处于 ‘minibuffer-mode’ 中。这是一个内部的 Emacs 模式，没有任何特殊功能。

不活动时，迷你缓冲区处于 ‘minibuffer-inactive-mode’ 中，在那里点击 ‘mouse-1’ 会显示 ‘*Messages*’ 缓冲区。如果你使用专用的迷你缓冲区框架，Emacs 还会识别那里的一些按键，例如 ‘n’ 用来创建新框架。

10.4 补全

你常常可以用一种称为“补全”（completion）的功能来帮助输入参数。这意味着，在你键入参数的一部分之后，Emacs 可以根据已键入的内容填入其余部分或其中一部分。

当补全可用时，某些键（通常是 <TAB>、<RET> 和 <SPC>）会在迷你缓冲区中被重新绑定到特殊的补全命令（参见“补全命令”）。这些命令尝试根据索要参数的命令所提供的一组“补全候选项”（completion alternatives）来补全迷你缓冲区中的文本。你通常可以键入 ‘?’ 让 Emacs 弹出一个缓冲区（名为 ‘*Completions*’），显示相关的补全候选项。如下文更详细的描述，你可以浏览这个缓冲区并从中选择想要的补全项。

虽然补全通常在迷你缓冲区中进行，但这一功能有时也可在普通缓冲区中使用。参见“符号名补全”。

10.4.1 补全示例

一个简单的例子可能有所帮助。‘M-x’ 用迷你缓冲区读取命令名，因此补全的工作方式是把迷你缓冲区文本与现有 Emacs 命令的名字进行匹配。假设你想运行命令 ‘auto-fill-mode’。你可以键入 ‘M-x auto-fill-mode <RET>’ 来做到，但使用补全更方便。

如果你键入 ‘M-x a u <TAB>’，<TAB> 会寻找以 ‘au’ 开头的补全候选项（在本例中是命令名）。这样的候选项有好几个，包括 ‘auto-fill-mode’ 和 ‘autoconf-mode’，但它们都以 ‘auto’ 开头，所以迷你缓冲区中的 ‘au’ 补全为 ‘auto’。（你的 Emacs 会话中可能定义了更多命令。例如，如果定义了一个名为 ‘authorize-me’ 的命令，Emacs 就只能补全到 ‘aut’。）

如果你马上再次键入 ‘<TAB>’，它无法确定下一个字符；那可能是 ‘-’、‘a’ 或 ‘c’。因此它不添加任何字符；相反，<TAB> 在另一个窗口中显示所有可能的补全项列表。

接下来，键入 ‘-f’。现在迷你缓冲区中包含 ‘auto-f’，以此开头的命令名只有 ‘auto-fill-mode’。如果你现在键入 ‘<TAB>’，补全会把参数的其余部分 ‘auto-fill-mode’ 填入迷你缓冲区。

因此，只键入 ‘a u <TAB> - f <TAB>’ 就能让你输入 ‘auto-fill-mode’。

<TAB> 在光标点不在迷你缓冲区末尾时也能工作。在这种情况下，它会同时在光标点处和迷你缓冲区末尾填入文本。如果你键入 ‘M-x autocm’，然后按 ‘C-b’ 把光标点移到 ‘m’ 之前，你可以键入 ‘<TAB>’ 在光标点处插入文本 ‘onf-’，并在迷你缓冲区末尾插入 ‘ode’，从而使迷你缓冲区包含 ‘autoconf-mode’。

10.4.2 补全命令

下面是补全可用时在迷你缓冲区中或补全缓冲区中定义的补全命令列表。

- ‘<TAB>’ 尽可能补全迷你缓冲区中的文本；若无法补全，则显示可能的补全项列表。
- ‘<SPC>’ 补全到光标点之前迷你缓冲区文本中的下一个单词为止。
- ‘C-x <UP>’ 使用迷你缓冲区历史补全迷你缓冲区中的文本。
- ‘C-x <DOWN>’ 使用迷你缓冲区默认值补全迷你缓冲区中的文本。
- ‘<RET>’ 把迷你缓冲区中的文本作为参数提交，可能先进行补全。参见“补全退出”。
- ‘?’ 显示补全项列表和几个有用的键绑定（‘minibuffer-completion-help’）。
- ‘M-<DOWN>’ ‘M-<UP>’ 在补全项列表中导航。
- ‘M-v’ ‘M-g M-c’ ‘<PageUp>’ ‘<prior>’ 在迷你缓冲区中时，选择显示补全列表的窗口。
- ‘<RET>’ 在补全缓冲区中，选择光标点处的补全项。
- ‘mouse-1’ ‘mouse-2’ 在补全缓冲区中，选择鼠标点击处的补全项。
- ‘<TAB>’ ‘n’ 在补全缓冲区中，移到下一个补全候选。
- ‘S-<TAB>’ ‘p’ 在补全缓冲区中，移到上一个补全候选。
- ‘<RIGHT>’ ‘<LEFT>’ 在补全缓冲区中，按列在补全列表中导航。
- ‘<DOWN>’ ‘<UP>’ 在补全缓冲区中，按行在补全列表中导航。
- ‘q’ 退出补全窗口并切换到迷你缓冲区窗口。
- ‘z’ 删除补全缓冲区并删除显示它的窗口。

<TAB>（‘minibuffer-complete’）是最基本的补全命令。它搜索所有与现有迷你缓冲区文本匹配的可能补全项，并尽可能多地补全。关于补全候选项如何选取，参见“补全候选项如何选定”。

<SPC>（‘minibuffer-complete-word’）像 <TAB> 一样补全，但只补到下一个连字符或空格为止。如果你的迷你缓冲区中有 ‘auto-f’，然后键入 <SPC>，它发现补全项是 ‘auto-fill-mode’，但只插入 ‘ill-’，得到 ‘auto-fill-’。此时再按一次 <SPC> 就会一路补全到 ‘auto-fill-mode’。对于经常包含空格的参数（例如文件名），此命令不可用。

如果 <TAB> 或 <SPC> 无法补全，它会在另一个窗口中显示匹配的补全候选项列表（如果有的话）以及几个用于选择补全候选的有用命令。你可以用 ‘?’（‘minibuffer-completion-help’）显示同样的补全列表和帮助。下列命令可用于补全列表：

在迷你缓冲区或补全列表缓冲区中时，‘M-<DOWN>’（‘minibuffer-next-completion’）和 ‘M-<UP>’（‘minibuffer-previous-completion’）在补全缓冲区中显示的补全项之间导航。这些命令对补全列表的格式敏感：当 ‘completions-format’ 的值为 ‘horizontal’ 时它们按列移动光标点，当其值为 ‘vertical’ 时按行移动（参见“补全选项”）。如果你把 ‘minibuffer-completion-auto-choose’ 设为非 ‘nil’，使用这些命令还会把当前补全候选插入迷你缓冲区。你可以用 ‘M-<RET>’（‘minibuffer-choose-completion’）或 ‘<RET>’（‘minibuffer-complete-and-exit’）选择选中的补全候选。默认情况下，这些命令会退出迷你缓冲区，但带前缀参数时（即 ‘C-u M-<RET>’ 或 ‘C-u <RET>’），它们会把当前选中的候选插入迷你缓冲区而不退出。

在迷你缓冲区中键入 ‘M-v’ 会选择显示补全列表的窗口（‘switch-to-completions’）。这为使用下面的命令铺平了道路。<PageUp>、<prior> 和 ‘M-g M-c’ 做同样的事。你也可以用其他方式选择窗口（参见“多窗口”）。

在补全列表缓冲区中时，‘<RET>’ 选择光标点处的补全候选（‘choose-completion’），而 ‘mouse-1’ 和 ‘mouse-2’ 选择鼠标点击处的补全项。带前缀参数时，‘C-u <RET>’ 把光标点处的补全项插入迷你缓冲区，但不退出迷你缓冲区——这样你可以改变主意并选择另一个候选。

在补全列表缓冲区中时，你可以用 ‘<TAB>’ 或 ‘n’ 把光标点移到下一个补全候选（‘next-completion’），用 ‘S-<TAB>’ 或 ‘p’ 把光标点移到上一个补全候选（‘previous-completion’）。与 ‘M-<DOWN>’ 和 ‘M-<UP>’ 一样，这些命令也对补全列表格式敏感。

相比之下，‘<RIGHT>’（‘next-column-completion’）和 ‘<LEFT>’（‘previous-column-completion’）无论补全列表格式如何，总是按列移动光标点。同样，‘<DOWN>’（‘next-line-completion’）和 ‘<UP>’（‘previous-line-completion’）总是按行移动光标点。

所有这些移动命令（以及 ‘M-<DOWN>’ 和 ‘M-<UP>’）都接受数字前缀参数 N，它使光标点移到后面或前面第 N 个补全候选。

你也可以使用为提示你的那个命令累积的迷你缓冲区输入历史来进行补全。‘C-x <UP>’（‘minibuffer-complete-history’）像 ‘<TAB>’ 一样工作，但它使用迷你缓冲区历史而不是通常的补全候选项进行补全。类似的命令 ‘C-x <DOWN>’（‘minibuffer-complete-defaults’）使用提示命令提供的默认输入项进行补全。

最后，‘q’ 退出显示它的窗口并选择显示迷你缓冲区的窗口（‘quit-window’），而 ‘z’ 删除补全缓冲区并删除显示它的窗口（‘kill-current-buffer’）。

如果把变量 ‘minibuffer-visible-completions’ 定制为值 ‘t’，它会改变绑定到方向键的命令：这些键不再在迷你缓冲区中移动，而是在补全候选之间移动，如同默认情况下元键加方向键所做的那样（但请注意，正如选中显示

补全列表的窗口时一样，这里‘<RIGHT>’和‘<LEFT>’也只按列移动光标点，‘<DOWN>’和‘<UP>’也只按行移动光标点，无论补全列表格式如何）。同样，‘<RET>’会像通常的‘M-<RET>’那样选择当前候选。‘C-g’隐藏补全窗口，但保持迷你缓冲区活动，因此你可以继续在提示符处键入。如果此变量的值为‘up-down’，则只有‘<UP>’和‘<DOWN>’方向键在补全候选之间移动光标点，而‘<RIGHT>’和‘<LEFT>’在迷你缓冲区窗口中移动光标点。

10.4.3 补全退出

当命令使用带补全的迷你缓冲区读取参数时，它也控制键入 <RET>（‘minibuffer-complete-and-exit’）提交参数时会发生什么。行为有四种：

- “严格补全”（strict completion）只接受精确的补全匹配。只有当迷你缓冲区文本是精确匹配、或能补全成一个精确匹配时，键入 <RET> 才会退出迷你缓冲区。否则 Emacs 拒绝退出迷你缓冲区；它转而尝试补全，如果无法进行任何补全，就在迷你缓冲区文本后短暂显示‘[No match]’。（你仍然可以键入‘C-g’取消命令来离开迷你缓冲区。）

使用这种行为的一个命令例子是‘M-x’，因为让它接受一个不存在的命令名是没有意义的。

- “谨慎补全”（cautious completion）与严格补全类似，只是仅当文本已经是精确匹配时 <RET> 才退出。如果文本能补全为一个精确匹配，<RET> 会执行该补全但还不退出；你必须再键入一次 <RET> 才能退出。

例如，读取那些必须已经存在的文件的文件名时就使用谨慎补全。

- “宽松补全”（permissive completion）允许任何输入；补全候选项只是建议。键入 <RET> 不会补全，它只是按你输入的样子提交参数。

- “带确认的宽松补全”（permissive completion with confirmation）与宽松补全类似，但有一个例外：如果你键入了 <TAB> 且它把文本补全到某种中间状态（即尚不是精确补全匹配的状态），紧接着键入 <RET> 不会提交参数。相反，Emacs 会在文本后短暂显示‘[Confirm]’来请求确认；再次键入 <RET> 以确认并提交文本。这能抓住一个常见错误：在还没意识到 <TAB> 没有补全到想要的程度之前就键入了 <RET>。

你可以通过定制变量‘confirm-nonexistent-file-or-buffer’来调整确认行为。默认值‘after-completion’给出我们刚才描述的行为。如果你把它改为‘nil’，Emacs 不再请求确认，退回宽松补全。如果你把它改为任何其他非‘nil’值，那么无论前一个命令是否为 <TAB>，Emacs 都会请求确认。

读取文件名的大多数命令（如‘C-x C-f’）和读取缓冲区名的命令（如‘C-x b’）都使用这种行为。

10.4.4 补全候选项如何选定

补全命令的工作方式是：把一份庞大的可能补全候选项列表缩小为与你在迷你缓冲区中键入的内容相匹配的较小子集。在参见“补全示例”中，我们给出了这种匹配的一个简单例子。确定什么算作匹配的过程相当复杂。Emacs 在大多数情况下都试图提供合理的补全项。

Emacs 使用一种或多种“补全风格”（completion style）来执行补全——补全风格是把迷你缓冲区文本与补全候选项匹配的一组标准。补全过程中，Emacs 依次尝试每种补全风格。如果某种风格产生一个或多个匹配，就把它用作补全候选项列表。如果某种风格没有产生匹配，Emacs 就退回到下一种风格。

列表变量 'completion-styles' 指定要使用的补全风格。每个列表元素要么是补全风格的名称（一个 Lisp 符号），要么是一个以补全风格名称开头、后跟 'let' 风格的绑定列表的列表，这些绑定将在该补全风格中生效。'completion-styles' 的多个元素可以用不同的变量绑定指定同一种补全风格。

可用的风格符号存放在变量 'completion-styles-alist' 中（参见《Emacs Lisp 参考手册》的"Completion Variables"）。默认的补全风格（按顺序）是：

- 'basic' 匹配的补全候选项必须与光标点之前迷你缓冲区中的文本具有相同的开头。此外，如果光标点之后的迷你缓冲区中还有文本，补全候选项的其余部分必须包含该文本作为子串。
- 'partial-completion' 这种激进的补全风格把迷你缓冲区文本按连字符或空格分成单词，并分别补全每个单词。（例如，在补全命令名时，'em-l-m' 补全为 'emacs-lisp-mode'。）

此外，迷你缓冲区文本中的 '*' 被当作"通配符"——它匹配补全候选项中对应位置上的任意字符串。

若 'completion-pcm-leading-wildcard' 被设为 't'，此风格总是表现得如同迷你缓冲区文本中每个词的开头都存在一个"通配符"，与 'substring' 风格类似。例如，'l-ode' 将补全为 'emacs-lisp-mode'。

'emacs22' 此补全风格与 'basic' 类似，只是它忽略迷你缓冲区中光标点之后的文本。它之所以这样命名，是因为它对应 Emacs 22 中的补全行为。

以下补全风格亦已定义，若你愿意，可以把它们加入 'completion-styles'（参见"自定义"）：

'substring' 匹配的补全候选项必须把光标点之前的迷你缓冲区文本和光标点之后的迷你缓冲区文本都作为子串包含在内（顺序相同）。

因此，若迷你缓冲区中的文本是 'foobar'，光标点位于 'foo' 与 'bar' 之间，则它匹配 'AfooBbarC'，其中 A、B、C 可以是任意字符串，包括空字符串。

'flex' 这种激进的补全风格又称 'fix' 或 'fuzzy' 或 'scatter' 补全，它尝试用按序出现的子串进行补全。例如，它可以认为 'foo' 匹配 'frodo' 或 'fbarbazoo'。

'initials' 这种非常激进的补全风格尝试补全首字母缩写和缩略语。例如，在补全命令名时，它把 'lch' 匹配到 'list-command-history'。

还有一种非常简单的补全风格叫 'emacs21'。在这种风格中，若迷你缓冲区中的文本是 'foobar'，则只考虑以 'foobar' 开头的匹配项。

通过设置变量 'completion-category-overrides'，你可以在不同情形下使用不同的补全风格。例如，默认设置规定缓冲区名只使用 'basic' 和 'substring' 补全。

10.4.5 补全选项

补全区分大小写的参数（如命令名）时，大小写是敏感的。例如，补全命令名时，'AU' 不会补全为 'auto-fill-mode'。补全大小写无关的参数时，大小写差异被忽略。

补全文件名时，若变量 'read-file-name-completion-ignore-case' 非 'nil'，则忽略大小写差异。在文件名区分大小写的系统（如 GNU/Linux）上，默认值为 'nil'；在文件名不区分大小写的系统（如 Microsoft Windows）上，默认值为非 'nil'。补全缓冲区名时，若变量 'read-buffer-completion-ignore-case' 非 'nil'，则忽略大小写差异；默认值为 'nil'。

补全文件名时，Emacs 通常省略某些被认为不太可能被选中的候选，这由列表变量 ‘completion-ignored-extensions’ 决定。该列表的每个元素都应是字符串；任何以这样的字符串结尾的文件名都被忽略，不作为补全候选。以斜杠（‘/’）结尾的元素代表子目录名。‘completion-ignored-extensions’ 的标准值包含若干元素，其中有 “'.o"’、“'.elc"’ 和 “'~"’。例如，若某目录包含 ‘foo.c’ 和 ‘foo.elc’，则 ‘foo’ 补全为 ‘foo.c’。然而，若*所有*可能的补全都以本来会被忽略的字符串结尾，则它们不会被忽略：在前面的例子中，‘foo.e’ 补全为 ‘foo.elc’。在补全列表中显示补全候选时，Emacs 不考虑 ‘completion-ignored-extensions’。

Shell 补全是文件名补全的扩展版本，参见“Shell 模式选项”。

若 ‘completion-auto-help’ 设为 ‘nil’，补全命令从不显示补全列表缓冲区；你必须输入 ‘?’ 才能显示该列表。若值为 ‘lazy’，Emacs 只在第二次尝试补全时才显示补全列表缓冲区。换言之，若没有什么可补全的，第一次 <TAB> 会回显 ‘Next char not unique’；第二次 <TAB> 才显示补全列表缓冲区。若值为 ‘always’，则每次尝试补全时都会显示补全列表缓冲区。

补全列表缓冲区在首次显示之后的显示行为也由 ‘completion-auto-help’ 控制。若值为 ‘t’ 或 ‘lazy’，当 Emacs 能够补全时，显示补全项的窗口会收起（若你继续输入一些文本后 Emacs 又无法补全，它可能再次弹出）；若值为 ‘always’，该窗口只在你退出补全时才收起。值 ‘visible’ 是混合体：在决定是否弹出显示补全列表缓冲区的窗口时，它的行为像 ‘t’；在决定是否让该窗口收起时，它的行为像 ‘always’。

Emacs 可以选择在显示补全项时选中显示它们的窗口。要启用此行为，把用户选项 ‘completion-auto-select’ 定制为 ‘t’，这会改变 Emacs 弹出补全项时 <TAB> 的行为：按 ‘<TAB>’ 会切换到补全列表缓冲区，然后你可以用光标移动命令移到某个候选，并用 ‘<RET>’ 选中它。若 ‘completion-auto-select’ 的值为 ‘second-tab’，则第一次 ‘<TAB>’ 会弹出补全列表缓冲区，第二次才切换到它。

若 ‘completion-cycle-threshold’ 非 ‘nil’，补全命令可以在补全候选之间循环。通常，若迷你缓冲区文本有多个补全候选，补全命令会补全到最长的公共子串。若你把 ‘completion-cycle-threshold’ 改为 ‘t’，补全命令则改为补全到这些补全候选中的第一个；此后每次调用补全命令，都会以循环方式用下一个补全候选替换它。若给 ‘completion-cycle-threshold’ 一个数值 N，则补全命令仅在候选数不超过 N 时才切换到这种循环行为。

用户选项 ‘completions-sort’ 控制 ‘*Completions*’ 缓冲区中补全项的排序顺序。在默认值 ‘alphabetical’ 下，补全项按字母顺序显示。值 ‘nil’ 禁用排序；值 ‘historical’ 先按字母顺序排序，然后按候选在迷你缓冲区历史中的顺序重新排列。该值也可以是一个函数，调用时传入补全列表，函数应返回按所需顺序排列的列表。

默认情况下，‘*Completions*’ 缓冲区水平列出排序后的补全项（即从左到右跨列，然后继续到下一行），并尽可能用满窗口宽度的列数。你可以通过定制用户选项 ‘completions-format’ 来改变这一点。若其值为 ‘vertical’，‘*Completions*’ 缓冲区竖直列出排序后的补全项（即在每一列内往下排列，然后继续到左边下一列）；若值为 ‘one-column’，则所有补全项都显示为单列。(1)

当光标点位于最后一个补全候选上时，按下一个把光标点移到下一个候选的键（按当前格式的排序顺序），默认会“回绕”，即把光标点移到补全缓冲区中的第一个候选。同样，当光标点位于第一个候选上时，按下移到前一个候选的键会把光标点放到缓冲区中的最后一个候选。你可以通过把用户选项 ‘completion-auto-wrap’ 定制为 ‘nil’ 来禁止这种回绕；这样在最后一个候选上输入 ‘<TAB>’ 或 ‘n’，或在第一个候选上输入 ‘S-<TAB>’ 或 ‘p’，都不会移动光标点。

使用方向键时，回绕行为取决于所用的补全格式。在水平格式中，可以使用 ‘<RIGHT>’ 和 ‘<LEFT>’；在垂直格式中，可以使用 ‘<DOWN>’ 和 ‘<UP>’；它们完全像按排序顺序在补全候选之间移动的格式相关键一样，回

绕行为也相同。相反，在水平格式中，‘<DOWN>’和‘<UP>’只在补全项的当前列内移动——若启用了回绕，也只在列内回绕；在垂直格式中，‘<RIGHT>’和‘<LEFT>’只在补全项的当前行内移动（若启用回绕，也只在行内回绕）。(2)

当‘completions-max-height’非‘nil’时，它会限制补全窗口的大小。它以行数指定，并包含模式行、标题行和底部分隔线（若有）。若要对补全窗口的显示属性做更复杂的控制，可以使用‘display-buffer-alist’（参见《Emacs Lisp 参考手册》的“Buffer Display Action Alists”）。

变量‘completions-header-format’是一个格式说明字符串，用于控制补全候选列表之前显示的信息行。若其中含有‘%s’结构，它会被替换为补全列表缓冲区中显示的补全项数目。若要抑制标题行的显示，请把此变量定制为‘nil’。此变量值所对应的字符串可以带有文本属性，以改变标题行的外观；其中一些有用的属性是‘face’或‘cursor-intangible’（参见《Emacs Lisp 参考手册》的“Special Properties”）。

当‘completions-highlight-face’指定一个字体（face）时，当前补全候选——即按‘<RET>’或用鼠标点击将会选中的那一个——会用该字体高亮显示。此变量的默认值是‘completions-highlight’；若值为‘nil’，则禁用这种高亮。此功能使用特殊文本属性‘cursor-face’。

> 注：若某个补全候选太长，使窗口中放不下一列以上，或者候选不超过三个，那么即使‘completions-format’的值为‘horizontal’或‘vertical’，所有补全项也会显示为单列。

> 注：即使采用水平或垂直格式，若补全缓冲区只包含单列，那么格式相关键和方向键都会把光标点移到下一个（相应地，前一个）候选，且仅在最后一个（相应地，第一个）候选处回绕，前提是‘completion-auto-wrap’的值非‘nil’。

10.5 迷你缓冲区历史

你在迷你缓冲区中输入的所有内容都保存在“迷你缓冲区历史列表”中，以便你日后可以轻松再次使用。这包括补全候选（如文件名、缓冲区名、命令名等）以及任何其他类型的迷你缓冲区输入。你可以使用以下命令快速把先前或备选的响应取入迷你缓冲区：

- ‘M-p’：移到迷你缓冲区历史中的上一项，即更早的参数（‘previous-history-element’）。
- ‘M-n’：移到迷你缓冲区历史中的下一项（‘next-history-element’）。
- ‘<UP>’、‘<DOWN>’：类似‘M-p’和‘M-n’，但在转到上一个或下一个历史项之前，会先移到多行项的上一行或下一行（‘previous-line-or-history-element’和‘next-line-or-history-element’）。
- ‘M-r REGEXP <RET>’：移到迷你缓冲区历史中匹配 REGEXP 的较早项（‘previous-matching-history-element’）。
- ‘M-s REGEXP <RET>’：移到迷你缓冲区历史中匹配 REGEXP 的较晚项（‘next-matching-history-element’）。

在迷你缓冲区中，‘M-p’（‘previous-history-element’）一次一项地在迷你缓冲区历史列表中移动。每次‘M-p’都从历史列表中取一个较早的项放入迷你缓冲区，替换其现有内容。输入‘M-n’（‘next-history-element’）则沿相反方向在迷你缓冲区历史列表中移动，把较晚的条目取入迷你缓冲区。

若你在迷你缓冲区中输入 'M-n' 时，迷你缓冲区历史中没有更晚的条目（例如你之前没有输入过 'M-p'），Emacs 会尝试从默认参数列表中取值：这些值是你可能输入的内容。你可以把它想成在"未来历史"中移动。向前搜索迷你缓冲区历史（参见"搜索迷你缓冲区"）也包括"未来历史"。

文件名的"未来历史"包含若干你可能会用到的备选，例如当前缓冲区中光标点处的文件名或 URL。此时放入"未来历史"的默认值由选项 'file-name-at-point-functions' 值中提到的函数控制。默认情况下，其值会调用 'ffap' 包（参见"在光标点处查找文件和 URL"），后者尝试根据光标点周围的文本猜测默认文件或 URL。要禁用这种猜测，请把该选项定制为 'nil'，这样文件名的"未来历史"将只包含当前缓冲区所访问的文件（若有）以及默认目录。

方向键 '<UP>' 和 '<DOWN>' 的作用类似 'M-p' 和 'M-n'，但若当前历史项长于一行，它们允许你在转到上一个或下一个历史项之前，先移到当前历史项的上一行或下一行。

若你编辑了 'M-p' 或 'M-n' 迷你缓冲区历史命令插入的文本，这不会改变它在历史列表中的条目。不过，当你提交时，编辑后的参数确实会被加到历史列表末尾。

你可以用 'M-r'（'previous-matching-history-element'）在历史列表中搜索较旧的元素，用 'M-s'（'next-matching-history-element'）搜索较新的条目。这两个命令都要求以"正则表达式"为参数，并把第一个匹配的条目取入迷你缓冲区。正则表达式的解释参见"正则表达式的语法"。数值前缀参数 N 表示取第 N 个匹配的条目。这些命令不同寻常之处在于，尽管是从迷你缓冲区中调用的，它们仍用迷你缓冲区来读取正则表达式参数。正则表达式中的大写字母使搜索区分大小写（参见"搜索时的宽松匹配"）。

你也可以使用增量搜索（'C-s' 和 'C-r'）搜索迷你缓冲区历史。参见"搜索迷你缓冲区"。

Emacs 为几种不同类型的参数保留各自独立的历史列表。例如，有一个文件名的列表，供所有读取文件名的命令使用。其他历史列表包括缓冲区名、命令名（供 'M-x' 使用）和命令参数（供 'query-replace' 之类的命令使用）。

变量 'history-length' 指定迷你缓冲区历史列表的最大长度；当列表变得过长时，加入新元素会删除最旧的元素。若值为 't'，则没有最大长度。

变量 'history-delete-duplicates' 指定是否删除历史中的重复项。若它非 'nil'，加入新元素时会从列表中删除所有与其相等的其他元素。默认值为 'nil'。

10.6 重复迷你缓冲区命令

每个使用过迷你缓冲区的命令都会连同其参数值记录在一个特殊的历史列表即"命令历史"中，以便你可以重复整个命令。特别是，每次使用 'M-x' 都会记录在那里，因为 'M-x' 用迷你缓冲区读取命令名。

- 'C-x <ESC> <ESC>'：从命令历史中重新执行最近的一个迷你缓冲区命令（'repeat-complex-command'）。
- 'M-x list-command-history'：显示整个命令历史，列出 'C-x <ESC> <ESC>' 可以重复的所有命令，最近的排在前面。

'C-x <ESC> <ESC>' 重新执行最近一个使用过迷你缓冲区的命令。不带参数时，它重复最后一个这样的命令。数值参数指定重复哪个命令；1 表示最后一个，2 表示前一个，依此类推。

‘C-x <ESC> <ESC>’ 的工作原理是把上一个命令转换成一个 Lisp 表达式，然后进入一个用该表达式的文本初始化的迷你缓冲区。即使你不懂 Lisp，大概也能明显看出显示出来准备重复的是哪个命令。若你只输入 <RET>，就会原样重复该命令。你也可以在执行前编辑该 Lisp 表达式来改变命令。所执行的命令会被加到命令历史的前端，除非它与最近的项完全相同。

一旦进入 ‘C-x <ESC> <ESC>’ 的迷你缓冲区，你就可以使用通常的迷你缓冲区历史命令（参见“迷你缓冲区历史”）在历史列表中移动。找到想要的先前命令后，你可以照常编辑其表达式，然后按 <RET> 执行。

严格说来，增量搜索并不使用迷你缓冲区。因此，尽管它的行为像复杂命令，它通常不会出现在 ‘C-x <ESC> <ESC>’ 的历史列表中。你可以通过把 ‘isearch-resume-in-command-history’ 设为非 ‘nil’ 值，使增量搜索命令出现在历史中。参见“增量搜索”。

先前使用迷你缓冲区的命令列表以 Lisp 列表形式存储在变量 ‘command-history’ 中。每个元素都是一个描述某个命令及其参数的 Lisp 表达式。Lisp 程序可以用 ‘command-history’ 的元素调用 ‘eval’ 来重新执行某个命令。

10.7 输入密码

有时你可能需要在 Emacs 中输入密码。例如，当你让 Emacs 通过 FTP 之类的网络协议访问另一台机器上的文件时，常常需要提供密码才能获准访问该机器（参见“远程文件”）。

输入密码类似于使用迷你缓冲区。Emacs 在回显区显示提示（如 ‘Password: ’）；输入所需密码后，按 <RET> 提交。为防止他人看到你的密码，你输入的每个字符都显示为星号（‘*’），而不显示为其通常形式。

与迷你缓冲区相关的大部分功能和命令在输入密码时*不能*使用。没有历史，也没有补全；在提交密码之前，你无法切换窗口，也无法用 Emacs 做任何其他事情。

输入密码时，你可以按 <DEL> 向后删除，删除最后输入的字符。‘C-u’ 删除你目前输入的全部内容。‘C-g’ 退出密码提示（参见“退出与中止”）。‘C-y’ 把当前删除的内容插入密码（参见“删除与移动文本”）。<TAB> 切换密码的可见性。你可以按 <RET> 或 <ESC> 提交密码。其他任何自插入字符键都会把相应字符插入密码，而所有其他输入都被忽略。

模式行中还有一个图标指示密码的可见性。用 ‘mouse-1’ 点击它同样可以切换密码的可见性。

10.8 是或否提示

Emacs 命令在执行过程中可能要求你回答一个是或否的问题。这类询问主要有两种。

第一种是或否询问的提示以 ‘(y or n)’ 结尾。你通过输入单个键 ‘y’ 或 ‘n’ 来回答，这会立即退出迷你缓冲区并给出响应。例如，若你输入 ‘C-x C-w’（‘write-file’）保存缓冲区，并输入一个已存在的文件名，Emacs 会发出如下提示：

文件 `foo.el' 已存在；是否覆盖？(y or n)

第二种是或否询问通常在给出错误回答会有严重后果时使用；因此它的提示更长，以 ‘(yes or no)’（或你定制过的 ‘yes-or-no-prompt’ 的值）结尾。例如，若你对一个访问文件且含有未保存修改的缓冲区调用 ‘C-x k’（‘kill-buffer’），Emacs 会激活迷你缓冲区并给出如下提示：

缓冲区 foo.el 已修改；仍要删除？(yes or no)

要回答，你必须在迷你缓冲区中输入‘yes’或‘no’，然后按 <RET>。

对两种是或否询问，迷你缓冲区的行为都与前面几节所述相同；你可以用‘C-l’重新居中选中的窗口，滚动该窗口（‘C-v’或‘PageDown’向前滚动，‘M-v’或‘PageUp’向后滚动），用‘C-x o’切换到另一个窗口，使用历史命令‘M-p’和‘M-n’等。输入‘C-g’可取消该询问，并退出迷你缓冲区与发出询问的命令（参见“退出与中止”）。

11 按名称执行命令

每个 Emacs 命令都有一个名字，你可以用它来运行该命令。为方便起见，许多命令还带有键绑定。你既可以键入按键来运行这些命令，也可以按名字运行它们。大多数 Emacs 命令没有键绑定，因此按名字运行是运行它们的唯一方式。（参见“定制键绑定”，了解如何设置键绑定。）

按惯例，命令名由一个或多个单词组成，单词之间用连字符分隔；例如 ‘auto-fill-mode’ 或 ‘manual-entry’。命令名大多使用完整的英文单词，以便于记忆。

要按名字运行命令，先按 ‘M-x’，键入命令名，然后用 <RET> 结束。‘M-x’ 使用迷你缓冲区读取命令名。字符串 ‘M-x’ 出现在迷你缓冲区开头，作为“提示符”提醒你输入要运行的命令名。<RET> 退出迷你缓冲区并运行该命令。关于迷你缓冲区的更多信息，参见“迷你缓冲区”。

你可以使用补全来输入命令名。例如，要调用命令 ‘forward-char’，你可以键入

```
M-x forward-char <RET>
```

或

```
M-x forw <TAB> c <RET>
```

请注意，‘forward-char’ 与用键 ‘C-f’ 调用的是同一个命令。键绑定的存在并不妨碍你按名字运行该命令。

当 ‘M-x’ 对命令进行补全时，它会忽略在以往任何 Emacs 主版本中已声明为“过时”的命令；对这些命令，你必须键入它们的完整名字。在当前 Emacs 版本中被标记为过时的命令会列出来。（过时命令是指那些已有更新、更好的替代命令，并计划在未来某个 Emacs 版本中移除的命令。）

此外，‘M-x’ 补全还可以排除与当前缓冲区的主模式（参见“主模式”）和次模式（参见“次模式”）无关、并且通常无法在其中工作的命令。默认不排除任何命令，但你可以自定义选项 ‘read-extended-command-predicate’，把那些无关命令从补全结果中排除。

反过来，Emacs 也可以只列出与当前缓冲区特别相关的命令，而排除其他所有命令。‘M-S-x’（即“meta shift x”）命令的工作方式和 ‘M-x’ 一样，但它不会列出 Emacs 所知道的所有（或大部分）命令，而只列出被标记为“属于”当前主模式或任何已启用次模式的命令。

若要取消 ‘M-x’ 而不运行命令，请键入 ‘C-g’，而不是输入命令名。这会把你带回命令层级。

若要向用 ‘M-x’ 调用的命令传递数字参数，请在 ‘M-x’ 之前指定数字参数。在读取命令名的过程中，参数值会显示在提示符里，最后 ‘M-x’ 会把该参数传递给那个命令。例如，要把数字参数 42 传给命令 ‘forward-char’，你可以键入 ‘C-u 42 M-x forward-char <RET>’。

当你用 ‘M-x’ 运行的命令有键绑定时，Emacs 会在运行该命令之后在回显区提及这一点。例如，如果你键入 ‘M-x forward-word’，消息会说明你可以键入 ‘M-f’ 来运行同一个命令。你可以通过把变量 ‘suggest-key-bindings’ 设为 ‘nil’ 来关闭这些消息。‘suggest-key-bindings’ 的值也可以是一个数字，此时 Emacs 会显示该绑定那么多秒，然后将其从显示中移除。默认行为是显示该绑定 2 秒。

此外，当 ‘suggest-key-bindings’ 为非 ‘nil’ 时，‘M-x’ 的补全列表会为所有具有等价键绑定的命令显示该绑定。

没有键绑定的命令，仍然可以在‘M-x’提示符下键入少于其完整名字的内容后调用。如果这类简写明显短于完整命令名，并且‘extended-command-suggest-shorter’为非‘nil’，Emacs 会在回显区提及它们。‘suggest-key-bindings’的设置也会影响这些提示。

在本手册中，当我们谈到按名字运行命令时，常常省略结束命令名的 <RET>。因此我们可能会说‘M-x auto-fill-mode’，而不是‘M-x auto-fill-mode <RET>’。只有在需要强调时才提及 <RET>，例如命令后面还要跟参数时。

‘M-x’的工作方式是运行命令‘execute-extended-command’，该命令负责读取另一个命令的名字并调用它。

12 帮助

Emacs 提供了各种各样的帮助命令，全部可以通过前缀键 ‘C-h’（等价地，功能键 <F1>）访问。以下各节将介绍这些帮助命令。你也可以键入 ‘C-h C-h’ 查看帮助命令列表（‘help-for-help’）。可以用 <SPC> 和 <DEL> 滚动该列表，然后键入你想要的帮助命令。要取消，请键入 ‘C-g’。

许多帮助命令把信息显示在一个特殊的“帮助缓冲区”中。在该缓冲区里，你可以键入 <SPC> 和 <DEL> 来滚动，键入 <RET> 来跟随超链接。参见“帮助模式命令”。

默认情况下，帮助命令在单独的窗口中显示帮助缓冲区，但不选中该窗口。变量 ‘help-window-select’ 控制这一行为：其默认值为 ‘nil’；若把它定制为 ‘t’，帮助命令会无条件选中帮助窗口；若其值为 ‘other’，则只有当所选框架上有两个以上窗口时才选中帮助窗口。

反过来，在 ‘*Help*’ 缓冲区中的许多命令会弹出新窗口来显示结果。例如，点击显示源代码的链接，或使用 <i> 命令显示手册条目，（默认）都会弹出新窗口。如果把 ‘help-window-keep-selected’ 改为非 ‘nil’ 值，则会改为重用显示 ‘*Help*’ 缓冲区的那个窗口。

如果你在寻找某个功能，但不知道它叫什么或该到哪里去找，我们推荐三种方法。首先尝试 apropos 命令，然后尝试搜索手册索引，再查看 FAQ 和包关键字，最后尝试列出外部包。

‘C-h a TOPICS <RET>’ 搜索名称匹配参数 TOPICS 的命令。该参数可以是一个关键字、由空白分隔的关键字列表，或一个正则表达式（参见“正则表达式的语法”）。参见“Apropos”。

‘C-h d TOPICS <RET>’ 类似，但搜索的是文档字符串的*文本*，而非命令和函数的名称。

‘C-h r i TOPIC <RET>’ 在 Emacs Info 手册的索引中搜索 TOPIC，显示找到的第一个匹配项。按 ‘,’ 查看后续匹配项。TOPIC 可以使用正则表达式。

‘C-h r s TOPIC <RET>’ 类似，但搜索的是手册的*文本*，而非索引。

‘C-h C-f’ 使用 Info 显示 Emacs FAQ。

‘C-h p’ 根据关键字显示可用的 Emacs 包。参见“包的关键字搜索”。

‘M-x list-packages’ 显示外部包的列表。参见“Emacs Lisp 软件包”。

‘C-h’ 或 <F1> 在许多其他上下文中也意味着“帮助”。例如，你可以在前缀键之后键入它们，以查看可跟随该前缀键的按键列表。（在此上下文中也可以使用 ‘?’。少数前缀键不支持以这种方式使用 ‘C-h’ 或 ‘?’，因为它们为这些输入定义了其他含义，但它们都支持 <F1>。）

12.1 帮助摘要

下面概要列出用于访问内置文档的帮助命令。其中大多数将在后续各节中更详细地介绍。

- ‘C-h a TOPICS <RET>’ 显示名称匹配 TOPICS 的命令列表（‘apropos-command’）。参见“Apropos”。
- ‘C-h b’ 显示所有活动键绑定；先是次模式绑定，然后是主模式绑定，最后是全局绑定（‘describe-bindings’）。参见“其他帮助命令”。
- ‘C-h C-q’ 切换显示一个窗口，其中列出常用命令及其键绑定。参见“其他帮助命令”。

- ‘C-h c KEY’ 显示键序列 KEY 所绑定到的命令名（‘describe-key-briefly’）。这里的 ‘c’ 代表 “character”（字符）。若想获得关于 KEY 的更详细信息，请使用 ‘C-h k’。参见“按键的文档”。
- ‘C-h d TOPICS <RET>’ 显示文档匹配 TOPICS 的命令和变量（‘apropos-documentation’）。参见 “Apropos”。
- ‘C-h e’ 显示 ‘*Messages*’ 缓冲区（‘view-echo-area-messages’）。参见“其他帮助命令”。
- ‘C-h f FUNCTION <RET>’ 显示名为 FUNCTION 的 Lisp 函数的文档（‘describe-function’）。由于命令也是 Lisp 函数，这同样适用于命令，但你也可以使用 ‘C-h x’。参见“按命令或变量名获取帮助”。
- ‘C-h h’ 显示 ‘HELLO’ 文件，其中展示了各种字符集的示例。
- ‘C-h i’ 运行 Info，即 GNU 文档浏览器（‘info’）。Emacs 手册可在 Info 中获得。参见“其他帮助命令”。
- ‘C-h k KEY’ 显示 KEY 所运行命令的名称和文档（‘describe-key’）。参见“按键的文档”。
- ‘C-h l’ 显示你最近 300 次击键的描述（‘view-lossage’）。参见“其他帮助命令”。
- ‘C-h m’ 显示当前主模式和次模式的文档（‘describe-mode’）。参见“其他帮助命令”。
- ‘C-h n’ 显示近期 Emacs 变更的新闻（‘view-emacs-news’）。参见“帮助文件”。
- ‘C-h o SYMBOL’ 显示名为 SYMBOL 的 Lisp 符号的文档（‘describe-symbol’）。这将显示各种符号的文档：函数、变量和字体（face）。参见“按命令或变量名获取帮助”。
- ‘C-h p’ 按主题关键字查找包（‘finder-by-keyword’）。参见“包的关键字搜索”。这会使用包菜单缓冲区列出包。参见“Emacs Lisp 软件包”。
- ‘C-h P PACKAGE <RET>’ 显示关于指定包的文档（‘describe-package’）。参见“包的关键字搜索”。
- ‘C-h r’ 在 Info 中显示 Emacs 手册（‘info-emacs-manual’）。
- ‘C-h s’ 显示当前“语法表”（syntax table）的内容（‘describe-syntax’）。参见“其他帮助命令”。语法表说明哪些字符是起始定界符、哪些是单词的组成部分，等等。参见《Emacs Lisp 参考手册》的“Syntax Tables, for details”
- ‘C-h t’ 进入 Emacs 交互式教程（‘help-with-tutorial’）。
- ‘C-h v VAR <RET>’ 显示 Lisp 变量 VAR 的文档（‘describe-variable’）。参见“按命令或变量名获取帮助”。
- ‘C-h w COMMAND <RET>’ 显示哪些键运行名为 COMMAND 的命令（‘where-is’）。参见“按键的文档”。
- ‘C-h x COMMAND <RET>’ 显示关于具名 COMMAND 的文档（‘describe-command’）。参见“按命令或变量名获取帮助”。
- ‘C-h C CODING <RET>’ 描述编码系统 CODING（‘describe-coding-system’）。参见“编码系统”。
- ‘C-h C <RET>’ 描述当前正在使用的编码系统。
- ‘C-h F COMMAND <RET>’ 进入 Info 并转到记录 Emacs 命令 COMMAND 的节点（‘Info-goto-emacs-command-node’）。参见“按命令或变量名获取帮助”。
- ‘C-h I METHOD <RET>’ 描述输入法 METHOD（‘describe-input-method’）。参见“选择输入法”。
- ‘C-h K KEY’ 进入 Info 并转到记录键序列 KEY 的节点（‘Info-goto-emacs-key-command-node’）。参见“按键的文档”。
- ‘C-h L LANGUAGE-ENV <RET>’ 显示关于语言环境 LANGUAGE-ENV 中所用字符集、编码系统和输入法的信息（‘describe-language-environment’）。参见“语言环境”。

- ‘C-h S SYMBOL <RET>’ 根据你正在编辑的编程语言，显示符号 SYMBOL 的 Info 文档（‘info-lookup-symbol’）。参见“其他帮助命令”。
- ‘C-h .’ 如果光标点位于某个特殊文本区域中，显示该区域的帮助消息（‘display-local-help’）。（例如，‘*Help*’ 缓冲区中的链接就属于这类区域。）参见“活动文本与工具提示的帮助”。如果带前缀参数调用此命令，即 ‘C-u C-h .’，且光标点位于按钮或控件上，此命令将弹出一个描述该按钮或控件的新缓冲区。

12.2 按键的文档

用于获取键序列信息的帮助命令是 ‘C-h c’（‘describe-key-briefly’）和 ‘C-h k’（‘describe-key’）。

‘C-h c KEY’ 在回显区显示 KEY 所绑定命令的名称。例如，‘C-h c C-f’ 显示 ‘forward-char’。

‘C-h k KEY’ 与之类似，但提供更多信息：它显示一个帮助缓冲区，其中包含该命令的“文档字符串”，精确描述该命令做什么。

‘C-h K KEY’ 显示 Emacs 手册中描述与 KEY 对应命令的那一节。

‘C-h c’、‘C-h k’ 和 ‘C-h K’ 适用于任何类型的键序列，包括功能键、菜单和鼠标事件（但 ‘C-h c’ 忽略鼠标移动事件）。例如，在 ‘C-h k’ 之后，你可以从菜单栏中选择一个菜单项，以查看它所运行命令的文档字符串。

‘C-h w COMMAND <RET>’ 列出绑定到 COMMAND 的键。它在回显区显示该列表。如果它说该命令不在任何键上，那意味着你必须用 ‘M-x’ 来运行它。‘C-h w’ 运行命令 ‘where-is’。

Emacs 中的某些模式使用各种按钮（参见《Emacs Lisp 参考手册》的“Buttons”）和控件（参见《Widget 手册》的“Introduction”），可以点击它们来执行某些操作。为了查明这些按钮最终调用的是哪个函数，Emacs 提供了 ‘button-describe’ 和 ‘widget-describe’ 命令，应在光标点位于按钮上时运行它们。

‘M-x which-key’ 是一个全局次模式，有助于发现键位图。它会在弹出窗口中显示跟随你当前已输入的不完整命令（前缀）的键绑定。

12.3 按命令或变量名获取帮助

‘C-h x COMMAND <RET>’（‘describe-command’）在窗口中显示具名 COMMAND 的文档。例如，

```
C-h x auto-fill-mode <RET>
```

显示 ‘auto-fill-mode’ 的文档。对于未绑定到任何键的命令（通常要用 ‘M-x’ 运行的那种命令），这就是获取其文档的方式。

‘C-h f FUNCTION <RET>’（‘describe-function’）显示 Lisp 函数 FUNCTION 的文档。此命令面向你在 Lisp 程序中使用的 Lisp 函数。例如，如果你刚写了表达式 ‘(make-vector len)’ 并想检查自己是否正确使用了 ‘make-vector’，请键入 ‘C-h f make-vector <RET>’。此外，由于所有命令都是 Lisp 函数，你也可以用此命令查看任何命令的文档。

如果你键入 ‘C-h f <RET>’，它会描述光标点周围缓冲区中最内层 Lisp 表达式所调用的函数，*前提是*该函数名是有效的、已定义的 Lisp 函数。（在你输入参数时，该名称会作为默认值出现。）例如，如果光标点位于

文本 ‘(make-vector (car x)’ 之后，包含光标点的最内层列表就是以 ‘(make-vector’ 开头的那个，因此 ‘C-h f <RET>’ 会描述函数 ‘make-vector’。

‘C-h f’ 也可用来验证函数名是否拼写正确。如果 ‘C-h f’ 的迷你缓冲区提示把缓冲区中的函数名作为默认值显示，那就意味着该名称已定义为 Lisp 函数。如果你其实并不想查看文档，键入 ‘C-g’ 取消 ‘C-h f’ 命令。

‘describe-function’ 显示的函数文档不仅包括文档字符串和函数签名，还包括辅助信息，如其类型、定义所在的文件、是否已被声明为过时；通常还可以通过点击或键入 <RET> 于文本的强调部分来获取更多信息。

如果已知函数类型，就用“函数类型说明符”（function type specifier）表示（参见《Emacs Lisp 参考手册》的“Type Specifiers”）；若该类型是由 Lisp 程序手动声明或由编译器推断出来的，就会给出。注意，函数类型推断仅在启用原生编译时有效（参见《Emacs Lisp 参考手册》的“native compilation”）。

如果你请求一个自动加载函数的帮助，而它的 ‘autoload’ 形式（参见《Emacs Lisp 参考手册》的“Autoload”）未提供文档字符串，则 ‘*Help*’ 缓冲区没有任何文档字符串可显示。在这种情况下，若 ‘help-enable-symbol-autoload’ 非 ‘nil’，Emacs 会尝试加载定义该函数的文件，看看那里是否有文档字符串。

你可以使用 ‘M-x shortdoc’ 命令获取与特定主题相关的函数的概览。它会提示你输入感兴趣的领域，如 ‘string’，然后把带你到一个缓冲区，其中列出了许多与处理字符串相关的函数。

你还可以要求 ‘C-h f’ 弹出的 ‘*Help*’ 缓冲区中显示的函数和命令文档包含用法示例。为此，请将以下内容加入你的初始化文件（参见“Emacs 初始化文件”）：

```
(add-hook 'help-fns-describe-function-functions
          #'shortdoc-help-fns-examples-function)
```

‘C-h v’（‘describe-variable’）类似 ‘C-h f’，但描述的是 Lisp 变量而非 Lisp 函数。其默认值是光标点周围或之前的 Lisp 符号——前提是该符号是已定义的 Lisp 变量名。参见“变量”。

描述 Emacs 变量和函数的帮助缓冲区通常带有指向相应源代码的超链接（如果你安装了源文件）（参见“超链接与网页浏览”）。

要在手册中查找某个命令的文档，请使用 ‘C-h F’（‘Info-goto-emacs-command-node’）。它了解各种手册，而不仅是 Emacs 手册，并能找到正确的那一个。

‘C-h o’（‘describe-symbol’）类似 ‘C-h f’ 和 ‘C-h v’，但它描述任何符号，无论是函数、变量还是字体（face）。如果该符号有不止一个定义，比如既有函数定义又有变量定义，此命令会依次显示所有这些定义的文档。

‘C-h 4 s’（‘help-find-source’）切换到访问帮助缓冲区中所描述对象源代码定义的缓冲区。

如果用户选项 ‘completions-detailed’ 非 ‘nil’，某些命令在显示补全时会提供候选值的详细信息。例如，‘C-h o TAB’ 届时会包含文档字符串的第一行，还会说明每个符号是函数还是变量（等等）。包含哪些详细信息因所用命令而异。

12.4 Apropos

“apropos” 命令回答诸如“处理文件的命令有哪些？”之类的问题。更精确地说，你把查询指定为一个“apropos 模式”（apropos pattern），它要么是一个词、由空白分隔的词列表，要么是一个正则表达式。

以下每个 apropos 命令都会在迷你缓冲区中读取一个 apropos 模式，搜索匹配该模式的项，并在另一个窗口中显示结果。

- ‘C-h a’ 搜索命令（‘apropos-command’）。带前缀参数时，也搜索非交互式函数。
- ‘M-x apropos’ 搜索函数和变量。交互式函数（命令）和非交互式函数都能被它找到。
- ‘C-h u’ 搜索用户可定制的变量。带前缀参数时，也搜索不可定制的变量。
- ‘M-x apropos-variable’ 搜索变量。带前缀参数时，只搜索可定制的变量。
- ‘M-x apropos-local-variable’ 搜索缓冲区局部变量。
- ‘M-x apropos-value’ 搜索值匹配指定模式的变量。带前缀参数时，还搜索定义匹配该模式的函数，以及属性匹配该模式的 Lisp 符号。
- ‘M-x apropos-local-value’ 搜索值匹配指定模式的缓冲区局部变量。
- ‘C-h d’ 搜索文档字符串匹配指定模式的函数和变量（‘apropos-documentation’）。

最简单的 apropos 模式是单个词。任何包含该词的内容都匹配该模式。因此，要查找处理文件的命令，请键入 ‘C-h a file <RET>’。这会显示所有包含 ‘file’ 的命令名列表，包括 ‘copy-file’、‘find-file’ 等。每个命令名都带有简要说明，以及你当前可以调用它的键列表。在我们的例子中，它会说你可以通过键入 ‘C-x C-f’ 来调用 ‘find-file’。

默认情况下，显示 apropos 缓冲区（含查询结果）的窗口不会被选中，但你可以把变量 ‘help-window-select’ 定制为任何非 ‘nil’ 值，使其被选中。

要了解 apropos 缓冲区中列出的函数定义、变量或符号属性的更多信息，你可以用 ‘mouse-1’ 或 ‘mouse-2’ 点击它，或把光标移到那里并键入 <RET>。

当你在 apropos 模式中指定多个词时，名称必须至少包含其中两个词才算匹配。因此，如果你在寻找删除光标点之前一段文本的命令，可以试试 ‘C-h a kill back backward behind before <RET>’。真正的命令名 ‘kill-backward’ 会匹配该模式；如果存在一个 ‘kill-text-before’ 命令，它也会匹配，因为它包含指定的词中的两个。

为了获得更大的灵活性，你可以指定正则表达式（参见“正则表达式的语法”）。如果 apropos 模式包含任何正则表达式特殊字符 ‘^$*+?.\[’，就会被当作正则表达式解释。

遵循 Emacs 命令的命名约定，下面是一些在 apropos 模式中很有用的词。在 ‘C-h a’ 中使用它们，你也会对命名约定有所体会。

char、line、word、sentence、paragraph、region、page、sexp、list、defun、rect、buffer、frame、window、face、file、dir、register、mode、beginning、end、forward、backward、next、previous、up、down、search、goto、kill、delete、mark、insert、yank、fill、indent、case、change、set、what、list、find、view、describe、default。

如果变量 ‘apropos-do-all’ 非 ‘nil’，大多数 apropos 命令的行为就像被给了前缀参数一样。有一个例外：不带前缀参数的 ‘apropos-variable’ 总是搜索所有变量，无论 ‘apropos-do-all’ 的值是什么。

默认情况下，除 ‘apropos-documentation’ 之外的所有 apropos 命令都按字母顺序列出结果。如果变量 ‘apropos-sort-by-scores’ 非 ‘nil’，这些命令则改为尝试推测每个结果的相关性，并优先显示最相关的结果。‘apropos-

documentation’ 命令默认按相关性顺序列出结果；要按字母顺序列出，请把变量 ‘apropos-documentation-sort-by-scores’ 改为 ‘nil’。

12.5 帮助模式命令

帮助缓冲区以帮助模式（Help mode）为主模式。帮助模式提供与视图模式（View mode）相同的命令（参见“视图模式”）；例如，<SPC> 向前滚动，<DEL> 或 ‘S-<SPC>’ 向后滚动。它还提供了一些特殊命令：

- ‘<RET>’ 跟随光标点处的交叉引用（‘help-follow’）。
- ‘<TAB>’ 把光标点前移到下一个超链接（‘forward-button’）。
- ‘S-<TAB>’ 把光标点后移到上一个超链接（‘backward-button’）。
- ‘mouse-1’ ‘mouse-2’ 跟随你点击的超链接。
- ‘n’ ‘p’ 在帮助缓冲区的各页之间前后移动。
- ‘C-c C-c’ 显示关于光标点处符号的所有文档（‘help-follow-symbol’）。
- ‘C-c C-f’ ‘r’ 在帮助命令历史中前进（‘help-go-forward’）。
- ‘C-c C-b’ ‘l’ 在帮助命令历史中后退（‘help-go-back’）。
- ‘s’ 查看当前帮助主题的源代码（如果有）（‘help-view-source’）。
- ‘i’ 在手册中查找当前主题（‘help-goto-info’）。
- ‘I’ 在 Emacs Lisp 手册中查找当前主题（‘help-goto-lispref-info’）。
- ‘c’ 定制该变量或字体（‘help-customize’）。

当函数名、变量名或字体名（参见“文本字体”）出现在帮助缓冲区的文档中时，它通常是一个带下划线的“超链接”。要查看相关的文档，请把光标点移到那里并键入 <RET>（‘help-follow’），或用 ‘mouse-1’ 或 ‘mouse-2’ 点击该超链接。这样做会替换帮助缓冲区的内容；要回溯你的步骤，请键入 ‘C-c C-b’ 或 ‘l’（‘help-go-back’）。在回溯时，你可以用 ‘C-c C-f’ 或 ‘r’（‘help-go-forward’）前进。

要在帮助缓冲区的超链接之间移动，用 <TAB>（‘forward-button’）前移到下一个超链接，用 ‘S-<TAB>’（‘backward-button’）后移到上一个超链接。这些命令循环生效；例如，在最后一个超链接处键入 <TAB> 会移回第一个超链接。

默认情况下，帮助缓冲区中的许多链接显示时两侧带有引号字符。如果用户选项 ‘help-clean-buttons’ 非 ‘nil’，这些引号字符会从缓冲区中移除。

某些帮助命令（如 ‘C-h b’，它显示很长的键绑定列表）产生的帮助缓冲区按 ‘^L’ 字符分页。在这类缓冲区中，‘n’（‘help-goto-next-page’）命令会把你带到下一页开头，‘p’（‘help-goto-previous-page’）命令会把你带到上一页开头。这样你就能在帮助缓冲区中不同种类的文档之间快速导航。

帮助缓冲区还可能包含指向 Info 手册、源代码定义和 URL（网页）的超链接。前两者在 Emacs 中打开，第三个则通过 ‘browse-url’ 命令使用网页浏览器打开（参见“跟随 URL”）。

要查看文本中任何符号的所有文档，请把光标点移到该符号处并键入 ‘C-c C-c’（‘help-follow-symbol’）。这会显示该符号所有含义的文档——作为变量、作为函数和/或作为字体。

12.6 包的关键字搜索

Emacs 中的大多数可选特性被归入“包”（package）。Emacs 包含数百个内置包，还可以通过网络安装更多（参见“Emacs Lisp 软件包”）。

为了更容易找到与某个主题相关的包，大多数包都根据其用途关联了一个或多个“关键字”。键入 ‘C-h p’（‘finder-by-keyword’）可调出包关键字列表，以及各关键字含义的说明。要查看给定关键字的包列表，请在该行上键入 <RET>；这会在包菜单缓冲区中显示该包的列表（参见“包菜单缓冲区”）。

‘C-h P’（‘describe-package’）提示输入包名（参见“Emacs Lisp 软件包”），并显示一个帮助缓冲区，描述该包的属性以及它实现的功能。该缓冲区以按钮的形式列出与该包相关的关键字。用 ‘mouse-1’ 或 ‘mouse-2’ 点击某个按钮，可查看与该关键字相关的其他包的列表。

12.7 国际语言支持的帮助

要了解特定语言环境的信息（参见“语言环境”），请键入 ‘C-h L’（‘describe-language-environment’）。这会显示一个帮助缓冲区，描述该语言环境所支持的语言，并列出相关的字符集、编码系统和输入法，以及该语言环境的一些示例文本。

命令 ‘C-h h’（‘view-hello-file’）显示文件 ‘etc/HELLO’，它通过展示如何用多种语言说“hello”来演示各种字符集。

命令 ‘C-h I’（‘describe-input-method’）描述输入法——要么是指定的输入法，要么默认是当前正在使用的输入法。参见“输入法”。

命令 ‘C-h C’（‘describe-coding-system’）描述编码系统——要么是指定的编码系统，要么是当前正在使用的那些。参见“编码系统”。

12.8 其他帮助命令

‘C-h i’（‘info’）运行 Info 程序，它浏览结构化的文档文件。‘C-h 4 i’（‘info-other-window’）做同样的事，但在另一个窗口中显示 Info 缓冲区。整本 Emacs 手册都能在 Info 中获得，此外还有许多 GNU 系统的其他手册。进入 Info 后键入 ‘h’ 可运行 Info 使用教程。

若带有数字参数 N，‘C-h i’ 会选中 Info 缓冲区 ‘*info*<N>’。如果你想同时浏览多个 Info 手册，这会很有用。如果只把前缀参数指定为 ‘C-u’，‘C-h i’ 会提示输入文档文件名，这样你就能浏览在顶层 Info 菜单里没有条目的文件。

前面介绍过的帮助命令 ‘C-h F FUNCTION <RET>’ 和 ‘C-h K KEY’ 会进入 Info，并直接跳到 FUNCTION 或 KEY 的文档。

编辑程序时，如果你有该编程语言手册的 Info 版本，可以用 ‘C-h S’（‘info-lookup-symbol’）在合适的手册中查找某个符号（关键字、函数或变量）的条目。该命令的工作细节取决于主模式。

如果发生了意外情况，而你不确定自己输入了什么，就用 ‘C-h l’（‘view-lossage’）。‘C-h l’ 会显示你最近输入的击键以及它们所调用的命令。默认情况下，Emacs 保存最近 300 次击键；如有需要，你可以用命令 ‘lossage-size’ 修改这个数目。如果看到不熟悉的命令，可以用 ‘C-h k’，或者按 ‘RET’、或者点击它们来查明其作用。

默认情况下，lossage 缓冲区一经显示，就不会显示你在此之后输入的击键和命令。但如果你把变量 ‘view-lossage-auto-refresh’ 设为 ‘t’，那么随着你继续输入命令，lossage 缓冲区会不断刷新。另一种办法是带前缀参数调用 ‘view-lossage’，例如 ‘C-u C-h l’，这会显示 lossage 缓冲区并在其后自动刷新。

要回顾最近的回显区消息，用 ‘C-h e’（‘view-echo-area-messages’）。它会显示保存这些消息的缓冲区 ‘*Messages*’。

每个 Emacs 主模式通常都会重定义一些键，并在编辑的工作方式上做其他改动。‘C-h m’（‘describe-mode’）显示关于当前主模式的文档，其中通常描述该模式下被改动的命令和特性，以及它的键绑定。

‘C-h b’（‘describe-bindings’）和 ‘C-h s’（‘describe-syntax’）显示 Emacs 内部当前环境的其他信息。‘C-h b’ 列出当前生效的所有键绑定：先是当前各次模式的局部绑定，然后是当前主模式定义的局部绑定，最后是全局绑定（参见“定制键绑定”）。‘C-h s’ 显示语法表的内容，并解释每个字符的语法（参见《Emacs Lisp 参考手册》的“Syntax Tables”）。

‘C-h C-q’（‘help-quick-toggle’）切换是否显示这样一个缓冲区：其中列出最常用的 Emacs 命令及各自对应的键绑定（俗称“速查表”）。该缓冲区的内容由命令 ‘help-quick’ 生成。此缓冲区中显示的每个键绑定都是一个按钮：用 ‘mouse-1’ 或 ‘mouse-2’ 点击它，就会显示该键序列所绑定命令的文档。

你可以在某个前缀键之后输入 ‘C-h’、‘?’ 或 <F1>（‘describe-prefix-bindings’），以获得该前缀键的子命令列表。（有少数前缀键对以上各键并非都管用——那些为自身提供了该键绑定的前缀键。其中之一是 <ESC>，因为 ‘<ESC> C-h’ 和 ‘<ESC> ?’ 实际上分别是 ‘C-M-h’（‘mark-defun’）和 ‘M-?’（‘xref-find-references’）。不过 ‘<ESC> <F1>’ 可以正常工作。）

最后，‘M-x describe-keymap’ 会提示输入键位图的名字（可补全），并列出该键位图中的所有键绑定。

12.9 帮助文件

除内置文档和手册之外，Emacs 还包含其他一些文件，描述诸如复制条件、发行说明、调试和报告缺陷的说明等主题。你可以用以下命令查看这些文件。除 ‘C-h g’ 外，它们的形式都是 ‘C-h C-CHAR’。

‘C-h C-c’ 显示你可以复制和再发行 Emacs 所依据的规则（‘describe-copying’）。‘C-h C-d’ 显示调试 Emacs 的帮助（‘view-emacs-debugging’）。‘C-h C-e’ 显示从哪里获取外部软件包的信息（‘view-external-packages’）。‘C-h C-f’ 显示 Emacs 常见问题解答列表（‘view-emacs-FAQ’）。‘C-h g’ 访问包含 GNU 工程信息的页面（https://www.gnu.org）（‘describe-gnu-project’）。‘C-h C-m’ 显示关于订购 Emacs 手册印刷本的信息（‘view-order-manuals’）。‘C-h C-n’ 显示新闻，其中列出此版 Emacs 的新特性（‘view-emacs-news’）。‘C-h C-o’ 显示如何订购或下载最新版 Emacs 及其他 GNU 软件（‘describe-distribution’）。‘C-h C-p’ 显示已知 Emacs 问题的列表，有时附有建议的变通办法（‘view-emacs-problems’）。‘C-h C-t’ 显示 Emacs 待办事项列表（‘view-emacs-todo’）。‘C-h C-w’ 显示关于 GNU Emacs 完全不提供担保的完整细节（‘describe-no-warranty’）。

12.10 活动文本与工具提示的帮助

在 Emacs 中，若干段“活动文本”（active text，指对鼠标点击或 <RET> 有特殊反应的文本）往往带有相关的帮助文本。这包括 Emacs 缓冲区中的超链接，也包括模式行的某些部分。在图形显示上，以及在某些支持鼠标

跟踪的文本终端上，把鼠标移到活动文本上会以“工具提示”（tooltip）的形式显示帮助文本。参见“工具提示”。

在不支持鼠标跟踪的终端上，你可以输入‘C-h .’（‘display-local-help’）来显示光标点处活动缓冲区文本的帮助文本。它会在回显区显示帮助文本。若要在光标点处有可用帮助文本时自动显示它，请把变量‘help-at-pt-display-when-idle’设为‘t’。如果你使用 Eldoc，请改为把变量‘eldoc-help-at-pt’设为‘t’。

13 标记与区域

Emacs 和许多其他应用程序一样，允许你选取缓冲区文本中任意的一部分，并调用作用于这些“选中文本”的命令。在 Emacs 中，我们把选中的文本称为“区域”（region）；对它的处理与其他程序中选中文本的处理非常相似，但也有重要的区别。

区域是缓冲区中“标记”与当前“光标点”之间的那部分文本。你可以在某个位置设置标记（例如用 ‘C-SPC’ 命令），然后把光标点移到你希望区域结束的地方，从而划定一个区域。（你也可以用鼠标划定区域。）

区域总是在光标点和标记之间延伸，无论二者中哪一个在文本中更靠前；每当你移动光标点，区域都会随之改变。

在文本中某个位置设置标记会“激活”它。当标记处于活动状态时，我们也说区域处于活动状态；Emacs 用 ‘region’ 字体高亮显示其中的文本来指示其范围（参见“自定义字体”）。

在某些非移动类命令之后（包括任何改变缓冲区文本的命令），Emacs 会自动“取消激活”标记；这同时关闭高亮显示。你也可以随时键入 ‘C-g’ 显式地取消激活标记（参见“退出与中止”）。

许多命令把它们作用的文本限制在活动区域之内。例如，‘M-%’ 命令（替换匹配的文本）通常作用于缓冲区的整个可访问部分，但如果你有活动区域，它就只作用于该区域。

即使标记未激活，它也是有用的。例如，你可以用标记环回到先前的标记位置。参见“标记环”。此外，某些命令即使对非活动区域也有作用（例如 “upcase-region”）。你还可以用 ‘C-x C-x’ 之类的命令重新激活区域。

上述行为是交互式会话中的默认行为，称为临时标记模式（Transient Mark mode）。禁用临时标记模式会让 Emacs 改用另一种行为，此时区域通常不会被高亮显示。参见“禁用临时标记模式”。

在一个缓冲区中设置标记不影响其他缓冲区中的标记。当你回到一个有活动标记的缓冲区时，标记仍在原来的位置。当多个窗口显示同一个缓冲区时，它们可以有不同的光标点值，因而有不同的区域，但它们共用一个标记位置。参见“多窗口”。通常只有选中的窗口会高亮显示其区域；不过，如果变量 ‘highlight-nonselected-windows’ 不为 ‘nil’，那么每个窗口都会高亮显示自己的区域。

还有另一种区域：矩形区域。参见“矩形”。

13.1 设置标记

以下是一些用于设置标记的命令：

- ‘C-<SPC>’ 在光标点处设置标记并激活它（‘set-mark-command’）。
- ‘C-@’ 同上。
- ‘C-x C-x’ 在光标点处设置标记并激活它，然后把光标点移到标记原来的位置（‘exchange-point-and-mark’）。
- ‘Drag-mouse-1’ 在你拖过的文本两端设置光标点和标记。
- ‘mouse-3’ 在光标点处设置标记，然后把光标点移到你单击的位置（‘mouse-save-then-kill’）。
- ‘Shifted cursor motion keys’ 若标记未激活，则在光标点处设置标记，然后再移动光标点。参见“Shift 选择”。

设置标记最常见的方式是用 'C-<SPC>'（'set-mark-command'）(1)。它会在光标点所在处设置标记并激活它。随后你可以把光标点移开，把标记留在原地。如果标记已经设置在光标点所在处，这条命令不会再设置一个新标记，而只是激活已有的标记。

例如，假设你想把缓冲区的一部分改成大写。为此，先走到所需文本的一端，键入 'C-<SPC>'，然后移动光标点，直到所需的那部分文本被高亮显示。现在键入 'C-x C-u'（'upcase-region'）。这会把区域中的文本改成大写，然后取消激活标记。

只要标记处于活动状态，你就可以键入 'C-g' 来取消激活它（参见"退出与中止"）。大多数作用于区域的命令也会自动取消激活标记，如上面例子中的 'C-x C-u'。

除了设置标记以便对区域进行操作之外，你也可以用它记住缓冲区中的某个位置（键入 'C-<SPC> C-<SPC>'），以后再跳回那里（键入 'C-u C-<SPC>'）。详情参见"标记环"。

命令 'C-x C-x'（'exchange-point-and-mark'）交换光标点和标记的位置。当你对光标点的位置满意、但想移动区域的另一端（标记所在处）时，'C-x C-x' 很有用。如有必要，再用一次 'C-x C-x' 就能把标记设在新位置，同时让光标点回到原来的位置。通常，如果标记未激活，这条命令会先在标记上一次设置的位置重新激活它，以确保区域保持高亮显示。不过，如果带前缀参数调用，它会保持标记未激活、区域不高亮显示；你可以借此以类似 'C-u C-<SPC>' 的方式跳到标记处。

你也可以用鼠标设置标记。如果你按下鼠标左键（'down-mouse-1'）并拖过一段文本，标记会设在你最初按下鼠标按钮的位置，光标点则放在你松开按钮的位置。另外，单击鼠标右键（'mouse-3'）会在光标点处设置标记，然后把光标点移到你单击的位置。关于这些鼠标命令的更详细说明，参见"鼠标编辑命令"。

最后，你还可以在键入某些光标移动命令（如 'S-<RIGHT>'、'S-C-f'、'S-C-n' 等）时按住 Shift 键来设置标记。这称为"Shift 选择"。它会在移动光标点之前于光标点处设置标记，但仅当此前没有通过 Shift 选择或鼠标命令设置过活动标记时才这样做。由鼠标命令和 Shift 选择设置的标记，行为与通常的标记略有不同：其后任何未按住 Shift 的光标移动命令都会自动取消激活它。详情参见"Shift 选择"。

许多插入文本的命令（如 'C-y'（'yank'，粘贴））会把标记设在所插入文本的另一端，但不激活它。这让你可以轻松回到那个位置（参见"标记环"）。当某条命令在回显区显示 'Mark set' 时，你就知道它这么做了。

在 X 下，每当活动区域发生变化，Emacs 都会把区域中的文本保存到"主选择"（primary selection）中。这让你可以通过 'mouse-2' 单击把该文本插入其他 X 应用程序。参见"与其他窗口应用剪贴"。

> 注：ASCII 中没有 'C-<SPC>' 这个字符；通常，在文本终端上键入 'C-<SPC>' 得到的是字符 'C-@'。这个键也绑定到 'set-mark-command'，所以除非你运气不好，碰上行为不同的文本终端，否则不妨把 'C-@' 就当作用 'C-<SPC>'。

13.2 标记文本对象的命令

以下命令用于围绕某个文本对象（如单词、列表、段落或页面）放置光标点和标记：

- 'M-@' 在下一个单词末尾设置标记（'mark-word'）。这不移动光标点。
- 'C-M-@' 在下一个平衡表达式末尾之后设置标记（'mark-sexp'）。这不移动光标点。
- 'M-h' 把光标点移到当前段落开头，并在末尾设置标记（'mark-paragraph'）。

- ‘C-M-h’ 把光标点移到当前 defun 开头，并在末尾设置标记（‘mark-defun’）。
- ‘C-x C-p’ 把光标点移到当前页开头，并在末尾设置标记（‘mark-page’）。
- ‘C-x h’ 把光标点移到缓冲区开头，并在末尾设置标记（‘mark-whole-buffer’）。

‘M-@’（‘mark-word’）把标记设在下一个单词的末尾（关于单词的信息，参见“单词”）。重复调用这条命令会一次前进一个单词，从而扩展区域。例外情况是：如果标记处于活动状态且位于光标点之前，‘M-@’ 会把标记从当前位置一次一个单词地向后移动。

这条命令还接受数字参数 N，意为把标记前进 N 个单词。负参数 −N 则把标记向后移动 N 个单词。

类似地，‘C-M-@’（‘mark-sexp’）把标记设在下一个平衡表达式的末尾（参见“平衡括号表达式”）。重复调用会把区域扩展到后续表达式，而正数或负数数字参数则把标记向前或向后移动指定数量的表达式。

上面列表中的其他命令会同时设置光标点和标记，从而界定缓冲区中的某个对象。‘M-h’（‘mark-paragraph’）标记段落（参见“段落”），‘C-M-h’（‘mark-defun’）标记顶层定义（参见“按 defun 移动”），‘C-x C-p’（‘mark-page’）标记页面（参见“页”）。重复调用同样作用相同，会把区域扩展到连续的对象；类似地，数字参数指定要把标记移动多少个对象。

‘C-x h’（‘mark-whole-buffer’）把整个缓冲区设为区域，即把光标点放在开头、标记放在末尾。

13.3 对区域进行操作

有了区域之后，下面是一些可以对它进行操作的方式：

- 用 ‘C-w’ 删除它（参见“删除与移动文本”）。
- 用 ‘M-w’ 把它复制到删除环（参见“粘贴”）。
- 用 ‘C-x C-l’ 或 ‘C-x C-u’ 转换大小写（参见“大小写转换命令”）。
- 用 ‘C-u C-/’ 撤销其中的改动（参见“撤销”）。
- 用 ‘M-%’ 替换其中的文本（参见“查询替换”）。
- 用 ‘C-x <TAB>’ 或 ‘C-M-\’ 对它缩进（参见“缩进”）。
- 用 ‘M-x fill-region’ 把它作为文本进行填充（参见“填充文本”）。
- 用 ‘M-$’ 检查其中单词的拼写（参见“拼写检查与纠正”）。
- 用 ‘M-x eval-region’ 把它作为 Lisp 代码求值（参见“求值 Emacs Lisp 表达式”）。
- 用 ‘C-x r s’ 把它保存到寄存器中（参见“寄存器”）。
- 把它保存到缓冲区或文件中（参见“累积文本”）。

某些命令在标记未激活时有默认行为，但标记激活时则作用于区域。例如，‘M-$’（‘ispell-word’）通常检查光标点处单词的拼写，但如果标记处于活动状态，它就检查区域中的文本（参见“拼写检查与纠正”）。通常，如果区域为空（即标记和光标点在同一位置），这类命令会采用默认行为。如果你希望它们对空区域也进行操作，请把变量 ‘use-empty-active-region’ 改为 ‘t’。

如参见“删除文本”所述，<DEL>（‘backward-delete-char’）和 <Delete>（‘delete-forward-char’）命令也是如此。如果标记处于活动状态，它们会删除区域中的文本。（例外情况是：如果你提供数字参数 N，且 N 不为 1，那么无论标记是否活动，这些命令都会删除 N 个字符。）如果你把变量 ‘delete-active-region’ 改为 ‘nil’，那

么标记活动时这些命令的行为就不再不同。如果你把该值改为‘kill’，这些命令就会“删除”（kill）区域而非删除（delete）它（参见“删除与移动文本”）。

其他命令总是作用于区域，没有默认行为。这类命令的名字中通常含有‘region’一词，例如‘C-w’（‘kill-region’）和‘C-x C-u’（‘upcase-region’）。如果标记未激活，它们就作用于“非活动区域”——即光标点与标记上一次设置的位置之间的文本（参见“标记环”）。要禁用这一行为，请把变量‘mark-even-if-inactive’改为‘nil’。这样，这些命令在标记未激活时就会改为发出错误信号。

默认情况下，即使标记处于活动状态，文本插入也照常进行——例如，键入‘α’会插入字符‘α’，然后取消激活标记。次模式 Delete Selection 模式会改变这一行为：如果启用该模式，那么在标记活动时插入文本会先删除区域中的文本。不过，你可以通过定制‘delete-selection-temporary-region’选项来调整这一行为。它的默认值是‘nil’，但你可以把它设为‘t’，此时只有临时活动的区域会被替换：即通过拖动鼠标（参见“设置标记”）或 Shift 选择（参见“Shift 选择”）设置的区域，以及在临时标记模式禁用时由‘C-u C-x C-x’设置的区域。你还可以进一步调整：把‘delete-selection-temporary-region’设为‘selection’，此时由‘C-u C-x C-x’产生的临时区域不会被替换，只有通过拖动鼠标或 Shift 选择激活的区域才会被替换。要打开或关闭 Delete Selection 模式，请键入‘M-x delete-selection-mode’。要仅在当前缓冲区中打开或关闭 Delete Selection 模式，请键入‘M-x delete-selection-local-mode’。

13.4 标记环

每个缓冲区都在“标记环”中记住标记先前的位置。设置标记的命令也会把旧标记压入这个环。标记环的用途之一就是记住你可能想回去的各个位置。

- ‘C-<SPC> C-<SPC>’设置标记，把它压入标记环，但不激活它。
- ‘C-u C-<SPC>’把光标点移到标记原来的位置，并从先前的标记环中恢复标记。

当你想用标记记住一个可能想返回的位置时，命令‘C-<SPC> C-<SPC>’很方便。它把当前光标点压入标记环，而不激活标记（激活标记会导致 Emacs 高亮显示区域）。这实际上是连续两次调用‘C-<SPC>’（‘set-mark-command’）：第一次‘C-<SPC>’设置标记，第二次‘C-<SPC>’取消激活它。（当临时标记模式关闭时，‘C-<SPC> C-<SPC>’则改为临时启用临时标记模式；参见“禁用临时标记模式”。）

要回到某个标记过的位置，请带前缀参数使用‘set-mark-command’：‘C-u C-<SPC>’。它把光标点移到标记原来的位置，并且如果标记原本处于活动状态，就取消激活它。此后每再执行一次‘C-u C-<SPC>’，都会跳到标记环中保存的一个更早的位置。你以这种方式经过的位置不会丢失，它们会被放到环的末尾。

如果你把‘set-mark-command-repeat-pop’设为非‘nil’，那么在键入‘C-u C-<SPC>’之后，你可以立即改用‘C-<SPC>’（而不必再键入‘C-u C-<SPC>’）来在标记环中循环。默认情况下，‘set-mark-command-repeat-pop’为‘nil’。

每个缓冲区都有自己的标记环。所有编辑命令都使用当前缓冲区的标记环。特别是，‘C-u C-<SPC>’总是停留在同一个缓冲区中。

变量‘mark-ring-max’指定标记环中最多保留多少个条目。默认是 16 个条目。如果已经有这么多条目，而又要压入一个，则列表中最早的那个会被丢弃。重复执行‘C-u C-<SPC>’会在环中当前的各个位置之间循环。

如果你想一次又一次地回到同一个地方，标记环可能不够方便。如果是这样，你可以把该位置记录到寄存器中，以后再去取回（参见“在寄存器中保存位置”）。

13.5 全局标记环

除了每个缓冲区各自拥有的普通标记环之外，Emacs 还有一个“全局标记环”。每当你设置标记时，如果自上次设置标记以来你切换过缓冲区，那么这个标记除了记入当前缓冲区自己的标记环之外，还会记入全局标记环。因此，全局标记环记录了你依次待过的一系列缓冲区，以及每个缓冲区中你设置标记的位置。全局标记环的长度由‘global-mark-ring-max’控制，默认是 16。

注意，只有当某条命令设置标记时，标记才会被记入全局标记环。如果只是激活一个已存在的标记，例如在标记已经设置的地方使用‘C-<SPC>’（参见“设置标记”），那不会把该标记压入全局标记环。

命令‘C-x C-<SPC>’（‘pop-global-mark’）会跳到全局标记环中最新条目所在的缓冲区和位置。它还会旋转这个环，因此连续使用‘C-x C-<SPC>’会依次带你回到更早的缓冲区和标记位置。

13.6 Shift 选择

如果你在按住 Shift 键的同时键入光标移动命令，它会在移动光标点之前设置标记，从而使区域从光标点的原始位置延伸到新位置。这一特性称为“Shift 选择”。它类似于其他编辑器中选中文本的方式。

通过 Shift 选择设置的标记，行为与我们上面描述的略有不同。首先，除了取消激活标记的通常方式（如修改缓冲区文本或键入‘C-g’）之外，任何*未按住 Shift* 的光标移动命令都会取消激活该标记。其次，此后任何*按住 Shift* 的光标移动命令都不会重新设置标记。因此，一连串按住 Shift 的光标移动命令会连续地调整区域。

Shift 选择只有在按住 Shift 的光标移动键尚未绑定到其他命令时才有效（参见“自定义”）。例如，如果你把‘S-C-f’绑定到另一条命令，那么键入‘S-C-f’会运行那条命令，而不会执行‘C-f’（‘forward-char’）的 Shift 选择版本。

通过鼠标命令设置的标记，行为与通过 Shift 选择设置的标记相同（参见“设置标记”）。例如，如果你通过拖动鼠标指定了区域，就可以继续用按住 Shift 的光标移动命令扩展该区域。在两种情况下，任何未按住 Shift 的光标移动命令都会取消激活标记。

要关闭 Shift 选择，请把‘shift-select-mode’设为‘nil’。这样做并不会禁用通过鼠标命令设置标记。如果你把‘shift-select-mode’设为‘permanent’，那么未被 Shift 转换的光标移动键就不会取消激活标记，因此，举例来说，先前命令设置的区域可以用 Shift 选择来扩展，而未按住 Shift 的光标移动键也会扩展由 Shift 选择设置的区域。

13.7 禁用临时标记模式

标记和区域的默认行为——设置标记即激活它并高亮显示区域——称为临时标记模式。这是一个次模式，在交互式会话中默认启用。你可以用‘M-x transient-mark-mode’来切换它，或者用‘Options’菜单中的‘Highlight Active Region’菜单项。关闭它会将 Emacs 切换到另一种操作方式：

- 用‘C-<SPC>’或‘C-x C-x’之类的命令设置标记时，不会高亮显示区域。因此，你无法通过看来判断标记在什么位置；你得记住它。

这个问题的通常解决办法是设置标记后尽快使用它，趁你还没忘记它在哪里。你也可以用‘C-x C-x’检查标记的位置，它会交换光标点和标记的位置（*note Setting Mark::）。

- 某些命令在标记活动时通常作用于区域，此时则不再如此。例如，正常情况下，如果标记处于活动状态，‘M-%’（‘query-replace’）会在区域内执行替换。当临时标记模式关闭时，它总是从光标点作用到缓冲区末尾。以这种方式行事的命令会在各自的文档中说明。

当临时标记模式关闭时，你可以用‘C-<SPC> C-<SPC>’或‘C-u C-x C-x’临时激活它。

- ‘C-<SPC> C-<SPC>’在光标点处设置标记（如同普通的‘C-<SPC>’），并只启用一次临时标记模式，直到标记被取消激活为止。（这并不是一条独立的命令；你是在两次使用‘C-<SPC>’命令。）
- ‘C-u C-x C-x’交换光标点和标记，激活标记并临时启用临时标记模式，直到标记下次被取消激活为止。（这就是带前缀参数的‘C-x C-x’命令，即‘exchange-point-and-mark’。）

这些命令设置或激活标记，并只启用临时标记模式直到标记被取消激活为止。你可能想使用它们的一个原因是：当临时标记模式关闭时，某些命令作用于整个缓冲区而不是区域。临时启用临时标记模式就为你提供了一种在区域上使用这些命令的办法。

当你用鼠标指定区域（参见“设置标记”）或用 Shift 选择指定区域（参见“Shift 选择”）时，同样会临时激活临时标记模式并高亮显示区域。

14 删除与移动文本

在 Emacs 中，“删除”（killing）指擦除文本并将其复制到“删除环”（kill ring）中。“粘贴”（yanking）指把删除环中的文本重新带回缓冲区。（有些应用程序用“剪切”和“粘贴”来称呼类似操作。）删除环之所以得名，是因为可以把它想象成一组按环形排列的文本块，你可以按循环顺序访问它们。参见“删除环”。

删除和粘贴是在 Emacs 内移动或复制文本最常见的方式。这种方式非常灵活，因为有各种命令可以删除许多不同类型的语法单元。

14.1 删除与删除环

大多数从缓冲区中擦除文本的命令都会将其保存在删除环中（参见“删除环”）。这些命令称为“kill”命令，其名称通常包含单词‘kill’（例如‘kill-line’）。删除环会保存最近删除的若干段文本，而不只是最后一段，因此删除是一项非常安全的操作：你不必太担心丢失先前删除的文本。删除环由所有缓冲区共享，因此在一个缓冲区中删除的文本可以粘贴到另一个缓冲区中。

当你使用‘C-/’（‘undo’）撤销某个 kill 命令时（参见“撤销”），被删除的文本会回到缓冲区中，但不会从删除环中移除。

在图形显示环境中，删除文本还会将其复制到系统剪贴板。参见“图形显示环境中的“剪切与粘贴”操作”。

擦除文本但不将其保存到删除环的命令称为“delete”命令；其名称通常包含单词‘delete’。其中包括每次只删除一个字符的‘C-d’（‘delete-char’）和‘<DEL>’（‘delete-backward-char’），以及只删除空格或换行符的命令。能够擦除大量重要数据的命令通常改为执行一次 kill 操作。

你也可以使用鼠标来删除和粘贴。参见“图形显示环境中的“剪切与粘贴”操作”。

- 删除：用于删除少量文本和空白区域的命令。
- 按行删除：如何一次删除整行文本。
- 其他删除命令：删除大块文本以及单词、句子等语法单元的命令。
- 删除选项：影响删除操作的选项。

14.1.1 基本删除

删除指擦除文本但不将其保存到删除环中。大多数情况下，Emacs 中删除文本的命令只擦除一个字符或仅擦除空白。

- ‘<DEL>’‘<BACKSPACE>’：删除前一个字符；若区域处于活动状态，则删除区域中的文本（‘delete-backward-char’）。
- ‘<Delete>’：删除下一个字符；若区域处于活动状态，则删除区域中的文本（‘delete-forward-char’）。
- ‘C-d’：删除下一个字符（‘delete-char’）。
- ‘M-\’：删除光标点周围的空格和制表符（‘delete-horizontal-space’）。
- ‘M-x just-one-space’：删除光标点周围的空格和制表符，只留下一个空格。
- ‘M-<SPC>’：以灵活的方式删除光标点周围的空格和制表符（‘cycle-spacing’）。

- ‘C-x C-o’：删除当前行周围的空行（‘delete-blank-lines’）。
- ‘M-^’：删除中间的换行符及其后的任何缩进，从而连接两行（‘delete-indentation’）。

我们已经介绍过基本的删除命令‘<DEL>’（‘delete-backward-char’）、‘<delete>’（‘delete-forward-char’）和‘C-d’（‘delete-char’）。参见“删除文本”。带数字参数时，它们删除指定数量的字符。若省略数字参数或参数为一，且区域处于活动状态，则‘<DEL>’和‘<delete>’删除区域中的全部文本（参见“对区域进行操作”）。

其他删除命令是那些只删除空白字符的命令：空格、制表符和换行符。‘M-\’（‘delete-horizontal-space’）删除光标点前后的所有空格和制表符。带前缀参数时，它只删除光标点之前的空格和制表符。

‘M-x just-one-space’删除光标点周围的制表符和空格，但会在光标点之前留下一个空格，不管此前有多少个空格（即使此前一个空格也没有）。带数字参数 N 时，若 N 为正，则在光标点前留下 N 个空格；若 N 为负，则除空格和制表符外还删除换行符，并在光标点前留下 −N 个空格。

命令‘cycle-spacing’（‘M-<SPC>’）的作用类似于‘just-one-space’的更灵活版本。若连续反复调用，它会以循环方式执行由‘cycle-spacing-actions’定义的不同空白清理动作。默认情况下，第一次调用与‘just-one-space’相同，第二次像‘delete-horizontal-space’一样删除光标点周围的所有空白字符，第三次恢复原来的空白字符；然后如此循环。若带前缀参数调用，则每个动作都会使用该参数值。用户选项‘cycle-spacing-actions’还可以包含其他成员；详情请参阅该选项的文档字符串。

‘C-x C-o’（‘delete-blank-lines’）删除当前行之后的所有空行。如果当前行是空行，它还会删除当前行之前的所有空行（留下一个空行，即当前行）。如果当前行是单独一个空行，则删除该行。

‘M-^’（‘delete-indentation’）通过删除一个换行符及其周围的空格，将当前行与上一行连接起来，通常留下一个空格。参见“缩进”。

命令‘delete-duplicate-lines’在区域中搜索相同的行，并只保留每一组相同行中的一份。通常它会保留每个重复行的第一个实例，但带‘C-u’前缀参数时会保留最后一个。带‘C-u C-u’前缀参数时，它只搜索相邻的相同行。这是一种更高效的操作方式，适用于这些行已经排好序的情况。带‘C-u C-u C-u’前缀参数时，它会保留重复的空行。

14.1.2 按行删除

- ‘C-k’：删除该行剩余部分，或删除一行或多行（‘kill-line’）。
- ‘C-S-backspace’：一次删除整行（‘kill-whole-line’）。

最简单的 kill 命令是‘C-k’（‘kill-line’）。若在行尾使用，它会删除行尾的换行符，把下一行并入当前行（因此，空行会被完全删除）。否则，‘C-k’会删除从光标点到行尾的所有文本；如果光标点原本在行首，这会使该行变为空行。

在判断属于哪种情况时，行尾的空格和制表符会被忽略。只要光标点位于该行最后一个非空白字符之后，就可以确定‘C-k’会删除换行符。要删除一整行非空行，请移到行首并键入两次‘C-k’。

这里的“行”指逻辑文本行，而不是屏幕行（参见“续行”）。

当给‘C-k’一个正参数 N 时，它删除 N 行以及这些行之后的换行符（当前行中光标点之前的文本不会被删除）。带负参数 −N 时，它删除当前行之前的 N 行，连同当前行中光标点之前的文本。参数为零的‘C-k’删除当前行中光标点之前的文本。

如果变量 'kill-whole-line' 为非 'nil'，那么在行首最开头处使用 'C-k' 会删除整行，包括其后的换行符。该变量通常为 'nil'。

'C-S-backspace'（'kill-whole-line'）会删除整行，包括其换行符，无论光标点在该行中的什么位置。注意，许多文本终端会阻止你键入键序列 'C-S-backspace'。

14.1.3 其他删除命令

- 'C-w'：删除区域（'kill-region'）。
- 'M-w'：把区域复制到删除环（'kill-ring-save'）。
- 'M-d'：删除下一个单词（'kill-word'）。参见"单词"。
- 'M-<DEL>'：向后删除一个单词（'backward-kill-word'）。
- 'C-x <DEL>'：删除到句子开头（'backward-kill-sentence'）。参见"句子"。
- 'M-k'：删除到句子末尾（'kill-sentence'）。
- 'C-M-k'：删除后面一个平衡表达式（'kill-sexp'）。参见"平衡括号表达式"。
- 'M-z CHAR'：删除到 CHAR 下一次出现处（包括该字符）（'zap-to-char'）。
- 'M-x zap-up-to-char CHAR'：删除到 CHAR 下一次出现之前（不包括该字符）。

常用的 kill 命令之一是 'C-w'（'kill-region'），它删除区域中的文本（参见"标记与区域"）。类似地，'M-w'（'kill-ring-save'）把区域中的文本复制到删除环，而不从缓冲区中移除。如果你键入 'C-w' 或 'M-w' 时标记未激活，该命令会作用于光标点与上次设置标记的位置之间的文本（参见"对区域进行操作"）。

Emacs 还提供命令来删除特定的语法单元：用 'M-<DEL>' 和 'M-d' 删除单词（参见"单词"）；用 'C-M-k' 删除平衡表达式（参见"平衡括号表达式"）；用 'C-x <DEL>' 和 'M-k' 删除句子（参见"句子"）。

命令 'M-z'（'zap-to-char'）把删除与搜索结合起来：它读取一个字符，并删除从光标点到该字符在缓冲区中下一次出现处（包括该字符）的文本。数字参数充当重复次数；负参数表示向后搜索并删除光标点之前的文本。会维护以前使用过的字符历史，可以通过 'M-p'/'M-n' 键访问。当需要使用的字符必须通过复杂的输入法输入时，这尤其有用。类似的命令 'zap-up-to-char' 删除从光标点到某个字符下一次出现之前的文本（不包括该字符），数字参数同样充当重复次数。

14.1.4 删除选项

某些专用缓冲区包含"只读文本"，这些文本无法修改，因此无法被删除。kill 命令在只读缓冲区中的工作方式很特殊：它们会移过文本并将其复制到删除环，但不实际从缓冲区中删除。通常，发生这种情况时它们还会发出蜂鸣声并显示错误消息。但如果你把变量 'kill-read-only-ok' 设置为非 'nil' 值，它们只会在回显区打印一条消息，说明文本为什么没有被擦除。

在把删除的内容保存到删除环之前，你可以使用 'kill-transform-function' 转换字符串。调用时会传入要删除的字符串，它应返回你希望保存的字符串。它也可以返回 'nil'，此时该字符串不会被保存到删除环。例如，如果你从不希望把纯空白字符串保存到删除环，可以这样写：

```
(setq kill-transform-function
      (lambda (string)
        (and (not (string-blank-p string))
```

```
            string)))
```

如果把变量‘kill-do-not-save-duplicates’改为非‘nil’值，后续相同的删除只会产生一个删除环条目，不会重复。

如果启用次模式‘kill-ring-deindent-mode’，保存到删除环的文本会按照保存文本第一行的缩进量减少缩进。也就是说，如果保存文本的第一行缩进了 N 列，该模式会从每个被保存行的缩进中移除这个列数。

14.2 粘贴

“粘贴”（yanking）指重新插入先前删除的文本。移动或复制文本的通常做法是：先删除它，再在别处粘贴它。

- ‘C-y’：把最近一次删除的内容粘贴到缓冲区中光标点处（‘yank’）。
- ‘M-y’：用更早删除的一批文本替换刚刚粘贴的文本（‘yank-pop’），或者从先前删除的各批文本列表中进行选择。参见“粘贴更早删除的内容”。
- ‘C-M-w’：使接下来的命令（若为 kill 命令）追加到上一次删除的内容之后（‘append-next-kill’）。参见“追加删除”。

基本的粘贴命令是‘C-y’（‘yank’）。它插入最近一次删除的内容，并把光标留在插入文本的末尾。它还会把标记设置在插入文本的开头，但不激活标记；这样，如果需要，你可以用‘C-u C-<SPC>’轻松跳到该位置（参见“标记环”）。

使用不带数字的普通前缀参数（‘C-u C-y’）时，该命令会把光标留在插入文本之前，并把标记设置在末尾。使用任何其他前缀参数都会指定更早的一次删除；例如，‘C-u 4 C-y’会重新插入最近第四次删除的内容。参见“粘贴更早删除的内容”。

在图形显示环境和支持该功能的文本模式显示环境中，‘C-y’会先检查是否有其他应用程序比 Emacs 最近一次删除更近地把文本放入了系统剪贴板。若是，则改为插入剪贴板中的文本。因此，Emacs 实际上把其他应用程序中执行的“剪切”或“复制”剪贴板操作当作 Emacs 的删除处理，只是这些操作不会记录到删除环中。详情参见“图形显示环境中的“剪切与粘贴”操作”。

14.2.1 删除环

“删除环”是一个列表，其中的元素是以前删除的文本块。只有一个删除环，由所有缓冲区共享，因此你可以在一个缓冲区中删除文本，再在另一个缓冲区中粘贴它。这是把一个缓冲区中的文本移到另一个缓冲区的常用方法。（还有其他几种方法：例如，可以把文本存到寄存器中，参见“寄存器”；有关移动文本的其他方法，参见“累积文本”。）

删除环中的最大条目数由变量‘kill-ring-max’控制。默认值为 120。达到此上限后若执行新的删除，Emacs 会删除删除环中最旧的条目以腾出空间。

删除环的实际内容存储在名为‘kill-ring’的变量中；你可以用‘C-h v kill-ring’查看删除环的全部内容。

14.2.2 粘贴更早删除的内容

如参见“粘贴”中所述，可以给‘C-y’一个数字参数，以粘贴不再是最新一次删除的文本。如果你记得自己想要哪个删除环条目，这会很有用。如果不记得，可以使用‘M-y’（‘yank-pop’）命令在可能的内容之间循环，或选择更早的某次删除。

如果上一个命令是粘贴命令，‘M-y’ 会取刚刚粘贴的文本，并用更早删除的文本替换它。因此，要找回倒数第二次删除的文本，先用 ‘C-y’ 粘贴最近一次删除的内容，再用 ‘M-y’ 把它替换为上一次删除的内容。这只在 ‘C-y’ 或另一个 ‘M-y’ 之后才有效。（如果在其他命令之后调用 ‘M-y’，它的工作方式不同，见下文。）

你可以借助一个“最近粘贴指针”（last-yank pointer）来理解 ‘M-y’ 的这种操作方式，该指针指向删除环中的某个条目。每次执行删除时，最近粘贴指针都会移到环前端新生成的条目。‘C-y’ 粘贴最近粘贴指针所指向的条目。在 ‘C-y’ 或另一个 ‘M-y’ 之后执行 ‘M-y’，会把最近粘贴指针移到前一个条目，缓冲区中的文本也随之改变。连续执行足够多的 ‘M-y’ 命令，可以把指针移到环中的任何条目，因此你可以把任何条目取到缓冲区中。最终指针会到达环的末端；再执行一次 ‘M-y’ 就会绕回第一个条目。

‘M-y’ 会让最近粘贴指针在环中移动，但不会改变环中条目的顺序；该顺序始终是从前端的最近一次删除到仍被记住的最旧条目。

在 ‘C-y’ 或 ‘M-y’ 之后使用时，‘M-y’ 可以接受数字参数，表示要把最近粘贴指针前进多少个条目。负参数会把指针移向环的前端；从环的前端继续移动时，它会绕到最后一个条目，并从那里继续向前。

一旦想要的文本被带入缓冲区，就可以停止执行 ‘M-y’ 命令，最后粘贴的文本会留在那里。它只是删除环条目的副本，因此在缓冲区中编辑它不会改变环中的内容。只要没有执行新的删除，最近粘贴指针就会留在删除环中的同一位置，因此重复 ‘C-y’ 会粘贴同一份先前删除内容的另一个副本。

带数字参数调用 ‘C-y’ 时，也会把最近粘贴指针设置为其所粘贴的条目。

你也可以在不是粘贴命令的命令之后调用 ‘M-y’。在这种情况下，‘M-y’ 会在迷你缓冲区中提示你选择以前删除的某段文本。你可以使用迷你缓冲区历史命令（参见“迷你缓冲区历史”）在删除环的各个条目中浏览或搜索，直到找到想要重新插入的条目。也可以使用补全命令（参见“补全命令”）对删除环条目列表中的条目进行补全，或者弹出 ‘*Completions*’ 缓冲区，列出可供选择的候选条目。选定删除环条目后，你可以选择在迷你缓冲区中编辑它。最后，键入 ‘RET’ 退出迷你缓冲区，并插入所选删除环条目的文本。与在另一个粘贴命令之后使用 ‘M-y’ 的情况一样，最近粘贴指针会停留在你刚刚粘贴的文本上，无论它是以前删除的文本之一，还是你在插入前编辑过的删除环条目。（在后一种情况下，编辑后的条目会被添加到删除环前端。）因此，在这里键入 ‘C-y’ 同样会粘贴刚插入文本的另一个副本。

在不是粘贴命令的命令之后，使用不带数字的普通前缀参数（‘C-u M-y’）调用 ‘M-y’ 时，它会像 ‘C-y’ 那样把光标留在插入文本之前，并把标记设置在末尾。

14.2.3 追加删除

通常，每个 kill 命令都会向删除环压入一个新条目。然而，连续两个或多个 kill 命令会把它们的文本合并为一个条目，因此一次 ‘C-y’ 就能把全部文本作为一个整体粘贴出来，就像删除前一样。

因此，如果想把文本作为一个整体粘贴，不必用一条命令删除全部内容；你可以一行接一行或一个单词接一个单词地删除，直到全部删完，仍然可以一次全部取回。

从光标点向前删除的命令会把文本加到前一次删除文本的末尾；从光标点向后删除的命令会把文本加到开头。这样，任意混用向前和向后 kill 命令的序列都会把所有删除的文本放入同一个条目，而不会重新排列。数字参数不会打断追加删除的序列。例如，假设缓冲区包含以下文本：

```
This is a line *of sample text.
```

其中 · 表示光标点。如果键入 'M-d M-<DEL> M-d M-<DEL>'，交替向前和向后删除，最终删除环中会有一个条目 'a line of sample'，而缓冲区中剩下 'This is text.'。（注意 'is' 和 'text' 之间有两个空格，你可以用 'M-<SPC>' 或 'M-q' 清理掉。）

删除同一段文本的另一种方法是：用 'M-b M-b' 向后移动两个单词，然后用 'C-u M-d' 向前删除全部四个单词。这在缓冲区和删除环中产生的结果完全相同。'M-f M-f C-u M-<DEL>' 也会删除同一段文本，只是全部向后进行；结果同样相同。删除环条目中的文本顺序始终与删除前在缓冲区中的顺序相同。

如果某个 kill 命令与上一个 kill 命令之间隔着其他命令（不只是数字参数），它就会在删除环上开始一个新条目。但你可以通过事先键入 'C-M-w'（'append-next-kill'）强制它与上次删除的文本合并。'C-M-w' 会告诉紧随其后的命令（如果它是 kill 命令）把这次删除视为之前删除序列的一部分。和往常一样，如果命令向前删除，删除的文本会追加到前一次删除的文本之后；如果向后删除，则前置到其前面。这样，你可以删除几段彼此分离的文本，并把它们累积起来，以便一次粘贴回同一处。

紧跟在 'M-w'（'kill-ring-save'）之后的 kill 命令不会追加到 'M-w' 复制进删除环的文本之后。

14.3 图形显示环境中的"剪切与粘贴"操作

在大多数图形桌面环境中，你可以使用称为"剪贴板"（clipboard）的系统设施在不同应用程序之间传输数据（通常是文本）。在 X 上，还有两个类似的设施可用：主选择（primary selection）和次选择（secondary selection）。当 Emacs 在图形显示环境中运行时，它的 kill 和粘贴命令会与这些设施集成，因此你可以轻松地在 Emacs 与其他图形应用程序之间传输文本。

默认情况下，Emacs 使用 UTF-8 作为程序间文本传输的编码系统。如果发现粘贴的文本不符合预期，可以键入 'C-x <RET> x' 或 'C-x <RET> X' 指定其他编码系统。也可以自定义 'x-select-request-type' 来请求不同的数据类型。参见"用于进程间通信的编码系统"。

14.3.1 使用剪贴板

"剪贴板"是大多数图形应用程序用于"剪切和粘贴"的设施。当剪贴板存在时，Emacs 中的 kill 和粘贴命令会利用它。

当你用 'C-w'（'kill-region'）之类的命令删除某些文本，或用 'M-w'（'kill-ring-save'）之类的命令将其复制到删除环时，该文本也会被放入剪贴板。

当 Emacs 的 kill 命令把文本放入剪贴板时，剪贴板中现有的内容通常会丢失。也可以选择让 Emacs 把剪贴板现有内容保存到删除环，以免丢失旧的剪贴板数据。如果 'save-interprogram-paste-before-kill' 被设置为一个数字，那么当数据（按字符数计）小于该数字时，就会把它复制过来。如果该变量是任何其他非 'nil' 值，则总是复制数据——风险是如果数据很大，会消耗大量内存。

诸如 'C-y'（'yank'）之类的粘贴命令也会使用剪贴板。如果另一个应用程序"拥有"剪贴板——也就是说，你在那里剪切或复制文本的时间比在 Emacs 中执行最后一次 kill 命令更近——那么 Emacs 会从剪贴板而不是删除环粘贴。

通常，用 'M-y'（'yank-pop'）轮换删除环不会改变剪贴板。但是，如果把 'yank-pop-change-selection' 改为 't'，那么 'M-y' 会把新粘贴的内容保存到剪贴板。

要阻止 kill 和粘贴命令访问剪贴板，请把变量 'select-enable-clipboard' 改为 'nil'。

程序可以在剪贴板上放置纯文本以外的东西。例如，网页浏览器通常允许你对图像选择"复制图像"，该图像就会被放入剪贴板。在支持的平台上，Emacs 可以用 'yank-media' 命令粘贴这些对象——但仅限于支持该命令的模式。默认情况下，它会自动选择剪贴板中可用的首选媒体类型，但可以通过给该命令加前缀参数来覆盖这一选择（参见《Emacs Lisp 参考手册》的"Yanking Media"）。

许多 X 桌面环境支持称为"剪贴板管理器"的功能。如果在 Emacs 是剪贴板数据当前"所有者"时退出，并且有剪贴板管理器正在运行，Emacs 会把剪贴板数据转交给剪贴板管理器，以免丢失。在某些情况下，这可能会导致退出 Emacs 时出现延迟；如果希望阻止 Emacs 向剪贴板管理器传输数据，请把变量 'x-select-enable-clipboard-manager' 改为 'nil'。

由于包含 NUL 字节的字符串在通过剪贴板传递时通常会被截断，Emacs 会在把它们传输到系统剪贴板之前，将这些字符替换为 '"\0"'。

在 Emacs 24 之前，删除和粘贴命令使用的都是主选区（primary selection）（参见"与其他窗口应用剪贴"），而不是剪贴板。若你偏好这种行为，请把 'select-enable-clipboard' 改为 'nil'，把 'select-enable-primary' 改为 't'，并把 'mouse-drag-copy-region' 改为 't'。在这种情况下，你可以用下列命令显式地操作剪贴板：'clipboard-kill-region' 删除区域并把它存入剪贴板；'clipboard-kill-ring-save' 把区域复制到删除环并存入剪贴板；'clipboard-yank' 在光标点处粘贴剪贴板的内容。

14.3.2 与其他窗口应用剪贴

在 X Window System、PGTK 和 Haiku 下，存在一个"主选区"，其中含有在 X 应用中最近选中的一段文本（通常通过拖动鼠标选中）。通常，可以按 'mouse-2' 把这段文本插入到其他 X 应用中。主选区与剪贴板彼此独立。它的内容更易失：每次你用鼠标选中文本时它都会被覆盖，而剪贴板只会被显式的剪切或复制命令覆盖。

在 X 下，只要区域处于活动状态（参见"标记与区域"），区域中的文本就会存入主选区。无论区域是靠拖动或点击鼠标（参见"鼠标编辑命令"）建立的，还是靠键盘命令（例如输入 'C-<SPC>' 并移动光标点；参见"设置标记"）建立的，都是如此。

若把变量 'select-active-regions' 改为 'only'，Emacs 只把临时活动的区域存入主选区，即用鼠标或按住 Shift 选择所建立的区域（参见"Shift 选择"）。若把 'select-active-regions' 改为 'nil'，Emacs 就完全不把活动区域存入主选区。

要把主选区插入到 Emacs 缓冲区，在你想插入的位置点击 'mouse-2'（'mouse-yank-primary'）。参见"鼠标编辑命令"。若 'select-enable-primary' 已设置，你也可以用普通的 Emacs 粘贴命令（'C-y'）插入这段文本（参见"使用剪贴板"）。

默认情况下，即使另一个程序选中了文本，Emacs 仍让区域保持活动；这与典型的 X 行为相反。若想让 Emacs 在另一个程序把数据放入主选区之后取消激活区域，请启用全局次模式 'lost-selection-mode'。

MS-Windows 没有主选区，但 Emacs 通过内部保存选中的文本，在单个 Emacs 会话内模拟它。因此，所有与主选区相关的功能和命令在 Windows 上都能像在 X 上一样工作，用于同一会话内的剪切与粘贴，但不能跨 Emacs 会话，也不能与其他应用程序配合使用。

14.3.3 辅助选区

除主选区之外，X Window System 还提供了另一个类似的设施，称为“辅助选区”。如今很少有 X 应用使用辅助选区，但你可以用下列 Emacs 命令访问它：

‘M-Drag-mouse-1’ 设置辅助选区，一端在按下鼠标按钮处，另一端在松开处（‘mouse-set-secondary’）。拖动时，选中的文本会用 ‘secondary-selection’ 字体（face）高亮显示。若把鼠标拖出窗口顶部或底部，窗口会自动滚动，就像 ‘mouse-set-region’ 一样（参见“鼠标编辑命令”）。

此命令不改变删除环。

‘M-mouse-1’ 为“辅助选区”设置一个端点（‘mouse-start-secondary’）；用 ‘M-mouse-3’ 设置另一端并完成选择。此命令在开始新的辅助选区时会取消任何已有的辅助选区。

‘M-mouse-3’ 设置辅助选区（‘mouse-secondary-save-then-kill’），一端在你点击 ‘M-mouse-3’ 的位置，另一端在先前用 ‘M-mouse-1’ 指定的位置。这还会把选中的文本放入删除环。在同一位置再按一次 ‘M-mouse-3’ 会删除刚建立的辅助选区所选中的文本。

‘M-mouse-2’ 在你点击的位置插入辅助选区，并把光标点放在粘贴文本的末尾（‘mouse-yank-secondary’）。

双击或三击 ‘M-mouse-1’ 会按单词和按行操作，很像 ‘mouse-1’。

若 ‘mouse-yank-at-point’ 为非 ‘nil’，‘M-mouse-2’ 在光标点处粘贴。这时你具体点击哪里并不重要，甚至点在该框架的哪个窗口上也不重要。参见“鼠标编辑命令”。这个用户选项也影响交互式搜索：若它为非 ‘nil’，在框架内任意位置用鼠标粘贴都会把文本加入搜索字符串。

14.4 累积文本

通常我们通过删除和粘贴来复制或移动文本，但还有其他方便的方法，可把一块文本复制到许多地方，或把许多分散的文本块复制到一个地方。这里介绍把分散的文本片段累积到缓冲区或文件中的命令。

‘M-x append-to-buffer’ 把区域追加到指定缓冲区的内容之后。‘M-x prepend-to-buffer’ 把区域添加到指定缓冲区的内容之前。‘M-x copy-to-buffer’ 把区域复制到指定缓冲区，并删除该缓冲区原有的内容。‘M-x insert-buffer’ 把指定缓冲区的内容插入当前缓冲区的光标点处。‘M-x append-to-file’ 把区域追加到指定文件的内容末尾。

要累积文本到缓冲区，使用 ‘M-x append-to-buffer’。它会读取一个缓冲区名，然后把区域的一份副本插入指定缓冲区。若你指定一个不存在的缓冲区，‘append-to-buffer’ 会创建它。文本插入到该缓冲区中光标点所在的位置。若你一直在用该缓冲区编辑，复制的文本会从当时光标点所在之处开始，插入到缓冲区文本的中间。

该缓冲区中的光标点留在被复制文本的末尾，所以连续使用 ‘append-to-buffer’ 会按复制的顺序把文本累积到指定缓冲区。严格说来，‘append-to-buffer’ 并不总是追加到缓冲区中已有的文本之后——只有当该缓冲区中的光标点在末尾时才追加。不过，若 ‘append-to-buffer’ 是你用来改动某个缓冲区的唯一命令，那么光标点就总在末尾。

‘M-x prepend-to-buffer’ 与 ‘append-to-buffer’ 类似，只是另一个缓冲区中的光标点留在被复制文本之前，所以连续使用此命令会以相反的顺序添加文本。‘M-x copy-to-buffer’ 类似，只是另一个缓冲区中已有的文本会被删除，因此该缓冲区最终只含有新复制进去的文本。

命令 ‘C-x x i’（‘insert-buffer’）可用于从另一个缓冲区取出累积的文本。它会提示输入缓冲区名，并把该缓冲区中所有文本的一份副本插入当前缓冲区的光标点处，光标点留在插入文本的开头。它还会把插入文本末尾的位

置加入标记环，但不激活标记。关于缓冲区的背景信息，参见“多缓冲区”。

除把文本累积到缓冲区之外，你还可以用‘M-x append-to-file’直接把文本追加到文件中。它会提示输入文件名，并把区域的文本添加到指定文件的末尾。文件会立即在磁盘上被修改。

你只应对*没有*在 Emacs 中访问的文件使用‘append-to-file’。对你在 Emacs 中编辑的文件使用它，会在 Emacs 不知情的情况下改动文件，可能导致你的部分编辑丢失。

另一种移动文本的办法是把它存入寄存器。参见“寄存器”。

14.5 矩形

“矩形”命令作用于文本的矩形区域：某几行范围内、某对列之间的所有字符。Emacs 有命令可删除矩形、粘贴被删除的矩形、清空矩形、用空白或文本填充矩形，或删除矩形。矩形命令对多栏格式的文本很有用，也适合把文本改成这种格式或从这种格式改回来。

要为命令指定操作的矩形，把标记设在一个角、把光标点设在相对的角。这样指定的矩形称为“区域矩形”（region-rectangle）。若光标点和标记在同一列，区域矩形为空。若它们在同一行，区域矩形只有一行高。

区域矩形的控制方式与区域大体相同。但要记住，给定的一对光标点和标记位置，既可以被解释为区域，也可以被解释为矩形，取决于使用它们的命令。

还可以用鼠标标记矩形区域：从矩形的一个角向对角点击并拖动‘C-M-mouse-1’。

‘C-x r k’删除区域矩形的文本，并把其内容保存为最后删除的矩形（‘kill-rectangle’）。‘C-x r M-w’把区域矩形的文本保存为最后删除的矩形（‘copy-rectangle-as-kill’）。‘C-x r d’删除区域矩形的文本（‘delete-rectangle’）。‘C-x r y’粘贴最后删除的矩形，使其左上角位于光标点（‘yank-rectangle’）。‘C-x r o’插入空白以填满区域矩形所占的空间（‘open-rectangle’），这会把区域矩形原先的内容向右推。‘C-x r N’沿区域矩形的左边缘插入行号（‘rectangle-number-lines’），这会把区域矩形原先的内容向右推。‘C-x r c’用空格替换区域矩形的全部内容以清空它（‘clear-rectangle’）。‘M-x delete-whitespace-rectangle’删除指定矩形每一行中的空白，从矩形的左边缘列开始。‘C-x r t STRING <RET>’在每一行上用 STRING 替换矩形内容（‘string-rectangle’）。‘M-x string-insert-rectangle <RET> STRING <RET>’在矩形的每一行上插入 STRING。‘C-x <SPC>’切换矩形标记模式（‘rectangle-mark-mode’）。该模式激活时，区域矩形会被高亮，可以缩小或扩大，标准的删除和粘贴命令也作用于它。

矩形操作分为两类：擦除或插入矩形的命令，以及建立空白矩形的命令。

擦除矩形中的文本有两种方式：‘C-x r d’（‘delete-rectangle’）直接删除文本，或‘C-x r k’（‘kill-rectangle’）删除文本并把它保存为“最后删除的矩形”。两种情况下，擦除区域矩形都相当于擦除矩形每一行上指定的文本；若该行后面还有文本，它会向后移动以填补空缺。

删除矩形并非通常意义上的删除；矩形不存入删除环，而是存入一个只记录最近删除的那个矩形的特殊位置。这是因为粘贴矩形与粘贴线性文本差别太大，必须使用不同的粘贴命令。矩形不支持连续粘贴轮换（yank-popping）。

‘C-x r M-w’（‘copy-rectangle-as-kill’）对矩形而言相当于‘M-w’：它把矩形记录为最后删除的矩形，而不从缓冲区中删除文本。

要粘贴最后删除的矩形，输入‘C-x r y’（‘yank-rectangle’）。矩形的第一行插入到光标点处，第二行插入到垂直下一行的相同水平位置，依此类推。受影响的行数由所保存矩形的高度决定。

例如，你可以先把两个单栏列表中的一个作为矩形删除，再把它粘贴到另一个列表旁边，从而把两个单栏列表变成一个双栏列表。

你也可以用‘C-x r r R’和‘C-x r i R’在寄存器与矩形之间复制。参见“在寄存器中保存矩形”。

有两个命令可用来建立空白矩形：‘C-x r c’（‘clear-rectangle’）把区域矩形中已有的文本清成空白，‘C-x r o’（‘open-rectangle’）插入一个空白矩形。

‘M-x delete-whitespace-rectangle’从特定列开始删除水平空白。它作用于矩形中的每一行，列由矩形的左边缘指定。矩形的右边缘对此命令没有影响。

命令‘C-x r N’（‘rectangle-number-lines’）沿区域矩形的左边缘插入行号。通常，编号从 1 开始（对应矩形的第一行）。带前缀参数时，该命令会提示输入起始编号，以及打印编号所用的格式字符串（参见《Emacs Lisp 参考手册》的“Formatting Strings”）。

命令‘C-x r t’（‘string-rectangle’）在每一行上用字符串替换区域矩形的内容。字符串的宽度不必与矩形的宽度相同。若字符串较窄，矩形之后的文本向左移动；若字符串比矩形宽，矩形之后的文本向右移动。

命令‘M-x string-insert-rectangle’与‘string-rectangle’类似，但在每一行上插入字符串，把原有文本向右推。

命令‘C-x <SPC>’（‘rectangle-mark-mode’）切换是突出显示区域矩形还是标准区域（必要时先激活区域）。启用该模式后，调整区域的命令（‘C-f’、‘C-n’等）会按矩形方式调整，删除和粘贴也作用于矩形。参见“删除与移动文本”。该模式只在区域活动期间持续。

区域矩形只在标记活动时有效。特别是在临时标记模式关闭时（参见“禁用临时标记模式”），除输入‘C-x <SPC>’外，你还需要激活标记。

与标准区域不同，区域矩形的角可以延伸到缓冲区末尾之外，或伸入光标点通常无法进入的空白区域，比如 TAB 字符的中间。

当区域处于活动状态（参见“标记与区域”）且处于矩形标记模式时，‘C-x C-x’运行命令‘rectangle-exchange-point-and-mark’，它在区域矩形的四个角之间循环。若你想在操作标记文本之前修改区域矩形的尺寸，这很方便。

14.6 CUA 绑定

命令‘M-x cua-mode’建立与许多其他应用程序所用的 Common User Access（CUA）系统兼容的键绑定。

启用 CUA 模式后，键‘C-x’、‘C-c’、‘C-v’和‘C-z’分别调用剪切（删除）、复制、粘贴（yank）和撤销命令。‘C-x’和‘C-c’仅在区域活动时执行剪切和复制。否则它们仍作为前缀键，因此像‘C-x C-c’这样的标准 Emacs 命令仍然可用。注意这意味着变量‘mark-even-if-inactive’对‘C-x’和‘C-c’没有影响（参见“对区域进行操作”）。

要在标记活动时输入像‘C-x C-f’这样的 Emacs 命令，可使用下列方法之一：按住‘Shift’与前缀键一起按，例如‘S-C-x C-f’，或快速连按两次前缀键，例如‘C-x C-x C-f’。

要禁止 CUA 模式覆盖标准 Emacs 键绑定，同时保留下面介绍的 CUA 模式的其他特性，请把变量 ‘cua-enable-cua-keys’ 设为 ‘nil’。

CUA 模式默认会激活 Delete-Selection 模式（参见“鼠标编辑命令”），使键入的文本替换活动区域。若想使用不带此行为的 CUA，请把变量 ‘cua-delete-selection’ 设为 ‘nil’。

CUA 模式提供增强的矩形支持，矩形高亮可见。用 ‘C-<RET>’ 开始一个矩形，用移动命令扩展它，用 ‘C-x’ 或 ‘C-c’ 剪切或复制它。<RET> 把光标移到矩形的下一个（顺时针）角，因此你可以轻松地向任意方向扩展矩形。你键入的普通文本会插入到矩形每一行的左侧或右侧（与光标同侧）。

你可以在不激活 CUA 的情况下使用这一矩形支持，只需调用命令 ‘cua-rectangle-mark-mode’。标准命令 ‘rectangle-mark-mode’ 也存在，参见“矩形”。

使用 CUA 时，只须为删除、复制和粘贴命令提供一个一位数字前缀，就能轻松地把文本和矩形复制进寄存器或从寄存器取出，例如 ‘C-1 C-c’ 把区域复制到寄存器 ‘1’，‘C-2 C-v’ 粘贴寄存器 ‘2’ 的内容。

CUA 模式还有一个全局标记特性，可轻松地在缓冲区之间移动和复制文本。用 ‘C-S-<SPC>’ 打开或关闭全局标记。全局标记打开时，你删除或复制的所有文本都会自动插入到全局标记处，你键入的文本也会插入到全局标记处而不是当前位置。

例如，要把各个缓冲区中的单词复制到某个缓冲区的单词表中，先在目标缓冲区设置全局标记，然后依次定位到你想要加入列表的每个单词，标记它（例如用 ‘S-M-f’），用 ‘C-c’ 或 ‘M-w’ 把它复制到列表中，并在目标列表中的该单词后按 <RET> 插入一个换行。

15 寄存器

Emacs 的“寄存器”（register）是一些隔间，你可以在其中保存文本、矩形、位置及其他东西，以供日后使用。一旦把文本或矩形存入寄存器，你就可以把它复制进缓冲区一次或多次；一旦把位置存入寄存器，你就可以跳回该位置一次或多次。

每个寄存器都有一个由单个字符组成的名字，我们用 R 来表示；R 可以是字母（比如‘a’），也可以是数字（比如‘1’）；大小写是有区别的，所以寄存器‘a’与寄存器‘A’并不相同。你也可以用非字母数字字符来设置寄存器，例如借助 <C-q>‘C-d’来设置寄存器‘C-d’。

寄存器可以保存一个位置、一段文本、一个矩形、一个数字、一个窗口或框架配置、一个缓冲区名，或者一个文件名，但在任一给定时刻只能保存其中一样东西。你存入寄存器的内容会一直留在那里，直到你往该寄存器里存入别的东西。要查看寄存器 R 里装的是什么，用‘M-x view-register’：

- ‘M-x view-register <RET> R’显示寄存器 R 所含内容的描述。

所有提示输入寄存器的命令，默认都会在短暂延迟之后显示一个预览窗口，列出已有的寄存器（如果有的话）及其当前值。这一行为以及提示输入寄存器的其他方面，都可以通过设置‘register-use-preview’的值来定制，该变量可以取以下值：

‘traditional’取这个值（也是默认值）时，Emacs 的行为与 Emacs 29 之前的所有版本一样：它会在延迟之后显示已有寄存器的预览，并允许你通过输入单个字符（即寄存器的名字）来覆盖已有寄存器的值。预览出现的延迟由可定制变量‘register-preview-delay’决定，该变量以秒为单位指定延迟；把它设为‘nil’会禁用预览（不过在 Emacs 提示输入寄存器时，你仍然可以按‘C-h’或 <F1> 显式请求一个预览窗口）。

‘t’这个值要求一种更灵活的已有寄存器预览。Emacs 提示输入寄存器时预览立即出现（因此‘register-preview-delay’不起作用），并且预览窗口提供导航功能：用‘C-n’和‘C-p’（或者‘<UP>’和‘<DOWN>’方向键），你可以在预览窗口中的各个寄存器之间移动。在这种模式下要覆盖已有寄存器的值，你需要先用导航或输入名字选中该寄存器，然后按 <RET>。

此外，预览所显示的寄存器会按触发预览的命令进行过滤：例如，‘insert-register’显示的预览只列出其值可以插入缓冲区的寄存器，而省略保存窗口配置、位置以及其他不可插入值的寄存器。

‘insist’这个值与‘t’类似，但此外你还可以再次按下与寄存器名相同的那个键来退出迷你缓冲区，而不必按 <RET>。

‘nil’这个值要求的行为与‘traditional’类似，但预览不经延迟就显示，并且按命令进行过滤。

‘never’这个值与‘nil’类似，但它禁用预览。

“书签”（bookmark）记录文件和文件中的位置，这样当你再次查看该文件时就能回到那些位置。书签在精神上类似寄存器，因此也放在本章讲述。

15.1 在寄存器中保存位置

- ‘C-x r <SPC> R’把光标点的位置和当前缓冲区记录到寄存器 R 中（‘point-to-register’）。
- ‘C-x r j R’跳转到寄存器 R 中保存的位置和缓冲区（‘jump-to-register’）。

输入‘C-x r <SPC>’（‘point-to-register’），再输入一个字符‘R’，就会把光标点的位置和当前缓冲区都保存到寄存器 R 中。寄存器会一直保留这一信息，直到你往它里面存入别的东西。

命令‘C-x r j R’会切换到寄存器 R 中记录的缓冲区，压入一个标记，并把光标点移到记录的位置。（如果光标点本来就在记录的位置上，或者是在连续多次调用该命令时，则不会压入标记。）寄存器的内容不会被改变，因此你可以任意多次跳转到保存的位置。

如果你用‘C-x r j’前往某个保存的位置，而保存该位置时所在的缓冲区已经被删除，那么‘C-x r j’会尝试通过访问同一个文件来重新创建该缓冲区。当然，这只对曾访问过文件的缓冲区有效。

15.2 在寄存器中保存文本

当你想把同一段文本插入多份时，从删除环中粘贴可能会不方便，因为此后每次删除都会把那个条目在环中推得更靠下。另一种办法是把文本存入寄存器，以后再取出来。

- ‘C-x r s R’把区域复制到寄存器 R 中（‘copy-to-register’）。
- ‘C-x r i R’插入寄存器 R 中的文本（‘insert-register’）。
- ‘M-x append-to-register <RET> R’把区域追加到寄存器 R 中的文本之后。

当寄存器 R 中含有文本时，你可以用‘C-x r +’（‘increment-register’）向该寄存器追加内容。注意，如果 R 中含有数字，命令‘C-x r +’的行为会有所不同。参见“在寄存器中保存数字”。

- ‘M-x prepend-to-register <RET> R’把区域前置到寄存器 R 中的文本之前。

‘C-x r s R’把区域文本的一份副本存入名为 R 的寄存器中。如果标记处于未激活状态，Emacs 会先在其上次设置的位置重新激活标记。该命令结束时标记会被取消激活。参见“标记与区域”。带前缀参数的‘C-u C-x r s R’是同一个命令，但它会把文本复制到寄存器 R 中，同时还把该文本从缓冲区中删除；你可以把这理解为把区域文本移入寄存器。

‘M-x append-to-register <RET> R’把区域中文本的副本追加到名为 R 的寄存器中已经保存的文本之后。若带前缀参数调用，它会在把区域追加到寄存器之后删除该区域。命令‘prepend-to-register’与此类似，只不过它把区域文本*前置*到寄存器中的文本之前，而不是*追加*到其后。

当你用‘append-to-register’和‘prepend-to-register’收集文本时，可能想用分隔符把各个收集到的片段隔开。这种情况下，可以配置一个‘register-separator’，并把分隔符文本存入那个寄存器。例如，要在收集过程中用两个换行符作为文本分隔符，可以用下面这个设置。

```
(setq register-separator ?+)
(set-register register-separator "\n\n")
```

‘C-x r i R’把寄存器 R 中的文本插入缓冲区。通常它会把光标点留在文本之后，并把标记设在文本之前，但不激活标记。带前缀参数时，它反而把光标点放在文本之前，把标记放在文本之后。

15.3 在寄存器中保存矩形

寄存器中可以存放矩形，而不是线性文本。如何在缓冲区中指定矩形，参见“矩形”。

- ‘C-x r r R’ 把区域矩形复制到寄存器 R 中（‘copy-rectangle-to-register’）。带前缀参数时还会把它删除。
- ‘C-x r i R’ 插入寄存器 R 中保存的矩形（如果其中保存的是矩形）（‘insert-register’）。

先前在参见“在寄存器中保存文本”中介绍过的 ‘C-x r i R’（‘insert-register’）命令，在寄存器中保存的是矩形时，会插入矩形而不是文本字符串。

15.4 在寄存器中保存窗口和框架配置

你可以把所选框架的窗口配置存入寄存器，甚至可以保存所有框架中所有窗口的配置，稍后再恢复该配置。有关窗口配置的信息，参见“窗口处理的便利功能”。

- ‘C-x r w R’ 把所选框架的窗口状态保存到寄存器 R 中（‘window-configuration-to-register’）。
- ‘C-x r f R’ 把所有框架的状态（包括它们的全部窗口，又称“框架组”）保存到寄存器 R 中（‘frameset-to-register’）。

用 ‘C-x r j R’ 来恢复窗口配置或框架配置。这与恢复光标位置用的是同一个命令。恢复框架配置时，配置中未包含的现存的框架会变为不可见。如果你希望改为删除这些框架，请用 ‘C-u C-x r j R’。

15.5 在寄存器中保存数字

有些命令可以把数字存入寄存器、以十进制把该数字插入缓冲区，以及递增该数字。这些命令在键盘宏中可能很有用（参见“键盘宏”）。

- ‘C-u NUMBER C-x r n R’ 把 NUMBER 存入寄存器 R（‘number-to-register’）。
- ‘C-u NUMBER C-x r + R’ 如果 R 中含有数字，就把该寄存器中的数字增加 NUMBER。注意，如果 R 中含有文本，命令 ‘C-x r +’（‘increment-register’）的行为会有所不同。参见“在寄存器中保存文本”。
- ‘C-x r i R’ 把寄存器 R 中的数字插入缓冲区。

‘C-x r i’ 也就是把其他任何种类的寄存器内容插入缓冲区时所用的那个命令。不带数字参数的 ‘C-x r +’ 会把寄存器中的值增加 1；不带数字参数的 ‘C-x r n’ 会把零存入寄存器。

15.6 在寄存器中保存文件名和缓冲区名

- ‘C-x r F R’ 把当前访问的文件或目录存入寄存器 R（‘file-to-register’）。
- ‘C-x r B R’ 把当前访问的缓冲区存入寄存器 R（‘buffer-to-register’）。

如果你频繁访问某些文件或缓冲区，把它们的名字放进寄存器可以让访问更方便。

输入 ‘C-x r F’（‘file-to-register’），再输入一个字符 ‘R’，就会把对当前访问文件的引用保存到寄存器 R 中。此后你可以用 ‘jump-to-register’（即输入 ‘C-x r j’ R）来访问该文件。带前缀参数时，‘file-to-register’ 会提示输入要存入寄存器的文件名。

输入 ‘C-x r B’（‘buffer-to-register’），再输入一个字符 ‘R’，就会把对当前访问缓冲区的引用保存到寄存器 R 中。此后你可以用 ‘jump-to-register’ 重新访问该缓冲区。带前缀参数时，‘buffer-to-register’ 会提示输入要存入寄存器的缓冲区名。

如果你把某个正在访问文件的缓冲区名存入寄存器，而该缓冲区随后被关闭，那么这个寄存器会自动转换为对文件的引用，让你能迅速重新打开已关闭的文件。

15.7 键盘宏寄存器

如果你需要频繁执行某个键盘宏（参见“键盘宏”），把它放进寄存器或保存起来会更方便（参见“命名并保存键盘宏”）。‘C-x C-k x R’（‘kmacro-to-register’）把最后一个键盘宏存入寄存器 R。

要执行寄存器 R 中的键盘宏，输入 ‘C-x r j R’。（这与跳转到某个位置或恢复框架组用的是同一个命令。）

15.8 书签

“书签”（bookmark）在某种程度上类似寄存器，因为它们都记录你可以跳转到的位置。与寄存器不同的是，书签有较长的名字，而且会自动从一个 Emacs 会话保留到下一个会话。书签最典型的用途是记录你在各个文件中读到了哪里。

- ‘C-x r m <RET>’ 在当前光标点处为所访问的文件设置书签。
- ‘C-x r m BOOKMARK <RET>’ 在当前光标点处设置名为 BOOKMARK 的书签（‘bookmark-set’）。
- ‘C-x r M BOOKMARK <RET>’ 类似 ‘C-x r m’，但不覆盖已有书签。
- ‘C-x r b BOOKMARK <RET>’ 跳转到名为 BOOKMARK 的书签（‘bookmark-jump’）。
- ‘C-x r l’ 列出所有书签（‘list-bookmarks’）。
- ‘M-x bookmark-save’ 把所有当前书签值保存到默认书签文件中。

要记录所访问文件中的当前位置，用命令 ‘C-x r m’，它以所访问文件的名字作为书签名字的默认值来设置书签。如果你按书签所指向的文件来给每个书签命名，就可以方便地用 ‘C-x r b’（‘bookmark-jump’）重新访问其中任一文件，同时移动到该书签所在的位置。

除了记录当前位置之外，在图形显示上，‘C-x r m’ 还会在与所记录位置对应的屏幕行的左侧边缘（fringe）放置一个特殊图像（参见“窗口边缘”），以表明那里有一个书签。这可以通过用户选项 ‘bookmark-fringe-mark’ 控制：把它定制为 ‘nil’ 即可禁用边缘标记。默认值是 ‘bookmark-mark’，也就是为此目的使用的位图。当你之后用 ‘C-x r b’ 跳回该书签时，边缘标记会再次显示在边缘上。

命令 ‘C-x r M’（‘bookmark-set-no-overwrite’）与 ‘C-x r m’ 类似，但如果指定的书签已经存在，它会报错，而不是覆盖它。

要在单独的缓冲区中显示你所有书签的列表，输入 ‘C-x r l’（‘list-bookmarks’）。如果你切换到那个缓冲区，就可以用它来编辑书签定义或为书签添加注释。在该书签缓冲区中输入 ‘C-h m’ 可了解有关其特殊编辑命令的更多信息。

当你退出 Emacs 时，如果你改动过任何书签值，Emacs 会保存你的书签。你也可以随时用 ‘M-x bookmark-save’ 命令保存书签。书签会保存到文件 ‘~/.emacs.d/bookmarks’ 中（为了与旧版本的 Emacs 兼容，如果你有一个名为 ‘~/.emacs.bmk’ 的文件，就会改用那个文件）。书签相关命令会自动加载你的默认书签文件。这种保存与加载正是书签能从一个 Emacs 会话保留到下一个会话的原因。

如果你把变量 'bookmark-save-flag' 设为 1，那么每条设置书签的命令也会保存你的书签；这样即使 Emacs 崩溃，你也不会丢失任何书签值。该值若为数字，就表示应在两次保存之间经历多少次书签修改。如果你把该变量设为 'nil'，Emacs 只在你显式使用 'M-x bookmark-save' 时才保存书签。

变量 'bookmark-default-file' 指定默认保存书签的文件。

如果你把变量 'bookmark-use-annotations' 设为 't'，设置书签时会询问注释。如果某个书签带有注释，那么当你跳转到该书签时，注释会自动显示在一个单独的窗口中。

书签的位置值与其周围的上下文一起保存，这样即使文件被稍作改动，'bookmark-jump' 也能找到正确的位置。变量 'bookmark-search-size' 表示在书签位置的每一侧记录多少个字符的上下文。（在访问加密文件的缓冲区中，无论该变量的值是什么，都不会在书签文件中保存上下文。）

下面是另外一些用于操作书签的命令：

- 'M-x bookmark-load <RET> FILENAME <RET>' 加载名为 FILENAME 的文件，其中包含一个书签值列表。你可以用这个命令以及 'bookmark-write'，在默认书签文件之外处理其他书签值文件。
- 'M-x bookmark-write <RET> FILENAME <RET>' 把所有当前书签值保存到文件 FILENAME 中。
- 'M-x bookmark-delete <RET> BOOKMARK <RET>' 删除名为 BOOKMARK 的书签。
- 'M-x bookmark-insert-location <RET> BOOKMARK <RET>' 把书签 BOOKMARK 所指向的文件名插入缓冲区。
- 'M-x bookmark-insert <RET> BOOKMARK <RET>' 把书签 BOOKMARK 所指向文件的内容插入缓冲区。

16 控制显示

由于大缓冲区只有一部分能放进窗口，Emacs 只能显示其中一部分。本章介绍用于指定你想查看文本的哪一部分、以及文本如何显示的命令和变量。

- 窄化：把显示与编辑限制在缓冲区的一部分。
- 视图模式：查看只读缓冲区。
- 跟随模式：跟随模式让两个窗口作为一个整体滚动。
- 字体：如何用字体改变显示样式。
- 颜色：为字体指定颜色。
- 标准字体：主要的预定义字体。
- 图标：如何改变图标的外观。
- 文本缩放：在缓冲区中增大或减小文本大小。
- 字体锁定：用字体进行语法高亮的次模式。
- 交互式高亮：告诉 Emacs 要高亮哪些文本。
- 边缘：启用或禁用窗口边缘。
- 显示边界：显示缓冲区的顶部与底部。
- 无用空白：显示可能是多余的尾随空白。
- 选择性显示：隐藏缩进很深的行。
- 可选模式行：可选的模式行显示特性。
- 文本显示：文本字符通常如何显示。
- 光标显示：显示光标的特性。
- 行截断：把行截断以适合屏幕宽度，而不是让它们继续到多个屏幕行。
- 视觉行模式：自动换行以及基于屏幕行的编辑。
- 显示定制：关于定制显示所用变量的信息。

16.1 滚动

如果窗口太小，无法显示其缓冲区中的全部文本，它就只显示其中一部分。“滚动”（scrolling）命令改变所显示的是缓冲区的哪一部分。

向前或向上滚动会把窗口中显示的缓冲区部分向后推进；等价地说，它使缓冲区文本相对于窗口向上移动。向后或向下滚动则显示缓冲区中更靠前的部分，并使文本相对于窗口向下移动。

在 Emacs 中，向上滚动或向下滚动指的是文本在窗口中移动的方向，*而不是*窗口相对于文本移动的方向。Emacs 采用这一术语时，现代意义上的“向上滚动”和“向下滚动”尚未普及。因此会出现奇怪的结果：<PageDown> 按 Emacs 的意义是向上滚动。

窗口中显示的缓冲区部分始终包含光标点。如果把光标点移出窗口底部或顶部，就会自动滚动，使它回到屏幕上（参见“自动滚动”）。也可以用下列命令显式滚动：

- ‘C-v’、‘<PageDown>’、‘<next>’：向前滚动几乎整整一屏（‘scroll-up-command’）。
- ‘M-v’、‘<PageUp>’、‘<prior>’：向后滚动（‘scroll-down-command’）。

‘C-v’（‘scroll-up-command’）向前滚动几乎整个窗口高度。其效果是把窗口底部的两行移到顶部，后面接着原先不可见的行。如果光标点原先位于滚出顶部的文本中，它最终落在窗口新的最顶行。<PageDown>（或<next>）键等价于‘C-v’。

‘M-v’（‘scroll-down-command’）以类似方式向后滚动。<PageUp>（或 <prior>）键等价于‘M-v’。

这些滚动命令留下的重叠行数由变量‘next-screen-context-lines’控制，默认值为 2。可以给命令提供数字前缀参数 N，使其滚动 N 行；Emacs 会尽量保持光标点不变，使文本和光标点一起向上或向下移动。带负参数的‘C-v’类似于‘M-v’，反之亦然。

默认情况下，如果窗口已到达缓冲区开头或末尾、无法再滚动，这些命令会发出错误信号（通过响铃或闪烁屏幕）。如果把变量‘scroll-error-top-bottom’改为‘t’，这些命令会把光标点移到尽可能远的位置。如果光标点已经在那里，命令就发出错误信号。

有些用户希望滚动命令把光标点保持在相同的屏幕位置，这样滚回同一屏幕时就能方便地让光标点回到原来的位置。可以通过变量‘scroll-preserve-screen-position’启用此行为。如果值为‘t’，则每当滚动命令把光标点移出窗口时，Emacs 都会调整光标点以把光标保持在相同的屏幕位置，而不是把它移到最顶行或最底行。若值为任何其他非‘nil’值，即使滚动命令把光标点留在窗口内，Emacs 也会这样调整光标点。该变量影响本节记录的所有滚动命令，也影响用鼠标滚轮滚动（参见“鼠标编辑命令”）；一般而言，它影响任何具有非‘nil’的‘scroll-command’属性的命令。参见《Emacs Lisp 参考手册》的“Property Lists”。同一属性还使 Emacs 在调用这些命令之一且‘isearch-allow-scroll’非‘nil’时不退出增量搜索（参见“不退出增量搜索”）。

有时，特别是在按住‘C-v’、‘M-v’这类键、触发键盘自动重复时，Emacs 跟不上请求的滚动速度；显示不再更新，Emacs 可能长时间对输入无响应。可以把变量‘fast-but-imprecise-scrolling’设为非‘nil’值来缓解这种迟钝。这会指示滚动命令不要对其滚过的未字体化（fontify）文本做字体化（参见“Font Lock 模式”），而是假定它使用默认字体。当所用字体大小不完全一致时，即使只做一次滚动操作（即不自动重复），这也可能导致 Emacs 滚动到略有偏差的缓冲区位置。

除了设置‘fast-but-imprecise-scrolling’，你可能更愿意启用 jit-lock 延迟字体化（参见“Font Lock 模式”）。为此，把‘jit-lock-defer-time’定制为小的正数，如 0.25，若打字很快甚至可为 0.1。这样按住‘C-v’时滚动更平滑，但任何滚动进入缓冲区新部分的操作之后，窗口内容会短暂地未字体化。

最后，这些变量的第三种替代方案是‘redisplay-skip-fontification-on-input’。如果该变量非‘nil’，则在有输入挂起时跳过一些字体化。这通常不影响显示，因为只要有输入挂起，重显示本来就会被完全跳过；但它通过避免不必要的字体化，可使滚动更平滑。

‘M-x scroll-up’和‘M-x scroll-down’命令的行为与‘scroll-up-command’和‘scroll-down-command’类似，只是不遵循‘scroll-error-top-bottom’。在 Emacs 24 之前，它们是向上和向下滚动的默认命令。‘M-x scroll-up-line’和‘M-x scroll-down-line’命令每次把当前窗口滚动一行。如果你打算使用其中任何命令，可能需要为它们绑定按键（参见“在初始化文件中重绑定按键”）。

在图形显示上，还可以用滚动条滚动窗口；参见“滚动条”。

16.2 重新居中

- ‘C-l’：滚动所选窗口，使当前行成为最接近中心的文本行；在后续连续调用中，依次使当前行成为顶行、底行，并按循环顺序继续。可能还会重显示屏幕（‘recenter-top-bottom’）。
- ‘C-M-S-l’：滚动另一个窗口；这等价于对另一个窗口执行 ‘C-l’。
- ‘M-x recenter’：滚动所选窗口，使当前行成为最接近中心的文本行。可能还会重显示屏幕。
- ‘C-M-l’：启发式地滚动，把有用的信息带到屏幕上（‘reposition-window’）。

‘C-l’（‘recenter-top-bottom’）命令把所选窗口“重新居中”，即滚动它，使当前屏幕行正好位于窗口中央，或尽可能接近中央。

连续输入两次 ‘C-l’（‘C-l C-l’）会把窗口滚动到光标点位于最顶屏幕行。第三次输入 ‘C-l’ 则把窗口滚动到光标点位于最底屏幕行。之后每次 ‘C-l’ 都在这三个位置间循环。

可以定制列表变量 ‘recenter-positions’ 来改变循环顺序。每个列表元素应是符号 ‘top’、‘middle’ 或 ‘bottom’，或一个数字；整数表示把该行移到指定的屏幕行，而 0.0 到 1.0 之间的浮点数指定从窗口顶部算起的屏幕空间百分比。默认值 ‘(middle top bottom)’ 就是上述循环顺序。此外，如果把变量 ‘scroll-margin’ 改为非零值 N，‘C-l’ 始终在光标点与窗口顶部或底部之间至少保留 N 个屏幕行（参见“自动滚动”）。

还可以给 ‘C-l’ 加前缀参数。单纯的前缀参数 ‘C-u C-l’ 只是把显示光标点的那一行重新居中。正参数 N 把显示光标点的行移到距窗口顶部 N 行的位置。参数为零则把光标点所在行移到窗口顶部。负参数 −N 把光标点所在行移到距窗口底部 N 行的位置。带参数时，‘C-l’ 不会清屏，也不会在不同屏幕位置间循环。

如果变量 ‘recenter-redisplay’ 的值为非 ‘nil’，则每次调用 ‘C-l’ 还会清屏并重显示屏幕；特殊值 ‘tty’（默认值）表示只在文本终端框架上这样做。若屏幕因某种原因变得混乱，重显示很有用（参见“屏幕乱码”）。

更原始的命令 ‘M-x recenter’ 行为类似 ‘recenter-top-bottom’，但不在不同屏幕位置间循环。

‘C-M-l’（‘reposition-window’）以启发式方式滚动当前窗口，目的是把有用信息带到屏幕上。例如在 Lisp 文件中，该命令会尽可能把整个当前 defun 带到屏幕上。

16.3 自动滚动

当光标点移出文本的可见部分时，Emacs 会执行“自动滚动”。通常自动滚动会把光标点在窗口中垂直居中，但有好几种方法可以改变这一行为。

如果把 ‘scroll-conservatively’ 设为较小的数 N，那么光标点稍微移出屏幕（不超过 N 行）时，Emacs 只滚动到刚好让光标点回到屏幕上；如果这样做仍不能让光标点可见，Emacs 就滚动到刚好使光标点在窗口中居中。如果把 ‘scroll-conservatively’ 设为大数（大于 100），则自动滚动从不使光标点居中，无论光标点移动多远；Emacs 总是只把文本滚动到刚好让光标点进入视野，根据滚动方向使其位于窗口顶部或底部。默认情况下 ‘scroll-conservatively’ 为 0，表示总是让光标点在窗口中居中。话虽如此，在迷你缓冲区窗口中，默认情况下滚动总是保守的，因为 ‘scroll-minibuffer-conservatively’ 非 ‘nil’，其优先级高于 ‘scroll-conservatively’。

控制自动滚动的另一种方法是定制变量 ‘scroll-step’。它的值决定当光标点移出屏幕时自动滚动的行数。如果滚动这么多行仍不能让光标点回到视野，就改为使光标点居中。默认值为零，默认情况下会使光标点在滚动后总是居中。

控制自动滚动的第三种方法是定制变量‘scroll-up-aggressively’和‘scroll-down-aggressively’，它们直接指定滚动后光标点的垂直位置。‘scroll-up-aggressively’的值应为‘nil’（默认值），或 0 到 1 之间的浮点数 F。后者表示当光标点落到窗口下边缘以下（即向前滚动）时，Emacs 滚动窗口，使光标点距离窗口下边缘为窗口高度的 F 部分。因此 F 越大，滚动越激进：更多新文本进入视野。默认值‘nil’等价于 0.5。

同样，当光标点移到窗口上边缘以上（即向后滚动）时使用‘scroll-down-aggressively’。其值指定滚动后光标点应距窗口上边缘多远。因此与‘scroll-up-aggressively’一样，值越大越激进。

注意，变量‘scroll-conservatively’、‘scroll-step’以及‘scroll-up-aggressively’/‘scroll-down-aggressively’以相互矛盾的方式控制自动滚动。因此，你最多只应选择其中一种方法来定制自动滚动。如果定制了多个变量，优先级顺序为：‘scroll-conservatively’，然后‘scroll-step’，最后‘scroll-up-aggressively’/‘scroll-down-aggressively’。

变量‘scroll-margin’限制光标点能多接近窗口顶部或底部（即使激进滚动指定的比例 F 大于上下边距之间的窗口部分）。其值是屏幕行数；如果光标点进入距窗口顶部或底部该行数以内，Emacs 就执行自动滚动。默认情况下‘scroll-margin’为 0。默认情况下有效边距大小限制为窗口高度的四分之一，但可以通过定制‘maximum-scroll-margin’把该限制增大到一半（或减小到零）。

16.4 水平滚动

“水平滚动”指在窗口中把所有的行向侧面平移，使靠近左边距的部分文本不再显示。窗口中的文本被水平滚动时，文本行会被截断而不是续行（参见“行截断”）。如果窗口显示被截断的行，那么每当光标点移出屏幕左边缘或右边缘，Emacs 就执行自动水平滚动。默认情况下窗口中所有行一起水平滚动，但如果把变量‘auto-hscroll-mode’设为特殊值‘current-line’，则只有显示光标的那一行会滚动。要完全禁用自动水平滚动，把变量‘auto-hscroll-mode’设为‘nil’。注意，关闭自动水平滚动后，如果光标点移出屏幕边缘，光标会消失以示这一点。（在文本终端上，光标则留在边缘处。）

变量‘hscroll-margin’控制自动滚动发生前光标点能多接近窗口左右边缘。它以列为单位。例如值为 5 时，光标点移到距边缘 5 列以内就会引发远离该边缘的水平滚动。

变量‘hscroll-step’决定当光标点太接近边缘时窗口滚动多少列。默认值零表示使光标点在窗口中水平居中。正整数指定滚动的列数。浮点数（值应在 0 到 1 之间）指定滚动的窗口宽度比例。

也可以用下列命令显式执行水平滚动：

- ‘C-x <’：把当前窗口中的文本向左滚动（‘scroll-left’）。
- ‘C-x >’：向右滚动（‘scroll-right’）。

‘C-x <’（‘scroll-left’）把所选窗口中的文本向左滚动整个窗口宽度少两列的距离。（换句话说，窗口中的文本相对于窗口向左移动。）带数字参数 N 时滚动 N 列。

如果文本已向左滚动，而光标点移出窗口左边缘，光标会冻结在窗口左边缘，直到光标点移回文本的显示部分。这与‘auto-hscroll-mode’的当前设置无关；对于向左滚动的文本，该设置只影响窗口右边缘的行为。

‘C-x >’（‘scroll-right’）类似地向右滚动。一旦窗口正常显示、每行都从窗口左边距开始，就不能再向右滚动；尝试这样做没有效果。这意味着你不必为‘C-x >’精确计算参数；任何足够大的参数都会恢复正常显示。

如果用这些命令水平滚动窗口，就为自动水平滚动设定了下界。自动滚动会继续滚动窗口，但绝不会比先前用‘scroll-left’设定的量更向右。当‘auto-hscroll-mode’设为‘current-line’时，除显示光标的那一行外，所有行都会按该最小量滚动。

在图形显示上，如果启用可选的‘horizontal-scroll-bar-mode’，可以用水平滚动条滚动窗口；参见“滚动条”。

16.5 窄化

“窄化”（narrowing）指聚焦于缓冲区的某一部分，使其余部分暂时无法访问。仍然可以访问的部分称为“可访问部分”。取消窄化、使整个缓冲区重新可访问称为“解除窄化”（widening）。缓冲区中生效的窄化边界称为该缓冲区的“限制”（restriction）。

窄化可以消除杂乱，便于集中处理单个子程序或段落。它也可用于限制替换命令或重复键盘宏的操作范围。

- ‘C-x n n’：窄化到光标点与标记之间（‘narrow-to-region’）。
- ‘C-x n w’：解除窄化，使整个缓冲区重新可访问（‘widen’）。
- ‘C-x n p’：窄化到当前页（‘narrow-to-page’）。
- ‘C-x n d’：窄化到当前 defun（‘narrow-to-defun’）。

窄化到缓冲区某部分后，该部分看起来就是全部内容。你看不到其余部分，无法移入其中（移动命令不会跑到可访问部分之外），也无法以任何方式修改它。不过它并没有消失，如果保存文件，所有不可访问的文本都会被保存。只要窄化生效，模式行上就会出现单词‘Narrow’。

主要的窄化命令是‘C-x n n’（‘narrow-to-region’）。它设置当前缓冲区的限制，使当前区域中的文本保持可访问，而区域之前或之后的文本不可访问。光标点和标记不变。

另一种方法是使用‘C-x n p’（‘narrow-to-page’）窄化到当前页。页的定义参见“页”。‘C-x n d’（‘narrow-to-defun’）窄化到包含光标点的 defun（参见“顶层定义（defun）”）。

取消窄化的方法是使用‘C-x n w’（‘widen’）解除窄化。这使缓冲区中所有文本重新可访问。

可以用‘C-x =’命令获取当前窄化到缓冲区哪一部分的信息。参见“光标位置信息”。

由于窄化很容易让不了解它的用户困惑，‘narrow-to-region’通常是禁用命令。尝试使用该命令时会请求确认，并提供启用它的选项；如果启用该命令，就不再需要确认。参见“禁用命令”。

16.6 视图模式

视图模式是一种次模式，让你逐屏浏览缓冲区。它提供方便地滚动浏览缓冲区的命令，但不提供修改缓冲区的命令。除了常用的 Emacs 光标移动命令外，可以输入 <SPC> 向前滚动一窗，输入‘S-<SPC>’或 <DEL> 向后滚动，输入‘s’开始增量搜索。

输入‘q’（‘View-quit’）会禁用视图模式，并切回启用视图模式之前的缓冲区和位置。输入‘e’（‘View-exit’）禁用视图模式，但保留当前缓冲区和位置。

‘M-x view-buffer’提示输入一个已存在的 Emacs 缓冲区，切换到它并启用视图模式。‘M-x view-file’提示输入文件，以启用视图模式的方式访问它。

16.7 跟随模式

“跟随模式”（Follow mode）是一种次模式，使两个显示同一缓冲区的窗口作为一个高的虚拟窗口一起滚动。要使用跟随模式，先到一个只有一个窗口的框架，用 ‘C-x 3’ 把它拆成两个并排的窗口，然后输入 ‘M-x follow-mode’。此后可以在任一窗口中编辑缓冲区，或滚动任一窗口；另一个窗口会跟随。

在跟随模式中，如果把光标点移出一个窗口的可见部分而进入另一个窗口的可见部分，就会选中另一个窗口——同样是把两个窗口当作一个大窗口的两部分。

要关闭跟随模式，再次输入 ‘M-x follow-mode’。

16.8 文本字体

Emacs 可以用若干不同的样式显示文本，这些样式称为“字体”（face）。每种字体可以指定各种“字体属性”（face attribute），如字体、高度、粗细、倾斜、前景色和背景色，以及下划线或上划线。大多数主模式通过字体锁定模式自动为文本分配字体。这些字体是如何分配的，参见“Font Lock 模式”。

要查看当前定义了哪些字体以及它们的外观，输入 ‘M-x list-faces-display’。带前缀参数时，它会提示输入正则表达式，只显示名称匹配该正则表达式的字体（参见“正则表达式的语法”）。

同一种字体在不同框架上看起来可能不同。例如，有些文本终端不支持所有字体属性，特别是字体、高度和宽度，有些只支持有限范围的色彩。此外，出于可读性考虑，大多数 Emacs 字体的定义使其属性在浅色和深色框架背景上有所不同。默认情况下，Emacs 根据框架当前的背景色自动选择在每个框架上显示哪一组字体属性。不过，可以把变量 ‘frame-background-mode’ 设为非 ‘nil’ 值来覆盖这一点。值 ‘dark’ 使 Emacs 把所有框架都当作深色背景处理，值 ‘light’ 则把所有框架都当作浅色背景处理。

可以定制字体以更改其属性，并把这些定制保存到以后的 Emacs 会话中。详见参见“自定义字体”。

‘default’ 字体是显示文本的默认字体，其所有属性都已指定。它的背景色也用作框架的背景色。参见“字体的颜色”。

另一种特殊字体是 ‘cursor’ 字体。在图形显示上，该字体的背景色用于绘制文本光标。该字体的其他属性都不起作用；光标下文本的前景色取自底层文本的背景色。在文本终端上，文本光标的外观由终端决定，而不是由 ‘cursor’ 字体决定。

也可以用 X 资源指定任何特定字体的属性。参见“X 资源”。

Emacs 可以显示变宽字体，但有些 Emacs 命令，特别是缩进命令，没有考虑字符显示宽度的变化。因此，我们建议大多数字体不要使用变宽字体，尤其是字体锁定模式所分配的字体。

16.9 字体的颜色

字体（face）可以有各种前景色和背景色。当你为某个字体指定颜色时——例如在定制该字体时（参见“自定义字体”）——可以使用“颜色名”（color name），也可以使用“RGB 三元组”（RGB triplet）。

16.9.1 颜色名称

颜色名是预先定义好的名称，例如 ‘dark orange’ 或 ‘medium sea green’。要查看颜色名列表，输入 ‘M-x list-colors-display’。要控制颜色的显示顺序，请定制 ‘list-colors-sort’。若在图形显示上运行该命令，它会显示 Emacs 所知的全部颜色名（这些是标准的 X11 颜色名，定义在 X 的 ‘rgb.txt’ 文件中）。若在文本终端上运行该命令，它只显示能在这种终端上安全显示的一小部分颜色。不过，即使在文本终端上，Emacs 也能理解 X11 颜色名；若给某个字体指定了由 X11 颜色名表示的颜色，Emacs 会用最接近的终端颜色来显示它。

16.9.2 RGB 三元组

RGB 三元组是形如 ‘#RRGGBB’ 的字符串。每个原色分量用 ‘00’（强度 0）到 ‘FF’（最大强度）之间的十六进制数表示。每个分量也可以使用一位、三位或四位十六进制数字，因此 ‘red’ 可以表示为 ‘#F00’、‘#fff000000’ 或 ‘#ffff00000000’。各分量必须使用相同的位数。十六进制数值 A 到 F 用大写或小写均可。

‘M-x list-colors-display’ 命令还会显示每个有名字的颜色的等效 RGB 三元组。例如，‘medium sea green’ 等效于 ‘#3CB371’。

你可以用 ‘M-x set-face-foreground’ 和 ‘M-x set-face-background’ 更改某个字体的前景色和背景色。这些命令在迷你缓冲区中提示输入字体名和颜色（带补全），然后把该字体设为使用指定的颜色。它们会影响所有框架上该字体的颜色，但与使用定制缓冲区或 X 资源不同，其效果不会在以后的 Emacs 会话中保留。你也可以用框架参数为特定框架设置前景色和背景色；参见“框架参数”。

16.10 标准字体

下面是用于指定文本外观的标准字体。当你想要它们产生的效果时，可以把它们应用到特定文本上。

‘default’ 此字体用于未指定任何字体的普通文本。它的背景色被用作框架的背景色。

‘bold’ 此字体使用默认字体的粗体变体。

‘italic’ 此字体使用默认字体的斜体变体。

‘bold-italic’ 此字体使用默认字体的粗斜体变体。

‘underline’ 此字体给文本加下划线。

‘fixed-pitch’ 此字体强制使用定宽字体。如果你愿意，把这个字体定制成使用另一种定宽字体是合理的，但不应该把它变成变宽字体。

‘fixed-pitch-serif’ 此字体与 ‘fixed-pitch’ 类似，只是所用字体有衬线，看起来更像传统的打字机字体。

‘variable-pitch’ 此字体强制使用变宽（即比例）字体。为这个字体选定的字号与为默认（通常是定宽）字体选定的字号相匹配。

‘variable-pitch-text’ 这与 ‘variable-pitch’ 字体（它继承自该字体）类似，但稍大一些。与等宽字体高度相同的比例字体在视觉上通常显得更小，因而可能更难读。显示较长的文本时，这个字体比（稍小一些的）‘variable-pitch’ 字体更合适。

‘shadow’ 此字体用于让文本比周围的普通文本更不显眼。通常可以通过使用灰色调、与黑色或白色的默认前景色形成对比来做到这一点。

下面是一个不完整的列表，列出用于为特定目的临时高亮部分文本的字体。（许多其他模式为此定义了自己的字体。）

‘highlight’ 此字体用于在各种场合下高亮文本，例如鼠标光标移到超链接上时。

‘isearch’ 此字体用于高亮当前的 Isearch 匹配（参见“增量搜索”）。

‘query-replace’ 此字体用于高亮当前的 Query Replace 匹配（参见“替换命令”）。

‘lazy-highlight’ 此字体用于高亮 Isearch 和 Query Replace 的惰性匹配（当前匹配之外的匹配）。

‘region’ 此字体用于显示活动区域（参见“标记与区域”）。若 Emacs 构建时带有 GTK+ 支持，其颜色取自当前的 GTK+ 主题。

‘secondary-selection’ 此字体用于显示辅助 X 选区（参见“辅助选区”）。

‘trailing-whitespace’ 这是当 ‘show-trailing-whitespace’ 为非 ‘nil’ 时用于高亮行尾多余空格和制表符的字体（参见“无用的空白”）。

‘escape-glyph’ 这是用于显示控制字符和转义序列的字体（参见“文本的显示”）。

‘homoglyph’ 这是用于显示形似字符的字体，即看起来像但并不等同于其所表示的字符的字符（参见“文本的显示”）。

‘nobreak-space’ 这是用于显示不断行空格字符的字体（参见“文本的显示”）。

‘nobreak-hyphen’ 这是用于显示不断行连字符的字体（参见“文本的显示”）。

下列字体控制 Emacs 框架各部分的外观：

‘mode-line’ 这是用于模式行、标题行，以及不使用工具包菜单时的菜单栏的基础字体。默认情况下，在图形显示上它以阴影绘制出凸起效果，而在文本终端上则以默认字体的反色绘制。

‘mode-line-active’ 和 ‘mode-line-inactive’ 字体（模式行上实际使用的正是它们）继承自这个字体。

‘mode-line-active’ 与 ‘mode-line’ 类似，但用于当前选中窗口的模式行。此字体继承自 ‘mode-line’，因此对该字体的改动会影响所有窗口的模式行。

‘mode-line-inactive’ 与 ‘mode-line’ 类似，但用于选中窗口之外的其他窗口的模式行（若 ‘mode-line-in-non-selected-windows’ 为非 ‘nil’）。此字体继承自 ‘mode-line’，因此对该字体的改动会影响所有窗口的模式行。

‘mode-line-highlight’ 与 ‘highlight’ 类似，但用于模式行上对鼠标敏感的文本部分。当鼠标指针悬停其上时，这类文本部分通常会弹出工具提示（参见“工具提示”）。

‘mode-line-buffer-id’ 此字体用于模式行中标识缓冲区的部分。

‘header-line’ 类似 ‘mode-line’，但用于窗口的标题行，它出现在窗口顶部，正如模式行出现在底部一样。大多数窗口没有标题行——只有某些特殊模式（如 Info 模式）会创建标题行。

实际上用在标题行上的 ‘header-line-active’ 和 ‘header-line-inactive’ 两个字体继承自这个字体。

‘header-line-active’ 与 ‘header-line’ 类似，但用于当前选中窗口的标题行。此字体继承自 ‘header-line’，因此对该字体的改动会影响所有窗口的标题行。

‘header-line-inactive’ 与 ‘header-line’ 类似，但用于选中窗口之外的其他窗口的标题行（若那些窗口有标题行）。此字体继承自 ‘header-line’，因此对该字体的改动会影响所有窗口的标题行。

‘header-line-highlight’ 与 ‘highlight’ 和 ‘mode-line-highlight’ 类似，但用于标题行上对鼠标敏感的文本部分。之所以为它单设一个字体，是因为 ‘header-line’ 字体可能被定制成与 ‘highlight’ 配合不佳的样子。

‘tab-line’ 类似 ‘mode-line’，但用于窗口的标签行，它出现在窗口顶部，其中的标签代表窗口的缓冲区。参见“窗口标签行”。

实际上用在标签行上的 ‘tab-line-active’ 和 ‘tab-line-inactive’ 两个字体继承自这个字体。

‘tab-line-active’ 与 ‘tab-line’ 类似，但用于当前选中窗口的标签行。此字体继承自 ‘tab-line’，因此对该字体的改动会影响所有窗口的标签行。

‘tab-line-inactive’ 与 ‘tab-line’ 类似，但用于选中窗口之外的其他窗口的标签行（若那些窗口有标签行）。此字体继承自 ‘tab-line’，因此对该字体的改动会影响所有窗口的标签行。

‘vertical-border’ 此字体用于文本终端上窗口之间的竖直分隔线。

‘minibuffer-prompt’ 此字体用于迷你缓冲区中显示的提示字符串。默认情况下，Emacs 会自动把这个字体加入 ‘minibuffer-prompt-properties’ 的值中，该变量是一个用于显示提示文本的文本属性列表（参见《Emacs Lisp 参考手册》的“Text Properties”）。（此变量在你进入迷你缓冲区时生效。）

‘fringe’ 这是图形显示中窗口左右两侧边缘（fringe）所用的字体。（边缘是 Emacs 框架中位于文本区与窗口左右边框之间的狭窄部分。）参见“窗口边缘”。

‘cursor’ 此字体的 ‘:background’ 属性指定文本光标的颜色。参见“光标的显示”。

‘tooltip’ 此字体用于工具提示文本。默认情况下，若 Emacs 构建时带有 GTK+ 支持，工具提示由 GTK+ 绘制，此字体不起作用。参见“工具提示”。

‘mouse’ 此字体决定鼠标指针的颜色。

下列字体同样控制 Emacs 框架各部分的外观，但只在文本终端上，或在 Emacs 构建于 X 且无工具包支持时才起作用。（在其他所有情况下，相应框架元素的外观由系统范围的设置决定。）

‘scroll-bar’ 此字体决定滚动条的视觉外观。参见“滚动条”。

‘tool-bar’ 此字体决定工具栏图标的颜色。参见“工具栏”。

‘tab-bar’ 此字体决定标签栏图标的颜色。参见“标签栏”。

‘menu’ 此字体决定 Emacs 菜单的颜色和字体。参见“菜单栏”。

‘tty-menu-enabled-face’ 此字体用于在文本模式终端上显示已启用的菜单项。

‘tty-menu-disabled-face’ 此字体用于在文本模式终端上显示已禁用的菜单项。

‘tty-menu-selected-face’ 此字体用于在文本模式终端上显示若你点击鼠标或按 <RET> 就会被选中的那个菜单项。

16.11 图标

Emacs 有时会显示可点击的按钮（或其他提示性图标），你可以定制它们在显示上的外观。

这里主要的定制点是用户选项 'icon-preference'。通过它，你可以告诉 Emacs 你对图标的总的偏好。它是一份图标类型列表，将使用其中第一个受支持的类型。受支持的类型有：

'image' 用图像作为图标。

'emoji' 用彩色 emoji 作为图标。

'symbol' 用单色符号作为图标。

'text' 用简单文本作为图标。

此外，每个单独的图标都可以用 'M-x customize-icon' 定制，主题还可以进一步改变图标的外观。

要快速了解某个图标的说明，请使用 'M-x describe-icon' 命令。

16.12 文本缩放

要增大当前缓冲区中 'default' 字体的字号，输入 'C-x C-+' 或 'C-x C-='。要减小，输入 'C-x C--'。要恢复默认（全局）字号，输入 'C-x C-0'。这些键都绑定到同一个命令 'text-scale-adjust'，它查看最后输入的键来决定执行哪个操作，并通过更改默认字体的高度来相应地调整字号。

大多数字体没有显式设置 ':height' 属性，因而从 'default' 字体继承高度。这些字体也会被上述命令缩放。

除 'default' 之外显式设置了 ':height' 属性的字体不受这些字号变化的影响。'header-line' 字体是个例外：即使它显式设置了 ':height' 属性，也会被缩放。

类似地，当鼠标指针位于缓冲区文本之上时，按住 'Ctrl' 修饰键滚动鼠标滚轮，将根据滚动方向增大或减小所影响字体的字号。

这些命令的最后一个键可以重复输入，而不带前导的 'C-x'，也不带修饰键。例如，'C-x C-= C-= C-=' 和 'C-x C-= = = =' 都会把字体高度增加三步。每一步把文本高度缩放为原来的 1.2 倍；要改变这个系数，请定制变量 'text-scale-mode-step'。给 'text-scale-adjust' 命令一个数值为 0 的前缀参数会恢复默认高度，与输入 'C-x C-0' 相同。

类似地，要全局改变各字体的字号，输入 'C-x C-M-+'、'C-x C-M-='、'C-x C-M--' 或 'C-x C-M-0'，或者同时按住 'Ctrl' 和 'Meta' 修饰键滚动鼠标滚轮。要在全局改变字号时启用框架尺寸调整，请定制变量 'global-text-scale-adjust-resizes-frames'（参见"简易自定义界面"）。

命令 'text-scale-increase' 和 'text-scale-decrease' 分别像 'C-x C-+' 和 'C-x C--' 一样增大或减小当前缓冲区中的字体大小。你可能会觉得把它们（而不是 'text-scale-adjust'）绑定到键上更方便。

命令 'text-scale-set' 把当前缓冲区的字体大小缩放到由其前缀参数指定的绝对级别。

若当前字体缩放比例不是 1，上述命令会自动启用次模式 'text-scale-mode'，否则会禁用它。

命令 'text-scale-pinch' 根据在触摸板上做捏合手势时手指之间的距离来增大或减小文本缩放比例，该手势的做法是把两根手指放在触摸板上并让它们相互靠近或分开。此命令只在某些具有受支持硬件的系统上可用。

命令‘mouse-wheel-text-scale’也会改变文本缩放比例。通常，它在你移动鼠标滚轮时按住 <Ctrl> 的情况下运行。滚轮向下移动时文本缩放比例增大，向上移动时则减小。

16.13 Font Lock 模式

Font Lock 模式是一种次模式，始终是某个特定缓冲区的局部模式，它为缓冲区中的文本指定字体（即进行“字体化”，fontify）。每个缓冲区的主模式告诉 Font Lock 模式要对哪些文本进行字体化；例如，编程语言模式会对注释、字符串和函数名等语法上相关的构造进行字体化。

支持 Font Lock 模式的主模式默认启用它。要在当前缓冲区中切换它，输入‘M-x font-lock-mode’。正的数值参数无条件启用 Font Lock 模式，负的或零参数则禁用它。

输入‘M-x global-font-lock-mode’可在所有缓冲区中切换 Font Lock 模式。要让此设置在以后的 Emacs 会话中继续生效，请定制变量‘global-font-lock-mode’（参见“简易自定义界面”），或在初始化文件中加入下面一行：

```
(global-font-lock-mode 0)
```

若你已禁用全局 Font Lock 模式，仍可以通过把函数‘font-lock-mode’加入模式钩子来为特定主模式启用 Font Lock（参见“钩子”）。例如，要为编辑 C 文件启用 Font Lock 模式，可以这样做：

```
(add-hook 'c-mode-hook 'font-lock-mode)
```

Font Lock 模式使用几个专门命名的字体来完成工作，包括‘font-lock-string-face’、‘font-lock-comment-face’等。找出全部这些字体最容易的办法是使用‘M-x customize-group <RET> font-lock-faces <RET>’。然后你可以用那个定制缓冲区定制这些字体的外观。参见“自定义字体”。

对非常大的缓冲区进行字体化可能耗时很长。为避免访问文件时出现长时间延迟，Emacs 起初只对缓冲区可见的部分进行字体化。当你在缓冲区中滚动时，每一部分一旦显示出来就立即被字体化；这类 Font Lock 称为“即时”（Just-In-Time，即“JIT”）锁定。你可以通过定制‘jit-lock’定制组中的变量来控制 JIT Lock 的行为，包括让它空闲时执行字体化。参见“自定义特定项目”。

主模式用来确定缓冲区文本的哪些部分要字体化以及使用哪些字体的信息，可以基于几种不同的文本分析方式：

- 基于正则表达式搜索关键字和其他文本模式（参见“正则表达式搜索”）。
- 基于内置语法表找出语法上不同的文本部分（参见《Emacs Lisp 参考手册》的“Syntax Tables”）。
- 使用由完整解析器产生的语法树，通过某个专用库（例如 tree-sitter 库，参见《Emacs Lisp 参考手册》的“Parsing Program Source”）或外部程序。

16.13.1 传统 Font Lock

提供 font-lock 信息的“传统”方法基于正则表达式搜索，以及基于 Emacs 内置语法表的语法分析。本小节介绍对使用这些传统方法的主模式而言，font-lock 的使用与定制。

对于支持该功能的主模式，你可以通过定制变量‘font-lock-maximum-decoration’来控制 Font Lock 模式所应用的字体化程度。此变量的值应是一个数字（1 表示最少的字体化；有些模式支持高达 3 的级别）；或者是‘t’，

意为“尽可能高”（默认值）。要对某个文件缓冲区生效，‘font-lock-maximum-decoration’ 的定制应该在访问该文件之前完成；若你定制此变量时该文件已在某个缓冲区中被访问，请杀掉该缓冲区，并在定制后重新访问该文件。

你也可以为特定的主模式指定不同的数字；例如，要对 C/C++ 模式使用级别 1，其他模式使用默认级别，可使用如下值

```
'((c-mode . 1) (c++-mode . 1)))
```

注释和字符串的字体化（或称“语法”字体化）依赖于对缓冲区文本语法结构的分析。为了速度，某些模式（包括 Lisp 模式）依赖一种特殊约定：位于最左列的左圆括号或左花括号总是定义一个 defun 的开头，因而总是位于任何字符串或注释之外。因此，如果左圆括号或左花括号位于字符串或注释内部，就应避免把它放在最左列。详见参见“左边界约定”。

大多数模式已有现成的 Font Lock 高亮模式，但你可能还想字体化其他模式。你可以用函数 ‘font-lock-add-keywords’ 为特定模式添加自己的高亮模式。例如，要高亮 C 注释中的 ‘FIXME:’ 字样，可使用：

```
(add-hook 'c-mode-hook
          (lambda ()
           (font-lock-add-keywords nil
            '(("\\<\\(FIXME\\):" 1
               font-lock-warning-face t)))))
```

要从 font-lock 高亮模式中移除关键字，请使用函数 ‘font-lock-remove-keywords’。参见《Emacs Lisp 参考手册》的“Search-based Fontification”。另外，你也可以通过定制 ‘font-lock-ignore’ 选项有选择地禁用由某些关键字引起的高亮，参见《Emacs Lisp 参考手册》的“Customizing Keywords”。

16.13.2 基于解析器的 Font Lock

若你的 Emacs 构建时带有 tree-sitter 库，它就可以利用该库解析缓冲区文本的结果来进行字体化。这通常比上一小节描述的“传统”方法更快、更准确，因为 tree-sitter 库为其支持的各种编程语言和其他格式文本提供了完整的解析器。利用 tree-sitter 库的主模式命名为 ‘FOO-ts-mode’，其中 ‘-ts-’ 部分表示使用了该库。本小节介绍基于 tree-sitter 库的 Font Lock 支持。

对于基于 tree-sitter 的主模式，你可以通过定制变量 ‘treesit-font-lock-level’ 来控制 Font Lock 模式所应用的字体化程度。其值是 1 到 4 之间的数字：

1. 第 1 级：这一级通常只对注释和函数定义中的函数名做字体化。
1. 第 2 级：这一级增加对关键字、字符串和数据类型的字体化。
1. 第 3 级：这是默认级别；它增加对赋值、数字等的字体化。
1. 第 4 级：这一级增加所有其他可以字体化的内容：运算符、定界符、括号、其他标点、函数调用中的函数名、属性查找、变量等。

上面提到的每个语法类别究竟包含什么，取决于主模式以及 tree-sitter 为该主模式的语言所使用的解析器文法。不过一般来说，这些类别遵循该主模式所支持的编程语言或文件格式的惯例。变量 ‘treesit-font-lock-

feature-list' 的缓冲区局部值保存着基于 tree-sitter 的主模式所支持的字体化特征，其中每个子列表显示相应字体化级别所提供的特征。

一旦你通过 'M-x customize-variable'（参见"自定义特定项目"）改变了 'treesit-font-lock-level' 的值，它就会立即在所有现有缓冲区中生效，并在同一会话中对你以后访问的文件生效。

16.13.3 语义 Font Lock

"语义高亮"（semantic highlighting）是一项功能：编辑器借助某种语义分析来理解程序的源代码，并把关于不同词元含义的有用信息传达给用户，做法是根据这些词元在程序中的具体角色对它们进行高亮。

语义高亮比传统的"语法高亮"更精细，后者只考虑词元的语法角色，即它如何影响代码的*解析*，而语义分析则考虑词元对程序*执行*的影响。例如，某个语义高亮的实现可能能够区分局部变量和全局变量，并给每个类别以不同的高亮，即使该语言的*语法*并不作这种区分。在那些语法构造会因出现语境不同而含义完全不同的语言（如 Lisp 和 Prolog）中，语义高亮尤其有益。在这类语言中，仅靠语法分析会漏掉大量程序员推理其程序所需的重要信息。

有些语言服务器提供语义高亮信息，Emacs 可以通过其 LSP 客户端 Eglot 利用这些信息。参见《Eglot 手册》的"Eglot Features"。

此外，Emacs 还为 Emacs Lisp 实现了语义高亮，作为 'emacs-lisp-mode' 的可选功能（参见"执行 Lisp 表达式"）。要启用它，请把选项 'elisp-fontify-semantically' 定制为非 'nil' 值。本小节余下部分介绍这一 Emacs Lisp 专用语义高亮支持的使用方法。

启用 'elisp-fontify-semantically' 后，'emacs-lisp-mode' 会分析你的代码，并根据符号的语义角色高亮符号，这是该模式通常的 Font Lock 高亮的一部分（参见"Font Lock 模式"）。它不影响字符串、注释以及其他语法元素（如括号）的高亮；'elisp-fontify-semantically' 只影响符号的高亮。

语义分析为每个符号指定一个"符号角色"（symbol role），例如"函数"、"局部变量"、"字体名"等。每个符号角色都有一个关联的字体属性，在语义高亮时该属性会应用到具有该角色的符号上。默认情况下，这些字体大多继承自合适的 'font-lock-*' 字体。例如，局部绑定的变量使用 'elisp-bound-variable' 字体，该字体继承自 'font-lock-variable-use-face'。

语义分析能区分 50 多种这样的符号角色，但你无需记住这么多字体的外观就能利用语义高亮：你可以用鼠标悬停在某个已高亮的符号上，查看一个提示，其中给出 Emacs 为该符号推断出的确切角色（参见"工具提示"）。如果你想禁用这一额外信息，请把 'elisp-add-help-echo' 定制为 'nil' 值。

使用 'elisp-fontify-semantically' 时还有几点需要注意：

- 语法错误会破坏语义分析，因此为了获得最佳效果，你可能想启用 'electric-pair-mode'，以便在编辑代码时保持语法正确。参见"匹配括号"。
- 该分析在某些情况下会使用宏展开，但默认情况下只在可信缓冲区中这样做（参见"主机安全"）。在_不可信_缓冲区中，某些宏参数可能不会被高亮。有关 Emacs 认为可安全展开以进行分析的宏的更多信息，参见 'elisp-scope-safe-macro-p' 的文档字符串。用户选项 'elisp-scope-safe-macros' 控制哪些宏在不可信缓冲区的分析过程中可以安全展开。
- 该分析以当前 Emacs 会话中的定义为依据，因此使用未加载库的代码可能会遗漏一些高亮。

- 该分析假定已生效的是词法绑定（lexical binding，参见《Emacs Lisp 参考手册》的“Selecting Lisp Dialect”）。
- 语义高亮需要在传统 Font Lock 之外做额外处理——当前实现在编辑非常大的 defun 时可能较慢。

16.14 交互式高亮

高亮变更模式（Highlight Changes mode）是一个次模式，它通过给缓冲区中最近被修改过的文本指定不同的字体来“高亮”这些部分。要启用或禁用高亮变更模式，请使用 ‘M-x highlight-changes-mode’。

Hi Lock 模式是一个次模式，它高亮与你指定的正则表达式匹配的文本。例如，你可以用它高亮程序源文件中某个变量的所有引用、高亮某个程序大量输出中的某些部分，或者高亮文章中的某些名字。要启用或禁用 Hi Lock 模式，请使用命令 ‘M-x hi-lock-mode’。要为所有缓冲区启用 Hi Lock 模式，请使用 ‘M-x global-hi-lock-mode’，或者把 ‘(global-hi-lock-mode 1)’ 放进你的 ‘.emacs’ 文件。

Hi Lock 模式的工作方式与 Font Lock 模式类似（参见“Font Lock 模式”），区别在于要由你显式指定要高亮的正则表达式。你可以用以下命令控制它们。（下面以 ‘C-x w’ 开头的键绑定已废弃，改用全局的 ‘M-s h’ 绑定，并将在未来的某个 Emacs 版本中移除。）

‘M-s h r REGEXP <RET> FACE <RET>’ ‘C-x w h REGEXP <RET> FACE <RET>’ 用字体 FACE 高亮与 REGEXP 匹配的文本（‘highlight-regexp’）。只要缓冲区处于加载状态，高亮就会一直保留。例如，要用默认字体（黄色背景）高亮单词“whim”的所有出现，请键入 ‘M-s h r whim <RET> <RET>’。任何字体都可用于高亮，Hi Lock 自带若干字体，它们已预加载到一个默认值列表中。在提示选择字体时，用 ‘M-n’ 和 ‘M-p’ 在这些字体间循环。数字前缀参数会把高亮限制在相应的子表达式上。

把选项 ‘hi-lock-auto-select-face’ 设为非 ‘nil’ 值，会使该命令（以及其他读取字体的 Hi Lock 命令）自动从默认列表中选择下一个字体，而不进行提示。

你可以多次使用该命令，指定各种正则表达式，以不同的方式高亮。

‘M-s h u REGEXP <RET>’ ‘C-x w r REGEXP <RET>’ 取消高亮 REGEXP（‘unhighlight-regexp’）。如果你从菜单中调用它，你会从一个列表中选择要取消高亮的表达式。如果你从键盘调用它，则使用迷你缓冲区。它会显示最近添加的正则表达式；用 ‘M-n’ 显示下一个更早的表达式，用 ‘M-p’ 选择下一个更新的表达式。（你也可以手工键入表达式，并支持补全。）当你想取消高亮的表达式出现在迷你缓冲区中时，按 ‘<RET>’ 退出迷你缓冲区并取消其高亮。

‘M-s h l REGEXP <RET> FACE <RET>’ ‘C-x w l REGEXP <RET> FACE <RET>’ 用字体 FACE 高亮包含 REGEXP 匹配的整行（‘highlight-lines-matching-regexp’）。

‘M-s h p PHRASE <RET> FACE <RET>’ ‘C-x w p PHRASE <RET> FACE <RET>’ 用字体 FACE 高亮 PHRASE 的匹配（‘highlight-phrase’）。PHRASE 可以是任何正则表达式，但空格会被替换为对空白字符的匹配，开头的小写字母会变得不区分大小写。

‘M-s h .’ ‘C-x w .’ 用下一个可用的字体高亮光标点附近的符号（‘highlight-symbol-at-point’）。

‘M-s h w’ ‘C-x w b’ 把所有当前的高亮正则表达式/字体对插入到光标点处的缓冲区中，并加上注释定界符，以免它们改变你的程序。（该键绑定运行 ‘hi-lock-write-interactive-patterns’ 命令。）

如果你调用 ‘M-x hi-lock-find-patterns’，或者在 Hi Lock 模式启用时访问该文件（因为那会运行 ‘hi-lock-find-patterns’），这些模式会（在合适时）从注释中提取出来。

‘M-s h f’ ‘C-x w i’ 从当前缓冲区的注释中提取正则表达式/字体对（‘hi-lock-find-patterns’）。因此，你可以用 ‘highlight-regexp’ 交互式地输入模式，用 ‘hi-lock-write-interactive-patterns’ 把它们存入文件，编辑它们（或许为匹配中不同的括号部分指定不同的字体），最后用该命令（‘hi-lock-find-patterns’）让 Hi Lock 高亮编辑后的模式。

变量 ‘hi-lock-file-patterns-policy’ 控制 Hi Lock 模式在访问文件时是否应自动提取并高亮文件中找到的模式。它的值可以是 ‘nil’（从不自动高亮）、‘ask’（询问用户），或者一个函数。如果是个函数，‘hi-lock-find-patterns’ 会把模式作为参数调用它；若该函数返回非 ‘nil’，就使用这些模式。默认值是 ‘ask’。注意，如果你直接调用 ‘hi-lock-find-patterns’，无论该变量的值是什么，模式总会被高亮。

另外，如果当前主模式的符号是列表 ‘hi-lock-exclude-modes’ 的成员，‘hi-lock-find-patterns’ 什么也不做。

16.15 窗口边缘

在图形显示中，每个 Emacs 窗口通常在左右两边有狭窄的“边缘”（fringes）。边缘用于显示提供窗口文本信息的符号。你可以键入 ‘M-x fringe-mode’ 来切换边缘的显示或修改它们的宽度。该命令影响所有框架中的边缘；要只修改所选框架上的边缘，请使用 ‘M-x set-fringe-style’。你可以通过定制变量 ‘fringe-mode’ 使对边缘的修改永久生效。

边缘最常见的用途是指示续行（参见“续行”）。当一行文本被拆成多个屏幕行时，左边缘会为除第一行之外的每个屏幕行显示一个弯曲箭头，表示这不是真正的行首。右边缘会为除最后一个之外的每个屏幕行显示一个弯曲箭头，表示这不是真正的行尾。如果行方向是从右到左（参见“双向编辑”），边缘中弯曲箭头的含义则相反。

边缘用短的横向箭头指示行截断（参见“行截断”），表示该行还有更多文本被水平滚动到视野之外。用鼠标点击其中一个箭头，会沿箭头方向水平滚动显示。

边缘还可以指示其他东西，例如缓冲区边界（参见“显示边界”）、窗口末尾附近的未使用行（参见“indicate-empty-lines”），以及你正在调试的程序的执行位置（参见“在 Emacs 中运行调试器”）。

如果当前行的宽度恰好与窗口相同且光标点位于行尾，边缘还用于绘制光标。要禁用此行为，请把变量 ‘overflow-newline-into-fringe’ 改为 ‘nil’；这会使 Emacs 对恰好与窗口同宽的行进行续行或截断。

如果你把 ‘fringe-mode’ 定制为去掉窗口显示一侧或两侧的边缘，那么显示在边缘上的功能将不可用。续行和截断的指示符是例外：当边缘不可用时，Emacs 使用最左和最右的字符单元，以特殊的 ASCII 字符来指示续行和截断，参见“续行”和参见“行截断”。这会减少每行可用于显示文本的宽度，因为用于截断和续行指示符的字符单元被保留用于此目的。由于缓冲区文本可以包含双向文本，从而同时包含从左到右和从右到左的段落（参见“双向编辑”），只去掉一侧边缘时仍会保留两个字符单元（窗口两侧各一个）用于截断和续行指示符，因为在从右到左的段落中这些指示符显示在窗口的相对两侧。

16.16 显示边界

Emacs 可以显示 'fill-column' 位置的指示（参见"显式填充命令"）。填充列指示符是一项有用的功能，尤其是在 'prog-mode' 及其派生模式中（参见"主模式"），用来指示对程序源代码的排版有特殊意义的某一列的位置。这假定缓冲区使用固定宽度字体，即所有字符（双宽字符可能例外）在显示上宽度相同。如果缓冲区使用可变宽度字体，不同行上的填充列指示符可能看起来没有对齐。

要激活填充列指示显示，请使用次模式 'display-fill-column-indicator-mode' 和 'global-display-fill-column-indicator-mode'，它们分别局部或全局地启用该指示符。

或者，你可以设置两个缓冲区局部变量 'display-fill-column-indicator' 和 'display-fill-column-indicator-character' 来激活指示符并控制用于指示的字符。注意，两个变量都必须为非 'nil'，指示才会显示。（开启该次模式会设置这两个变量。）

有两个缓冲区局部变量和一个字体可用于定制该模式：

'display-fill-column-indicator-column' 指定应设置指示符的列号。它可以取正的列号数值，或者特殊值 't'，表示将使用变量 'fill-column' 的值。

任何其他值都会禁用指示符。默认值是 't'。

'display-fill-column-indicator-character' 指定用于指示符的字符。该字符可以是任何有效字符，若字体支持，也可以包括 Unicode 字符。值 'nil' 会禁用指示符。当通过函数 'display-fill-column-indicator-mode' 或 'global-display-fill-column-indicator-mode' 启用该模式时，如果该变量为非 'nil'，它们会使用该变量指定的字符；否则 Emacs 将使用字符 U+2502 BOX DRAWINGS LIGHT VERTICAL，如果 U+2502 无法显示则回退到 '|'。

'fill-column-indicator' 指定用于显示指示符的字体。它从字体 'shadow' 继承默认值，但不带背景色。要改变指示符的颜色，你只需设置该字体的前景色。

在图形显示中，Emacs 可以在边缘中指示缓冲区边界。如果启用该功能，第一行和最后一行会在边缘中标记为角形图像。这可以与表示是否能滚动窗口的上下箭头图像结合使用。

缓冲区局部变量 'indicate-buffer-boundaries' 控制如何在边缘中指示缓冲区边界和窗口滚动。如果值为 'left' 或 'right'，则角形和箭头位图都分别在左边缘或右边缘中显示。

如果值是关联列表（参见《Emacs Lisp 参考手册》的"Association Lists"），每个元素 '(INDICATOR . POSITION)' 指定其中一个指示符的位置。INDICATOR 必须是 'top'、'bottom'、'up'、'down' 或 't' 之一；'t' 指定关联列表中未出现的指示符的默认位置。POSITION 是 'left'、'right' 或 'nil' 之一，'nil' 表示不显示该指示符。

例如，'((top . left) (t . right))' 把顶部角形位图放在左边缘，把底部角形位图放在右边缘，并把两个箭头位图都放在右边缘。如果只想在左边缘显示角形位图而不显示箭头位图，请使用 '((top . left) (bottom . left))'。

16.17 无用的空白

很容易在不知不觉中于行尾留下不必要的空格，或在缓冲区末尾留下空行。多数情况下，这种"行尾空白"没有影响，但有时会带来麻烦。

你可以通过把缓冲区局部变量 'show-trailing-whitespace' 设为 't' 使行尾空白可见。这样 Emacs 就会用字体 'trailing-whitespace' 显示行尾空白。

当光标点位于含该空白的行尾时，此功能不适用。严格说来，那仍然是行尾空白，但在那种情况下特别显示它，在你键入新文本时会显得难看。在这种特殊情况下，光标点的位置足以向你表明空格的存在。

键入 'M-x delete-trailing-whitespace' 可删除所有行尾空白。该命令会删除缓冲区中每一行末尾的所有多余空格，以及缓冲区末尾的所有空行；要忽略后者，请把变量 'delete-trailing-lines' 改为 'nil'。如果区域处于活动状态，该命令则改为删除区域中每一行末尾的多余空格。如果你启用缓冲区局部模式 'delete-trailing-whitespace-mode'，Emacs 将在你每次保存缓冲区时自动调用 'delete-trailing-whitespace'。

在图形显示中，Emacs 可以用左边缘中的一个小图像指示窗口末尾的未使用行（参见"窗口边缘"）。该图像出现在没有对应任何缓冲区文本的屏幕行上，因此缓冲区末尾的空行会因缺少该图像而显得突出。要启用该功能，请把缓冲区局部变量 'indicate-empty-lines' 设为非 'nil' 值。你可以通过设置该变量的默认值（例如 '(setq-default indicate-empty-lines t)'）为所有新缓冲区启用或禁用该功能。

空白模式（Whitespace mode）是一个缓冲区局部次模式，它让你可以把缓冲区中许多种类的空白可视化，办法是用特殊字体绘制空白字符，或把它们显示为特殊字形。要切换该模式，请键入 'M-x whitespace-mode'。被可视化的空白种类由列表变量 'whitespace-style' 决定。你可以通过键入 'M-x whitespace-toggle-options' 在当前缓冲区中打开或关闭该列表中的各个元素。下面是可能元素的部分列表（完整列表见该变量的文档）：

'face' 启用所有使用特殊字体的可视化。该元素有特殊含义：如果它不在列表中，除 'space-mark'、'tab-mark' 和 'newline-mark' 之外，其他可视化都不会生效。

'trailing' 高亮行尾空白。

'tabs' 高亮制表字符。

'spaces' 高亮空格和不换行空格字符。

'lines' 高亮长于 80 列的行。要改变列数限制，请定制变量 'whitespace-line-column'。

'newline' 高亮换行符。

'missing-newline-at-eof' 如果缓冲区末尾没有换行字符，则高亮最后一个字符。

'empty' 高亮缓冲区开头和/或末尾的空行。

'big-indent' 高亮过深的缩进。默认情况下，任何至少 4 个连续制表字符或 32 个连续空格字符的序列都会被高亮。要改变这一点，请定制正则表达式 'whitespace-big-indent-regexp'。

'space-mark' 用特殊字形绘制空格和不换行字符。

'tab-mark' 用特殊字形绘制制表字符。

'newline-mark' 用特殊字形绘制换行字符。

全局空白模式（Global Whitespace mode）是一个全局次模式，它让你可以把所有缓冲区中的空白可视化。要切换各个功能，请使用 'M-x global-whitespace-toggle-options'。

16.18 选择性显示

Emacs 能够隐藏缩进超过给定列数的行。你可以用这一点来获得程序某部分的概览。

要在当前缓冲区中隐藏行，请键入 'C-x $'（'set-selective-display'）并带一个数字参数 N。这样，缩进至少为 N 列的行就会从屏幕上消失。它们存在的唯一迹象是：在每一条后面跟着一个或多个隐藏行的可见行末尾出现三个点（'...'）。

命令 'C-n' 和 'C-p' 会跨过隐藏的行，就好像它们不存在一样。

隐藏的行仍然留在缓冲区中，大多数编辑命令照常把它们视为存在，因此你可能会发现光标点位于隐藏文本的中间。发生这种情况时，光标出现在前一行末尾、三个点之后。如果光标点位于可见行的末尾、即结束该行的换行符之前，光标则出现在三个点之前。

要使所有行再次可见，请不带参数地键入 'C-x $'。

如果你把变量 'selective-display-ellipses' 设为 'nil'，三个点就不会出现在隐藏行之前的行末尾。这样就没有隐藏行的可见迹象了。设置该变量时，它会自动成为局部的。

隐藏缓冲区中部分文本的另一种方法，参见"大纲模式"。

16.19 可选的模式行功能

缓冲区百分比 POS 表示窗口顶部以上的部分占缓冲区的百分比。你还可以通过键入 'M-x size-indication-mode' 打开大小指示模式（Size Indication mode），从而显示缓冲区的大小。大小会紧跟在缓冲区百分比之后显示，像这样：

```
POS of SIZE
```

这里的 SIZE 是缓冲区字符数的易读表示，也就是说会用 'k' 表示 10^3、'M' 表示 10^6、'G' 表示 10^9 等等来简写。

启用行号模式（Line Number mode）时，光标点当前的行号会出现在模式行中。使用命令 'M-x line-number-mode' 打开或关闭该模式；通常它是打开的。行号出现在缓冲区百分比 POS 之后，并带有字母 'L' 以表明它是什么。

类似地，你可以用 'M-x column-number-mode' 打开列号模式（Column Number mode）来显示当前列号。列号用字母 'C' 表示。不过，当这两个模式都启用时，行号和列号会显示在括号中，行号在前，而不用 'L' 和 'C'。例如：'(561,2)'。（参见"次模式"，其中有关于次模式以及如何使用这些命令的更多信息。）

在列号模式中，显示的列号从窗口左边缘的一列开始从零计数。如果你希望显示的列号从一开始计数，可以把 'column-number-indicator-zero-based' 设为 'nil'。

如果你对缓冲区进行了窄化（参见"窄化"），显示的行号是相对于缓冲区可访问部分的。因此，它不适合作为 'goto-line' 的参数。（命令 'what-line' 显示相对于整个文件的行号。）你可以使用 'goto-line-relative' 命令把光标点移到相对于窄化缓冲区可访问部分的行。

如果缓冲区非常大（大于 'line-number-display-limit' 的值），Emacs 将不会计算行号，因为那会太慢；因此，行号不会出现在模式行上。要取消该限制，请把 'line-number-display-limit' 设为 'nil'。

如果缓冲区中的行太长，行号计算也可能很慢。为此，如果光标点附近各行的平均宽度（以字符计）大于‘line-number-display-limit-width’的值，Emacs 就不显示行号。默认值是 200 个字符。

Emacs 可以选择在所有模式行中显示时间和系统负载。要启用该功能，请键入‘M-x display-time’或定制选项‘display-time-mode’。添加到模式行的信息看起来像这样：

```
HH:MMPM L.LL
```

这里的 HH 和 MM 是小时和分钟，后面总是跟着‘AM’或‘PM’。L.LL 是最近几分钟内采集的整个系统中正在运行或已就绪可运行（即正在等待可用处理器）的进程数的平均值。（如果你的操作系统不支持某些字段，它们可能会缺失。）如果你更喜欢 24 小时格式的时间显示，请把变量‘display-time-24hr-format’设为‘t’。

如果有你尚未阅读的邮件，负载级别之后会出现单词‘Mail’。在图形显示中，你可以通过定制‘display-time-use-mail-icon’用一个图标代替‘Mail’；这可以节省模式行上的一些空间。你可以定制‘display-time-mail-face’使邮件指示符醒目。使用‘display-time-mail-file’指定要检查的邮件文件，或者设置‘display-time-mail-directory’指定要检查新到邮件的目录（该目录中任何非空的普通文件都被视为新到的邮件）。

在笔记本电脑上运行 Emacs 时，可以用命令‘display-battery-mode’在模式行上显示电池电量，也可以自定义变量‘display-battery-mode’。变量‘battery-mode-line-format’决定电池电量的显示方式；具体的模式行消息取决于操作系统，通常以占总电量的百分比显示当前电池电量。‘battery-update-functions’中的函数会在模式行更新之后运行，可用于根据电池状态触发相应操作。

在图形显示上，模式行被绘制成一个 3D 盒子。若不喜欢这种效果，可以自定义‘mode-line’字体（face）并将其‘box’属性设为‘nil’来禁用它。参见“自定义字体”。

默认情况下，非选中窗口的模式行用另一种字体显示，名为‘mode-line-inactive’。只有被选中的窗口才用‘mode-line’字体显示。这有助于表明哪个窗口被选中。当迷你缓冲区被选中时，由于它没有模式行，激活迷你缓冲区的那个窗口的模式行会用‘mode-line’显示；因此，正常进入迷你缓冲区不会改变任何模式行。

你可以把变量‘mode-line-in-non-selected-windows’设为‘nil’来禁用‘mode-line-inactive’；此后所有模式行都用‘mode-line’字体显示。

你可以针对每种行尾格式自定义模式行的显示方式，方法是将变量‘eol-mnemonic-unix’、‘eol-mnemonic-dos’、‘eol-mnemonic-mac’和‘eol-mnemonic-undecided’分别设置为你偏好的字符串。

有些模式在模式行中放入大量数据，把模式行末尾的元素挤到右边。若变量‘mode-line-compact’非‘nil’，Emacs 可以“压缩”模式行，把连续的空格变成单个空格。若该变量为‘long’，则只在模式行比当前选中的窗口更宽时才这样做。（该计算是近似的，基于字符数而非字符的显示宽度。）该变量可以设为缓冲区局部，从而只压缩某些缓冲区中的模式行。要进一步“压缩”模式行，可以把选项‘mode-line-collapse-minor-modes’自定义为非‘nil’值，Emacs 就会把模式行上的一些次模式指示符折叠成单个可点击按钮，从而隐藏它们。

默认情况下，大多数缓冲区都显示模式行，但若模式行在某个缓冲区中用处不大，可以用命令‘M-x mode-line-invisible-mode’隐藏当前缓冲区中的模式行，模式行会一直保持隐藏，直到再次调用该命令。要在进入某个主模式时总是隐藏模式行，可把‘mode-line-invisible-mode’加入钩子（参见“钩子”）。

16.20 文本的显示

大多数字符是“打印字符”（printing characters）：当它们出现在缓冲区中时，会原样显示在屏幕上。打印字符包括 ASCII 数字、字母和标点字符，以及许多非 ASCII 字符。

ASCII 字符集包含非打印的“控制字符”。其中两个被特殊显示：换行符（Unicode 码点 U+000A）通过另起一行来显示，而制表符（U+0009）显示为一个延伸到下一个制表位列的空白（通常每 8 列一个制表位）。每个制表符对应的空格数由缓冲区局部变量 ‘tab-width’ 控制，其值必须是 1 到 1000（含）之间的整数。注意，缓冲区中制表符的显示方式与 <TAB> 作为命令的定义无关。

其他 ASCII 控制字符的编码低于 U+0020（八进制 40，十进制 32），它们显示为脱字符（‘^’）后跟该字符的非控制版本，使用 ‘escape-glyph’ 字体。例如，‘control-A’ 字符（U+0001）显示为 ‘^A’。

编码从 U+0080（八进制 200）到 U+009F（八进制 237）的原始字节显示为“八进制转义序列”，使用 ‘escape-glyph’ 字体。例如，字符编码 U+0098（八进制 230）显示为 ‘\230’。若把缓冲区局部变量 ‘ctl-arrow’ 改为 ‘nil’，ASCII 控制字符也会显示为八进制转义序列，而不是脱字符转义序列。（你也可以要求以十六进制显示原始字节，参见“显示的自定义”）

有些非 ASCII 字符的外观与 ASCII 空格或连字符（减号）相同。若这些字符在你未察觉的情况下被输入缓冲区（例如通过粘贴），可能会造成问题；例如，源代码编译器通常不把非 ASCII 空格当作空白字符。为处理这个问题，Emacs 会特殊显示此类字符：它用 ‘nobreak-space’ 字体显示 U+00A0 NO-BREAK SPACE 以及 Unicode 水平空格类中的其他字符，用 ‘nobreak-hyphen’ 字体显示 U+00AD SOFT HYPHEN、U+2010 HYPHEN 和 U+2011 NON-BREAKING HYPHEN。要禁用这一功能，可将变量 ‘nobreak-char-display’ 改为 ‘nil’。若给该变量一个非 ‘nil’ 且非 ‘t’ 的值，Emacs 会把这些字符显示为高亮的反斜杠后跟一个空格或连字符。

你可以借助显示表（display table）自定义任何特定字符编码的显示方式。参见《Emacs Lisp 参考手册》的“Display Tables”

在图形显示上，有些字符在 Emacs 可用的任何字体中都没有字形。这些“无字形字符”（glyphless characters）通常显示为内部带有十六进制字符编码的方框。类似地，在文本终端上，无法用终端编码显示的字符（参见“终端 I/O 的编码系统”）通常显示为问号。你可以通过自定义变量 ‘glyphless-char-display-control’ 来控制显示方式。也可以自定义 ‘glyphless-char’ 字体，使这些字符在显示时更醒目。参见《Emacs Lisp 参考手册》的“Glyphless Chars, for details”

次模式 ‘glyphless-display-mode’ 可用于切换当前缓冲区中无字形字符的显示方式。无字形字符将显示为内部带有其名称缩写的方框。

Emacs 会尝试判断当前显示设备能否显示弯引号 ‘ 和 ’。默认情况下，若判断结果为可以，Emacs 在消息和帮助文本中遇到 ASCII 引号（‘`’ 和 “'”）时，就会把它们转换为这些弯引号。你可以通过自定义用户选项 ‘text-quoting-style’ 来影响或禁止这种转换（参见《Emacs Lisp 参考手册》的“Keys in Documentation”）。

若已知弯引号 ‘、’、“ 和 ” 的外观与 ASCII 字符完全相同，则用 ‘homoglyph’ 字体显示它们。已知无法显示的弯引号则以其 ASCII 近似形式 ‘`’、“'” 和 “"” 用 ‘homoglyph’ 字体显示。

16.21 光标的显示

在文本终端上，光标的外观由终端控制，基本上不受 Emacs 掌控。某些终端提供两种不同的光标：可见的静态光标和非常显眼的闪烁光标。默认情况下，Emacs 使用非常显眼的那种光标，并在启动或恢复 Emacs 时切换到它。若 Emacs 启动或恢复时变量 'visible-cursor' 为 'nil'，则使用普通光标。

在图形显示和许多兼容 Xterm 的文本终端上，可以改变文本光标的颜色和形状。要自定义其颜色，可修改名为 'cursor' 的字体的 ':background' 属性（参见"自定义字体"）。（该字体的其他属性没有效果；光标之下显示的文本使用框架的背景色绘制。）要改变其形状，可自定义缓冲区局部变量 'cursor-type'；可能的值有 'box'（默认值）、'(box . SIZE)'（当被遮罩的图像在任一维度上大于 SIZE 像素时，实心框变为空心框）、'hollow'（空心框）、'bar'（竖条）、'(bar . N)'（N 像素宽的竖条）、'hbar'（横条）、'(hbar . N)'（N 像素高的横条）或 'nil'（完全不显示光标）。

在兼容 Xterm 的文本终端上，光标自定义由用户选项 'xterm-update-cursor' 控制。有效值为：'t' 更新光标的颜色和形状，'type' 只更新光标的形状，'color' 只更新光标的颜色，'nil' 不更新光标的外观。文本终端无法显示空心框，而是改用实心框。类似地，所有文本终端都忽略 'bar' 和 'hbar' 的像素尺寸。

要让光标更加醒目，可以使用高亮行模式（HL Line mode），这是一个高亮包含光标点的那一行的次模式。使用 'M-x hl-line-mode' 在当前缓冲区中启用或禁用它。'M-x global-hl-line-mode' 全局启用或禁用同一模式。

其余控制项只在 Emacs 能完全控制光标外观的图形显示上有效。

默认情况下，若 Emacs 在这段时间内没有收到任何输入，光标闪烁 10 次后便停止；任何输入事件都会重新开始计数。你可以自定义变量 'blink-cursor-blinks' 来控制这一点：其值表示在没有输入的情况下闪烁多少次后停止。把该变量设为 0 或负值会使光标永远闪烁。要完全禁用光标闪烁，可将变量 'blink-cursor-mode' 改为 'nil'（参见"简易自定义界面"），或把下面这行

```
(blink-cursor-mode 0)
```

加入你的初始化文件。或者，你可以通过自定义列表变量 'blink-cursor-alist' 来改变光标闪烁熄灭时的外观。列表中的每个元素都应采用 '(ON-TYPE . OFF-TYPE)' 形式；这意味着，若光标闪烁亮起时显示为 ON-TYPE（其中 ON-TYPE 是上文所述光标类型之一），则闪烁熄灭时显示为 OFF-TYPE。

有些字符（如制表符）特别宽。当光标位于此类字符之上时，通常按默认字符宽度绘制。你可以把变量 'x-stretch-cursor' 改为非 'nil' 值，让光标拉伸以覆盖宽字符。

光标在非选中窗口中通常显示为不闪烁的空心框。（对于竖条光标，则显示为更细的竖条。）要关闭非选中窗口中的光标，可将变量 'cursor-in-non-selected-windows' 改为 'nil'。

16.22 行截断

作为续行的替代方案（参见"续行"），Emacs 可以通过"截断"（truncation）显示长行。这意味着所有无法容纳在屏幕或窗口宽度内的字符完全不显示。在图形显示上，行两端边缘（fringe）中的小直箭头表示发生了截断。在文本终端上，则用最右和/或最左列中的 '$' 符号表示。

水平滚动会自动导致行截断（参见"水平滚动"）。你可以用命令 'C-x x t'（'toggle-truncate-lines'）显式地为某个缓冲区启用行截断。其工作方式是局部地改变变量 'truncate-lines'。若该变量非 'nil'，长行被截断；若为

‘nil’，则续行到多个屏幕行上。以任何方式设置变量 ‘truncate-lines’ 都会使其成为当前缓冲区的局部变量；在此之前，生效的是默认值，通常为 ‘nil’。

由于行截断与词换行（下一节介绍）相互矛盾，‘toggle-truncate-lines’ 在启用行截断时会禁用词换行。

若拆分后的窗口变得太窄，Emacs 可能会自动启用行截断。控制这一行为的变量 ‘truncate-partial-width-windows’ 见参见“拆分窗口”。

16.23 可视行模式

普通续行（参见“续行”）的另一种替代方案是使用“词换行”（word wrap）。此时，每个长逻辑行被分成两个或多个屏幕行，或称“可视行”，与普通续行类似。不过，Emacs 会尝试在靠近窗口右边缘的单词边界处换行。（若行的方向是从右到左，则在左窗口边缘处换行。）这使文本更易阅读，因为换行不会发生在单词中间。

词换行由可视行模式（Visual Line mode）启用，这是一个可选的次模式。要在当前缓冲区中打开可视行模式，输入 ‘M-x visual-line-mode’；重复该命令会关闭它。也可以使用菜单栏打开可视行模式：在 Options 菜单中选择 ‘Line Wrapping in this Buffer’ 子菜单，然后选择 ‘Word Wrap (Visual Line mode)’ 菜单项。启用可视行模式时，模式行的模式显示中会显示字符串 ‘wrap’。命令 ‘M-x global-visual-line-mode’ 在所有缓冲区中切换可视行模式。

由于词换行与行截断（上一节介绍）相互矛盾，打开 ‘visual-line-mode’ 会禁用行截断。

在可视行模式中，某些编辑命令作用于屏幕行而非逻辑行：‘C-a’（‘beginning-of-visual-line’）移动到屏幕行开头，‘C-e’（‘end-of-visual-line’）移动到屏幕行末尾，而 ‘C-k’（‘kill-visual-line’）删除到屏幕行末尾的文本。

要按逻辑行移动，使用命令 ‘M-x next-logical-line’ 和 ‘M-x previous-logical-line’。它们分别把光标点移到下一逻辑行和上一逻辑行，无论是否启用可视行模式。若你经常使用这些命令，为它们分配键绑定可能比较方便。参见“在初始化文件中重绑定按键”。

默认情况下，换行后的行不显示边缘指示符。可视行模式常用于编辑包含许多长逻辑行的文件，因此为每个换行的行显示边缘指示符会在视觉上分散注意力。你可以通过自定义变量 ‘visual-line-fringe-indicators’ 来改变这一点。

默认情况下，Emacs 只在 <SPC> 和 <TAB> 这类空白字符之后断行，而不会在 <EN QUAD> 这类空白字符之后断行。Emacs 提供了一个名为 ‘word-wrap-whitespace-mode’ 的次模式，它在当前主模式中打开词换行，并根据用户选项 ‘word-wrap-whitespace-characters’ 设定以哪些字符作为换行点。还有一个该模式的全局化版本，名为 ‘global-word-wrap-whitespace-mode’。

只在空白字符之后断行，在 CJK 与拉丁文本混排时会产生不正确的结果（因为 CJK 字符不用空白分隔单词）。你可以自定义选项 ‘word-wrap-by-category’，让 Emacs 在具有 ‘|’ 类别的任何字符之后断行（参见《Emacs Lisp 参考手册》的“Categories”），从而更好地支持 CJK 字符。此外，若使用 Customize 设置该变量，Emacs 会自动加载 ‘kinsoku.el’。加载 ‘kinsoku.el’ 后，Emacs 断行时会遵循禁则（kinsoku）规则。这意味着具有 ‘>’ 类别的字符不会出现在行首（例如 U+FF0C FULLWIDTH COMMA），而具有 ‘<’ 类别的字符不会出现在行尾（例如 U+300A LEFT DOUBLE ANGLE BRACKET）。你可以使用命令 ‘char-category-set’ 和 ‘category-set-mnemonics’ 查看某个字符的类别集合，或者在光标位于该字符上时输入 ‘C-u C-x =’ 并查看报告中的“category”部分。你可以使用命令 ‘modify-category-entry’ 为字符添加类别。

16.24 显示的自定义

本节介绍控制 Emacs 屏幕外观各个方面（杂项）的变量。初学者可以跳过。

若你想让 Emacs 为缓冲区中的每一行显示行号，可自定义缓冲区局部变量 ‘display-line-numbers’；它默认为 ‘nil’。该变量可以取几个不同的值，以支持各种行号显示模式：

- ‘t’：在每个显示缓冲区文本的非续行屏幕行之前显示（绝对）行号。若该行是续行，或整个屏幕行显示的是显示字符串或覆盖层字符串，则该行不编号。
- ‘relative’：在显示缓冲区文本的非续行之前显示相对行号。行号相对于显示光标点的那一行，因此当行离当前行越远时，数字向上和向下都增大。
- ‘visual’：该值使 Emacs 以可视方式计数行：只计算实际显示在屏幕上的行（忽略文本不可见部分中的任何行），而换行后占据多个屏幕行的行会被编号多次。显示的数字是相对的，与上面的 ‘relative’ 值一样。这在折叠文本的模式（如大纲模式，参见“大纲模式”）中，以及在你需要按精确的屏幕行数移动时很方便。
- 其他任何值：任何其他非 ‘nil’ 值都当作 ‘t’ 处理。

命令 ‘M-x display-line-numbers-mode’ 提供了打开行号显示的便捷方式。该模式有一个全局化变体 ‘global-display-line-numbers-mode’。用户选项 ‘display-line-numbers-type’ 控制这些模式将激活上文所述哪种行号显示子模式。

注意，即使全局打开 ‘display-line-numbers-mode’，行号也不会在迷你缓冲区和工具提示中显示。

当 Emacs 显示相对行号时，你可以控制当前行（即显示光标点的那一行）之前所显示的数字。默认情况下，Emacs 在那里显示当前行的绝对行号，尽管其他所有行号都是相对的。若把变量 ‘display-line-numbers-current-absolute’ 自定义为 ‘nil’ 值，当前行显示的数字将为零。若你不在意当前行的行号，并希望在大型缓冲区中为文本留出更多横向空间，这会很方便。

在窄化过的缓冲区中（参见“窄化”），行通常从窄化范围的起始处开始编号。不过，若把变量 ‘display-line-numbers-widen’ 自定义为非 ‘nil’ 值，行号将忽略任何窄化，从缓冲区的第一个字符开始编号。

若 ‘display-line-numbers-offset’ 的值非零，它会被加到每个绝对行号上，并且行从缓冲区开头开始计数，就如同 ‘display-line-numbers-widen’ 非 ‘nil’ 一样。当它设为零，或行号不是绝对行号时，它没有效果。

在选择性显示模式（参见“选择性显示”）以及其他从显示中隐藏许多行的模式（如大纲模式和 Org 模式）中，你可能希望自定义变量 ‘display-line-numbers-width-start’ 和 ‘display-line-numbers-grow-only’，或者把 ‘display-line-numbers-width’ 设为一个足够大的值，以避免偶尔错误计算为行号预留的空间。

行号用特殊字体 ‘line-number’ 显示。当前行的行号用另一种字体 ‘line-number-current-line’ 显示，因此你可以让当前行的行号具有独特外观，这有助于定位显示光标点的那一行。额外的字体 ‘line-number-major-tick’ 和 ‘line-number-minor-tick’ 可用于高亮那些行号为某些数字之倍数的行号。分别自定义 ‘display-line-numbers-major-tick’ 和 ‘display-line-numbers-minor-tick’ 来设置这些数字。

变量 ‘line-spacing’ 控制行与行之间的垂直间距。它可以设为整数（指定像素）或浮点数（指定相对于默认框架字体高度的间距）。你也可以把该变量设为整数或浮点数的 cons 单元，如 ‘(TOP . BOTTOM)’。设为 cons 单元

时，间距分布在行的上方和下方，使文本能在行高内垂直居中。另见 参见《Emacs Lisp 参考手册》的“Line Height”。

若变量 ‘visible-bell’ 非 ‘nil’，Emacs 在通常要发出可闻响铃时会尝试让整个屏幕闪烁。若你的终端无法使屏幕闪烁，该变量没有效果。

变量 ‘echo-keystrokes’ 控制多字符键的回显；其值是需要暂停的秒数，达到该时长后才开始回显，值为零表示完全不回显。该值在有内容需要回显时才会生效。参见“回显区”。

若变量 ‘echo-keystrokes-help’ 非 ‘nil’（默认值），则按 ‘echo-keystrokes’ 显示的多字符键序列回显中会包含一段简短帮助文本，介绍用于调用 ‘describe-prefix-bindings’ 的按键（参见“其他帮助命令”），以显示你已经输入的前缀的命令列表。关于相关的帮助功能，参见“which-key”。

在图形显示上，若 Emacs 繁忙，会把鼠标指针显示为沙漏。要禁用此功能，将变量 ‘display-hourglass’ 设为 ‘nil’。变量 ‘hourglass-delay’ 决定显示沙漏前的繁忙时间秒数；默认值为 1。

若鼠标指针位于某个 Emacs 框架内，则每次你输入字符以插入文本时，Emacs 都会使其不可见，以防它遮挡文本。（准确地说，隐藏发生在你输入自插入字符时。参见“插入文本”。）移动鼠标指针会使其重新可见。要禁用此功能，将变量 ‘make-pointer-invisible’ 设为 ‘nil’。

在图形显示上，变量 ‘underline-minimum-offset’ 决定带下划线文本的基线与下划线之间的最小距离，以像素计。默认值为 1；增大它可改善某些字体下带下划线文本的可读性。（不过，Emacs 绝不会在当前行区域之下绘制下划线。）变量 ‘x-underline-at-descent-line’ 决定如何绘制带下划线的文本。默认值为 ‘nil’，表示在字体的基线高度处绘制；若改为 ‘t’，Emacs 会在与字体下降线相同的高度绘制下划线。（若为带下划线的文本指定了非默认行距，参见《Emacs Lisp 参考手册》的“Line Height”，Emacs 会在额外间距之下绘制下划线。）

变量 ‘overline-margin’ 指定上划线在文本上方的垂直位置，包括上划线自身的高度，以像素计；默认值为 2。

在某些文本终端上，粗体与反显同时使用会导致文本难以阅读。此时以非 ‘nil’ 参数调用函数 ‘tty-suppress-bold-inverse-default-colors’ 即可抑制粗体的效果。

原始字节默认以八进制格式显示，例如十进制值为 128 的字节显示为 ‘\200’。要改为十六进制格式 ‘\x80’ 显示，可将变量 ‘display-raw-bytes-as-hex’ 设为 ‘t’。在从包含 Emacs 会话的终端复制文本时解释原始字节，或终端的 ‘escape-glyph’ 字体看起来与默认字体相同时，可能需要小心。例如，默认情况下 Emacs 显示四个字符 ‘\’、‘2’、‘0’、‘0’ 所用的字符，与它显示十进制值为 128 的字节所用的字符相同。使用十六进制显示时问题可能更严重：原始字节 128 后跟字符 ‘7’ 会显示为 ‘\x807’，而 Emacs Lisp 会把它读作单个字符 U+0807 SAMARITAN LETTER IT；相应的八进制显示 ‘\2007’ 则不会出现这种混淆，因为八进制转义最多包含三个数字。

17 搜索与替换

与其他编辑器一样，Emacs 提供了搜索某个字符串出现位置的命令。Emacs 也有用一个字符串替换另一个字符串的命令。还有一些命令做同样的事，但搜索的是模式而不是固定字符串。

你还可以在 'xref' 的控制下（参见"用标识符搜索与替换"）、或者通过 Dired 的 'A' 命令（参见"对文件进行操作"）搜索多个文件，也可以让 'grep' 程序去做（参见"在 Emacs 中用 Grep 搜索"）。

- Lax Search：搜索会忽略相似字符之间的一些区别，例如字母大小写。
- Replace：搜索并替换部分或全部匹配项。
- Other Repeating Search：对某个正则表达式的所有匹配项进行操作。
- Search Customizations：各种搜索定制。

17.1 增量搜索

Emacs 中主要的搜索命令是"增量式"的：只要你键入搜索字符串的第一个字符，它就开始搜索。随着你键入搜索字符串，Emacs 会显示出到目前为止所键入的字符串会在哪里被找到。当你键入的字符已足以确定你想要的位置时，就可以停下。至于是否需要显式地用 <RET> 终止搜索，则取决于你接下来打算做什么。

'C-s' 向前增量搜索（'isearch-forward'）。'C-r' 向后增量搜索（'isearch-backward'）。

你也可以从菜单栏的 'Edit->Search' 菜单调用增量搜索。

- Isearch Yank：把文本取入搜索字符串、或者编辑搜索字符串的命令。
- Error in Isearch：当你的字符串找不到时。
- Special Isearch：增量搜索中的特殊输入。
- Not Exiting Isearch：前缀参数与滚动命令。
- Isearch Minibuffer：增量搜索迷你缓冲区的历史。

17.1.1 增量搜索基础

'C-s' 开始向前增量搜索（'isearch-forward'）。'C-r' 开始反向增量搜索（'isearch-backward'）。

'C-s'（'isearch-forward'）启动向前增量搜索。它从键盘读取字符，并把光标点移到缓冲区中下一处出现这些字符的位置之后。

例如，如果你键入 'C-s'，然后键入 'F'，光标就会移到缓冲区中起始位置之后第一个 'F' 的后面。接着键入 'O'，光标就移到第一个 'FO' 之后；这个 'FO' 中的 'F' 未必是先前找到的那个 'F'。再键入一个 'O'，光标便移到第一个 'FOO' 之后。

每一步中，Emacs 都用 'isearch' 字体（face）高亮显示"当前匹配"——即与搜索字符串匹配的缓冲区文本（参见"文本字体"）。关于定制这种高亮显示的各种选项，参见"定制搜索以满足需要"。当前的搜索字符串也会显示在回显区。

如果在键入搜索字符串时出错，请键入 <DEL>（'isearch-delete-char'）。每次 <DEL> 都会取消搜索期间最后输入的一个输入项。只要某个命令改变了搜索字符串、光标点的位置、搜索的成功或失败、搜索的方向、当前

搜索结果另一端的位置，或者搜索的“环绕”状态，Emacs 就会记录一个新的“输入项”。关于处理搜索失败，参见“增量搜索中的错误”。

当你对到达的位置满意时，键入 <RET>（‘isearch-exit’）。这会停止搜索，并把光标留在搜索把它带到的地方。此外，任何在搜索中没有特殊含义的命令都会停止搜索，然后被执行。因此，键入 ‘C-a’ 会退出搜索，然后移到行首；键入某个方向键会退出搜索，并执行相应的移动命令；等等。只有当你想键入的下一个命令是打印字符、<DEL>、<RET>，或者在搜索中有特殊含义的其他字符（‘C-q’、‘C-w’、‘C-r’、‘C-s’、‘C-y’、‘M-y’、‘M-r’、‘M-c’、‘M-e’ 以及下文描述的其他一些字符）时，才需要 <RET>。你可以微调哪些命令会退出搜索；参见“不退出增量搜索”。

作为一个特殊的例外，当搜索字符串为空时键入 <RET> 会启动非增量搜索（参见“非增量搜索”）。（这一点可以定制；参见“定制搜索以满足需要”。）

要放弃搜索并回到出发的地方，请键入 ‘<ESC> <ESC> <ESC>’（‘isearch-cancel’）或 ‘C-g C-g’（‘isearch-abort’）。

退出增量搜索时，它会把光标点的原值加入标记环，但不激活标记；因此你可以用 ‘C-u C-<SPC>’ 或 ‘C-x C-x’ 回到开始搜索之前所在的位置。参见“标记环”。（Emacs 只在标记尚未激活时才这样做；如果开始搜索时标记已激活，那么 ‘C-u C-<SPC>’ 和 ‘C-x C-x’ 都会跳到标记处。）

要向后搜索，请用 ‘C-r’（‘isearch-backward’）代替 ‘C-s’ 来启动搜索。向后搜索找到的匹配结束于起始点之前，正如向前搜索找到的匹配开始于起始点之后。

17.1.2 重复增量搜索

假设你向前搜索 ‘FOO’ 并找到了一个匹配，但不是你期望的那个：你想要的 ‘FOO’ 出现在缓冲区中更靠后的位置。这时，再键入一个 ‘C-s’（‘isearch-repeat-forward’）就能移到搜索字符串的下一处出现，或者键入 ‘C-r’（‘isearch-repeat-backward’）移到上一处出现。这些命令可以重复任意多次。另一种做法是给 ‘C-s’ 和 ‘C-r’ 提供数字前缀参数 N，以找到第 N 个下一处或上一处出现。如果越过了目标，可以用 <DEL> 取消若干次 ‘C-s’ 命令。同样，在向后增量搜索中，每个 ‘C-r’（‘isearch-repeat-backward’）都会重复那次向后搜索。

如果在增量搜索期间稍作停顿，Emacs 会高亮显示屏幕上出现的该搜索字符串的所有其他可能匹配。这有助于你预判键入 ‘C-s’ 或 ‘C-r’ 重复搜索能到达哪里。其他匹配用可定制的字体（face）‘lazy-highlight’ 以不同于当前匹配的方式高亮显示（参见“文本字体”）。如果你不喜欢这一特性，可以把 ‘isearch-lazy-highlight’ 设为 ‘nil’ 来禁用它。关于高亮显示匹配的其他定制，参见“定制搜索以满足需要”。

退出搜索之后，只需键入 ‘C-s C-s’ 就能再次搜索同一个字符串。第一个 ‘C-s’ 是调用增量搜索的键，第二个 ‘C-s’ 表示再次搜索上次的搜索字符串。类似地，‘C-r C-r’ 会向后搜索上次的搜索字符串。在确定上次的搜索字符串时，该字符串当初是用 ‘C-s’ 还是 ‘C-r’ 搜索的并无影响。

如果你正在向前搜索，却意识到要找的东西在起始点之前，可以键入 ‘C-r’ 切换到向后搜索，搜索字符串保持不变。类似地，在向后搜索中键入 ‘C-s’ 会切换到向前搜索。

改变搜索方向时，默认情况下你键入的第一个命令仍停留在同一个匹配上，光标会移到该匹配的另一端。若要立即移到另一个匹配，请把变量 ‘isearch-repeat-on-direction-change’ 定制为 ‘t’。

如果搜索失败，而你通过再键入一个 'C-s' 要求重复搜索，它就会从缓冲区开头重新开始。用 'C-r' 重复失败的向后搜索则会从末尾重新开始。这称为"环绕"（wrapping around）；一旦发生，搜索提示中就会出现 'Wrapped'。如果你继续越过搜索最初的起始点，它就会变成 'Overwrapped'，意味着你在重访已经见过的匹配。

你可以通过定制用户选项 'isearch-wrap-pause' 来控制没有更多匹配时发生什么。若它的值为 't'（默认），则会发出错误。（重复搜索会环绕。）若为 'no'，则在到达最后一个匹配后响一声（'ding'）并立即环绕。若为 'no-ding'，则立即环绕，但不响铃。取值为 'no' 和 'no-ding' 时，键入字符时搜索也会尝试环绕。最后，若为 'nil'，则从不环绕，只是停在最后一个匹配处。

若要重用先前的搜索字符串，请使用"搜索环"（search ring）。命令 'M-p'（'isearch-ring-retreat'）和 'M-n'（'isearch-ring-advance'）在环中移动，以挑选一个要重用的搜索字符串。这些命令会把选中的搜索环元素留在迷你缓冲区中，你可以在那里编辑它。键入 'C-s'/'C-r' 或 <RET> 以接受该字符串并开始搜索它。搜索环中保存的最近使用过的搜索字符串数量由变量 'search-ring-max' 指定，默认为 16。

若要编辑当前的搜索字符串，而不是用搜索环中的项把它替换掉，请键入 'M-e'（'isearch-edit-string'）或点击迷你缓冲区中的 'mouse-1'。键入 <RET>、'C-s' 或 'C-r' 完成编辑并搜索该字符串。键入 'C-f' 或 '<RIGHT>' 可以把开始搜索的那个缓冲区中光标点之后的字符加入搜索字符串。

17.1.3 Isearch 的粘贴

很多情况下，你会想用光标点处或附近的文本作为搜索字符串。本小节描述的命令能让你方便地做到这一点。

'C-w'（'isearch-yank-word-or-char'）把光标点处的下一个字符或单词追加到搜索字符串。这是搜索光标点处文本另一处出现的一种便捷方法。（是复制一个字符还是一个单词，按启发式规则决定。）带数字前缀参数 N 时，追加接下来的 N 个字符或单词。

'C-M-w'（'isearch-yank-symbol-or-char'）把光标点处的下一个字符或符号追加到搜索字符串。这是搜索光标点处符号另一处出现的一种便捷方法。（是复制一个字符还是一个符号，按启发式规则决定。）带数字前缀参数 N 时，追加接下来的 N 个字符或符号。

'M-s C-e'（'isearch-yank-line'）把当前行的其余部分追加到搜索字符串。如果光标点已在行尾，则追加下一行。带前缀参数 N 时，追加接下来的 N 行。

类似地，'C-M-z'（'isearch-yank-until-char'）把从光标点到指定字符下一次出现之间的全部内容（不包括该字符）追加到搜索字符串。这对键盘宏特别有用，例如在编程语言或标记语言中，该字符标记一个词法单元的边界。带数字前缀参数 N 时，该命令追加从光标点到指定字符第 N 次出现之间的全部内容。

在增量搜索中，'C-y'（'isearch-yank-kill'）把当前删除的内容追加到搜索字符串。如果在增量搜索中于 'C-y' 之后调用 'M-y'（'isearch-yank-pop'），它会用更早的一次删除替换追加的文本，类似于通常的 'M-y'（'yank-pop'）命令。在回显区点击 'mouse-2' 会把当前的 X 选区（参见"与其他窗口应用剪贴"）追加到搜索字符串（'isearch-yank-x-selection'）。

'C-M-d'（'isearch-del-char'）从搜索字符串中删除最后一个字符，而 'C-M-y'（'isearch-yank-char'）把光标点之后的字符追加到搜索字符串。另一种添加光标点之后字符的方法是：用 'M-e' 进入迷你缓冲区（参见"重复增量搜索"），并在迷你缓冲区中搜索字符串的末尾键入 'C-f' 或 '<RIGHT>'。你每键入一次 'C-f' 或 '<RIGHT>'，就把光标点之后的另一个字符加入搜索字符串。

通常，当搜索不区分大小写时，粘贴进搜索字符串的文本会被转换为小写，以使搜索保持不区分大小写（参见“搜索时的宽松匹配”）。不过，如果变量 ‘search-upper-case’（参见“搜索时的宽松匹配”）的值不是 ‘not-yanks’，就会禁用这种小写转换。

若要以光标点附近的文本作为初始搜索字符串开始一次新的增量搜索，请键入 ‘M-s M-.’，它运行命令 ‘isearch-forward-thing-at-point’。如果区域处于活动状态，它就把区域中的文本粘贴进搜索字符串。否则，它会尝试粘贴光标点附近找到的一个 URL、一个符号或一个表达式。粘贴什么由用户选项 ‘isearch-forward-thing-at-point’ 定义。

17.1.4 增量搜索中的错误

如果完全找不到你的字符串，回显区会显示 ‘Failing I-Search’，光标会移到 Emacs 所能找到的、与你的字符串相匹配的那一部分之后。因此，如果你搜索 ‘FOOT’ 而缓冲区中没有 ‘FOOT’，你可能会看到光标停在 ‘FOOL’ 中的 ‘FOO’ 之后。在回显区中，搜索字符串里未能匹配的部分用字体（face）‘isearch-fail’ 高亮显示。

此时你有几种做法。如果字符串输错了，可以用 <DEL> 取消上一个输入项（参见“增量搜索基础”），用 ‘C-M-d’ 一次擦除一个字符，或用 ‘M-e’ 编辑它。如果你喜欢找到的位置，可以键入 <RET> 留在那里。也可以键入 ‘C-g’，它会把找不到的字符（‘FOOT’ 中的 ‘T’）从搜索字符串中移除，保留已找到的字符（‘FOOT’ 中的 ‘FOO’）。此时再按一次 ‘C-g’ 会完全取消搜索，把光标点返回搜索开始时的位置。

退出命令 ‘C-g’ 在搜索期间会做特殊的事；它究竟做什么取决于搜索的状态。如果搜索已经找到了你指定的内容并在等待输入，‘C-g’ 会取消整个搜索，把光标移回你开始搜索的地方。如果在搜索字符串中还有尚未找到的字符——因为 Emacs 仍在搜索它们，或者因为它没能找到它们——此时键入 ‘C-g’，那么未找到的搜索字符串字符会从搜索字符串中被丢弃。这些字符去掉之后，搜索现在成功了并在等待更多输入，因此第二次 ‘C-g’ 会取消整个搜索。

17.1.5 增量搜索的特殊输入

除前面几个小节描述的字符之外，你在增量搜索期间键入的其他一些字符也有特殊效果。这里对它们加以说明。

要切换宽松空格匹配（参见“搜索时的宽松匹配”），请键入 ‘M-s <SPC>’。

要切换搜索是否区分大小写，请键入 ‘M-c’ 或 ‘M-s c’。参见“搜索时的宽松匹配”。如果搜索字符串中含大写字母，默认情况下搜索区分大小写。

要切换搜索是否把相近且等价的字符视为匹配，请键入 ‘M-s ’’。参见“搜索时的宽松匹配”。如果搜索字符串中含带重音的字符，那么在这次搜索中会禁用字符折叠。

要切换搜索是否会在由覆盖层变得不可见的文本中找到匹配，请键入 ‘M-s i’（‘isearch-toggle-invisible’）。参见“Outline Search”。若要让所有增量搜索都在不可见文本内部找到匹配（无论不可见是由文本属性还是覆盖层属性造成的），请把 ‘search-invisible’ 定制为值 ‘t’。

要在非正则表达式增量搜索与正则表达式增量搜索之间切换，请键入 ‘M-r’ 或 ‘M-s r’（‘isearch-toggle-regexp’）。参见“正则表达式搜索”。

要切换符号模式，请键入 ‘M-s _’。参见“符号搜索”。

要搜索换行符，请把 ‘C-j’ 作为搜索字符串的一部分键入。

要搜索非 ASCII 字符，请在增量搜索期间使用下列方法之一：

- 键入 ‘C-q’（‘isearch-quote-char’），后面跟一个非图形字符或一串八进制数字。这会向搜索字符串添加一个字符，类似于用 ‘C-q’ 向缓冲区插入字符（参见“插入文本”）。例如，在增量搜索中键入 ‘C-q C-s’ 会把 ‘control-S’ 字符加入搜索字符串。

- 使用输入法（参见“输入法”）。如果你开始搜索时当前缓冲区中启用了输入法，那么你键入搜索字符串时，迷你缓冲区中也会激活同一个输入法。在键入搜索字符串的过程中，你可以用 ‘C-\’（‘isearch-toggle-input-method’）切换输入法。你还可以用 ‘C-^’（‘isearch-toggle-specified-input-method’）打开一个非默认输入法，它会提示输入输入法的名称。当增量搜索期间有输入法处于活动状态时，搜索提示中会含有输入法的助记符，如下所示：

```
I-search [IM]:
```

其中 IM 是活动输入法的助记符。你在增量搜索期间启用的任何输入法，之后都会在当前缓冲区中保持启用。最后，你还可以用 ‘C-x \’（‘isearch-transient-input-method’）临时启用一个瞬态输入法（*note transient input method::），以借助输入法向搜索字符串插入单个字符，之后自动禁用该输入法。

- 键入 ‘C-x 8 <RET>’（‘isearch-char-by-name’），后面跟 Unicode 名称或十六进制码点。这会把指定的字符加入搜索字符串，类似于通常的 ‘insert-char’ 命令（参见“插入文本”）。

你也可以在搜索字符串中包含 Emoji 序列。键入 ‘C-x 8 e <RET>’（‘isearch-emoji-by-name’），后面跟某个 Emoji 的 Unicode 名称（例如 ‘smiling face’ 或 ‘heart with arrow’）。这会把指定的 Emoji 加入搜索字符串。如果你不知道要搜索的 Emoji 的名称，可以用 ‘C-x 8 e l’（‘emoji-list’）和 ‘C-x 8 e d’（‘emoji-describe’）（参见“输入法”）。

在增量搜索中键入 ‘M-s o’ 会调用 ‘isearch-occur’，它用当前的搜索字符串运行 ‘occur’。参见“其他搜索与循环命令”。

在增量搜索中键入 ‘M-%’（‘isearch-query-replace’）会调用 ‘query-replace’ 或 ‘query-replace-regexp’（取决于搜索模式），并把当前搜索字符串用作要替换的字符串。负的前缀参数表示向后替换。参见“查询替换”。键入 ‘C-M-%’（‘isearch-query-replace-regexp’）会调用 ‘query-replace-regexp’，并把当前搜索字符串用作要替换的正则表达式。

在增量搜索中键入 ‘M-<TAB>’ 会调用 ‘isearch-complete’，它尝试用搜索环（你以前用过的搜索字符串）作为补全候选列表来补全搜索字符串。参见“补全”。在许多操作系统中，‘M-<TAB>’ 键序列会被窗口管理器截获；这时如果你想使用它，就需要把 ‘isearch-complete’ 重新绑定到另一个键序列（参见“交互式更改键绑定”）。

你可以键入 ‘M-s h r’（‘isearch-highlight-regexp’）退出搜索，同时让匹配保持高亮显示。这会运行 ‘highlight-regexp’（参见“交互式高亮”），把由搜索字符串导出的正则表达式传给它，并提示你选择用于高亮的字体（face）。要高亮含有匹配的*整行*（而不是*仅仅*匹配的文本），请键入 ‘M-s h l’（‘isearch-highlight-lines-matching-regexp’）。无论哪种情况，要移除高亮，请键入 ‘M-s h u’（‘unhighlight-regexp’）。

当增量搜索处于活动状态时，你可以键入 ‘C-h C-h’（‘isearch-help-map’）来访问交互式帮助选项，其中包括特殊键绑定的列表。这些键绑定是键位图 ‘isearch-mode-map’ 的一部分（参见“键位图”）。

当增量搜索处于活动状态时，键入 'M-s M->' 会跳到搜索字符串的最后一处出现，而 'M-s M-<' 会跳到第一处出现。带数字前缀参数 N 时，这些命令分别跳到从缓冲区开头或末尾数起的第 N 处出现。

17.1.6 不退出增量搜索

本小节说明如何控制：键入一个在搜索中没有特殊含义的命令时，是否先退出搜索再执行该命令。它还描述了你可以不退出当前增量搜索就键入的三类命令，尽管这些命令本身并不属于增量搜索。

通常，键入一个未由增量搜索绑定的命令会先退出搜索，再执行该命令。因此，该命令作用于你调用搜索的那个缓冲区。不过，如果你把变量 'search-exit-option' 定制为 'append'，那么你键入的、未被增量搜索解释的字符就会直接追加到搜索字符串。这样你就能把控制字符（例如 'C-a'）包含进搜索字符串，而它们通常会退出搜索并在缓冲区上调用绑定给它们的命令。

前缀参数 在增量搜索中，当你键入指定前缀参数的命令（参见"数字参数"）时，默认情况下它会作用于搜索中的下一个动作，或者作用于退出搜索的那个命令。换句话说，输入前缀参数本身不会终止搜索。

在早先的 Emacs 版本中，输入前缀参数总是终止搜索。你可以把变量 'isearch-allow-prefix' 设为 'nil' 来恢复这种行为。

当 'isearch-allow-scroll' 非 'nil' 时（见下文），前缀参数总是具有上面描述的默认行为，即它们不会终止搜索，即使 'isearch-allow-prefix' 为 'nil'。

滚动命令 通常，滚动命令会退出增量搜索。但如果你把变量 'isearch-allow-scroll' 改为非 'nil' 值，就可以在不退出搜索的情况下使用滚动条，以及像 'C-v'、'M-v' 和 'C-l' 这样具有非 'nil' 'scroll-command' 属性的键盘滚动命令（参见"滚动"）。这只适用于通过它们绑定的键序列来调用这些命令——键入 'M-x' 仍会退出搜索。你可以照常给这些命令加前缀参数。这一特性通常不会让你把当前匹配滚动到看不见的地方；但如果你把 'isearch-allow-scroll' 定制为特殊值 'unlimited'，这一限制就会被解除。

'isearch-allow-scroll' 特性还影响其他一些命令，例如 'C-x 2'（'split-window-below'）和 'C-x ^'（'enlarge-window'），它们并不严格算是滚动，但确实影响文本在屏幕上出现的位置。事实上，它影响任何具有非 'nil' 'isearch-scroll' 属性的命令。因此，你可以通过修改这些属性来控制哪些命令受影响。

例如，若想让 'C-h l' 在今后所有 Emacs 会话中都能在增量搜索内使用，可以先用 'C-h c' 查出它运行哪个命令（*note Key Help::），即 'view-lossage'。然后你可以把下面这一行放进初始化文件（*note Init File::）：

```
(put 'view-lossage 'isearch-scroll t)
```

这一特性可应用于任何不会永久改变光标点、缓冲区内容、匹配数据、当前缓冲区，或者选中的窗口和框架的命令。该命令本身不能试图进行增量搜索。如果 'isearch-allow-scroll' 为 'nil'（默认即如此），这一特性就被禁用。

同样，如果你把变量 'isearch-allow-motion' 改为非 'nil' 值，就可以使用键盘移动命令 'M-<'、'M->'、'C-v' 和 'M-v'，它们分别移到当前搜索字符串在缓冲区中的第一处出现、最后一处出现、当前窗口之后的第一次出现，以及当前窗口之前的最后一次出现。使用这些移动命令时搜索方向不变，除非你把变量 'isearch-motion-changes-direction' 改为非 'nil' 值；在那种情况下，'M-<' 和 'C-v' 之后搜索方向为向前，'M->' 和 'M-v' 之后为向后。

运动命令：当把 'isearch-yank-on-move' 定制为 'shift' 时，可以在键入光标移动命令的同时按住 Shift 键来扩展搜索字符串。它会在当前缓冲区中移动光标点之后，粘贴以新位置结尾的文本。

当 'isearch-yank-on-move' 为 't' 时，无需为光标移动命令使用 Shift 键也能扩展搜索字符串，但它仅适用于符号上带有 'isearch-move' 属性的某些移动命令。

17.1.7 搜索迷你缓冲区

如果在迷你缓冲区活动时启动增量搜索，Emacs 会像搜索其他缓冲区那样搜索迷你缓冲区的内容。与搜索普通缓冲区不同，搜索字符串不显示在回显区，因为回显区用于显示迷你缓冲区。

如果迷你缓冲区中的增量搜索未能找到匹配，它会尝试搜索迷你缓冲区历史（参见"迷你缓冲区历史"）。Emacs 把迷你缓冲区及其历史看作一系列页面，最早的历史元素在第一页，当前迷你缓冲区在最后一页。向前搜索 'C-s' 向后面的页面搜索；反向搜索 'C-r' 向前面的页面搜索。与普通缓冲区搜索一样，失败的搜索可以绕回，从最后一页到第一页，反之亦然。

当迷你缓冲区中的增量搜索找到的匹配是某个历史元素的一部分时，该历史元素会被提取到迷你缓冲区并显示出来。如果正常退出增量搜索（例如键入 <RET>），它此后仍留在迷你缓冲区中。用 'C-g' 取消搜索会把迷你缓冲区的内容恢复为开始搜索时的样子。

17.2 非增量搜索

Emacs 也有传统的非增量搜索命令，它们要求你在搜索开始之前键入完整的搜索字符串。

- 'C-s <RET> STRING <RET>' 搜索 STRING。
- 'C-r <RET> STRING <RET>' 向后搜索 STRING。

要开始非增量搜索，首先键入 'C-s <RET>'。这会进入迷你缓冲区读取搜索字符串；用 <RET> 结束字符串，随后进行搜索。如果找不到该字符串，搜索命令会发出错误信号。

当你键入 'C-s <RET>' 时，'C-s' 照常调用增量搜索。如果指定的字符串为空，该命令会被特意编写为调用非增量搜索的命令。（否则这样的空参数毫无用处。）'C-r <RET>' 同样如此，调用非增量向后搜索命令。

非增量搜索也可以从菜单栏的 'Edit->Search' 菜单调用。

你也可以使用两个更简单的命令：'M-x search-forward' 和 'M-x search-backward'。这些命令查找你指定的字面字符串，除大小写折叠外，不支持任何宽松搜索特性（参见"搜索时的宽松匹配"）。

17.3 单词搜索

"单词搜索"（word search）查找一个单词序列，而不考虑它们之间标点的类型。例如，如果你输入的搜索字符串由两个用单个空格分隔的单词组成，搜索会匹配这两个单词之间用一个或多个空格、换行符或其他标点字符分隔的任意序列。这对搜索文本文档特别有用，因为你不必担心要找的单词之间是换行还是空格。注意，编程语言的主模式或其他专用模式可以修改单词的定义，以满足其语法需要。

- 'M-s w' 若增量搜索处于活动状态，切换单词搜索模式（'isearch-toggle-word'）；否则开始向前增量单词搜索（'isearch-forward-word'）。

- ‘M-s w <RET> WORDS <RET>’ 使用向前非增量单词搜索查找 WORDS。
- ‘M-s w C-r <RET> WORDS <RET>’ 使用非增量单词搜索向后查找 WORDS。
- ‘M-s M-w’ 在 Web 上搜索区域中的文本。

要开始向前增量单词搜索，键入 ‘M-s w’。如果增量搜索尚未活动，这会运行命令 ‘isearch-forward-word’。如果增量搜索已经活动（无论是向前还是向后搜索），‘M-s w’ 会运行命令 ‘isearch-toggle-word’，它在保持搜索方向和当前搜索字符串不变的情况下切换到单词搜索。再次键入 ‘M-s w’ 可以切换回普通搜索。

要开始非增量单词搜索，向前搜索键入 ‘M-s w <RET>’，向后搜索键入 ‘M-s w C-r <RET>’。它们分别运行命令 ‘word-search-forward’ 和 ‘word-search-backward’。

增量单词搜索和非增量单词搜索在寻找匹配的方式上略有不同。在非增量单词搜索中，搜索字符串中的每个单词都必须与一个完整单词精确匹配。在增量单词搜索中，匹配更为宽松：在你键入搜索字符串的过程中，它的第一个和最后一个单词不必匹配完整单词。这是为了让匹配能在你键入时增量进行。这种额外的宽松不适用于延迟高亮（参见“增量搜索”），后者总是匹配完整单词。在你键入搜索字符串期间，搜索提示中会出现 ‘Pending’，直到你使用 ‘C-s’ 这样的搜索重复键。

单词搜索命令不执行字符折叠，切换宽松空白匹配（参见“搜索时的宽松匹配”）对它们没有影响。

要在 Web 上搜索区域中的文本，键入 ‘M-s M-w’。该命令使用由变量 ‘eww-search-prefix’ 指定 URL 的搜索引擎，对区域中的单词进行 Internet 搜索（参见《《eww》手册》的“Basics”）。如果区域未激活，或不含任何单词，该命令会提示用户输入要搜索的 URL 或关键词。

你也可以通过查询词典服务器来查找单词的定义，所用协议是 RFC 2229 定义的 DICT 协议。Emacs 包含该协议的一个客户端。键入 ‘M-x dictionary-search <RET>’ 连接到 DICT 服务器，并要求它提供某个单词的可用定义。该命令提示要查找的单词（默认使用光标点处的单词），然后要求 DICT 服务器在一个或多个词典中提供该单词的定义。默认情况下，该命令先尝试连接本机上安装的 DICT 服务器，如果失败，则在请求确认后尝试 ‘dict.org’；定制变量 ‘dictionary-server’，以字符串形式指定要使用的单个服务器的 URL（如果只想查询本地服务器，就使用 ‘localhost’）。通常，‘dictionary-search’ 告诉服务器在该服务器可用的所有词典中查找该单词，但如果带前缀参数调用该命令，它会提示选择一个词典。服务器可用的词典列表，可以通过按下 ‘*Dictionary*’ 缓冲区中显示的 ‘Select dictionary’ 按钮来显示，详见下文。

首次使用 ‘dictionary-search’ 时，它会创建一个新的 ‘*Dictionary*’ 缓冲区，并在其中开启一个特殊模式。该缓冲区显示用于选择词典、搜索另一个单词的定义等按钮。后续的 ‘dictionary-search’ 命令会复用该缓冲区。要创建另一个这样的缓冲区（例如查找其他单词，也许是在另一个词典中），键入 ‘M-x dictionary <RET>’。

如果在某个缓冲区中开启 ‘dictionary-tooltip-mode’，Emacs 会查找鼠标指针处单词的定义，并在工具提示中显示这些定义。当你阅读含有许多意思不确定的单词的文本时，这很方便。

关于 dictionary-search 的其他选项，参见 ‘dictionary’ 定制组（参见“自定义特定项目”）。

17.4 符号搜索

“符号搜索”（symbol search）与普通搜索很像，只不过搜索的边界必须与符号的边界一致。此处“符号”的含义取决于主模式，通常指源代码中的记号（token），例如 Emacs Lisp 模式中的 Lisp 符号。例如，如果对 Lisp 符

号‘forward-word’执行增量符号搜索，它不会匹配‘isearch-forward-word’。因此，该特性主要用于搜索源代码。

- ‘M-s _’若增量搜索处于活动状态，切换符号搜索模式（‘isearch-toggle-symbol’）；否则开始向前增量符号搜索（‘isearch-forward-symbol’）。
- ‘M-s .’向前开始符号增量搜索，初始搜索字符串中加入在光标点附近找到的符号。
- ‘M-s _ <RET> SYMBOL <RET>’非增量地向前搜索 SYMBOL。
- ‘M-s _ C-r <RET> SYMBOL <RET>’非增量地向后搜索 SYMBOL。

要开始向前增量符号搜索，键入‘M-s ’*（如果要搜索的符号在光标点附近，则键入‘M-s .’）。如果增量搜索尚未活动，‘M-s ’*运行命令‘isearch-forward-symbol’，‘M-s .’运行命令‘isearch-forward-symbol-at-point’。带数值前缀参数 N 时，‘M-s .’会搜索光标点处符号第 N 次后续出现的位置；N 为负值时向后搜索。如果增量搜索已经活动，‘M-s ’*切换到符号搜索，保持搜索方向和当前搜索字符串不变；再次键入‘M-s ’*可以禁用符号搜索。在增量符号搜索中，在你键入搜索字符串期间，只要求搜索字符串的开头匹配符号的开头，并且在你使用‘C-s’这样的搜索重复键之前，搜索提示中会出现‘Pending’。

要开始非增量符号搜索，向前搜索键入‘M-s *<RET>’，向后搜索键入‘M-s* C-r <RET>’。在非增量符号搜索中，搜索字符串的开头和结尾分别要与符号的开头和结尾匹配。

符号搜索命令不执行字符折叠，切换宽松空白匹配（参见“搜索时的宽松匹配”）对它们没有影响。

17.5 正则表达式搜索

“正则表达式”（regular expression，简称“regexp”）是表示一类可供匹配的备选字符串的模式。Emacs 同时提供增量和非增量的方式来搜索正则表达式的匹配。正则表达式的语法在下一节说明。

- ‘C-M-s’开始增量正则表达式搜索（‘isearch-forward-regexp’）。
- ‘C-M-r’开始反向增量正则表达式搜索（‘isearch-backward-regexp’）。

正则表达式的增量搜索可以通过键入‘C-M-s’（‘isearch-forward-regexp’）、带前缀参数调用‘C-s’（参数值无关紧要），或在向前增量搜索中键入‘M-r’来完成。该命令像‘C-s’那样增量地读取搜索字符串，但它把搜索字符串当作正则表达式，而不是在缓冲区文本中寻找精确匹配。每次向搜索字符串添加文本，你都使正则表达式变长，并搜索这个新的正则表达式。要向后搜索正则表达式，使用‘C-M-r’（‘isearch-backward-regexp’）、带前缀参数的‘C-r’，或在向后增量搜索中键入‘M-r’。

普通增量搜索中的所有特殊键序列（参见“增量搜索的特殊输入”）在增量正则表达式搜索中做类似的事情。例如，在开始搜索后立即键入‘C-s’会取回上次使用的增量搜索正则表达式并向前搜索它。增量正则表达式搜索和非正则表达式搜索有各自独立的默认值。它们也有各自的搜索环，可以用‘M-p’和‘M-n’访问。搜索环中保存的搜索正则表达式最大数量由‘regexp-search-ring-max’的值决定，默认为 16。

与普通增量搜索不同，增量正则表达式搜索默认不使用宽松空白匹配。要切换该特性，使用‘M-s <SPC>’（‘isearch-toggle-lax-whitespace’）。此后在增量正则表达式搜索中键入的任何 <SPC> 都将匹配一个或多个空白字符组成的任意序列。变量‘search-whitespace-regexp’指定用于宽松空白匹配的正则表达式（参见“增量搜索的特殊输入”）。

另外，与普通增量搜索不同，增量正则表达式搜索不能使用字符折叠（参见“搜索时的宽松匹配”）。（如果在增量正则表达式搜索期间用 ‘M-s ’’ 切换字符折叠，搜索会变成非正则表达式搜索，你键入的搜索模式被解释为字面字符串。）

在某些情况下，向增量正则表达式搜索中的正则表达式添加字符会使光标后退并重新开始。例如，如果你已经搜索了 ‘foo’，然后添加 ‘\|bar’，光标会后退，以防第一个 ‘bar’ 位于第一个 ‘foo’ 之前。（提示会变为显示“Pending”，以通知用户已发生这种重新计算。）参见“正则表达式的语法”。

向前和向后正则表达式搜索并不对称，因为 Emacs 中的正则表达式匹配总是从正则表达式的开头向前进行。因此，向前正则表达式搜索向前扫描，在每个可能的起始位置尝试向前匹配。向后正则表达式搜索向后扫描，在每个可能的起始位置尝试向前匹配。这些搜索方法并非彼此镜像。

正则表达式的非增量搜索由命令 ‘re-search-forward’ 和 ‘re-search-backward’ 完成。你可以用 ‘M-x’ 调用它们，也可以通过 ‘C-M-s <RET>’ 和 ‘C-M-r <RET>’ 经由增量正则表达式搜索来调用。用 ‘M-x’ 调用这些命令时，它们搜索你指定的确切正则表达式，因此除大小写折叠外不支持任何宽松搜索特性（参见“搜索时的宽松匹配”）。

如果带前缀参数使用增量正则表达式搜索命令，它们会执行普通的字符串搜索，就像 ‘isearch-forward’ 和 ‘isearch-backward’ 那样。参见“增量搜索”。

17.6 正则表达式的语法

本节（以及整本手册）描述用户通常使用的正则表达式特性。关于主要用于 Lisp 程序的其他特性，参见《Emacs Lisp 参考手册》的“Regular Expressions”。

正则表达式有一种语法，其中少数几个字符是“特殊构造”，其余的是“普通字符”。普通字符匹配它本身，不匹配其他任何字符。特殊字符是 ‘$^.*+?[\’。字符 ‘]’ 在结束方括号表达式时是特殊的（见下文）。字符 ‘-’ 在方括号表达式内部是特殊的。正则表达式中出现的任何其他字符都是普通的，除非它前面有 ‘\’。（在 Lisp 程序中使用正则表达式时，每个 ‘\’ 都必须写两次，参见本节末尾附近的示例。）

例如，‘f’ 不是特殊字符，所以它是普通的，因此 ‘f’ 是一个匹配字符串 ‘f’ 而不匹配其他字符串的正则表达式。（它不匹配字符串 ‘ff’。）同样，‘o’ 是只匹配 ‘o’ 的正则表达式。（当忽略大小写时，这些正则表达式也匹配 ‘F’ 和 ‘O’，但我们把这看作“同一字符串”的推广，而不是例外。）

任意两个正则表达式 A 和 B 可以连接。结果是一个正则表达式：若 A 匹配某个字符串开头的一部分，且 B 匹配该字符串的其余部分，它就匹配该字符串。举个简单的例子，连接正则表达式 ‘f’ 和 ‘o’ 得到正则表达式 ‘fo’，它只匹配字符串 ‘fo’。要做不那么简单的事，你需要使用某个特殊字符。下面是它们的列表。

‘.’（句点）是匹配除换行符以外的任意单个字符的特殊字符。例如，正则表达式 ‘a.b’ 匹配以 ‘a’ 开头、以 ‘b’ 结尾的任意三字符字符串。

‘*’ *本身不是构造；它是一个后缀运算符，表示重复匹配前一个正则表达式任意多次，并尽可能多。因此，‘o*’* 匹配任意多个 ‘o’，包括零个 ‘o’。

‘*’ *总是作用于尽可能小的前一个表达式。因此，‘fo*’* 中重复的是 ‘o’，而不是 ‘fo’。它匹配 ‘f’、‘fo’、‘foo’ 等等。

匹配器处理 ‘*’ *构造时，会立即匹配尽可能多的重复次数。然后它继续处理模式的其余部分。如果失败，就会发生回溯，丢弃 ‘*’* 修饰的构造的一些匹配，以防这样能使模式的其余部分匹配成功。例如，在用 ‘ca*ar’ 匹配字

符串 ‘caaar’ 时，‘a*’ 首先尝试匹配全部三个 ‘a’；但模式的其余部分是 ‘ar’，只剩下 ‘r’ 可以匹配，所以这次尝试失败。下一个选择是让 ‘a*’ 只匹配两个 ‘a’。作此选择时，正则表达式的其余部分匹配成功。

‘+’ 是后缀运算符，与 ‘*’ 类似，但它必须至少匹配前一个表达式一次。因此，‘ca+r’ 匹配字符串 ‘car’ 和 ‘caaaar’，但不匹配字符串 ‘cr’，而 ‘ca*r’ 匹配这三个字符串。

‘?’ 是后缀运算符，与 ‘*’ 类似，但它可以匹配前一个表达式一次或不匹配。因此，‘ca?r’ 匹配 ‘car’ 或 ‘cr’，不匹配其他内容。

‘?’、‘+?’、‘??’ 是上述运算符的非“贪婪”变体。普通运算符 ‘’、‘+’、‘?’* 在整体正则表达式仍能匹配的前提下尽可能多地匹配。后面跟一个 ‘?’ 时，它们会尽可能少地匹配。

因此，‘ab*’ 和 ‘ab*?’ 都能匹配字符串 ‘a’ 和字符串 ‘abbbb’；但如果用它们匹配文本 ‘abbb’，‘ab*’ 会匹配整个文本（最长的有效匹配），而 ‘ab*?’ 只匹配 ‘a’（最短的有效匹配）。

非贪婪运算符从给定起始点开始匹配最短可能的字符串；不过在向前搜索中，总是选择最早可能的匹配起始点。因此，如果用 ‘a.*?$’ 搜索后面跟着换行符的文本 ‘abbab’，它会匹配整个字符串。因为它可以从第一个 ‘a’ 开始匹配，所以就那样做了。

‘[…]’ 是“方括号表达式”（也称“备选字符集”），它匹配一组字符中的一个。

最简单的情况下，两个方括号之间的字符就是这个集合能匹配的内容。因此，‘[ad]’ 匹配一个 ‘a’ 或一个 ‘d’，而 ‘[ad]*’ 匹配仅由 ‘a’ 和 ‘d’ 组成的任意字符串（包括空字符串）。由此，‘c[ad]*r’ 匹配 ‘cr’、‘car’、‘cdr’、‘caddaar’ 等。

你也可以在字符集中包含字符范围，方法是在起始和结束字符之间写一个 ‘-’。因此，‘[a-z]’ 匹配任意小写 ASCII 字母。范围可以与单个字符自由混用，如 ‘[a-z$%.]’，它匹配任意小写 ASCII 字母或 ‘$’、‘%’ 或句点。另一个例子，‘[α-ωι]’ 匹配所有小写希腊字母。

你也可以在字符集中包含某些特殊的“字符类”。‘[:’ 和与之配对的 ‘:]’ 在方括号表达式内括出一个字符类。例如，‘[[:alnum:]]’ 匹配任意字母或数字。字符类的列表参见《Emacs Lisp 参考手册》的“Char Classes”。

要在字符集中包含 ‘]’，必须把它放在第一个字符的位置。例如，‘[]a]’ 匹配 ‘]’ 或 ‘a’。要包含 ‘-’，把它写成集合的最后一个字符，不过你也可以把它放在最前面或某个范围之后。因此，‘[]-]’ 同时匹配 ‘]’ 和 ‘-’。

要在集合中包含 ‘^’，把它放在除集合开头之外的任何位置。（放在开头时，它对集合取补——见下文。）

在大小写不敏感的搜索中使用范围时，应把范围的两端都写成大写，或都写成小写，或两端都写成非字母字符。像 ‘A-z’ 这样的混合大小写范围的行为相当不明确，并且可能在未来的 Emacs 版本中改变。

要用正则表达式搜索原始字节（参见“国际字符集简介”），在方括号表达式中指定它们的码点，如 ‘C-M-s [C-x 8 <RET> 3fff80 <RET> - C-x 8 <RET> 3fffff <RET>]’。Emacs 会在回显区显示等价的 `Regexp I-search: [\200-\377]`。

‘[^ …]’ ‘[^’ 开始一个“补集字符集”，它匹配除指定字符之外的任何字符。因此，‘[^a-z0-9A-Z]’ 匹配 ASCII 字母和数字除外的所有字符。

除非 ‘^’ 是第一个字符，否则它在字符集中不是特殊的。‘^’ 后面的字符被当作第一个字符处理（换句话说，在那里 ‘-’ 和 ‘]’ 不是特殊的）。

补集字符集可以匹配换行符，除非换行符被列为不匹配的字符之一。这与‘grep’等程序中对正则表达式的处理相反。

‘^’是匹配空字符串的特殊字符，但只在被匹配文本中行的开头。在其他位置它匹配不了任何东西。因此，‘^foo’匹配出现在行首的‘foo’。

出于历史兼容性原因，‘^’只有用在正则表达式的开头，或用在‘\(’或‘\|’之后时，才具有这种含义。

‘$’与‘^’类似，但只在行尾匹配。因此，‘x+$’匹配行尾由一个或更多‘x’组成的字符串。

出于历史兼容性原因，‘$’只有用在正则表达式的末尾，或用在‘\)’或‘\|’之前时，才具有这种含义。

‘\’有两个功能：它引用特殊字符（包括‘\’），并引入额外的特殊构造。

因为‘\’引用特殊字符，‘\$’是只匹配‘$’的正则表达式，‘\[’是只匹配‘[’的正则表达式，等等。

以‘\’开头的特殊构造见下一节。

注意：为了历史兼容性，如果特殊字符出现在其特殊含义没有意义的上下文中，它们会被当作普通字符。例如，*‘foo’把‘*’当作普通字符，因为前面没有可供‘*’作用的表达式。依赖这种行为是不好的做法；无论如何，最好把特殊字符引用起来，不管它出现在哪里。

由于‘\’在方括号表达式内部不是特殊的，它永远无法去除‘-’、‘^’或‘]’的特殊含义。当这些字符没有特殊含义时，不应引用它们。这不会澄清任何事情，因为在它们有特殊含义的地方，反斜杠可以正当地放在它们前面，如‘[^\]’（Lisp 字符串语法写作‘"[^\\]"’），它匹配除反斜杠外的任意单个字符。

17.7 正则表达式中的反斜杠

大多数情况下，‘\’后跟任何字符都只匹配该字符。不过有几个例外：以‘\’开头的双字符序列具有特殊含义。序列中的第二个字符单独使用时总是普通字符。下面是‘\’构造的列表。

‘\|’指定一个二选一。两个正则表达式 A 和 B 中间加‘\|’组成一个表达式：若 A 匹配某段文本或 B 匹配它，该表达式就匹配这段文本。它的工作方式是先尝试匹配 A，如果失败，再尝试匹配 B。

因此，‘foo\|bar’匹配‘foo’或‘bar’，不匹配其他字符串。

‘\|’作用于尽可能大的外围表达式。只有外围的‘\(…\)’分组才能限制‘\|’的分组作用范围。

为处理‘\|’的多处使用，系统具备完整的回溯能力。

‘\(…\)’是一种分组构造，有三个用途：

1. 为其他操作括起一组‘\|’备选项。因此，‘\(foo\|bar\)x’匹配‘foox’或‘barx’。

1. 括起一个复杂表达式，供后缀操作符‘*’、‘+’和‘?’作用。因此，‘ba\(na\)*’匹配‘bananana’等，即含有任意（零个或多个）个‘na’字符串。

1. 记录匹配到的子串，供以后引用。

最后一个用途并非括号分组这一想法的必然结果；它是另一项功能，被赋为同一‘\(…\)’构造的第二重含义。实践中这两种含义通常不会冲突；一旦冲突，可以使用下文介绍的隐藏分组。

‘\(?: ... \)’ 指定一个“隐藏分组”（shy group），它不记录匹配的子串；你不能用 ‘\D’ 反向引用它（见下文）。这在以机械方式组合正则表达式时很有用，因为你可以为语法上的目的添加分组，而不干扰那些需要被引用的分组的编号。

‘\D’ 匹配与第 D 个 ‘\(... \)’ 构造所匹配文本相同的文本。这称为“反向引用”（back reference）。

在 ‘\(... \)’ 构造结束之后，匹配器会记住该构造所匹配文本的起点和终点。随后，在正则表达式中更靠后的位置，你可以用 ‘\’ 后跟数字 D 来表示“匹配与第 D 个 ‘\(... \)’ 构造相同的文本”。

正则表达式中前九个 ‘\(... \)’ 构造所匹配的字符串，按左括号在正则表达式中出现的顺序被编号为 1 到 9。因此你可以用 ‘\1’ 到 ‘\9’ 引用相应 ‘\(... \)’ 构造所匹配的文本。

例如，‘\(.\)\1’ *匹配任何由两个相同部分组成的、不含换行的字符串。*‘\(.\)’ 匹配前半部分，它可以是任何内容，但紧随其后的 ‘\1’ 必须匹配完全相同的文本。

若某个 ‘\(... \)’ 构造匹配了不止一次（当它后面跟着 ‘*’ 时很容易发生），只记录最后一次匹配。

‘\{M\}’ 是指定 M 次重复的后缀操作符——即前面的正则表达式必须连续精确匹配 M 次。例如，‘x\{4\}’ 匹配字符串 ‘xxxx’，不匹配其他内容。

‘\{M,N\}’ 是指定 M 到 N 次重复的后缀操作符——即前面的正则表达式必须至少匹配 M 次，但不超过 N 次。若省略 N，则没有上限，但前面的正则表达式必须至少匹配 M 次。‘\{0,1\}’ 等价于 ‘?’。‘\{0,\}’ 等价于 ‘*’。‘\{1,\}’ 等价于 ‘+’。

‘\`’ 匹配空字符串，但只在被匹配的字符串或缓冲区（或其可访问部分）的开头。

‘\'’ 匹配空字符串，但只在被匹配的字符串或缓冲区（或其可访问部分）的末尾。

‘\=’ 匹配空字符串，但只在光标点处。

‘\b’ 匹配空字符串，但只在单词的开头或结尾。因此，‘\bfoo\b’ 匹配作为独立单词出现的任何 ‘foo’。‘\bballs?\b’ 匹配作为独立单词的 ‘ball’ 或 ‘balls’。

‘\b’ 无论旁边出现什么文本，都在缓冲区的开头或结尾处匹配。

‘\B’ 匹配空字符串，但*不*在单词的开头或结尾处匹配。

‘\<’ 匹配空字符串，但只在单词的开头。仅当后面跟着构成单词的字符时，‘\<’ 才在缓冲区开头处匹配。

‘\>’ 匹配空字符串，但只在单词的结尾。仅当内容以构成单词的字符结尾时，‘\>’ 才在缓冲区末尾处匹配。

‘\w’ 匹配任何构成单词的字符。哪些字符属于此类由语法表决定。参见《Emacs Lisp 参考手册》的“Syntax Tables”

‘\W’ 匹配任何不构成单词的字符。

‘_<’ *匹配空字符串，但只在符号的开头。符号是一个或多个构成符号的字符组成的序列。构成符号的字符是语法为 ‘w’ 或 ‘_’* 的字符。仅当后面跟着构成符号的字符时，‘_<’ 才在缓冲区开头处匹配。与单词一样，哪些字符构成符号由语法表决定。

‘_>’ *匹配空字符串，但只在符号的结尾。仅当内容以构成符号的字符结尾时，*‘\>’ 才在缓冲区末尾处匹配。

‘\sC’ 匹配语法为 C 的任何字符。这里的 C 是表示某个语法类的字符：例如 ‘w’ 表示构成单词的字符，‘-’ 或 ‘ ’ 表示空白，‘.’ 表示普通标点，等等。参见《Emacs Lisp 参考手册》的“Syntax Class Table”

‘\SC’ 匹配语法不是 C 的任何字符。

‘\cC’ 匹配属于类别 C 的任何字符。例如，‘\cc’ 匹配汉字，‘\cg’ 匹配希腊字符，等等。要查看已知类别的说明，输入 ‘M-x describe-categories <RET>’。

‘\CC’ 匹配*不*属于类别 C 的任何字符。

与单词和语法有关的这些构造由语法表的设置控制。参见《Emacs Lisp 参考手册》的“Syntax Tables”

17.8 正则表达式示例

下面是一个正则表达式的例子——它类似于 Emacs 默认用来识别句子结尾（不含其后的空格）的正则表达式，即变量 ‘sentence-end-base’：

```
[.?!][]\"')}]*
```

它由前后相接的两部分组成：一个匹配句点、‘?’ 或 ‘!’ 的字符集，以及一个匹配右括号、引号或圆括号、并重复零次或多次的字符集。

17.9 搜索时的宽松匹配

通常，你会希望搜索命令忽略所输入的搜索字符串与被搜索文本之间的某些细微差异。例如，长度不同的空白字符序列通常被视为等价；字母大小写差异通常无所谓；等等。这称为“字符等价”（character equivalence）。

本节介绍 Emacs 的宽松搜索功能，以及如何按需定制它们。

默认情况下，搜索命令执行“宽松空格匹配”（lax space matching）：每个空格或空格序列匹配文本中一个或多个空白字符的任意序列。更准确地说，Emacs 会把搜索字符串中的每个空格序列与用户选项 ‘search-whitespace-regexp’ 所指定的正则表达式相匹配。该选项的默认值把任意空格和制表符序列视为空白。因此，‘foo bar’ 匹配 ‘foo bar’、‘foo bar’、‘foo bar’ 等等（但不匹配 ‘foobar’）。若想让空格也能匹配由换行符组成的序列而不仅是空格和制表符，可定制该选项，把它的值设为正则表达式 ‘[\t\n]+’。（增量正则表达式搜索的默认行为不同；参见“正则表达式搜索”。）

若希望空白字符精确匹配，可以在增量搜索中输入 ‘M-s <SPC>’（‘isearch-toggle-lax-whitespace’）关闭宽松空格匹配。再输入一次 ‘M-s <SPC>’ 会重新开启。要对所有搜索禁用宽松空白匹配，把 ‘search-whitespace-regexp’ 改为 ‘nil’；这样搜索字符串中的每个空格都精确匹配一个空格。

Emacs 中的搜索默认忽略所搜索文本的大小写——前提是你以小写形式给出搜索字符串。因此，如果你指定搜索 ‘foo’，那么 ‘Foo’ 和 ‘fOO’ 也会匹配。正则表达式，尤其是字符集，行为类似：‘[ab]’ 匹配 ‘a’ 或 ‘A’ 或 ‘b’ 或 ‘B’。此功能称为“大小写折叠”（case folding），增量搜索和非增量搜索模式都支持它。

搜索字符串中任意位置出现大写字母都会使搜索区分大小写。因此，搜索 ‘Foo’ 不会找到 ‘foo’ 或 ‘FOO’。这既适用于正则表达式搜索，也适用于字面字符串搜索。如果你从搜索字符串中删去该大写字母，这一效果就会消失。变量 ‘search-upper-case’ 控制此行为：若其值非 ‘nil’，搜索字符串中的大写字符会使搜索区分大小写；把它设为 ‘nil’ 会禁用大写字符的这一效果。该变量的默认值是 ‘not-yanks’，它使搜索在搜索字符串含大写字母时

区分大小写，同时把粘贴进搜索字符串的文本（参见“Isearch 的粘贴”）转为小写，从而这类搜索默认不区分大小写。

若把变量 ‘case-fold-search’ 设为 ‘nil’，则所有字母都必须精确匹配，包括大小写。这是一个缓冲区局部变量；通常修改它只影响当前缓冲区，除非你修改它的默认值。参见“局部变量”。该变量也适用于非增量搜索，包括替换命令（参见“替换命令”）和迷你缓冲区历史匹配命令（参见“迷你缓冲区历史”）所执行的搜索。

在增量搜索中输入 ‘M-c’ 或 ‘M-s c’（‘isearch-toggle-case-fold’）可切换该次搜索的大小写敏感性。此效果不会超出当前这次增量搜索，但它会覆盖在当前搜索中添加或删除大写字母所产生的效果。

若干相关变量控制特定命令或活动在搜索和匹配时的大小写敏感性。例如，‘tags-case-fold-search’ 控制 ‘find-tag’ 的大小写敏感性。要找到这些变量，可执行 ‘M-x apropos-variable <RET> case-fold-search <RET>’。

大小写折叠忽略字符之间的大小写区别，使大写字符匹配小写变体，反之亦然。大小写折叠的推广是“字符折叠”（character folding），它忽略相似字符之间更广泛的类别差异。例如，在字符折叠下，字母 ‘a’ 匹配它所有带变音符号的同类，如 ‘ä’ 和 ‘á’，也就是说，这种匹配忽略区分这些变体的变音符号。此外，‘a’ 还匹配其他与它相似、或图形表示中包含它的字符，如 U+00AA FEMININE ORDINAL INDICATOR 和 U+24D0 CIRCLED LATIN SMALL LETTER A（看起来像圆圈里的一个小 ‘a’）。类似地，ASCII 双引号字符 ‘"’ 匹配 Unicode 标准定义的所有其他双引号变体。最后，字符折叠还能使一个或多个字符的序列匹配另一个长度不同的序列：例如，两个字符的序列 ‘ff’ 匹配 U+FB00 LATIN SMALL LIGATURE FF，序列 ‘(a)’ 匹配 U+249C PARENTHESIZED LATIN SMALL LETTER A。彼此并不相同但在字符折叠下匹配的字符序列称为“等价字符序列”（equivalent character sequences）。

一般来说，Emacs 中的搜索命令默认不执行字符折叠来匹配等价字符序列。你可以通过把变量 ‘search-default-mode’ 定制为 ‘char-fold-to-regexp’ 来启用此行为。参见“定制搜索以满足需要”。在增量搜索中，输入 ‘M-s '’（‘isearch-toggle-char-fold’）可切换字符折叠，但仅对该次搜索有效。（替换命令有不同的默认值，由单独的选项控制；参见“替换命令与宽松匹配”。）

默认情况下，在搜索字符串中输入字符的显式变体（如 ‘ä’）并不会匹配其基本字符（如 ‘a’）。但若把变量 ‘char-fold-symmetric’ 定制为 ‘t’，搜索命令就会同等对待等价字符，搜索字符串中一组等价字符里的任意一个都会在被搜索文本中找到其中任意一个，因此输入带变音符号的字符 ‘ä’ 会匹配字母 ‘a’ 以及 ‘á’ 等所有其他变体。

你可以用可定制变量 ‘char-fold-include’ 添加新的折叠，或用可定制变量 ‘char-fold-exclude’ 删除已有的折叠。你还可以把 ‘char-fold-override’ 定制为 ‘t’，以禁用所有字符等价关系，只保留你用 ‘char-fold-include’ 自己添加的那些。

17.10 替换命令

Emacs 提供若干命令来执行查找并替换的操作。除了简单的 ‘M-x replace-string’ 命令外，还有 ‘M-%’（‘query-replace’），它逐个呈现搜索模式的每次出现，并询问你是否替换。

替换命令通常作用于从光标点到缓冲区末尾的文本。当区域处于活动状态时，它们改为作用于该区域（参见“标记与区域”）。基本的替换命令用一个“替换字符串”替换一个“搜索字符串”（或正则表达式）。也可以用命令 ‘expand-region-abbrevs’ 并行执行多处替换（参见“控制缩写展开”）。

17.10.1 无条件替换

‘M-x replace-string <RET> STRING <RET> NEWSTRING <RET>’ 把 STRING 的每次出现都替换为 NEWSTRING。

要把光标点之后每一处 ‘foo’ 都替换为 ‘bar’，使用命令 ‘M-x replace-string’，两个参数分别为 ‘foo’ 和 ‘bar’。替换只发生在光标点之后的文本中，所以若想覆盖整个缓冲区，必须先移动到开头。光标点到缓冲区末尾之间的所有出现都会被替换；若要把替换限制在缓冲区的一部分，就在该部分周围激活区域。当区域处于活动状态时，替换被限制在该区域内（参见“标记与区域”）。

当 ‘replace-string’ 退出时，它把光标点留在最后被替换的那处出现上。它把光标点原先的位置（即发出 ‘replace-string’ 命令的位置）加入标记环，但不激活标记；用 ‘C-u C-<SPC>’ 可以回到那里。参见“标记环”。

前缀参数把替换限制在被单词边界包围的匹配上。

关于替换命令中大小写敏感性和字符折叠的细节，参见“替换命令与宽松匹配”。

17.10.2 正则表达式替换

‘M-x replace-string’ 命令替换单个字符串的精确匹配。类似的命令 ‘M-x replace-regexp’ 替换指定正则表达式模式的任何匹配（参见“正则表达式的语法”）。

‘M-x replace-regexp <RET> REGEXP <RET> NEWSTRING <RET>’ 把 REGEXP 的每个匹配都替换为 NEWSTRING。

在 ‘replace-regexp’ 中，NEWSTRING 不必是常量：它可以引用 REGEXP 所匹配内容的全部或一部分。NEWSTRING 中的 ‘\&’ 表示被替换的整个匹配。NEWSTRING 中的 ‘\D’（其中 D 是从 1 开始的数字）表示 REGEXP 中第 D 个括号分组所匹配的内容。（这称为“反向引用”。）‘\#’ 表示此命令中已完成替换的次数，用十进制数表示。在第一次替换时，‘\#’ 表示 ‘0’；第二次表示 ‘1’；依此类推。例如，

```
M-x replace-regexp <RET> c[ad]+r <RET> \&-safe <RET>
```

会把（例如）‘cadr’ 替换为 ‘cadr-safe’，把 ‘cddr’ 替换为 ‘cddr-safe’。

```
M-x replace-regexp <RET> \(c[ad]+r\)-safe <RET> \1 <RET>
```

执行相反的变换。若要在替换成的文本中包含 ‘\’，必须输入 ‘\\’。

若希望每次手工输入替换字符串的一部分，可在替换字符串中使用 ‘\?’。每次替换都会在迷你缓冲区中要求你编辑替换字符串，并把光标点放在 ‘\?’ 所在的位置。

本小节剩余内容面向专门任务，需要 Lisp 知识。大多数读者可以跳过。

你可以用 Lisp 表达式计算替换字符串的各个部分。为此，在替换字符串中写 ‘\,’，后跟该表达式。每次替换都会计算该表达式的值，并将其转换为文本而不加引号（若它是字符串，就意味着使用该字符串的内容），然后用它代替表达式本身放入替换字符串。若表达式是符号，则符号名之后替换字符串中的一个空格随符号名一起被替换，因此该值替换两者。

在此类表达式内部，你可以使用一些特殊序列。‘\&’ 和 ‘\D’ 在这里照旧分别指整个匹配（作为字符串）和某个子匹配（作为字符串）。D 可以是多位数字；若第 D 个括号分组没有匹配，则 ‘\D’ 的值为 ‘nil’。你还可以

用 '\#&' 和 '\#D' 以数字形式引用这些匹配（当匹配或子匹配具有数字形式时这有效）。这里的 '\#' 同样表示已完成替换的次数。

例如，我们可以这样交换 'x' 和 'y':

```
M-x replace-regexp <RET> \(x\)\|y <RET>
\,(if \1 "y" "x") <RET>
```

为 '\,' 计算替换字符串时，'format' 函数常常很有用（参见《Emacs Lisp 参考手册》的"Formatting Strings"）。例如，要在第 73 到 80 列添加像 'ABC00042' 这样连续编号的字符串（除非这些列已被占用），可以使用

```
M-x replace-regexp <RET> ^.\{0,72\}$ <RET>
\,(format "%-72sABC%05d" \& \#) <RET>
```

17.10.3 替换命令与宽松匹配

本小节介绍替换命令在宽松匹配（参见"搜索时的宽松匹配"）方面的行为，以及如何定制它。一般来说，替换命令的默认匹配大多比对应的搜索命令更严格。

与增量搜索不同，替换命令默认不使用宽松空格匹配（参见"搜索时的宽松匹配"）。要为替换启用宽松空格匹配，把变量 'replace-lax-whitespace' 改为非 'nil'。（这仅影响 Emacs 如何查找要替换的文本，不影响替换文本。）

与之配套的变量 'replace-regexp-lax-whitespace' 控制 'query-replace-regexp' 在搜索模式时是否使用宽松空白匹配。

若替换命令的第一个参数全为小写，则命令在搜索要替换的出现时忽略大小写——前提是 'case-fold-search' 非 'nil' 且 'search-upper-case' 也非 'nil'。若 'search-upper-case'（参见"搜索时的宽松匹配"）为 'nil'，则搜索是否忽略大小写只由 'case-fold-search' 决定，与命令第一个参数的大小写无关。若 'case-fold-search' 被设为 'nil'，则所有搜索中大小写始终有意义。

此外，当替换命令的第二个参数全为小写或部分为小写时，替换命令会尽量保留每处出现的大小写模式。因此，命令

```
M-x replace-string <RET> foo <RET> bar <RET>
```

会把小写的 'foo' 替换为小写的 'bar'，把全大写的 'FOO' 替换为 'BAR'，把首字母大写的 'Foo' 替换为 'Bar'。（这三种形式——小写、全大写、首字母大写——是 'replace-string' 所能区分的全部形式。）注意，Emacs 通过分析被替换文本中的每个单词来决定是把替换文本转为大写还是首字母大写，并且仅当被替换文本的*所有*单词都使用相同的大小写形式时，才保留其大小写。因此，命令

```
M-x replace-string <RET> foo bar <RET> baz quux <RET>
```

把 'Foo Bar' 替换为 'Baz Quux'，因为 'Foo Bar' 中的两个单词都首字母大写。相反，同样的命令把 'Foo bar' 替换为 'baz quux'，也就是说，由于 'Foo bar' 中两个单词的大小写不同，它保持替换文本的大小写不变。究竟什么算一个"单词"取决于当前缓冲区中生效的语法表（参见《Emacs Lisp 参考手册》的"Syntax Tables"）；因此，'Foo_Bar' 在 Text 模式下是两个单词，但在某些支持编程语言的主模式下可能是一个单词。

若替换字符串中出现了大写字母，则每次插入该文本时它们都保持大写。若第一个参数中出现了大写字母，则第二个参数总是按给定内容原样替换，不做大小写转换。同样，若 'case-replace' 或 'case-fold-search' 被设为 'nil'，替换就不做大小写转换。

替换命令默认在查找要替换的文本时不使用字符折叠（参见"搜索时的宽松匹配"）。要为 'query-replace' 和 'replace-string' 的匹配启用字符折叠，把变量 'replace-char-fold' 设为非 'nil' 值。（此设置不影响替换文本，只影响 Emacs 如何查找要替换的文本。它也不影响 'replace-regexp'。）

17.10.4 查询替换

'M-% STRING <RET> NEWSTRING <RET>' 替换 STRING 的部分出现。'C-M-% REGEXP <RET> NEWSTRING <RET>' 替换 REGEXP 的部分匹配。

若只想把 'foo' 的一部分出现改为 'bar'，而不是全部，使用 'M-%'（'query-replace'）。该命令逐个查找 'foo' 的出现，显示每处出现并询问你是否替换。除询问之外，'query-replace' 的工作方式与 'replace-string' 相同（参见"无条件替换"）。特别地，只要 'case-replace' 非 'nil'（通常如此），它就保留大小写（参见"替换命令与宽松匹配"）。数字参数表示只考虑被单词分隔字符包围的出现。负前缀参数表示向后替换。

'C-M-%' 执行正则表达式搜索与替换（'query-replace-regexp'）。它的工作方式与 'replace-regexp' 相同，只是像 'query-replace' 那样进行询问。

用这些命令可以重用先前的替换。当 'query-replace' 或 'query-replace-regexp' 提示输入搜索字符串时，用 'M-p' 和 'M-n' 以 'FROM -> TO' 的形式显示先前的替换，其中 FROM 是搜索模式，TO 是它的替换文本，两者之间的分隔符由变量 'query-replace-from-to-separator' 的值决定。输入 <RET> 选择想要的替换。若该变量的值为 'nil'，替换不会被加入命令历史，也就无法重用。

这些命令用字体（face）'query-replace' 高亮当前匹配。你可以把变量 'query-replace-highlight' 设为 'nil' 来禁用这一高亮。它们像增量搜索那样用 'lazy-highlight' 高亮其他匹配（参见"增量搜索"）；把 'query-replace-lazy-highlight' 设为 'nil' 可以禁用这一高亮。默认情况下，'query-replace-regexp' 会在迷你缓冲区中显示当前匹配替换后的字符串。若想保持特殊序列 '\&' 和 '\N' 不被展开，可定制变量 'query-replace-show-replacement'。正如 'search-highlight-submatches' 高亮增量搜索中的子表达式（参见"定制搜索以满足需要"），变量 'query-replace-highlight-submatches' 决定是否在正则表达式替换命令中高亮子表达式。

变量 'query-replace-skip-read-only' 若设为非 'nil'，会让替换命令忽略只读文本中的匹配。默认不忽略它们。

在向你显示字符串或正则表达式的匹配时，你可以输入的字符有：

'<SPC>' 'y' 用 NEWSTRING 替换本次出现。

'<DEL>' '<Delete>' '<BACKSPACE>' 'n' 跳到下一次出现而不替换本次出现。

', (Comma)' 替换本次出现并显示结果。随后会再要求你输入一个字符，说明接下来做什么。由于替换已经完成，此情形下 <DEL> 和 <SPC> 等价；两者都移到下一次出现。

此时你可以输入 'C-r'（见下文）来修改被替换的文本。你也可以用 'undo' 命令撤销替换（例如输入 'C-x u'；*note Undo::）；这会退出 'query-replace'，所以若要继续替换，必须用 'C-x <ESC> <ESC> <RET>' 重新开始（*note Repetition::）。

'<RET>' 'q' 退出，不再做任何替换。

‘. (Period)’ 替换本次出现，然后退出，不再搜索更多出现。

‘!’ 替换所有剩余的出现，不再询问。

‘^’ 回到上一次出现（或曾经是出现）的位置，以防你误改了它或想重新检查它。

‘u’ 撤销上一次替换，并回到做出该替换的位置。

‘U’ 撤销全部替换，并回到第一次替换发生的位置。

‘d’ 显示把全部出现都改成其替换文本的 diff 缓冲区。

‘C-r’ 进入递归编辑层，以防该出现需要编辑，而不只是用 NEWSTRING 替换。完成后，用 ‘C-M-c’ 退出递归编辑层，继续处理下一次出现。参见“递归编辑层级”。

‘C-w’ 删除该出现，然后像 ‘C-r’ 那样进入递归编辑层。用递归编辑插入文本来替换被删掉的 STRING 出现。完成后，用 ‘C-M-c’ 退出递归编辑层，继续处理下一次出现。

‘e’ 在迷你缓冲区中编辑替换字符串。当你输入 <RET> 退出迷你缓冲区时，迷你缓冲区的内容会替换模式的当前出现。它也会成为后续所有出现的新替换字符串。

‘E’ 与 ‘e’ 类似，但下一次替换会严格按大小写进行。也就是说，如果你用 ‘query-replace’ 把 ‘foo’ 换成 ‘bar’，那么像 ‘Foo’ 这样的文本通常会被替换成 ‘Bar’。用此命令可对当前替换严格按大小写进行。

‘d’ 把当前匹配的替换以缓冲区形式显示出来，其中是原文与替换匹配后文本之间的差异。

‘C-l’ 重绘屏幕。之后你必须再输入一个字符，指明如何处理该出现。

‘Y (Upper-case)’ 在多缓冲区替换（例如对选中文件执行查询替换的 Dired ‘Q’ 命令）中，替换所有剩余缓冲区中的所有剩余出现。它对本次询问以及后续所有询问都回答“是”，无需再与用户交互。

‘N (Upper-case)’ 在多缓冲区替换中跳到下一个缓冲区，不替换当前缓冲区中的剩余出现。它对本次询问回答“否”，放弃当前缓冲区的后续询问，并继续序列中的下一个缓冲区。

‘C-h’ ‘?’ ‘<F1>’ 显示一条概述这些选项的消息。之后你必须再输入一个字符，指明如何处理该出现。

除此之外，任何其他字符都会退出 ‘query-replace’，然后被重新读取为键序列的一部分。因此，如果你输入 ‘C-k’，它会退出 ‘query-replace’，然后删除到行尾。特别是，‘C-g’ 只是退出 ‘query-replace’。

一旦退出 ‘query-replace’，要重新启动它，可用 ‘C-x <ESC> <ESC>’，它会重复 ‘query-replace’，因为它用迷你缓冲区读取了参数。参见“重复迷你缓冲区命令”

选项 ‘search-invisible’ 决定 ‘query-replace’ 如何处理不可见文本。参见“Outline Search”。

参见“对文件进行操作”，其中介绍了对选中文件执行查询替换的 Dired ‘Q’ 命令。另见 参见“在 Dired 中转换文件名”，其中介绍了通过替换文件名中的正则表达式匹配来重命名、复制或链接文件的 Dired 命令。

17.11 其他搜索与循环命令

下面是另一些为正则表达式查找匹配的命令。若模式中不含大写字母且 ‘case-fold-search’ 非 ‘nil’，它们在匹配时都忽略大小写。除了总是搜索整个缓冲区的 ‘multi-occur’ 和 ‘multi-occur-in-matching-buffers’ 之外，所有命令都作用于从光标点到缓冲区末尾的文本，或者，若区域处于活动状态，则作用于区域。

‘M-x multi-isearch-buffers’ 提示输入一个或多个缓冲区名，以 <RET> 结束；然后在这些缓冲区中开始多缓冲区增量搜索。（若在某个缓冲区中搜索失败，下一次 ‘C-s’ 会尝试搜索下一个指定的缓冲区，依此类推。）带前缀参数时，提示输入一个正则表达式，并在名称匹配该正则表达式的缓冲区中开始多缓冲区增量搜索。

‘M-x multi-isearch-buffers-regexp’ 此命令与 ‘multi-isearch-buffers’ 相同，只是执行增量正则表达式搜索。

‘M-x multi-isearch-files’ 提示输入一个或多个文件名，以 <RET> 结束；然后在这些文件中开始多文件增量搜索。（若在某个文件中搜索失败，下一次 ‘C-s’ 会尝试搜索下一个指定的文件，依此类推。）带前缀参数时，提示输入一个正则表达式，并在文件名匹配该正则表达式的文件中开始多文件增量搜索。

‘M-x multi-isearch-files-regexp’ 此命令与 ‘multi-isearch-files’ 相同，只是执行增量正则表达式搜索。

在某些设置了缓冲区局部变量 ‘multi-isearch-next-buffer-function’ 的模式中（例如 Change Log 模式），多文件增量搜索会自动启用。

‘M-x occur’ ‘M-s o’ 提示输入一个正则表达式，并显示一个列表，列出缓冲区中包含其匹配的每一行。若在提示处输入 ‘M-n’，可以重用先前增量搜索的搜索字符串。匹配的文本用 ‘match’ 字体（face）高亮。数字参数 N 表示在每个匹配行前后显示 N 行上下文。

上下文行数的默认值由变量 ‘list-matching-lines-default-context-lines’ 指定。当 ‘list-matching-lines-jump-to-current-line’ 非 ‘nil’ 时，当前行会以字体（face）‘list-matching-lines-current-line-face’ 高亮显示，且光标点被置于该行之后的第一个匹配处。

在增量搜索处于活动状态时也可以运行 ‘M-s o’；这会使用当前的搜索字符串。

注意，你输入的正则表达式的匹配会扩展到完整的行，而一个在前一个匹配结束之前开始的匹配不算作匹配。

‘*Occur*’ 缓冲区以 Occur 模式为主模式。你可以用 ‘n’ 和 ‘p’ 键移到下一个或上一个匹配；带前缀数字参数时，这些命令移动那么多匹配。数字键绑定到 ‘digit-argument’，所以 ‘5 n’ 移到往后第五个匹配（不必输入 ‘C-u’）。<SPC> 和 <DEL> 上下滚动 ‘*Occur*’ 缓冲区。点击某个匹配，或把光标点移到那里并输入 <RET>，会访问被搜索的原始缓冲区中对应的位置。‘o’ 和 ‘C-o’ 在另一个窗口中显示该匹配；‘C-o’ 不选中那个窗口。或者，你可以用 ‘M-g M-n’（‘next-error’）命令逐个访问这些出现（*note Compilation Mode::）。最后，‘q’ 退出显示 ‘*Occur*’ 缓冲区的窗口并把该缓冲区埋藏起来。

在 ‘*Occur*’ 缓冲区中输入 ‘e’ 会使该缓冲区可写，并进入 Occur Edit 模式，其中你可以编辑匹配行，且这些编辑会反映到来源缓冲区的文本中。输入 ‘C-c C-c’ 退出 Occur Edit 模式并回到 Occur 模式。

命令 ‘M-x list-matching-lines’ 是 ‘M-x occur’ 的同义词。

‘M-x multi-occur’ 此命令与 ‘occur’ 相同，只是它能搜索多个缓冲区。它会要求你逐个指定缓冲区名。

‘M-x multi-occur-in-matching-buffers’ 此命令与 ‘multi-occur’ 类似，只是要搜索的缓冲区由一个匹配所访问文件名的正则表达式指定。带前缀参数时，它改用该正则表达式匹配缓冲区名。

‘M-x how-many’ 提示输入一个正则表达式，并打印光标点之后缓冲区中其匹配的数目。若区域处于活动状态，则改为作用于区域。

‘M-x flush-lines’ 提示输入一个正则表达式，删除包含其匹配的每一行，作用于光标点之后的文本。命令结束时，它会打印被删除的匹配行数。

若当前行包含一个从光标点之后开始的匹配，此命令会删除当前行。若区域处于活动状态，则改为作用于区域；若某行部分包含在区域内，且它包含一个完全在区域内的匹配，则该行会被删除。

若一个匹配跨越多行，‘flush-lines’会删除所有这些行。它在开始寻找下一个匹配之前先删除这些行；因此，它会忽略在同一行上、紧接另一个匹配结束处开始的匹配。

‘M-x keep-lines’提示输入一个正则表达式，删除*不*包含其匹配的每一行，作用于光标点之后的文本。若光标点不在行首，此命令总是保留当前行。若区域处于活动状态，则命令改为作用于区域；它从不删除只有部分在区域内的行（结束一行的换行符算作该行的一部分）。

若一个匹配跨越多行，此命令会保留所有这些行。

‘M-x kill-matching-lines’与‘flush-lines’类似，但还会把匹配行加入删除环。该命令把匹配行作为单个字符串加入删除环，其中包含分隔这些行的换行符。

‘M-x copy-matching-lines’与‘kill-matching-lines’类似，但不把匹配行从缓冲区中移除。

17.12 定制搜索以满足需要

本节介绍其他章节未涉及的、与搜索有关的各种定制。

增量搜索的默认搜索模式由变量‘search-default-mode’指定。它可以是‘nil’、‘t’或一个函数。若为‘nil’，默认模式是进行字面搜索，不做字符折叠，但按‘case-fold-search’和‘search-whitespace-regexp’分别决定是否做大小写折叠与宽松空白匹配（参见“搜索时的宽松匹配”）。若值为‘t’，增量搜索默认进行正则表达式搜索。默认值指定一个只做大小写折叠和宽松空白匹配的函数。

正在进行的增量搜索的当前匹配用‘isearch’字体（face）高亮。把变量‘search-highlight’设为‘nil’可以禁用这种高亮。

在搜索正则表达式时（例如用‘C-M-s’），子表达式会根据变量‘search-highlight-submatches’获得特殊高亮。若该变量的值为‘nil’，不做特殊高亮；但若值非‘nil’，则匹配正则表达式中‘\(...\)’结构（又称“子表达式”）的文本会用不同的字体（face）高亮。默认定义了两个不同的字体（face），名为‘isearch-group-1’和‘isearch-group-2’。使用这两个字体（face）时，奇数号子表达式用‘isearch-group-1’字体（face）高亮，偶数号子表达式用‘isearch-group-2’字体（face）高亮。例如，搜索‘foo-\([0-9]+\)\([a-z]+\)’时，由‘[0-9]+’匹配的部分会用‘isearch-group-1’字体（face）高亮，由‘[a-z]+’匹配的部分会用‘isearch-group-2’字体（face）高亮。若你用同样的编号方案定义额外的字体（face），即‘isearch-group-3’、‘isearch-group-4’、……，则字体（face）‘isearch-group-M’将用于高亮第 M 个、第‘N+M’个、第‘2N+M’个……子表达式，其中 N 是形如‘isearch-group-M’的字体（face）总数。

搜索字符串的其余匹配中，显示在屏幕上的那些用‘lazy-highlight’字体（face）高亮。把变量‘isearch-lazy-highlight’设为‘nil’可禁用这种高亮。下面是另一些定制延迟高亮的变量：

‘lazy-highlight-initial-delay’高亮可见匹配之前等待的秒数。仅当搜索字符串长度小于‘lazy-highlight-no-delay-length’个字符时才适用。

‘lazy-highlight-no-delay-length’对于长度至少为该变量值的搜索字符串，匹配的延迟高亮立即开始。

‘lazy-highlight-interval’高亮相继匹配之间的秒数。

‘lazy-highlight-max-at-a-time’ 在检查输入之前最多高亮的匹配数。数值大时高亮要花些时间，所以如果你在此期间想继续搜索并输入 ‘C-s’ 或 ‘C-r’，Emacs 要等到把这些匹配全部高亮完才会响应。因此，较小的值让 Emacs 响应更灵敏。

‘isearch-lazy-count’ 在搜索提示中显示当前匹配编号和匹配总数。

‘lazy-count-prefix-format’ ‘lazy-count-suffix-format’ 这两个变量决定 ‘isearch-lazy-count’ 显示当前匹配编号和匹配总数的格式。

‘lazy-highlight-buffer’ 若非 ‘nil’，延迟高亮会高亮整个缓冲区中的匹配，而不只是当前窗口显示的那些（因此，例如，它们也会在显示同一缓冲区的其他窗口中变为可见）。

‘lazy-highlight-buffer-max-at-a-time’ 与 ‘lazy-highlight-max-at-a-time’ 类似，但在 ‘lazy-highlight-buffer’ 非 ‘nil’ 时用于高亮当前窗口中不可见的匹配。默认值为 200；设为 ‘nil’ 可高亮缓冲区中的所有匹配而不检查输入。*警告：*在搜索大缓冲区时这可能会让 Emacs 失去响应。

通常，在增量搜索中当搜索字符串为空时输入 <RET> 会启动非增量搜索。（实际上，它让你编辑搜索字符串，再下一次 <RET> 才进行搜索。）然而，若你把变量 ‘search-nonincremental-instead’ 定制为 ‘nil’，输入 <RET> 将总是退出增量搜索，即使搜索字符串为空。

默认情况下，增量搜索和 query-replace 命令会匹配不可见文本，但一旦当前匹配移离不可见文本，就会隐藏这些匹配。若你把变量 ‘isearch-hide-immediately’ 定制为 ‘nil’，则找到过匹配的任何不可见文本会一直显示，直到搜索或替换命令退出。

在慢速终端上增量搜索（例如通过慢速连接连到远程机器的显示器）可能很烦人，因为搜索推进时需要重绘显示的大部分内容。Emacs 为慢速终端提供了一种特殊的显示模式，搜索会弹出一个独立的小窗口，并在该窗口中显示匹配周围的文本。小窗口显示更快，因而缓解了慢速带来的烦人效果。变量 ‘search-slow-speed’ 决定波特率阈值，低于该阈值时 Emacs 会使用这种显示模式。变量 ‘search-slow-window-lines’ 控制 Emacs 为显示搜索结果而弹出的窗口的行数；默认是 1 行。通常，该窗口会弹出在显示你开始搜索的缓冲区的窗口底部，但若 ‘search-slow-window-lines’ 的值为负，则表示把窗口放在顶部，并给它 ‘search-slow-window-lines’ 绝对值那么多的行数。

18 纠错命令

本章介绍在编辑过程中发现错误时用得上的命令。这些命令中最基础的是撤销命令‘C-/’（也绑定到‘C-x u’和‘C-_’）。它撤销单个命令、命令的一部分（如‘query-replace’的情形），或者若干次连续的字符插入。连续重复‘C-/’会依次撤销越来越早的改动，一直回溯到现有撤销信息的限度。

除本章描述的命令外，你还可以用 <DEL>（‘delete-backward-char’）之类的删除命令抹去文本。这些内容本手册前面已经介绍过。参见“删除文本”。

18.1 撤销

“撤销”（undo）命令逆转缓冲区文本中最近的改动。每个缓冲区各自记录自己的改动，撤销命令总是作用于当前缓冲区。只要缓冲区的记录还在，你就可以撤销其中所有改动。通常每个编辑命令在撤销记录中占一条记录，但有些命令（如‘query-replace’）为了撤销时更灵活，会把自己的改动拆成多条记录。连续的字符插入命令通常合并成一条撤销记录，以免撤销过程过于繁琐。

- ‘C-/’、‘C-x u’、‘C-_’：撤销当前缓冲区撤销记录中的一条记录（‘undo’）。

要开始撤销，输入‘C-/’（或其别名‘C-_’、‘C-x u’）(1)。这会撤销缓冲区中最近的一处改动，并把光标点移回该改动之前所在的位置。连续重复‘C-/’（或其别名），就会依次撤销当前缓冲区中越来越早的改动。如果所有已记录的改动都已被撤销，撤销命令就会报错。

除撤销命令之外的任何命令都会打断撤销命令序列。从那一刻起，你刚刚执行的一整串撤销命令本身会被放进撤销记录。因此，若要重新应用你已经撤销的改动，先输入‘C-f’或其他任何能无害地打断撤销序列的命令，然后再输入一次或多次‘C-/’来撤销其中一些撤销命令。

另一种做法是，若你想继续撤销而不重做之前的撤销命令，可以使用‘M-x undo-only’。它与‘undo’类似，但不会重做你刚刚撤销的改动。与之互补，‘M-x undo-redo’会撤销先前的撤销命令（且不会把自己记为可撤销的命令）。

如果你发现缓冲区被意外修改了，最简单的恢复办法是反复输入‘C-/’，直到模式行前面的星号消失（参见“模式行”）。每当撤销命令让模式行上的星号消失，就说明缓冲区内容与上次读入或保存该文件时相同。如果你记不清某个改动是不是有意为之，就输入一次‘C-/’。看到你做的最后一处改动被撤销，你就能判断它是否是有意做的。若是意外，就让它保持撤销状态；若是有意，则按上面的说明重做该改动。

或者，你也可以用‘M-x revert-buffer’丢弃自上次访问或保存该缓冲区以来的全部改动（参见“还原缓冲区”）。

当存在活动区域时，任何‘undo’的使用都执行“选择性撤销”：它撤销区域内最近的一处改动，而不是整个缓冲区的改动。不过，当临时标记模式关闭时（参见“禁用临时标记模式”），‘C-/’总是作用于整个缓冲区，忽略区域。这种情况下，你可以给‘undo’命令一个前缀参数来执行选择性撤销：‘C-u C-/’。若要继续撤销同一区域内的更早改动，重复‘undo’命令即可（无需前缀参数）。

有些专用缓冲区不产生撤销记录。名字以空格开头的缓冲区从不产生撤销记录；这类缓冲区由 Emacs 内部使用，用来保存用户通常不会查看或编辑的文本。

当缓冲区的撤销信息变得过大时，Emacs 会不时（在“垃圾回收”期间）丢弃最旧的记录。你可以通过设置变量 ‘undo-limit’、‘undo-strong-limit’ 和 ‘undo-outer-limit’ 来指定保留多少撤销信息。它们的值以字节为单位。

变量 ‘undo-limit’ 设定一个软性上限：Emacs 会保留足够多命令的撤销数据，使之达到甚至略微超过该大小，但不会保留更早命令的数据。它的默认值是 160000。变量 ‘undo-strong-limit’ 设定更严格的上限：任何使大小超过该数量的先前命令（最近的那条除外）都会被遗忘。‘undo-strong-limit’ 的默认值是 240000。

无论这些变量的值如何，最近的一处改动永远不会被丢弃，除非它超过 ‘undo-outer-limit’（通常为 24,000,000）。到了那一步，Emacs 会丢弃该撤销数据并提醒你。这是唯一一种你无法撤销上一条命令的情形。若发生这种情况，你可以增大 ‘undo-outer-limit’ 的值，使将来更不容易发生。但如果你本以为该命令不会产生如此庞大的撤销数据，那多半是个缺陷，应当报告它。参见“报告 Bug”。

> 注：除 ‘C-/’ 外，‘undo’ 命令还绑定到 ‘C-x u’，因为这样对初学者来说更容易记住：‘u’ 代表“undo”。它还绑定到 ‘C-’，*因为在某些文本终端上输入 ‘C-/’ 实际会送入 ‘C-’*。

18.2 调换文本

- ‘C-t’：调换两个字符（‘transpose-chars’）。
- ‘M-t’：调换两个单词（‘transpose-words’）。
- ‘C-M-t’：调换两个平衡表达式（‘transpose-sexps’）。
- ‘C-x C-t’：调换两行（‘transpose-lines’）。
- ‘M-x transpose-sentences’：调换两个句子（‘transpose-sentences’）。
- ‘M-x transpose-paragraphs’：调换两个段落（‘transpose-paragraphs’）。
- ‘M-x transpose-regions’：调换两个区域。

常见的编辑失误之一是把字符、单词或句子的顺序敲错了。要修正它，你可以删掉敲错的文本再重新输入，也可以把位置不对的文本删除（参见“删除文本”）再粘贴（参见“粘贴”）到正确位置。但这样做效率不高。Emacs 提供了方便的调换字符、单词、句子等的命令，本节将逐一介绍。

两个相邻字符位置颠倒这种常见错误，可以用 ‘C-t’ 命令（‘transpose-chars’）修正。通常，‘C-t’ 会调换光标点两侧的两个字符。当在行尾使用时，它不会把行末字符与换行符调换（那样毫无用处），而是调换该行最后两个字符。所以，如果你立刻发现调换错误，只需一个 ‘C-t’ 就能修好。如果发现得没那么快，你就必须先移动光标，回到两个被调换的字符之间，再输入 ‘C-t’。如果你是把一个空格与其前面单词的最后一个字符调换了，用单词移动命令（‘M-f’、‘M-b’ 等）是不错的到位方式。否则，反向搜索（‘C-r’）常常是最好的办法。参见“搜索与替换”。

‘M-t’ 调换光标点之前的单词与光标点之后的单词（‘transpose-words’）。它把光标点向前移过一个单词，同时把光标点之前或光标点所在的单词也一起向前拖。两个单词之间的标点字符不移动。例如，‘FOO, BAR’ 会调换成 ‘BAR, FOO’，而不是 ‘BAR FOO,’。当光标点在行尾时，它会把光标点之前的单词与该行下一行的第一个单词调换。

‘C-M-t’（‘transpose-sexps’）是调换两个表达式的类似命令（参见“平衡括号表达式”），而 ‘C-x C-t’（‘transpose-lines’）交换两行。‘M-x transpose-sentences’ 和 ‘M-x transpose-paragraphs’ 分别调换句子和段落。这些命令的工作方式与 ‘M-t’ 相同，只是所调换的文本单位不同。

调换命令的数字参数用作重复次数：它告诉调换命令把光标点之前或光标点所在的字符（或单词、表达式、行）移过若干个其他字符（或单词、表达式、行）。例如，‘C-u 3 C-t’ 把光标点之前的字符向前移过另外三个字符。它会把 ‘f*oobar’ 变成 ‘oobf*ar’。这等价于重复三次 ‘C-t’。‘C-u - 4 M-t’ 把光标点之前的单词向后移过四个单词。‘C-u - C-M-t’ 则会抵消普通 ‘C-M-t’ 的效果。

数字参数为零时有特殊含义（因为否则重复次数为零的命令什么也不做）：把结束于光标点之后的字符（或单词、表达式、行）与结束于标记之后的那个字符（或单词、表达式、行）调换。因此，要把 ‘Bob and Mary’ 变成 ‘Mary and Bob’，可先在 ‘Bob’ 上设置标记（参见“设置标记”），再把光标点移到 ‘Mary’，然后输入 ‘C-u 0 M-t’。

‘M-x transpose-regions’ 把光标点与标记之间的文本，与标记环中最近推入的两个标记之间的文本调换（参见“设置标记”）。带数字前缀参数时，它与标记环中往回数那么多条的两个相继标记之间的文本调换。这个命令最适合一次性调换多个字符（或单词、句子、段落）。

18.3 大小写转换

- ‘M-- M-l’：把最后一个单词转换为小写。注意 ‘Meta--’ 是 Meta 减号。
- ‘M-- M-u’：把最后一个单词全部转换为大写。
- ‘M-- M-c’：把最后一个单词转换为首字母大写、其余小写。

一种很常见的错误是把单词的大小写敲错。因此，单词大小写转换命令 ‘M-l’、‘M-u’ 和 ‘M-c’ 在带负参数时有特殊之处：它们不移动光标。一旦你发现最后一个单词敲错了，直接对它做大小写转换，然后继续输入即可。参见“大小写转换命令”。

18.4 拼写检查与纠正

本节介绍检查单个单词或缓冲区某一部分拼写的命令。这些命令只有在安装了拼写检查程序——Hunspell、Aspell、Ispell 或 Enchant 之一——之后才有效。这些程序并不属于 Emacs，但在 GNU/Linux 及其他自由操作系统上通常都装好了其中之一。参见《《aspell》手册》

如果你只安装了其中一个拼写检查程序，Emacs 会在你第一次调用本节描述的命令时找到它。如果你装了多个，可以通过定制变量 ‘ispell-program-name’ 来控制使用哪一个。

- ‘M-$’：检查并纠正光标点处的单词拼写（‘ispell-word’）。若区域处于活动状态，则改为检查区域内所有单词。
- ‘C-u M-$’：若上一次拼写操作被中断，继续该操作（‘ispell-continue’）。
- ‘M-x ispell’：检查并纠正缓冲区中所有单词的拼写。若区域处于活动状态，则改为检查区域内所有单词。
- ‘M-x ispell-buffer’：检查并纠正缓冲区中的拼写。
- ‘M-x ispell-region’：检查并纠正区域内的拼写。
- ‘M-x ispell-message’：检查并纠正一封邮件草稿中的拼写，排除引用材料。
- ‘M-x ispell-comments-and-strings’：检查并纠正缓冲区或区域中注释和字符串的拼写。
- ‘M-x ispell-comment-or-string-at-point’：检查光标点处的注释或字符串。
- ‘M-x ispell-change-dictionary <RET> DICT <RET>’：用 DICT 作为词典重启拼写检查进程。

- ‘M-x ispell-kill-ispell’：杀掉拼写检查子进程。
- ‘M-<TAB>’、‘<ESC> <TAB>’、‘C-M-i’：根据拼写词典及其他补全源补全光标点之前的单词（‘completion-at-point’）。
- ‘M-x flyspell-mode’：启用 Flyspell 模式，高亮所有拼错的单词。
- ‘M-x flyspell-prog-mode’：只对注释和字符串启用 Flyspell 模式。

要检查光标点周围或之前的单词拼写，并可选地加以纠正，输入 ‘M-$’（‘ispell-word’）。若区域处于活动状态，‘M-$’ 检查区域内所有单词的拼写。参见“标记与区域”。（当临时标记模式关闭时，‘M-$’ 总是作用于光标点周围或之前的单词，忽略区域；参见“禁用临时标记模式”。）带前缀参数调用时，‘C-u M-$’ 会调用 ‘ispell-continue’，继续此前被 ‘X’ 或 ‘C-g’ 中断的拼写操作。

类似地，命令 ‘M-x ispell’ 在有活动区域时对区域进行拼写检查，否则对整个缓冲区进行。命令 ‘M-x ispell-buffer’ 和 ‘M-x ispell-region’ 则分别显式地对整个缓冲区或区域做拼写检查。要检查你正在写的邮件中的拼写，用 ‘M-x ispell-message’；该命令检查整个缓冲区，但会排除缩进的材料以及看起来是从其他邮件引用的材料。参见“发送邮件”。处理源代码时，可以用 ‘M-x ispell-comments-and-strings’ 或 ‘M-x ispell-comment-or-string-at-point’ 只检查注释或字符串字面量。

当这些命令遇到看起来拼错的单词时，会询问你该怎么办。它通常会显示一个带编号的“近似词”列表——即与错误单词相近的词。然后你必须输入一个单字符的答复。以下是有效的答复：

- ‘DIGIT’：仅这一次，用所显示的某个近似词替换该词。每个近似词都列有一个数字；输入该数字即可选中它。
- ‘<SPC>’：跳过这个词——仍然认为它拼写不正确，但在此处不做改动。
- ‘r NEW <RET>’：仅这一次，用 NEW 替换该词。（替换后的字符串会再次接受拼写检查。）
- ‘R NEW <RET>’：用 NEW 替换该词，并执行一次 ‘query-replace’，以便你愿意时替换缓冲区中其他位置的同一个词。（替换结果会再次接受拼写检查。）
- ‘a’：接受这个拼错的词——把它当作正确拼写，但仅在本次编辑会话中有效。
- ‘A’：接受这个拼错的词——把它当作正确拼写，但仅在本次编辑会话中、且仅对这个缓冲区有效。
- ‘i’：把这个词插入你的个人词典文件，从此以后它都会被视为正确拼写，将来的会话中也是如此。
- ‘m’：类似 ‘i’，但你还可以指定词典补全信息。
- ‘u’：把这个词的小写形式插入你的个人词典文件。
- ‘l WORD <RET>’：在词典中查找与 WORD 匹配的词。这些词成为新的近似词列表；你可以输入数字选中其中一个作为替换。你可以在 WORD 中使用 ‘*’ 作为通配符。
- ‘C-g’、‘X’：中断交互式拼写检查，把光标点留在正被检查的那个词上。之后你可以用 ‘C-u M-$’ 重新开始检查。
- ‘x’：退出交互式拼写检查，并把光标点移回你开始拼写检查时所处的位置。
- ‘q’：退出交互式拼写检查并杀掉拼写检查子进程。
- ‘C-r’：进入递归编辑（参见“递归编辑层级”）。当你用 ‘C-M-c’ 退出递归编辑时，交互式拼写检查会继续。这样你就可以查阅缓冲区文本而不打断拼写检查。请 *不要* 在递归编辑中修改缓冲区，尤其不要修改拼错

的词，因为退出递归编辑时这些编辑会被撤销。如果你需要编辑那个拼错的词，改用‘r’或‘R’，或者用‘X’，编辑缓冲区，然后用‘C-u M-$’继续。

- ‘C-z’：挂起 Emacs，或把选中的框架图标化。
- ‘?’：显示选项列表。

你可以让 Ispell 记住你的拼写纠正，使得在缩写模式启用时它们会被自动应用（参见“缩写”）。若把‘ispell-save-corrections-as-abbrevs’定制为非 nil 值，那么每当你纠正一个拼错的词，Emacs 就把该纠正保存为全局缩写。此后，在启用了缩写模式的缓冲区中，每当你输入这个拼错的写法再加上一个分词符（<SPC>、逗号等），Emacs 就会把该错误拼写展开为正确的写法。你可以在选择替换词时提供‘C-u’前缀参数，以推翻这一行为并禁止保存某次特定的纠正。如果‘ispell-save-corrections-as-abbrevs’取其默认值 nil，前缀参数的含义就反过来：在选择替换词之前输入‘C-u’反而 *会* 保存一个全局缩写。

使用命令‘M-<TAB>’（‘completion-at-point’）可以补全光标点处的单词。先输入一个单词的开头，然后输入‘M-<TAB>’从补全列表中挑选。（如果你的窗口管理器截获了‘M-<TAB>’，可以输入‘<ESC> <TAB>’或‘C-M-i’。）

拼写检查子进程一旦启动就会继续运行，等待任务，这样后续的拼写检查命令能更快完成。如果你想摆脱这个进程，用‘M-x ispell-kill-ispell’。通常没有必要这样做，因为除了做拼写纠正时，该进程不占用处理器时间。

拼写检查程序会查两部词典：标准词典和你的个人词典。标准词典由变量‘ispell-local-dictionary’指定，若它为‘nil’，则由变量‘ispell-dictionary’指定。若两者都是‘nil’，则使用拼写程序的默认词典。命令‘M-x ispell-change-dictionary’为该缓冲区设置标准词典并重启子进程，使其改用另一部标准词典。你的个人词典由变量‘ispell-personal-dictionary’指定。若它为‘nil’，拼写程序会在默认位置查找个人词典，具体位置因拼写检查程序而异。

通常，拼写检查程序使用的词典针对某一种特定语言。默认语言由系统环境和区域设置决定。如果你想检查另一种语言的文本拼写，标准词典和个人词典都应当更换。为此可以使用‘ispell-change-dictionary’命令。

Hunspell 的特殊之处在于它支持同时使用多部不同的词典进行拼写检查。要使用这一特性，在针对某组特定词典组合开始使用 Hunspell 之前，先调用‘M-x ispell-hunspell-add-multi-dic’命令。该命令会提示输入词典组合，它应当是以逗号分隔的若干特定语言词典名，例如‘en_US,de_DE,ru_RU’。此后，你就可以检查混合使用这些语言的文本，而不必每次更换词典。（注意：当几种语言使用同一种文字时，有可能某个在一种语言中拼错的词在另一种语言的词典中被当作有效拼写，从而被漏掉。）

单词补全使用单独的一部词典。变量‘ispell-complete-word-dict’指定该词典的文件名。补全词典必须不同，因为它无法使用词的词根和词缀信息，而拼写检查正是靠这些信息来识别词的各种变体。对某些语言来说，有拼写检查词典但没有单词补全词典。

Flyspell 模式是一个次模式，它在你输入文字时自动对其进行拼写检查。当它发现不认识的单词时，就会高亮该词。输入‘M-x flyspell-mode’可在当前缓冲区中切换 Flyspell 模式。要在所有文本模式缓冲区中启用 Flyspell 模式，把‘flyspell-mode’加入‘text-mode-hook’。参见“钩子”。注意，由于 Flyspell 模式需要检查你移动经过的每一个单词，它会拖慢光标移动和滚动命令。它也不会自动检查你没有输入、也没有移动经过的文本；对此请使用‘flyspell-region’或‘flyspell-buffer’。

正常情况下，Flyspell 模式会高亮你输入或修改过的拼错单词，以及你移动经过但未改动的单词。但如果你把变量‘flyspell-check-changes’定制为非‘nil’值，Flyspell 模式就只检查你输入或以某种方式编辑过的单词。

当 Flyspell 模式把某个词高亮为拼写错误时，你可以用‘mouse-2’（‘flyspell-correct-word’）点击它，弹出可能的纠正与操作菜单。如果你希望这个菜单出现在‘mouse-3’上，就启用‘context-menu-mode’。此外，‘C-.’或‘<ESC> <TAB>’（‘flyspell-auto-correct-word’）会为光标点处的单词依次给出各种可能的纠正，而‘C-c $’（‘flyspell-correct-word-before-point’）会弹出可能纠正的菜单。当然，你随时可以用任何喜欢的方式手动编辑拼错的词来纠正它。

Flyspell Prog 模式的作用与普通 Flyspell 模式相同，只是它只检查注释和字符串常量中的单词。这一特性在编辑程序时很有用。输入‘M-x flyspell-prog-mode’可在当前缓冲区中启用或禁用该模式。要在所有编程模式缓冲区中启用该模式，把‘flyspell-prog-mode’加入‘prog-mode-hook’（参见“钩子”）。

19 键盘宏

本章介绍如何录制一串编辑命令，以便日后方便地重复执行。

“键盘宏”（keyboard macro）是 Emacs 用户定义的一种命令，用来代表另一串按键。例如，如果你发现自己正要输入 ‘C-n M-d C-d’ 四十次，就可以定义一个执行 ‘C-n M-d C-d’ 的键盘宏，然后把它再执行 39 次，从而加快工作速度。

定义键盘宏的办法是执行并录制构成其定义的那些命令。换句话说，在定义键盘宏时，定义本身就在第一次执行。这样你就能看到命令的效果，不必在脑子里推演。等你结束定义时，键盘宏就定义好了，而且实际上已经执行过一次。此后，你只要调用该宏，就能把整套操作再做一遍。

键盘宏与普通的 Emacs 命令的区别在于，它们是用 Emacs 命令语言而不是 Lisp 编写的。这使初学者更容易编写键盘宏，也使它们作为临时权宜手段更为方便。不过，Emacs 命令语言作为一种编程语言还不够强大，无法用来编写任何有智能或通用性的东西。要做这类事情，必须使用 Lisp。

- 键盘宏查询：让键盘宏每次做不同的事。
- 保存键盘宏：给键盘宏命名；把它们保存到文件中。
- 编辑键盘宏：编辑键盘宏。
- 键盘宏单步编辑：交互式地执行并编辑键盘宏。
- Kmacro 菜单：用于列出和编辑键盘宏以及键盘宏环的界面。

19.1 基本用法

- ‘<F3>’：开始定义键盘宏（‘kmacro-start-macro-or-insert-counter’）。
- ‘<F4>’：若正在定义键盘宏，则结束定义；否则执行最近的键盘宏（‘kmacro-end-or-call-macro’）。
- ‘C-u <F3>’：重新执行上一个键盘宏，然后向它的定义追加按键。
- ‘C-u C-u <F3>’：向最后一个键盘宏追加按键，但不重新执行它。
- ‘C-x C-k r’：在区域中起始的每一行上运行最后一个键盘宏（‘apply-macro-to-region-lines’）。
- ‘C-x (’：开始定义键盘宏（旧式）（‘kmacro-start-macro’）；带前缀参数时向最后一个宏追加按键。
- ‘C-x)’：结束宏定义（旧式）（‘kmacro-end-macro’）；前缀参数作为执行该宏的重复次数。
- ‘C-x e’：执行最近定义的键盘宏（‘kmacro-end-and-call-macro’）；前缀参数作为重复次数。

要开始定义键盘宏，输入 <F3>。从那时起，你按的键仍会被执行，但同时也会成为宏定义的一部分。模式行上会出现 ‘Def’，提醒你当前正在做什么。完成后，输入 <F4>（‘kmacro-end-or-call-macro’）来结束定义。例如，

```
<F3> M-f foo <F4>
```

定义的宏会向前移动一个单词，然后插入 ‘foo’。注意 <F3> 和 <F4> 不会成为宏的一部分。

定义好宏之后，你可以用 <F4> 调用它。对上面的例子来说，这与再输入一遍 ‘M-f foo’ 效果相同。（注意 <F4> 命令有双重作用：如果你正在定义宏，它就结束定义；否则它调用上一个宏。）你还可以给 <F4> 一个

数值前缀参数 'n'，表示调用该宏 'n' 次。参数为零会无限次重复该宏，直到出错或你输入 'C-g'（在 MS-DOS 上则是 'C-<Break>'）。

上面的例子演示了使用键盘宏的一个简便技巧：如果你想在文本中按固定间隔重复某项操作，就把移动命令作为宏的一部分。在这个例子里，重复该宏就会在每一个相继的单词后面插入字符串 'foo'。

结束键盘宏的定义后，你可以输入 'C-u <F3>' 向其定义追加更多的按键。这等价于直接按 <F3>，再把到目前为止的整个定义重新输入一遍。因此，它会重新执行先前定义的那个宏。如果你把变量 'kmacro-execute-before-append' 改为 'nil'，那么在追加之前就不会重新执行已有的宏（默认值为 't'）。你也可以输入 'C-u C-u <F3>'，把按键加到最后一个键盘宏定义的末尾，而无需重新执行它。

当某条命令用迷你缓冲区读取参数时，你的迷你缓冲区输入会连同该命令一起成为宏的一部分。因此，当你重放该宏时，该命令会得到与录制宏时相同的参数。例如，

```
<F3> C-a C-k C-x b foo <RET> C-y C-x b <RET> <F4>
```

定义的宏会删除当前行，把它粘贴到缓冲区 'foo' 中，然后回到原来的缓冲区。

大多数键盘命令在键盘宏定义中都照常工作，但有几个例外。输入 'C-g'（'keyboard-quit'）会退出键盘宏定义。输入 'C-M-c'（'exit-recursive-edit'）可能不可靠：如果它退出的是在宏内部开始的递归编辑，则符合你的预期；但如果它退出的是在你调用键盘宏之前就已开始的递归编辑，那就必然也会退出键盘宏。鼠标事件同样不可靠，尽管你可以在键盘宏中使用它们：宏重放鼠标事件时，使用的是该事件最初的鼠标位置，也就是你定义宏时鼠标所在的位置。其效果可能很难预料。

命令 'C-x C-k r'（'apply-macro-to-region-lines'）会在区域中起始的每一行上重复最后定义的键盘宏。它逐行进行：把光标点移到行首，然后执行该宏。

除了上面介绍的 <F3> 和 <F4> 命令之外，Emacs 还支持一套较旧的、用于定义和执行键盘宏的键绑定。要开始宏定义，输入 'C-x ('（'kmacro-start-macro'）；与 <F3> 一样，前缀参数会把这段定义追加到最后一个键盘宏上。要结束宏定义，输入 'C-x)'（'kmacro-end-macro'）。要执行最近的宏，输入 'C-x e'（'kmacro-end-and-call-macro'）。如果你在定义宏的过程中输入 'C-x e'，该宏就会被终止并立即执行。输入 'C-x e' 之后，你可以立刻重复输入 'e'，把该宏立即再重复一次或多次。你还可以给 'C-x e' 一个重复参数，就像 <F4>（用于执行宏时）那样。

'C-x)' 可以接受一个重复次数作为参数。这意味着在定义完宏之后立即重复它。宏定义本身算作第一次重复，因为它在定义时就被执行了，所以 'C-u 4 C-x)' 会立即把该宏额外执行 3 次。

在执行运行时间较长的键盘宏时，有时触发一次重显示（以显示已经进行到哪一步）会很有用。'C-x C-k d' 命令可用于此。举一个不太有用的例子：'C-x (M-f C-x C-k d C-x)' 会创建一个宏，当你说 'C-u 42 C-x e' 时，它会在每次迭代时重显示一次。

19.2 键盘宏环

所有定义过的键盘宏都记录在"键盘宏环"（keyboard macro ring）中。只有一个键盘宏环，由所有缓冲区共享。

- 'C-x C-k C-k'：执行宏环头部的键盘宏（'kmacro-end-or-call-macro-repeat'）。

- ‘C-x C-k C-n’：把键盘宏环转到下一个宏（更早定义的）（‘kmacro-cycle-ring-next’）。
- ‘C-x C-k C-p’：把键盘宏环转到上一个宏（更晚定义的）（‘kmacro-cycle-ring-previous’）。

所有操作键盘宏环的命令都使用相同的前缀 ‘C-x C-k’。这些命令大多可以在彼此之后立即执行并重复，而不必重复 ‘C-x C-k’ 前缀。例如，

```
C-x C-k C-p C-p C-k C-k C-k C-n C-n C-k C-p C-k C-d
```

会把键盘宏环转到倒数第二个宏，把转到的头部宏执行三次，再转回原来的头部宏，执行一次，转到上一个宏，执行它，最后把它从宏环中删除。

命令 ‘C-x C-k C-k’（‘kmacro-end-or-call-macro-repeat’）执行宏环头部的键盘宏。你可以立即重复该宏：再输入一个 ‘C-k’；也可以立即转动宏环：输入 ‘C-n’ 或 ‘C-p’。

正在定义键盘宏时，‘C-x C-k C-k’ 的行为与 <F4> 相同，区别在于此后你可以立即使用本节中的大多数键绑定，而不带 ‘C-x C-k’ 前缀。例如，再输入一个 ‘C-k’ 就会重新执行该宏。

命令 ‘C-x C-k C-n’（‘kmacro-cycle-ring-next’）和 ‘C-x C-k C-p’（‘kmacro-cycle-ring-previous’）转动宏环，把下一个或上一个键盘宏带到宏环头部。新头部宏的定义会显示在回显区。你可以只重复 ‘C-n’ 和 ‘C-p’ 来继续立即转动宏环，直到想要的宏位于宏环头部。要立即执行新的宏环头部宏，只需输入 ‘C-k’。

注意，Emacs 把宏环头部当作最后定义的键盘宏。例如，<F4> 会执行该宏，而 ‘C-x C-k n’ 会给它命名。

键盘宏环中最多存放多少个宏，由可自定义变量 ‘kmacro-ring-max’ 决定。

19.3 键盘宏计数器

每个键盘宏都有一个关联的计数器，在你开始定义该宏时初始化为 0。这个“当前计数器”让你可以向缓冲区中插入一个取决于宏已被调用次数的数字。计数器通常在其值被插入缓冲区时递增。

除了当前计数器，键盘宏还维护“前一个计数器”，它记录当前计数器上一次递增或设置时的值。注意，把当前计数器递增零（例如用 ‘C-u 0 C-x C-k C-i’）也会把当前计数器的值记录为前一个计数器的值。

- ‘<F3>’：在键盘宏定义中，把键盘宏计数器的值插入缓冲区（‘kmacro-start-macro-or-insert-counter’）。
- ‘C-x C-k C-i’：把键盘宏计数器的值插入缓冲区（‘kmacro-insert-counter’）。
- ‘C-x C-k C-c’：设置键盘宏计数器（‘kmacro-set-counter’）。
- ‘C-x C-k C-a’：把前缀参数加到键盘宏计数器上（‘kmacro-add-counter’）。
- ‘C-x C-k C-f’：指定插入键盘宏计数器所用的格式（‘kmacro-set-format’）。

在定义键盘宏时，命令 <F3>（‘kmacro-start-macro-or-insert-counter’）会把键盘宏计数器的当前值插入缓冲区，并把计数器加 1。（如果你没有在定义宏，<F3> 会改为开始一个宏定义。参见“基本用法”。）你可以用数值前缀参数指定不同的增量。如果你只给出 ‘C-u’ 前缀，则会插入前一个计数器的值，而不改变当前值。

作为例子，我们演示如何用键盘宏计数器来构造带编号的列表。考虑下面的键序列：

```
<F3> C-a <F3> . <SPC> <F4>
```

作为这个键盘宏定义的一部分，字符串 '0. ' 被插入当前行的开头。如果你现在把光标移到缓冲区中的其他地方并输入 <F4> 调用该宏，字符串 '1. ' 就会被插入该行的开头。后续调用会插入 '2. '、'3. ' 等等。

命令 'C-x C-k C-i'（'kmacro-insert-counter'）与 <F3> 作用相同，但可以在键盘宏定义之外使用。当没有键盘宏正在定义或执行时，它会插入并递增键盘宏环头部那个宏的计数器。

命令 'C-x C-k C-c'（'kmacro-set-counter'）把当前宏计数器设为数值参数的值。如果你在宏内部使用它，它会在宏的每次重复中生效。如果你在执行宏时只给出 'C-u' 作为前缀，那会把计数器重置为它在当前这次宏重复开始时的值（撤销本次重复到目前为止的所有递增）。

命令 'C-x C-k C-a'（'kmacro-add-counter'）把前缀参数加到当前宏计数器上。若只给 'C-u' 作为参数，它会把计数器重置为任何键盘宏最近插入的值。（通常使用它时，最近一次插入来自同一个宏，并且是同一个计数器。）

命令 'C-x C-k C-f'（'kmacro-set-format'）提示输入插入宏计数器时使用的格式。默认格式是 '%d'，意思是把数字按十进制插入，不做任何填充。用空的迷你缓冲区退出可以把格式重置为这个默认值。你可以指定 'format' 函数接受的、且对单个整数额外参数有意义的任何格式字符串（参见《Emacs Lisp 参考手册》的"Formatting Strings"）。在迷你缓冲区中输入该格式字符串时，不要把它放在双引号里。

如果在没有键盘宏正在定义或执行时使用此命令，新格式会影响所有后续的宏定义。已有宏继续使用它们定义时生效的格式。如果你在定义键盘宏时设置格式，那么从那时起该格式会影响正在定义的宏，但不影响后续的宏。执行该宏时，每一步都会使用其定义过程中该步生效的格式。在执行宏的过程中改变宏格式，与定义过程中相应的改变一样，对后续的宏没有影响。

'C-x C-k C-f' 设置的格式不影响插入寄存器中存储的数字。

如果你把寄存器用作计数器，并在宏的每次重复中递增它，那也能达到与键盘宏计数器相同的效果。参见"在寄存器中保存数字"。对大多数用途来说，使用键盘宏计数器更简单。

19.4 带变化地执行宏

在键盘宏中，你可以制造出与 'query-replace' 类似的效果：宏每一轮都询问你是否要做改动。

- 'C-x q'：宏执行中到达此处时，请求确认（'kbd-macro-query'）。

定义宏时，在希望出现询问的位置输入 'C-x q'。在宏定义过程中，'C-x q' 什么也不做，但当你之后运行该宏时，'C-x q' 会交互式地问你是否继续。

'C-x q' 提问时的有效回应有：

- <SPC>（或 'y'）：继续执行键盘宏。
- <DEL>（或 'n'）：跳过本次宏重复的剩余部分，并立即开始下一次重复。
- <RET>（或 'q'）：跳过本次宏重复的剩余部分，并取消后续的重复。
- 'C-r'：进入一个递归编辑层，在其中可以进行不属于该宏的编辑。当你用 'C-M-c' 退出该递归编辑时，会再次询问你如何继续执行键盘宏。此时若输入 <SPC>，就会执行宏定义余下的部分。你需要负责让光标点和文本处于合适的状态，使宏的其余部分能按你的意愿工作。

‘C-u C-x q’ 即带前缀参数的 ‘C-x q’，其作用完全不同。无论是在定义宏时输入它，还是在宏执行时执行它，它都会进入一个从键盘读取输入的递归编辑。在定义期间，你在递归编辑中所做的编辑不会成为宏的一部分。在宏执行期间，递归编辑让你有机会在每次重复时做一些特殊化的编辑。参见“递归编辑层级”。

19.5 命名并保存键盘宏

- ‘C-x C-k n’：给最近定义的键盘宏一个命令名（仅在本次 Emacs 会话期间有效）（‘kmacro-name-last-macro’）。
- ‘C-x C-k b’：把最近定义的键盘宏绑定到一个键序列（仅在本次会话期间有效）（‘kmacro-bind-to-key’）。
- ‘M-x insert-kbd-macro’：把某个键盘宏的定义以 Lisp 代码插入缓冲区。

如果你想保存键盘宏供以后使用，可以用 ‘C-x C-k n’（‘kmacro-name-last-macro’）给它命名。该命令用迷你缓冲区读取一个名字作为参数，并把该名字定义为执行最后一个键盘宏（按其当前形式）。（如果你之后向该宏的定义追加内容，并不会改变这个名字作为宏的定义。）宏名是一个 Lisp 符号，以这种方式定义它，就使它成为可用 ‘M-x’ 调用的合法命令名，也可用 ‘keymap-global-set’ 把键绑定到它（参见“键位图”）。如果你指定的名字已有非键盘宏的先前定义，则会显示错误信息，并且不做任何改动。

你还可以用 ‘C-x C-k b’（‘kmacro-bind-to-key’），后跟你想要绑定的键序列，把最后一个键盘宏（按其当前形式）绑定到一个键上。你可以绑定全局键位图中的任何键序列，但由于大多数键序列已有其他绑定，选取键序列时应格外小心。如果你试图绑定到一个已有绑定（在任何键位图中）的键序列，该命令会在替换现有绑定之前请求你确认。

为避免覆盖现有绑定引起的问题，键序列 ‘C-x C-k 0’ 到 ‘C-x C-k 9’ 以及 ‘C-x C-k A’ 到 ‘C-x C-k Z’ 保留给你自己的键盘宏绑定。事实上，要绑定到这些键序列之一，你只需输入相应的数字或字母，而不必输入完整的键序列。例如，

```
C-x C-k b 4
```

会把最后一个键盘宏绑定到键序列 ‘C-x C-k 4’。

你可以把宏的定义保存到文件中。这样它就能在另一次编辑会话中使用。首先，访问你打算保存定义的那个文件。然后使用下面这条命令：

```
M-x insert-kbd-macro <RET> MACRONAME <RET>
```

这会插入一些 Lisp 代码，日后执行这些代码时，就会定义出同一个宏，其定义与现在相同。（你无需理解 Lisp 代码也能做到这一点，因为 ‘insert-kbd-macro’ 会替你写好 Lisp 代码。）然后保存该文件。你稍后可以用 ‘load-file’ 加载该文件（参见“Emacs 的 Lisp 代码库”）。如果你保存到的文件是你的初始化文件 ‘~/.emacs’（参见“Emacs 初始化文件”），那么每次运行 Emacs 时都会定义该宏。

如果你给 ‘insert-kbd-macro’ 一个前缀参数，它会额外生成一些 Lisp 代码，记录你绑定到 MACRONAME 的键（如果有的话），这样当你加载该文件时，该宏会被重新分配相同的键。

如果你不给 ‘insert-kbd-macro’ 提供宏名，它会插入 Lisp 代码来恢复 ‘last-kdb-macro’。

19.6 编辑键盘宏

- ‘C-x C-k C-e’：编辑最后定义的键盘宏（‘kmacro-edit-macro’）。
- ‘C-x C-k e NAME <RET>’：编辑先前定义的键盘宏 NAME（‘edit-kbd-macro’）。
- ‘C-x C-k l’：把最近 300 次击键当作键盘宏来编辑（‘kmacro-edit-lossage’）。

你可以输入 ‘C-x C-k C-e’ 或 ‘C-x C-k <RET>’（‘kmacro-edit-macro’）来编辑最后一个键盘宏。这会把宏定义格式化到一个缓冲区中，并进入一个专门用于编辑它的主模式。在该缓冲区中输入一次 ‘C-h m’，可显示如何编辑该宏的详细信息。编辑完成后，输入 ‘C-c C-c’。

‘kmacro-edit-macro’ 所用的主模式 ‘edmacro-mode’ 提供了若干命令，使编辑格式化后的宏更容易。用 ‘C-c C-q’（‘edmacro-insert-key’）可以把接下来输入的下一个键序列以正确的格式插入缓冲区，类似于 ‘C-q’（‘quoted-insert’）。用 ‘C-c C-r’（‘edmacro-set-macro-to-region-lines’）可以用区域中的文本替换宏文本。如果区域不是从行首开始，或者不是到行尾结束，则区域会扩展以包含完整的行。如果区域结束于某一行的开头，则最后那一行不包括在内。

你可以输入 ‘C-x C-k e’（‘edit-kbd-macro’）来编辑一个有名字的键盘宏或绑定到某个键的宏。随后输入你用来调用该宏的键盘输入——‘C-x e’ 或 ‘M-x NAME’ 或其他某个键序列。

你可以输入 ‘C-x C-k l’（‘kmacro-edit-lossage’）把最近 300 次击键当作宏来编辑。默认情况下，你最近的击键列在缓冲区底部。要让宏的键序列按相反顺序列出，可把 ‘edmacro-reverse-macro-lines’ 设为 ‘t’。

19.7 逐步编辑键盘宏

你可以输入 ‘C-x C-k <SPC>’（‘kmacro-step-edit-macro’），一次一条命令地交互式重放并编辑最后一个键盘宏。除非你用 ‘q’ 或 ‘C-g’ 退出宏，否则编辑后的宏会替换宏环上的最后一个宏。

这个宏编辑功能会在迷你缓冲区中显示最后一个宏，以及将要执行的第一条（或下一条）命令，并提示你选择一个操作。你可以输入 ‘?’ 来获取各选项的概要。可用的操作有：

- <SPC> 和 ‘y’：执行当前命令，并前进到键盘宏中的下一条命令。
- ‘n’、‘d’ 和 <DEL>：跳过并删除当前命令。
- ‘f’：在键盘宏的本次执行中跳过当前命令，但不把它从宏中删除。
- <TAB>：执行当前命令，以及紧跟在当前命令之后的所有同类命令；例如，<TAB> 可用于插入一串字符（对应于一串 ‘self-insert-command’ 命令）。
- ‘c’：继续执行（不再做进一步编辑），直到键盘宏结束。如果执行正常终止，编辑后的宏会替换原来的键盘宏。
- ‘C-k’：跳过并删除键盘宏的剩余部分，结束逐步编辑，并用编辑后的宏替换原来的键盘宏。
- ‘q’ 和 ‘C-g’：取消对键盘宏的逐步编辑；丢弃对键盘宏所做的任何改动。
- ‘i KEY... C-j’：读取并执行一系列键序列（不包括最后的 ‘C-j’），把它们插入键盘宏中当前命令之前，而不越过当前命令向前推进。
- ‘I KEY...’：读取一个键序列，执行它，并把它插入键盘宏中当前命令之前，而不越过当前命令向前推进。

- ‘r KEY... C-j’：读取并执行一系列键序列（不包括最后的‘C-j’），用它们替换键盘宏中的当前命令，并越过所插入的键序列向前推进。
- ‘R KEY...’：读取一个键序列，执行它，并用该键序列替换键盘宏中的当前命令，越过所插入的键序列向前推进。
- ‘a KEY... C-j’：执行当前命令，然后读取并执行一系列键序列（不包括最后的‘C-j’），并把它们插入键盘宏中当前命令之后；随后越过当前命令和所插入的键序列向前推进。
- ‘A KEY... C-j’：执行键盘宏中其余的命令，然后读取并执行一系列键序列（不包括最后的‘C-j’），并把它们追加到键盘宏末尾；随后结束逐步编辑，并用编辑后的宏替换原来的键盘宏。

19.8 列出与编辑键盘宏

要显示现有键盘宏的列表，输入‘M-x list-keyboard-macros <RET>’。这会在名为‘*Keyboard Macro List*’的缓冲区中弹出“Kmacro Menu”。列表中的每一行显示一个宏的位置、计数器值、计数器格式、按该格式格式化后的计数器值，以及宏的按键。下面是宏列表的一个例子：

```
Position  Counter  Format  Formatted  Keys
0               8  %02d    08         N : SPC <F3> RET
1               0  %d      0          l o n g SPC p h r a s e
```

宏按这样的顺序列出：当前宏位于最上面，位置编号为零；较早的宏则按照它们在键盘宏环中出现的顺序排列（参见“键盘宏环”）。借助 Kmacro 菜单，你可以改变宏的顺序，并修改它们的计数器、计数器格式和按键。Kmacro 菜单是只读缓冲区，只能通过本节介绍的特殊命令来修改。命令执行之后，Kmacro 菜单会显示相应的变化，以反映宏属性与宏环的新值。在该缓冲区中，你可以使用通常的光标移动命令，也可以使用用于在表格中导航的特殊移动命令。要查看特殊命令的列表，请在 Kmacro 菜单中输入‘C-h m’或‘?’（‘describe-mode’）。

你可以使用以下命令修改宏的属性：

- ‘#’改变当前行上宏的位置（参见“键盘宏环”）。
- ‘C-x C-t’把当前行上的宏移到上一行，如同‘transpose-lines’那样。
- ‘c’改变当前行上宏的计数器值（参见“键盘宏计数器”）。
- ‘f’改变当前行上宏的计数器格式。
- ‘e’使用‘edit-kbd-macro’改变当前行上宏的按键（参见“编辑键盘宏”）。
- ‘<RET>’使用上述命令改变当前行上宏在当前列中的值。

以下命令删除或复制列表中的宏：

- ‘d’‘d (Kmacro Menu)’把当前行上的宏标记为待删除，然后把光标点移到下一行（‘kmacro-menu-flag-for-deletion’）。删除标志由行首的字符‘D’表示。只有当你输入‘x’命令时才会真正执行删除（见下文）。

若区域处于活动状态，该命令会把区域内的所有宏都标记为待删除。

- ‘x’‘x (Kmacro Menu)’删除列表中所有已被标记为待删除的宏（‘kmacro-menu-do-flagged-delete’）。

- ‘m’ ‘m (Kmacro Menu)’ 标记当前行上的宏，然后把光标点移到下一行（‘kmacro-menu-mark’）。被标记的宏由行首的字符 ‘*’ 表示。已标记的宏可以用 ‘C’ 和 ‘D’ 命令来操作（见下文）。

若区域处于活动状态，该命令会标记区域内的所有宏。

- ‘C’ ‘C (Kmacro Menu)’ 该命令在宏于列表中的当前位置插入其副本，从而复制这些宏（‘kmacro-menu-do-copy’）。例如，对位置编号为零的宏运行此命令，会把该宏的一份副本插入位置编号一，并把其余的宏依次下移。

若区域处于活动状态，该命令会复制区域内的宏。否则，若存在已标记的宏，该命令会复制已标记的宏。若既没有活动区域，也没有已标记的宏，该命令会复制当前行上的宏。在前两种情况下，命令会在复制之前请求确认。

- ‘D’ ‘D (Kmacro Menu)’ 该命令删除宏，把它们从宏环中移除（‘kmacro-menu-do-delete’）。例如，对位置编号为零的宏运行此命令，会删除当前宏，然后让宏环中的第一个宏（原先位于位置编号一）成为新的当前宏，并将它从环中弹出。

若区域处于活动状态，该命令会删除区域内的宏。否则，若存在已标记的宏，该命令会删除已标记的宏。若既没有活动区域，也没有已标记的宏，该命令会删除当前行上的宏。在所有情况下，命令都会在删除之前请求确认。

该命令是 ‘d’ 和 ‘x’ 命令的替代方式（见上文）。

- ‘u’ ‘u (Kmacro Menu)’ 取消当前行上宏的标记和待删除标志，然后把光标点下移到下一行（‘kmacro-menu-unmark’）。若存在活动区域，该命令会取消区域内所有宏的标记和待删除标志。

- ‘<DEL>’ ‘<DEL> (Kmacro Menu)’ 与 ‘u’ 命令类似（见上文），但没有活动区域时会把光标点上移到上一行（‘kmacro-menu-unmark-backward’）。

- ‘U’ ‘U (Kmacro Menu)’ 取消列表中所有宏的标记和待删除标志（‘kmacro-menu-unmark-all’）。

20 文件处理

操作系统把数据永久存储在具名的“文件”（file）中，因此你用 Emacs 编辑的大多数文本都来自某个文件，最终也存储在某个文件中。

要编辑文件，必须让 Emacs 读取该文件，并准备一个包含该文件文本副本的缓冲区。这称为“访问”（visiting）该文件。编辑命令直接作用于缓冲区中的文本，也就是 Emacs 内部的副本。只有当你把缓冲区“保存”（save）回文件时，你的改动才会出现在文件本身中。

除了访问和保存文件，Emacs 还能删除、复制、重命名、追加文件，保留文件的多个版本，以及对文件目录进行操作。

20.1 文件名

许多操作文件的 Emacs 命令都要求你指定文件名，这要通过迷你缓冲区完成（参见“用于文件名的迷你缓冲区”）。

在迷你缓冲区中，你可以使用通常的补全和历史命令（参见“迷你缓冲区”）。注意，文件名补全会忽略扩展名出现在变量 ‘completion-ignored-extensions’ 中的文件名（参见“补全选项”）。还要注意，大多数命令在读取文件名时使用带确认的宽松补全：你被允许提交一个不存在的文件名，但如果你在补全到不存在的文件名后立即输入 <RET>，Emacs 会显示 ‘[Confirm]’，你必须再输入一次 <RET> 确认。详见参见“补全退出”。

迷你缓冲区历史命令为读取文件名提供了一些特殊功能，参见“迷你缓冲区历史”。

每个缓冲区都有一个“默认目录”，保存在缓冲区局部变量 ‘default-directory’ 中。每当 Emacs 用迷你缓冲区读取文件名时，它通常会把默认目录作为初始内容插入迷你缓冲区。你可以通过把变量 ‘insert-default-directory’ 改为 ‘nil’ 来禁止这种插入（参见“用于文件名的迷你缓冲区”）。无论如何，Emacs 始终假定任何相对文件名都是相对于默认目录的，例如，输入不带目录的文件名即指定默认目录中的文件。

当你访问一个文件时，Emacs 会把访问缓冲区的 ‘default-directory’ 设为该文件所在的目录。当你通过 ‘C-x b’ 之类的命令创建不访问文件的新缓冲区时，其默认目录通常复制自当时为当前缓冲区的那个缓冲区（参见“创建与选择缓冲区”）。你可以用命令 ‘M-x pwd’ 查看当前缓冲区中 ‘default-directory’ 的值。命令 ‘M-x cd’ 提示输入目录名，并把缓冲区的 ‘default-directory’ 设为该目录（这样做不会更改缓冲区的文件名，如果有的话）。

举例来说，当你访问文件 ‘/u/rms/gnu/gnu.tasks’ 时，默认目录被设为 ‘/u/rms/gnu/’。若你调用读取文件名的命令，在迷你缓冲区中只输入 ‘foo’（省略目录），则指定文件 ‘/u/rms/gnu/foo’；输入 ‘../.login’ 指定 ‘/u/rms/.login’；输入 ‘new/foo’ 指定 ‘/u/rms/gnu/new/foo’。

在迷你缓冲区中输入文件名时，你可以使用几个快捷方式：双斜杠会忽略这对斜杠中第二个斜杠之前的一切，而 ‘~/’ 是你的主目录。参见“用于文件名的迷你缓冲区”。

字符 ‘$’ 用于把环境变量替换到文件名中。环境变量的名称由 ‘$’ 之后的所有字母数字字符组成；或者，它可以放在 ‘$’ 之后的花括号中。例如，若你用过 shell 命令 ‘export FOO=rms/hacks’ 来设置名为 ‘FOO’ 的环境变量，那么 ‘/u/$FOO/test.c’ 和 ‘/u/${FOO}/test.c’ 都是 ‘/u/rms/hacks/test.c’ 的缩写。若环境变量未定义，则不会发生替换，因此字符 ‘$’ 代表它自身。注意，在 Emacs 之外设置的环境变量只有在 Emacs 启动之前应用才会影响 Emacs。

要访问名字中含‘$’的文件，若‘$’会引起展开，就输入‘$$’。这对字符会在为单个‘$’执行变量替换的同一时刻被转换为单个‘$’。或者，用‘/:’引用整个文件名（参见“引用的文件名”）。以字面‘~’开头的文件名应当用‘/:./’引用。

文件名中可以包含非 ASCII 字符。参见“文件名的编码系统”。

20.2 访问文件

- ‘C-x C-f’：访问文件（‘find-file’）。
- ‘C-x C-r’：以只读方式访问文件，不允许修改它（‘find-file-read-only’）。
- ‘C-x C-v’：访问另一个文件来替代上次访问的文件（‘find-alternate-file’）。
- ‘C-x 4 f’：在另一个窗口中访问文件（‘find-file-other-window’）。不改变所选窗口中显示的内容。
- ‘C-x 5 f’：在新框架中访问文件（‘find-file-other-frame’）。不改变所选框架中显示的内容。
- ‘M-x find-file-literally’：访问文件，不对内容做任何转换。

“访问”（visiting）文件指把其内容读入 Emacs 缓冲区，以便你编辑它们。Emacs 为你访问的每个文件创建一个新缓冲区。

要访问文件，输入‘C-x C-f’（‘find-file’），并在迷你缓冲区中输入所需文件的名称。在迷你缓冲区中，你可以输入‘C-g’来中止该命令。关于在迷你缓冲区中输入文件名的细节，参见“文件名”。

若指定的文件存在但系统不允许你读取，回显区会显示一条错误消息（在 GNU 和 Unix 系统上，你也许能用‘su’或‘sudo’方法访问这类文件；参见“远程文件”）。否则，你可以通过屏幕上出现的新文本以及模式行中显示的缓冲区名，判断‘C-x C-f’已成功完成（参见“模式行”）。Emacs 通常从文件名构造缓冲区名，省略目录名。例如，名为‘/usr/rms/emacs.tex’的文件在名为‘emacs.tex’的缓冲区中访问。如果已存在同名缓冲区，Emacs 会构造一个唯一的名字；通常的方法是根据目录名添加后缀（如‘<rms>’、‘<tmp>’等），但你也可以选择其他方法。参见“使缓冲区名称唯一”。

要创建新文件，只需用同一命令‘C-x C-f’访问它。Emacs 在回显区显示‘(New file)’，但在其他方面表现得就像你访问了一个已存在的空文件。

访问文件之后，你用编辑命令所做的改动都在 Emacs 缓冲区中进行。在你“保存”缓冲区（参见“保存文件”）之前，它们不会在所访问的文件中生效。若缓冲区含有尚未保存的改动，我们说该缓冲区“已修改”（modified）。这意味着如果不保存该缓冲区，某些改动将会丢失。模式行在左边缘附近显示两个星号，表示缓冲区已修改。

若你访问的文件已在 Emacs 中，‘C-x C-f’会切换到已有缓冲区，而不是再制作一份副本。在此之前，它会检查该文件自你上次访问或保存以来是否已改变。若文件已改变，Emacs 会提议重新读取它。

若你试图访问的文件大于‘large-file-warning-threshold’（默认值为 10000000，约合 10 兆字节），Emacs 会先请你确认。你可以回答‘y’继续访问该文件，或回答‘l’以字面方式访问该文件（见下文）。以字面方式访问大文件可加快这类文件的浏览和编辑，因为各种可能开销很大的功能会被关闭。但要注意，Emacs 无法访问大于 Emacs 缓冲区大小上限的文件，该上限受 Emacs 可分配的内存量和 Emacs 能表示的整数所限制（参见“多缓冲区”）。如果你尝试这样做，Emacs 会显示一条错误消息，说明已超出最大缓冲区大小。

若你访问的文件，其主模式（参见“主模式”）使用 tree-sitter 解析库，当文件的字节大小大于变量 ‘treesit-max-buffer-size’ 的值时，Emacs 会显示警告。该变量在 64 位 Emacs 上的默认值为 40 兆字节，在 32 位 Emacs 上为 15 兆字节。这样可通过阻止在这类大缓冲区中激活基于 tree-sitter 的主模式，避免 Emacs 耗尽内存的危险，因为典型的 tree-sitter 解析器所需内存约为其所解析文本的 10 倍。

若你指定的文件名含有 shell 风格的通配符字符，Emacs 会访问所有匹配的文件。（在不区分大小写的文件系统上，Emacs 匹配通配符时忽略字母大小写。）通配符包括 ‘?’、‘*’ 和 ‘[...]’ 序列。要在迷你缓冲区中的文件名里输入通配符 ‘?’，需要输入 ‘C-q ?’。关于如何访问名字中确实含有通配符字符的文件，参见“引用的文件名”。你可以通过定制 ‘find-file-wildcards’ 来禁用通配符功能。

若你请求访问的文件已在某缓冲区中访问，但该文件在外部已改变，Emacs 通常会询问你是否要从磁盘重新读取该文件。但若你把 ‘query-about-changed-file’ 设为 ‘nil’，Emacs 不会询问你，而只是显示改动之前的缓冲区内容，并在回显区显示一条消息，告知你如何从文件还原缓冲区。

若你因输错文件名而无意中访问了错误的文件，输入 ‘C-x C-v’（‘find-alternate-file’）来访问你真正想要的文件。‘C-x C-v’ 类似于 ‘C-x C-f’，但它会杀掉当前缓冲区（若该缓冲区已修改，会先提议保存它）。当 ‘C-x C-v’ 读取要访问的文件名时，它会把整个默认文件名插入缓冲区，光标点紧跟在目录部分之后；如果你在输入名字时犯了小错，这很方便。

若你访问的文件实际上是一个目录，Emacs 会调用 Dired，即 Emacs 的目录浏览器。参见“Dired 目录编辑器”。你可以通过把变量 ‘find-file-run-dired’ 设为 ‘nil’ 来禁用该行为；这种情况下，试图访问目录会报错。

实际上是其他文件之集合的文件，即“文件归档”（file archives），会在特殊模式中访问，这些模式会启用类似 Dired 的环境，以便对归档成员进行操作。关于这些功能的更多内容，参见“文件归档”。

若你访问的文件操作系统不允许你修改，或被标记为只读，Emacs 也会把缓冲区设为只读，以免你继续做出之后难以保存的改动。你可以用 ‘C-x C-q’（‘read-only-mode’）使缓冲区可写。参见“其他缓冲区操作”。

若你想以只读方式访问文件，以保护自己不会意外输入改动，请改用命令 ‘C-x C-r’（‘find-file-read-only’）而不是 ‘C-x C-f’ 来访问它。

‘C-x 4 f’（‘find-file-other-window’）与 ‘C-x C-f’ 类似，只是包含指定文件的缓冲区在另一个窗口中被选中。在 ‘C-x 4 f’ 之前选中的窗口继续显示它原先显示的缓冲区。若在只显示一个窗口时使用该命令，该窗口会被分成两个（参见“‘display-buffer’ 的工作方式”），一个窗口显示和以前相同的缓冲区，另一个显示新请求的文件。参见“多窗口”。

‘C-x 5 f’（‘find-file-other-frame’）类似，但它打开一个新框架，或选择任何显示指定文件的已有框架。参见“框架与图形显示”。

在图形显示上，还有两种访问文件的额外方法。首先，当 Emacs 以合适的 GUI 工具包构建时，用鼠标调用的命令（通过点击菜单栏或工具栏）会使用该工具包的标准文件选择对话框，而不是在迷你缓冲区中提示输入文件名。在 GNU/Linux 和 Unix 平台上，Emacs 在以 GTK+、LessTif 和 Motif 工具包构建时如此；在 MS-Windows 和 Mac 上，GUI 版本默认如此。关于如何定制这一点，参见“使用对话框”。

其次，Emacs 支持拖放：把文件拖放到普通 Emacs 窗口中，会用该窗口访问该文件。作为例外，把文件拖放到显示 Dired 缓冲区的窗口中，会把该文件移动或复制到所显示的目录中。细节参见“拖放” 和 参见“Dired 的其他功能”。

在文本模式终端上，以及 Emacs 未以 GUI 工具包构建的图形显示上，你可以通过菜单栏的 'File' 菜单访问文件，该菜单中有 'Visit New File' 和 'Open File' 项。

每次访问文件时，Emacs 都会自动扫描其内容，以检测它使用何种字符编码和行尾约定，并把它们转换为缓冲区中 Emacs 内部的编码和行尾约定。保存缓冲区时，Emacs 执行相反的转换，以文件原有的编码和行尾约定把它写回磁盘。参见"编码系统"。

若你希望把文件当作一串 ASCII 字符来编辑，不做特殊编码或转换，请使用 'M-x find-file-literally' 命令。它像 'C-x C-f' 一样访问文件，但不做格式转换（参见《Emacs Lisp 参考手册》的"Format Conversion"）、字符编码转换（参见"编码系统"）或自动解压缩（参见"访问压缩文件"），也不会因为 'require-final-newline' 而添加结尾换行（参见"自定义文件的保存"）。若你已用通常（非字面）方式访问过同一文件，该命令会询问你是否改为以字面方式访问它。

文件有时（松散地）与其他文件关联，你可以把这类文件称为"兄弟文件"（sibling files）。例如，编辑 C 文件时，如果你有一个名为 '"foo.c"' 的文件，通常还会有一个名为 '"foo.h"' 的文件，后者可以是它的兄弟文件。或者你可能有同一文件的不同版本，例如 '"src/emacs/emacs-27/lisp/allout.el"' 和 '"src/emacs/emacs-28/lisp/allout.el"' 可能被视为兄弟。Emacs 提供 'find-sibling-file' 命令在兄弟文件之间跳转，但无法猜测用户希望把哪些文件视为兄弟，所以 Emacs 让你通过修改 'find-sibling-rules' 用户选项自由配置这一点。这是一个匹配/展开元素的列表。

例如，要实现 '".c"' 到 '".h"' 的映射，你可以写：

```
(setq find-sibling-rules
      '(("\\([^/]+\\)\\.c\\'" "\\1.h")))
```

（'ff-find-related-file' 提供类似功能，尤其针对 C 文件，参见"C 模式的其他命令"。）

或者，若你想把 '"src/emacs/DIR/file-name"' 下的所有文件视为其他 DIR 的兄弟，可以写：

```
(setq find-sibling-rules
      '(("src/emacs/[^/]+/\\(.*\\)\\'" "src/emacs/.*/\\1")))
```

如你所见，这是一个 '(MATCH EXPANSION...)' 元素的列表。MATCH 是匹配所访问文件名的正则表达式，每个 EXPANSION 可以用 '\\1' 等引用匹配组。然后，得到的展开字符串会应用到文件系统，看是否有文件匹配该展开（按正则表达式解释）。

有两个特殊的钩子变量允许扩展改变访问文件的操作。访问不存在的文件会运行 'find-file-not-found-functions' 中的函数；该变量保存一个函数列表，它们逐个被调用（不带参数），直到其中一个返回非 'nil'。这不是普通钩子，其名字以 '-functions' 而非 '-hook' 结尾，正说明了这一点。

任何文件的成功访问，无论文件是否存在，都会不带参数调用 'find-file-hook' 中的函数。该变量是普通钩子。对于不存在的文件，先运行 'find-file-not-found-functions'。参见"钩子"。

有几种方法可以自动指定编辑该文件的主模式（参见"选择文件模式"），以及指定为该文件定义的局部变量（参见"文件中的局部变量"）。

20.3 保存文件

在 Emacs 中“保存”缓冲区，指把其内容写回该缓冲区所访问的文件。

- Interlocking：Emacs 如何防止两个用户同时编辑同一个文件。
- Shadowing（File Shadowing）：自动把文件复制到“影子”文件。
- Time Stamps：Emacs 可以在保存文件时更新时间戳。

20.3.1 保存文件的命令

这些是与保存和写出文件相关的命令。

- ‘C-x C-s’：把当前缓冲区保存到其文件（‘save-buffer’）。
- ‘C-x s’：保存任意或全部缓冲区到它们的文件（‘save-some-buffers’）。
- ‘M-~’：忘记当前缓冲区已更改（‘not-modified’）。带前缀参数（‘C-u’）时，把当前缓冲区标记为已更改。
- ‘C-x C-w’：以指定的文件名保存当前缓冲区（‘write-file’）。
- ‘M-x set-visited-file-name’：更改当前缓冲区将被保存到的文件名。
- ‘M-x rename-visited-file’：与 ‘M-x set-visited-file-name’ 相同，但同时重命名缓冲区所访问的文件（如果有）。

当你希望保存文件并使改动永久生效时，输入 ‘C-x C-s’（‘save-buffer’）。保存完成后，‘C-x C-s’ 显示类似这样的消息：

```
Wrote /u/rms/gnu/gnu.tasks
```

若当前缓冲区未被修改（自缓冲区创建或上次保存以来没有做出任何改动），则实际不会执行保存，因为那不会产生任何效果。相反，‘C-x C-s’ 会在回显区显示类似这样的消息：

```
(No changes need to be saved)
```

带前缀参数时，‘C-u C-x C-s’，Emacs 还会把缓冲区标记为在下次保存时进行备份。参见“备份文件”。

命令 ‘C-x s’（‘save-some-buffers’）会提议保存任意或全部已修改的缓冲区。它对每个缓冲区询问你要怎么做。可能的回答类似于 ‘query-replace’ 的回答：

- ‘y’ ‘<SPC>’：保存这个缓冲区，并继续询问其余缓冲区。
- ‘n’ ‘<DEL>’：不保存这个缓冲区，但继续询问其余缓冲区。
- ‘!’：保存这个缓冲区以及其余所有缓冲区，不再询问。
- ‘q’ ‘<RET>’：终止 ‘save-some-buffers’，不再保存任何缓冲区。
- ‘.’：保存这个缓冲区，然后退出 ‘save-some-buffers’，甚至不询问其他缓冲区。
- ‘M-~’：不保存这个缓冲区，并把它标记为未修改。这类似 ‘not-modified’ 命令（参见“保存文件的命令”）。
- ‘C-r’：查看当前正被询问的缓冲区。退出 View 模式后，你回到 ‘save-some-buffers’，它会再次提问。
- ‘C-f’：退出 ‘save-some-buffers’ 并访问当前正被询问的缓冲区。

- ‘d’：把缓冲区与其对应的文件做差异比较，以便你看到将要保存哪些改动。这会调用命令‘diff-buffer-with-file’（参见“比较文件”）。
- ‘C-h’：显示关于这些选项的帮助消息。

你可以定制‘save-some-buffers-default-predicate’的值，以控制 Emacs 会询问哪些缓冲区。

退出 Emacs 的键序列‘C-x C-c’会调用‘save-some-buffers’，因此会问同样的问题。

若你改动了一个缓冲区却不想保存这些改动，应当采取某种措施加以防止。否则，每次你使用‘C-x s’或‘C-x C-c’时，都可能误保存该缓冲区。你可以做的一件事是输入‘M-~’（‘not-modified’），它会清除缓冲区已修改的指示。这样做了之后，任何保存命令都不会认为该缓冲区需要保存。（‘~’常用作表示“非”的数学符号；因此‘M-~’就是加了 Meta 的“非”。）或者，你可以通过再次从文件读取文本，取消自访问或保存该文件以来所做的所有改动。这称为“还原”（reverting）。参见“还原缓冲区”。（你也可以重复撤销命令‘C-x u’直到撤销所有改动；但还原更方便。）

‘M-x set-visited-file-name’更改当前缓冲区所访问文件的名称。它用迷你缓冲区读取新文件名。然后它把缓冲区标记为访问该文件名，并相应地更改缓冲区名。‘set-visited-file-name’不会把缓冲区保存到新访问的文件中；它只是更改 Emacs 内部的记录，以备你以后确实保存。它还会把缓冲区标记为已修改，以便该缓冲区中的‘C-x C-s’会保存。

若你希望把缓冲区标记为访问另一个文件并立即保存，请使用‘C-x C-w’（‘write-file’）。这等价于先执行‘set-visited-file-name’再执行‘C-x C-s’，只是若文件已存在，‘C-x C-w’会要求确认。对未访问任何文件的缓冲区使用‘C-x C-s’与‘C-x C-w’效果相同；即它会读取一个文件名，把缓冲区标记为访问该文件，并保存到那里。未访问文件的缓冲区中的默认文件名，由缓冲区名与缓冲区的默认目录组合而成（参见“文件名”）。

若新文件名暗示某个主模式，则‘C-x C-w’在大多数情况下会切换到该主模式。命令‘set-visited-file-name’也会这样做。参见“选择文件模式”。

若你希望把当前缓冲区保存到另一个文件而不访问该文件，请使用‘mark-whole-buffer’（‘C-x h’），然后‘M-x write-region’（参见“其他文件操作”）。

若 Emacs 正要保存文件时发现磁盘上最新版本的日期与 Emacs 上次读取或写入所记录的不一致，它会把这个情况通知你，因为这可能表明存在同时编辑导致的问题，需要你立即处理。参见“防止同时编辑”

20.3.2 备份文件

在大多数操作系统上，重写文件会自动销毁该文件原先内容的一切记录。因此，从 Emacs 保存文件会丢弃文件的旧内容——或者说本来会如此，只是 Emacs 会在真正保存之前，小心地把旧内容复制到另一个称为“备份”（backup）文件的文件里。

Emacs 只在文件第一次从访问它的缓冲区中被保存时，才为该文件创建备份。此后无论你保存该文件多少次，其备份都保持不变。不过，如果你杀掉缓冲区然后再次访问该文件，就会创建新的备份文件。

对大多数文件，变量‘make-backup-files’决定是否创建备份文件。在大多数操作系统上，其默认值为‘t’，因此 Emacs 确实会写备份文件。

对于由版本控制系统管理的文件（参见“版本控制”），变量‘vc-make-backup-files’决定是否创建备份文件。默认情况下它为‘nil’，因为当你把所有以前的版本都保存在版本控制系统中时，备份文件是多余的。参见“通用

选项”。

你可以选择让 Emacs 为每个文件只保留一个备份，或者为你编辑的每个文件生成一系列带编号的备份文件。参见“单个备份与编号备份”。

变量 ‘backup-enable-predicate’ 的默认值会阻止在为临时文件所用的目录（由 ‘temporary-file-directory’ 或 ‘small-temporary-file-directory’ 指定）中为文件写备份文件。

你可以显式告诉 Emacs 从缓冲区再创建一个备份文件，即使该缓冲区此前已保存过。若你用 ‘C-u C-x C-s’ 保存缓冲区，那么再次保存该缓冲区时，这样保存的版本会被做成备份文件。‘C-u C-u C-x C-s’ 保存缓冲区，但会先把之前的文件内容做成新的备份文件。‘C-u C-u C-u C-x C-s’ 做两件事：从之前的内容创建备份，并安排在你再次保存时用新保存的内容再创建一个备份。

你可以自定义变量 ‘backup-directory-alist’，指定与某些模式匹配的文件应当备份到特定的目录中。一种典型的用法是添加元素 ‘("." . DIR)’，把所有备份都放到绝对名称为 DIR 的目录里。Emacs 会修改备份文件的名字，以避免来自不同目录的同名文件之间发生冲突。另一种做法是添加 ‘("." . ".~")’，把备份放到原文件所在目录的不可见子目录 ‘.~’ 中。必要时 Emacs 会创建该目录以便存放备份。

20.3.2.1 单个备份与编号备份

当 Emacs 创建备份文件时，其名称通常是在被编辑的文件名后追加 ‘~’；因此 ‘eval.c’ 的备份文件就是 ‘eval.c~’。

如果访问控制使 Emacs 无法以通常的名称写备份文件，它会把备份写成 ‘~/.emacs.d/%backup%~’。这样的文件只能存在一个，因此只有最近创建的那一份备份可用。

Emacs 也能创建“编号备份文件”。编号备份文件名在原始文件名之后包含 ‘.~’、编号以及另一个 ‘~’。因此 ‘eval.c’ 的备份文件会被称为 ‘eval.c.~1~’、‘eval.c.~2~’，如此下去，直至 ‘eval.c.~259~’ 之类的名字乃至更大的编号。

变量 ‘version-control’ 决定创建单个备份文件还是多个编号备份文件。其可能取值为：

- ‘nil’：对已有编号备份的文件创建编号备份，否则创建单个备份。这是默认值。
- ‘t’：创建编号备份。
- ‘never’：从不创建编号备份，总是创建单个备份。

设置此变量的通常方式是通过初始化文件或 customization 缓冲区进行全局设置。不过你也可以在单个缓冲区中局部设置 ‘version-control’，以控制该缓冲区所对应文件如何创建备份（参见“局部变量”）。某些模式（例如 Rmail 模式）会设置此变量。你也可以让 Emacs 在访问某个给定文件时局部设置 ‘version-control’（参见“文件中的局部变量”）。

如果你设置了环境变量 ‘VERSION_CONTROL’，以告诉各种 GNU 工具如何处理备份文件，那么 Emacs 启动时也会遵循该环境变量，相应地设置 Lisp 变量 ‘version-control’。如果环境变量的值为 ‘t’ 或 ‘numbered’，则 ‘version-control’ 变为 ‘t’；如果值为 ‘nil’ 或 ‘existing’，则 ‘version-control’ 变为 ‘nil’；如果值为 ‘never’ 或 ‘simple’，则 ‘version-control’ 变为 ‘never’。

如果你把变量 ‘make-backup-file-name-function’ 设为一个合适的 Lisp 函数，就可以覆盖 Emacs 构造备份文件名的通常方式。

20.3.2.2 自动删除备份

为防止过多占用磁盘空间，Emacs 可以自动删除编号备份版本。一般而言，Emacs 保留最早的几个备份和最近的几个备份，删除介于其间的所有备份。每次创建新备份时都会进行这种删除。

变量 'kept-old-versions' 和 'kept-new-versions' 控制这种删除。它们的值分别是每次创建新备份时要保留的最旧（编号最小）备份的数目和最新（编号最大）备份的数目。中间的那些备份（除去最旧和最新的）就是多余的中间版本——这些备份会被删除。这些变量的值在需要删除多余版本时使用，也就是在创建了一个新备份版本之后；新创建的备份计入 'kept-new-versions' 的数目中。默认情况下，两个变量的值都是 2。

如果 'delete-old-versions' 为 't'，Emacs 会静默地删除多余的备份文件。如果它为 'nil'（默认值），Emacs 会询问你是否应当删除多余的备份版本。如果它具有其他任何值，Emacs 就从不会自动删除备份。

Dired 的 '.'（句点）命令也可用于删除旧版本。参见"一次为多个文件加旗标"。

20.3.2.3 复制与重命名

备份文件可以通过复制旧文件或重命名旧文件来创建。当旧文件有多个名字（硬链接）时，这两种方式会有所不同。如果旧文件被重命名为备份文件，那么其他名字就变成了备份文件的名字。如果改为复制旧文件，那么其他名字仍然是你正在编辑的文件的名字，而通过这些名字访问到的内容将是新内容。

创建备份文件的方法还可能影响文件的所有者和组。如果使用复制，这两者不会改变。如果使用重命名，你将成为文件的所有者，文件的组则变为默认组（不同操作系统对组的默认设置不同）。

选择重命名还是复制，按如下规则决定：

- 如果变量 'backup-by-copying' 非 'nil'（默认为 'nil'），使用复制。
- 否则，如果变量 'backup-by-copying-when-linked' 非 'nil'（默认为 'nil'），并且文件有多个名字，使用复制。
- 否则，如果变量 'backup-by-copying-when-mismatch' 非 'nil'（默认为 't'），并且重命名会改变文件的所有者或组，使用复制。

如果你把 'backup-by-copying-when-mismatch' 改为 'nil'，Emacs 会检查文件所有者的数字用户 ID 和文件所属组的数字组 ID。如果其中任何一个不大于 'backup-by-copying-when-privileged-mismatch'，那么它的行为就好像 'backup-by-copying-when-mismatch' 仍然是非 'nil' 一样。

- 否则，默认选择重命名。

当文件由版本控制系统管理时（参见"版本控制"），Emacs 通常不会为该文件创建常规备份。但"提交"（committing，亦称"检入"checking in，参见"版本控制的概念"）文件的新版本在某些方面与创建备份相似。一个不幸的相似之处是，这些操作通常会断开硬链接，使你所访问的文件名与同一文件的任何其他名字脱钩。这与 Emacs 无关——是版本控制系统造成的。

某些文件存储服务支持"文件版本化"（file versioning）：它们记录文件先前版本的历史，并允许回退到那些先前的版本。如果你想在用 Emacs 编辑由这类服务托管的文件时也能这样做，请把 'backup-by-copying' 自定义为非 'nil' 值。

在编辑珍贵文件时，复制旧文件作为备份也很有用，因为这样可以确保：如果在做备份与保存你的修改之间出了什么差错，旧文件仍保留原来的名字。或者，你也可以把 'file-precious-flag' 自定义为非 'nil' 值，这既意味着以复制方式做备份，也能在保存修改时防止 I/O 错误。

20.3.3 自定义文件的保存

如果变量 'require-final-newline' 的值为 't'，那么在保存或写入文件时，如果末尾还没有换行符，Emacs 会静默地在末尾放一个。如果值为 'visit'，Emacs 在访问任何缺少末尾换行符的文件之后，会立即给它加上换行符。（这会把缓冲区标记为已修改，你可以撤销该操作。）如果值为 'visit-save'，Emacs 在访问和保存时都会添加这样的换行符。如果值为 'nil'，Emacs 不对文件末尾做任何改动；其他任何非 'nil' 值都表示 Emacs 会询问你是否要添加换行符。默认值为 'nil'。

有些主模式是为特定类型的文件设计的，这类文件的末尾总是应当有换行符。这样的主模式会把变量 'require-final-newline' 设为 'mode-require-final-newline' 的值，后者默认为 't'。通过设置后一个变量，你可以控制这些模式如何处理文件末尾的换行符。

如果此选项非 'nil'，而你正通过符号链接访问某个文件，那么当 'file-precious-flag' 的值非 'nil' 时，Emacs 在保存缓冲区时会断开该符号链接，并把缓冲区写入一个与符号链接同名的文件（参见《Emacs Lisp 参考手册》的"Saving Buffers"）。如果你希望 Emacs 在这些情况下把缓冲区保存到符号链接所指向的文件（从而保留该链接），请把变量 'file-preserve-symlinks-on-save' 自定义为 't'。

通常，当程序写文件时，操作系统会先把文件数据短暂缓存到主存中，然后再把数据提交到二级存储。虽然这可以极大地提高性能，但如果系统在提交缓存之前断电，就有丢失数据的风险；而且在某些平台上，其他进程可能不会立即察觉到文件的变化。

为减轻这种风险，Emacs 可以在保存文件后调用 'fsync' 系统调用。使用 'fsync' 并不能消除数据丢失或通知迟缓的风险，部分原因是许多系统并不正确支持 'fsync'，部分原因是 Emacs 的保存文件过程通常还依赖于目录更新，而即使 'fsync' 正常工作，目录更新也可能在崩溃中丢失。

变量 'write-region-inhibit-fsync' 控制 Emacs 在保存文件后是否调用 'fsync'。该变量的默认值为 't'。

Emacs 在写自动保存文件时从不使用 'fsync'，因为这些文件本来就可能丢失数据。

20.3.4 防止同时编辑

当两个用户访问同一个文件、都做了修改、然后都保存时，就发生了同时编辑。如果没有人被告知这种情况正在发生，那么先保存的用户后来会发现自己的修改丢失了。

在某些系统上，当第二个用户开始修改文件时，Emacs 会立即察觉并马上发出警告。在所有系统上，Emacs 都会在你保存文件时进行检查，如果你将要覆盖另一个用户的修改，它就会发出警告。你可以采取适当的补救措施而不是保存文件，从而避免丢失另一个用户的工作。

当你在访问某个文件的 Emacs 缓冲区中做第一次修改时，Emacs 会记录该文件已被你"锁定"。（它的做法是在同一目录中创建一个名称特殊、内容也特殊的符号链接[1]。更多细节参见《Emacs Lisp 参考手册》的"File Locks"。）你在保存修改时，Emacs 会移除该锁。其思想是：只要访问某文件的 Emacs 缓冲区有未保存的修改，该文件就是被锁定的。

你可以通过把变量‘create-lockfiles’设为‘nil’来阻止创建锁文件。*注意：*这样做会失去此特性带来的好处。你还可以使用‘lock-file-name-transforms’变量来控制锁文件写到哪里。

如果你在访问的文件已被他人锁定时开始修改缓冲区，这就构成一次“冲突”。当 Emacs 检测到冲突时，它会调用 Lisp 函数‘ask-user-about-lock’询问你该怎么办。你可以为实现自定义而重新定义这个函数。该函数的标准定义会向你提一个问题，并接受三种可能的回答：

- ‘s’：夺取锁。原本正在修改该文件的人失去锁，由你获得锁。
- ‘p’：继续。尽管文件已被他人锁定，仍继续编辑该文件。
- ‘q’：退出。这会引发错误（‘file-locked’），缓冲区内容保持不变——你试图做的修改并不会真正发生。

如果 Emacs 或操作系统崩溃，可能会留下过期的锁文件，因此你偶尔会收到关于虚假冲突的警告。当你确定冲突是虚假的时，只需用‘p’告诉 Emacs 照常进行。

注意，锁定是基于文件名进行的；如果文件有多个名字，Emacs 不会阻止两个用户以不同的名字同时编辑它。

在某些情况下无法写入锁文件，例如 Emacs 缺少系统权限，或由于其他原因无法创建锁文件。在这些情况下，Emacs 仍能在你尝试保存文件时通过检查文件的最后修改日期来检测冲突。如果文件自 Emacs 上次访问或保存以来已经改变，就说明修改是以某种别的方式做出的，若 Emacs 继续保存就会丢失这些修改。于是 Emacs 会显示一条警告消息，并在保存前请求确认；回答‘yes’即保存，回答‘no’或‘C-g’则取消保存。

如果你被告知同时编辑已经发生，比较缓冲区与其文件的一种方法是使用‘M-x diff-buffer-with-file’命令。参见“比较文件”。

你可以通过把变量‘remote-file-name-inhibit-locks’设为‘t’来阻止创建远程锁文件。

以交互方式调用的次模式‘lock-file-mode’会切换当前缓冲区中‘create-lockfiles’的局部值。

> 注：如果你的文件系统不支持符号链接，则使用普通文件。

20.3.5 影子文件

你可以安排把某些文件完全相同的“影子”副本保存在不止一个地方——可能是在不同的机器上。要做到这一点，首先必须建立一个“影子文件组”（shadow file group），它是一组在若干站点之间共享的同名文件。文件组是永久的，对以后的 Emacs 会话以及当前会话都有效。组建立之后，每次退出 Emacs 时，它都会把你编辑过的文件复制到组内的其他文件处。你也可以不退出 Emacs 就完成复制，只需键入‘M-x shadow-copy-files’。

“影子集群”（shadow cluster）是一组共享目录的主机，因此向其中一台复制或从其中一台复制，就足以更新所有这些主机上的文件。每个影子集群都有一个名字，并指定一台主主机（我们向其复制文件的那台）的网络地址，以及一个匹配该集群中所有其他主机名的主正则表达式。你可以用‘M-x shadow-define-cluster’定义影子集群。

- ‘M-x shadow-initialize’：设置文件影子化。
- ‘M-x shadow-define-literal-group’：声明在多个站点之间共享单个文件。
- ‘M-x shadow-define-regexp-group’：使匹配一组文件中每个文件的那些文件都在各主机之间共享。
- ‘M-x shadow-define-cluster <RET> NAME <RET>’：定义名为 NAME 的影子文件集群。
- ‘M-x shadow-copy-files’：复制所有待处理的影子文件。

- ‘M-x shadow-cancel’：取消对某些文件进行影子化的指示。

要建立影子文件组，请使用 ‘M-x shadow-define-literal-group’ 或 ‘M-x shadow-define-regexp-group’。更多信息请参见它们的文档字符串。

在把文件复制到其影子副本之前，Emacs 会请求确认。你可以回答“no”来跳过这一次对该文件的复制。如果你想永久取消对某个文件的影子化，请用 ‘M-x shadow-cancel’ 删除或修改影子文件组。

文件影子化在 MS Windows 上不可用。

20.3.6 自动更新时间戳

在文件文本中放置时间戳，可以保证文件被写入的时间得以保留，即使文件被复制，或以丢失文件系统修改时间的方式被转换。时间戳有时也称为日期戳或最后修改时间。你可以安排每次保存文件时自动更新文件中的时间戳。

设置自动时间戳有两个步骤。首先，文件需要一个时间戳模板。默认情况下，模板出现在前八行中的某处，形如：

```
Time-stamp: <>
```

或者（随你选择）形如：

```
Time-stamp: " "
```

有了这个模板，你就可以用命令 ‘M-x time-stamp’ 立即更新当前缓冲区的时间戳一次。Emacs 编辑器会查找模板；如果找到模板，Emacs 就会在尖括号或引号之间写入当前的日期、时间、作者和/或其他信息。第一次生成时间戳之后，该行可能形如：

```
Time-stamp: <1993-07-06 11:05:14 terryg>
```

其次，配置你的 Emacs，使其在每次保存文件时运行 ‘time-stamp’，办法是把 ‘time-stamp’ 加入 ‘before-save-hook’（参见“钩子”）。这一步有两种做法：你可以

- 使用 ‘M-x customize-option’（参见“自定义特定项目”）来定制选项 ‘before-save-hook’，或者
- 编辑你的初始化文件（参见“Emacs 初始化文件”），添加下面这一行：

```
(add-hook 'before-save-hook 'time-stamp)
```

现在，每次保存文件时，Emacs 都会查找时间戳。如果缓冲区中没有模板，‘time-stamp’ 什么也不做；任何确实带有时间戳的文件都会自动保持其时间戳最新。

20.3.6.1 自定义时间戳

要自定义某个特定文件中的时间戳，请在该文件的局部变量列表中设置变量 ‘time-stamp-pattern’（参见“指定文件变量”）。你可以用 ‘time-stamp-pattern’ 改变 ‘time-stamp’ 用来识别模板所匹配的模式，以及文件中查找该模式的位置；详情请见该变量的内置文档（用 ‘C-h v’ 查看，参见“按命令或变量名获取帮助”）。

举个简单的例子，假设你想让一份手稿写明出版的年份和城市。你希望在你做修订时年份会更新。你可以在文件开头附近放这样一行：

```
publishing_year_and_city = "Published nnnn in Boston, Mass.";
```

并在同一文件末尾放下面这段注释，告诉 'time-stamp' 如何识别和更新那个自定义模板：

```
// Local variables:
// time-stamp-pattern: "Published %Y in Boston"
// End:
```

这个模式表示：时间戳起点之前的文本是"Published "，终点之后的文本是" in Boston"。如果 'time-stamp' 在前八行中的某一行里同时找到起点和终点，那么两者之间的内容就会按格式所指定的方式更新，本例中的格式是 '%Y'。由于 '%Y' 请求的是年份，结果可能形如：

```
publishing_year_and_city = "Published 2025 in Boston, Mass.";
```

通过指定 '%Y' 格式，我们恰好只替换年份；默认格式的其他部分（日期、时间和作者）不属于这个示例模式，因此不会出现在结果中。

修改 'time-stamp-pattern'（或任何文件局部变量）的值之后，请键入 'M-x normal-mode'，以便 Emacs 察觉这一改动。

下面是另一个例子，其中时间戳被插入 HTML 文档的最后一段。由于这个模板位于文档末尾，而不在前八行中，'time-stamp-pattern' 以 '-10/' 开头，告诉 'time-stamp' 查看最后 10 行。'%%' 请求使用默认格式（由 'time-stamp-format' 指定）。

```
...
<p>Last modified: </p>
</body>
</html>
<!--
Local variables:
time-stamp-pattern: "-10/Last modified: %%</p>$"
End:
-->
```

默认情况下，时间戳按你的 locale 设置（参见"环境变量"）和时区（参见《Emacs Lisp 参考手册》的"Time of Day"）格式化。设置 'time-stamp-time-zone' 可以覆盖所使用的时区。关于格式化的细节以及其他影响格式化的变量，请参见变量 'time-stamp-format' 的内置文档。

20.3.6.2 为单个文件强制启用时间戳

如果你正在处理一份有多位作者的文件，而无法确定其他作者是否已在其 Emacs 初始化文件中全局启用了时间戳，你可以通过在该文件的局部变量列表中把 'time-stamp' 加入该缓冲区的 'before-save-hook'，强制为这个特定文件启用时间戳。下面扩展了前面的一个例子：

```
// Local variables:
// eval: (add-hook 'before-save-hook 'time-stamp nil t)
// time-stamp-pattern: "Published %Y in Boston"
```

```
// End:
```

尽管这个例子把两者放在一起设置，但如果你喜欢默认的模式，也可以只使用 'eval' 而不设置 'time-stamp-pattern'。

这里传给 'add-hook' 的额外参数 'nil' 和 't'，是让所添加的钩子只影响本缓冲区所必需的。

20.4 还原缓冲区

如果你对某个访问文件的缓冲区做了大量修改，然后又改变主意，你可以"还原"（revert）这些修改，回到文件的已保存版本。为此请键入 'C-x x g'。由于无意中还原可能会丢失大量工作，如果缓冲区已被修改，Emacs 会先请求确认。

'revert-buffer' 命令会尽量把光标点放在这样的位置：如果文件只被轻微编辑过，你会大致回到与之前相同的文本位置。但如果你做了重大改动，光标点可能落在完全不同的位置。

还原会把缓冲区标记为未修改。不过，它会把还原带来的改动作为单独一次修改加入缓冲区的撤销历史（参见"撤销"）。因此，还原之后，如果你碰巧又改变主意，可以键入 'C-/' 或其别名把被还原的改动恢复回来。

要更保守地还原缓冲区，可以使用命令 'revert-buffer-with-fine-grain'。该命令的作用与 'revert-buffer' 类似，但它尽可能采用非破坏性的方式，努力保留缓冲区中的所有标记、属性和覆盖层。由于在你做了大量修改时这种还原方式可能非常慢，你可以修改变量 'revert-buffer-with-fine-grain-max-seconds' 来指定以这种方式替换缓冲区内容最多应花费的秒数。注意，这并不保证 'revert-buffer-with-fine-grain' 的整个执行过程不会超过这个时间。

有些与文件无关的缓冲区（例如 Dired 缓冲区）也可以还原。对它们来说，还原意味着重新计算其内容。用 'C-x b' 显式创建的缓冲区无法还原；如果你尝试还原，'revert-buffer' 会报错。

当你编辑一个会自动且频繁变化的文件时——例如一个持续运行的进程的输出日志——让 Emacs 在不询问你的情况下还原文件可能很有用。要请求这种行为，请把变量 'revert-without-query' 设为一个正则表达式列表。当某个文件名匹配其中一个正则表达式时，如果该文件已经改变，且缓冲区本身未被修改，'find-file' 和 'revert-buffer' 就会自动还原它。（如果你编辑过文本，丢弃你的改动就是不对的。）

'C-x x g' 按键绑定到 'revert-buffer-quick' 命令。它类似 'revert-buffer' 命令，但询问更少。与 'revert-buffer' 不同，如果当前缓冲区访问的是文件且缓冲区未被修改，它不会询问。它还遵循 'revert-buffer-quick-short-answers' 用户选项。如果该选项非 'nil'，则使用较短的 'y/n' 询问，而不是较长的 'yes/no' 询问。

你也可以让 Emacs 在缓冲区所访问的文件在磁盘上发生变化时自动还原缓冲区；参见"自动还原：自动保持缓冲区最新"。

注意，还原缓冲区会启用适合访问该缓冲区文件的主模式，如参见"选择文件模式"所述。因此，还原缓冲区实际启用的主模式取决于模式重映射，如果你在此期间定制了 'major-mode-remap-alist'，结果可能与原来的模式不同。

'C-x x @' 键绑定到 'tramp-revert-buffer-with-sudo' 命令。它再次访问该文件，但以超级用户（root）权限。若带 'C-u' 前缀参数调用，它会提示输入另一种 Tramp 方法，以替代默认的 'sudo'。参见《Tramp 手册》的"Ad-hoc multi-hops"

20.5 自动还原：自动保持缓冲区最新

如果缓冲区访问的文件被另一个程序改动，该缓冲区就可能与磁盘上的文件不同步。为了让它保持最新，你可以键入 'M-x auto-revert-mode' 启用自动还原模式。当它访问的文件在磁盘上发生变化时，该模式会自动还原缓冲区。要对所有文件缓冲区做同样的事，请键入 'M-x global-auto-revert-mode' 启用全局自动还原模式。

如果缓冲区有未保存的改动，或者它在磁盘上的文件已被删除或重命名，自动还原就不会还原该缓冲区。

自动还原模式的一种用途是"跟踪"（tail）诸如系统日志之类的文件，使其他程序对该文件所做的改动持续显示出来。要做到这一点，只需把光标点移到缓冲区末尾，它就会随文件内容的变化而留在那里。不过，如果你确定该文件只会在末尾增长，请改用自动还原跟踪模式（'auto-revert-tail-mode'），它对这种情况更高效。自动还原跟踪模式同样适用于远程文件。

缓冲区被自动还原时会生成一条消息。把 'auto-revert-verbose' 设为 'nil' 可以抑制它。

自动还原的各模式不检查也不还原远程文件，因为这通常太慢。把变量 'auto-revert-remote-files' 设为非 'nil' 值可以改变这一行为。

默认情况下，自动还原模式借助"文件通知"工作，即文件系统中的变化由操作系统报告给 Emacs。你可以把变量 'auto-revert-use-notify' 定制为 'nil' 值来禁用文件通知，此后 Emacs 将每五秒轮询一次文件变化。你可以通过变量 'auto-revert-interval' 改变轮询间隔。

并非所有系统都支持文件通知；在不支持的系统上，'auto-revert-use-notify' 默认为 'nil'。

默认情况下，即使使用文件通知，自动还原模式也会定期轮询文件变化。许多情况下轮询并无必要，关掉它可以仅依靠通知来省电。为此，请把变量 'auto-revert-avoid-polling' 设为非 'nil'。但在某些文件系统上通知无效，主要是类 Unix 机器上的网络文件系统，那里的文件可能被其他机器改动。对这类文件系统，轮询可能是必需的。要在 'auto-revert-avoid-polling' 为非 'nil' 时强制轮询，可把 'auto-revert-notify-exclude-dir-regexp' 设为匹配那些应排除在通知之外的文件的模式。

在 Dired 缓冲区中（参见"Dired 目录编辑器"），当缓冲区所在目录中有文件被创建或删除时，自动还原模式会刷新该缓冲区。

参见"撤销版本控制操作"，了解还原到版本控制下文件早期版本的命令。参见"版本控制与模式行"，了解访问版本控制下文件时自动还原的特殊之处。

20.5.1 自动还原非文件缓冲区

全局自动还原模式通常只还原文件缓冲区。有两种方法可以自动还原某些非文件缓冲区：在这些缓冲区中启用自动还原模式（使用 'M-x auto-revert-mode'）；以及把 'global-auto-revert-non-file-buffers' 设为非 'nil' 值。后者会为所有已实现自动还原的缓冲区类型启用它（列在下面的菜单中）。

与文件缓冲区一样，非文件缓冲区通常不应在你正在处理它们时还原，也不应在其中含有还原后可能丢失的信息时还原。因此，如果它们已被修改，就不会还原。这可能有点棘手，因为判断非文件缓冲区何时应标记为已修改，通常比文件缓冲区更难。

另一个棘手的细节是：出于效率考虑，自动还原往往不会尝试探测缓冲区中所有可能的变化，只探测重大或易于探测的变化。因此，为非文件缓冲区启用自动还原并不总能保证缓冲区中的所有信息都是最新的，也未必能使手动还原变得无用。

另一方面，某些缓冲区每隔 'auto-revert-interval' 秒就自动还原一次。（目前这只适用于缓冲区菜单。）在这种情况下，即使 'auto-revert-verbose' 为非 'nil'，自动还原在还原时也不打印任何消息。

有些非文件缓冲区可以可靠地通过对其默认目录的文件通知来更新；Dired 缓冲区就是一例。主模式可以在该缓冲区中把 'buffer-auto-revert-by-notification' 设为非 'nil' 值来表明这一点，从而使自动还原不必定期轮询。这样的通知不包括该目录中文件的变化，只包括目录本身的变化。

具体细节取决于缓冲区类型，并在相应章节中说明。

20.5.1.1 自动还原缓冲区菜单

如果启用了非文件缓冲区的自动还原，缓冲区菜单（参见"对多个缓冲区操作"）会每隔 'auto-revert-interval' 秒自动还原一次，无论是否需要。（检查是否需要所花的时间可能比实际还原还长。）

如果缓冲区菜单被不恰当地标记为已修改，只需用 'g' 手动还原它，自动还原就会恢复。不过，如果你把某些缓冲区标记为待删除或待显示，就要小心，因为还原会抹掉所有标记。添加标记会置位缓冲区的已修改标志，这一事实可防止自动还原自动抹掉这些标记。

20.5.1.2 自动还原 Dired 缓冲区

Dired 缓冲区只在该缓冲区主目录的文件列表发生变化（例如新增或删除文件）时才自动还原。当某个特定文件的信息变化（例如大小改变）或所插入的子目录变化时，它们不会自动还原。要确保所列出的_全部_信息都是最新的，你必须用 'g' 手动还原，_即使_该 Dired 缓冲区中已启用自动还原。有时你可能会觉得，修改或保存主目录下列出的文件确实会引发自动还原。这是因为改动或保存文件常常会导致目录本身发生变化，例如通过备份文件或自动保存文件。不过，这并无保证。

如果 Dired 缓冲区被标记为已修改，而没有你想保护的改动，那么大多数情况下你可以用 'g' 手动还原该缓冲区，使自动还原恢复。有一个例外。如果你给文件加了标志或标记，你可以放心地还原缓冲区。这不会抹掉标志或标记（当然，除非被标记的文件已被删除）。然而，还原之后缓冲区仍会保持已修改状态，自动还原不会恢复。这是因为，如果你给文件加了标志或标记，你可能正在处理该缓冲区，可能不希望它在毫无提示的情况下发生变化。如果希望在存在标记和标志时仍能恢复自动还原，请用 'M-~' 把缓冲区标记为未修改。不过，添加、删除或改变标记或标志会再次把它标记为已修改。

远程 Dired 缓冲区目前不会自动还原。用 shell 通配符或文件参数只列出部分文件的 Dired 缓冲区也不会。'*Find*' 和 '*Locate*' 缓冲区同样不会自动还原。

注意，自动还原 Dired 缓冲区在某些系统上可能无法令人满意地工作。

20.6 自动保存：灾难防护

Emacs 会不时把每个访问的文件自动保存到一个单独的文件中，而不改动你实际使用的文件。这称为"自动保存"。如果系统崩溃，它可以防止你损失超过有限的工作量。

当 Emacs 判定该自动保存时，它会检查每个缓冲区，如果某个缓冲区启用了自动保存，且自上次自动保存以来已被改动，就自动保存它。当变量 'auto-save-no-message' 被设为 'nil'（默认值）时，如果确有文件被自动保存，自动保存期间会在回显区显示消息 'Auto-saving...'；要禁用这些消息，请把该变量定制为非 'nil' 值。自动保存期间发生的错误会被捕获，以免干扰你正在键入的命令的执行。

- 文件：自动保存文件。在你保存该文件之前，自动保存的改动实际写入的文件。
- 控制：自动保存控制。控制自动保存的时机与频率。
- 恢复：从自动保存文件恢复文本。

20.6.1 自动保存文件

自动保存通常不写入你所访问的那些文件，因为把你不希望永久保留的改动保存下来可能非常糟糕。相反，自动保存是在另一个名为“自动保存文件”的文件中进行的，只有当你明确请求保存时（例如用 ‘C-x C-s’），才会改动所访问的文件。

通常，自动保存文件名是在所访问文件名的前后各加一个 ‘#’。因此，访问文件 ‘foo.c’ 的缓冲区会保存在文件 ‘#foo.c#’ 中。大多数不访问文件的缓冲区只在你明确请求时才自动保存；当它们被自动保存时，自动保存文件名是在缓冲区名的前后各加 ‘#’，然后在末尾加上数字和字母以保证唯一。例如，你用来撰写待发送消息的 ‘*mail*’ 缓冲区可能被自动保存在名为 ‘#*mail*#704juu’ 的文件中。除非你重新编程 Emacs 的某些部分（函数 ‘make-auto-save-file-name’ 和 ‘auto-save-file-name-p’）以做不同的事，自动保存文件名就是这样生成的。缓冲区中用于自动保存的文件名，是在该缓冲区中开启自动保存时计算出来的。

变量 ‘auto-save-file-name-transforms’ 允许对自动保存文件名有一定程度的控制。你可以指定一系列正则表达式和替换，来变换自动保存文件名。默认值把远程文件的自动保存文件（参见“远程文件”）放到本机的临时文件目录中。

当你删除大缓冲区中相当一部分文本时，该缓冲区中的自动保存会暂时关闭。这是因为，如果你是无意删除了文本，自动保存文件若包含被删掉的文本，你可能会觉得它更有用。此后要重新启用自动保存，请用 ‘C-x C-s’ 保存缓冲区，或使用 ‘C-u 1 M-x auto-save-mode’。

如果你希望自动保存写入所访问的文件而不是单独的自动保存文件，请启用全局次模式 ‘auto-save-visited-mode’。在此模式下，自动保存与显式保存相同。注意，该模式与上述 ‘auto-save’ 模式是彼此独立的；你可以同时启用两者。不过，如果某个缓冲区中 ‘auto-save’ 模式处于活动状态，且已过时的变量 ‘auto-save-visited-file-name’ 被设为非 ‘nil’ 值，该缓冲区就不受 ‘auto-save-visited-mode’ 影响。

你可以用变量 ‘auto-save-visited-interval’ 定制 ‘auto-save-visited-mode’ 中自动保存操作之间的间隔；默认是五秒。‘auto-save-interval’ 和 ‘auto-save-timeout’ 对 ‘auto-save-visited-mode’ 没有影响。关于这些变量的细节，参见“控制自动保存”。

当你在缓冲区所访问的文件中保存该缓冲区时，缓冲区的自动保存文件会被删除。（把变量 ‘delete-auto-save-files’ 设为 ‘nil’ 可以阻止这一点。）用 ‘C-x C-w’ 或 ‘set-visited-file-name’ 改变所访问的文件名，会把任何自动保存文件重命名，以与新的访问名相配。

默认情况下，杀死缓冲区不会删除该缓冲区的自动保存文件。如果 ‘kill-buffer-delete-auto-save-files’ 为非 ‘nil’，杀死一个有自动保存文件的缓冲区时，Emacs 会提示用户是否应删除该自动保存文件。（若 ‘delete-auto-save-files’ 为 ‘nil’，则不会这样。）

20.6.2 控制自动保存

每次你访问文件时，如果变量 ‘auto-save-default’ 为非 ‘nil’，就会为该文件的缓冲区开启自动保存（但在批处理模式下不会；参见“初始选项”）。该变量的默认值为 ‘t’，所以自动保存是文件缓冲区的惯常做法。要切换当前

缓冲区中的自动保存，请键入 'M-x auto-save-mode'。自动保存模式表现为缓冲区局部次模式（参见"次模式"）。

Emacs 根据你自上次自动保存以来键入的字符数定期自动保存。变量 'auto-save-interval' 指定两次自动保存之间有多少字符。默认是 300。Emacs 不接受太小的值：如果你把 'auto-save-interval' 定制为小于 20 的值，Emacs 会把它当作 20。

当你停止键入一段时间后，也会进行自动保存。默认情况下，它在空闲 30 秒后这样做（此时 Emacs 也可能执行垃圾回收；参见《Emacs Lisp 参考手册》的"Garbage Collection"）。要改变这个间隔，请定制变量 'auto-save-timeout'。如果当前缓冲区很长，实际的时间段会更长；这是一种启发式规则，意在编辑长缓冲区（其中自动保存耗费的时间相当可观）时尽量不打扰你。空闲时自动保存有两个作用：第一，如果你离开终端一段时间，它确保你的全部工作都被保存；第二，它可能避免在你实际键入时进行某些自动保存。

启用 'auto-save-visited-mode' 时，Emacs 会在空闲五秒后自动保存访问文件的缓冲区。你可以定制变量 'auto-save-visited-interval' 来改变这个空闲时间间隔。

Emacs 在发生致命错误时也会自动保存。这包括用 'kill %emacs' 之类的 shell 命令杀死 Emacs 作业，或断开电话线或网络连接。

你可以用命令 'M-x do-auto-save' 显式执行自动保存。

20.6.3 从自动保存恢复数据

你可以用命令 'M-x recover-file <RET> FILE <RET>' 利用自动保存文件的内容从数据丢失中恢复。它会访问 FILE，然后（在你确认后）从其自动保存文件 '#FILE#' 恢复内容。之后你可以用 'C-x C-s' 保存，把恢复的文本写回 FILE 本身。例如，要从自动保存文件 '#foo.c#' 恢复文件 'foo.c'，请这样做：

```
M-x recover-file <RET> foo.c <RET>
yes <RET>
C-x C-s
```

在请求确认之前，'M-x recover-file' 会显示一个目录列表，描述指定文件和自动保存文件，以便你比较它们的大小和日期。如果自动保存文件更旧，'M-x recover-file' 不会提议读取它。

当 'M-x recover-file' 请求确认时，如果你回答 'diff' 或 '='，它会显示 FILE 与其自动保存文件 '#FILE#' 之间的差异，并再次提示你确认。

如果 Emacs 或计算机崩溃，你可以用命令 'M-x recover-session' 从自动保存文件恢复你当时正在编辑的所有文件。它首先显示一个记录下来的被中断会话的列表。把光标点移到你选定的那个上，然后键入 'C-c C-c'。

接着 'recover-session' 会询问该会话期间正在编辑的每个文件，问是否要恢复该文件。如果你回答 'y'，它就调用 'recover-file'，后者按常规方式工作。它会显示原文件及其自动保存文件的日期，并再次询问是否要恢复该文件。

当 'recover-session' 完成后，你选择恢复的文件会出现在 Emacs 缓冲区中。然后你应该保存它们。只有保存它们，才会更新文件本身。

Emacs 把被中断会话的信息记录在目录 '~/.emacs.d/auto-save-list/' 下名为 '.saves-PID-HOSTNAME~' 的文件中。该目录由变量 'auto-save-list-file-prefix' 决定。如果你把 'auto-save-list-file-prefix' 设为 'nil'，就不会记录会

话以供恢复。

20.7 文件名别名

符号链接和硬链接都能让多个文件名指向同一个文件。硬链接是直接指向该文件的替代名称；所有这些名称都同样有效，其中没有哪个更受偏好。相比之下，符号链接是一种有定义的别名：当 'foo' 是指向 'bar' 的符号链接时，两个名字都可以用来指该文件，但 'bar' 是真实名称，而 'foo' 只是别名。更复杂的情况出现在符号链接指向目录时。

通常，如果你访问的文件 Emacs 已经以另一个名字访问过，Emacs 会在回显区显示一条消息，并使用访问该文件的现有缓冲区。这可能发生在支持硬链接或符号链接的系统上，或者你在会截断长文件名的系统上使用了长文件名，或者在大小写不敏感的文件系统上。把变量 'find-file-suppress-same-file-warnings' 设为非 'nil' 值可以抑制该消息。把变量 'find-file-existing-other-name' 设为 'nil' 可以完全禁用这一特性：这样如果你以两个不同的名字访问同一个文件，每个文件名都会得到各自的缓冲区。

如果变量 'find-file-visit-truename' 为非 'nil'，那么为缓冲区记录的文件名是该文件的"真实名"（truename，即把所有符号链接替换为其目标名后得到的名字），而不是你指定的名字。设置 'find-file-visit-truename' 也会隐含 'find-file-existing-other-name' 的效果。

有时，某个目录平时是通过符号链接访问的，你可能希望 Emacs 优先显示其链接名。为此，请定制 'directory-abbrev-alist'。该列表中的每个元素都应具有 '(FROM . TO)' 的形式，意思是每当 FROM 出现在目录名中时，就把 FROM 替换为 TO。FROM 字符串是正则表达式（参见"正则表达式的语法"）。它是从第一个字符开始锚定与目录名匹配的，并且应以 '\' 开头（以支持内嵌换行符的目录名，那会使 '^' 失效）。TO 字符串应是指向同一目录的普通绝对目录名。不要在 TO 字符串中用 '~' 代表主目录；Emacs 会另行进行这些替换。下面是一个示例，来自某个系统，其中 '/home/fsf' 平时通过名为 '/fsf' 的符号链接访问：

```
(("\\`/home/fsf" . "/fsf"))
```

20.8 文件目录

文件系统把文件组织成"目录"。"目录列表"是某个目录中所有文件的列表。Emacs 提供创建和删除目录的命令，以及生成简短格式（仅文件名）和详细格式（包含大小、日期和其他属性）目录列表的命令。Emacs 还包含一个名为 Dired 的目录浏览器功能，你可以用 'C-x d' 调用它；参见"Dired 目录编辑器"。

- 'C-x C-d DIR-OR-PATTERN <RET>' 显示简短的目录列表（'list-directory'）。
- 'C-u C-x C-d DIR-OR-PATTERN <RET>' 显示详细的目录列表。
- 'M-x make-directory <RET> DIRNAME <RET>' 创建名为 DIRNAME 的新目录。
- 'M-x delete-directory <RET> DIRNAME <RET>' 删除名为 DIRNAME 的目录。如果它不是空的，你将被询问是否要递归删除它。

显示目录列表的命令是 'C-x C-d'（'list-directory'）。它用迷你缓冲区读取一个文件名，该文件名可以是要列出的目录，也可以是含通配符的待列文件模式。例如，

```
C-x C-d /u2/emacs/etc <RET>
```

会列出目录‘/u2/emacs/etc’中的所有文件。下面是一个指定文件名模式的例子：

```
C-x C-d /u2/emacs/src/*.c <RET>
```

通常，‘C-x C-d’显示仅含文件名的简短目录列表。数字参数（无论其值）告诉它生成详细列表，包含大小、日期和所有者（类似‘ls -l’）。

目录列表的文本主要是在一个子进程中运行‘ls’得到的。两个 Emacs 变量控制传给‘ls’的开关：‘list-directory-brief-switches’是一个字符串，给出简短列表中使用的开关（默认为‘"-CF"’），‘list-directory-verbose-switches’是一个字符串，给出详细列表中使用的开关（默认为‘"-l"’）。

在详细目录列表中，Emacs 会添加关于包含该目录的磁盘上可用空间大小的信息。

命令‘M-x delete-directory’用迷你缓冲区提示输入目录名，并在目录为空时删除它。如果目录非空，你将被询问是否要递归删除它。在具备“回收站”（Trash 或 Recycle Bin）功能的系统上，你可以把变量‘delete-by-moving-to-trash’改为‘t’，使该命令把指定目录移到回收站，而不是直接删除。关于使用回收站的更多信息，参见“其他文件操作”。

20.9 比较文件

命令‘M-x diff’用迷你缓冲区提示输入两个文件名，并在名为‘*diff*’的缓冲区中显示两个文件的差异。它通过运行‘diff’程序来完成工作，使用的选项取自变量‘diff-switches’。‘diff-switches’的值应是字符串；默认值为‘"-u"’，以指定统一上下文差异。参见《《diffutils》手册》的“Top, for more information about the ‘diff’ program”

‘diff’命令的输出使用一种名为 Diff 模式的主模式显示。参见“Diff 模式”。

另一种（复杂得多的）替代方法是‘M-x ediff’（参见《Ediff 手册》）。

命令‘M-x diff-backup’把指定文件与其最近的备份相比较。如果你指定的是备份文件名，‘diff-backup’会把它与其作为备份来源的源文件相比较。在其他方面，它的行为与‘M-x diff’相同。

命令‘M-x diff-buffer-with-file’把指定缓冲区与其对应的文件相比较。这会向你显示如果你保存该缓冲区会对文件做出哪些改动。

命令‘M-x diff-buffers’比较两个指定缓冲区的内容。

命令‘M-x compare-windows’把当前窗口中的文本与在你选定当前窗口之前所选窗口中的文本相比较。（关于 Emacs 中窗口的更多信息，参见“多窗口”。）比较从各窗口中的光标点开始，先把每个初始光标点值压入各自缓冲区的标记环（参见“标记环”）。然后它在每个窗口中逐字符向前移动光标点，直到遇到不匹配的字符。随后命令退出。

如果命令启动时两个窗口中光标点之后的文本并不匹配，‘M-x compare-windows’会启发式地向前推进到两个窗口中相匹配的文本处，然后退出。因此，如果你反复使用‘M-x compare-windows’（参见“重复执行命令”），每次它要么跳过一个匹配区间，要么找到另一个匹配区间的开头。

带数字参数时，‘compare-windows’忽略空白字符的改动。若变量‘compare-ignore-case’非‘nil’，比较还会忽略大小写差异。若变量‘compare-ignore-whitespace’非‘nil’，‘compare-windows’默认忽略空白字符的改动，但前缀参数会在该次调用命令时关闭这一行为。

你可以用‘M-x smerge-mode’打开 Smerge 模式，这是一个用于编辑‘diff3’程序输出的次模式。这种输出通常是在 VC 之外执行版本控制系统的更新时，由于对某个文件的修改发生冲突而导致合并失败的产物。Smerge 模式提供的命令可以通过选择特定的改动来解决冲突。

参见“用 Emerge 合并文件”，了解 Emerge 工具，它提供了功能强大的文件合并接口。

20.10 Diff 模式

Diff 模式是一种主模式，用于显示‘M-x diff’及其他类似命令的输出。这类输出称为“补丁”（patch），因为它可以传给‘patch’命令，以自动应用其中指定的改动。要手动选择 Diff 模式，请键入‘M-x diff-mode’。

补丁中指定的改动被划分为“块”（hunk），块是包含一行或多行改动内容的连续文本片段。块通常还包含未改动的行，为改动提供上下文。每个块之前都有一个“块头”（hunk header），它指明该块的改动所在的旧行号和新行号。Diff 模式会高亮每个块头，以把它与块的实际内容区分开。

补丁中的第一个块之前有一个文件头，它显示该文件的新版本与旧版本的名称及其时间戳。如果某个补丁显示多个文件的改动，那么每个文件都会在该文件改动的第一个块之前有这样一个头部。

你可以像编辑其他任何缓冲区一样编辑 Diff 模式缓冲区。（如果它是只读的，你需要先使之可写；参见“其他缓冲区操作”。）每当你编辑某个块时，Diff 模式都会尝试自动更正块头中的行号，以确保补丁仍然正确，并且仍可由‘patch’应用。要禁用自动行号更正，请把变量‘diff-update-on-the-fly’改为‘nil’。

Diff 模式会安排让块被‘M-g M-n’及其他处理错误消息的命令当作编译器错误消息处理（参见“编译模式”）。这样，你就可以用编译模式的命令访问相应的源码位置。

此外，Diff 模式还提供以下命令来浏览、操作和应用补丁的各个部分：

- ‘M-n’移动到下一个块的起始处（‘diff-hunk-next’）。带前缀参数 N 时，向前移动到第 N 个后继块。

默认情况下，Diff 模式在 Emacs 显示块时会自动“精化”（refine）它们，以更细的粒度高亮其中的改动。另一种做法是：若你把‘diff-refine’设为符号‘navigation’，Diff 模式只精化你用此命令或用‘diff-hunk-prev’所移动到的那个块。最后，你可以把‘diff-refine’设为‘nil’，并用‘C-c C-b’（‘diff-refine-hunk’）手动精化块，该命令将在下面说明。

默认情况下，以任何方式精化某个块时，都会在精化区域的两个边界之一显示一个“影子光标”（shadow cursor），以便更好地显示精化区域从何处开始或结束。这可以通过定制变量‘smerge-refine-shadow-cursor’来控制。

- ‘M-p’移动到上一个块的起始处（‘diff-hunk-prev’）。带前缀参数 N 时，向后移动到第 N 个前面的块。与‘M-n’一样，如果你把‘diff-refine’设为符号‘navigation’，该命令会精化你所移动到的块。
- ‘M-}’在多文件补丁中移动到下一个文件的开头（‘diff-file-next’）。带前缀参数 N 时，向前移动到第 N 个后继文件的开头。
- ‘M-{’在多文件补丁中移动到上一个文件的开头（‘diff-file-prev’）。带前缀参数 N 时，向后移动到第 N 个前面文件的开头。
- ‘M-k’删除光标点处的块（‘diff-hunk-kill’）。如果区域处于活动状态，则删除该区域所覆盖的所有块。
- ‘M-K’在多文件补丁中，删除当前文件部分（‘diff-file-kill’）。

- ‘C-c C-a’ 把此块应用到它的目标文件（‘diff-apply-hunk’）。带前缀参数 ‘C-u’ 时，还原此块，即应用该块的反向改动，把“新”版本变成“旧”版本。若 ‘diff-jump-to-old-file’ 非 ‘nil’，则改为把该块应用到文件的“旧”版本。

‘diff-apply-hunk’ 在删除文件以及把块应用到备份文件时会提示你确认，并且对于已经应用过的块，它会提议反向应用。不过，如果区域处于活动状态，它就会应用该区域覆盖的所有块，而不做任何提示。在这种模式下，如果有任何块不能干净地应用，它就干脆失败，并且不会确认删除或把块应用到备份文件。

- ‘C-c M-u’ 还原此块，然后从差异中移除该块（‘diff-revert-and-kill-hunk’）。请保存访问目标文件的缓冲区。当区域处于活动状态时，该命令会反向应用并删除该区域覆盖的块。

此命令在由 ‘C-x v =’ 和 ‘C-x v D’ 生成的缓冲区中很有用（*note Old Revisions::）。这些缓冲区向你呈现你所做改动的视图，你可以用此命令撤销你无意做出或不再想要的改动。

这是一项破坏性操作，因此默认情况下，该命令会要求你确认确实要还原并删除这些块。你可以定制 ‘diff-ask-before-revert-and-kill-hunk’ 来控制这一点。

- ‘C-c <RET> a’ 应用缓冲区中的所有块（‘diff-apply-buffer’）。如果区域处于活动状态，则应用该区域覆盖的所有块；否则应用所有块。带前缀参数时，反向应用这些块。如果差异应用成功，则保存被改动的缓冲区。
- ‘C-c <RET> n’ 删除当前块以外的其他块。如果区域处于活动状态，该命令删除该区域覆盖的块；否则删除当前块以外的所有块。该命令在窄化的缓冲区中不起作用，因为安全地删除块需要访问缓冲区中指定各块文件名的那些部分。
- ‘C-c C-b’ 以更细的粒度高亮光标点处块中的改动（‘diff-refine-hunk’）。这让你能看清每个被改动行中究竟哪些部分真的被改动了。精化某个块时，默认会在精化区域的两个边界之一显示一个“影子光标”。

默认情况下，Diff 模式在 Emacs 显示块时就会精化它们，因此如果你把 ‘diff-refine’ 定制为非默认值，你可能会发现此命令很有用。

- ‘C-c C-c’ 转到此块对应的源文件和行（‘diff-goto-source’）。默认情况下，这会跳到文件的“新”版本，即文件头上最先显示的版本。带前缀参数时，改为跳到“旧”版本。若 ‘diff-jump-to-old-file’ 非 ‘nil’，该命令默认跳到“旧”文件，而前缀参数的含义则相反。如果前缀参数是大于 8 的数字（例如你键入 ‘C-u C-u C-c C-c’），那么该命令还会为下一次调用设置 ‘diff-jump-to-old-file’。如果源文件受版本控制（参见“版本控制”），默认会跳到工作文件。带前缀参数时，若光标点位于旧行上，则跳到文件的“旧”修订版（参见“查看并比较旧修订”），否则跳到“新”修订版。
- ‘C-c C-e’ 用该补丁启动一个 Ediff 会话（‘diff-ediff-patch’）。参见《Ediff 手册》
- ‘C-c C-n’ 把视图限制在当前块（‘diff-restrict-view’）。参见“窄化”。带前缀参数时，把视图限制在多文件补丁的当前文件。要再次解除限制，请用 ‘C-x n w’（‘widen’）。
- ‘C-c C-r’ 反转整个缓冲区的比较方向（‘diff-reverse-direction’）。带前缀参数时，只在当前区域内部反转方向（参见“标记与区域”）。反转方向意味着更改各块和文件起始头部，以生成一个能把“新”版本变成“旧”版本的补丁。
- ‘C-c C-s’ 把光标点处的块拆分为两个独立的块（‘diff-split-hunk’）。这会插入一个块头，并修改当前块的头部。该命令对人工编辑补丁很有用，并且只适用于由 ‘diff’ 程序的 ‘-u’ 或 ‘--unified’ 选项产生的“统一差异格

式”（unified diff format）。如果你需要在‘diff’的‘-c’或‘--context’选项产生的“上下文差异格式”（context diff format）中拆分块，请先用‘C-c C-u’把缓冲区转换为统一差异格式。

- ‘C-c C-d’把整个缓冲区转换为“上下文差异格式”（‘diff-unified->context’）。带前缀参数时，只转换区域内的块。
- ‘C-c C-u’把整个缓冲区转换为统一差异格式（‘diff-context->unified’）。带前缀参数时，把统一格式转换为上下文格式。当标记处于活动状态时，只转换区域内的块。
- ‘C-c C-l’重新生成当前块（‘diff-refresh-hunk’）。
- ‘C-c C-w’重新生成当前块，忽略空白字符的改动。带非‘nil’前缀参数时，重新生成所有块（‘diff-ignore-whitespace-hunk’）。这会以‘diff-ignore-whitespace-switches’调用‘diff-command’，该变量默认为‘-b’，意思是只忽略空白字符的改动。
- ‘C-x 4 A’为每个块生成一个 ChangeLog 条目（‘diff-add-change-log-entries-other-window’），方式与‘C-x 4 a’相同（参见“变更日志”）。这会创建一份改动日志的骨架，你随后可以在其中填入改动的实际描述。在 Diff 模式下，‘C-x 4 a’本身针对当前块所属的文件操作，但从补丁本身获取函数名。这对为被补丁删除的函数编写日志条目很有用。

补丁有时会在被改动的行上包含行尾空白，这是一种无意且不想要的改动。处理这个问题有两种办法。首先，如果你在 Diff 缓冲区中启用空白模式（参见“无用的空白”），它会自动高亮被改动行中的行尾空白。其次，你可以用命令‘M-x diff-delete-trailing-whitespace’，它在补丁改动的行中搜索行尾空白，并在补丁和被补丁的源文件中都删除这些空白。该命令不会保存它所做的修改，因此你可以决定是否保存这些改动（被修改的文件列表显示在回显区）。带前缀参数时，它尝试修改原始（“旧”）源文件，而不是打过补丁的（“新”）源文件。

若‘diff-font-lock-syntax’非‘nil’，块中的源码片段会按相应的主模式高亮。

20.11 复制、命名和重命名文件

Emacs 提供若干用于复制、命名和重命名文件的命令。它们都用迷你缓冲区读取两个文件名，OLD（或 TARGET）与 NEW，然后相应地复制文件或调整文件名；它们不接受通配符文件名。

在所有这些命令中，如果参数 NEW 只是一个目录名（参见《Emacs Lisp 参考手册》的“Directory Names”），那么真正的新名字就在该目录中，其非目录部分与 OLD 的相同。例如，命令‘M-x rename-file <RET> ~/foo <RET> /tmp/ <RET>’把‘~/foo’重命名为‘/tmp/foo’。在 GNU 及其他类 POSIX 系统上，目录名以‘/’结尾。

当新文件名已经存在时，所有这些命令都会请求确认。

- ‘M-x copy-file’把文件 OLD 的内容复制到文件 NEW。
- ‘M-x copy-directory’复制目录，类似‘cp -r’shell 命令。如果 NEW 是目录名，它创建 OLD 目录的副本并放入 NEW。否则，它把 OLD 的全部内容复制到名为 NEW 的新目录中。若‘copy-directory-create-symlink’非‘nil’且 OLD 是符号链接，该命令会复制该符号链接。若为‘nil’，该命令会跟随该链接并改为复制其内容。（这是默认行为。）
- ‘M-x rename-file’把文件 OLD 重命名为 NEW。如果文件名 NEW 已经存在，你必须用‘yes’确认，否则不会进行重命名；这是因为重命名会丢失名字 NEW 原有的含义。如果 OLD 和 NEW 位于不同的文件系统上，则复制文件 OLD 然后删除它。

如果文件受版本控制（参见“版本控制”），你应改用 ‘C-x v R’ 而不是 ‘M-x rename-file’ 来重命名它。参见“删除与重命名受版本控制的文件”。

- ‘M-x add-name-to-file’ 为已有文件增加另一个名字，而不删除原有的名字。新名字是作为已有文件的硬链接创建的。新名字必须与文件位于同一个文件系统上。在 MS-Windows 上，该命令仅在文件位于 NTFS 文件系统时有效。在 MS-DOS 以及某些远程系统类型上，它通过复制文件来实现。
- ‘M-x make-symbolic-link’ 创建名为 NEW 的符号链接，它指向 TARGET。其效果是：以后试图打开文件 NEW 时，将引用打开那一刻名为 TARGET 的文件；如果那时名字 TARGET 不存在，则会出错。该命令不展开参数 TARGET，因而允许你指定相对名字作为链接目标。不过，该命令会展开 TARGET 开头的 ‘~’，以便你方便地指定主目录，并会去掉开头的 ‘/:’，以便你可以指定以字面 ‘~’ 或 ‘/:’ 开头的相对名字。参见“引用的文件名”。在 MS-Windows 上，该命令仅在 MS Windows Vista 及更高版本上有效。当 NEW 是远程文件时，其是否有效取决于系统类型。

20.12 其他文件操作

Emacs 还有对文件执行许多其他操作的命令。所有命令都只操作一个文件；它们不接受通配符文件名。

- ‘M-x delete-file’ 提示输入一个文件并删除它。如果你要删除同一目录中的许多文件，使用 Dired 可能比 ‘delete-file’ 更方便。参见“用 Dired 删除文件”。
- ‘M-x move-file-to-trash’ 把文件移入系统的“Trash”（或“Recycle Bin”）。这是大多数操作系统上可用的功能；移入 Trash 的文件以后你若改变主意，还可以取回。（恢复已移入 Trash 的文件的方法因系统而异。）

默认情况下，Emacs 的删除命令*不*使用 Trash。要让常见的删除命令使用 Trash（在可用时），请把变量 ‘delete-by-moving-to-trash’ 改为 ‘t’。这会影响命令 ‘M-x delete-file’ 和 ‘M-x delete-directory’（参见“文件目录”），以及 Dired 中的删除命令（参见“用 Dired 删除文件”）。给 ‘M-x delete-file’ 或 ‘M-x delete-directory’ 提供前缀参数，会使它们直接删除，而不使用 Trash，无论 ‘delete-by-moving-to-trash’ 的值如何。

如果你设置了 ‘delete-by-moving-to-trash’，又想从 Trash 目录中手工删除 Emacs 里的文件，那么用 ‘D’（‘dired-do-delete’）之类的命令在 Trash 目录中并不能很好地工作（它只会给文件起个新名字，而不会删除任何东西）。如果你希望能够这样做，就应在 Trash 目录中创建一个 ‘.dir-locals.el’ 文件，内容类似下面这样：

```
((dired-mode . ((delete-by-moving-to-trash . nil))))
```

但要注意，如果你使用系统的“清空 Trash”命令，它很可能也会删除这个 ‘.dir-locals.el’ 文件，因此只有在你手工从 Trash 目录删除文件时才应这样做。

若变量 ‘remote-file-name-inhibit-delete-by-moving-to-trash’ 非 ‘nil’，远程文件永远不会被移入 Trash。它们会被直接删除。

若文件处于版本控制之下（参见“版本控制”），应改用 ‘M-x vc-delete-file’ 而非 ‘M-x delete-file’ 来删除它。参见“删除与重命名受版本控制的文件”。

‘M-x insert-file’（也可用 ‘C-x i’）会把指定文件内容的副本插入当前缓冲区中光标点处，并让光标点在插入内容之前保持不动。插入内容之后的位置会被加入标记环，但不会激活标记（参见“标记环”）。

‘M-x insert-file-literally’ 与 ‘M-x insert-file’ 类似，区别在于文件是按字面插入的：它被当作一串 ASCII 字符处理，不做任何特殊编码或转换，类似于 ‘M-x find-file-literally’ 命令（参见“访问文件”）。

‘M-x write-region’ 是 ‘M-x insert-file’ 的逆操作；它把区域的内容复制到指定文件中。‘M-x append-to-file’ 把区域的文本追加到指定文件的末尾。参见“累积文本”。变量 ‘write-region-inhibit-fsync’ 适用于这些命令以及保存文件；参见“自定义文件的保存”。

‘M-x set-file-modes’ 读取一个文件名和一个“文件模式”，并把该文件模式应用到指定文件。文件模式又称“文件权限”，决定文件能否被读取、写入或执行，以及由谁执行。此命令读取文件模式时使用与 ‘chmod’ 命令相同的符号格式或八进制格式；例如，‘u+x’ 表示为文件属主增加执行权限。在不支持文件模式的操作系统上，它不起作用。‘chmod’ 是这个函数的便捷别名。

20.13 访问压缩文件

访问压缩文件时，Emacs 会自动解压；若你改动并保存了它，Emacs 又会自动重新压缩。Emacs 依据文件名识别压缩文件。以 ‘.gz’ 结尾的文件名表示用 ‘gzip’ 压缩的文件，其他结尾则表示其他压缩程序。

自动解压与压缩适用于 Emacs 使用文件内容的所有操作，包括访问文件、保存文件、把文件内容插入缓冲区、加载文件以及对文件进行字节编译。

要禁用此特性，请键入命令 ‘M-x auto-compression-mode’。你也可以通过定制变量 ‘auto-compression-mode’ 来永久禁用。

20.14 文件归档

名字以 ‘.tar’ 结尾的文件通常是由 ‘tar’ 程序制作的“归档文件”（archive）。Emacs 在一种称为 Tar 模式的特殊模式中查看这类文件，该模式提供类似 Dired 的内容列表（参见“Dired 目录编辑器”）。你可以像在 Dired 中一样在列表里移动，并访问归档中所含的子文件。不过，并非所有 Dired 命令在 Tar 模式中都可使用。

如果启用了自动压缩模式（参见“访问压缩文件”），那么 Tar 模式也用于压缩归档，即扩展名为 ‘.tgz’、‘.tar.Z’ 和 ‘.tar.gz’ 的文件。

按键 ‘e’、‘f’ 和 <RET> 都会把某个成员文件解出到它自己的缓冲区。你可以在那里编辑它；若你保存该缓冲区，编辑后的版本将替换 Tar 缓冲区中的那个版本。在 Tar 缓冲区中用鼠标点击文件名也会做同样的事。‘v’ 把文件解出到一个处于 View 模式的缓冲区（参见“视图模式”）。‘o’ 把文件解出并显示在另一个窗口中，这样你就能在编辑该文件的同时操作归档。

按键 ‘I’ 向归档中加入一个新的（普通）文件。该文件最初是空的，但可以很方便地用上述命令编辑。此命令把新文件插在当前文件之前，因此在 Tar 缓冲区的第一行使用它，会使新文件成为归档中的第一个文件；而在缓冲区末尾使用它，则会使其成为最后一个文件。

‘d’ 标记文件以便稍后用 ‘x’ 删除，‘u’ 取消标记，与 Dired 中相同。‘C’ 把文件从归档复制到磁盘，‘R’ 在归档内重命名文件。‘g’ 从磁盘上的归档还原缓冲区。按键 ‘M’、‘G’ 和 ‘O’ 分别更改文件的权限位、所属组和属主。

保存 Tar 缓冲区会把归档的新版本连同你对各成员所做的改动一起写入磁盘。

使用 Tar 模式不需要 ‘tar’ 程序——Emacs 直接读取归档。不过，访问压缩归档需要相应的解压程序。

另一种独立的类似模式 Archive 模式用于 'arc'、'jar'、'lzh'、'zip'、'rar'、'7z' 和 'zoo' 归档，以及自解压可执行文件 'exe'。

Archive 模式的键绑定与 Tar 模式类似，另外增加了 'm' 键（标记文件以供后续操作）和 'M-<DEL>'（取消所有已标记文件的标记）。此外，对于那些无法在一行内显示完整详细信息的归档类型，'a' 键用于切换详细文件信息的显示。重命名子文件、更改其模式或属主之类的操作，只在部分归档格式中受支持。

与 Tar 模式不同，Archive 模式会运行归档程序来解包和重新打包归档。不过，查看归档的目录表并不需要这些程序，只有在解出或操作归档中的子文件时才需要。程序名及其选项的细节可以在 'Archive' 定制组中设置（参见"自定义组"）。

20.15 远程文件

你可以用一种特殊的文件名语法引用其他机器上的文件：

```
/METHOD:HOST:FILENAME
/METHOD:USER@HOST:FILENAME
/METHOD:USER@HOST#PORT:FILENAME
```

为了完成这种请求，Emacs 使用 'ssh' 之类的远程登录程序。你必须在文件名中始终指明使用哪种方法——例如，'/ssh:USER@HOST:FILENAME' 使用 'ssh'。当你在文件名中指定伪方法 '-' 时，Emacs 按如下方式选择方法：

1. 若主机名以 'ftp.'（带点）开头，Emacs 使用 FTP。
2. 若用户名为 'ftp' 或 'anonymous'，Emacs 使用 FTP。
3. 若变量 'tramp-default-method' 被设为 'ftp'，Emacs 使用 FTP。
4. 若 'ssh-agent' 正在运行，Emacs 使用 'scp'。
5. 否则，Emacs 使用 'ssh'。

你可以通过运行 'M-x inhibit-remote-files' 彻底关闭远程文件名特性。对于个别情况，可以用 '/:' 引用文件名来关闭该特性（参见"引用的文件名"）。

通过 FTP 的远程文件访问由 Ange-FTP 包处理，下文会加以说明。通过其他方法的远程文件访问由 Tramp 包处理，它有自己的手册。参见《Tramp 手册》

使用 Ange-FTP 包时，如果远程文件名中指定了 USER，Emacs 就用该名字通过 FTP 登录。如果未指定 USER，Emacs 就用你在本地系统上的用户名登录；但如果你把变量 'ange-ftp-default-user' 设为一个字符串，则改用该字符串。登录时，Emacs 也可能询问密码。

出于性能考虑，Emacs 默认不会为通过 FTP 访问的文件生成备份文件。若要让它生成，请把变量 'ange-ftp-make-backup-files' 改为非 'nil' 值。

默认情况下，远程文件的自动保存文件会生成在本地机器的临时文件目录中，具体位置由变量 'auto-save-file-name-transforms' 指定。参见"自动保存文件"。

要访问可通过匿名 FTP 获取的文件，需使用特殊用户名 'anonymous' 或 'ftp'。这些用户名的密码有专门的处理方式。变量 'ange-ftp-generate-anonymous-password' 控制具体行为：若该变量的值是一个字符串，则该字符串

被用作密码；若为非‘nil’（默认情况），则使用‘user-mail-address’的值；若为‘nil’，则 Emacs 像平常一样提示你输入密码（参见“输入密码”）。

有时你可能无法访问远程机器上的文件，因为中间的“防火墙”出于安全原因阻止了连接。如果你能登录到一台“网关”机器，而目标文件从该网关机器*确实*可以访问，且该网关的 FTP 服务器支持网关功能，那么你仍然可以使用远程文件名；你只需通过设置变量‘ange-ftp-gateway-host’指定网关机器的名称，并把‘ange-ftp-smart-gateway’设为‘t’。否则，你也可能让远程文件名正常工作，但过程较为复杂。你可以通过输入‘M-x finder-commentary <RET> ange-ftp <RET>’阅读相关说明。

20.16 引用的文件名

你可以“引用”一个绝对文件名，以防止其中的特殊字符和语法产生特殊效果。做法是在开头加上‘/:’。

例如，你可以引用一个看起来像远程文件名的本地文件名，以免它被当作远程文件名处理。因此，如果你有一个名为‘/foo:’的目录，其中有一个名为‘bar’的文件，那么你在 Emacs 中可以用‘/:/foo:/bar’指代该文件。

如果你只想引用远程文件名中本地部分的特殊字符，可以只引用本地部分。‘/ssh:baz:/:/foo:/bar’指的是主机‘baz’上目录‘/foo:’中的文件‘bar’。

‘/:’还可以防止‘~’被当作用户主目录的特殊字符。例如，‘/:/tmp/~hack’指的是目录‘/tmp’中名为‘~hack’的文件。如果‘~’出现在文件名的开头，则必须用‘/:./’引用它，例如‘/:./~foo/bar’。

用‘/:’引用也是在迷你缓冲区中输入含‘$’的文件名的一种办法。要使这种做法生效，‘/:’必须位于迷你缓冲区内容的开头。（你也可以把每个‘$’写成两个；参见“File Names with $”。）

你还可以用‘/:’引用通配符字符，以便访问文件。例如，‘/:/tmp/foo*bar’会访问文件‘/tmp/foo*bar’。

获得相同结果的另一种方法是输入‘/tmp/foo[*]bar’，这是一个只匹配‘/tmp/foo*bar’的通配符写法。不过在多数情况下无需引用通配符字符，因为即便不加引用，它们也会给出正确结果。例如，若‘/tmp’中以‘foo’开头并以‘bar’结尾的文件名只有‘foo*bar’，那么指定‘/tmp/foo*bar’就只会访问‘/tmp/foo*bar’。

20.17 文件名缓存

你可以使用“文件名缓存”来方便地按名称定位文件，而不必准确记住它所在的位置。在迷你缓冲区中输入文件名时，‘C-<TAB>’（‘file-cache-minibuffer-complete’）会使用文件名缓存对它进行补全。如果重复按‘C-<TAB>’，则会在你最初输入内容的可能补全之间循环。（但请注意，在大多数文本终端上无法输入‘C-<TAB>’字符。）

文件名缓存不会自动填充。你需要用下列命令把文件名载入缓存：

- ‘M-x file-cache-add-directory <RET> DIRECTORY <RET>’把 DIRECTORY 中的每个文件名加入文件名缓存。
- ‘M-x file-cache-add-directory-using-find <RET> DIRECTORY <RET>’把 DIRECTORY 及其所有嵌套子目录中的每个文件名加入文件名缓存。
- ‘M-x file-cache-add-directory-using-locate <RET> DIRECTORY <RET>’使用‘locate’找出 DIRECTORY 及其所有嵌套子目录中的每个文件名，并把它们加入文件名缓存。

- ‘M-x file-cache-add-directory-list <RET> VARIABLE <RET>’ 把 VARIABLE 中列出的每个目录里的每个文件名加入文件名缓存。VARIABLE 应是一个 Lisp 变量，其值为目录列表，如 ‘load-path’。
- ‘M-x file-cache-clear-cache <RET>’ 清空缓存，即从中移除所有文件名。

文件名缓存不是持久的：它只在 Emacs 会话期间保存和维护。你可以用 ‘file-cache-display’ 命令查看缓存的内容。

20.18 查找文件的便利功能

本节介绍一些用于查找最近打开的文件、从缓冲区读取文件名的便利设施。

如果你用 ‘M-x recentf-mode’ 启用 Recentf 模式，Emacs 会维护一个最近打开文件的列表。要从该列表打开文件，使用 ‘M-x recentf-open’ 命令。启用此模式后，‘File’ 菜单中会包含一个子菜单，你可以用它访问这些文件之一。‘M-x recentf-save-list’ 把当前的 ‘recentf-list’ 保存到文件，而 ‘M-x recentf-edit-list’ 则编辑它。

如果你使用远程文件，可能还需要定制 ‘remote-file-name-access-timeout’，它是在停止检查某个远程文件是否应在 Recentf 中使用之前等待的秒数。这可以防止 Emacs 被阻塞。

‘M-x ffap’ 命令以更强大的启发式默认值对 ‘find-file’ 做了推广（参见“在光标点处查找文件和 URL”），这些默认值常常基于光标点处的文本。部分补全模式提供了扩展 ‘find-file’ 的其他功能，它们可以与 ‘ffap’ 一起使用。参见“补全选项”。

20.19 查看图像文件

访问图像文件会自动选中 Image 模式。在这个主模式中，你可以输入 ‘C-c C-c’（‘image-toggle-display’），在把文件作为图像显示在 Emacs 缓冲区中与显示其底层文本（或原始字节）表示之间切换。此外，你还可以输入 ‘C-c C-x’（‘image-toggle-hex-display’），在把文件作为图像显示与以十六进制表示显示之间切换。只有 Emacs 在编译时支持显示这类图像，把文件作为图像显示才可行。

如果显示的图像比显示它的窗口更宽或更高，通常的光标点移动键（‘C-f’、‘C-p’ 等）会使图像的不同部分显示出来。不过，默认情况下图像会自动缩放以适合窗口，因此只有在你用选项 ‘image-auto-resize’ 和 ‘image-auto-resize-on-window-resize’ 定制了默认行为时，这样做才是必要的。

要手动缩放图像，可以使用绑定到 ‘s w’ 的命令 ‘image-transform-fit-to-window’，它使图像同时适合窗口的高度和宽度。要按原始尺寸的百分比缩放图像，使用绑定到 ‘s p’ 的命令 ‘image-transform-set-percent’。要指定缩放因子来缩放图像，使用绑定到 ‘s s’ 的命令 ‘image-transform-set-scale’。要把所有变换重置为初始状态，使用绑定到 ‘s 0’ 的 ‘image-transform-reset-to-initial’，或绑定到 ‘s o’ 的 ‘image-transform-reset-to-original’。

你可以按 ‘n’（‘image-next-file’）和 ‘p’（‘image-previous-file’），分别访问同一目录中的下一个和上一个图像文件。这些命令会查询“父”Dired 缓冲区，以确定下一个/上一个图像文件是哪个。从归档文件（如 zip 或 tar 文件）中打开文件时这些命令同样可用，此时它们改为查询归档模式缓冲区。如果既找不到归档“父”缓冲区，也找不到 Dired“父”缓冲区，则会打开一个 Dired 缓冲区。

浏览图像时，有时能标记文件供以后处理会很有用（例如，你想选出一组图像复制到别处）。‘m’（‘image-mode-mark-file’）命令会在显示当前文件所在目录的任何 Dired 缓冲区中标记当前文件。如果没有打开这样的

缓冲区，则会在新缓冲区中打开该目录。要取消标记文件，使用‘u’（‘image-mode-unmark-file’）命令。最后，如果你只想把当前缓冲区的文件名复制到删除环，可以使用‘w’（‘image-mode-copy-file-name-as-kill’）命令。

如果图像可以是动画，命令 <RET>（‘image-toggle-animation’）会启动或停止动画。除非选项‘image-animate-loop’为非‘nil’，动画只播放一次。用‘f’（‘image-next-frame’）和‘b’（‘image-previous-frame’）可以逐帧浏览。这两个命令都接受数字前缀参数，以便一次跨过若干帧。你可以用‘F’（‘image-goto-frame’）跳到指定的帧。帧从 1 开始编号。输入‘a +’（‘image-increase-speed’）加快动画速度，‘a -’（‘image-decrease-speed’）减慢速度，‘a r’（‘image-reverse-speed’）则反向。命令‘a 0’（‘image-reset-speed’）把速度重置为原始值。

除了上述 Image 模式专用的键绑定之外，当光标点位于图像上或图像内部时，任何 Emacs 缓冲区中显示的图像都有特殊的键绑定：

- ‘i +’把图像尺寸增大 20%（‘image-increase-size’）。前缀数字参数控制增量；值 N 表示把尺寸乘以‘1 + N / 10’的因子，因此‘C-u 5 i +’表示把尺寸增大 50%。
- ‘i -’把图像尺寸减小 20%（‘image-decrease-size’）。前缀数字参数控制减量；值 N 表示把尺寸乘以‘1 - N / 10’的因子，因此‘C-u 3 i -’表示把尺寸减小 30%。
- ‘i r’把图像顺时针旋转 90 度（‘image-rotate’）。带前缀参数时，改为逆时针旋转 90 度。注意，此命令不适用于切片（sliced）图像。
- ‘i h’水平翻转图像（‘image-flip-horizontally’）。这使图像看起来像在竖直镜面中反射一样。注意，此命令不适用于切片图像。
- ‘i v’垂直翻转图像（‘image-flip-vertically’）。这使图像看起来像在水平镜面中反射一样。注意，此命令不适用于切片图像。
- ‘i o’把图像保存到文件（‘image-save’）。此命令会提示你输入保存图像的文件名。
- ‘i c’裁剪图像（‘image-crop’）。仅当你的系统安装了可用于裁剪和切割图像的外部程序时，此命令才可用；用户选项‘image-crop-crop-command’决定使用哪个程序，默认是 ImageMagick 的‘convert’程序。该命令会显示图像并在其上叠加一个矩形框，让你用鼠标移动和调整该框的大小。输入‘m’可使鼠标移动改为移动该框而不是调整其大小；输入‘s’则改为移动正方形框。当你对裁剪框的位置和大小满意后，输入‘<RET>’实际裁剪框下的部分；或输入‘q’退出而不裁剪。然后你可以用‘i o’或‘M-x image-save’保存裁剪后的图像。
- ‘i x’从图像中切出一个矩形（‘image-cut’）。其工作方式与‘image-crop’相同（同样需要由变量‘image-crop-cut-command’定义的外部程序来执行切割），但它不裁剪图像，而是移除框内的部分并用‘image-cut-color’指定的颜色填充该部分。带前缀参数时，该命令会提示输入要使用的颜色。

尺寸和旋转命令是“重复”的，这意味着你可以不使用‘i’前缀而继续调整图像。

如果 Emacs 在编译时带上了对 ImageMagick 库的支持，它就能用 ImageMagick 渲染多种多样的图像。变量‘imagemagick-enabled-types’列出 Emacs 可以用 ImageMagick 渲染的图像类型；列表中的每个元素都应是某种图像类型的 ImageMagick 内部名称，形式为符号或等效的字符串（例如对‘.bmp’图像用‘BMP’）。要对所有可能的图像类型启用 ImageMagick，请把‘imagemagick-enabled-types’改为‘t’。变量‘imagemagick-types-inhibit’列出无论如何都不应使用 ImageMagick 渲染的图像类型，不论‘imagemagick-enabled-types’的值是什么（默认列表包含‘C’和‘HTML’等类型，ImageMagick 能把它们渲染成图像，但 Emacs 不应这样做）。要完全禁用 ImageMagick，请把‘imagemagick-types-inhibit’改为‘t’。

如果 Emacs 对相关图像格式没有原生支持，而 'image-use-external-converter' 为非 'nil'，则 Emacs 会在显示之前尝试确定是否有可用于把该图像转换为 PNG 的外部工具。目前支持用 GraphicsMagick、ImageMagick 和 'ffmpeg' 进行图像转换。

此外，你可能还想为某些图像格式添加特殊处理程序。这些可以通过 'image-converter-add-handler' 函数添加。例如，要允许把 Krita 文件作为简单图像查看，可以写成类似这样：

```
(image-converter-add-handler
 "kra"
 (lambda (file data-p)
   (if data-p
       (error "Can't decode non-files")
     (call-process "unzip" nil t nil
                   "-qq" "-c" "-x" file "mergedimage.png"))))
```

该函数接受两个参数，第一个是文件名的后缀，第二个是执行"转换"的函数。这个函数接受两个参数，第一个是文件名或包含数据的字符串，第二个说明第一个参数是否为数据，并且它应在当前缓冲区中输出一个 'image-convert-to-format' 格式的图像。

Image-Dired 包也可用来把图像作为缩略图查看。参见"在 Dired 中查看图像缩略图"。

20.20 文件集

如果你经常编辑某一组文件，可以把它们定义为一个"文件集"（fileset）。这让你能对所有文件一次执行某些操作，例如访问、'query-replace' 和 shell 命令。要用文件集，你必须先把表达式 '(filesets-init)' 加入初始化文件（参见"Emacs 初始化文件"）。这会在菜单栏的 'File' 菜单中添加一个 'Filesets' 子菜单。

定义文件集的最简单方法是一次添加一个文件。要把文件加入名为 NAME 的文件集，请访问该文件并输入 'M-x filesets-add-buffer <RET> NAME <RET>'。如果不存在名为 NAME 的文件集，则会新建一个，它最初只包含当前文件。命令 'M-x filesets-remove-buffer' 把当前文件从文件集中移除。

你也可以用 'M-x filesets-edit'（或从 'Filesets' 菜单中选择 'Edit Filesets'）直接编辑文件集列表。编辑在 Customize 缓冲区中进行（参见"简易自定义界面"）。通常，文件集就是一个简单的文件列表，但你也可以把文件集定义为匹配文件名的正则表达式。定制缓冲区中给出了一些这类较复杂文件集的示例。如果你想在以后的 Emacs 会话中使用同样的文件集，请记得选择 'Save for future sessions'。

你可以用命令 'M-x filesets-open' 访问文件集中的所有文件，用 'M-x filesets-close' 关闭它们。用 'M-x filesets-run-cmd' 对文件集中的所有文件运行一个命令（例如 'multi-isearch-files' 或 'grep'）。这些命令在 'filesets-commands' 中指定，也可以从 'Filesets' 菜单中使用，其中每个已有的文件集都由一个子菜单表示。

关于文件集的另一种概念——为版本控制操作而捆绑在一起的文件组——参见"版本控制"。那种类型的文件集是匿名的，且不会在 Emacs 会话之间持久保留。

21 多缓冲区

你在 Emacs 中编辑的文本存放在一个称为“缓冲区”（buffer）的对象里。每次访问文件，都会用一个缓冲区来容纳该文件的文本。每次调用 Dired，都会用一个缓冲区来容纳目录列表。若用 ‘C-x m’ 发送消息，则会用一个缓冲区来容纳消息正文。当你查看某个命令的文档时，该文档显示在名为 ‘*Help*’ 的缓冲区中。

缓冲区在使用期间一直存在，不再需要时就被删除（“杀死”），删除者可以是你（参见“删除缓冲区”），也可以是 Emacs（例如退出 Emacs 时，参见“退出 Emacs”）。

每个缓冲区都有一个唯一的名称，名称长度不限。当缓冲区显示在某个窗口中时，其名称会显示在模式行上（参见“模式行”）。缓冲区名区分大小写。大多数缓冲区是通过访问文件创建的，其名称由文件名派生而来；不过你也可以创建一个任意名称的空缓冲区。刚启动的 Emacs 有几个缓冲区，其中包括名为 ‘*scratch*’ 的缓冲区，它可用于求值 Lisp 表达式，且不与任何文件关联（参见“Lisp 交互缓冲区”）。

在任何时刻，有且只有一个缓冲区是“选中的”；我们称它为“当前缓冲区”。我们有时说某个命令作用于“该缓冲区”，这实际上是指它作用于当前缓冲区。当只有一个 Emacs 窗口时，该窗口中显示的缓冲区就是当前缓冲区。当有多个窗口时，“选中窗口”中显示的缓冲区是当前缓冲区。参见“多窗口”。

缓冲区的“内容”由一系列字符组成，每个字符都可以选择性地携带一组文本属性（参见“国际字符集简介”），这些属性可以说明关于该字符的更多信息。

除了文本内容之外，每个缓冲区还记录若干信息，例如它正在访问哪个文件（如果有）、它是否已被修改、以及生效的是哪个主模式和哪些次模式（参见“主模式与次模式”）。这些信息存放在“缓冲区局部变量”中——即可以在每个缓冲区中取不同值的变量。参见“局部变量”。

缓冲区的大小不能超过某个上限，该上限由“Emacs 整数”所能表示的最大缓冲区位置决定。这是因为 Emacs 用该数据类型来记录缓冲区位置。对于典型的 64 位机器，最大缓冲区大小为 2^{61} - 2 字节，约合 2 EiB。对于典型的 32 位机器，上限通常是 2^{29} - 2 字节，约合 512 MiB。缓冲区大小还受系统内存量的限制。

- Several Buffers:: 如何遍历所有缓冲区的列表，并对其中若干缓冲区执行各种操作。* Indirect Buffers:: 间接缓冲区共享另一个缓冲区的文本。* Buffer Convenience:: 缓冲区处理的便利与定制特性。

21.1 创建与选择缓冲区

‘C-x b BUFFER <RET>’ 选择或创建名为 BUFFER 的缓冲区（‘switch-to-buffer’）。‘C-x 4 b BUFFER <RET>’ 类似，但在另一个窗口中选中 BUFFER（‘switch-to-buffer-other-window’）。‘C-x 5 b BUFFER <RET>’ 类似，但在另一个框架中选中 BUFFER（‘switch-to-buffer-other-frame’）。‘C-x <LEFT>’ 选中缓冲区列表中的上一个缓冲区（‘previous-buffer’）。‘C-x <RIGHT>’ 选中缓冲区列表中的下一个缓冲区（‘next-buffer’）。‘C-u M-g M-g’ ‘C-u M-g g’ 读取数字 N，并在另一个窗口中移到最近选中的、当前缓冲区之外的那个缓冲区的第 N 行。

‘C-x b’（‘switch-to-buffer’）命令用迷你缓冲区读取一个缓冲区名。然后它使该缓冲区成为当前缓冲区，并在当前选中的窗口中显示它。输入为空时，指定的是最近曾是当前缓冲区、且当前未显示在任何窗口中的那个缓冲区。

输入缓冲区名时，可以使用通常的补全与历史命令（参见“迷你缓冲区”）。注意，‘C-x b’ 及相关命令在迷你缓冲区补全中使用“带确认的宽松补全”：若你在迷你缓冲区文本指向一个不存在的缓冲区时输入 <RET>，Emacs

会显示 '[Confirm]'，你必须再输入一次 <RET> 才会提交该缓冲区名。详见参见"补全退出"。关于其他补全选项与特性，参见"补全选项"。

若你指定的缓冲区不存在，'C-x b' 会创建一个不访问任何文件的新空缓冲区，并选中它进行编辑。变量 'major-mode' 的默认值决定新缓冲区的主模式；该默认值是 Fundamental 模式。参见"主模式"。创建新缓冲区的一个用途是拿它做临时笔记。若你试图保存它，Emacs 会询问要用的文件名，并会参照该文件名重新确定缓冲区的主模式（参见"选择文件模式"）。

若想在少数几个缓冲区之间方便地切换，可使用命令 'C-x <LEFT>' 和 'C-x <RIGHT>'。'C-x <LEFT>'（'previous-buffer'）选中上一个缓冲区（按当前框架中最近选择的顺序），而 'C-x <RIGHT>'（'next-buffer'）则沿相反方向遍历缓冲区。两个命令都支持数字前缀参数作为重复次数。

要在当前窗口之外的窗口中选中缓冲区（参见"多窗口"），输入 'C-x 4 b'（'switch-to-buffer-other-window'）。它会用迷你缓冲区提示输入缓冲区名，在另一个窗口中显示该缓冲区，并选中那个窗口。

类似地，'C-x 5 b'（'switch-to-buffer-other-frame'）提示输入缓冲区名，在另一个框架中显示该缓冲区（参见"框架与图形显示"），并选中那个框架。若该缓冲区已经显示在另一个框架的某个窗口中，Emacs 会改为选中那个窗口及其框架，而不创建新框架。

关于 'C-x 4 b' 和 'C-x 5 b' 如何获得用于显示的窗口和/或框架，参见"在窗口中显示缓冲区"。

此外，'C-x C-f' 以及任何其他访问文件的命令，也可用于切换到已存在的访问文件的缓冲区。参见"访问文件"。

'C-u M-g M-g'，即带普通前缀参数的 'goto-line'，会用迷你缓冲区读取数字 N，然后在另一个窗口中选中最近选中的、当前缓冲区之外的那个缓冲区，再把光标点移到该缓冲区第 N 行的开头。这主要在一个引用另一缓冲区行号的缓冲区中有用：若光标点位于某个数字上或紧接其后，'goto-line' 会把该数字用作 N 的默认值。注意，除单纯的 'C-u' 之外的前缀参数行为不同。'C-u 4 M-g M-g' 直接跳到*当前*缓冲区的第 4 行，而不从迷你缓冲区读取数字。（记住，不带前缀参数的 'M-g M-g' 会读取数字 N，然后移到当前缓冲区的第 N 行。参见"移动光标点的位置"。）

Emacs 内部使用以空格开头的缓冲区名。它对这些缓冲区有少许特殊处理——例如，默认情况下它们不记录撤销信息。最好自己避免使用这样的缓冲区名。

21.2 列出已有缓冲区

'C-x C-b' 列出已有的缓冲区（'list-buffers'）。

要显示已有缓冲区的列表，输入 'C-x C-b'。这会在名为 '*Buffer List*' 的缓冲区中弹出一个缓冲区菜单。列表的每一行显示一个缓冲区的名称、大小、主模式以及所访问的文件。缓冲区按其曾为当前缓冲区的顺序列出；最近曾是当前缓冲区的排在前面。本节说明缓冲区列表如何显示，以及如何解读列表中的各种标记；关于 '*Buffer List*' 缓冲区中的特殊模式及可用命令，参见"对多个缓冲区操作"。

一行第一栏中的 '.' 表示该缓冲区是当前缓冲区。'%' 表示只读缓冲区。'*' 表示该缓冲区已被修改。若有多个缓冲区被修改，也许该用 'C-x s' 保存其中几个了（参见"保存文件的命令"）。下面是缓冲区列表的一个例子：

```
CRM Buffer                Size  Mode             File
. * .emacs                3294  Elisp/l          ~/.emacs
 %  *Help*                 101  Help
    search.c              86055  C                ~/cvs/emacs/src/search.c
 %  src                   20959  Dired by name    ~/cvs/emacs/src/
  * *mail*                  42  Mail
 %  HELLO                 1607  Fundamental      ~/cvs/emacs/etc/HELLO
 %  NEWS                481184  Outline          ~/cvs/emacs/etc/NEWS
    *scratch*              191  Lisp Interaction
  * *Messages*            1554  Messages
```

缓冲区‘*Help*’是由帮助请求产生的（参见“帮助”）；它不访问任何文件。缓冲区‘src’是 Dired 在目录‘~/cvs/emacs/src/’上产生的。若给该命令一个前缀参数，如‘C-u C-x C-b’，则只列出访问文件的缓冲区。

默认情况下，‘list-buffers’会略去名称以空格开头的缓冲区，除非它们访问文件：这类缓冲区由 Emacs 内部使用（但‘I’命令可撤销这一限制，参见“对多个缓冲区操作”）。

21.3 其他缓冲区操作

‘C-x C-q’切换缓冲区的只读状态（‘read-only-mode’）。‘C-x x r <RET> BUFFER <RET>’更改当前缓冲区的名称。‘C-x x u’通过添加‘<NUMBER>’后缀来重命名当前缓冲区。‘M-x view-buffer <RET> BUFFER <RET>’滚动浏览缓冲区 BUFFER。参见“视图模式”。

缓冲区可以是“只读的”，这意味着不允许执行插入或删除其文本的命令。（不过，其他命令如‘C-x <RET> f’仍可把它标记为已修改，参见“为文件文本指定编码系统”）。模式行在靠近左边缘处用‘%%’或‘%*’指示只读缓冲区。参见“模式行”。只读缓冲区通常由 Dired、Rmail 等子系统创建，这些子系统有专门命令来操作其文本。访问一个访问控制表明你不可写的文件，也会使缓冲区只读。

命令‘C-x C-q’（‘read-only-mode’）使只读缓冲区变为可写，使可写缓冲区变为只读。其做法是设置变量‘buffer-read-only’，该变量在每个缓冲区中都有局部值，若其值非‘nil’就使缓冲区只读。若你把选项‘view-read-only’改为非‘nil’值，则用‘C-x C-q’使缓冲区只读时，也会在该缓冲区中启用 View 模式（参见“视图模式”）。

‘C-x x r’（‘rename-buffer’）更改当前缓冲区的名称。你在迷你缓冲区中给出新名称作为参数；没有默认值。若你指定的名称已被其他缓冲区使用，就会出错，且不会进行重命名。

‘C-x x u’（‘rename-uniquely’）把当前缓冲区重命名为相似的名称，并加上数字后缀，使其既不同又唯一。该命令不需要参数。它对于创建多个 shell 缓冲区很有用：若你重命名‘*shell*’缓冲区，然后再次执行‘M-x shell’，它会创建一个名为‘*shell*’的新 shell 缓冲区；与此同时，旧的 shell 缓冲区以新名称继续存在。这个方法对邮件缓冲区、编译缓冲区，以及大多数会创建带特定名称的特殊缓冲区的 Emacs 特性都很好用。（对于其中某些特性，如‘M-x compile’、‘M-x grep’，你需要先切换到其他缓冲区再使用该命令，否则尽管名称已改，它仍会复用当前缓冲区。）

命令‘M-x append-to-buffer’和‘C-x x i’（‘insert-buffer’）也可用于把文本从一个缓冲区复制到另一个缓冲区。参见“累积文本”。

21.4 删除缓冲区

若 Emacs 会话持续一段时间，你可能会积累大量缓冲区。这时你也许会发现，把不再需要的缓冲区“删除”（kill）会很方便。（有些其他编辑器把这一操作称为“关闭”，并说“关闭缓冲区”或“关闭该缓冲区所访问的文件”。）在大多数操作系统上，删除缓冲区会把 Emacs 用于该缓冲区的内存归还给操作系统，以便其他程序使用。以下是一些删除缓冲区的命令：

‘C-x k BUFFER <RET>’ 删除缓冲区 BUFFER（‘kill-buffer’）。‘M-x kill-some-buffers’ 逐一询问是否删除每个缓冲区。‘M-x kill-matching-buffers’ 询问是否删除所有名称匹配某正则表达式的缓冲区。‘M-x kill-matching-buffers-no-ask’ 类似 ‘kill-matching-buffers’，但不要求确认。

‘C-x k’（‘kill-buffer’）删除一个缓冲区，其名称由你在迷你缓冲区中指定。若你在迷你缓冲区中只输入 <RET>，默认删除当前缓冲区。若你删除了当前缓冲区，就会有另一个缓冲区成为当前缓冲区：它是最近曾为当前缓冲区、但现在未显示在任何窗口中的那个缓冲区。若你要删除的访问文件的缓冲区已被修改，则必须用 ‘yes’ 确认后才会删除该缓冲区。

命令 ‘M-x kill-some-buffers’ 逐一询问每个缓冲区。回答 ‘yes’ 表示删除该缓冲区，与 ‘kill-buffer’ 相同。该命令忽略名称以空格开头、由 Emacs 内部使用的缓冲区。

命令 ‘M-x kill-matching-buffers’ 提示输入一个正则表达式，并删除所有名称匹配该表达式的缓冲区。参见“正则表达式的语法”。与 ‘kill-some-buffers’ 一样，它在每次删除前都要求确认。该命令通常忽略名称以空格开头、由 Emacs 内部使用的缓冲区。要同时删除内部缓冲区，请带前缀参数调用 ‘kill-matching-buffers’。命令 ‘M-x kill-matching-buffers-no-ask’ 与 ‘kill-matching-buffers’ 类似，但在删除每个匹配的缓冲区之前不要求确认。

缓冲区菜单特性也便于删除各种缓冲区。参见“对多个缓冲区操作”。

若你想在每次删除缓冲区时做点特别的事，可以把钩子函数加入钩子 ‘kill-buffer-hook’（参见“钩子”）。

若你像许多人那样让一个 Emacs 会话连续运行数天，它会堆积起你几天前用过的缓冲区。命令 ‘M-x clean-buffer-list’ 是清理它们的一种便捷方式；它会删除所有长时间未使用且未修改的缓冲区。普通缓冲区若已三天未被显示就会被删除；不过，你可以指定某些缓冲区绝不应被自动删除，另一些缓冲区则只要一小时未使用就应删除。这些默认值以及该命令行为的其他方面，可通过定制 ‘clean-buffer-list’ 文档字符串中描述的若干选项来控制。

你还可以启用 Midnight 模式，让它每天为你清理一次缓冲区。Midnight 模式每天午夜运行；届时它会运行 ‘clean-buffer-list’，或运行你放入普通钩子 ‘midnight-hook’ 中的任何函数（参见“钩子”）。要启用 Midnight 模式，请用定制缓冲区把变量 ‘midnight-mode’ 设为 ‘t’。参见“简易自定义界面”。

21.5 对多个缓冲区操作

‘M-x buffer-menu’ 开始编辑列出所有 Emacs 缓冲区的缓冲区列表。‘M-x buffer-menu-other-window’ 类似，但在另一个窗口中进行。

由 ‘C-x C-b’ 打开的“缓冲区菜单”（参见“列出已有缓冲区”）不只是列出缓冲区。它还让你通过类似 Dired 的界面（参见“Dired 目录编辑器”）对缓冲区执行各种操作。你可以保存缓冲区、删除它们（这里称为“删除”，以与 Dired 保持一致），或显示它们。

要使用缓冲区菜单，输入‘C-x C-b’并切换到显示‘*Buffer List*’缓冲区的窗口。你也可以输入‘M-x buffer-menu’在选中窗口中打开缓冲区菜单。另外，命令‘M-x buffer-menu-other-window’会在另一个窗口中打开缓冲区菜单，并选中那个窗口。

缓冲区菜单是只读缓冲区，只能通过本节所述的特殊命令来更改。该缓冲区中可以使用通常的光标移动命令。下列命令作用于当前行所述的缓冲区：

‘d’把该缓冲区标记为删除（杀死），然后把光标点移到下一行（‘Buffer-menu-delete’）。删除标记由该行缓冲区名之前的字符‘D’指示。只有在你输入‘x’命令时（见下）才会真正删除。

‘C-d’类似‘d’，但光标点向上移而不是向下移（‘Buffer-menu-delete-backwards’）。

‘s’把该缓冲区标记为保存（‘Buffer-menu-save’）。保存标记由该行缓冲区名之前的字符‘S’指示。只有在你输入‘x’时才会真正保存。你可以对同一缓冲区同时请求保存和删除。

‘x’执行所有已标记的删除和保存（‘Buffer-menu-execute’）。

‘u’去掉当前行的所有标记，并向下移动（‘Buffer-menu-unmark’）。带前缀参数时，去掉标记后向上移动。

‘<DEL>’移到上一行并去掉该行的所有标记（‘Buffer-menu-backup-unmark’）。

‘M-<DEL>’从所有行上去掉某一种标记（‘Buffer-menu-unmark-all-buffers’）。它会询问一个字符，然后取消带有该字符标记的缓冲区；输入 <RET> 可去掉所有标记。

‘U’去掉所有行上的所有标记（‘Buffer-menu-unmark-all’）。

去掉标记的命令‘d’和‘C-d’接受数字参数作为重复次数。

下列命令立即作用于当前行列出的缓冲区。它们也接受数字参数作为重复次数。

‘~’把缓冲区标记为未修改（‘Buffer-menu-not-modified’）。参见“保存文件的命令”。

‘%’切换缓冲区的只读状态（‘Buffer-menu-toggle-read-only’）。参见“其他缓冲区操作”。

‘t’把该缓冲区作为标签表访问（‘Buffer-menu-visit-tags-table’）。参见“选择标签表”。

下列命令用于选择另一个或另一些缓冲区：

‘q’退出缓冲区菜单（‘quit-window’）。最近曾可见的缓冲区会显示在原处。

‘<RET>’‘f’选中该行的缓冲区，替换其窗口中的‘*Buffer List*’缓冲区（‘Buffer-menu-this-window’）。

‘o’在另一个窗口中选中该行的缓冲区，如同使用‘C-x 4 b’，并让‘*Buffer List*’保持可见（‘Buffer-menu-other-window’）。

‘C-o’在另一个窗口中显示该行的缓冲区，但不选中它（‘Buffer-menu-switch-other-window’）。

‘1’在全框架窗口中选中该行的缓冲区（‘Buffer-menu-1-window’）。

‘2’在当前框架上设置两个窗口，其中一个选中该行的缓冲区，另一个显示先前为当前的缓冲区（‘*Buffer List*’除外）（‘Buffer-menu-2-window’）。

‘b’埋藏该行的缓冲区（‘Buffer-menu-bury’），即把它移到缓冲区列表的末尾。

‘m’ 把该行的缓冲区标记为：在你用 ‘v’ 命令退出时显示在另一个窗口中（‘Buffer-menu-mark’）。显示标记由行首的字符 ‘>’ 指示。（单个缓冲区不能同时带有删除标记和显示标记。）

‘v’ 选中该行的缓冲区，并把所有用 ‘m’ 命令标记的缓冲区也显示在其他窗口中（‘Buffer-menu-select’）。若你没有标记任何缓冲区，该命令等价于 ‘1’。

下列命令影响整个缓冲区列表：

‘S’ 按光标点所在列的值对缓冲区菜单条目排序。带数字前缀参数 N 时，按第 N 列排序（‘tabulated-list-sort’）。

‘}’ 把当前列宽增加 N（前缀数字参数）个字符。

‘{’ 把当前列宽减少 N（前缀数字参数）个字符。

‘T’ 删除或重新插入非文件缓冲区的行（‘Buffer-menu-toggle-files-only’）。该命令切换是否在缓冲区列表中包含这类缓冲区。

‘I’ 切换内部缓冲区（即名称以空格开头的缓冲区）的显示。

通常，当缓冲区被创建和删除时，‘*Buffer List*’ 缓冲区不会自动更新；其内容只是文本。若你创建、删除或重命名了缓冲区，要更新 ‘*Buffer List*’ 以反映你的改动，可输入 ‘g’（‘revert-buffer’）。若你在该缓冲区中启用 Auto Revert 模式（只要它未被标记为已修改），就可以让它每隔 ‘auto-revert-interval’ 秒自动执行一次。全局 Auto Revert 模式仅在 ‘global-auto-revert-non-file-buffers’ 非 ‘nil’ 时才作用于 ‘*Buffer List*’ 缓冲区。参见“Auto Reverting the Buffer Menu, for details”

21.6 间接缓冲区

“间接缓冲区”（indirect buffer）共享另一个缓冲区的文本，后者称为该间接缓冲区的“基础缓冲区”（base buffer）。在某些方面，它类似于文件间符号链接的缓冲区版本。

‘M-x make-indirect-buffer <RET> BASE-BUFFER <RET> INDIRECT-NAME <RET>’ 创建名为 INDIRECT-NAME、基础缓冲区为 BASE-BUFFER 的间接缓冲区。‘M-x clone-indirect-buffer <RET>’ 创建当前缓冲区的孪生副本作为间接缓冲区。‘C-x 4 c’ 创建当前缓冲区的孪生副本作为间接缓冲区，并在另一个窗口中选中它（‘clone-indirect-buffer-other-window’）。

间接缓冲区的文本始终与其基础缓冲区的文本相同；编辑其中任一个所做的改动都会立即在另一个中可见。这里的“文本”既包括字符，也包括它们的文本属性。但在其他所有方面，间接缓冲区与其基础缓冲区完全独立。它们可以有不同的名称、不同的光标点位置、不同的窄化范围、不同的标记、不同的覆盖层、不同的主模式和不同的局部变量。

间接缓冲区不能访问文件，但其基础缓冲区可以。若你试图保存间接缓冲区，实际上是通过保存其基础缓冲区来完成。删除基础缓冲区实际上会删除间接缓冲区，但删除间接缓冲区对其基础缓冲区没有影响。

使用间接缓冲区的一种方式是显示大纲的多个视图。参见“在多个视图中查看单个大纲”。

创建间接缓冲区的一种快捷方法是使用命令 ‘C-x 4 c’（‘clone-indirect-buffer-other-window’）。它创建并选中一个以当前缓冲区为基础缓冲区的间接缓冲区。带数字参数时，它会提示输入间接缓冲区的名称；否则它使用当前缓冲区的名称，并加上 ‘<N>’ 后缀。

创建间接缓冲区的更通用方法是使用命令‘M-x make-indirect-buffer’。它从缓冲区 BASE-BUFFER 创建名为 INDIRECT-NAME 的间接缓冲区，两者都用迷你缓冲区提示输入。

创建间接缓冲区的函数会在创建间接缓冲区之后运行钩子‘clone-indirect-buffer-hook’。该钩子运行时，新创建的间接缓冲区是当前缓冲区。

注意：当对缓冲区文本做了修改时，修改钩子只会在基础缓冲区中运行，因为那些钩子上的大多数函数并未准备好在间接缓冲区中正确工作。因此，若你需要在间接缓冲区中使用某个修改钩子函数，你需要手动把该函数加入*基础缓冲区*中的钩子，然后让该函数在所需的间接缓冲区中起作用。

21.7 缓冲区处理的便利功能与定制

本节介绍若干模式与功能，它们让在缓冲区之间切换更为方便。

21.7.1 使缓冲区名称唯一

当多个缓冲区访问同名文件时，Emacs 必须给这些缓冲区取不同的名称。默认方法会添加一个后缀，该后缀基于包含这些文件的目录名称。例如，若你同时访问文件‘/foo/bar/mumble/name’和‘/baz/quux/mumble/name’，它们的缓冲区将分别命名为‘name<bar/mumble>’和‘name<quux/mumble>’。Emacs 会添加所需数量的目录部分，以构成唯一的名称。

通过定制选项‘uniquify-buffer-name-style’，你可以从多种不同的风格中选择如何构造唯一的缓冲区名称。

‘forward’命名方法把文件目录名的一部分放在缓冲区名称的开头；使用这种方法，访问文件‘/u/rms/tmp/Makefile’和‘/usr/projects/zaphod/Makefile’的缓冲区将命名为‘tmp/Makefile’和‘zaphod/Makefile’。

相比之下，‘post-forward’命名方法会把这两个缓冲区称为‘Makefile|tmp’和‘Makefile|zaphod’。默认方法‘post-forward-angle-brackets’与‘post-forward’类似，只是把唯一路径用尖括号括起来。‘reverse’命名方法会把它们称为‘Makefile\tmp’和‘Makefile\zaphod’。当你仅用一个目录名不足以区分两个文件时，‘post-forward’与‘reverse’之间的细微差别就显现出来：此时‘reverse’按相反顺序排列目录名，于是‘/top/middle/file’变成‘file\middle\top’，而‘post-forward’则按正序把它们放在文件名之后，如‘file|top/middle’。若‘uniquify-buffer-name-style’设为‘nil’，则缓冲区名称只会被追加‘<2>’、‘<3>’等。

‘uniquify-buffer-name-style’的值可以设为一个定制函数，该函数接受两个参数 BASE 和 EXTRA-STRINGS，其中 BASE 是字符串，EXTRA-STRINGS 是字符串列表。例如，‘post-forward-angle-brackets’当前的实现可以是：

```
(defun my-post-forward-angle-brackets (base extra-string)
  (concat base \"<\" (mapconcat #'identity extra-string \"/\") \">\"))
```

如果你打算在键入某个缓冲区名称之前先*看*一眼缓冲区名称，那么采用哪条规则把目录名放进缓冲区名称并不很重要。但作为有经验的用户，如果你了解规则，就不必再看。于是你可能会发现，某条规则对你来说更容易记住并快速应用。

21.7.2 快速迷你缓冲区选择

Icomplete 模式提供了一种便捷方式，可快速从迷你缓冲区中的候选补全里选择一个元素。启用后，在迷你缓冲区中键入内容会持续显示与所键入字符串匹配的候选补全列表。

任何时刻你都可以键入 'C-j' 来选择列表中的第一个补全。因此，选择某个特定补全的办法就是让它成为列表中的第一项。有两种做法：你可以多键入一些补全名称，从而缩小列表范围，排除所需项之上的不想要的补全；或者，你可以用 'C-.' 和 'C-,' 轮转列表，直到所需缓冲区排在第一位。

'M-<TAB>' 会选择列表中的第一个补全，与 'C-j' 类似，但不退出迷你缓冲区，因此你可以继续编辑它。这通常用于输入文件名时，此时可多次使用 'M-<TAB>' 逐层深入目录层级。

要为迷你缓冲区启用 Icomplete 模式，键入 'M-x icomplete-mode'，或把变量 'icomplete-mode' 定制为 't'（参见"简易自定义界面"）。

你还可以通过把变量 'icomplete-in-buffer' 定制为 't'，为 'C-M-i'（命令 'completion-at-point'）额外启用 Icomplete 模式。对于缓冲区内的补全，变量 'completion-auto-help' 控制 Icomplete 模式何时显示可能的补全。其默认值 't' 表示在你首次键入 'C-M-i' 时就显示可能的补全。

默认情况下，当你按下 'C-M-i' 时，Icomplete 模式在缓冲区中显示的候选补全和 '*Completions*' 缓冲区都会出现。如果你使用了 'icomplete-in-buffer'，那么你可能希望抑制 '*Completions*' 缓冲区的这种出现。为此，请把以下内容加入你的初始化文件（参见"Emacs 初始化文件"）：

```
(advice-add 'completion-at-point :after #'minibuffer-hide-completions)
```

Icomplete 模式的替代方案是 Fido 模式。它与 Icomplete 模式非常相似，但保留了名为 Ido 模式的流行扩展的某些功能（事实上该名称源自"Fake Ido"）。除其他方面外，在 Fido 模式中，'C-s' 和 'C-r' 也用于轮转补全列表，'C-k' 可用于在列表中删除文件和删除缓冲区。另一个值得注意的方面是，默认补全风格为 'flex'（参见"补全候选项如何选定"）。要改变这一点，请把以下内容加入你的初始化文件（参见"Emacs 初始化文件"）：

```
(defun my-icomplete-styles ()
  (setq-local completion-styles '(initials flex)))
(add-hook 'icomplete-minibuffer-setup-hook 'my-icomplete-styles)
```

要启用 Fido 模式，键入 'M-x fido-mode'，或把变量 'fido-mode' 定制为 't'（参见"简易自定义界面"）。

默认情况下，Icomplete 模式和 Fido 模式在提示符所在的那一行显示可能的补全。要在提示符下方垂直显示补全候选，键入 'M-x icomplete-vertical-mode'，或把变量 'icomplete-vertical-mode' 定制为 't'（参见"简易自定义界面"）。

21.7.3 定制缓冲区菜单

- 'M-x bs-show'：类似于 'M-x list-buffers' 列出缓冲区，但显示方式可定制。
- 'M-x ibuffer'：列出缓冲区，并以类似 Dired 的方式对它们进行操作。

'M-x bs-show' 弹出与通常由 'C-x C-b' 显示的缓冲区列表相似的缓冲区列表，但其显示可以更灵活地定制。例如，你可以指定要显示的缓冲区属性列表、缓冲区名称列的最小和最大宽度、用于匹配永不显示的缓冲区名称的正则表达式，以及始终显示的缓冲区名称的正则表达式，等等。如果你更喜欢它而不是通常的缓冲区列表，

可以把该命令绑定到 'C-x C-b'。要定制该缓冲区列表，使用 'bs' 定制组（参见"简易自定义界面"），或调用 'bs-customize'。

MSB 全局次模式（"MSB" 代表 "mouse select buffer"）提供了另一种可定制的鼠标缓冲区菜单，你可能会更喜欢。它用自己的命令替换了通常绑定到 'C-Down-mouse-1' 和 'C-<F10>' 的 'mouse-buffer-menu' 命令，并且还会修改菜单栏的缓冲区菜单。你可以在 'msb' 定制组中定制该菜单。

IBuffer 是一个主模式，用于查看缓冲区列表并以类似 Dired 的方式对它们进行操作（参见"Dired 目录编辑器"），包括过滤、标记、以多种方式排序以及对缓冲区执行操作。

22 多窗口

Emacs 可以把一个框架拆分成两个或多个窗口。多个窗口可以显示不同缓冲区的内容，也可以显示同一个缓冲区的不同部分。多个框架总是意味着多个窗口，因为每个框架都有自己的一组窗口。每个窗口属于且仅属于一个框架。

22.1 Emacs 窗口的概念

每个 Emacs 窗口在任何时刻都显示一个 Emacs 缓冲区。同一个缓冲区可能出现在多个窗口中；若如此，其文本的任何改动都会显示在它出现的所有窗口中。但这些窗口可以显示缓冲区的不同部分，因为每个窗口都有自己的光标点值。

在任何时刻，都有一个 Emacs 窗口是“选中窗口”（selected window）；该窗口显示的缓冲区就是当前缓冲区。在图形显示上，光标点在选中窗口中表现为实心闪烁的光标，在未选中窗口中表现为空心方框。在文本终端上，只在选中窗口中绘制光标。参见“光标的显示”。

移动光标点的命令只影响选中 Emacs 窗口中的光标点值。它们不改变其他 Emacs 窗口（即使是显示同一缓冲区的窗口）中的光标点值。诸如 ‘C-x b’ 这样的切换缓冲区命令也是如此；它们完全不影响其他窗口。不过，还有其他命令，例如 ‘C-x 4 b’，会选择另一个窗口并在其中切换缓冲区。此外，所有在窗口中显示信息的命令，包括（例如）‘C-h f’（‘describe-function’）和 ‘C-x C-b’（‘list-buffers’），通常都在非选中窗口中显示缓冲区，而不影响选中窗口。

当多个窗口显示同一个缓冲区时，它们可以有不同的区域，因为它们可以有不同的光标点值。不过它们的标记值都相同，因为每个缓冲区只有一个标记位置。

每个窗口都有自己的模式行，显示该窗口中所显示缓冲区的名称、修改状态以及主模式和次模式。选中窗口的模式行以不同的颜色显示。详见参见“模式行”。

22.2 拆分窗口

- ‘C-x 2’：把选中窗口拆分为上下两个窗口（‘split-window-below’）。
- ‘C-x 3’：把选中窗口拆分为并排的两个窗口（‘split-window-right’）。
- ‘C-mouse-2’：在某个窗口的模式行中点击，拆分该窗口。

‘C-x 2’（‘split-window-below’）把选中窗口拆分为上下两个窗口。拆分后，选中窗口是上面那个，新拆出的窗口在下面。两个窗口具有与之前相同的光标点值，并显示缓冲区的同一部分（或尽可能接近的部分）。如有必要，窗口会滚动以使光标点保持在屏幕上。默认情况下，两个窗口各占原窗口高度的一半。正数值参数指定给上面窗口多少行；负数值参数指定给下面窗口多少行。

如果你把变量 ‘split-window-keep-point’ 改为 ‘nil’，‘C-x 2’ 就会改为调整两个窗口所显示的缓冲区部分以及各窗口中光标点的值，以便让屏幕上的文本尽可能接近原来的样子；此外，如果光标点原先在原窗口的下半部分，则会选中下面那个窗口而不是上面那个。

‘C-x 3’（‘split-window-right’）把选中窗口拆分为并排的两个窗口。左边窗口是选中窗口；右边窗口显示同一缓冲区的同一部分，并具有相同的光标点值。正数值参数指定给左边窗口多少列；负数值参数指定给右边窗口多少列。

用 ‘C-x 3’ 拆分窗口时，得到的每个窗口都占不满框架的整个宽度。若窗口变得太窄，在使用续行时缓冲区可能难以阅读（参见“续行”）。因此，若窗口宽度窄于 50 列，Emacs 会自动切换到截断行。这种截断与变量 ‘truncate-lines’ 的值无关（参见“行截断”）；它改由变量 ‘truncate-partial-width-windows’ 控制。若该变量的值为正整数（默认是 50），它指定了在自动截断行发生之前部分宽度窗口的最小总宽度；若值为 ‘nil’，则禁用自动截断行；对于其他任何非 ‘nil’ 值，Emacs 会在每个部分宽度窗口中截断行，无论其宽度如何。窗口的总宽度以 ‘window-total-width’ 所报告的列单位计（参见《Emacs Lisp 参考手册》的“Window Sizes”），它包括边缘、续行与截断字形、边距以及滚动条。

在文本终端上，并排的窗口由一条竖分隔线隔开，该分隔线用 ‘vertical-border’ 字体（face）绘制。

如果你在某个窗口的模式行上点击 ‘C-mouse-2’，就会拆分该窗口，并在你点击的位置放置一条竖分隔线。取决于 Emacs 的编译方式，你也可以通过在滚动条上点击 ‘C-mouse-2’ 来拆分窗口，这会在点击的位置放置一条水平分隔线（当 Emacs 使用 GTK+ 滚动条时此功能不可用）。

默认情况下，拆分窗口时，Emacs 会让得到的每个窗口的尺寸成为框架默认字体大小的整数倍。这可能使屏幕空间在得到的窗口之间分配不均。若你把变量 ‘window-resize-pixelwise’ 设为非 ‘nil’ 值，Emacs 会给每个窗口相同的像素数（若初始尺寸为奇数像素，则相差一个像素）。注意，当框架的像素尺寸不是框架字符尺寸的整数倍时，即使该选项为 ‘nil’，也至少会有一个窗口被按像素调整大小。

22.3 使用其他窗口

- ‘C-x o’：选中另一个窗口（‘other-window’）。
- ‘C-x O’：向后选中另一个窗口（‘other-window-backward’）。
- ‘C-M-v’：把下一个窗口向上滚动（‘scroll-other-window’）。
- ‘C-M-S-v’：把下一个窗口向下滚动（‘scroll-other-window-down’）。
- ‘C-M-S-l’：把下一个窗口重置到中央（‘recenter-other-window’）。
- ‘mouse-1’：在窗口的文本区中，‘mouse-1’ 会选中该窗口并把光标点移到点击的位置。在模式行中点击则选中该窗口，但不移动其中的光标点。

使用键盘时，你可以输入 ‘C-x o’（‘other-window’）来切换窗口。那是 “other” 的 ‘o’，不是零。当窗口多于两个时，该命令按循环顺序遍历所有窗口，通常从上到下、从左到右。经过最右边和最下面的窗口之后，它会回到左上角的窗口。数值参数表示按窗口的循环顺序移动若干步。负参数则按相反顺序沿循环移动。当迷你缓冲区处于活动状态时，迷你缓冲区窗口是循环中的最后一个窗口；你可以从迷你缓冲区窗口切换到其他某个窗口，之后再切回来，完成所要求的迷你缓冲区参数输入。参见“在迷你缓冲区中编辑”。‘C-x O’ 与之类似，但默认按相反顺序。

‘other-window’ 命令通常只在当前框架中切换到下一个窗口（除非另有配置）。如果你在多框架环境中工作，并希望所有框架中的窗口都参与循环，可以把 ‘C-x o’ 重新绑定到 ‘next-window-any-frame’ 命令。（关于如何重新绑定命令，参见“交互式更改键绑定”。）

通常的滚动命令（参见“控制显示”）只作用于选中窗口，但也有一些命令用于滚动下一个窗口。‘C-M-v’（‘scroll-other-window’）滚动‘C-x o’会选中的那个窗口。在其他方面，该命令的行为与‘C-v’类似；两者都把缓冲区文本相对于窗口向上移动，并接受正参数和负参数。（在迷你缓冲区中，‘C-M-v’滚动与迷你缓冲区关联的帮助窗口（若有），而不是标准循环顺序中的下一个窗口；参见“在迷你缓冲区中编辑”。）‘C-M-S-v’（‘scroll-other-window-down’）以类似方式把下一个窗口向下滚动。同样，‘C-M-S-l’（‘recenter-other-window’）在下一个窗口中的行为与‘C-l’（‘recenter-top-bottom’）类似。

若你把‘mouse-autoselect-window’设为非‘nil’值，把鼠标移到另一个窗口上就会选中该窗口。此功能默认关闭。

22.4 在另一个窗口中显示

‘C-x 4’是一个前缀键，用于各种在另一个窗口中切换到缓冲区的命令——那个窗口可以是另一个已存在的窗口，也可以是拆分选中窗口而新建的窗口。关于 Emacs 如何选择或创建要用的窗口，参见“‘display-buffer’的工作方式”。

- ‘C-x 4 b BUFNAME <RET>’：在另一个窗口中选中缓冲区 BUFNAME（‘switch-to-buffer-other-window’）。参见“创建与选择缓冲区”。
- ‘C-x 4 C-o BUFNAME <RET>’：在某个窗口中显示缓冲区 BUFNAME，但不试图选中它（‘display-buffer’）。关于窗口如何选取的细节，参见“在窗口中显示缓冲区”。
- ‘C-x 4 f FILENAME <RET>’：访问文件 FILENAME，并在另一个窗口中选中其缓冲区（‘find-file-other-window’）。参见“访问文件”。
- ‘C-x 4 d DIRECTORY <RET>’：在另一个窗口中为目录 DIRECTORY 选中一个 Dired 缓冲区（‘dired-other-window’）。参见“Dired 目录编辑器”。
- ‘C-x 4 m’：开始撰写邮件消息，类似‘C-x m’（参见“发送邮件”），但在另一个窗口中进行（‘compose-mail-other-window’）。
- ‘C-x 4 .’：查找某个标识符的定义，类似‘M-.’（参见“查找标识符引用”），但在另一个窗口中进行（‘xref-find-definitions-other-window’）。
- ‘C-x 4 r FILENAME <RET>’：以只读方式访问文件 FILENAME，并在另一个窗口中选中其缓冲区（‘find-file-read-only-other-window’）。参见“访问文件”。
- ‘C-x 4 4’：一个更通用的前缀命令，影响此前缀命令之后调用的下一个命令所显示的缓冲区（‘other-window-prefix’）。它要求把后续命令显示的缓冲区显示在另一个窗口中。
- ‘C-x 4 1’：这个通用前缀命令要求把下一个命令的缓冲区显示在同一个窗口中。

22.5 删除与调整窗口大小

- ‘C-x 0’：删除选中窗口（‘delete-window’）。
- ‘C-x 1’：删除选中框架中除选中窗口之外的所有窗口（‘delete-other-windows’）。
- ‘C-x 4 0’：删除选中窗口，并杀死其中原先显示的缓冲区（‘kill-buffer-and-window’）。该键序列的最后一个字符是零。
- ‘C-x w 0 <RET> BUFFER <RET>’：删除显示指定 BUFFER 的窗口。

- ‘C-x ^’：使选中窗口变高（‘enlarge-window’）。
- ‘C-x }’：使选中窗口变宽（‘enlarge-window-horizontally’）。
- ‘C-x {’：使选中窗口变窄（‘shrink-window-horizontally’）。
- ‘C-x -’：若窗口的缓冲区不需要那么多行，则缩小该窗口（‘shrink-window-if-larger-than-buffer’）。
- ‘C-x +’：平衡选中框架中所有窗口的大小（‘balance-windows’）。

要删除选中窗口，输入‘C-x 0’（‘delete-window’）。（那是零。）窗口一旦被删除，它所占的空间就交给相邻的窗口（但不交给迷你缓冲区窗口，即使当时它处于活动状态）。删除窗口对它原先显示的缓冲区没有影响；缓冲区继续存在，你仍可以用‘C-x b’切换到它。选项‘delete-window-choose-selected’控制改为选中哪个窗口作为新的选中窗口（参见《Emacs Lisp 参考手册》的“Deleting Windows”）。

‘C-x 4 0’（‘kill-buffer-and-window’）是比‘C-x 0’更强的命令；它杀死当前缓冲区，然后删除选中窗口。

‘C-x 1’（‘delete-other-windows’）删除*除选中窗口之外*的所有窗口；选中窗口扩展以使用整个框架。（当迷你缓冲区窗口活动时不能使用此命令；试图这样做会报错。）

‘M-x delete-windows-on’删除显示特定缓冲区的窗口。它会提示输入缓冲区，默认是当前缓冲区。带零前缀参数‘C-u 0’时，该命令只删除当前显示器各框架上的窗口。

‘C-x ^’（‘enlarge-window’）命令使选中窗口高一行，从竖直相邻的窗口取空间，而不改变框架高度。带正数值参数时，该命令按那么多行增加窗口高度；带负参数时，按那么多行减少高度。若没有竖直相邻的窗口（即该窗口已是整个框架的高度），则会报错。若你试图把任何窗口的高度减到低于变量‘window-min-height’所指定的某个最小行数（默认是 4），该命令也会报错。

类似地，‘C-x }’（‘enlarge-window-horizontally’）使选中窗口变宽，‘C-x {’（‘shrink-window-horizontally’）使它变窄。若你试图把任何窗口的宽度减到低于变量‘window-min-width’所指定的某个最小列数（默认是 10），这些命令会报错。

在模式行上（参见“模式行鼠标命令”）或在窗口分隔线上（参见“窗口分隔条”）点击鼠标，是改变窗口高度以及拆分或删除窗口的另一种办法。

‘C-x -’（‘shrink-window-if-larger-than-buffer’）在选中窗口高于显示其缓冲区全部文本所需的高度时，减小该窗口的高度。它把多出的行让给框架中的其他窗口。

你还可以用‘C-x +’（‘balance-windows’）来平衡选中框架中所有窗口的大小（迷你缓冲区窗口除外，参见“迷你缓冲区”）。该命令使每一对水平相邻的窗口高度相同，每一对竖直相邻的窗口宽度相同。

22.6 在窗口中显示缓冲区

响应用户命令而显示或弹出某个缓冲区，是 Emacs 中常见的操作。命令执行这种操作的方式有几种不同的途径。

许多命令，如‘C-x C-f’（‘find-file’），默认通过“接管”选中窗口来显示缓冲区，它预期用户的注意力会转向那个缓冲区。

有些命令试图智能地显示，尽量不接管选中窗口，例如拆分出一个新窗口并在那里显示所需的缓冲区。这类命令包括各种帮助命令（参见“帮助”），它们在内部调用‘display-buffer’来工作。细节参见“‘display-buffer’的工

作方式”。

另一些命令做与 ‘display-buffer’ 相同的事，并额外选中显示窗口，以便你可以开始编辑其中的缓冲区。命令 ‘M-g M-n’（‘next-error’）就是一个例子（参见“编译模式”）。这类命令在内部调用函数 ‘pop-to-buffer’ 来工作。参见《Emacs Lisp 参考手册》的“Switching Buffers”。

名字以 ‘-other-window’ 结尾的命令行为类似 ‘display-buffer’，只是它们从不在选中窗口中显示，可能会拆分选中窗口以创建新窗口（参见“‘display-buffer’ 的工作方式”）。其中若干命令绑定在 ‘C-x 4’ 前缀键中（参见“在另一个窗口中显示”）。

名字以 ‘-other-frame’ 结尾的命令行为类似 ‘display-buffer’，只是它们 (i) 从不在选中窗口中显示，(ii) 更愿意创建新框架或使用其他框架上的窗口来显示所需的缓冲区。其中若干命令绑定在 ‘C-x 5’ 前缀键中。

有时，某个窗口“专用于”其当前缓冲区。参见《Emacs Lisp 参考手册》的“Dedicated Windows”。‘display-buffer’ 大多数时候会避免复用专用窗口。这在模式行中以 ‘d’ 表示（参见“模式行”）。窗口也可以是强专用的，这会阻止对窗口中显示的缓冲区做任何更改。这在模式行中以 ‘D’ 表示。

通常，专用窗口用于显示专用缓冲区，但专用化有时在交互中很有用。例如，用 ‘M-g M-n’ ‘next-error’ 查看错误时，新显示的源代码可能替换掉你想参考的缓冲区。若你把某个窗口专用于那个缓冲区，该命令（通过 ‘display-buffer’）就会更愿意改用别的窗口。

你可以用命令 ‘C-x w d’（‘toggle-window-dedicated’）来切换选中窗口是否专用于当前缓冲区。带前缀参数时，它使该窗口成为强专用。

22.6.1 ‘display–buffer’ 的工作方式

‘display-buffer’ 命令（以及在内部调用它的命令）按下面给出的步骤选择要显示的窗口。参见《Emacs Lisp 参考手册》的“Choosing Window, for details about how to alter this sequence of steps”

- 若无论其他因素如何，缓冲区都应显示在选中窗口中，就复用选中窗口。默认情况下会跳过这一步，但你可以把匹配该缓冲区名称的正则表达式以及对 ‘display-buffer-same-window’ 动作函数的引用（参见《Emacs Lisp 参考手册》的“Buffer Display Action Functions”）加入选项 ‘display-buffer-alist’（参见《Emacs Lisp 参考手册》的“Choosing Window”），以告诉 Emacs 不要跳过它。例如，要把缓冲区 ‘*scratch*’ 优先显示在选中窗口中，可以写：

```
(setopt
 display-buffer-alist
 '(("\\*scratch\\*" (display-buffer-same-window))))
```

默认情况下，‘display-buffer-alist’ 为 ‘nil’。

- 否则，若缓冲区已经显示在某个已存在的窗口中，就复用那个窗口。通常只考虑选中框架上的窗口，但若你使用相应的 ‘reusable-frames’ 动作列表项（参见《Emacs Lisp 参考手册》的“Buffer Display Action Alists”），其他框架上的窗口也可以复用。关于如何做到这一点，见下一步骤中的示例。
- 否则，可以选择创建新框架并在其中显示缓冲区。默认情况下会跳过这一步。要启用它，按如下方式更改选项 ‘display-buffer-base-action’ 的值（参见《Emacs Lisp 参考手册》的“Choosing Window”）：

```
(setopt
 display-buffer-base-action
 '((display-buffer-reuse-window display-buffer-pop-up-frame)
   (reusable-frames . 0)))
```

这种定制还会试着让前一步骤在所有可见或图标化的框架上搜索可复用的窗口。

- 否则，尝试通过拆分选中框架上的某个窗口来创建新窗口，并在该新窗口中显示缓冲区。

拆分可以是竖直的，也可以是水平的，取决于变量‘split-height-threshold’和‘split-width-threshold’。这些变量应取整数值。若‘split-height-threshold’小于所选窗口的高度，拆分就把新窗口放在下面。否则，若‘split-width-threshold’小于窗口的宽度，拆分就把新窗口放在右边。若两个条件都不成立，Emacs 会试着让新窗口位于下面——但仅当该窗口此前未被拆分过时（以避免过度拆分）。当两个条件都成立时，Emacs 是先尝试竖直拆分还是水平拆分，由‘split-window-preferred-direction’的值决定。其默认值是‘longest’，意思是：若窗口所在框架高度大于宽度（“竖版”框架），就竖直拆分；若宽度大于高度（“横版”框架），就水平拆分。‘vertical’和‘horizontal’两个值则总是分别偏好竖直或水平拆分。

- 否则，在先前显示过该缓冲区的某个窗口中显示该缓冲区。通常只考虑选中框架上的窗口，但若有合适的‘reusable-frames’动作列表项（见上文），该窗口也可能位于其他框架上。
- 否则，在选中框架上的某个已存在窗口中显示该缓冲区。
- 若上述所有方法因某种原因都失败，则创建新框架并在其中显示缓冲区。

22.6.2 显示不可编辑的缓冲区

有些缓冲区显示在窗口中是为了供人阅读，而非供人编辑。帮助命令（参见“帮助”）通常为此使用名为‘*Help*’的缓冲区，迷你缓冲区补全（参见“补全”）使用名为‘*Completions*’的缓冲区，等等。这类缓冲区通常只显示很短一段时间。

通常，Emacs 通过‘display-buffer’为这类临时显示选择窗口，如前一小节所述。另一方面，‘*Completions*’缓冲区通常显示在选中框架底部的一个窗口中，无论该框架上已经显示了多个窗口。

如果你希望 Emacs 以不同方式显示临时缓冲区，请适当地定制变量‘display-buffer-alist’（参见《Emacs Lisp 参考手册》的“Choosing Window”）。例如，要让‘*Completions*’总是显示在选中窗口下方，可在初始化文件（参见“Emacs 初始化文件”）中使用如下形式：

```
(setopt
 display-buffer-alist
 '(("\\*Completions\\*" display-buffer-below-selected)))
```

‘*Completions*’缓冲区还有一个特殊之处：Emacs 通常尽量使其窗口大小刚好足以显示其全部内容。要调整显示其他临时显示内容（例如‘*Help*’缓冲区）的窗口大小，请开启次模式（参见“次模式”）‘temp-buffer-resize-mode’（参见《Emacs Lisp 参考手册》的“Temporary Displays”）。

‘temp-buffer-resize-mode’所调整窗口的最大尺寸，可通过定制选项‘temp-buffer-max-height’和‘temp-buffer-max-width’来控制（参见《Emacs Lisp 参考手册》的“Temporary Displays”），且不能超过包含它的框架的尺寸。

显示警告的缓冲区（例如字节编译警告，参见《Emacs Lisp 参考手册》的“Compilation Functions”）默认也显示在选中框架底部的一个窗口中。你可以用变量 ‘warning-display-at-bottom’ 控制这一点：若设为 ‘nil’，Emacs 将改用 ‘display-buffer’ 的常规逻辑（参见“‘display-buffer’ 的工作方式”），而你可以通过 ‘display-buffer-alist’ 定制它。

22.7 窗口处理的便利功能

Winner 模式是一个全局次模式，它记录窗口配置的改动（即框架如何划分成窗口），以便你可以撤销这些改动。你可以用 ‘M-x winner-mode’ 切换 Winner 模式，或通过定制变量 ‘winner-mode’ 来切换。启用该模式后，‘C-c left’（‘winner-undo’）撤销上一次窗口配置改动。若你在撤销过程中改变主意，可以用 ‘C-c right’（‘M-x winner-redo’）重做已撤销的改动。要阻止 Winner 模式绑定 ‘C-c left’ 和 ‘C-c right’，可把变量 ‘winner-dont-bind-my-keys’ 定制为非 ‘nil’ 值。默认情况下，Winner 模式为每个框架最多存储 200 个窗口配置，但你可以通过修改变量 ‘winner-ring-size’ 来改变这一点。若有些缓冲区的窗口你不希望 Winner 模式恢复，把它们的名字加入列表变量 ‘winner-boring-buffers’ 或正则表达式 ‘winner-boring-buffers-regexp’。

Follow mode（‘M-x follow-mode’）会同步同一缓冲区上的多个窗口，使它们始终显示该缓冲区中相邻的部分。参见“跟随模式”。

Windmove 包定义了一些命令，用于在框架中相邻的窗口之间按方向移动。‘M-x windmove-right’ 选择紧靠当前选中窗口右侧的窗口，左、上、下方向也有对应的命令。‘windmove-default-keybindings’ 把这些命令绑定到 ‘S-right’ 等按键上；这样做会禁用这些按键的 shift 选择功能（参见“Shift 选择”）。与为按方向选择窗口的命令定义键绑定类似，你可以用 ‘windmove-display-default-keybindings’ 为这样一类命令定义键绑定：它们指定下一个命令要显示的缓冲区在哪个方向上显示。此外还有 ‘windmove-delete-default-keybindings’，用于为按方向删除窗口的命令定义键绑定；以及 ‘windmove-swap-states-default-keybindings’，用于为这样一类命令定义键绑定：把选中窗口的窗口内容与指定方向上的窗口互换。

命令 ‘M-x compare-windows’ 让你比较不同窗口中显示的文本。参见“比较文件”。

Scroll All 模式（‘M-x scroll-all-mode’）是一个全局次模式，它使滚动命令和光标点移动命令作用于每一个窗口。

22.8 窗口标签行

命令 ‘global-tab-line-mode’ 切换在每个窗口顶部屏幕行上显示“标签行”（tab line）。标签行会为每个曾在该窗口中显示过的缓冲区显示特殊按钮（“标签”），点击对应的按钮即可切换到其中任何一个缓冲区。点击 ‘+’ 图标会向该窗口局部的缓冲区标签行中添加一个新缓冲区，点击某个标签的 ‘x’ 图标则会删除它。标签行上的鼠标滚轮可以水平滚动标签。

触摸屏输入（参见“触摸屏输入与虚拟键盘”）也可用于与“标签行”交互。长按（参见“在触摸屏上使用 Emacs”）某个标签会显示一个上下文菜单，其中的菜单项对所按的标签进行操作；点击标签本身会切换到该标签的缓冲区，而点击标签行上的按钮则等同于用 ‘mouse-1’ 点击。

选择前一个窗口局部标签等同于输入 ‘C-x <LEFT>’（‘previous-buffer’），选择下一个标签等同于 ‘C-x <RIGHT>’（‘next-buffer’）。这两个命令都支持数字前缀参数作为重复次数。

你可以自定义变量‘tab-line-tabs-function’来定义标签行首选的内容。默认情况下，它显示该窗口中此前访问过的所有缓冲区，如上所述。但你也可以把它设置为显示与当前缓冲区具有相同主模式的缓冲区列表，或者显示按主模式分组的缓冲区：此时点击第一个标签中的模式名称会显示所有主模式的列表，你可以在其中选择另一组缓冲区。

注意标签行与标签栏（Tab Bar）不同（参见“标签栏”）。每个框架顶部的标签栏中的标签用于在包含若干窗口及其缓冲区的窗口配置之间切换，而每个窗口顶部的标签行中的标签则用于在该窗口的缓冲区之间切换。

另请注意，标签行显示在与窗口工具栏相同的空间中，因此在任一时刻只能显示其中之一，除非你在 Lisp 中自定义‘tab-line-format’的值，把‘(:eval (tab-line-format))’添加到‘tab-line-format’中。参见《Emacs Lisp 参考手册》的“Mode Line Format”。

22.9 窗口工具栏

命令‘global-window-tool-bar-mode’切换在每个窗口顶部显示工具栏。启用后，多个窗口可以同时显示各自的工具栏。为节省空间，如果没有按钮可显示，即‘tool-bar-map’为‘nil’，窗口工具栏会被隐藏。

如果你只想为某些缓冲区切换窗口工具栏的显示，请在这些缓冲区中运行命令‘window-tool-bar-mode’。把它放进模式钩子中很有用。例如，如果你希望窗口工具栏只出现在不对应文件且带有自定义工具栏的缓冲区中，可以把以下代码添加到初始化文件中（参见“Emacs 初始化文件”）：

```
(add-hook 'special-mode-hook 'window-tool-bar-mode)
```

在图形显示上，窗口工具栏可以用几种不同的样式显示。默认情况下，窗口工具栏只把各项显示为图像。要指定某种样式，请自定义变量‘window-tool-bar-style’。

在纯文本显示上，窗口工具栏只显示每个按钮的文本，即使指定了其他样式也是如此。

Emacs 也可以在框架顶部显示单个工具栏（参见“工具栏”）。

注意窗口工具栏与标签行显示在相同的空间中，因此在任一时刻只能显示其中之一，除非你自定义‘tab-line-format’的值，把‘(:eval (window-tool-bar-string))’添加到‘tab-line-format’中。参见《Emacs Lisp 参考手册》的“Mode Line Format”。

23 框架与图形显示

当 Emacs 在图形显示环境（例如 X Window System）中启动时，它会占用一个图形系统级别的显示区域。本手册称之为“框架”（frame），而把“窗口”（window）一词保留给框架中用于显示缓冲区的部分。一个框架初始时包含一个窗口，但可以细分为多个窗口（参见“多窗口”）。框架通常还包含菜单栏、工具栏和回显区。

你也可以创建额外的框架（参见“创建框架”）。同一个 Emacs 会话中创建的所有框架都访问相同的底层缓冲区和其他数据。例如，若某个缓冲区显示在多个框架中，在其中任何一个框架里对它所做的修改都会立即出现在其他框架中。

输入 ‘C-x C-c’ 会关闭当前显示器上的所有框架，若 Emacs 会话在其他显示器上没有打开的框架，则结束该会话（参见“退出 Emacs”）。要只关闭选中的框架，输入 ‘C-x 5 0’（是数字零，不是字母 ‘o’）。

本章介绍 Emacs 专用于图形显示环境的特性（尤其是鼠标命令），以及管理多个框架的特性。在文本终端上，其中许多特性不可用。不过，在文本终端上仍然可以创建多个框架；这类框架一次只显示一个，占满整个终端屏幕（参见“文本终端”）。在某些文本终端上也可以使用鼠标（在 GNU 和 Unix 系统上的用法参见“在文本终端中使用鼠标”；在 MS-DOS 上的用法参见“MS-DOS 上的鼠标使用”）。所有文本终端都支持菜单。

23.1 鼠标编辑命令

‘mouse-1’ 把光标点移到你点击的位置（‘mouse-set-point’）。

‘Drag-mouse-1’ 激活你拖动选中的文本周围的区域，并把该文本放入主选择（primary selection）（‘mouse-set-region’）。

‘S-mouse-1’ 调整已经选中的区域。

‘mouse-2’ 把光标点移到你点击的位置，并在那里插入主选择的内容（‘mouse-yank-primary’）。

‘mouse-3’ 若区域处于活动状态，把区域中较近的一端移到点击位置；否则，把标记设在光标点当前的值处，并把光标点设在点击位置。把得到的区域存入删除环；再次点击会删除它（‘mouse-save-then-kill’）。

‘C-M-mouse-1’ 激活你拖动选中的文本周围的矩形区域。参见“矩形”。

最基础的鼠标命令是 ‘mouse-set-point’，在窗口的文本区用鼠标左键 ‘mouse-1’ 点击即可调用。它把光标点移到你点击的位置。若该窗口原本不是选中窗口，它将成为选中窗口。你也可以双击 ‘mouse-1’ 来激活区域（参见“面向单词和行的鼠标命令”）。

通常，若你点击的框架原本不是选中框架，那么除了选中窗口并设定光标之外，它还会成为选中框架。在 X Window System 上，你可以把变量 ‘x-mouse-click-focus-ignore-position’ 设为 ‘t’ 来改变这一点。此时，对未选中框架的第一次点击只选中该框架，不做其他事；再点击一次才会选中窗口并设定光标位置。

按住 ‘mouse-1’ 并在一段文本上拖动鼠标，会激活该文本周围的区域（‘mouse-set-region’），把标记放在你按下鼠标键的位置，把光标点放在你松开的位置（参见“标记与区域”）。此外，区域中的文本会成为主选择（参见“与其他窗口应用剪贴”）。

若把变量 ‘mouse-drag-copy-region’ 改为非 ‘nil’ 值，在一段文本上拖动鼠标还会把该文本加入删除环。默认值为 ‘nil’。

若该变量的值为 'non-empty'，则只有在区域非空时才复制到删除环。例如，若你拖动的区域不足半个字符，通常会在删除环中得到空字符串，而取 'non-empty' 时，这样短的拖动不会影响删除环。

若你启用了 Mouse-Shift-Adjust 次模式，按住 'Shift' 修饰键点击鼠标左键（'S-down-mouse-1'）会调整（按点击位置扩展或收缩）已选中的区域。

若你在拖动时把鼠标移出窗口的上边或下边，窗口会以恒定速率滚动，直到你把鼠标移回窗口内。这样你就能选中无法完全容纳在屏幕上的区域。每一步滚动的行数取决于鼠标离开窗口边缘多远；变量 'mouse-scroll-min-lines' 指定最小步长。

若你启用了选项 'mouse-drag-mode-line-buffer'，并且窗口系统支持拖动文件，那么在模式行上显示缓冲区名的部分拖动鼠标，会该缓冲区的文件拖到其他程序或框架中。

用鼠标中键 'mouse-2' 点击，会把光标点移到你点击的位置，并插入主选择的内容（'mouse-yank-primary'）。参见"与其他窗口应用剪贴"。这一行为与其他 X 应用程序一致。另一种做法是把 'mouse-2' 重新绑定到 'mouse-yank-at-click'，后者在你点击的位置执行粘贴。

若把变量 'mouse-yank-at-point' 改为非 'nil' 值，'mouse-2' 不会移动光标点；它把文本插入到光标点处，而不论你点击的是什么位置，甚至不论你点击的是框架中的哪个窗口。该变量同时影响 'mouse-yank-primary' 和 'mouse-yank-at-click'。

用鼠标右键 'mouse-3' 点击会运行命令 'mouse-save-then-kill'。它根据你点击的位置和区域的状态执行若干动作：

- 若没有活动区域，点击 'mouse-3' 会激活区域，把标记放在光标点原来所在的位置，把光标点放在点击位置。
- 若有活动区域，点击 'mouse-3' 会把区域中较近的一端移到点击位置，从而调整区域。调整后区域的文本会复制到删除环；若原区域中的文本已经在删除环中，则替换掉它。
- 若你最初是用双击或三击 'mouse-1' 指定区域的，因而该区域由整词或整行构成（参见"面向单词和行的鼠标命令"），那么用 'mouse-3' 调整区域时也以整词或整行为单位进行。
- 若你在同一位置连续第二次使用 'mouse-3'，就会删除已选中的区域。因此，用鼠标删除文本最简单的方法是：在一端点击 'mouse-1'，然后在另一端点击两次 'mouse-3'。要把文本复制到删除环而不从缓冲区中删除，只需按一次 'mouse-3'——或者直接用 'mouse-1' 在文本上拖过。之后你就可以把它粘贴到别处。

'mouse-save-then-kill' 命令也遵循变量 'mouse-drag-copy-region'（见上文）。若该值非 'nil'，那么每当该命令设定或调整活动区域时，区域中的文本也会加入删除环。若删除环中最近一项是以同样方式加入的，则替换该项，而不是新建一项。

无论你用上述哪条鼠标命令设定区域，随后任何不带 Shift 的光标移动命令都会取消标记的激活状态，这与取消标记激活的其他常规方式相同。参见"Shift 选择"。

有些鼠标带有可用于滚动的"滚轮"。在大多数图形显示环境中，Emacs 默认支持用鼠标滚轮滚动窗口。要切换该特性，使用 'M-x mouse-wheel-mode'。变量 'mouse-wheel-follow-mouse' 和 'mouse-wheel-scroll-amount' 决定滚动哪些缓冲区以及滚动多少。变量 'mouse-wheel-progressive-speed' 决定滚动速度是否与你转动滚轮的快慢相关。该模式还支持增大或减小字体大小，默认绑定到按住 <Ctrl> 修饰键滚动。启用该模式后，鼠标滚轮会产

生 ‘wheel-up’ 和 ‘wheel-down’ 这类特殊事件。（有些较老的系统把它们报告为 ‘mouse-4’ 和 ‘mouse-5’。）若鼠标有水平滚轮，还会产生 ‘wheel-left’ 和 ‘wheel-right’ 事件。

Emacs 也支持按住 <Shift> 修饰键进行水平滚动。在开始水平滚动之前输入数字前缀参数（例如 ‘M-5’），会改变由用户选项 ‘mouse-wheel-scroll-amount-horizontal’ 定义的步长值。

若鼠标的滚轮可以倾斜，或你的触摸板支持倾斜，那么你还可以把变量 ‘mouse-wheel-tilt-scroll’ 自定义为非 ‘nil’ 值来启用水平滚动。默认情况下，向某一方向倾斜滚轮会沿倾斜方向水平滚动窗口的视图：例如，向右倾斜使窗口向右滚动，于是窗口中显示的文本向左水平移动。若你希望反转水平滚动的方向，可把变量 ‘mouse-wheel-flip-direction’ 自定义为非 ‘nil’ 值。

当鼠标指针位于 Image 模式下的图像上时（参见“查看图像文件”），按住 <Ctrl> 修饰键滚动鼠标滚轮会缩放鼠标指针下的图像，按住 <Shift> 修饰键滚动鼠标滚轮会水平平移该图像。

23.2 面向单词和行的鼠标命令

这些 ‘mouse-1’ 的变体一次选中整个单词或整行。Emacs 会激活所选文本周围的区域，该文本同时被复制到删除环。

‘Double-mouse-1’ 选中你点击的单词或字符周围的文本。

双击具有符号语法（例如 C 模式中的下划线）的字符，会选中该字符周围的符号。双击具有左括号或右括号语法的字符，会选中该字符开始或结束的括号分组。双击具有字符串定界符语法（例如 C 中的单引号或双引号）的字符，会选中该字符串常量（Emacs 用启发式方法判断该字符是字符串的开头还是结尾）。

双击括号分组的开头或字符串定界符的开头，会把光标点移到区域的末尾；必要时向前滚动缓冲区显示，以显示光标点的新位置。双击括号分组的结尾或字符串定界符的结尾，默认把光标点留在区域末尾，因此若区域的开头在窗口顶部以上，它就不可见；把用户选项 ‘mouse-select-region-move-to-beginning’ 设为非 ‘nil’ 会改为把光标点移到区域开头，必要时向后滚动显示。

‘Double-Drag-mouse-1’ 以整词为单位选中你拖过的文本。

‘Triple-mouse-1’ 选中你点击的那一行。

‘Triple-Drag-mouse-1’ 以整行为单位选中你拖过的文本。

23.3 用鼠标跟随引用

有些 Emacs 缓冲区包含“按钮”或“超链接”：即一些文本片段，在激活时（例如点击它们时）执行某种动作（例如跟随某个引用）。通常，按钮的文本在视觉上会有强调：它带下划线，或周围画有方框。若你把鼠标移到按钮上，鼠标指针的形状会改变，按钮会亮起。若把变量 ‘mouse-highlight’ 改为 ‘nil’，Emacs 会禁用这种高亮。

你可以把光标点移到按钮上并按 <RET> 来激活它，也可以用 ‘mouse-1’ 或 ‘mouse-2’ 点击该按钮。例如，在 Dired 缓冲区中，每个文件名都是一个按钮；激活它会让 Emacs 访问该文件（参见“Dired 目录编辑器”）。在 ‘*Compilation*’ 缓冲区中，每条错误消息都是一个按钮，激活它会访问该错误对应的源代码（参见“在 Emacs 中运行编译”）。

虽然在按钮上点击‘mouse-1’通常会激活按钮，但若你在松开鼠标键之前按住一段时间（具体说，超过 450 毫秒），Emacs 会把光标点移到你点击的位置，而不激活按钮。这样，你就可以用鼠标把光标点移到按钮上而不激活它。在按钮上或拖过按钮拖动鼠标仍保持设定区域的常规行为，不会激活按钮。

你可以通过自定义变量‘mouse-1-click-follows-link’来改变‘mouse-1’作用于按钮的方式。若该值为正整数，它决定你需要按住鼠标键多长时间（以毫秒计）才会取消按钮激活；默认值为 450，如上一段所述。若该值为‘nil’，‘mouse-1’只在你点击的位置设定光标点，不激活按钮。若该值为‘double’，双击会激活按钮，而单击只设定光标点。

通常，在按钮上使用‘mouse-1’会激活按钮，即使该按钮位于未选中的窗口中。若把变量‘mouse-1-click-in-non-selected-windows’改为‘nil’，则对未选中窗口中按钮使用‘mouse-1’会把光标点移到点击位置并选中该窗口，而不激活按钮。

23.4 打开菜单的鼠标点击

若干带 <Ctrl> 和 <SHIFT> 修饰键的鼠标点击可弹出菜单。

‘C-mouse-1’该菜单用于选择缓冲区。

MSB（“mouse select buffer”）全局次模式使该菜单更智能、更可定制。参见“定制缓冲区菜单”。

‘C-mouse-2’该菜单包含查看字体（face）及其他文本属性以及设定它们的菜单项（后者主要在编辑富文本时有用；参见“富文本”）。

‘C-mouse-3’该菜单是模式相关的。对大多数模式，若 Menu-bar 模式开启，该菜单的菜单项与所有模式相关的菜单栏菜单合在一起相同。有些模式可能为该按键指定不同的菜单。若 Menu Bar 模式关闭，该菜单包含菜单栏中本来会出现的所有菜单项——不只是模式相关的那些——这样你不必显示菜单栏就能访问它们。

‘S-mouse-1’该菜单用于更改窗口缓冲区中的默认字体（face）。参见“文本缩放”。

许多 GUI 应用程序用‘mouse-3’显示“上下文菜单”：即提供对鼠标点击位置与上下文中各种相关设置和操作的访问的菜单。若你在 Emacs 中更喜欢这样，而不喜欢‘mouse-3’的默认功能（它绑定到‘mouse-save-then-kill’命令，参见“鼠标编辑命令”），你可以启用次模式‘context-menu-mode’。之后当你点击‘mouse-3’时，Emacs 会显示上下文菜单。这些上下文菜单的确切内容取决于当前主模式以及鼠标点击位置周围的缓冲区内容。要自定义上下文菜单的内容，可以使用变量‘context-menu-functions’（参见《Emacs Lisp 参考手册》的“Major Mode Conventions”）。你也可以按‘S-<F10>’来调用上下文菜单。

23.5 模式行鼠标命令

你可以用鼠标点击窗口的模式行来选择并操作窗口。

模式行的某些区域（例如缓冲区名，以及主模式和次模式名）有自己特殊的鼠标绑定。当你把鼠标停留在这些区域上时，它们会高亮，并显示关于这些特殊绑定的信息（参见“工具提示”）。本节中的命令不适用于这些区域。

‘mouse-1’在模式行上使用‘mouse-1’会选中该模式行所属的窗口。在模式行上拖动‘mouse-1’可以移动它，从而改变上下窗口的高度。以这种方式用鼠标改变高度绝不会删除窗口，只是拒绝把任何窗口变得比最小高度更

小。

‘mouse-2’ 在模式行上使用 ‘mouse-2’ 会把该窗口扩展到填满其框架。

‘mouse-3’ 在模式行上使用 ‘mouse-3’ 会删除该模式行所属的窗口。若框架只有一个窗口，则什么也不做。

‘C-mouse-2’ 在模式行上使用 ‘C-mouse-2’ 会拆分该窗口，产生两个并排的窗口，其分界线经过点击位置（参见“拆分窗口”）。

此外，在两条并排模式行之间的分隔条上点击并拖动 ‘mouse-1’，可以把垂直分界线向左或向右移动。

注意，调整窗口大小受 ‘window-resize-pixelwise’ 的值影响，参见“拆分窗口”。

23.6 创建框架

前缀键 ‘C-x 5’ 与 ‘C-x 4’ 类似。‘C-x 4’ 的各命令在选中框架的另一个窗口中弹出缓冲区（参见“在另一个窗口中显示”），而 ‘C-x 5’ 的各命令则使用另一个框架。若某个已存在且可见或已图标化（即“最小化”，参见《Emacs Lisp 参考手册》的“Visibility of Frames”）的框架已经显示了所请求的缓冲区，则抬起该框架并取消图标化（“取消最小化”）；否则，在当前显示终端上创建一个新框架。

各个 ‘C-x 5’ 命令的区别在于它们如何查找或创建要选中的缓冲区：

‘C-x 5 2’ 使用默认框架参数创建一个新框架（‘make-frame-command’）。

‘C-x 5 c’ 使用当前框架的窗口配置和框架参数创建一个新框架（‘clone-frame’）。

‘C-x 5 b BUFNAME <RET>’ 在另一个框架中选中缓冲区 BUFNAME。它运行 ‘switch-to-buffer-other-frame’。

‘C-x 5 f FILENAME <RET>’ 访问文件 FILENAME 并在另一个框架中选中它的缓冲区。它运行 ‘find-file-other-frame’。参见“访问文件”。

‘C-x 5 d DIRECTORY <RET>’ 在另一个框架中为目录 DIRECTORY 选中一个 Dired 缓冲区。它运行 ‘dired-other-frame’。参见“Dired 目录编辑器”。

‘C-x 5 m’ 在另一个框架中开始撰写邮件消息。它运行 ‘compose-mail-other-frame’。它是 ‘C-x m’ 的另一个框架变体。参见“发送邮件”。

‘C-x 5 .’ 在另一个框架中查找标识符的定义。它运行 ‘xref-find-definitions-other-frame’，即 ‘M-.’ 的多框架变体。参见“查找标识符引用”。

‘C-x 5 r FILENAME <RET>’ 以只读方式访问文件 FILENAME，并在另一个框架中选中它的缓冲区。它运行 ‘find-file-read-only-other-frame’。参见“访问文件”。

‘C-x 5 5’ 一个更通用的前缀命令，影响此后缀命令之后所调用命令显示的缓冲区（‘other-frame-prefix’）。它请求把后续命令要显示的缓冲区显示在另一个框架中。

你可以通过指定“框架参数”来控制新建框架的外观和行为。参见“框架参数”。

23.7 框架命令

以下命令用于删除和操作框架：

‘C-x 5 0’ 删除选中的框架（‘delete-frame’）。若只有一个框架，它会报错。

‘C-x 5 u’ 启用 ‘undelete-frame-mode’ 时，恢复最近删除的 16 个框架之一。不带前缀参数时，恢复最近删除的那个框架。带 1 到 16 之间的数字前缀参数时（其中 1 表示最近删除的框架），恢复相应的已删除框架。

‘C-z’ 最小化（或图标化）选中的 Emacs 框架（‘suspend-frame’）。参见“退出 Emacs”。

‘C-x 5 o’ 选中另一个框架并抬起它。若重复该命令，它会在你终端上的所有框架之间循环。

‘C-x 5 1’ 删除当前终端上除选中框架以外的所有框架。

‘M-<F10>’ 切换当前框架的最大化状态。框架最大化时会填满屏幕。

‘<F11>’ 切换当前框架的全屏模式。（全屏与最大化的区别通常在于：前者隐藏窗口管理器的装饰，把略多一些的屏幕空间留给 Emacs 本身。）注意，是否允许调整全屏框架的大小以及 Emacs 如何处理这一点，由用户选项 ‘alter-fullscreen-frames’ 控制。

注意，在某些窗口管理器下，你可能需要把变量 ‘frame-resize-pixelwise’ 自定义为非 ‘nil’ 值，才能让框架真正最大化或全屏。该变量设为非 ‘nil’ 值时，通常允许以像素分辨率调整框架大小，而不是以行和列的整数倍调整。

‘C-x 5 0’（‘delete-frame’）命令删除选中的框架。不过，它会拒绝删除 Emacs 会话中的最后一个框架，以防你失去与 Emacs 会话交互的能力。注意，当 Emacs 以守护进程方式运行时（参见“把 Emacs 用作服务器”），在所有普通的交互式框架被删除之后，总会留下一个虚拟框架。在这种情况下，‘C-x 5 0’ 可以删除最后一个交互式框架；你可以用 ‘emacsclient’ 重新连接到该 Emacs 会话。

‘C-x 5 1’（‘delete-other-frames’）命令删除当前终端上的所有其他框架（这里的终端指图形显示器或文本终端；参见“文本终端”）。若该 Emacs 会话在其他图形显示器或文本终端上打开了框架，那些框架不会被删除。

‘C-x 5 o’（‘other-frame’）命令选中当前终端上的下一个框架。若你在 X Window System 上使用 Emacs，而窗口管理器会选中（或“把焦点给予”）鼠标指针所在的任何框架，你必须把变量 ‘focus-follows-mouse’ 改为 ‘t’ 才能让该命令正常工作。此时调用 ‘C-x 5 o’ 还会把鼠标指针移到所选框架。

23.8 字体

默认情况下，Emacs 在图形显示器上用 10 点的等宽字体显示文本，字体大小可以交互地更改（参见“文本缩放”）。

有几种不同的方式指定不同的字体：

- 点击 ‘Options’ 菜单中的 ‘Set Default Font’。这会把所选字体设为所有已存在图形框架的默认字体。要把它保存下来供以后的会话使用，点击 ‘Options’ 菜单中的 ‘Save Options’。
- 在初始化文件中添加一行，修改变量 ‘default-frame-alist’ 以指定 ‘font’ 参数（参见“框架参数”），如下所示：

```
(add-to-list 'default-frame-alist
             '(font . "DejaVu Sans Mono-10"))
```

这会使该字体成为用该初始化文件重启 Emacs 之后创建的所有图形框架的默认字体。

- 在 X 资源文件中添加 'emacs.font' X 资源设置，如下所示：

```
emacs.font: DejaVu Sans Mono-12
```

你必须重启 X，或使用 'xrdb' 命令，X 资源文件才会生效。参见"X 资源"。不要在 X 资源文件中给字体名加引号。

- 如果你在 GNOME 桌面或 Haiku 上运行 Emacs，可以把变量 'font-use-system-font' 设为 't'（默认值为 'nil'），让 Emacs 在系统默认字体改变时相应调整框架的默认字体。要使其生效，Emacs 编译时必须带有对 Gsettings（或较旧的 Gconf）的支持。（具体而言，所用的 Gsettings 配置名为 'org.gnome.desktop.interface monospace-font-name' 和 'org.gnome.desktop.interface font-name'。）

- 使用命令行选项 '-fn'（或 '--font'）。参见"字体指定选项"。

要查看当前正在使用的字体，'C-u C-x =' 命令会很有帮助。它描述光标点处的字符，并给出渲染该字符所用的字体名称。

字体名有四种不同的写法。第一种是使用"Fontconfig 模式"（Fontconfig pattern）。Fontconfig 模式的格式如下：

```
FONTNAME[-FONTSIZE][:NAME1=VALUES1][:NAME2=VALUES2]...
```

在这种格式中，方括号内的元素都可以省略。其中，FONTNAME 是字体的"族名"（family name），例如 'Monospace' 或 'DejaVu Sans Mono'；FONTSIZE 是字体的"点大小"（point size）（一个"印刷点"约为 1/72 英寸）；'NAME=VALUES' 形式的条目则指定字体的倾斜度、粗细度等设置。每个 VALUES 可以是单个值，也可以是用逗号分隔的值列表。此外，有些属性值只对某一种属性名有效，此时可以省略 'NAME=' 部分。

以下是常见字体属性的列表：

'slant' 取值为 'italic'、'oblique' 或 'roman' 之一。

'weight' 取值为 'light'、'medium'、'demibold'、'bold' 或 'black' 之一。

'style' 有些字体会定义特殊样式，它们是倾斜度和粗细度的组合。例如，'Dejavu Sans' 定义了 'book' 样式，该样式会覆盖 slant 和 weight 属性。

'width' 取值为 'condensed'、'normal' 或 'expanded' 之一。

'spacing' 取值为 'monospace'、'proportional'、'dual-width' 或 'charcell' 之一。

下面是几个 Fontconfig 模式的例子：

```
Monospace
Monospace-12
Monospace-12:bold
DejaVu Sans Mono:bold:italic
Monospace-12:weight=bold:slant=italic
```

关于 Fontconfig 模式的更详细说明，参见 Fontconfig 手册，该手册随 Fontconfig 一起发布，也可在 <https://fontconfig.org/fontconfig-user.html> 在线获取。

在 MS-Windows 上，对所有字体只支持 FONTNAME[-FONTSIZE] 这一子集形式；完整的 Fontconfig 模式未必对所有字体都有效。

第二种指定字体的方法是使用“GTK 字体模式”（GTK font pattern）。其语法为

```
FONTNAME [PROPERTIES] [FONTSIZE]
```

其中 FONTNAME 是族名，PROPERTIES 是用空格分隔的属性值列表，FONTSIZE 是点大小。GTK 字体模式中可以指定的属性如下：

- 倾斜度属性：‘Italic’ 或 ‘Oblique’。若省略，则隐含默认的（roman）倾斜度。
- 粗细度属性：‘Bold’、‘Book’、‘Light’、‘Medium’、‘Semi-bold’ 或 ‘Ultra-light’。若省略，则隐含 ‘Medium’ 粗细度。
- 宽度属性：‘Semi-Condensed’ 或 ‘Condensed’。若省略，则使用默认宽度。

下面是几个 GTK 字体模式的例子：

```
Monospace 12
Monospace Bold Italic 12
```

在 MS-Windows 上只支持 FONTNAME 这一子集。

第三种指定字体的方法是使用“XLFD”（“X Logical Font Description”，X 逻辑字体描述）。这是在 X 下指定字体的传统方法，MS-Windows 也支持。每个 XLFD 由十四个单词或数字组成，彼此用连字符分隔，如下所示：

```
-misc-fixed-medium-r-semicondensed--13-*-*-*-c-60-iso8859-1
```

XLFD 中的通配符字符（‘*’）匹配任意字符序列（包括空序列），‘?’ 匹配任意单个字符。不过，匹配方式依赖于具体实现，当通配符匹配长名称中的连字符时可能不准确。为得到可靠结果，请给出全部 14 个连字符，并且只在字段内部使用通配符。XLFD 中大小写无关。XLFD 的语法如下：

```
-MAKER-FAMILY-WEIGHT-SLANT-WIDTHTYPE-STYLE...
...-PIXELS-HEIGHT-HORIZ-VERT-SPACING-WIDTH-REGISTRY-ENCODING
```

各条目的含义如下：

MAKER 字体制造商的名称。

FAMILY 字体族的名称（例如 ‘courier’）。

WEIGHT 字体粗细度——通常是 ‘bold’、‘medium’ 或 ‘light’。有些字体名支持其他值。

SLANT 字体倾斜度——通常是 ‘r’（roman，正体）、‘i’（italic，斜体）、‘o’（oblique，倾斜体）、‘ri’（reverse italic，反向斜体）或 ‘ot’（other，其他）。有些字体名支持其他值。

WIDTHTYPE 字体宽度——通常是 ‘normal’、‘condensed’、‘semicondensed’ 或 ‘extended’。有些字体名支持其他值。

STYLE 可选的附加样式名。通常为空——大多数 XLFD 在此处有两个连续的连字符。样式名也可以指定两位的 ISO-639 语言名，例如 ‘ja’ 或 ‘ko’；某些支持 CJK 文种的字体在样式名部分就写明了这一点。

PIXELS 字体高度，以像素为单位。

HEIGHT 屏幕上字体的高度，以十分之一印刷点为单位。它是字体点大小乘以十。对于给定的垂直分辨率，HEIGHT 与 PIXELS 成正比；因此常见的做法是只指定其中之一，另一个用 '*'。

HORIZ 字体所面向的屏幕的水平分辨率，以每英寸像素数为单位。

VERT 字体所面向的屏幕的垂直分辨率，以每英寸像素数为单位。通常系统上字体的分辨率对屏幕来说就是合适的值；因此一般把它和 HORIZ 都指定为 '*'。

SPACING 取值为 'm'（monospace，等宽）、'p'（proportional，比例）或 'c'（character cell，字符单元）。

WIDTH 平均字符宽度，以像素为单位，再乘以十。

REGISTRY ENCODING 字体所描绘的 X 字体字符集。（X 字体字符集与 Emacs 字符集并不相同，但二者相似。）可以用 'xfontsel' 程序查看有哪些可选项。通常 REGISTRY 应使用 'iso8859'，ENCODING 应使用 '1'。

第四种、也是最后一种指定字体的方法是使用字体绰号。某些字体有更短的绰号，可以用它代替常规的字体规格。例如，'6x13' 等价于

```
-misc-fixed-medium-r-semicondensed--13-*-*-*-c-60-iso8859-1
```

这种形式在 MS-Windows 上不受支持。

在 X 上，Emacs 识别两类字体："客户端字体"（client-side font），由 Xft 和 Fontconfig 库提供；"服务器端字体"（server-side font），由 X 服务器本身提供。大多数客户端字体支持抗锯齿、次像素微调等高级字体特性，服务器端字体则不支持。Fontconfig 和 GTK 模式只匹配客户端字体。

你可能想使用固定宽度的默认字体——即所有字符宽度都相同的字体。对于 Xft 和 Fontconfig 字体，可以用 'fc-list' 命令列出可用的固定宽度字体，如下所示：

```
fc-list :spacing=mono
fc-list :spacing=charcell
```

对于服务器端 X 字体，可以用 'xlsfonts' 程序列出可用的固定宽度字体，如下所示：

```
xlsfonts -fn '*x*' | grep -E '^[0-9]+x[0-9]+'
xlsfonts -fn '*-*-*-*-*-*-*-*-*-*-*-m*'
xlsfonts -fn '*-*-*-*-*-*-*-*-*-*-*-c*'
```

XLFD 的 SPACING 字段中含有 'm' 或 'c' 的字体都是固定宽度字体。要查看某个具体字体的样子，可使用 'xfd' 命令。例如：

```
xfd -fn 6x13
```

会显示 '6x13' 字体的全部内容。

在运行 Emacs 时，你还可以设置某一类文本所用的字体（参见"文本字体"），或某个特定框架所用的字体（参见"框架参数"）。

23.9 Speedbar 框架

“speedbar”是一种特殊的框架，用于方便地浏览或操作另一个框架。speedbar 存在时总是与某个特定的框架相关联，该框架称为它的“附加框架”（attached frame）；所有 speedbar 操作都作用于该框架。

输入‘M-x speedbar’可创建 speedbar 并将其与当前框架关联。要关闭 speedbar，再次输入‘M-x speedbar’，或选中 speedbar 后输入‘q’。（你也可以像删除其他 Emacs 框架那样删除 speedbar 框架。）若想把 speedbar 与另一个框架关联，请先关闭它，再在那个框架中调用‘M-x speedbar’。

speedbar 可以在多种模式下工作。其默认模式是“文件显示”（File Display）模式，它显示附加框架中所选窗口的当前目录下的文件，每行一个文件。点击非目录项会在附加框架的所选窗口中访问该文件，点击目录项则会在 speedbar 中显示该目录（参见“用鼠标跟随引用”）。每一行还有一个方框，‘[+]’或‘<+>’，点击它可以“展开”该项的内容。展开目录会把该目录的内容添加到 speedbar 显示中，位于该目录自身那一行下面。展开普通文件会把该文件中标签的列表添加到 speedbar 显示中；点击某个标签名，就会在附加框架的所选窗口中跳到该标签处。文件或目录被展开后，‘[+]’变为‘[-]’；点击该方框可以“折叠”该项，隐藏其内容。

你也可以用键盘在 speedbar 中浏览。当光标点位于 speedbar 的某一行时，输入 <RET> 等价于点击当前行上的项，<SPC> 则展开或折叠该项。‘U’显示当前目录的父目录。要复制、删除或重命名当前行上的文件，分别输入‘C’、‘D’和‘R’。要新建目录，输入‘M’。

另一种通用 speedbar 模式是“缓冲区显示”（Buffer Display）模式；在此模式下，speedbar 显示 Emacs 缓冲区列表。要切换到该模式，在 speedbar 中输入‘b’。要回到文件显示模式，输入‘f’。你也可以在 speedbar 窗口的任意位置点击‘mouse-3’（或在模式行上点击‘mouse-1’），然后在弹出菜单中选择‘Displays’来改变显示模式。

某些主模式，包括 Rmail 模式、Info 和 GUD，有专门的方式把有用的条目放入 speedbar 供你选择。例如，在 Rmail 模式下，speedbar 显示 Rmail 文件列表，并让你通过点击某个文件的‘<M>’方框把当前消息移到另一个 Rmail 文件。

speedbar 还可以选择显示为窗口而非单独的框架，在终端和图形模式下都可以。

关于 speedbar 的使用与编程的更多细节，参见《Speedbar 手册》。

23.10 多个 X 显示

一个 Emacs 可以与多个 X 显示通信。最初，Emacs 只使用一个显示——即由‘DISPLAY’环境变量或‘--display’选项指定的那个（参见“初始选项”）。要连接到另一个显示，使用命令‘make-frame-on-display’：

- ‘M-x make-frame-on-display <RET> DISPLAY <RET>’在显示 DISPLAY 上创建一个新框架。

一个 X 服务器可以处理多个屏幕。当你在属于同一服务器的两个屏幕上打开框架时，Emacs 知道它们共用一个键盘，并把来自这些屏幕的所有命令当作单一输入流处理。

当你在不同的 X 服务器上打开框架时，Emacs 为每个服务器建立独立的输入流。每个服务器也各有自己选中的框架。你通过某个 X 服务器输入的命令作用于该服务器选中的框架。

在多监视器显示上，可以使用命令‘make-frame-on-monitor’：

- ‘M-x make-frame-on-monitor <RET> MONITOR <RET>’在监视器 MONITOR 上创建新框架，其屏幕区域是当前显示的一部分。

23.10.1 单键盘模式

每个显示都关联一个键盘。通常 Emacs 假定这些键盘在物理上彼此独立，因此一个人可以在一个键盘上输入，另一个人可以在另一个键盘上输入，分别作用于不同终端上的不同框架。然而在某些情况下，Emacs 会进入“单键盘模式”（single-keyboard mode），此时除一个键盘之外，来自其他所有键盘的输入都被阻止。这可以防止在一个键盘上键入的字符干扰在另一个键盘上启动的操作。所适用的主要操作是进入递归编辑（参见“递归编辑层级”），其中包括所有迷你缓冲区提示（参见“使用迷你缓冲区”）。

当存在多个不同的显示、因而有多个不同的键盘，但只有一个用户和一个物理键盘在控制 Emacs 时，单键盘模式可能带来不便。在使用 X 转发时会出现这种情况：有一个远程 Emacs 守护进程，并用 ‘ssh -X daemon-host emacsclient -c’ 这样的命令创建多个框架，那么从远程 Emacs 守护进程的角度看，每执行一次 ‘ssh -X daemon-host’ 命令就有一个终端和一个键盘，但实际上由本地同一个 X 服务器显示所有框架，且只有一个物理键盘。

在这种情况下，你可能希望不同框架的表现就像它们是用 ‘C-x 5 2’ 创建的一样（参见“创建框架”）。这样，在一个框架中启动递归编辑并不意味着其他框架的键盘输入被阻止。把变量 ‘multiple-terminals-merge-keyboards’ 定制为非 ‘nil’ 值即可实现这一点。

23.11 框架参数

你可以在变量 ‘default-frame-alist’ 中指定默认的“框架参数”列表，以控制所有框架的默认外观和行为。它的值应是一个条目列表，每个条目指定一个参数名和该参数的值。每当 Emacs 创建新框架（包括初始框架）时，这些条目就会生效。

例如，你可以在初始化文件中加入以下几行（参见“Emacs 初始化文件”），把默认框架宽度设为 90 个字符列、默认框架高度设为 40 个字符行、默认字体设为 ‘Monospace-10’：

```
(add-to-list 'default-frame-alist '(width  . 90))
(add-to-list 'default-frame-alist '(height . 40))
(add-to-list 'default-frame-alist '(font . "Monospace-10"))
```

框架参数的列表及其作用，参见《Emacs Lisp 参考手册》的“Frame Parameters”。

你还可以定制变量 ‘initial-frame-alist’，指定只适用于初始框架的框架参数列表。

如果 Emacs 编译为使用 X 工具包，那么指定颜色和字体的框架参数不会影响菜单和菜单栏，因为它们是由工具包绘制的，而不是由 Emacs 直接绘制的。

框架的外观和行为还可以通过 X 资源来定制（参见“附录 D X 选项与资源”）；这些设置会覆盖初始化文件中为初始框架指定的参数。

注意，如果你使用 desktop 库保存和恢复会话，则待恢复的框架连同其参数会记录在 desktop 文件中。这些框架被恢复时，记录下来的参数优先于初始化文件中由 ‘default-frame-alist’ 和 ‘initial-frame-alist’ 指定的框架参数。如何避免这种情况，参见“保存 Emacs 会话”。

23.12 滚动条

在图形显示上，每个 Emacs 窗口的一侧都有一个“竖直滚动条”。点击滚动条上下按钮上的 ‘mouse-1’ 会使窗口一次滚动一行（但有些工具包允许你定制滚动条，使其没有这些按钮）。在滚动条内部滑块的上方或下方点击 ‘mouse-1’，会使窗口分别像 ‘M-v’ 和 ‘C-v’ 那样滚动接近整个窗口的高度（参见“移动光标点的位置”）。（在某些工具包上，这一点也可能表现不同。）拖动内部滑块可以连续滚动。

如果 Emacs 编译于 X 窗口系统且不带 X 工具包支持，滚动条的行为会有所不同。此时在滚动条任意位置点击 ‘mouse-1’ 会像 ‘C-v’ 一样向前滚动，而点击 ‘mouse-3’ 会像 ‘M-v’ 一样向后滚动。在滚动条中点击 ‘mouse-2’ 可以上下拖动内部滑块。

要切换竖直滚动条的使用，输入 ‘M-x scroll-bar-mode’。该命令作用于所有框架，包括尚未创建的框架。要只为选中的框架切换竖直滚动条，使用命令 ‘M-x toggle-scroll-bar’。

要在启动时控制竖直滚动条的使用，定制变量 ‘scroll-bar-mode’（参见“自定义”）。它的值应为 ‘right’（把滚动条放在窗口右侧）、‘left’（放在左侧）或 ‘nil’（禁用竖直滚动条）。默认情况下，如果 Emacs 是在 X 窗口系统上带 GTK+ 支持编译的，或在 MS-Windows 或 macOS 上，它就把滚动条放在右侧；如果是在 X 窗口系统上不带 GTK+ 支持编译的，就把滚动条放在左侧（沿用 X 应用程序的旧惯例）。

你也可以使用 X 资源 ‘verticalScrollBars’ 来启用或禁用滚动条（参见“X 资源”）。要控制滚动条宽度，修改 ‘scroll-bar-width’ 框架参数（参见《Emacs Lisp 参考手册》的“Frame Parameters”）。

如果你在 X 上使用 Emacs（带 GTK+ 或 Motif），可以定制变量 ‘scroll-bar-adjust-thumb-portion’ 来控制滚动条的“过度滚动”（overscrolling），即在缓冲区末尾已经可见时仍把滑块向下拖。若其值非 ‘nil’，即使缓冲区末尾已显示，滚动条仍可向下拖动；若为 ‘nil’，则缓冲区末尾显示时滑块位于底部。当整个缓冲区都可见时，无法过度滚动。

滚动条的视觉外观由 ‘scroll-bar’ 字体（face）控制。（某些工具包，例如 GTK+ 和 MS-Windows，会忽略这个字体（face）；在这些工具包上，滚动条的外观只能在系统范围内定制，对 GTK+ 而言参见“GTK+ 资源”。）

在图形框架上，竖直滚动条事实上还起到在视觉上分隔并排窗口的作用。当竖直滚动条被禁用时，Emacs 默认借助一像素宽的“竖直边框”（vertical border）来分隔这类窗口。该边框占据右侧窗口的第一个像素列，因此可能覆盖显示在那里的任何字形最左边的像素。如果这些像素承载重要信息，可以启用窗口分隔条使它们可见，参见“窗口分隔条”。要复制竖直边框的外观，请把框架的 ‘right-divider-width’ 参数设为 1，并让 ‘window-divider’ 字体（face）继承 ‘vertical-border’ 的字体（face），参见《Emacs Lisp 参考手册》的“Window Dividers”。

在支持工具包的图形显示上，Emacs 还可以在每个窗口底部提供“水平滚动条”。点击该滚动条左右按钮上的 ‘mouse-1’ 会使窗口一次水平滚动一列。（注意，某些工具包允许对滚动条进行定制，使这些按钮不显示。）在滚动条内部滑块的左侧或右侧点击 ‘mouse-1’ 会使窗口滚动四列。拖动内部滑块可以使窗口连续滚动。

注意，这种水平滚动可能使窗口中光标点的位置在左侧或右侧消失。输入字符以插入文本，或用键盘命令移动光标点，通常会让它重新回到视野中。

要切换水平滚动条的使用，输入 ‘M-x horizontal-scroll-bar-mode’。该命令作用于所有框架，包括尚未创建的框架。要只为选中的框架切换水平滚动条，使用命令 ‘M-x toggle-horizontal-scroll-bar’。

要在启动时控制水平滚动条的使用，定制变量 ‘horizontal-scroll-bar-mode’。

你也可以使用 X 资源 'horizontalScrollBars' 来启用或禁用水平滚动条（参见"X 资源"）。要控制滚动条高度，修改 'scroll-bar-height' 框架参数（参见《Emacs Lisp 参考手册》的"Frame Parameters"）。

23.13 窗口分隔条

在图形显示上，你可以使用"窗口分隔条"（window divider）在视觉上分隔窗口。窗口分隔条是可以用鼠标拖动的条，因此你可以方便地调整相邻窗口的大小。

要切换窗口分隔条的显示，使用命令 'M-x window-divider-mode'。

要定制分隔条出现的位置，使用选项 'window-divider-default-places'。其值应为 'bottom-only'（只在窗口底部显示分隔条）、'right-only'（只在窗口右侧显示分隔条）或 't'（在底部和右侧都显示）。

要调整此模式所显示的窗口分隔条的宽度，请定制选项 'window-divider-default-bottom-width' 和 'window-divider-default-right-width'。

当竖直滚动条被禁用时，分隔条还可用于让窗口的第一个像素列可见，否则该像素列会被用来分隔并排窗口的竖直边框覆盖（参见"滚动条"）。

关于窗口分隔条的更多细节，参见《Emacs Lisp 参考手册》的"Window Dividers"。

23.14 拖放

在大多数图形桌面环境中，Emacs 对"拖放"（drag and drop）操作有基本支持。例如，把文本拖放到 Emacs 框架上，会在放下文本的位置插入该文本。把文件拖放到 Emacs 框架上会访问该文件。作为特例，把文件拖放到 Dired 缓冲区上，会按该文件来源应用程序的惯例把文件移动或复制到该缓冲区所显示的目录中。

拖放文件通常会在你放下它的窗口中访问该文件。如果你希望在这种情况下于新窗口中访问该文件，请定制变量 'dnd-open-file-other-window'。

目前支持 XDND 和 Motif 拖放协议，以及旧的 KDE 1.x 协议。

在把文本拖到 Emacs 窗口上时，可能难以滚动窗口或确定放下的文本将插入何处。把选项 'dnd-indicate-insertion-point' 设为非 'nil' 值，可在拖动过程中鼠标于窗口内移动时让光标点移到将要插入放下文本的位置；把 'dnd-scroll-margin' 设为整数值，则当拖动过程中鼠标移到距窗口顶部或底部该行数以内时，窗口会滚动。

Emacs 还可以选择用鼠标把区域拖到本缓冲区或另一个缓冲区的其他位置。要启用此功能，请把变量 'mouse-drag-and-drop-region' 定制为非 'nil' 值。通常，当目标与源是同一缓冲区时，文本会被移动，即剪切并粘贴；把区域放到另一个缓冲区上则会复制文本。如果该变量的值指定了修饰键名，例如 'shift'、'control' 或 'alt'，那么在放下文本时按下该修饰键就会复制而不是剪切，即使你把它放到文本来源的同一个缓冲区上也是如此。

为了在源缓冲区与目标缓冲区不同时也能剪切文本，请把选项 'mouse-drag-and-drop-region-cut-when-buffers-differ' 设为非 'nil' 值。默认情况下，在图形显示上，拖拽过程中选中的文本会显示在工具提示中，且光标点随鼠标指针一起移动。要抑制这类行为，请把选项 'mouse-drag-and-drop-region-show-tooltip' 和/或 'mouse-drag-and-drop-region-show-cursor' 设为 'nil'。

要把文本从 Emacs 拖到其他程序，请把选项 'mouse-drag-and-drop-region-cross-program' 设为非 'nil' 值。

在 X 窗口系统上，有些程序可以把文件拖放到 Emacs 上，并期望 Emacs 保存它们。通常，Emacs 会提示输入用于保存该文件的文件名，然后打开该文件，但可以通过修改变量 'x-dnd-direct-save-function' 来改变这一行为。参见《Emacs Lisp 参考手册》的"Drag and Drop"。

23.15 菜单栏

可以用 'M-x menu-bar-mode' 开关菜单栏的使用。不带参数时，该命令切换菜单栏模式（Menu Bar mode，一个全局次模式）的开关状态。带参数时，若参数为正则打开菜单栏模式，若参数不为正则关闭。要控制启动时是否使用菜单栏，请定制变量 'menu-bar-mode'。

熟练用户常常关闭菜单栏，在文本终端上尤其如此，因为这样可多出一行用于显示文本。若菜单栏已关闭，在支持弹出菜单的显示上，你仍可以用 'C-mouse-3' 弹出包含其内容的菜单。或者可以启用 'context-menu-mode' 并定制变量 'context-menu-functions'，以用 'mouse-3' 弹出上下文菜单。参见"打开菜单的鼠标点击"。

关于如何用菜单栏调用命令，参见"菜单栏"。关于如何定制菜单栏菜单的外观，参见"附录 D X 选项与资源"。

23.16 工具栏

在图形显示上，Emacs 会在每个框架顶部、菜单栏正下方放置一个"工具栏"（tool bar）。它是一行带图标的按钮，你可以用鼠标点击它们来调用各种命令。Emacs 还可以选择在每个窗口顶部显示工具栏（参见"窗口工具栏"）。

全局（默认）工具栏包含通用命令。某些主模式定义自己的工具栏；只要当前缓冲区处于这类主模式，该模式的工具栏就会替换全局工具栏。要阻止这种替换发生，请定制变量 'tool-bar-always-show-default'。

要开关工具栏的使用，请键入 'M-x tool-bar-mode'。该命令作用于所有框架，包括尚未创建的框架。要控制启动时是否使用工具栏，请定制变量 'tool-bar-mode'。

当 Emacs 编译时带有 GTK+ 支持时，每个工具栏项可以由一个图像、一个文本标签或两者共同组成。默认情况下，Emacs 遵循 Gnome 桌面的工具栏样式设置；若未定义该设置，则只以图像形式显示工具栏项。要强制使用特定工具栏样式，请定制变量 'tool-bar-style'。

对于 GTK+ 工具栏，你还可以用框架参数 'tool-bar-position' 控制工具栏的位置。参见《Emacs Lisp 参考手册》的"Frame Parameters"。

NS 构建把工具栏视为窗口装饰，因此当窗口未被装饰时不显示它。参见《Emacs Lisp 参考手册》的"Frame Parameters"。在 macOS 上，框架进入全屏时工具栏会被隐藏，但把鼠标指针移到屏幕顶部即可显示它。

键盘常常缺少 Emacs 用户可能想用的一个或多个修饰键（参见"修饰键"），这使得输入带这些修饰键的键序列很困难甚至不可能。例如，许多键盘缺少 Hyper 和 Super 修饰键，智能手机通常也缺少 Ctrl 和 Alt 修饰键。Emacs 可以选择显示一个由按钮组成的工具栏，这些按钮可代为按下修饰键；这个额外的工具栏称为"修饰键栏"（modifier bar）。点击修饰键栏中的按钮，会使该按钮上所示的修饰键作用于 Emacs 接下来读取的那个键盘事件。修饰键栏在全局次模式 'modifier-bar-mode' 启用时显示；要启用它，请键入 'M-x modifier-bar-mode'。

23.17 标签栏

在图形显示和文本终端上，Emacs 可选地在每个框架顶部显示“标签栏”（Tab Bar），它位于菜单栏正下方（参见“菜单栏”），并在工具栏之上或之下（参见“工具栏”），具体取决于变量 ‘tab-bar-position’。标签栏是一行“标签”（tab）——即按钮，你可以点击它们来在窗口配置之间切换。

标签栏上的每个标签代表其所属框架的一个具名持久窗口配置，即该框架如何被分割成窗口，以及每个窗口中显示哪个缓冲区。标签名由该窗口配置的各窗口中所显示的缓冲区名称列表组合而成。点击某个标签会切换到该标签所记录的窗口配置；那是在该标签为当前标签时，此框架中此前使用过的窗口与缓冲区配置。

如果你使用 desktop 库来保存和恢复会话（参见“保存 Emacs 会话”），标签栏中的标签会连同其关联的窗口配置一起记录在 desktop 文件中，并在恢复会话后可用。

注意标签栏与标签行（参见“窗口标签行”）不同。标签行位于每个窗口顶部，其中的标签用于在该窗口的缓冲区之间切换；而标签栏位于每个框架顶部，其中的标签用于在包含若干窗口的窗口配置之间切换，这些窗口显示一个或多个缓冲区。

要开关标签栏的使用，请键入 ‘M-x tab-bar-mode’。该命令作用于所有框架，包括尚未创建的框架。要控制启动时是否使用标签栏，请定制变量 ‘tab-bar-mode’ 并保存你的定制。

变量 ‘tab-bar-show’ 控制是否自动打开标签栏模式。若值为 ‘t’，则在使用创建新标签的命令时启用 ‘tab-bar-mode’。值为 ‘1’ 时，标签只有一个时隐藏标签栏，创建更多标签时再次显示它。更一般地，值为非负整数时，只有当标签数大于该整数时才显示标签栏。值为 ‘nil’ 时总是隐藏标签栏；此时仍可通过 ‘M-x tab-next’、‘M-x tab-switcher’ 以及其他对标签名提供补全的命令，在不显示标签栏的情况下在具名窗口配置之间切换。此外，也可以用命令 ‘M-x tab-new’、‘M-x tab-close’ 等在不显示标签栏的情况下创建和关闭标签。

注意，‘tab-bar-show’ 的数值可能导致标签栏在某些框架上显示，而在其他框架上不显示，具体取决于每个框架上创建的标签数。

要只在选中的框架上开关标签栏，请键入 ‘M-x toggle-frame-tab-bar’。该命令便于在某些框架上选择性地启用标签栏，而在其他框架上保持禁用，且不受 ‘tab-bar-mode’ 和 ‘tab-bar-show’ 取值的影响。

前缀键 ‘C-x t’ 类似于 ‘C-x 5’。每个 ‘C-x 5’ 命令都在不同框架中弹出缓冲区（参见“创建框架”），而 ‘C-x t’ 命令则在选中的框架中使用带有不同窗口配置的不同标签。

各种 ‘C-x t’ 命令在如何查找或创建要选中的缓冲区方面有所不同。以下命令可用于在新标签中选择缓冲区：

- ‘C-x t 2’ 添加一个新标签（‘tab-new’）。你可以通过定制变量 ‘tab-bar-new-tab-choice’ 来控制新标签中显示哪个缓冲区。你可以通过定制变量 ‘tab-bar-tab-name-function’ 来控制默认给新标签起的名称。
- ‘C-x t b BUFNAME <RET>’ 在另一个标签中选择缓冲区 BUFNAME。这会运行 ‘switch-to-buffer-other-tab’。
- ‘C-x t f FILENAME <RET>’ 访问文件 FILENAME（参见“访问文件”），并在另一个标签中选择其缓冲区。这会运行 ‘find-file-other-tab’。
- ‘C-x t d DIRECTORY <RET>’ 在另一个标签中编辑指定的 DIRECTORY（参见“Dired 目录编辑器”）。这会运行 ‘dired-other-tab’。
- ‘C-x t t’ 这是一个前缀命令（‘other-tab-prefix’），影响紧随该前缀命令之后调用的下一个命令。它要求下一个命令所显示的缓冲区显示在另一个标签中。

默认情况下，新标签以调用添加新标签的命令之前所处的当前缓冲区开始。要用其他缓冲区开始一个新标签，请定制变量‘tab-bar-new-tab-choice’。

变量‘tab-bar-new-tab-to’定义新标签放在哪里。默认情况下，新标签添加在当前标签的右侧。

以下命令可用于删除标签：

- ‘C-x t 0’关闭选中的标签（‘tab-close’）。若只有一个标签，则此命令没有效果，除非变量‘tab-bar-close-last-tab-choice’被定制为非默认值。
- ‘C-x t 1’关闭选中框架上除选中标签之外的所有标签。

变量‘tab-bar-close-tab-select’定义关闭当前标签之后选择哪个标签。默认情况下，它选择最近用过的标签。

命令‘tab-undo’恢复最后关闭的标签。

以下命令可用于在标签之间切换：

- ‘C-x t o’‘C-<TAB>’切换到下一个标签（‘tab-next’）。若重复执行该命令，它会在选中框架的所有标签之间循环。带正数值参数 N 时，它切换到向后第 N 个标签；带负参数 −N 时，它切换回向前第 N 个标签。
- ‘S-C-<TAB>’切换到上一个标签（‘tab-previous’）。带正数值参数 N 时，它切换到向前第 N 个标签；带负参数 −N 时，它切换到向后第 N 个标签。
- ‘C-x t <RET> TABNAME <RET>’按其名称切换到某个标签（‘tab-switch’），并对所有标签名提供补全。默认值以及标签名的“未来历史”按最近使用程度排序，因此你可以用‘M-n’（‘next-history-element’）取得最后访问的标签名、倒数第二个，依此类推。
- ‘MODIFIER-TAB-NUMBER’按其编号 TAB-NUMBER 切换到某个标签（‘tab-select’）。定制变量‘tab-bar-select-tab-modifiers’以指定一个或多个 MODIFIER 键之后，你就可以用所指定的某个修饰键与要选择的标签编号组合，按标签的序号选择它。编号 9 可用于选择最后一个标签。你可以选择 Emacs 支持的任何修饰键，参见“修饰键”。要在标签名旁显示标签编号，你可以定制另一个变量‘tab-bar-tab-hints’。这将帮助你决定按哪个数字键来按编号选择标签。
- ‘MODIFIER-9’切换到最后一个标签（‘tab-last’）。该组合键由‘tab-bar-select-tab-modifiers’定义的修饰键与键‘9’组成。带数值参数 N 时，切换到倒数第 N 个标签。
- ‘MODIFIER-0’切换到最近的标签（‘tab-recent’）。该组合键由‘tab-bar-select-tab-modifiers’定义的修饰键与键‘0’组成。带数值参数 N 时，切换到第 N 个最近的标签。

以下命令可用于操作标签：

- ‘C-x t r TABNAME <RET>’把当前标签重命名为 TABNAME（‘tab-rename’）。
- ‘C-x t m’把当前标签向右移动一个位置（‘tab-move’）。带正数值参数 N 时，向右移动 N 个位置；带负参数 −N 时，向左移动 N 个位置。

你可以用鼠标操作标签。点击‘mouse-2’会关闭标签。点击‘mouse-3’会弹出上下文菜单，其中的菜单项对所点击的标签进行操作。用‘mouse-1’拖动标签会把它移到标签栏上的另一个位置。鼠标滚轮滚动会切换到下一个或上一个标签。滚动时按住 <SHIFT> 键会把标签向左或向右移动。

触摸屏输入（参见“触摸屏输入与虚拟键盘”）也可用于操作标签。长按（参见“在触摸屏上使用 Emacs”）某个标签会显示一个上下文菜单，其中的菜单项对所按的标签进行操作；长按标签栏本身会显示一个上下文菜单，

让你创建和删除标签；点击标签本身会选中该标签的窗口配置，而点击标签栏上的按钮则等同于用 ‘mouse-1’ 点击它。

你可以启用 ‘tab-bar-history-mode’ 来记住每个标签中用过的窗口配置，并在之后恢复它们。

- ‘M-x tab-bar-history-back’ 恢复当前标签中此前用过的窗口配置。这会在窗口配置历史中向后导航。
- ‘M-x tab-bar-history-forward’ 取消对上一个窗口配置的恢复。这会在窗口配置历史中向前移动。

可以通过用户选项 ‘tab-bar-format’ 定制标签栏上所显示的项。

23.18 使用系统 GUI 任务栏功能

Emacs 可以利用你的 GUI 系统任务栏，在 Emacs 任务栏图标上显示徽章覆盖层、进度条报告，并提醒用户某个 Emacs 会话需要关注。注意：系统任务栏可能被称为 dock、启动器（launcher）或类似的名称。

在符合条件的 GNU/Linux GUI 桌面上，系统任务栏效果会出现在由你的 shell 扩展所决定的桌面目标上，最常见的是应用程序启动器或 dock 面板，或顶部面板。这些效果对 Emacs 实例是全局的。

注意：GNU/Linux 实现通过 D-Bus 向 GUI 发送系统任务栏消息。你可能需要安装或配置一些 shell 扩展，例如 <https://extensions.gnome.org/extension/307/dash-to-dock/>，它实现了 Ubuntu 的 Unity D-Bus 启动器规范，你可以在这里了解更多：<https://wiki.ubuntu.com/Unity/LauncherAPI>。参见《《dbus》手册》。

在 macOS/GNUstep 10.5+ 上，系统任务栏效果出现在 Dock 和应用程序切换器中。这些效果对 Emacs 实例是全局的。macOS/GNUstep 无需特殊配置。

在 MS-Windows 7+ 上，任务栏效果出现在 Windows 系统任务栏上。这些效果与发起它们的那个框架相关联。MS-Windows 无需特殊配置。

在使用系统任务栏之前必须先初始化它。为此，请键入 ‘M-x system-taskbar-mode’。

用户选项 ‘system-taskbar-use-progress-reporter’ 把 ‘system-taskbar-mode’ 与 Emacs 的进度报告函数集成起来，许多耗时较长的函数用这些函数来指示其工作进度。进度报告会出现在回显区和系统任务栏上的 Emacs 图标中。该变量默认为 ‘t’。请在启用 ‘system-taskbar-mode’ 之前定制该变量。参见《Emacs Lisp 参考手册》的 “Progress”

用户选项 ‘system-taskbar-clear-attention-on-frame-focus’ 打开一个在 GNU/Linux D-Bus 平台上有用的辅助功能，它会在任何 Emacs 框架获得焦点时自动清除系统任务栏的关注指示器。这对 macOS/GNUstep 或 MS-Windows 没有效果。它默认为 ‘t’。请在启用 ‘system-taskbar-mode’ 之前定制该变量。

用户选项 ‘system-taskbar-dbus-desktop-file-name’ 帮助 GNU/Linux 上的 D-Bus 识别所启动的 Emacs 实例。它默认为 ‘emacsclient’，根据你的 GNU/Linux 配置，可能需要把它改为 ‘emacs’。

用户选项 ‘system-taskbar-dbus-timeout’ 是一个排错工具，很可能无需定制。它默认为 ‘nil’，即使用 D-Bus 的默认超时值 25,000ms，即 25 秒。

参见《Emacs Lisp 参考手册》的“System Taskbar”

23.19 使用对话框

对话框是一种特殊的菜单，用于向你提出是/否问题或其他特殊问题。如果你用鼠标调用了导致该问题的命令，许多 Emacs 命令就会用对话框来提出是/否问题。

要禁用对话框，请把变量 'use-dialog-box' 改为 'nil'。在这种情况下，Emacs 总是通过回显区和键盘输入来进行是/否提示。该变量还控制是否使用文件选择窗口（但并非所有平台都支持它们）。

文件选择窗口是一种特殊的对话框，用于询问文件名。即使你仍想要其他类型的对话框，也可以通过定制变量 'use-file-dialog' 来禁止使用文件选择窗口。如果你已用变量 'use-dialog-box' 禁止了所有对话框，则该变量没有效果。

当 Emacs 编译时带有 GTK+ 支持时，它会使用 GTK+ 文件选择对话框。Emacs 在此对话框中添加了一个额外的切换按钮，你可以用它在该对话框中启用或禁用隐藏文件（以点开头的文件）的显示。如果你希望默认启用该切换，请把变量 'x-gtk-show-hidden-files' 改为 't'。此外，Emacs 还会在 GTK+ 文件选择对话框中添加帮助文本；要禁用该帮助文本，请把变量 'x-gtk-file-dialog-help-text' 改为 'nil'。

23.20 工具提示

"工具提示"（tooltip）是在当前鼠标位置显示文本信息的小型特殊框架。当鼠标在窗口中的某段有意义的文本、模式行，或 Emacs 框架的其他部分（如工具栏按钮或菜单项）上暂停移动时，它们就会激活。

可以用命令 'M-x tooltip-mode' 开关工具提示的使用。当工具提示模式被禁用时，帮助文本改为在回显区显示。要控制启动时是否使用工具提示，请定制变量 'tooltip-mode'。

以下变量为工具提示的显示提供了定制选项：

- 'tooltip-delay' 该变量指定 Emacs 在显示第一个工具提示之前应等待多久。值以秒为单位。
- 'tooltip-short-delay' 该变量指定在已经显示第一个工具提示之后，Emacs 在不同项上显示后续工具提示之前应等待多久。值以秒为单位。
- 'tooltip-hide-delay' 显示工具提示后，若鼠标不动，经过多少秒后隐藏它。
- 'tooltip-x-offset' 'tooltip-y-offset' 工具提示左上角相对于鼠标指针位置的 X 和 Y 偏移量，以像素为单位。注意，若 'tooltip-frame-parameters' 被定制为分别包含 'left' 和 'top' 参数，则这些偏移量会被忽略。偏移量的取值应当选择为使工具提示不覆盖鼠标指针的热点，否则可能妨碍鼠标点击。
- 'tooltip-frame-parameters' 用于显示工具提示的框架参数。参见《Emacs Lisp 参考手册》的"Frame Parameters"，另参见《Emacs Lisp 参考手册》的"Tooltips"。

关于显示工具提示的其他定制选项，请使用 'M-x customize-group <RET> tooltip <RET>'。

如果 Emacs 是带有 GTK+ 工具包、Nextstep 窗口系统或 Haiku 窗口系统支持而构建的，则它通过工具包显示工具提示，使用该工具包工具提示的默认外观。(1) 要禁用这一点，请把变量 'use-system-tooltips' 改为 'nil'。若你这样做，或者 Emacs 构建时没有相应的窗口系统支持，则工具提示文本的大多数属性由 'tooltip' 字体（face）以及 X 资源指定（参见"附录 D X 选项与资源"）。

"GUD 工具提示"是特殊的工具提示，在用 GUD 调试程序时显示变量的值。参见"调试器操作"。

> 注：(1) 在 Nextstep 上，工具包创建的工具提示的前景色和背景色也可以通过设置 'tooltip-frame-parameters' 中的 'foreground' 和 'background' 框架参数来定制。

23.21 鼠标回避

在图形终端上，鼠标指针可能遮挡 Emacs 框架中的文本。Emacs 提供两种方法来避免这个问题。

首先，每当键入自插入字符时，若鼠标指针位于某个 Emacs 框架内，Emacs 就会隐藏鼠标指针；移动鼠标指针会使其再次可见。要禁用此功能，请把变量 'make-pointer-invisible' 设为 'nil'。参见"显示的自定义"。

其次，你可以使用鼠标回避模式（Mouse Avoidance mode），一个次模式，来让鼠标指针远离光标点。要使用鼠标回避模式，请定制变量 'mouse-avoidance-mode'。你可以把它设为各种值，以多种方式移动鼠标：

- 'banish' 任何按键时把指针移到框架的一个角落。你可以定制变量 'mouse-avoidance-banish-position' 来指定指针被放逐后去往何处。
- 'exile' 仅当光标靠得太近时才放逐指针，并在光标移开后允许它返回。
- 'jump' 若光标离指针太近，则以随机的距离和方向移开指针。
- 'animate' 同 'jump'，但显示沿途的步骤，以造成运动的错觉。
- 'cat-and-mouse' 与 'animate' 相同。
- 'proteus' 同 'animate'，但还会改变鼠标指针的形状。

你也可以用命令 'M-x mouse-avoidance-mode' 来启用该模式。每当鼠标回避模式移动鼠标时，它也会置顶该框架。

23.22 文本终端

在文本终端上，Emacs 一次只能显示一个 Emacs 框架。不过，你仍然可以创建多个 Emacs 框架，并在它们之间切换。在这些终端上切换框架很像在不同的窗口配置之间切换。

用 'C-x 5 2' 创建新框架并切换到它；用 'C-x 5 o' 在现有框架之间循环；用 'C-x 5 0' 删除当前框架。

每个框架都有一个编号以区分它们。如果你的终端一次只能显示一个框架，选中框架的编号 N 会以 'FN' 的形式出现在模式行开头附近。

'FN' 实际上是框架的初始名称。若愿意，你可以给框架起更有意义的名字，并且可以按名称选择框架。用命令 'M-x set-frame-name <RET> NAME <RET>' 为选中框架指定新名称，用 'M-x select-frame-by-name <RET> NAME <RET>' 按名称选择框架。你指定的名称会在该框架被选中时显示在模式行中。

23.23 在文本终端中使用鼠标

有些文本终端支持在终端窗口中进行鼠标点击。

在兼容 'xterm' 的终端模拟器中，你可以用 'M-x xterm-mouse-mode' 让 Emacs 控制鼠标的简单用法——基本上只支持不带修饰键的单击。较新版本的 'xterm' 也支持鼠标跟踪。按住 <SHIFT> 键再按鼠标键，仍可使用 'xterm' 对此类点击的常规鼠标功能。Xterm Mouse 模式是一个全局次模式（参见"次模式"），若 Emacs 在启动时检测到终端支持它，则默认启用。再次执行该命令会关闭该模式。

在 GNU/Linux 的控制台中，可以用 ‘M-x gpm-mouse-mode’ 启用鼠标支持。要让该功能生效，系统上必须安装并运行 gpm 服务器。注意，启用该模式后，无法用鼠标在 Emacs 与使用 GPM 的其他程序之间传送文本。这是 GPM 和 Linux 内核的限制所致。

关于 MS-DOS 上的鼠标支持，参见“MS-DOS 上的鼠标使用”。

24 国际字符集支持

Emacs 支持多种多样的国际字符集，包括拉丁字母的欧洲变体和越南语变体，以及阿拉伯文字、婆罗米系文字（用于孟加拉语、印地语、泰语等语言）、西里尔文、埃塞俄比亚文、格鲁吉亚文、希腊文、汉字（用于汉语和日语）、谚文（用于朝鲜语）、希伯来文和 IPA。Emacs 还支持其他国际化软件（如文字处理器和邮件程序）对这些字符所用的各种编码。

Emacs 通过支持所有相关活动，使你能够编辑含有国际字符的文本：

- 你可以访问含有非 ASCII 字符的文件，保存非 ASCII 文本，并在 Emacs 与它调用的程序（如编译器、拼写检查器和邮件程序）之间传递非 ASCII 文本。设置语言环境（参见“语言环境”）会针对特定的语言或文化设置好编码系统及其他选项。另一种做法是，你可以逐条命令地指定 Emacs 应如何编码或解码文本；参见“为文件文本指定编码系统”。

- 你可以显示用各种文字编码的非 ASCII 字符。这要靠图形显示上使用合适的字体（参见“定义字体集”），以及向文本终端发送特殊代码（参见“终端 I/O 的编码系统”）来实现。如果某些字符显示不正确，请参阅参见“无法显示的字符”，其中描述了可能出现的问题并说明了解决办法。

- 对于文字的天然顺序是从右到左的那些字符集，Emacs 会在显示时重新排序（参见“双向编辑”）。这类文字包括阿拉伯文、希伯来文、叙利亚文、塔安那文（Thaana）以及其他几种。

- 你可以插入非 ASCII 字符，也可以搜索它们。为此，你可以指定适合自己语言的 Emacs 输入法（参见“选择输入法”），或者使用选择语言环境时设置好的默认输入法。如果你的键盘能产生非 ASCII 字符，你可以选择合适的键盘编码系统（参见“终端 I/O 的编码系统”），Emacs 就会接受这些字符。在图形显示上，现代系统通常提供自己的本机输入法；此外，Latin-1 字符也可以用 ‘C-x 8’ 前缀输入，参见“单字节编辑模式”。

在 X Window 系统下，应把 locale 设为适当的值，以确保 Emacs 正确解释键盘输入；参见“语言环境”中关于 locales 的说明，以及参见“X 键盘输入的编码系统”。

本章其余部分详细讲述这些问题。

- 编码系统（Coding Systems）：读写文件等时的字符集转换。
- 识别编码（Recognize Coding）：Emacs 如何判断该使用哪种转换。
- 指定编码（Specify Coding）：显式指定文件的编码系统。
- 输出编码（Output Coding）：为输出选择编码系统。
- 文本编码（Text Coding）：为文件文本选择要使用的转换。
- 通信编码（Communication Coding）：用于进程间通信的编码系统。
- 文件名编码（File Name Coding）：用于文件*名*的编码系统。
- X 编码（X Coding）：用于 X 输入法的编码系统。
- 终端编码（Terminal Coding）：指定转换终端输入和输出的编码系统。
- 字体集（Fontsets）：字体集是覆盖全部字符范围的字体集合。
- 定义字体集（Defining Fontsets）：定义新的字体集。

- 修改字体集（Modifying Fontsets）：修改已有的字体集。
- 无法显示的字符（Undisplayable Characters）：字符无法显示时怎么办。
- 单字节模式（Unibyte Mode）：可以挑选一种欧洲字符集，在不用多字节字符的情况下使用。
- 字符集（Charsets）：Emacs 如何对其内部字符代码分组。
- 双向编辑（Bidirectional Editing）：对从右到左文字的支持。

24.1 国际字符集简介

国际字符集和文字的使用者为保存文件制定了许多大体上标准的编码系统。这些编码系统通常是“多字节”（multibyte）的，意思是说，用两个或更多字节的序列来表示单个非 ASCII 字符。

在内部，Emacs 使用自己的多字节字符编码，它是“Unicode”标准的超集。这种内部编码允许把几乎每一种已知文字的字符混用在同一个缓冲区或字符串中。Emacs 在读写文件以及与子进程交换数据时，会在这种多字节字符编码与其他各种编码系统之间进行转换。

命令 ‘C-h h’（‘view-hello-file’）显示文件 ‘etc/HELLO’，该文件通过展示多种语言中“hello”的说法来说明各种文字。如果某些字符无法在你的终端上显示，它们会显示为 ‘?’ 或空心方框（参见“无法显示的字符”）。

即使在大量使用这些字符集的国家，键盘通常也没有包含其中全部字符的按键。你可以用 ‘C-x 8 <RET>’（‘insert-char’）插入键盘不支持的字符。参见“插入文本”。有些常见字符有简写输入方式；例如，输入 ‘C-x 8 [’ 可以插入左单引号 ‘，而在 Electric Quote 模式下通常只需键入 ‘`’。参见“引号”。Emacs 还支持各种“输入法”（input method），通常每种文字或语言有一种，它们使输入该文字的字符更为容易。参见“输入法”。

前缀键 ‘C-x <RET>’ 用于与多字节字符、编码系统和输入法相关的命令。

命令 ‘C-x =’（‘what-cursor-position’）显示光标点处字符的信息。除了字符位置（已在参见“光标位置信息”中介绍）之外，该命令还会显示该字符是如何编码的。例如，对于字符 ‘c’，它在回显区显示下面这一行：

```
Char: c (99, #o143, #x63) point=28062 of 36168 (78%) column=53
```

‘Char:’ 之后的四个值描述光标点之后的那个字符：先把它显示出来，然后给出它的字符代码，依次为十进制、八进制和十六进制。对于非 ASCII 多字节字符，如果缓冲区的编码系统能安全地用单个字节编码该字符，那么这些值之后还会跟着 ‘file’ 以及该字符在缓冲区编码系统中的十六进制表示（参见“编码系统”）。如果该字符的编码长于一个字节，Emacs 就显示 ‘file ...’。

在少数情况下，Emacs 会遇到“原始字节”（raw byte）：即值在 128（八进制 0200）到 255（八进制 0377）之间的单个字节，Emacs 无法把它解释为某个已知非 ASCII 字符编码的一部分。这样的原始字节被当作属于一个特殊的字符集 ‘eight-bit’；Emacs 把它们显示为转义形式的八进制代码（这一点可以自定义；参见“显示的自定义”）。在这种情况下，‘C-x =’ 显示 ‘raw-byte’ 而不是 ‘file’。此外，‘C-x =’ 显示的原始字节字符代码就像它们位于 ‘#x3FFF80..#x3FFFFF’ 范围内一样——Emacs 把它们映射到这个范围，以此区别于 ‘#x0080..#x00FF’ 范围内的 Unicode 字符。

带上前缀参数（‘C-u C-x =’）时，该命令还会调用 ‘describe-char’ 命令，显示该字符的详细说明：

- 字符集名称，以及在该字符集中标识该字符的代码；ASCII 字符被标识为属于 ‘ascii’ 字符集。

- 该字符的文字、语法和类别（category）。
- 在当前输入法下输入该字符要按什么键（如果该输入法支持该字符）。
- 该字符的编码，既有缓冲区内部的编码，也有把它保存到文件时的外部编码。
- 如果 Emacs 运行在图形显示上，则是该字符的字体名和字形代码。如果 Emacs 运行在文本终端上，则是发送给终端的代码。
- 如果该字符在显示时与后面的字符组合成一个或多个字形簇（grapheme cluster），则是组合信息：若框架位于图形显示上，为字体的字形；以及参与组合的字符。
- 该字符的文本属性（参见《Emacs Lisp 参考手册》的“Text Properties”），包括显示该字符所使用的任何非默认字体（face），以及包含它的任何覆盖层（参见《Emacs Lisp 参考手册》的“Overlays”）。

下面是一个例子，其中有些行为了适应本手册的版式而折行了：

```
             position: 1 of 1 (0%), column: 0
            character: ê (displayed as ê) (codepoint 234, #o352, #xea)
    preferred charset: unicode (Unicode (ISO10646))
code point in charset: 0xEA
               script: latin
               syntax: w        which means: word
             category: .:Base, L:Left-to-right (strong), c:Chinese,
                       j:Japanese, l:Latin, v:Viet
             to input: type "C-x 8 RET ea" or
                       "C-x 8 RET LATIN SMALL LETTER E WITH CIRCUMFLEX"
          buffer code: #xC3 #xAA
            file code: #xC3 #xAA (encoded by coding system utf-8-unix)
              display: by this font (glyph code)
    xft:-PfEd-DejaVu Sans Mono-normal-normal-
        normal-*-15-*-*-*-m-0-iso10646-1 (#xAC)
```

```
Character code properties: customize what to show
  name: LATIN SMALL LETTER E WITH CIRCUMFLEX
  old-name: LATIN SMALL LETTER E CIRCUMFLEX
  general-category: Ll (Letter, Lowercase)
  decomposition: (101 770) ('e' '^')
```

24.2 语言环境

只要启用了多字节字符，所有受支持的字符集都能在 Emacs 缓冲区中使用；不必为了显示某种语言的字符而先选定该语言。不过，为了设定各种默认值，选择“语言环境”（language environment）很重要。粗略地说，语言环境代表的是对首选文字的选择，而不是对语言的选择。

语言环境控制在读取文本时应识别哪些编码系统（参见“识别编码系统”）。这适用于文件、收到的邮件，以及任何其他读入 Emacs 的文本。它还可以指定创建文件时使用的默认编码系统。每个语言环境还指定一种默认输入法。

要选择语言环境，可以自定义 ‘current-language-environment’，或使用命令 ‘M-x set-language-environment’。使用该命令时当前是哪个缓冲区并无影响，因为其效果全局作用于整个 Emacs 会话。受支持的语言环境列表见

变量‘language-info-alist’；要了解语言环境 LANG-ENV 的更多信息，可使用命令‘C-h L LANG-ENV <RET>’（‘describe-language-environment’）。受支持的语言环境包括：

```
ASCII, Arabic, Belarusian, Bengali, Brazilian Portuguese,
Bulgarian, Burmese, Cham, Chinese-BIG5, Chinese-CNS,
Chinese-EUC-TW, Chinese-GB, Chinese-GB18030, Chinese-GBK, Croatian,
Cyrillic-ALT, Cyrillic-ISO, Cyrillic-KOI8, Czech, Devanagari,
Dutch, English, Esperanto, Ethiopic, French, Georgian, German,
Greek, Gujarati, Hebrew, IPA, Italian, Japanese, Kannada, Khmer,
Korean, Lao, Latin-1, Latin-2, Latin-3, Latin-4, Latin-5, Latin-6,
Latin-7, Latin-8, Latin-9, Latvian, Lithuanian, Malayalam, Oriya,
Persian, Polish, Punjabi, Romanian, Russian, Sinhala, Slovak,
Slovenian, Spanish, Swedish, TaiViet, Tajik, Tamil, Telugu, Thai,
Tibetan, Turkish, UTF-8, Ukrainian, Vietnamese, Welsh, and
Windows-1255.
```

要在图形显示上显示你的语言环境所用的文字，需要有合适的字体。有关设置字体的更多细节，参见“字体集”。

有些操作系统允许你通过设置 locale 环境变量‘LC_ALL’、‘LC_CTYPE’或‘LANG’来指定自己使用的字符集 locale。（如果其中不止一个被设置了，则以第一个非空的值作为此用途的 locale。）启动时，Emacs 在系统的 locale 别名表中查找你的字符集 locale 的名称，把它的规范名称与变量‘locale-charset-language-names’和‘locale-language-names’的值中的条目进行匹配（前者优先于后者），若找到匹配项，就选择相应的语言环境。它还会调整显示表和终端编码系统、locale 编码系统、该 locale 所需的优先编码系统，以及——最后但同样重要——Emacs 解码键盘所发送非 ASCII 字符的方式。

如果你在运行 Emacs 期间（用‘M-x setenv’）修改了‘LC_ALL’、‘LC_CTYPE’或‘LANG’环境变量，之后也许需要调用‘set-locale-environment’命令，根据新的 locale 重新调整语言环境。

‘set-locale-environment’函数通常使用语言环境所确立的优先编码系统来解码系统消息。但如果你的 locale 匹配变量‘locale-preferred-coding-systems’中的某个条目，Emacs 就会改用相应的编码系统。例如，如果 locale ‘ja_JP.PCK’在‘locale-preferred-coding-systems’中匹配‘japanese-shift-jis’，那么即使通常可能会用‘utf-8’，Emacs 也使用那种编码。

你可以显式使用命令‘set-language-environment’，或在初始化文件中自定义‘current-language-environment’，来覆盖启动时选定的语言环境。

要显示某个语言环境 LANG-ENV 的效果信息，可使用命令‘C-h L LANG-ENV <RET>’（‘describe-language-environment’）。它会告诉你该语言环境适用于哪些语言，并列出与之配套的字符集、编码系统和输入法。它还会显示一些示例文本，以说明该语言环境所用的文字。如果给 LANG-ENV 输入空值，该命令就会描述当前选定的语言环境。

你可以用普通钩子‘set-language-environment-hook’自定义任何语言环境。命令‘set-language-environment’在设置好新的语言环境之后运行该钩子。钩子函数可以通过检查变量‘current-language-environment’来判断当前是某个特定的语言环境。针对特定语言环境的非默认设置（例如键盘输入和终端输出所用的编码系统、默认输入法等）应当放在这个钩子里。

在开始设置新的语言环境之前，‘set-language-environment’会先运行钩子‘exit-language-environment-hook’。这个钩子适合用来撤消通过‘set-language-environment-hook’所做的自定义。例如，如果你用‘set-language-environment-hook’在某个特定语言环境中设置了特殊的键绑定，就应当设置‘exit-language-environment-hook’来恢复该键的常规绑定。

24.3 输入法

“输入法”（input method）是一种专为交互式输入而设计的字符转换。本节介绍 Emacs 自带的输入法；至于底层操作系统提供的本机输入法，参见“单字节编辑模式”。

在 Emacs 中，通常每种语言都有自己的输入法；有时几种使用相同字符的语言可以共用一个输入法。少数语言支持多种输入法。

最简单的输入法把 ASCII 字母映射为另一种字母表；这使你能够用另一种字母表代替 ASCII。希腊文和俄文输入法就是这样工作的。

更强大的技术是组合（composition）：把字符序列转换成一个字母。许多欧洲语言的输入法都使用组合，由一个字母后跟重音字符（或反过来）组成的序列产生单个非 ASCII 字母。例如，有些输入法把序列‘o ^’转换成单个带重音的字母。这类输入法没有自己的特殊命令；它们所做的只是把可打印字符的序列组合起来。

用于音节文字的输入法通常先做映射，再做组合。泰文和朝鲜文的输入法就是这样工作的。首先，把字母映射为表示特定读音或声调符号的符号；然后，把组成完整音节的这些符号序列映射为一个音节符号。

汉语和日语需要更复杂的方法。在中文输入法中，你先输入汉语词的拼音拼写（例如输入法‘chinese-py’及其他一些输入法），或者输入字符各部分的序列（输入法‘chinese-4corner’、‘chinese-sw’等）。一个输入序列通常对应许多可能的汉字。你用‘C-f’、‘C-b’、‘C-n’、‘C-p’等键（或方向键）以及数字键从中选出所要的那一个，这些键在这种情况下有特殊含义。

这些候选字符在概念上排成若干行，每行最多容纳 10 个候选项。通常 Emacs 一次只在回显区显示一行；行首会出现‘(I/J)’字样，表示这是总共 J 行中的第 I 行。输入‘C-n’或‘C-p’可显示下一行或上一行。

输入‘C-f’和‘C-b’可在当前行的候选项之间前后移动。移动时，Emacs 用特殊颜色高亮当前候选项；输入‘C-<SPC>’可选中当前候选项并将其作为输入。行中的候选项还带有编号；编号出现在候选项之前。输入一个数字即可选中当前行中相应的候选项并将其作为输入。

在这些中文输入法中，<TAB> 会显示一个列出所有候选字符的缓冲区；然后在其中某个字符上点击‘mouse-2’即可选中该候选项。‘C-f’、‘C-b’、‘C-n’、‘C-p’和数字键照常工作，但它们是在显示候选字符的缓冲区中进行高亮，而不是在回显区。

如果想按“pīnyīn”转写方法输入字符，请使用‘chinese-sisheng’输入法。这是一种基于组合的方法，例如，‘pi1’会得到‘pī’。

在日语输入法中，你先用拼音拼写输入整个词；等词进入缓冲区后，Emacs 再用一部大词典把它转换成一个或多个字符。一个拼音拼写对应若干个不同的日语词；要从中选一个，可用‘C-n’和‘C-p’在候选项之间循环。

有时，中断输入法的处理过程是有用的，这样你刚输入的字符就不会与后续字符组合。例如，在输入法‘latin-1-postfix’中，序列‘o ^’会组合成带重音的‘o’。如果你想把它们作为两个独立字符输入，该怎么办呢？

一种办法是把重音符号输入两次；这是用于输入独立字母和重音符号的特殊功能。例如，‘o ^ ^’ 会得到 ‘o^’ 这两个字符。另一种办法是在 ‘o’ 之后输入另一个不会与之组合的字母，然后立即把它删掉。例如，你可以输入 ‘o o <DEL> ^’，得到分开的 ‘o’ 和 ‘^’。还有一种更通用但不那么容易输入的方法，是在两个字符之间使用 ‘C-\ C-\’ 来阻止它们组合。这就是把命令 ‘C-\’（‘toggle-input-method’）连用两次。参见“选择输入法”。

‘C-\ C-\’ 在增量搜索中特别有用，因为它会停止等待更多字符来组合，并开始搜索你已经输入的内容。

要弄清如何在当前输入法下输入光标点之后的字符，请键入 ‘C-u C-x =’。参见“光标位置信息”。

变量 ‘input-method-highlight-flag’ 和 ‘input-method-verbose-flag’ 控制输入法如何说明正在发生的事情。如果 ‘input-method-highlight-flag’ 非 ‘nil’，部分序列会在缓冲区中高亮显示（对大多数输入法而言如此——有些输入法禁用此功能）。如果 ‘input-method-verbose-flag’ 非 ‘nil’，则会在回显区显示接下来可输入的候选字符列表（但在迷你缓冲区中不会显示）。

你可以通过修改添加到钩子变量 ‘quail-activate-hook’ 的函数，来改变输入法的工作方式。参见“钩子”。例如，你可以用 ‘keymap-set’ 在函数 ‘quail-translation-keymap’ 返回的键位图中定义键绑定，从而重新定义输入法的一些按键。参见“在初始化文件中重绑定按键”。

如果缓冲区中的文本因某种原因只读，输入法就会被抑制。这样，即使输入法处于活动状态，单字符键绑定在把缓冲区文本或其一部分设为只读的模式（如 ‘read-only-mode’ 和 ‘image-mode’）中仍然有效。

另一种输入键盘上没有的字符的手段，是用 ‘C-x 8 <RET>’（‘insert-char’）按字符的 Unicode 名称或码点插入单个字符；参见“插入文本”。

有一些专门用于插入 Emoji 的命令，它们位于 ‘C-x 8 e’ 键位图上。‘C-x 8 e e’（‘emoji-insert’）让你在各个 Emoji 类别中浏览，然后选择一个。‘C-x 8 e l’（‘emoji-list’）会弹出一个新缓冲区并列出所有 Emoji；在一个 Emoji 字符上点击（或使用 ‘RET’）就会把它插入当前缓冲区。最后，‘C-x 8 e s’（‘emoji-search’）让你根据名称搜索 Emoji。

‘describe-char’ 会显示光标点处字符或字形（包括 emoji）的大量信息。有时你需要快速了解名称，可以用 ‘C-x 8 e d’（‘emoji-describe’）命令来做这件事。它主要是为了帮助区分不同的 Emoji 变体（它们看起来可能非常相似），但它也会告诉你非 Emoji 字符的名称。

24.4 选择输入法

‘C-\’ 启用或禁用所选输入法（‘toggle-input-method’）。

‘C-x <RET> C-\ METHOD <RET>’ 为当前缓冲区选择新的输入法（‘set-input-method’）。

‘C-x \ METHOD <RET>’ 临时启用所选的一次性输入法；插入一个字符后它会自动禁用（‘activate-transient-input-method’）。

‘C-h I METHOD <RET>’ ‘C-h C-\ METHOD <RET>’ 描述输入法 METHOD（‘describe-input-method’）。默认情况下，它描述当前的输入法（如果有）。这份说明应当给出使用任一特定输入法的完整细节。

‘M-x list-input-methods’ 列出所有受支持的输入法。

要为当前缓冲区选择输入法，请使用 ‘C-x <RET> C-\’（‘set-input-method’）。该命令从迷你缓冲区读取输入法名称；名称通常以它打算配用的语言环境开头。变量 ‘current-input-method’ 记录当前选中的输入法。

输入法使用各种 ASCII 字符序列来代表非 ASCII 字符。有时暂时关闭输入法是有用的。为此，请键入 ‘C-\’（‘toggle-input-method’）。要重新启用输入法，再次键入 ‘C-\’。

如果你键入 ‘C-\’ 而尚未选择输入法，它会提示你指定一个。这与用 ‘C-x <RET> C-\’ 指定输入法的效果相同。

带数字参数调用时（如 ‘C-u C-\’），‘toggle-input-method’ 总会提示你输入一个输入法，并把最近选过的那个作为默认值。

选择一个语言环境，就为各种缓冲区指定了一个默认输入法。有了默认输入法之后，你可以在当前缓冲区中键入 ‘C-\’ 来选用它。变量 ‘default-input-method’ 指定默认输入法（值为 ‘nil’ 表示没有）。

有些语言环境支持多种不同的输入法，你可能想使用与 ‘set-language-environment’ 所选定者不同的输入法。如有需要，你可以借助 ‘set-language-environment-hook’ 为某个语言环境指定另一个默认输入法（参见“语言环境”）。例如：

```
(defun my-chinese-setup ()
  "Set up my private Chinese environment."
  (if (equal current-language-environment "Chinese-GB")
      (setq default-input-method "chinese-tonepy")))
(add-hook 'set-language-environment-hook 'my-chinese-setup)
```

这样，每当你选择 Chinese-GB 语言环境时，默认输入法就被设为 ‘chinese-tonepy’。

你还可以让 Emacs 自动启用某个输入法。例如：

```
(add-hook 'text-mode-hook
  (lambda () (set-input-method "german-prefix")))
```

这会在文本模式（Text mode）中自动启用输入法 ‘german-prefix’。

有些用于表音文字的输入法，其工作方式实际上是把键盘重新映射，以模拟这些文字常用的各种键盘布局。如何正确地进行这种重映射，取决于你实际的键盘布局。要指定你的键盘采用哪种布局，请使用命令 ‘M-x quail-set-keyboard-layout’。

你可以使用命令 ‘M-x quail-show-key’ 显示：在当前选用的键盘布局下，要输入光标点之后的那个字符应当键入哪个键（或键序列）。命令 ‘C-u C-x =’ 除了给出该字符的其他信息之外，也会显示这一信息。

‘M-x list-input-methods’ 显示所有受支持的输入法列表。列表中给出每种输入法的信息，其中包括在模式行中代表它的字符串。

有时临时启用输入法只为了插入单个字符会很方便。键入 ‘C-x \’（‘activate-transient-input-method’）会暂时启用一个输入法，让你按该输入法的规则插入一个字符，然后自动禁用该输入法。如果尚未选定过临时输入法，‘C-x \’ 会提示你输入一个输入法；此后再次调用该命令，就会启用所选定的临时输入法。要选择另一个临时输入法，请键入 ‘C-u C-x \’。你可以选择一个与用 ‘C-u C-\’ 所选定的输入法不同的临时输入法。

24.5 编码系统

使用各种语言的用户已经为表示这些语言建立了许多多少算是标准的编码系统。Emacs 在内部并不使用这些编码系统；相反，它在读取数据时把各种编码系统转换成自己的体系，在写出数据时再把内部编码系统转换成其

他编码系统。无论是读写文件、从终端收发数据，还是与子进程交换数据，都可以进行这种转换。

Emacs 为每个编码系统指定一个名字。多数编码系统用于一种语言，其名字以语言名开头。有些编码系统用于多种语言；它们的名字通常以 'iso' 开头。此外还有一些特殊的编码系统，例如 'no-conversion'、'raw-text' 和 'emacs-internal'。

有一类特殊的编码系统，统称为"代码页"（codepage），是为支持由 MS-Windows 和 MS-DOS 软件编码的文本而设计的。这些编码系统的名字是 'cpNNNN'，其中 NNNN 是代码页的 3 位或 4 位编号。你可以像使用其他任何编码系统那样使用这些编码；例如，要访问以代码页 850 编码的文件，请键入 'C-x <RET> c cp850 <RET> C-x C-f FILENAME <RET>'。

除了转换非 ASCII 字符的各种表示形式之外，编码系统还可以进行行尾转换。对于文件中如何分隔各行，Emacs 处理三种不同的约定：换行符（Unix）、回车符后跟换行符（DOS），以及只用回车符（Mac）。

- 'C-h C CODING <RET>'：描述编码系统 CODING（'describe-coding-system'）。
- 'C-h C <RET>'：描述当前正在使用的编码系统（'describe-coding-system'）。
- 'M-x list-coding-systems'：显示所有受支持的编码系统列表。

命令 'C-h C'（'describe-coding-system'）显示有关特定编码系统的信息，其中包括这些编码系统所指定的行尾转换。你可以把编码系统名作为参数给出；若不给出参数，它则描述当前为各种用途选定的编码系统——既包括当前缓冲区中的，也包括作为默认值的——以及识别编码系统所用的优先级列表（参见"识别编码系统"）。

要显示所有受支持的编码系统列表，请键入 'M-x list-coding-systems'。该列表给出每个编码系统的信息，其中包括在模式行中代表它的字母（参见"模式行"）。

这个列表中出现的每个编码系统——除了 'no-conversion'（表示不做任何种类的转换）之外——都规定了是否以及如何转换可打印字符，但把行尾转换的选择留待根据每个文件的内容来决定。例如，若文件看起来用回车符加换行符的序列分隔各行，就会使用 DOS 行尾转换。

列表中每个编码系统都有三个变体，它们精确规定行尾转换该怎么做：

- '...-unix'：不做任何行尾转换；假定文件用换行符分隔各行。（这是 Unix 和 GNU 系统以及 macOS 上通常使用的约定。）
- '...-dos'：假定文件用回车符后跟换行符来分隔各行，并进行相应的转换。（这是 Microsoft 系统上通常使用的约定。(1)）
- '...-mac'：假定文件用回车符分隔各行，并进行相应的转换。（这是 Classic Mac OS 中使用的约定，但如今在旧式软件之外已很少见。）

为简洁起见，这些变体编码系统不出现在 'list-coding-systems' 的显示中，因为它们完全可预测。例如，编码系统 'iso-latin-1' 有变体 'iso-latin-1-unix'、'iso-latin-1-dos' 和 'iso-latin-1-mac'。

编码系统 'unix'、'dos' 和 'mac' 分别是 'undecided-unix'、'undecided-dos' 和 'undecided-mac' 的别名。这些编码系统只规定行尾转换，而把字符代码的转换留待从文本本身推断。

编码系统 'raw-text' 适用于这样的文件：它主要是 ASCII 文本，但可能含有高于 127 的字节值，而这些字节值并非用来编码非 ASCII 字符。使用 'raw-text' 时，Emacs 原样复制这些字节值，并在当前缓冲区中把 'enable-

multibyte-characters’ 设为 ‘nil’，以便正确解释它们。‘raw-text’ 按通常方式处理行尾转换，依据所遇到的数据，并且有通常的三个变体来指定使用哪种行尾转换。

相比之下，编码系统 ‘no-conversion’ 完全不做字符代码转换——既不对非 ASCII 字节值做转换，也不对行尾做转换。这对读取或写出二进制文件、tar 文件以及其他必须逐字检查的文件很有用。它同样把 ‘enable-multibyte-characters’ 设为 ‘nil’。

不做任何转换地编辑文件，最简单的方法是使用 ‘M-x find-file-literally’ 命令。它使用 ‘no-conversion’，还会抑制其他可能在你看到文件内容之前就对其进行转换的 Emacs 功能。参见“访问文件”。

编码系统 ‘emacs-internal’（或与之等价的 ‘utf-8-emacs’）表示文件中的非 ASCII 字符是用 Emacs 内部编码存储的。该编码系统依据所遇到的数据处理行尾转换，并有通常的三个变体来指定行尾转换的种类。

> 注：(1) 它也被规定用于 MIME 的 ‘text/*’ 正文，以及其他网络传输场合。它与 SGML 参考语法中 record-start/record-end 格式不同，后者 Emacs 并不直接支持。

24.6 识别编码系统

每当 Emacs 读取某段文本时，它都会设法识别应使用哪个编码系统。这适用于正在读取的文件、子进程的输出、来自 X 选区的文本等等。多数情况下 Emacs 都能自动选出正确的编码系统——前提是你已经指定了自己的偏好。

有些编码系统可以通过数据中出现的字节序列来识别或区分。然而，有些编码系统是无法区分的，甚至原则上无法区分。例如，根本无法区分 Latin-1 与 Latin-2；它们使用相同的字节值而含义不同。

Emacs 通过编码系统的优先级列表来处理这种情况。每当 Emacs 读取文件时，如果你没有指定要用的编码系统，它就会用数据逐一检验每个编码系统，从优先级最高的开始沿列表向下，直到找到一个与数据相符的编码系统。然后它就假定文件内容是用该编码系统表示的，并据此转换。

编码系统的优先级列表取决于选定的语言环境（参见“语言环境”）。例如，如果你使用法语，大概会希望 Emacs 优先采用 Latin-1 而不是 Latin-2；如果你使用捷克语，大概会希望优先采用 Latin-2。这也是应当指定语言环境的理由之一。

不过，你可以用命令 ‘M-x prefer-coding-system’ 详细地改动编码系统优先级列表。该命令从迷你缓冲区读取一个编码系统名，并把它加到优先级列表的前端，从而使它优先于所有其他的。如果你多次使用该命令，每次使用都会把一个元素加到优先级列表的前端。

如果你使用的编码系统规定了行尾转换类型，例如 ‘iso-8859-1-dos’，那么这意味着：Emacs 应优先设法识别 ‘iso-8859-1’，而一旦确实识别出 ‘iso-8859-1’，就应使用 DOS 行尾转换。

有时文件名会指示该文件应使用哪个编码系统。变量 ‘file-coding-system-alist’ 规定这种对应关系。有一个专用函数 ‘modify-coding-system-alist’ 用于向该列表添加元素。例如，若要读写所有 ‘.txt’ 文件时都使用编码系统 ‘chinese-iso-8bit’，可以执行下面这个 Lisp 表达式：

```
(modify-coding-system-alist 'file "\\.txt\\'" 'chinese-iso-8bit)
```

第一个参数应当是‘file’，第二个参数应当是一个正则表达式，用以确定该设置适用于哪些文件，第三个参数说明这些文件要使用哪个编码系统。

Emacs 会根据文件内容识别该使用哪种行尾转换：如果它看到的只是回车符，或者只是回车符后跟换行符的序列，就相应地选择行尾转换。你可以通过把变量‘inhibit-eol-conversion’设为非‘nil’来禁止自动使用行尾转换。若这样做，DOS 风格的文件在缓冲区中会显示出可见的‘^M’字符；有些人更喜欢这样，而不喜欢模式行左端附近那个不太显眼的‘(DOS)’行尾类型指示（参见“模式行”）。

默认情况下，编码系统的自动检测对转义序列是敏感的。如果 Emacs 看到以转义字符开头的一串字符，而该序列作为 ISO-2022 代码是合法的，这就告诉 Emacs 用某种 ISO-2022 编码来解码该文件。

然而，有些情况下你想把文件中的转义序列按原样读入。此时你可以把变量‘inhibit-iso-escape-detection’设为非‘nil’。于是代码检测会忽略任何转义序列，绝不使用 ISO-2022 编码。结果是所有转义序列都在缓冲区中变得可见。

‘inhibit-iso-escape-detection’的默认值是‘nil’。我们建议你不要永久地改动它，只在某一次特定操作中改动。这是因为 Emacs 发行版中的某些 Emacs Lisp 源文件含有以编码系统‘iso-2022-7bit’编码的非 ASCII 字符；如果你抑制了转义序列检测，访问这些文件时它们就无法被正确解码。

变量‘auto-coding-alist’和‘auto-coding-regexp-alist’分别是为某些文件名模式、或为含有某些模式的文件指定编码系统的最强方式。这两个变量甚至会覆盖文件本身的‘*--coding:--*’标记（参见“指定文件的编码系统”）。例如，Emacs 对 tar 文件和归档文件使用‘auto-coding-alist’，以免被归档成员中的‘*--coding:--*’标记搞混，误以为它适用于整个归档文件。

指定编码系统的另一种方式是使用变量‘auto-coding-functions’。例如，内置的‘auto-coding-functions’之一能检测 XML 文件的编码。与前两个不同，该变量不会覆盖任何‘*--coding:--*’标记。

24.7 指定文件的编码系统

如果 Emacs 错误地识别了某个文件的编码，你可以用‘C-x <RET> r’（‘revert-buffer-with-coding-system’）以正确的编码系统重新读取该文件。该命令会提示输入要用的编码系统。要查看 Emacs 实际用了哪个编码系统来解码该文件，请看模式行左端附近的编码系统助记字母（参见“模式行”），或键入‘C-h C’（‘describe-coding-system’）。

你可以在文件本身中为该文件指定编码系统，方法是在开头使用‘*--...--*’结构，或在结尾使用局部变量列表（参见“文件中的局部变量”）。为此，你要为名为‘coding’的“变量”定义一个值。Emacs 实际上并没有名为‘coding’的变量；这样做并不是设置变量，而是为该文件使用所指定的编码系统。例如，‘*--mode: C; coding: latin-1; --*’指定使用 Latin-1 编码系统，同时也指定使用 C 模式。当你在文件中显式指定编码时，该指定会覆盖‘file-coding-system-alist’。

24.8 为输出选择编码系统

一旦 Emacs 为某个缓冲区选定了编码系统，它就把该编码系统存入‘buffer-file-coding-system’。这使它成为从该缓冲区写出到文件的操作（例如‘save-buffer’和‘write-region’）的默认值。你可以用‘set-buffer-file-coding-system’为该缓冲区后续的文件输出指定另一个编码系统（参见“为文件文本指定编码系统”）。

你可以把 Emacs 支持的任何字符插入任何 Emacs 缓冲区，但多数编码系统只能处理这些字符的一个子集。因此，你插入的字符有可能无法用将要保存该缓冲区时所用的编码系统编码。例如，你可能访问一个用 'iso-8859-2' 编码的波兰语文本文件，并向其中加入一些俄语词。当你保存该缓冲区时，Emacs 不能使用 'buffer-file-coding-system' 的当前值，因为你加入的字符无法用该编码系统编码。

出现这种情况时，Emacs 会尝试最优先的编码系统（由 'M-x prefer-coding-system' 或 'M-x set-language-environment' 设定）。如果该编码系统能安全地编码缓冲区中的所有字符，Emacs 就使用它，并把它的值存入 'buffer-file-coding-system'。否则，Emacs 会显示一个适合编码该缓冲区内容的编码系统列表，并请求你从中选择一个。

如果你在不合适的字符是插入邮件消息中的，Emacs 的行为会略有不同。它还会检查最优先的编码系统是否被推荐用于 MIME 消息；如果不是，它就告知你这一事实，并提示你换一个编码系统。这样你就不会在不知不觉中发出一个收件人的邮件软件难以解码的消息。（如果你在提示处输入其名字，仍然可以使用不合适的编码系统。）

当你发送邮件消息时（参见"发送邮件"），Emacs 有四种不同的方式来确定用于编码消息正文的编码系统。它首先尝试缓冲区自己的 'buffer-file-coding-system' 值（如果该值非 'nil'）。否则，如果 'sendmail-coding-system' 的值非 'nil'，就使用该值。第三，使用 'default-sendmail-coding-system' 的值。如果这三个值全都是 'nil'，Emacs 就用新文件的默认编码系统（即 'buffer-file-coding-system' 的默认值）来编码发出的邮件，而该默认值由你所选的语言环境控制。

24.9 为文件文本指定编码系统

在 Emacs 没有自动为文件内容选对编码系统的情况下，你可以使用下列命令来指定一个：

- 'C-x <RET> f CODING <RET>'：用编码系统 CODING 保存或重新访问当前缓冲区中的文件（'set-buffer-file-coding-system'）。
- 'C-x <RET> c CODING <RET>'：为紧接其后的命令指定编码系统 CODING（'universal-coding-system-argument'）。
- 'C-x <RET> r CODING <RET>'：用编码系统 CODING 重新访问当前文件（'revert-buffer-with-coding-system'）。
- 'M-x recode-region <RET> RIGHT <RET> WRONG <RET>'：转换一个曾用编码系统 WRONG 解码的区域，改用编码系统 RIGHT 来解码。

命令 'C-x <RET> f'（'set-buffer-file-coding-system'）设定当前缓冲区的文件编码系统（即保存或还原该文件时要用的编码系统）。你使用迷你缓冲区指定是哪个编码系统。你也可以用 'mouse-3' 点击模式行中的编码系统指示符来调用该命令（参见"模式行"）。

如果你指定的编码系统无法处理缓冲区中的所有字符，那么当你试图保存该缓冲区时，Emacs 会就这些麻烦的字符向你发出警告，并请求你另选一个编码系统（参见"为输出选择编码系统"）。

你也可以用该命令指定编码当前缓冲区时所用的行尾转换（参见"编码系统"）。例如，'C-x <RET> f dos <RET>' 会使 Emacs 以 DOS 风格的回车符后跟换行符的行尾来保存当前缓冲区的文本。

指定文件编码系统的另一种时机是在访问该文件之时。首先使用命令‘C-x <RET> c’（‘universal-coding-system-argument’）；该命令用迷你缓冲区读入一个编码系统名。退出迷你缓冲区之后，所指定的编码系统就用于*紧接其后的命令*。

因此，例如紧接其后的命令若是‘C-x C-f’，它就用该编码系统读取文件（并记下该编码系统，供以后保存该文件时使用）。而紧接其后的命令若是‘C-x C-w’，它就用该编码系统写出文件。以这种方式为保存指定编码系统时，若缓冲区含有该编码系统无法处理的字符，不会有任何警告，这点与用‘C-x <RET> f’时不同。

其他受所指定编码系统影响的文件命令包括‘C-x i’和‘C-x C-v’，以及‘C-x C-f’的各种在另一窗口中的变体。‘C-x <RET> c’也会影响启动子进程的命令，包括‘M-x shell’（参见“在 Emacs 中运行 Shell 命令”）。如果紧接其后的命令并不使用编码系统，那么‘C-x <RET> c’最终不产生任何效果。

不做转换地访问文件，一个简便方法是使用‘M-x find-file-literally’命令。参见“访问文件”。

变量‘buffer-file-coding-system’的默认值规定你创建新文件时选用哪个编码系统。它适用于你访问一个新文件的情形，也适用于你创建缓冲区然后再把它存入文件的情形。选择一个语言环境通常会把该变量设成适合该语言环境的默认编码系统。

如果你用错误的编码系统访问了文件，可以用‘C-x <RET> r’（‘revert-buffer-with-coding-system’）来纠正。这会用你指定的编码系统再次访问当前文件。

如果一段文本已经用错误的编码系统插入了缓冲区，你可以用‘M-x recode-region’重新对它解码。该命令先提示你输入正确的编码系统，然后提示输入实际用过的错误编码系统，再执行转换。它先用错误的编码系统编码该区域，然后用正确的编码系统重新解码。

24.10 用于进程间通信的编码系统

本节说明如何为与其他进程通信指定编码系统。

- ‘C-x <RET> x CODING <RET>’：用编码系统 CODING 与其他图形应用程序之间传送选区（‘set-selection-coding-system’）。
- ‘C-x <RET> X CODING <RET>’：用编码系统 CODING 与另一个图形应用程序之间传送*一个*选区——即下一个选区（‘set-next-selection-coding-system’）。
- ‘C-x <RET> p INPUT-CODING <RET> OUTPUT-CODING <RET>’：在当前缓冲区中为子进程的输入和输出分别使用编码系统 INPUT-CODING 和 OUTPUT-CODING（‘set-buffer-process-coding-system’）。

命令‘C-x <RET> x’（‘set-selection-coding-system’）指定把选中的文本发送给其他窗口系统应用程序、以及接收在其他应用程序中所做选区的文本时所用的编码系统。该命令适用于此后所有的选区，直到你再次使用该命令将其覆盖为止。命令‘C-x <RET> X’（‘set-next-selection-coding-system’）指定在 Emacs 中做出的下一个选区、或由 Emacs 读取的下一个选区所用的编码系统。

变量‘x-select-request-type’指定从 X Window System 接收其他应用程序的文本选区时所要请求的数据类型。如果该值是‘nil’（默认值），Emacs 会按此顺序尝试‘UTF8_STRING’和‘COMPOUND_TEXT’，并用各种启发式方法从两个结果中选出更合适的那个；如果这些都不成功，Emacs 就退回到‘STRING’。如果‘x-select-request-type’的值是符号‘COMPOUND_TEXT’、‘UTF8_STRING’、‘STRING’或‘TEXT’之一，Emacs 就只使用该请求

类型。如果该值是其中若干符号组成的列表，Emacs 就只按顺序尝试列表中的请求类型，直到某个成功为止，或者直到列表用尽。

命令 'C-x <RET> p'（'set-buffer-process-coding-system'）指定与某个子进程之间输入和输出所用的编码系统。该命令作用于当前缓冲区；通常每个子进程都有自己的缓冲区，因此你可以在相应的缓冲区中发出该命令，从而为与某个特定子进程之间的转换进行指定。

也可以在运行或启动子进程的命令之前紧接使用 'C-x <RET> c'（'universal-coding-system-argument'），以指定与该子进程通信所用的编码系统。参见"为文件文本指定编码系统"。

进程输入输出的转换默认值取决于当前语言环境。

变量 'locale-coding-system' 指定在编码和解码系统字符串（如系统错误消息，以及 'format-time-string' 的格式与时间戳）时使用的编码系统。该编码系统还可能用于在 X Window System 上解码非 ASCII 键盘输入，也会在批处理模式下用于编码发送到标准输出和标准错误流的文本。你应当选择与底层系统的文本表示相兼容的编码系统，后者通常由环境变量 'LC_ALL'、'LC_CTYPE' 和 'LANG' 之一指定。（按上述顺序，第一个值非空者即决定文本表示。）

24.11 文件名的编码系统

'C-x <RET> F CODING <RET>' 用编码系统 CODING 对文件名进行编码与解码（'set-file-name-coding-system'）。

命令 'C-x <RET> F'（'set-file-name-coding-system'）指定用于编码文件*名称*的编码系统。它对读写文件*内容*没有影响。

实际上，该命令所做的只是设置变量 'file-name-coding-system' 的值。若把该变量设为一个编码系统名（Lisp 符号或字符串），Emacs 在所有文件操作中都用该编码系统编码文件名。这样就可以在文件名中使用非 ASCII 字符——至少是所指定编码系统能够编码的那些非 ASCII 字符。

若 'file-name-coding-system' 为 'nil'，Emacs 使用由所选语言环境决定的默认编码系统，它保存在变量 'default-file-name-coding-system' 中（通常为 UTF-8）。

当 Emacs 运行在属于 NT 家族后代的 MS-Windows 版本（Windows 2000、XP 及所有更高版本）上时，'file-name-coding-system' 的值在很大程度上被忽略，因为 Emacs 默认使用可直接传递 Unicode 文件名的 API。相反，在 Windows 9X 上，文件名用 'file-name-coding-system' 编码，应把它设为与当前系统区域设置相关的代码页（codepage，参见"编码系统"）。变量 'w32-unicode-filenames' 的值控制 Emacs 在调用接受文件名的操作系统函数时是否使用 Unicode API。启动代码在 Windows 9X 上把该变量设为 'nil'，在较新的 MS-Windows 版本上设为 't'。

警告： 若在 Emacs 会话中途更改 'file-name-coding-system'（或语言环境），而你已经访问过某些文件、其文件名是用先前的编码系统编码并且在新编码系统下无法编码（或编码结果不同），就可能出问题。若你试图按已访问的文件名保存这些缓冲区之一，保存时可能用错文件名，或遇到错误。若发生此类问题，请用 'C-x C-w' 为该缓冲区指定新的文件名。

若编码文件名时出错，可用命令 'M-x recode-file-name' 更改文件名的编码系统。它会提示输入现有文件名、其旧编码系统，以及你希望转换成的编码系统。

24.12 X 键盘输入的编码系统

X Window System 下的输入法会指定各自必须用于解码键盘输入的编码系统。默认情况下，Emacs 在建立与输入法服务器的连接时自动确定每种输入法所用的编码系统，并用该特定编码系统解码键盘输入。然而，这种判定有时会失败；此时改用区域设置编码系统（参见“用于进程间通信的编码系统”）。

若输入法未能正确声明其编码文本所用的编码系统，就必须手工指定 Emacs 解码来自输入法的文本所用的编码系统。变量‘x-input-coding-system’的值若为符号，就无条件地用作解码来自输入法的键盘输入的编码系统。

24.13 终端 I/O 的编码系统

‘C-x <RET> t CODING <RET>’用编码系统 CODING 进行终端输出（‘set-terminal-coding-system’）。

‘C-x <RET> k CODING <RET>’用编码系统 CODING 进行键盘输入（‘set-keyboard-coding-system’）。

命令‘C-x <RET> t’（‘set-terminal-coding-system’）指定终端输出的编码系统。若为终端输出指定了某种字符编码，则输出到终端的所有字符都转换成该编码系统。

此功能对某些为支持特定语言或字符集而构建的纯字符终端很有用——例如支持某种 ISO Latin 字符集的欧洲终端。使用多字节文本时需要指定终端编码系统，以便 Emacs 知道终端实际能处理哪些字符。

默认情况下，除非 Emacs 能从终端类型或区域设置规范推断出合适的编码系统（参见“语言环境”），输出到终端的内容完全不做转换。

命令‘C-x <RET> k’（‘set-keyboard-coding-system’）或变量‘keyboard-coding-system’指定键盘输入的编码系统。对于其按键发送非 ASCII 图形字符的终端（例如某些为 ISO Latin-1 或其子集设计的终端），键盘输入的字符编码转换很有用。

默认情况下，键盘输入依据系统区域设置进行转换。若终端并不真正支持区域设置所隐含的编码（例如你发现输入‘M-i’会插入一个非 ASCII 字符），就需要把‘keyboard-coding-system’设为‘nil’以关闭编码转换。可以在初始化文件中加入

```
(set-keyboard-coding-system nil)
```

来做到这一点。

设置‘keyboard-coding-system’在 MS-Windows 上没有效果，旧式 Windows 9X 系统除外，此时编码必须与 MS-Windows 控制台的当前代码页一致，后者可通过调用‘w32-set-console-codepage’修改。

为键盘输入使用编码系统转换与使用输入法有相似之处：两者都定义可转换为单个字符的键盘输入序列。然而，输入法是为方便人交互使用而设计的，被转换的序列通常是 ASCII 可打印字符序列；编码系统通常转换的是非图形字符序列。

24.14 字体集

一种字体通常只为单一字母表或文字定义字形。因此，要显示 Emacs 支持的全部文字范围，需要许多字体的集合。在 Emacs 中，这样的集合称为“字体集”（fontset）。字体集由一组字体规格定义，每条规格负责一定范围的字符编码，并且对于它所指定的字体未覆盖的字符，可以回退到另一个字体集。

每个字体集都像字体一样有名字。但字体存储在系统中，可用字体名由系统定义，而字体集是在 Emacs 内部定义的。定义字体集之后，就可以在 Emacs 中任何能使用单个字体的地方通过名字使用它。当然，Emacs 字体集只能使用你的系统所支持的字体。若某些字符在屏幕上显示为空框或十六进制码，说明用于它们的字体集中没有覆盖这些字符的字体。此时，或者字符虽能显示但效果不理想时，你可能需要安装额外的字体，或修改字体集以使用系统上已安装的特定字体（见下文）。你的操作系统可能带有可选字体可供安装；也可以安装 GNU Intlfonts 包，它包含大多数受支持文字所需的字体。（见注 1）

Emacs 自动创建三个字体集：“标准字体集”（standard fontset）、“启动字体集”（startup fontset）和“默认字体集”（default fontset）。默认字体集最可能包含大量各种非 ASCII 字符的字体，并且是另两个字体集的默认回退，也是在你设置默认字体而非字体集时使用的回退。但它不指定字体族名，因此直接使用它时结果可能有些随意。可以在启动 Emacs 时用 ‘-fn’ 选项指定某个字体集。例如，

```
emacs -fn fontset-standard
```

也可以用 ‘Font’ 资源指定字体集（参见“附录 D X 选项与资源”）。

若未指定使用的字体集，Emacs 使用一个 ASCII 字体，并以 ‘fontset-default’ 作为该字体未覆盖字符的回退。标准字体集尽管名为“标准”，也只有显式请求时才会使用。

要显示特定字体集的信息，使用 ‘M-x describe-fontset’ 命令。它提示输入字体集名，默认是当前框架所用的那个，然后显示该字体集中所有字符子范围以及分配给它们的字体。要查看在未指定字体集的情况下启动的会话（通常即如此）中 Emacs 使用了哪些字体，在提示处输入 ‘fontset-default <RET>’，或直接输入 ‘<RET>’ 以描述当前框架所用的字体集。

字体集不必为每个字符编码都指定字体。若字体集没有为某个字符指定字体，或指定的字体在系统上不存在，就无法正确显示该字符，而会把它显示为十六进制码、薄空格或空框。（详情参见“Text Display, for details”）或者字体集为某个字符范围指定了字体，但你可能不喜欢它们的外观。若如此，可以修改字体集；修改方法参见“修改字体集”。

> 注：若在 X 上运行 Emacs，可能需要用如下命令告知 X 服务器新安装字体的位置：`xset fp+ /usr/local/share/emacs/fonts` 和 `xset fp rehash`。

24.15 定义字体集

在 X 上运行时，Emacs 依据 ‘standard-fontset-spec’ 的值自动创建标准字体集。该字体集的名字是

```
-*-fixed-medium-r-normal-*-16-*-*-*-*-*-fontset-standard
```

或简写为 ‘fontset-standard’。

在 GNUstep 和 macOS 上，标准字体集用 ‘ns-standard-fontset-spec’ 的值创建；在 MS Windows 上则用 ‘w32-standard-fontset-spec’ 的值创建。

标准字体集的粗体、斜体和粗斜体变体会自动创建。它们的名字分别把 ‘medium’ 换成 ‘bold’、把 ‘r’ 换成 ‘i’，或两者都换。

Emacs 会根据你用 'Font' 资源或 '-fn' 参数指定的默认 ASCII 字体，或启动时找到的默认字体，自动生成一个字体集。这就是"启动字体集"，其名字为 'fontset-startup'。Emacs 生成它的方法是：把 CHARSET_REGISTRY 字段替换为 'fontset'，把 CHARSET_ENCODING 字段替换为 'startup'，然后用所得的字符串指定字体集。

例如，若以下面这种形式的字体启动 Emacs，

```
emacs -fn "*courier-medium-r-normal--14-140-*-iso8859-1"
```

Emacs 会生成下列字体集，并将其用于初始的 X 窗口框架：

```
-*-courier-medium-r-normal-*-14-140-*-*-*-*-fontset-startup
```

启动字体集对该字体支持的所有字符都使用你指定的字体（或具有不同注册与编码的变体），对其他字符回退到 'fontset-default'。

用 X 资源 'Emacs.Font'，可以像指定实际字体名那样指定字体集名。但要注意不要在 'Emacs*Font' 这类通配资源中指定字体集名——这种通配写法会匹配菜单等其他各种资源，而菜单无法处理字体集。参见"附录 D X 选项与资源"。

可以用名为 'Fontset-N' 的 X 资源指定额外的字体集，其中 N 是从 0 开始的整数。该资源的值应具有如下形式：

```
FONTPATTERN, [CHARSET:FONT]...
```

其中 FONTPATTERN 应具有标准 X 字体名的形式（见前面的 fontset-startup 示例），但最后两个字段除外。它们应具有 'fontset-ALIAS' 的形式。

每个字体集有两个名字，一长一短。长名是 FONTPATTERN。短名是 'fontset-ALIAS'，即长名的最后两个字段（例如启动时自动创建的字体集为 'fontset-startup'）。用任一名字都可以引用该字体集。

构造 'CHARSET:FONT' 指定（在此字体集中）某个特定字符集使用哪种字体。这里 CHARSET 是字符集名，FONT 是用于该字符集的字体。在定义一个字体集时，这种构造可以使用任意多次。

对于其他字符集，Emacs 根据 FONTPATTERN 选择字体。它把 'fontset-ALIAS' 替换为描述该字符集的值。对于 ASCII 字符字体，'fontset-ALIAS' 被替换为 'ISO8859-1'。

此外，当连续多个字段都是通配符时，Emacs 会把它们合并为单个通配符。这是为了防止使用自动缩放的字体。由放大较大字体所得的字体不适合编辑，缩小较小字体也没有用处，因为按其原有尺寸使用较小字体更好，Emacs 正是这样做的。

因此，若 FONTPATTERN 为

```
-*-fixed-medium-r-normal-*-24-*-*-*-*-*-fontset-24
```

则 ASCII 字符的字体规格为：

```
-*-fixed-medium-r-normal-*-24-*-ISO8859-1
```

而中文 GB2312 字符的字体规格为：

```
-*-fixed-medium-r-normal-*-24-*-gb2312*-*
```

你可能没有与上述字体规格匹配的中文字体。多数 X 发行版只包含 FAMILY 字段为 'song ti' 或 'fangsong ti' 的中文字体。这种情况下，可以把 'Fontset-N' 指定为：

```
Emacs.Fontset-0: -*-fixed-medium-r-normal-*-24-*-*-*-*-*-fontset-24,\
        chinese-gb2312:-*-*-medium-r-normal-*-24-*-gb2312*-*
```

于是，除中文 GB2312 字符外，其他所有字符的字体规格的 FAMILY 字段都是 'fixed'，而中文 GB2312 字符的字体规格的 FAMILY 字段是通配符 '*'。

处理字体集资源值以创建字体集的函数名为 'create-fontset-from-fontset-spec'。也可以显式调用该函数来创建字体集。

字体命名的更多信息参见"字体"。

24.16 修改字体集

字体集不必总要从头创建。若只需少量改动，修改现有字体集（通常是 'fontset-default'）可能更方便。修改 'fontset-default' 还会影响以它为回退的其他字体集，因此这可能是修正 Emacs 为特定文字所选字体问题的有效方法。

可以用函数 'set-fontset-font' 修改字体集，需指定要修改字体的字符、字符集、文字或字符范围，以及要使用的字体的规格。一些例子如下：

```
;; Prefer a big5 font for han characters.
(set-fontset-font "fontset-default"
                  'han (font-spec :registry "big5")
                  nil 'prepend)
```

```
;; Use "Noto Color Emoji" for the emoji script (this is the default).
(set-fontset-font "fontset-default" 'emoji
                  '("Noto Color Emoji" . "iso10646-1")
                  nil 'prepend)
```

```
;; Display the "heart" character using a color font.
(set-fontset-font "fontset-default"
                  #x2764 "Noto Color Emoji")
```

```
;; Use MyPrivateFont for the Unicode private use area.
(set-fontset-font "fontset-default"  '(#xe000 . #xf8ff)
                  "MyPrivateFont")
```

```
;; Use Liberation Mono for latin-3 charset.
(set-fontset-font "fontset-default" 'iso-8859-3
                  "Liberation Mono")
```

```
;; Use DejaVu Sans Mono as a fallback in fontset-startup
;; before resorting to fontset-default.
(set-fontset-font "fontset-startup" nil "DejaVu Sans Mono"
                  nil 'append)
```

修改‘symbol’文字的字体集时，‘use-default-font-for-symbols’的值会影响该字体集是否实际被使用。

关于使用‘set-fontset-font’函数的更多细节，参见《Emacs Lisp 参考手册》的“Fontsets”。

若不知道字符的代码点或它所属的文字，可以询问 Emacs。把光标点移到该字符上，输入‘C-u C-x =’（‘what-cursor-position’），该信息以及更多内容会显示在 Emacs 弹出的‘*Help*’缓冲区中。参见“光标位置信息”。例如，日文字符属于‘kana’文字，但日文文本还会混用汉字，所以下面的例子用‘han’文字来设置 Emacs 为日文文本使用‘Kochi Gothic’字体：

```
(set-fontset-font "fontset-default" 'han "Kochi Gothic")
```

（为方便起见，Emacs 中的‘han’文字被设置为支持所有中文、日文和韩文（即 CJK）字符，而不只是汉字。）

已知文字的列表见变量‘script-representative-chars’。

像上面那样的字体集设置只影响默认字体不支持的字符，因此若‘Kochi Gothic’字体覆盖拉丁字符，它也不会被用于显示拉丁文字，因为 Emacs 使用的默认字体通常覆盖基本拉丁字符。

系统上安装的某些字体可能有缺陷，或对使用它们的字符产生不佳效果，你可能希望让 Emacs 在搜索显示某字符所需的合适字体时完全忽略它们。可以把有问题的字体加入变量‘face-ignored-fonts’的值（一个列表）来做到这一点。下面是可放入‘~/.emacs’的一个例子：

```
(add-to-list 'face-ignored-fonts "Some Bad Font")
```

24.17 无法显示的字符

可能有些非 ASCII 字符你的终端无法显示。多数文本终端只支持单一字符集（用变量‘default-terminal-coding-system’告诉 Emacs 是哪一个，参见“终端 I/O 的编码系统”）；无法用该编码系统编码的字符默认显示为‘?’。

图形显示能显示更多种类的字符，但你未必为所有这些字符安装了字体；没有字体的字符显示为空心方框。

若你使用 Latin-1 字符但终端无法显示 Latin-1，可以改为显示助记的 ASCII 序列，例如用‘"o’表示 o-umlaut。加载‘iso-ascii’库即可做到这一点。

若终端能显示 Latin-1，你可以混合使用等效的 Latin-1 字符和 ASCII 助记符来显示其他欧洲字符集中的字符。自定义变量‘latin1-display’即可启用。这些助记 ASCII 序列大多与前缀输入法中的序列相对应。

24.18 单字节编辑模式

ISO 8859 Latin-N 字符集定义了八进制 0240 到 0377（十进制 160 到 255）范围内的字符编码，用于处理各种欧洲语言（以及一些非欧洲语言）所需的带重音字母和标点。注意 Emacs 把此范围内的字节视为原始字节而非字符，即使在单字节缓冲区中（即禁用多字节字符时）也是如此。不过，Emacs 仍能在同一时刻把这些字符编

码当作属于*某个*单字节字符集来处理。要指定使用其中*哪一个*编码，可调用 'M-x set-language-environment' 并指定合适的语言环境，如 'Latin-N'。参见《Emacs Lisp 参考手册》的"Disabling Multibyte"

只要所用终端或字体支持，Emacs 也能把 160 到 255 范围内的字节显示为可读字符。这是自动进行的。在图形显示上，Emacs 还可以通过字体集显示单字节字符，实际上就是按当前语言环境显示等效的多字节字符。要请求这一行为，把变量 'unibyte-display-via-language-environment' 设为非 'nil' 值。注意此设置只影响这些字节的显示方式，并不改变 Emacs 把它们视为原始字节而非字符这一基本事实。

若终端不支持 Latin-1 字符集的显示，Emacs 可以把这些字符显示为 ASCII 序列，至少让你清楚这些字符是什么。为此，加载 'iso-ascii' 库。其他 Latin-N 字符集的类似库可以实现，但迄今尚未实现。

通常，非 ISO-8859 字符（十进制 128 到 159，含端点）显示为八进制转义。对非标准的 ISO-8859 字符集扩展版本，可以用 'disp-table' 库中的函数 'standard-display-8bit' 改变这一行为。

输入单字节非 ASCII 字符有两种方法：

- 可以为所选语言环境使用输入法。参见"输入法"。在单字节缓冲区中使用输入法时，用它指定的非 ASCII 字符会被转换为单字节。
- 若你的键盘能产生表示非 ASCII 字符的十进制 128 及以上的字符编码，可以直接输入这些字符编码。

在图形显示上，使用这些键应当不需要做任何特殊处理，它们应该直接可用。在文本终端上，应当使用命令 'M-x set-keyboard-coding-system' 或自定义变量 'keyboard-coding-system' 来指定你的键盘所用的编码系统（参见"终端 I/O 的编码系统"）。启用此功能很可能需要你用 <ESC> 输入 Meta 字符；不过，在控制台终端或 'xterm' 之类的终端模拟器上，你可以让 Meta 转换为 <ESC>，同时仍能输入键盘上直接具有的 8 位字符，或用 <Compose> 或 <AltGr> 键输入它们。参见"输入类型"。

许多现代系统为许多语言提供了"原生输入法"，这些语言的字符并没有分配键盘按键。若 Emacs 在编译时带有对这些原生输入法的支持，你就可以激活这种输入法并输入它所支持的字符。如何激活和使用这些输入法取决于系统与具体的输入法，此处不再描述；请查阅系统文档。这里我们介绍一些控制原生输入法使用的 Emacs 功能。

在使用 GTK 工具库构建的 Emacs 中，变量 'x-gtk-use-native-input' 控制 Emacs 是否接收由 GTK 输入法产生的字符。若该值为 'nil'（默认值），Emacs 使用 X 输入法（XIM），否则使用 GTK 输入法。'useXIM' X 资源控制是否使用 XIM，'inputStyle' X 资源控制原生输入法生成的预览文本在 X 上的显示方式；参见"Emacs 的 X 资源表"。

在 MS-Windows 上，Emacs 支持由 IMM（Input Method Manager，输入法管理器）提供的原生输入法，但必要时可以关闭这一支持；参见"MS-Windows 上的键盘用法"。

- 你可以把键 'C-x 8' 用作组合字符前缀，用来输入非 ASCII 的 Latin-1 字符和其他可打印字符。'C-x 8' 适合用于插入（在迷你缓冲区以及其他缓冲区中均可）、搜索，以及其他任何允许键序列的场合。

'C-x 8' 通过加载 'iso-transl' 库来工作。该库一经加载，如果键盘上有 <Alt> 修饰键，它就起到与 'C-x 8' 相同的作用：把 <Alt> 与某个重音字符一起使用，以修饰紧随其后的字母。此外，如果键盘上有 Latin-1 死重音（dead accent）字符对应的按键，那么一旦 'iso-transl' 被加载，这些键也被定义为与后一个字符组合。

用 'C-x 8 C-h' 列出所有可用的 'C-x 8' 转换。

可用转换的集合可以借助 'M-x iso-transl-set-language' 命令，用某些特定语言中的字符加以扩充。目前支持的语言有：'French'、'German'、'Portuguese'、'Spanish' 和 'Esperanto'。详情参见 'iso-transl-language-alist'。

24.19 字符集

在 Emacs 中，"charset"是"character set"（字符集）的简称。Emacs 支持大多数流行的字符集（如 'ascii'、'iso-8859-1'、'cp1250'、'big5' 和 'unicode'），此外还支持一些它自己的字符集（如 'emacs'、'unicode-bmp' 和 'eight-bit'）。所有受支持的字符都属于一个或多个字符集。

Emacs 通常在字符集方面会自动做正确的事，因此你不必为它们操心。不过，有时了解字符集的一些底层细节会有所帮助。

一个例子是字体选择（参见"字体"）。每种语言环境（参见"语言环境"）都为各种字符集定义了优先级列表。在查找字体时，Emacs 起初会尝试找到能显示最高优先级字符集的字体。例如，在日语语言环境中，字符集 'japanese-jisx0208' 具有最高优先级，因此 Emacs 会尝试使用 'registry' 属性为 'JISX0208.1983-0' 的字体。

有两个命令可用于获取字符集的信息。命令 'M-x list-charset-chars' 提示输入字符集名称，并显示该字符集中的所有字符。命令 'M-x describe-character-set' 提示输入字符集名称，并显示该字符集的信息，包括它在 Emacs 内部的表示形式。

'M-x list-character-sets' 显示所有受支持字符集的列表。该列表给出字符集的名称以及用于识别各个字符集的附加信息；更多细节见日本信息处理学会 / 日本信息技术标准委员会（IPSJ/ITSCJ）（https://www.itscj.ipsj.or.jp/itscj_english/）维护的 ISO 国际编码字符集登记表（ISO-IR）（https://www.itscj.ipsj.or.jp/itscj_english/iso-ir/ISO-IR.pdf）。在该列表中，字符集分为两类：先列出"普通字符集"（normal charsets），随后列出"补充字符集"（supplementary charsets）。补充字符集是指用于定义另一个字符集（作为父集或子集）的字符集，或用于为旧版 Emacs 提供向后兼容性的字符集。

要查明缓冲区中某个字符属于哪个字符集，请把光标点放在它前面并输入 'C-u C-x ='（参见"国际字符集简介"）。

24.20 双向编辑

Emacs 支持编辑用阿拉伯文、波斯文和希伯来文等文字书写的文本，这些文字显示水平文本的自然顺序是从右到左。不过，嵌入这些文字中的数字和拉丁文本仍从左到右显示。在拉丁文文档中嵌入少量阿拉伯文或希伯来文文本也很常见，例如作为程序源文件中的注释和字符串。由于这些原因，使用这些文字的文本实际上是"双向的"：它由从左到右和从右到左的字符片段混合而成。

本节介绍 Emacs 为编辑双向文本提供的设施和选项。

Emacs 以所谓的"逻辑"（或"阅读"）顺序存储从右到左的文本和双向文本：你读到的第一个字符在缓冲区或字符串中的位置先于下一个字符。双向文本重排为"视觉"顺序发生在显示时。因此，字符位置不再随其在屏幕上的位置单调递增。Emacs 实现了 Unicode 标准附件 #9（Unicode Standard Annex #9）（https://unicode.org/reports/tr9/）中描述的 Unicode 双向算法（Unicode Bidirectional Algorithm，UBA），用于在显示时重排双向文本。它与 UBA 的唯一偏离在于：当文本方向与段落基准方向相反时如何显示续行，例如当一行很长的英文文本出现在从右到左的段落中时。

缓冲区局部变量 'bidi-display-reordering' 控制缓冲区中的文本是否在显示时被重排。若其值非 'nil'，Emacs 在显示时重排具有从右到左方向性的字符。默认值为 't'。

双向文本的每个段落都可以有自己的"基准方向"，即从右到左或从左到右。从左到右的段落中的文本在屏幕上从窗口左边界开始，到达右边界时被截断或续行。相反，从右到左的段落中的文本从右边界开始显示，在左边界处续行或截断。默认情况下，段落边界是空行，即完全由空白字符组成的行。要改变这一点，你可以自定义两个变量 'bidi-paragraph-start-re' 和 'bidi-paragraph-separate-re'，它们的值应当是正则表达式（字符串）；例如，要让单个换行符开始一个新段落，就把这两个变量都设为 '"^"'。这两个变量是缓冲区局部的（参见"局部变量"）。

Emacs 根据段落开头的文本动态确定每个段落的基准方向。不过，有时缓冲区可能需要为其段落强制指定某个基准方向。若变量 'bidi-paragraph-direction' 非 'nil'，就禁用基准方向的动态确定，转而强制缓冲区中的所有段落都具有其缓冲区局部值所指定的方向。该值可以是 'right-to-left' 或 'left-to-right'。任何其他值都被解释为 'nil'。

另一种办法是，通过在段落前面插入特殊的格式化字符来控制段落的基准方向。特殊字符 'RIGHT-TO-LEFT MARK'（即 RLM）强制其后的段落采用从右到左的方向，而 'LEFT-TO-RIGHT MARK'（即 LRM）则强制采用从左到右的方向。（你可以用 'C-x 8 <RET>' 插入这些字符。）在图形界面会话中，LRM 和 RLM 字符显示为非常细的空白字符；在文本终端上它们显示为空格。

由于字符在显示时被重排，按逻辑顺序或按连续缓冲区位置区间操作的 Emacs 命令可能会产生不寻常的效果。例如，命令 'C-f' 和 'C-b' 按逻辑顺序移动光标点，因此当光标点穿过被重排的双向文本时，光标有时会跳跃。类似地，如果覆盖一段连续字符位置的加亮区域跨越了被重排的文本，它看起来可能不连续。这是正常的，与其他支持双向文本的程序的行为类似。

绑定到方向键的光标移动命令，如 <LEFT> 和 'C-<RIGHT>'，对当前段落的基准方向敏感。在从左到右的段落中，绑定到 <RIGHT>（无论是否带修饰键）的命令在缓冲区文本中向_前_移动，但在从右到左的段落中它们反而向_后_移动。这反映了一个事实：在从右到左的段落中，在显示上向左移动时，缓冲区位置主要递增。

当你移出一个段落时，如果前一段或后一段的基准方向与你离开的段落不同，方向键的含义可能会改变。发生这种情况时，你需要按新的基准方向调整所按的方向键。

默认情况下，<LEFT> 和 <RIGHT> 按逻辑顺序移动，但如果 'visual-order-cursor-movement' 非 'nil'，这些命令会移动到当前屏幕位置上相应处于左侧或右侧的字符处，并按需移到下一屏幕行或上一屏幕行。注意，取决于周围的双向上下文，这可能使光标点移动很多个缓冲区位置。

双向文本有时使用特殊的格式化字符来影响文本在显示时的重排。上面提到的 LRM 和 RLM 字符就是两个这样的字符，但还有更多。默认情况下，它们在图形框架上显示为细空格字形，在文本模式框架上显示为普通空格。如果你希望留意这些特殊控制字符，以免它们对显示的影响让你感到意外，可以打开 'glyphless-display-mode'（参见"文本的显示"）。这个次模式会使这些格式化字符显示为小方框内的缩写，从而在显示中凸显出来，并使其作用更易于理解。

25 主模式与次模式

Emacs 包含许多“编辑模式”（editing modes），它们以有用的方式改变 Emacs 的基本行为。这些模式分为“主模式”（major modes）和“次模式”（minor modes）。

主模式为处理某种特定类型的文件（例如 C 源文件，参见“编辑程序”）或某种特定类型的非文件缓冲区（例如 shell 缓冲区，参见“在 Emacs 中运行 Shell 命令”）提供专门的功能。主模式彼此互斥；任何时刻每个缓冲区有且只有一个主模式。

次模式是可选的特性，你可以开启或关闭它们，它们不必与某种文件或缓冲区类型相关。例如，自动填充模式（Auto Fill mode）是一种次模式，在它开启时你键入 <SPC> 会在词之间断行（参见“自动填充模式”）。次模式彼此独立，也与当前选中的主模式无关。

- Minor Modes：每个次模式都是一项可以独立于其他模式开启的特性。
- Choosing Modes：访问文件时如何选择模式。

25.1 主模式

每个缓冲区都有一个主模式，它决定该缓冲区为当前缓冲区时 Emacs 的编辑行为。模式行通常以括号形式显示当前主模式的名称（参见“模式行”）。

最不专门化的主模式称为“基本模式”（Fundamental mode）。该模式没有针对模式的重新定义或变量设置，因此每条 Emacs 命令都以最通用的方式行事，每个用户选项变量都处于默认状态。

要编辑 Emacs 所知的某种特定类型的文本（如 Lisp 代码或英文文本），你通常使用更专门化的主模式，如 Lisp 模式或文本模式（Text mode）。大多数主模式归入三大类。第一类包含用于普通文本的模式，这些文本或是纯文本，或是带标记的文本，包括文本模式、HTML 模式、SGML 模式、TeX 模式和提纲模式（Outline mode）。第二类包含用于特定编程语言的模式，包括 Lisp 模式（有若干变体）、C 模式、Fortran 模式等。第三类由不直接与文件关联的主模式组成；它们用于 Emacs 为特定目的创建的缓冲区。例如，用于 Dired 所建缓冲区的 Dired 模式（参见“Dired 目录编辑器”）、用于 ‘C-x m’ 所建缓冲区的 Message 模式（参见“发送邮件”），以及用于与下级 shell 进程通信的缓冲区的 Shell 模式（参见“交互式子 shell”）。

通常，主模式由 Emacs 在你首次访问文件或创建缓冲区时自动设置（参见“选择文件模式”）。你可以用 ‘M-x’ 命令显式选择新的主模式。取模式名并加上 ‘-mode’，就得到选择该模式的命令名（例如 ‘M-x lisp-mode’ 进入 Lisp 模式）。由于每个缓冲区恰好只有一个主模式，无法“关闭”某个主模式；你必须切换到另一个主模式。

缓冲区局部变量 ‘major-mode’ 的值是一个符号，其名称与主模式命令相同（例如 ‘lisp-mode’）。该变量是自动设置的；你不应自行修改它。

‘major-mode’ 的默认值决定用于没有指定主模式的文件以及用 ‘C-x b’ 新建的缓冲区的主模式。通常，该默认值是符号 ‘fundamental-mode’，即指定基本模式。你可以通过定制界面改变这个默认值（参见“简易自定义界面”），或者在初始化文件中加入如下一行（参见“Emacs 初始化文件”）：

```
(setq-default major-mode 'text-mode)
```

若 ‘major-mode’ 的默认值为 ‘nil’，则主模式取自先前为当前的那个缓冲区。

专门化的主模式常常改变某些键的含义，以便做更适合该模式的事。例如，编程语言模式把 <TAB> 绑定为按该语言的规则缩进当前行（参见“缩进”）。通常被改变的键有 <TAB>、<DEL> 和 ‘C-j’。许多模式还定义自己的特殊命令，通常绑定到以 ‘C-c’ 为前缀键的键序列（参见“按键”）。主模式还可能改变用户选项和变量；例如，编程语言模式通常为变量 ‘comment-start’ 设置缓冲区局部值，该变量决定源代码注释如何界定（参见“操作注释”）。

要查看当前主模式的文档，包括其键绑定列表，键入 ‘C-h m’（‘describe-mode’）。参见“其他帮助命令”。

除基本模式之外，每个主模式都定义一个“模式钩子”（mode hook），即一个可定制的 Lisp 函数列表，每次在某个缓冲区中启用该模式时就会运行。关于钩子的更多信息，参见“钩子”。每个模式钩子以其主模式命名，例如 Fortran 模式有 ‘fortran-mode-hook’。此外，所有基于文本的主模式都运行 ‘text-mode-hook’，而许多编程语言模式(1)（包括随 Emacs 分发的全部这类模式）在运行各自的模式钩子之前会先运行 ‘prog-mode-hook’。钩子函数可以查看变量 ‘major-mode’ 的值，以了解实际进入的是哪个模式。

模式钩子常用于启用次模式（参见“次模式”）。例如，你可以把下面几行放入初始化文件，以便在所有基于文本的主模式中启用 Flyspell 次模式（参见“拼写检查与纠正”），并在 Emacs Lisp 模式中启用 ElDoc 次模式（参见“编程语言文档查询”）：

```
(add-hook 'text-mode-hook 'flyspell-mode)
(add-hook 'emacs-lisp-mode-hook 'eldoc-mode)
```

> 注：更准确地说，是指那些“派生”自 ‘prog-mode’ 的模式（参见《Emacs Lisp 参考手册》的“Derived Modes”）。

25.2 次模式

次模式是一种可选的编辑模式，它以某种定义明确的方式改变 Emacs 的行为。与主模式不同，任意数目的次模式都可以同时生效。有些次模式是“缓冲区局部的”（buffer-local），可以在某些缓冲区中开启（启用），而在其他缓冲区中关闭（禁用）。另一些次模式是“全局的”（global）：它们启用期间会影响你在 Emacs 会话中所做的一切，作用于所有缓冲区。大多数次模式默认禁用，但有少数默认启用。

大多数缓冲区局部次模式在启用时会在模式行中显示，位置紧接在主模式指示符之后。例如，模式行中的 ‘Fill’ 表示自动填充模式已启用。参见“模式行”。

与主模式一样，每个次模式都与一个“模式命令”相关联，其名称由模式名后接 ‘-mode’ 组成。例如，自动填充模式的模式命令是 ‘auto-fill-mode’。但与仅仅启用该模式的主模式命令不同，次模式的模式命令既可以启用它，也可以禁用它：

- 若你直接调用模式命令而不带前缀参数（通过 ‘M-x’，或把它绑定到某个键然后键入该键；参见“定制键绑定”），则该命令“切换”该次模式：原本关闭就开启，原本开启就关闭。
- 若带前缀参数调用模式命令，则该参数为零或负数时无条件关闭次模式；否则无条件开启次模式。
- 若通过 Lisp 调用模式命令，则省略参数或参数为 ‘nil’ 时无条件开启次模式。这使你很容易从某个主模式的模式钩子中开启次模式（参见“主模式”）。非 ‘nil’ 的参数按上述交互式前缀参数的方式处理。

大多数次模式还有一个“模式变量”（mode variable），其名称与模式命令相同。模式启用时它的值为非 ‘nil’，禁用时为 ‘nil’。一般来说，你不应试图通过在 Lisp 中直接改变模式变量的值来启用或禁用模式；而应运行模式命令。不过，通过定制界面设置模式变量（参见“简易自定义界面”）总能正确地启用或禁用模式，因为定制界面会自动为你运行模式命令。

下面列出一些缓冲区局部次模式：

- 缩写模式（Abbrev mode）根据预定义的缩写定义自动展开文本。参见“缩写”。
- 自动填充模式在你就输入时插入换行，以防各行变得过长。参见“填充文本”。
- 自动保存模式（Auto Save mode）定期把缓冲区内容保存到磁盘，以减少崩溃时可能丢失的工作量。参见“自动保存：灾难防护”。
- 电引号模式（Electric Quote mode）自动转换引号。例如，它把键入的 ``like this'' 重新引用为 ‘like this’。你可以控制它在何种文本中起作用，也可以在各个缓冲区中完全禁用它。参见“引号”。
- 富文本模式（Enriched mode）允许编辑和保存带格式的文本。参见“富文本”。
- Flyspell 模式自动高亮拼错的词。参见“拼写检查与纠正”。
- Font-Lock 模式自动高亮程序中的某些文本单元。它默认全局启用，但你可以在个别缓冲区中禁用它。参见“文本字体”。
- 显示行号模式（Display Line Numbers mode）是围绕 ‘display-line-numbers’ 的便捷封装，用 ‘display-line-numbers-type’ 的值来设置它。参见“显示的自定义”。
- 提纲次模式（Outline minor mode）提供与名为提纲模式（Outline mode）的主模式类似的功能。参见“大纲模式”。
- 覆盖模式（Overwrite mode）使普通打印字符替换已有文本，而不是把已有文本向右推。例如，若光标点位于 ‘FOOBAR’ 中 ‘B’ 的前面，那么在覆盖模式下键入 ‘G’ 会把它变成 ‘FOOGAR’，而不是像通常那样产生 ‘FOOGBAR’。在覆盖模式下，命令 ‘C-q’ 插入下一个字符，无论它是什么，即使是数字——这给你提供了一种插入字符而非替换已有字符的方法。该模式命令 ‘overwrite-mode’ 绑定在 <Insert> 键上。
- 二进制覆盖模式（Binary Overwrite mode）是覆盖模式的一个变体，用于编辑二进制文件；它把换行符和制表符当作其他字符一样对待，使它们可以覆盖其他字符，也可以被其他字符覆盖。在二进制覆盖模式下，‘C-q’ 之后的数字照常指定八进制字符编码。
- 视觉行模式（Visual Line mode）执行自动换行，使长行在词的边界处折行。参见“可视行模式”。

下面是一些有用的全局次模式：

- 列号模式（Column Number mode）在模式行中显示当前列号。参见“模式行”。
- 删除选区模式（Delete Selection mode）使文本插入先删除区域中的文本（如果区域处于活动状态）。参见“对区域进行操作”。
- Icomplete 模式在你处于迷你缓冲区且补全处于活动状态时显示可用补全的提示。参见“快速迷你缓冲区选择”。
- 行号模式（Line Number mode）在模式行中显示当前行号。它默认启用。参见“模式行”。
- 菜单栏模式（Menu Bar mode）为每个框架提供菜单栏。它默认启用。参见“菜单栏”。

- 滚动条模式（Scroll Bar mode）为每个窗口提供滚动条。它默认启用，但滚动条只在图形终端上显示。参见“滚动条”。
- 工具栏模式（Tool Bar mode）为每个框架提供工具栏。它默认启用，但工具栏只在图形终端上显示。参见“工具栏”。
- 窗口工具栏模式（Window Tool Bar mode）为窗口提供工具栏。参见“窗口工具栏”。
- 标签栏模式（Tab Bar mode）为每个框架提供标签栏。参见“标签栏”。
- 标签行模式（Tab Line mode）为每个窗口提供标签行。参见“窗口标签行”。
- 临时标记模式（Transient Mark mode）高亮区域，并使许多 Emacs 命令在标记处于活动状态时对区域操作。它默认启用。参见“标记与区域”。

25.3 选择文件模式

访问文件时，Emacs 会自动选择主模式。通常它根据文件名来选择——例如，名称以 ‘.c’ 结尾的文件通常用 C 模式编辑——但有时它根据文件中的特殊文本来选择主模式。这种特殊文本也可用于启用缓冲区局部次模式。

具体过程如下：

首先，Emacs 检查文件中是否含有文件局部模式变量（file-local mode variables）。参见“文件中的局部变量”。若存在指定主模式的文件局部变量，则 Emacs 使用该主模式，忽略所有其他判据。用文件局部变量指定主模式有几种方法；最简单的是把模式名放在第一个非空行中，并在其前后各加 ‘-*-’。该行还可以出现其他文本。例如，

```
; -*-Lisp-*-
```

告诉 Emacs 使用 Lisp 模式。注意这里用分号使 Lisp 把该行当作注释。你也可以等价地写成

```
; -*- mode: Lisp;-*-
```

你也可以用 ‘eval’ 规范，通过文件局部变量指定缓冲区局部次模式。例如，下面这个第一非空行使缓冲区进入 Lisp 模式并启用自动填充模式：

```
; -*- mode: Lisp; eval: (auto-fill-mode 1); -*-
```

但请注意，这样启用次模式通常并不合适，因为大多数次模式代表的是个人偏好。如果你本人想对某种特定文件类型使用某个次模式，最好通过主模式的模式钩子来启用该次模式（参见“主模式”）。

其次，Emacs 检查文件的扩展名是否匹配任何目录局部的 ‘auto-mode-alist’ 中的条目。这些条目通过 ‘.dir-locals.el’ 设施找到（参见“按目录的局部变量”）。

第三，若没有指定主模式的文件变量，Emacs 检查文件内容是否以 ‘#!’ 开头。若是，则表明该文件可以充当可执行的 shell 命令，其工作方式是运行在该文件第一行中指定的解释器（文件的其余部分用作该解释器的输入）。因此，Emacs 尝试用解释器名来选择模式。例如，以 ‘#!/usr/bin/perl’ 开头的文件会在 Perl 模式下打开。变量 ‘interpreter-mode-alist’ 指定解释器程序名与主模式之间的对应关系。

当第一行以‘#!’开头时，你通常无法在第一行使用‘--’特性，*因为系统在运行解释器时会搞不清楚。因此，在这类文件中，Emacs 除了第一行之外还会在第二行查找‘-’*。以魔法字符串“\"”开头、用于指定 troff 预处理器列表的 man 手册页也是如此。

第四，Emacs 尝试根据变量‘magic-mode-alist’，查看缓冲区开头的文本来确定主模式。默认情况下，该变量为‘nil’（一个空列表），因此 Emacs 跳过这一步；不过你可以在初始化文件中定制它（参见“Emacs 初始化文件”）。它的值应该是一个元素列表，每个元素形如

```
(REGEXP . MODE-FUNCTION)
```

其中 REGEXP 是正则表达式（参见“正则表达式的语法”），MODE-FUNCTION 是主模式命令。若文件开头的文本匹配 REGEXP，Emacs 就选择 MODE-FUNCTION 所指定的主模式。

另外，‘magic-mode-alist’的元素也可以具有如下形式

```
(MATCH-FUNCTION . MODE-FUNCTION)
```

其中 MATCH-FUNCTION 是一个在缓冲区开头被调用的 Lisp 函数；若该函数返回非‘nil’，Emacs 就用 MODE-FUNCTION 设置主模式。

第五——若 Emacs 仍未找到合适的主模式——它查看文件名。文件名与主模式之间的对应关系由变量‘auto-mode-alist’控制。它的值是一个列表，其中每个元素具有如下形式

```
(REGEXP . MODE-FUNCTION)
```

或如下形式

```
(REGEXP MODE-FUNCTION FLAG)
```

例如，该列表中通常有一个元素的形如‘("\\.c\\'" . c-mode)’，它负责为名称以‘.c’结尾的文件选择 C 模式。（注意，Lisp 语法中需要‘\\’才能在字符串中包含一个‘\’，必须用它来抑制‘.’在正则表达式中的特殊含义。）

若元素的形式为‘(REGEXP MODE-FUNCTION FLAG)’且 FLAG 为非‘nil’，则在调用 MODE-FUNCTION（若它非‘nil’）之后，Emacs 丢弃匹配 REGEXP 的后缀，并在列表中再次搜索另一处匹配。这种“递归剥离扩展名”用于具有多个扩展名的文件，其“外层”扩展名掩盖了真正指定正确模式的“内层”扩展名。例如，备份文件以及扩展名为‘.gpg’的 GPG 加密文件就使用这一特性。

在 GNU/Linux 以及其他文件名区分大小写的系统上，Emacs 在‘auto-mode-alist’中执行区分大小写的搜索；若该搜索失败，它再对该 alist 执行一次不区分大小写的搜索。要抑制第二次搜索，把变量‘auto-mode-case-fold’改为‘nil’。在文件名不区分大小写的系统（如 Microsoft Windows）上，Emacs 在‘auto-mode-alist’中只执行一次不区分大小写的搜索。

最后，若 Emacs 仍然没有找到可用的主模式，它把缓冲区开头的文本与变量‘magic-fallback-mode-alist’相比较。该变量的工作方式与上述‘magic-mode-alist’类似，只是它仅在‘auto-mode-alist’被查阅之后才被查阅。默认情况下，‘magic-fallback-mode-alist’包含检查图像文件、HTML/XML/SGML 文件、PostScript 文件和 Unix 风格 Conf 文件的形式。

一旦找到主模式，Emacs 会做最后一次检查，看该模式是否被 ‘major-mode-remap-alist’ “重映射”过，若是则改用重映射后的模式。这用于同一文件类型可以使用若干不同主模式的情形，以便你指定自己偏好的模式。注意，这种重映射会影响上述所有方法找到的主模式，因此，例如文件第一行所指定的模式未必就是访问该文件的缓冲区中实际启用的模式。（这种重映射也会影响 ‘revert-buffer’，参见“还原缓冲区”。）当同一文件类型有多个可用模式时，你可以通过定制 ‘major-mode-remap-alist’ 告诉 Emacs 你的主模式偏好。例如，把下面这行放入你的 ‘~/.emacs’ 初始化文件（参见“Emacs 初始化文件”）

```
(add-to-list 'major-mode-remap-alist '(c-mode . c-ts-mode))
```

以强制 Emacs 在 ‘auto-mode-alist’ 或文件局部变量指定 ‘c-mode’ 时调用 ‘c-ts-mode’。反之，

```
(add-to-list 'major-mode-remap-alist '(c-mode))
```

将强制 Emacs 永不把 ‘c-mode’ 重映射为任何其他模式。

‘major-mode-remap-alist’ 的默认值为 ‘nil’，因此不会发生重映射。不过，加载某些 Lisp 包或特性可能引入模式重映射，因为 Emacs 假定加载它们意味着用户偏好使用替代模式。因此，为了获得可预期的行为，我们建议你总是定制 ‘major-mode-remap-alist’ 来表达你确定的偏好，因为该变量会覆盖 Emacs 在内部可能决定进行的任何重映射。

作为基于 tree-sitter 库启用主模式的便捷特性（参见《Emacs Lisp 参考手册》的“Parsing Program Source”），你可以定制用户选项 ‘treesit-enabled-modes’ 来选择性地启用或禁用基于 tree-sitter 的模式：若其值为 ‘t’，则启用所有可用的基于 tree-sitter 的模式；若其值为模式名列表，则只启用那些模式。定制该选项会把相应的映射加入 ‘major-mode-remap-alist’，例如从 ‘c-mode’ 重映射到 ‘c-ts-mode’（如果你启用后者）。默认情况下，该选项的值为 ‘nil’，因此不启用任何基于 tree-sitter 的模式。

启用基于 tree-sitter 的模式意味着，访问相应编程语言的文件时会自动启用该模式，而不启用同一语言的任何非 tree-sitter 模式。例如，若你启用 ‘c-ts-mode’，访问 C 源文件将启用 ‘c-ts-mode’ 而非 ‘c-mode’。

若你改变了某个缓冲区的主模式，可以通过键入 ‘M-x normal-mode’ 回到 Emacs 本会自动选择的主模式。这也是 ‘find-file’ 用来选择主模式的同一个函数。若该缓冲区正在访问文件，此命令还会处理文件的 *‘--’ 行和文件局部变量列表（若有）。参见“文件中的局部变量”。若缓冲区不访问文件，该命令只处理 ‘–’* 行和文件局部变量列表中可能存在的的主模式规范。‘M-x normal-mode’ 会考虑模式重映射，因此，若你在 Emacs 选定该缓冲区的主模式之后定制了 ‘major-mode-remap-alist’，‘normal-mode’ 可能启用与 Emacs 原先选择不同的模式。

命令 ‘C-x C-w’ 和 ‘set-visited-file-name’ 在新文件名暗示某种模式时会切换到新的主模式（参见“保存文件”）。（若缓冲区未访问文件，‘C-x C-s’ 也会如此。）不过，若缓冲区内容指定了主模式，或某些特殊主模式不允许改变模式，则不会发生这种切换。你可以把 ‘change-major-mode-with-file-name’ 设为 ‘nil’ 来关闭这一模式切换特性。

26 缩进

“缩进”（indentation）指在文本行开头插入或调整“空白字符”（空格和/或制表符）。本章介绍 Text 模式及相关模式与编程语言模式所共有的缩进命令和选项。关于编程模式中缩进的更多文档，参见“程序的缩进”。

执行缩进最简单的方法是 <TAB> 键。在大多数主模式中，它运行命令 ‘indent-for-tab-command’。（在 C 及相关模式中，<TAB> 运行命令 ‘c-indent-line-or-region’，其行为类似，参见“C 缩进命令”）。

<TAB> 以适合模式的方式插入空白，或缩进当前行（‘indent-for-tab-command’）。若区域处于活动状态，则缩进区域内的所有行。

<TAB> 的确切行为取决于主模式。在 Text 模式及相关主模式中，<TAB> 通常插入空格和制表符的某种组合，以把光标点推进到下一个制表位（参见“制表位”）。为此，前一行的第一个非空白字符的位置被当作一个额外的制表位，因此你可以用 <TAB> 让光标点与上一行对齐。若区域处于活动状态（参见“对区域进行操作”），<TAB> 的表现比较特殊：它会缩进区域内的每一行，使其第一个非空白字符与上一行对齐。

在编程模式中，<TAB> 根据前几行代码的合理方式来缩进当前行代码。若区域处于活动状态，则区域中的所有行都按此方式缩进。若光标点最初位于当前行的缩进部分之内，则它会被重新定位到该行的第一个非空白字符处。

若你只想在缓冲区中插入一个制表符，请键入 ‘C-q <TAB>’（参见“插入文本”）。

26.1 缩进命令

除了 ‘<TAB>’（‘indent-for-tab-command’）命令之外，Emacs 还提供了多种以其他方式执行缩进的命令。

‘C-M-o’ 在光标点处拆分当前行（‘split-line’）。光标点之后的文本成为新的一行，并缩进到光标点所在的列。此命令会先让光标点前移越过所有空格和制表符。之后，光标点位于所插入的换行符之前。

‘M-m’ 移动到当前行的第一个非空白字符处（可向前或向后）（‘back-to-indentation’）。若该行没有非空白字符，则移动到行尾。

‘M-i’ 缩进光标点处的空白，直到下一个制表位（‘tab-to-tab-stop’）。参见“制表位”。

‘M-x indent-relative’ 在光标点处插入空白，直到光标点与上一行（实际上是最后一个非空行）的第一个非空白字符对齐。若光标点已经比该位置更靠右，则改为运行 ‘tab-to-tab-stop’——除非带数值参数调用，此时什么也不做。

‘M-^’ 合并上一行和当前行（‘delete-indentation’）。它把当前行开头的缩进连同行边界替换为一个空格，从而干净地把两行连接起来。

作为一种特殊情况（对 Lisp 代码很有用），若要连接的两个字符是相邻的开括号和闭括号，或者连接处紧随另一个换行符，则省略这个空格。

若存在填充前缀（fill prefix），且它出现在被删除的换行符之后，则 ‘M-^’ 会删除该填充前缀。参见“填充前缀”。

带前缀参数时，把当前行与下一行连接。若区域处于活动状态且未给出前缀参数，则改为连接区域中的所有行。

‘C-M-\’ 缩进区域中的所有行，就像你在每行开头键入了 ‘<TAB>’ 一样（‘indent-region’）。

若提供了数值参数，则把区域中的每一行缩进到该列号。

‘C-x <TAB>’ 缩进区域中开始的所有行，把受影响的行作为一个整体刚性移动（‘indent-rigidly’）。

若不带参数调用，此命令会激活一个临时模式，以交互方式调整受影响行的缩进。该临时模式活动期间，键入 ‘<LEFT>’ 或 ‘<RIGHT>’ 会分别向左或向右缩进一个空格。你也可以键入 ‘S-<LEFT>’ 或 ‘S-<RIGHT>’ 向左或向右缩进到下一个制表位（参见“制表位”）。键入任何其他键都会停用该临时模式，随后该键按正常方式生效。

若带前缀参数 N 调用，此命令把各行向前缩进 N 个空格（不启用临时模式）。N 为负值时向后缩进，因此可以用一个很大的负数参数去掉区域中各行所有的缩进，如下所示：

```
C-u -999 C-x <TAB>
```

26.2 制表位

Emacs 把某些列号定义为“制表位”（tab stop）。当 <TAB> 在 Text 模式及相关模式中插入空白时，以及 ‘M-i’ 之类的命令，都以这些位置作为停靠点（参见“缩进”；参见“缩进命令”）。变量 ‘tab-stop-list’ 控制这些位置。默认值为 ‘nil’，表示每隔 8 列一个制表位。该值也可以是一个从 0 开始计数的列号列表（按递增顺序），指明在哪些列放置制表位。Emacs 会通过重复最后两个元素之差，把这个列表无限延伸下去。

除了直接定制变量 ‘tab-stop-list’ 之外，查看和设置制表位的便捷方式是命令 ‘M-x edit-tab-stops’。这会切换到一个包含制表位设置说明的缓冲区，其内容如下所示：

```
        :       :       :       :       :       :
0         1         2         3         4
0123456789012345678901234567890123456789012345678
To install changes, type C-c C-c
```

第一行在每个制表位处都有一个冒号。后两行上的数字只是用来标明冒号所在的位置。若 ‘tab-stop-list’ 的值为 ‘nil’（默认即如此），则最初不显示任何冒号。

你可以编辑这个缓冲区，把冒号放在期望的列上，从而指定不同的制表位。该缓冲区使用 Overwrite 模式（参见“次模式”）。请记住，Emacs 会通过重复你明确放置的最后两个制表位之差，无限延伸制表位列表。完成后，键入 ‘C-c C-c’ 使新的制表位生效。通常，新的制表位设置会应用于所有缓冲区。然而，若你已让 ‘tab-stop-list’ 变量在你调用 ‘M-x edit-tab-stops’ 的缓冲区中成为局部变量（参见“局部变量”），则新的制表位设置只应用于该缓冲区。要为将来的 Emacs 会话保存制表位设置，请使用 Customize 界面保存 ‘tab-stop-list’ 的值（参见“简易自定义界面”）。

请注意，本节讨论的制表位与制表符在缓冲区中如何显示毫无关系。制表符总是显示为延伸到下一个“显示制表位”（display tab stop）的空白。参见“文本的显示”。

26.3 制表符与空格

通常，缩进命令会插入（或删除）尽可能短的制表符和空格字符序列，以对齐到期望的列。制表符显示为延伸到下一个“显示制表位”的一段空白。默认情况下，每隔‘tab-width’列（默认值为 8）有一个显示制表位。参见“文本的显示”。

若你愿意，可以只用空格来进行所有缩进。为此，请把缓冲区局部变量‘indent-tabs-mode’设为‘nil’。关于设置缓冲区局部变量的信息，参见“局部变量”。但请注意，‘C-q <TAB>’总是插入一个制表符，无论‘indent-tabs-mode’的值如何。

把‘indent-tabs-mode’设为‘nil’的一个原因是，并非所有编辑器都以相同方式显示制表符。Emacs 用户也可能定制了不同的‘tab-width’值。只用空格，就能确保你的文件看起来始终一致。若你只关心它在 Emacs 内的外观，解决这个问题的另一种方法是在文件局部变量中设置‘tab-width’变量（参见“文件中的局部变量”）。

还有一些命令可以在制表符与空格之间转换，并始终保留所有非空白文本所在的列。‘M-x tabify’扫描区域中的空格序列，在不改变缩进的前提下，尽可能把至少两个空格的序列转换为制表符。‘M-x untabify’把区域中的所有制表符改为适当数量的空格。

26.4 缩进的便利功能

变量‘tab-always-indent’微调 <TAB>（‘indent-for-tab-command’）命令的行为。默认值‘t’给出参见“缩进”中描述的行为。若把该值改为符号‘complete’，则 <TAB> 先尝试缩进当前行；若该行已经缩进过，则尝试补全光标点处的文本（参见“符号名补全”）。若该值为‘nil’，则只有当光标点位于左边界或该行的缩进部分之内时，<TAB> 才缩进当前行；否则它插入一个制表符。

若‘tab-always-indent’为‘complete’，则还可以通过‘tab-first-completion’变量进一步定制是展开还是缩进。例如，若该变量为‘eol’，则只有当光标点位于行尾时才补全。更多细节参见《Emacs Lisp 参考手册》的“Mode-Specific Indent”。

Electric Indent 模式是一个全局次模式，它在你每次键入 <RET> 之后自动缩进该行。该模式默认启用。要切换这个次模式，请键入‘M-x electric-indent-mode’。要在单个缓冲区中切换该模式，请使用‘M-x electric-indent-local-mode’。

26.5 代码对齐

“对齐”（alignment）指调整区域中一系列行的空白，使得在所有行中某些部分都从同一列开始的过程。这通常是为了增强某段文本或代码的可读性。经典的例子是对齐类 C 编程语言中的一系列赋值：

```
int a = 1;
short foo = 2;
double blah = 4;
```

通常会将其对齐为：

```
int    a    = 1;
short  foo  = 2;
double blah = 4;
```

你可以使用命令‘M-x align’来对齐当前区域中的行。此命令了解许多标记语言和编程语言中常见的对齐模式。它把这些模式编码为一组“对齐规则”（alignment rule），用以说明在不同上下文中如何对齐不同种类的文本。

用户选项‘align-rules-list’说明‘M-x align’应参考哪些对齐规则。该选项的值是一个列表，其元素描述对齐规则。每个元素是一个 cons 单元‘(TITLE . ATTRIBUTES)’：TITLE 是作为符号的对齐规则名，ATTRIBUTES 是规则属性列表，这些属性定义该规则何时适用以及如何划分和对齐各行。每个规则属性是一个 cons 单元‘(ATTRIBUTE . VALUE)’：ATTRIBUTE 是属性名，VALUE 是它的值。唯一必需的属性是‘regexp’，其值是一个正则表达式，其中的子表达式匹配每行中‘M-x align’应扩展或收缩空白的位置（参见“正则表达式中的反斜杠”）。关于可能的各种对齐规则属性的完整说明，参见‘align-rules-list’的文档字符串（‘C-h v align-rules-list <RET>’）。默认情况下，该选项被设置为一个很长的对齐规则列表，涵盖 Emacs 所支持的许多语言。默认规则使用‘modes’规则属性来指定‘M-x align’应在哪些主模式中应用它们。主模式也可以通过把缓冲区局部变量‘align-mode-rules-list’设为非‘nil’的对齐规则列表来覆盖‘align-rules-list’。当‘align-mode-rules-list’非‘nil’时，‘M-x align’参考它而不是‘align-rules-list’。

除了对齐规则之外，‘M-x align’还使用另一类称为“排除规则”（exclusion rule）的规则。排除规则说明区域中哪些部分‘M-x align’不应对齐，而应保持原样。用户选项‘align-exclude-rules-list’指定这些排除规则。与‘align-rules-list’类似，‘align-exclude-rules-list’的值也是一个描述排除规则的 cons 单元列表。默认情况下，‘align-exclude-rules-list’包含的规则会在 Lisp、C 及其他语言中排除对引用字符串和注释的对齐。除了‘align-exclude-rules-list’中的默认排除规则之外，主模式还可以通过把‘align-mode-exclude-rules-list’设为非‘nil’的规则列表来定义专门的排除规则；正如‘align-mode-rules-list’覆盖‘align-rules-list’那样，这也会覆盖‘align-exclude-rules-list’。

‘M-x align’把区域切分成一系列“段”（section），通常是若干非空行的序列，并按照所有匹配的对齐规则，通过扩展或收缩空白段来对齐每个段。‘M-x align’会一致地对齐单个段内的所有行，但可能以不同方式对齐区域中的不同段。用户选项‘align-region-separate’指定‘M-x align’如何把区域分隔成段。该选项可以是符号‘entire’、‘group’之一，也可以是一个正则表达式。若‘align-region-separate’为‘entire’，Emacs 把整个区域作为单个段来对齐。若该选项为‘group’，Emacs 把区域中每一组连续的非空行作为单独的段来对齐。若‘align-region-separate’是正则表达式，‘M-x align’会扫描区域中与该正则表达式匹配的位置，并把它们当作段分隔符。默认情况下，‘align-region-separate’被设置为一个正则表达式，匹配空行以及只含空白和一个花括号（‘{’或‘}’）的行。对于正则表达式不够精确的特殊情况，你也可以把‘align-region-separate’设为一个函数，由它说明如何把区域分隔成对齐段。更多细节参见‘align-region-separate’的文档字符串。特定的对齐规则可以通过指定‘separate’规则属性来覆盖‘align-region-separate’的值，并定义自己的段分隔符。

若带前缀参数（‘C-u’）调用‘M-x align’，它会启用更多对齐规则，这些规则常常有用，但有时可能过于侵入。例如，在含下列形式的 Lisp 缓冲区中：

```
(set-face-attribute 'mode-line-inactive nil
                    :box nil
                    :background nil
                    :underline "black")
```

键入（‘C-u M-x align’）会得到：

```
(set-face-attribute 'mode-line-inactive nil
                    :box                nil
                    :background         nil
                    :underline          "black")
```

大多数情况下，你应当先不带前缀参数试用 ‘M-x align’；若结果不正确，可以用 ‘C-/’ 撤销，然后用 ‘C-u M-x align’ 再试一次。

你可以使用命令 ‘M-x align-highlight-rule’ 在当前区域中可视化某条特定对齐规则或排除规则的效果。此命令会提示你输入规则的标题，并高亮该规则在区域中影响的部分。对于对齐规则，此命令高亮 ‘M-x align’ 会扩展或收缩的空白；对于排除规则，此命令高亮 ‘M-x align’ 会排除在对齐之外的部分。要移除该命令创建的高亮，请键入 ‘M-x align-unhighlight-rule’。

命令 ‘M-x align-current’ 与 ‘M-x align’ 类似，不同之处在于它只处理包含光标点的那个对齐段，而不顾当前区域。此命令根据 ‘align-region-separate’ 定义的段分隔符确定当前段的边界。‘M-x align-entire’ 是 ‘M-x align’ 的另一种变体，它忽略 ‘align-region-separate’，把整个区域作为单个对齐段一致地对齐。若你把 ‘align-region-separate’ 设为 ‘entire’，则 ‘M-x align’ 默认就像 ‘M-x align-entire’ 一样工作。为说明把整个区域作为单个对齐段对齐的效果，请考虑下列代码：

```
one = 1;
foobarbaz = 2;
```

```
spam = 3;
emacs = 4;
```

当区域覆盖所有这些行时，键入 ‘M-x align’ 会得到：

```
one       = 1;
foobarbaz = 2;
```

```
spam  = 3;
emacs = 4;
```

另一方面，‘M-x align-entire’ 把所有行作为单个段来对齐，因此 ‘=’ 在所有行中都出现在同一列：

```
one       = 1;
foobarbaz = 2;
```

```
spam      = 3;
emacs     = 4;
```

命令 ‘M-x align-regexp’ 让你用临时定义的对齐规则来对齐当前区域，而不使用 ‘align-rules-list’ 中的预定义规则。‘M-x align-regexp’ 会提示你输入一个正则表达式，并把该表达式用作一条临时对齐规则的 ‘regexp’ 属性，此命令用这条规则来对齐当前区域。默认情况下，该命令调整与所指定正则表达式的第一个子表达式匹配的空白。若带前缀参数调用 ‘M-x align-regexp’，它还会提示你指定要使用的子表达式，并让你指定用作填充的空白量，以及是否对每行中该正则表达式的所有匹配重复应用该规则。关于正则表达式及其子表达式的更多信息，参见“正则表达式中的反斜杠”。

若用户选项 'align-indent-before-aligning' 非 'nil'，Emacs 会先缩进区域，然后再用 'M-x align' 对齐它。参见"缩进"。默认情况下，'align-indent-before-aligning' 被设为 'nil'。

用户选项 'align-to-tab-stop' 说明对齐的部分是否应从制表位开始（参见"制表位"）。若该选项为 'nil'，'M-x align' 只使用刚好够对齐的空白，而不考虑制表位。若这是一个非 'nil' 的符号，'M-x align' 会检查该符号的值；若该值非 'nil'，则 'M-x align' 对齐到制表位。默认情况下，该选项被设为 'indent-tabs-mode'，因此在使用制表符进行缩进的缓冲区中，对齐会遵循制表位。参见"制表符与空格"。

用户选项 'align-default-spacing' 指定 'M-x align' 及其相关命令在对齐每行时，用作各行不同部分之间填充的默认空白量。当 'align-to-tab-stop' 为 'nil' 时，'align-default-spacing' 的值是要用作填充的空格数；当 'align-to-tab-stop' 非 'nil' 时，'align-default-spacing' 的值则代表要用作填充的制表位数。每条对齐规则都可以用 'spacing' 属性规则覆盖 'align-default-spacing' 指定的默认值。

27 人类语言命令

本章介绍对“文本”进行操作的 Emacs 命令，这里的“文本”指人类语言中的字符序列（而不是，比方说，计算机编程语言的字符序列）。这些命令的工作方式会顾及人类语言的句法和文体惯例：涉及单词、句子、段落和大小写的惯例。另有用于“填充”（filling）的命令，指重新排列段落中的各行，使它们的长度大致相等。这些命令虽然主要面向文本编辑，但在编辑程序时也常常很有用。

Emacs 有若干用于编辑人类语言文本的主模式。如果文件包含普通文本，请使用 Text 模式，它会针对文本的句法惯例对 Emacs 做小幅定制。Outline 模式提供专门命令来操作具有大纲结构的文本。参见“大纲模式”。

Org 模式扩展了 Outline 模式，把 Emacs 变成功能完备的组织工具：你可以管理 TODO 列表、保存笔记，并以多种格式发布它们。参见《Org 手册》

Emacs 还有其他主模式，用于处理内嵌命令的文本，例如 TeX 和 LaTeX（参见“TeX 模式”）；HTML 和 SGML（参见“SGML 与 HTML 模式”）；XML（参见《nXML 手册》）；以及 Groff 和 Nroff（参见“Nroff 模式”）。

如果你需要编辑由文本字符拼成的 ASCII 字符画，请使用 Picture 模式，这是一个专门编辑此类图片的主模式。参见“编辑图片”。

写作时，自动输入功能可能很有用。参见《Autotype 手册》

27.1 单词

Emacs 定义了几个用于在单词间移动或对单词进行操作的命令：

- ‘M-f’ 向前移动一个单词（‘forward-word’）。
- ‘M-b’ 向后移动一个单词（‘backward-word’）。
- ‘M-d’ 删除到单词末尾（‘kill-word’）。
- ‘M-<DEL>’ 删除回单词开头（‘backward-kill-word’）。
- ‘M-@’ 在下一个单词末尾设置标记（‘mark-word’）。
- ‘M-t’ 调换两个单词，或把一个单词拖过其他单词（‘transpose-words’）。

注意这些键如何构成一个系列，与基于字符的 ‘C-f’、‘C-b’、‘C-d’、<DEL> 和 ‘C-t’ 相对应。‘M-@’ 与 ‘C-@’ 同源，后者是 ‘C-<SPC>’ 的别名。

‘M-f’（‘forward-word’）和 ‘M-b’（‘backward-word’）命令在单词之间前后移动。这些基于 <Meta> 的键序列类似于在单个字符间移动的 ‘C-f’ 和 ‘C-b’。这种类比也延伸到数值参数，它们用作重复次数。带负参数的 ‘M-f’ 向后移动，带负参数的 ‘M-b’ 向前移动。向前移动停在单词最后一个字母之后，向后移动停在第一个字母之前。

‘M-d’（‘kill-word’）删除光标点之后的单词。准确地说，它删除从光标点到 ‘M-f’ 将会移到的位置之间的所有内容。因此，如果光标点位于单词中间，‘M-d’ 只删除光标点之后的部分。如果光标点和下一个单词之间有标点，标点会随该单词一起被删除。（如果你只想删除下一个单词而不删除它前面的标点，可以直接用 ‘M-f’ 到达末尾，再用 ‘M-<DEL>’ 反向删除该单词。）‘M-d’ 接受参数的方式与 ‘M-f’ 相同。

‘M-<DEL>’（‘backward-kill-word’）删除光标点之前的单词。它删除从光标点回到 ‘M-b’ 将会移到的位置之间的所有内容。例如，如果光标点在 ‘FOO, BAR’ 中空格的后面，它删除 ‘FOO, ’。如果你只想删除 ‘FOO’ 而不删除逗号和空格，请用 ‘M-b M-d’ 代替 ‘M-<DEL>’。

‘M-t’（‘transpose-words’）把光标点之前或包含光标点的单词与随后的单词交换。单词之间的分隔符字符不移动。例如，‘FOO, BAR’ 调换成 ‘BAR, FOO’，而不是 ‘BAR FOO,’。关于调换的更多信息，参见“调换文本”。

若要用作用于区域的命令来操作单词，请使用 ‘M-@’（‘mark-word’）命令。该命令把标记设置在 ‘M-f’ 将会移到的位置。关于该命令的更多信息，参见“标记文本对象的命令”。

单词命令对单词边界的理解由语法表控制。例如，可以把任何字符声明为单词分隔符。参见《Emacs Lisp 参考手册》的“Syntax Tables”。

此外，关于统计并报告区域或缓冲区中单词数的 ‘M-=’（‘count-words-region’）和 ‘M-x count-words’ 命令，参见“光标位置信息”。

27.2 句子

用于操作句子和段落的 Emacs 命令大多在 Meta 键上，与处理单词的命令类似。

- ‘M-a’ 向后移到句首（‘backward-sentence’）。
- ‘M-e’ 向前移到句末（‘forward-sentence’）。
- ‘M-k’ 向前删除到句末（‘kill-sentence’）。
- ‘C-x <DEL>’ 向后删除到句首（‘backward-kill-sentence’）。

‘M-a’（‘backward-sentence’）和 ‘M-e’（‘forward-sentence’）分别移动到当前句子的开头和结尾。它们的绑定被选得与 ‘C-a’ 和 ‘C-e’ 相似，后者移动到行首和行尾。与它们不同的是，重复执行时 ‘M-a’ 和 ‘M-e’ 会越过一个又一个句子。

向后越过一个句子会把光标点放在该句子第一个字符之前；向前移动则把光标点放在结束该句的标点之后。两者都不会越过句子边界处的空白。

正如 ‘C-a’ 和 ‘C-e’ 有配套的删除命令 ‘C-k’，‘M-a’ 和 ‘M-e’ 也有相应的删除命令：‘M-k’（‘kill-sentence’）从光标点删除到句末。带正数值参数 N 时，它删除后面 N 个句子；带负参数 −N 时，它向后删除到前面第 N 个句子的开头。

‘C-x <DEL>’（‘backward-kill-sentence’）向后删除到句首。

句子命令假定你遵循美国打字员的习惯：在句末放两个空格。也就是说，句子结束于出现 ‘.’、‘?’ 或 ‘!’ 且其后跟行尾或两个空格的地方，中间允许出现任意数量的 ‘)’、‘]’、‘'’ 或 ‘"’ 字符。段落开始或结束之处，句子也随之开始或结束。遵循这一惯例很有用，因为这样 Emacs 的句子命令就能区分结束句子的句点和表示缩写的句点。

如果你希望在句子之间只用单个空格，可以把变量 ‘sentence-end-double-space’ 设为 ‘nil’，让句子命令在单个空格处停下。但这样做有个缺点：无法区分结束句子的句点和表示缩写的句点。因此，为了便捷而可靠的编辑，我们建议你遵循双空格惯例。变量 ‘sentence-end-double-space’ 也会影响填充（参见“显式填充命令”）。

变量 'sentence-end' 控制如何识别句子的结尾。若不为 'nil'，其值应是一个正则表达式，用于匹配句子的最后几个字符以及句子后面的空白（参见"正则表达式的语法"）。如果值为 'nil'（默认），则 Emacs 根据各种标准计算句子结尾，例如 'sentence-end-double-space' 的值。

有些语言（如泰语）不使用句点表示句子结束。在这种情况下，请把变量 'sentence-end-without-period' 设为 't'。

尽管上面提到的句子移动命令是基于人类语言的，其他 Emacs 模式也可以设置这些命令以获得类似功能（参见"按句子移动"）。

27.3 段落

用于操作段落的 Emacs 命令也在 Meta 键上。

- 'M-{' 向后移到前一段开头（'backward-paragraph'）。
- 'M-}' 向前移到下一段末尾（'forward-paragraph'）。
- 'M-h' 把光标点和标记放到本段或下一段周围（'mark-paragraph'）。

'M-{'（'backward-paragraph'）移动到当前段或上一段的开头，取决于调用命令时光标点的位置（段落的定义见下文）。'M-}'（'forward-paragraph'）类似地移动到当前段或下一段的末尾。如果段落之前有一个空行，'M-{' 会移动到该空行。

当你想对某个段落进行操作时，输入 'M-h'（'mark-paragraph'）来设置围绕它的区域。例如，'M-h C-w' 删除光标点所在或之后的段落。'M-h' 把光标点放在段落开头、标记放在段落末尾，该段落即光标点原来所在的段落。如果光标点位于段落之间（在一串空行中，或处在边界上），'M-h' 设置围绕光标点之后那一段的区域。如果段落第一行之前有空行，这些空行中的某一行会被包含在区域内。如果区域已处于活动状态，该命令只设置标记而不移动光标点，之后每按一次 'M-h' 都会把标记再推进一个段落。

段落的定义取决于主模式。在 Fundamental 模式以及 Text 模式和相关模式中，段落与相邻段落之间由一个或多个"blank lines"（空行）分隔——这些行要么是空的，要么仅由空格、制表符和/或换页符组成。在编程语言模式中，段落通常以类似方式定义，因此即使程序里并没有真正意义上的段落，你也可以使用段落命令。

注意，在 Text 模式中，缩进的行本身并不构成段落分隔。如果你希望用缩进的行分隔段落，请改用 Paragraph-Indent Text 模式。参见"文本模式"。

如果设置了填充前缀，那么段落由所有不以填充前缀开头的行来界定。参见"填充文本"。

段落边界的确切定义由变量 'paragraph-separate' 和 'paragraph-start' 控制。'paragraph-start' 的值是一个正则表达式，应匹配开始段落或分隔段落的行（参见"正则表达式的语法"）。'paragraph-separate' 的值是另一个正则表达式，应匹配分隔段落但不属于任何段落的行（例如空行）。开始一个新段落并且属于该段落一部分的行必须只匹配 'paragraph-start'，而不匹配 'paragraph-separate'。例如，在 Fundamental 模式中，'paragraph-start' 是 *'"\f\\|[\t]$"'*，*'paragraph-separate' 是 '"[\t\f]$"'*。

注意，'paragraph-start' 和 'paragraph-separate' 是与左侧边距处的文本相匹配的，那里不一定是行的开头，所以这些正则表达式不应使用 '^' 作为锚点，以保证在按边距设置缩进的文本区域内段落函数同样正常工作。

27.4 页

在某些文本文件中，文本被分为若干“页”，由“换页符”（formfeed character，ASCII 码 12，也写作‘control-L’）分隔，Emacs 将其显示为转义序列‘^L’（参见“文本的显示”）。传统上，这类文本文件打印到纸面时，每个换页符都会强制分页。大多数 Emacs 命令把它当作普通字符对待，因此你可以用‘C-q C-l’插入它，用 <DEL> 删除它，等等。此外，Emacs 还提供了在页之间移动并对页进行操作的命令。

- ‘M-x what-page’显示光标点所在的页号，以及在该页中的行号。
- ‘C-x [’把光标点移到上一页边界（‘backward-page’）。
- ‘C-x]’把光标点移到下一页边界（‘forward-page’）。
- ‘C-x C-p’把光标点和标记放到本页（或其他页）周围（‘mark-page’）。
- ‘C-x l’统计本页的行数（‘count-lines-page’）。

‘M-x what-page’从文件开头开始数页，并数出页内的行号，把两个数字都显示在回显区。

‘C-x [’（‘backward-page’）命令把光标点移到上一个分页符之后。如果光标点已经紧跟在分页符之后，它会跳过这一个，停在上一个分页符之后。数值参数用作重复次数。‘C-x]’（‘forward-page’）命令向前移动越过下一个分页符。

‘C-x C-p’（‘mark-page’）命令把光标点放在当前页开头（即页面开头那个分页符之后），把标记放在页末尾（即末尾分页符之后）。

‘C-x C-p C-w’是删除一页以便把它移到别处的便捷方法。如果你先用‘C-x [’和‘C-x]’移到另一个分页符处，再粘贴所删除的页，所有页会重新被正确分隔。‘C-x C-p’之所以在区域中只包含末尾的那个分页符，就是为了确保这种操作按预期工作。

给‘C-x C-p’一个数值参数可指定要转到哪一页（相对于当前页）。零表示当前页，一表示下一页，−1 表示上一页。

‘C-x l’（‘count-lines-page’）命令有助于决定在何处把一页分成两页。它在回显区显示当前页的总行数，然后将其分为当前行之前的行和当前行之后的行，如下所示：

```
Page has 96 (72+25) lines
```

注意总和相差一；如果光标点不在行首，这个结果是正确的。

变量‘page-delimiter’控制页从何处开始。其值是一个正则表达式，匹配分隔各页的行的开头（参见“正则表达式的语法”）。该变量的常规值是‘"^\f"’，匹配行首的换页符。

27.5 引号

一种常见的引用方式是打字机惯例，用直撇号“like this”或直双引号‘"like this"’来引用。另一种常见方式是弯引号惯例，使用左右单引号‘like this’或左右双引号“like this”(1)。在文本文件中，打字机式引号简单且可移植；弯引号歧义更少，通常也更美观。

Electric Quote 模式使输入弯引号更容易。在你输入字符时，它会选择性地把‘’ 转换为 ‘，把 ‘'’ 转换为 ’，把 ‘`’ 转换为 “，把 ‘"’ 转换为 ”。可以通过定制变量‘electric-quote-chars’来更改上面列出的默认引号，该变量

是一个包含四个字符的列表，各项依次对应左单引号、右单引号、左双引号和右双引号，其默认值为 “(?‘ ?’ ?“ ?”)’。

你可以通过定制控制其生效位置的变量来定制 Electric Quote 模式的行为。如果 ‘electric-quote-paragraph’ 非 ‘nil’，它在文本段落中生效；如果 ‘electric-quote-comment’ 非 ‘nil’，它在编程语言注释中生效；如果 ‘electric-quote-string’ 非 ‘nil’，它在编程语言字符串中生效。默认情况下，‘electric-quote-string’ 为 ‘nil’，其他变量为 ‘t’。

你也可以把选项 ‘electric-quote-replace-double’ 设为非 ‘nil’ 值。这样，输入 ‘"’ 时会根据上下文插入合适的弯双引号：在缓冲区开头，或在换行、空白、左括号或引号字符之后插入 “，其他情况下插入 ”。

Electric Quote 模式默认禁用。要在单个缓冲区中切换它，使用 ‘M-x electric-quote-local-mode’。要全局切换，输入 ‘M-x electric-quote-mode’。要抑制一次转换，输入 ‘C-q ’ 或 ‘C-q '’ 而不是 ‘’’ 或 “’”。即使 Electric Quote 被禁用或未生效，你也可以输入 ‘C-x 8 [’ 得到 ‘，输入 ‘C-x 8]’ 得到 ’，输入 ‘C-x 8 {’ 得到 “，输入 ‘C-x 8 }’ 得到 ”。参见“插入文本”。注意，‘electric-quote-chars’ 的值不影响这些键绑定，它们不是 ‘electric-quote-mode’ 的键绑定，而是在 ‘global-map’ 中绑定的。

> 注：弯单引号字符是 U+2018 LEFT SINGLE QUOTATION MARK 和 U+2019 RIGHT SINGLE QUOTATION MARK；弯双引号是 U+201C LEFT DOUBLE QUOTATION MARK 和 U+201D RIGHT DOUBLE QUOTATION MARK。在无法显示这些字符的文本终端上，Info 阅读器可能把它们显示为打字机式的 ASCII 引号字符。

27.6 填充文本

“填充”文本指按指定宽度把它断成若干行。Emacs 用两种方式进行填充。在自动填充模式下，用自插入字符插入文本时也会自动填充。此外还有可在编辑文本时使用的显式填充命令。

27.6.1 自动填充模式

“自动填充”模式是一种缓冲区局部的次模式（参见“次模式”），当行变得过宽并且你输入 ‘<SPC>’ 或 ‘<RET>’ 时，它会自动把行断开。

- ‘M-x auto-fill-mode’ 启用或禁用自动填充模式。
- ‘<SPC>’ ‘<RET>’ 在自动填充模式下，于适当时机断行。

模式命令 ‘M-x auto-fill-mode’ 在当前缓冲区中切换自动填充模式。像任何其他次模式一样，带正数值参数时它启用自动填充模式，带负参数时禁用它。要在某些主模式中自动启用自动填充模式，请把 ‘auto-fill-mode’ 加入这些模式的钩子（参见“主模式”）。启用自动填充模式后，模式行中会出现模式指示符 ‘Fill’（参见“模式行”）。

只要行长超过所需宽度，自动填充模式就会在适当位置自动断行。这种断行只在你输入 ‘<SPC>’ 或 ‘<RET>’ 时发生。如果你想插入空格或换行而不允许断行，请分别输入 ‘C-q <SPC>’ 或 ‘C-q C-j’。另外，‘C-o’ 插入换行而不断行。

自动填充断行的位置取决于行中的字符。对于 ASCII、拉丁字母和大多数其他文字的字符，Emacs 在空格字符处断行，以保持单词完整。但对于 CJK 文字，行可以在任意两个字符之间断开。（如果加载了 ‘kinsoku’ 库，

Emacs 会避免在某些 CJK 字符对之间断行，那里特殊规则禁止断行。）

自动填充模式断行时，会尽量遵循“自适应填充前缀”：如果能从当前段落的第一行和/或第二行推出填充前缀，就把它插入新行（参见“自适应填充”）。否则，新行会被缩进，就好像你在该行输入了 <TAB> 一样（参见“缩进”）。在编程语言模式中，如果行在注释中间断开，会视情况插入新的注释分隔符来拆分注释。

自动填充模式不会重新填充整个段落；它只断行，不合并行。因此，在段落中间编辑可能导致段落填充不正确。要填充它，请调用显式填充命令（参见“显式填充命令”）。

一个类似的功能是在显示时自动折行超长的行，即 Visual Line 模式（参见“可视行模式”）。

27.6.2 显式填充命令

- ‘M-q’ 填充当前段落（‘fill-paragraph’）。
- ‘C-x f’ 设置填充列（‘set-fill-column’）。
- ‘M-x fill-region’ 填充区域内的每个段落。
- ‘M-x fill-region-as-paragraph’ 把区域当作一个段落来填充。
- ‘M-x unfill-paragraph’ 取消填充当前段落。
- ‘M-x center-line’ 使一行居中。

‘M-q’（‘fill-paragraph’）命令“填充”当前段落。它重新分配段落内的换行，并删除段落中出现的多余空格和制表符，使各行最终都在某个最大宽度之内。像自动填充模式一样，这个命令和其他填充命令通常在空格字符处断行，但对于 CJK 字符，这些命令几乎可以在任意两个字符之间断行，并且也能遵守 kinsoku 规则。参见“自动填充模式”。

通常，‘M-q’ 作用于光标点所在的段落；但如果光标点位于段落之间，它作用于光标点之后的段落。如果区域处于活动状态，它改为作用于区域中的文本。你也可以调用 ‘M-x fill-region’ 专门填充区域中的文本。

‘M-q’ 和 ‘fill-region’ 使用 Emacs 寻找段落边界的常规标准（参见“段落”）。要获得更多控制，你可以使用 ‘M-x fill-region-as-paragraph’，它把光标点和标记之间的所有内容作为单个段落重新填充。该命令的行为由变量 ‘fill-region-as-paragraph-function’ 控制。默认情况下，它会删除区域内的所有空行，于是分开的文本块最终合并成一块。

给 ‘M-q’ 一个数值参数会让它在填充的同时对文本进行“两端对齐”。这意味着插入额外的空格，使右边界正好对齐到填充列。要去除多余的空格，请不带参数使用 ‘M-q’。（‘fill-region’ 同理。）

填充的最大行宽由缓冲区局部变量 ‘fill-column’ 指定。其默认值（参见“局部变量”）是 70。在当前缓冲区中设置 ‘fill-column’ 最简单的方法是用命令 ‘C-x f’（‘set-fill-column’）。带数值参数时，它把该值作为新的填充列。只把 ‘C-u’ 作为参数时，它把 ‘fill-column’ 设为光标点当前的水平位置。注意，就其本质而言，‘fill-column’ 以列为单位度量；该列在图形显示器上的实际位置取决于所用的字体。特别是，使用变宽字体会导致 ‘fill-column’ 在显示时于不同行占据不同的水平位置。

‘M-x unfill-paragraph’ 命令是 ‘M-q’ 的逆操作。它把当前段落的各行连接起来。使用该命令相当于从段落最后一行开始，反复输入 ‘M-^’（参见“缩进命令”），直到所有行都连接起来。带数值前缀参数时，对该数量的段落执行此操作。例如，数值前缀参数为 2 时，连接本段落的所有行和下一段落的所有行，但保留两个段落之间的分隔。没有前缀参数时，它的行为与 ‘M-q’ 一样，作用于活动区域。

‘M-x center-line’ 命令在当前填充列范围内使当前行居中。带参数 N 时，它逐行使 N 行居中并移到它们之后。这个绑定由 Text 模式建立，只在该模式及相关模式中可用（参见“文本模式”）。

默认情况下，Emacs 认为句点后跟两个空格或一个换行是句子结尾；句点后只跟一个空格表示缩写，而不是句子结尾。因此，填充命令不会在仅跟一个空格的句点之后断行。如果你把变量 ‘sentence-end-double-space’ 设为 ‘nil’，填充命令会在句点后跟一个空格处断行，并在每个句点后只放一个空格。关于这一点的其他影响和可能的缺点，参见“句子”。

如果变量 ‘colon-double-space’ 非 ‘nil’，填充命令会在冒号后放两个空格。

要指定不允许断行的其他条件，请定制异常钩子变量 ‘fill-nobreak-predicate’（参见“钩子”）。该钩子中的每个函数都在不带参数的情况下被调用，调用时光标点位于 Emacs 正在考虑断行的位置。如果函数返回非 ‘nil’ 值，Emacs 就不会在那里断行。可以在其中使用的函数包括：‘fill-single-word-nobreak-p’（不在句子第一个单词之后或最后一个单词之前断行）；‘fill-single-char-nobreak-p’（不在前面是空白字符的单字母单词之后断行）；‘fill-french-nobreak-p’（不在 ‘(’ 之后或 ‘)’、‘:’、‘?’ 之前断行）；以及 ‘fill-polish-nobreak-p’（不在单字母单词之后断行，即使它前面是非空白字符）。

Emacs 可以在 ‘fill-column’ 位置显示一个指示符，这由 Display fill column indicator 模式实现（参见“显示边界”）。

27.6.3 填充前缀

“填充前缀”（fill prefix）功能允许在填充段落时让每一行都以一个特殊的字符串开头（例如一串空格，从而得到缩进的段落）。你可以显式指定填充前缀；否则 Emacs 会尝试自动推断一个（参见“自适应填充”）。

- ‘C-x .’ 设置填充前缀（‘set-fill-prefix’）。
- ‘M-q’ 使用当前填充前缀填充段落（‘fill-paragraph’）。
- ‘M-x fill-individual-paragraphs’ 填充区域，把每一处缩进变化都视为新段落的开始。
- ‘M-x fill-nonuniform-paragraphs’ 填充区域，只把段落分隔行视为新段落的开始。

要为当前缓冲区指定填充前缀，先移到以所需前缀开头的一行，把光标点放在该前缀末尾，然后键入 ‘C-x .’（‘set-fill-prefix’）。（这是 ‘C-x’ 之后的一个句点。）要关闭填充前缀，可以键入 ‘C-u C-x .’，也可以指定一个空前缀：把光标点放在行首，然后键入 ‘C-x .’。

当填充前缀生效时，填充命令会在填充之前去掉段落每一行的填充前缀，填充之后再把它插回每一行。（段落第一行的开头保持不变，因为那里常常是有意与众不同。）Auto Fill 模式在另起新行时也会自动插入填充前缀（参见“自动填充模式”）。‘C-o’ 命令在行首使用时，会在它创建的新行上插入填充前缀（参见“空行”）。反过来，命令 ‘M-^’ 在删除换行符时，也会删除该换行符之后的前缀（如果存在的话）（参见“缩进”）。

例如，若 ‘fill-column’ 为 40，且你把填充前缀设为 ‘;; ’，那么对下面这段文字执行 ‘M-q’

```
;; This is an
;; example of a paragraph
;; inside a Lisp-style comment.
```

就会得到：

```
;; This is an example of a paragraph
;; inside a Lisp-style comment.
```

不以填充前缀开头的行被视为段落的开始，这在 'M-q' 和段落命令中都是如此；对于悬挂缩进的段落（除第一行外每行都缩进）来说，这样做效果很好。去掉前缀后为空或仍有缩进的行也会分隔或开始段落；若你在编写每行都带注释定界符的多段注释，这正是你想要的。

你可以用 'M-x fill-individual-paragraphs' 自动为每个段落设置填充前缀。该命令把区域划分成段落，把缩进量的每一处变化都当作新段落的开始，然后逐一填充这些段落。这样，同一段落中所有行的缩进量相同，而该缩进量就作为该段落的填充前缀。

'M-x fill-nonuniform-paragraphs' 是一个类似的命令，只是以不同方式把区域划分成段落。它只把段落分隔行（由 'paragraph-separate' 定义）视为新段落的开始。这意味着一个段落中各行的缩进量可以不同，因此所用的填充前缀取该段落所有行中最小的缩进量。对于首行缩进比其余各行多或少一些的排版风格，这样效果很好。

填充前缀保存在变量 'fill-prefix' 中。其值为一个字符串，没有填充前缀时为 'nil'。这是一个缓冲区局部变量；修改该变量只影响当前缓冲区，但还存在一个默认值，你同样可以更改。参见"局部变量"。

'indentation' 文本属性提供了另一种控制段落缩进量的方法。参见"富文本中的缩进"。

27.6.4 自适应填充

在某些情况下，填充命令可以自动推断出段落合适的填充前缀：行首的空白或某些标点字符会被传播到段落的所有行。

若段落有两行或更多行，填充前缀取自段落的第二行，但前提是它也出现在第一行上。

若段落只有一行，填充命令**可能**从该行取用前缀。这个决策很复杂，因为在这种情形下有三种合理的做法：

- 把第一行的前缀用于段落的所有行。
- 用空白缩进后续各行，使它们与第一行前缀之后的文字对齐，但并不真正复制第一行的前缀。
- 对第二行及后续各行不做任何特殊处理。

这三种排版风格都很常用。因此填充命令会根据出现的前缀和主模式来推断你想要哪一种。具体如下。

若在第一行上找到的前缀匹配 'adaptive-fill-first-line-regexp'，或者它看起来像注释起始序列（这取决于主模式），则用找到的前缀来填充该段落，前提是它在后续行上不会充当段落起始。

否则，把找到的前缀转换成等量的空格，并用这些空格作为其余各行的填充前缀，前提是它们在后续行上不会充当段落起始。

在文本模式以及其他只用空行和分页符分隔段落的模式中，自适应填充选出的前缀从不充当段落起始，因此总能用于填充。

变量 'adaptive-fill-regexp' 决定什么样的行首可以作为填充前缀：行首匹配该正则表达式的所有字符都会被使用。若把变量 'adaptive-fill-mode' 设为 'nil'，则从不自动选择填充前缀。

把变量 'adaptive-fill-function' 设为一个函数，可以指定更复杂的自动选择填充前缀的方式。该函数在行的左边距之后以光标点的位置调用，应基于该行返回合适的填充前缀。若它返回 'nil'，则由 'adaptive-fill-regexp' 再尝试寻找前缀。

27.7 大小写转换命令

Emacs 有把单个单词或任意范围的文本转换为大写或小写的命令。

- ‘M-l’ 把后面的单词转为小写（‘downcase-word’）。
- ‘M-- M-l’ 把前一个/最后一个单词转为小写。注意：‘Meta--’ 即 Meta 减号。
- ‘M-u’ 把后面的单词转为大写（‘upcase-word’）。
- ‘M-- M-u’ 把前一个/最后一个单词转为全大写。
- ‘M-c’ 使后面的单词首字母大写（‘capitalize-word’）。
- ‘M-- M-c’ 把前一个/最后一个单词转为首字母大写、其余小写。
- ‘C-x C-l’ 把区域转为小写（‘downcase-region’）。
- ‘C-x C-u’ 把区域转为大写（‘upcase-region’）。

‘M-l’（‘downcase-word’）把光标点之后的单词转为小写，并把光标点移过该词。因此重复 ‘M-l’ 可依次转换后面的单词。‘M-u’（‘upcase-word’）改为转为全大写，而 ‘M-c’（‘capitalize-word’）把单词首字母变为大写、其余变为小写。这些命令若给出参数，可一次转换多个单词。它们特别便于把大量全大写文本转为大小写混合，因为你可以用 ‘M-l’、‘M-u’ 或 ‘M-c’ 逐个单词按需要转换，偶尔改用 ‘M-f’ 跳过一个单词。

当给出负参数时（如 ‘C-u - 5 M-c’），单词大小写转换命令作用于光标点之前相应数目的单词，但不移动光标点。若你刚把某个单词的大小写打错，这很方便：给出 ‘M-- M-u’ 这样的转换命令，然后继续键入即可。

若在单词中间执行单词大小写转换命令，它只作用于该单词中光标点之后的部分。（这与 ‘M-d’（‘kill-word’）的行为类似。）带负参数时，转换只作用于光标点之前的单词部分。

其他大小写转换命令是 ‘C-x C-u’（‘upcase-region’）和 ‘C-x C-l’（‘downcase-region’），它们把光标点与标记之间的全部内容转换为指定的大小写。光标点和标记都不移动。

区域大小写转换命令 ‘upcase-region’ 和 ‘downcase-region’ 默认处于禁用状态。这意味着尝试使用它们时会要求确认。确认之后，你可以启用该命令，此后它就不再要求确认。参见“禁用命令”。

27.8 文本模式

文本模式（Text mode）是用于编辑人类语言文本文件的主模式。文件名以 ‘.txt’ 结尾的文件通常用文本模式打开（参见“选择文件模式”）。要显式切换到文本模式，请键入 ‘M-x text-mode’。

在文本模式中，只有空行和分页符分隔段落。因此段落可以缩进，而填充段落时由自适应填充决定使用何种缩进。参见“自适应填充”。

在文本模式中，<TAB>（‘indent-for-tab-command’）命令通常插入空白直到下一个制表位，而不是缩进当前行。详情见参见“缩进”。

文本模式会关闭与注释有关的功能，除非你显式调用它们。它修改语法表，使撇号被视为单词的一部分（例如 ‘don't’ 算作一个单词）。不过，若单词以撇号开头，则在处理大小写时该撇号被视为前缀（例如 ‘M-c’ 会把 "hello" 转换为 "Hello"，符合预期）。

若你缩进段落的首行，那么应当使用段落缩进文本模式（‘M-x paragraph-indent-text-mode’）而不是文本模式。在该模式下，段落之间无需空行，因为首行缩进足以开始一个新段落；但不支持每行都缩进的段落。对于不应

改变主模式的场合（例如撰写邮件），可用 'M-x paragraph-indent-minor-mode' 启用一个等效的次模式。

命令 'M-<TAB>'（'completion-at-point'）对缓冲区中光标点之前的残缺单词进行补全，默认以拼写词典作为可能的单词空间。参见"拼写检查与纠正"。若你的窗口管理器把 'M-<TAB>' 定义为切换窗口，可以改键入 '<ESC> <TAB>' 或 'C-M-i'。要禁用这种补全，请把变量 'text-mode-ispell-word-completion' 定制为 'nil'。

进入文本模式会运行模式钩子 'text-mode-hook'（参见"主模式"）。

以下几节介绍若干"派生"自文本模式的主模式。这些派生模式共享上文所述文本模式的大部分功能。特别是，文本模式的派生模式会先运行 'text-mode-hook'，再运行它们自己的模式钩子。

27.9 大纲模式

大纲模式（Outline mode）是派生自文本模式的主模式，专门用于编辑大纲。它提供在大纲结构的条目之间移动的命令，以及让缓冲区某些部分暂时不可见的命令，以便更轻松地查看大纲结构。键入 'M-x outline-mode' 切换到大纲模式。进入大纲模式会先运行钩子 'text-mode-hook'，再运行钩子 'outline-mode-hook'（参见"钩子"）。

当你用大纲模式命令让某一行不可见时（参见"大纲可见性命令"），该行会从屏幕上消失。前一个可见行的末尾会显示一个省略号（连续三个句点），以表示有隐藏文本。连续多个不可见行只产生一个省略号。

对行进行操作的编辑命令，如 'C-n' 和 'C-p'，会把不可见行的文本当作前一个可见行的一部分。删除可见行末尾的省略号，确实会删除与该省略号相关的所有后续不可见文本。

27.9.1 大纲次模式

大纲次模式是缓冲区局部的次模式，提供与主模式大纲模式相同的命令，但可以与其他主模式配合使用。你可以键入 'M-x outline-minor-mode' 在当前缓冲区中开关大纲次模式，或通过文件局部变量设置在特定文件中启用它（参见"文件中的局部变量"）。

主模式大纲模式在 'C-c' 前缀上提供特殊的键绑定。大纲次模式以 'C-c @' 为前缀提供类似的绑定；这样做是为了减少与主模式特殊命令的冲突。（变量 'outline-minor-mode-prefix' 控制所用的前缀。）

若 'outline-minor-mode-use-buttons' 非 'nil'，大纲次模式除省略号外，还会在标题行开头使用按钮，以表示某一节被隐藏。用鼠标点击按钮可切换该节的显示。若该变量的值为 'insert'，按钮会直接插入缓冲区文本中，于是在按钮上按 <RET> 也会像鼠标点击一样切换该节的显示。在可编辑缓冲区中不建议使用 'insert' 值，因为它会修改缓冲区。若值为 'in-margins'，大纲次模式会使用窗口边缘来指示某一节被隐藏。这些按钮可以按图标定制（参见"图标"）。

若用户选项 'outline-minor-mode-cycle' 非 'nil'，则在大纲标题行上启用循环切换可见性的 'TAB' 和 'S-<TAB>' 键（参见"大纲可见性命令"）。'TAB' 在当前节中循环切换隐藏、显示子标题和全部显示三种状态。'S-<TAB>' 对整个缓冲区做同样的事。

27.9.2 大纲的格式

大纲模式假定缓冲区中的行有两类："标题行"（heading line）和"正文行"（body line）。标题行代表大纲中的一个主题。标题行以一个或多个星号（'*'）*字符开头；星号数目决定该标题在大纲结构中的深度。因此，带一*

*个 * 的标题行是一个主主题；在它与下一个单 ‘* 标题之间所有带两个 * 的标题行都是它的子主题；依此类推。任何不是标题行的行都是正文行。正文行属于它前面的标题行。下面是一个例子：

- 食物 这是正文， 讲述关于食物这一主题的一些内容。

** 美味的食物 这是二级标题的正文。

** 难吃的食物 这个也可以有 正文，包含 好几行。

*** 宿舍食物

- 栖身之所 另一个一级主题，带着它的标题行。

标题行连同其后的所有正文行合称一个“条目”（entry）。标题行连同其后的所有更深层标题行及其正文行合称一个“子树”（subtree）。

你可以通过设置变量 ‘outline-regexp’ 来定制区分标题行的标准。（推荐的做法是在主模式函数中设置，或使用文件局部变量。）任何行首匹配该正则表达式的行都被视为标题行。在行中间（不在左边界）开始的匹配不算。

匹配文本的长度决定标题的层级；匹配越长，嵌套层级越深。例如，若某个文本格式化程序用 ‘@chapter’、‘@section’ 和 ‘@subsection’ 命令把文档分为章和节，你可以把 ‘outline-regexp’ 设为 ‘"@chap\\|@\\(sub\\)*section"’，使这些行算作标题行。注意这个技巧：‘chapter’ 和 ‘section’ 两个词一样长，但把正则表达式定义为只匹配 ‘chap’，就能确保在章标题上匹配到的文本更短，从而让大纲模式知道节包含在章之内。只要没有其他命令以 ‘@chap’ 开头，这样就行得通。

除了设置变量 ‘outline-regexp’，还可以把变量 ‘outline-search-function’ 设为一个函数，由它匹配当前标题并搜索下一个标题（参见《Emacs Lisp 参考手册》的“Outline Minor Mode”）。

你可以通过设置变量 ‘outline-level’ 显式指定计算标题层级的规则。‘outline-level’ 的值应为一个不接受参数、并返回当前标题层级的函数。推荐在主模式命令中设置该变量，或使用文件局部变量。

27.9.3 大纲移动命令

大纲模式提供特殊的移动命令，可向前和向后移动到标题行。

- ‘C-c C-n’ 把光标点移到下一个可见标题行（‘outline-next-visible-heading’）。
- ‘C-c C-p’ 把光标点移到上一个可见标题行（‘outline-previous-visible-heading’）。
- ‘C-c C-f’ 把光标点移到与光标点所在标题同层的下一个可见标题行（‘outline-forward-same-level’）。
- ‘C-c C-b’ 把光标点移到同层的上一个可见标题行（‘outline-backward-same-level’）。
- ‘C-c C-u’ 把光标点向上移到更低层（更包容）的可见标题行（‘outline-up-heading’）。

以上所有命令都接受数字参数作为重复次数。例如，‘C-c C-f’ 带参数时会向前移动相应数目的同层可见标题行，而 ‘C-c C-u’ 带参数时会向外退出相应数目的嵌套层级。

27.9.4 大纲可见性命令

大纲模式提供若干命令，按大纲结构暂时隐藏或显示缓冲区的某些部分。这些命令不能撤销；其效果不会被撤销机制记录，因此你可以直接撤销越过它们（参见“撤销”）。

其中许多命令作用于当前标题行。若光标点在标题行上，该行就是当前标题行；若光标点在正文行上，当前标题行就是它前面最近的一个标题行。

- ‘C-c C-c’ 使当前标题行的正文不可见（‘outline-hide-entry’）。
- ‘C-c C-e’ 使当前标题行的正文可见（‘outline-show-entry’）。
- ‘C-c C-d’ 使当前标题下的所有内容不可见，但不包括标题本身（‘outline-hide-subtree’）。
- ‘C-c C-s’ 使当前标题下的所有内容可见，包括正文、子标题及其正文（‘outline-show-subtree’）。
- ‘C-c C-l’ 使当前标题行及其所有子标题的正文不可见（‘outline-hide-leaves’）。
- ‘C-c C-k’ 使当前标题行各个层级的所有子标题可见（‘outline-show-branches’）。
- ‘C-c C-i’ 使当前标题行直接下属的（低一层的）子标题可见（‘outline-show-children’）。
- ‘C-c C-t’ 使缓冲区中所有正文行不可见（‘outline-hide-body’）。
- ‘C-c C-a’ 使缓冲区中所有行可见（‘outline-show-all’）。
- ‘C-c C-q’ 除最上面 N 层的标题行外全部隐藏（‘outline-hide-sublevels’）。
- ‘C-c C-o’ 除光标点所在的标题或正文、以及从那里向上直到大纲顶层的各级标题之外，其他全部隐藏（‘outline-hide-other’）。
- ‘C-c / h REGEXP <RET>’ 隐藏匹配 REGEXP 的标题的正文（‘outline-hide-by-heading-regexp’）。
- ‘C-c / s REGEXP <RET>’ 显示匹配 REGEXP 的标题的正文（‘outline-show-by-heading-regexp’）。

这些命令中最简单的是 ‘C-c C-c’（‘outline-hide-entry’），它隐藏紧跟在当前标题行之后的正文行；以及 ‘C-c C-e’（‘outline-show-entry’），它显示这些正文行。子标题及其正文不受影响。

命令 ‘C-c C-d’（‘outline-hide-subtree’）和 ‘C-c C-s’（‘outline-show-subtree’）功能更强。它们作用于当前标题行的“子树”：它的正文、它的所有子标题（直接的和间接的），以及这些子标题的全部正文。

命令 ‘C-c C-l’（‘outline-hide-leaves’）隐藏当前标题行的正文以及其子树中的所有正文；子标题本身保持可见。
命令 ‘C-c C-k’（‘outline-show-branches’）显示出子标题，前提是它们此前被隐藏了（例如被 ‘C-c C-d’ 隐藏）。
命令 ‘C-c C-i’（‘outline-show-children’）是这一命令的较弱版本；它只显示直接子标题，即低一层的那些。

命令 ‘C-c C-o’（‘outline-hide-other’）隐藏除光标点所在的条目、其父级（从那里向上直到大纲顶层的各级标题）以及顶层标题之外的一切。它还会显示缓冲区中第一个标题之前的正文行。

命令 ‘C-c / h’（‘outline-hide-by-heading-regexp’）提示输入一个正则表达式，并隐藏所有匹配该正则表达式的标题的正文行。命令 ‘C-c / s’ 同样提示输入一个正则表达式，并显示匹配标题的正文。

其余命令影响整个缓冲区。‘C-c C-t’（‘outline-hide-body’）使所有正文行不可见，于是你只看到大纲结构（作为特例，它不会隐藏文件顶部、第一个标题行之前的行，尽管按严格定义这些也是正文行）。‘C-c C-a’（‘outline-show-all’）使所有行可见。‘C-c C-q’（‘outline-hide-sublevels’）隐藏除位于当前标题行层级及以上的顶层标题之外的所有内容（若光标点不在标题上，则默认为 1 层）；带数字参数 N 时，它隐藏除最上面 N 层标题行之外的一切。注意它会完全显示最上面的 N 层以及第一个标题之前的正文行。

大纲模式还提供两个便捷命令，用于循环切换每一节以及整个缓冲区的可见性。在标题上键入 ‘<TAB>’（‘outline-cycle’）会在“全部隐藏”“子标题”“全部显示”三种状态之间循环切换当前节。键入 ‘S-

<TAB>’（‘outline-cycle-buffer’）会在“仅顶层标题”“所有标题和子标题”“全部显示”三种状态之间循环切换整个缓冲区。

当增量搜索找到被大纲模式隐藏的文本时，它会让缓冲区的那部分可见。若你在该位置退出搜索，文本会保持可见。要切换活动中的增量搜索能否匹配隐藏文本，请键入‘M-s i’。要更改以后搜索的默认行为，请定制选项‘search-invisible’。（该选项也影响‘query-replace’及相关函数处理隐藏文本的方式，参见“查询替换”。）你还可以使用 Reveal 模式（‘M-x reveal-mode’）——一个缓冲区局部的次模式——在浏览文本时自动使其可见。

变量‘outline-default-state’控制打开大纲模式后哪些标题可见。若为非‘nil’，初始时会隐藏部分标题。若等于一个数字，则只显示到相应层级（含该层级）的标题。若等于‘outline-show-all’，则显示缓冲区的所有文本。若等于‘outline-show-only-headings’，则只显示标题，不管其层级如何。若等于一个 lambda 函数或函数名，则启用该模式后会无参数调用此函数，由它切换标题的可见性。

27.9.5 在多个视图中查看单个大纲

你可以在不同窗口中同时显示单个大纲的两个视图。为此，必须用‘M-x make-indirect-buffer’创建间接缓冲区。该命令的第一个参数是已有的大纲缓冲区名，第二个参数是新间接缓冲区要使用的名字。参见“间接缓冲区”。

间接缓冲区创建后，就可以按常规方式用‘C-x 4 b’或其他 Emacs 命令在窗口中显示它。Outline 模式中显示和隐藏部分文本的命令对每个缓冲区独立生效；因此，每个缓冲区可以有各自的视图。如果你希望同一大纲有超过两个视图，就再创建额外的间接缓冲区。

27.9.6 折叠编辑

Foldout 包用折叠命令扩展了 Outline 模式和 Outline 次模式。折叠的思路是：放大（zoom in）大纲中某个嵌套的部分，同时隐藏它在更高层级的亲属节点。

设想一个 Outline 模式缓冲区，其中 level-1 标题下的所有正文和子标题都被隐藏。要查看其中一个标题下隐藏的内容，可以用‘C-c C-e’（‘M-x outline-show-entry’）展开正文，或用‘C-c C-i’展开子（level-2）标题。

使用 Foldout 时，用‘C-c C-z’（‘M-x foldout-zoom-subtree’）。它会展开正文和子标题，并窄化缓冲区，使只有 level-1 标题、正文和 level-2 标题可见。现在要查看某个 level-2 标题下的内容，把光标移到该标题上，再次使用‘C-c C-z’。这会展开 level-2 正文及其 level-3 子标题，并再次窄化缓冲区。可以按需要不断放大到后续的子标题。模式行中的字符串会显示你已进入的深度。

在放大的标题上，若只想看到子标题，可指定数字参数：‘C-u C-c C-z’。也可以指定子标题的层数（比较‘M-x outline-show-children’），例如‘M-2 C-c C-z’会展开两层子标题。另外，可以用负参数指定只显示正文：‘M-- C-c C-z’。给出零参数可以展开整个子树，类似于‘C-c C-s’（‘M-x outline-show-subtree’）：‘M-0 C-c C-z’。

在放大状态下，仍可使用 Outline 模式的展开和隐藏函数，而不会干扰 Foldout。此外，由于缓冲区已被窄化，全局编辑操作只会影响放大标题下的文本。这对于把修改限制在文档的某一章或某一节很有用。

要取消放大（退出）某个折叠，使用‘C-c C-x’（‘M-x foldout-exit-fold’）。它会隐藏顶层标题下的所有文本和子标题，并让你回到缓冲区先前的视图。指定数字参数则退出那么多层折叠。指定零参数则退出所有折叠。指定负参数（使退出的折叠仍保持可见）会保留光标点和窗口视图的位置。

若要取消折叠的窄化而不隐藏文本和子标题，可指定负参数。例如，‘M--2 C-c C-x’退出两层折叠，同时保持文本和子标题展开。

在折叠内工作时，你可能想使用 Emacs 标准的窄化命令，如‘C-x n n’（‘narrow-to-region’）或‘C-x n d’（‘narrow-to-defun’）。使用这些命令后，‘foldout-widen-to-current-fold’允许你解除窄化回到当前折叠层级，而不是整个缓冲区。如果当前不在折叠中，它的行为与‘widen’相同。

Foldout 模式还提供用于进入和退出折叠以及显示和隐藏文本的鼠标命令：

- ‘C-M-mouse-1’：放大点击的标题。单击：展开正文。双击：展开子标题。三击：展开正文和子标题。四击：展开整个子树。
- ‘C-M-mouse-2’：展开点击的标题下的文本。单击：展开正文。双击：展开子标题。三击：展开正文和子标题。四击：展开整个子树。
- ‘C-M-mouse-3’：隐藏点击的标题下的文本或退出折叠。单击：隐藏子树。双击：退出折叠并隐藏文本。三击：退出折叠但不隐藏文本。四击：退出所有折叠并隐藏文本。

可以通过设置‘foldout-mouse-modifiers’来指定不同的修饰键（代替‘<Ctrl>-<Meta>-’）；但如果你已经加载了‘foldout.el’库，必须重新加载它才能让这一设置生效。

要使用 Foldout 包，可以输入‘M-x load-library <RET> foldout <RET>’；也可以在初始化文件中放入以下内容，让它自动完成：

```
(with-eval-after-load "outline"
  (require 'foldout))
```

27.10 Org 模式

Org 模式是 Outline 模式的一个变体，用于把 Emacs 当作组织工具和/或写作系统使用。文件名以‘.org’扩展名结尾的文件会在 Org 模式中打开（参见“选择文件模式”）。要显式切换到 Org 模式，输入‘M-x org-mode’。

在 Org 模式中，与 Outline 模式一样，每个条目都有一个以一个或多个‘*’字符开头的标题行。参见“大纲的格式”。此外，任何以‘#’字符开头的行都被视为注释。

Org 模式提供的命令可以方便地查看和操作大纲结构。这些命令中最简单的是 <TAB>（‘org-cycle’）。如果在标题行上调用，它会在子树的几种可见状态之间循环：(i) 只显示该标题行；(ii) 只显示该标题行及其直接子标题的标题行（如果有）；(iii) 显示整个子树。如果在正文行上调用，则执行 <TAB> 的全局绑定。

在 Org 模式缓冲区的任何位置输入‘S-<TAB>’（‘org-shifttab’），都会在整个大纲结构的可见状态之间循环：(i) 只显示顶层标题行；(ii) 显示所有标题行但不显示正文行；(iii) 显示全部内容。

在标题行上输入‘M-<UP>’（‘org-metaup’）或‘M-<DOWN>’（‘org-metadown’），可以在缓冲区中把整个条目（包括其正文行和子树，如果有）上移或下移。类似地，可以用‘M-<LEFT>’（‘org-metaleft’）和‘M-<RIGHT>’（‘org-metaright’）提升或降低标题行的层级。如果在正文行上调用，这些命令会执行其全局绑定。

以下小节给出把 Org 模式用作组织工具和写作系统的基本说明。详细信息参见《Org 手册》。

27.10.1 用 Org 做组织工具

在条目中的任何位置输入‘C-c C-t’（‘org-todo’），就能把某个 Org 条目标记为“TODO”项。这会在标题行添加关键字‘TODO’。再次输入‘C-c C-t’会把关键字切换为‘DONE’；再输入一次‘C-c C-t’则完全删除该关键字，依此

类推。可以通过变量‘org-todo-keywords’自定义‘C-c C-t’使用的关键字。

除了把条目标记为 TODO，还可以在条目中输入‘C-c C-s’（‘org-schedule’）为它附加日期。这会弹出 Emacs 日历（参见“日历与日记”）提示输入日期，然后在标题行下方添加标签‘SCHEDULED’和所选日期。命令‘C-c C-d’（‘org-deadline’）效果相同，只是使用标签‘DEADLINE’。

一旦在 Org 文件中规划了一些 TODO 项，就可以输入‘C-c [’（‘org-agenda-file-to-front’）把该文件加入“agenda 文件”列表。Org 模式的设计让你可以轻松维护多个 agenda 文件，例如用来组织生活中不同方面的事务。agenda 文件列表保存在变量‘org-agenda-files’中。

要查看来自 agenda 文件的项目，输入‘M-x org-agenda’。该命令会提示你想看什么：本周待办事项列表、具有特定关键字的 TODO 项列表等。详细信息参见《Org 手册》的“Agenda Views”。

27.10.2 用 Org 做写作系统

你可能希望把 Org 笔记排版得漂亮些，并为导出和发布做好准备。要导出当前缓冲区，在 Org 缓冲区的任何位置输入‘C-c C-e’（‘org-export-dispatch’）。该命令提示选择导出格式；目前支持的格式包括 HTML、LaTeX、Texinfo、OpenDocument（‘.odt’）、iCalendar、Markdown、man-page 和 PDF。有些格式（如 PDF）需要安装特定的系统工具。

要把多个文件一次导出到某个目录（本地或通过网络），必须通过变量‘org-publish-project-alist’定义项目列表。详见其文档。

Org 支持一套简单的标记方案，用来为导出的文档应用文本格式：

- 这段文字是 /强调的/
- 这段文字是 *粗体*
- 这段文字是 _带下划线的_
- 这段文字使用 =等宽字体=

```
#+begin_quote
``This is a quote.''
#+end_quote
```

```
#+begin_example
This is an example.
#+end_example
```

更多细节参见《Org 手册》的“Exporting”和参见《Org 手册》的“Publishing”。

27.11 TeX 模式

TeX 是 Donald Knuth 编写的强大文本排版程序；与 GNU Emacs 一样，它是自由软件。TeX 格式有几个变体，包括 LaTeX（TeX 的一种简化输入格式）、DocTeX（编写 LaTeX 源文件的一种特殊文件格式，把源码与文档结合起来），以及 SliTeX（LaTeX 的一种过时的特殊形式）(1)。

Emacs 为这些变体各提供一个 TeX 主模式：Plain TeX 模式、LaTeX 模式、DocTeX 模式和 SliTeX 模式。Emacs 通过查看缓冲区的内容来选择适当的模式。（这是通过调用‘tex-mode’命令完成的，通常在访问类 TeX

文件时会自动调用。参见“选择文件模式”。）如果内容不足以判断，Emacs 会选择由变量‘tex-default-mode’指定的模式；其默认值是‘latex-mode’。如果 Emacs 猜错了，你可以用命令‘plain-tex-mode’、‘latex-mode’、‘slitex-mode’或‘doctex-mode’选择正确的 TeX 模式变体。

以下各节介绍 TeX 模式及其变体的功能。另有几个与 TeX 相关的 Emacs 包未在本手册中介绍：

- BibTeX 模式是用于 BibTeX 文件的主模式，这种文件通常用于保存 LaTeX 文档的参考文献。更多信息参见命令‘bibtex-mode’的文档字符串。
- RefTeX 包提供一个次模式，可与 LaTeX 模式配合管理参考文献。参见《《reftex》手册》
- AUCTeX 包为编辑 TeX 及其相关格式提供更高级的功能，包括在 Emacs 缓冲区中预览 TeX 公式的能力。与 BibTeX 模式和 RefTeX 包不同，AUCTeX 默认不随 Emacs 分发。它可以通过 Package Menu 下载（参见“Emacs Lisp 软件包”）；安装后，参见《《auctex》手册》。

> 注：它已被 LaTeX 附带的‘slides’文档类取代。

27.11.1 TeX 编辑命令

- ‘"’：根据上下文插入‘``’、‘''’或‘"’（‘tex-insert-quote’）。
- ‘C-j’：插入分段（两个换行），并检查前一段落是否有不配对的花括号或美元符号（‘tex-terminate-paragraph’）。
- ‘M-x tex-validate-region’：检查区域中每个段落是否有不配对的花括号或美元符号。
- ‘C-c {’：插入‘{}’并把光标点放在两者之间（‘tex-insert-braces’）。
- ‘C-c }’：向前移动越过下一个未匹配的右花括号（‘up-list’）。

在 TeX 中，字符‘"’通常不使用；引文以‘``’开始、以‘''’结束。因此 TeX 模式把‘"’键绑定到‘tex-insert-quote’命令。它在空白或左花括号之后插入‘``’，在反斜杠之后插入‘"’，在任何其他字符之后插入‘"’。

作为特殊的例外，如果在光标点之前的文本是‘``’或‘"’时输入‘"’，Emacs 会把前面的文本替换为单个‘"’字符。因此，如果你确实需要，可以输入‘""’来插入‘"’。（也可以用‘C-q "’插入该字符。）

在 TeX 模式中，‘$’有一种特殊的语法代码，试图理解 TeX 数学模式定界符的匹配方式。当你插入一个用于退出数学模式的‘$’时，进入数学模式的匹配‘$’的位置会显示一秒钟。这与插入右花括号时显示匹配的左花括号是同一个功能。然而，无法判断一个‘$’是进入还是离开数学模式；所以当你插入进入数学模式的‘$’时，会显示前一个‘$’的位置，仿佛它是匹配的，尽管实际上两者无关。

TeX 使用必须配对的花括号作为定界符。有些用户喜欢始终保持花括号平衡，而不是逐个插入。使用‘C-c {’（‘tex-insert-braces’）插入一对花括号。它把光标点留在两个花括号之间，以便插入其中的文本。之后，用命令‘C-c }’（‘up-list’）向前移动越过右花括号。你也可以在标记了一些文本之后调用‘C-c {’：这时命令会把标记的文本括在花括号中。

有两个检查花括号配对的命令。‘C-j’（‘tex-terminate-paragraph’）检查光标点之前的段落，并插入两个换行以开始新段落。如果发现任何不匹配，它会在回显区输出一条消息。‘M-x tex-validate-region’逐段检查一个区域。错误列在‘*Occur*’缓冲区中；你可以在该缓冲区中使用通常的 Occur 模式命令，例如‘C-c C-c’，来访问某个不匹配处（参见“其他搜索与循环命令”）。

注意，在 TeX 模式中，Emacs 命令会把方括号和圆括号也计入配对，而不只是花括号。就检查 TeX 语法而言，这并不严格正确。不过，圆括号和方括号在文本中很可能被用作匹配的定界符，而且让各种移动命令和自动匹配显示对其生效很有用。

27.11.2 LaTeX 编辑命令

LaTeX 模式提供了一些不适用于 plain TeX 的额外功能：

- ‘C-c C-o’：为 LaTeX 块插入‘\begin’和‘\end’，并把光标点放在两者之间的一行上（‘latex-insert-block’）。
- ‘C-c C-e’：关闭最内层尚未关闭的 LaTeX 块（‘latex-close-block’）。

在 LaTeX 输入中，‘\begin’和‘\end’标签用来分组文本块。要插入一个块，输入‘C-c C-o’（‘latex-insert-block’）。它会提示输入块类型，并插入相应的匹配‘\begin’和‘\end’标签，两者之间留一空行并把光标点移到那里。

在给‘C-c C-o’输入块类型参数时，可以使用通常的补全命令（参见“补全”）。默认补全列表包含标准的 LaTeX 块类型。如果你想要额外的块类型用于补全，请自定义列表变量‘latex-block-names’。

在 LaTeX 输入中，‘\begin’和‘\end’标签必须平衡。你可以用‘C-c C-e’（‘latex-close-block’）插入一个与最后一个未匹配‘\begin’匹配的‘\end’标签。它还会缩进‘\end’以与相应的‘\begin’对齐，并且如果光标点在行首，就在‘\end’标签之后插入一个换行。次模式‘latex-electric-env-pair-mode’会在你输入其中一个标签时自动为你插入对应的‘\end’或‘\begin’标签。

27.11.3 TeX 打印命令

你可以把 TeX 作为 Emacs 的子进程调用，向它提供整个缓冲区的内容或只提供其中一部分（例如大部头文档中的某一章）。

- ‘C-c C-b’：对整个当前缓冲区调用 TeX（‘tex-buffer’）。
- ‘C-c C-r’：对当前区域连同缓冲区的头部调用 TeX（‘tex-region’）。
- ‘C-c C-f’：对当前文件调用 TeX（‘tex-file’）。
- ‘C-c C-v’：预览上一次‘C-c C-b’、‘C-c C-r’或‘C-c C-f’命令的输出（‘tex-view’）。
- ‘C-c C-p’：打印上一次‘C-c C-b’、‘C-c C-r’或‘C-c C-f’命令的输出（‘tex-print’）。
- ‘C-c <TAB>’：对当前文件调用 BibTeX（‘tex-bibtex-file’）。
- ‘C-c C-l’：重新居中显示 TeX 输出的窗口，使最后一行可见（‘tex-recenter-output-buffer’）。
- ‘C-c C-k’：杀掉 TeX 子进程（‘tex-kill-job’）。
- ‘C-c C-c’：对整个当前缓冲区调用其他某种编译命令（‘tex-compile’）。

要让当前缓冲区经过 TeX 处理，输入‘C-c C-b’（‘tex-buffer’）。格式化后的输出写入一个临时文件，通常是‘.dvi’文件。之后可以输入‘C-c C-v’（‘tex-view’）启动外部程序（如‘xdvi’）查看该输出文件。也可以输入‘C-c C-p’（‘tex-print’）打印输出文件的硬拷贝。

默认情况下，‘C-c C-b’在当前目录中运行 TeX。TeX 的输出也在该目录中生成。要在其他目录中运行 TeX，可把变量‘tex-directory’改为所需的目录。如果你的环境变量‘TEXINPUTS’包含相对名称，或者你的文件含有带相

对文件名的‘\input’命令，那么‘tex-directory’**必须是**‘"."’，否则会得到错误的结果。否则，可以安全地指定其他目录，例如‘"/tmp"’。

缓冲区的 TeX 变体决定‘C-c C-b’实际运行哪个 shell 命令。在 Plain TeX 模式中，它由变量‘tex-run-command’指定，默认值为‘"tex"’。在 LaTeX 模式中，由‘latex-run-command’指定，默认值为‘"latex"’。‘C-c C-v’为查看 .dvi 输出而运行的 shell 命令由变量‘tex-dvi-view-command’决定，与 TeX 变体无关。‘C-c C-p’为打印输出而运行的 shell 命令由变量‘tex-dvi-print-command’决定。变量‘tex-print-file-extension’可设置为查看和打印 TeX 编译文件所需的文件扩展名。例如，可以把它设为‘.pdf’，并相应地更新‘tex-dvi-view-command’和‘tex-dvi-print-command’，以及‘latex-run-command’或‘tex-run-command’。

通常，Emacs 会自动把输出文件名追加到上一段所述的 shell 命令字符串之后。例如，如果‘tex-dvi-view-command’是‘"xdvi"’，那么‘C-c C-v’会运行‘xdvi OUTPUT-FILE-NAME’。但在某些情况下，文件名需要嵌入命令之中，例如需要把文件名作为某个命令的参数、而该命令的输出又要管道给另一个命令时。可以用命令字符串中的‘*’指定文件名放在哪里。例如：

```
(setq tex-dvi-print-command "dvips -f * | lpr")
```

TeX 的终端输出（包括任何错误消息）出现在名为‘*tex-shell*’的缓冲区中。如果 TeX 出错，你可以切换到该缓冲区并向它输入（工作方式与 Shell 模式相同；参见“交互式子 shell”）。无需切换到该缓冲区，也可以输入‘C-c C-l’滚动它，使其最后一行可见。

如果发现 TeX 的输出不再有用，输入‘C-c C-k’（‘tex-kill-job’）杀掉 TeX 进程。使用‘C-c C-b’或‘C-c C-r’也会杀掉仍在运行的 TeX 进程。

你也可以输入‘C-c C-r’（‘tex-region’）让任意区域经过 TeX 处理。不过这样做很棘手，因为大多数 TeX 输入文件开头都含有设置参数和定义宏的命令，没有这些命令，文件后面的部分就无法正确排版。为解决这个问题，‘C-c C-r’允许你指定文件的一部分作为必要命令；这部分会在指定区域之前作为 TeX 输入的一部分被包含进来。文件中指定的这部分称为“头部”（header）。

在 Plain TeX 模式中，要标明头部的边界，需在文件中插入两个特殊字符串。在头部之前插入‘%**start of header’，在其后插入**‘%end of header’。每个字符串必须完全位于一行，但该行前后可以有其他文本。包含这两个字符串的行本身也包含在头部中。如果‘%**start of header’没有出现在缓冲区前 100 行内，‘C-c C-r’就假定没有头部。

在 LaTeX 模式中，头部以‘\documentclass’或‘\documentstyle’开始，以‘\begin{document}’结束。这些是 LaTeX 无论如何都要求使用的命令，因此无需特别处理来标识头部。

命令（‘tex-buffer’）和（‘tex-region’）都在临时目录中完成全部工作，无法使用 TeX 做交叉引用所需的任何辅助文件；这些命令通常不适合用来生成最终副本，因为最终副本要求所有交叉引用都正确。

当你需要交叉引用所需的辅助文件时，使用‘C-c C-f’（‘tex-file’），它会针对当前缓冲区所属的文件、在该文件所在的目录中运行 TeX。运行 TeX 之前，它会提议保存任何已修改的缓冲区。一般来说，需要执行两次（‘tex-file’）才能让交叉引用正确。

变量‘tex-start-options’的值指定 TeX 运行的选项。

变量‘tex-start-commands’的值指定启动 TeX 的 TeX 命令。默认值使 TeX 以非停止模式运行。要以交互方式运行 TeX，请把该变量设为“""”。

大型 TeX 文档常被拆成几个文件——一个主文件加上若干子文件。对子文件运行 TeX 通常不行；必须在主文件上运行。为了使‘tex-file’在编辑子文件时仍然有用，可以把变量‘tex-main-file’设为主文件名。这样‘tex-file’就会对该文件运行 TeX。

使用‘tex-main-file’最方便的方法是在每个子文件的局部变量列表中指定它。参见“文件中的局部变量”。

对于 LaTeX 文件，可以使用 BibTeX 处理当前缓冲区所属文件的辅助文件。BibTeX 在数据库中查找文献引用，并为参考文献部分准备所引用的文献。命令‘C-c <TAB>’（‘tex-bibtex-file’）运行 shell 命令（‘tex-bibtex-command’），为当前缓冲区所属的文件生成‘.bbl’文件。通常需要先执行一次‘C-c C-f’（‘tex-file’）生成‘.aux’文件，然后执行‘C-c <TAB>’（‘tex-bibtex-file’），再重复‘C-c C-f’（‘tex-file’）两次以使交叉引用正确。

要对当前 TeX 缓冲区调用其他某种编译程序，输入‘C-c C-c’（‘tex-compile’）。该命令知道如何向许多常见程序传递参数，包括‘pdflatex’、‘yap’、‘xdvi’和‘dvips’。可以用标准补全键选择所需的编译程序（参见“补全”）。

27.11.4 TeX 模式杂项

进入任何 TeX 模式变体都会运行钩子‘text-mode-hook’和‘tex-mode-hook’。然后它会运行‘plain-tex-mode-hook’、‘doctex-mode-hook’、‘latex-mode-hook’或‘slitex-mode-hook’中适当的那个。启动 TeX shell 会运行钩子‘tex-shell-hook’。参见“钩子”。

命令 ‘M-x iso-iso2tex’、‘M-x iso-tex2iso’、‘M-x iso-iso2gtex’ 和 ‘M-x iso-gtex2iso’ 可用于在 Latin-1 编码的文件与相应的 TeX 编码文件之间相互转换。

27.12 SGML 与 HTML 模式

用于 SGML 和 HTML 的主模式提供缩进支持，以及操作标签的命令。

HTML 由两种模式组成——一种是名为 ‘html-mode’ 的基本模式，它是 SGML 模式经过少许定制的变体。另一种名为 ‘mhtml-mode’，HTML 文件默认使用它，它力图正确处理包含在 ‘<script>’ 元素中的 Javascript 以及嵌入在 ‘<style>’ 元素中的 CSS。

- ‘C-c C-n’ 交互式指定一个特殊字符，并插入代表该字符的 SGML ‘&’ 命令（‘sgml-name-char’）。
- ‘C-c C-t’ 交互式指定一个标签及其属性（‘sgml-tag’）。该命令向你询问标签名和各个属性的值，然后同时插入起始标签和结束标签，并把光标点留在两者之间。

若带前缀参数 N，该命令把标签包住缓冲区中光标点之后已有的 N 个单词。当区域处于活动状态时，它把标签包住该区域（Transient Mark 模式关闭时，需提供数值参数 −1 才会这样做）。

- ‘C-c C-a’ 交互式为当前标签插入属性值（‘sgml-attributes’）。
- ‘C-c C-f’ 向前跳过一组配对的标签（从起始标签到与之对应的结束标签）（‘sgml-skip-tag-forward’）。数值参数用作重复次数。
- ‘C-c C-b’ 向后跳过一组配对的标签（从起始标签到与之对应的结束标签）（‘sgml-skip-tag-backward’）。数值参数用作重复次数。

- ‘C-c C-d’ 删除光标点处或其后方的标签，并同时删除与之匹配的标签（‘sgml-delete-tag’）。若光标点处或其后的标签是起始标签，则同时删除结束标签；若它是结束标签，则同时删除起始标签。
- ‘C-c ? TAG <RET>’ 显示标签 TAG 含义的说明（‘sgml-tag-help’）。若参数 TAG 为空，则描述光标点处的标签。
- ‘C-c /’ 为最内层尚未闭合的标签插入一个结束标签（‘sgml-close-tag’）。若在标签或注释内部调用，则改为关闭该标签或注释，而不插入结束标签。
- ‘C-c 8’ 切换一个次模式：在该模式下，输入 Latin-1 字符时插入代表该字符的相应 SGML 命令，而不是字符本身（‘sgml-name-8bit-mode’）。
- ‘C-c C-v’ 运行一条 shell 命令（须由你指定），以检验当前缓冲区是否为合法的 SGML（‘sgml-validate’）。（在 HTML 模式下，此键序列运行另一条命令。）
- ‘C-c <TAB>’ 切换缓冲区中已有标签的可见性。这可以当作一种简易的预览手段（‘sgml-tags-invisible’）。

用于编辑 XML 文档的主模式称为 nXML 模式。这是一种功能强大的主模式，它能识别许多已有的 XML schema，并借助它们通过 ‘M-<TAB>’ 提供 XML 元素的补全，还能在输入时动态检验 XML 并以高亮标出错误。要在已有缓冲区中启用 nXML 模式，键入 ‘M-x nxml-mode’，或者等价地键入 ‘M-x xml-mode’。对于扩展名为 ‘.xml’ 的文件，Emacs 使用 nXML 模式。对于扩展名为 ‘.xhtml’ 的 XHTML 文件，Emacs 默认使用 HTML 模式；你可以通过定制变量 ‘auto-mode-alist’ 让它改用 nXML 模式（参见“选择文件模式”）。nXML 模式在其自己的手册中描述：参见《nXML 手册》

你也可以选择用功能较弱的 SGML 模式来编辑 XML，因为 XML 是 SGML 的严格子集。要在已有缓冲区中启用 SGML 模式，键入 ‘M-x sgml-mode’。启用 SGML 模式时，Emacs 会检查该缓冲区以判断它是否为 XML；若是，则把变量 ‘sgml-xml-mode’ 设为非 ‘nil’ 值。这会使上面所述的 SGML 模式标签插入命令总是同时插入显式的结束标签。

27.13 Nroff 模式

Nroff 模式是一种派生自文本模式的主模式，专用于编辑 nroff 文件（如 Unix man 手册页）。键入 ‘M-x nroff-mode’ 进入该模式。进入 Nroff 模式会运行钩子 ‘text-mode-hook’，然后运行 ‘nroff-mode-hook’（参见“钩子”）。

在 Nroff 模式中，nroff 命令行被当作段落分隔符，页面由 ‘.bp’ 命令分隔，注释以反斜杠加双引号开头。该模式还定义了以下命令：

‘M-n’ 移到下一条不是 nroff 命令的行首（‘nroff-forward-text-line’），参数为重复次数。‘M-p’ 与 ‘M-n’ 类似，但向上移动（‘nroff-backward-text-line’）。‘M-?’ 在回显区显示区域中文本行（即不是 nroff 命令的行）的行数（‘nroff-count-text-lines’）。

Electric Nroff 模式是一种可与 Nroff 模式配合使用的缓冲区局部次模式。要切换该次模式，键入 ‘M-x nroff-electric-mode’（参见“次模式”）。该模式开启时，每当你键入 <RET> 结束一个含有开启某种分组结构的 nroff 命令的行，用于关闭该分组的 nroff 命令就会自动插入到下一行。

若把提纲次模式与 Nroff 模式一起使用（参见“大纲模式”），则标题行是形如 ‘.H’ 后跟一个数字（标题层级）的行。

27.14 富文本

富文本模式（Enriched mode）是一种以所见即所得（WYSIWYG，What You See Is What You Get）方式编辑格式化文本文件的次模式。启用富文本模式后，你可以给缓冲区中的文本施加各种格式化属性，如字体和颜色；保存缓冲区时，这些属性会随文本一起保存，使用 MIME 的 'text/enriched' 文件格式。

富文本模式通常与文本模式配合使用（参见"文本模式"）。它_不_兼容 Font Lock 模式，后者被许多主模式（包括大多数编程语言模式）用于语法高亮（参见"Font Lock 模式"）。与富文本模式不同，Font Lock 模式根据当前缓冲区内容自动设置文本属性；那些属性不会被保存到磁盘。

Emacs 的 'data-directory' 中的文件 'enriched.txt' 可用作富文本模式各项特性的示例。

- Enriched Justification：居中、使文本与左边距或右边距对齐等。
- Enriched Properties："特殊文本属性"子菜单。

27.14.1 富文本模式

富文本模式是一种缓冲区局部次模式（参见"次模式"）。当你访问一个以 'text/enriched' 格式保存的文件时，Emacs 会自动启用富文本模式，并把文件中的格式化信息应用到缓冲区文本上。保存启用了富文本模式的缓冲区时，它会以 'text/enriched' 格式保存，其中包含格式化信息。

要创建新的格式化文本文件，访问一个不存在的文件并键入 'M-x enriched-mode'。该命令实际上是切换富文本模式。带前缀参数时，若参数为正则启用富文本模式，否则禁用它。若禁用富文本模式，Emacs 就不再以 'text/enriched' 格式保存该缓冲区；已经加到缓冲区上的格式化属性仍留在缓冲区中，但不会被保存到磁盘。

富文本模式并不保存 Emacs 的全部文本属性，只保存变量 'enriched-translations' 中指定的那些属性。这些属性包括用于字体、颜色、缩进和对齐方式的属性。

若你访问某个文件而 Emacs 未能识别出它是 'text/enriched' 格式，键入 'M-x format-decode-buffer'。该命令提示输入一种文件格式，并以该格式重新读取文件。指定 'text/enriched' 格式会自动启用富文本模式。

要以原始形式查看 'text/enriched' 文件（即带标记标签的纯文本，而非格式化后的文本），使用 'M-x find-file-literally'（参见"访问文件"）。

关于 Emacs 如何识别并转换 'text/enriched' 之类的文件格式的细节，参见《Emacs Lisp 参考手册》的"Format Conversion, for details of how Emacs recognizes and converts file formats like 'text/enriched'"；关于文本属性的更多信息，参见《Emacs Lisp 参考手册》的"Text Properties"。

27.14.2 硬换行与软换行

在富文本模式中，Emacs 区分两种不同的换行："硬"换行和"软"换行。你也可以在其他缓冲区中通过键入 'M-x use-hard-newlines' 启用或禁用这一特性。

硬换行用于分隔段落，或用于任何无论文本如何填充都必须断行的位置；软换行则用于填充。 <RET>（'newline'）和 'C-o'（'open-line'）命令插入硬换行。各种填充命令（包括自动填充，参见"自动填充模式"）只插入软换行，也只删除软换行，不会动硬换行。

因此，在用富文本模式编辑时，不应使用 <RET> 或 'C-o' 在已填充段落的中间断行。应改用自动填充模式或显式的填充命令（参见"显式填充命令"）。在换行应始终保留的地方（如表格和列表中），才使用 <RET> 或

‘C-o’。对于这类行，你可能还想把对齐方式设为 ‘unfilled’（参见“富文本中的对齐”）。

27.14.3 编辑格式信息

修改属性最简单的方法是用 ‘Text Properties’ 菜单。你可以从菜单栏的 ‘Edit’ 菜单进入该菜单（参见“菜单栏”），也可以按 ‘C-mouse-2’（参见“打开菜单的鼠标点击”）。‘Text Properties’ 菜单中的部分命令列在下面（你也可以用 ‘M-x’ 调用它们）：

- ‘Remove Face Properties’ 从区域中移除字体（face）属性（‘facemenu-remove-face-props’）。
- ‘Remove Text Properties’ 从区域中移除所有文本属性，包括字体属性（‘facemenu-remove-all’）。
- ‘Describe Properties’ 列出光标点之后那个字符的所有文本属性及其他信息（‘describe-text-properties’）。
- ‘Display Faces’ 显示已定义字体的列表（‘list-faces-display’）。参见“文本字体”。
- ‘Display Colors’ 显示已定义颜色的列表（‘list-colors-display’）。参见“字体的颜色”。

其他菜单项在下面各节中描述。

27.14.4 富文本中的字体

以下命令可用于添加或移除字体（face）（参见“文本字体”）。若标记处于活动状态，每条命令都作用于区域中的文本；若标记不活动，则作用于下一个自插入字符。带前缀参数时，即使区域处于活动状态，每条命令也作用于下一个自插入字符。

- ‘M-o d’ 移除所有 ‘face’ 属性（‘facemenu-set-default’）。
- ‘M-o b’ 应用 ‘bold’ 字体（‘facemenu-set-bold’）。
- ‘M-o i’ 应用 ‘italic’ 字体（‘facemenu-set-italic’）。
- ‘M-o l’ 应用 ‘bold-italic’ 字体（‘facemenu-set-bold-italic’）。
- ‘M-o u’ 应用 ‘underline’ 字体（‘facemenu-set-underline’）。
- ‘M-o o FACE <RET>’ 应用字体 FACE（‘facemenu-set-face’）。
- ‘M-x facemenu-set-foreground’ 提示输入一种颜色（参见“字体的颜色”），并将其作为前景色应用。
- ‘M-x facemenu-set-background’ 提示输入一种颜色，并将其作为背景色应用。

这些命令也可以通过 Text Properties 菜单使用。

自插入字符通常会从缓冲区中前一个字符继承字体属性（以及大多数其他文本属性）。若你用上述某条命令为下一个自插入字符指定字体，该字符不会继承前一个字符的字体属性，但仍会继承其他文本属性。

富文本模式定义了两个额外的字体：‘excerpt’ 和 ‘fixed’。它们对应于 text/enriched 文件格式中使用的编码。‘excerpt’ 字体意在用于引文；默认情况下它显示得与 ‘italic’ 一样。‘fixed’ 字体指定定宽文本；默认情况下它显示得与 ‘bold’ 一样。

27.14.5 富文本中的缩进

在富文本模式中，你可以为段落或段落的一部分的左、右边距指定不同的缩进量。这些边距也会影响像 ‘M-q’ 这样的填充命令（参见“填充文本”）。

Text Properties 的 Indentation 子菜单提供了指定缩进的命令：

- ‘Indent More’ 把区域缩进 4 列（‘increase-left-margin’）。在富文本模式中，该命令也可用 ‘C-x <TAB>’ 调用；若提供数值参数，它表示要给边距增加多少列（负参数则减少列数）。
- ‘Indent Less’ 从区域中去掉 4 列缩进。
- ‘Indent Right More’ 在右边距缩进 4 列，使文本变窄。
- ‘Indent Right Less’ 从右边距去掉 4 列缩进。

变量 ‘standard-indent’ 指定这些命令应当增加或减少多少列缩进，默认值为 4。富文本模式默认的右边距照常由变量 ‘fill-column’ 控制。

你也可以键入 ‘C-c [’（‘set-left-margin’）和 ‘C-c]’（‘set-right-margin’）来设置左、右边距。可以用数值参数指定边距宽度；否则这些命令通过迷你缓冲区提示输入一个值。

填充前缀（如果有）在指定的段落缩进之外另行起作用：‘C-x .’ 不会把指定缩进的空白算入填充前缀的新值，而填充命令在每一行上查找缩进之后的填充前缀。参见“填充前缀”。

27.14.6 富文本中的对齐

在富文本模式中，你可以使用以下命令为填充指定各种“对齐方式”（justification style）。这些命令作用于包含光标点的段落；若区域处于活动状态，则作用于与区域重叠的所有段落。

- ‘M-j l’ 使各行与左边距对齐（‘set-justification-left’）。
- ‘M-j r’ 使各行与右边距对齐（‘set-justification-right’）。
- ‘M-j b’ 使各行与两端边距都对齐，为此在行中间插入空格（‘set-justification-full’）。
- ‘M-j c’ ‘M-S’ 使各行在边距之间居中（‘set-justification-center’）。
- ‘M-j u’ 完全关闭填充（‘set-justification-none’）。对具有此设置的文本，填充命令不起作用。不过你仍然可以缩进左边距。

你也可以用 Text Properties 菜单中的 Justification 子菜单指定对齐方式。默认的对齐方式由缓冲区局部变量 ‘default-justification’ 指定。其值应是符号 ‘left’、‘right’、‘full’、‘center’ 或 ‘none’ 之一；它们的含义与上述命令相对应。

27.14.7 设置其他文本属性

Text Properties 的 Special Properties 子菜单中有添加或移除另外四种文本属性的条目：‘read-only’（禁止改动文本）、‘invisible’（隐藏文本）、‘intangible’（禁止在文本中移动光标点）和 ‘charset’（对于为显示某个字符选择合适的字体很重要）。‘Remove Special’ 菜单项从区域中的文本上移除所有这些特殊属性。

‘invisible’ 和 ‘intangible’ 属性不会被保存。

富文本模式还支持保存和恢复 ‘display’ 属性（参见《Emacs Lisp 参考手册》的“Display Property”），这些属性影响文本在屏幕上的显示方式，还允许显示来自缓冲区文本之外来源的图片和字符串。‘display’ 属性还支持在处理该属性以便显示时执行任意 Lisp 形式，从而提供了一种手段，可以动态地针对只有在显示时才能知道的条件来调整显示。由于执行任意 Lisp 会使 Emacs 面临潜在攻击，尤其当富文本的来源在 Emacs 之外、甚至在你的系统之外时（例如，它来自收到的电子邮件消息），这种执行在富文本模式中默认被禁用。你可以通过把变量 ‘enriched-allow-eval-in-display-props’ 定制为非 ‘nil’ 值来启用它。

27.15 编辑基于文本的表格

‘table’ 包提供的命令可以方便地编辑基于文本的表格。下面是这样一张表格的示例：

```
+-----------------+--------------------------------+-----------------+
|     Command     |          Description           |   Key Binding   |
+-----------------+--------------------------------+-----------------+
|  forward-char   |Move point right N characters   |       C-f       |
|                 |(left if N is negative).        |                 |
|                 |                                |                 |
+-----------------+--------------------------------+-----------------+
|  backward-char  |Move point left N characters    |       C-b       |
|                 |(right if N is negative).       |                 |
|                 |                                |                 |
+-----------------+--------------------------------+-----------------+
```

当 Emacs 认出这样一段文本是表格时（参见“表格识别”），只要单元格内容变得太大而放不下，编辑各个单元格的内容就会自动调整表格大小。你可以使用下面各节中定义的命令来浏览和编辑表格布局。

键入 ‘M-x table-fixed-width-mode’ 可切换表格自动调整大小这一特性。

27.15.1 什么是基于文本的表格?

“表格”由一块矩形文本区域构成，该区域被划分为若干“单元格”。每个单元格的宽度和高度至少为一个字符，边框线不计在内。一个单元格可以再细分为更多单元格，但这些单元格不能重叠。

单元格的边框线用三个特殊字符绘制，它们由以下变量指定：

- ‘table-cell-vertical-char’ 用于竖线的字符，默认为 ‘|’。
- ‘table-cell-horizontal-chars’ 用于横线的字符，默认为 “"-="”。
- ‘table-cell-intersection-char’ 用于横线与竖线交点的字符，默认为 ‘+’。

下面是*不合法*表格的示例：

```
+-----+       +--+    +-++--+
|     |       |  |    | ||  |
|     |       |  |    | ||  |
+--+  |    +--+--+    +-++--+
|  |  |    |  |  |    +-++--+
|  |  |    |  |  |    | ||  |
+--+--+    +--+--+    +-++--+
   a          b          c
```

从左到右：

1. 不允许单元格重叠或单元格非矩形。
2. 边框必须是矩形。
3. 单元格的宽度和高度至少为一个字符。

27.15.2 创建表格

要从头创建一张基于文本的表格，键入‘M-x table-insert’。该命令提示输入表格列数、表格行数、单元格宽度和单元格高度。单元格宽度和高度不包括单元格边框；两者都可以指定为一个整数（表示每个单元格都取相同的宽度/高度），或指定为以空格或逗号分隔的整数序列（分别指定各表格列/行的宽度/高度，表格列从左到右数，表格行从上到下数）。随后，指定的表格被插入到光标点处。

由‘M-x table-insert’插入的表格带有特殊的文本属性，它们告诉 Emacs 要把它特别当作基于文本的表格来处理。如果你把缓冲区保存到文件并在以后再次访问它，这些属性就会丢失，表格在 Emacs 看来只是一段普通文本。如何把它重新转换为表格，见下一节。

27.15.3 表格识别

缓冲区中已有的基于文本的表格若缺少‘M-x table-insert’所施加的特殊文本属性，就不会被特别当作表格处理。要施加这些文本属性，键入‘M-x table-recognize’。该命令扫描当前缓冲区，“识别”出合法的表格单元格，并施加相关的文本属性。反之，键入‘M-x table-unrecognize’可“取消识别”当前缓冲区中的所有表格，移除那些特殊文本属性，使表格重新变回纯文本。

你也可以用以下命令有选择地识别或取消识别表格：

- ‘M-x table-recognize-region’识别当前区域内的表格。
- ‘M-x table-unrecognize-region’取消识别当前区域内的表格。
- ‘M-x table-recognize-table’识别光标点处的表格并使其生效。
- ‘M-x table-unrecognize-table’使光标点处的表格失效。
- ‘M-x table-recognize-cell’识别光标点处的单元格并使其生效。
- ‘M-x table-unrecognize-cell’使光标点处的单元格失效。

识别表格的另一种方法，参见“纯文本与表格之间的转换”。

27.15.4 表格单元格命令

命令‘M-x table-forward-cell’和‘M-x table-backward-cell’把光标点从当前单元格移到相邻单元格。次序是循环的：当光标点位于表格的最后一个单元格时，‘M-x table-forward-cell’移到第一个单元格；同样，当光标点位于第一个单元格时，‘M-x table-backward-cell’移到最后一个单元格。

‘M-x table-span-cell’提示输入一个方向——右、左、上或下——并把当前单元格与该方向上的相邻单元格合并。若合并会导致不合法的单元格布局，该命令会报错。

‘M-x table-split-cell’把当前单元格纵向或横向拆分，并通过迷你缓冲区提示输入方向。要按指定方向拆分，可使用‘M-x table-split-cell-vertically’和‘M-x table-split-cell-horizontally’。纵向拆分时，原单元格的内容会自动分到两个新单元格中。横向拆分时，若单元格非空，会提示你如何分配单元格内容，选项有‘split’（在光标点处分割内容）、‘left’（把所有内容放入左边单元格）和‘right’（把所有内容放入右边单元格）。

以下命令扩大或缩小单元格。默认情况下，它们按一行或一列调整大小；若提供数值参数，则该参数指定按多少行或多少列调整。

- ‘M-x table-heighten-cell’：垂直增大当前单元格。

- ‘M-x table-shorten-cell’：垂直缩小当前单元格。
- ‘M-x table-widen-cell’：水平加宽当前单元格。
- ‘M-x table-narrow-cell’：水平收窄当前单元格。

27.15.5 单元格对齐

‘M-x table-justify’ 命令对基于文本的表格中的一个或多个单元格施加“对齐”（justification）。对齐方式决定单元格中的文本相对于单元格边缘如何排列。表格中的每个单元格都可以单独设置对齐方式。

‘M-x table-justify’ 首先询问要对什么设置对齐；选项有 ‘cell’（只对当前单元格）、‘column’（当前表格列中的所有单元格）和 ‘row’（当前表格行中的所有单元格）。然后该命令询问对齐样式；选项有 ‘left’、‘center’、‘right’、‘top’、‘middle’、‘bottom’ 或 ‘none’（表示不做垂直对齐）。

水平和垂直对齐样式是分别指定的，而且两种样式可以同时生效；例如，你可以调用 ‘M-x table-justify’ 两次，一次指定 ‘right’ 对齐，一次指定 ‘bottom’ 对齐，从而把某个单元格的内容对齐到右下角。

对齐样式以文本属性的形式保存在缓冲区中，当你删除缓冲区或退出 Emacs 时就会丢失。不过，表格识别命令（例如 ‘M-x table-recognize’，参见“表格识别”）会通过检查单元格的内容，尝试判定并重新应用每个单元格的对齐样式。要禁用此特性，请把变量 ‘table-detect-cell-alignment’ 改为 ‘nil’。

27.15.6 表格的行与列

‘M-x table-insert-row’ 在当前表格行之前插入一行单元格。当前行连同光标点一起被下推到新行之下。要在表格最底下一行之后插入一行，请在表格下方、紧靠底边之下的位置调用此命令。使用数字前缀参数可以一次插入多行。

类似地，‘M-x table-insert-column’ 在当前表格列的左侧插入一列单元格。要在最右列的右侧插入一列，请把光标点放在最右列右侧、表格之外的位置调用此命令。数字前缀参数指定要插入的列数。

‘M-x table-delete-column’ 删除光标点处的那一列单元格。类似地，‘M-x table-delete-row’ 删除光标点处的那一行单元格。给这两个命令加数字前缀参数即可指定要删除的列数或行数。

27.15.7 纯文本与表格之间的转换

‘M-x table-capture’ 命令捕获区域中的纯文本，把它变成表格。与 ‘M-x table-recognize’（参见“表格识别”）不同，原始文本不必具有表格的外观；它只需具有逻辑上的类表格结构。

例如，假设我们有下面这些数字，它们分成三行，并用逗号水平分隔：

```
1, 2, 3, 4
5, 6, 7, 8
, 9, 10
```

对该文本调用 ‘M-x table-capture’ 会生成下面这个表格：

```
+-----+-----+-----+-----+
|1    |2    |3    |4    |
+-----+-----+-----+-----+
|5    |6    |7    |8    |
```

```
+-----+-----+-----+-----+
|     |9    |10   |     |
+-----+-----+-----+-----+
```

‘M-x table-release’ 做相反的事：它把表格转换回纯文本，去掉单元格边框。

这对命令的一个应用场合是按版面编辑文本。看看下面三个段落：

```
table-capture is a powerful command.
Here are some things it can do:
```

```
Parse Cell Items   Using row and column delimiter regexps,
                   it parses the specified text area and
                   extracts cell items into a table.
```

把 ‘table-capture’ 用于包含上述文本的区域，并把列分隔符和行分隔符的正则表达式都设为空字符串，就会创建一个只含一个单元格的表格，如下所示。

```
+-------------------------------------------------------------+
|table-capture is a powerful command.                         |
|Here are some things it can do:                              |
|                                                             |
|Parse Cell Items   Using row and column delimiter regexps,|
|                   it parses the specified text area and  |
|                   extracts cell items into a table.      |
+-------------------------------------------------------------+
```

然后我们可以用拆分单元格的命令（参见“表格单元格命令”）细分该表格，使每个段落各占一个单元格：

```
+-------------------------------------------------------------+
|table-capture is a powerful command.                         |
|Here are some things it can do:                              |
+-----------------+-------------------------------------------+
|Parse Cell Items | Using row and column delimiter regexps,|
|                 | it parses the specified text area and  |
|                 | extracts cell items into a table.      |
+-----------------+-------------------------------------------+
```

现在每个单元格都可以独立编辑，而不影响其他单元格的版面。完成后，我们可以调用 ‘M-x table-release’ 把表格转换回纯文本。

27.15.8 表格杂项

‘table-query-dimension’ 命令报告表格以及光标点所在单元格的布局。下面是其输出示例：

```
Cell: (21w, 6h), Table: (67w, 16h), Dim: (2c, 3r), Total Cells: 5
```

这表明当前单元格宽 21 个字符、高 6 行，表格宽 67 个字符、高 16 行，有 2 列、3 行，共 5 个单元格。

‘M-x table-insert-sequence’ 遍历表格的各个单元格，一边走一边把一串文本字符串插入每个单元格。它询问该序列的基础字符串，然后通过“递增”基础字符串来生成序列：若基础字符串以数字字符结尾，就按数值递增，

否则按 ASCII 顺序递增。除基础字符串外，该命令还会询问序列中的元素个数、增量、单元格间隔，以及每个单元格中文本的对齐方式。

‘M-x table-generate-source’ 生成按特定标记语言排版的表格。它询问语言（必须是 ‘html’、‘latex’、‘cals’、‘wiki’ 或 ‘mediawiki’ 之一）、用于存放结果的目标缓冲区以及表格标题，然后把生成的表格插入指定的缓冲区。默认的目标缓冲区是 ‘table.LANG’，其中的 LANG 是你指定的语言。

27.16 双栏编辑

双栏模式（Two-column mode）让你方便地编辑并排的两栏文本。它使用两个并排的窗口，每个窗口显示自己的缓冲区。进入双栏模式有三种方式：

- ‘<F2> 2’ 或 ‘C-x 6 2’：进入双栏模式，当前缓冲区在左边，右边是一个名字基于当前缓冲区名的缓冲区（‘2C-two-columns’）。若右边的缓冲区尚不存在，则它一开始是空的；当前缓冲区的内容不会改变。

当当前缓冲区为空，或只含一栏而你希望再加上一栏时，适合使用此命令。

- ‘<F2> s’ 或 ‘C-x 6 s’：把当前含有双栏文本的缓冲区拆分成两个缓冲区，并排显示（‘2C-split’）。当前缓冲区成为左边的缓冲区，但右栏中的文本被移到右边的缓冲区。当前栏指定拆分点。拆分从当前行开始，一直进行到缓冲区末尾。

当你有一个已经含有双栏文本的缓冲区，并希望暂时把两栏分开时，适合使用此命令。

- ‘<F2> b BUFFER <RET>’ ‘C-x 6 b BUFFER <RET>’：以当前缓冲区作为左边的缓冲区，以缓冲区 BUFFER 作为右边的缓冲区，进入双栏模式（‘2C-associate-buffer’）。

‘<F2> s’ 或 ‘C-x 6 s’ 会寻找栏分隔符，即出现在每行两栏之间的那个字符串。你可以用给 ‘<F2> s’ 的数字参数指定分隔符的宽度；光标点之前的那么多个字符就构成分隔符字符串。默认宽度为 1，因此栏分隔符就是光标点之前的那个字符。

当某一行的分隔符位于适当位置时，‘<F2> s’ 把分隔符之后的文本放进右边的缓冲区，并删除分隔符。分隔符不在适当位置的行则保持不拆分；它们留在左边的缓冲区，而右边的缓冲区得到一个空行与之对应。（在双栏模式下，若要写一行横跨两栏的内容，办法就是：在左边的缓冲区中写它，并在右边的缓冲区中放一个空行。）

‘<F2> <RET>’ 或 ‘C-x 6 <RET>’（‘2C-newline’）命令在两个缓冲区的对应位置各插入一个换行符。这是在拆分缓冲区中编辑双栏文本时添加新行的最简便方法。

当你按自己的意愿编辑好两个缓冲区之后，用 ‘<F2> 1’ 或 ‘C-x 6 1’（‘2C-merge’）合并它们。这会把右边缓冲区中的文本作为第二栏复制到另一个缓冲区中。要回到双栏编辑，请使用 ‘<F2> s’。

用 ‘<F2> d’ 或 ‘C-x 6 d’ 解除两个缓冲区的关联，让它们各自保持原样（‘2C-dissociate’）。若另一个缓冲区（即你键入 ‘<F2> d’ 时不是当前缓冲区的那个）为空，‘<F2> d’ 会删除它。

28 编辑程序

本章介绍 Emacs 中便于编辑程序的功能。这些功能可以做的事情包括：

- 查找或越过顶层定义（参见“顶层定义（defun）”）。
- 应用该语言惯用的缩进约定（参见“程序的缩进”）。
- 平衡圆括号（参见“编辑括号的命令”）。
- 插入、删除或对齐注释（参见“操作注释”）。
- 高亮程序语法（参见“Font Lock 模式”）。

- Defuns：操作程序中主要顶层部分的命令。
- Program Indent：调整缩进以显示嵌套。
- Parentheses：操作圆括号的命令。
- Comments：插入、删除和对齐注释。
- Documentation：获取你打算调用的函数的文档。
- Hideshow：有选择地显示代码块。
- Symbol Completion：补全程序或语言中的符号名。
- MixedCase Words：处理 identifiersLikeThis 这类标识符。
- Semantic：一套基于源代码解析的编辑工具。
- Misc for Programs：其他对编辑程序有用的 Emacs 功能。
- C Modes：C、C++、Objective-C、Java、IDL、Pike 和 AWK 模式的特殊命令。
- Asm Mode：汇编模式及其特殊功能。
- Fortran：Fortran 模式及其特殊功能。

28.1 编程语言的主模式

Emacs 为许多编程语言提供了专门的主模式（参见“主模式”）。编程语言模式通常规定表达式的语法、惯用的缩进规则、如何为该语言做语法高亮，以及如何找到函数定义的开始或结束。它往往还具有编译和调试程序的功能。每种语言的主模式都以该语言命名；例如，C 编程语言的主模式是‘c-mode’。

Emacs 为下列语言提供编程语言模式：Lisp、Scheme、基于 Scheme 的 DSSSL 表达式语言、Ada、ASM、AWK、C、C++、C#、Elixir、Fortran、Icon、IDL（CORBA）、HEEx、Java、Javascript、Lua、M4、Makefile、Metafont（TeX 附带的字体创建工具）、Modula2、Object Pascal、Objective-C、Octave、Pascal、Perl、PHP、Pike、PostScript、Prolog、Python、Ruby、Simula、SQL、Tcl、TypeScript、Verilog 和 VHDL。Perl 推荐的模式称为 CPerl 模式。此外还有用于常见 GNU 和 Unix shell 的脚本语言，以及 MS-DOS/MS-Windows ‘BAT’ 文件、JSON、DNS 主文件、CSS（层叠样式表）、Dockerfile、CMake 文件和各种配置文件的模式。

理想情况下，Emacs 应为你可能想编辑的每种编程语言都提供一个主模式。如果它没有你喜爱的语言的模式，该模式可能实现在某个未随 Emacs 分发的包中（参见“Emacs Lisp 软件包”）；你也可以贡献一个。

如果 Emacs 在编译时加入了 'tree-sitter' 库，它会提供若干基于该库的可选编辑模式，这些模式利用 'tree-sitter' 提供的增量解析能力。这些模式的名称中含有 '-ts-'；例如 'c-ts-mode'、'python-ts-mode' 等。

编程语言的主模式可以通过 Eglot 包提供的设施使用"语言服务器"（language server）的服务。Eglot 实现了 LSP，即"语言服务器协议"（language server protocol），它让 Emacs 能接收特定于语言的信息和服务，从而丰富并扩展源代码编辑能力。参见《Eglot 手册》的"Eglot Features"。

在大多数编程语言中，缩进应逐行变化，以体现程序的结构。因此，在大多数编程语言模式中，键入 '<TAB>' 会更新当前行的缩进（参见"程序的缩进"）。此外，'<DEL>' 通常绑定到 'backward-delete-char-untabify'，它向后删除文本，删除时把每个制表符当作等量的空格，这样你就可以删除一列缩进，而无需担心该空白由空格还是制表符组成。

进入编程语言模式会先运行钩子变量 'prog-mode-hook' 中指定的自定义 Lisp 函数，然后运行该模式自己的模式钩子中指定的函数（参见"主模式"）。例如，进入 C 模式会运行钩子 'prog-mode-hook' 和 'c-mode-hook'。关于钩子的信息，参见"钩子"。

对于 Ada（参见《《ada-mode》手册》）、C/C++/Objective C/Java/Corba IDL/Pike/AWK（参见《《ccmode》手册》）、Octave 和 VHDL 的模式，另有单独的手册可供参阅。

28.2 顶层定义（defun）

在 Emacs 中，缓冲区顶层的主要定义（如函数）称为"defun"。这个名字来自 Lisp，但在 Emacs 中我们用它指所有语言中的这类定义。

- Left Margin Paren：位于左边界的左圆括号或类似的开定界符会开始一个 defun。
- Moving by Defuns：越过或标记主要定义的命令。
- Moving by Sentences：越过某些代码单元的命令。
- Imenu：把缓冲区索引做成菜单。
- Which Function：Which Function 模式显示你当前所在的函数。

28.2.1 左边界约定

许多编程语言模式历来假定：任何出现在左边界的左圆括号或左花括号都是顶层定义（即 defun）的开始。因此，默认情况下，寻找 defun 开头的命令会把这样的定界符视为该位置的标志。

如果你想推翻这一约定，可以把用户选项 'open-paren-in-column-0-is-defun-start' 设为 'nil'。若该选项为 't'（默认值），寻找 defun 开头的命令会停在第零列、且不在注释或字符串中的左圆括号或左花括号处。当它为 'nil' 时，则通过搜索最外层的括号或花括号来寻找 defun。由于 Emacs 的底层例程不再依赖这一约定，你通常无需把 'open-paren-in-column-0-is-defun-start' 从默认值改掉。

28.2.2 按 defun 移动

这些命令根据顶层主要定义（也称为"defun"）移动光标点或设置区域。

'C-M-a' 移到当前或前一个 defun 的开头（'beginning-of-defun'）。'C-M-e' 移到当前或后一个 defun 的结尾（'end-of-defun'）。'C-M-h' 把区域设在当前或后一个完整 defun 周围（'mark-defun'）。

移到当前 defun 开头和结尾的命令是 'C-M-a'（'beginning-of-defun'）和 'C-M-e'（'end-of-defun'）。若重复其中某个命令，或使用正数值参数，则每重复一次就沿移动方向移到下一个 defun。

带负参数 −N 的 'C-M-a' 向前移动 N 次，到达下一个 defun 的开头。这与带参数 N 的 'C-M-e' 所到之处并不完全相同；一个 defun 的结尾通常并不正好等于后一个 defun 的开头。（空白、注释，也许还有声明，可以把它们隔开。）同样，带负参数的 'C-M-e' 向后移到一个 defun 的结尾，这与带正参数的 'C-M-a' 也不完全相同。

要对当前 defun 进行操作，使用 'C-M-h'（'mark-defun'），它把标记设在当前 defun 的结尾，并把光标点放在其开头。参见"标记文本对象的命令"。这是为了把 defun 移到文件中另一位置而准备好删除它的最简便方法。若该 defun 前面紧接有注释（中间没有空行），这些注释也会被标记。若在光标点位于两个 defun 之间时使用该命令，它使用后一个 defun。若使用该命令时标记已经处于活动状态，它会把区域结尾扩展为多包含一个 defun。带前缀参数时，它会标记相应数目的 defun，或按相应数目的 defun 扩展区域。带负前缀参数时，它向相反方向标记 defun，并改变后续使用 'mark-defun' 时选择的方向。

在 C 模式中，'C-M-h' 运行函数 'c-mark-function'，它与 'mark-defun' 几乎相同；区别在于它会越过参数声明、函数名和返回的数据类型，使整个 C 函数都落在区域之内。这是主模式调整标准键绑定、以便以更适合特定语言的方式完成其标准工作的一个例子。其他主模式也可能为此目的替换其中任意或全部键绑定。

有些编程语言支持"嵌套 defun"，即一个 defun（如函数、方法或类）可以定义在另一个 defun 内部（也就是作为其函数体的一部分）。上述命令默认查找光标点周围"最内层"defun 的开头和结尾。基于 tree-sitter 库的主模式可以控制这一行为：默认情况下，'treesit-defun-tactic' 的值被设为 'nested'；若设为 'top-level'，defun 命令会改为查找"最外层"defun；若该值设为 'parent-first'，defun 命令总是试图移出当前所在的包围 defun，而不是移到光标点周围的前一个或后一个 defun；若没有包围的 defun，则移到前一个或后一个 defun。

28.2.3 按句子移动

这些命令根据代码单元（也称为"句子"）移动光标点或设置区域。尽管"句子"通常是在撰写人类语言时才考虑的概念，Emacs 也可以用同样的命令越过编程语言中的某些构造（参见"句子"、参见"按 defun 移动"）。在编程语言中，一个句子通常是一个完整的语言构造，它比 defun 小，但比 sexp 大（参见《Emacs Lisp 参考手册》的"List Motion"）。在这种情况下，句子究竟是什么取决于编程语言，但通常它是一条完整的语句，例如变量定义加初始化，或条件语句。C 语言中句子的一个例子可能是

```
int x = 5;
```

而在 JavaScript 语言中，它可能形如

```
const thing = () => console.log("Hi");
const foo = [1] == '1'
  ? "No way"
  : "...";
```

'M-a' 移到当前或前一个句子的开头（'backward-sentence'）。'M-e' 移到当前或后一个句子的结尾（'forward-sentence'）。

移到当前句子开头和结尾的命令是 'M-a'（'backward-sentence'）和 'M-e'（'forward-sentence'）。若重复其中某个命令，或使用正数值参数，则每重复一次就沿移动方向移到下一个句子。

带负参数 −N 的 'M-a' 向前移动 N 次，到达下一个句子的结尾。同样，带负参数的 'M-e' 向后移到一个句子的开头。

28.2.4 Imenu

Imenu 设施提供了一种按名称查找文件中主要定义的方法。它在编程语言主模式中很有用（其中的定义是变量、函数等），在文本格式化工具的主模式中也很有用（它把每一章、节等当作一个定义）。（参见"查找标识符引用"，了解能同时处理多个文件的更强大特性。）

若键入 'M-g i'（'imenu'），它会用迷你缓冲区读取一个定义的名称，然后把光标点移到该定义处。你可以用补全来指定名称；该命令会在补全缓冲区中显示匹配的合法名称列表。若索引是分层的，那么默认情况下补全候选也按层次显示为嵌套列表：你先选择一个节，再选择一个子节，等等，最后选择定义的名称。不过，若把 'imenu-flatten' 设为非 'nil' 值，你就可以从扁平化的定义列表中选择补全候选，而不再使用嵌套菜单。节和子节在扁平化的补全候选列表中如何显示，取决于 'imenu-flatten' 的值，它可以是下列之一：

- 'prefix'：每个候选项前面加上其所属节、子节、次子节等的名称，各层之间用 'imenu-level-separator' 的值分隔，默认为 ':'。
- 'annotation'：把节名作为注解显示在每个定义名之后。
- 'group'：按所在的节对补全候选分组显示。

另外，你也可以把 'imenu' 命令绑定到鼠标点击。这样它会显示鼠标菜单，供你选择定义名。你还可以通过调用 'imenu-add-menubar-index' 把缓冲区的索引加入菜单栏。若想让该菜单栏项对某种主模式下的所有缓冲区都可用，可以把 'imenu-add-menubar-index' 加入该模式的模式钩子。但若你这样做了，每次访问该模式下的文件时都得稍等片刻，等 Emacs 找出该缓冲区中的所有定义。

当你改变缓冲区内容时，若增加或删除了定义，可以通过调用菜单中的 '*Rescan*' 项，根据新内容更新缓冲区的索引。若把 'imenu-auto-rescan' 设为非 'nil' 值，则重新扫描会自动进行。文本的小改动无需重新扫描。

对于大于 'imenu-auto-rescan-maxout' 字节的缓冲区，'imenu-auto-rescan' 会被禁用；若扫描耗时超过 'imenu-max-index-time' 秒，扫描会被停止。

你可以通过设置变量 'imenu-sort-function' 定制菜单的排序方式。默认情况下，名称按其出现在缓冲区中的顺序排列；若想按字母顺序排序，可用符号 'imenu--sort-by-name' 作为值。你也可以编写 Lisp 代码来定义自己的比较函数。

你还可以通过修改变量 'completion-category-overrides' 并设置 'imenu' 类别的 'display-sort-function'，来定制 Imenu 补全的排序方式。

若当前缓冲区的项目（参见"使用项目"）和当前缓冲区的主模式已为 Eglot 激活，Eglot 会提供自己的设施，依据管理当前缓冲区的语言服务器对程序源代码的分析来生成缓冲区索引。参见《Eglot 手册》的"Eglot Features"。

Imenu 提供的信息可供 Which Function 模式使用（参见"Which Function 模式"）。Speedbar 也可以使用它（参见"Speedbar 框架"）。

28.2.5 Which Function 模式

Which Function 模式是一种全局次模式（参见“次模式”），它在模式行或标题行中显示当前函数名，并随你在缓冲区中的移动而更新。

要启用或禁用 Which Function 模式，使用命令‘M-x which-function-mode’。Which Function 模式是一种全局次模式。默认情况下，它在所有知道如何支持它的主模式（即所有支持 Imenu 的主模式）中生效。你可以把变量‘which-func-modes’的值从‘t’（意为支持所有可用的主模式）改为一个主模式名列表，从而把它限制在特定的主模式列表中。

默认情况下，Which Function 模式使用模式行显示当前函数名。把‘which-func-display’定制为‘header’、‘mode’或‘mode-and-header’，即可分别使用标题行、模式行或两者。

28.3 程序的缩进

保持程序缩进得当的最佳方法是，在修改程序时用 Emacs 重新缩进。Emacs 有命令可以缩进一行、指定数目的行，或某个括号分组内的所有行。

关于缩进的一般信息，参见“缩进”。本节介绍编程语言模式特有的缩进功能。

把 Lisp 对象重新格式化为美观的缩进。参见《Emacs Lisp 参考手册》的“Output Functions”

28.3.1 基本的程序缩进命令

‘<TAB>’调整当前行的缩进（‘indent-for-tab-command’）。‘<RET>’插入换行，然后调整下一行的缩进（‘newline’）。

基本的缩进命令是‘<TAB>’（‘indent-for-tab-command’），参见“缩进”中已有说明。在编程语言模式中，‘<TAB>’根据前面各行的缩进和语法内容缩进当前行；若区域处于活动状态，‘<TAB>’会缩进区域内的每一行，而不只是当前行。

命令‘<RET>’（‘newline’）在 参见“插入文本”中已有说明，它的作用等同于‘C-j’后接‘<TAB>’：插入新行，然后调整该行的缩进。

当缩进的行起始于某个括号分组内部时，Emacs 通常会把该行行首放在该分组中前一行的下方，或放在该括号之后的文本下方。如果你手动给其中某行设置了非标准的缩进（例如为了美观），其下面的行会跟随它。

大多数编程语言模式的缩进命令假定，位于左边界的左圆括号、左花括号或其他开定界符是函数的开始。如果你编辑的代码违背了这一假定——即使这些定界符出现在字符串或注释中——你必须把‘open-paren-in-column-0-is-defun-start’设为‘nil’，缩进才能正常工作。参见“左边界约定”。

28.3.2 缩进多行

有时你可能会想一次重新缩进若干行代码。方法之一是使用标记：当标记处于活动状态且区域非空时，‘<TAB>’会缩进区域中的每一行。另外，命令‘C-M-\’（‘indent-region’）会缩进区域中的每一行，无论标记是否处于活动状态（参见“缩进命令”）。

此外，Emacs 还提供下列命令来缩进大段代码：

‘C-M-q’ 重新缩进一个括号分组内的所有行。‘M-q’ 填充 defun 中的单个段落，或重新缩进该 defun 内的所有行。‘C-u <TAB>’ 把整个括号分组整体侧移，使其第一行缩进正确。‘M-x indent-code-rigidly’ 把区域中的所有行整体侧移，但不改动起始于注释和字符串内部的行。

要重新缩进单个括号分组的内容，把光标点定位在该分组开始之前，然后键入 ‘C-M-q’。这会改变分组内部的相对缩进，而不影响其整体缩进（即该分组起始行的缩进）。‘C-M-q’ 运行的函数取决于主模式；在 Lisp 模式中是 ‘indent-pp-sexp’，在 C 模式中是 ‘c-indent-exp’，等等。若要同时修正整体缩进，先键入 ‘<TAB>’。

要重新缩进光标点周围的整个 defun，键入 ‘M-q’（‘prog-fill-reindent-defun’）。若光标点位于注释或字符串中，该命令改为填充并缩进该注释或字符串。究竟什么算作注释、字符串或 defun 取决于主模式：defun 的边界由变量 ‘beginning-of-defun-function’ 和 ‘end-of-defun-function’ 决定（参见《Emacs Lisp 参考手册》的“List Motion”），填充机制由 ‘fill-paragraph-function’ 决定（参见《Emacs Lisp 参考手册》的“Filling”）。

如果你喜欢某个分组内的相对缩进，但不满意其第一行的缩进，把光标点移到那一行并键入 ‘C-u <TAB>’。在 Lisp、C 及某些其他主模式中，带数值参数的 ‘<TAB>’ 会照常重新缩进当前行，然后按相同的量重新缩进从当前行开始的括号分组中的所有行。不过它很聪明，不会改动起始于字符串内部的行。在 C 模式中它也不会改动 C 预处理行，但会重新缩进可能附着于这些行的任何续行。

命令 ‘M-x indent-code-rigidly’ 像 ‘indent-rigidly’ 那样把区域中的所有行整体侧移（参见“缩进命令”）。它不会改动起始于字符串内部的行，除非区域也起始于该字符串内部。前缀参数指定缩进的列数。

28.3.3 定制 Lisp 缩进

Lisp 表达式的缩进模式可以取决于该表达式所调用的函数。对每个 Lisp 函数，你可以在若干预定义的缩进模式中选择，也可以用 Lisp 程序定义任意一种模式。

标准的缩进模式如下：表达式的第二行缩进到第一个参数之下（如果该参数与表达式的开头在同一行）；否则，第二行缩进到函数名之下。其后的每一行缩进到前一个嵌套深度相同的行之下。

若变量 ‘lisp-indent-offset’ 非 ‘nil’，它会覆盖表达式第二行通常的缩进模式，使这些行总是比所在的列表多缩进 ‘lisp-indent-offset’ 列。

某些函数会覆盖标准模式。名称以 ‘def’ 开头的函数把第二行当作“函数体”的开始，使第二行比开启该表达式的左圆括号多缩进 ‘lisp-body-indent’ 列。

你可以按函数名的 ‘lisp-indent-function’ 属性，为各个函数以多种方式覆盖标准模式。这通常用于宏定义，借助 ‘declare’ 构造完成。参见《Emacs Lisp 参考手册》的“Defining Macros”。

在 Emacs Lisp 中，列表通常按类似函数调用的形式缩进：

```
(setq foo '(bar zot
               gazonk))
```

然而，如果在左圆括号之后加一个空格，这就告诉 Emacs 它是一个数据列表而不是一段代码，于是 Emacs 会这样缩进它：

```
(setq foo '( bar zot
             gazonk))
```

28.3.4 C 缩进命令

下面是 C 模式及相关模式中缩进的特殊功能：

‘C-c C-q’ 重新缩进当前顶层函数定义或聚合类型声明（CC 模式中的 ‘c-indent-defun’，基于 tree-sitter 的 ‘c-ts-mode’ 中的 ‘c-ts-mode-indent-defun’）。

‘C-M-q’ 重新缩进光标点之后平衡表达式（参见“平衡括号表达式”，也称为“sexp”）中的每一行。在 CC 模式中，它调用 ‘c-indent-exp’；在基于 tree-sitter 的 ‘c-ts-mode’ 中，它调用更通用的 ‘prog-indent-sexp’。前缀参数会抑制有关无效语法的警告消息。

‘<TAB>’ 重新缩进当前行、活动区域，或从本行开始的代码块（‘c-indent-line-or-region’）。带前缀参数时，若当前行需要重新缩进，则整体重新缩进从当前行开始的平衡表达式。

若 ‘c-tab-always-indent’ 为 ‘t’，该命令总是重新缩进当前行，不做其他事情。这是默认值。

若该变量为 ‘nil’，该命令只在光标点位于左边界或位于该行的缩进部分时才重新缩进当前行；否则，它插入一个制表符（若 ‘indent-tabs-mode’ 为 ‘nil’，则插入等量的空格）。

任何其他值（不是 ‘nil’ 或 ‘t’）都表示总是重新缩进该行，并且如果该行位于注释或字符串内部，还会插入一个制表符。

要重新缩进整个当前缓冲区，输入 ‘C-x h C-M-\’。这会先把整个缓冲区选为区域，然后重新缩进该区域。

要重新缩进当前代码块，使用 ‘C-M-u C-M-q’。这会移到该代码块的开头，然后把它整个重新缩进。

28.3.5 自定义 C 缩进

C 模式及相关模式使用一套灵活的机制来自定义缩进。C 模式分两步缩进源代码行：首先，它根据该行的内容和上下文对之做语法分类；其次，它确定你所选“风格”（style）中与该语法结构关联的缩进偏移量，并将其加到“锚点语句”（anchor statement）的缩进上。

‘C-c . STYLE <RET>’ 选择一个预定义风格 STYLE（在 CC 模式中为 ‘c-set-style’，在基于 tree-sitter 的 ‘c-ts-mode’ 中为 ‘c-ts-mode-set-style’）。

“风格”是一组命名的自定义设置集合，可用于 C 模式及相关模式。完整说明见 参见《《ccmode》手册》的“Styles”。Emacs 自带若干预定义风格，包括 ‘gnu’、‘k&r’、‘bsd’、‘stroustrup’、‘linux’、‘python’、‘java’、‘whitesmith’、‘ellemtel’ 和 ‘awk’。其中有些风格主要面向某一种语言，但任何一种风格都可用于这些模式所支持的任何语言。要了解某种风格是什么样子，可以选定它并重新缩进一些代码，例如在函数定义的开头输入 ‘C-M-q’。

要为当前缓冲区选择风格，使用命令 ‘C-c .’。以风格名作为参数指定（大小写不敏感）。此命令只影响当前缓冲区，并且只影响此后调用的缩进命令；它不会重新缩进缓冲区中已有的代码。若想按新风格重新缩进整个缓冲区，可以输入 ‘C-x h C-M-\’。

使用 CC 模式时，还可以设置变量 ‘c-default-style’ 来指定各个主模式的默认风格。其值应为风格名（字符串），或一个 alist，其中每个元素指定一个主模式以及该模式要使用的缩进风格。例如，

```
(setq c-default-style
      '((java-mode . "java")
        (awk-mode . "awk")
```

```
        (other . "gnu")))
```

这就为 Java 和 AWK 模式指定了明确的选择，并为其他类 C 模式指定了默认的 'gnu' 风格。（这些设置实际上就是默认值。）此变量在你选择某个类 C 主模式时生效；因此，如果你为 Java 模式指定了新的默认风格，可以通过在该缓冲区中输入 'M-x java-mode' 让它在已有的 Java 模式缓冲区中生效。

使用基于 tree-sitter 的 'c-ts-mode' 时，可以通过自定义变量 'c-ts-mode-indent-style' 来设置默认缩进风格。

'gnu' 风格指定 GNU 项目推荐用于 C 的格式；它是默认值，以鼓励使用我们推荐的风格。

关于自定义 C 及相关模式缩进的更多信息，包括如何覆盖现有风格的部分设置以及如何定义自己的风格，参见 《《ccmode》手册》的"Indentation Engine Basics"和 参见《《ccmode》手册》的"Customizing Indentation"。

除了指定风格之外，你还可以让 Emacs 猜测风格：在示例代码缓冲区中输入 'M-x c-guess'。然后可以用 'M-x c-guess-install' 把猜出的风格应用到其他缓冲区。详情参见 《《ccmode》手册》的"Guessing the Style"。

28.4 编辑括号的命令

本节介绍那些利用程序中的括号结构、或帮助保持括号配对的命令和功能。

在讨论这些功能时，"括号"一词也包含花括号、方括号，或任何被定义为成对匹配的分隔符。哪些分隔符有意义由主模式通过语法表控制（参见《Emacs Lisp 参考手册》的"Syntax Tables"）。在 Lisp 中只有圆括号算数；在 C 中，这些命令也适用于花括号和方括号。

可以用 'M-x check-parens' 找出缓冲区中任何不配对的括号和不配对的字符串引号。

- **Moving by Parens**：在括号结构中向上、向下和横向移动的命令。
- **Matching**：插入闭合分隔符时闪显与之匹配的开分隔符。

28.4.1 平衡括号表达式

每种编程语言模式对"平衡表达式"（balanced expression）都有自己的定义。平衡表达式通常包括单个符号、数字和字符串常量，也包括用一对匹配分隔符括起来的代码片段。以下命令用于处理平衡表达式（在 Emacs 内部，这类表达式称为"sexp"(1)）。

- 'C-M-f' 向前越过一个平衡表达式（'forward-sexp'）。
- 'C-M-b' 向后越过一个平衡表达式（'backward-sexp'）。
- 'C-M-k' 向前删除平衡表达式（'kill-sexp'）。
- 'C-M-t' 交换表达式的位置（'transpose-sexps'）。
- 'C-M-@' 'C-M-<SPC>' 把标记放到后面的表达式之后（'mark-sexp'）。

要向前越过一个平衡表达式，使用 'C-M-f'（'forward-sexp'）。如果光标点之后的第一个有效字符是开分隔符（例如 C 中的 '('、'[' 或 '{'），该命令会越过与之匹配的闭分隔符。如果该字符开始一个符号、字符串或数字，该命令就越过它。

命令 'C-M-b'（'backward-sexp'）向后越过一个平衡表达式——与 'C-M-f' 类似，只是方向相反。如果该表达式前面有任何前缀字符（在 Lisp 中是单引号、反引号和逗号），该命令也会越过它们。

带参数的 'C-M-f' 或 'C-M-b' 会把该操作重复指定次数；负数参数表示向相反方向移动。在大多数模式中，这两个命令把注释当作空白一样越过。请注意，它们的按键 'C-M-f' 和 'C-M-b' 与按字符移动的 'C-f' 和 'C-b'（参见"移动光标点的位置"）、以及按单词移动的 'M-f' 和 'M-b'（参见"单词"）相类似。

要删除整个平衡表达式，输入 'C-M-k'（'kill-sexp'）。它删除 'C-M-f' 会越过的文本。

'C-M-t'（'transpose-sexps'）交换前一个平衡表达式和后一个平衡表达式的位置。它类似于交换字符的 'C-t' 命令（参见"调换文本"）。'C-M-t' 的参数用作重复次数，把前一个表达式依次越过其后的那么多个表达式。负数参数把前一个平衡表达式向后移动，越过它前面的那些表达式。参数为零时并不什么都不做，而是交换结束于光标点或位于光标点之后的平衡表达式与标记。

要配合对区域起作用的命令来操作平衡表达式，输入 'C-M-<SPC>'（'mark-sexp'）。它把标记设在 'C-M-f' 会移到的地方。当标记处于活动状态时，每次连续调用此命令都会把标记移过一个表达式，从而扩展区域。正数或负数数值参数把标记向前或向后移动指定个数的表达式。别名 'C-M-@' 等同于 'C-M-<SPC>'。有关此命令及相关命令的更多信息，参见"标记文本对象的命令"。

在 C 这类使用中缀运算符的语言中，不可能识别出所有平衡表达式，因为在给定位置上可能有多种可能。例如，C 模式不把 'foo + bar' 当成单个表达式，尽管它_确实_是一个 C 表达式；相反，它把 'foo' 识别为一个表达式、把 'bar' 识别为另一个表达式，而 '+' 是两者之间的标点。不过，C 模式把 '(foo + bar)' 识别为单个表达式，因为有括号。

> 注："sexp"一词用来指 Lisp 中的一个表达式。

28.4.2 在括号结构中移动

以下命令越过由括号（或你所处理的编程语言中充当分隔符的其他字符）界定的分组。它们忽略字符串和注释，包括其中的任何括号，也忽略用转义字符引用的括号。这些命令主要面向编辑程序，但也可用于编辑任何含括号的文本。它们在内部被称为"列表命令"（list commands），因为在 Lisp 中这些分组就是列表。

这些命令假定起始点不在字符串或注释内部。如果你在字符串或注释内部调用它们，结果不可靠。

- 'C-M-n' 向前越过一个括号分组（'forward-list'）。
- 'C-M-p' 向后越过一个括号分组（'backward-list'）。
- 'C-M-u' 在括号结构中向上移动（'backward-up-list'）。
- 'C-M-d' 在括号结构中向下移动（'down-list'）。

列表命令 'C-M-n'（'forward-list'）和 'C-M-p'（'backward-list'）向前或向后越过一个（或 N 个）括号分组。

'C-M-n' 和 'C-M-p' 会尽量停留在括号结构中的同一层。要_向上_移动一层（或 N 层），使用 'C-M-u'（'backward-up-list'）。'C-M-u' 向后越过一个未配对的开分隔符而上升。正数参数用作重复次数；负数参数反转移动方向，使该命令向前并向上移动一层或多层。

要在括号结构中_向下_移动，使用 'C-M-d'（'down-list'）。在 Lisp 模式中，'(' 是唯一的开分隔符，因此这几乎等同于搜索一个 '('。参数指定要下移的层数。

28.4.3 匹配括号

Emacs 有许多"括号匹配"功能，让人容易看出括号（或其他成对分隔符）是如何配对的、是否配对。

每当你输入一个作为闭合分隔符的自插入字符时，只要与之匹配的开分隔符在屏幕上，Emacs 就会短暂指出它的位置。如果它不在屏幕上，Emacs 就在回显区显示其附近的一些文本。无论哪种方式，你都能知道正在结束的是哪个分组。如果开分隔符与闭分隔符不匹配——例如 '[x)' 中那样——回显区会显示一条警告消息。

有三个变量控制匹配括号的显示：

- 'blink-matching-paren' 打开或关闭该功能：'nil' 禁用它，默认值为 't'，即启用它。把它设为 'jump' 可让指示方式改为暂时把光标移到匹配的开分隔符上。把它设为 'jump-offscreen' 可让光标跳跃，即使开分隔符在屏幕之外。

- 'blink-matching-delay' 指定持续指示匹配的开分隔符多少秒。它可以是整数或浮点数；默认值为 1。

- 'blink-matching-paren-distance' 指定向后搜索匹配的开分隔符时最多搜索多少个字符。如果在该距离内没有找到匹配项，Emacs 就停止扫描，不显示任何内容。默认值为 102400。

Show Paren 模式是一种次模式，提供更强大的自动匹配功能。当光标点位于开分隔符之前或闭分隔符之后时，该分隔符、与之匹配的分隔符，以及可选的二者之间的文本都会被高亮显示。要全局切换 Show Paren 模式，输入 'M-x show-paren-mode'。要只在当前缓冲区中切换它，输入 'M-x show-paren-local-mode'。

默认情况下，该模式在所有用于编辑的缓冲区中开启，但不在显示数据的缓冲区中启用。这由用户选项 'show-paren-predicate' 控制。

要自定义该模式，输入 'M-x customize-group <RET> paren-showing'。控制该模式运行的可自定义选项包括：

- 'show-paren-highlight-openparen' 控制当光标点恰在开括号之前、其位置反正已由光标标出时，是否高亮该开括号。默认值非 'nil'（是）。

- 'show-paren-style' 控制是只高亮两个括号，还是也高亮二者之间的文本。有效选项有：'parenthesis'（显示匹配的括号）、'expression'（高亮括号所括的整个表达式）和 'mixed'（若匹配的括号在窗口中可见就高亮该括号，否则高亮表达式）。

- 'show-paren-when-point-inside-paren' 若非 'nil'，则当光标点位于括号内部时也进行高亮。默认值为 'nil'。

- 'show-paren-when-point-in-periphery' 若非 'nil'，则当光标点位于行首的空白处、且该行第一个或最后一个非空白位置上有括号时，或者当光标点位于行尾、且该行最后一个非空白位置上有括号时，也进行高亮。

- 'show-paren-context-when-offscreen' 若非 'nil'，则当光标点位于闭分隔符中、而开分隔符在屏幕之外时，在回显区显示一些上下文。上下文通常是包含开分隔符的那一行，除非开分隔符独自占一行，此时上下文还包括前面最近的非空行。

- 若 'show-paren-not-in-comments-or-strings' 的值为 'all'，则注释和字符串内部的分隔符不会被高亮。否则，若该值被设为 'on-mismatch'，则注释内部不匹配的分隔符不会被高亮。默认值为 'nil'，因此注释和字符串内部的分隔符总会被高亮。

Electric Pair 模式是一种次模式，提供一种便捷地插入成对匹配分隔符的方式：圆括号、花括号、方括号、引号等（什么算作匹配分隔符取决于主模式）。要全局切换 Electric Pair 模式，输入 'M-x electric-pair-mode'。要只在当前缓冲区中切换它，输入 'M-x electric-pair-local-mode'。

启用该模式后，输入一个开分隔符会同时插入该字符以及紧跟在它后面的匹配闭分隔符，光标点留在二者之间。这样在大多数情况下就不必再输入匹配的闭分隔符。不过如果你还是输入了一个，Emacs 只会插入该字符，除非光标点恰在同类闭分隔符之前；在后一种情况下，光标点移到该闭分隔符之后，并且不再插入额外的闭分隔符。因此，输入序列 <opening delimiter>、<matching closing delimiter> 是输入序列 <opening delimiter>、'C-f' 的另一种也许更方便的选择。

当区域处于活动状态时（参见"标记与区域"），Electric Pair 模式的行为有所不同：插入开分隔符或闭分隔符都会把区域中的字符包进由此产生的那一对匹配分隔符中，光标点留在你所输入的分隔符之后（这便于你继续在开分隔符之后的文本之前、或在闭分隔符之后输入）。

如果你在插入开分隔符时提供前缀参数——若区域处于活动状态，在插入闭分隔符时也一样——就会插入与该前缀参数数值一样多的嵌套匹配分隔符对。

有几个用户选项可以修改 Electric Pair 模式的行为：

- 'electric-pair-preserve-balance' 若非 'nil'（默认值），输入分隔符时会保持开分隔符与闭分隔符之间的平衡。因此，如果你输入一个开分隔符，而缓冲区中后面有一个未配对的匹配闭分隔符，那么只会插入开分隔符（不会紧跟着插入匹配的闭分隔符）；同样，如果存在未配对的开分隔符，那么以后在缓冲区中输入匹配的闭分隔符时，即使后一个字符是另一个匹配的闭分隔符，也会插入该字符。

当它被设为 'nil' 时，输入开分隔符只插入该字符，但仅在光标点恰在匹配开分隔符之前或恰在其之后、或恰在字母或数字之前时如此；在所有其他位置，插入开分隔符都会自动在其后紧跟一个匹配的闭分隔符，即使缓冲区中后面有一个未配对的匹配闭分隔符。而在同类闭分隔符之前紧挨着输入闭分隔符时，不插入该字符，而是如上所述移动光标点，即使缓冲区中前面有一个未配对的匹配开分隔符也是如此。

如果区域处于活动状态，该变量不起作用。

- 'electric-pair-delete-adjacent-pairs' 若非 'nil'（默认值），用 <DEL> 键（通常就是 <BACKSPACE> 键；参见"若 <DEL> 无法删除"）删除开分隔符时，会自动同时删除紧跟在它后面的匹配闭分隔符（但如果这对分隔符之间有任何字符——包括只有空白——则不然）。当它被设为 'nil' 时，输入 <BACKSPACE> 只删除开分隔符。（输入 <BACKSPACE> 删除闭分隔符时，始终只删除该字符。）

- 当 'electric-pair-open-newline-between-pairs' 非 'nil'（默认值）且光标点位于开分隔符与其后紧跟的匹配闭分隔符之间时，输入换行会自动在光标点之后多插入一个换行（视主模式而定，可能还会缩进光标点所在的空行）。当它被设为 'nil' 时，输入换行只在光标点之前插入一个换行，与通常一样。

- 当 'electric-pair-skip-whitespace' 取其默认的非 'nil' 值，且光标点与某个闭分隔符之间只隔着空白时，输入同类闭分隔符不会插入该字符，而是把光标点移到已存在的该闭分隔符之后。你也可以把此选项设为额外删除光标点移动经过的任何空白。当它被设为 'nil' 时，输入闭分隔符只是插入该字符（即使这会让后面的同类闭分隔符变得不配对）。

28.5 操作注释

由于注释是编程中如此重要的一部分，Emacs 为编辑和插入注释提供了专门的命令。它还能配合 Flyspell Prog 模式对注释做拼写检查（参见"拼写检查与纠正"）。

有些主模式对不同种类的注释的缩进有特殊规则。例如在 Lisp 代码中，以两个分号开头的注释按代码行的方式缩进，而以三个分号开头的注释则应左对齐到页边，常用于分节。Emacs 了解这些约定；例如在注释行上输入 '<TAB>' 会把该注释缩进到合适的位置。

```
;; This function is just an example.
;;; Here either two or three semicolons are appropriate.
(defun foo (x)
;;;  And now, the first part of the function:
  ;; The following line adds one.
  (1+ x))           ; This line adds one.
```

28.5.1 注释命令

以下命令用于操作注释：

- 'M-;' 插入或重新对齐当前行的注释；如果区域处于活动状态，则改为注释或取消注释该区域（'comment-dwim'）。
- 'C-x C-;' 注释或取消注释当前行（'comment-line'）。如果区域处于活动状态，则改为注释或取消注释区域中的各行。
- 'C-u M-;' 删除当前行的注释（'comment-kill'）。
- 'C-x ;' 设置注释列（'comment-set-column'）。
- 'C-M-j' 'M-j' 相当于 '<RET>' 之后插入并对齐一条注释（'default-indent-new-line'）。参见"多行注释"。
- 'M-x comment-region' 'C-c C-c'（在类 C 模式中）为区域中的所有行添加注释分隔符。

创建或对齐注释的命令是 'M-;'（'comment-dwim'）。"dwim"一词是"Do What I Mean"的首字母缩写；它表示此命令可用于与注释有关的许多不同工作，具体做什么取决于你使用它的场合。

当区域处于活动状态时（参见"标记与区域"），'M-;' 要么给该区域添加注释分隔符，要么删除它们。如果区域中每一行都已经是注释，它就删除各行的注释分隔符，从而取消注释。否则，它添加注释分隔符，把区域中的文本括起来。

如果在区域处于活动状态时给 'M-;' 提供前缀参数，该参数指定要添加或删除的注释分隔符个数。正参数 N 添加 N 个分隔符，负参数 −N 删除 N 个分隔符。

如果区域不活动，且当前行上没有已有注释，'M-;' 会为当前行添加一条新注释。如果该行是空行（即空行或只含空白字符），注释缩进到 '<TAB>' 会缩进到的同一位置（参见"基本的程序缩进命令"）。如果该行非空，注释放在该行最后一个非空白字符之后。Emacs 会尽可能把注释放在变量 'comment-column' 和 'comment-fill-column' 所指定的列之间（参见"控制注释的选项"）。否则，它会选择某个其他合适的位置，通常与非注释文本之间至少隔一个空格。无论哪种情况，Emacs 都把光标点放在注释起始分隔符之后，这样你可以立即开始输入注释内容。

你也可以用 'M-;' 对齐已有的注释。如果某行已经含有注释起始字符串，'M-;' 会把它重新对齐到常规位置，并把光标点移到注释起始分隔符之后。作为例外，从第 0 列开始的注释不会被移动。即使已有注释本来就对齐得当，'M-;' 仍可用于直接跳到注释文本的开头。

‘C-x C-;’（‘comment-line’）注释或取消注释整行。当区域处于活动状态时（参见“标记与区域”），‘C-x C-;’ 要么注释、要么取消注释区域中的各行。如果区域不活动，该命令注释或取消注释光标点所在的行。带正前缀参数 N 时，它处理从当前行开始的 N 行；带负参数 −N 时，它影响前面 N 行。用负参数调用此命令之后，后续带正参数的调用会作用于前面的行，就好像参数被取了负号一样。

当区域不活动时，‘C-u M-;’（带前缀参数的 ‘comment-dwim’）会删除当前行上的任何注释，连同其前面的空白。由于该注释被存入删除环，你可以把它重新插入到另一行：移到那一行的末尾，执行 ‘C-y’，然后用 ‘M-;’ 重新对齐该注释。输入 ‘M-x comment-kill’ 也能获得与 ‘C-u M-;’ 相同的效果（当给定前缀参数时，‘comment-dwim’ 实际上是调用 ‘comment-kill’ 作为子程序）。在没有活动区域时用数值前缀参数调用 ‘comment-dwim’，如 ‘C-u N M-;’，会告诉 ‘comment-kill’ 删除 N 行上的注释。

命令 ‘M-x comment-region’ 等同于对活动区域调用 ‘M-;’，区别在于它总是作用于区域，即使标记不活动。在 C 模式及相关模式中，此命令绑定到 ‘C-c C-c’。命令 ‘M-x uncomment-region’ 取消注释区域中的每一行；数值前缀参数指定要删除的注释分隔符个数（负参数指定要添加的注释分隔符个数）。

对于类 C 的模式，可以通过设置变量 ‘c-indent-comment-alist’ 和 ‘c-indent-comments-syntactically-p’ 来配置 ‘M-;’ 的确切效果。例如，在以右花括号结尾的行上，‘M-;’ 会把注释放在花括号之后一个空格处，而不是放在 ‘comment-column’ 处。完整细节参见《《ccmode》手册》的“Comment Commands”。

28.5.2 多行注释

如果在键入一条注释时想把它延续到另一行，可以键入 ‘M-j’ 或 ‘C-M-j’（‘default-indent-new-line’）。这会断开当前行，并插入延续该注释所需的注释定界符和缩进。

对于带有结束注释定界符的语言（例如 C 中的 ‘*/’），‘M-j’ 的确切行为取决于变量 ‘comment-multi-line’ 的值。若值为 ‘nil’，该命令在旧行上结束该注释，并在新行上开始一条新注释。否则，它会在当前注释定界符内部另起一行。

Auto Fill 模式开启时，键入注释的过程中越过填充列也会延续注释，方式与显式调用 ‘M-j’ 相同。

要把已有的行变成注释行，可以在区域处于活动状态时使用 ‘M-;’，或使用 ‘M-x comment-region’（参见“注释命令”）。

可以这样配置 C 模式：当你在多行块注释的行首键入 ‘/’ 时，该注释即被关闭。为此启用 ‘comment-close-slash’ 清理项。参见《《ccmode》手册》的“Clean-ups”。

28.5.3 控制注释的选项

如参见“注释命令”中所述，当 ‘M-j’ 命令为某行添加注释时，它会尽量把注释放在由缓冲区局部变量 ‘comment-column’ 和 ‘comment-fill-column’ 指定的列之间（若后者为 ‘nil’，则用 ‘fill-column’ 的值，参见“显式填充命令”）。可以用通常的方式设置这些缓冲区局部变量的局部值或默认值（参见“局部变量”）。另一种做法是键入 ‘C-x ;’（‘comment-set-column’），把当前缓冲区中 ‘comment-column’ 的值设为光标点当前所在的那一列。‘C-u C-x ;’ 把注释列设为与缓冲区中光标点之前最后一条注释相匹配的列，然后执行一次 ‘M-;’，把当前行的注释对齐到上一条注释之下。

注释命令依据变量 ‘comment-start-skip’ 的值（一个正则表达式）来识别注释。要确保该正则表达式不匹配空串。它匹配的内容可以多于严格意义上的注释起始定界符；例如在 C 模式中，该变量的值可以是 ‘"/\\+\

\t\\|//+[\t]"，*它除了匹配 '/' 本身之外还匹配其后的多余星号和空格，并且也接受 C++ 风格的（'//'）注*释。（注意，在 Lisp 语法中需要写成 '\\' 才能在字符串里包含一个 '\'，而在正则表达式语法中需要这个 '\' 来取消第一个星号的特殊含义。参见"正则表达式中的反斜杠"。）

当注释命令新建一条注释时，它插入 'comment-start' 的值作为起始注释定界符。它还在光标点之后插入 'comment-end' 的值作为结束注释定界符。例如在 Lisp 模式中，'comment-start' 是 '";"'，'comment-end' 是 '""'（空串）。在 C 模式中，'comment-start' 是 '"/* "'，'comment-end' 是 '" */"'。

变量 'comment-padding' 指定注释命令应插在注释定界符与注释正文之间的字符串。默认值 '" "' 表示单个空格。该值也可以是一个数字，表示相应数量的空格，或者为 'nil'，表示完全不留空格。

变量 'comment-multi-line' 控制 'M-j' 和 Auto Fill 模式如何把注释延续到多行。参见"多行注释"。

变量 'comment-indent-function' 应含有一个函数，该函数被调用来计算新插入注释的对齐位置，或对已有注释进行对齐。不同主模式对它的设置不同。调用该函数时不带参数，但光标点位于注释开头，或者（若将要插入新注释）位于行尾。它应返回注释应当起始的列。例如，默认函数根据已有注释由多少个注释字符开头来作出决定。

Emacs 还会尽量让相邻行上的注释对齐。要覆盖这一点，该函数可以返回一个由两个（可能相等的）整数构成的 cons，以指示可接受的缩进范围。

28.6 文档查询

Emacs 提供了若干功能，可用于查阅你打算在程序中使用的函数、变量和命令的文档。

28.6.1 Info 文档查询

对于所适用的语言在 Info 中有文档的主模式，可以使用 'C-h S'（'info-lookup-symbol'）查看程序中某个符号的 Info 文档。你在迷你缓冲区中指定该符号；默认值是缓冲区中光标点处的符号。例如在 C 模式中，这会到《C 库手册》中查找该符号。只有当相应手册的 Info 文件已安装时，该命令才有效。

Emacs 根据主模式决定到何处查找该符号的文档——查阅哪些 Info 文件、搜索哪些索引。你也可以使用 'M-x info-lookup-file' 查找某个文件名的文档。

如果在不支持该命令的主模式中使用 'C-h S'，它会要求你指定符号帮助模式。你应输入诸如 'c-mode' 这样的命令，以选择一个 'C-h S' 支持的主模式。

28.6.2 man 手册页查询

在 Unix 上，在线文档的主要形式是"手册页"（manual page，或"man page"）。在 GNU 操作系统中，我们的目标是用组织得更好、可以用 Info 浏览的手册取代手册页（参见"其他帮助命令"）。这一过程尚未完成，因此读手册页仍然有用。

可以用 'M-x man' 命令阅读操作系统命令、库函数或系统调用的手册页。它会提示输入主题并支持补全（参见"补全"），然后运行 'man' 程序来格式化相应的手册页。若系统允许，它会异步运行 'man'，这样在手册页格式化的同时你可以继续编辑。结果显示在名为 '*Man TOPIC*' 的缓冲区中。这些缓冲区使用一种特殊的主模式——Man 模式，它便于滚动以及跳转到其他手册页。详情请在 Man 模式缓冲区中键入 'C-h m'。

每个手册页属于十个或更多“节”（section）之一，每节用一位数字或一位数字加一个字母命名。有时不同节中存在同名的手册页。要阅读某一节中的手册页，在 ‘M-x man’ 提示输入主题时键入 ‘TOPIC(SECTION)’ 或 ‘SECTION TOPIC’。例如，C 库函数 ‘chmod’ 的手册页在第 2 节，但有一个同名的 shell 命令，其手册页在第 1 节；要查看前者，键入 ‘M-x man <RET> chmod(2) <RET>’。

若不指定节，‘M-x man’ 通常只显示找到的第一个手册页。在某些系统上，‘man’ 程序接受 ‘-a’ 命令行选项，告诉它显示指定主题的所有手册页。要利用这一点，把变量 ‘Man-switches’ 的值改为 ‘"-a"’。之后在 Man 模式缓冲区中，你可以键入 ‘M-n’ 和 ‘M-p’ 在不同节的手册页之间切换。模式行会显示有多少个手册页可用。

默认情况下，‘M-x man’ 异步调用 ‘man’ 程序。把 ‘Man-prefer-synchronous-call’ 定制为非 ‘nil’ 值，可以强制以同步方式调用。

若用户选项 ‘Man-support-remote-systems’ 为非 ‘nil’，且 ‘default-directory’ 指向某个远程系统（参见“远程文件”），则从该远程系统获取手册页。用 ‘C-u M-x man’ 这样的前缀调用 ‘man’ 命令，会为该次调用反转 ‘Man-support-remote-systems’ 的值。

阅读手册页的另一种方式是 ‘M-x woman’ 命令。与 ‘M-x man’ 不同，它不运行任何外部程序来格式化和显示手册页；格式化由 Emacs 完成，因此在 ‘man’ 程序可能不可用的系统（如 MS-Windows）上也能工作。它提示输入一个手册页，并在名为 ‘*WoMan SECTION TOPIC’ 的缓冲区中显示。

‘M-x woman’ 在第一次调用该命令时计算手册页的补全列表。带数字参数时它会重新计算该列表；若你增加或删除了手册页，这会很有用。

如果你键入某个手册页的名称，而 ‘M-x woman’ 发现不同节中存在多个同名手册页，它会弹出一个窗口列出候选者，请你从中选择一个。

注意，‘M-x woman’ 尚不支持现代手册页的最新特性，因此如果你的系统上有 ‘M-x man’，我们建议使用它。

关于设置和使用 ‘M-x woman’ 的更多信息，参见《WoMan 手册》。

28.6.3 编程语言文档查询

编辑 Emacs Lisp 代码时，可以使用命令 ‘C-h f’（‘describe-function’）和 ‘C-h v’（‘describe-variable’）查看你要使用的 Lisp 函数和变量的内置文档。参见“按命令或变量名获取帮助”。

ElDoc(1) 是一个缓冲区局部的次模式，帮助查阅程序中符号（函数、方法、类、变量等）的文档。启用该模式后，只要光标点处有带文档的符号，回显区就会显示有用的信息。例如，在 Emacs Lisp 模式的缓冲区中，它显示光标点处函数的参数列表；对 Lisp 变量则显示该变量文档字符串的第一行。

要开关 ElDoc 模式，键入 ‘M-x eldoc-mode’。此外还有 Global ElDoc 模式，它默认开启，并会为那些主模式设置了下文所述变量的缓冲区开启 ElDoc 模式。使用 ‘M-x global-eldoc-mode’ 可全局关闭它。

各种主模式会配置 Global ElDoc 模式以使用各自的文档函数。例子包括 Emacs Lisp 模式、Python 模式和 Cfengine 模式。此外，为若干主模式提供支持的 Emacs 功能也会配置 ElDoc 使用它们检索文档的设施。例子包括 Eglot（参见《Eglot 手册》的“Eglot Features”），它基于语言服务器的信息提供文档；Semantic 的 Idle Summary 模式（参见《Semantic 手册》的“Idle Summary Mode”）；以及 Flymake，它使用 ElDoc 显示光标点处的诊断信息（参见《《flymake》手册》的“Finding diagnostics”）。

ElDoc 模式的工作方式是：在 Emacs 空闲一小段时间后，安排显示光标点处符号可用的文档。这样，当你快速不间断地输入时，可避免回显区或模式行中文档消息令人厌烦的闪烁。

你也可以使用命令 'M-x eldoc-print-current-symbol-info' 触发显示光标点处符号的文档。

以下变量可用于配置 ElDoc 模式：

- 'eldoc-idle-delay'：该用户选项的值控制显示光标点处文档之前的空闲时间。应把它设成要等待的秒数；值为零表示不延迟、立即显示。默认值为 0.5 秒。
- 'eldoc-print-after-edit'：若该用户选项为非 'nil'，ElDoc 只在某些编辑命令（如插入或删除某些文本）之后显示文档。若你希望 Emacs 只显示你键入的符号的文档，而不显示缓冲区中已有符号的文档（这样仅仅阅读源代码不会显示文档），这会很方便。默认值为 'nil'。若改变该值，需要把 'eldoc-mode' 关掉再打开。
- 'eldoc-echo-area-use-multiline-p'：该用户选项控制当文档文本长于回显区能作为单个屏幕行显示的长度时，是否以及如何截断它。若值为正数，它指定 ElDoc 允许在回显区中显示多少屏幕行而不截断文档。正整数指定可使用的绝对最大屏幕行数；浮点数指定相对于框架高度的屏幕行数比例。值 't' 表示从不截断文档（回显区将被调高到 'max-mini-window-height' 允许的高度，参见"在迷你缓冲区中编辑"），而值 'nil' 表示若文档长于单个屏幕行就截断。最后，特殊值 'truncate-sym-name-if-fit'（默认值）表示：如果截掉文档中表示符号名的部分能让文档放进单个屏幕行，就截掉这部分。
- 'eldoc-echo-area-display-truncation-message'：若非 'nil'（默认值），且回显区中显示的文档因过长而被截断，则在文档之后附上如何查看完整文档文本的说明。若为 'nil'，则只用 '...' 表示文档已被截断。
- 'eldoc-echo-area-prefer-doc-buffer'：若该用户选项的值为 't'，当带有该文档的 ElDoc 缓冲区已在某个窗口中显示时，ElDoc 就不在回显区中显示文档。（你可以随时用命令 'M-x eldoc-doc-buffer' 显示 ElDoc 缓冲区。）若该选项的值为符号 'maybe'，则当 ElDoc 缓冲区已在某个窗口中显示，且该文档若在回显区显示就不得不被截断时，不在回显区中显示文档。最后，值 'nil'（默认值）表示总在回显区中显示文档。
- 'eldoc-documentation-strategy'：该可定制变量的值是用于检索并显示光标点处符号文档的函数。文档由钩子 'eldoc-documentation-functions' 中的函数生成。'eldoc-documentation-strategy' 的默认值规定 ElDoc 应显示 'eldoc-documentation-functions' 钩子中函数生成的第一个文档文本，但你可以定制 'eldoc-documentation-strategy'，使其以其他方式工作，例如把所有文档文本连接起来显示。
- 'eldoc-documentation-functions'：该异常钩子的值是一个函数列表，这些函数能按照当前缓冲区主模式的相应方式，为光标点处的符号生成文档。这些函数充当 ElDoc 的一组后端。主模式通过把其函数添加到该变量的缓冲区局部值中，向 ElDoc 注册自己的文档查询函数。

> 注："ElDoc"这个名字是历史偶然：该模式最初只支持 Emacs Lisp 缓冲区。

28.7 Hideshow 次模式

Hideshow 模式是一个缓冲区局部的次模式，让你能有选择地显示程序的某些部分，这些部分称为"块"（block）。键入 'M-x hs-minor-mode' 可开关该次模式（参见"次模式"）。

当你用 Hideshow 模式隐藏一个块时，该块从屏幕上消失，代之以一个省略号（连续三个句点）。究竟什么构成一个块取决于主模式。在 C 模式及相关模式中，块由花括号界定；而在 Lisp 模式中，块由圆括号界定。多行注释也算作块。

此外，Hideshow 模式支持可选的、基于缩进的隐藏/显示。默认情况下它是禁用的；要启用它，请打开缓冲区局部的次模式 'hs-indentation-mode'。启用 'hs-indentation-mode' 并不要求 'hs-minor-mode' 已经启用。

Hideshow 模式提供以下命令（定义在 'hs-prefix-map' 中）：

- 'C-c @ C-h'、'C-c @ C-d'：隐藏当前块（'hs-hide-block'）。
- 'C-c @ C-s'：显示当前块（'hs-show-block'）。
- 'C-c @ C-c'、'C-c @ C-e'、'S-mouse-2'：隐藏或显示当前块（'hs-toggle-hiding'）。
- 'C-c @ C-M-h'、'C-c @ C-t'：隐藏所有顶层块（'hs-hide-all'）。
- 'C-c @ C-M-s'、'C-c @ C-a'：显示缓冲区中的所有块（'hs-show-all'）。
- 'C-u N C-c @ C-l'：隐藏此块之下 N 层的所有块（'hs-hide-level'）。
- 'C-c @ TAB'：在当前块的各可见性状态之间循环（'hs-cycle'）。
- 'C-c @ <backtab>'：隐藏或显示当前缓冲区中的所有块（'hs-toggle-all'）。

以下变量可用于定制 Hideshow 模式：

- 'hs-hide-comments-when-hiding-all'：若非 'nil'，'hs-hide-all'、'hs-cycle' 和 'hs-hide-level' 也会隐藏注释。
- 'hs-hide-block-behavior'：该变量控制 'hs-hide-block'、'hs-cycle'、'hs-hide-level' 和 'hs-toggle-hiding' 应如何隐藏一个块。可用的值为 'after-bol'（隐藏当前行所属的最内层块）或 'after-point'（隐藏光标点之后的块）。
- 'hs-display-lines-hidden'：若非 'nil'，在省略号旁边显示隐藏的行数。
- 'hs-show-indicators'：该变量控制 Hideshow 模式是否显示隐藏块和显示块的指示符。这些指示符还允许切换每个块的隐藏/显示状态。若值为非 'nil'，则启用指示符。默认值为 'nil'，即禁用指示符。
- 'hs-indicator-type'：该变量控制（若已启用指示符）在何处显示指示符。你可以把它们显示在边缘区（fringe，参见"窗口边缘"）或窗口的边距中。默认是若有边缘区则使用边缘区，否则使用边距。
- 'hs-indicator-maximum-buffer-size'：该变量把 hideshow 指示符的显示限制在不太大的缓冲区中。（较大的缓冲区可能对重绘性能产生不利影响。）默认情况下，大于 2MB 的缓冲区会禁用指示符；值 'nil' 则不论缓冲区大小都启用指示符。
- 'hs-cycle-filter'：该变量控制：在含有可隐藏块的行上，键入 '<TAB>' 键会在该行的什么位置循环切换块的可见性。依据其非 'nil' 的值，<TAB> 可以在此类行的不同部分生效。行上其他任何位置，<TAB> 都保持其默认键绑定。值 'nil' 表示 <TAB> 不能在标题行的任何位置循环切换块可见性。
- 'hs-isearch-open'：该变量指定在何种条件下，当增量搜索在某个隐藏块内匹配到文本时应当取消隐藏该块。其值应为 'code'（只取消隐藏代码块）、'comment'（只取消隐藏注释）、't'（代码块和注释都取消隐藏）或 'nil'（代码块和注释都不取消隐藏）。默认值为 'code'。

- ‘hs-indentation-respect-end-block’：该变量控制块的结尾是否应与被隐藏的区域一起隐藏。这仅在启用 ‘hs-indentation-mode’ 时才有作用。

28.8 符号名补全

补全通常在迷你缓冲区中进行（参见“补全”），但你也可以在普通 Emacs 缓冲区中补全符号名。

在大多数编程语言模式中，‘C-M-i’（或 ‘M-<TAB>’(1)）调用命令 ‘completion-at-point’，它为光标点处的符号生成可能的补全列表。该命令使用可用的支持设施来得出补全候选：

- 若针对当前缓冲区的项目（参见“使用项目”）以及当前缓冲区的主模式激活了 Eglot，该命令会尝试使用相应的语言服务器来生成补全候选列表。参见《Eglot 手册》的“Eglot Features”。
- 若启用了 Semantic 模式（参见“Semantic”），该命令会尝试使用 Semantic 的解析器数据来补全。
- 若未启用 Semantic 模式，或它执行补全失败，该命令会尝试使用选定的标签表来补全（参见“标签表”）；要让这生效，需要用 ‘M-x visit-tags-table’ 访问该标签表。
- 在 Emacs Lisp 模式中，该命令使用当前 Emacs 会话中定义的函数、变量或属性名来补全。

在其他所有方面，缓冲区内的符号补全与迷你缓冲区补全行为相同。例如，若 Emacs 无法补全到唯一符号，它会在另一个窗口中显示补全候选列表。之后你可以用 ‘M-<DOWN>’ 和 ‘M-<UP>’ 键在补全缓冲区中显示的候选之间浏览，而不必离开原缓冲区；用 ‘M-<RET>’ 键把当前高亮的候选插入缓冲区。参见“补全”。

在文本模式及相关模式中，‘M-<TAB>’ 基于拼写检查器的词典补全单词。参见“拼写检查与纠正”。

补全预览模式（Completion Preview mode）是一个次模式，在你键入时显示补全建议。你可以用 ‘M-x completion-preview-mode’ 为当前缓冲区启用它，或用 ‘M-x global-completion-preview-mode’ 全局启用。补全预览模式开启时，Emacs 会自动把光标点附近文本的建议补全，以行内预览的形式显示在光标点之后；键入 <TAB> 可接受该建议。

> 注：在图形显示上，‘M-<TAB>’ 键通常被窗口管理器保留用于切换图形窗口，因此你应改为键入 ‘C-M-i’ 或 ‘<ESC> <TAB>’。

28.9 混合大小写单词

某些编程风格使用混合大小写（或称“CamelCase”，驼峰式）的符号，如 ‘unReadableSymbol’。（在 GNU 工程中，我们建议在标识符内用下划线分隔单词，而不要用大小写区分。）Emacs 有各种功能，使处理这类符号更容易。

Glasses 模式是一个缓冲区局部的次模式，它通过改变这类符号的显示方式使其更易读。默认情况下，它在每个小写字母与其后的大写字母之间显示额外的下划线。这不会改变缓冲区文本，只改变显示方式。

要开关 Glasses 模式，键入 ‘M-x glasses-mode’（参见“次模式”）。启用 Glasses 模式时，模式行中会出现次模式指示符 ‘o^o’。关于 Glasses 模式的更多信息，键入 ‘C-h P glasses <RET>’。

子词模式（Subword mode）是另一个缓冲区局部的次模式。在子词模式中，Emacs 的单词命令把 ‘StudlyCapsIdentifiers’ 中的大写字母识别为单词边界。启用子词模式时，模式行中会出现次模式指示符 ‘,’。另

请参见类似的‘superword-mode’（参见“编辑程序的其他有用功能”）。

28.10 Semantic

Semantic 是一个包，它基于“源代码解析器”（source code parsers）提供能感知语言的编辑命令。本节简要介绍 Semantic；完整细节参见《Semantic 手册》。

Emacs 中大多数能感知语言的功能，例如 Font Lock 模式（参见“Font Lock 模式”），都依赖于经验规则(1)，这些规则通常效果不错，但从来都不完全精确。相比之下，Semantic 使用的解析器能精确理解编程语言的语法。这使 Semantic 能够提供强大而精确的搜索、导航和补全命令。

要开始使用 Semantic，输入‘M-x semantic-mode’，或点击‘Tools’菜单中名为‘Source Code Parsers (Semantic)’的菜单项。这会启用 Semantic 模式，它是一种全局次模式。

启用 Semantic 模式后，Emacs 会自动尝试解析你访问的每个文件。目前 Semantic 能理解 C、C++、HTML、Java、Javascript、Make、Python、Scheme、SRecode 和 Texinfo。在每个已解析的缓冲区中，可以使用以下命令：

‘C-c , j’提示输入当前文件中定义的函数名，并把光标点移到那里（‘semantic-complete-jump-local’）。

‘C-c , J’提示输入 Emacs 已解析的任意文件中定义的函数名，并把光标点移到那里（‘semantic-complete-jump’）。

‘C-c , <SPC>’显示光标点处符号的可能补全列表（‘semantic-complete-analyze-inline’）。这还会激活一组用于选择补全的特殊键绑定：‘<RET>’接受当前补全，‘M-n’和‘M-p’在可能的补全之间循环，‘<TAB>’尽可能补全然后循环，而‘C-g’或任何其他键会中止补全。

‘C-c , l’在另一个窗口中显示光标点处符号的可能补全列表（‘semantic-analyze-possible-completions’）。

除上述命令外，Semantic 包还提供多种利用解析器信息的方式。例如，你可以用它来在 Emacs 空闲时显示补全列表。参见《Semantic 手册》的“Top, for details”

> 注：正则表达式与语法表。

28.11 编辑程序的其他有用功能

有些 Emacs 命令并非专为编辑程序而设计，但用来编辑程序仍然很有用。

用于操作单词、句子和段落的 Emacs 命令对编辑代码很有用。多数符号名中含有单词（参见“单词”），而句子可以在字符串和注释中找到（参见“句子”）。至于段落，在多数编程语言模式中，段落被定义为以空行开始和结束（参见“段落”）。因此，有意识地使用空行使程序更清晰，同时也会为段落命令提供可操作的文本块。若在编程语言主模式中启用自动填充模式（Auto Fill mode），它会把所创建的新行缩进。

Superword 模式是一种缓冲区局部次模式，它使编辑和移动命令把符号（例如‘this_is_a_symbol’）当作单词处理。启用 Superword 模式时，模式行中会出现次模式指示符‘²’。另请参见类似的‘subword-mode’（参见“混合大小写单词”）。

电布局模式（Electric Layout mode，‘M-x electric-layout-mode’）是一种全局次模式，在你输入某些字符时会自动插入换行；例如在 Javascript 模式中的 ‘{’、‘}’ 和 ‘;’。

除了 Hideshow 模式（参见“Hideshow 次模式”），另一种有选择地显示程序各部分的方法是使用选择性显示功能（参见“选择性显示”）。编程模式通常还支持大纲次模式（参见“大纲模式”），它可以与 Foldout 包一起使用（参见“折叠编辑”）。

自动输入功能对编写程序也可能有用。参见《Autotype 手册》

美化符号模式（Prettify Symbols mode）是一种缓冲区局部次模式，它出于显示目的把某些字符串替换为更美观的版本。例如在 Emacs Lisp 模式中，它把字符串 ‘lambda’ 替换为希腊字母 lambda ‘λ’。在 TeX 缓冲区中，它会把 ‘\alpha’……‘\omega’ 以及其他数学宏替换为对应的 Unicode 字符。你也可能希望在非编程模式中使用它。可以通过向 ‘prettify-symbols-alist’ 添加更多条目来自定义该模式。如果 ‘prettify-symbols-compose-predicate’ 的默认值 ‘prettify-symbols-default-compose-p’ 不合适，也可以自定义该变量来实现更精细的定制。此外还有全局版本 ‘global-prettify-symbols-mode’，它在所有支持该模式的缓冲区中启用它。

光标点处的符号可以以其原始形式显示。这由变量 ‘prettify-symbols-unprettify-at-point’ 控制：若其值非 ‘nil’，则只要光标点停留在该符号上，它就会恢复为原始形式。

28.12 C 及相关模式

本节简要介绍 C、C++、Objective-C、Java、CORBA IDL、Pike 和 AWK 模式中可用的特殊功能。（这些模式统称为“C 模式及相关模式”。）更多细节参见《《ccmode》手册》。

- Other C Commands:: 填充注释、查看宏展开以及其他实用功能。

28.12.1 C 模式的移动命令

本节介绍在 C 模式及相关模式中移动光标点的命令。

‘C-M-a’ ‘C-M-e’ 把光标点移到当前函数或顶层定义的开头或结尾。在具有外围作用域的语言中（如 C++ 的类），“当前函数”指最内层的那个，它可能位于某个作用域内部。否则它指由最外层包围花括号所定义的那个函数。（相比之下，‘beginning-of-defun’ 和 ‘end-of-defun’ 会搜索位于第 0 列的花括号。）参见“按 defun 移动”。

‘C-c C-u’ 把光标点向后移到包含它的预处理条件处，并留下标记。前缀参数用作重复次数。参数为负时，则把光标点向前移到包含它的预处理条件的末尾。

‘#elif’ 等价于 ‘#else’ 后跟 ‘#if’，因此向后移动时函数会停在 ‘#elif’ 处，而向前移动时不会。

‘C-c C-p’ 把光标点向后移过一个预处理条件，并留下标记。前缀参数用作重复次数。参数为负时，则向前移动。

‘C-c C-n’ 把光标点向前移过一个预处理条件，并留下标记。前缀参数用作重复次数。参数为负时，则向后移动。

‘M-a’ 把光标点移到最内层 C 语句的开头（‘c-beginning-of-statement’）。若光标点已在语句开头，则移到前一条语句的开头。带前缀参数 N 时，向后移动 N − 1 条语句。

在注释或跨越多行的字符串中，此命令按句子而不是按语句移动。

‘M-e’ 把光标点移到最内层 C 语句或句子的末尾；与 ‘M-a’ 类似，只是移动方向相反（‘c-end-of-statement’）。

28.12.2 C 模式中的电击式字符

在 C 模式及相关模式中，某些可打印字符是“电击式”（electric）的——除插入自身之外，它们还会重新缩进当前行，并且可选地插入换行。这些电击式字符是 ‘{’、‘}’、‘:’、‘#’、‘;’、‘,’、‘<’、‘>’、‘/’、‘*’、‘(’ 和 ‘)’。

如果你正在编辑缩进混乱的代码，可能会觉得电击式缩进不便。如果你是 CC 模式的新手，可能会对此感到不适。可以用命令 ‘C-c C-l’ 开关电击式行为；启用时，模式行中模式名之后会出现 ‘/Cl’（其中 C 若出现，则为 ‘*’ 或 ‘/’，取决于注释风格是块注释还是行注释）。关于 CC 模式中模式行指示符的更多信息，参见《《ccmode》手册》的“Minor Modes”。

‘C-c C-l’ 开关电击式行为（‘c-toggle-electric-state’）。前缀参数为正时，此命令启用点击式行为；为负时禁用它。

只有当电击状态之外还启用了“自动换行”（auto-newline）功能时，电击式字符才会插入换行（模式行中模式名之后显示 ‘/Cla’ 即表示已启用）。可以用命令 ‘C-c C-a’ 打开或关闭此功能：

‘C-c C-a’ 开关自动换行功能（‘c-toggle-auto-newline’）。带前缀参数时，参数为正则打开自动换行功能，为负则关闭。

通常由 CC 模式的风格（style）来配置 Emacs 插入自动换行的确切情形。你也可以直接配置。参见《《ccmode》手册》的“Custom Auto-newlines”。

28.12.3 C 模式中的饥饿删除功能

如果想删除光标点处的一整块空白，可以使用“饥饿删除”（hungry deletion）。它在一次操作中删除光标点之前或之后的全部连续空白。这里的“空白”包括制表符和换行，但不包括注释或预处理命令。

‘C-c C-<DEL>’ ‘C-c <DEL>’ 删除光标点之前的整块空白（‘c-hungry-delete-backwards’）。

‘C-c C-d’ ‘C-c C-<Delete>’ ‘C-c <Delete>’ 删除光标点之后的整块空白（‘c-hungry-delete-forward’）。

作为上述命令的替代，你可以启用“饥饿删除模式”。启用此功能时（模式行中模式名之后的 ‘/’ 后面显示 ‘h’），单次 ‘<DEL>’ 会删除之前的所有空白，而不仅仅是一个空格；单次 ‘C-d’（但 *不是* 普通的 ‘<Delete>’）会删除之后的所有空白。

‘M-x c-toggle-hungry-state’ 开关饥饿删除功能（‘c-toggle-hungry-state’）。带前缀参数时，参数为正则打开饥饿删除功能，为负则关闭。

变量 ‘c-hungry-delete-key’ 控制是否启用饥饿删除功能。

28.12.4 C 模式的其他命令

‘M-x c-context-line-break’ 此命令插入换行，并按适合上下文的方式缩进新行。在普通代码中，它完成 ‘<RET>’（‘newline’）的工作；在 C 预处理行中，它还会在换行处插入 ‘\’；在注释内部，它等同于 ‘M-j’（‘c-indent-new-comment-line’）。

‘c-context-line-break’ 默认没有绑定到任何键，但只有绑定后才用得上。以下代码把它绑定到 ‘<RET>’。这里我们使用 ‘c-initialization-hook’，以确保在尝试修改键位图之前它已被加载。

```
(defun my-bind-clb ()
  (keymap-set c-mode-base-map "RET"
              'c-context-line-break))
(add-hook 'c-initialization-hook 'my-bind-clb)
```

‘C-M-h’ 把标记放在函数定义末尾，并把光标点放在开头（‘c-mark-function’）。

‘M-q’ 填充段落，处理 C 和 C++ 注释（‘c-fill-paragraph’）。若当前行的任何部分属于注释或位于注释内部，此命令会填充该注释或光标点所在的注释段落，并保留注释的缩进与注释定界符。

‘C-c C-e’ 对区域中的文本运行 C 预处理器并显示结果，其中包含所有宏调用的展开（‘c-macro-expand’）。为了让在区域之前定义的宏生效，区域之前的缓冲区文本也会参与预处理，但这部分的输出不会显示。

在调试使用宏的 C 代码时，有时很难弄清宏究竟是如何展开的。有了这条命令，你不必自己去想明白，可以直接看到展开结果。

‘C-c C-\’ 在区域各行的行尾插入或对齐 ‘\’ 字符（‘c-backslash-region’）。这在编写或编辑 C 宏定义之后很有用。

若某行已以 ‘\’ 结尾，此命令会调整其前面的空白数量。否则，它插入一个新的 ‘\’。不过，区域中的最后一行被特殊处理：该行不插入 ‘\’，且其中原有的 ‘\’ 会被删除。

‘M-x cpp-highlight-buffer’ 根据文本的预处理条件来高亮文本的各个部分。此命令会显示另一个名为 ‘*CPP Edit*’ 的缓冲区，它充当图形化菜单，用于选择如何显示特定种类的条件及其内容。修改各项设置后，点击 ‘[A]pply these settings’（或切换到该缓冲区并输入 ‘a’），即可相应地重新高亮 C 模式缓冲区。

‘C-c C-s’ 显示关于当前源代码行的语法信息（‘c-show-syntactic-information’）。这些信息决定该行如何缩进。

‘M-x cwarn-mode’ ‘M-x global-cwarn-mode’ CWarn 次模式会高亮某些可疑的 C 和 C++ 结构：

- 表达式内部的赋值。
- 紧跟在 ‘if’、‘for’ 和 ‘while’ 之后的分号（‘do ... while’ 语句之后除外）；
- 带引用参数的 C++ 函数。

可以用命令 ‘M-x cwarn-mode’ 为单个缓冲区启用该模式，或用命令 ‘M-x global-cwarn-mode’、或通过自定义变量 ‘global-cwarn-mode’ 为所有合适的缓冲区启用。还必须启用字体锁定模式（Font Lock mode），它才能工作。

‘M-x hide-ifdef-mode’ Hide-ifdef 次模式隐藏 ‘#if’ 和 ‘#ifdef’ 预处理块中的选定代码。若把变量 ‘hide-ifdef-shadow’ 改为 ‘t’，则 Hide-ifdef 次模式会通过以不太显眼的字体（face）显示预处理块来遮蔽它们，而不是完全隐藏。更多信息参见 ‘hide-ifdef-mode’ 的文档字符串。

‘M-x ff-find-related-file’ 查找与当前缓冲区所访问文件有特殊关联的文件。通常这是与某个 C/C++ 源文件对应的头文件，或反之。变量 ‘ff-related-file-alist’ 指定如何计算相关文件名。

28.13 Asm 模式

Asm 模式是用于编辑汇编代码文件的主模式。它定义了以下命令：

‘<TAB>’ ‘tab-to-tab-stop’。

‘C-j’ 插入换行，然后用 ‘tab-to-tab-stop’ 缩进。

‘:’ 插入冒号，然后去掉冒号前那个标号之前的缩进，再执行 ‘tab-to-tab-stop’。

‘;’ 插入或对齐注释。

变量 ‘asm-comment-char’ 指定汇编语法中以哪个字符开始注释。

28.14 Fortran 模式

Fortran 模式用于编辑固定格式（以及制表符格式）的源代码（通常是 Fortran 77）。要编辑更现代的自由格式源代码（Fortran 90、95、2003、2008），请使用 F90 模式（‘f90-mode’）。对于扩展名为 ‘.f’、‘.F’ 或 ‘.for’ 的文件，Emacs 通常使用 Fortran 模式；对于扩展名 ‘.f90’、‘.f95’、‘.f03’ 和 ‘.f08’，则使用 F90 模式。可自定义 ‘auto-mode-alist’ 以添加更多扩展名。GNU Fortran 同时支持自由格式和固定格式。本手册主要记述 Fortran 模式，但相关处也会提及 F90 模式中相应的功能。

Fortran 模式为 Fortran 语句和子程序提供了专门的移动命令，以及能理解 Fortran 嵌套、行号和续行约定的缩进命令。Fortran 模式支持自动填充模式，它把过长的行拆分为符合 Fortran 规范的续行。Fortran 模式还支持 Hideshow 次模式（参见“Hideshow 次模式”）和 Imenu（参见“Imenu”）。

由于 Fortran 的注释与其他语言不同，因此提供了专门的注释命令。内置缩写可选择性地在你插入 Fortran 关键字时省去打字。

使用 ‘M-x fortran-mode’ 切换到此主模式。此命令会运行钩子 ‘fortran-mode-hook’。参见“钩子”。

28.14.1 移动命令

除了用于按 defun 移动和操作的常规命令（Fortran 子程序——函数和子程序，以及 F90 模式中的模块，使用命令 ‘fortran-end-of-subprogram’ 和 ‘fortran-beginning-of-subprogram’）之外，Fortran 模式还提供了按语句和其他程序单元移动的专门命令。

‘C-c C-n’ 移到下一条语句的开头（‘fortran-next-statement’/‘f90-next-statement’）。

‘C-c C-p’ 移到上一条语句的开头（‘fortran-previous-statement’/‘f90-previous-statement’）。若没有上一条语句（即从缓冲区中的第一条语句处调用），则移到缓冲区开头。

‘C-c C-e’ 把光标点向前移到下一个代码块的开头，或当前代码块的末尾，以先到者为准（‘f90-next-block’）。代码块可以是子程序、‘if’-‘endif’ 语句等等。此命令仅存在于 F90 模式，Fortran 模式没有。带数值参数时，向前移动相应数量的代码块。

‘C-c C-a’ 把光标点向后移到上一个代码块（‘f90-previous-block’）。这与 ‘f90-next-block’ 类似，但向后移动。

‘C-M-n’ 移到当前代码块的末尾（‘fortran-end-of-block’/‘f90-end-of-block’）。带数值参数时，向前移动相应数量的代码块。移动光标点之前会先设置标记。此命令的 F90 模式版本会检查块类型和标号（若存在）是否一致，但不检查最外层代码块，因为它可能并不完整。

‘C-M-p’ 移到当前代码块的开头（‘fortran-beginning-of-block’/‘f90-beginning-of-block’）。这与 ‘fortran-end-of-block’ 类似，但向后移动。

命令 ‘fortran-beginning-of-subprogram’ 和 ‘fortran-end-of-subprogram’ 分别移到当前子程序的开头或末尾。命令 ‘fortran-mark-do’ 和 ‘fortran-mark-if’ 标记当前 ‘do’ 或 ‘if’ 块的末尾，并把光标点移到开头。

28.14.2 Fortran 缩进

缩进固定格式（或制表符格式）的 Fortran 代码需要专门的命令与功能，以确保各种语法要素（行号、注释行指示符和续行标志）出现在规定的列上。

28.14.2.1 Fortran 缩进与填充命令

‘C-M-j’ 在光标点处断开当前行，并建立续行（‘fortran-split-line’）。

‘M-^’ 把本行接到上一行（‘fortran-join-line’）。

‘C-M-q’ 缩进光标点所在子程序的所有行（‘fortran-indent-subprogram’）。

‘M-q’ 填充注释块或语句（使用 ‘fortran-fill-paragraph’ 或 ‘fortran-fill-statement’）。

按键 ‘C-M-q’ 运行 ‘fortran-indent-subprogram’，该命令重新缩进包含光标点的 Fortran 子程序（函数或子程序）的所有行。

按键 ‘C-M-j’ 运行 ‘fortran-split-line’，它以适合 Fortran 的方式拆分一行。在非注释行中，后半部分成为续行并相应缩进。在注释行中，两半各自成为独立的注释行。

‘M-^’ 或 ‘C-c C-d’ 运行命令 ‘fortran-join-line’，它把续行接回上一行，大致相当于 ‘fortran-split-line’ 的逆操作。调用此命令时光标点必须位于续行上。

Fortran 模式中的 ‘M-q’ 填充光标点所在的注释块或语句。这会去掉多余的语句续行。

28.14.2.2 续行

多数 Fortran 77 编译器允许两种书写续行的方式。若一行中第一个非空格字符位于第 5 列，则该行是上一行的续行。我们称这种方式为“固定格式”。（在 GNU Emacs 中我们总是从 0 开始计数列；但要注意 Fortran 标准是从 1 开始计数。你可以自定义变量 ‘column-number-indicator-zero-based’，使列号显示方式与 Fortran 一致；参见“可选的模式行功能”。）变量 ‘fortran-continuation-string’ 指定在第 5 列放入什么字符。以制表符开头、后跟除 ‘0’ 之外的任意数字的行也是续行。我们称这种续行风格为“制表符格式”。（Fortran 90 引入了自由格式续行。）

Fortran 模式可以使用任一种续行风格。进入 Fortran 模式时，它会尝试根据缓冲区内容自动推断合适的续行风格。做法是从缓冲区开头起最多扫描 ‘fortran-analyze-depth’ 行（默认 100 行）。第一行以制表符或六个空格开头者决定所选风格。若扫描失败（例如缓冲区是新建的、因而为空），则使用 ‘fortran-tab-mode-default’ 的值（固定格式为 ‘nil’，制表符格式为非 ‘nil’）。模式行中的 ‘/t’（‘fortran-tab-mode-string’）表示已选用制表符格式。Fortran 模式会相应地设置 ‘indent-tabs-mode’ 的值。

若某行的文本以 Fortran 续行标记 ‘$’ 开头，或在第 5 列以任意非空白字符开头，Fortran 模式就把它视为续行。用 ‘<TAB>’ 缩进续行时，它会把该行转换为当前的续行风格。用 ‘C-M-j’ 拆分 Fortran 语句时，新行上的续行标记会按续行风格创建。

续行风格的设置会影响 Fortran 模式中编辑的若干其他方面。在固定格式模式中，语句主体的最小列号为 6。Fortran 块中缩进到更大列号的行，其空白必须只使用空格字符。在制表符格式模式中，语句主体的最小列号为 8，且第 8 列之前的空白必须由一个制表符构成。

28.14.2.3 行号

若一行中第一个非空白字符是数字，Fortran 缩进就假定它是行号，并把它移到第 0 至第 4 列。（在 Emacs 中列总是从 0 计数，但把 'column-number-indicator-zero-based' 设为 'nil' 可以改变这一点，参见"可选的模式行功能"。）

四位或更少的行号通常缩进一个空格。变量 'fortran-line-number-indent' 控制这一点；它指定行号可以有的最大缩进量。该变量的默认值为 1。Fortran 模式会设法防止行号的数字越过第 4 列，必要时把缩进减小到指定最大值以下。若 'fortran-line-number-indent' 的值为 5，则行号右对齐，结束于第 4 列。

仅仅插入行号就足以按这些规则对它缩进。每插入一个数字，缩进都会重新计算。要关闭此功能，把变量 'fortran-electric-line-number' 设为 'nil'。

28.14.2.4 语法约定

Fortran 模式假定你遵循某些约定，这些约定简化了充分理解 Fortran 程序以便正确缩进它的任务：

- 两个嵌套的 'do' 循环绝不共用一个 'continue' 语句。
- Fortran 关键字如 'if'、'else'、'then'、'do' 等书写时中间不夹空白或换行。

Fortran 编译器通常忽略字符串常量之外的空白，但若这些关键字不连续，Fortran 模式就不会识别它们。'else if' 或 'end do' 之类的构造是可以接受的，但第二个词应与第一个词在同一行，而不能放在续行上。

若不遵守这些约定，缩进命令可能会把某些行缩进得很难看。不过，即使没有遵守这些约定，正确的 Fortran 程序在重新缩进后也仍能保持其含义。

28.14.2.5 Fortran 缩进所用的变量

还有几个额外的变量控制 Fortran 缩进的工作方式：

- 'fortran-do-indent'：每一层 'do' 语句内部的额外缩进量（默认值为 3）。
- 'fortran-if-indent'：每一层 'if'、'select case' 或 'where' 语句内部的额外缩进量（默认值为 3）。
- 'fortran-structure-indent'：每一层 'structure'、'union'、'map' 或 'interface' 语句内部的额外缩进量（默认值为 3）。
- 'fortran-continuation-indent'：续行主体的额外缩进量（默认值为 5）。
- 'fortran-check-all-num-for-matching-do'：在 Fortran 77 中，带标号的 'do' 语句由任何行号与之匹配的语句终止。为此通常（但并非必须）使用 'continue' 语句。若此变量的值为非 'nil'，则缩进任何带标号的语句时都必须检查是否有 'do' 在那里结束。如果你总是用一行 'continue'（或者使用更现代的 'enddo'）来结束 'do' 语句，那么把此变量设为 'nil'（默认值）就可以加快缩进速度。
- 'fortran-blink-matching-if'：若此变量为 't'，则缩进 'endif'（或 'enddo'）语句时会短暂地把光标移到与之匹配的 'if'（或 'do'）语句处，以显示它的位置。默认值为 'nil'。

- ‘fortran-minimum-statement-indent-fixed’：使用固定格式续行风格时 Fortran 语句的最小缩进量。语句主体的缩进绝不会小于此值。默认值为 6。
- ‘fortran-minimum-statement-indent-tab’：使用制表符格式续行风格时 Fortran 语句的最小缩进量。语句主体的缩进绝不会小于此值。默认值为 8。

下一节介绍控制注释缩进的变量。

28.14.3 Fortran 注释

通常的 Emacs 注释命令假定注释可以跟在一行代码之后。而在 Fortran 77 中，标准注释语法要求整行都只是注释。因此，Fortran 模式替换了标准的 Emacs 注释命令，并定义了一些新变量。

Fortran 模式也能处理 Fortran 90 的注释语法，其中注释以 ‘!’ 开头，并且可以跟在其他文本之后。由于只有部分 Fortran 77 编译器接受这种语法，除非你事先声明要这样做，否则 Fortran 模式不会插入这类注释。为此，请把变量 ‘fortran-comment-line-start’ 设为 “"!"”。若你使用不寻常的值，可能还需要修改 ‘fortran-comment-line-start-skip’。

- ‘M-;’：对齐注释或插入新注释（‘comment-dwim’）。
- ‘C-x ;’：只适用于非标准的 ‘!’ 注释（‘comment-set-column’）。
- ‘C-c ;’：把区域中所有行变成注释；带参数时则把它们恢复为真正的代码（‘fortran-comment-region’）。

在 Fortran 模式下，‘M-;’ 运行标准的 ‘comment-dwim’。它能识别任何类型的现有注释并适当地对齐其文本；若没有现有注释，则插入一条注释并对齐之。插入和对齐注释在 Fortran 模式下的做法与其他模式不同。

当必须插入一条新注释时，若当前行为空行，则插入一条整行注释。在非空行上，若你已声明要使用非标准的 ‘!’ 注释，就插入这样的注释。否则，在当前行的前一行插入一条整行注释。

非标准的 ‘!’ 注释像其他语言中的注释那样对齐，但整行注释则不同。在标准的整行注释中，注释定界符本身必须始终出现在第零列。可以对齐的是注释内部的文本。通过把变量 ‘fortran-comment-indent-style’ 设为下列值之一，你可以从三种对齐样式中选择：

- ‘fixed’：把文本对齐到固定的列，该列是 ‘fortran-comment-line-extra-indent’ 与最小语句缩进之和。这是默认值。最小缩进在使用制表符格式续行风格时为 ‘fortran-minimum-statement-indent-tab’，在使用固定格式风格时为 ‘fortran-minimum-statement-indent-fixed’。
- ‘relative’：像对待一行代码那样对齐文本，但要额外增加 ‘fortran-comment-line-extra-indent’ 列的缩进。
- ‘nil’：不自动移动整行注释中的文本。

此外，你还可以把变量 ‘fortran-comment-indent-char’ 设为你想要使用的单字符字符串，以指定在整行注释内部用于缩进的字符。

编译器指示行（或预处理行）的外观与注释行大致相同。不过重要的是，无论 ‘fortran-comment-indent-style’ 的值是什么，这类行都绝不能被缩进。变量 ‘fortran-directive-re’ 是一个正则表达式，用于指定哪些行是指示。匹配的行绝不会被缩进，并会获得独特的字体锁定。

正常的 Emacs 注释命令 ‘C-x ;’（‘comment-set-column’）并未被重新定义。若你使用 ‘!’ 注释，此命令可用于这类注释。否则，它在 Fortran 模式下毫无用处。

命令 ‘C-c ;’（‘fortran-comment-region’）把区域中所有行都变成注释，办法是在每一行开头插入字符串 ‘c$$$’。带数值参数时，它从区域中每一行开头删除 ‘c$$$’，从而把区域恢复为活代码。用于这类注释的字符串可以通过设置变量 ‘fortran-comment-region’ 来控制。请注意，这里有一个命令和一个变量同名的例子；该名字的这两种用法从不冲突，因为在 Lisp 和 Emacs 中，总是可以从上下文清楚地看出指的是哪一个。

28.14.4 Fortran 模式下的自动填充

Fortran 模式对自动填充模式（Auto Fill mode）有专门支持，该次模式会在你输入语句时，自动把变得过宽的语句拆开。拆分语句需要利用 ‘fortran-continuation-string’ 生成续行（参见“续行”）。当你键入 ‘<SPC>’、‘<RET>’ 或 ‘<TAB>’ 时，以及 Fortran 缩进命令中，都会发生这种拆分。你可以按常规方式在 Fortran 模式下启用自动填充。参见“自动填充模式”。

当行长超过所需宽度（‘fill-column’ 的值）时，自动填充会在空格或定界符处断行。自动填充可以断行的定界符（空白字符除外）有 ‘+’、‘-’、‘/’、‘*’、‘=’、‘<’、‘>’ 和 ‘,’。若变量 ‘fortran-break-before-delimiters’ 为 ‘nil’，断行点位于定界符之后。否则（这也是默认情况），断行点位于定界符之前。

要在所有 Fortran 缓冲区中启用自动填充，请把 ‘auto-fill-mode’ 加入 ‘fortran-mode-hook’。参见“钩子”。

28.14.5 检查 Fortran 中的列

在标准的 Fortran 77 中，第 72 列之后的一切都被忽略。大多数编译器提供一个选项来改变这一点（例如 gfortran 中的 ‘-ffixed-line-length-N’）。定制变量 ‘fortran-line-length’ 可以改变 Fortran 模式中的行长。超过这个位置的一切都会按注释进行字体锁定。（除非它位于字符串内部：超出 ‘fortran-line-length’ 的字符串会使字体锁定产生混乱。）

- ‘C-c C-r’：在当前行上方短暂显示一个列标尺（‘fortran-column-ruler’）。
- ‘C-c C-w’：临时把当前窗口水平拆分，使其宽度为 ‘fortran-line-length’ 列（‘fortran-window-create-momentarily’）。这有助于你避免让行超出 Fortran 编译器所施加的限制。
- ‘C-u C-c C-w’：把当前窗口水平拆分，使其宽度为 ‘fortran-line-length’ 列（‘fortran-window-create’）。然后你就可以继续编辑。
- ‘M-x fortran-strip-sequence-nos’：删除第 ‘fortran-line-length’ 列及之后的全部文本。

命令 ‘C-c C-r’（‘fortran-column-ruler’）在当前行上方短暂显示一个列标尺。该列标尺是两行文本，向你显示 Fortran 程序中具有特殊意义的各列的位置。方括号显示用于行号的列的范围，花括号显示用于语句主体的列的范围。列号显示在它们上方。

请注意，列号从零开始计数，这在 GNU Emacs 中一向如此（但定制 ‘column-number-indicator-zero-based’ 可以改变列号显示，使之与 Fortran 的一致；参见“可选的模式行功能”）。因此，这些列号可能比你所熟悉的列号小一；但它们所指示的行内位置对 Fortran 来说是标准的。

用于显示列标尺的文本取决于变量 ‘indent-tabs-mode’ 的值。若 ‘indent-tabs-mode’ 为 ‘nil’，则使用变量 ‘fortran-column-ruler-fixed’ 的值作为列标尺。否则，显示变量 ‘fortran-column-ruler-tab’ 的值。通过改变这些变量，你可以改变列标尺的显示。

‘C-c C-w’（‘fortran-window-create-momentarily’）临时把当前窗口水平拆分，得到一个宽 ‘fortran-line-length’ 列的窗口，以便你看到任何过长的行。键入一个空格即可恢复正常宽度。

你也可以水平拆分窗口，并让拆分保持原样继续编辑。为此，使用‘C-u C-c C-w’（‘M-x fortran-window-create’）。在这个窗口中编辑时，一旦某行过宽而不再是正确的 Fortran，你就能立即看到。

命令‘M-x fortran-strip-sequence-nos’删除当前缓冲区中所有行上第‘fortran-line-length’列及之后的全部文本。这是去掉旧序号的最简便办法。

28.14.6 Fortran 关键字缩写

Fortran 模式为常见关键字和声明提供了许多内置缩写。这些缩写与你自己定义的缩写属于同一类。要使用它们，必须先启用缩写模式（Abbrev mode）。参见“缩写”。

内置缩写在一点上与众不同：它们都以分号开头。例如，一个内置的 Fortran 缩写是代表‘continue’的‘;c’。若你插入‘;c’，然后插入一个空格或换行符之类的标点字符，那么在缩写模式已启用的前提下，‘;c’会自动展开为‘continue’。

键入‘;?’或‘;C-h’可显示所有内置 Fortran 缩写的列表以及它们所代表的含义。

29 编译与测试程序

上一章讨论了对修改程序有用的 Emacs 命令。本章讨论协助编译和测试程序过程的命令。

- 编译（Compilation）：用 Lisp 之外的语言（C、Pascal 等）编译程序。
- 编译模式（Compilation Mode）：用于访问编译器错误的模式。
- 编译用的 Shell（Compilation Shell）：为在编译缓冲区中使用而正确配置你的 shell。
- Grep 搜索（Grep Searching）：用 grep 搜索。
- Flymake：即时查找语法错误。
- 调试器（Debuggers）：为非 Lisp 程序运行符号调试器。
- 执行 Lisp（Executing Lisp）：编辑 Lisp 程序的各种模式，各带运行 Lisp 程序的不同设施。
- 库（Lisp Libraries）：Lisp 程序如何加载进 Emacs。
- 求值（Lisp Eval）：在 Emacs 中执行单个 Lisp 表达式。
- 交互（Lisp Interaction）：在 Emacs 缓冲区中执行 Lisp。
- 外部 Lisp（External Lisp）：通过 Emacs 与独立的 Lisp 通信。

29.1 在 Emacs 中运行编译

Emacs 能为 C、Fortran 之类的语言运行编译器，把编译日志送入一个 Emacs 缓冲区。它还能解析错误消息，并向你显示错误发生的位置。

- ‘M-x compile’：在 Emacs 下异步运行编译器，错误消息送往 ‘*compilation*’ 缓冲区。
- ‘M-x recompile’、‘g’（Compilation mode）：用与上次调用 ‘M-x compile’ 相同的命令调用编译器。
- ‘M-x kill-compilation’：杀掉正在运行的编译子进程。

要运行 ‘make’ 或其他编译命令，键入 ‘M-x compile’。它用迷你缓冲区读取一条 shell 命令行，然后通过把 shell 当作 Emacs 的子进程（“劣质进程”，inferior process）运行来执行该命令。输出被插入名为 ‘*compilation*’ 的缓冲区。当前缓冲区的默认目录被用作执行该命令的工作目录，因此默认情况下编译就在该目录中进行。

以带前缀参数的方式调用时，‘*compilation*’ 缓冲区以 Comint 模式作为其主模式（参见“Shell 模式”）。默认情况下，Comint 模式有一个很好的性质：它会在其内容中查找任何凭据提示，一旦出现就让 Emacs 询问密码。当编译命令需要这类凭据时，这很有用。

默认的编译命令是 ‘make -k’，对于用 ‘make’ 工具编译的程序通常是对的（‘-k’ 标志告诉 ‘make’ 在出错后尽可能继续编译）。参见《《make》手册》 如果你以前执行过 ‘M-x compile’，你指定的命令会自动保存在变量 ‘compile-command’ 中；下次键入 ‘M-x compile’ 时它被用作默认值。文件也可以为 ‘compile-command’ 指定文件局部值（参见“文件中的局部变量”）。

启动编译会在另一个窗口中显示 ‘*compilation*’ 缓冲区，但不选中它。编译运行期间，‘*compilation*’ 缓冲区的主模式指示符中显示 ‘run’ 一词，所有模式行中都出现 ‘Compiling’ 一词。编译运行时你不必让 ‘*compilation*’ 缓冲区保持可见；无论如何它都会继续。编译结束时，无论出于何种原因，‘*compilation*’ 缓冲区的模式行都会改为显示 ‘exit’（后跟退出码：正常退出为 ‘[0]’），或者 ‘signal’（如果有信号终止了进程）。

如果你想在编译记录出现时观看它，切换到‘*compilation*’缓冲区并把光标点移到缓冲区末尾。当光标点在末尾时，新的编译输出会插入到光标点上方，而光标点仍留在末尾。否则，光标点保持不动，而编译输出被添加到缓冲区末尾。

编译进行期间，模式行显示编译器到目前为止发出的错误、警告和信息消息的数目。

如果你把变量‘compilation-scroll-output’改为非‘nil’值，‘*compilation*’缓冲区会自动滚动以跟随输出。若值为‘first-error’，滚动在第一个错误出现时停止，并把光标点留在该错误处。对于任何其他非‘nil’值，滚动会持续到不再有输出为止。

要用上次相同的命令重新运行编译，键入‘M-x recompile’。它重用上次调用‘M-x compile’时的编译命令。它也重用‘*compilation*’缓冲区，并在其默认目录中启动编译，该目录就是上一次编译启动时所在的目录。在‘*compilation*’缓冲区中，该命令绑定到‘g’。

启动新的编译也会杀掉‘*compilation*’中已在运行的任何编译，因为该缓冲区一次只能处理一个编译。不过，‘M-x compile’和‘M-x recompile’在真正杀掉正在运行的编译之前会请求确认；若想始终自动杀掉编译而不询问，把变量‘compilation-always-kill’改为‘t’。你也可以用命令‘M-x kill-compilation’杀掉编译进程。

要同时运行两个编译，先启动第一个，然后重命名‘*compilation*’缓冲区（或许用‘rename-uniquely’；参见“其他缓冲区操作”），再切换缓冲区并启动另一个编译。这会创建一个新的‘*compilation*’缓冲区。

你可以用变量‘compilation-environment’控制传给编译命令的环境。它的值是一组环境变量设置构成的列表；每个元素应是形如‘"ENVVARNAME=VALUE"’的字符串。这些环境变量设置会覆盖通常的设置。

在编译输出中显示极长的行可能拖慢 Emacs。长于‘compilation-max-output-line-length’的行，超出该限制的部分会被隐藏在一个按钮后面，点击该按钮即可显示隐藏的部分。把此变量设为‘nil’就从不隐藏任何内容。

29.2 编译模式

‘*compilation*’缓冲区使用一种名为编译模式（Compilation mode）的主模式。编译模式把缓冲区中的每条错误消息变成超链接；你可以把光标点移到它上面并键入 <RET>，或者用鼠标点击它（参见“用鼠标跟随引用”），以在另一个窗口中访问该错误消息的“位置”（locus）。位置是文件中该错误发生的具体地点。

‘*compilation*’缓冲区的外观可以通过定制用于高亮‘*compilation*’缓冲区各部分的字体（face）来控制，例如‘compilation-error’或‘compilation-warning’，分别用于错误消息和警告消息。注意，由于这些字体继承自‘error’和‘warning’字体，也可以直接定制父字体。

用‘M-x customize-group RET compilation’查看定制变量和字体的完整列表。

如果你把变量‘compilation-auto-jump-to-first-error’改为非‘nil’值，Emacs 会自动访问‘*compilation*’缓冲区中出现的第一个错误消息的位置。（此变量也可以取‘if-location-known’和‘first-known’值，它们会修改自动访问错误位置的条件。）

错误消息的实际位置由‘compilation-search-path’决定，它是一个目录名列表。默认情况下，Emacs 在编译发生所在目录下访问错误消息中指明的文件，该目录在‘compilation-search-path’的默认值中用特殊目录名‘nil’表示（即‘(nil)’）。要让 Emacs 尝试在另一个目录下查找该文件（例如编译嵌套项目时），把该目录加入‘compilation-search-path’的定制值中。若要做目录局部定制（参见“按目录的局部变量”），最好改为设置

‘compilation-search-extra-path’，Emacs 会把它与 ‘compilation-search-path’ 一并考虑。‘compilation-search-extra-path’ 中的条目优先于 ‘compilation-search-path’ 中的条目。

编译模式提供以下额外命令。这些命令也可用在 ‘*grep*’ 缓冲区中，那里的超链接是搜索匹配项而非错误消息（参见“在 Emacs 中用 Grep 搜索”）。

- ‘M-g M-n’、‘M-g n’、‘C-x `’：访问下一个错误消息或匹配项的位置（‘next-error’）。
- ‘M-g M-p’、‘M-g p’：访问上一个错误消息或匹配项的位置（‘previous-error’）。
- ‘M-n’：把光标点移到下一个错误消息或匹配项，但不访问其位置（‘compilation-next-error’）。
- ‘M-p’：把光标点移到上一个错误消息或匹配项，但不访问其位置（‘compilation-previous-error’）。
- ‘M-}’：把光标点移到出现在不同文件中的下一个错误消息或匹配项（‘compilation-next-file’）。
- ‘M-{’：把光标点移到出现在不同文件中的上一个错误消息或匹配项（‘compilation-previous-file’）。
- ‘C-c C-f’：切换 Next Error Follow 次模式，它使编译缓冲区中的光标移动自动显示源代码。
- ‘g’：重新运行其输出显示在 ‘*compilation*’ 缓冲区中的上一条命令。
- ‘M-x next-error-select-buffer’：选择一个缓冲区，供下次调用 ‘next-error’ 和 ‘previous-error’ 时使用。

要顺序访问各个错误，键入 ‘C-x `’（‘next-error’），或者等价的 ‘M-g M-n’ 或 ‘M-g n’。此命令可以从任何缓冲区调用，而不只是编译模式缓冲区。一次编译之后第一次调用它时，它访问第一个错误消息的位置。此后每次 ‘M-g M-n’ 都以类似方式访问下一个错误。如果你在 ‘*compilation*’ 缓冲区中用 <RET> 或鼠标点击访问了某个特定错误，后续的 ‘M-g M-n’ 命令就从那里向前推进。当 ‘M-g M-n’ 找不到更多可访问的错误消息时，它会发出错误信号。‘C-u M-g M-n’ 从编译缓冲区开头重新开始，并访问第一个位置。

‘M-g M-p’ 或 ‘M-g p’（‘previous-error’）沿相反方向遍历错误。

‘next-error’ 和 ‘previous-error’ 命令并不只作用于 ‘*compilation*’ 和 ‘*grep*’ 缓冲区中列出的错误或匹配项；它们也知道如何遍历其他命令产生的错误或匹配项列表，例如 ‘M-x occur’（参见“其他搜索与循环命令”）。如果当前缓冲区包含错误消息或匹配项，这些命令就遍历它们；否则，Emacs 先在所选框架的窗口中寻找包含错误消息或匹配项的缓冲区（前提是变量 ‘next-error-find-buffer-function’ 被定制为值 ‘next-error-buffer-on-selected-frame’），然后寻找先前被 ‘next-error’ 或 ‘previous-error’ 使用过的缓冲区，最后是其他所有缓冲区。这些命令所遍历的任何缓冲区若当前未显示在窗口中，就会被显示出来。你可以用 ‘next-error-select-buffer’ 命令切换到另一个缓冲区，供后续调用 ‘next-error’ 使用。

默认情况下，‘next-error’ 和 ‘previous-error’ 命令会跳过不太重要的消息。变量 ‘compilation-skip-threshold’ 控制这一点。默认值 1 表示跳过任何比警告更不重要的内容。值为 2 表示跳过任何比错误更不重要的内容，而 0 表示不跳过任何消息。

当 Emacs 访问某条错误消息的位置时，它会短暂高亮相关的源代码行。该高亮的持续时间由变量 ‘next-error-highlight’（用于所选缓冲区中的位置）和 ‘next-error-highlight-no-select’（用于非所选缓冲区中的位置）决定。此外，你还可以定制变量 ‘next-error-message-highlight’，它定义如何在包含消息的缓冲区中高亮当前错误消息。

如果 ‘*compilation*’ 缓冲区显示在带有左边缘（fringe）的窗口中（参见“窗口边缘”），访问位置的命令会在边缘中放置一个箭头，指向当前错误消息。如果窗口没有左边缘，例如在文本终端上，这些命令会滚动窗口，使当前消息位于窗口顶部。如果你把变量 ‘compilation-context-lines’ 改为 ‘t’，则改为在第零列之前插入一个可见

的箭头。如果你把该变量改为整数值 N，这些命令会滚动窗口，使当前错误消息距顶部 N 行，无论是否有边缘；默认值 'nil' 给出上面描述的行为。

编译输出有时可能非常冗长，其中许多内容对用户并非特别有用。用户选项 'compilation-hidden-output' 应当是单个正则表达式或正则表达式列表，匹配到的输出将被隐藏。例如，要隐藏递归 makefile 的冗长输出，可以这样写：

```
(setq compilation-hidden-output
      '("^make[^\n]+\n"))
```

为解析来自编译器的消息，编译模式使用变量 'compilation-error-regexp-alist'，它列出各种错误消息格式，并告诉 Emacs 如何从每种格式中提取位置。一个类似的变量 'grep-regexp-alist' 告诉 Emacs 如何解析 'grep' 命令的输出（参见"在 Emacs 中用 Grep 搜索"）。

有时 'compilation-error-regexp-alist' 不能正确判定作为错误来源的文件名。请使用用户选项 'compilation-transform-file-match-alist' 做必要的调整，例如添加或更改目录部分，甚至把某些编译器消息完全不当成错误消息。

编译模式还定义了按键 <SPC> 和 <DEL> 用于按屏滚动；'M-n'（'compilation-next-error'）和 'M-p'（'compilation-previous-error'）用于移动到下一条或上一条错误消息；以及 'M-{'（'compilation-next-file'）和 'M-}'（'compilation-previous-file'）用于移动到属于另一源文件的下一条或上一条错误消息。

你可以键入 'C-c C-f' 切换 Next Error Follow 模式。在此次模式中，编译缓冲区中的普通光标移动会自动更新源缓冲区，也就是说，把光标移到某条错误消息上会显示该错误的位置。

编译模式的功能也以一种名为编译次模式（Compilation Minor mode）的次模式提供。这让你可以解析任何缓冲区中的错误消息，而不只是普通的编译输出缓冲区。键入 'M-x compilation-minor-mode' 启用该次模式。例如，在 Rlogin 缓冲区中（参见"远程主机 Shell"），编译次模式会自动通过 FTP 访问远程源文件（参见"文件名"）。

29.3 供编译使用的子 shell

本节包含在编译缓冲区中使用 shell 及其特性的各种技巧和建议。这些材料针对本地编译，对于默认目录位于远程主机上的编译缓冲区，很可能不适用（或与之无关）。

'M-x compile' 命令用 shell 运行编译命令，但会指定非交互式 shell 的选项。这尤其意味着 shell 启动时不应有提示符。如果你发现惯用的 shell 提示符难看地出现在 '*compilation*' 缓冲区中，那说明你在 shell 的初始化文件中无条件设置了提示符，犯了一个错误。（该初始化文件可能名为 '.bashrc'、'.profile'、'.cshrc'、'.shrc' 等，取决于你使用哪种 shell。）shell 初始化文件应只在已有提示符时才设置提示符。在 bash 中可这样做：

```
if [ "${PS1+set}" = set ]
then PS1=...
fi
```

在 csh 中可这样做：

```
if ($?prompt) set prompt = ...
```

如果你想定制传给编译子 shell 的 'TERM' 环境变量的值，请定制变量 'comint-terminfo-terminal'（参见"Shell 模式选项"）。

Emacs 不指望编译器进程启动异步子进程；如果它这样做了，而这些子进程在主编译器进程终止后仍继续运行，Emacs 可能杀掉它们，或者它们的输出可能无法到达 Emacs。要避免这个问题，让主编译进程等待其子进程结束。在 shell 脚本中，可以用 '$!' 和 'wait' 做到这一点，像这样：

```
(sleep 10; echo 2nd)& pid=$!  # Record pid of subprocess
echo first message
wait $pid                     # Wait for subprocess
```

如果后台进程不向编译缓冲区输出，因而你只需防止它在主编译进程终止时被杀掉，那么这样写就足够了：

```
nohup COMMAND; sleep 1
```

在 MS-DOS 上不支持异步子进程，因此 'M-x compile' 会同步运行编译命令（也就是说，你必须等到命令结束才能在 Emacs 中做别的事）。参见"Emacs 与 MS-DOS"。

29.4 在 Emacs 中用 Grep 搜索

正如你可以在 Emacs 中运行编译器然后访问出现编译错误的行，你也可以运行 'grep' 然后访问找到匹配的行。其工作方式是把 'grep' 报告的匹配项当作错误来处理。输出缓冲区使用 Grep 模式，它是编译模式的一个变体（参见"编译模式"）。

- 'M-x grep'、'M-x lgrep'：在 Emacs 下异步运行 'grep'，在名为 '*grep*' 的缓冲区中列出匹配的行。
- 'M-x grep-find'、'M-x find-grep'、'M-x rgrep'：通过 'find' 运行 'grep'，并把输出收集到 '*grep*' 缓冲区中。
- 'M-x zrgrep'：运行 'zgrep'，并把输出收集到 '*grep*' 缓冲区中。
- 'M-x kill-grep'：杀掉正在运行的 'grep' 子进程。

要运行 'grep'，键入 'M-x grep'，然后输入一条指明如何运行 'grep' 的命令行。使用你在正常运行时本会传给 'grep' 的同样参数：一个 'grep' 风格的正则表达式（通常放在单引号内以引用 shell 的特殊字符），后跟文件名，文件名中可以使用通配符。如果你为 'M-x grep' 指定前缀参数，它会找到光标点附近缓冲区中的标识符（参见"查找标识符引用"），并把它放进默认的 'grep' 命令中。

你的命令不必只是运行 'grep'；你可以使用任何以相同格式产生输出的 shell 命令。例如，你可以这样串联 'grep' 命令：

```
grep -nH -e foo *.el | grep bar | grep toto
```

'grep' 的输出进入 '*grep*' 缓冲区。你可以用 'M-g M-n'、<RET> 等命令在原始文件中找到对应的行，就像对待编译错误一样。参见"编译模式"，其中有关于 '*grep*' 缓冲区中可用命令和键绑定的详细说明。

有些 grep 程序接受 '--color' 选项，以便在匹配项周围输出特殊标记用于高亮。你可以把 'grep-highlight-matches' 设为 't' 来利用这一特性。在源缓冲区中显示匹配项时，将高亮确切的匹配文本，而不是整个源代码行。高亮是通过匹配 'grep' 发出的 ANSI 转义序列实现的。对这些序列的匹配由 'grep-match-regexp' 控制，可以定制它以适应不同的 'grep' 程序。

与编译命令一样（参见“在 Emacs 中运行编译”），grep 命令运行期间，模式行显示到目前为止找到并高亮的匹配项数目。

‘grep’ 命令在运行前会提议保存缓冲区。这由变量 ‘grep-save-buffers’ 控制。可能的值为 ‘nil’（不保存）、‘ask’（保存前询问），或一个用作谓词的函数（它以文件名为参数被调用，若该缓冲区应被保存则应返回非 ‘nil’）。任何其他非 ‘nil’ 值表示应不经询问保存所有缓冲区。默认值是 ‘ask’。

默认情况下，grep 匹配项显示时会在每行前面加上文件名。但如果变量 ‘grep-use-headings’ 被定制为非 ‘nil’ 值，匹配项会被分成若干节，每个有匹配项的文件一节，文件名则用特殊字体 ‘grep-heading’ 显示在各节标题中。

命令 ‘M-x grep-find’（也可用 ‘M-x find-grep’）与 ‘M-x grep’ 类似，但它为该命令提供不同的初始默认值——该默认值同时运行 ‘find’ 和 ‘grep’，以搜索目录树中的每个文件。另请参见 “Dired 与 ‘find’” 中的 ‘find-grep-dired’ 命令。

命令 ‘M-x lgrep’（局部 grep）和 ‘M-x rgrep’（递归 grep）是 ‘grep’ 和 ‘grep-find’ 更友好的版本，它们分别询问要匹配的正则表达式、要搜索的文件以及搜索的基础目录。搜索是否区分大小写由 ‘case-fold-search’ 的当前值控制。命令 ‘M-x zrgrep’ 与 ‘M-x rgrep’ 类似，但它调用 ‘zgrep’ 而不是 ‘grep’ 来搜索经 gzip 压缩文件的内容。

这些命令基于变量 ‘grep-template’（用于 ‘lgrep’）和 ‘grep-find-template’（用于 ‘rgrep’）构建 shell 命令。要搜索的文件可以使用变量 ‘grep-files-aliases’ 中定义的别名。

变量 ‘grep-find-ignored-directories’ 中列出的目录会被 ‘M-x rgrep’ 自动跳过。默认值包含各种版本控制系统使用的数据目录。

默认情况下，为 ‘lgrep’、‘rgrep’ 和 ‘zgrep’ 构建的 shell 命令在显示时会被缩略，方法是隐藏其中包含一长串要忽略的文件和目录的部分。你可以点击表示它们的带省略号的按钮来显示被隐藏的部分。你也可以键入 ‘M-x grep-find-toggle-abbreviation’ 以交互方式切换是否查看被隐藏的部分。要禁用这种对 shell 命令的缩略，请把选项 ‘grep-find-abbreviate’ 定制为 ‘nil’ 值。

在 ‘*grep*’ 缓冲区中键入 ‘e’ 会使缓冲区可写并进入 Grep Edit 模式。与 Occur Edit 模式类似（参见“其他搜索与循环命令”），你可以编辑 ‘grep’ 报告的匹配行，并让这些改动反映到访问原始文件的缓冲区中。键入 ‘C-c C-c’ 离开 Grep Edit 模式并返回 Grep 模式。

29.5 即时查找语法错误

Flymake 模式是一种次模式，它为许多编程语言和标记语言执行即时语法检查，包括 C、C++、Perl、HTML 和 TeX/LaTeX。它有点类似于 Flyspell 模式，后者以类似方式为普通人类语言执行拼写检查（参见“拼写检查与纠正”）。在你编辑文件时，Flymake 模式使用缓冲区的临时副本在后台运行合适的语法检查工具。然后它解析错误和警告消息，并高亮缓冲区中出错的行。所用的语法检查工具取决于语言；例如对于 C/C++ 文件，这通常是 C 编译器。Flymake 也可以使用 ‘make’ 之类的构建工具来检查复杂的项目。

要启用 Flymake 模式，键入 ‘M-x flymake-mode’。你可以用 ‘M-x flymake-goto-next-error’ 和 ‘M-x flymake-goto-prev-error’ 跳转到它发现的错误。要显示当前缓冲区诊断信息的详细概览，使用命令 ‘M-x flymake-show-buffer-diagnostics’；要显示整个项目诊断信息的类似概览（参见“使用项目”），使用 ‘M-x flymake-show-project-diagnostics’。

关于使用 Flymake 的更多细节，参见《《flymake》手册》

29.6 在 Emacs 中运行调试器

GUD（Grand Unified Debugger，大统一调试器）库为众多符号调试器提供了 Emacs 接口。它可以运行 GNU 调试器（GDB），也可以运行 LLDB、DBX、SDB、XDB、Guile REPL 调试命令、Perl 的调试模式、Python 调试器 PDB 以及 Java 调试器 JDB。

Emacs 为 GDB 提供了专门的接口，它使用额外的 Emacs 窗口来显示被调试程序的状态。参见“GDB 图形界面”。

Emacs 还为 Emacs Lisp 程序提供了内置调试器。参见《Emacs Lisp 参考手册》的“Debugging”。

- GDB Graphical Interface:: 一种增强模式，利用 GDB 的功能构建图形化调试环境。

29.6.1 启动 GUD

有若干命令可以启动调试器子进程，每条命令对应一个特定的调试器程序。

- ‘M-x gdb’：以子进程方式运行 GDB，并通过类似 IDE 的 Emacs 界面与之交互。关于此命令的更多信息，参见“GDB 图形界面”。
- ‘M-x gud-gdb’：运行 GDB，使用 GUD 交互缓冲区向 GDB 子进程发送输入并接收其输出（参见“调试器操作”）。若这样的缓冲区已存在，就切换到它；否则创建该缓冲区并切换过去。

此列表中的其他命令对其他调试器程序做同样的事。

- ‘M-x lldb’：运行 LLDB 调试器。
- ‘M-x perldb’：以调试模式运行 Perl 解释器。
- ‘M-x jdb’：运行 Java 调试器。
- ‘M-x pdb’：运行 Python 调试器。
- ‘M-x guiler’：运行 Guile REPL，用于调试 Guile Scheme 程序。
- ‘M-x dbx’：运行 DBX 调试器。
- ‘M-x xdb’：运行 XDB 调试器。
- ‘M-x sdb’：运行 SDB 调试器。

这些命令都会在迷你缓冲区中读取一行命令行来调用调试器。迷你缓冲区的初始内容包含调试器的标准可执行文件名和选项，有时还包含对你想调试的可执行文件名的猜测。对于 ‘pdb’、‘perldb’ 和 ‘guiler’，若当前缓冲区正在访问某个文件，则在提示符处按 ‘M-n’（‘next-history-element’）会建议一条包含该文件名的完整命令行。此命令行中不允许使用 shell 通配符和变量。Emacs 假定第一个不以 ‘-’ 开头的命令参数是可执行文件名。

Tramp 提供了远程调试功能，调试器和被调试的程序都位于同一台远程主机上。详情参见《Tramp 手册》的“Running a debugger on a remote host”。这与 GDB 的远程调试特性不同：后者中程序与调试器运行在不同的机器上（参见《《gdb》手册》的“Remote Debugging”）。

29.6.2 调试器操作

“GUD 交互缓冲区”是一个 Emacs 缓冲区，用来向调试器子进程发送文本命令并记录其输出。这是与调试器交互的基本界面，‘M-x gud-gdb’ 以及参见“启动 GUD” 中列出的其他命令都使用它。‘M-x gdb’ 命令扩展了这一界面，为控制断点、栈帧以及调试器状态的其他方面增加了专门的缓冲区（参见“GDB 图形界面”）。

GUD 交互缓冲区使用 Shell 模式的一个变体，因此 Shell 模式定义的 Emacs 命令都可用（参见“Shell 模式”）。大多数调试器命令都支持补全（参见“补全”），你也可以用通常的 Shell 模式历史命令重复它们。关于可在 GUD 交互缓冲区中使用的特殊命令，参见“GUD 的命令”。

调试程序时，Emacs 会在 Emacs 缓冲区中访问相关的源文件来显示它们，并在左侧边缘（fringe）用一个箭头指示当前执行行。（在文本终端上，箭头显示为 ‘=>’，叠加在前两个文本列上。）在这种缓冲区中移动光标点不会移动箭头。你可以随意编辑这些源文件，但要注意插入或删除行会使箭头的位置失准，因为 Emacs 无法判断被编辑过的源代码行与调试器子进程报告的行如何对应。要更新这一信息，通常必须重新编译并重启程序。

此外，GUD 还能以两种方式之一在窗口文本内直观地标出当前执行行：第一种在启用用户选项 ‘gud-highlight-current-line’ 时生效，它用一个覆盖层（overlay）显示该行，其外观由字体（face）‘gud-highlight-current-line-face’ 提供。另一种在启用 HL Line 模式（参见“光标的显示”）时生效，它把 HL Line 模式引入的覆盖层短暂移到执行行上，直到后续的编辑命令把它重新放回光标下方。

GUD Tooltip 模式是一个全局次模式，为 GUD 增加工具提示（tooltip）支持。要切换该模式，输入 ‘M-x gud-tooltip-mode’。它默认被禁用。若启用，你可以把鼠标指针移到变量、函数或宏（统称“标识符”）上，在工具提示中显示它们的值（参见“工具提示”）。如果仅把鼠标指针放在某个表达式上并不能显示你想要的那个表达式的值，你可以把鼠标拖过该表达式、然后让鼠标停留在标记区域内，从而更明确地告诉 Emacs 要求值哪个表达式。GUD Tooltip 模式在 GUD 交互缓冲区中生效，也在所有主模式列于变量 ‘gud-tooltip-modes’ 中的源缓冲区中生效。如果你关闭了工具提示模式，值会改为显示在回显区而不是工具提示中。

在 ‘M-x gud-gdb’ 中使用 GUD Tooltip 模式时，在 GDB 中显示表达式的值有时会展开宏，从而可能对被调试程序产生副作用。因此，‘gud-gdb’ 中的工具提示被禁用。若使用 ‘M-x gdb’ 界面，则不会出现这个问题，因为有专门的代码来避免副作用；此外，程序未在运行时你也可以显示与某标识符相关的宏定义。

29.6.3 GUD 的命令

GUD 提供设置和清除断点、选择栈帧以及在程序中单步执行的命令。

- ‘C-x C-a C-b’：在光标点所在的源代码行上设置断点。

在源缓冲区中调用 ‘C-x C-a C-b’（‘gud-break’）时，会在当前源代码行上设置一个调试器断点。该命令只在启动 GUD 之后才可用。若在与任何调试器子进程都不关联的缓冲区中调用它，它会报错。

以下命令在 GUD 交互缓冲区和全局都可用，但键绑定不同。以 ‘C-c’ 开头的按键只在 GUD 交互缓冲区中可用，而以 ‘C-x C-a’ 开头的按键在全局可用。其中有些命令也可通过工具栏使用；有些则不受某些调试器支持。

- ‘C-c C-l’ ‘C-x C-a C-l’：在另一个窗口中显示 GUD 交互缓冲区里最近提到的源代码行（‘gud-refresh’）。

- ‘C-c C-s’ ‘C-x C-a C-s’：执行下一行代码（‘gud-step’）。若该行包含函数调用，则在进入被调用函数后停止执行。
- ‘C-c C-n’ ‘C-x C-a C-n’：执行下一行代码，越过函数调用，不进入函数内部停止（‘gud-next’）。
- ‘C-c C-i’ ‘C-x C-a C-i’：执行一条机器指令（‘gud-stepi’）。
- ‘C-c C-p’ ‘C-x C-a C-p’：对光标点处的表达式求值（‘gud-print’）。若 Emacs 打印的不是你想要的那个表达式，请先把它标记为区域。
- ‘C-c C-r’ ‘C-x C-a C-r’：不指定任何停止点继续执行。程序将一直运行到遇到断点、终止，或收到调试器正在监视的信号为止（‘gud-cont’）。
- ‘C-c C-d’ ‘C-x C-a C-d’：删除当前源代码行上的断点（若有）（‘gud-remove’）。若在 GUD 交互缓冲区中使用该命令，则作用于程序上次停止的那一行。
- ‘C-c C-t’ ‘C-x C-a C-t’：在当前源代码行上设置临时断点（若有）（‘gud-tbreak’）。若在 GUD 交互缓冲区中使用该命令，则作用于程序上次停止的那一行。
- ‘C-c <’ ‘C-x C-a <’：选择下一层外层栈帧（‘gud-up’）。这等价于 GDB 命令 ‘up’。
- ‘C-c >’ ‘C-x C-a >’：选择下一层内层栈帧（‘gud-down’）。这等价于 GDB 命令 ‘down’。
- ‘C-c C-u’ ‘C-x C-a C-u’：继续执行到当前行（‘gud-until’）。程序将一直运行到遇到断点、终止、收到调试器正在监视的信号，或到达光标当前所在的行。
- ‘C-c C-f’ ‘C-x C-a C-f’：运行程序直到选定的栈帧返回或因其他原因停止（‘gud-finish’）。

若你使用的是 GDB，还可使用以下额外的键绑定：

- ‘C-x C-a C-j’：只在源缓冲区中有用，‘gud-jump’ 把程序的执行点转移到当前行。换句话说，程序接下来执行的行就是你发出该命令时所在的行。若新的执行行与原来的执行行处于不同的函数中，GDB 会提示确认，因为结果可能很怪异。详情参见 GDB 手册中关于 ‘jump’ 的条目。
- ‘<TAB>’：在 GDB 中补全符号名（‘gud-gdb-complete-command’）。该键只在 GUD 交互缓冲区中可用。

在合理的情况下，这些命令会把数字参数解释为重复次数。

由于 `<TAB>` 用作补全命令，你无法用它向正在用 GDB 调试的程序输入制表符。请改为输入 ‘C-q <TAB>’ 来输入制表符。

29.6.4 GUD 定制

启动时，GUD 会运行下列钩子之一：若使用 GDB，则为 ‘gdb-mode-hook’；若使用 DBX，则为 ‘dbx-mode-hook’；若使用 SDB，则为 ‘sdb-mode-hook’；若使用 XDB，则为 ‘xdb-mode-hook’；Guile REPL 调试用 ‘guiler-mode-hook’；Perl 调试模式用 ‘perldb-mode-hook’；PDB 用 ‘pdb-mode-hook’；JDB 用 ‘jdb-mode-hook’。参见“钩子”。

‘gud-def’ Lisp 宏（参见《Emacs Lisp 参考手册》的“Defining Macros”）提供了一种便捷方式，用来定义一条向调试器发送特定命令字符串的 Emacs 命令，并为它在 GUD 交互缓冲区中设置键绑定：

```
(gud-def FUNCTION CMDSTRING BINDING DOCSTRING)
```

它定义一条名为 FUNCTION 的命令，该命令把 CMDSTRING 发送给调试器进程，并为其指定文档字符串 DOCSTRING。随后你可以在任何缓冲区中使用命令 FUNCTION。若 BINDING 非 'nil'，'gud-def' 还会在 GUD 缓冲区的模式中把该命令绑定到 'C-c BINDING'，并一般性地绑定到 'C-x C-a BINDING'。

命令字符串 CMDSTRING 可以包含某些 '%' 序列，它们代表在调用 FUNCTION 时才填入的数据：

- '%f'：当前源文件的名字。若当前缓冲区是 GUD 缓冲区，则当前源文件就是程序停止所在的文件。
- '%l'：当前源代码行的行号。若当前缓冲区是 GUD 缓冲区，则当前源代码行就是程序停止所在的行。
- '%e'：在 transient-mark-mode 下为区域中的文本（若区域处于活动状态）。否则为光标点处或紧邻光标点处的 C 左值或函数调用表达式的文本。
- '%a'：光标点处或紧邻光标点处的十六进制地址的文本。
- '%p'：被调用函数的数字参数，以十进制数表示。若命令未带数字参数使用，则 '%p' 代表空字符串。

若你在命令字符串中不使用 '%p'，你定义的命令会忽略任何数字参数。

- '%d'：当前源文件所在目录的名字。
- '%c'：由光标点周围的表达式导出的完全限定类名（仅限 jdb）。

29.6.5 GDB 图形界面

命令 'M-x gdb' 在类似 IDE 的界面中启动 GDB，其中有专门的缓冲区来控制断点、栈帧以及调试器状态的其他方面。它还提供了另外一些用鼠标控制调试会话的方式，例如在源缓冲区的边缘单击以在那里设置断点。

若只想使用 GUD 交互缓冲区界面运行 GDB、不要这些额外特性，请使用 'M-x gud-gdb'（参见"启动 GUD"）。

在内部，'M-x gdb' 会告知 GDB 其屏幕尺寸不受限制；为正确操作，在调试会话期间你不得改变 GDB 的屏幕高度和宽度值。

- Source Buffers:: 在边缘/边距中用鼠标控制你的程序。* Breakpoints Buffer:: 一个断点控制面板。* Threads Buffer:: 显示线程。* Stack Buffer:: 从调用栈中选择一个栈帧。* Other GDB Buffers:: 用于控制 GDB 状态的其他缓冲区。* Watch Expressions:: 在 speedbar 中监视变量值。* Multithreaded Debugging:: 调试有多线程的程序。

29.6.5.1 GDB 用户界面布局

若变量 'gdb-many-windows' 为 'nil'（默认值），'M-x gdb' 通常只显示 GUD 交互缓冲区。然而，若变量 'gdb-show-main' 也为非 'nil'，它启动时会有两个窗口：一个显示 GUD 交互缓冲区，另一个显示你所调试程序的 'main' 函数的源代码。

若 'gdb-many-windows' 为非 'nil'，则 'M-x gdb' 显示如下框架布局：

```
+--------------------------------+--------------------------------+
|   GUD interaction buffer       |   Locals/Registers buffer      |
|--------------------------------+--------------------------------+
|   Primary Source buffer        |   I/O buffer for debugged pgm  |
|--------------------------------+--------------------------------+
|   Stack buffer                 |   Breakpoints/Threads buffer   |
+--------------------------------+--------------------------------+
```

你可以基于上述布局定制窗口布局，并用‘gdb-save-window-configuration’把该布局保存到文件。之后你可以用‘gdb-load-window-configuration’把这个布局重新载入。（在内部，Emacs 使用“窗口配置”一词而不是“窗口布局”。）你可以通过定制‘gdb-default-window-configuration-file’，把你自定义的布局设为‘gdb-many-windows’使用的默认布局。若它不是绝对文件名，GDB 会在‘gdb-window-configuration-directory’下查找该文件。‘gdb-window-configuration-directory’默认为‘user-emacs-directory’（参见“Emacs 如何查找你的初始化文件”）。

若你改变了窗口布局，可以输入‘M-x gdb-restore-windows’恢复默认布局。要在多窗口布局与只有 GUD 交互缓冲区和源文件的简单布局之间切换，输入‘M-x gdb-many-windows’。

若你有精心设置的窗口安排，而不希望‘gdb-many-windows’打乱它，最好一开始就在单独的框架中调用‘M-x gdb’，这样你原来框架上的窗口安排不会受影响。如果你在文本模式终端上工作、窗口的屏幕空间可能很紧张，那么为 GDB 会话使用单独的框架会特别方便。若你选择在同一个框架中启动 GDB，可以考虑把‘gdb-restore-window-configuration-after-quit’设为非‘nil’值。这样 GDB 退出后会恢复你原来的布局。用‘t’表示总是恢复；用‘if-gdb-many-windows’表示仅当‘gdb-many-windows’为非‘nil’时恢复；用‘if-gdb-show-main’表示仅当‘gdb-show-main’为非‘nil’时恢复。

你也可以指定要显示的额外 GDB 相关缓冲区，它们可在同一框架中，也可在另一个框架中。输入‘M-x gdb-display-BUFFERTYPE-buffer’或‘M-x gdb-frame-BUFFERTYPE-buffer’来选择你想要的缓冲区，其中 BUFFERTYPE 是相关的缓冲区类型，例如‘breakpoints’或‘io’。你也可以从菜单栏的‘GUD’菜单下的‘GDB-Windows’和‘GDB-Frames’子菜单中做同样的事。

默认情况下，GDB 至多使用一个窗口来显示源文件。你可以通过定制‘gdb-max-source-window-count’让它使用更多窗口。你还可以定制‘gdb-display-source-buffer-action’来控制 GDB 如何显示源文件。

调试结束后，用‘C-x k’杀掉 GUD 交互缓冲区，这也会杀掉与该会话关联的所有缓冲区。不过，如果你在 Emacs 中编辑并重新编译源代码之后还想继续调试，就不必这样做。当你重新开始执行时，GDB 会自动找到新的可执行文件。保留 GUD 交互缓冲区的好处是既保留了 shell 历史，也保留了 GDB 的断点。你确实需要检查最近编辑过的源文件中的断点是否仍在正确的位置上。

29.6.5.2 源缓冲区

- ‘mouse-1’（在边缘中）：在该行上设置或清除断点（‘gdb-mouse-set-clear-breakpoint’）。
- ‘C-mouse-1’（在边缘中）：启用或禁用该行上的断点（‘gdb-mouse-toggle-breakpoint-margin’）。
- ‘mouse-3’（在边缘中）：继续执行到该行（‘gdb-mouse-until’）。
- ‘C-mouse-3’（在边缘中）：跳转到该行（‘gdb-mouse-jump’）。

在图形显示下，你可以在源缓冲区的边缘单击‘mouse-1’，从而在该行上设置断点（参见“窗口边缘”）。你单击处的边缘会出现一个红点。若那里已有断点，单击会删除它。‘C-mouse-1’单击会启用或禁用已有的断点；已禁用但未取消设置的断点由灰点表示。

在文本终端上，或当边缘被禁用时，已启用的断点由窗口左边距中的字符‘B’表示。已禁用的断点由‘b’表示。（只有当存在断点时才显示边距。）

源缓冲区左侧边缘中的实心箭头指示被调试程序停下的最内层栈帧所在的行。空心箭头指示更高层栈帧的当前执行行。若你用‘mouse-1’拖动边缘中的箭头，就会使执行推进到你释放按钮时所在的行。另一种做法是在边缘中单击‘mouse-3’推进到该行。你可以在边缘中单击‘C-mouse-3’跳转到该行，而不执行中间的各行。该命令允许你往回走，这在运行已经执行过的代码以便更细致地检查其执行过程时很有用。

默认情况下，被调试程序中的源文件名和非 ASCII 字符串用默认编码系统解码。若你喜欢别的解码方式——也许是因为被调试程序使用了不同的字符编码——请把变量‘gdb-mi-decode-strings’设为合适的编码系统，或设为‘nil’以让非 ASCII 字符保持为未解码的八进制转义形式。

29.6.5.3 断点缓冲区

GDB 断点缓冲区显示调试会话中的断点、监视点和捕获点。参见《《gdb》手册》的“Breakpoints”。它提供下列命令，其中大多数作用于“当前断点”（即光标点所在的断点）：

- ‘<SPC>’：启用/禁用当前断点（‘gdb-toggle-breakpoint’）。在图形显示下，这会改变该行源缓冲区边缘中圆点的颜色。断点启用时圆点为红色，禁用时为灰色。
- ‘D’：删除当前断点（‘gdb-delete-breakpoint’）。
- ‘<RET>’：访问当前断点对应的源代码行（‘gdb-goto-breakpoint’）。
- ‘mouse-2’：访问你单击的断点所对应的源代码行（‘gdb-goto-breakpoint’）。

当‘gdb-many-windows’为非‘nil’时，GDB 断点缓冲区与 GDB 线程缓冲区共用一个窗口。要用其中一个切换到另一个，用‘mouse-1’单击标题行中的相应按钮。若‘gdb-show-threads-by-default’为非‘nil’，则默认显示 GDB 线程缓冲区。

29.6.5.4 线程缓冲区

GDB 线程缓冲区显示被调试程序中线程的概要。参见《《gdb》手册》的“Threads”。要选择某个线程，把光标点移到那里并按 <RET>（‘gdb-select-thread’），或用‘mouse-2’在其上单击。这还会显示相关的源缓冲区，并更新其他 GDB 缓冲区的内容。

你可以定制‘gdb-buffers’组中的变量，以选择 GDB 线程缓冲区中要包含的字段。

- ‘gdb-thread-buffer-verbose-names’：显示长线程名，如‘Thread 0x4e2ab70 (LWP 1983)’。
- ‘gdb-thread-buffer-arguments’：显示线程顶层栈帧的参数。
- ‘gdb-thread-buffer-locations’：显示文件信息或库名。
- ‘gdb-thread-buffer-addresses’：在线程缓冲区中显示线程栈帧的地址。

要同时查看多个线程的信息，可在 GDB 线程缓冲区中使用以下命令。

- ‘d’：为当前行处的线程显示反汇编缓冲区（‘gdb-display-disassembly-for-thread’）。
- ‘f’：为当前行处的线程显示 GDB 栈缓冲区（‘gdb-display-stack-for-thread’）。
- ‘l’：为当前行处的线程显示 GDB 局部变量缓冲区（‘gdb-display-locals-for-thread’）。
- ‘r’：为当前行处的线程显示 GDB 寄存器缓冲区（‘gdb-display-registers-for-thread’）。

它们对应的大写形式 ‘D’、‘F’、‘L’ 和 ‘R’ 会在新框架中显示相应的缓冲区。

当你创建了显示某个特定线程信息的缓冲区后，该缓冲区就绑定到那个线程，并在你调试程序期间持续显示实际信息。每个 GDB 缓冲区的模式指示器显示该缓冲区所显示信息所属的线程号。线程号也包含在每个已绑定缓冲区的名字中。

GDB 线程缓冲区中还有更多命令可用，它们取决于控制程序执行的 GDB 模式。参见“多线程调试”。

29.6.5.5 栈缓冲区

GDB 栈缓冲区显示“调用栈”，调试会话中每层嵌套的子程序调用（“栈帧”）占一行。参见《《gdb》手册》的“Backtrace”

在图形显示上，被选中的栈帧由边缘区中的箭头指示。在文本终端上，或禁用边缘区时，被选中的栈帧以反色显示。要选择某个栈帧，把光标点移到它所在的行并输入 <RET>（‘gdb-frames-select’），或在其上点击 ‘mouse-2’。这样做还会更新 Locals 缓冲区（参见“其他 GDB 缓冲区”）。

若希望每个栈帧都显示其帧地址，请把变量 ‘gdb-stack-buffer-addresses’ 定制为非 ‘nil’ 值。

29.6.5.6 其他 GDB 缓冲区

‘M-x gdb’ 提供的其他缓冲区中，可以选择性地请求显示以下这些：

Locals 缓冲区 本缓冲区显示当前栈帧中简单数据类型的局部变量值（参见《《gdb》手册》的“Frame Info”）。若想编辑某个值，请在其上按 <RET> 或点击 ‘mouse-2’。

数组和结构体只显示其类型。使用 GDB 6.4 或更高版本时，按 <RET> 或点击 ‘mouse-2’ 即可查看光标点处的局部变量的值。使用更早版本的 GDB 时，请在类型描述（‘[struct/union]’ 或 ‘[array]’）上使用 <RET> 或 ‘mouse-2’。参见“监视表达式”。

要显示 Locals 缓冲区，请输入 ‘M-x gdb-display-locals-buffer’。

I/O 缓冲区 如果你正在调试的程序使用标准输入和输出流与用户交互，或向标准输出发出大量输出，你可能希望把它的 I/O 与同 GDB 的交互分开。使用命令 ‘M-x gdb-display-io-buffer’ 显示一个窗口，其中的缓冲区接收 Emacs 从你正在调试的程序重定向来的输入和输出。不过，若变量 ‘gdb-display-io-buffer’ 为 ‘nil’，Emacs 不会创建也不会显示单独的 I/O 缓冲区，而是把程序的交互重定向到 GUD 交互缓冲区。

Registers 缓冲区 本缓冲区显示各寄存器中保存的值（参见《《gdb》手册》的“Registers”）。用命令 ‘M-x gdb-display-registers-buffer’ 请求显示此缓冲区。若想编辑某个寄存器的值，请在其上按 <RET> 或点击 ‘mouse-2’。使用 GDB 6.4 或更高版本时，最近改变过的寄存器值以 ‘font-lock-warning-face’ 显示。

汇编缓冲区 汇编缓冲区以机器码形式显示当前栈帧。箭头指向当前指令，你可以像在源代码缓冲区中那样设置和删除断点。断点图标也会出现在边缘区或边距中。要请求显示此缓冲区，请使用 ‘M-x gdb-display-

disassembly-buffer’。

内存缓冲区 内存缓冲区让你查看程序内存的各个区段（参见《《gdb》手册》的“Memory”）。在标题行相应部位点击‘mouse-1’可改变起始地址或缓冲区显示的数据项数目。也可以分别使用‘S’和‘N’。在标题行上点击‘mouse-3’可选择这些数据项的显示格式或单位大小。使用‘M-x gdb-display-memory-buffer’请求显示此缓冲区。

当‘gdb-many-windows’为非‘nil’时，locals 缓冲区与 registers 缓冲区共用一个窗口，就像断点缓冲区和线程缓冲区那样。要在这两者之间切换，请用‘mouse-1’点击标题行中相应的按钮。

29.6.5.7 监视表达式

若想查看某个变量在程序每次停下时的变化，请把光标点移到该变量名中，然后点击工具栏上的监视图标（‘gud-watch’），或输入‘C-x C-a C-w’。若指定前缀参数，你可以在迷你缓冲区中输入变量名。

每个监视表达式都显示在 speedbar 中（参见“Speedbar 框架”）。数组、结构体和联合体等复杂数据类型以树形格式表示。叶子和简单数据类型显示表达式的名称及其值，并且在 speedbar 框架被选中时，以工具提示的形式显示类型。更高层次对于指针显示名称、类型和地址值，对其他情况只显示名称和类型。根表达式还会以工具提示的形式显示帧地址，以帮助识别它们是在哪个栈帧中定义的。

要展开或收缩某个复杂数据类型，请点击该表达式左侧的标签，或在其上按 <SPC>。如果该表达式的直接子项数目超过变量‘gdb-max-children’的值，Emacs 会在展开该表达式之前请求确认。

要删除某个复杂监视表达式，请把光标点移到 speedbar 中的根表达式上，然后输入‘D’（‘gdb-var-delete’）。

要编辑简单数据类型的变量或复杂数据类型的简单元素，请把光标点移到 speedbar 中相应位置，然后输入 <RET>（‘gdb-edit-value’）。你也可以在某值上点击‘mouse-2’来编辑它。这两种方式都会用迷你缓冲区读取新值。

若把变量‘gdb-show-changed-values’设为非‘nil’（默认值），Emacs 会用‘font-lock-warning-face’高亮最近改变过的值，并用‘shadow’字体（face）使已超出作用域的变量不那么显眼。变量超出作用域后，你就不能编辑它的值。

若变量‘gdb-delete-out-of-scope’为非‘nil’（默认值），Emacs 会自动删除超出作用域的监视表达式。有时，当你的程序多次重新进入同一个函数时，把这个值设为‘nil’可能很有用，这样你就不必重新创建监视表达式。

若变量‘gdb-use-colon-colon-notation’为非‘nil’，Emacs 使用‘FUNCTION::VARIABLE’格式。这使用户能够显示共享同一变量名的多个监视表达式。默认值为‘nil’。

若想每次监视表达式的显示更新时都自动升起 speedbar，请把‘gdb-speedbar-auto-raise’设为非‘nil’。如果你在整屏的 Emacs 框架中调试，这可能很有用。

29.6.5.8 多线程调试

在 GDB 的“全停模式”（all-stop mode）中，每当你的程序停下，所有执行线程都停下。同样，每当你重新启动程序，所有线程都开始执行。参见《《gdb》手册》的“All-Stop Mode”。对某些多线程目标，GDB 还支持另一种运行模式，称为“不停模式”（non-stop mode），在其中你可以在调试器里检查已停止的程序线程，而其他线程继续自由执行。参见《《gdb》手册》的“Non-Stop Mode”。7.0 之前的 GDB 版本不支持不停模式，而且它并非在所有目标上都能工作。

变量 'gdb-non-stop-setting' 决定 Emacs 是以全停模式还是不停模式运行 GDB。默认值为 't'，意思是如果不停模式可用就尝试使用它。若你把值改为 'nil'，或不停模式不可用，Emacs 就以全停模式运行 GDB。该变量在 Emacs 开始调试会话时生效；若你改变了它的值，应重新启动任何活动的调试会话。

在不停模式下，当某个线程停下时，Emacs 通常切换到该线程。若已经选中了另一个已停止的线程，而你不希望 Emacs 做这种切换，请把变量 'gdb-switch-when-another-stopped' 改为 'nil'。

Emacs 可以依据导致停止的原因决定是否切换到停下的线程。定制变量 'gdb-switch-reasons' 以选择将导致线程切换的停止原因。

变量 'gdb-stopped-functions' 让你在某个线程停下时执行自己的函数。

在不停模式下，你可以切换 GUD 执行控制命令的不同模式。

"Non-stop/A"

当 'gdb-gud-control-all-threads' 为 't'（默认值）时，中断和继续命令作用于所有线程，因此你可以分别用 'gud-stop-subjob' 和 'gud-cont' 以一条命令暂停或继续你所有的线程。当至少有一个线程停下时，工具栏上显示 'Go' 按钮；而当至少有一个线程正在运行时，显示 'Stop' 按钮。

"Non-stop/T"

当 'gdb-gud-control-all-threads' 为 'nil' 时，只有当前线程被暂停/继续。GUD 工具栏上的 'Go' 和 'Stop' 按钮根据当前线程的状态显示。

你可以从工具栏或从 'GUD->GDB-MI' 菜单改变 'gdb-gud-control-all-threads' 的当前值。

单步命令始终作用于当前线程。

在不停模式下，你可以在不选中线程的情况下中断/继续线程。在线程缓冲区中按 'i' 会中断光标点处的线程，'c' 继续它，'s' 单步执行它。将来可能会添加更多此类命令。

注意，当你中断某个线程时，它以 'signal received' 原因停下。若该原因包含在你的 'gdb-switch-reasons' 中（默认情况下确实包含），Emacs 将切换到该线程。

29.7 执行 Lisp 表达式

Emacs 为若干 Lisp 变体提供了主模式。它们使用与其他编程语言模式相同的编辑命令（参见"编辑程序"）。此外，它们还提供执行 Lisp 表达式的专用命令。

Emacs Lisp 模式 用于编辑 Emacs Lisp 源文件的模式。它定义 'C-M-x' 来求值当前的顶层 Lisp 表达式。参见"求值 Emacs Lisp 表达式"。

Lisp Interaction 模式 用于交互式 Emacs Lisp 会话的模式。它定义 'C-j' 来求值光标点之前的表达式，并把其值插入缓冲区。参见"Lisp 交互缓冲区"。

Lisp 模式 用于编辑在 Emacs Lisp 之外的 Lisp 中运行的程序源文件的模式。它定义 'C-M-x' 来在外部 Lisp 中求值当前的顶层表达式。参见"运行外部 Lisp"。

Inferior Lisp 模式 用于与作为 Emacs 子进程（或"下级进程"）运行的外部 Lisp 进行交互式会话的模式。参见"运行外部 Lisp"。

Scheme 模式 类似 Lisp 模式，但用于 Scheme 程序。

Inferior Scheme 模式 类似 Inferior Lisp 模式，但用于 Scheme。

29.8 Emacs 的 Lisp 代码库

Emacs Lisp 代码存放在文件名习惯上以 '.el' 结尾的文件中。这类文件会自动以 Emacs Lisp 模式访问。

Emacs Lisp 代码可以编译成字节码，字节码加载更快、占用空间更少、执行更快。按惯例，编译后的 Emacs Lisp 代码放在单独的文件中，文件名以 '.elc' 结尾。例如，'foo.el' 的编译代码放在 'foo.elc' 中。参见《Emacs Lisp 参考手册》的"Byte Compilation"。

Emacs Lisp 代码也可以编译成"原生代码"（native code）：与 C 或 Fortran 编译器生成的机器码类似的机器码。原生代码运行得比字节码更快。原生编译的 Emacs Lisp 代码存放在文件名以 '.eln' 结尾的文件中。参见《Emacs Lisp 参考手册》的"Native Compilation"。

要"加载"一个 Emacs Lisp 文件，请输入 'M-x load-file'。该命令用迷你缓冲区读取文件名，并把该文件的内容作为 Emacs Lisp 代码执行。不必先访问该文件；此命令直接从磁盘读取文件，而不是从已有的 Emacs 缓冲区读取。

若某个 Emacs Lisp 文件安装在 Emacs Lisp"加载路径"（见下文定义）中，你可以用 'M-x load-library' 而不是 'M-x load-file' 来加载它。'M-x load-library' 命令提示的是"库名"而不是文件名；它会搜索 Emacs Lisp 加载路径中的每个目录，试图找到与该库名匹配的文件。若库名为 'FOO'，它会尝试查找名为 'FOO.elc'、'FOO.el' 和 'FOO' 的文件。（若 Emacs 在构建时启用了原生编译，'load-library' 会查找与 'FOO.el' 对应的 '.eln' 文件，并加载它而不是 'FOO.elc'。）默认行为是加载找到的第一个文件。该命令优先选择 '.eln' 文件而不是 '.elc' 文件，优先选择 '.elc' 文件而不是 '.el' 文件，因为编译后的文件加载和运行更快。若它发现 'LIB.el' 比 'LIB.elc' 更新，它会发出警告，以防有人修改了 '.el' 文件却忘了重新编译，但仍会加载 '.elc' 文件。（由于这一行为，你可以把未完成的修改保存到 Emacs Lisp 源文件中，而不重新编译，直到你的修改可供使用。）不过，若你把选项 'load-prefer-newer' 设为非 'nil' 值，那么 Emacs 就不按上述过程行事，而是加载该文件最新的那个版本。若 Emacs 在构建时启用了原生编译，而它找不到与 'LIB.el' 对应的 '.eln' 文件，它会加载 'LIB.elc' 并在后台启动对 'LIB.el' 的原生编译，然后在编译完成后加载 '.eln' 文件。

Emacs Lisp 程序通常用 'load' 函数加载 Emacs Lisp 文件。它类似 'load-library'，但层次更低，并接受额外的参数。参见《Emacs Lisp 参考手册》的"How Programs Do Loading"。

Emacs Lisp 加载路径由变量 'load-path' 指定。它的值应是一个目录（字符串）列表。'M-x load-library' 命令、层次更低的 'load' 函数以及其他查找 Emacs Lisp 库的 Emacs 函数都按指定顺序搜索这些目录。'load-path' 中的一项也可以是特殊值 'nil'，表示当前的默认目录，但使用它几乎总是不好的主意，因为当 Emacs 使用 'load-path' 时，它的含义取决于当时当前的缓冲区。（如果你发现自己希望列表中有 'nil'，很可能你真正想要的是使用 'M-x load-file'。）

'load-path' 的默认值是存放 Emacs 自身 Lisp 代码的那些目录组成的列表。如果你在另一个目录中有自己的库，可以把该目录添加到加载路径中。与本手册中描述的大多数其他变量不同，'load-path' 不能通过 Customize 界面修改（参见"简易自定义界面"），但你可以通过在初始化文件中放入如下的一行来添加目录（参见"Emacs 初始化文件"）：

```
(add-to-list 'load-path "/path/to/my/lisp/library")
```

习惯上把本地安装的库放在‘site-lisp’目录中，该目录已经在‘load-path’的默认值里，或放在‘site-lisp’的某个子目录中。这样你就不必修改‘load-path’的默认值。

与‘load-path’类似，Emacs 查找含原生编译 Lisp 代码的‘*.eln’文件的目录列表由变量‘native-comp-eln-load-path’指定。

有些命令是“自动加载的”；当你运行它们时，Emacs 会先自动加载相关的库。例如，‘M-x compile’命令（参见“在 Emacs 中运行编译”）是自动加载的；若你调用它，Emacs 会先自动加载‘compile’库。相反，命令‘M-x recompile’不是自动加载的，所以在加载‘compile’库之前它不可用。

当你查看某个自动加载命令的文档时，也可能发生自动加载（参见“按命令或变量名获取帮助”），前提是该文档引用了其库中的其他函数和变量（加载该库使 Emacs 能正确设置‘*Help*’缓冲区中的超链接）。要禁用此特性，请把变量‘help-enable-autoload’改为‘nil’。

在为‘describe-variable’和‘describe-function’补全名称时也会发生自动加载，依据是正在补全的前缀。要禁用此特性，请把变量‘help-enable-completion-autoload’改为‘nil’。

一旦你把库放在 Emacs 能找到并加载的目录中，你可能希望让它在启动时即可用。当该库定义了应当按需自动可用的特性，因而手动加载该库很不方便时，这很有用。在这些情况下，请通过向初始化文件中添加合适的形式来确保该库会被加载：要么用‘load’或‘require’（如果你总是需要在启动时加载该库），要么用‘autoload’（如果你需要在某个命令或函数被调用时才让 Emacs 加载该库）。例如：

```
;; Loads my-shining-package.elc unconditionally.
(require 'my-shining-package)
;; Will load my-shining-package.elc when my-func is invoked.
(autoload 'my-func "my-shining-package")
```

注意，用‘package-install’安装包（参见“包的安装”）会自动把该包的 Lisp 文件放在 Emacs 能找到的目录中，并按需扩展‘load-path’，因此对此类包而言，上述手工定制是不必要的。

29.9 求值 Emacs Lisp 表达式

Emacs Lisp 模式是编辑 Emacs Lisp 的主模式。它的模式命令是‘M-x emacs-lisp-mode’。

Emacs 提供若干命令来求值 Emacs Lisp 表达式。你可以在 Emacs Lisp 模式中使用这些命令，在编写 Emacs Lisp 代码的过程中测试它。例如，在重写某个函数之后，你可以求值该函数定义，使其对后续的函数调用生效。这些命令也可全局使用，能在 Emacs Lisp 模式之外使用。

```
‘M-:’ Read a single Emacs Lisp expression in the minibuffer, evaluate it, and print the value
in the echo area (‘eval-expression’). ‘C-x C-e’ Evaluate the Emacs Lisp expression before poin
t, and print the value in the echo area (‘eval-last-sexp’). ‘C-M-x’ (in Emacs Lisp mode) ‘M-x
eval-defun’ Evaluate the defun containing or after point, and print the value in the echo area
(‘eval-defun’). ‘M-x eval-region’ Evaluate all the Emacs Lisp expressions in the region. ‘M-x
eval-buffer’ Evaluate all the Emacs Lisp expressions in the buffer.
```

‘M-:’（‘eval-expression’）用迷你缓冲区读取一个表达式并求值它。（在求值该表达式之前，当前缓冲区会切换回你输入 ‘M-:’ 时所在的缓冲区，而不是你输入该表达式的迷你缓冲区。）

命令 ‘C-x C-e’（‘eval-last-sexp’）求值缓冲区中位于光标点之前的 Emacs Lisp 表达式，并在回显区显示其值。当求值结果是整数时，它还会连同该值一起以其他格式（八进制、十六进制，以及在下面描述的 ‘eval-expression-print-maximum-character’ 允许时以字符形式）显示。

若给 ‘M-:’ 或 ‘C-x C-e’ 一个前缀参数，它会把值插入当前缓冲区中光标点处，而不是在回显区显示。若前缀参数为零，任何整数输出都会连同其值以其他格式（八进制、十六进制和字符）插入。这样的前缀参数还会阻止根据变量 ‘eval-expression-print-level’ 和 ‘eval-expression-print-length’（见下）对输出做缩写。同样，前缀参数 ‘-1’ 会覆盖 ‘eval-expression-print-length’ 的效果。

‘C-x C-e’（‘eval-last-sexp’）对 ‘defvar’ 表达式作特殊处理。通常，若 ‘defvar’ 表达式所定义的变量已有值，求值该表达式什么也不做。但此命令会无条件地把该变量重置为 ‘defvar’ 指定的初始值；这对调试 Emacs Lisp 程序很方便。‘defcustom’ 和 ‘defface’ 表达式的处理方式类似。注意，本节记录的其他命令（‘eval-defun’ 除外）没有这一特殊功能。

‘eval-defun’ 命令在 Emacs Lisp 模式中绑定到 ‘C-M-x’。它求值包含光标点或位于光标点之后的顶层 Lisp 表达式，并在回显区显示其值。在这里，顶层表达式被称为“defun”，但它不必是真正的 ‘defun’（函数定义）。

此命令处理 ‘defvar’/‘defcustom’/‘defface’ 形式的方式与 ‘eval-last-sexp’ 相同。

带前缀参数时，‘C-M-x’ 会为 Edebug（Emacs Lisp 调试器）对函数定义进行插桩。参见《Emacs Lisp 参考手册》的“Instrumenting”。

命令 ‘M-x eval-region’ 把区域的文本解析为一个或多个 Lisp 表达式，逐个求值它们。‘M-x eval-buffer’ 类似，但求值整个缓冲区。

选项 ‘eval-expression-print-level’ 和 ‘eval-expression-print-length’ 控制求值命令结果中在缩写之前打印的列表的最大深度和长度。给 ‘eval-expression’ 或 ‘eval-last-sexp’ 提供零前缀参数会使列表完整打印。‘eval-expression-debug-on-error’ 控制使用这些命令时求值错误是否调用调试器；其默认值为 ‘t’。‘eval-expression-print-maximum-character’ 防止比它大的整数被显示为字符。

29.10 Lisp 交互缓冲区

Emacs 启动时，包含一个名为 ‘*scratch*’ 的缓冲区，它用于交互式地求值 Emacs Lisp 表达式。它的主模式是 Lisp Interaction 模式。你也可以输入 ‘M-x lisp-interaction-mode’ 来启用 Lisp Interaction 模式。

若你删除了 ‘*scratch*’ 缓冲区，可以用 ‘M-x scratch-buffer’ 命令重新创建它。

在 ‘*scratch*’ 缓冲区以及其他 Lisp Interaction 模式缓冲区中，‘C-j’（‘eval-print-last-sexp’）求值光标点之前的 Lisp 表达式，并把其值插入光标点处。因此，当你在缓冲区中输入表达式并在每个表达式之后按 ‘C-j’ 时，缓冲区会记录求值过的表达式及其值的一份记录。Lisp Interaction 模式中所有其他命令都与 Emacs Lisp 模式中相同。

启动时，‘*scratch*’ 缓冲区包含一条以 Lisp 注释形式给出的简短消息，说明它的用途。该消息由变量 ‘initial-scratch-message’ 控制，其值应是一个文档字符串，或 ‘nil’（表示不显示该消息）。

交互式求值 Emacs Lisp 表达式的另一种方式是使用 Inferior Emacs Lisp 模式，它提供一个很像 Shell 模式（参见“Shell 模式”）的界面来求值 Emacs Lisp 表达式。输入 ‘M-x ielm’ 可创建一个使用该模式的 ‘*ielm*’ 缓冲区。更多信息见该命令的文档。

默认情况下，‘*scratch*’ 缓冲区不与任何文件关联。因此，当你退出 Emacs 时，缓冲区中的任何文本都会丢失。如果你愿意，可以使用 Remember 模式的笔记缓冲区特性，让 Emacs 保存并重新加载 ‘*scratch*’ 的内容。为此，请把变量 ‘initial-buffer-choice’ 定制为符号 ‘remember-notes’，并把变量 ‘remember-notes-buffer-name’ 定制为字符串 ‘"*scratch*"’。另见参见《《remember》手册》的“Quick Start”。

29.11 运行外部 Lisp

Lisp 模式是编辑用通用 Lisp 方言（如 Common Lisp）编写的程序的主模式。它的模式命令是 ‘M-x lisp-mode’。对于文件名以 ‘.l’、‘.lsp’ 或 ‘.lisp’ 结尾的文件，Emacs 会自动使用 Lisp 模式。

你可以把外部 Lisp 会话作为 Emacs 的子进程或“下级进程”运行，并向它传递表达式以求值。要开始外部 Lisp 会话，请输入 ‘M-x run-lisp’。这会运行名为 ‘lisp’ 的程序，并把它设置成使输入和输出都经过一个名为 ‘*inferior-lisp*’ 的 Emacs 缓冲区。要改变 ‘M-x run-lisp’ 所运行的 Lisp 程序名，请改变变量 ‘inferior-lisp-program’。

‘*lisp*’ 缓冲区的主模式是 Inferior Lisp 模式，它兼具 Lisp 模式和 Shell 模式的特征（参见“Shell 模式”）。若要把输入发送到该 Lisp 会话，请转到 ‘*lisp*’ 缓冲区的末尾，键入输入内容，然后按 <RET>。来自该 Lisp 会话的终端输出会自动插入到该缓冲区中。

在 Lisp 模式下编辑 Lisp 程序时，你可以键入 ‘C-M-x’（‘lisp-eval-defun’），把 Lisp 模式缓冲区中的某个表达式发送到你此前用 ‘M-x run-lisp’ 启动的 Lisp 会话。所发送的表达式是光标点处或其后第一个顶层 Lisp 表达式。求值结果照例进入 ‘*inferior-lisp*’ 缓冲区。请注意，因此在 Lisp 模式下 ‘C-M-x’ 的效果与它在 Emacs Lisp 模式下的效果非常相似（参见“求值 Emacs Lisp 表达式”），区别只在于表达式被发送到另一个 Lisp 环境中求值，而不是在 Emacs 内部求值。

用于编辑 Scheme 代码以及把表达式发送到 Scheme 子进程的功能也非常相似。Scheme 源文件在 Scheme 模式下编辑，该模式可以用 ‘M-x scheme-mode’ 显式启用。你可以键入 ‘M-x run-scheme’ 启动一个 Scheme 会话（用于与 Scheme 交互的缓冲区名为 ‘*scheme*’），并键入 ‘C-M-x’ 把表达式发送给它。

30 维护大型项目

本章介绍用于维护中大型程序和软件包的 Emacs 特性。这些特性包括：

- 用于支持记录源文件变更历史的版本控制系统（VCS）的统一界面。
- 用于处理编程项目的命令。
- 一种专门用于维护 'ChangeLog' 文件的模式，这类文件按时间顺序记录程序的变更。
- Xref，一组用于显示符号（又称"标识符"）的定义及其引用的命令。
- EDE，管理编程项目的另一种工具集，尤其用于构建和调试。
- 一种用于合并不同开发分支上对程序源代码所做变更的模式。
- 一种次模式，用于高亮缺陷引用，并在相应的议题跟踪器中查看被引用的缺陷报告。

如果你正在维护大型 Lisp 程序，那么除这里描述的特性外，你可能还会发现 Emacs Lisp 回归测试（ERT）库很有用（参见《《ert》手册》）。

- Xref：查找程序中任何函数、方法、结构体、宏等的定义和引用。
- EDE：管理编程项目的另一种工具。
- Emerge：合并程序两个版本的便捷方式。
- Bug Reference：高亮对缺陷报告的引用，并在其议题跟踪器中浏览这些报告。

30.1 版本控制

"版本控制系统"是一种能记录源文件多个版本的程序，它保存每个版本的创建时间、创建者以及对所做更改的描述等信息。

Emacs 的版本控制接口称为"VC"。VC 命令可与多种不同的版本控制系统配合工作；目前它支持 Bazaar、CVS、Git、Mercurial、RCS、SRC、SCCS/CSSC 和 Subversion。其中，GNU 项目分发 CVS、RCS 和 Bazaar。

当你访问受版本控制系统管理的文件时，VC 会自动启用。若要完全禁用 VC，请把可定制变量 'vc-handled-backends' 设为 'nil'（参见"定制 VC"）。

若要更新当前缓冲区所访问文件的 VC 状态信息，请使用命令 'vc-refresh-state'。当你在 Emacs 之外（例如在 shell 提示符下）执行版本控制命令，或把该缓冲区的文件置于另一个版本控制系统之下，或把它完全移出版本控制时，这个命令很有用。

在显示文件由 VCS 管理的目录的 Dired 缓冲区中（参见"Dired 目录编辑器"），VC 也会自动启用。本节描述的所有 VC 命令都可以在显示含 VC 管理文件的目录的任意 Dired 缓冲区中调用；Dired 缓冲区中已标记的文件（参见"Dired 标记与旗标之别"）被视为属于当前文件集，VC 命令作用于该文件集中的文件。这样，你就能构造包含任意文件的 VC 文件集，而不论这些文件的 VC 状态如何。（若从 Dired 缓冲区调用 VC 命令时没有标记任何文件，则缓冲区当前行所显示的文件被视为文件集中唯一的文件。）

30.1.1 版本控制简介

VC 让你可以在 Emacs 内部使用版本控制系统，使版本控制操作与编辑平滑地融为一体。它为许多版本控制系统中的常见操作提供了统一的界面。

某些不常见或复杂的版本控制操作，例如修改仓库设置，在 VC 中不受支持。这类任务应在 VC 之外完成，例如在命令行上执行。

本节概述版本控制，并介绍 VC 所支持的版本控制系统。如果你已经熟悉想要使用的版本控制系统，可以跳过本节。

30.1.1.1 理解它所解决的问题

版本控制系统为你提供三项重要能力：

- “可回退性”：当发现所做的某项修改是错误或糟糕的想法时，能够退回到先前的状态。
- “并发性”：让许多人同时修改同一批文件，并且知道冲突的修改可以被检测和解决。
- “历史”：能够为你的数据附加历史信息，例如说明每次更改背后意图的注释。即使对独自工作的程序员来说，变更历史也是帮助记忆的重要手段；对多人项目而言，它更是开发者之间至关重要的沟通形式。

30.1.1.2 受支持的版本控制系统

VC 目前可与许多不同的版本控制系统配合工作，它把这些系统称为“后端”：

- Git 是一种去中心化的版本控制系统，最初由 Linus Torvalds 发明，用于支持 Linux（他的内核）的开发。VC 支持 Git 的所有常见操作，但值得注意的是，除单纯编辑提交信息之外的 Git rebase 必须在命令行中完成。
- CVS 是一种自由的版本控制系统，直到大约 2008 年，它一直被大多数自由软件项目使用。此后，它已被更新的系统所取代。CVS 允许在本地或通过网络进行并发的多用户开发。与更新的系统不同，它不支持原子提交和文件的移动/重命名。VC 支持 CVS 下的所有基本编辑操作。
- Subversion（svn）是一种自由的版本控制系统，设计上与 CVS 相似但没有它的那些问题（例如，它支持文件集的原子提交，并支持目录、符号链接、元数据、重命名、复制和删除的版本化）。
- SCCS 是有史以来第一个版本控制系统，很久以前就被更先进的系统取代。对于 SCCS 中缺失的某些特性（例如发布版的标签名），VC 通过自己实现来弥补。其他 VC 特性，例如多分支，则完全不可用。由于 SCCS 是非自由的，我们建议避免使用它。
- CSSC 是 SCCS 的自由替代品。只有当出于某种原因无法使用更新、设计更好的版本控制系统时，才应使用 CSSC。
- RCS 是 VC 最初围绕其构建的自由版本控制系统。它相对原始：不能通过网络使用，并且工作在单个文件的层面上。用 RCS 几乎能做的所有事情都可以通过 VC 完成。
- Mercurial（hg）是一种去中心化的版本控制系统，大体上与 Git 相似。VC 支持大多数 Mercurial 命令。
- Bazaar（bzr）是一种去中心化的版本控制系统，既支持基于仓库的版本管理，也支持去中心化的版本管理。VC 支持 Bazaar 下的大多数基本编辑操作。

- SRC（src）是重装上阵的 RCS——一种专为仅由一人处理的单文件项目设计的版本控制系统。它允许一个目录中存在多个具有独立版本控制历史的文件，因此特别适合维护小型文档、脚本和 dotfile。它虽然使用 RCS 存储修订版本，但提供了现代的用户界面，具有无锁操作和整数顺序版本号。VC 支持几乎所有的 SRC 操作。

30.1.1.3 版本控制的概念

当文件处于版本控制之下时，我们说它已在版本控制系统中“注册”。系统有一个“仓库”，既保存文件的当前状态，也保存它的变更历史——足以重建当前版本或任何更早的版本。仓库还包含其他信息，例如描述对每个文件所做更改的“日志条目”。

你实际编辑的那份受版本控制的文件副本称为“工作文件”。你可以像修改普通文件一样修改每个工作文件。一组更改完成后，你可以“提交”（commit，或称“检入”check in）这些更改；这会把更改连同描述性的日志条目一起记录到仓库中。

由工作文件构成的目录树称为“工作树”。

每次提交都会在仓库中创建一个新的“修订版本”。版本控制系统跟踪所有过去的修订版本以及每个修订中所做的更改。每个修订版本由一个“修订 ID”命名，其格式取决于版本控制系统；最简单的情况下，它就是一个整数。

要超越这些基本概念，你需要理解版本控制系统相互区别的三个层面。如下面三节所述，它们可以是基于锁的或基于合并的；基于文件的或基于变更集的；集中式的或去中心化的。VC 能处理所有这些操作模式，但无法隐藏这些差异。

30.1.1.4 基于合并与基于锁的版本控制

版本控制系统通常有某种机制，在想要修改同一文件的用户之间进行协调。有两种做法：合并与加锁。

在使用合并的版本控制系统中，每个用户都可以随时修改工作文件。系统允许你把工作文件（其中可能含有尚未提交的更改）与他人已提交的最新更改“合并”。

较老的版本控制系统则使用“加锁”方案。在这种方案中，工作文件通常是只读的。要编辑文件，你需要请版本控制系统“锁定”该文件，使其对你可写；在任一时刻，只有一个用户可以锁定某个给定文件。这个过程与 Emacs 用来检测普通文件同时编辑的加锁机制类似（参见“防止同时编辑”），但两者并不相同。当你提交更改后，文件会被解锁，工作文件重新变为只读。之后其他用户可以锁定该文件，做出他们自己的更改。

当多个用户试图同时修改同一个文件时，加锁系统和合并系统都可能出现问题。加锁系统会有“锁冲突”；用户可能想检出某个文件，却因为它被锁定而无法检出。在合并系统中，“合并冲突”发生在你提交对某文件的更改，而该更改与在你检出之后别人提交的更改相冲突的时候。两类冲突都必须靠人的判断和沟通来解决。经验表明，无论从开发者使用的便利性看，还是从尽量减少实际发生的冲突数量和严重程度看，合并都优于加锁。

SCCS 始终使用加锁。RCS 默认基于锁，但也可以让它以合并方式工作。CVS 和 Subversion 默认基于合并，但也可以让它们以加锁模式工作。去中心化的版本控制系统，例如 Git 和 Mercurial，则只基于合并。

VC 模式同时支持加锁式和合并式的版本控制。较新的版本控制系统使用“提交”（commit）和“更新”（update）这两个术语；较老的基于锁的系统则使用“检入”（check in）和“检出”（check out）。VC 尽可能隐藏它们之间的差异。

30.1.1.5 基于变更集与基于文件的版本控制

在 SCCS、RCS、CVS 以及其他早期版本控制系统（SRC 也是如此）中，版本控制操作是“基于文件的”：每个文件都有自己的注释和修订历史，与其他所有文件彼此独立。从 Subversion 开始，较新的系统是“基于变更集的”：一次提交可以包含对多个文件的更改，整个更改集合作为一个单元处理。与这次更改相关的任何注释都不属于单个文件，而属于变更集本身。

基于变更集的版本控制比基于文件的版本控制更灵活、更强大；通常，当需要撤销对多个文件的某项更改时，能够方便地识别并移除整个更改是很有好处的。

30.1.1.6 去中心化与集中式仓库

早期的版本控制系统围绕着“集中式”模型设计，每个项目只有一个仓库供所有开发者使用。SCCS、RCS、CVS、Subversion 和 SRC 都采用这种模型。其缺点之一是，仓库在可靠性和效率上成为瓶颈。

GNU Arch 开创了“分布式”或“去中心化”版本控制的概念，后来由 Git、Mercurial 和 Bazaar 实现。一个项目可以有几个不同的仓库，这些系统支持仓库之间某种超级合并，以调和它们的变更历史。实际上，每个开发者都有一个仓库，仓库合并取代了提交操作。

VC 帮助你管理个人工作文件与仓库之间的往来。仓库是单一的主仓库，还是对等仓库网络中的一个，并不是 VC 需要关心的事情。

30.1.1.7 日志文件的类型

使用版本控制系统的项目可以有两种变更日志。一种是由版本控制系统维护的日志：每次提交更改时，你都要为这次更改填写“日志条目”（参见“日志条目缓冲区的特性”）。这称为“版本控制日志”。

另一种日志是 ‘ChangeLog’ 文件（参见“变更日志”）。它按时间顺序记录程序很大一部分（通常是一个目录及其子目录）的所有更改。小程序使用一个 ‘ChangeLog’ 文件；大程序可能每个主要目录都有一个 ‘ChangeLog’ 文件。参见“变更日志”。早在版本控制系统出现之前很久，程序员就已在使用变更日志。

基于变更集的版本系统通常为整个系统维护基于变更集的修改日志，这使变更日志文件多少有些冗余。它们仍然保留的一个好处是，有时把单个目录的变更历史与其他目录的分开查看很有用。另一个好处是，在许多版本控制系统中，提交日志里的错误无法修正。

用版本控制维护的项目可以只使用版本控制日志，也可以同时使用两种日志。它可以对某些文件采用一种方式，对另一些文件采用另一种方式。每个项目都有自己的政策，你应当遵循它。

当政策是两者并用时，通常你希望每次更改只写一次条目，然后把它放进两个日志中。你可以在 ‘ChangeLog’ 中写好条目，然后在提交更改时用 ‘C-c C-a’ 把它复制到日志缓冲区（参见“日志条目缓冲区的特性”）。也可以在提交更改时（借助 ‘C-c C-w’）在日志缓冲区中写条目，之后再使用 ‘C-x v a’ 命令把它复制到 ‘ChangeLog’（参见“变更日志与 VC”）。

30.1.2 版本控制与模式行

当你访问受版本控制的文件时，Emacs 会在模式行上指出这一点。例如，‘Bzr-1223’ 表示该文件使用 Bazaar，当前修订 ID 为 1223。

后端名称与修订 ID 之间的字符表示工作文件的“版本控制状态”。在基于合并的版本控制系统中，‘-’ 表示工作文件未被修改，‘:’ 表示它已被修改。‘!’ 表示该文件因最近的合并操作而包含冲突（参见“合并分支”），或者该

文件已从版本控制中移除，或者它受版本控制但同时被“忽略”——这通常不应发生（参见“忽略版本控制文件”）。最后，‘?’ 表示文件受版本控制，但在工作树中缺失。

在基于锁的系统中，‘-’ 表示文件未锁定，‘:’ 表示文件已锁定；如果文件被其他用户锁定（例如 ‘jim’），则显示为 ‘RCS:jim:1.3’。‘@’ 表示文件是在本地添加的，但尚未提交到主仓库。

在图形显示环境中，你可以把鼠标移到这个模式行指示符上，弹出显示版本控制状态更详细描述的工具提示。在指示符上按 ‘mouse-1’ 会弹出 VC 命令菜单，与菜单栏上的 ‘Tools / Version Control’ 相同。

当自动还原模式（Auto Revert mode，参见“还原缓冲区”）还原受版本控制的缓冲区时，它会更新模式行中的版本控制信息。然而，如果版本控制状态在没有更改工作文件的情况下从当前 Emacs 会话之外发生变化，自动还原模式可能无法正确更新这一信息。如果把 ‘auto-revert-check-vc-info’ 设为 ‘t’，自动还原模式会每隔 ‘auto-revert-interval’ 秒更新一次版本控制状态信息，即使工作文件本身没有变化。由此产生的 CPU 占用取决于版本控制系统，但通常不会过高。

30.1.3 版本控制下的基本编辑

大多数 VC 命令作用于“VC 文件集”。VC 文件集是 VC 操作所作用的一个或多个文件的集合。当你在访问受版本控制文件的缓冲区中输入 VC 命令时，VC 文件集就是那一个文件。当你在 VC Directory 缓冲区中输入这些命令，且其中有些文件已被标记时，VC 文件集由这些被标记的文件组成（参见“VC 目录模式”）。同样，当你从 Dired 缓冲区调用 VC 命令时，VC 文件集由被标记的文件组成（参见“Dired 标记与旗标之别”）；若没有标记任何文件，则默认为当前行显示的文件。

对于现代的基于变更集的版本控制系统（参见“基于变更集与基于文件的版本控制”），例如 Git、Mercurial 和 Bazaar，VC 命令把多文件 VC 文件集作为一个组来处理。例如，提交一个多文件 VC 文件集会生成单个修订版本，其中包含对所有那些文件的更改。在 CVS 这类较老的基于文件的版本控制系统中，多文件 VC 文件集中的每个文件都单独处理；因此，提交一个文件集会为文件集中每个发生更改的文件各生成一个修订版本。

‘C-x v v’ 对当前 VC 文件集执行下一个适当的版本控制操作。

主要的 VC 命令是一个多用途命令 ‘C-x v v’（‘vc-next-action’），它对当前 VC 文件集执行最适当的操作：把它注册到版本控制系统，或提交它，或解锁它，或把更改合并进去。各种情形下的确切动作将在后续小节中详细描述。你可以在访问文件的缓冲区、Dired 缓冲区或 VC Directory 缓冲区中使用 ‘C-x v v’；在后两种情况下，命令作用于由被标记文件组成的文件集。你也可以在处于 Diff 模式且含有补丁的缓冲区中使用 ‘C-x v v’（参见“Diff 模式”），此时命令作用于缓冲区中显示其差异的文件。

请注意，VC 文件集不同于用于按功能分组查看和访问文件的具名文件集（参见“文件集”）。与具名文件集不同，VC 文件集没有名称，也不会跨会话保留。

30.1.3.1 使用合并的基本版本控制

对于现代的基于合并的版本控制系统（例如 Git 和 Hg；参见“基于合并与基于锁的版本控制”），当从访问受版本控制文件的缓冲区、VC Directory 缓冲区或 Dired 缓冲区调用 ‘C-x v v’ 时，它会做以下事情：

- 如果 VC 文件集中有多个文件，而这些文件的版本控制状态不一致，则发出错误信号。（但请注意，一个文件集可以同时包含新添加的文件、已移除的文件和已修改的文件；参见“为版本控制注册文件”。）如果文件集中的任何文件被版本控制忽略，也会发出错误信号。

- 如果 VC 文件集中的每个文件都已注册，且相对于上一个修订版本没有变化，则什么也不做。
- 如果 VC 文件集中的文件都未注册到版本控制系统，则注册文件集中的新添加文件，即把它们置于版本控制之下。参见“为版本控制注册文件”。如果 Emacs 找不到可以注册的系统，它会提示输入仓库类型，创建一个新仓库，并把该 VC 文件集注册到其中。你也可以显式指定系统，参见“‘C-x v v’ 中的高级控制”。注意，注册文件并不会提交它们；你必须再次调用 ‘C-x v v’ 来提交，见下一点。
- 如果 VC 文件集中的每个文件都是新添加的或已修改的，则提交这些发生更改的文件。为此，Emacs 会弹出 ‘*vc-log*’ 缓冲区；输入你想要的更改日志条目，然后按 ‘C-c C-c’ 提交。参见“日志条目缓冲区的特性”。

如果从处于 Diff 模式的缓冲区调用 ‘C-x v v’，该命令会把缓冲区视为含有一个或多个文件的一组补丁。然后它把更改应用到相应的文件，并在弹出 ‘*vc-log*’ 缓冲区让你输入合适的提交日志消息之后提交这些更改。

一旦你输入 ‘C-x v v’，在通过在 ‘*vc-log*’ 缓冲区中键入 ‘C-c C-k’ 取消提交之前，文件集或补丁都不能更改。例如，如果在 Emacs 已经弹出 ‘*vc-log*’ 缓冲区之后，你更改了 ‘*vc-dir*’ 缓冲区中被标记的文件，那么旧的文件集仍将对本次提交生效。（这与非提交补丁时对文件集中文件*内容*所做的更改形成对比：所有这些更改都会包含在提交中，即使它们是在 Emacs 弹出 ‘*vc-log*’ 缓冲区之后才做出的。）

取消提交时，Emacs 会保存你的日志消息。这意味着如果你需要调整文件集或补丁，很容易重新开始提交流程：输入 ‘C-c C-k C-x v v M-p’。这里 ‘C-c C-k’ 取消提交，‘C-x v v’ 用新的文件集或补丁发起另一次提交，最后 ‘M-p’ 调出你之前的日志消息。

对于现代的去中心化版本控制系统（Git、Mercurial 等），更改只在本地提交，不会自动传播到上游仓库（上游仓库通常位于远程主机上）。在这种情况下，如果仓库自你上次更新以来已发生变化，提交可能会失败。此时你必须先从上游更新，然后再试一次。为此可使用 ‘C-x v +’（参见 Pulling / Pushing）或 ‘C-x v m’（参见 Merging）。

对于集中式的版本控制系统，如果提交因上游更改而失败，再次输入 ‘C-x v v’ 以合并上游仓库的更改。

- 最后，如果你使用的是集中式版本控制系统，并且 VC 文件集中的任何文件相对于上游仓库已过时，则提示你从仓库更新该文件集。

当你以非加锁模式使用 RCS 时，这些规则同样适用，只是更改不会自动从仓库合并过来。如果在你开始编辑之后其他用户提交了对同一文件的更改，你不会得到任何通知；当你提交自己的修订版本时，那位用户的更改会被移除（不过它们仍留在仓库中，因此并未不可挽回地丢失）。因此，在提交你的更改之前，必须确认当前修订版本没有变化。此外，即使在这种模式下，RCS 也支持加锁：对未修改的文件使用 ‘C-x v v’ 会锁定该文件，就像 RCS 在其常规加锁模式下所做的那样（参见“使用加锁的基本版本控制”）。

30.1.3.2 使用加锁的基本版本控制

对于基于加锁的版本控制系统（如 SCCS，以及默认模式下的 RCS），‘C-x v v’ 执行以下操作：

- 如果 VC 文件集中有多个文件，且这些文件的版本控制状态不一致，则报错。如果文件集中有任何文件被版本控制忽略，也会报错。
- 如果 VC 文件集中的每个文件都未在版本控制系统中注册，则注册文件集中新添加的文件。参见“为版本控制注册文件”。如果 Emacs 找不到可用来注册的系统，它会提示输入仓库类型，创建一个新仓库，并把该 VC 文件集注册到其中。你也可以显式指定系统，参见“‘C-x v v’ 中的高级控制”。

- 如果每个文件都已注册且未加锁，则检出这些文件：给每个文件加锁并使其可写，这样你就能开始编辑它。
- 如果每个文件都由你加锁且含有改动，则提交（commit，又称“check-in”）这些改动。为此，Emacs 会弹出‘*vc-log*’缓冲区；为新版本输入所需的日志条目，然后输入‘C-c C-c’提交（参见“日志条目缓冲区的特性”）。
- 如果每个文件都由你加锁，但你并未修改它，则释放锁并再次把文件设为只读。这会撤销先前对自检出以来未改动文件所做的检出操作。
- 如果每个文件由其他用户加锁，则询问你是否要夺取该锁。若回答是，文件就变为由你加锁，同时会向原先给该文件加锁的用户发送一条警告消息。
- 如果文件集中的文件未加锁，但相对于其最后修订有改动，则询问是否为每个这样的文件申请锁，或把文件还原到最后一次检入的修订。（这种情形属于例外，通常不应发生。）

这些规则在你以加锁模式使用 CVS 时同样适用，只是 CVS 不支持夺取锁。

30.1.3.3 ‘C-x v v’中的高级控制

当你给‘vc-next-action’一个前缀参数时（‘C-u C-x v v’），它仍然执行下一个合乎逻辑的版本控制操作，但接受额外的参数来精确指定如何执行该操作。

- 你可以指定版本控制系统的名称。当该文件集可由多个版本控制系统管理，而 Emacs 未能检测出正确的那一个时，这很有用。
- 否则，如果使用 CVS、RCS 或 SRC，你可以指定修订号。

如果文件集已被修改（或已加锁），这会让 Emacs 用该修订号提交这些文件。你可以通过提供合适的修订号来创建一个新分支（*note Branches::）。

如果文件集未被修改（且未加锁），这会把指定的修订检出到工作树中。你也可以通过给出另一分支上的修订号或分支 ID 来指定该分支上的一个修订（*note Switching Branches::）。空参数（即‘C-u C-x v v <RET>’）会检出当前分支上最新的（又称“head”）修订。

以这种方式指定修订号会被分布式版本控制系统静默忽略。这些系统不允许你指定自己的修订号，也不使用检出单个文件的概念。

30.1.4 日志条目缓冲区的特性

当你让 VC 提交某项改动时，它会弹出一个名为‘*vc-log*’的缓冲区。在此缓冲区中，你应编写一条“日志条目”（log entry）来描述你所做的改动（参见“理解它所解决的问题”）。完成后，输入‘C-c C-c’（‘log-edit-done’）退出缓冲区，并连同你的日志条目一起提交该改动。

‘*vc-log*’缓冲区的主模式是 Log Edit 模式，它是文本模式的一种变体（参见“文本模式”）。进入 Log Edit 模式时，Emacs 会运行钩子‘text-mode-hook’和‘vc-log-mode-hook’（参见“钩子”）。

在‘*vc-log*’缓冲区中，你可以写一行或多行“标题行”（header line），用来指定要提供给版本控制系统的附加信息。每个标题行必须占据缓冲区顶部的一行；第一个不是标题行的行被视为日志条目的开始。例如，下面的标

题行说明本次改动并非由你编写，而是由另一位开发者编写：

```
Author: J. R. Hacker <jrh@example.com>
```

除了‘Author’标题，Emacs 还识别‘Summary’（变更集的一行摘要）、‘Date’（手动指定的提交时间）和‘Fixes’（对本次改动所修复的缺陷的引用）这几个标题。并非所有版本控制系统都识别所有标题。如果你为不支持某个标题的系统指定了该标题，该标题会被当作日志条目的一部分。

在‘*vc-log*’缓冲区中，当前 VC 文件集被视为你输入‘C-c C-c’时将要提交的文件集。要查看该 VC 文件集中的文件列表，输入‘C-c C-f’（‘log-edit-show-files’）。要查看 VC 文件集与你开始编辑时所基于的版本之间改动的差异，输入‘C-c C-d’（‘log-edit-show-diff’）（参见“查看并比较旧修订”）。

为帮助生成 ChangeLog 条目，输入‘C-c C-w’（‘log-edit-generate-changelog-from-diff’），以便根据 VC 文件集的差异生成 ChangeLog 条目的骨架，列出所有改动的文件名和函数名。留空的连续条目会被‘M-q’（‘fill-paragraph’）合并。默认情况下，骨架只包含文件名，不含任何前置目录。如果你希望前置到 VC 根目录为止的各级目录，请定制‘diff-add-log-use-relative-names’。

如果 VC 文件集包含一个或多个‘ChangeLog’文件（参见“变更日志”），输入‘C-c C-a’（‘log-edit-insert-changelog’）把相关条目取到‘*vc-log*’缓冲区中。如果每个‘ChangeLog’中最上面的一项是在当前日期以你的用户名添加的，该命令会在该项中搜索与要提交的文件匹配的条目并把它们插入。如果你使用 CVS 或 RCS，参见“变更日志与 VC”，了解相反的工作方式——从 Log Edit 缓冲区生成 ChangeLog 条目。

要中止提交，只需在该缓冲区中不输入‘C-c C-c’。你可以切换缓冲区并进行其他编辑。只要不尝试进行另一次提交，你正在编辑的条目就会留在‘*vc-log*’缓冲区中，你可以随时回到该缓冲区完成提交。

你还可以浏览以往日志条目的历史，以复制某条提交注释。当你想用相似的注释做多次提交时，这很有用。用于此目的的‘M-n’、‘M-p’、‘M-s’和‘M-r’命令的工作方式与迷你缓冲区历史命令相同（参见“迷你缓冲区历史”），只是它们在迷你缓冲区之外使用。

30.1.5 为版本控制注册文件

‘C-x v i’为访问的文件注册版本控制。

‘C-x v i’（‘vc-register’）命令“注册”当前 VC 文件集中的每个文件，把它置于版本控制之下。这实质上等同于对未注册的 VC 文件集执行‘C-x v v’的操作（参见“版本控制下的基本编辑”），区别在于：如果该 VC 文件集已经注册，‘C-x v i’会报错，而‘C-x v v’会执行某种其他操作。

要注册文件，Emacs 必须选择一种版本控制系统。对于多文件的 VC 文件集，由 VC Directory 缓冲区指定要使用的系统（参见“VC 目录模式”）。对于单文件的 VC 文件集，如果该文件所在目录已包含在某个版本控制系统中注册的文件，或该目录是某个版本控制系统所控制的目录树的一部分，Emacs 就会选择该系统。如果有多个版本控制系统都适用，Emacs 使用‘vc-handled-backends’变量中排在最前面的那个（参见“定制 VC”）。如果 Emacs 找不到可用来注册该文件的版本控制系统，它会提示输入仓库类型，创建一个新仓库，并把该文件注册到该仓库中。

对于大多数版本控制系统，用‘C-x v i’或‘C-x v v’注册文件会把它加入工作树，但不会提交它，即不会把它加入仓库。这类文件在 VC Directory 缓冲区中标记为“added”，访问这类文件的缓冲区的模式行显示的修订号为

‘@@’。要使注册在仓库中生效，你必须提交新添加的文件（参见“版本控制下的基本编辑”）。注意，一次提交可以既包含添加文件，也包含对 VCS 已知文件的编辑。

对于基于加锁的版本控制系统（参见“基于合并与基于锁的版本控制”），注册文件后该文件保持未加锁且只读。输入 ‘C-x v v’ 检出该文件并开始编辑它。

30.1.6 查看并比较旧修订

‘C-x v =’ 将当前 VC 文件集中的工作文件与你开始编辑时所基于的版本进行比较（‘vc-diff’）。带前缀参数时，提示输入当前 VC 文件集的两个修订并比较它们。你也可以从 Dired 缓冲区调用此命令（参见“对文件进行操作”）。

‘M-x vc-ediff’ 与 ‘C-x v =’ 类似，但使用 Ediff。参见《Ediff 手册》

‘C-x v D’ 将整个工作树与你开始编辑时所在的修订进行比较（‘vc-root-diff’）。带前缀参数时，提示输入两个修订并比较它们对应的目录树。

‘C-x v ~’ 提示输入当前文件的一个修订，并在单独的缓冲区中访问它（‘vc-revision-other-window’）。

‘C-x v g’ 显示当前文件的注解版本：对每一行，显示它最后一次被修改所在的修订（‘vc-annotate’）。

‘C-x v =’（‘vc-diff’）显示一个“差异”（diff），它把当前 VC 文件集中的每个工作文件与你开始编辑时所基于的版本进行比较。该差异显示在另一个窗口里名为 ‘*vc-diff*’ 的 Diff 模式缓冲区中（参见“Diff 模式”）。此缓冲区中可使用通常的 Diff 模式命令。特别是，‘g’（‘revert-buffer’）命令会重新执行文件比较，生成新的差异。

要比较当前 VC 文件集的任意两个修订，带前缀参数调用 ‘vc-diff’：‘C-u C-x v =’。这会提示输入两个修订号（参见“版本控制的概念”），并显示该文件集这两个版本之间的差异。对于多文件的 VC 文件集，如果版本控制系统是基于文件而非基于变更集的（例如 CVS），这不能可靠工作，因为那样不同文件的修订号之间没有任何有意义的关联。

有些版本控制系统允许你用其他格式而不只是修订号来指定修订。例如，在 Bazaar 下，你可以为 ‘C-u C-x v =’（以及相关命令）的参数输入 ‘date:yesterday’，以指定昨天之后提交的第一个修订。详情请参阅该版本控制系统的文档。

如果你从 Dired 缓冲区调用 ‘C-x v =’ 或 ‘C-u C-x v =’（参见“Dired 目录编辑器”），当前行列出的文件被视为当前 VC 文件集。该 VC 文件集也可以包含若干已标记的文件。

‘M-x vc-ediff’ 的工作方式与 ‘C-x v =’ 相同，只是它使用 Ediff 会话。参见《Ediff 手册》

‘C-x v D’（‘vc-root-diff’）与 ‘C-x v =’ 类似，但它显示整个当前工作树（即包含当前 VC 文件集的工作树）中的改动。如果你从 Dired 缓冲区调用此命令，它作用于包含该目录的工作树。

要比较整棵目录树的任意两个修订，带前缀参数调用 ‘vc-root-diff’：‘C-u C-x v D’。这会提示输入两个修订号（参见“版本控制的概念”），并显示整个受版本控制的目录树这两个版本之间的差异（RCS、SCCS、CVS 和 SRC 不支持此特性）。

你可以定制 ‘C-x v =’ 和 ‘C-x v D’ 生成差异时使用的 ‘diff’ 选项。所用选项依次取自变量 ‘vc-BACKEND-diff-switches’、‘vc-diff-switches’ 和 ‘diff-switches’ 中第一个非 ‘nil’ 的值（参见“比较文件”）。这里 BACKEND 代表相关的版本控制系统，例如 Bazaar 对应 ‘bzr’。由于 ‘nil’ 表示检查序列中的下一个变量，前两个变量中的任何一

个都可以使用值 't' 来表示完全不使用任何开关。大多数 'vc-BACKEND-diff-switches' 变量默认为 'nil'，但有些默认为 't'；后者适用于其 'diff' 实现不接受常见 diff 选项的版本控制系统，例如 Subversion。

要直接查看文件的某个旧版本，访问该工作文件并输入 'C-x v ~ REVISION <RET>'（'vc-revision-other-window'）。这会取出与 REVISION 对应的文件版本，把它保存到 'FILENAME.~REVISION~'，并在单独的窗口中访问它（参见"'display-buffer' 的工作方式"）。

许多版本控制系统允许你输入 'C-x v g'（'vc-annotate'）来查看带有逐行修订信息的"注解"（annotated）文件。这会创建一个新的"注解"缓冲区来显示文件文本，其中每一行都着色以显示它有多旧。红色文本是新的，蓝色是旧的，中间颜色表示中间的新旧程度。默认情况下，颜色在全部时间范围内缩放，使最旧的改动为蓝色，最新的改动为红色。如果变量 'vc-annotate-background-mode' 非 'nil'，表示每行新旧程度的颜色会应用于背景色，前景色则保持默认颜色。

你可以通过定制 'vc-BACKEND-annotate-switches' 和 'vc-annotate-switches' 来定制 'C-x v g' 使用的 'annotate' 选项。它们的作用与上文所述的 'vc-BACKEND-diff-switches' 和 'vc-diff-switches' 类似。

当你给 'C-x v g' 一个前缀参数时，Emacs 使用迷你缓冲区读取两个参数：要显示并注解的修订（代替当前文件内容），以及颜色范围应覆盖的天数跨度。

在"注解"缓冲区中，这些以及其他颜色缩放选项可从 'VC-Annotate' 菜单获得。在此缓冲区中，你还可以使用以下按键浏览以往修订的注解、查看差异或查看日志条目：

'p' 注解前一个修订，即当前所注解修订之前的那个修订。数字前缀参数表示重复次数，因此 'C-u 10 p' 会带你回到 10 个修订之前。

'n' 注解下一个修订，即当前所注解修订之后的那个修订。数字前缀参数表示重复次数。

'j' 注解当前行所指明的修订。

'a' 注解当前行所指明的修订之前的那个修订。这有助于查看当前行的改动发生之前文件所处的状态。

'f' 在缓冲区中显示当前行所指明的文件修订。

'd' 显示当前行所在修订与前一个修订之间的差异。这有助于查看当前行所在修订实际在文件中改动了什么。

'D' 显示当前行所在修订与前一个修订之间针对变更集中所有文件的差异（适用于支持变更集的 VC 系统）。这有助于查看当前行所在修订实际在目录树中改动了什么。

'l' 显示当前行所在修订的日志。这有助于查看作者对当前行所在修订中改动的描述。

'w' 注解工作修订——即你正在编辑的那个修订。如果你用 'p' 和 'n' 浏览到了其他修订，用此键回到你的工作修订。

'v' 切换注解的可见性。这有助于只看文件内容，而不受注解干扰。

30.1.7 VC 变更日志

'C-x v l' 显示当前文件集的变更历史（'vc-print-log'）。

'C-x v L' 显示当前仓库的变更历史（'vc-print-root-log'）。

'C-u 1 C-x v L REVISION <RET>' 显示单个 REVISION 的日志条目和改动（差异）（'vc-print-root-log'）。

‘C-x v b l’ 显示当前文件集在另一个分支上的变更历史（‘vc-print-fileset-branch-log’）。

‘C-x v b L’ 显示当前仓库在另一个分支上的变更历史（‘vc-print-root-branch-log’）。

‘C-x v I’ 显示“拉取”（pull）操作将获取的改动的日志条目（‘vc-root-log-incoming’）。

如果你把 ‘vc-use-incoming-outgoing-prefixes’ 定制为非 ‘nil’，‘C-x v I’ 就变成前缀键，‘vc-root-log-incoming’ 则绑定到 ‘C-x v I L’。

‘M-x vc-root-diff-incoming’ 显示一次拉取操作将获取的所有改动的差异。

如果你把 ‘vc-use-incoming-outgoing-prefixes’ 定制为非 ‘nil’，此命令可在 ‘C-x v I D’ 上使用。

‘M-x vc-diff-incoming’ 显示一次拉取操作将获取的改动的差异，但仅限于当前文件集。

如果你把 ‘vc-use-incoming-outgoing-prefixes’ 定制为非 ‘nil’，此命令可在 ‘C-x v I =’ 上使用。

‘C-x v O’ 显示下一次“推送”（push）操作将发送的改动的日志条目（‘vc-root-log-outgoing’）。

如果你把 ‘vc-use-incoming-outgoing-prefixes’ 定制为非 ‘nil’，‘C-x v O’ 就变成前缀键，‘vc-root-log-outgoing’ 则绑定到 ‘C-x v O L’。

‘M-x vc-root-diff-outgoing’ 显示下一次推送操作将发送的所有改动的差异。

如果你把 ‘vc-use-incoming-outgoing-prefixes’ 定制为非 ‘nil’，此命令绑定到 ‘C-x v O D’。

‘M-x vc-diff-outgoing’ 显示下一次推送操作将发送的改动的差异，但仅限于当前文件集。

如果你把 ‘vc-use-incoming-outgoing-prefixes’ 定制为非 ‘nil’，此命令可在 ‘C-x v O =’ 上使用。

‘C-x v E D’ 显示下一次推送操作将发送的所有改动再加上任何未提交改动的合并差异。

‘C-x v E =’ 显示下一次推送操作将发送的所有改动再加上任何未提交改动的合并差异，但仅限于当前文件集。

‘C-x v h’ 显示当前缓冲区所访问文件在区域中所做改动的历史（‘vc-region-history’）。

‘M-x vc-log-search <RET>’ 在变更历史中搜索指定的模式。

‘C-x v l’（‘vc-print-log’）显示名为 ‘*vc-change-log*’ 的缓冲区，以详细形式展示对当前文件集所做改动的历史，包括改动者、日期以及每项改动的日志条目（这些日志条目与你通过 ‘*vc-log*’ 缓冲区输入的相同；参见“日志条目缓冲区的特性”）。如果从访问某个文件的缓冲区调用，当前文件集就由那一个文件组成，所显示的 ‘*vc-change-log*’ 缓冲区中的光标点会居中于该文件的修订处。如果从 VC Directory 缓冲区（参见“VC 目录模式”）或 Dired 缓冲区（参见“对文件进行操作”）调用，文件集由所有已标记的文件组成；若没有标记任何文件，则默认为目录缓冲区中当前行显示的那个文件。

如果文件集包含一个或多个目录，所生成的 ‘*vc-change-log*’ 缓冲区会显示改动的简短日志（每项改动一行），前提是该 VC 后端支持这样做；否则它以详细形式显示日志。

带前缀参数时，该命令会提示输入要在 ‘*vc-change-log*’ 缓冲区中居中的修订，以及要显示的修订数目上限。

‘C-x v L’（‘vc-print-root-log’）显示 ‘*vc-change-log*’ 缓冲区，展示整个受版本控制目录树的历史（RCS、SCCS、CVS 和 SRC 不支持此特性）。带前缀参数时，该命令提示输入要显示的修订数目上限。数字前缀参数则直接指定修订数目上限，不再提示。

如果数字前缀参数为 1，例如 'C-1 C-x v L' 或 'C-u 1 C-x v L'，该命令会提示输入修订号，并显示该修订的日志条目以及它引入的改动（差异）。（某些能力较弱的版本控制系统，如 RCS 和 CVS，没有显示修订日志及其差异的命令；对它们，该命令只显示日志条目，你可以输入 'd' 或 'D' 请求显示差异，见下文。）

'C-x v L' 的历史以紧凑形式显示，通常只显示每个日志条目的第一行。不过，你可以在 '*vc-change-log*' 缓冲区中输入 <RET>（'log-view-toggle-entry-display'），以展开光标点处修订的整个日志条目。再次输入 <RET> 会重新隐藏它。

'C-x v b l BRANCH-NAME <RET>'（'vc-print-fileset-branch-log'）显示 '*vc-change-log*' 缓冲区，以详细形式展示对当前文件集所做改动的历史，类似 'vc-print-log'，但它显示的是当前分支之外另一分支的历史；它会提示输入你想查看其历史的分支名。'C-x v b L BRANCH-NAME <RET>'（'vc-print-root-branch-log'）与之类似，只是它像 'vc-print-root-log' 那样显示受版本控制目录树的历史。

对于适合这样做的版本控制系统，你也可以用修订号代替分支名作为 BRANCH-NAME，Emacs 将从该修订号开始打印日志。你还可以指定 VCS 支持的用于特定修订的其他名称，例如标签名（参见"修订标签"），或 Git 支持的远程分支引用。

在分布式版本控制系统中，'C-x v I'（'vc-root-log-incoming'）命令显示一个日志缓冲区，展示你下次运行该版本控制系统的 pull 命令、从另一个远程位置获取新修订时将要应用的改动（参见"把改动拉入/推出分支"）。这个远程位置是版本控制系统所定义的默认拉取来源；带前缀参数时，'vc-root-log-incoming' 会提示输入特定的远程位置。类似地，'C-x v O'（'vc-root-log-outgoing'）显示你下次运行 push 命令时将要发送到另一个远程位置的改动；带前缀参数时，它会提示输入特定目的地，对某些版本控制系统而言该目的地可以是分支名。

与之密切相关的命令 'vc-root-diff-incoming' 和 'vc-root-diff-outgoing' 是 'vc-root-log-incoming' 和 'vc-root-log-outgoing' 的差异版本。它们显示报告将要拉取或推送的改动的差异缓冲区。你也可以在这里使用前缀参数来指定特定的远程位置。'vc-root-diff-outgoing' 可用于预览你的推送，并快速检查是否所有、且仅有你希望包含的改动已被提交并将被推送。

'vc-diff-incoming' 和 'vc-diff-outgoing' 命令非常相似。它们也显示将要拉取或推送的改动。区别在于所报告的差异仅限于当前文件集。不要忘记，实际的拉取和推送操作总是影响整个工作树，而不仅仅是当前文件集。

最后，还有 'C-x v E D'（'vc-root-diff-outgoing-and-edited'），它显示一份合并的差异，包含所有将被推送的改动以及任何未提交的改动。它适合展示所有只存在于本地的工作。还有 'C-x v E ='（'vc-diff-outgoing-and-edited'），作用相同，但只限于当前文件集。

在 '*vc-change-log*' 缓冲区中，可以使用下列按键在修订与文件的日志之间移动，并查看、比较过去的修订（参见"查看并比较旧修订"）：

'p' 移动到上一个修订条目（'log-view-msg-prev'）。（日志缓冲区中的修订条目通常按逆时间顺序排列，因此上一个修订条目通常对应较新的修订。）数字前缀参数表示重复次数。

'n' 移动到下一个修订条目（'log-view-msg-next'）。数字前缀参数表示重复次数。

'a' 为当前行的修订添加注释（'log-view-annotate-version'；参见"查看并比较旧修订"）。

'e' 修改显示在光标点处的变更注释（'log-view-modify-change-comment'）。注意并非所有 VC 系统都支持修改变更注释。

‘f’ 访问当前行所指的修订（‘log-view-find-revision’）。

‘d’ 显示当前行修订与其上一个较早修订之间针对特定文件的差异（‘log-view-diff’）。

‘D’ 显示当前行修订与其上一个较早修订之间的变更集差异（‘log-view-diff-changeset’）。这会显示该修订对所有文件所做的改动。

‘w’ 把光标点处日志条目的修订 ID 复制到删除环（参见“删除环”），效果如同使用了 ‘M-w’（‘log-view-copy-revision-as-kill’）。若标记了多个修订，该命令会把它们的 ID 全部复制到删除环，以空格分隔。

‘<RET>’ 在紧凑风格的日志缓冲区（例如 ‘C-x v L’ 创建的那个）中，切换显示或隐藏光标点处修订的完整日志条目（‘log-view-toggle-entry-display’）。

‘M-<RET>’ 若相关，会像 ‘RET’ 那样显示完整的日志条目，同时像 ‘D’ 那样显示差异，但不选中显示差异的窗口（‘log-view-display-entry-and-diff’）。若完整日志条目文本已经显示，则绝不隐藏它。

‘M-p’ 相当于先 ‘p’ 再 ‘M-<RET>’：移动到上一个条目，若相关则以完整形式显示它，并显示其差异（‘log-view-msg-and-diff-prev’）。数字前缀参数只作为移动部分的重复次数；最多只有一个条目会以完整形式显示。

‘M-n’ 相当于先 ‘n’ 再 ‘M-<RET>’：移动到上一个条目，若相关则以完整形式显示它，并显示其差异（‘log-view-msg-and-diff-next’）。数字前缀参数只作为移动部分的重复次数；最多只有一个条目会以完整形式显示。

‘m’ 标记光标点处的条目（‘log-view-mark-entry’）。

‘u’ 取消标记光标点处的条目（‘log-view-unmark-entry’）。

‘U’ 取消标记所有已标记的条目（‘log-view-unmark-all-entries’）。

‘C’ 把改动复制到当前检出的分支；可以是光标点处修订的改动，也可以是所有已标记修订的改动（‘log-view-cherry-pick’）。

‘R’ 撤销旧修订的影响；可以是光标点处的修订，也可以是所有已标记的修订（‘log-view-revert-or-delete-revisions’）。

‘x’ 从当前分支删除比光标点处修订更新的修订，而不触碰工作树（‘log-view-uncommit-revisions-from-end’）。

‘X’ 从当前分支删除比光标点处修订更新的修订（‘log-view-delete-revisions-from-end’）。

因为获取大量日志条目可能很慢，‘*vc-change-log*’ 缓冲区默认最多显示 2000 个修订。变量 ‘vc-log-show-limit’ 指定这个上限；若把它的值设为零，则取消该限制。你也可以点击缓冲区末尾的 ‘Show 4X entries’ 或 ‘Show unlimited entries’ 按钮，增加已有 ‘*vc-change-log*’ 缓冲区中显示的修订数量。不过，RCS、SCCS、CVS 和 SRC 不支持此特性。

查看变更历史还有一种有用的变体，由命令 ‘vc-region-history’ 提供（默认绑定到 ‘C-x v h’），它显示一个 ‘*VC-history*’ 缓冲区，其中包含当前缓冲区文件中光标点与标记之间区域的变更历史（参见“标记与区域”）。变更历史既包括提交日志消息，也包括以 Diff 格式呈现的改动本身。

先在当前缓冲区中标记你感兴趣的那段区域，然后再调用该命令。在它弹出的 ‘*VC-history*’ 缓冲区中，你可以使用前面介绍过的 ‘*vc-change-log*’ 缓冲区里全部可用命令，还可以使用 Diff 模式定义的命令（参见“Diff 模式”）。

该命令目前只在 Git 和 Mercurial（hg）下可用。

命令‘vc-log-search’用于在变更日志中搜索模式。它提示输入一个模式（正则表达式），并显示变更历史中日志消息匹配该模式的所有条目。带前缀参数调用时，该命令还会提示输入一个特定的 VCS shell 命令，用于此目的。

30.1.8 撤销版本控制操作

‘C-x v u’‘C-x v @’把当前 VC 文件集中的工作文件还原到最后一个修订（‘vc-revert’）。

‘M-x vc-revert-or-delete-revision’撤销较早提交的影响。

‘M-x vc-revert-revision’新建一个提交，以撤销较早提交所做的改动。

‘M-x vc-delete-revision’从修订历史中删除一个尚未推送的提交。

如果你想丢弃对当前 VC 文件集所做的全部改动，输入‘C-x v u’或‘C-x v @’（‘vc-revert’）。在丢弃改动之前，它会要求你确认。若你同意，就会还原该文件集。

若‘vc-revert-show-diff’不为‘nil’，该命令会显示工作文件与开始编辑时所在修订之间的差异。之后，差异缓冲区要么被删除（若该变量的值为‘kill’），要么被埋藏（任何其他非‘nil’值）。如果你不希望‘vc-revert’显示差异，就把该变量设为‘nil’（你仍可用‘C-x v =’直接查看差异；参见“查看并比较旧修订”）。

在基于锁定的版本控制系统中，‘vc-revert’会让文件处于未锁定状态；你必须再次锁定才能继续编辑。如果你锁定了某个文件之后又决定不做修改，也可以用‘vc-revert’解锁该文件。

要丢弃已经提交的改动（无论由你自己还是别人提交），可以使用‘M-x vc-revert-or-delete-revision’。这称为“还原”（revert）一个提交。该命令提示输入要还原的修订，然后由 VC 后端执行还原。

多数后端通过新建一个提交来撤销该修订所做的改动，从而实现这一点。对于分布式 VCS，若该提交是你做的并且尚未推送，Emacs 会改为提议把它整个删除。带前缀参数时，该命令只会尝试彻底删除该修订；若它已被推送，则操作失败。

访问这一功能的另一种方式是命令‘log-view-revert-or-delete-revisions’，它在 Log View 模式缓冲区中绑定到‘R’（参见“VC 变更日志”）。与直接使用‘M-x vc-revision revert’相比，这样更容易确保你还原的正是你想还原的那个修订。

更具体、更专用的命令是‘M-x vc-revert-revision’和‘M-x vc-delete-revision’。前者提示输入一个修订，然后总是新建一个提交来撤销该修订所做的改动，无论使用哪种 VCS，也无论该修订是否已推送。后者提示输入一个修订，然后总是删除它，不过若该提交已被推送，它会停下来。通常‘vc-revert-or-delete-revision’已经够用，但‘vc-revert-revision’和‘vc-delete-revision’有时对构造特定的版本控制历史很有用。

30.1.9 忽略版本控制文件

‘C-x v G’在当前版本控制系统中忽略某个文件（‘vc-ignore’）。

许多源码树都包含一些无需纳入版本控制的文件，例如编辑器备份文件、目标文件或字节码文件，以及编译出的程序。你完全可以不添加它们，但它们总会作为未知文件冒出来。你也可以把它们加入源码树顶层的忽略文件，从而告诉版本控制系统忽略这些文件。‘C-x v G’（‘vc-ignore’）可以帮你做到这一点。带前缀参数调用时，你可以从忽略文件列表中移除某个文件。

30.1.10 VC 目录模式

“VC 目录缓冲区”是一种专门的缓冲区，用于查看目录树中各文件的版本控制状态，并针对这些文件执行版本控制操作。特别地，它用于为 ‘C-x v v’ 这类命令指定多文件 VC 文件集（参见“VC 目录命令”）。

要使用 VC 目录缓冲区，输入 ‘C-x v d’（‘vc-dir’）。它会用迷你缓冲区读取目录名，并切换到该目录的 VC 目录缓冲区。默认情况下，该缓冲区名为 ‘*vc-dir*’。其内容见参见“VC 目录缓冲区”。

‘vc-dir’ 命令会自动检测指定目录中要使用的版本控制系统。若该目录中使用了不止一种系统，你应该带前缀参数调用该命令，‘C-u C-x v d’；它会提示输入 VC 目录缓冲区应使用的版本控制系统。

除 VC 目录缓冲区之外，Emacs 还有一个类似的设施 PCL-CVS，它专用于 CVS。参见《《pcl-cvs》手册》

你也可以从 Dired 缓冲区中调用 VC 命令（参见“Dired 目录编辑器”）。在这种情况下，你调用的任何 VC 命令都会把已标记的文件视为当前文件集（参见“版本控制下的基本编辑”）；若没有标记任何文件，则默认为当前行上的文件。

30.1.10.1 VC 目录缓冲区

VC 目录缓冲区包含一个受版本控制的文件及其版本控制状态的列表。它列出当前目录（即你调用 ‘C-x v d’ 时指定的目录）及其子目录中的文件，但只列出状态值得注意的那些文件。已是最新状态（即与仓库中的相同）或被忽略的文件会被省略。若某个子目录中的所有文件都已是最新状态，该子目录也不会列出。作为例外，若某个文件因为某条 VC 命令而直接变为最新状态，默认情况下它会被列出。

下面是 VC 目录缓冲区列表的一个例子：

```
                  ./
    edited        configure.ac
*   added         README
    unregistered  temp.txt
                  src/
*   edited        src/main.c
```

有两个工作文件已被修改但尚未提交：当前目录中的 ‘configure.ac’，以及 ‘src/’ 子目录中的 ‘main.c’。名为 ‘README’ 的文件已被添加但尚未提交，而 ‘temp.txt’ 不在版本控制之下（参见“为版本控制注册文件”）。

‘README’ 和 ‘src/main.c’ 条目旁的 ‘*’ 字符表示用户已把这些文件标记为当前 VC 文件集（参见“VC 目录命令”）。

上述例子对 Bazaar、Git 或 Mercurial 这样的分布式版本控制系统来说是典型的（参见“去中心化与集中式仓库”）。其他系统可能显示别的状态。例如，若仓库中有尚未应用到工作文件的改动，CVS 会显示 ‘needs-update’ 状态。RCS 和 SCCS 会把锁定文件的用户名显示为其状态。

如上所述，默认情况下，在 VC 目录缓冲区首次显示之后变为最新状态或被忽略的文件与目录仍会保持显示。若要自动把这类文件从显示中移除，请把变量 ‘vc-dir-auto-hide-up-to-date’ 定制为值 ‘t’。你也可以把它定制为值 ‘revert’，此时这类文件只在你刷新 VC 目录显示时才会被移除，例如输入 ‘g’（参见“VC 目录命令”）。

在 CVS 上，‘vc-dir’ 命令通常会联系仓库（它可能位于远程机器上）来检查更新。若你把变量 ‘vc-cvs-stay-local’ 改为 ‘nil’（参见“CVS 专有选项”），Emacs 在生成 VC 目录缓冲区时就会避免联系远程仓库（在必要时它仍会联系，例如执行提交时）。若你在离线工作或网络很慢，这样做可能是可取的。

VC 目录缓冲区会省略‘vc-directory-exclusion-list’变量中列出的子目录。它的默认值包含版本控制系统内部使用的目录。

在缓冲区顶部，Emacs 会显示一些关于仓库的信息，例如所用后端的名称和工作目录。此外，对于分布式 VCS，若你有待推送的提交（参见“VC 变更日志”），Emacs 会显示一行“Outgoing : N unpushed revisions”，其中 N 是一个数字。你可以点击这段文字来执行‘vc-root-log-outgoing’命令（参见“VC 变更日志”）。

Emacs 会尽量利用缓存信息来确定未推送修订的数量，但对某些后端来说这做不到。在这些情况下，Emacs 必须不时从远程仓库获取数据才能确定数量。若你到远程仓库的连接很慢，这可能使刷新 VC 目录缓冲区慢得无法接受。若你受此影响，可以把‘vc-dir-show-outgoing-count’定制为‘nil’，以完全禁用未推送修订计数。你也可以用目录局部变量按仓库分别设置它（参见“按目录的局部变量”）。

30.1.10.2 VC 目录命令

Emacs 提供若干命令，用于在 VC 目录缓冲区中导航，把文件标记为属于当前 VC 文件集，以及对 VC 文件集进行操作。

‘n’‘<SPC>’把光标点移到下一个条目（‘vc-dir-next-line’）。

‘p’把光标点移到上一个条目（‘vc-dir-previous-line’）。

‘<TAB>’移到下一个目录条目（‘vc-dir-next-directory’）。

‘S-<TAB>’移到上一个目录条目（‘vc-dir-previous-directory’）。

‘<RET>’‘f’访问当前行列出的文件或目录（‘vc-dir-find-file’）。

‘o’在另一个窗口中访问当前行上的文件或目录（‘vc-dir-find-file-other-window’）。

‘m’标记当前行上的文件或目录（‘vc-dir-mark’），把它加入当前 VC 文件集。若区域处于活动状态，则标记区域中的所有文件。

若某个文件已经位于已标记的目录或其某个子目录中，就不能用该命令标记它。同样，若某个目录树中有任何文件已被标记，就不能用该命令标记该目录。

‘M’若光标点在文件条目上，则标记所有状态相同的文件；若光标点在目录条目上，则标记该目录树中的所有文件（‘vc-dir-mark-all-files’）。带前缀参数时，标记所有列出的文件和目录。

‘% m’‘* %’你可以用该命令按正则表达式标记文件（‘vc-dir-mark-by-regexp’）。若给出前缀参数，则改为取消标记。

‘* r’你可以用该命令标记处于已注册状态（包括 edited、added 或 removed）的文件（‘vc-dir-mark-registered-files’）。

‘G’把光标点处的文件加入 VC 应当忽略的文件列表（‘vc-dir-ignore’）。例如，若 VC 是 Git，它会把这个文件追加到‘.gitignore’文件中。若给出前缀参数，则对所有已标记的文件执行此操作。

‘@’丢弃你对当前文件集所做的全部改动（‘vc-revert’）。

‘V’对 VC 目录缓冲区中显示的所有文件执行下一个合乎逻辑的版本控制操作（‘vc-dir-root-next-action’）。这类似于‘C-x v v’（参见“版本控制下的基本编辑”），只是它会忽略任何标记以及光标点的位置。

‘u’ 取消标记当前行上的文件或目录。若区域处于活动状态，则取消标记区域中的所有文件（‘vc-dir-unmark’）。

‘U’ 若光标点在文件条目上，则取消标记所有状态相同的文件；若光标点在目录条目上，则取消标记该目录树中的所有文件（‘vc-dir-unmark-all-files’）。带前缀参数时，取消标记所有文件和目录。

‘x’ 隐藏状态为 ‘up-to-date’ 或 ‘ignored’ 的文件（‘vc-dir-hide-up-to-date’）。带前缀参数时，隐藏状态与光标点处条目相同的那些条目。

‘g’ 刷新 VC 目录缓冲区的显示（‘revert-buffer’）。

‘q’ 退出 VC 目录缓冲区并把该缓冲区埋藏起来（‘quit-window’）。

在 VC 目录缓冲区中，你用 ‘m’（‘vc-dir-mark’）或 ‘M’（‘vc-dir-mark-all-files’）标记的所有文件都属于当前 VC 文件集。若你用 ‘m’ 标记某个目录条目，该目录树中所有列出的文件都属于当前 VC 文件集。属于当前 VC 文件集的文件和目录，在 VC 目录缓冲区中会在其 VC 状态旁用一个 ‘*’ 字符标出。这样，你就能建立一个多文件的 VC 文件集，供 ‘C-x v v’（参见“版本控制下的基本编辑”）、‘C-x v =’（参见“查看并比较旧修订”）和 ‘C-x v u’（参见“撤销版本控制操作”）等 VC 命令操作。

VC 目录缓冲区还为带 ‘C-x v’ 前缀的 VC 命令定义了一些单键快捷方式：‘=’、‘+’、‘l’、‘i’、‘D’、‘L’、‘G’、‘I’、‘O’ 和 ‘v’。

例如，你可以打开一个 VC 目录缓冲区（其中的文件以 ‘edited’ 状态列出），标记这些文件，然后输入 ‘v’ 或 ‘C-x v v’（‘vc-next-action’），从而提交一组已修改的文件。若版本控制系统是基于变更集的，Emacs 会把这些文件提交为单个修订。

在 VC 目录缓冲区中，你还可以用下列命令对当前 VC 文件集执行搜索和替换：

‘S’ 搜索文件集（‘vc-dir-search’）。

‘Q’ 对文件集执行正则表达式查询替换（‘vc-dir-query-replace-regexp’）。

‘M-s a C-s’ 对文件集执行增量搜索（‘vc-dir-isearch’）。

‘M-s a C-M-s’ 对文件集执行增量正则表达式搜索（‘vc-dir-isearch-regexp’）。

除了作用于多个文件之外，这些命令的行为与它们对应的单缓冲区版本很相似（参见“搜索与替换”）。

VC 目录缓冲区还定义了一些与分支相关的命令，它们以前缀 ‘b’ 开头：

‘b c’ 新建一个分支（‘vc-create-branch’）。参见“创建新分支”。

‘b L’ 提示输入分支名，并显示该分支的变更历史（‘vc-print-root-branch-log’）。

‘b s’ 切换到某个分支（‘vc-switch-branch’）。参见“在分支之间切换”。

‘d’ 删除已标记的文件；若没有标记，则删除当前文件（‘vc-dir-delete-file)’）。若这些文件已被注册，它们会在版本控制系统中被标记为已删除。

上述命令也可以通过菜单栏访问，或通过 ‘mouse-2’ 弹出的上下文菜单访问。此外，某些 VC 后端会利用菜单提供额外的后端专有命令。例如，Git 和 Bazaar 允许你操作“stashes”和“shelves”（它们是暂时搁置未提交改动、稍后再取回的一种方式）。

30.1.11 版本控制分支

版本控制的用途之一是支持多条相互独立的开发路线，它们称为“分支”（branch）。除其他用途外，分支可用于维护程序的稳定版本和开发版本，也可用于彼此隔离地开发互不相关的特性。

VC 对分支操作的支持目前相当有限。对于分布式版本控制系统，它提供命令，用另一个分支的内容“更新”某个分支，以及“合并”对两个不同分支所做的改动（参见“合并分支”）。对于集中式版本控制系统，它支持检出不同的分支，以及提交到新的或不同的分支。

30.1.11.1 在分支之间切换

各种版本控制系统实现分支的方式不同，这些差异无法被 VC 完全掩盖。

在某些分布式版本控制系统中（包括 Bazaar 和处于常规运行模式的 Mercurial），每个分支都有自己的工作目录树，因此切换分支只涉及切换目录。在 Git 上，分支通常“同处一地”（co-located）于同一目录中，切换分支要用 ‘git checkout’ 命令完成，它会把工作树的内容改为与你切换到的分支一致。Bazaar 也支持同处一地的分支，此时 ‘bzr switch’ 命令会在当前目录中切换分支。使用 Subversion 时，用 ‘svn switch’ 命令切换到另一个分支。使用 Mercurial 时，用命令 ‘hg update’ 切换到另一个分支。

在当前目录中切换到另一个分支的 VC 命令是 ‘C-x v b s BRANCH-NAME <RET>’（‘vc-switch-branch’）。

在集中式版本控制系统中，你也可以在一个最新状态的工作文件中输入 ‘C-u C-x v v’（参见“‘C-x v v’ 中的高级控制”），并输入另一个分支上某个修订的修订 ID，从而在分支之间切换。例如，在 CVS 中，“主干”（trunk，即主要开发线）上的修订通常形如 1.1、1.2、1.3、……的 ID，而从（比如）修订 1.2 创建的第一个分支的修订 ID 为 1.2.1.1、1.2.1.2、……，从修订 1.2 创建的第二个分支的修订 ID 为 1.2.2.1、1.2.2.2、……，依此类推。你也可以指定“分支 ID”，即省略最后一个组成部分的分支修订 ID（例如 1.2.1），以切换到该分支上的最新修订。

在基于锁定的系统中，切换到不同的分支还会对工作树解锁（写保护）。

一旦切换到某个分支，VC 命令就会作用于该分支，直到你切换离开；例如，你提交的任何 VC 文件集都会提交到那个特定的分支。

30.1.11.2 把改动拉入/推出分支

‘C-x v P’ 使用分布式版本控制系统时，用当前分支中已在本地提交的改动更新另一个仓库（也就是“推送”改动）。集中式版本控制系统没有这个概念。

- ‘C-x v +’ 在去中心化版本控制系统中，通过从另一个仓库“拉入”（pull in）变更来更新本地仓库的当前分支。在集中式版本控制系统中，从仓库更新当前 VC 文件集。

在去中心化版本控制系统中，命令 ‘C-x v P’（‘vc-push’）把当前分支上已在本地提交的变更推送到另一个位置，该位置通常称为“上游仓库”（upstream repository）。带前缀参数时，它会提示输入要运行的确切版本控制命令，从而让你指定把变更推送到哪里；默认情况下，Bazaar 用 ‘bzr push’，Git 用 ‘git push’，Mercurial 用 ‘hg push’。这些默认命令总是推送到由版本控制系统根据你的分支配置所确定的默认位置上的仓库。

推送之前，你可以用 ‘C-x v O’（‘vc-root-log-outgoing’）查看将要发送到上游的变更的日志缓冲区。参见“VC 变更日志”。

此命令目前仅受 Bazaar、Git 和 Mercurial 支持。“推送”这一概念在集中式版本控制系统中并不存在——在那类系统里，此操作是提交变更集的一部分——因此在集中式 VCS 上调用此命令会报错。在 Bazaar 的“绑定分支”（bound branch）中尝试该命令也会报错；在绑定分支里，提交变更集会自动把变更推送到本地分支所绑定的远程仓库。

在去中心化版本控制系统中，命令 ‘C-x v +’（‘vc-pull’）用上游仓库中所做的变更更新本地仓库的当前分支及其工作树。它通常用于更新远程分支的一份副本（又称“克隆”，clone）。若提供前缀参数，该命令会提示输入要使用的确切版本控制命令，从而让你指定从何处拉取变更；否则，它从版本控制系统根据你的分支配置所确定的默认位置的仓库拉取。

在去中心化版本控制系统中，‘C-x v +’目前仅受 Bazaar、Git 和 Mercurial 支持。在 Bazaar 下，对普通分支它调用 ‘bzr pull’（把主分支的变更拉入镜像分支），对绑定分支则调用 ‘bzr update’（从中央仓库拉取）。在 Git 下，它调用 ‘git pull’ 从远程仓库获取变更并把它合并到当前分支。在 Mercurial 下，它调用 ‘hg pull -u’ 从默认远程仓库获取变更集并更新工作目录。

拉取之前，你可以用 ‘C-x v I’（‘vc-root-log-incoming’）查看将要应用的变更的日志缓冲区。参见“VC 变更日志”。

在 CVS 这样的集中式版本控制系统中，‘C-x v +’从仓库更新当前 VC 文件集。

30.1.11.3 合并分支

- ‘C-x v m’在去中心化版本控制系统中，把另一个分支的变更合并到当前分支。在集中式版本控制系统中，把另一个分支的变更合并到当前 VC 文件集。

开发某个分支时，你有时需要把另一个分支中已经做出的变更“合并”（merge）进来。这并非轻而易举的操作，因为两个分支上可能已经做出了重叠且冲突的变更。

在去中心化版本控制系统中，用命令 ‘C-x v m’（‘vc-merge’）合并变更。在 Bazaar 下，它会提示输入要传给 ‘bzr merge’ 命令的确切参数，若有可能则给出一个合理的默认值。在 Git 下，它会提示输入要从中合并的分支名，并支持补全（基于当前仓库已知的分支名）。在 Mercurial 下，它会提示输入要传给 ‘hg merge’ 的参数。运行合并命令的输出显示在单独的缓冲区中。

在 CVS 这样的集中式版本控制系统中，‘C-x v m’会提示输入分支 ID，或一对修订版本 ID（参见“在分支之间切换”）；然后它找出该分支的变更，或你所指定的两个修订版本之间的变更，并把这些变更合并到当前 VC 文件集中。若在提示处直接输入 ‘<RET>’，Emacs 就只合并自你检出该文件以来同一分支上做出的变更。

刚执行完合并时，只有工作树被修改，你可以用 ‘C-x v D’ 及相关命令查看合并产生的变更（参见“查看并比较旧修订”）。如果两个分支含有重叠的变更，合并会产生“冲突”（conflict）；合并命令的输出中会出现警告，并且每个受影响的工作文件中都会插入“冲突标记”（conflict markers），把两组相互冲突的变更包围起来。随后你必须通过编辑冲突文件来解决冲突；默认情况下，Emacs 会把含有 VC 冲突的缓冲区置于特殊的 Smerge 模式，该模式提供用于解决合并冲突的特殊命令。解决冲突并保存了冲突已解决的文件之后，必须按常规方式提交这些文件，合并才会生效（参见“版本控制下的基本编辑”）。

30.1.11.4 创建新分支

在 CVS 这样的集中式版本控制系统中，Emacs 支持在提交操作中创建新分支。提交修改过的 VC 文件集时，输入 ‘C-u C-x v v’（带前缀参数的 ‘vc-next-action’；参见“‘C-x v v’ 中的高级控制”）。然后 Emacs 提示输入新修

订版本的修订版本 ID。你应当为从当前修订版本开始的分支指定合适的变分支 ID。例如，若当前修订版本是 2.5，则分支 ID 应为 2.5.1、2.5.2 等，具体取决于该处已有分支的数量。

此过程对 Git 或 Mercurial 这样的分布式版本控制系统不适用。对这些系统，你应改用命令 'vc-create-branch'（'C-x v b c BRANCH-NAME <RET>'）。

要在较旧的修订版本（即已不再是某个分支头部的修订版本）上创建新分支，先选中该修订版本（参见"在分支之间切换"）。接下来的过程取决于你使用的是锁定式还是基于合并的 VCS。

在锁定式 VCS 上，你需要用 'C-x v v' 锁定旧修订版本所在的分支。锁定旧修订版本时，系统会要求你确认确实要创建新分支——若回答否，则会给你机会改为锁定最新的修订版本。在基于合并的 VCS 上，你可以跳过这一步。

然后做出你的修改，再次输入 'C-x v v' 提交新修订版本。这样就从选中的修订版本开始创建了一个新分支。

分支创建之后，后续提交都会在该分支上产生新的修订版本。要离开该分支，必须用 'C-u C-x v v' 显式选择另一个修订版本。

30.1.11.5 在分支之间复制修订版本所做的变更

- 'M-x vc-cherry-pick' 把一个修订版本复制到当前工作树中检出的分支上。

有时把某个修订版本从一个分支复制到另一个分支很有用。这意味着创建一个新修订版本，其变更、日志消息和作者信息都与另一个分支上已有的某个修订版本相同。这通常称为把修订版本从一个分支"拣选"（cherry-pick）到另一个分支。

最常见的情况是把一个不会被合并的分支（参见"合并分支"）中的修订版本复制到你的当前分支。例如，你的项目可能有一个只接受错误修复的特性冻结分支。某人（可能就是你）在主开发分支上修复了一个错误。然后你可以把该修订版本拣选到特性冻结分支上，以便在那里也修复该错误。这称为该修订版本的"向后移植"（backport），或修复的向后移植。

你可以用命令 'M-x vc-cherry-pick' 拣选修订版本。它会提示输入要拣选的修订版本，然后弹出一个缓冲区，供你编辑新修订版本的日志消息。通常 VC 后端会生成一条日志消息，其中包含对要复制的修订版本的引用，以便这次复制能够被追溯。如果你愿意，可以在输入 'C-c C-c' 结束拣选之前删掉这个引用。

或者，你也可以带前缀参数调用该命令，即 'C-u M-x vc-cherry-pick'。在这种情况下，源修订版本的日志消息被原样使用，拣选立即完成，不会弹出用于编辑日志消息的缓冲区。

访问此功能的另一种方式是 'log-view-cherry-pick' 命令，它绑定到 Log View 模式缓冲区中的 'C'（参见"VC 变更日志"）。与直接使用 'M-x vc-cherry-pick' 相比，这样做更容易确保你拣选的正是你想要的修订版本。

30.1.12 VC 的其他命令与特性

本节介绍 VC 中不常使用的特性。

30.1.12.1 变更日志与 VC

如果你为某个程序使用 RCS 或 CVS，并且该程序带有一个 'ChangeLog' 文件（参见"变更日志"），你就可以根据以往提交的版本控制日志条目生成变更日志条目。

注意，这只对 RCS 或 CVS 有效。在现代基于变更集的版本控制系统中，这个过程尤其不正确，因为在那类系统里对 'ChangeLog' 文件的修改通常作为变更集的一部分提交。在这种情况下，你应当先写好变更日志条目，然后在提交时把它们拉入 '*vc-log*' 缓冲区（参见"日志条目缓冲区的特性"）。

- 'C-x v a' 访问当前目录的 'ChangeLog' 文件，并为该目录中已注册的文件，为自最近一条变更日志条目以来所提交的版本创建新条目（'vc-update-change-log'）。
- 'C-u C-x v a' 同上，但只查找当前缓冲区文件的条目。

例如，假设 'ChangeLog' 的第一行日期是 1999-04-10，且此后唯一一次检入是 Nathaniel Bowditch 在 1999-05-22 对 'rcs2log' 的检入，其日志条目为 'Ignore log messages that start with '#'.'。那么 'C-x v a' 会插入如下 'ChangeLog' 条目：

```
1999-05-22  Nathaniel Bowditch  <nat@apn.org>

* rcs2log: Ignore log messages that start with '#'.
```

如果版本控制日志条目指定了函数名（写在行首的括号中），该信息就会反映在 'ChangeLog' 条目里。例如，若 'vc.el' 的日志条目是 '(vc-do-command): Check call-process status.'，则 'ChangeLog' 条目为：

```
1999-05-06  Nathaniel Bowditch  <nat@apn.org>

* vc.el (vc-do-command): Check call-process status.
```

当 'C-x v a' 一次添加若干变更日志条目时，如果相关的日志条目都是由同一作者在几乎同一时间检入的，它就把它们归为一组。如果若干此类文件的日志条目文本完全相同，它就把它们合并为单独一条条目。

30.1.12.2 删除与重命名受版本控制的文件

- 'C-x v x' 提示输入文件名，从工作树中删除该文件，并安排该删除以便随后提交。
- 'C-x v R' 提示输入两个文件名 OLD 和 NEW，在工作树中重命名它们，并安排该重命名以便随后提交。若 OLD 文件受 VC 管理，则默认取当前缓冲区的文件名。

如果你想删除受版本控制的文件，输入 'C-x v x'（'vc-delete-file'）。它会提示输入文件名，并通过版本控制系统删除该文件。文件从工作树中被移除，在 VC 目录缓冲区（参见"VC 目录模式"）中显示为 'removed' 状态。当你提交时，该删除在仓库中生效。

要重命名受版本控制的文件，输入 'C-x v R'（'vc-rename-file'）。它提示输入两个参数：要重命名的文件名和新名称；然后它通过版本控制系统执行重命名。重命名立即在工作树中生效，并在你提交被重命名的文件时在仓库中生效。

在内置支持重命名的现代版本控制系统中，被重命名的文件保留原文件的完整变更历史。在 CVS 及更旧的版本控制系统中，'vc-rename-file' 命令实际上是先用新名称创建旧文件的副本，将其注册，再删除旧文件。在这种情况下，变更历史不会被保留。

30.1.12.3 修订标签

大多数版本控制系统允许你给受版本控制的树的某个特定版本加上“修订标签”（revision tag）。在现代基于变更集的版本控制系统中，修订标签就是一个特定修订版本的符号名。在 CVS 这样的较旧的基于文件的系统中，每个标签都会加到整个受版本控制的文件集合上，使它们可以作为一个单元来处理。修订标签常用于标识分发给用户的发行版本。

标签有两个基本命令：一个用给定的名称创建标签，另一个取回指定的标签。

- ‘C-x v s NAME <RET>’ 把当前目录中或其下每个已注册文件的工作修订版本定义为名为 NAME 的标签（‘vc-create-tag’）。
- ‘C-x v r NAME <RET>’ 对当前目录层级及其下所有已注册文件，取回加有标签 NAME 的修订版本。如果 NAME 是分支名且你的 VCS 区分分支与标签，此命令会切换到该分支。（‘vc-retrieve-tag’）。

如果当前目录中或其下有文件被锁定，此命令会报错而不做任何改动；这是为了避免覆盖正在进行的工作。

你可以把标签名或分支名作为参数传给 ‘C-x v =’ 或 ‘C-x v ~’（参见“查看并比较旧修订”）。因此，你可以用它把某个加标签的版本与当前文件比较，或把两个加标签的版本相互比较。

在 SCCS 上，VC 自己实现标签；这些标签只有通过 VC 才能看到。大多数后来的系统（包括 CVS、Subversion、bzr、git 和 hg）都自带标签机制，VC 在可用时使用它；即使你绕过 VC，这些标签也是可见的。

在基于文件的版本控制系统中，重命名已注册文件时，你需要同时重命名它的主文件（master）；命令 ‘C-x v R’（‘vc-rename-file’）会自动完成这件事（参见“删除与重命名受版本控制的文件”）。如果你使用 SCCS，还必须更新标签的记录，用新名称来指代该文件（‘C-x v R’ 也会做这件事）。如果一个旧标签所指的主文件已不在记录的名称下存在，该标签就失效了；VC 再也无法取回它。要说明如何手工更新标签，需要对 RCS 和 SCCS 做足够的解释，这超出了本手册的范围。使用 ‘C-x v R’ 可使标签保持有效、仍可被取回，但这并不能解决所有问题。例如，你程序中的某些文件可能按名称引用其他文件。至少 makefile 很可能提到了你重命名的那个文件。如果你取回一个旧标签，被重命名的文件会以它的新名称被取回，而那不是 makefile 所期望的名称。所以取回后的程序实际上无法正常工作。

30.1.12.4 合并基点

- ‘C-x v M D’ 报告某个分支自与另一分支分叉以来所做变更的差异（‘vc-diff-mergebase’）。
- ‘C-x v M L’ 显示某个分支自与另一分支分叉以来的修订版本的日志消息（‘vc-log-mergebase’）。

两个分支的“合并基点”（merge base）是同时存在于两个分支上的最新修订版本。如果两个分支从未互相合并过（参见“合并分支”），那么合并基点就是较新的分支从较旧分支创建出来时、较旧分支所处的修订版本（参见“创建新分支”）。如果其中一个分支曾被合并到另一个分支，那么合并基点就是最近的一次合并点。

本节介绍的命令目前只为去中心化版本控制系统实现（参见“去中心化与集中式仓库”）。

合并基点对于在分支之间做某些比较很有用，Emacs 提供两个命令来做这种比较。‘C-x v M D’（‘vc-diff-mergebase’）和 ‘C-x v M L’（‘vc-log-mergebase’）都会提示输入两个分支，找出它们的合并基点，然后把该合并基点与两个分支中的第二个相比较。这两个命令分别报告差异和显示变更历史。

这些命令的典型用例是：其中一个分支最初是从另一个分支创建的，而你或协作者至少一次把其中一个分支合并到了另一个分支。这样你就可以用这些命令查看一个分支上哪些变更尚未合并到另一个分支。

把含有你感兴趣的变更的那个分支称为“源分支”（source branch），把这些变更尚未合并进去的那个分支称为“目标分支”（target branch）。在提示输入“较旧修订版本”时指定目标分支，在提示输入“较新修订版本”时指定源分支。(1) 然后 ‘C-x v M D’ 会向你预览：如果把源分支合并到目标分支，目标分支上会发生什么变化；而 ‘C-x v M L’ 会显示源分支上尚未合并到目标分支的变更日志。

> 注：(1) 合并基点的概念可以从分支推广到任意两个修订版本。两个修订版本的合并基点是同时出现在这两个修订版本的修订历史中的最新修订版本。‘C-x v M D’ 和 ‘C-x v M L’ 接受任意两个修订版本，而不仅限于分支。比较两个分支就等同于比较两个分支末端的修订版本。（事实上这个概念可以推广到任意多个修订版本，但 Emacs 的合并基点命令只处理两个，因此我们仅限于此。）

30.1.12.5 查看所有未集成变更的命令

- ‘C-x v T =’ 显示 VC 文件集自本分支与其上游对应分支的合并基点以来的变更差异（‘vc-diff-unintegrated’）。
- ‘C-x v T D’ 显示自本分支与其上游对应分支的合并基点以来的所有变更的差异（‘vc-root-diff-unintegrated’）。
- ‘C-x v T l’ 显示 VC 文件集自本分支与其上游对应分支的合并基点以来的变更的日志消息（‘vc-log-unintegrated’）。
- ‘C-x v T L’ 显示自本分支与其上游对应分支的合并基点以来的所有变更的日志消息（‘vc-root-log-unintegrated’）。
- ‘C-x v T R =’ 显示 VC 文件集自本主题分支与其上游对应分支的合并基点以来的远程变更的差异（‘vc-diff-remote-unintegrated’）。
- ‘C-x v T R D’ 显示自本主题分支与其上游对应分支的合并基点以来的所有远程变更的差异（‘vc-root-diff-remote-unintegrated’）。
- ‘C-x v T R l’ 显示 VC 文件集自本主题分支与其上游对应分支的合并基点以来的远程变更的日志消息（‘vc-log-remote-unintegrated’）。
- ‘C-x v T R L’ 显示自本主题分支与其上游对应分支的合并基点以来的所有远程变更的日志消息（‘vc-root-log-remote-unintegrated’）。

对于去中心化版本控制系统（参见“去中心化与集中式仓库”），这些命令提供了 ‘C-x v M L’ 和 ‘C-x v M D’ 的专门版本（参见“合并基点”），它们还会考虑上游仓库的状态。无论你是在单个分支上工作，还是在单独的分支上开发特性（参见“版本控制分支”），这些命令都很有用。这两种情形在概念上是不同的，因此我们分别介绍它们。

首先考虑在单个分支上工作的情况。“未集成变更”（unintegrated changes）指你尚未推送到上游的变更。这既包括尚未推送的提交，也包括工作树中尚未提交的变更。在很多情况下，这些变更尚未推送的原因是它们还没完成：到目前为止提交的变更单独来看并不成立。

输入 ‘C-x v T D’（‘vc-root-diff-unintegrated’）可显示所有这些变更（已提交的和未提交的）的摘要。这个摘要以差异的形式呈现：它展示提交并推送（参见“把改动拉入/推出分支”）所有这些变更会对上游仓库做什么。你可以改用 ‘C-x v T =’（‘vc-diff-unintegrated’）把变更的显示限制在当前 VC 文件集上。（‘C-x v T D’ 与 ‘C-x v T =’ 的区别，就像 ‘C-x v D’ 与 ‘C-x v =’ 的区别（参见“查看并比较旧修订”）。）⑴

输入 ‘C-x v T L’（‘vc-root-log-unintegrated’）可以修订日志的形式显示同一批变更的摘要；它不包含未提交的变更。你可以改用 ‘C-x v T l’（‘vc-log-unintegrated’）把变更的显示限制在当前 VC 文件集上。

其次考虑在单独的分支上开发特性的情况。把这个分支称为“主题分支”（topic branch），⑵ 并把主题分支最初被创建时所依据的分支称为“主干”（trunk）或“开发主干”（development trunk）。

在这种情况下，“未集成变更”是一个比主题分支上未推送和未提交的变更更具体的概念。直到变更被合并进主干并推送出去，你才算完成与协作者的共享工作。因此，在这个例子中，未集成变更就是尚未包含在上游仓库开发主干中的那些变更。这意味着主题分支上尚未合并进主干的已提交变更，加上未提交的变更。

当当前分支是主题分支时，输入 ‘C-x v T D’ 后，Emacs 会显示相对于当前分支将要合并进去的那个主干而言尚未处理的所有变更的摘要。这个摘要以差异的形式呈现：它展示提交并推送所有变更、*并且*随后合并主题分支会对主干做什么。同上，你可以改用 ‘C-x v T =’ 把变更的显示限制在当前 VC 文件集上。‘C-x v T L’ 和 ‘C-x v T l’ 显示相应的修订日志，并如上所述排除未提交的变更。

在主题分支上，有时也有必要查看同样的信息，但针对远程仓库中该主题分支的版本。这是主题分支上已推送但尚未合并的未完成工作。它对协作者可见，但在其他方面仍未集成。我们称这样的工作为“远程未集成变更”（remote unintegrated changes）。前面讨论的四个命令各有一个对应命令，只是作用于远程仓库中当前主题分支的版本：‘C-x v R T =’（‘vc-diff-remote-unintegrated’）、‘C-x v R T D’（‘vc-root-diff-remote-unintegrated’）、‘C-x v R T l’（‘vc-log-remote-unintegrated’）和 ‘C-x v R T L’（‘vc-root-log-remote-unintegrated’）。

所有这些功能都依赖于 Emacs 正确判断当前分支是主干还是主题分支，并在属于后者时正确确定该主题分支最终将被合并到哪个分支。如果自动检测没有得到正确结果，有若干选项可以调整和覆盖它。

变量 ‘vc-trunk-branch-regexps’ 和 ‘vc-topic-branch-regexps’ 分别包含一组正则表达式，用于匹配那些应始终视为主干分支和主题分支的分支名。你也可以给 ‘C-x v T ...’ 指定前缀参数。下面是使用这些控制手段的简要说明：

1. 如果问题在于 Emacs 把你的主题分支当成了主干分支，你可以把它的名字，或匹配其名字的正则表达式，加入 ‘vc-topic-branch-regexps’ 变量（参见“正则表达式的语法”）。有几种特殊的取值可以简化常见用法：

- 若某个元素不含正则表达式中具有特殊含义的字符，则该正则表达式会隐式地在两端锚定，即它只匹配名字恰好等于该名字的分支。
- 若 ‘vc-topic-branch-regexps’ 的第一个元素是符号 ‘not’，则 ‘vc-topic-branch-regexps’ 的含义反转：凡名字不匹配 ‘vc-topic-branch-regexps’ 任何元素的分支，Emacs 都视作主题分支。
- 若 ‘vc-topic-branch-regexps’ 变量不是正则表达式列表，而是取特殊值 ‘t’，则凡 ‘vc-trunk-branch-regexps’ 变量未明确认定为主干的分支，Emacs 都视作主题分支。
- 关于如何为单个 VC 仓库指定 ‘vc-topic-branch-regexps’ 与 ‘vc-trunk-branch-regexps’ 的值，参见“按目录的局部变量”。

1. 如果问题在于 Emacs 把你的主干分支当成了主题分支，你可以把它的名字，或匹配其名字的正则表达式，加入 'vc-trunk-branch-regexps' 变量。其工作方式与 'vc-topic-branch-regexps' 完全相同，特殊取值也如我们刚才所述。例如，若 'vc-trunk-branch-regexps' 的值为 't'，则凡 'vc-topic-branch-regexps' 变量未认定为主题分支的分支，Emacs 都视作主干。

1. 提供一个双重前缀参数，即 'C-u C-u C-x v T ...'，Emacs 无论如何都会把当前分支当作主干。当你只想获得所有待推送改动（参见"VC 变更日志"）加上未提交改动的差异时，这很有用。

1. 最后，你可以通过提供一个单重前缀参数，即 'C-u C-x v T ...'，完全手动控制。Emacs 会提示你输入"待推送基点"（outgoing base），即这些改动在不再待推送之后所要发往的上游位置。

要把当前分支当作主干，请指定当前分支上游版本的引用，即你和协作者推送已完成工作的位置。要把当前分支当作主题分支，请指定主干上游版本的引用，该主题分支稍后将合并到该主干中。

究竟如何指定分支上游版本的引用，取决于所用的版本控制系统。例如，在 Git 中，要引用分支 FOO 的上游版本，你应提供 'origin/FOO'。因此，如果 FOO 是当前分支，那么要把它当作主干，就输入待推送基点 'origin/FOO'；而要把 FOO 当作稍后将合并进名为 BAR 的主干的主题分支，就输入待推送基点 'origin/BAR'。

如果有默认选项，那么它就是 Emacs 认为你需要输入以便把当前分支当作主题分支的内容。如果没有默认值，那么在提示符处不输入任何内容就表示把当前分支当作主干。

> 注：⑴ 另一点可作比较的是，这些命令类似于 'C-x v O ='（'vc-fileset-diff-outgoing'）和 'C-x v O D'（'vc-root-diff-outgoing'），区别在于它们报告的差异中包含未提交的改动。与其他那些命令一样，你可以用前缀参数来指定特定的上游位置。

> 注：⑵ 我们所说的主题分支，是指任何生命周期较短、用于某项工作且该工作稍后将被合并进某个生命周期较长的分支的分支。主题分支有时也称为"功能分支"（feature branch）。还常见的情形是，"功能分支"一词被专门用于某一类主题分支，即另一个分支或其他多个分支会反复合并进的那类分支。

30.1.12.6 单个仓库的多个工作树

有些 VCS 支持在同一个后备仓库（backing repository）或版本存储之上拥有多个工作树。这意味着你可以同时在多个不同的工作树中检出不同的版本或分支（参见"版本控制分支"），而所有版本历史、分支、标签和其他元数据都是共享的。

假设你的项目有一个主干，你在其中开发软件的新版本 3，还有一个稳定发布分支，你从中发布版本 2 的小版本。有人给你发来一个补丁，修复版本 2 中存在的一个缺陷。你的主工作树中检出的是版本 3，而你正处在一次大重构的中途，不想被打扰。于是你输入 'C-x v w c'（见下文），创建一个新的工作树，并按提示在那里检出版本 2 分支。

你用 'C-x v w a'（见下文）把补丁应用到那个工作树，构建并测试。满意之后，你在另一个工作树中使用 'C-x v P'（参见"把改动拉入/推出分支"）。在测试补丁的过程中，你意识到该缺陷在软件的版本 3 中也存在。于是你切回第一个工作树，用 'C-x v m'（参见"合并分支"）合并你在另一个工作树中检出的那个分支。现在你的主干版本已经合并了版本 2 的全部修复，但你还没有推送，因为你在重构中。稍后你会用 'C-x v P'。

当存在其他工作树时，像 ‘C-x v v’（参见“版本控制下的基本编辑”）和 ‘C-x v d’（参见“VC 目录模式”）这样的普通 VC 命令的行为并无不同，只是它们创建的提交、分支及其他 VC 产物会从所有工作树中可见。换个说法：在任何工作树中执行、会在 VCS 后备存储中创建新产物的操作，都会从其他工作树可见；而只影响工作文件的操作则不会。因此，如果你把一个补丁应用到某个工作树中的若干工作文件，那只影响该工作树。但如果你提交应用该补丁所做的改动，那就在后备存储中创建了一个新版本，于是该操作会以“其他工作树看到的仓库历史中多了一次提交”的方式影响其他工作树。

以下特殊命令让你在不同工作树之间切换并修改它们。不在 VC 工作树内使用这些命令是错误的；也就是说，你必须在一个访问受 VCS 控制文件的缓冲区中，或在一个 ‘default-directory’（参见“文件名”）位于某个 VC 工作树内的缓冲区中使用它们。

- ‘C-x v w c’ 添加一个新的工作树。
- ‘C-x v w w’ 在另一个工作树中访问此文件或目录。
- ‘C-x v w k’ 删除在其他工作树中访问此文件的缓冲区。
- ‘C-x v w s’ 调用 ‘C-x p p’（‘project-switch-project’），但仅限于其他工作树。
- ‘C-x v w a’ 把文件集的改动复制或移动到另一个工作树。
- ‘C-x v w A’ 把全部改动复制或移动到另一个工作树。
- ‘C-x v w x’ 删除你不再需要的工作树。
- ‘C-x v w R’ 把工作树迁移到另一个文件名。

你可以用命令 ‘C-x v w c’（‘vc-add-working-tree’）创建一个新的工作树，从而开始使用多个工作树。它会提示你指定目标目录，该目录标识了此工作树，并将存放新的工作文件集合。

不同 VCS 对于其他工作树中允许检出什么、必须检出什么有不同的规则，因此视所用 VCS 的不同，可能会有额外的提示。例如，Git 要求每个分支在同一时间只能在一个工作树中检出，所以在使用 Git 时，Emacs 还会提示你输入要在新工作树中检出的分支名。

一旦你的仓库有了其他工作树，你就可以用命令 ‘C-x v w w’（‘vc-switch-working-tree’）在它们之间切换。它会尝试在另一个工作树中找到当前缓冲区的文件或目录的对应物。通常，不同工作树下的工作文件集合在文件内容上的差异，大于在哪些文件存在、哪些不存在上的差异。换句话说，当前缓冲区所访问的文件或目录很可能在其他工作树中也存在，而此命令让你切换到该文件的那些版本。

‘C-x v w w’ 在 Diff 模式（参见“Diff 模式”）和 Log View 模式（参见“VC 变更日志”）中也可用。该命令不会切换到另一个缓冲区，而是把缓冲区的默认目录改为另一个工作树下的对应目录。这在由 ‘C-x v L’（参见“VC 变更日志”）之类命令生成的 Log View 模式缓冲区中特别有用。在用 ‘C-x v w w’ 改变了 Log View 缓冲区的默认目录之后，你可以把光标点移到感兴趣的版本上并输入 ‘=’（‘log-view-diff’），打开一个显示该版本改动的 Diff 模式缓冲区。然后你就可以用 ‘C-c C-a’（‘diff-apply-hunk’）之类的标准 Diff 模式命令把补丁块应用到另一个工作树。

在访问文件的缓冲区中使用 ‘C-x v w w’ 之后，你会有多个访问同名文件的缓冲区（参见“使缓冲区名称唯一”）。你可以输入 ‘C-x v w k’（‘vc-kill-other-working-tree-buffers’）杀掉除一个之外的所有这些缓冲区。此命令会杀掉在其他工作树中访问当前缓冲区文件的各个版本的缓冲区，并保留当前缓冲区。它在不访问文件的缓冲区中不起作用。

另一种在工作树之间切换的方式是 'C-x v w s'（'vc-working-tree-switch-project'）。它会提示你选择一个工作树，然后显示一个可对其执行操作的命令菜单。这实际上就是 'C-x p p'（'project-switch-project'）（参见"切换项目"），只是把项目的选择范围限制在其他工作树。'C-x v w w' 与 'C-x v w s' 的主要区别在于：前者在另一个工作树中寻找当前缓冲区的对应物，而后者把另一个工作树当作一个整体、独立的项目来考虑。

命令 'C-x v w a'（'vc-apply-to-other-working-tree'）提示你选择另一个工作树，然后把改动从当前工作树复制到那个工作树。带前缀参数时，它移动改动而不只是复制。通常该命令作用于当前 VC 文件集的本地（未提交）改动。在 Diff 模式（参见"Diff 模式"）下的缓冲区中调用时，它作用于该缓冲区内容所指定的改动。若其中任何改动无法应用到目标工作树，该命令就停止，什么也不做。

'C-x v w a' 可用于把改动复制到一个临时工作树以便测试。它也可用于把修复复制回主工作树以便检入。例如，你可能在一个临时工作树中反复琢磨某个缺陷并把它修复。然后你会想把该修复复制或移动回主工作树，以便检入并推送。

命令 'C-x v w A' 作用类似，区别在于它总是把整个工作树的全部本地改动复制或移动，而不只是当前 VC 文件集的改动或 Diff 模式缓冲区内容所表示的改动。带两个前缀参数时，此命令会显示待复制改动的预览，让你用 'C-c C-a' 和 'C-c <RET> a' 之类的标准 Diff 模式命令来应用它们（参见"Diff 模式"）。（'C-u C-u C-x v w A' 大致相当于先输入 'C-x v D'，再输入 'C-x v w w'。）

命令 'C-x v w x'（'vc-delete-working-tree'）和 'C-x v w R'（'vc-move-working-tree'）用于对其他工作树执行维护任务，让你删除、移动和重命名它们。删除其他工作树尤其有用，因为多个工作树的一个常见用途是创建仓库的一次性副本，以便快速测试改动，同时不干扰你在主工作树中可能正在进行的工作。

30.1.12.7 插入版本控制头

在 Subversion、CVS、RCS 和 SCCS 上，你可以把称为"版本头"（version header）的特定特殊字符串放入工作文件。当文件被提交时，版本控制系统会自动把版本号、进行提交的用户名以及其他相关信息放入版本头。

VC 通常不使用版本头中的信息。作为例外，使用 RCS 时，如果有版本头，Emacs 会用它来确定文件版本，因为它常常比 RCS 主文件更可靠。要禁止这样使用版本头，请把变量 'vc-consult-headers' 改为 'nil'。此后 VC 总是使用文件权限（如果它应当信任这些权限），否则检查主文件。

要把合适的头字符串插入当前缓冲区，请用命令 'M-x vc-insert-headers'。此命令只在 Subversion、CVS、RCS 和 SCCS 上有效。变量 'vc-BACKEND-header' 含有要插入版本头的关键字列表；例如，CVS 使用 'vc-cvs-header'，其默认值是 '("\$Id\$")'（参见《《cvs》手册》的"Keyword substitution"）。（多余的反斜杠可防止该字符串常量在定义它的 Emacs Lisp 文件本身受版本控制时被当作头来解释。）'vc-insert-headers' 命令会把列表中的每个关键字插在光标点处的新行上，两侧以制表符包围，必要时还放在注释定界符之内。

变量 'vc-static-header-alist' 指定根据缓冲区名字进一步添加的字符串。它的值应是由形如 '(REGEXP . FORMAT)' 的元素组成的列表。每当 REGEXP 匹配缓冲区名字时，FORMAT 也会作为版本头的一部分被插入。FORMAT 中的 '%s' 会被替换为该文件的版本控制类型。

30.1.12.8 编辑 VC 命令

你可以用前缀命令 'C-x v !'（'vc-edit-next-command'）来编辑 VC 即将运行的 shell 命令行。这主要是为了能向 VCS 命令添加可选的命令行参数，而不必给 VC 命令集及其与后端的接口带来不必要的复杂性。

例如，Git 能生成多个分支的日志，但 ‘C-x v b L’（‘vc-print-root-branch-log’）只提示输入一个分支名。要获得多个分支的日志，你可以输入 ‘C-x v ! C-x v b L’，然后把其他分支名追加到 VC 即将运行的 ‘git log’ 命令末尾。

30.1.12.9 准备补丁

在项目上协作时，通过电子邮件发送补丁以分享改动很常见。你可以用 VC 的 ‘vc-prepare-patch’ 命令来做这件事。它会提示你输入要分享的版本，以及要使用哪些目标电子邮件地址。版本之间用 CRM-SEPARATOR 的值分隔，默认是逗号。随后该命令会用你的 MUA（邮件用户代理）准备好这些版本，供你审阅并发送。

在带标记版本的 Log View 缓冲区中以交互方式调用时，将使用那些已标记的版本。

根据用户选项 ‘vc-prepare-patches-separately’ 的值，‘vc-prepare-patch’ 会生成一条或多条消息。默认值 ‘t’ 表示为每个版本依次准备并显示一条消息。值 ‘nil’ 表示生成一条消息，把所有补丁都附在正文中。

如果你预期会经常贡献补丁，可以把用户选项 ‘vc-default-patch-addressee’ 设为你希望使用的地址。调用 ‘vc-prepare-patch’ 时，它将作为默认值使用。项目维护者可以考虑把它设为目录局部变量（参见“按目录的局部变量”）。

30.1.12.10 自动还原访问受跟踪文件的缓冲区

当 Emacs 执行它知道可能改变受跟踪文件内容的 VCS 操作时，它会还原访问这些文件的缓冲区（参见“还原缓冲区”）。它以能感知 VCS 的方式进行还原，即使 VCS 操作导致 VCS 关键字被展开（参见“插入版本控制头”），也能保留光标点和标记的位置。

此特性的一个重要局限是：当你在 Emacs 之外执行其他 VCS 操作时，例如在 shell 提示符处或通过脚本执行，Emacs 不会知道要还原缓冲区。如果你经常这样做，并且你使用的 VCS 没有关键字展开（所有现代 VCS 在没有特殊配置时都是如此），那你也许更愿意改用 ‘vc-auto-revert-mode’，办法是把该变量定制为非 ‘nil’ 值。

此模式与 ‘global-auto-revert-mode’（参见“自动还原：自动保持缓冲区最新”）完全相同，只是仅限于访问受 VCS 跟踪文件的那些文件。它确保当 VCS 操作改变了缓冲区内容时 Emacs 总会还原缓冲区，无论这些操作是否由 Emacs 发起。

关于访问受跟踪文件的缓冲区中的自动还原模式的细节（‘vc-auto-revert-mode’ 所启用的正是该模式），参见“版本控制与模式行”。

30.1.12.11 回退分支

- ‘M-x vc-delete-revisions-from-end’ 从当前分支末尾删除版本。
- ‘M-x vc-uncommit-revisions-from-end’ 从当前分支末尾删除版本，而不改动工作树。

对于去中心化的版本控制系统（参见“去中心化与集中式仓库”），这些命令提供了把当前分支退回到较早版本的方法。‘vc-delete-revisions-from-end’ 提示输入一个版本，然后从分支末尾删除所有版本，直到（但不包括）指定的那个版本。我们说该分支被“回退”（rewound）到了指定版本。

此命令会从工作树中移除那些版本所做的改动。因此，如果存在任何未提交的改动，必须先还原它们（参见“撤销版本控制操作”）。必要时此命令会提示你这么做。如果你提供前缀参数，Emacs 将不经提示就删除未提交的改动。

“取消提交”（uncommit）一个版本，是指把它从版本历史中移除，但不把它的改动从工作树中移除。这就好像你已经做了那些改动但还没有检入它们。命令‘vc-uncommit-revisions-from-end’提示输入一个版本，然后取消提交从分支末尾直到（但不包括）指定版本的所有版本。分支被回退到指定版本，但改动留在工作树中。

回退当前分支时，如果从版本历史中删除的所有版本都在你已拉取或推送的版本之中，那么这些操作不会永久删除任何东西：简单执行一次‘C-x v +’（参见“把改动拉入/推出分支”）就能把那些版本找回来。另一方面，如果分支末尾有尚未推送的新版本，那么这些命令会永久删除它们。Emacs 会尝试检测这种情况，并询问你是否确定要删除它们。

访问此功能的另一种方式是‘log-view-uncommit-revisions-from-end’和‘log-view-delete-revisions-from-end’命令，它们在 Log View 模式缓冲区中分别绑定到‘x’和‘X’（参见“VC 变更日志”）。与直接使用这里描述的命令相比，Log View 模式的命令更容易让你确认自己回退到的正是想要的那个版本。

30.1.13 定制 VC

变量‘vc-handled-backends’决定 VC 应处理哪些版本控制系统。默认值是‘(RCS CVS SVN SCCS SRC Bzr Git Hg)’，因此它包含目前支持的所有版本系统。如果你希望 VC 忽略其中某个或某些系统，就把它的名字从列表中排除。要完全禁用 VC，请把此变量设为‘nil’。

列表中系统的顺序很重要：当你访问一个登记在多个系统中的文件时，VC 默认使用在‘vc-handled-backends’中靠前的那个系统。首次登记文件时（参见“为版本控制注册文件”），顺序也很重要。

30.1.13.1 通用选项

Emacs 通常不为受版本控制维护的源文件保存备份文件。如果你希望即使对使用版本控制的文件也创建备份文件，请把变量‘vc-make-backup-files’设为非‘nil’值。

如果你不知道底层文件受版本控制，通过符号链接编辑受版本控制的文件可能会导致意外结果。变量‘vc-follow-symlinks’控制当你试图访问指向受版本控制文件的符号链接时 Emacs 怎么做。若值为‘ask’（默认值），Emacs 会请求确认。若为‘nil’，Emacs 只显示一条警告消息。若为‘t’，Emacs 会自动跟随链接并改为访问真正的文件。

若‘vc-suppress-confirm’非‘nil’，则‘C-x v v’和‘C-x v i’可以不经询问就保存当前缓冲区，‘C-x v u’也无需请求确认即可操作。

VC 模式的工作很大一部分是通过运行相应版本控制系统的 shell 命令来完成的。若‘vc-command-messages’的值为‘log’，VC 会把消息记录到‘*Messages*’缓冲区，以说明它运行了哪些 shell 命令，并在命令完成时记录附加消息。若该变量取任何其他非‘nil’值，Emacs 既记录又显示这些消息。

VC 模式会异步运行某些 shell 命令，让你可以在命令完成期间继续用 Emacs 做别的事。这主要用于访问网络的命令，例如为分布式 VCS 拉取和推送（参见“把改动拉入/推出分支”）。命令启动时，VC 会显示一个缓冲区，命令的输出将写入其中。若‘vc-display-failed-async-commands’非‘nil’，VC 还会确保当命令运行结束但失败时显示此缓冲区。这意味着你可以放心关掉显示命令缓冲区的窗口，并知道在命令运行失败的情况下它会再次弹出。

通常检入操作是同步完成的；也就是说，Emacs 会等到检入完成后才做别的事。对于检入操作很慢的仓库，这可能不方便，例如检入改动到超大文件的 Git 仓库，或文件数量极多的 Mercurial 仓库。

对于那些支持的后端，把‘vc-async-checkin’设为非‘nil’可改为异步执行检入操作。在检入操作很慢的仓库中，把它作为目录局部变量特别有用（参见《Emacs Lisp 参考手册》的“Directory Local Variables”）。

在一个异步检入操作进行期间，如果你用‘C-x C-s’保存一个访问当前 VC 树内任何文件的缓冲区，那么该操作会退回到同步检入，Emacs 会等到它完成后再保存缓冲区。这是为了避免在究竟哪些改动会被检入这一点上出现不确定性。

30.1.13.2 RCS 与 SCCS 的选项

默认情况下，RCS 使用锁定来协调多个用户的活动，但有一种称为“非严格锁定”（non-strict locking）的模式，在其中你不必先锁定文件就可以检入改动。请用‘rcs -U’为特定文件切换到非严格锁定，详见‘rcs’手册页。

在推断 RCS 文件的版本控制状态时，VC 首先会在文件中查找 RCS 版本头字符串（参见“插入版本控制头”）。如果没有版本头字符串，VC 通常会查看工作文件的文件权限；这种方式很快。但有些情况下文件权限并不可信。此时就必须查询主文件，而这相当昂贵。此外，主文件只能告诉你文件上*是否*存在锁，却不能告诉你你的工作文件是否真的包含那个被锁定的版本。

你可以通过把‘vc-consult-headers’设为‘nil’来告诉 VC 不要用版本头来确定文件状态。此后 VC 总是使用文件权限（如果它应当信任这些权限），否则就检查主文件。

VC 确定 SCCS 下文件的版本控制状态的方式与 RCS 大体相同。不过它不考虑 SCCS 版本头。因此，变量‘vc-consult-headers’对 SCCS 的使用没有影响。

30.1.13.3 CVS 专有选项

你可以用变量‘vc-cvs-global-switches’指定要传递给所有 CVS 操作的附加命令行选项。这些开关会紧接在‘cvs’命令之后、要调用的操作名之前插入。

在远程机器上使用 CVS 仓库时，VC 可以设法把网络交互保持在最少。这由变量‘vc-cvs-stay-local’控制。若‘vc-cvs-stay-local’为‘only-file’（默认值），VC 仅依据本地 CVS 子目录中的条目以及先前 CVS 命令返回的信息来确定每个文件的版本控制状态。因此，如果你修改了某个文件，而别人又提交了别的改动，那么直到你尝试提交时才会被告知冲突。

如果你把‘vc-cvs-stay-local’改为‘nil’，VC 会在决定‘vc-next-action’（‘C-x v v’）中做什么之前先查询远程仓库，就像它对本地仓库所做的那样。

你也可以把‘vc-cvs-stay-local’设为一个与仓库主机名匹配的正则表达式；此时 VC 只对来自匹配该模式的主机的仓库保持本地。

使用远程仓库时，Emacs 通常会为每个被编辑文件的原始版本制作“自动版本备份”。这些本地备份在你首次保存对文件的修改时制作，并在你把修改提交到仓库之后删除。（注意这些备份与普通的 Emacs 备份文件不同；参见“备份文件”。）‘C-x v =’和‘C-x v u’之类的命令在可能时会利用自动版本备份，以避免访问网络。

把‘vc-cvs-stay-local’设为‘nil’会禁止制作自动版本备份。

自动版本备份的名字形如‘FILE.~VERSION.~’。这与‘C-x v ~’保存旧版本时使用的名字相似（参见“查看并比较旧修订”），区别只在于版本后面多了一个点（‘.’）。相关的 VC 命令可以同时使用这两种版本备份。主要区别在于，由‘C-x v ~’制作的手动版本备份在提交时不会被自动删除。

CVS 默认不使用锁定，但有办法通过其 'CVSREAD' 或"watch"特性启用类似锁定的行为；详情参见 CVS 文档。在这种情况下，你可以在 Emacs 中用 'C-x v v' 切换锁定，就像对基于锁定的版本控制系统所做的那样（参见"使用加锁的基本版本控制"）。

30.2 使用项目

"项目"（project）是用于生成一个或多个程序的文件集合。属于项目的文件通常存放在一个目录层次中；该层次的顶层目录称为"项目根目录"。

某个目录是否是某个项目的根，由项目特定的基础设施决定，这种基础设施称为"项目后端"。Emacs 目前支持两种这样的后端：感知 VC 的后端（参见"版本控制"），即把 VCS 仓库视为一个项目；以及 EDE（参见"Emacs 开发环境"）。预计将来会扩展以支持更多类型的项目。

哪些文件属于项目、哪些不属于，也由项目后端决定。例如，感知 VC 的后端不把"被忽略"的文件（参见"忽略版本控制文件"）视为项目的一部分。参见下面它的条目以了解说明和相关选项。

-- 用户选项：project-mode-line 若此用户选项非 'nil'，Emacs 会在模式行上显示当前项目（如果有）的名字；用 'mouse-1' 点击项目名会弹出包含项目相关命令的菜单。默认值为 'nil'。若值为 'non-remote'，Emacs 只对本地文件显示项目名；当因网络延迟而更新远程系统上项目的模式行很慢时，这很方便。

30.2.1 操作文件的项目命令

- 'C-x p f'：访问属于当前项目的文件（'project-find-file'）。
- 'C-x p g'：在当前项目的所有文件中查找正则表达式的匹配（'project-find-regexp'）。
- 'M-x project-search'：在当前项目的所有文件中交互式搜索正则表达式的匹配。
- 'C-x p r'：在当前项目的所有文件中针对正则表达式执行 query-replace（'project-query-replace-regexp'）。
- 'C-x p D'：在当前项目的根目录中运行 Dired（'project-dired'）。
- 'C-x p v'：在当前项目的根目录中运行 'vc-dir'（'project-vc-dir'）。
- 'C-x p s'：在当前项目的根目录中启动一个子 shell（'project-shell'）。
- 'C-x p e'：在当前项目的根目录中启动 Eshell（'project-eshell'）。
- 'C-x p c'：在当前项目的根目录中运行编译（'project-compile'）。
- 'C-x p !'：在当前项目的根目录中运行 shell 命令（'project-shell-command'）。
- 'C-x p &'：在当前项目的根目录中异步运行 shell 命令（'project-async-shell-command'）。
- 'C-x p o'：在当前项目中运行接下来的命令（'project-any-command'）。

Emacs 提供了方便地处理项目文件的命令。本小节介绍这些命令。

这里描述的所有命令都共享"当前项目"这一概念。当前项目由命令被调用时的当前缓冲区的 'default-directory' 决定（参见"文件名"）。如果该目录看起来不属于任何可识别的项目，这些命令会提示你输入项目目录。

命令 'C-x p f'（'project-find-file'）是访问（参见"访问文件"）属于当前项目的文件的便捷方式。与 'C-x C-f' 不同，此命令不要求输入要访问文件的完整文件名，你只需输入文件的基名（即省略前面的目录）。此外，该命令考虑的补全候选只包括属于当前项目的文件，不包括其他任何文件。如果光标点处有文件名，此命令会把该

文件作为“未来历史”（future history）的第一个元素提供出来。若带前缀参数，则包括项目根目录下的所有文件，但‘vc-directory-exclusion-list’中列出的 VCS 目录除外。

命令‘C-x p g’（‘project-find-regexp’）类似于‘rgrep’（参见“在 Emacs 中用 Grep 搜索”），但它只搜索属于当前项目的文件。该命令提示输入要搜索的正则表达式，并弹出带有搜索结果的 Xref 模式缓冲区，你可以在其中用 Xref 模式命令选择某个匹配（参见“‘*xref*’缓冲区中可用的命令”）。带前缀参数调用时，该命令还会提示输入开始搜索的基目录；例如这可以让你只搜索项目根目录下某个子目录中的项目文件。此命令显示匹配的方式受‘xref-auto-jump-to-first-xref’的值影响（参见“用标识符搜索与替换”）。

‘M-x project-search’是‘project-find-regexp’的顺序变体。它提示输入要在当前项目文件中搜索的正则表达式，但它不是找出所有匹配并显示出来，而是在找到匹配时停下，并在匹配所在位置访问匹配的文件，让你可以编辑该文件。要查找其余的匹配，输入‘M-x fileloop-continue <RET>’。

‘C-x p r’（‘project-query-replace-regexp’）类似于‘project-search’，但它会像‘query-replace’那样（参见“查询替换”）逐个询问你是否替换找到的每个匹配，并在你回答后继续下一个匹配。如果你的回答导致 Emacs 退出 query-replace 循环，你可以稍后用‘M-x fileloop-continue <RET>’继续。

命令‘C-x p d’（‘project-find-dir’）提示你通过补全选择当前项目内的一个目录，并打开列出其中文件的 Dired 缓冲区（参见“Dired 目录编辑器”）。

命令‘C-x p D’（‘project-dired’）打开一个 Dired 缓冲区（参见“Dired 目录编辑器”），列出当前项目根目录中的文件。

命令‘C-x p v’（‘project-vc-dir’）打开一个 VC 目录缓冲区（参见“VC 目录模式”），列出当前项目根目录下某个目录树中各文件的版本控制状态。

命令‘C-x p s’（‘project-shell’）以当前项目根目录为工作目录，在新缓冲区中启动一个 shell 会话（参见“在 Emacs 中运行 Shell 命令”）。

命令‘C-x p e’（‘project-eshell’）以当前项目根目录为工作目录，在新缓冲区中启动一个 Eshell 会话。参见《Eshell 手册》

命令‘C-x p c’（‘project-compile’）在当前项目根目录中运行编译（参见“在 Emacs 中运行编译”）。

命令‘C-x p !’（‘project-shell-command’）在当前项目根目录中运行‘shell-command’。

命令‘C-x p &’（‘project-async-shell-command’）在当前项目根目录中运行‘async-shell-command’。

最后，命令‘C-x p o’（‘project-any-command’）会在当前项目中运行你接下来输入的命令，无论它是否与文件相关。

30.2.2 操作缓冲区的项目命令

- ‘C-x p b’：切换到属于当前项目的另一个缓冲区（‘project-switch-to-buffer’）。
- ‘C-x p C-b’：列出项目缓冲区（‘project-list-buffers’）。
- ‘C-x p k’：删除所有属于当前项目的活动缓冲区（‘project-kill-buffers’）。
- ‘C-x p o’：在当前项目中运行接下来的命令（‘project-any-command’）。

在一个项目上的工作可能涉及许多访问属于该项目文件的缓冲区，还可能涉及属于该项目但不访问任何文件的缓冲区（例如‘project-compile’创建的‘*compilation*’缓冲区）。命令‘C-x p b’（‘project-switch-to-buffer’）通过提示你选择要切换的缓冲区、并且只把当前项目的缓冲区作为补全候选，帮助你在属于当前项目的缓冲区之间切换。

与命令‘list-buffers’（参见“列出已有缓冲区”）类似，命令‘C-x p C-b’（‘project-list-buffers’）显示现有缓冲区的列表，但只限于属于当前项目的缓冲区。

当你在项目上的工作结束时，你可能希望删除属于该项目的所有缓冲区，以保持 Emacs 会话更小。命令‘C-x p k’（‘project-kill-buffers’）可以做到这一点：它删除所有属于当前项目且满足‘project-kill-buffer-conditions’中任一条件的缓冲区。若‘project-kill-buffers-display-buffer-list’非‘nil’，将先显示要删除的缓冲区。

最后，命令‘C-x p o’（‘project-any-command’）会在当前项目中运行你接下来输入的命令，无论它是否与缓冲区相关。

30.2.3 切换项目

- ‘C-x p p’：为另一个项目运行 Emacs 命令（‘project-switch-project’）。

对项目文件进行操作的命令（参见“操作文件的项目命令”）在没有当前项目时会方便地提示你输入项目目录。当你在某个项目内，但想操作另一个项目时，使用‘C-x p p’命令（‘project-switch-project’）。此命令提示你从已知的项目根目录中选择一个目录，然后显示可用来操作你所选项目的命令菜单。变量‘project-switch-commands’控制菜单中有哪些命令可用，以及哪个键调用每条命令。

变量‘project-list-file’指定 Emacs 记录已知项目列表的文件。它默认为‘user-emacs-directory’中的文件‘projects’（参见“Emacs 如何查找你的初始化文件”）。

30.2.4 管理项目列表文件

- ‘M-x project-remember-project’：把当前项目加入‘project-list-file’。
- ‘M-x project-forget-project’：从‘project-list-file’中移除一个已知项目。

通常 Emacs 会自动向‘project-list-file’添加项目或从中移除项目，但有时你可能想手动编辑可用的项目。‘M-x project-remember-project’记住当前项目，‘M-x project-forget-project’提示你从可用项目中选择一个，然后把它从文件中移除。

用户选项‘project-list-exclude’可以设为总是忽略某些项目，使其不被记住、也不被保存到‘project-list-file’。它是一个由针对项目根目录和项目对象的正则表达式与谓词函数组成的列表。正则表达式与项目根目录匹配；谓词应接受项目对象作为唯一参数，若该项目*不应*保存到‘project-list-file’，就返回非‘nil’。

30.2.5 感知 VC 的项目后端

此后端是 Emacs 的一部分，默认启用。（其他后端可能需要安装附加包并进行适当配置。）

它根据 VCS 仓库的配置（如果有）确定项目的内容，并把“被忽略”的文件从结果中排除。

对某些流行的 VCS 系统（目前是 Git 和 Mercurial），它在列出文件时有一些性能优化。

它还使用基于时间的缓存。如果模式行显示的项目信息已过期，你可以输入 'M-x project-remember-projects-under RET' 来刷新过期的缓存信息。

-- 用户选项：project-vc-include-untracked 默认情况下，既未被 VCS 注册、也未被 VCS 忽略的文件被视为项目的一部分。把此变量自定义为 nil 即可改变这一点。

-- 用户选项：project-vc-ignores 使用此变量，你可以为项目添加更多忽略模式，从而把更多文件排除在项目的文件列表之外。其值是一个 glob 字符串列表。它们既可以匹配普通文件，也可以匹配目录。要把某个条目锚定到项目根目录，就以 './' 开头。要只匹配目录，就以 '/' 结尾。当此变量具有目录局部值时，它只应用于对应的目录子树。

-- 用户选项：project-vc-name 此变量让你把自动检测到的项目名改成你选择的字符串。默认情况下，名字是其根目录的基名。

-- 用户选项：project-vc-extra-root-markers 此变量让你在此后端中设置对非 VC 项目的检测，也可以让某些子目录被检测为单独的项目。其值是一个列表。

每个元素要么是基文件名，要么是用于这类文件名的 glob 模式。

示例值：'.dir-locals.el'、'package.json'、'requirements.txt'、'*.gemspec'。

30.3 变更日志

许多软件项目维护一份"变更日志"（change log）。它是一个文件，通常名为 'ChangeLog'，按时间顺序记录程序何时以及如何被更改。有时这些文件是由存储在版本控制系统中的变更日志条目自动生成的，或者被用来生成这些变更日志条目。有时会有多个变更日志文件，每个记录一个目录或目录树中的更改。

30.3.1 变更日志命令

Emacs 命令 'C-x 4 a' 为你正在编辑的文件向变更日志文件添加一个新条目（'add-change-log-entry-other-window'）。如果该文件实际上是备份文件，它就为该文件所对应的原始文件（parent）制作合适的条目——这对于为在当前版本中已被删除的函数写日志条目很有用。

'C-x 4 a' 在所选窗口之外的另一个窗口中访问变更日志文件（参见"'display-buffer' 的工作方式"），并创建新条目，除非最近的条目就是今天的日期且是你的名字。它还会为当前文件创建一个新项。对许多语言，它甚至能猜出被更改的函数或其他对象的名称。

为找到变更日志文件，Emacs 从你正在编辑的文件开始向上搜索目录树。默认情况下，如果找到看起来是版本控制仓库根的目录，它就停下。要改变这一点，自定义 'change-log-directory-files'。

当变量 'add-log-keep-changes-together' 非 'nil' 时，'C-x 4 a' 会把内容添加到该文件已有的项中，而不是新建一项。

你可以合并多个性质相同的更改。如果在最初的 'C-x 4 a' 之后没有输入任何文本，那么后续的 'C-x 4 a' 会向变更日志条目中再添加一个符号。

若 'add-log-always-start-new-record' 非 'nil'，'C-x 4 a' 总是创建新条目，即使最后一个条目是你本人在同一天制作的。

若变量‘change-log-version-info-enabled’的值非‘nil’，‘C-x 4 a’会把文件的版本号添加到变更日志条目中。它通过搜索文件的前百分之十、使用变量‘change-log-version-number-regexp-list’中的正则表达式来找到版本号。

变更日志文件在 Change Log 模式下访问。在这个主模式中，每一组分组项算作一个段落，每个条目被视为一页。这便于编辑条目。‘C-j’和自动填充会把每个新行像前一行那样缩进；这便于输入条目的内容。

在 Change Log 模式开启时，你可以使用命令‘change-log-goto-source’（默认绑定到‘C-c C-c’）转到光标点附近变更日志条目所对应的源码位置。之后调用‘next-error’命令（默认绑定到‘M-g M-n’和‘C-x `’）将在变更日志的条目之间移动。你会跳到文件中实际被更改的位置，而不只是跳到下一个变更日志条目。你还可以用‘previous-error’在变更日志条目之间向后移动。

你可以使用命令‘M-x change-log-merge’把其他日志文件合并到一个处于 Change Log 模式的缓冲区中，同时保持条目按日期排序。

版本控制系统是跟踪程序更改并维护变更日志的另一种方式。如今许多使用 VCS 的项目不再保留单独的受版本控制的变更日志文件，因此你可能希望避免让仓库中有这样一个文件。若‘add-log-dont-create-changelog-file’的值非‘nil’，像‘C-x 4 a’（‘add-change-log-entry-other-window’）这样的命令会在这样的文件尚不存在时，把更改记录到名字合适的临时缓冲区中而不是文件中。

无论你是有变更日志文件，还是用临时缓冲区记录变更日志，都可以在 VC Log 缓冲区中输入‘C-c C-a’（‘log-edit-insert-changelog’）来插入相关的变更日志条目，前提是这些条目存在。参见“日志条目缓冲区的特性”。

30.3.2 ChangeLog 的格式

变更日志条目以一行头部开始，其中包含当前日期、你的名字（取自变量‘add-log-full-name’）和你的电子邮件地址（取自变量‘add-log-mailing-address’）。除这些头部行以外，变更日志中的每一行都以空格或制表符开头。条目的大部分由“项”（item）组成，每一项都以一行以空白和星号开头的行开始。下面是两个条目，日期都在 1993 年 5 月，分别包含两项和一项。

```
1993-05-25  Richard Stallman  <rms@gnu.org>
```

```
* man.el: Rename symbols 'man-*' to 'Man-*'.
(manual-entry): Make prompt string clearer.
```

```
* simple.el (blink-matching-paren-distance):
Change default to 12,000.
```

```
1993-05-24  Richard Stallman  <rms@gnu.org>
```

```
* vc.el (minor-mode-map-alist): Don't use it if it's void.
(vc-cancel-version): Doc fix.
```

一个条目可以描述多项更改；每项更改都应有自己的项，或在某个项中占自己的一行。通常项与项之间应有一个空行。当各项相互关联时（同一更改的不同部分，位于不同位置），就让它们之间不留空行，从而把它们分组。

你应在变更日志文件的末尾放置版权声明和许可声明。下面是一个例子：

```
Copyright 1997--1998, 2025--2026 Free Software Foundation, Inc.
Copying and distribution of this file, with or without modification, are
permitted provided the copyright notice and this notice are preserved.
```

当然，你应该替换为适当的年份和版权持有者。

30.4 查找标识符引用

“标识符”（identifier）是程序某个语法子单元的名字：函数、子程序、方法、类、数据类型、宏等。在编程语言中，每个标识符都是该语言语法中的一个符号。标识符也称为“标签”（tag）。

程序开发和维护需要能够快速找到每个标识符被定义和被引用的位置、在整个项目中重命名标识符等等。这些能力对于在除为支持编程语言而定义的主模式之外的主模式中查找引用也很有用。例如，文本或 TeX 文档的章、节、附录等也可以被当作子单元，它们的名字可以用作标识符。在本章中，我们用术语“标识符”统称任何种类子单元的名字，无论在程序源码中还是在其他种类的文本中。

Emacs 为这些能力提供了统一的接口，称为 ‘xref’。

为完成其工作，‘xref’ 需要利用信息并采用特定于主模式的方法。要在哪些文件中搜索标识符、如何查找对标识符的引用、如何对标识符进行补全——这一切以及更多都是特定于模式的知识。‘xref’ 把它工作中特定于模式的部分委托给由主模式提供的“后端”；对于不提供自己后端的模式，它也为其中一些命令包含默认实现。

后端可以通过多种方式实现其能力。下面举几个例子：

1. 一些主模式为查找语言符号提供了内置手段。例如，Emacs Lisp 符号可以通过搜索由 Emacs Lisp 解释器维护的包加载历史、以及查阅内置文档字符串来识别；Emacs Lisp 模式在其后端中使用这些设施来查找符号的定义。（这类后端的一个缺点是，它只知道已加载到解释器中的子单元。）

b. 如果 Eglot 已针对当前缓冲区的项目（参见“使用项目”）和当前缓冲区的主模式激活，Eglot 会查询外部语言服务器程序，并提供该服务器给出的、关于项目中各标识符定义的数据。参见《Eglot 手册》的“Eglot Features”。

c. 外部程序可以通过扫描相关文件来提取引用，并建立这些引用的数据库。后端随后可以在需要列出或查找引用时访问该数据库。Emacs 发行版包含 ‘etags’，这是一个为程序中的标识符定义打标签的命令，它通过把引用提取到“标签表”（tags tables）中，支持许多编程语言以及其他主模式（如 HTML）。‘etags’ 所支持语言的主模式可以用标签表作为其后端的基础。启用 ‘etags-regen-mode’ 即可为当前项目中受支持的文件类型生成标签表，并在编辑时自动更新。或者，你也可以手工构建标签表，以控制文件集合及其更新时机，参见“创建标签表”。

- Find Identifiers:: 用于查找标识符定义或引用位置、列出标识符等的命令。 * Tags Tables:: 标签表记录哪个文件定义了某个符号。 * Select Tags Table:: 如何访问特定的标签表。

30.4.1 查找标识符

本小节介绍查找标识符引用以及执行各种标识符查询的命令。每一处这样的引用都可能是_定义_一个标识符，例如给出某个程序子单元的实现或文档章节的正文；也可能是_使用_该标识符，例如调用函数或方法、给变量赋值、在交叉引用中提及某一章，等等。

30.4.1.1 查找标识符定义

‘xref’ 让你能做的最重要的事，就是找出特定标识符的定义。

- ‘M-.’：查找标识符的定义（‘xref-find-definitions’）。
- ‘C-M-. PATTERN <RET>’：查找名称匹配 PATTERN 的所有标识符（‘xref-find-apropos’）。
- ‘C-x 4 . <RET>’：查找标识符的定义，但在另一个窗口中显示（‘xref-find-definitions-other-window’）。
- ‘C-x 5 . <RET>’：查找标识符的定义，并在新框架中显示（‘xref-find-definitions-other-frame’）。
- ‘M-x xref-find-definitions-at-mouse’：查找鼠标点击处的标识符的定义。
- ‘M-,’：回到你先前调用 ‘M-.’ 及类似命令的位置（‘xref-go-back’）。
- ‘C-M-,’：前进到你先前调用 ‘M-,’ 的位置（‘xref-go-forward’）。
- ‘M-x xref-etags-mode’：让 ‘xref’ 改用 ‘etags’ 后端。

‘M-.’（‘xref-find-definitions’）显示光标点处标识符的定义。带前缀参数，或光标点处没有标识符时，它会提示输入标识符。（若希望它总是提示，请把 ‘xref-prompt-for-identifier’ 定制为 ‘t’。）

向 ‘M-.’ 输入标识符参数时，可以使用通常的迷你缓冲区补全命令（参见“补全”），已知的标识符名就是补全候选。

它使用当前的 Xref 后端；若未配置任何后端，它会报错，并给出一些建议。

与大多数可以切换缓冲区的命令一样，‘xref-find-definitions’ 有一个在另一个窗口中显示新缓冲区的变体（参见“‘display-buffer’ 的工作方式”），还有一个为它新建框架的变体。前者是 ‘C-x 4 .’（‘xref-find-definitions-other-window’），后者是 ‘C-x 5 .’（‘xref-find-definitions-other-frame’）。

命令 ‘xref-find-definitions-at-mouse’ 的工作方式与 ‘xref-find-definitions’ 相同，但它查找的是鼠标事件所在位置处或附近的标识符名。次模式 ‘xref-mouse-mode’ 会把该命令绑定到 ‘C-mouse-1’。

命令 ‘C-M-.’（‘xref-find-apropos’）类似于针对标签的 ‘apropos’（参见“Apropos”）。它列出所选标签表中名称匹配指定 REGEXP 的标识符。这与 ‘M-.’ 基本相同，区别在于它对标识符做正则表达式匹配，而不是把符号名当作固定字符串来匹配。默认情况下，该命令像 ‘M-.’ 一样弹出 ‘*xref*’ 缓冲区，但你可以通过定制变量 ‘tags-apropos-additional-actions’ 来显示附加输出；详情参见其文档。

若上述任一命令找到多个匹配的定义，默认会弹出 ‘*xref*’ 缓冲区显示匹配的候选并选中该缓冲区的窗口。（‘C-M-.’ 只要找到至少一个匹配，就_总是_弹出 ‘*xref*’ 缓冲区。）每个候选通常在该缓冲区中显示为一个文件名以及该文件中匹配的标识符。在该缓冲区中，你可以选择任一候选来显示，还可以使用若干附加命令，见 参见“‘*xref*’ 缓冲区中可用的命令”。不过，若变量 ‘xref-auto-jump-to-first-definition’ 的值为 ‘move’，Emacs 会自动把光标点移到 ‘*xref*’ 缓冲区中的第一个候选，于是直接键入 <RET> 就会显示该候选的定义。若该变量的值为 ‘t’ 或 ‘show’，会自动在自己的窗口中显示第一个候选；‘t’ 还会选中显示第一个候选定义的窗口，而 ‘show’ 则让 ‘*xfer*’ 缓冲区的窗口保持选中。默认值为 ‘nil’，此时只在 ‘*xref*’ 缓冲区中显示候选，既不选中任何候选，也不显示其定义，直到你在 ‘*xref*’ 缓冲区中选择某个候选。

若你离开显示多个候选的 ‘*xref*’ 缓冲区所在窗口，可以用命令 ‘M-g M-n’（‘next-error’）和 ‘M-g M-p’（‘previous-error’）在各个候选之间移动。参见“编译模式”。

若要回到你_从那里_显示定义的那些位置，请使用 'M-,'（'xref-go-back'）。它会跳回到最后一次调用 'M-.' 时的光标点。于是你可以用 'M-.' 找到并查看某个东西的定义，再用 'M-,' 回到原来的位置。'M-,' 让你沿着位置历史中前进的步骤逐步回溯，一直回到历史中第一个位置，也就是你最初调用 'M-.' 的地方，或回到其间任何位置。

如果你先前用 'M-,' 回溯得太远，或者想重新查看你从其回溯的某个位置，可以用 'C-M-,'（'xref-go-forward'）再次前进。这与使用 'M-.' 类似，只是你无需在每一步都把光标点移到想查找其定义的标识符上。'C-M-,' 让你沿着位置历史中回溯的步骤逐步重走，一直走到历史中最后一个位置，也就是你调用 'M-,' 的地方，或走到其间任何位置。

有些主模式安装了 'xref' 支持设施，它们有时可能找不到某些标识符。例如，在 Emacs Lisp 模式中（参见"求值 Emacs Lisp 表达式"），'M-.' 默认只能找到已加载到当前 Emacs 会话中或自动加载（参见《Emacs Lisp 参考手册》的"Autoload"）的 Lisp 包中的函数和变量。若 'M-.' 找不到某些标识符，你可以尝试强制 'xref' 使用 'etags' 后端（参见"查找标识符引用"）。为此，请用 'M-x xref-etags-mode' 打开 Xref Etags 次模式，然后再次调用 'M-.'。（为使其生效，务必运行 'etags' 在源文件所在目录树中创建标签表，参见"创建标签表"。）

30.4.1.2 '*xref*' 缓冲区中可用的命令

特殊的 Xref 模式在 '*xref*' 缓冲区中提供以下命令：

- '<RET>'、'mouse-1'：显示当前行上的引用（'xref-goto-xref'）。带前缀参数时，还会把 '*xref*' 缓冲区放到下层。
- 'mouse-2'：与 'mouse-1' 相同，但让显示 '*xref*' 缓冲区的窗口成为选中窗口（'xref-select-and-show-xref'）。
- 'n'、'.'：移到下一个引用并在另一个窗口中显示（'xref-next-line'）。
- 'N'：移到下一个引用组的第一个引用并在另一个窗口中显示（'xref-next-group'）。
- 'p'、','：移到上一个引用并在另一个窗口中显示（'xref-prev-line'）。
- 'P'：移到上一个引用组的第一个引用并在另一个窗口中显示（'xref-prev-group'）。
- 'C-o'：在另一个窗口中显示当前行上的引用（'xref-show-location-at-point'）。
- 'r PATTERN <RET> REPLACEMENT <RET>'：对匹配 PATTERN 的引用执行交互式 query-replace，把匹配处替换为 REPLACEMENT（'xref-query-replace-in-results'）。该命令只能用在显示某个标识符在所有相关文件中的全部匹配的 '*xref*' 缓冲区中。参见"用标识符搜索与替换"。
- 'g'：刷新 '*xref*' 缓冲区的内容（'revert-buffer'）。参见"还原缓冲区"。
- 'M-,'：关闭显示 '*xref*' 缓冲区的窗口，然后跳到 Xref 栈中的上一个位置（'xref-quit-and-pop-marker-stack'）。
- 'q'：关闭显示 '*xref*' 缓冲区的窗口（'xref-quit'）。

此外，常用的移动命令（如方向键、'C-n' 和 'C-p'）可在缓冲区中移动，而不显示引用。

30.4.1.3 用标识符搜索与替换

本小节的命令或对标识符本身、或对引用这些标识符的文件执行各种搜索与替换操作。

- 'M-?'：查找光标点处标识符的所有引用。

- ‘r’、‘M-x xref-query-replace-in-results <RET> REPLACEMENT <RET>’、‘C-u M-x xref-query-replace-in-results <RET> REGEXP <RET> REPLACEMENT <RET>’：在‘*xref*’缓冲区所示所有标识符的名称中，交互式地把 REGEXP 替换为 REPLACEMENT。
- ‘M-x xref-find-references-and-replace <RET> FROM <RET> TO <RET>’：交互式地把标识符 FROM 的所有实例改名为新名字 TO。
- ‘M-x tags-search <RET> REGEXP <RET>’：在所选标签表的各文件中搜索 REGEXP。
- ‘M-x tags-query-replace <RET> REGEXP <RET> REPLACEMENT <RET>’：对所选标签表中的每个文件执行‘query-replace-regexp’。
- ‘M-x fileloop-continue’：从当前光标点位置重新开始上述最后两个命令之一。

‘M-?’查找光标点处标识符的所有引用，必要时会带补全地提示输入标识符。视当前后端而定（参见“查找标识符引用”），即使光标点处找到了有效标识符，该命令也可能提示。带前缀参数调用时，它总是提示输入标识符。（若希望它总是提示，请把变量‘xref-prompt-for-identifier’的值定制为‘t’；或者设为‘nil’，以便仅在光标点处没有可用标识符时才提示。）随后该命令用‘*xref*’缓冲区展示对该标识符的所有引用，显示文件名以及标识符被引用的行。该缓冲区中可以使用 Xref 模式的命令，参见“‘*xref*’缓冲区中可用的命令”。

在某个主模式使用‘etags’后端的缓冲区中调用时，‘M-?’搜索主模式与原缓冲区相同文件。它根据文件扩展名推测该模式，因此若‘M-?’似乎跳过了相关的缓冲区或文件，可尝试定制变量‘semantic-symref-filepattern-alist’（若你的缓冲区主模式在此表中已有条目）或‘auto-mode-alist’（若没有），从而把缺失的扩展名告知‘xref’（参见“选择文件模式”）。

若变量‘xref-auto-jump-to-first-xref’的值为‘t’，‘xref-find-references’会自动跳到‘*xref*’缓冲区中的第一个结果，并选中显示该引用的窗口；你可以用‘M-g M-n’（‘next-error’）和‘M-g M-p’（‘previous-error’）选择其他结果（参见“编译模式”）。若值为‘show’，会显示第一个结果，但显示‘*xref*’缓冲区的窗口保持选中。若值为‘move’，会在‘*xref*’缓冲区中选中第一个结果，但不显示它；随后你可以用 <RET> 实际显示该引用。默认值为‘nil’，此时只在‘*xref*’缓冲区中显示结果，既不选中任何结果，也不显示引用本身。

‘r’（‘xref-query-replace-in-results’）读取一个 REPLACEMENT 字符串，就像普通的‘M-x query-replace-regexp’一样。然后它在所有文件中引用这些标识符的所有位置，把‘*xref*’缓冲区中显示的标识符改名，使其新名字为 REPLACEMENT。当你在重构过程中重命名标识符时，这很有用。该命令应在‘M-?’生成的‘*xref*’缓冲区中调用。默认情况下，该命令把每个标识符的整个名字替换为 REPLACEMENT；但带前缀参数调用时，它会提示输入一个正则表达式来匹配标识符名，并只把标识符名中该正则表达式的匹配部分替换为 REPLACEMENT。

‘M-x xref-find-references-and-replace’的工作方式与‘xref-query-replace-in-results’类似，但当你想重命名由名字 FROM 指定的单个标识符时，它更方便。

在‘*xref*’缓冲区中键入‘e’会使该缓冲区可写并进入 Xref Edit 模式。与 Occur Edit 模式（参见“其他搜索与循环命令”）类似，你可以编辑 Xref 后端报告的匹配行，并让这些修改反映到访问原始文件的缓冲区中。键入‘C-c C-c’即可离开 Xref Edit 模式，返回 Xref 模式。

‘M-x tags-search’用迷你缓冲区读取一个正则表达式，然后在所选标签表的所有文件中逐个文件地搜索匹配。它会显示正在搜索的文件名，以便你跟踪进度。一旦找到一处匹配，‘tags-search’就返回。该命令要求标签表可用（参见“标签表”）。

用 'tags-search' 找到一处匹配后，你大概想找出其余所有匹配。'M-x fileloop-continue' 会继续 'tags-search'，再找一处匹配。它会搜索当前缓冲区的其余部分，然后搜索标签表中其余的文件。

'M-x tags-query-replace' 通过标签表中的所有文件执行一次 'query-replace-regexp'。它像普通的 'M-x query-replace-regexp' 一样读取要搜索的正则表达式和要替换成的字符串。它的搜索方式与 'M-x tags-search' 很相似，但是反复进行，按你的输入逐个处理匹配。关于 query replace 的更多信息，参见"查询替换"。

你可以通过定制变量 'tags-case-fold-search' 的值来控制标签搜索命令的大小写敏感性。默认使用与 'case-fold-search' 的值相同的设置（参见"搜索时的宽松匹配"）。

只调用一次 'M-x tags-query-replace' 就有可能走完标签表中的所有文件。但常常有用的是暂时退出，你可以用任何没有特殊 query replace 含义的输入事件来做到。随后你可以键入 'M-x fileloop-continue' 恢复该 query replace；该命令会恢复你最近执行的标签搜索或替换命令。例如，要跳过当前文件的其余部分，你可以键入 'M-> M-x fileloop-continue'。

注意，上述命令执行的搜索范围远比 'xref-find-definitions' 系列宽。'xref-find-definitions' 命令只搜索匹配你的字符串或正则表达式的标识符的定义。而 'xref-find-references'、'tags-search' 和 'tags-query-replace' 会找出该标识符或正则表达式的每一处出现，正如普通搜索命令和替换命令在当前缓冲区中所做的那样。

作为 'xref-find-references' 和 'tags-search' 的替代方案，你可以把 'grep' 作为子进程运行，并让 Emacs 逐条向你显示匹配的行。参见"在 Emacs 中用 Grep 搜索"。

30.4.1.4 标识符查询

- 'C-M-i'、'M-<TAB>'：对光标点周围的文本执行补全，若已加载标签表，可能会使用所选的标签表（'completion-at-point'）。
- 'M-x list-tags <RET> FILE <RET>'：列出程序文件 FILE 中定义的标识符。
- 'C-M-. REGEXP <RET>'：列出所有匹配 REGEXP 的标识符（'xref-find-apropos'）。参见"查找标识符定义"。
- 'M-x tags-next-file'：访问所选标签表中记录的文件。

在大多数编程语言模式中，你可以键入 'C-M-i' 或 'M-<TAB>'（'completion-at-point'）来补全光标点处的符号。有些模式为该命令提供了针对本模式定制的专门补全；对于没有提供的模式，如果已加载标签表，该命令可以用它生成补全候选。参见"符号名补全"。

'M-x list-tags' 带补全地读取所选标签表所覆盖的某个文件的文件名，并列出该文件中定义的标签；它把当前缓冲区的文件名作为要列出其标签的默认文件。除非标签表中记录的文件名包含目录，否则不要把目录作为文件名的一部分。该命令只对 etags 后端有效，并要求该项目的标签表可用。参见"标签表"。

'M-x tags-next-file' 访问所选标签表所覆盖的文件。第一次调用时，它访问该表覆盖的第一个文件。以后每次调用都访问下一个被覆盖的文件，除非提供了前缀参数，此时它回到第一个文件。该命令要求已选定标签表。

30.4.2 标签表

"标签表"（tags table）记录着通过扫描某个程序或某份文档的源代码而提取出的标签(1)。从生成文件里提取的标签引用的是原始文件，而不是在提取标签时扫描的那些生成文件。生成文件的例子包括：由 Cweb 源文件、

由 Yacc 解析器或由 Lex 扫描器定义生成的 C 文件；预处理后的 '.i' C 文件；以及由预处理 '.fpp' 源文件产生的 Fortran 文件。

要生成标签表，你需要对文档或源代码文件运行 'etags' shell 命令。'etags' 程序把标签写入"标签表文件"，简称"标签文件"。标签文件的惯用名是 'TAGS'。参见"创建标签表"。（也可以用其他包中能生成相同格式标签表的命令之一来创建标签表。）若启用全局次模式 'etags-regen-mode'，Emacs 会在需要时自动生成并更新标签表。

Emacs 通过 'etags' 包使用标签表，把它作为 'xref' 所支持的后端之一。由于标签表是由 Emacs 发行版自带的 'etags' 命令生成的，我们在此更详细地描述标签表。

Ebrowse 设施与 'etags' 类似，但专门为 C++ 定制。参见《Ebrowse 手册》 Semantic 包提供了另一种生成和使用标签的方法，独立于 'etags' 设施。参见"Semantic"。

> 注："标签"（tag）是标识符引用的同义词。基于 'etags' 包的命令和特性传统上就在这个含义上使用"标签"一词，本小节沿用这一传统。

30.4.2.1 源文件标签语法

以下是最流行的几种语言中标签语法的定义方式：

- 在 C 代码中，任何 C 函数或 typedef 都是标签，'struct'、'union' 和 'enum' 的定义也是。'#define' 宏定义、'#undef' 和 'enum' 常量也是标签，除非你在生成标签表时指定 '--no-defines'。类似地，全局变量是标签，除非你指定 '--no-globals'，结构体成员也是，除非你指定 '--no-members'。使用 '--no-globals'、'--no-defines' 和 '--no-members' 可以显著减小标签表文件。

给 'etags' 提供 '--declarations' 选项，就可以在函数定义之外，也为函数声明和外部变量打标签。

- 在 C++ 代码中，除了 C 代码的所有标签构造外，成员函数也会被识别；成员变量也会被识别，除非你使用 '--no-members' 选项。'operator' 定义的标签名形如 'operator+'。若指定 '--class-qualify' 选项，类中变量和函数的标签名为 'CLASS::VARIABLE' 和 'CLASS::FUNCTION'。默认情况下，类方法和成员不带类限定，这有助于在源码中更准确地识别它们的名字。

- 在 Java 代码中，标签包括 C++ 中能识别的所有构造，再加上 'interface'、'extends' 和 'implements' 构造。类中变量和函数的标签名形如 'CLASS.VARIABLE' 和 'CLASS.FUNCTION'。

- 在 LaTeX 文档
  中，'\chapter'、'\section'、'\subsection'、'\subsubsection'、'\eqno'、'\label'、'\ref'、'\Ref'、'\footref'、'\cite'、'\bibitem'、'\part'、'\appendix'、'\entry'、'\index'、'\def'、'\edef'、'\gdef'、'\xdef'、'\newcommand'、'\renewcommand'、'\newenvironment'、'\renewenvironment'、'\DeclareRobustCommand'、'\newrobustcmd'、'\renewrobustcmd'、'\providecommand'、'\providerobustcmd'、'\NewDocumentCommand'、'\RenewDocumentCommand'、'\ProvideDocumentCommand'、'\DeclareDocumentCommand'、'\NewExpandableDocumentCommand'、'\RenewExpandableDocumentCommand'、'\ProvideExpandableDocumentCommand'、'\DeclareExpandableDocumentCommand'、'\NewDocumentEnvironment'、'\RenewDocumentEnvironment'、'\ProvideDocumentEnvironment'、'\DeclareDocumentEnvironment'、'\csdef'、'\csedef'、'\csgdef'、'\csxdef'、'\csletcs'、'\cslet'、'\letcs'、'\let'、'\cs_new_protected_nopar'、'\cs_new_protected'、'\cs_new_nopar'、'\cs_new_eq'、'\cs_new'、'\cs_set_protected_nopar'、'\cs_set_protected'、'\cs_set_nopar'、'\cs_set_eq'、'\cs_set'、

'\cs_gset_protected_nopar'、'\cs_gset_protected'、'\cs_gset_nopar'、'\cs_gset_eq'、'\cs_gset'、'\cs_generate_from_arg_count' 和 '\cs_generate_variant' 的参数都是标签。这些命令的任何带星号变体的参数也是如此。

若在调用 'etags' 之前把这些命令指定在环境变量 'TEXTAGS' 中，其他命令也可以生成标签。该环境变量的值应当是以冒号分隔的命令名列表。例如，

```
TEXTAGS="mycommand:myothercommand"
export TEXTAGS
```

（使用 Bourne shell 语法）指定了 '\mycommand' 和 '\myothercommand' 这些命令也定义标签。

- 在 Lisp 代码中，用 'defun' 定义的任何函数、用 'defvar' 或 'defconst' 定义的任何变量，以及一般而言任何在第零列以 '(def' 开头的表达式的第一个参数，都是标签。作为例外，形如 '(defvar FOO)' 的表达式被视为声明，只有在给出 '--declarations' 选项时才打标签。
- 在 Scheme 代码中，标签包括用 'def' 定义的任何东西，或用名字以 'def' 开头的构造定义的任何东西。它们还包括在文件顶层用 'set!' 设置的变量。

另外还支持其他几种语言：

- 在 Ada 代码中，函数、过程、包、任务和类型都是标签。使用 '--packages-only' 选项可以只为包创建标签。

在 Ada 中，同一个名字可以用于不同种类的实体（例如，既用于一个过程，也用于一个函数）。此外，对于包、过程和函数这类东西，还有规范（spec，即接口）和主体（body，即实现）之分。为了便于挑选你想要的定义，Ada 的标签名带有表示实体类型的后缀：

- '/b'：包体（package body）。
- '/f'：函数（function）。
- '/k'：任务（task）。
- '/p'：过程（procedure）。
- '/s'：包规范（package spec）。
- '/t'：类型（type）。

因此，'M-x find-tag <RET> bidule/b <RET>' 会直接跳到包 'bidule' 的包体，而 'M-x find-tag <RET> bidule <RET>' 则只是查找任何名为 'bidule' 的标签。

- 在汇编代码中，出现在行首且后面跟冒号的标号是标签。
- 在 Bison 或 Yacc 输入文件中，每条规则都把它所构造的非终结符定义为一个标签。文件中包含 C 代码的部分按 C 代码解析。
- 在 Cobol 代码中，标签是段落名；即任何从第 8 列开始并以句点结尾的单词。
- 在 Erlang 代码中，标签是文件中定义的函数、记录和宏。
- 在 Fortran 代码中，函数、子程序和块数据是标签。
- 在 Go 代码中，包、函数和类型是标签。

- 在 HTML 输入文件中，标签是 ‘title’ 以及 ‘h1’、‘h2’、‘h3’ 标题。此外，锚点中的 ‘name=’ 和所有出现的 ‘id=’ 也是标签。
- 在 Lua 输入文件中，所有函数都是标签。
- 在 makefile 中，目标是标签；另外，除非你指定 ‘--no-globals’，否则变量也是标签。
- 在 Objective C 代码中，标签包括类、类类别、方法和协议的 Objective C 定义。类中变量和函数的标签分别命名为 ‘CLASS::VARIABLE’ 和 ‘CLASS::FUNCTION’。
- 在 Pascal 代码中，标签是文件中定义的函数和过程。
- 在 Perl 代码中，标签是由 ‘package’、‘sub’、‘use constant’、‘my’ 和 ‘local’ 关键字定义的包、子程序和变量。若想标记全局变量，请使用 ‘--globals’。子程序的标签命名为 ‘PACKAGE::SUB’。默认包中定义的子程序，其名称为 ‘main::SUB’。
- 在 PHP 代码中，标签是函数、类和 define。变量也是标签，除非你使用 ‘--no-members’ 选项。
- 在 PostScript 代码中，标签是函数。
- 在 Prolog 代码中，标签是位于行首的谓词和规则。
- 在 Python 代码中，位于行首的 ‘def’ 或 ‘class’ 会生成一个标签。
- 在 Ruby 代码中，位于行首的 ‘def’、‘class’ 或 ‘module’ 会生成一个标签。常量也会生成标签。
- 在 Rust 代码中，由 ‘fn’、‘enum’、‘struct’ 或 ‘macro_rules!’ 定义的任何内容都是标签。

你也可以基于正则表达式匹配来生成标签（参见“Etags 正则表达式”），以处理其他格式和语言。

30.4.2.2 创建标签表

‘etags’ 程序用于创建标签表文件。它了解若干种语言的语法，如参见“源文件标签语法”所述。运行 ‘etags’ 的方法如下：

```
etags INPUTFILES...
```

‘etags’ 程序读取指定的文件，并在当前工作目录中写出一个名为 ‘TAGS’ 的标签表。你可以选用 ‘--output=FILE’ 选项为标签表指定不同的文件名；把文件名指定为 ‘-’ 则会把标签表打印到标准输出。你还可以使用 ‘--append’ 选项把新创建的标签表追加到已有文件中。

如果指定的文件不存在，‘etags’ 会寻找它们的压缩版本，解压后再读取。在 MS-DOS 上，如果在命令行中给了 ‘mycode.c’ 而 ‘mycode.c’ 并不存在，‘etags’ 还会寻找像 ‘mycode.cgz’ 这样的文件名。

如果由于标签表所描述的文件发生变化而使标签表过时，你可以重新运行 ‘etags’ 程序来更新它。如果标签表没有记录某个标签，或把它记录在错误的文件中，那么在更新标签表之前 Emacs 无法找到该定义。但如果标签表中记录的位置稍有偏差（由于其他编辑），Emacs 仍能找到正确的位置，只是略有延迟。

因此，不必在每次编辑后都更新标签表。当你定义了希望列入的新标签，或把标签定义从一个文件移到另一个文件，或改动相当大时，才应当更新标签表。

如果启用了下面描述的 ‘etags-regen-mode’ 次模式，Emacs 会在需要时自动让标签表保持最新。

通过把 ‘--include=FILE’ 选项传给 ‘etags’，你可以让一个标签表“包含”另一个标签表。这样它既涵盖自己的文件，也涵盖被包含的标签文件所涵盖的所有文件。

如果在运行 'etags' 时用相对文件名指定源文件，标签表中将包含相对于最初写入标签表所在目录的文件名。这样，你可以整体移动包含标签表和源文件的整个目录树，而标签表仍能正确指向源文件。不过，如果标签表是 '-' 或位于 '/dev' 目录中，文件名则相对于当前工作目录。例如，在把标签写入标准输出时，这很有用。

使用相对文件名时，它不应是指向另一目录中标签表的符号链接，因为那通常会使这些文件名失效。

如果给 'etags' 指定绝对文件名作为参数，那么标签表将包含绝对文件名。这样，只要源文件留在原处，即使移动标签表，它仍指向同样的文件。绝对文件名以 '/' 开头，在 MS-DOS 和 MS-Windows 上则以 'DEVICE:/' 开头。

当你要从大量文件生成标签表时，在命令行中列出它们可能会有问题，因为某些系统对命令行长度有限制。你可以通过在文件名处写一个短横线，让 'etags' 从标准输入读取文件名，从而规避该限制，像这样：

```
find . -name "*.[chCH]" -print | etags -
```

'etags' 根据输入文件的文件名和内容来识别其所用语言。它首先尝试把文件名和扩展名与某些语言常用的名字相匹配。有些语言有名称已知的解释器（例如 Perl 的 'perl'、Prolog 的 'pl'），因此 'etags' 接下来会在输入文件的第一行寻找形如 '#!INTERP' 的解释器说明，并与已知解释器匹配。如果这些都不奏效，或者你想覆盖自动的语言检测，可以用 '--language=NAME' 选项显式指定语言。这些选项可以与文件名混用；每个选项作用于其后出现的文件名。指定 '--language=auto' 可让 'etags' 恢复根据文件名和文件内容猜测语言。指定 '--language=none' 则完全关闭针对具体语言的处理；此时 'etags' 仅通过正则表达式匹配来识别标签（参见"Etags 正则表达式"）。当输入文件使用的语言尚不受 'etags' 支持，而你想避免 'etags' 退回把 Fortran 和 C 作为默认语言时，这很方便。

如果你指定 '--no-fallback-lang' 选项，也可以阻止 'etags' 退回 Fortran 和 C。选项 '--fallback-lang' 则取消该设置。

'--parse-stdin=FILE' 选项主要在从程序中调用 'etags' 时有用。它可以在命令行中（仅一次）代替文件名。'etags' 将从标准输入读取，并把生成的标签标记为属于文件 FILE。

对于 C 和 C++，如果源文件不遵守 GNU 编码规范中"只在函数和 'struct' 定义等顶层定义中把花括号（'{' 和 '}'）放在第 0 列"的约定，我们建议你使用 '--ignore-indentation' 选项，以免 'etags' 错误地解释位于第 0 列的右花括号。

'etags --help' 会输出 'etags' 已知的语言列表，以及猜测语言所用的文件名规则。它还打印所有可用 'etags' 选项的列表及简短说明。如果后面跟一个或多个 '--language=LANG' 选项，它会输出关于如何为 LANG 生成标签的详细信息。

默认情况下，'etags' 会把扫描过的所有文件都包含进它生成的标签表，包括在其中完全没有找到标签的文件。指定 '--no-empty-file-entries' 可避免这种情况；这样没有标签的文件就不会出现在标签表中。但要注意，处理标签表所提及文件的命令（例如 'tags-search'，参见"用标识符搜索与替换"）仍会处理那些因此从标签表中略去的文件。选项 '--empty-file-entries' 则取消该设置。

除了手动调用 'etags' 来创建和更新标签表，你还可以让 Emacs 自动为你完成。全局次模式 'etags-regen-mode' 若启用，会在需要时自动生成标签表，并在你编辑任何提供标签的源文件时负责更新它们。该模式使用当前项

目配置（参见“使用项目”）来确定把哪些文件交给‘etags’以重新生成项目的标签表。你可以用下列用户选项定制该次模式的工作方式：

- ‘etags-regen-program’：用于重新生成标签表的程序；默认为‘etags’。
- ‘etags-regen-program-options’：传给重新生成标签表程序的命令行选项。
- ‘etags-regen-ignores’：通配符模式列表，指定重新生成标签表时要忽略的文件。

如果你用‘M-x visit-tags-table’（参见“选择标签表”）手动选择了一个标签表，‘etags-regen-mode’实际上会禁用自身：它不再自动创建和更新标签表，因为它假定你更愿意自己管理标签表。调用命令‘tags-reset-tags-tables’可以取消使用‘visit-tags-table’带来的这一效果。

30.4.2.3 Etags 正则表达式

‘etags’的‘--regex’选项允许通过正则表达式匹配来识别标签。你可以把这个选项与文件名混用；每个选项作用于其后出现的源文件。如果指定多个‘--regex’选项，它们会全部并行使用。语法是：

```
--regex=[{LANGUAGE}]/TAGREGEXP/[NAMEREGEXP/]MODIFIERS
```

选项值的核心部分是 TAGREGEXP，即用于匹配标签的正则表达式。它总是以锚定方式使用，也就是说，只在行首匹配。如果你想允许带缩进的标签，请使用能匹配行首空白字符的正则表达式；以‘[\t]*’开头即可。

在这些正则表达式中，‘\’引用下一个字符，并且支持所有 C 字符转义序列：‘\a’表示响铃，‘\b’表示退格，‘\e’表示 escape，‘\f’表示换页，‘\n’表示换行，‘\r’表示回车，‘\t’表示制表符，‘\v’表示垂直制表符。此外，‘\d’代表‘DEL’字符。除此之外，正则表达式语法与 Emacs 相同，只是反斜杠转义与 GNU grep 相同（这意味着，例如不支持 shy group），并且不支持‘[:ascii:]’、‘[:multibyte:]’、‘[:nonascii:]’、‘[:word:]’和‘[:unibyte:]’。

理想情况下，TAGREGEXP 匹配的字符不应多于识别你想要标记的内容所需。如果语法要求你把 TAGREGEXP 写成匹配标签本身之外的更多字符，就应当再加上 NAMEREGEXP，以便仅仅挑出标签。这会让 Emacs 更准确地找到标签，也更可靠地对标签名进行补全。在 NAMEREGEXP 中，使用“反向引用”（参见“正则表达式中的反斜杠”）来引用 TAGREGEXP 中带括号的分组‘\(…\)’往往很方便。例如，‘\1’指第一个这样的带括号分组。下面你可以找到一些例子。

MODIFIERS 是由零个或多个字符组成的序列，用于修改‘etags’进行匹配的方式。不带修饰符的正则表达式按区分大小写的方式，依次作用于输入文件的每一行。修饰符及其含义如下：

- ‘i’：匹配此正则表达式时忽略大小写。
- ‘m’：对整个文件匹配此正则表达式，因此可以进行多行匹配。
- ‘s’：对整个文件匹配此正则表达式，并允许 TAGREGEXP 中的‘.’匹配换行符。

‘-R’选项取消前面所有‘--regex’选项定义的正则表达式。它同样作用于其后出现的文件名。下面是一个例子：

```
etags --regex=/REG1/i voo.doo --regex=/REG2/m \
    bar.ber -R --lang=lisp los.er
```

这里‘etags’根据‘voo.doo’和‘bar.ber’的内容为其选择解析语言。‘etags’还使用 REG1 在‘voo.doo’中识别额外的标签，并使用 REG1 和 REG2 在‘bar.ber’中识别额外的标签。REG1 以不区分大小写的方式逐行检查

‘voo.doo’ 和 ‘bar.ber’，而 REG2 以区分大小写的方式检查整个 ‘bar.ber’ 文件，允许进行多行匹配。对于 ‘los.er’，‘etags’ 只使用 Lisp 的标签规则，不使用任何用户指定的正则表达式匹配。

你可以用可选的前缀 {LANGUAGE} 把 ‘--regex’ 选项限制为只匹配给定语言的文件。（‘etags --help’ 会打印 ‘etags’ 能识别的语言列表。）当把许多预定义的正则表达式存放在一个文件中供 ‘etags’ 使用时，这尤其有用。下面的例子只针对 C 语言，为 Emacs 源文件中的 ‘DEFVAR’ 宏生成标签：

```
--regex='{c}/[ \t]*DEFVAR_[A-Z_ \t(]+"\([^"]+\)"/\1/'
```

当正则表达式很复杂时，你可以把它们存放在文件中。下面的选项语法指示 ‘etags’ 读取两个正则表达式文件。第二个文件中的正则表达式在匹配时不区分大小写。

```
--regex=@CASE-SENSITIVE-FILE --ignore-case-regex=@IGNORE-CASE-FILE
```

用于 ‘etags’ 的正则表达式文件每行含一个正则表达式。空行以及以空格或制表符开头的行会被忽略。当一行中的第一个字符是 ‘@’ 时，‘etags’ 认为该行其余部分是另一个正则表达式文件的名称；因此，这样的文件可以包含另一个文件。所有其他行都被当作正则表达式。如果一行中最先出现的非空白文本是 ‘--’，该行就是注释。

例如，我们可以创建一个名为 ‘emacs.tags’ 的文件，内容如下：

```
        -- This is for GNU Emacs C source files
{c}/[ \t]*DEFVAR_[A-Z_ \t(]+"\([^"]+\)"/\1/
```

然后这样使用它：

```
etags --regex=@emacs.tags *.[ch] */*.[ch]
```

下面还有几个例子。正则表达式都加了引号，以免被 shell 解释。

- 为 Octave 文件生成标签：

```
etags --language=none \
      --regex='/[ \t]*function.*=[ \t]*\([^ \t]*\)[ \t]*(/\1/' \
      --regex='/###key \(.*\)/\1/' \
      --regex='/[ \t]*global[ \t].*/' \
      *.m
```

注意，不会为脚本生成标签，因此如果你想跳转到某个脚本，必须自己添加一行形如 ‘###key SCRIPTNAME’ 的内容。

- 为 Tcl 文件生成标签：

```
etags --language=none --regex='/proc[ \t]+\([^ \t]+\)/\1/' *.tcl
```

- 为 VHDL 文件生成标签：

```
etags --language=none \
  --regex='/[ \t]*\(ARCHITECTURE\|CONFIGURATION\) +[^ ]* +OF/' \
  --regex='/[ \t]*\(ATTRIBUTE\|ENTITY\|FUNCTION\|PACKAGE\
  \( BODY\)?\|PROCEDURE\|PROCESS\|TYPE\)[ \t]+\([^ \t(]+\)/\3/'
```

30.4.3 选择标签表

Emacs 在任何时候至多有一个“选定的”标签表。所有处理标签表的命令都优先使用选定的那一个。要选定标签表，请键入 ‘M-x visit-tags-table’，它把标签表文件名作为参数，默认值为 ‘TAGS’：从默认目录向上递归搜索时遇到的第一个包含名为 ‘TAGS’ 的文件的目录。

如果你启用了 ‘etags-regen-mode’ 全局次模式，它会在需要时自动找到并访问标签表文件。如果你随后手动调用 ‘visit-tags-table’ 来选定标签表，‘etags-regen-mode’ 将禁用标签表的自动重新生成。参见“创建标签表”。

在你尝试使用标签表内容之前，Emacs 实际上并不读取它；‘visit-tags-table’ 所做的只是把文件名存入变量 ‘tags-file-name’，仅此而已。该变量的初始值是 ‘nil’；这个值告诉所有处理标签表的命令，它们必须询问要使用的标签表文件名。

除选定的标签表之外，Emacs 还维护一个供你一起使用的多个标签表的列表。例如，如果你正在开发一个使用某个库的程序，你可能希望程序和库的标签表都可用，这样 Emacs 就能方便地从两者中找到标识符。如果选定的标签表没有某个标识符，或没有提到某个标签命令所需的源文件，该命令就会尝试使用当前标签表列表中的所有其他标签表。

在已经加载了另一个标签表时用 ‘visit-tags-table’ 加载新标签表，会让你做出选择：你可以把新标签表加入当前标签表列表，也可以丢弃当前列表并开始一个新列表。如果开始新列表，新标签表会_代替_其他标签表被使用。如果把新表加入当前列表，它会与其他标签表_一并_被使用。

你可以通过把变量 ‘tags-table-list’ 设为目录名列表，来指定一个确切的标签表列表，像这样：

```
(setq tags-table-list
      '("~/.emacs.d" "/usr/local/lib/emacs/src"))
```

这告诉标签命令去查看 ‘~/.emacs.d’ 目录和 ‘/usr/local/lib/emacs/src’ 目录中的 ‘TAGS’ 文件。使用顺序取决于你当前在哪个文件中，以及哪个标签表提到了该文件。

不要同时设置 ‘tags-file-name’ 和 ‘tags-table-list’。

30.5 Emacs 开发环境

EDE（“Emacs Development Environment”，Emacs 开发环境）是一个包，它简化了用 Emacs 创建、构建和调试大型程序的任务。它在 Emacs 中提供了 IDE（“Integrated Development Environment”，集成开发环境）的部分功能。

本节简要介绍 EDE 的用法。完整细节参见《EDE 手册》。

EDE 实现为一个全局次模式（参见“次模式”）。要启用它，请键入 ‘M-x global-ede-mode’，或单击 ‘Tools’ 菜单中的 ‘Project Support (EDE)’ 项。你也可以在每次启动 Emacs 时启用 EDE，方法是在初始化文件中加入下面这一行：

```
(global-ede-mode t)
```

激活 EDE 会在菜单栏中增加一个名为 'Development' 的菜单。许多 EDE 命令（包括下面描述的命令）都可以从这个菜单调用。

EDE 把文件组织成"项目"（project），项目对应于目录树。"项目根目录"（project root）是项目最顶层的目录。要定义新项目，请访问所需项目根目录中的某个文件，并键入 'M-x ede-new'。该命令会提示输入"项目类型"（project type），它指的是 EDE 用来管理项目的底层方法（参见《EDE 手册》的"Creating a project"）。最常见的项目类型是使用 Makefile 的 'Make'，以及使用 GNU Automake 的 'Automake'（参见《《automake》手册》）。在这两种情况下，EDE 还会创建一个名为 'Project.ede' 的文件，用于存储关于该项目的信息。

一个项目可以包含一个或多个"目标"（target）。目标可以是目标文件、可执行程序或其他类型的文件，它由项目中的一个或多个文件构建而成。

要向项目中添加新"目标"，请键入 'C-c . t'（'M-x ede-new-target'）。该命令还会询问你是否想把当前文件加入该目标，这意味着该目标将由该文件构建。定义目标之后，你可以键入 'C-c . a'（'ede-add-file'）向其中添加更多文件。

要构建某个目标，请键入 'C-c . c'（'ede-compile-target'）。要构建项目中的所有目标，请键入 'C-c . C'（'ede-compile-project'）。EDE 根据文件类型来推测应当如何构建该目标。

30.6 用 Emerge 合并文件

程序员们偶尔会搞错方向，把同一个程序朝两个不同方向修改，这并不罕见。为了从这种混乱中恢复，你需要合并这两个版本。Emerge 使这变得更容易。比较文件的其他方法，参见"比较文件"和参见《Ediff 手册》。

- 差异状态：为每个差异指定状态 A 或 B 来完成合并。
- 合并命令：用于选择差异、更改差异状态等的命令。
- 退出 Emerge：合并完成后该做什么。
- 在 Emerge 中合并：如何为某个差异保留两种选择。
- Emerge 的细节：其他杂项问题。

30.6.1 Emerge 概述

要启动 Emerge，请运行下列四个命令之一：

- 'M-x emerge-files'：合并两个指定文件。
- 'M-x emerge-files-with-ancestor'：参照一个共同祖先合并两个指定文件。
- 'M-x emerge-buffers'：合并两个缓冲区。
- 'M-x emerge-buffers-with-ancestor'：参照第三个缓冲区中的共同祖先合并两个缓冲区。

Emerge 命令比较两个文件或缓冲区，并在三个缓冲区中显示比较结果：每个输入文本一个（"A 缓冲区"和"B 缓冲区"），再加上一个进行合并的缓冲区（"合并缓冲区"）。合并缓冲区显示完整的合并后文本，而不只是差异。凡是两个输入文本不同的地方，你都可以选择把其中哪一个纳入合并缓冲区。

如果取自现有缓冲区的输入被窄化了，Emerge 命令只使用那些缓冲区的可访问部分。参见"窄化"。

如果存在一个共同祖先版本，两个待合并文本都由它派生而来，Emerge 就可以用它来推测哪个选择是正确的。凡是某个当前版本与祖先一致的地方，Emerge 就假定另一个当前版本是有意做出的改动，应当保留在合并后的版本中。如果你想指定共同的祖先文本，请使用带 'with-ancestor' 的命令。这些命令读取三个文件或缓冲区名——变体 A、变体 B 以及共同祖先。

比较完成、缓冲区准备好之后，交互式合并就开始了。你在合并缓冲区中键入特殊的"合并命令"来控制合并（参见"合并命令"）。对于输入文本之间的每一段连续差异，你可以选择保留其中哪一个，或者把它们一起编辑。

合并缓冲区使用一种特殊的主模式，即 Emerge 模式，其中提供了用于做出这些选择的命令。但你也可以用普通的 Emacs 命令编辑该缓冲区。

在任何时刻，Emerge 都只关注某一个特定的差异，称为"选中的"（selected）差异。该差异在三个缓冲区中这样标出：

```
vvvvvvvvvvvvvvvvvvvv
TEXT THAT DIFFERS
^^^^^^^^^^^^^^^^^^^^
```

Emerge 按顺序给所有差异编号，模式行总是显示当前选中差异的编号。

通常，合并缓冲区起初采用文本的 A 版本。但若某个差异的 A 版本与共同祖先一致，则该差异一开始优先采用 B 版本。

退出时，Emerge 会把合并后的文本留在合并缓冲区中。此时你可以用 'C-x C-w' 把它保存到文件。如果给 'emerge-files' 或 'emerge-files-with-ancestor' 一个数字参数，它会用迷你缓冲区读取输出文件名。（这是这些命令读取的最后一个文件名。）此后退出 Emerge 时，合并后的文本就会保存到该输出文件。

通常，Emerge 命令在退出时会保存输出缓冲区到其文件。若用 'C-]' 中止 Emerge，该 Emerge 命令不会保存输出缓冲区，但你可以自行保存。

30.6.2 Emerge 的子模式

你可以从两种给出合并命令的方式中选择一种：快速模式（Fast mode）和编辑模式（Edit mode）。在快速模式下，基本的合并命令都是单个字符，但普通的 Emacs 命令被禁用。如果你只用合并命令，这很方便。在编辑模式下，所有合并命令都以前缀键 'C-c C-c' 开头，同时也可用常规的 Emacs 命令。这允许你编辑合并缓冲区，但会减慢 Emerge 的操作。

用 'e' 切换到编辑模式，用 'C-c C-c f' 切换到快速模式。模式行分别用 'E' 和 'F' 表示编辑模式和快速模式。

Emerge 还有两个附加子模式，影响某些合并命令的行为：自动前进模式（Auto Advance）和跳过优先项模式（Skip Prefers）。

若自动前进模式生效，'a' 和 'b' 命令会前进到下一个差异。只要你是从输入中简单地挑一个备选版本，这就能让你更快地完成合并。模式行用 'A' 表示自动前进模式。

若跳过优先项模式生效，'n' 和 'p' 命令会跳过处于 "prefer-A" 和 "prefer-B" 状态的差异（参见"差异的状态"）。这样你只会看到两个版本都未被推定为正确的差异。模式行用 'S' 表示跳过优先项模式。该模式只在存在祖先版本时才有意义。

用命令‘s a’（‘emerge-auto-advance’）设置或清除自动前进模式。用‘s s’（‘emerge-skip-prefers’）设置或清除跳过优先项模式。这些命令在参数为正时打开该模式，在参数为负或零时关闭该模式，在无参数时切换该模式。

30.6.3 差异的状态

在合并缓冲区中，差异用由‘v’和‘^’字符组成的行标出。每个差异处于以下七种状态之一：

- A：该差异显示的是 A 版本。‘a’命令总是产生此状态；模式行用‘A’表示。
- B：该差异显示的是 B 版本。‘b’命令总是产生此状态；模式行用‘B’表示。
- default-A、default-B：该差异默认显示 A 状态或 B 状态，因为你还没有做出选择。除某个备选版本被优先考虑的差异（见下文）外，所有差异都从 default-A 状态开始（因此合并缓冲区是 A 缓冲区的副本）。

当你选中一个差异时，其状态会从 default-A 或 default-B 变为普通的 A 或 B。因此，被选中的差异从不会有 default-A 或 default-B 状态，这两个状态也从不在模式行中显示。

命令‘d a’选择 default-A 作为默认状态，命令‘d b’选择 default-B。所选的默认状态适用于所有你从未选中过的差异，以及没有哪个备选版本被优先考虑的差异。如果你是顺序地遍历合并，那么你尚未选中的差异就是选中项之后的那些差异。因此，在顺序遍历时，通过在各个区段之间使用‘d a’和‘d b’，你可以有效地把合并缓冲区中某些区段的默认设为 A 版本，另一些区段的默认设为 B 版本。

- prefer-A、prefer-B：该差异显示 A 状态或 B 状态，因为它是“优先的”（preferred）。这意味着你并未做出明确选择，但其中一个备选版本看起来很可能正确，因为另一个备选版本与共同祖先一致。因此，在 A 缓冲区与共同祖先一致之处，B 版本会被优先采用，因为它很可能才是实际被修改过的版本。

这两个状态在模式行中显示为‘A*’和‘B*’。

- combined：该差异显示 A 状态与 B 状态的组合，这是‘x c’或‘x C’命令的结果。

差异一旦处于此状态，‘a’和‘b’命令对它就不起作用，除非你给它们一个数字参数。

模式行将此状态显示为‘comb’。

30.6.4 合并命令

以下是快速模式下的合并命令；在编辑模式下，需先按‘C-c C-c’前缀：

- ‘p’选中前一个差异。
- ‘n’选中下一个差异。
- ‘a’选择该差异的 A 版本。
- ‘b’选择该差异的 B 版本。
- ‘C-u N j’选中编号为 N 的差异。
- ‘.’选中包含光标点的差异。
- ‘q’退出——完成合并。
- ‘C-]’中止——退出合并，并且不保存输出。
- ‘f’进入快速模式。（在编辑模式下，实际是‘C-c C-c f’。）
- ‘e’进入编辑模式。

- ‘l’ 将三个窗口全部重新居中（如同 ‘C-l’）。带参数时，重建默认的三窗口显示。
- ‘-’ 指定前缀数字参数的一部分。
- ‘DIGIT’ 同样指定前缀数字参数的一部分。
- ‘d a’ 从合并缓冲区此处往下，选择 A 版本作为默认。
- ‘d b’ 从合并缓冲区此处往下，选择 B 版本作为默认。
- ‘c a’ 将该差异的 A 版本复制到删除环。
- ‘c b’ 将该差异的 B 版本复制到删除环。
- ‘i a’ 在光标点处插入该差异的 A 版本。
- ‘i b’ 在光标点处插入该差异的 B 版本。
- ‘m’ 围绕该差异设置光标点和标记。
- ‘^’ 将三个窗口全部向下滚动（如同 ‘M-v’）。
- ‘v’ 将三个窗口全部向上滚动（如同 ‘C-v’）。
- ‘<’ 将三个窗口全部向左滚动（如同 ‘C-x <’）。
- ‘>’ 将三个窗口全部向右滚动（如同 ‘C-x >’）。
- ‘|’ 重置三个窗口的水平滚动。
- ‘x 1’ 将合并窗口缩小到一行。（用 ‘C-u l’ 把它恢复为完整大小。）
- ‘x c’ 组合该差异的两个版本（参见“组合两个版本”）。
- ‘x f’ 在 Help 窗口中显示 Emerge 正在操作的文件/缓冲区的名称。（用 ‘C-u l’ 恢复窗口。）
- ‘x j’ 将该差异与下一个差异合并。（‘C-u x j’ 将该差异与前一个差异合并。）
- ‘x s’ 将该差异拆分为两个差异。使用此命令之前，先要在三个缓冲区中把光标点定位到你想要拆分差异的位置。
- ‘x t’ 从差异的顶部和底部裁去相同的行。当 A、B 版本相同而与祖先版本不同时，就会出现这样的行。

30.6.5 退出 Emerge

‘q’ 命令（‘emerge-quit’）完成合并，若你指定了输出文件，则把结果存入该文件。它会把 A、B 缓冲区恢复为原本的内容；若它们是由 Emerge 创建且你未曾修改过，则将其杀掉。它还会禁用合并缓冲区中的 Emerge 命令，因为稍后执行这些命令可能破坏各个缓冲区的内容。

‘C-]’ 中止合并。这意味着退出而不写出输出文件。若你没有指定输出文件，那么中止与完成合并之间并无实质区别。

若 Emerge 命令是由另一个 Lisp 程序调用的，则其返回值为：成功完成时为 ‘t’，中止时为 ‘nil’。

30.6.6 组合两个版本

有时你想把某个差异的两个备选版本*都*保留下来。为此，使用 ‘x c’，它会像这样编辑合并缓冲区：

```
#ifdef NEW
VERSION FROM B BUFFER
#else /* not NEW */
```

```
VERSION FROM A BUFFER
#endif /* not NEW */
```

这个例子用 C 预处理条件指令来分隔两个备选版本，但你可以把变量 'emerge-combine-versions-template' 设成自己选择的字符串，来指定要使用的字符串。在该字符串中，'%a' 表示放置 A 版本的位置，'%b' 表示放置 B 版本的位置。默认设置产生的正是上面所示的结果，它看起来像这样：

```
"#ifdef NEW\n%b#else /* not NEW */\n%a#endif /* not NEW */\n"
```

30.6.7 Emerge 的细节

合并期间，你千万不要自行编辑 A、B 缓冲区。Emerge 会临时修改它们，但最终会把它们恢复原样。

你可以同时进行任意多个合并——只是不要用同一个缓冲区同时作为多个合并的输入，因为这些缓冲区中所作的临时修改会相互干扰。

启动 Emerge 可能耗时较长，因为它需要完整地比较文件。在 'diff' 完成之前，Emacs 无法做别的任何事情。也许将来有人会修改 Emerge，使得在输入文件很大时在后台进行比较——那时你就可以在 Emerge 准备好接受命令之前继续用 Emacs 做其他事情。

建立合并之后，Emerge 会运行钩子 'emerge-startup-hook'。参见"钩子"。

30.7 缺陷引用

用户数量达到一定规模的项目大多会在某种问题跟踪软件中跟踪缺陷报告，这类软件会为每份报告分配一个唯一而简短的编号或标识符。人们用这些编号来引用某个缺陷，例如在修复某缺陷的代码上方的源码注释中、在文档文件中，或在某个邮件列表或 IRC 频道的讨论中。

次模式 'bug-reference-mode' 和 'bug-reference-prog-mode' 会高亮这类缺陷引用，并让你能够顺着它们跳转到项目问题跟踪器中对应的缺陷报告。'bug-reference-prog-mode' 是 'bug-reference-mode' 的变体，只高亮源码注释和字符串中的缺陷引用。

为了工作，缺陷引用模式需要知道缺陷引用的语法（'bug-reference-bug-regexp'），以及可查询缺陷报告的问题跟踪器 URL（'bug-reference-url-format'）。由于这些通常因项目而异，因此把它们指定在参见"按目录的局部变量"或参见"文件中的局部变量"中是合理的。

例如，假设在我们的项目里，通常把缺陷报告的引用写成 bug#1234 或 Bug-1234，且该缺陷在问题跟踪器上的页面是 <https://project.org/issues/1234>，那么下面这个局部变量段就可以了。

```
;; Local Variables:
;; bug-reference-bug-regexp: "\\([Bb]ug[#-]\\([0-9]+\\)\\)"
;; bug-reference-url-format: "https://project.org/issues/%s"
;; End:
```

第一个正则表达式组所捕获的字符串定义了 bug-reference 所创建的覆盖层的边界，即被高亮并变为可点击的那部分。

'bug-reference-bug-regexp' 中第二个正则表达式组所捕获的字符串，用来替换 'bug-reference-url-format' 中的 '%s' 模板。

注意，‘bug-reference-url-format’ 也可以是一个函数，以便应对更复杂的情形，例如需要用缺陷引用的不同部分来区分问题（issue）与合并请求（merge request），从而得到不同的 URL。

自动设置

如果 ‘bug-reference-mode’ 已被激活、‘bug-reference-mode-hook’ 已经运行，而 ‘bug-reference-bug-regexp’ 或 ‘bug-reference-url-format’ 仍为 ‘nil’，则该模式会依次调用 ‘bug-reference-auto-setup-functions’ 中的函数，尝试自动为这两个变量找到合适的值，直到某个函数成功为止。

目前有三类设置函数。

1. 针对受版本控制的文件的设置，可通过变量 ‘bug-reference-forge-alist’ 和 ‘bug-reference-setup-from-vc-alist’ 配置。默认配置能够为以下情形完成设置：以 <https://debbugs.gnu.org> 作为问题跟踪器、问题通常写作 ‘bug#13’（但也考虑许多不同的写法）的 GNU 项目，以及 GitLab、Gitea、SourceHut 和 GitHub 等若干其他种类的软件托管平台。如果你部署了这类平台的自托管实例，把该情况告知 bug-reference 的最简单方式是通过 ‘bug-reference-forge-alist’。

1. 针对根据邮件文件夹/mbox 名称和邮件头字段值进行推测的设置，可通过变量 ‘bug-reference-setup-from-mail-alist’ 配置。支持内置的新闻阅读器与邮件阅读器参见“用 Gnus 收发邮件与新闻”和参见“用 Rmail 读信”。

1. 针对 IRC 频道的设置，可通过变量 ‘bug-reference-setup-from-irc-alist’ 配置。支持内置的 IRC 客户端 Rcirc、*Note Rcirc: (rcirc)Top，以及 ERC，参见《《erc》手册》的“Top, are supported”

对于几乎所有这些模式，只需简单地启用 ‘bug-reference-mode’ 就够了；只有 Rmail 需要略微不同的设置。

```
;; Use VC-based setup if file is under version control.
(add-hook 'prog-mode-hook #'bug-reference-prog-mode)
```

```
;; Gnus (summary & article buffers)
(add-hook 'gnus-mode-hook #'bug-reference-mode)
```

```
;; Rmail
(add-hook 'rmail-show-message-hook #'bug-reference-mode-force-auto-setup)
```

```
;; Rcirc
(add-hook 'rcirc-mode-hook #'bug-reference-mode)
```

```
;; ERC
(add-hook 'erc-mode-hook #'bug-reference-mode)
```

在 Rmail 的情形下，不能用模式钩子，而要把 ‘rmail-show-message-hook’ 与函数 ‘bug-reference-mode-force-auto-setup’ 结合使用，后者会激活 ‘bug-reference-mode’ 并强制进行自动设置。原因在于，在 Rmail 中所有邮件都位于同一个缓冲区，而每当显示另一封邮件时都需要执行一次设置。

为第三方包添加支持

为 bug-reference 添加自动设置支持通常相当直接：写一个不接受参数的设置函数，由它收集所需信息（例如，对于 MUA 而言是 List-Id/To/From/Cc 等邮件头字段的值），然后调用下列辅助函数之一：

- ‘bug-reference-maybe-setup-from-vc’，根据 ‘bug-reference-setup-from-vc-alist’ 进行设置；
- ‘bug-reference-maybe-setup-from-mail’，根据 ‘bug-reference-setup-from-mail-alist’ 进行设置；以及
- ‘bug-reference-maybe-setup-from-irc’，根据 ‘bug-reference-setup-from-irc-alist’ 进行设置。

若设置函数能够为 bug-reference 模式完成设置，它应返回非 ‘nil’ 值；如果该函数所做的最后一件事是调用上面的某个辅助函数，情况就是如此。

最后，必须把该设置函数加入 ‘bug-reference-auto-setup-functions’。

注意，这些自动设置函数应首先检查自身是否适用，例如检查 ‘major-mode’ 的值。

与 debbugs 包的集成

如果你的项目的问题在 <https://debbugs.gnu.org> 服务器上跟踪，你可以用 ‘debbugs’ 包直接在 Emacs 中浏览并回复报告，该包可通过 Package Menu 下载（参见“Emacs Lisp 软件包”）。该包增加了次模式 ‘debbugs-browse-mode’，可以像下面这样在 ‘bug-reference-mode’ 和 ‘bug-reference-prog-mode’ 之上激活它：

```
(add-hook 'bug-reference-mode-hook 'debbugs-browse-mode)
(add-hook 'bug-reference-prog-mode-hook 'debbugs-browse-mode)
```

31 缩写

一个已定义的“缩写”（abbrev）是这样一个单词：插入它时，它会“展开”成另外一些文本。缩写由用户定义，以特定的方式展开。例如，你可以把 ‘foo’ 定义为一个展开成 ‘find outer otter’ 的缩写。此后，只要键入 ‘f o o <SPC>’，就能把 ‘find outer otter ’ 插入缓冲区。

第二类缩写设施称为“动态缩写展开”（dynamic abbrev expansion）。你通过显式命令来使用动态缩写展开：在缓冲区中查找以光标点前的那些字母开头的其他单词，以此来展开它们。参见“动态缩写展开”。

第三类称为“嬉皮展开”（hippie expansion），它推广了缩写展开。参见《Autotype 手册》的“Hippie Expand”。

31.1 缩写概念

“缩写”（abbrev）是定义为展开成某个指定“展开文本”（expansion）的单词。当你在缩写之后插入一个单词分隔符字符时，该缩写就会展开——用它的展开文本替换该缩写。例如，若 ‘foo’ 被定义为展开成 ‘find outer otter’ 的缩写，那么键入 ‘f o o .’ 会插入 ‘find outer otter.’。

只有在启用 Abbrev 模式（一种缓冲区局部的次模式）时，缩写才会展开。禁用 Abbrev 模式不会使缩写定义被遗忘，但在再次启用 Abbrev 模式之前它们不会展开。命令 ‘M-x abbrev-mode’ 切换当前缓冲区中的 Abbrev 模式；带数值参数时，若参数为正则打开 Abbrev 模式，否则关闭（参见“次模式”）。要在所有缓冲区中默认启用 Abbrev 模式，请把变量 ‘abbrev-mode’ 定制为非 ‘nil’ 值（参见“简易自定义界面”）。

缩写可以有“模式特定的”（mode-specific）定义，仅在某一个主模式中有效。缩写也可以有在所有主模式中都有效的“全局”（global）定义。同一个缩写可以既有一个全局定义，又有针对不同主模式的各种模式特定定义。针对当前主模式的模式特定定义会覆盖全局定义。

你可以在编辑会话期间交互地定义缩写，无论 Abbrev 模式是否启用。你也可以把缩写定义列表保存在文件中，以便在以后的会话中重新加载使用。

31.2 定义缩写

- ‘C-x a g’：定义一个缩写，用光标点之前的一个或多个单词作为它的展开文本（‘add-global-abbrev’）。
- ‘C-x a l’：类似，但定义一个专属于当前主模式的缩写（‘add-mode-abbrev’）。
- ‘C-x a i g’：把缓冲区中的某个单词定义为一个缩写（‘inverse-add-global-abbrev’）。
- ‘C-x a i l’：把缓冲区中的某个单词定义为一个模式特定的缩写（‘inverse-add-mode-abbrev’）。
- ‘M-x define-global-abbrev <RET> ABBREV <RET> EXP <RET>’：把 ABBREV 定义为展开成 EXP 的缩写。
- ‘M-x define-mode-abbrev <RET> ABBREV <RET> EXP <RET>’：把 ABBREV 定义为展开成 EXP 的模式特定缩写。
- ‘M-x kill-all-abbrevs’：丢弃所有缩写定义，留下空白状态。

定义缩写的通常方式是：先输入你想要的展开文本，把光标点移到它之后，然后键入 ‘C-x a g’（‘add-global-abbrev’）。该命令用迷你缓冲区读取缩写本身，然后把它定义为光标点之前一个或多个单词的缩写。用数值

参数指定把光标点之前的多少个单词作为展开文本。例如，要定义前面提到的缩写 ‘foo’，请插入文本 ‘find outer otter’，然后键入 ‘C-u 3 C-x a g f o o <RET>’。

如果你在使用 ‘transient-mark-mode’（默认即如此），活动的区域将被用作所定义缩写的展开文本。否则，给 ‘C-x a g’ 一个零参数即表示使用区域的内容。

命令 ‘C-x a l’（‘add-mode-abbrev’）与此类似，但它为当前主模式定义一个模式特定的缩写。参数的作用与 ‘C-x a g’ 相同。

‘C-x a i g’（‘inverse-add-global-abbrev’）和 ‘C-x a i l’（‘inverse-add-mode-abbrev’）完成相反的任务：若缩写文本已经在缓冲区中，你就用这些命令在迷你缓冲区中指定展开文本，从而定义一个缩写。这些命令会展开用于该定义的那个缩写文本。

使用命令 ‘define-global-abbrev’，你可以既不在缓冲区中插入缩写、也不插入它的展开文本而定义一个缩写。它读取两个参数——缩写及其展开文本。命令 ‘define-mode-abbrev’ 对模式特定的缩写做同样的事。

要改变某个缩写的定义，只需建立一个新的定义即可。当某个缩写已有先前的定义时，缩写定义命令会在替换它之前请求确认。

要删除某个缩写定义，请给缩写定义命令一个负参数：‘C-u - C-x a g’ 或 ‘C-u - C-x a l’。前者删除全局定义，后者删除模式特定的定义。‘M-x kill-all-abbrevs’ 删除所有缩写定义，包括全局的和局部的。

31.3 控制缩写展开

启用 Abbrev 模式时，只要某个缩写恰好位于缓冲区中光标点之前，而你键入了一个自插入的空白或标点字符（<SPC>、逗号等），该缩写就会展开。更精确地说，任何不是单词构成成分的字符都会展开缩写，而任何单词构成成分字符都可以作为缩写的一部分。使用缩写最常见的方式是：插入缩写，然后插入一个标点或空白字符使其展开。

缩写展开会保留大小写形式：‘foo’ 展开为 ‘find outer otter’，而 ‘Foo’ 展开为 ‘Find outer otter’。默认情况下 ‘FOO’ 展开为 ‘Find Outer Otter’，但若你把变量 ‘abbrev-all-caps’ 改为非 ‘nil’ 值，它就会展开为 ‘FIND OUTER OTTER’。

以下命令用于控制缩写展开：

- ‘M-’’：把前缀与后面要展开的缩写分开（‘abbrev-prefix-mark’）。
- ‘C-x a e’：展开光标点之前的缩写（‘expand-abbrev’）。即使未启用 Abbrev 模式，该命令也有效。
- ‘M-x unexpand-abbrev’：撤销上一次展开的缩写的展开。
- ‘M-x expand-region-abbrevs’：展开区域中发现的全部或部分缩写。

你可能希望展开某个缩写并把一个前缀附加到展开文本上；例如，若 ‘cnst’ 展开为 ‘construction’，你可能想用它输入 ‘reconstruction’。键入 ‘recnst’ 是不行的，因为它不一定是一个已定义的缩写。你可以做的是：在前缀 ‘re’ 与缩写 ‘cnst’ 之间使用命令 ‘M-’’（‘abbrev-prefix-mark’）。先插入 ‘re’，然后键入 ‘M-’’；这会在缓冲区中插入一个连字符，表示它已经完成了自己的工作。接着插入缩写 ‘cnst’，此时缓冲区中包含 ‘re-cnst’。现在插入一个非单词字符，把缩写 ‘cnst’ 展开为 ‘construction’。这个展开步骤还会删掉那个表示已使用过 ‘M-’’ 的连字符。结果就是想要的 ‘reconstruction’。

如果你真正想要的是缓冲区中出现缩写的原文，而不是它的展开文本，可以用 'C-q' 插入后面的标点来做到这一点。因此，'foo C-q ,' 会在缓冲区中留下 'foo,'，而不展开它。

如果你误展开了某个缩写，可以键入 'C-/'（'undo'）来撤销这次展开。参见"撤销"。这会撤销缩写展开文本的插入，并恢复缩写原文。若你想要的结果是终止用的非单词字符加上未展开的缩写，就必须重新插入该终止字符，并用 'C-q' 引用它。你也可以使用命令 'M-x unexpand-abbrev' 取消上一次展开，同时不删除终止字符。

'M-x expand-region-abbrevs' 在区域中搜索已定义的缩写，并对找到的每一个询问是否要用其展开文本替换。如果你在用缩写输入文本时忘了先打开 Abbrev 模式，这个命令就有用。它也可能与一组特殊的缩写定义配合，用于一次完成多处全局替换。即使未启用 Abbrev 模式，该命令也有效。

函数 'expand-abbrev' 通过调用 'abbrev-expand-function' 所指定的函数来完成展开。通过改变这个函数，你可以对缩写展开做任意的改动。参见《Emacs Lisp 参考手册》的"Abbrev Expansion"。

31.4 缩写建议

当你手动输入文本，而当前存在一个对此有效的已定义缩写时，你可以得到缩写建议。例如，若存在一个展开为 'find outer otter' 的缩写 'foo'，而你手动输入了 'find outer otter'，Emacs 可以注意到这一点，并在你停止键入时于回显区显示提示。

要启用缩写建议功能，请把选项 'abbrev-suggest' 定制为非 'nil' 值。

变量 'abbrev-suggest-hint-threshold' 控制何时向用户建议某个缩写。该变量定义了 Emacs 建议使用某缩写所需的最小节省量（以用户不必键入的字符数计）。例如，若用户键入 'foo bar'（七个字符），而有一个已定义的缩写 'fubar'（五个字符），那么除非阈值设为数字 2 或更小，用户不会得到任何建议。在默认值 3 之下，此例中用户不会得到任何建议，因为使用该缩写节省的字符数低于阈值。如果你总是想得到缩写建议，请把该变量的值设为零。

命令 'abbrev-suggest-show-report' 会显示一个缓冲区，其中列出当前编辑会话期间显示过的所有缩写建议。如果你得到了若干条缩写建议而又记不全，这会很有用。

31.5 查看与编辑缩写

- 'M-x list-abbrevs'：显示所有缩写定义的列表。带数值参数时，只列出局部缩写。
- 'M-x edit-abbrevs'：编辑缩写列表；你可以添加、修改或删除定义。

'M-x list-abbrevs' 的输出类似这样：

```
VARIOUS OTHER TABLES...
(python-mode-skeleton-abbrev-table)
"class" (sys)   0 "" python-skeleton-class
(lisp-mode-abbrev-table)
"ks"          0    "keymap-set"
(global-abbrev-table)
"dfn"         0    "definition"
```

（一些没有语义意义的空行，以及另一些缩写表，都被省略了。）

括号中包含名字的一行是某个特定缩写表中各缩写的标题行；‘global-abbrev-table’ 包含所有全局缩写，其他以主模式命名的缩写表则包含模式特定的缩写。

在每个缩写表中，每一非空行都定义一个缩写。行首的单词是缩写。紧随其后的数字是该缩写已被展开的次数。Emacs 记录这一信息是为了帮你看出自己实际使用了哪些缩写，以便删掉那些不常用的。行末的字符串是展开文本。

有些缩写带有 ‘(sys)’ 标记。这些“系统缩写”（参见《Emacs Lisp 参考手册》的“Abbrevs”）由各种模式预先定义，不会被保存到你的缩写文件中。要禁用某个系统缩写，请定义一个同名、展开为其自身的缩写，并把它保存到你的缩写文件中。系统缩写带有一个关联的钩子函数，调用它来执行缩写展开；该函数的名字跟在 ‘list-abbrevs’ 所显示缓冲区中的缩写展开文本之后。

‘M-x edit-abbrevs’ 让你可以在 Emacs 缓冲区中编辑缩写列表，从而添加、修改或删除缩写定义。该列表的格式与上面描述的相同。缩写缓冲区名为 ‘*Abbrevs*’，处于 Edit-Abbrevs 模式。在该缓冲区中键入 ‘C-c C-c’，即可按缓冲区中的规定安装缩写定义——并删除任何未被列出的缩写定义。

命令 ‘edit-abbrevs’ 实际上与 ‘list-abbrevs’ 相同，区别在于前者会选中缓冲区 ‘*Abbrevs*’，而 ‘list-abbrevs’ 只是在另一个窗口中显示它。

31.6 保存缩写

以下命令让你在各次编辑会话之间保留缩写定义。

- ‘M-x write-abbrev-file <RET> FILE <RET>’：写出一个描述所有已定义缩写的文件 FILE。
- ‘M-x read-abbrev-file <RET> FILE <RET>’：读取文件 FILE，并按其中规定定义缩写。
- ‘M-x define-abbrevs’：根据当前缓冲区中的定义来定义缩写。
- ‘M-x insert-abbrevs’：把所有缩写及其展开文本插入当前缓冲区。

‘M-x write-abbrev-file’ 用迷你缓冲区读取文件名，然后把当前所有缩写定义的描述写入该文件。这用于保存缩写定义，供以后的会话使用。存入文件的文本是一系列 Lisp 表达式，执行它们就会定义你当前拥有的那些缩写。

‘M-x read-abbrev-file’ 用迷你缓冲区读取文件名，然后读取该文件，按文件内容定义缩写。函数 ‘quietly-read-abbrev-file’ 与此类似，只是它不在回显区显示消息；你不能交互地调用它，它主要用于你的初始化文件（参见“Emacs 初始化文件”）。若这两个函数中的任何一个以 ‘nil’ 作为参数被调用，它就使用变量 ‘abbrev-file-name’ 所给出的文件，该变量默认为 ‘~/.emacs.d/abbrev_defs’。这是你的标准缩写定义文件，Emacs 启动时会自动从它加载缩写。（作为例外，以批处理模式启动时 Emacs 不会加载该缩写文件。关于批处理模式的说明，参见“初始选项”。）

如果你改动了任何缩写，那么每当 Emacs 提议保存所有文件时（针对 ‘C-x s’ 或 ‘C-x C-c’），它也会提议自动保存缩写。它把它们保存在 ‘abbrev-file-name’ 所指定的文件中。把变量 ‘save-abbrevs’ 设为 ‘nil’ 可以禁止这一特性；把它设为 ‘silently’，则会在不询问的情况下自动保存缩写。

命令 ‘M-x insert-abbrevs’ 和 ‘M-x define-abbrevs’ 与前面几个命令类似，但作用于 Emacs 缓冲区中的文本。‘M-x insert-abbrevs’ 在光标点之后把描述当前所有缩写定义的文本插入当前缓冲区；‘M-x define-abbrevs’ 解析整个当前缓冲区，并据此定义缩写。

31.7 动态缩写展开

上面描述的缩写设施在你插入文本时自动工作，但所有缩写都必须显式定义。相比之下，“动态缩写”允许从缓冲区的内容自动确定缩写的含义，但动态缩写展开只在你显式请求时才发生。

- ‘M-/’：通过搜索以该缩写开头的单词，把光标点之前缓冲区中的单词作为“动态缩写”展开（‘dabbrev-expand’）。
- ‘C-M-/’：把光标点之前的单词作为一个动态缩写进行补全（‘dabbrev-completion’）。

例如，若缓冲区中包含 ‘does this follow ’，而你键入 ‘f o M-/’，效果是插入 ‘follow’，因为它是缓冲区中以 ‘fo’ 开头的最后一个单词。给 ‘M-/’ 一个数值参数，表示取从光标点向后查找时遇到的第二个、第三个等不同的展开。重复 ‘M-/’ 会向更早的位置查找，以寻找另一种展开。扫描完光标点之前的全部文本之后，它会扫描光标点之后的文本。变量 ‘dabbrev-limit’ 若非 ‘nil’，则指定在缓冲区中搜索展开文本的范围有多远。

扫描完当前缓冲区之后，‘M-/’ 通常还会搜索其他缓冲区。变量 ‘dabbrev-check-all-buffers’ 和 ‘dabbrev-check-other-buffers’ 可用来确定是否搜索其他缓冲区、搜索哪些其他缓冲区。主模式派生自 ‘dabbrev-ignored-buffer-modes’ 中任一模式的缓冲区会被忽略。

要更精细地控制扫描哪些缓冲区，可定制变量 ‘dabbrev-ignored-buffer-names’ 和 ‘dabbrev-ignored-buffer-regexps’。前者的值是要跳过的缓冲区名列表。后者的值是正则表达式列表；若某个缓冲区的名字与其中任一正则表达式匹配，动态缩写展开就会跳过该缓冲区。

给 ‘M-/’ 一个负参数，例如 ‘C-u - M-/’，表示先搜索光标点之后的展开，再搜索其他缓冲区，而把光标点之前的展开只当作最后的选择。如果你重复 ‘M-/’ 以寻找另一种展开，请不要指定参数。重复 ‘M-/’ 会依次循环光标点之后的所有展开，然后是光标点之前的展开。

展开某个动态缩写之后，你可以复制其原始上下文中紧跟该展开文本的更多单词。对每个想要复制的单词，只需键入 ‘<SPC> M-/’。单词之间的空格和标点也会随单词一起复制。

你可以控制 ‘M-/’ 确定要展开的单词的方式以及展开方式，参见“定制动态缩写”。

命令 ‘C-M-/’（‘dabbrev-completion’）完成动态缩写的补全。它不逐个尝试可能的展开，而是找出全部展开，然后插入它们共有的文本。如果它们没有共有文本，‘C-M-/’ 会显示一个补全列表，你可以按通常方式从中选择一个。参见“补全”。

动态缩写展开与 Abbrev 模式完全无关；用 ‘M-/’ 展开某个单词，与该单词是否被定义为普通缩写完全无关。

31.8 定制动态缩写

通常，动态缩写展开在搜索展开文本时会忽略大小写。也就是说，展开文本不必与你要展开的单词在大小写上一致。

这一特性由变量 ‘dabbrev-case-fold-search’ 控制。若它为 ‘t’，则在此搜索中忽略大小写；若为 ‘nil’，则单词与展开文本必须在大小写上匹配。若值为 ‘case-fold-search’（默认值），则由变量 ‘case-fold-search’ 控制搜索展开文本时是否忽略大小写（参见“搜索时的宽松匹配”）。

通常，动态缩写展开会保留*你所展开的动态缩写的*大小写模式，即把展开文本转换成该大小写模式。

变量 'dabbrev-case-replace' 控制是否保留动态缩写的大小写模式。若它为 't'，则在多数情况下保留动态缩写的大小写模式；若为 'nil'，则展开文本总是逐字照抄。若值为 'case-replace'（默认值），则由变量 'case-replace' 控制是否逐字照抄展开文本（参见"替换命令与宽松匹配"）。

不过，若展开文本包含复杂的混合大小写模式，而动态缩写与之匹配到相同程度，那么无论上述变量如何设置，展开文本总是逐字照抄。因此，例如，若缓冲区中包含 'variableWithSillyCasePattern'，而你键入 'v a M-/'，它会逐字照抄该展开文本，包括其大小写模式。

变量 'dabbrev-abbrev-char-regexp' 若非 'nil'，则控制就动态展开而言哪些字符被视为单词的一部分。该正则表达式必须只匹配一个字符，绝不能匹配两个或更多字符。同一个正则表达式还决定哪些字符是展开文本的一部分。（默认）值 'nil' 有特殊含义：动态缩写（即光标点处的单词）由单词构成字符组成，但它们的展开文本则按单词字符和符号字符的序列来查找。这通常适用于展开程序源码中的符号，也适用于许多语言的自然语言文本，但在包含异常标点字符的文本缓冲区中可能并非你所想要的；在这种情况下，值 '"\\sw"' 可能产生更好的结果。

在 shell 脚本和 makefile 中，变量名有时带 '$' 前缀，有时不带。用于这类文本的主模式可以通过设置变量 'dabbrev-abbrev-skip-leading-regexp' 来定制动态缩写展开，以处理可选前缀。它的值应当是一个正则表达式，匹配动态缩写展开应当忽略的可选前缀。默认值为 'nil'，表示不跳过任何字符。

32 Dired 目录编辑器

Dired 会生成一个 Emacs 缓冲区，其中包含某个目录（以及可选的若干子目录）的列表。你可以用普通的 Emacs 命令在这个缓冲区中移动，并用专门的 Dired 命令对列出的文件进行操作。Dired 既适用于本地目录，也适用于远程目录。

Dired 缓冲区通常是只读的，不允许在其中插入文本（不过 Wdired 模式允许这样做，参见“编辑 Dired 缓冲区”）。像 ‘d’ 和 ‘x’ 这样的普通打印字符被重新定义为特殊的 Dired 命令。有些 Dired 命令会“标记”（mark）或“加旗标”（flag）“当前文件”（即当前行上的文件）；另一些命令则对已标记的文件或已加旗标的文件进行操作。你先标记某些文件，为的是用一条命令对它们全部进行操作。

Dired-X 包为 Dired 模式提供了各种额外特性。参见《《dired-x》手册》

你也可以用 ‘C-x C-d’（‘list-directory’）查看某个目录中的文件列表。与 Dired 不同，该命令不允许你对列出的文件进行操作。参见“文件目录”。

32.1 进入 Dired

要调用 Dired，键入 ‘C-x d’（‘dired’）。它用迷你缓冲区读取目录名，并打开一个列出该目录中文件的“Dired 缓冲区”。你也可以在迷你缓冲区参数中给出通配符文件名模式，此时 Dired 缓冲区会列出所有匹配该模式的文件。通配符也可以出现在目录部分。例如，

```
C-x d  ~/foo/*.el  <RET>
C-x d  ~/foo/*/*.el  <RET>
```

前者列出目录 ‘foo’ 中所有扩展名为 ‘.el’ 的文件。后者列出 ‘foo’ 的所有子目录中扩展名为 ‘.el’ 的文件。

在 Posix 系统上，当系统 shell 支持“globstar”（一种递归通配特性）且该支持已启用时，你可以在 Dired 中使用递归通配：

```
C-x d  ~/foo/**/*.el  <RET>
```

此命令会生成一份目录列表，其中包含所有扩展名为 ‘.el’ 的文件，并递归地深入 ‘foo’ 的所有子目录。注意，不同 shell 对 globstar 的实现之间存在细微差异。请查阅你的 shell 手册以了解预期行为。

如果 shell 支持 globstar，但该支持默认被禁用，你仍可通过把 ‘dired-maybe-use-globstar’ 定制为非 ‘nil’ 值来让 Dired 使用这一特性；此后 Dired 会为那些它知道如何启用的 shell 启用 globstar（这些 shell 的列表见 ‘dired-enable-globstar-in-shell’）。

迷你缓冲区中可以使用通常的历史和补全命令；特别是，‘M-n’ 会把所访问文件的名称（如果有的话）放入迷你缓冲区（参见“迷你缓冲区历史”）。

你也可以通过给 ‘C-x C-f’（‘find-file’）提供目录名来调用 Dired。

你可以让 Emacs 对任意缓冲区的 default-directory（参见“文件名”）调用 Dired，方法是键入 ‘C-x C-j’（‘dired-jump’）。如果该缓冲区访问的是某个文件，此命令会把光标点移到它显示的 Dired 缓冲区中该文件所在的行；否则，光标点会落在目录列表的第一个文件上。作为例外，如果你在 Dired 缓冲区中键入 ‘C-x C-j’，

Emacs 会显示父目录的目录列表，并把光标点放在与你调用 'dired-jump' 时所在目录相对应的行上。键入 'C-x 4 C-j'（'dired-jump-other-window'）效果相同，但在一个新窗口中显示 Dired 缓冲区（参见"'display-buffer' 的工作方式"）。

变量 'dired-listing-switches' 指定列出目录时传给 'ls' 的选项；该字符串 *必须* 含 '-l'。如果对 'dired' 命令使用前缀参数，你就可以在输入目录规范之前用迷你缓冲区指定 'ls' 开关。无论以何种方式指定，'ls' 开关都可以包含不需要参数的短选项（即单个字符），以及参数用 '=' 指定的长选项（以 '--' 开头）。

你可以把 'dired-listing-switches' 声明为连接局部变量，以便调整其值，使其符合远程系统的预期（参见"按连接的局部变量"）。

当文件名中含有换行符时，Dired 默认把它显示为字面换行符，因此该文件名的显示会在 Dired 缓冲区中占据不止一行。如果你对这样的文件列表调用 Dired 操作，许多情况下会失败并报错。因此，当 Dired 显示含有字面换行符的文件名时，Emacs 会识别出这一点并自动弹出一个带有提示信息的警告缓冲区。对于这类文件名，Dired 提供了另一种显示方式，即使用 'ls' 开关 '-b'，此时换行符表示为 '\n'，Dired 对该文件的列表照常只占一行，于是你可以对它执行所有适用的 Dired 操作。(1)

Emacs 提供两种不同方式让 Dired 使用 '-b' 开关：

- 你可以在调用 'dired' 之前把 '-b' 加入 'dired-listing-switches'。由于该变量是用户选项，你可以用定制界面（参见"保存自定义"）或在初始化文件中使用 'setopt' 宏（参见"查看与设置变量"）来持久地改变其值。(2) 你也可以只对下一次 'dired' 调用添加 '-b'，方法是键入 'C-u C-x d'。

- 如果你启用用户选项 'dired-auto-toggle-b-switch'，那么当你访问某个目录、而该目录中有文件名带换行符的文件时，Emacs 会自动添加 '-b' 开关并重新显示该目录，在文件名中显示 '\n'。如果你编辑该文件名并去掉 '\n' 字符，那么在编辑完成时 Emacs 会自动移除 '-b' 开关并重新显示 Dired 缓冲区，于是文件名中的制表符或空格现在会显示为不带反斜杠的字面空格。如果你在访问了含有带换行符文件名的目录之后才启用或禁用 'dired-auto-toggle-b-switch'，Emacs 会酌情添加或移除 '-b' 开关并自动重新显示 Dired 缓冲区。

Dired 会在模式行中显示调用 'ls' 时所用开关的说明。默认情况下，Dired 会尝试判断这些开关表示按名称排序还是按日期排序，并在模式行中说明。如果 'dired-switches-in-mode-line' 变量的值为 'as-is'，则原样显示开关。如果该变量的值为整数，则开关显示会被截断到该长度。该变量也可以是函数，调用时以 'dired-actual-switches' 作为唯一参数，并且应返回要显示在模式行中的字符串。

如果你的 'ls' 程序支持 '--dired' 选项，Dired 会自动把该选项传给它；这会让 'ls' 对某些不常见的文件名输出特殊转义序列，否则 Dired 将无法解析这些名称。在某个 Emacs 会话中第一次运行 Dired 时，它会带该选项调用一次 'ls'，以检查 'ls' 是否支持 '--dired' 选项。如果退出码为 0，Dired 随后就会使用 '--dired' 选项；否则不使用。你可以通过定制变量 'dired-use-ls-dired' 来禁止这项检查。值 'unspecified'（默认值）表示执行检查；任何其他非 'nil' 值表示使用 '--dired' 选项；'nil' 表示不使用 '--dired' 选项。

在 MS-Windows 和 MS-DOS 系统上，以及在某些远程系统上，Emacs 会模拟 'ls'。有关这种模拟的选项和特殊之处，参见"在 MS-Windows 上模拟 'ls'"。

要在另一个窗口中显示 Dired 缓冲区，使用 'C-x 4 d'（'dired-other-window'）。'C-x 5 d'（'dired-other-frame'）在单独的框架中显示 Dired 缓冲区。参见"'display-buffer' 的工作方式"。

键入‘q’（‘quit-window’）会把 Dired 缓冲区埋入后台，并且如果该窗口是为该缓冲区专门创建的，还会删除该窗口。

> 注：⑴ 注意，使用‘-b’开关时，Dired 会把文件名中的制表符显示为‘\t’，并对文件名中的其他控制字符以及空格用‘\’进行转义。

> 注：⑵ 如果你在对含有带换行符文件名的目录调用 dired 时，‘dired-listing-switches’中已含有‘-b’，那么换行符会显示为‘\n’，因此 Emacs 不会弹出警告。

32.2 在 Dired 缓冲区中导航

所有通常的 Emacs 光标移动命令在 Dired 缓冲区中都可用。键‘C-n’和‘C-p’被重新定义为分别运行‘dired-next-line’和‘dired-previous-line’，它们把光标放在行上文件名的开头，而不是放在行首。

为方便起见，Dired 中的 <SPC> 和‘n’等价于‘C-n’。‘p’等价于‘C-p’。（按行移动在 Dired 中如此常见，理应易于键入。）<DEL>（向上移动并取消旗标）也常常仅用于向上移动（参见“用 Dired 删除文件”）。

‘j’（‘dired-goto-file’）用迷你缓冲区提示输入文件名，并把光标点移到 Dired 缓冲区中描述该文件的行。

‘M-s f C-s’（‘dired-isearch-filenames’）在 Dired 缓冲区中执行向前增量搜索，只在文件名中查找匹配，而忽略缓冲区中的其余文本。‘M-s f M-C-s’（‘dired-isearch-filenames-regexp’）做同样的事，但使用正则表达式搜索。如果你把变量‘dired-isearch-filenames’改为‘t’，那么通常的搜索命令也会把自己限制在文件名中；例如‘C-s’的行为就像‘M-s f C-s’。如果值为‘dwim’，那么只有当光标点最初位于文件名上时，搜索命令才匹配文件名。有关增量搜索的信息，参见“搜索与替换”。

当 Dired 缓冲区包含多个目录时，还有一些额外的导航命令可用。参见“在子目录间移动”。

32.3 用 Dired 删除文件

Dired 最常见的用途之一，是先为文件“加旗标”以待删除，再删除已加旗标的文件。

‘d’为该文件加删除旗标（‘dired-flag-file-deletion’）。‘u’移除删除旗标（‘dired-unmark’）。‘<DEL>’把光标点移到上一行并移除该行的删除旗标（‘dired-unmark-backward’）。‘x’删除已加删除旗标的文件（‘dired-do-flagged-delete’）。

你可以移到描述某文件的行并键入‘d’（‘dired-flag-file-deletion’）来为该文件加删除旗标。删除旗标显示为行首的‘D’。此命令会把光标点移到下一行，因此重复使用‘d’命令可以依次为多个文件加旗标。数值前缀参数用作重复次数；负计数表示为前面的文件加旗标。

如果区域是活动的，‘d’命令会为区域中的所有文件加删除旗标；此时该命令不移动光标点，并忽略任何前缀参数。

为文件加删除旗标而不是立即删除文件，是为了降低误删文件的风险。在你指示 Dired 删除已加旗标的文件之前，可以用‘u’和 <DEL> 命令移除删除旗标。‘u’（‘dired-unmark’）的工作方式与‘d’类似，但移除旗标而不是加旗标。<DEL>（‘dired-unmark-backward’）向上移动并移除旗标；它相当于带参数 −1 的‘u’。给这两个命令的数值前缀参数用作重复次数，负计数表示向相反方向取消旗标。如果区域是活动的，这两个命令则改为取消区域中所有文件的旗标，而不移动光标点。

要删除已加旗标的文件，键入 'x'（'dired-do-flagged-delete'）。此命令会显示所有已加删除旗标的文件名列表，并要求用 'yes' 确认。如果你确认，Dired 会删除这些已加旗标的文件，然后从 Dired 缓冲区的文本中删除它们的行。行数有所减少的 Dired 缓冲区仍保持选中状态。

如果在要求确认时你回答 'no' 或用 'C-g' 退出，你会立即返回 Dired，缓冲区中仍保留删除旗标，且实际没有删除任何文件。

你可以像删除其他文件一样删除空目录，但通常 Dired 不能删除非空目录。不过，如果变量 'dired-recursive-deletes' 为非 'nil'，那么 Dired 可以删除非空目录及其全部内容。这可能有一定风险。如果该变量的值为 'always'，Dired 将递归删除非空目录，这风险更大。

即使你把 'dired-recursive-deletes' 设为 'nil'，有时你可能也想递归删除目录，而不必对每个目录都请求确认。例如，当你标记了许多目录以待删除，并且非常确定它们都能安全删除时。对于每一个要求你确认删除的非空目录，如果你回答 'all'，那么所有剩余的目录都将被删除，不再有任何进一步询问。

如果你把变量 'delete-by-moving-to-trash' 改为 't'，上述删除命令会把受影响的文件或目录移入操作系统的回收站，而不是直接删除它们。参见"其他文件操作"。

另一种删除文件的方式是用 'm' 标记它们，再用 'D' 删除，参见"对文件进行操作"。

32.4 一次为多个文件加旗标

'#'、'~'、'.'、'% &' 和 '% d' 命令依据文件名一次为多个文件加删除旗标：

- '#' 为所有自动保存文件（名称以 '#' 开头和结尾的文件）加删除旗标（参见"自动保存：灾难防护"）。
- '~' 为所有备份文件（名称以 '~' 结尾的文件）加删除旗标（参见"备份文件"）。
- '.'（句点）为多余的编号备份文件加删除旗标。任何一个文件最旧和最新的少数几个备份文件会被豁免；中间的会被加旗标。
- '% &' 为所有具有某些类型名称的文件加删除旗标，这些名称暗示你可以很容易地重新创建这些文件。
- '% d REGEXP <RET>' 为所有名称匹配正则表达式 REGEXP 的文件加删除旗标。

'#'（'dired-flag-auto-save-files'）为所有名称看起来像自动保存文件的文件加旗标——即名称以 '#' 开头和结尾的文件。参见"自动保存：灾难防护"。

'~'（'dired-flag-backup-files'）为所有名称表明是备份文件的文件加旗标——即名称以 '~' 结尾的文件。参见"备份文件"。

'.'（句点，'dired-clean-directory'）只为一部分备份文件加删除旗标：对任何一个文件，除了最旧的几个和最新的几个备份之外，其余的都加旗标。通常，每个文件保留的最新版本数量由变量 'dired-kept-versions' 给出（*不是* 'kept-new-versions'；后者只在保存时适用）。要保留的最旧版本数量由变量 'kept-old-versions' 给出。

带正数值参数的句点命令，如 'C-u 3 .'，指定要保留的最新版本数量，覆盖 'dired-kept-versions'。负数值参数覆盖 'kept-old-versions'，用该参数值的相反数指定每个文件要保留的最旧版本数量。

'% &'（'dired-flag-garbage-files'）为名称匹配变量 'dired-garbage-files-regexp' 所指定正则表达式的文件加旗标。默认情况下，它匹配 TeX 产生的某些文件、'.bak' 文件，以及 'patch' 产生的 '.orig' 和 '.rej' 文件。

‘% d’ 为所有名称匹配指定正则表达式的文件加旗标（‘dired-flag-files-regexp’）。匹配时只使用文件名的非目录部分。你可以用 ‘^’ 和 ‘$’ 锚定匹配。在使用 ‘% d’ 时，你可以通过隐藏某些子目录来把它们排除在标记之外。参见“隐藏子目录”。

32.5 在 Dired 中访问文件

Dired 中有几个命令用于访问或查看 Dired 缓冲区中列出的文件。它们都作用于当前行的文件；如果该文件实际上是一个目录，这些命令会在该子目录上调用 Dired（创建一个单独的 Dired 缓冲区）。

- ‘f’ 访问当前行描述的文件，如同键入 ‘C-x C-f’ 并提供该文件名（‘dired-find-file’）。参见“访问文件”。
- ‘<RET>’ ‘e’ 等价于 ‘f’。
- ‘o’ 与 ‘f’ 类似，但使用另一个窗口（参见“‘display-buffer’ 的工作方式”）来显示该文件的缓冲区（‘dired-find-file-other-window’）。Dired 缓冲区仍在第一个窗口中保持可见。这类似于用 ‘C-x 4 C-f’ 访问文件。参见“多窗口”。
- ‘C-o’ 访问当前行描述的文件，并在另一个窗口中显示其缓冲区，但不选择那个窗口（‘dired-display-file’）。
- ‘mouse-1’ ‘mouse-2’ 访问你点击其名称的文件（‘dired-mouse-find-file-other-window’）。这使用另一个窗口显示该文件，如同 ‘o’ 命令。
- ‘v’ 以 View 模式查看当前行描述的文件（‘dired-view-file’）。View 模式提供便利的命令来浏览缓冲区，但禁止修改它；参见“视图模式”。
- ‘@’ 以超级用户（root）权限打开当前行描述的文件（‘tramp-dired-find-file-with-sudo’）。如果带 ‘C-u’ 前缀参数调用，它会提示输入默认值 ‘sudo’ 之外的另一种 Tramp 方法。参见《Tramp 手册》的“Ad-hoc multi-hops”。
- ‘^’ 访问当前目录的父目录（‘dired-up-directory’）。这等价于移到 ‘..’ 所在行并在那里键入 ‘f’。

用户选项：dired-kill-when-opening-new-dired-buffer

在 Dired 中访问新的子目录时，Emacs（默认）会打开一个新缓冲区来显示这个新目录，并让旧的 Dired 缓冲区保持原样。如果该用户选项为非 ‘nil’，则旧的 Dired 缓冲区会在选定新目录之后被杀死。这意味着如果你在 Dired 中遍历目录结构，最终不会有多于一个 Dired 缓冲区。

32.6 Dired 标记与旗标之别

你可以不用 ‘D’ 为文件加旗标，而是用其他某个字符（通常是 ‘*’）*来“标记”文件。大多数对文件进行操作的 Dired 命令都使用以 * 标记的文件。*唯一对已加旗标的文件进行操作的命令是 ‘x’，它删除这些文件。

下面是一些用 ‘*’ 标记、取消标记以及对标记进行操作的命令。（有关为文件加旗标和取消旗标的命令，参见“用 Dired 删除文件”。）

- ‘m’ ‘* m’ 用 ‘*’ 标记当前文件（‘dired-mark’）。如果区域是活动的，则改为标记区域中的所有文件；否则，如果给出了数值参数 N，则改为标记从当前文件开始的接下来 N 个文件（如果 N 为负，则标记前面 −N

个文件）。如果在子目录标题行上调用（参见“Dired 中的子目录”），此命令会标记该子目录中的所有文件。

- ‘* N’ 报告已标记文件的数量和大小（‘dired-number-of-marked-files’）。
- ‘* *’ 用 ‘*’ 标记所有可执行文件（‘dired-mark-executables’）。带数值参数时，取消所有这些文件的标记。
- ‘* @’ 用 ‘*’ 标记所有符号链接（‘dired-mark-symlinks’）。带数值参数时，取消所有这些文件的标记。
- ‘* /’ 用 ‘*’ 标记所有作为目录的文件，但 ‘.’ 和 ‘..’ 除外（‘dired-mark-directories’）。带数值参数时，取消所有这些文件的标记。
- ‘* s’ 标记当前子目录中的所有文件，‘.’ 和 ‘..’ 除外（‘dired-mark-subdir-files’）。
- ‘u’ ‘* u’ 移除此行上的任何标记（‘dired-unmark’）。如果区域是活动的，则改为取消区域中所有文件的标记；否则，如果给出了数值参数 N，则改为取消从当前文件开始的接下来 N 个文件的标记（如果 N 为负，则取消前面 −N 个文件的标记）。
- ‘<DEL>’ ‘* <DEL>’ 把光标点移到上一行并移除该行上的任何标记（‘dired-unmark-backward’）。如果区域是活动的，则改为取消区域中所有文件的标记；否则，如果给出了数值参数 N，则改为取消当前文件之前的 N 个文件的标记（如果 N 为负，则取消接下来的 −N 个文件的标记）。
- ‘* !’ ‘U’ 移除此 Dired 缓冲区中所有文件的所有标记（‘dired-unmark-all-marks’）。
- ‘* ? MARKCHAR’ ‘M-<DEL>’ 移除所有使用字符 MARKCHAR 的标记（‘dired-unmark-all-files’）。如果带 ‘M-<DEL>’ 调用，该命令会提示输入 MARKCHAR。MARKCHAR 是单个字符——不要用 <RET> 来结束它。参见下面 ‘* c’ 命令的说明，它让你把一个标记字符替换为另一个。

带数值参数时，此命令会逐个询问每个已标记的文件，询问是否移除其标记。你可以回答 ‘y’ 表示是，‘n’ 表示否，或 ‘!’ 表示不再询问而移除剩余文件的标记。

- ‘* C-n’ ‘M-}’ 向下移到下一个已标记的文件（‘dired-next-marked-file’）。只要文件有任意种类的标记，它就算“已标记”。
- ‘* C-p’ ‘M-{’ 向上移到上一个已标记的文件（‘dired-prev-marked-file’）。
- ‘t’ ‘* t’ 切换所有标记（‘dired-toggle-marks’）：用 ‘*’ *标记的文件变为未标记，未标记的文件被* ‘*’ 标记。以任何其他方式标记的文件不受影响。
- ‘* c OLD-MARKCHAR NEW-MARKCHAR’ 把所有使用字符 OLD-MARKCHAR 的标记替换为使用字符 NEW-MARKCHAR 的标记（‘dired-change-marks’）。此命令是创建或使用 ‘*’ 或 ‘D’ 之外的标记的主要方式。参数是单个字符——不要用 <RET> 来结束它们。

借助此命令，你几乎可以用任何字符作为标记字符，以区分各类文件。如果 OLD-MARKCHAR 是空格（‘ ’），那么该命令作用于所有未标记的文件；如果 NEW-MARKCHAR 是空格，那么该命令会取消它所作用文件的标记。

为说明此命令的强大之处，下面演示如何给所有没有标记的文件加上 'D' 旗标，同时取消所有已有 'D' 旗标的文件的旗标：

```
* c D t  * c <SPC> D  * c t <SPC>
```

这里假设没有文件已经被 't' 标记。

- '% m REGEXP <RET>' '* % REGEXP <RET>' 用 *'' 标记所有名称匹配正则表达式 REGEXP 的文件（'dired-mark-files-regexp'）。此命令与 '% d' 类似，区别是用 '* 标记文件，而不是用 'D' 加旗标。*

匹配时只使用文件名的非目录部分。用 '^' 和 '$' 锚定匹配。你可以通过临时隐藏子目录来排除它们（参见"隐藏子目录"）。

- '% g REGEXP <RET>' 用 '*' 标记所有 *内容* 中含有与正则表达式 REGEXP 匹配的文本的文件（'dired-mark-files-containing-regexp'）。此命令与 '% m' 类似，区别是搜索文件内容而不是文件名。注意，如果某个文件已在 Emacs 缓冲区中访问过，并且 'dired-always-read-filesystem' 为 'nil'（默认值），此命令会在该缓冲区中查找而不重新访问文件，因此如果文件内容自上次访问以来已改变，结果可能与磁盘上的文件不一致。如果你不希望这样，可以在调用此命令之前，恢复缓冲区中已访问的文件，或在那些缓冲区中开启 Auto-Revert 模式。参见"还原缓冲区"。如果你希望此命令总是重新访问文件，而不必由你恢复文件或启用 Auto-Revert 模式，可以把 'dired-always-read-filesystem' 设为非 'nil'。

'C-/'、'C-x u'、'C-_' 撤销 Dired 缓冲区中的更改，例如添加或删除标记（'dired-undo'）。此命令不会还原实际的文件操作，也不会恢复丢失的文件！它只是撤销缓冲区自身的更改。与 'undo' 一样，数字前缀参数指定重复次数。

有时，在对文件进行操作的命令之后使用此命令会带来麻烦。例如，在重命名一个或多个文件之后，'dired-undo' 会在 Dired 缓冲区中恢复原来的名字，这会让 Dired 缓冲区与目录的实际内容不同步。

'touchscreen-hold' 进入"点击选择"模式（'dired-click-to-select-mode'），在该模式下，在文件名上使用鼠标键 'mouse-2' 会切换其标记。此模式在使用触摸屏设备进行文件管理时很有用。

当在文件名上检测到"按住"（hold）手势（*note Touchscreens::）时会启用它；一旦某个操作被标记文件的 Dired 命令结束，它就会自动禁用。

32.7 对文件进行操作

本节介绍对一个或多个文件进行操作的基本 Dired 命令。这些命令中有许多是大写字母；它们全都在执行之前使用迷你缓冲区，或读取参数，或请求确认。它们都允许你按以下方式指定要操作的文件：

- 若给命令一个数字前缀参数 N，它就从当前文件开始，对接下来 N 个文件进行操作。（若 N 为负，则命令对当前行之前的 −N 个文件进行操作。）
- 否则，若某些文件用 '*' 标记，则命令对所有这些文件进行操作。
- 否则，命令只对当前文件进行操作。

某些其他 Dired 命令（如 '!' 和 '%' 系列命令）也用同样的约定来决定对哪些文件操作。

除了这里描述的 Dired 命令，你还可以对 Dired 缓冲区中显示的一个或多个文件调用版本控制（VC）命令。参见“版本控制”。

那些要求给出目标目录的命令（例如复制、重命名文件或为它们创建链接的命令）会尝试猜测该操作的默认目标目录。通常它们建议 Dired 缓冲区的默认目录，但若选项 ‘dired-dwim-target’ 非 ‘nil’，且某个窗口中显示着另一个 Dired 缓冲区，则改为建议那个缓冲区的目录。你可以自定义 ‘dired-dwim-target’，让它优先选择下一个显示 Dired 缓冲区的窗口，或最近使用过的显示 Dired 缓冲区的窗口，或使用任何其他函数。当值是一个函数时，它会被无参数调用，并应返回一个目录列表用作默认值（即默认目标和“future history”）。

以下是操作文件的文件操作类 Dired 命令。

‘C NEW <RET>’ 复制指定的文件（‘dired-do-copy’）。参数 NEW 是要复制到的目录，或者（若只复制一个文件）新名字。这类似 shell 命令 ‘cp’。

选项 ‘dired-create-destination-dirs’ 控制 Dired 在复制/重命名文件时是否应在目标位置创建不存在的目录。默认值 ‘nil’ 表示 Dired 从不创建这种缺失的目录；值 ‘always’ 表示 Dired 自动创建它们；值 ‘ask’ 表示 Dired 在创建前向你请求确认。

若除 ‘dired-create-destination-dirs’ 之外，选项 ‘dired-create-destination-dirs-on-trailing-dirsep’ 也非 ‘nil’，则目标目录末尾的目录分隔符会被特殊处理。这种情况下，当复制到 ‘test/’ 且尚不存在目录 ‘test’ 时，会创建该目录，并把指定的源文件或目录复制到新建的目录中。

若 ‘dired-copy-preserve-time’ 非 ‘nil’，则用此命令复制时会保留原文件的修改时间，如同 ‘cp -p’。

变量 ‘dired-recursive-copies’ 控制是否递归复制目录（类似 ‘cp -r’）。默认值为 ‘top’，意思是递归复制目录前先询问。

变量 ‘dired-copy-dereference’ 控制是把符号链接作为链接复制，还是在解引用后复制（类似 ‘cp -L’）。默认值为 ‘nil’，意思是符号链接通过创建新的符号链接来复制。

用户选项 ‘dired-keep-marker-copy’ 控制此命令如何处理文件标记。默认行为是用 ‘C’ 标记所有新复制出的文件。

‘D’ 删除指定的文件（‘dired-do-delete’）。这类似 shell 命令 ‘rm’。

与本节的其它命令一样，此命令对标记过的文件，或接下来 N 个文件进行操作。相比之下，‘x’（‘dired-do-flagged-delete’）删除所有“已加旗标”的文件。

‘E’ 用外部程序“打开”指定的文件（‘dired-do-open’）。程序按系统惯例选择，由变量 ‘shell-command-guess-open’ 决定。

‘R NEW <RET>’ 重命名指定的文件（‘dired-do-rename’）。若重命名单个文件，参数 NEW 是文件的新名字。若重命名多个文件，参数 NEW 是要把文件移入的目录（这类似 shell 命令 ‘mv’）。

选项 ‘dired-create-destination-dirs’ 控制 Dired 是否应创建 NEW 中不存在的目录。

若除 ‘dired-create-destination-dirs’ 之外还设置了选项 ‘dired-create-destination-dirs-on-trailing-dirsep’，则它控制目标末尾的目录分隔符是否被特殊处理。这种情况下，当把目录 ‘old’ 重命名为 ‘new/’ 且尚不存在目录 ‘new’ 时，会创建该目录并把 ‘old’ 移入新建的目录。否则，‘old’ 被重命名为 ‘new’。

Dired 会自动修改与被重命名文件相关联的缓冲区所访问的文件名，使它们指向新名字。

若变量 ‘dired-vc-rename-file’ 的值非 ‘nil’，则使用底层 VCS 的命令重命名文件，即通过 ‘vc-rename-file’（*note VC Delete/Rename::）。

‘H NEW <RET>’ 为指定的文件创建硬链接（‘dired-do-hardlink’）。这类似 shell 命令 ‘ln’。参数 NEW 是要在其中创建链接的目录，或者（若只创建一个链接）给链接起的名字。

‘S NEW <RET>’ 为指定的文件创建符号链接（‘dired-do-symlink’）。这类似 ‘ln -s’。参数 NEW 是要在其中创建链接的目录，或者（若只创建一个链接）给链接起的名字。

‘Y NEW <RET>’ 为指定的文件创建相对符号链接（‘dired-do-relsymlink’）。参数 NEW 是要在其中创建链接的目录，或者（若只创建一个链接）给链接起的名字。这类似 ‘dired-do-symlink’，但创建的是相对符号链接。例如：

```
foo -> ../bar/foo
```

它不会创建这样的绝对链接：

```
foo -> /path/that/may/change/any/day/bar/foo
```

‘M MODESPEC <RET>’ 更改指定文件的模式（也称“权限位”）（‘dired-do-chmod’）。MODESPEC 可以用八进制或符号记法，如同 ‘chmod’ 程序处理的参数。此命令不跟随符号链接，因此在符号链接的模式不可变的平台上，若你试图更改符号链接的模式，它会报错。

‘G NEWGROUP <RET>’ 把指定文件的组改为 NEWGROUP（‘dired-do-chgrp’）。

‘O NEWOWNER <RET>’ 把指定文件的所有者改为 NEWOWNER（‘dired-do-chown’）。（在大多数系统上，只有超级用户能做这件事。）

变量 ‘dired-chown-program’ 指定用于完成此项工作的程序名。（这个变量是必要的，因为不同系统把 ‘chown’ 放在不同的位置。）

‘T TIMESTAMP <RET>’ 触碰（touch）指定的文件（‘dired-do-touch’）。也就是说，把它们的修改时间更新为 TIMESTAMP，默认是当前时间。这类似 shell 命令 ‘touch’。

‘P COMMAND <RET>’ 打印指定的文件（‘dired-do-print’）。你必须指定打印用的命令，但迷你缓冲区会以利用变量 ‘lpr-command’ 和 ‘lpr-switches’ 做出的合适猜测作为初始内容（与 ‘lpr-buffer’ 所用的变量相同；参见“打印”）。

‘Z’ 压缩指定的文件（‘dired-do-compress’）。若文件看起来已经是压缩文件，则改为解压它。每个被标记的文件都被压缩成各自的归档；若 ‘gzip’ 程序可用就用它，否则用 ‘compress’。

对目录名执行此命令时，会根据用户选项 ‘dired-compress-directory-default-suffix’ 生成压缩归档。默认是包含该目录所有文件的 ‘.tar.gz’ 归档，做法是运行 ‘tar’ 命令并把输出通过管道传给 ‘gzip’。为了能解压压缩过的目录，在 ‘.tar.gz’ 或 ‘.tgz’ 归档文件上输入 ‘Z’ 会把归档中的所有文件解包到一个目录中，该目录的名字是去掉扩展名后的归档名。

‘c’ 把指定的文件压缩（‘dired-do-compress-to’）到文件系统上任意位置的单个归档中。默认归档由用户选项 ‘dired-compress-directory-default-suffix’ 控制。另请参见 ‘dired-compress-files-alist’。

‘C-x v v’、‘C-x v = (Dired)’、‘C-x v l (Dired)’ 若你访问的目录处于版本控制之下（参见“版本控制”），则常规的 VC 命令会对选中的文件进行操作。例如，‘C-x v v’ 调用 ‘dired-vc-next-action’，它在访问受版本控制文件的缓冲区中与 ‘vc-next-action’ 做同样的事（参见“版本控制下的基本编辑”）。类似地，‘C-x v =’ 显示被标记文件与其已提交版本之间的差异，‘C-x v l’ 显示被标记文件的 VC 更改历史。

‘:d’ 解密指定的文件（‘epa-dired-do-decrypt’）。参见《EasyPG 手册》的“Dired integration”。

‘:v’ 验证指定文件上的数字签名（‘epa-dired-do-verify’）。参见《EasyPG 手册》的“Dired integration”。

‘:s’ 对指定的文件进行数字签名（‘epa-dired-do-sign’）。参见《EasyPG 手册》的“Dired integration”。

‘:e’ 加密指定的文件（‘epa-dired-do-encrypt’）。参见《EasyPG 手册》的“Dired integration”。

‘L’ 加载指定的 Emacs Lisp 文件（‘dired-do-load’）。参见“Emacs 的 Lisp 代码库”。

‘B’ 字节编译指定的 Emacs Lisp 文件（‘dired-do-byte-compile’）。参见《Emacs Lisp 参考手册》的“Byte Compilation”

‘I’ 对此文件运行 Info（假定它是 Info 格式的文件）。

‘N’ 对此文件运行 man（假定它是 ‘nroff’ 格式的文件）。

‘A REGEXP <RET>’ 在所有指定的文件中搜索正则表达式 REGEXP（‘dired-do-find-regexp’）。

此命令是 ‘xref-find-references’ 的变体（*note Identifier Search::），它显示 ‘*xref*’ 缓冲区，你可以在其中借助 *note Xref Commands:: 中描述的命令在各匹配之间移动，并按需显示它们。

若被标记的文件中有目录，则此命令会递归地搜索这些目录中的所有文件及其所有子目录，但文件名匹配 ‘grep-find-ignored-files’ 的文件和名字匹配 ‘grep-find-ignored-directories’ 的子目录除外。

‘Q REGEXP <RET> TO <RET>’ 对每个指定的文件执行 ‘query-replace-regexp’，把与 REGEXP 匹配的内容替换为字符串 TO（‘dired-do-find-regexp-and-replace’）。

此命令是 ‘xref-query-replace-in-results’ 的变体。它呈现一个 ‘*xref*’ 缓冲区，列出 REGEXP 的所有匹配，你可以使用该缓冲区中的特殊命令（*note Xref Commands::）。特别地，若你退出查询替换循环，可以在该缓冲区中用 ‘r’ 替换更多匹配。参见“用标识符搜索与替换”。

与 ‘dired-do-find-regexp’ 类似，若被标记的文件中有目录，则此命令会递归地在这些目录及其所有子目录的所有文件中执行替换，但文件名匹配 ‘grep-find-ignored-files’ 的文件和名字匹配 ‘grep-find-ignored-directories’ 的子目录除外。

32.8 Dired 中的 shell 命令

Dired 命令 ‘!’（‘dired-do-shell-command’）在迷你缓冲区中读取一个 shell 命令字符串，并在一个或多个文件上运行该 shell 命令。shell 命令所操作的文件按 Dired 命令的通常方式确定（参见“对文件进行操作”）。命令 ‘X’ 是 ‘!’ 的同义词。

命令‘&’（‘dired-do-async-shell-command’）做同样的事，只是它异步地运行 shell 命令。（你也可以通过在 shell 命令末尾追加‘&’字符来用‘!’做到这一点。）当命令对多个文件操作时，它会运行指定 shell 命令的多个并行副本，每个文件一个。作为例外，若指定的 shell 命令以‘;’或‘;&’结尾，则该 shell 命令会依次在每个文件上后台运行；Emacs 等待每个调用的 shell 命令结束，然后再运行下一个。

对‘!’和‘&’两者而言，shell 命令的工作目录都是 Dired 缓冲区的顶层目录。

若你让‘!’或‘&’对多个文件操作，shell 命令字符串决定这些文件如何传给 shell 命令：

- 若在命令字符串中使用两侧有空白的‘*’，*则命令只运行一次，以文件名列表替换* ‘*’。文件名的顺序就是它们在 Dired 缓冲区中出现的顺序。

因此，‘! tar cf foo.tar * <RET>’对整个文件名列表运行‘tar’，把它们放进一个 tar 文件‘foo.tar’中。

若你想把‘*’*当作两侧带空白的 shell 通配符使用，可写成* ‘"*"’。在 shell 中这等价于‘*’*；但由于* ‘*’两侧没有空白，Dired 不会特殊对待它。这样做时 Emacs 会请求确认，除非‘dired-confirm-shell-command’为‘nil’。

- 否则，若命令字符串包含两侧有空白的‘?’或‘`?`’，Emacs 会为每个文件各运行一次 shell 命令，每次都把当前文件名替换到‘?’和‘`?`’处。你可以在命令中多次使用‘?’和‘`?`’；同一个文件名替换每一处出现。若把它们与‘*’混用，命令会报错。

- 若命令字符串既不包含‘*’，也不包含‘?’或‘`?`’，Emacs 会为每个文件运行一次 shell 命令，并把文件名加在末尾。例如，‘! uudecode <RET>’对每个文件运行‘uudecode’。

若要以更复杂的方式遍历文件名，你可能更愿意使用显式的 shell 循环。例如，下面是对每个文件进行 uuencode、通过在输入文件名后追加‘.uu’来生成输出文件名的方法：

```
for file in * ; do uuencode "$file" "$file" >"$file".uu; done
```

用‘`?`’记法写成的同一例子：uuencode ? ? > `?`.uu

‘!’和‘&’命令不会尝试更新 Dired 缓冲区来显示新增或修改过的文件，因为它们并不知道会改动哪些文件。请用‘g’命令更新 Dired 缓冲区（参见“更新 Dired 缓冲区”）。

关于在 Dired 之外运行 shell 命令的信息，参见“单条 Shell 命令”。

32.9 猜测 shell 命令

Dired 会根据文件名尝试猜测你可能想对它应用什么 shell 命令。例如，若光标点位于名为‘foo.tar’的文件上，你按下‘!’，Dired 会猜你想运行‘tar xvf’，并把它作为默认 shell 命令建议给你。

你可以输入‘M-n’把默认值取到迷你缓冲区中编辑。若某个文件有多个命令，则多次输入‘M-n’可以依次看到每个匹配的命令。

Dired 只尝试为单个文件猜测命令，从不为被标记的文件列表猜测。

-- 变量：dired-guess-shell-alist-default 此变量规定用于猜测适用于某些文件的 shell 命令的预定义规则。把它设为‘nil’可关闭猜测。由用户定义的‘dired-guess-shell-alist-user’中的元素会覆盖这些规则。

-- 变量：dired-guess-shell-alist-optional 此变量类似 'dired-guess-shell-alist-default'，但包含针对各种媒体格式的外部查看器和播放器。把它设为 'nil' 会关闭猜测。变量 'dired-guess-shell-alist-user' 和 'dired-guess-shell-alist-default' 会覆盖这些规则。

-- 变量：dired-guess-shell-alist-user 若此变量非 'nil'，它规定用户定义的文件正则表达式及其建议命令的关联列表。运行 'dired-do-shell-command' 时，这些规则的优先级高于变量 'dired-guess-shell-alist-default' 和 'dired-guess-shell-alist-optional' 中的预定义规则。默认值为 'nil'。

关联列表的每个元素形如

```
(REGEXP COMMAND...)
```

其中每个 COMMAND 可以是字符串，也可以是求值为字符串的 Lisp 表达式。若给出多个命令，它们都会临时被压入历史列表。

shell 命令中的 *''代表与 REGEXP 匹配的文件名。当 Emacs 调用 COMMAND 时，它把每一处 '* 替换为匹配到的文件名。

要为 '.foo' 和 '.bar' 文件扩展名添加规则，请把下面这段加入你的初始化文件：

```
(setq dired-guess-shell-alist-user
      (list
       (list "\\.foo$" "FOO-COMMAND")  ; fixed rule
       ;; possibly more rules...
       (list "\\.bar$"  ; rule with condition test
             '(if CONDITION
                  "BAR-COMMAND-1"
                "BAR-COMMAND-2"))))
```

这会覆盖相同扩展名的任何预定义规则。

你可以用 'M-x customize-group <RET> dired-guess <RET>' 找到更多用户选项。

32.10 在 Dired 中转换文件名

本节介绍以系统化方式改动文件名的 Dired 命令。每个命令都对部分或全部被标记的文件进行操作，用转换现有名字得到的新名字。

与基本的 Dired 文件操作命令一样（参见"对文件进行操作"），这里描述的命令要么对接下来 N 个文件操作，要么对所有用 '*' 标记的文件操作，要么对当前文件操作。（要给文件加标记，请使用 参见"Dired 标记与旗标之别" 中描述的命令。）

本节描述的所有命令都是交互式工作的：它们会为每个候选文件请求你确认该操作。因此，你可以选中比你实际需要操作的更多的文件（例如用一个能匹配许多文件的正则表达式），然后在命令提示确认时输入 'y' 或 'n' 来筛选选中的名字。

'% u' 把每个选中的文件重命名为大写名字（'dired-upcase'）。若旧文件名为 'Foo' 和 'bar'，新名字为 'FOO' 和 'BAR'。

‘% l’ 把每个选中的文件重命名为小写名字（‘dired-downcase’）。若旧文件名为 ‘Foo’ 和 ‘bar’，新名字为 ‘foo’ 和 ‘bar’。

‘% R FROM <RET> TO <RET>’、‘% C FROM <RET> TO <RET>’、‘% H FROM <RET> TO <RET>’、‘% S FROM <RET> TO <RET>’、‘% Y FROM <RET> TO <RET>’ 这五个命令分别重命名、复制、创建硬链接、创建软链接和创建相对软链接，每种情况下都通过对旧文件名做正则表达式替换来算出新名字。

这四个正则表达式替换命令实际上是对选中的文件名执行查找并替换。它们读取两个参数：正则表达式 FROM 和替换模式 TO；它们用 FROM 匹配每个旧文件名，然后把匹配的部分替换为 TO。你可以在 TO 中使用 ‘\&’ 和 ‘\DIGIT’ 来引用模式在旧文件名中匹配到的全部或部分内容，如同 ‘replace-regexp’ 中那样（参见“正则表达式替换”）。若正则表达式在一个文件名中匹配多次，只有第一个匹配会被替换。

例如，‘% R ^.$ <RET> x-\& <RET>’ *通过在名字前加上 ‘x-’ 来重命名每个选中的文件。反过来，从每个文件名前面删掉 ‘x-’ 也是可行的：一种方法是 ‘% R ^x-\(.\)$ <RET> \1 <RET>’*；另一种是 ‘% R ^x- <RET> <RET>’。（用 ‘^’ 和 ‘$’ 来锚定应当覆盖整个文件名的匹配。）

通常，替换过程不考虑文件的目录名；它只作用于目录内的文件名。若你指定数字参数为零，则替换会影响包含目录名在内的整个绝对文件名。（非零参数指定要操作的文件数。）

你可能想用之后操作这些文件时所用的同一个正则表达式 FROM 来选出要操作的文件集。要做到这一点，先用 ‘% m FROM <RET>’ 标记那些文件，然后在操作文件的命令中使用同一个正则表达式。为了方便，操作文件的 ‘%’ 命令会把任何 ‘%’ 命令中最后指定的正则表达式用作默认值。

32.11 用 Dired 比较文件

‘=’（‘dired-diff’）命令用 ‘diff’ 程序把当前文件（光标点处的文件）与另一个文件（用迷你缓冲区读取）相比较。用迷你缓冲区指定的文件是 ‘diff’ 的第一个参数，光标点处的文件是第二个参数。‘diff’ 程序的输出显示在一个使用 Diff 模式的缓冲区中（参见“比较文件”）。

若区域处于活动状态，则用迷你缓冲区读取的文件默认是标记处的文件（即普通的 Emacs 标记，而非 Dired 标记；参见“设置标记”）。否则，若光标点处的文件有备份文件（参见“备份文件”），则那是默认值。

你也可以用 ‘ediff-files’ 比较文件，参见《Ediff 手册》的“Major Entry Points”。

32.12 Dired 中的子目录

Dired 缓冲区通常只显示一个目录，但你也可以选择把它的子目录一并包含进来。

在一个 Dired 缓冲区中包含多个目录的最简单方法，是为运行 ‘ls’ 指定选项 ‘-lR’。（若在运行 Dired 时给出数字参数，就可以在迷你缓冲区中指定这些选项。）这样会产生递归的目录列表，显示所有层级的所有子目录。

更常见的情况是，你只想显示特定的子目录。你可以用 ‘i’（‘dired-maybe-insert-subdir’）做到这一点：

‘i’ 在缓冲区后面插入子目录的内容。

如果在描述某个文件（该文件是个目录）的行上使用这条命令，它会把该目录的内容插入同一个 Dired 缓冲区，并移动到那里。插入的子目录内容跟在 Dired 缓冲区的顶层目录之后，就像 ‘ls -lR’ 的输出那样。

如果该子目录的内容已经存在于缓冲区中，‘i’ 命令就只是移动到那里。

无论哪种情况，‘i’ 都会在移动之前设置 Emacs 标记，因此 ‘C-u C-<SPC>’ 可以回到你在 Dired 缓冲区中先前的位置（参见“设置标记”）。你也可以用 ‘^’ 回到同一个 Dired 缓冲区中的父目录（参见“在 Dired 中访问文件”）。

用 ‘l’ 命令（‘dired-do-redisplay’）更新子目录的内容，并在子目录标题行上使用 ‘C-u k’ 删除子目录列表（参见“更新 Dired 缓冲区”）。你也可以隐藏和显示已插入的子目录（参见“隐藏子目录”）。

32.13 Dired 中的子目录开关

使用 ‘C-u i’ 时，可以在 Dired 缓冲区中插入带指定 ‘ls’ 开关的子目录。使用 ‘C-u l’ 可以更改光标点处已插入子目录的 ‘ls’ 开关。

如果还原缓冲区，Dired 会保留这些开关。删除某个子目录则会忘掉它的开关。

用 ‘dired-undo’（参见“Dired 标记与旗标之别”）重新插入或删除以显式开关插入的子目录，可能会绕过 Dired 记住（或忘掉）开关的机制。用 ‘dired-undo’ 删除子目录不会忘掉它的开关。此后用 ‘i’ 重新插入时，会使用它原来的开关重新插入。用 ‘dired-undo’ 重新插入一个用常规 Dired 命令（而非 ‘dired-undo’）删除的子目录时，最初会用它原来的开关插入。不过，还原缓冲区时会用缓冲区的默认开关重新列出它。如果这造成了问题，你可以很容易地用 ‘C-u i’ 或 ‘C-u l’ 纠正。

Dired 不记住 ‘R’ 开关。插入带 ‘R’ 开关（以及其余开关）的子目录，等价于用所有剩余的开关分别插入它的每个子目录。例如，更新或删除一个用 ‘R’ 开关插入的子目录，不会更新或删除它的子目录。

缓冲区的默认开关不影响用显式指定的开关插入的子目录。特别地，像 ‘s’ 这样更改缓冲区开关的命令不影响这类子目录。（不过，它们影响没有显式指定开关的子目录。）

你可以用 ‘M-x dired-reset-subdir-switches’ 让 Dired 忘掉所有子目录开关，并用缓冲区的默认开关重新列出所有子目录。这同时也会还原 Dired 缓冲区。

32.14 在子目录间移动

当 Dired 缓冲区列出子目录时，你可以使用翻页移动命令 ‘C-x [’ 和 ‘C-x]’ 按整个目录移动（参见“页”）。

以下命令在同一个 Dired 缓冲区内的目录树中横向、向上和向下移动。它们移动到“目录标题行”（directory header line），即目录内容开头给出目录名的那一行。

‘C-M-n’ 移动到下一个子目录标题行，不论层级（‘dired-next-subdir’）。

‘C-M-p’ 移动到上一个子目录标题行，不论层级（‘dired-prev-subdir’）。

‘C-M-u’ 向上移动到父目录的标题行（‘dired-tree-up’）。

‘C-M-d’ 在目录树中向下移动，移动到第一个子目录的标题行（‘dired-tree-down’）。

‘<’ 向上移动到前一个目录文件行（‘dired-prev-dirline’）。这些行把目录描述为其父目录中的一个文件。

‘>’ 向下移动到下一个目录文件行（‘dired-next-dirline’）。

‘M-G’ 提示输入一个目录，并移动到它的目录文件行（‘dired-goto-subdir’）。

32.15 隐藏子目录

“隐藏”一个子目录，是指让它不可见，只保留它的标题行。

‘$’ 隐藏或显示光标点所在的子目录，并把光标点移到下一个子目录（‘dired-hide-subdir’）。这是一个开关。数字参数用作重复次数。

‘M-$’ 隐藏此 Dired 缓冲区中的所有子目录，只留下它们的标题行（‘dired-hide-all’）。或者，如果当前有任何子目录被隐藏，就让所有子目录重新可见。在很深的目录树中，你可以用这条命令获得概览，或快速移动到远处的子目录。

常规 Dired 命令从不考虑隐藏子目录内的文件。例如，对已标记文件进行操作的命令会忽略隐藏目录中的文件，即使它们被标记了。因此，你可以用隐藏来临时把某些子目录排除在操作之外，而不必去掉这些子目录中文件上的 Dired 标记。

关于如何插入子目录列表，参见“Dired 中的子目录”；关于如何删除它，参见“更新 Dired 缓冲区”。

32.16 更新 Dired 缓冲区

本节介绍用于更新 Dired 缓冲区、使其反映目录和文件在 Dired 之外发生的变化的命令，以及删除 Dired 缓冲区一部分的命令。

‘g’ 更新 Dired 缓冲区的全部内容（‘revert-buffer’）。

‘l’ 更新指定的文件（‘dired-do-redisplay’）。指定 ‘l’ 所用文件的方式与文件操作命令相同。

‘k’ 删除指定的*文件行*——不是文件，只是这些行（‘dired-do-kill-lines’）。

‘s’ 在字母顺序与日期/时间顺序之间切换（‘dired-sort-toggle-or-edit’）。

‘C-u s SWITCHES <RET>’ 使用 SWITCHES 作为 ‘dired-listing-switches’ 刷新 Dired 缓冲区。

键入 ‘g’（‘revert-buffer’），根据所列目录和文件的变化更新 Dired 缓冲区的内容。这会保留所有标记，但已消失文件上的标记除外。隐藏的子目录会被更新，但仍保持隐藏。

若要只更新部分文件，键入 ‘l’（‘dired-do-redisplay’）。与 Dired 的文件操作命令一样，这条命令作用于接下来的 N 个文件（或前 −N 个文件），若有已标记的文件则作用于它们，否则作用于当前文件。更新文件是指读取它们的当前状态，然后更新缓冲区中相应的行以反映该状态。

如果在子目录标题行上使用 ‘l’，它会更新相应子目录的内容。

如果你用 ‘C-x d’ 或其他 Dired 命令访问一个已经在某个 Dired 缓冲区中显示的目录，Dired 会切换到那个缓冲区，但不会更新它。如果该缓冲区不是最新的，Dired 会显示警告，告诉你键入 ‘g’ 来更新它。你也可以通过把变量 ‘dired-auto-revert-buffer’ 设为非 ‘nil’ 值，让 Emacs 在你重新访问每个 Dired 缓冲区时自动还原它。

要从缓冲区中删除*文件行*——而不实际删除文件——键入 ‘k’（‘dired-do-kill-lines’）。与文件操作命令一样，这条命令作用于接下来的 N 个文件，若有已标记的文件则作用于它们。不过，它不作用于当前文件，因为否则误按 ‘k’ 会很恼人。

如果你用 ‘k’ 删除某个目录文件的目录行，而该目录是你作为子目录插入 Dired 缓冲区的（参见“Dired 中的子目录”），它也会移除相应的子目录列表。在子目录的标题行上键入 ‘C-u k’，也会从 Dired 缓冲区中移除该子

目录行。

‘g’ 命令会恢复你以这种方式删除的各个单独的行，但不会恢复子目录——你必须用 ‘i’ 重新插入子目录。

Dired 缓冲区中的文件通常按文件名的字母顺序列出。Dired 也可以按日期/时间排序。Dired 命令 ‘s’（‘dired-sort-toggle-or-edit’）在这两种排序方式之间切换。Dired 缓冲区的模式行会显示当前是按名称还是按日期排序。

‘C-u s SWITCHES <RET>’ 让你为 ‘dired-listing-switches’ 指定新值。

32.17 Dired 与 ‘find’

你可以用 ‘find’ 工具选择文件，从而更灵活地挑选一组文件显示在 Dired 缓冲区中。

要用通配符模式搜索名称匹配的文件，使用 ‘M-x find-name-dired’。它读取参数 DIRECTORY 和 PATTERN，并选出 DIRECTORY 或其子目录中各个名称匹配 PATTERN 的所有文件。

这样选出的文件会显示在一个 Dired 缓冲区中，其中可以使用常规 Dired 命令。

如果你想检查文件的内容而不是名称，使用 ‘M-x find-grep-dired’。这条命令读取两个迷你缓冲区参数 DIRECTORY 和 REGEXP；它选出 DIRECTORY 或其子目录中内容含有 REGEXP 匹配的所有文件。它的工作方式是运行 ‘find’ 和 ‘grep’ 程序。另请参见“在 Emacs 中用 Grep 搜索”中的 ‘M-x grep-find’。记住要为 ‘grep’ 而不是为 Emacs 编写正则表达式。（显示内容匹配给定正则表达式的文件的另一种方法是 ‘% g REGEXP’ 命令，参见“Dired 标记与旗标之别”。）

这一系列命令中最通用的是 ‘M-x find-dired’，它让你指定 ‘find’ 能检验的任何条件。它接受两个迷你缓冲区参数 DIRECTORY 和 FIND-ARGS；它在 DIRECTORY 中运行 ‘find’，并把 FIND-ARGS 传给 ‘find’ 以告诉它检验什么条件。要使用这条命令，你需要知道如何使用 ‘find’。

这些命令生成的列表格式由变量 ‘find-ls-option’ 控制。它是一个由两个选项组成的对；第一个指定如何调用 ‘find’ 以生成文件列表，第二个告诉 Dired 如何解析输出。

命令 ‘M-x locate’ 为 ‘locate’ 程序提供了类似的接口。‘M-x locate-with-filter’ 类似，但只保留名称匹配给定正则表达式的文件。

这些缓冲区并不完全像普通 Dired 缓冲区那样工作：文件操作可用，但不总是自动更新缓冲区。用 ‘g’ 还原缓冲区会删除所有已插入的子目录，并擦除所有标记和标志。

32.18 编辑 Dired 缓冲区

Wdired 是一种特殊模式，让你通过直接编辑 Dired 缓冲区来执行文件操作（“Wdired”中的“W”代表“writable”，即“可写”）。要进入 Wdired 模式，在 Dired 缓冲区中键入 ‘C-x C-q’（‘dired-toggle-read-only’）。或者，使用 ‘Immediate / Edit File Names’ 菜单项。

在 Wdired 模式中，你可以通过编辑 Dired 缓冲区中显示的文件名来重命名文件。所有常规 Emacs 编辑命令，包括矩形操作和 ‘query-replace’，都可以用于此目的。编辑完成后，键入 ‘C-c C-c’（‘wdired-finish-edit’）。这会应用你的更改，并切换回普通 Dired 模式。

除了简单地重命名文件，你还可以键入新的文件名（绝对或相对）把文件移动到另一个目录。要把文件标记为删除，删除整个文件名。要更改符号链接的目标，编辑链接名旁边出现的链接目标名。

如果你通过编辑文件名创建新的子目录，Wdired 会自动创建这些新目录。要禁止这种行为，把 'wdired-create-parent-directories' 设为 'nil'。

缓冲区中的其余文本，例如文件大小和修改日期，被标记为只读，因此你不能编辑它们。不过，如果把 'wdired-allow-to-change-permissions' 设为 't'，你就可以编辑文件权限。例如，你可以把 '-rw-r--r--' 改成 '-rw-rw-rw-'，使文件对所有人可写。这些更改同样在你键入 'C-c C-c' 时生效。

32.19 在 Dired 中查看图像缩略图

Image-Dired 是浏览图像文件的设施。它提供以缩略图或完整大小查看图像的方式，既可在 Emacs 内部查看，也可通过外部查看器查看。这与用于在 Emacs 缓冲区中访问图像文件的 Image 模式不同（参见"查看图像文件"）。

要进入 Image-Dired，先在 Dired 缓冲区中用 'm' 照常标记你想查看的图像文件。然后键入 'C-t d'（'image-dired-display-thumbs'）。这会创建并切换到一个包含 Image-Dired 的缓冲区，对应那些已标记的文件。

你也可以直接键入 'M-x image-dired' 进入 Image-Dired。它会提示输入一个目录；请指定含有图像文件的目录。这会为该目录中的所有图像创建缩略图，并把它们全部显示在缩略图缓冲区中。缩略图在后台生成，生成好一个就加载一个。

Image-Dired 支持三种生成和存储缩略图的方法，由选项 'image-dired-thumbnail-storage' 的值控制：

- 'image-dired'：此方法把缩略图作为 JPEG 图像存放在变量 'image-dired-dir' 指定的单个目录中。这是默认值。此时缩略图文件的名称按 'image-dired-thumb-naming' 的值构造。
- 'standard'、'standard-large'、'standard-x-large'、'standard-xx-large'：这些方法由 Thumbnail Managing Standard（https://specifications.freedesktop.org/thumbnail-spec/thumbnail-spec-latest.html）规定，把缩略图作为 PNG 图像存放在环境变量 'XDG_CACHE_HOME'（默认为 '~/.cache'）所指定目录的 'thumbnails' 子目录下。
- 'per-directory'：此方法把每个目录的缩略图作为 jpeg 图像存放在该目录的 '.image-dired' 子目录中。

你可以通过定制变量 'image-dired-thumb-size' 控制缩略图的大小。默认是 128 像素。该变量只对上述第一种和第三种缩略图存储方法有效；Thumbnail Managing Standard 使用由方法决定的固定大小：'standard' 使用 128 像素，'standard-large' 使用 256 像素，依此类推。如果你在某些缩略图已经创建之后更改大小，需要从缩略图目录中删除缩略图文件，新大小才会生效。

光标点在缩略图缓冲区中时，你可以键入 <RET>（'image-dired-display-this'）在另一个窗口中显示该图像。使用标准的 Emacs 移动键绑定或方向键在缩略图缓冲区中移动。为便于浏览，用 <SPC>（'image-dired-display-next'）前进并显示下一个图像。键入 <DEL>（'image-dired-display-previous'）退回上一个缩略图并改为显示它。

键入 'C-<RET>'（'image-dired-thumbnail-display-external'）在外部查看器中显示图像。你必须先配置 'image-dired-external-viewer'。

你也可以通过 Image-Dired 删除图像。键入 ‘d’（‘image-dired-flag-thumb-original-file’）在 Dired 缓冲区中把该图像文件标记为删除。或者，你可以从缩略图缓冲区中移除某个图像的缩略图而不把该图像标记为删除，方法是键入 ‘C-d’（‘image-dired-delete-char’）。

你也可以把 Image-Dired 用于“内联”操作（即直接在 Dired 缓冲区中）。键入 ‘C-t C-t’，Dired 中所选图像的缩略图就会出现在它们的名称之前（‘image-dired-dired-toggle-marked-thumbs’）。‘C-t i’ 和 ‘C-t x’ 分别会在 Emacs 中或使用外部查看器显示光标点处的图像。

更高级的功能包括“图像标签”（image tag），即用于给图像文件分类的元数据。标签存放在由 ‘image-dired-tags-db-file’ 配置的纯文本文件中。

要给图像文件加标签，先在 Dired 缓冲区中标记它们（你也可以在缩略图缓冲区中键入 ‘m’ 来标记 Dired 中的文件），然后键入 ‘C-t t’（‘image-dired-tag-files’）。这会在迷你缓冲区中读取标签名。要标记具有某个标签的文件，键入 ‘C-t f’（‘image-dired-mark-tagged-files’）。用某个标签标记了图像文件之后，你可以用 ‘C-t d’ 查看它们。

你也可以在缩略图缓冲区中键入 ‘t t’ 直接给文件加标签，并可以键入 ‘t r’ 移除标签。每个文件还有一个称为“comment”的特殊标签（它并不完全等同于其他标签，处理方式略有不同）。它用于输入关于图像的注释或描述。在缩略图缓冲区中键入 ‘c’ 可以为文件添加注释。系统会提示你输入注释。在 Dired 中键入 ‘C-t c’ 可以添加注释（‘image-dired-dired-comment-files’）。‘C-t e’ 会调出一个用于编辑注释和标签的缓冲区（‘image-dired-dired-edit-comment-and-tags’）。

如果你安装了 ‘exiftool’ 程序，可以在光标点位于某图像文件的缩略图上时调用 ‘image-dired-thumbnail-set-image-description’ 命令，来设置该图像文件的 EXIF ‘ImageDescription’ 标签。这条命令提示输入图像的描述，并为其添加 EXIF 标签。

如果 ‘image-dired-thumb-visible-marks’ 为非 ‘nil’（这是默认值），在 Dired 中标记的文件在 Image-Dired 中也会被标记。

Image-Dired 还提供简单的图像操作。在缩略图缓冲区中，键入 ‘L’ 把原始图像逆时针旋转 90 度，键入 ‘R’ 顺时针旋转 90 度。这种旋转是无损的，使用一个名为 ‘jpegtran’ 的外部工具，你需要先安装它。

32.20 Dired 的其他功能

默认情况下，Dired 会在目录列表的第一行、目录名之后显示该目录所在磁盘的可用空间。你可以通过定制变量 ‘dired-free-space’ 控制这一显示。它的默认值是 ‘first’，即在目录名之后显示可用空间。如果你改为把它定制成值 ‘separate’，Dired 就会在目录名那一行之后的单独一行中显示磁盘空间信息，并且在这一行中既包含当前目录中文件所占用的空间，也包含可用磁盘空间。如果你把它设为 ‘nil’，就完全不显示可用磁盘空间信息。

Dired 会对 Dired 缓冲区中的条目进行字体化（fontify）。如果该缓冲区的 ‘default-directory’ 是远程的，对符号链接做这件事可能代价很高，因为需要访问它们的 ‘file-truename’。为了避免因此导致 Dired 变慢，对于连接缓慢的远程目录，请把用户选项 ‘dired-check-symlinks’ 设为 ‘nil’。可以把该变量声明为连接局部变量，以匹配远程系统的可连接方式（参见“按连接的局部变量”）。

命令 ‘+’（‘dired-create-directory’）读取目录名，并创建该目录。如果目录已存在，它会报错。

命令（‘dired-create-empty-file’）读取文件名，并创建该文件。如果文件已存在，它会报错。

命令‘M-s a C-s’（‘dired-do-isearch’）在已标记的文件上开始多文件增量搜索。如果搜索在某个文件末尾失败，键入‘C-s’会前进到下一个已标记的文件并重复搜索；在最后一个已标记文件的末尾，搜索会回绕到第一个已标记文件。命令‘M-s a M-C-s’（‘dired-do-isearch-regexp’）对正则表达式搜索做同样的事。关于搜索的重复，参见“重复增量搜索”。

命令‘w’（‘dired-copy-filename-as-kill’）把已标记（或接下来 N 个）文件的名称放入删除环，就像你用‘C-w’删除了它们一样。这些名称以一个空格分隔。

使用零前缀参数时，它使用每个已标记文件的绝对文件名。前缀值为 1 时，它使用相对于项目根目录的名称。仅用‘C-u’作为前缀参数时，它使用相对于 Dired 缓冲区默认目录的文件名。（如果在子目录中，这些名称仍可能包含斜杠。）特殊情况是：如果光标点在目录标题行上，‘w’会给出该目录的绝对名称。这种情况下会忽略任何前缀参数或已标记文件。

这条命令的主要用途是让你把文件名粘贴到其他 Emacs 命令的参数中。它还会显示自己添加到删除环中的内容，因此你可以用它把当前已标记文件的列表显示在回显区。

如果文件列表中有 HTML 文件，用浏览器查看该文件会很有用。‘W’（‘browse-url-of-dired-file’）命令会使用配置好的标准浏览器查看该文件。

命令‘(’（‘dired-hide-details-mode’）切换当前 Dired 缓冲区中是否显示细节，例如所有者或文件权限。默认情况下，它还会隐藏符号链接的目标，以及除标题行和文件/目录列表之外的所有行。要改变这一点，请分别定制选项‘dired-hide-details-hide-symlink-targets’和‘dired-hide-details-hide-information-lines’。

命令‘M-x dired-compare-directories’用于把当前 Dired 缓冲区与另一个目录比较。它会标记两个目录之间有差异的所有文件。它把标记放进列出这些文件的所有 Dired 缓冲区中，当然包括当前缓冲区。

默认的比较方法（在提示处键入 <RET> 时使用）只比较文件名——若某文件名没有出现在另一个目录中，则文件名不同。输入一个 Lisp 表达式可以指定更严格的比较，该表达式可以引用变量‘size1’和‘size2’（各自的文件大小）、‘mtime1’和‘mtime2’（最后修改时间，以浮点数的秒数表示）以及‘fa1’和‘fa2’（各自的文件属性列表，由函数‘file-attributes’返回）。该表达式会针对每一对同名文件求值，若表达式的值为非‘nil’，则文件不同。

例如，序列‘M-x dired-compare-directories <RET> (> mtime1 mtime2) <RET>’会标记在此目录中比在另一个目录中更新的文件，以及标记在另一个目录中比在此目录中更旧的文件。它也照例标记两个目录中没有对应文件的文件。

在 X Window System 上，Emacs 支持拖放协议。你可以从另一个程序拖动一个文件对象，把它放到 Dired 缓冲区上；这会把该文件移动到该目录、复制到该目录，或在其中创建指向它的链接。具体执行哪种操作由来源程序决定。也支持把文件从 Dired 缓冲区拖出：启用用户选项‘dired-mouse-drag-files’后，可以用鼠标把文件拖到其他程序上。当设为‘link’时，它会让另一个程序（通常是文件管理器）创建指向该文件的符号链接；当设为‘move’时，它会让另一个程序把文件移动到新位置；设为其他任何非‘nil’值时，它会让另一个程序打开该文件或创建其副本。拖放操作期间按下的键盘修饰键也可以控制另一个程序对该文件采取什么操作。

33 日历与日记

Emacs 提供台历的功能，并带有一份记录计划中或已发生事件的日记。它还能管理你的约会，并记录你在某些项目上投入了多少时间。

要进入日历，输入 ‘M-x calendar’。这会显示一个以当前月份为中心的三月日历，光标点位于当前日期上。若带数字参数，如 ‘C-u M-x calendar’，它会提示你输入作为三月日历中心的月份和年份。日历使用自己的缓冲区，其主模式是 Calendar 模式。

在日历中，‘mouse-3’ 会弹出一个针对特定日期的操作菜单；‘mouse-2’ 会弹出一个常用日历功能的菜单，这些功能与具体日期无关。要退出日历，输入 ‘q’。

- 计算天数：两个日期之间相隔多少天?
- 通用日历：退出日历或重新计算日历。
- 写出日历文件：把日历写入各种格式的文件。
- 节假日：显示节假日的日期。
- 日出/日落：显示日出和日落的本地时间。
- 月相：显示月亮的相位。
- 其他历法：把日期转换为其他历法系统。
- 日记：显示日记中的事件。
- 夏令时：如何指定夏令时何时生效。
- 时间间隔：记录时间间隔。
- 日历/日记的高级用法：日历/日记的高级定制。

33.1 日历中的移动

Calendar 模式提供的命令按日、周、月、年等合乎逻辑的时间单位在日历中移动。如果你移到最初显示的月份之外，日历显示会自动随时间滚动，使选中的日期可见。移到某个日期后，你可以查看该日期的节假日或日记条目，或把它转换为其他历法；按长时间跨度移动在单纯滚动日历时也很有用。

- 指定日期：移动到当前日期或其他特定日期。

33.1.1 按标准时间长度移动

日历缓冲区中用于移动的命令与用于在文本中移动的命令相类似。你可以按日、周、月、年前后移动。

- ‘C-f’：光标点前移一天（‘calendar-forward-day’）。
- ‘C-b’：光标点后移一天（‘calendar-backward-day’）。
- ‘C-n’：光标点前移一周（‘calendar-forward-week’）。
- ‘C-p’：光标点后移一周（‘calendar-backward-week’）。
- ‘M-}’、‘}’：光标点前移一个月（‘calendar-forward-month’）。
- ‘M-{’、‘{’：光标点后移一个月（‘calendar-backward-month’）。

- ‘C-x]’、‘]’：光标点前移一年（‘calendar-forward-year’）。
- ‘C-x [’、‘[’：光标点后移一年（‘calendar-backward-year’）。

按日和按周的命令自然对应于 Emacs 中常用的按字符和按行移动的命令。正如 ‘C-n’ 通常移到下一行的同一列，在 Calendar 模式中它绑定到 ‘calendar-forward-week’，即移到下一周的同一日。‘C-p’（‘calendar-backward-week’）则移到上一周的同一日。‘C-f’（‘calendar-forward-day’）和 ‘C-b’（‘calendar-backward-day’）按日前后移动。

方向键等价于 ‘C-f’、‘C-b’、‘C-n’ 和 ‘C-p’，正如它们在其他模式中通常那样。

按月和按年移动的命令与按周的命令类似，只是移动的距离更大。按月命令 ‘M-}’（‘calendar-forward-month’）和 ‘M-{’（‘calendar-backward-month’）前后移动整整一个月。按年命令 ‘C-x]’（‘calendar-forward-year’）和 ‘C-x [’（‘calendar-backward-year’）前后移动整整一年。

记住这些命令最容易的办法，是把月和周分别类比为文本的段落和页面。但日历移动命令本身并不完全与在文本中移动的命令平行：通常的 Emacs 段落命令移到段落的开头或结尾，而这些按月、按年的命令移动整整一个月或整整一年，同时保持在该月或该年中的同一日期。

你也可以使用敲起来更简单的键 ‘}’ 和 ‘{’ 按月移动，用 ‘]’ 和 ‘[’ 按年移动。

所有这些命令都接受数字参数作为重复次数。为方便起见，在 Calendar 模式中，数字键和减号即使没有 Meta 修饰键也能指定数字参数。例如，‘100 C-f’ 把光标点从当前位置前移 100 天。

33.1.2 周、月或年的开始与结束

一周（或一月、一年）并不只是一段天数；我们认为周（月、年）始于特定的日期。因此 Calendar 模式提供命令，移到一周、一月或一年的开始或结束：

- ‘C-a’：光标点移到本周开始（‘calendar-beginning-of-week’）。
- ‘C-e’：光标点移到本周结束（‘calendar-end-of-week’）。
- ‘M-a’：光标点移到本月开始（‘calendar-beginning-of-month’）。
- ‘M-e’：光标点移到本月结束（‘calendar-end-of-month’）。
- ‘M-<’：光标点移到本年开始（‘calendar-beginning-of-year’）。
- ‘M->’：光标点移到本年结束（‘calendar-end-of-year’）。

这些命令也接受数字参数作为重复次数，重复次数表示向后或向前移动多少周、多少月或多少年。

默认情况下，一周从星期日开始。要改为从星期一开始，把变量 ‘calendar-week-start-day’ 设为 1。要改变哪些星期的表头被高亮显示为周末，设置变量 ‘calendar-weekend-days’。

33.1.3 指定日期

Calendar 模式提供的命令可以移到以各种方式指定的特定日期。

- ‘g d’：光标点移到指定日期（‘calendar-goto-date’）。
- ‘g D’：光标点移到指定的一年的第几天（‘calendar-goto-day-of-year’）。
- ‘g w’：光标点移到指定的一年的第几周（‘calendar-iso-goto-week’）。

- ‘o’：让日历以指定月份为中心（‘calendar-other-month’）。
- ‘.’：光标点移到今天的日期（‘calendar-goto-today’）。

‘g d’（‘calendar-goto-date’）提示输入年、月和日，然后移到该日期。由于日历包含当前纪元开始以来的所有日期，你必须输入完整的年份；也就是说，输入‘2010’，而不是‘10’。

‘g D’（‘calendar-goto-day-of-year’）提示输入年份和天数，然后移到该日期。负的天数从年末向前计算。‘g w’（‘calendar-iso-goto-week’）提示输入年份和周数，然后移到那一周。

‘o’（‘calendar-other-month’）提示输入月份和年份，然后让三月日历以该月为中心。

你可以用‘.’（‘calendar-goto-today’）回到今天的日期。

33.2 日历中的滚动

当你移出可见部分时，日历显示会自动随时间滚动。你也可以手动滚动它。设想日历窗口中有一长条印着各月份的纸带。滚动日历意味着水平移动这条纸带，使新的月份在窗口中变得可见。

- ‘>’：日历向前滚动一个月（‘calendar-scroll-left’）。
- ‘<’：日历向后滚动一个月（‘calendar-scroll-right’）。
- ‘C-v’、‘<PageDown>’、‘<next>’：日历向前滚动（‘calendar-scroll-calendar-left’）。
- ‘M-v’、‘<PageUp>’、‘<prior>’：日历向后滚动（‘calendar-scroll-calendar-right’）。
- ‘C-l’：把光标点处的日期重新居中。

最基本的日历滚动命令每次滚动一个月。‘>’（‘calendar-scroll-left’）把日历内容在时间上向前滚动一个月。‘<’（‘calendar-scroll-right’）把内容在时间上向后滚动一个月。

命令‘C-v’（‘calendar-scroll-calendar-left’）和‘M-v’（‘calendar-scroll-calendar-right’）滚动整个屏幕的日历——与这些命令的通常含义类似。‘C-v’使较晚的日期可见，‘M-v’使较早的日期可见。这些命令接受数字参数作为重复次数。

功能键 <PageDown>（或 <next>）和 <PageUp>（或 <prior>）等价于‘C-v’和‘M-v’，正如它们在其他模式中那样。

命令‘C-l’（‘calendar-recenter’）滚动当前显示的日历，使光标点处日期所在的月份水平居中。再次调用此命令会把该月份放到最左边的位置，又一次调用会把它放到最右边的位置。之后再调用会按同样的顺序循环重复。

33.3 计算天数

‘M-=’显示当前区域中的天数（‘calendar-count-days-region’）。

要确定一个范围中的天数，先用‘C-<SPC>’在某个日期上设置标记，把光标点移到另一个日期，然后输入‘M-=’（‘calendar-count-days-region’）。显示的日期数是包含两端的；也就是说，它包含标记和光标点所指定的那两个日期。

33.4 其他日历命令

- ‘p d’：显示一年中的第几天（‘calendar-print-day-of-year’）。
- ‘C-c C-l’：重新生成日历窗口（‘calendar-redraw’）。
- ‘<SPC>’：把下一个窗口向上滚动（‘scroll-other-window’）。
- ‘<DEL>’、‘S-<SPC>’：把下一个窗口向下滚动（‘scroll-other-window-down’）。
- ‘q’：退出日历（‘calendar-exit’）。
- ‘M-x calendar-show-more-months’：显示更多月份。
- ‘M-x calendar-show-fewer-months’：显示更少月份。

要显示自年初以来已过去的天数，或一年中剩余的天数，输入‘p d’命令（‘calendar-print-day-of-year’）。这会在回显区同时显示这两个数字。已过去的天数包含选中的日期。剩余的天数不包含该日期。

如果日历窗口的文本出现错乱，输入‘C-c C-l’（‘calendar-redraw’）重新绘制它。（只有在你使用非 Calendar 模式的编辑命令时才会发生这种情况。）

在 Calendar 模式中，你可以用 <SPC>（‘scroll-other-window’）和 <DEL>（‘scroll-other-window-down’）分别把另一个窗口（如果有）向上或向下滚动。当你在另一个窗口中显示节假日或日记条目的列表时，这很方便。

要退出日历，输入‘q’（‘calendar-exit’）。这会把所有与日历相关的缓冲区都收入后台，并选择其他缓冲区。（如果某个框架只包含一个专用的日历窗口，则退出日历时会根据‘calendar-remove-frame-by-deleting’的值删除或图标化该框架。）

命令‘M-x calendar-show-more-months’在日历右侧显示更多月份。只要有足够的空间，日历最多可以显示 12 个月。命令‘M-x calendar-show-fewer-months’显示更少的月份。日历始终至少显示 3 个月。

33.5 写出日历文件

你可以把日历和日记条目写入 HTML 和 LaTeX 文件。

Calendar 的 HTML 命令会生成包含日历、节假日和日记条目的 HTML 代码文件。每个文件对应一个月，名字格式为‘YYYY-MM.html’，其中 YYYY 和 MM 分别是四位数的年份和两位数的月份。变量‘cal-html-directory’指定 HTML 文件的默认输出目录。要禁止显示节假日，定制‘cal-html-holidays’。

用‘<’和‘>’括起来的日记条目会被解释为 HTML 标签（例如：这是一条带有 some red text 的日记条目）。你可以通过 HTML 文件所在目录中的样式表‘cal.css’改变所显示 HTML 页面的整体外观（例如各种页面元素的颜色、表头样式）（相关样式设置见变量‘cal-html-css-default’的值）。

- ‘H m’：生成一个月的日历（‘cal-html-cursor-month’）。
- ‘H y’：为一年中的每个月各生成一个日历文件，并生成一个索引页（‘cal-html-cursor-year’）。默认情况下，此命令把文件写入 YEAR 子目录，其中 YEAR 是光标所在的年份——如果改变这一点，年份之间的某些超链接将无法工作。

如果变量‘cal-html-print-day-number-flag’为非‘nil’，则每月日历会显示一年中的第几天。变量‘cal-html-year-index-cols’指定年度索引页的列数。

Calendar 的 LaTeX 命令生成一个 LaTeX 代码缓冲区，打印出来就是一份日历。取决于你使用的命令，打印出的日历覆盖光标点所在的日、周、月或年。

- ‘t m’：生成一个月的日历（‘cal-tex-cursor-month’）。
- ‘t M’：生成横向打印的一个月的日历（‘cal-tex-cursor-month-landscape’）。
- ‘t d’：生成一天的日历（‘cal-tex-cursor-day’）。
- ‘t w 1’：生成一周的一页日历，带小时（‘cal-tex-cursor-week’）。
- ‘t w 2’：生成一周的两页日历，带小时（‘cal-tex-cursor-week2’）。
- ‘t w 3’：生成一周的 ISO 风格日历，不带小时（‘cal-tex-cursor-week-iso’）。
- ‘t w 4’：生成从星期一开始的一周的日历，带小时（‘cal-tex-cursor-week-monday’）。
- ‘t w W’：生成一周的两页日历，不带小时（‘cal-tex-cursor-week2-summary’）。
- ‘t f w’：生成 Filofax 风格的两周概览日历（‘cal-tex-cursor-filofax-2week’）。
- ‘t f W’：生成 Filofax 风格的一周概览日历（‘cal-tex-cursor-filofax-week’）。
- ‘t y’：生成一年的日历（‘cal-tex-cursor-year’）。
- ‘t Y’：生成横向打印的一年的日历（‘cal-tex-cursor-year-landscape’）。
- ‘t f y’：生成 Filofax 风格的一年的日历（‘cal-tex-cursor-filofax-year’）。

其中有些命令会横向（横向模式）打印日历，从而使宽度大于长度。有些使用 Filofax 纸张尺寸（3.75in x 6.75in）。所有这些命令都接受前缀参数，它指定打印多少天、多少周、多少月或多少年（总是从选中的那个开始）。

如果变量 ‘cal-tex-holidays’ 为非 ‘nil’（默认值），则打印出的日历会显示 ‘calendar-holidays’ 中的节假日。如果变量 ‘cal-tex-diary’ 为非 ‘nil’（默认值为 ‘nil’），也包含日记条目（仅限每月、Filofax 和 ISO 周历）。如果变量 ‘cal-tex-rules’ 为非 ‘nil’（默认值为 ‘nil’），日历会在空间充足的样式中显示带横线的页面。请查阅各个 cal-tex 函数的文档，了解哪些日历支持哪些功能。

如有需要，你可以用变量 ‘cal-tex-preamble-extra’ 在所生成文档的前导部分插入额外的 LaTeX 命令。

33.6 节假日

Emacs 日历知道许多主要的和次要的节假日，并能显示它们。你可以把自己的节假日加入默认列表。

- ‘mouse-3 Holidays’、‘h’：显示所选日期的节假日（‘calendar-cursor-holidays’）。
- ‘x’：在日历窗口中标记节假日（‘calendar-mark-holidays’）。
- ‘u’：取消日历窗口中的标记（‘calendar-unmark’）。
- ‘a’：在另一个窗口中列出所显示月份的节假日（‘calendar-list-holidays’）。
- ‘M-x holidays’：在另一个窗口中列出今天前后三个月的节假日。
- ‘M-x list-holidays’：在另一个窗口中列出指定年份范围内的节假日。

要查看某个给定日期是否有节假日，把光标点移到日历窗口中的该日期上，并使用 ‘h’（‘calendar-cursor-holidays’）命令。另一种做法是用 ‘mouse-3’ 在该日期上点击，然后从弹出的菜单中选择 ‘Holidays’。两种方式都会显示该日期的节假日：如果放得下就显示在回显区，否则显示在一个单独的窗口中。

要查看日历中所示所有日期的节假日分布，使用‘x’（‘calendar-mark-holidays’）命令。这会把属于节假日的日期用一种不同的字体（face）显示。参见“定制日历”该命令既作用于当前可见的月份，也作用于之后通过滚动变为可见的其他月份。要关闭标记并清除当前的标记，输入‘u’（‘calendar-unmark’），这也会清除所有日记标记（参见“日记”）。如果变量‘calendar-mark-holidays-flag’为非‘nil’，则创建或更新日历时会自动标记节假日。

要获得更详细的信息，使用‘a’（‘calendar-list-holidays’）命令，它会显示一个单独的缓冲区，其中列出当前日历范围内的所有节假日。你可以在日历窗口中使用 <SPC> 和 <DEL> 分别把该列表向上和向下滚动。

命令‘M-x holidays’显示当前月份以及前一个月和后一个月的节假日列表；即使你没有日历窗口，这也能用。如果变量‘calendar-view-holidays-initially-flag’为非‘nil’，创建日历时会以这种方式显示节假日。如果你想要以另一个月份为中心的节假日列表，使用‘C-u M-x holidays’，它会提示输入月份和年份。

Emacs 所知的节假日包括美国的节假日，以及巴哈伊教、中国、基督教、伊斯兰教和犹太教的主要节假日；此外还有冬至、夏至和春分、秋分。

命令‘M-x holiday-list’显示某一年份范围内的节假日列表。此函数会询问起始年份和结束年份，并允许你选择所有节假日或若干类别之一的节假日。即使你没有日历窗口，也可以使用此命令。

Emacs 用于节假日的日期基于当前的做法，而非历史事实。例如，退伍军人节始于 1919 年，但在更早的年份中也会显示。

33.7 日出与日落时间

特殊的日历命令可以告诉你任意日期日出和日落的时间，误差在一两分钟之内。

- ‘mouse-3 Sunrise/sunset’、‘S’：显示所选日期的日出和日落时间（‘calendar-sunrise-sunset’）。
- ‘M-x sunrise-sunset’：显示今天的日出和日落时间。
- ‘C-u M-x sunrise-sunset’：显示指定日期的日出和日落时间。
- ‘M-x calendar-sunrise-sunset-month’：显示所选月份的日出和日落时间。

在日历中，要在回显区显示日出和日落的本地时间，把光标点移到你想要的日期，然后输入‘S’（‘calendar-sunrise-sunset’）。另一种做法是在该日期上点击‘mouse-3’，然后从弹出的菜单中选择‘Sunrise/sunset’。命令‘M-x sunrise-sunset’可在日历之外使用，显示今天或某个指定日期的这一信息。要指定今天以外的日期，使用‘C-u M-x sunrise-sunset’，它会提示输入年、月和日。

你可以用‘C-u C-u M-x sunrise-sunset’显示任意地点、任意日期的日出和日落时间。它会询问经度、纬度、与世界协调时的分钟差以及日期，然后告诉你该地点在该日期的日出和日落时间。

由于日出和日落的时间取决于在地球上的位置，在使用这些命令之前，你需要告诉 Emacs 你的纬度、经度和地点名称。下面是一个设置示例：

```
(setq calendar-latitude 40.1)
(setq calendar-longitude -88.2)
(setq calendar-location-name "Urbana, IL")
```

‘calendar-latitude’和‘calendar-longitude’的值应保留一位小数。

你的时区也会影响日出和日落的本地时间。Emacs 通常从操作系统获取时区信息，但如果这些值不是你想要的（或者操作系统没有提供它们），你必须自己设置。下面是一个例子：

```
(setq calendar-time-zone -360)
(setq calendar-standard-time-zone-name "CST")
(setq calendar-daylight-time-zone-name "CDT")
```

'calendar-time-zone' 的值是你的本地标准时间与世界协调时（又称"格林尼治时间"）之间相差的分钟数。'calendar-standard-time-zone-name' 和 'calendar-daylight-time-zone-name' 的值是你的时区所使用的缩写。Emacs 显示的日出和日落时间已按夏令时校正。至于如何确定夏令时，参见"夏令时"。

如果你想显示数字形式的时区（如 '"+0100"'）而不是符号形式的时区（如 '"CET"'），把变量 'calendar-time-zone-style' 设为 'numeric'。

作为用户，你可能会觉得在自己的 '.emacs' 文件中为你通常所处的实际地点设置日历位置变量很方便。如果你是系统管理员，可能想在 'default.el' 文件中为所有用户设置这些变量（参见"Emacs 初始化文件"）。

33.8 月相

本节所述的日历命令显示月亮各相位（新月、上弦月、满月、下弦月）的日期和时间。这一功能对于调试依赖于月相的问题很有用。

- 'M'：显示可见时段内月亮所有相位的日期和时间（'calendar-lunar-phases'）。
- 'M-x lunar-phases'：显示今天前后三个月的月亮各相位的日期和时间。

在日历中，使用 'M'（'calendar-lunar-phases'）命令显示一个单独的缓冲区，其中是当前可见范围内的月相。所列的日期和时间精确到几分钟之内。

在日历之外，使用命令 'M-x lunar-phases' 显示当前月份以及前一个月和后一个月的月相列表。要了解另一个月份的信息，使用 'C-u M-x lunar-phases'，它会提示输入月份和年份。

给出的月相日期和时间是本地时间（在适当情况下已按夏令时校正）。参见上一节的讨论（参见"日出与日落时间"）。

33.9 与其他历法的相互转换

所显示的 Emacs 日历始终是"公历"，有时称为"新历"，今天世界上大多数地方都在使用它。然而，这种历法在十六世纪之前并不存在，在十八世纪之前也未被广泛使用；直到二十世纪初，它才完全取代儒略历并得到普遍接受。Emacs 日历可以显示当前纪元第 1 年 1 月以来的任何月份，但所显示的日历始终是公历，即使对于公历尚不存在的日期也是如此。

Emacs 虽然不能显示其他历法，但可以把日期与另外几种历法相互转换。

- 历法系统：Emacs 所理解的（公历之外的）历法。
- 转换为其他历法：把选中的日期转换为各种历法。
- 从其他历法转换：移动到以另一种历法指定的日期。

33.9.1 支持的历法系统

ISO 商业历法常用于商业活动。

儒略历（Julian calendar）得名于尤利乌斯·凯撒，整个中世纪时期欧洲一直使用它，许多国家直到 19 世纪仍在使用。

天文学家使用一种简单的计数方式：从公元前 4713 年 1 月 1 日（儒略历）星期一正午起经过的天数。自那时起经过的天数称为“儒略日数”（Julian day number）或“天文日数”（Astronomical day number）。

希伯来历（Hebrew calendar）是犹太教传统使用的历法。Emacs 日历程序使用希伯来历确定犹太节日的日期。希伯来历的日期以日落为起点和终点。

伊斯兰历（Islamic calendar）在许多以伊斯兰教为主的国家使用。Emacs 用它确定伊斯兰节日的日期。伊斯兰世界对历法没有普遍一致的看法；Emacs 采用一个被广泛接受的版本，但伊斯兰节日的确切日期往往取决于宗教权威的宣布，而不是计算。因此，实际遵守的日期可能与 Emacs 计算出的日期略有出入。伊斯兰历的日期以日落为起点和终点。

法国大革命历（French Revolutionary calendar）由雅各宾派在 1789 年革命之后创制，用以表达一种更世俗、更基于自然的年度周期观，并作为一种类似公制的合理化措施推行 10 日一周。法国政府于 1805 年底正式废弃了这一历法。

中美洲的玛雅人使用三种相互独立而又有重叠的历法：*长纪年*（long count）、*卓尔金历*（tzolkin）和*哈布历*（haab）。Emacs 了解这三种历法。专家们对玛雅历与公历之间的确切对应关系存在争议；Emacs 在计算中使用 Goodman-Martinez-Thompson 对应关系。

科普特人使用的历法基于古埃及太阳历。他们的历法由十二个 30 天的月组成，其后另加五天。每四年，他们在这额外的一段中加一个闰日，使其成为六天。埃塞俄比亚历在结构上与之相同，但年号和月名不同。

波斯人使用的太阳历基于 Omar Khayyam 的设计。他们的历法由十二个月组成，其中前六个月有 31 天，接下来的五个月有 30 天，最后一个月在平年为 29 天、在闰年为 30 天。闰年以每四年或五年一次的复杂模式出现。此处实现的历法是 Birashk 所主张的算术波斯历，基于 2820 年的周期。它与以天文事件为基础的天文波斯历不同。截至本文写作时，预计第一个将来的差异将出现在 2025 年 3 月 20 日。目前尚不清楚届时伊朗的官方历法将会是什么。

中国历法是把农历月排入太阳年的一套复杂系统。年以六十为一周期，每年在平年含十二个月、在闰年含十三个月；每月有 29 天或 30 天。年、普通月份和日都由十个“天干”之一与十二个“地支”之一组合命名，共六十个名称，以六十为周期循环。

巴哈伊历法系统基于一个含 19 个月、每月 19 天的太阳周期。剩余的那些插入日（称为 Ayyám-i-Há）被放在第 18 个月与第 19 个月之间。在 2014 年世界正义院进行历法改革之后，Naw-Rúz（新年）的日期现由从德黑兰观测到的春分确定，双圣诞辰（巴孛诞辰与巴哈欧拉诞辰）的日期则由 Naw-Rúz 之后的第八个新月推算。对于 172 BE（公元 2015 年）之前的日期，该历法遵循改革前的固定算法。

33.9.2 转换到其他历法

以下命令用各种其他历法描述所选日期（即光标点处的日期）：

- ‘mouse-3 Other calendars’、‘p o’：用各种其他历法显示所选日期（‘calendar-print-other-dates’）。

- ‘p c’：显示所选日期对应的 ISO 商业历日期（‘calendar-iso-print-date’）。
- ‘p j’：显示所选日期的儒略历日期（‘calendar-julian-print-date’）。
- ‘p a’：显示所选日期的天文（儒略）日数（‘calendar-astro-print-day-number’）。
- ‘p h’：显示所选日期的希伯来历日期（‘calendar-hebrew-print-date’）。
- ‘p i’：显示所选日期的伊斯兰历日期（‘calendar-islamic-print-date’）。
- ‘p f’：显示所选日期的法国大革命历日期（‘calendar-french-print-date’）。
- ‘p b’：显示所选日期的巴哈伊历日期（‘calendar-bahai-print-date’）。
- ‘p C’：显示所选日期的中国历日期（‘calendar-chinese-print-date’）。
- ‘p k’：显示所选日期的科普特历日期（‘calendar-coptic-print-date’）。
- ‘p e’：显示所选日期的埃塞俄比亚历日期（‘calendar-ethiopic-print-date’）。
- ‘p p’：显示所选日期的波斯历日期（‘calendar-persian-print-date’）。
- ‘p m’：显示所选日期的玛雅历日期（‘calendar-mayan-print-date’）。

否则，就把光标点移到想要转换的日期上，然后输入上表中以 ‘p’ 开头的相应命令。前缀 ‘p’ 是 "print" 的助记符，因为 Emacs 是在回显区“打印”出对应的日期。‘p o’（‘calendar-print-other-dates’）以 Emacs 所知的全部形式显示该日期。你也可以使用 ‘mouse-3’ 然后从弹出的菜单中选择 ‘Other calendars’。这会以菜单的形式显示该日期在 Emacs 所理解的所有历法中的对应形式。（从该菜单中选择某一项实际上不会做任何事——该菜单仅用于显示。）

33.9.3 从其他历法转换

你可以用其他受支持的历法来指定要移到的日期。本节介绍用玛雅历之外的历法做这件事的命令；关于玛雅历，参见下一节。

- ‘g c’：移到 ISO 商业历中指定的日期（‘calendar-iso-goto-date’）。
- ‘g w’：移到 ISO 商业历中指定的某一周（‘calendar-iso-goto-week’）。
- ‘g j’：移到儒略历中指定的日期（‘calendar-julian-goto-date’）。
- ‘g a’：移到用天文（儒略）日数指定的日期（‘calendar-astro-goto-day-number’）。
- ‘g b’：移到巴哈伊历中指定的日期（‘calendar-bahai-goto-date’）。
- ‘g h’：移到希伯来历中指定的日期（‘calendar-hebrew-goto-date’）。
- ‘g i’：移到伊斯兰历中指定的日期（‘calendar-islamic-goto-date’）。
- ‘g f’：移到法国大革命历中指定的日期（‘calendar-french-goto-date’）。
- ‘g C’：移到中国历中指定的日期（‘calendar-chinese-goto-date’）。
- ‘g p’：移到波斯历中指定的日期（‘calendar-persian-goto-date’）。
- ‘g k’：移到科普特历中指定的日期（‘calendar-coptic-goto-date’）。
- ‘g e’：移到埃塞俄比亚历中指定的日期（‘calendar-ethiopic-goto-date’）。

这些命令会先向你询问另一种历法中的日期，把光标点移到与之等价的公历日期上，并在回显区显示该另一历法中的日期。每当要求你输入月名时，Emacs 都会使用严格补全（参见“补全退出”），因此你不必担心希伯来历、伊斯兰历或法国大革命历名称的拼写。

与希伯来历有关的一个常见问题是忌日纪念日（称为“yahrzeit”）的计算。Emacs 日历包含了进行这类计算的功能。如果你正处于日历中，命令 ‘M-x calendar-hebrew-list-yahrzeits’ 会向你询问一个年份范围，然后列出光标点处日期在这些年份中的忌日纪念日。如果你不在日历中，该命令会先询问死亡日期和年份范围，然后列出忌日纪念日的清单。

33.10 日记

Emacs 日记与日历配合，按天记录约会或其他事件。要使用日记功能，你必须先创建一个日记文件，其中包含事件及其日期的列表。此后 Emacs 就能自动挑出并显示今天、最近的将来或任意指定日期的事件。

尽管你很可能从手工创建日记开始，但 Emacs 提供了许多命令，让你查看、添加和修改日记条目。

33.10.1 日记文件

“日记文件”是记录与特定日期相关联的文件的文件。日记文件的名称由变量 ‘diary-file’ 指定。默认值是 ‘~/.emacs.d/diary’，不过为了与旧版本兼容，如果 ‘~/diary’ 存在，Emacs 会使用它。

日记文件中的每个条目描述一个事件，由一个或多个行组成。条目总是以位于左边界的日期说明开头。条目的其余部分就是描述该事件的文本。如果条目多于一行，那么第一行之后的行必须以空白字符开头，以表明它们延续前一个条目。既不以有效日期开头、又不延续前一条目的行会被忽略。下面是一个例子：

```
12/22/2015  Twentieth wedding anniversary!
10/22       Ruth's birthday.
* 21, *:    Payday
Tuesday--weekly meeting with grad students at 10am
         Supowit, Shen, Bitner, and Kapoor to attend.
1/13/89     Friday the thirteenth!!
thu 4pm     squash game with Lloyd.
mar 16      Dad's birthday
April 15, 2016 Income tax due.
* 15        time cards due.
```

这个例子用额外的空格把大多数条目的事件描述对齐。这种排版纯粹是个人喜好问题。

你也可以采用这样的格式：日记条目的第一行只包含日期或星期名（后面不跟空白或标点）。例如：

```
02/11/2012
      Bill B. visits Princeton today
      2pm Cognitive Studies Committee meeting
      2:30-5:30 Liz at Lawrenceville
      4:00pm Dentist appt
      7:30pm Dinner at George's
      8:00-10:00pm concert
```

如果你使用简单的日记显示（参见“日记显示”），这个条目看起来会有所不同。简单的日记显示会省去开头的日期行，只出现那些续行。当你只显示某一天的条目时，这种条目样式看起来更整洁，但如果你要查看不止一天的条目，就可能造成混淆。

33.10.2 显示日记

创建了日记文件之后，你就可以用日历查看它。你也可以在日历模式之外查看今天的事件。下面提到的键绑定都是针对日历缓冲区的。

- ‘mouse-3 Diary’、‘d’：显示所选日期的全部日记条目（‘diary-view-entries’）。
- ‘s’：显示整个日记文件（‘diary-show-all-entries’）。
- ‘m’：标记所有可见的、有日记条目的日期（‘diary-mark-entries’）。
- ‘u’：取消日历窗口中的标记（‘calendar-unmark’）。
- ‘M-x diary-print-entries’：把当前样式的日记显示打印成硬拷贝。
- ‘M-x diary’：显示今天日期的全部日记条目。
- ‘M-x diary-mail-entries’：把即将到来的日记条目以电子邮件提醒的形式发送给你自己。

用‘d’（‘diary-view-entries’）显示日记条目时，会在一个单独的缓冲区中显示日历中所选日期的日记条目。新缓冲区的模式行显示这些日记条目的日期。节日会显示在该缓冲区或模式行中，取决于你选择的显示方式（参见“日记显示”）。如果你为‘d’指定了数字参数，它就会显示随后那么多天的全部日记条目。因此，‘2 d’会显示所选日期以及后一天的全部条目。

显示某日日记条目的另一种方法是在该日期上点击‘mouse-3’，然后从弹出的菜单中选择‘Diary entries’。如果变量‘calendar-view-diary-initially-flag’的值非‘nil’，那么创建日历时就会列出当前日期的日记条目（前提是当前日期可见）。

要更宽泛地了解哪些日子在日记中被提到，请使用‘m’（‘diary-mark-entries’）命令。它用不同的字体（face）标记有日记条目的日期。参见“定制日历”

该命令既作用于当前可见的月份，也作用于之后滚动才变为可见的月份。要关闭标记并清除当前的标记，请按‘u’（‘calendar-unmark’），这同时也会关闭节日标记（参见“节假日”）。如果变量‘calendar-mark-diary-entries-flag’的值非‘nil’，那么创建或更新日历时会自动标记日记中的日期。

若要阻止某个日记条目在日历中被标记，请在条目的开头、日期之前插入‘diary-nonmarking-symbol’所指定的字符串（默认为‘&’）。这对该条目在日记缓冲区中的显示没有影响，只影响日历中的日期标记。非标记条目对于那些否则会标记许多不同日期的通用条目很有用。

要查看完整的日记文件，而不是只看其中一部分条目，请使用‘s’（‘diary-show-all-entries’）命令。

命令‘M-x diary’显示当前日期的日记条目，与日历显示无关，并且可以选择性地显示随后几天的条目；变量‘diary-number-of-entries’指定要包含多少天。参见“定制日记”

如果你把‘(diary)’放进你的‘.emacs’文件，那么启动 Emacs 时就会自动显示一个带有当天日记条目的窗口。

有些人喜欢通过电子邮件接收日记中事件的通知。要把这类邮件发送给自己，请使用命令‘M-x diary-mail-entries’。前缀参数指定要检查多少天（从今天开始）；否则由变量‘diary-mail-days’说明检查多少天。

33.10.3 日期格式

下面是一些日记条目示例，演示了格式化日期的不同方式。这些例子都以美式顺序（月、日、年）显示日期，但日历模式作为选项也支持欧洲顺序（日、月、年）和 ISO 顺序（年、月、日）。

```
4/20/12  Switch-over to new tabulation system
apr. 25  Start tabulating annual results
4/30  Results for April are due
*/25  Monthly cycle finishes
Friday  Don't leave without backing up files
```

第一条只出现一次，即 2012 年 4 月 20 日。第二条和第三条在每年指定的日期出现，第四条用通配符（星号）代替月份，因此它在每个月的 25 日出现。最后一条在每周五出现。

你可以只用数字来表示日期，如 'MONTH/DAY' 或 'MONTH/DAY/YEAR'。其后必须跟一个非数字字符。在日期本身中，MONTH 和 DAY 是一位或两位的数字。可选的 YEAR 也是数字，可以缩写为最后两位数字；也就是说，你可以用 '11/12/2012' 或 '11/12/12'。

日期也可以写成 'MONTHNAME DAY' 或 'MONTHNAME DAY, YEAR' 的形式，其中月份名可以拼写完整或缩写（带不带句点均可）。月和日名称的首选缩写可以用变量 'calendar-abbrev-length'、'calendar-month-abbrev-array' 和 'calendar-day-abbrev-array' 设定。默认情况下，取名称的前三个字母作为其缩写。大小写不敏感。

日期可以是"通用的"，即部分未指定。这样该条目就适用于所有匹配该说明的日期。如果日期中不含年份，它就是通用的，适用于任何年份。或者，MONTH、DAY 或 YEAR 也可以是 *''；它们分别匹配任何月份、任何日子或任何年份。因此，日记条目 '3//' 匹配任何年份三月中的任何一天；'march'* 也是如此。

如果你更喜欢欧洲式的日期写法（日在月之前），或 ISO 式写法（顺序为年、月、日），可以在日历中输入 'M-x calendar-set-date-style'，或定制变量 'calendar-date-style'。这会影响日记日期如何被解释、日期的显示方式，以及某些命令期望其参数按什么顺序给出。

你可以用一个星期中某天的名称作为通用日期，它适用于落在该星期几的任何日期。你可以按上文所述缩写星期几的名称，也可以拼写完整；大小写不敏感。

33.10.4 向日记添加条目的命令

在日历中时，有若干命令可用于创建日记条目。这里列出基本命令；更复杂的命令在下一节（参见"特殊日记条目"）。条目也可以基于非公历的历法。参见"使用非公历的日记条目"。

- 'i d'：为所选日期添加一个日记条目（'diary-insert-entry'）。
- 'i w'：为所选的星期几添加一个日记条目（'diary-insert-weekly-entry'）。
- 'i m'：为所选的一个月中的某日添加一个日记条目（'diary-insert-monthly-entry'）。
- 'i y'：为所选的一年中的某日添加一个日记条目（'diary-insert-yearly-entry'）。

你可以在日历窗口中选择某个日期，然后输入 'i d'（'diary-insert-entry'）命令，从而为这个特定日期创建日记条目。该命令会在另一个窗口中显示你的日记文件的末尾并插入日期；随后你就可以输入该日记条目的其余内容。

如果你想创建适用于某个特定星期几的日记条目，就选中该星期几（哪一次出现都可以），然后输入 'i w'（'diary-insert-weekly-entry'）。这会把星期几作为通用日期插入；随后你就可以输入该条目的其余内容。你可以用同样的方式创建每月一次的日记条目：选中月中的某日，使用 'i m'（'diary-insert-monthly-entry'）命

令，然后输入条目的其余内容。类似地，你可以用‘i y’（‘diary-insert-yearly-entry’）命令插入每年一次的日记条目。

上述所有命令默认都会标记日记条目。要创建非标记的日记条目，请给命令加前缀参数。例如，‘C-u i w’会创建非标记的每周日记条目。

修改日记文件时，务必在退出 Emacs 之前保存该文件。在使用了上述任一插入命令之后保存日记文件，如果条件合适，会自动更新日历窗口中的日记标记。你可以随时使用‘calendar-redraw’命令强制更新。

33.10.5 特殊日记条目

除基于日历日期的条目之外，日记文件还可以包含用于周年纪念等规律性事件的“sexp 条目”。这类条目基于 Lisp 表达式（sexp），Emacs 在扫描日记文件时会求值它们。sexp 条目不包含日期，而是包含‘%%’后跟一个必须以圆括号开头和结尾的 Lisp 表达式。该 Lisp 表达式决定条目适用于哪些日期。

日历模式提供了插入某些常用 sexp 条目的命令：

- ‘i a’：为所选日期添加周年纪念日记条目（‘diary-insert-anniversary-entry’）。
- ‘i b’：为当前区域添加块日记条目（‘diary-insert-block-entry’）。
- ‘i c’：添加从该日期开始的循环日记条目（‘diary-insert-cyclic-entry’）。

如果你想创建适用于某个特定日期的周年纪念的日记条目，请把光标点移到该日期并使用‘i a’（‘diary-insert-anniversary-entry’）命令。这会在另一个窗口中显示你的日记文件的末尾并插入周年纪念说明；随后你就可以输入该日记条目的其余内容。该条目看起来像这样：

```
%%(diary-anniversary 10 31 1988) Arthur's birthday
```

该条目适用于 1988 年之后任何一年的 10 月 31 日；‘10 31 1988’指定了该日期。（如果你使用的是欧洲或 ISO 日历风格，月、日、年的输入顺序会有所不同。）这个表达式之所以需要起始年份，是因为高级日记函数可以用它来计算经过的年数。

“块”日记条目适用于指定的连续日期范围。下面是一个适用于 2012 年 6 月 24 日到 2012 年 7 月 10 日全部日期的块日记条目：

```
%%(diary-block 6 24 2012 7 10 2012) Vacation
```

‘6 24 2012’表示起始日期，‘7 10 2012’表示终止日期。（同样，如果你使用的是欧洲或 ISO 日历风格，月、日、年的输入顺序会有所不同。）

要插入块条目，请把光标点和标记放在作为该范围起止的两个日期上，然后输入‘i b’（‘diary-insert-block-entry’）。该命令会在另一个窗口中显示你的日记文件的末尾并插入块说明；随后你就可以输入该日记条目。

“循环”日记条目按固定的天数间隔重复。要创建一个这样的条目，请选择起始日期并使用‘i c’（‘diary-insert-cyclic-entry’）命令。该命令会提示输入间隔长度，然后插入条目，其形式如下：

```
%%(diary-cyclic 50 3 1 2012) Renew medication
```

该条目适用于 2012 年 3 月 1 日以及其后的每第 50 天；‘3 1 2012’ 指定起始日期。（如果你使用的是欧洲或 ISO 日历风格，月、日、年的输入顺序会有所不同。）

这三个命令都会标记日记条目。要插入非标记条目，请给命令加前缀参数。例如，‘C-u i a’ 会创建非标记的周年纪念日记条目。

在日历中标记 sexp 日记条目可能很耗时，因为日历窗口中可见的每个日期都必须单独检查。因此，在可能的情况下，最好用 ‘&’ 把 sexp 日记条目设为非标记的。

另一种复杂形式的 sexp 条目是“浮动”日记条目，它以天、周和月为单位指定偏移量，用来描述规律性发生的事件。它可以与由 ‘cron’ 工具解释的 crontab 条目相比。下面是一个适用于 11 月第四个星期四的非标记浮动日记条目：

```
&%%(diary-float 11 4 4) American Thanksgiving
```

其中的 11 表示十一月（第十一个月），4 表示星期四（一周中的第四天，星期日编号为零），第二个 4 表示第四个星期四（1 表示“第一个”，2 表示“第二个”，−2 表示“倒数第二个”，依此类推）。月份可以是单个的月份，也可以是月份的列表。因此你可以把上面的 11 改成 “(1 2 3)’，使该条目适用于一月、二月和三月的最后一个星期四。如果月份为 ‘t’，则该条目适用于一年中的所有月份。

```
%%(diary-offset '(diary-float t 3 4) 2) Monthly committee meeting
```

该条目适用于每月第三个星期四之后的那个星期六。其中的 2 指定当 sexp “(diary-float t 3 4)’ 求值为 ‘t’ 之后经过的天数。例如，当你的组织在每个月第三个星期四举行的会议之后两天召开委员会会议时，或者当你希望参加因不同时区而日期不同的虚拟会议时，这就很有用。

每个标准的 sexp 日记条目都接受一个可选参数，用来指定在日历中标记该条目时使用的字体（face）名称或单字符字符串。更一般地说，sexp 日记条目可以执行任意计算来决定自己何时适用。参见“S 表达式条目与花式日记显示”。

33.10.6 约会提醒

如果你为某个约会写了日记条目，并且该条目以一个可识别的时间开头，Emacs 就能提前提醒你有一个约会即将到来。Emacs 会按变量 ‘appt-display-format’ 指定的、你所选定的格式显示一条消息来提醒你。若 ‘appt-audible’ 的值非 ‘nil’，提醒中还会包含声音提示。此外，若 ‘appt-display-mode-line’ 非 ‘nil’，Emacs 会在模式行上显示距该约会还有多少分钟。

若 ‘appt-display-format’ 的值为 ‘window’，则变量 ‘appt-display-duration’ 控制提醒窗口可见多长时间；变量 ‘appt-disp-window-function’ 和 ‘appt-delete-window-function’ 分别给出用于创建和销毁该窗口的函数名。

要启用约会提醒，输入 ‘M-x appt-activate’。正参数启用提醒；负参数禁用提醒；无参数则切换。启用提醒还会根据日记文件为当天建立一份约会列表，列出所有带可识别时间的日记条目，并在每个约会之前提醒你。

例如，假设日记文件包含以下内容：

```
Monday
  9:30am Coffee break
 12:00pm Lunch
```

那么在星期一，大约 9:20am 会提醒你喝咖啡休息，大约 11:50am 会提醒你吃午饭。变量 'appt-message-warning-time' 指定提前多少分钟（默认 12）提醒你。这是一个默认的提醒时间。每个约会都可以通过添加一段匹配 'appt-warning-time-regexp' 的内容来指定不同的提醒时间（细节见该变量的文档）。

时间可以用 am/pm 风格书写（其中 '12:00am' 表示午夜，'12:00pm' 表示正午），也可以用 24 小时的欧洲/军用风格。你无需保持一致；日记文件里可以混用两种风格。时间必须位于日记条目的开头才会被识别。

Emacs 会在刚过午夜时自动根据日记文件更新约会列表。你也可以随时重新启用约会提醒来强制更新。这两种操作还会显示当天的日记缓冲区，除非你把 'appt-display-diary' 设为 'nil'。每当日记文件（或它包含的文件；参见"花式日记显示"）被保存时，约会列表也会更新。如果你使用 Org Mode，并把约会保存在 Org 议程文件中，可以用 'org-agenda-to-appt' 命令把这些约会加入列表。参见《Org 手册》的"Weekly/daily agenda, for more about that command"

你还可以把约会提醒功能当作闹钟使用。命令 'M-x appt-add' 可以把条目加入约会列表，而不影响你的日记文件。用 'M-x appt-delete' 可以从约会列表删除条目。

33.10.7 日记条目的导入与导出

你可以在 Emacs 日记文件与其他格式之间转移日记条目。

33.10.7.1 把 iCalendar 数据导入为日记条目

"iCalendar"是一种用于交换日历数据的互联网标准格式。许多日历应用都能导出和导入 iCalendar 格式的数据。iCalendar 数据也常作为邮件附件发送。iCalendar 数据通常使用 '.ics' 文件扩展名，在邮件中以 'text/calendar' MIME 类型发送。（关于 MIME 和邮件附件的更多信息，参见"邮件杂项"。）

'diary-icalendar' 包让你可以在 Emacs 日记中使用 iCalendar 数据。你可以在 iCalendar 格式与 Emacs 日记文件之间导入和导出数据，也可以直接在日记中显示 iCalendar 数据。

以下命令会把 iCalendar 数据导入你的日记文件：

'diary-icalendar-import-file' 把 iCalendar 文件导入 Emacs 日记文件。

'diary-icalendar-import-buffer' 把当前缓冲区中的 iCalendar 数据导入 Emacs 日记文件。

'diary-icalendar-import-buffer' 也适合从邮件附件导入 iCalendar 数据。例如，使用 Rmail 邮件客户端时，你可以这样写：

```
(add-hook 'rmail-show-message-hook #'diary-icalendar-import-buffer)
```

日记导入依赖若干用户可定制的变量，它们位于 'diary-icalendar-import' 定制组中。你可以用 'M-x customize-group' 查看并定制这些变量。参见"自定义组"。

iCalendar 数据被组织成若干"组件"（component），分别表示日历事件（VEVENT 组件）、任务（VTODO）和其他文本数据（VJOURNAL）。由于这些组件包含不同类型的数据，它们由不同的函数导入，具体由以下变量决定：

'diary-icalendar-vevent-format-function' 用于为日记格式化 VEVENT 组件的函数。

'diary-icalendar-vjournal-format-function' 用于为日记格式化 VJOURNAL 组件的函数。

‘diary-icalendar-vtodo-format-function’ 用于为日记格式化 VTODO 组件的函数。

你可以编写自己的格式化函数来定制导入的日记条目的格式。把这类函数写成名为“骨架”（skeleton）的模板会很方便（但并非必需）。参见《Autotype 手册》的“Top, for more about skeletons”

例如，假设你只想导入每个日历事件的日期、时间、摘要和地点，并把它们写在这样一行里：

```
2025/11/11 Summary @ Some Location
```

那么你可以把导入格式化函数写成一个骨架，并把它设为 ‘diary-icalendar-vevent-format-function’ 的值，如下所示：

```
(require 'skeleton)
```

```
(defun simple-vevent (_)
  "Format a VEVENT summary and location on a single line"
  (skeleton-insert
    '(nil
      ical-start-to-end & " " & ical-summary & " "
      (when ical-location "@ ") & ical-location "\n")))
```

```
(setopt diary-icalendar-vevent-format-function #'simple-vevent)
```

此例中的变量 ‘ical-start-to-end’、‘ical-summary’ 和 ‘ical-location’ 在调用骨架时会被动态绑定到相应的值。更多信息见 ‘diary-icalendar-vevent-format-function’ 的文档字符串。

导入过程中遇到的任何错误都会报告在名为 ‘*icalendar-errors*’ 的缓冲区中。你可以用 ‘next-error’ 命令逐条查看这些错误。参见“编译模式”。如果你经常需要导入格式不正确的 iCalendar 数据，为此提供了若干钩子；参见 ‘icalendar-parser’ 定制组。

33.10.7.2 在日记中显示 iCalendar 条目

如果你的日历数据主要存放在 Emacs 之外，但你仍想在 Emacs 日历和日记中看到它们，可以让日记文件包含一个 iCalendar 文件。

例如，假设你从外部服务器把日历下载到一个名为 ‘Appointments.ics’ 的文件中。那么你可以在日记文件里写入这样一行，把这个文件包含进来：

```
#include "path/to/Appointments.ics"
```

你还必须设置一些钩子，以便把该文件中的数据作为日记条目显示出来，并在日历中标记它们：

```
(add-hook 'diary-mark-entries-hook
          #'diary-mark-included-diary-files)
(add-hook 'diary-nongregorian-marking-hook
          #'diary-icalendar-mark-entries)
(add-hook 'diary-list-entries-hook
          #'diary-include-other-diary-files)
(add-hook 'diary-nongregorian-listing-hook
          #'diary-icalendar-display-entries)
```

这样，当你从日历中显示日记时，‘Appointments.ics’ 中的事件、任务和日志条目就会出现在相应的日期上。参见“显示日记”。

这样做的好处是，你不需要在日历服务器和日记文件之间同步数据。这比定期在日记格式与 iCalendar 格式之间导入导出更简单、更可靠。

你还可以用函数 ‘diary-icalendar-mailcap-viewer’ 在邮件消息中显示 iCalendar 附件，而不必把它们导入日记文件。例如，你可以把这个函数加入变量 ‘mailcap-user-mime-data’；更多信息见其文档字符串。

显示 iCalendar 条目与导入它们使用相同的基础设施，因此定制导入格式也会改变所显示条目的格式。参见“把 iCalendar 数据导入为日记条目”。

33.10.7.3 把日记条目导出为 iCalendar

以下命令会以 iCalendar 格式导出日记条目：

‘diary-icalendar-export-file’ 把日记文件导出为 iCalendar 格式。

‘diary-icalendar-export-region’ 把一段日记文本导出为 iCalendar 格式。

iCalendar 导出依赖若干用户可定制的变量，它们位于 ‘diary-icalendar-export’ 定制组中。你可以用 ‘M-x customize-group’ 查看并定制这些变量。参见“自定义组”。

把日记条目导出为 iCalendar 要求你在日记中遵循某些约定，这样才能从日记条目中解析出 iCalendar 属性。

默认情况下，导出器把条目的第一行（日期和时间之后的部分）用作 iCalendar 摘要，把条目的其余部分用作 iCalendar 描述。其他 iCalendar 属性也可以在条目中分行编码，像这样：

```
2025/11/11 Bender's birthday bash
  Location: Robot House
  Attendees:
    Fry <philip.fry@mars.edu>
    Günter <guenter@mars.edu>
```

这种格式与默认导入函数生成的格式一致。

如果你定制了导入格式，或者想以另一种格式导出日记条目，就需要定制导出变量，以识别你日记条目的格式。最常见的 iCalendar 属性是用正则表达式从日记条目中解析出来的。要修改这些属性的解析方式，参见 ‘diary-icalendar-export’ 定制组中名为 ‘diary-icalendar-*-regexp’ 的变量。

如果你需要导出其他 iCalendar 属性，或者需要更复杂的解析，可以定义一个函数来完成，并把它设为变量 ‘diary-icalendar-other-properties-parser’ 的值；细节见其文档字符串。

默认情况下，导出器假定每个日记条目表示一个 iCalendar 事件。如果你喜欢采用每天一个条目、把不同事件写在同一条目后续行上的日记格式，仍然可以把这类条目导出为彼此不同的 iCalendar 事件。为此，请把变量 ‘diary-icalendar-export-linewise’ 设为非 nil 值。

例如，设置该变量之后，像这样的条目：

```
2025-05-03
  9AM Lab meeting
    Günter to present on new assay
```

```
Start experiment A
12:30-1:30PM Lunch with Phil
16:00 Experiment A finishes; move to freezer
```

会被导出为四个事件，它们都在同一天，但开始时间各不相同（第二个事件“Start experiment A”除外，它没有开始时间）。更多信息见‘diary-icalendar-export-linewise’的文档字符串。

33.10.7.4 把 Outlook 约会导入为日记条目

你还可以从 Outlook 生成的约会消息中导入日记条目。在 Rmail 或 Gnus 中查看此类消息时，执行‘M-x diary-from-outlook’即可导入该条目。你可以通过定制变量‘diary-outlook-formats’让此命令识别更多约会消息格式。其他邮件客户端可以把‘diary-from-outlook-function’设为适当的值。

33.11 夏令时

Emacs 懂得标准时间与夏令时之间的区别——给出的日出、日落、二至点、二分点以及月相的时间都考虑到了这一点。夏令时的规则因地点而异，历史上也逐年变化。为了正确处理，Emacs 需要知道该用哪套规则。

一些操作系统会记录适用于你所在地点的规则；在这些系统上，Emacs 会自动从系统获取所需信息。如果这些信息缺少一部分或全部，Emacs 会用马萨诸塞州剑桥市（Cambridge, Massachusetts）当前使用的规则来填补。如果得到的规则不合你意，你可以通过设置某些变量来告诉 Emacs 该用哪些规则：‘calendar-daylight-savings-starts’和‘calendar-daylight-savings-ends’。

这些值应当是引用变量‘year’的 Lisp 表达式，求值结果为夏令时开始或（相应地）结束时的公历日期，形式为列表‘(MONTH DAY YEAR)’。如果你所在地区不使用夏令时，这些值应为‘nil’。

Emacs 用这些表达式来确定假日列表中夏令时的起始日期，并用于校正太阳和月亮计算中的时刻。

马萨诸塞州剑桥市的取值如下：

```
(calendar-nth-named-day 2 0 3 year)
(calendar-nth-named-day 1 0 11 year)
```

也就是说，‘year’所指定年份的第三个月（三月）的第二个第 0 日（星期日），以及该年第十一个月（十一月）的第一个星期日。如果夏令时改为从 10 月 1 日开始，你就要把‘calendar-daylight-savings-starts’设为：

```
(list 10 1 year)
```

如果你所在地点没有夏令时，或者你希望所有时间都用标准时间，请把‘calendar-daylight-savings-starts’和‘calendar-daylight-savings-ends’设为‘nil’。

变量‘calendar-daylight-time-offset’指定夏令时与标准时间之差，以分钟计。马萨诸塞州剑桥市的值为 60。

最后，两个变量‘calendar-daylight-savings-starts-time’和‘calendar-daylight-savings-ends-time’指定当地午夜之后多少分钟应发生进入和退出夏令时的切换。马萨诸塞州剑桥市这两个变量的值都是 120。

33.12 时间区间累计

timeclock 包会把时间区间累加起来，因此你可以（例如）记录自己花在特定项目上的时间。（更高级的替代方案是使用 Org Mode 的时间计时功能，参见《Org 手册》的“Clocking Work Time”。）

开始处理某个项目时使用 ‘M-x timeclock-in’ 命令，做完时使用 ‘M-x timeclock-out’ 命令。每次这样做，都会给该项目的记录增加一个时间区间。你可以用 ‘M-x timeclock-change’ 切换到另一个项目。

当你从若干时间区间收集到数据之后，可以用 ‘M-x timeclock-workday-remaining’ 查看今天还剩多少工作时间（按每天 8 小时的典型平均值计算），用 ‘M-x timeclock-when-to-leave’ 计算你什么时候可以下班。

如果你想让 Emacs 在模式行上显示工作日剩余的时间，可以定制变量 ‘timeclock-mode-line-display’ 并把它的值设为 ‘t’，或者调用 ‘M-x timeclock-mode-line-display’ 命令。

终止当前的 Emacs 会话未必意味着你已经停止处理该项目；默认情况下 Emacs 会询问你。不过，你可以把变量 ‘timeclock-ask-before-exiting’ 的值定制为 ‘nil’ 来避免这个询问；这样，只有显式的 ‘M-x timeclock-out’ 或 ‘M-x timeclock-change’ 才会告诉 Emacs 当前区间已经结束。

timeclock 函数通过把数据累积到名为 ‘~/.emacs.d/timelog’ 的文件中来工作。你可以通过定制变量 ‘timeclock-file’ 来为这个文件指定别的名字。如果你手工编辑 timeclock 文件，或者改动了 timeclock 任何可定制变量的值，就应当运行 ‘M-x timeclock-reread-log’ 命令，以便根据文件更新 Emacs 中的数据。

33.13 日历与日记的更多高级特性

本节介绍日历和日记的一些较为高级/专门的特性。首先介绍按个人喜好定制日历和日记的诸多方法中的一部分。

33.13.1 定制日历

遗憾的是，日历的显示宽度无法从三个月改变，但你可以通过设置下列变量来定制所用的空白：‘calendar-left-margin’、‘calendar-day-header-width’、‘calendar-day-digit-width’、‘calendar-column-width’ 和 ‘calendar-intermonth-spacing’。要在各月_之间_显示文本，例如周数，请按其文档所述定制变量 ‘calendar-intermonth-header’ 和 ‘calendar-intermonth-text’。

变量 ‘calendar-month-header’ 控制日历中每个月上方显示的文本。默认显示月份和年份。变量 ‘calendar-day-header-array’ 控制每个月每天各列上方显示的文本。默认显示每天名称的前两个字母。

变量 ‘calendar-holiday-marker’ 指定如何标记属于节日的日期。其值可以是一个单字符字符串（插入日期旁边），也可以是一个用于显示该日期的字体名。同样，变量 ‘diary-entry-marker’ 指定如何标记有日记条目的日期。函数 ‘calendar-mark-today’ 使用 ‘calendar-today-marker’ 来标记今天的日期。默认情况下，日历为此使用名为 ‘holiday’、‘diary’ 和 ‘calendar-today’ 的字体。

启动日历时会运行普通钩子 ‘calendar-initial-window-hook’。重新计算日历显示不会运行这个钩子。但如果你用 ‘q’ 命令离开日历再重新进入，该钩子会再次运行。

变量 ‘calendar-today-visible-hook’ 是一个普通钩子，在用日历准备好日历缓冲区之后、且当前日期在窗口中可见时运行。这个钩子的一个用途是标记今天的日期；要做到这一点，请使用函数 ‘calendar-mark-today’ 或 ‘calendar-star-date’：

```
(add-hook 'calendar-today-visible-hook 'calendar-mark-today)
```

如果当前日期在窗口中_不_可见，则会运行一个类似的普通钩子 'calendar-today-invisible-hook'。

每个日历光标移动命令在移动光标之后都会运行钩子 'calendar-move-hook'。

33.13.2 定制节日

有几个变量列出 Emacs 所知的默认节日。它们是：'holiday-general-holidays'、'holiday-local-holidays'、'holiday-solar-holidays'、'holiday-bahai-holidays'、'holiday-christian-holidays'、'holiday-hebrew-holidays'、'holiday-islamic-holidays'、'holiday-oriental-holidays' 和 'holiday-other-holidays'。这些名字应当不言自明；例如，'holiday-solar-holidays' 列出与太阳和月亮有关的节日。

你可以按下文所述删除或添加节日，把这些节日列表定制为自己的需要。把其中任何一个设为 'nil' 就不再显示相应的节日。

一般节日默认是全美国通行的节日。与此相对，'holiday-local-holidays' 和 'holiday-other-holidays' 默认都是空的。前者用于系统范围的设置，后者供你个人使用。

默认情况下，Emacs 并不包含它所知的各宗教的全部节日，只包含世俗日历中常见的那些。若要更广泛地收录宗教节日，可以把变量 'calendar-bahai-all-holidays-flag'、'calendar-christian-all-holidays-flag'、'calendar-hebrew-all-holidays-flag' 或 'calendar-islamic-all-holidays-flag' 中的任意一个（或全部）设为 't'。

每个节日变量都是"节日形式"（holiday form）的列表，每个形式描述一个节日（有时是一组节日）。下表列出节日形式可能的各种类型。日号和月号从 1 开始计数，但"dayname"编号把星期日算作 0。参数 STRING 始终是节日的描述，为字符串。

'(holiday-fixed MONTH DAY STRING)' 公历上的固定日期。

'(holiday-float MONTH DAYNAME K STRING &optional DAY)' 公历日期 MONTH、DAY 之后或之前的第 K 个 DAYNAME（DAYNAME=0 表示星期日，依此类推）。K 为负数表示从月末往前数。可选参数 DAY 在 K 为正时默认为 1，否则默认为 MONTH 的最后一天。

'(holiday-chinese MONTH DAY STRING)' 中国农历上的固定日期。

'(holiday-hebrew MONTH DAY STRING)' 希伯来历上的固定日期。

'(holiday-islamic MONTH DAY STRING)' 伊斯兰历上的固定日期。

'(holiday-julian MONTH DAY STRING)' 儒略历上的固定日期。

'(holiday-sexp SEXP STRING)' 由 Lisp 表达式 SEXP 计算出的日期。该表达式应使用变量 'year' 计算并返回节日的日期，形式为列表 '(MONTH DAY YEAR)'；如果该节日当年不发生，则返回 'nil'。

'(if CONDITION HOLIDAY-FORM)' 仅在 CONDITION 为真时发生的节日。

'(FUNCTION [ARGS])' 由函数 FUNCTION 以参数 ARGS 调用所计算出的日期列表。

例如，假设你想加入法国在 7 月 14 日（即第七个月的第十四天）庆祝的巴士底日。你可以这样做：

```
(setq holiday-other-holidays '((holiday-fixed 7 14 "Bastille Day")))
```

许多节日发生在某个月份的某个特定星期几、特定时间。下面这个节日形式描述维尔京群岛在 7 月第四个星期一庆祝的飓风祈祷日（Hurricane Supplication Day）：

```
(holiday-float 7 1 4 "Hurricane Supplication Day")
```

这里 7 表示七月，1 表示星期一（星期日是 0，星期二是 2，依此类推），4 表示该月第四次出现（1 表示第一次出现，2 表示第二次出现，−1 表示最后一次出现，−2 表示倒数第二次出现，依此类推）。

你也可以指定巴哈伊历、中国农历、希伯来历、伊斯兰历和儒略历中固定日期的节日。例如，

```
(setq holiday-other-holidays
      '((holiday-hebrew 10 2 "Last day of Hanukkah")
        (holiday-islamic 3 12 "Mohammed's Birthday")
        (holiday-julian 4 2 "Jefferson's Birthday")))
```

这段代码添加了光明节的最后一天（因为希伯来月份从尼散月起以 1 编号）、庆祝穆罕默德生日的伊斯兰节日（因为伊斯兰月份从穆哈兰月起以 1 编号），以及托马斯·杰斐逊的生日，按儒略历为 1743 年 4 月 2 日。

若要按条件加入节日，可使用 Emacs Lisp 的 'if' 或 'holiday-sexp' 形式。例如，美国总统选举在能被 4 整除的年份的 11 月第一个星期一之后的第一个星期二举行：

```
(holiday-sexp '(if (zerop (% year 4))
                   (calendar-gregorian-from-absolute
                    (1+ (calendar-dayname-on-or-before
                         1 (+ 6 (calendar-absolute-from-gregorian
                                 (list 11 1 year)))))))
              "US Presidential Election")
```

或者

```
(if (zerop (% displayed-year 4))
    (holiday-fixed 11
           (calendar-extract-day
             (calendar-gregorian-from-absolute
               (1+ (calendar-dayname-on-or-before
                     1 (+ 6 (calendar-absolute-from-gregorian
                             (list 11 1 displayed-year)))))))
           "US Presidential Election"))
```

有些节日因为确定日期时要进行特殊计算，无法套用上述任何形式。这种情况下必须编写 Lisp 函数来完成计算。例如要加入日月食，可把 '(eclipses)' 加到 'holiday-other-holidays'，并编写 Emacs Lisp 函数 'eclipses'，让它返回一个（可能为空的）列表，其中是日历窗口中可见范围内相关的公历日期及其描述字符串，如下所示：

```
(((6 4 2012) "Lunar Eclipse") ((11 13 2012) "Solar Eclipse") ... )
```

33.13.3 从玛雅历转换

以下命令用于按玛雅历选择日期：

- 'g m l'：移动到长纪年历指定的日期（'calendar-mayan-goto-long-count-date'）。

- ‘g m n t’：移动到 tzolkin 历中某个位置的下一出现（‘calendar-mayan-next-tzolkin-date’）。
- ‘g m p t’：移动到 tzolkin 历中某个位置的上一次出现（‘calendar-mayan-previous-tzolkin-date’）。
- ‘g m n h’：移动到 haab 历中某个位置的下一出现（‘calendar-mayan-next-haab-date’）。
- ‘g m p h’：移动到 haab 历中某个位置的上一次出现（‘calendar-mayan-previous-haab-date’）。
- ‘g m n c’：移动到历法周期中某个位置的下一出现（‘calendar-mayan-next-calendar-round-date’）。
- ‘g m p c’：移动到历法周期中某个位置的上一次出现（‘calendar-mayan-previous-calendar-round-date’）。

要理解这些命令，需要先了解玛雅历法。“长纪年”（long count）是按以下单位计算天数：

```
1 kin = 1 day   1 uinal = 20 kin   1 tun = 18 uinal
1 katun = 20 tun   1 baktun = 20 katun
```

因此，长纪年日期 12.16.11.16.6 表示 12 个 baktun、16 个 katun、11 个 tun、16 个 uinal 和 6 个 kin。Emacs 日历最早能处理 7.17.18.13.3 的玛雅长纪年日期，再早则不行。使用 ‘g m l’ 命令时，输入玛雅长纪年日期，baktun、katun、tun、uinal 和 kin 之间用句点分隔。

玛雅 tzolkin 历是由 13 天和 20 天两个独立周期构成的 260 天周期。由于该周期无限重复，Emacs 提供了向前和向后移动到周期中上一个或下一个位置的命令。输入 ‘g m p t’ 可转到上一个 tzolkin 日期；Emacs 会询问 tzolkin 日期，并把光标点移到该日期的上一次出现。类似地，输入 ‘g m n t’ 转到某个 tzolkin 日期的下一次出现。

玛雅 haab 历是一个 365 天周期，由 18 个每月 20 天的月份及其后的 5 天无月期组成。与 tzolkin 周期一样，该周期无限重复，同样有向前和向后移动到周期中上一个或下一个位置的命令。输入 ‘g m p h’ 可转到上一个 haab 日期；Emacs 会询问 haab 日期，并把光标点移到该日期的上一次出现。类似地，输入 ‘g m n h’ 转到某个 haab 日期的下一次出现。

玛雅人还使用 tzolkin 日期与 haab 日期的组合。这种组合构成约 52 年的周期，称为*历法周期*（calendar round）。若输入 ‘g m p c’，Emacs 会同时询问 haab 日期和 tzolkin 日期，然后把光标点移到该组合的上一次出现。使用 ‘g m n c’ 可把光标点移到某个组合的下一次出现。若输入的 haab/tzolkin 日期组合不可能存在，这些命令会报错。

Emacs 在要求输入玛雅名称时总是使用严格补全（参见“补全退出”），因此不必担心拼写。

33.13.4 日期显示格式

通过设置 ‘calendar-date-display-form’，可以定制日记、模式行和消息中日期的显示方式。该变量保存一个表达式列表，其中的表达式可以使用变量 ‘month’、‘day’ 和 ‘year’（三者都是字符串形式的数字），以及 ‘monthname’ 和 ‘dayname’（二者都是字母字符串）。在美国风格下，该列表的默认值如下：

```
((if dayname (concat dayname ", ")) monthname " " day ", " year)
```

而在欧洲风格下，默认值为：

```
((if dayname (concat dayname ", ")) day " " monthname " " year)
```

默认的 ISO 日期表示为：

```
((format "%s-%.2d-%.2d" year (string-to-number month)
         (string-to-number day)))
```

另一种典型的美式格式是：

```
(month "/" day "/" (substring year -2))
```

33.13.5 时间显示格式

日历和日记默认以美国惯用风格显示一天中的时间：小时为 1 到 12，加上分钟，以及 'am' 或 'pm'。如果你更喜欢欧洲风格（在美国也称为军用时间），即小时从 00 到 23，可以修改变量 'calendar-time-display-form'。该变量是一个表达式列表，其中的表达式可以使用变量 '12-hours'、'24-hours' 和 'minutes'（三者都是字符串形式的数字），以及 'am-pm' 和 'time-zone'（二者都是字母字符串）。默认值为：

```
(12-hours ":" minutes am-pm
          (if time-zone " (") time-zone (if time-zone ")"))
```

下面这个值提供欧洲风格的时间：

```
(24-hours ":" minutes
          (if time-zone " (") time-zone (if time-zone ")"))
```

注意，返回一天中时间的日历函数很少（目前只有太阳相关的函数）。

33.13.6 定制日记

通常，日记窗口会在模式行或缓冲区本身中提示与日记条目日期重合的节日。检查节日的过程可能很慢，具体取决于已定义的节日。在这种情况下，把 'diary-show-holidays-flag' 设为 'nil' 可以加快日记的显示速度。

变量 'diary-number-of-entries' 控制一次显示多少天的日记条目。它影响 'calendar-view-diary-initially-flag' 为 't' 时的初始显示，也影响命令 'M-x diary'。例如，值为 1（默认值）时只显示当天的日记条目，而值为 2 时还会显示第二天的条目。该值也可以是一个包含七个整数的向量：例如，若值为 '[0 2 2 2 2 4 1]'，则星期日不出现日记条目，星期一至星期四显示当天和第二天的条目，星期五显示星期五至星期一的条目，而星期六只显示当天的条目。

通过设置变量 'diary-date-forms'，可以定制日记文件中日期的形式。该变量是一个用于识别日期的模式列表。每个日期模式是一个列表，其元素可以是正则表达式（参见《Emacs Lisp 参考手册》的"Regular Expressions"），也可以是符号 'month'、'day'、'year'、'monthname' 和 'dayname'。所有这些元素都充当模式，匹配日记文件中特定类型的文本。要使整个日期模式匹配，它的所有元素必须连续匹配。

日期模式中的正则表达式按通常方式匹配，使用的是经过修改的标准语法表，其中 '*' 是单词组成字符。

符号 'month'、'day'、'year'、'monthname' 和 'dayname' 分别匹配所考虑日期的月份数字、日数字、年份数字、月份名称和星期名称。匹配数字的符号允许有前导零；匹配名称的符号允许大写和缩写（由 'calendar-month-abbrev-array' 和 'calendar-day-abbrev-array' 指定）。所有符号都能匹配 *''；由于日记条目中的 * 表示*"任意一天""任意一月"等，因此无论所考虑的日期是什么，它都应匹配。

美式风格下 'diary-date-forms' 的默认值由 'diary-american-date-forms' 提供：

```
((month "/" day "[^/0-9]")
 (month "/" day "/" year "[^0-9]")
 (monthname " *" day "[^,0-9]")
 (monthname " *" day ", *" year "[^0-9]")
 (dayname "\\W"))
```

变量 'diary-european-date-forms' 和 'diary-iso-date-forms' 提供其他默认风格。

列表中的日期模式必须*互斥*，并且不能匹配日记条目本身的内容，只能匹配日期和一个空白字符。如果为了使各模式互斥，模式必须匹配日记条目文本中超出结束日期的那处空白之后的部分，那么该日期模式的第一个元素*必须*是 'backup'。这会使日期识别器在完成匹配后回退到日记条目中当前单词的开头。即使使用了 'backup'，日期模式也绝对不能匹配到日记条目第一个单词之外的内容。例如，'diary-european-date-forms' 的默认值是：

```
((day "/" month "[^/0-9]")
 (day "/" month "/" year "[^0-9]")
 (backup day " *" monthname "\\W+\\<\\([^*0-9]\\|\\([0-9]+[:aApP]\\)\\)")
 (day " *" monthname " *" year "[^0-9]")
 (dayname "\\W"))
```

注意第三个模式中使用了 'backup'，因为它需要匹配日期本身之后某个单词的一部分，以便与第四个模式区分开。

33.13.7 使用非公历的日记条目

除基于标准公历的条目外，日记中还可以有基于巴哈伊历、中国农历、希伯来历或伊斯兰历日期的条目。不过，识别这类条目可能很耗时，而且大多数人并不使用它们，因此必须显式启用。例如，若希望日记识别希伯来日期条目，必须这样做：

```
(add-hook 'diary-nongregorian-listing-hook 'diary-hebrew-list-entries)
(add-hook 'diary-nongregorian-marking-hook 'diary-hebrew-mark-entries)
```

类似地，对于伊斯兰历、巴哈伊历和中国农历条目，要添加 'diary-islamic-list-entries' 和 'diary-islamic-mark-entries'、'diary-bahai-list-entries' 和 'diary-bahai-mark-entries'，或 'diary-chinese-list-entries' 和 'diary-chinese-mark-entries'。

这些日记条目的格式与公历日期日记条目相同，只是巴哈伊历日期前必须加 'diary-bahai-entry-symbol'（默认为 'B'），中国农历日期前加 'diary-chinese-entry-symbol'（默认为 'C'），希伯来历日期前加 'diary-hebrew-entry-symbol'（默认为 'H'），伊斯兰历日期前加 'diary-islamic-entry-symbol'（默认为 'I'）。此外，非公历的月份名称不能缩写（因为前三个字母常常并不唯一）。（另请注意，若要表示平年希伯来历的 Adar 月，必须使用 "Adar I"。）例如，希伯来历日期 Heshvan 25 的日记条目可以写成这样：

```
HHeshvan 25 Happy Hebrew birthday!
```

它会出现在希伯来历 Heshvan 25 所对应的任何日期的日记中。下面是一个匹配 Dhu al-Qada 25 的伊斯兰历日期日记条目：

```
IDhu al-Qada 25 Happy Islamic birthday!
```

与公历日期日记条目一样，如果非公历条目前面有‘diary-nonmarking-symbol’（默认为‘&’），则该条目不标记日历。

下表列出日历中用于创建日记条目的命令，这些条目与所选日期以及巴哈伊历、中国农历、希伯来历或伊斯兰历中相似的日期相匹配：

- ‘i h d’：‘diary-hebrew-insert-entry’
- ‘i h m’：‘diary-hebrew-insert-monthly-entry’
- ‘i h y’：‘diary-hebrew-insert-yearly-entry’
- ‘i i d’：‘diary-islamic-insert-entry’
- ‘i i m’：‘diary-islamic-insert-monthly-entry’
- ‘i i y’：‘diary-islamic-insert-yearly-entry’
- ‘i B d’：‘diary-bahai-insert-entry’
- ‘i B m’：‘diary-bahai-insert-monthly-entry’
- ‘i B y’：‘diary-bahai-insert-yearly-entry’
- ‘i C d’：‘diary-chinese-insert-entry’
- ‘i C m’：‘diary-chinese-insert-monthly-entry’
- ‘i C y’：‘diary-chinese-insert-yearly-entry’
- ‘i C a’：‘diary-chinese-insert-anniversary-entry’

这些命令的工作方式与普通日记条目的对应命令很相似：它们作用于日历窗口中光标点所在的日期，所做的只是把日记条目的日期部分插入到日记文件末尾。此后你必须自行插入该日记条目的其余部分。基本命令为特定的非公历日期添加条目，‘monthly’ 命令为每个月中给定的非公历月内日期添加条目，‘yearly’ 命令为每年的给定非公历日和月份添加条目。

33.13.8 日记显示

日记显示的工作方式是：先准备好日记条目列表，然后运行变量 ‘diary-display-function’ 指定的函数。默认值 ‘diary-fancy-display’ 通过把日记条目和节日复制到一个仅为显示而存在的特殊缓冲区来显示它们。把日记条目复制到单独的缓冲区，提供了修改显示文本使之更美观的机会——例如，按条目适用的日期排序。

通常，花式日记缓冲区不显示没有日记条目的日子，即使那天是节日。若希望这类日子也显示在花式日记缓冲区中，请把变量 ‘diary-list-include-blanks’ 设为 ‘t’。

花式日记缓冲区启用了 View 模式（参见“视图模式”）。

另一种显示方法 ‘diary-simple-display’ 显示实际的日记缓冲区，并用不可见文本隐藏不适用的条目。节日显示在模式行中。这种方法的优点是可以编辑缓冲区并把修改直接保存到日记文件。不过，这种方法不如花式方法灵活。例如，它不能对条目排序。另一个缺点是不可见文本可能造成混淆。例如，若复制一段文本以便粘贴到别处，不可见文本可能被一并包含。类似地，由于你看到的日记缓冲区只是一种假象，直接打印该缓冲区可能打印不出屏幕上显示的内容。

因此，有一个专门的命令用于按*其显示的样子*打印日记缓冲区的硬拷贝；该命令是‘M-x diary-print-entries’。它对两种显示方法都适用，不过在花式显示下你也可以像打印其他缓冲区一样打印它。要打印一周的逐日日记硬拷贝，请把光标点放在该周的第一天，输入‘7 d’，然后执行‘M-x diary-print-entries’。和平时一样，包含节日会使显示稍微变慢；把变量‘diary-show-holidays-flag’设为‘nil’可以加快速度。

该命令会准备一个临时缓冲区，其中只包含当前在日记缓冲区中可见的日记条目。与简单显示不同，其他无关条目是真正不存在的，而不只是被隐藏。准备好缓冲区后，它运行钩子‘diary-print-entries-hook’。该钩子的默认值用命令‘lpr-buffer’把数据直接送到打印机（参见“打印”）。若想改用其他命令打印，只需修改该钩子的值。其他用途可能包括，例如，把各行按日期和时间重新排序。

你可以编辑简单日记窗口中显示的日记条目，但务必记住，所显示的缓冲区包含*整个*日记文件，只是其中一部分被隐藏起来。例如，这意味着‘C-f’（‘forward-char’）命令可以把光标点放到看起来是行尾的位置，而实际上那是某个隐藏行的中间。

*在简单显示中编辑日记条目时要小心！*插入额外的行，或在可见行中间增删字符，都不会引起问题，但在行尾进行编辑可能得不到预期的结果。删除一行可能会删除它后面其他不可见的条目。在编辑简单日记缓冲区之前，最好先用‘s’（‘diary-show-all-entries’）显示整个文件。

33.13.9 花式日记显示

以下功能只在花式日记显示下有效。

可以使用普通钩子‘diary-list-entries-hook’按一天中的时间对每天的日记条目排序。方法如下：

```
(add-hook 'diary-list-entries-hook 'diary-sort-entries t)
```

对于每一天，这会按时间对以可识别的时间开头的日记条目排序。没有时间的日记条目在每一天中排在最前面。注意排序命令被放在钩子列表的末尾，以防列表中较早的成员改变日记条目的顺序或添加条目。

通过把变量‘diary-comment-start’和‘diary-comment-end’设为界定注释的字符串，可以在日记条目中写“注释”。花式显示不打印注释。你可能想把供其他包（例如预约包，参见“约会提醒”）使用的元数据放在注释中。

主日记文件可以包含其他文件。这使得一群人能够共享一个日记文件，用于记录与他们所有人都相关的事件。日记文件中以‘diary-include-string’开头的行：

```
#include "FILENAME"
```

会把文件 FILENAME 中的日记条目包含进花式日记缓冲区。包含机制是递归的，因此被包含的文件还可以包含其他文件，依此类推（当然，必须小心不要形成循环包含）。启用包含功能的方法如下：

```
(add-hook 'diary-list-entries-hook 'diary-include-other-diary-files)
(add-hook 'diary-mark-entries-hook 'diary-mark-included-diary-files)
```

包含机制只在花式日记显示下有效，因为简单日记显示直接从日记文件中显示条目。

33.13.10 S 表达式条目与花式日记显示

S 表达式日记条目让你能做的不仅是设置日记条目适用的复杂条件。在日记文件中，S 表达式条目前面应加‘diary-sexp-entry-symbol’（默认为‘%%’）。在花式日记显示下，S 表达式条目可以根据日期本身生成条目的文本。

例如，周年纪念日记条目可以把自周年日期以来的年数插入日记条目的文本中。于是下面这个日记条目中的‘%d’：

```
%%(diary-anniversary 10 31 1948) Arthur's birthday (%d years old)
```

会被替换为年龄，因此在 1990 年 10 月 31 日，该条目在花式日记缓冲区中显示如下：

```
Arthur's birthday (42 years old)
```

如果日记文件中改为包含这个条目：

```
%%(diary-anniversary 10 31 1948) Arthur's %d%s birthday
```

那么 1990 年 10 月 31 日在花式日记缓冲区中的条目显示如下：

```
Arthur's 42nd birthday
```

类似地，循环日记条目可以插入已经发生的重复次数：

```
%%(diary-cyclic 50 1 1 2012) Renew medication (%d%s time)
```

显示如下：

```
Renew medication (5th time)
```

这是 2012 年 9 月 7 日在花式日记显示中的样子。

有一种提前提醒的日记 S 表达式，它不仅把条目包含在发生当天，也包含在更早的日期。例如，若想在周年纪念日一周之前得到提醒，可以使用

```
%%(diary-remind '(diary-anniversary 12 22 1968) 7) Ed's anniversary
```

花式日记会在 12 月 15 日和 12 月 22 日都显示‘Ed's anniversary’。

函数‘diary-date’适用于由月、日、年组合描述的日期，其中每一项可以是整数、整数列表或‘t’（表示所有值）。例如，

```
%%(diary-date '(10 11 12) 22 t) Rake leaves
```

会使花式日记显示

```
Rake leaves
```

即每年的 10 月 22 日、11 月 22 日和 12 月 22 日。

函数 'diary-float' 允许你描述适用于"11 月的第三个星期五"或"4 月的最后一个星期二"这类日期的日记条目。参数是 MONTH、DAYNAME 和一个索引 N。条目出现在 MONTH 第一天之后的第 N 个 DAYNAME 上，其中 DAYNAME=0 表示星期日，1 表示星期一，依此类推。若 N 为负数，则从 MONTH 末尾向前计数。MONTH 的值可以是一个月份列表、单个月份，或 't'（指定所有月份）。还可以使用可选参数 DAY 来指定 MONTH 中 DAY 当天或之后/之前的第 N 个 DAYNAME；当 N 为正数时 DAY 默认为 1，当 N 为负数时默认为 MONTH 的最后一天。例如，

```
%%(diary-float t 1 -1) Pay rent
```

会使花式日记显示

```
Pay rent
```

即每月的最后一个星期一。

S 表达式日记条目的通用性使你可以指定任何能够用算法描述的日记条目。S 表达式日记条目包含一个表达式，用于计算该条目是否适用于给定日期。若其值非 'nil'，该条目适用于该日期；否则不适用。表达式可以使用变量 'date' 来获得所考虑的日期；它的值是一个列表 (MONTH DAY YEAR)，指的是公历。

当表达式的值非 'nil' 时，S 表达式日记条目适用于该日期，但有些值有更具体的含义。若值是字符串，该字符串就是该日期所发生事件的描述。值也可以具有 '(MARK . STRING)' 形式；此时 MARK 指定如何在日历中标记该日期，STRING 是事件的描述。若 MARK 是单字符字符串，该字符会出现在日历中该日期旁边。若 MARK 是一个字体（face）名称，该日期会以该字体显示。若 MARK 为 'nil'，表示对该日期不做特别高亮。

假设你在每月的 21 日领薪，前提是该日是工作日；若 21 日落在周末，则在它之前的那个星期五领薪。下面是一个能匹配这些日期的 sexp 日记条目的写法：

```
&%%(let ((dayname (calendar-day-of-week date))
         (day (cadr date)))
      (or (and (= day 21) (memq dayname '(1 2 3 4 5)))
          (and (memq day '(19 20)) (= dayname 5)))
         ) Pay check deposited
```

下列 sexp 日记条目利用了（花式日记显示中的）一种能力：编造出文本随日期而变化的日记条目：

- '%%(diary-sunrise-sunset)' 为今天的本地日出和日落时间生成一条日记条目。
- '%%(diary-lunar-phases)' 为月亮的位相（四分相）生成一条日记条目。
- '%%(diary-day-of-year)' 生成一条日记条目，给出今天在当前年份中的日序号以及当前年份剩余的天数。
- '%%(diary-iso-date)' 生成一条日记条目，给出今天对应的 ISO 商业日期。
- '%%(diary-julian-date)' 生成一条日记条目，给出今天对应的儒略历日期。
- '%%(diary-astro-day-number)' 生成一条日记条目，给出今天对应的天文学（儒略）日数。
- '%%(diary-bahai-date)' 生成一条日记条目，给出今天对应的巴哈伊历日期。
- '%%(diary-chinese-date)' 生成一条日记条目，给出今天对应的中国农历日期。
- '%%(diary-coptic-date)' 生成一条日记条目，给出今天对应的科普特历日期。
- '%%(diary-ethiopic-date)' 生成一条日记条目，给出今天对应的埃塞俄比亚历日期。

- ‘%%(diary-french-date)’ 生成一条日记条目，给出今天对应的法国革命历日期。
- ‘%%(diary-hebrew-date)’ 生成一条日记条目，给出今天对应的希伯来历日期。
- ‘%%(diary-islamic-date)’ 生成一条日记条目，给出今天对应的伊斯兰历日期。
- ‘%%(diary-mayan-date)’ 生成一条日记条目，给出今天对应的玛雅历日期。
- ‘%%(diary-persian-date)’ 生成一条日记条目，给出今天对应的波斯历日期。

例如，写入下面这条日记条目

```
&%%(diary-hebrew-date)
```

就会使每一天的日记显示都包含对应的希伯来历日期，前提是你使用花式日记显示。（若使用简单日记显示，则无论哪个日期，日记中出现的都是字面行 ‘&%%(diary-hebrew-date)’。）

该函数被用来构造若干标准的希伯来 sexp 日记条目：

- ‘%%(diary-hebrew-rosh-hodesh)’ 生成一条日记条目，说明每个希伯来新月的到来及其仪式宣告。
- ‘%%(diary-hebrew-parasha)’ 生成一条星期六的日记条目，说明当周犹太会堂的经文诵读。
- ‘%%(diary-hebrew-sabbath-candles)’ 生成一条星期五的日记条目，说明安息日点蜡烛的_本地时间_。
- ‘%%(diary-hebrew-omer)’ 生成一条日记条目，在适当的时候给出俄梅珥（omer）计数。
- ‘%%(diary-hebrew-yahrzeit MONTH DAY YEAR) NAME’ 生成一条日记条目，标记某个忌日的周年。该日期是_格里高利_（公历）的死亡日期。条目会出现在对应的希伯来历周年纪念日以及它的前一天。（参数顺序随日历日期风格而变；例如欧洲风格下为 DAY、MONTH、YEAR。）
- ‘%%(diary-hebrew-birthday MONTH DAY YEAR)’ 为希伯来历上的生日生成一条日记条目。

上面记录的所有函数都接受一个可选参数 MARK，它指定如何在日历显示中标记该日期。若其中某个函数判定自己适用于某一日期，它就返回一个含有 MARK 的值，如前所述。

34 发送邮件

要在 Emacs 中发送电子邮件，请键入 ‘C-x m’。这会切换到一个名为 ‘*unsent mail*’ 的缓冲区，你可以在其中编辑邮件的正文和邮件头。完成后，键入 ‘C-c C-s’ 或 ‘C-c C-c’ 将其发出。

- ‘C-x m’ 开始撰写邮件（‘compose-mail’）。
- ‘C-x 4 m’ 同样，但在另一个窗口中（‘compose-mail-other-window’）。
- ‘C-x 5 m’ 同样，但在新框架中（‘compose-mail-other-frame’）。
- ‘C-c C-s’ 在邮件缓冲区中发送邮件（‘message-send’）。
- ‘C-c C-c’ 在邮件缓冲区中发送邮件并掩埋该缓冲区（‘message-send-and-exit’）。

邮件缓冲区是普通的 Emacs 缓冲区，因此你在撰写邮件时仍可切换到其他缓冲区。如果你想在完成当前邮件之前再发送另一封邮件，可以再次键入 ‘C-x m’ 打开一个新的邮件缓冲区，其名称带有不同的数字后缀（参见“其他缓冲区操作”）。（这仅在你使用默认的 Message 模式撰写邮件时才有效；参见“邮件命令”。）如果你知道自己想继续编辑之前那封未发送的邮件，可以带前缀参数调用此命令，即 ‘C-u C-x m’，Emacs 会切换到你上次使用的邮件缓冲区，让你从中断处继续编辑该邮件。

命令 ‘C-x 4 m’（‘compose-mail-other-window’）与 ‘C-x m’ 相同，只是它在另一个窗口中显示邮件缓冲区。命令 ‘C-x 5 m’（‘compose-mail-other-frame’）则在新框架中这样做。参见“‘display-buffer’ 的工作方式”。

当你键入 ‘C-c C-c’ 或 ‘C-c C-s’ 发送邮件时，Emacs 可能会询问你应当如何投递该邮件——是直接通过 SMTP，还是使用某种其他方法。详情参见“发送邮件”。

34.1 邮件缓冲区的格式

下面是邮件缓冲区内容的一个例子：

```
To: subotai@example.org
CC: mongol.soldier@example.net, rms@gnu.org
Subject: Re: What is best in life?
From: conan@example.org
--text follows this line--
To crush your enemies, see them driven before you, and to
hear the lamentation of their women.
```

邮件缓冲区的顶部是一组“邮件头字段”（header field），用于指定关于邮件收件人、主题等信息。上面的缓冲区含有 ‘To’、‘CC’、‘Subject’ 和 ‘From’ 的邮件头字段。某些邮件头字段会在适当时被自动预置到邮件缓冲区中。

内容为 ‘--text follows this line--’ 的那一行把邮件头字段与邮件的“正文”（body，或称“文本”）分隔开。该行之上的所有内容都被当作邮件头的一部分；其下的一切都被当作正文。分隔行本身不会出现在实际发送的邮件中。

你可以用普通编辑命令插入和编辑邮件头字段。关于专用于编辑邮件头字段的命令，参见“编辑邮件头”。某些邮件头，例如 ‘Date’ 和 ‘Message-Id’，通常不在邮件缓冲区中出现，而是在发送邮件时自动生成。

34.2 邮件头字段

邮件缓冲区中的邮件头字段以行首的字段名开始，以冒号结束。字段名不区分大小写。冒号及其后的可选空白之后是字段的内容。

你可以为邮件头字段使用任何喜欢的名称，但通常人们只使用含义公认的标准字段名。

'From' 邮件头字段标明发送该邮件的人（也就是你）。它应当是有效的邮件地址，因为回复通常发送到那里。该字段的默认内容由变量 'user-full-name'（指定你的全名）和 'user-mail-address'（你的邮件地址）计算得出。在某些操作系统上，Emacs 会用环境变量初始化这两个变量（参见"通用变量"）。如果这些信息不可用或有误，你应当自行定制这些变量（参见"简易自定义界面"）。

除 'From' 之外，下面是常用字段的列表：

- 'To' 邮件的收件地址。要列出多个地址，用逗号分隔它们。
- 'Subject' 邮件的主题。
- 'CC' 发送该邮件的附加地址。它与 'To' 类似，只是这些读者不应把邮件看作直接针对他们的。
- 'BCC' 发送该邮件的附加地址，这些地址不应出现在实际发送的邮件头中。'BCC' 代表"密送副本"（blind carbon copies）。
- 'FCC' 一个文件名，已发送邮件的副本应追加到该文件。Emacs 以 mbox 格式写入邮件，除非该文件是 Babyl 格式（Emacs 23 之前 Rmail 使用的格式），此时 Emacs 以 Babyl 格式写入。如果有 Rmail 缓冲区正在访问该文件，Emacs 会相应地更新它。要指定多个文件，请使用多个 'FCC' 字段，每个字段中写一个文件名。
- 'Reply-To' 应当把回复发送到的地址，而非 'From'。如果由于某种原因你的 'From' 地址无法接收回复，就使用它。
- 'Mail-Reply-To' 此字段优先于 'Reply-To'。使用它是因为某些邮件列表出于自身目的设置 'Reply-To' 字段（这是一种颇有争议的做法）。
- 'Mail-Followup-To' 一个或多个地址，用作后续消息的默认收件人。通常在你回复自己所订阅的邮件列表中的某封邮件，并希望回复发往该列表而不额外抄送给你时使用。
- 'In-Reply-To' 你所回复的那封邮件的标识符。多数邮件阅读器用这一信息把相关邮件归为一组。通常，当你在 Emacs 内置的任何邮件程序中回复邮件时，这个邮件头会自动填好。
- 'References' 此前相关邮件的标识符。与 'In-Reply-To' 一样，它通常会为你自动填好。

'To'、'CC' 和 'BCC' 字段可以出现任意多次，每个这样的邮件头字段可以包含多个地址，用逗号分隔。这样你就可以指定任意多个发送地点。这些字段还可以有续行：紧跟在字段起始行之后、以一个或多个空白开头的一行或多行被视为该字段的一部分。下面是一个带续行的 'To' 字段示例：

```
To: foo@example.net, this@example.net,
  bob@example.com
```

你可以通过把变量 'mail-default-headers' 设为一个字符串，让 Emacs 把某些默认邮件头插入邮件缓冲区。这样 'C-x m' 就会把这个字符串插入邮件头中。例如，下面是为每封邮件添加 'Reply-To' 和 'FCC' 邮件头的方法：

```
(setq mail-default-headers
      "Reply-To: foo@example.com\nFCC: ~/Mail/sent")
```

如果默认邮件头字段不适合某封特定邮件，在发送之前按需编辑它们即可。

34.3 邮件别名

你可以定义“邮件别名”（mail alias），即代表一个或多个邮件地址的简短助记名称。默认情况下，邮件别名定义在文件‘~/.mailrc’中。你可以通过设置变量‘mail-personal-alias-file’来指定使用其他文件名。

要在‘~/.mailrc’中定义别名，请写这样一行：

```
alias NICK FULLADDRESSES
```

这表示 NICK 应展开为 FULLADDRESSES，其中 FULLADDRESSES 既可以是单个地址，也可以是用空格分隔的多个地址。例如，要让‘maingnu’代表‘gnu@gnu.org’加上你自己的一个本地地址，请写入这一行：

```
alias maingnu gnu@gnu.org local-gnu
```

如果地址中含有空格，请用一对双引号把整个地址引起来，像这样：

```
alias jsmith "John Q. Smith <none@example.com>"
```

注意，你无需在地址的各个部分（例如人的全名）周围加双引号。Emacs 会在需要时加上它们。例如，它会把上面的地址插入为‘"John Q. Smith" <none@example.com>’。

在编辑‘~/.mailrc’文件之后，或者如果该文件在 Emacs 之外被修改过，你可以用‘M-x rebuild-mail-abbrevs <RET>’更新正在运行的 Emacs 会话所使用的邮件别名。该命令会提示输入要使用的文件名，默认值是‘mail-personal-alias-file’的值。类似的命令‘merge-mail-abbrevs’会提示输入一个含有邮件别名的文件，然后把该文件中的别名与已有的别名合并。

另一种方式是使用 Emacs 命令定义邮件别名。命令‘define-mail-abbrev’会提示输入别名和完整地址，并把该别名定义为展开成该完整地址。Emacs 在提议保存所有文件时（针对‘C-x s’或‘C-x C-c’），会像对待其他缩写一样保存新增的别名（参见“保存缩写”）。

Emacs 也识别‘~/.mailrc’中的包含命令。它们看起来像这样：

```
source FILENAME
```

‘~/.mailrc’文件并非 Emacs 专用；许多其他邮件阅读程序也用它保存邮件别名，其中可以包含各种其他命令。不过，Emacs 除别名定义和包含命令之外会忽略一切。

邮件别名的展开方式与缩写相同——也就是说，在你于别名之后键入分词字符后立即展开（参见“缩写”）。这种展开只在‘To’、‘From’、‘CC’、‘BCC’和‘Reply-To’邮件头字段（以及它们的‘Resent-’变体）中发生；在诸如‘Subject’等其他邮件头字段中不会发生。

你也可以用命令‘M-x mail-abbrev-insert-alias’直接插入别名地址。该命令会（带补全地）读取一个别名，并在光标点处插入其定义。

命令‘mail-abbrev-complete-alias’对光标点之前的邮件别名进行补全。

34.4 邮件命令

‘*mail*’缓冲区的默认主模式称为 Message 模式。它在许多方面表现得像 Text 模式，但在‘C-c’前缀上提供了若干额外命令，使编辑邮件更方便。

本节将介绍 Message 模式中一些最常用的命令。Message 模式也有自己的手册，其中更详细地描述了它的功能。参见《Message 手册》。

34.4.1 发送邮件

- ‘C-c C-c’发送邮件，并掩埋邮件缓冲区（‘message-send-and-exit’）。
- ‘C-c C-s’发送邮件，并让邮件缓冲区保持为当前缓冲区（‘message-send’）。

常用的发送命令是‘C-c C-c’（‘message-send-and-exit’）。它发送邮件，然后掩埋邮件缓冲区，把它的重选优先级降到最低。如果你希望改为杀死邮件缓冲区，请把变量‘message-kill-buffer-on-exit’改为‘t’。

命令‘C-c C-s’（‘message-send’）发送邮件并让该缓冲区保持为当前缓冲区。如果你想修改邮件（也许是换成新的收件人）并再次发送，就用这个命令。

发送邮件会运行钩子‘message-send-hook’。它还会把邮件缓冲区标记为未修改，但邮件缓冲区同时又是访问文件的缓冲区时除外（在那种情况下，只有保存文件才会把它标记为未修改，而且你若试图把同一封邮件发送两次，也不会收到警告）。

变量‘message-send-mail-function’控制邮件的投递方式（Mail 模式使用‘send-mail-function’）。‘send-mail-function’的值应当是下列函数之一：

- ‘sendmail-query-once’询问投递方式（本列表中的其他条目之一），并对本邮件使用该方式；随后把该方式保存到‘send-mail-function’，以便今后的投递使用。这是默认值，除非你已经设置了通过‘smtpmail-send-it’发送邮件所需的变量（见下）。
- ‘smtpmail-send-it’通过外部邮件主机发送邮件，例如你的 Internet 服务提供商的对外 SMTP 邮件服务器。如果你没有告诉 Emacs 如何联系该 SMTP 服务器，它会提示输入这些信息，并保存到变量‘smtpmail-smtp-server’和文件‘~/.authinfo’中。参见《《smtpmail》手册》。
- ‘sendmail-send-it’使用系统的默认‘sendmail’程序或等价程序发送邮件。这要求系统已设置好直接通过 SMTP 投递邮件。
- ‘mailclient-send-it’把邮件缓冲区交给系统指定的邮件客户端。细节见文件‘mailclient.el’的注释部分。
- ‘feedmail-send-it’与‘sendmail-send-it’类似，但允许你把邮件排队以便稍后发送。细节见文件‘feedmail.el’的注释部分。

当你发送含有非 ASCII 字符的邮件时，它们需要用编码系统编码（参见“编码系统”）。通常编码系统由你所选的语言环境自动指定（参见“语言环境”）。你可以通过设置变量‘sendmail-coding-system’显式指定外发邮件的编码系统（参见“识别编码系统”）。如果由此确定的编码系统无法处理某封特定邮件中的字符，Emacs 会要求你选择要使用的编码系统，并显示一个可能的编码系统列表。参见“为输出选择编码系统”。

34.4.2 编辑邮件头

Message 模式提供以下专用命令，用于移动到特定的邮件头字段以及在邮件头中补全地址。

- ‘C-c C-f C-t’ 移到 ‘To’ 邮件头（‘message-goto-to’）。
- ‘C-c C-f C-s’ 移到 ‘Subject’ 邮件头（‘message-goto-subject’）。
- ‘C-c C-f C-c’ 移到 ‘CC’ 邮件头（‘message-goto-cc’）。
- ‘C-c C-f C-b’ 移到 ‘BCC’ 邮件头（‘message-goto-bcc’）。
- ‘C-c C-f C-r’ 移到 ‘Reply-To’ 邮件头（‘message-goto-reply-to’）。
- ‘C-c C-f C-f’ 移到 ‘Mail-Followup-To’ 邮件头字段（‘message-goto-followup-to’）。
- ‘C-c C-f C-w’ 添加一个新的 ‘FCC’ 邮件头字段，带文件名补全（‘message-goto-fcc’）。
- ‘C-c C-b’ 移到邮件正文的开头（‘message-goto-body’）。
- ‘<TAB>’ 补全邮件地址（‘message-tab’）。

把光标点移到特定邮件头字段的那些命令都以 ‘C-c C-f’ 为前缀（‘C-f’ 代表 “field”）。如果所问的字段不存在，命令就会创建一个（例外是 ‘mail-fcc’，它每次都创建一个新字段）。

命令 ‘C-c C-b’（‘message-goto-body’）把光标点移到邮件头分隔行之后，也就是移到正文的开头。

在编辑含有地址的邮件头字段（如 ‘To:’、‘CC:’ 和 ‘BCC:’）时，你可以键入 <TAB>（‘message-tab’）来补全地址。这会尝试基于几种方法插入与该地址对应的全名，其中包括 EUDC——一个能识别多种目录服务器协议的库（参见《《eudc》手册》）。如果这些方法都失败，它就会尝试把该地址作为邮件别名展开（参见“邮件别名”）。如果光标点位于不接受地址的邮件头字段上，或者位于邮件正文中，那么 <TAB> 只插入一个制表符。

34.4.3 引用邮件

- ‘C-c C-y’ 从邮件阅读器中粘贴所选邮件，作为引用（‘message-yank-original’）。
- ‘C-c C-q’ 对引用自另一封邮件的每个段落进行填充（‘message-fill-yanked-message’）。

你可以用命令 ‘C-c C-y’（‘message-yank-original’）来“引用”你正在回复的邮件。这会把那封邮件的文本插入邮件缓冲区。此命令仅在邮件缓冲区是由在 Emacs 中运行的邮件阅读器（如 Rmail）调起时才有效。

默认情况下，Emacs 在被引用文本的每一行前插入字符串 ‘>’；这个前缀字符串由变量 ‘message-yank-prefix’ 指定。如果你带前缀参数调用 ‘message-yank-original’，则不会插入引用前缀。

使用 ‘C-c C-y’ 之后，你可以键入 ‘C-c C-q’（‘message-fill-yanked-message’）来填充所引邮件的段落。一次 ‘C-c C-q’ 会逐个填充所有这样的段落。要填充所引邮件中的单个段落，请使用 ‘M-q’。如果填充不能自动处理你使用的引用前缀类型，可以尝试显式设置填充前缀。参见“填充文本”。

你可以通过钩子 ‘mail-citation-hook’ 定制邮件引用。例如，你可以使用 Supercite 包，它提供更灵活的引用方式（参见《《sc》手册》的“Introduction”）。

34.4.4 邮件杂项

你可以在邮件缓冲区中键入 ‘C-c C-a’（‘mml-attach-file’），把文件“附加”到外发邮件上。附加功能是使用多用途 Internet 邮件扩展（MIME）标准完成的。

‘mml-attach-file’ 命令会提示输入文件名，以及附件的“内容类型”（content type）、“描述”（description）和“处置方式”（disposition）。内容类型通常会被自动检测；只需键入 <RET> 接受默认值。描述是一行文本，收件人会看到它显示在附件旁边；你也可以让它留空。处置方式要么是 ‘inline’，表示收件人会在邮件正文内看到指向附件的链接；要么是 ‘attachment’，表示该链接与正文分开。

‘mml-attach-file’ 命令专用于 Message 模式；在 Mail 模式中请改用 ‘mail-add-attachment’。它只提示输入文件名，并会自动确定内容类型和处置方式。如果你想对附加的文件作些说明，请在邮件正文中键入这些说明。

附加文件的实际内容不会插入邮件缓冲区。取而代之的是，一些占位文本会被插入邮件缓冲区，像这样：

```
<#part type="text/plain" filename="~/foo.txt" disposition=inline>
<#/part>
```

当你键入 ‘C-c C-c’ 或 ‘C-c C-s’ 发送邮件时，附加的文件会随之一同投递。

撰写邮件时，你可以键入 ‘M-x ispell-message’ 对邮件文本进行拼写检查。如果你曾把收到的邮件粘贴到外发草稿中，该命令会跳过被粘贴的部分，但会检查你自己键入的文本（它依据缩进或 ‘mail-yank-prefix’ 来区分引用行和你输入的内容）。参见“拼写检查与纠正”。

打开 Message 模式（‘C-x m’ 会自动这样做）会运行常规钩子 ‘text-mode-hook’ 和 ‘message-mode-hook’。初始化一封新的外发邮件会运行常规钩子 ‘message-setup-hook’；如果你想改变邮件缓冲区的外观，可以使用这个钩子。参见“钩子”。

这些钩子之间的主要区别只在于它们被调用的时机。每当你键入 ‘C-x m’，‘message-mode-hook’ 会在邮件缓冲区创建后立即运行。然后 ‘message-setup’ 函数插入缓冲区的默认内容。在这些默认内容插入之后，‘message-setup-hook’ 运行。

如果你用 ‘C-x m’ 继续一封已有的撰写信件，‘message-mode-hook’ 会在切换到该邮件缓冲区后立即运行。如果该缓冲区未被修改，或者你决定清空它重新开始，那么 ‘message-setup-hook’ 会在默认内容插入后运行。

34.5 邮件签名

你可以把一段标准文本——你的“邮件签名”（mail signature）——加到每封邮件的末尾。这个签名可以包含你的电话号码或你的实际位置等信息。变量 ‘message-signature’ 决定 Emacs 如何处理邮件签名。

‘message-signature’ 的默认值是 ‘t’；这表示在文件 ‘~/.signature’ 中查找你的邮件签名。如果该文件存在，其内容会被自动插入邮件缓冲区的末尾。你可以通过变量 ‘message-signature-file’ 更改签名文件。

如果你把 ‘message-signature’ 改为一个字符串，那就直接指定了签名的文本。

如果你把 ‘message-signature’ 改为 ‘nil’，Emacs 就不会自动插入你的邮件签名。你可以在邮件缓冲区中键入 ‘C-c C-w’（‘message-insert-signature’）来插入邮件签名。Emacs 会在签名文件中查找你的签名。

如果你撰写邮件时使用 Mail 模式而不是 Message 模式，那么决定签名如何发送的对应变量是 ‘mail-signature’ 和 ‘mail-signature-file’。

按照惯例，邮件签名应以内容为 ‘-- ’ 的一行作为标记。如果你的签名缺少这个前缀，Emacs 会为你加上。签名的其余部分不应超过四行。

34.6 邮件娱乐

‘M-x spook’ 会向外发邮件中添加一行随机选出的关键词。这些关键词从一组词中选出，那些词暗示你正在谈论某种颠覆性的东西。

此功能背后的想法是怀疑 NSA(1) 和其他情报机构会窥探所有含有他们可能觉得有趣的关键词的电子邮件。（这些机构说他们不会，但那正是他们*会*说的话。）这个想法是：如果许多人都往邮件里加上可疑的词，这些机构就会被大量虚假输入弄得忙不过来，最终不得不放弃阅读全部内容。无论这是否属实，它至少让一些人觉得有趣。

你可以用 ‘fortune’ 程序把一条“幸运饼干”消息放进外发邮件。为此，把 ‘fortune-to-signature’ 加到 ‘mail-setup-hook’ 中：

```
(add-hook 'mail-setup-hook 'fortune-to-signature)
```

使用这个之前，你大概需要设置变量 ‘fortune-file’。

> 注：(1) 美国国家安全局。

34.7 邮件撰写方法

本章我们介绍了 Emacs 中用于编辑和发送邮件的常用模式——Message 模式。它只是若干可用模式之一。在 Emacs 23.2 之前，默认模式是 Mail 模式，它在许多方面与 Message 模式相似，但功能不那么丰富；例如，它只支持基本的 MIME：允许你添加附件，但缺少更复杂的 MIME 功能。另一个可用的模式是 MH-E（参见《《mh-e》手册》）。

你可以选择这些“邮件用户代理”（mail user agent）中的任何一个，作为你编辑和发送邮件的首选方式。命令 ‘C-x m’、‘C-x 4 m’ 和 ‘C-x 5 m’ 会使用你所指定的那个代理；Emacs 中其他发送邮件的部分（例如错误报告器）也是如此（参见“报告 Bug”）。要指定邮件用户代理，请定制变量 ‘mail-user-agent’。目前，合法的值包括 ‘message-user-agent’（Message 模式）、‘sendmail-user-agent’（Mail 模式）、‘gnus-user-agent’ 和 ‘mh-e-user-agent’。可能还有更多选项可用；详情请查阅你所用的邮件用户代理包的手册。你也可以用 ‘define-mail-user-agent’ 定义另一个邮件用户代理。

若你选择了不同的邮件撰写方式，本章关于邮件缓冲区与 Message 模式的信息便不再适用；其他方式在另一个缓冲区中使用不同的文本格式，其命令也各不相同。

同样，要指定你偏好的读邮件方式，请定制变量 ‘read-mail-command’。默认值为 ‘rmail’（参见“用 Rmail 读信”）。

35 用 Rmail 读信

Rmail 是 Emacs 的一个子系统，用于阅读和处理收到的邮件。Rmail 把邮件消息存放在称为 Rmail 文件的文件中。在 Rmail 文件中阅读消息是在一个特殊的主模式——Rmail 模式——中进行的，该模式重新定义了大多数字母键，使它们运行管理邮件的命令。

Emacs 还附带一个复杂得多、也灵活得多的读邮件子系统，称为 Gnus。Gnus 是一个非常大的软件包，因此在它自己的手册中介绍，参见《Gnus 手册》。

- 其他邮箱格式：从各种格式的本地邮箱中取信。

35.1 Rmail 基本概念

以最简单的方式使用 Rmail 时，你只有一个 Rmail 文件‘~/RMAIL’，你的全部邮件都保存在其中。它被称为你的“主 Rmail 文件”。命令‘M-x rmail’读取你的主 Rmail 文件，从收件箱合并新邮件进来，显示你尚未阅读的第一条消息，并让你开始阅读。变量‘rmail-file-name’指定主 Rmail 文件的名称。

Rmail 一次只显示 Rmail 文件中的一条消息。被显示的消息称为“当前消息”。Rmail 模式的特殊命令可以完成诸如删除当前消息、把它复制到另一个文件、发送回复或移到另一条消息等操作。你也可以创建多个 Rmail 文件（参见“文件处理”），并用 Rmail 在它们之间移动消息（参见“把消息复制到文件”）。

在 Rmail 文件内，消息通常按接收顺序依次排列；你可以指定其他排序方式（参见“排序 Rmail 文件”）。消息用连续整数标识，这些整数就是它们的“消息编号”。当前消息的编号显示在 Rmail 的模式行中，其后是文件中消息的总数。你可以用‘j’键指定消息编号来移到某条消息（参见“在消息之间移动”）。

按照 Emacs 的通常惯例，对 Rmail 文件的改动只有在你保存文件后才永久生效。你可以用‘s’（‘rmail-expunge-and-save’）保存，它还会先把已删除的消息从文件中清除（参见“删除消息”）。若只想保存而不清除，请使用‘C-x C-s’。Rmail 在从收件箱文件合并新邮件后会自动保存 Rmail 文件（参见“Rmail 文件与收件箱”）。

你可以用‘q’（‘rmail-quit’）退出 Rmail；它会清除并保存 Rmail 文件，然后掩藏 Rmail 缓冲区以及它的摘要缓冲区（如果存在）（参见“摘要”）。但你不必正式退出。如果你从 Rmail 切换到其他缓冲区中编辑，并且再也不切换回来，你就已经退出了。只要确保最终保存 Rmail 文件（就像任何其他你改动过的文件一样）。‘C-x s’是这样做的一种合适方式（参见“保存文件的命令”）。Rmail 命令‘b’（‘rmail-bury’）掩藏 Rmail 缓冲区及其摘要缓冲区，而不清除和保存 Rmail 文件。

35.2 在消息内滚动

当 Rmail 显示的消息在屏幕上放不下时，你必须滚动浏览才能读完其余部分。你可以用通常的滚动命令做到这一点：‘C-v’、‘M-v’和‘M-<’（参见“滚动”），但在 Rmail 中滚动如此频繁，值得让它更方便。

- ‘<SPC>’：向前滚动（‘scroll-up-command’）。
- ‘<DEL>’、‘S-<SPC>’：向后滚动（‘scroll-down-command’）。
- ‘.’：滚动到消息开头（‘rmail-beginning-of-message’）。
- ‘/’：滚动到消息末尾（‘rmail-end-of-message’）。

由于阅读邮件时最常见的事情是按屏滚动浏览，Rmail 让 <SPC> 和 <DEL>（或 'S-<SPC>'）分别等同于 'C-v'（'scroll-up-command'）和 'M-v'（'scroll-down-command'）。

命令 '.'（'rmail-beginning-of-message'）滚动回到所选消息的开头。这与 'M-<' 并不完全相同：一方面，它不设置标记；另一方面，如果你改动过当前消息的缓冲区边界（例如通过编辑，参见"在邮件内部编辑"），它会重置这些边界。类似地，命令 '/'（'rmail-end-of-message'）向前滚动到所选消息的末尾。

35.3 在消息之间移动

对一条消息最基本的操作就是阅读它。在 Rmail 中，做法是让该消息成为当前消息。通常的实践是按顺序在文件中移动，因为这就是消息接收的顺序。进入 Rmail 时，你位于你尚未设为当前消息的第一条消息处（也就是第一条具有 'unseen' 属性的消息；参见"Rmail 属性"）。向前移动可查看其他新消息；向后移动可重新查看旧消息。

- 'n'：移到下一条未删除的消息，跳过中间被删除的消息（'rmail-next-undeleted-message'）。
- 'p'：移到上一条未删除的消息（'rmail-previous-undeleted-message'）。
- 'M-n'：移到下一条消息，包括已删除的消息（'rmail-next-message'）。
- 'M-p'：移到上一条消息，包括已删除的消息（'rmail-previous-message'）。
- 'C-c C-n'：移到下一条与当前消息主题相同的消息（'rmail-next-same-subject'）。
- 'C-c C-p'：移到上一条与当前消息主题相同的消息（'rmail-previous-same-subject'）。
- 'j'：移到第一条消息。带参数 N 时，移到编号为 N 的消息（'rmail-show-message'）。
- '>'：移到最后一条消息（'rmail-last-message'）。
- '<'：移到第一条消息（'rmail-first-message'）。
- 'M-s REGEXP <RET>'：移到下一条包含与 REGEXP 匹配的文本的消息（'rmail-search'）。
- '- M-s REGEXP <RET>'：移到上一条包含与 REGEXP 匹配的文本的消息。（这是带负参数的 'M-s'。）

'n' 和 'p' 是在 Rmail 中在消息之间移动的常用方式。它们按顺序遍历消息，但跳过已删除的消息，这通常正是你想要的。它们的命令定义名为 'rmail-next-undeleted-message' 和 'rmail-previous-undeleted-message'。如果你不想跳过已删除的消息——例如，你想移到某条消息以取消删除它——请使用变体 'M-n' 和 'M-p'（'rmail-next-message' 和 'rmail-previous-message'）。给这些命令中的任何一个加数字参数都用作重复次数。

在 Rmail 中，只输入数字就可以指定数字参数，不必先输入 'C-u'。你也可以只输入 '-' 来指定负参数。

'M-s'（'rmail-search'）命令是 Rmail 版的搜索。常用的增量搜索命令 'C-s' 在 Rmail 中也能用，但它只在当前消息内搜索。'M-s' 的目的是搜索另一条消息。它以非增量方式读取一个正则表达式（参见"正则表达式的语法"），然后从下一条消息的开头开始搜索匹配，随后选中那条消息。如果 REGEXP 为空，'M-s' 会重用上一次使用的正则表达式。

要在文件中向后搜索另一条消息，给 'M-s' 一个负参数。在 Rmail 中，你可以用 '- M-s' 做到这一点。它从上一条消息的末尾开始搜索。

也可以根据标签搜索消息。参见"标签"。

‘C-c C-n’（‘rmail-next-same-subject’）命令移到下一条与当前消息主题相同的消息。前缀参数用作重复次数。带负参数时，此命令向后移动，行为类似 ‘C-c C-p’（‘rmail-previous-same-subject’）。比较主题时，这些命令忽略通常加在回复主题前的那些前缀。这些命令用于阅读与同一主题（又称“线索”）相关的全部消息。

要移到由绝对消息编号指定的消息，请用 ‘j’（‘rmail-show-message’）并以消息编号为参数。不带参数时，‘j’ 选中第一条消息。‘<’（‘rmail-first-message’）也选中第一条消息。‘>’（‘rmail-last-message’）选中最后一条消息。

35.4 删除消息

当你不再需要保留某条消息时，可以“删除”它。这会把它标记为可忽略，某些 Rmail 命令会假装它已不存在；但它仍在 Rmail 文件中有其位置，并且仍有自己的消息编号。

对 Rmail 文件进行“清除”（expunge）才会真正移除已删除的消息。其余消息会重新连续编号。

- ‘d’：删除当前消息，并移到下一条未删除的消息（‘rmail-delete-forward’）。
- ‘C-d’：删除当前消息，并移到上一条未删除的消息（‘rmail-delete-backward’）。
- ‘u’：取消删除当前消息，或向后移到上一条已删除的消息并取消删除它（‘rmail-undelete-previous-message’）。
- ‘x’：清除 Rmail 文件（‘rmail-expunge’）。

有两个用于删除消息的 Rmail 命令。两者都删除当前消息并选中另一条。‘d’（‘rmail-delete-forward’）移到后一条消息，跳过已经删除的消息；而 ‘C-d’（‘rmail-delete-backward’）移到上一条未删除的消息。如果在指定方向上没有未删除的消息可移到，则刚删除的消息仍为当前消息。数字前缀参数用作重复次数，以便在单条命令中删除多条消息。负参数会颠倒 ‘d’ 和 ‘C-d’ 的含义。

每当 Rmail 删除一条消息时，它都会运行钩子 ‘rmail-delete-message-hook’。钩子函数被调用时，消息已被标记为删除，但它仍是 Rmail 缓冲区中的当前消息。

要使所有已删除的消息最终从 Rmail 文件中消失，请输入 ‘x’（‘rmail-expunge’）。在此之前，你仍可以“取消删除”已删除的消息。取消删除命令 ‘u’（‘rmail-undelete-previous-message’）旨在大多数情况下抵消 ‘d’ 命令的效果。如果当前消息已删除，它就取消删除当前消息。否则它向后移到之前的消息，直到找到一条已删除的消息，并取消删除该消息。数字前缀参数用作重复次数，以便在单条命令中取消删除多条消息。

通常你可以用 ‘u’ 撤销 ‘d’，因为 ‘u’ 会向后移回并取消删除 ‘d’ 所删除的那条消息。但当 ‘d’ 跳过了紧随被删除消息之后的几条已删除消息时，这就不奏效了；此时 ‘u’ 命令会取消删除被跳过的最后一条消息。没有干净的办法避免这个问题。不过，通过反复执行 ‘u’ 命令，你最终可以回到你想取消删除的那条消息。你也可以用 ‘M-p’ 命令选中某条已删除的消息，然后输入 ‘u’ 来取消删除它。

已删除的消息具有 ‘deleted’ 属性，因此当当前消息已删除时，模式行中会显示 ‘deleted’。事实上，删除或取消删除消息不过是添加或移除这个属性。参见“Rmail 属性”。

35.5 Rmail 文件与收件箱

当你在本地收到邮件时，操作系统会把你的来信放在一个我们称为“收件箱”的文件中。启动 Rmail 时，它会运行一个名为 ‘movemail’ 的 C 程序，把新消息从你的收件箱复制到你的主 Rmail 文件中，该文件还包含此前

Rmail 会话中保存的其他消息。你实际上是在这个文件中用 Rmail 阅读邮件。这个操作称为“取新邮件”。在 Rmail 中，你可以随时输入‘g’来取新邮件。

变量‘rmail-primary-inbox-list’包含作为主 Rmail 文件收件箱的文件列表。如果你没有显式设置这个变量，Rmail 会使用‘MAIL’环境变量，或者，作为最后的手段，使用基于‘rmail-spool-directory’的默认收件箱。默认收件箱文件取决于你的操作系统；通常是‘/var/mail/USERNAME’、‘/var/spool/mail/USERNAME’或‘/usr/spool/mail/USERNAME’。

你可以用命令‘set-rmail-inbox-list’为当前会话中的任何 Rmail 文件指定收件箱文件；参见“多个 Rmail 文件”。

把 Rmail 文件与收件箱分开有两个原因。

1. 收件箱文件的格式因操作系统以及所用的其他邮件软件而异。Rmail 只需要有一部分了解这些不同格式，而且它只需知道如何把它们全部转换成 Rmail 自己的格式。

1. 访问收件箱文件而不冒丢失邮件的风险非常麻烦，因为必须与邮件投递互锁。而且，不同的操作系统使用不同的互锁技术。一旦把邮件从收件箱一次性地移到单独的 Rmail 文件中，这种策略就避免了在 Rmail 其余部分中需要互锁，因为只有 Rmail 会操作 Rmail 文件。

Rmail 使用 Unix 和 GNU 系统为收件箱文件引入的标准‘mbox’格式，作为 Rmail 文件的内部格式。（实际上，有几种略有不同的 mbox 格式。这些差异并不很重要，但你可以设置变量‘rmail-mbox-format’来告诉 Rmail 你的系统使用哪种形式。详见该变量的文档。）

取新邮件时，Rmail 先把新邮件从收件箱文件复制到 Rmail 文件；然后保存 Rmail 文件；然后清空收件箱文件。这样，系统崩溃可能造成收件箱与 Rmail 文件之间邮件重复，但不会丢失邮件。如果‘rmail-preserve-inbox’非‘nil’，那么 Rmail 在取新邮件时不清空收件箱文件。例如，你可能希望在旅行时通过 POP 查收邮件的便携式计算机上设置它，这样你的邮件会留在服务器上，你以后可以在主桌面工作站上保存它。

在某些情况下，Rmail 会间接地从收件箱文件复制新邮件。首先它运行‘movemail’程序，把邮件从收件箱移到与 Rmail 文件同目录下名为‘.newmail-INBOXNAME’的中间文件。然后 Rmail 从该文件合并新邮件，保存 Rmail 文件，之后才删除该中间文件。如果在错误的时刻发生崩溃，这个文件会继续存在，Rmail 下次从该收件箱取新邮件时会再次使用它。

如果 Rmail 无法把‘.newmail-INBOXNAME’中的数据转换成 mbox 格式，它会把该文件重命名为‘RMAILOSE.N’（N 是为使名称唯一而选定的整数），这样 Rmail 就不会再次因这些数据而出问题。你应当查看该文件，找出让 Rmail 困惑的那条消息（可能是一条包含 control-underscore 字符、八进制代码 037 的消息），并删除它。然后你可以用‘1 g’从修正后的文件中取新邮件。

35.6 多个 Rmail 文件

默认情况下，Rmail 操作你的“主 Rmail 文件”，它名为‘~/RMAIL’，从系统收件箱文件接收你的来信。但你也可以有其他 Rmail 文件并用 Rmail 编辑它们。这些文件可以通过各自的收件箱接收邮件，或者你可以用显式的 Rmail 命令把消息移入其中（参见“把消息复制到文件”）。

- ‘i FILE <RET>’：把 FILE 读入 Emacs 并对它运行 Rmail（‘rmail-input’）。
- ‘g’：从当前 Rmail 文件的收件箱合并新邮件（‘rmail-get-new-mail’）。

- ‘C-u g FILE <RET>’：从收件箱文件 FILE 合并新邮件。

要对主 Rmail 文件之外的文件运行 Rmail，你可以在 Rmail 中使用 ‘i’（‘rmail-input’）命令。这会在 Rmail 模式下访问该文件。即使不在 Rmail 中，你也可以使用 ‘M-x rmail-input’，但输入 ‘C-u M-x rmail’ 更容易，它做同样的事。

用 ‘i’ 读取的文件通常应当是有效的 mbox 文件。如果不是，Rmail 会尝试把其文本转换成 mbox 格式，并在缓冲区中访问转换后的文本。如果你保存该缓冲区，就会转换该文件。

如果你指定的文件名不存在，‘i’ 会初始化一个新缓冲区，用于创建新的 Rmail 文件。

你也可以从菜单中选择一个 Rmail 文件。在 Classify 菜单中，选择 Input Rmail File 项；然后选择你想要的 Rmail 文件。变量 ‘rmail-secondary-file-directory’ 和 ‘rmail-secondary-file-regexp’ 指定菜单中提供哪些文件：第一个变量说明在哪个目录中查找；第二个说明该目录中提供哪些文件（所有匹配该正则表达式的文件）。如果没有文件匹配，你就不能选择这个菜单项。这些变量也适用于选择输出文件（参见“把消息复制到文件”）。

要使用的收件箱文件由变量 ‘rmail-inbox-list’ 指定，它在 Rmail 模式中是缓冲区局部的。作为特殊的例外，如果你没有为主 Rmail 文件指定任何收件箱文件，它会使用 ‘MAIL’ 环境变量，或你的标准系统收件箱。

‘g’ 命令（‘rmail-get-new-mail’）把邮件从其收件箱合并到当前 Rmail 文件。如果该 Rmail 文件没有收件箱，‘g’ 什么也不做。命令 ‘M-x rmail’ 也会把新邮件合并到你的主 Rmail 文件。

要从不是通常收件箱的文件合并邮件，给 ‘g’ 键一个数字参数，如 ‘C-u g’。然后它读取一个文件名，并从该文件合并邮件。使用带参数的 ‘g’ 时，收件箱文件不会被删除或以任何方式改变。因此，这是把一个消息文件合并到另一个文件中的通用方法。

35.7 把消息复制到文件

这些命令把消息从 Rmail 文件复制到另一个文件。

- ‘o FILE <RET>’：把当前消息的完整副本追加到文件 FILE（‘rmail-output’）。
- ‘C-o FILE <RET>’：把当前消息按显示的样子复制一份，追加到文件 FILE（‘rmail-output-as-seen’）。
- ‘w FILE <RET>’：只把消息正文输出到文件 FILE，默认文件名取自消息的 ‘Subject’ 头。

‘o’ 和 ‘C-o’ 命令把当前消息复制到指定文件，添加在其末尾。正前缀参数用作重复次数：从当前消息开始，连续复制指定数量的消息到指定文件，并忽略已删除的消息。

这两个命令的主要区别在于复制多少内容：‘o’ 复制完整的消息头，即使它们并非全部可见；而 ‘C-o’ 恰好只复制当前显示的那些头，不多复制。参见“邮件的显示”。此外，如果文件是 Babyl 格式，‘o’ 会把消息转换成 Babyl 格式（Emacs 22 及更早版本的 Rmail 使用该格式）；‘C-o’ 则完全不能输出到 Babyl 文件。

如果输出文件当前正在某个 Emacs 缓冲区中访问，输出命令会把消息追加到该缓冲区。最终把它保存到文件由你负责。

有时你收到的消息正文包含某个文件的内容。你可以用 ‘w’ 命令（‘rmail-output-body-to-file’）把正文（不包括消息头）保存到文件。这类消息常在 ‘Subject’ 字段中包含预期的文件名，因此 ‘w’ 命令用 ‘Subject’ 字段作为输出文件名的默认值（在替换某些无法可移植地用于文件名的字符之后）。不过，文件名是用迷你缓冲区读取的，所以如果你愿意，可以指定不同的名字。

你也可以把消息输出到通过菜单选择的 Rmail 文件。在 Classify 菜单中，选择 Output Rmail File 菜单项；然后选择你想要的 Rmail 文件。这会把当前消息输出到该文件，就像 'o' 命令那样。变量 'rmail-secondary-file-directory' 和 'rmail-secondary-file-regexp' 指定菜单中提供哪些文件：第一个变量说明在哪个目录中查找；第二个说明该目录中提供哪些文件（所有匹配该正则表达式的文件）。如果没有文件匹配，你就不能选择这个菜单项。

用 'o' 或 'C-o' 复制消息会给消息的原件赋予 'filed' 属性，因此当这样的消息为当前消息时，模式行中会显示 'filed'。

如果你希望每条邮件消息只保留一份副本，请把变量 'rmail-delete-after-output' 设为 't'；这样 'o'、'C-o' 和 'w' 命令在复制后会删除原消息。（如果愿意，你之后可以取消删除它，参见"删除消息"。）

默认情况下，'o' 会让它输出的消息保留原消息的删除状态；因此，在输出前已删除的消息在输出文件中会显示为已删除。把变量 'rmail-output-reset-deleted-flag' 设为非 'nil' 值可以取消这一行为：消息副本的删除状态会被重置，因此该消息在输出文件中会显示为未删除。此外，当这个变量非 'nil' 时，给 'o' 指定正参数在寻找要连续输出的消息时不会忽略已删除的消息。

变量 'rmail-output-file-alist' 让你根据当前消息的内容为输出文件指定智能默认值。其值应当是一个列表，元素形式如下：

```
(REGEXP . NAME-EXP)
```

如果当前消息中有匹配 REGEXP 的文本，则输出的默认文件名是 NAME-EXP。如果有多个元素匹配该消息，第一个匹配的元素决定默认文件名。子表达式 NAME-EXP 可以是一个给出文件名的字符串常量，更一般地，它可以是任何求值为文件名字符串的 Lisp 表达式。'rmail-output-file-alist' 同时适用于 'o' 和 'C-o'。

Rmail 可以根据变量 'rmail-automatic-folder-directives' 的值，自动把主 Rmail 文件（即 'rmail-file-name' 指定的那个）中的消息保存到其他文件。该变量是一个元素（"指令"）列表，说明哪些消息保存到哪里。每条指令是一个列表，由一个输出文件以及随后的一对或多对头名称和正则表达式组成。如果某条消息有一个头匹配指定的正则表达式，该消息就保存到给定的文件。如果指令中有多个头条目，则所有条目都必须匹配。Rmail 在显示来自文件 'rmail-file-name' 的消息时检查这些指令，并应用第一条匹配的指令（如果有的话）。如果输出文件是 'nil'，则删除该消息而不保存。例如，你可以用这个功能把来自某个特定地址、或带有某个特定主题的消息保存到专用文件。

35.8 标签

每条消息都可以分配各种"标签"作为分类手段。每个标签有一个名字；不同名字是不同的标签。任何给定的标签在某条特定消息上要么存在，要么不存在。少数标签名有标准含义，由 Rmail 在适当时自动赋予消息；这些特殊标签称为"属性"。（参见"Rmail 属性"。）所有其他标签只由用户分配。

- 'a LABEL <RET>'：给当前消息分配标签 LABEL（'rmail-add-label'）。
- 'k LABEL <RET>'：从当前消息移除标签 LABEL（'rmail-kill-label'）。
- 'C-M-n LABELS <RET>'：移到下一条具有 LABELS 中某个标签的消息（'rmail-next-labeled-message'）。
- 'C-M-p LABELS <RET>'：移到上一条具有 LABELS 中某个标签的消息（'rmail-previous-labeled-message'）。

- ‘l LABELS <RET>’、‘C-M-l LABELS <RET>’：为所有含有 LABELS 中任一标签的消息创建摘要（‘rmail-summary-by-labels’）。

‘a’（‘rmail-add-label’）和 ‘k’（‘rmail-kill-label’）命令让你在当前消息上分配或移除任意标签。如果 LABEL 参数为空，则表示分配或移除最近一次分配或移除的那个标签。

一旦你按自己的意愿给邮件打上标签以便分类，就有三种使用标签的方式：移动、摘要和排序。

‘C-M-n LABELS <RET>’（‘rmail-next-labeled-message’）移动到下一封带有 LABELS 中某个标签的邮件。参数 LABELS 指定一个或多个标签名，以逗号分隔。‘C-M-p’（‘rmail-previous-labeled-message’）类似，但向后移动到之前的邮件。给这两个命令加数字参数可作为重复次数。

命令 ‘C-M-l LABELS <RET>’（‘rmail-summary-by-labels’）显示一个摘要，其中只包含至少带有指定标签集合中某一个标签的邮件。参数 LABELS 是一个或多个标签名，以逗号分隔。关于摘要的信息，参见“摘要”。

如果 ‘C-M-n’、‘C-M-p’ 或 ‘C-M-l’ 的 LABELS 参数为空，则表示使用这些命令中最近一次指定的那组标签。

关于按标签排序邮件的信息，参见“排序 Rmail 文件”。

35.9 Rmail 属性

有些标签具有内置含义，例如 ‘deleted’ 和 ‘filed’，Rmail 会在适当的时候自动把它们赋予邮件；这些标签称为“属性”（attribute）。下面是 Rmail 属性列表：

- ‘unseen’ 表示该邮件从未成为当前邮件。邮件来自收件箱文件时赋予该属性，邮件成为当前邮件时移除。启动 Rmail 时，它最初显示第一封带有该属性的邮件。
- ‘deleted’ 表示邮件已被删除。由删除命令赋予，由取消删除命令移除（参见“删除消息”）。
- ‘filed’ 表示邮件已被复制到某个其他文件。由 ‘o’ 和 ‘C-o’ 文件输出命令赋予（参见“把消息复制到文件”）。
- ‘answered’ 表示你已就该邮件发出回复。由 ‘r’ 命令（‘rmail-reply’）赋予。参见“发送回复”。
- ‘forwarded’ 表示你已转发该邮件。由 ‘f’ 命令（‘rmail-forward’）赋予。参见“发送回复”。
- ‘edited’ 表示你在 Rmail 中编辑过该邮件的文本。参见“在邮件内部编辑”。
- ‘resent’ 表示你已重发该邮件。由命令 ‘M-x rmail-resend’ 赋予。参见“发送回复”。
- ‘retried’ 表示你重试过一封发送失败的邮件。由命令 ‘M-x rmail-retry-failure’ 赋予。参见“发送回复”。

所有其他标签只能由用户赋予或移除，没有标准含义。

35.10 发送回复

Rmail 有若干条发送外发邮件的命令。关于使用 Message 模式（包括某些专为与 Rmail 配合而设计的功能）的信息，参见“发送邮件”。本节记述的是 Rmail 的一些特殊命令，它们用于进入编写外发邮件的邮件缓冲区。注意，发送邮件的常用按键——‘C-x m’、‘C-x 4 m’ 和 ‘C-x 5 m’——在 Rmail 模式中也正常工作。

- ‘m’ 发送一封邮件（‘rmail-mail’）。
- ‘c’ 继续编辑已经动手编写的外发邮件（‘rmail-continue’）。
- ‘r’ 对当前 Rmail 邮件发送回复（‘rmail-reply’）。

- ‘f’ 把当前邮件转发给其他用户（‘rmail-forward’）。
- ‘C-u f’ 把当前邮件重发给其他用户（‘rmail-resend’）。
- ‘M-m’ 尝试再次发送一封退回的邮件（‘rmail-retry-failure’）。

在 Rmail 中发送邮件最常见的原因，是回复你正在阅读的邮件。要做到这一点，输入 ‘r’（‘rmail-reply’）。这会在另一个窗口中显示一个邮件编写缓冲区，很像 ‘C-x 4 m’，但会根据你正在回复的邮件预先初始化 ‘Subject’、‘To’、‘CC’、‘In-Reply-To’ 和 ‘References’ 首部字段。‘To’ 字段起始于你所收到邮件的发件人地址，‘CC’ 字段起始于该邮件的所有其他收件人。

‘Subject’ 首部字段可能包含一个或多个 ‘Re:’ 或其本地化变体。如果它们匹配 ‘rmail-re-abbrevs’（其初值来自 ‘mail-re-regexps’），就会被移除，并在前面加上 ‘rmail-reply-prefix’。

你可以用变量 ‘mail-dont-reply-to-names’ 排除某些收件人，使他们不被自动纳入回复。它的值应是一个正则表达式；任何匹配的收件人都会被排除在 ‘CC’ 字段之外。他们也会被排除在 ‘To’ 字段之外，除非这样会使该字段为空。如果该变量为 ‘nil’，那么在你第一次编写回复时，它会被初始化为一个匹配你自己地址的默认值。

若只想回复原邮件的发件人，可以带数字参数输入回复命令：‘C-u r’ 或 ‘1 r’。这会为这次回复完全省略 ‘CC’ 字段。

邮件编写缓冲区初始化之后，编辑和发送邮件就照常进行（参见“发送邮件”）。如果预先提供的首部字段不合你的意，你可以编辑它们。你还可以使用 ‘C-c C-y’ 之类的命令，把你正在回复的邮件粘贴进来（参见“邮件命令”）。你也可以切换到 Rmail 缓冲区，在那里选择另一封邮件，再切换回来，然后粘贴新的当前邮件。

有时邮件无法到达目的地。邮件系统通常会把失败的邮件退回给你，并把它包在一封“失败邮件”中。Rmail 命令 ‘M-m’（‘rmail-retry-failure’）为再次发送同一封邮件做准备：它建立一个邮件编写缓冲区，其正文和首部字段与原来相同。如果你立即输入 ‘C-c C-c’，就会把邮件完全按第一次的样子再发一次。你也可以先编辑正文或首部字段再发送。变量 ‘rmail-retry-ignored-headers’ 的格式与 ‘rmail-ignored-headers’ 相同（参见“邮件的显示”），它控制在重试时从失败邮件中剥除哪些首部。

在 Rmail 中发送邮件的另一个常见原因，是把当前邮件“转发”给其他用户。‘f’（‘rmail-forward’）使这件事变得容易：它预先以当前邮件作为正文、以 ‘[FROM: SUBJECT]’ 形式的主题来初始化邮件编写缓冲区，其中 FROM 和 SUBJECT 是原邮件的发件人和主题。你只需填好收件人然后发送。转发邮件时，收件人收到的是一封来自你的邮件，其内容中包含原邮件。

Rmail 提供两种转发邮件的格式。默认使用 MIME 格式（参见“邮件的显示”），它把原邮件作为单独的一部分包含进来。如果你更喜欢，也可以把变量 ‘rmail-enable-mime-composing’ 设为 ‘nil’，从而使用更简单的格式。在这种情况下，Rmail 只是把原邮件夹在两行分隔符之间。它还会修改每一行以短横线开头的行，在行首插入 ‘- ’。当你收到这种格式的转发邮件时，如果它包含普通文本之外的内容——例如程序源代码——你可能会发现撤销这种变换很有用。方法是选中该转发邮件并输入 ‘M-x unforward-rmail-message’。该命令会提取原始的被转发邮件，删除插入的 ‘- ’ 字符串，并把它作为一封独立邮件插入 Rmail 文件，紧接在当前邮件之后。

“重发”（resend）是类似转发的一种替代方式；区别在于重发发送的是一封来自原发件人的邮件，就像它到达你那里时一样——只是额外加了几个首部字段（‘Resent-From’ 和 ‘Resent-To’）以表明它经由你转来。要在 Rmail 中重发邮件，使用 ‘C-u f’。（‘f’ 运行 ‘rmail-forward’，如果你提供数字参数，它会调用 ‘rmail-resend’。）

使用‘m’（‘rmail-mail’）命令开始编辑一封并非回复的外发邮件。它会让首部字段保持为空。它与‘C-x 4 m’的唯一区别是，它让 Rmail 缓冲区可被‘C-c C-y’访问，就像‘r’那样。

‘c’（‘rmail-continue’）命令恢复编辑邮件编写缓冲区，以完成你已经在编写的外发邮件，或修改一封你已经发出的邮件。

如果你把变量‘rmail-mail-new-frame’设为非‘nil’值，那么所有开始发送邮件的 Rmail 命令都会新建一个框架来在其中编辑。发送邮件时该框架会被删除（但如果它是当前显示器上唯一可见的框架，或者它是文本模式框架，则不会删除）。如果在发送邮件时无法删除该框架，Emacs 会尝试把它复用于编写后续邮件。

所有发送邮件的 Rmail 命令都使用你所选定的邮件编写方法（参见“邮件撰写方法”）。

35.11 摘要

“摘要”（summary）是一个缓冲区，其中每封邮件占一行，让你对某个 Rmail 文件中的邮件有个总览。每行显示邮件编号和日期、发件人、行数、标签和主题。在摘要缓冲区中移动光标点，就会随着你移到各摘要行而选中相应邮件。几乎所有 Rmail 命令在摘要缓冲区中也有效；在摘要缓冲区中使用时，它们作用于当前摘要行所描述的邮件。

一个摘要缓冲区只对应一个 Rmail 文件；如果你在编辑多个 Rmail 文件，每个文件都可以有自己的摘要缓冲区。摘要缓冲区的名称是在 Rmail 缓冲区名称后加上‘-summary’构成。通常一次只显示一个摘要缓冲区。

35.11.1 创建摘要

下面是为当前 Rmail 缓冲区创建摘要的命令。Rmail 缓冲区有了摘要之后，Rmail 缓冲区中的变化（例如删除或清除邮件、收取新邮件）会自动更新摘要。

- ‘h’、‘C-M-h’汇总所有邮件（‘rmail-summary’）。
- ‘l LABELS <RET>’、‘C-M-l LABELS <RET>’汇总带有指定标签中一个或多个的邮件（‘rmail-summary-by-labels’）。
- ‘C-M-r RCPTS <RET>’汇总与指定收件人匹配的邮件（‘rmail-summary-by-recipients’）。
- ‘C-M-t TOPIC <RET>’汇总主题中匹配指定正则表达式 TOPIC 的邮件（‘rmail-summary-by-topic’）。
- ‘C-M-s REGEXP <RET>’汇总其首部匹配指定正则表达式 REGEXP 的邮件（‘rmail-summary-by-regexp’）。
- ‘C-M-f SENDERS <RET>’汇总与指定发件人匹配的邮件（‘rmail-summary-by-senders’）。

‘h’或‘C-M-h’（‘rmail-summary’）命令用当前 Rmail 缓冲区中所有邮件的摘要填充该缓冲区的摘要缓冲区。然后它在另一个窗口中显示并选中该摘要缓冲区。

‘C-M-l LABELS <RET>’（‘rmail-summary-by-labels’）生成部分摘要，只列出带有一个或多个标签 LABELS 的邮件。LABELS 应包含以逗号分隔的标签名。

‘C-M-r RCPTS <RET>’（‘rmail-summary-by-recipients’）生成部分摘要，只列出有一名或多名收件人匹配正则表达式 RCPTS 的邮件。匹配针对‘To’、‘From’和‘CC’首部进行（提供前缀参数可排除‘CC’首部）。

‘C-M-t TOPIC <RET>’（‘rmail-summary-by-topic’）生成部分摘要，只列出主题匹配正则表达式 TOPIC 的邮件。带前缀参数时，匹配针对整封邮件，而不只是主题。

‘C-M-s REGEXP <RET>’（‘rmail-summary-by-regexp’）生成部分摘要，只列出其首部（包括日期和主题行）匹配正则表达式 REGEXP 的邮件。

‘C-M-f SENDERS <RET>’（‘rmail-summary-by-senders’）生成部分摘要，只列出其 ‘From’ 字段匹配正则表达式 SENDERS 的邮件。

注意，任何 Rmail 缓冲区都只有一个摘要缓冲区；生成任何种类的摘要都会丢弃先前的摘要。

变量 ‘rmail-summary-window-size’ 规定摘要窗口使用多少行。变量 ‘rmail-summary-line-count-flag’ 控制邮件的摘要行是否应包含该邮件的行数。把该选项设为 ‘nil’ 可能会加快摘要的生成。

Rmail 使用选项 ‘rmail-summary-sender-function’ 和 ‘rmail-summary-recipient-function’ 来格式化摘要行中的发件人和收件人字段。对发件人，可选值包括显示地址（默认），或显示发件人姓名并在没有姓名时回退到地址。对收件人，可选值包括显示 ‘To:’ 字段第一行中的地址（默认）、显示第一个收件人的姓名并在没有姓名时回退到地址，或显示所有收件人的姓名并在没有姓名时回退到地址。这两个选项也都可以设为一个自定义函数。

选项 ‘rmail-summary-address-width’ 控制这些字段的宽度。

35.11.2 在摘要中编辑

你可以用 Rmail 摘要缓冲区做几乎任何能在 Rmail 缓冲区本身中做的事。事实上，一旦有了摘要缓冲区，就没有必要再切回 Rmail 缓冲区。

你只需把摘要缓冲区中的光标点移到不同的行，就能从摘要缓冲区选中并显示 Rmail 缓冲区中的各封邮件。用什么 Emacs 命令移动光标点都无所谓；命令结束时光标点位于哪一行，Rmail 缓冲区中就选中哪封邮件。

几乎所有 Rmail 命令在摘要缓冲区中和在 Rmail 缓冲区中一样有效。因此，在摘要缓冲区中 ‘d’ 删除当前邮件，‘u’ 取消删除，‘x’ 清除。（不过在摘要缓冲区中，如果相关方向上不再有未删除的邮件，删除命令会转到第一封或最后一封邮件，而不是停留在当前邮件上。）‘o’ 和 ‘C-o’ 把当前邮件输出到一个 FILE；‘r’ 开始回复它；等等。你可以用 <SPC> 和 <DEL> 在停留在摘要缓冲区中的同时滚动当前邮件。不过，在摘要缓冲区中用 <SPC> 或 <DEL> 滚过一封邮件的末尾或开头时，会分别转到下一封或上一封未删除的邮件。把 ‘rmail-summary-scroll-between-messages’ 选项定制为 ‘nil’ 可以禁用滚动到下一封/上一封邮件。

‘M-u’（‘rmail-summary-undelete-many’）取消删除摘要中所有被删除的邮件。前缀参数表示取消删除此前那么多封被删除的邮件。

在邮件之间移动的 Rmail 命令在摘要缓冲区中也有效，但有一点不同：它们在摘要所包含的那组邮件中移动。它们还会确保 Rmail 缓冲区出现在屏幕上（不像光标移动命令，那些命令会更新 Rmail 缓冲区的内容，但除非它已经显示，否则不会在窗口中显示它）。下面是这些命令的列表：

- ‘n’ 移到下一行，跳过标着 “deleted” 的行，并选中其邮件（‘rmail-summary-next-msg’）。
- ‘p’ 移到上一行，跳过标着 “deleted” 的行，并选中其邮件（‘rmail-summary-previous-msg’）。
- ‘M-n’ 移到下一行并选中其邮件（‘rmail-summary-next-all’）。
- ‘M-p’ 移到上一行并选中其邮件（‘rmail-summary-previous-all’）。
- ‘>’ 移到最后一行，并选中其邮件（‘rmail-summary-last-message’）。
- ‘<’ 移到第一行，并选中其邮件（‘rmail-summary-first-message’）。

- ‘j’、‘<RET>’ 选中当前行上的邮件（确保 Rmail 缓冲区出现在屏幕上；‘rmail-summary-goto-msg’）。带参数 N 时，选中编号为 N 的邮件并把摘要缓冲区中的光标点移到它的行；如果该邮件没有列在摘要缓冲区中，则会报错。
- ‘M-s PATTERN <RET>’ 从当前邮件开始在邮件中搜索 PATTERN；选中找到的邮件，并把摘要缓冲区中的光标点移到该邮件的行（‘rmail-summary-search’）。前缀参数作为重复次数；负参数表示向后搜索（等价于 ‘rmail-summary-search-backward’）。
- ‘C-M-n LABELS <RET>’ 移到下一封至少带有指定标签中一个的邮件（‘rmail-summary-next-labeled-message’）。LABELS 是以逗号分隔的标签列表。前缀参数作为重复次数。
- ‘C-M-p LABELS <RET>’ 移到上一封至少带有指定标签中一个的邮件（‘rmail-summary-previous-labeled-message’）。
- ‘C-c C-n <RET>’ 移到下一封主题与当前邮件相同的邮件（‘rmail-summary-next-same-subject’）。前缀参数作为重复次数。
- ‘C-c C-p <RET>’ 移到上一封主题与当前邮件相同的邮件（‘rmail-summary-previous-same-subject’）。

删除、取消删除、收取新邮件，甚至在 Rmail 缓冲区中选择另一封邮件，当你在 Rmail 缓冲区中执行它们时都会更新摘要缓冲区。如果变量 ‘rmail-redisplay-summary’ 为非 ‘nil’，这些操作还会把摘要缓冲区重新带回屏幕上。

用完摘要后，输入 ‘Q’（‘rmail-summary-wipe’）删除摘要缓冲区的窗口。你也可以在摘要中退出 Rmail：‘q’（‘rmail-summary-quit’）删除摘要窗口，然后通过保存 Rmail 文件并切换到另一个缓冲区来退出 Rmail。另外，‘b’（‘rmail-summary-bury’）只是把 Rmail 摘要和缓冲区埋起来。

35.12 排序 Rmail 文件

- ‘C-c C-s C-d’、‘M-x rmail-sort-by-date’：按日期排序当前 Rmail 缓冲区中的邮件。
- ‘C-c C-s C-s’、‘M-x rmail-sort-by-subject’：按主题排序当前 Rmail 缓冲区中的邮件。
- ‘C-c C-s C-a’、‘M-x rmail-sort-by-author’：按作者姓名排序当前 Rmail 缓冲区中的邮件。
- ‘C-c C-s C-r’、‘M-x rmail-sort-by-recipient’：按收件人姓名排序当前 Rmail 缓冲区中的邮件。
- ‘C-c C-s C-c’、‘M-x rmail-sort-by-correspondent’：按通信对方的姓名排序当前 Rmail 缓冲区中的邮件。
- ‘C-c C-s C-l’、‘M-x rmail-sort-by-lines’：按行数排序当前 Rmail 缓冲区中的邮件。
- ‘C-c C-s C-k <RET> LABELS <RET>’、‘M-x rmail-sort-by-labels <RET> LABELS <RET>’：按标签排序当前 Rmail 缓冲区中的邮件。参数 LABELS 应为以逗号分隔的标签列表。这些标签的顺序规定了邮件的顺序；带第一个标签的邮件排在最前，带第二个标签的排在其次，依此类推。不带这些标签中任何一个的邮件排在最后。

Rmail 的排序命令执行的是*稳定排序*：如果两封邮件之间没有理由偏好哪一个，它们的相对顺序保持不变。你可以利用这一点按多个标准排序。例如，如果你先使用 ‘rmail-sort-by-date’ 再使用 ‘rmail-sort-by-author’，那么来自同一作者的邮件就按日期排列。

带前缀参数时，所有这些命令都会反转比较顺序。这意味着它们按从新到旧、从大到小或逆字母顺序排序邮件。

摘要缓冲区中的相同按键运行类似的函数；例如 'C-c C-s C-l' 运行 'rmail-summary-sort-by-lines'。这些命令总是排序整个 Rmail 缓冲区，即使摘要只显示邮件的一个子集。

注意，排序无法撤销，所以你可能希望在排序之前先保存 Rmail 缓冲区。

35.13 邮件的显示

本节介绍 Rmail 如何显示邮件首部、MIME 各部分与附件、URL 以及加密邮件。

't' 切换完整首部的显示（'rmail-toggle-header'）。

在首次显示每封邮件之前，Rmail 会重新格式化其首部，隐藏不重要的首部字段以减少杂乱。't'（'rmail-toggle-header'）命令切换这一点，在显示重新格式化后的首部字段与显示完整原始首部之间来回切换。带正前缀参数时，该命令显示重新格式化后的首部；带零或负前缀参数时，显示完整首部。再次选中该邮件时，如有必要也会重新格式化它。

变量 'rmail-ignored-headers' 保存一个正则表达式，指定要隐藏的首部字段；任何匹配的首部行都会被隐藏。变量 'rmail-nonignored-headers' 覆盖它：任何匹配该正则表达式的首部字段，即使也匹配 'rmail-ignored-headers'，仍会被显示。变量 'rmail-displayed-headers' 是这两个变量的另一种选择；若不为 'nil'，它应是一个正则表达式，指定要显示哪些首部（默认值为 'nil'）。

Rmail 会高亮某些特别值得注意的首部字段——默认是 'From' 和 'Subject' 字段。这种高亮使用 'rmail-highlight' 字体（face）。变量 'rmail-highlighted-headers' 保存一个正则表达式，指定要高亮的首部字段；如果它匹配某个首部字段的开头，整个字段都会被高亮。要禁用此功能，把 'rmail-highlighted-headers' 设为 'nil'。

如果邮件是 MIME（Multipurpose Internet Mail Extensions）格式并包含多个部分（MIME 实体），Rmail 会把每个部分连同"标签行"（tagline）一起显示。标签行概括该部分的序号、大小和内容类型。视内容类型而定，它还可能包含一个或多个按钮；这些按钮执行诸如把该部分保存到文件之类的操作。

把 MIME 附件保存到文件时，Rmail 可以根据 'rmail-mime-save-action' 的值执行后续操作。默认情况下，该值为 'nil'，表示 Rmail 什么也不做。其他预定义值有：

- 'visit-file' 在 Emacs 中访问保存的文件。
- 'visit-directory' 在 Dired 中访问该文件所在的目录，并把光标点移到该文件上。
- 'open-external' 用适合该文件类型的外部程序打开该文件。

你也可以把 'rmail-mime-save-action' 设为一个自定义函数。该函数将以所保存附件的绝对文件名为唯一参数被调用。

- '<RET>' 隐藏或显示光标点处的 MIME 部分（'rmail-mime-toggle-hidden'）。
- '<TAB>' 把光标点移到下一个 MIME 标签行按钮（'rmail-mime-next-item'）。
- 'S-<TAB>' 把光标点移到上一个 MIME 部分（'rmail-mime-previous-item'）。
- 'v' 在 MIME 显示和原始邮件之间切换（'rmail-mime'）。

每个纯文本 MIME 部分最初直接显示在其标签行之后，作为 Rmail 缓冲区的一部分（除非该邮件有 HTML 部分，见下文），而其他类型的 MIME 部分仅以其标签行表示，实际内容被隐藏。无论哪种情况，你都可以在该部分的任意位置——或在其标签行的任意位置（若有用于其他操作的按钮，则不包括这些按钮）——输入

<RET> 来切换该 MIME 部分的显示与隐藏状态。输入 <RET>（或用鼠标点击）可激活标签行按钮，输入 <TAB> 可在各标签行按钮之间循环移动光标点。

‘v’（‘rmail-mime’）命令在上述默认的 MIME 显示与显示未解码 MIME 数据的原始显示之间切换。带前缀参数时，该命令只切换光标点处实体的显示。

如果邮件有 HTML MIME 部分，那么只要 Emacs 能够渲染 HTML(1)，Rmail 就会优先显示它而不是纯文本部分。要阻止这一点并改为显示纯文本部分，请把变量 ‘rmail-mime-prefer-html’ 定制为 ‘nil’ 值。

若要阻止 Rmail 处理经 MIME 解码的邮件，请把变量 ‘rmail-enable-mime’ 改为 ‘nil’。在这种情况下，‘v’（‘rmail-mime’）命令会改为创建一个临时缓冲区来显示当前的 MIME 邮件。

若当前邮件是加密邮件，可用命令 ‘C-c C-d’（‘rmail-epa-decrypt’）解密，它使用 EasyPG 库（参见《EasyPG 手册》）。

你可以用 Goto Address 模式在 Rmail 缓冲区中高亮并激活 URL：

```
(add-hook 'rmail-show-message-hook 'goto-address-mode)
```

然后你便可以通过用 ‘mouse-2’（或快速按 ‘mouse-1’）点击这些 URL，或移动到某个 URL 上并输入 ‘C-c <RET>’ 来浏览它们。参见“激活 URL”

> 注：此功能要求 Emacs 在构建时带有 ‘libxml2’ 支持，或者你已安装 Lynx 浏览器。

35.14 Rmail 与编码系统

Rmail 会自动解码包含非 ASCII 字符的邮件，就像 Emacs 处理你访问的文件和子进程输出那样。Rmail 使用邮件中标准的 ‘charset=CHARSET’ 头部（如果有的话）来判断发件人当初是如何编码这封邮件的。它把 CHARSET 映射为对应的 Emacs 编码系统（参见“编码系统”），并用该编码系统解码邮件文本。如果邮件头部没有 ‘charset’ 说明，或者 CHARSET 无法识别，Rmail 就用通常的 Emacs 启发式规则和默认值来选择编码系统（参见“识别编码系统”）。

偶尔邮件会被错误解码，这或者是因为在没有 ‘charset’ 说明时 Emacs 猜错了编码系统，或者是因为该说明本身不准确。例如，配置错误的邮件程序可能发出一封头部为 ‘charset=iso-8859-1’ 而实际用 ‘koi8-r’ 编码的邮件。当你看到邮件正文乱码，或其中一些字符显示为十六进制码或空方框时，可能就是这种情况。

如果你能推断或猜出哪个编码系统正确，就可以用它重新解码邮件来修正问题。做法是调用 ‘M-x rmail-redecode-body’ 命令。它会读取一个编码系统名称，然后按你指定的编码系统重新解码邮件。若你指定得正确，结果应当可读。

当你在 Rmail 中收到新邮件时，每封邮件都像单独的文件一样，自动从其书写所用的编码系统转换过来。这使用你指定的编码系统优先级列表。若 MIME 邮件指定了字符集，Rmail 遵从该说明。对于读取和保存 Rmail 文件本身，Emacs 使用变量 ‘rmail-file-coding-system’ 指定的编码系统。默认值是 ‘nil’，意味着 Rmail 文件不被转换（它们以 Emacs 内部字符码读写）。

35.15 在邮件内部编辑

大多数常用的 Emacs 键绑定在 Rmail 模式中可用，尽管少数如 ‘C-M-n’ 和 ‘C-M-h’ 被 Rmail 重定义为其他用途。然而 Rmail 缓冲区通常是只读的，大多数字母键被重定义为 Rmail 命令。若要编辑邮件的文本，必须使用 Rmail 命令 ‘e’。

‘e’ 把当前邮件当作普通文本编辑。

‘e’ 命令（‘rmail-edit-current-message’）从 Rmail 模式切换到 Rmail Edit 模式，这是另一个与 Text 模式几乎相同的主模式。模式行会显示这一变化。

在 Rmail Edit 模式中，字母键照常自行插入，Rmail 命令不可用。你可以编辑邮件正文和头部字段。编辑完成后，输入 ‘C-c C-c’（‘rmail-cease-edit’）切回 Rmail 模式。或者，你也可以输入 ‘C-c C-]’（‘rmail-abort-edit’）返回 Rmail 模式，同时取消你所做的任何编辑。

进入 Rmail Edit 模式会运行钩子 ‘text-mode-hook’；然后它运行钩子 ‘rmail-edit-mode-hook’（参见“钩子”）。若你在邮件中做了任何修改，返回普通 Rmail 模式时会为该邮件加上 ‘edited’ 属性（参见“Rmail 属性”）。

35.16 摘要邮件

“摘要邮件”（digest message）是为容纳和承载其他若干邮件而存在的邮件。有些邮件列表使用摘要；在某段时间（例如一天）内到达该列表的所有邮件都被放进一个摘要中，然后发送给订阅者。发送单个摘要所用的计算机时间比发送各封邮件少，即使总大小相同，因为网络邮件传输中每封邮件都有开销。

当你收到摘要邮件时，最方便的阅读方式是“解摘要”（undigestify）：把它重新变成许多单独的邮件。然后你就可以按需阅读和删除这些单独的邮件。做法是选中该摘要邮件并输入命令 ‘M-x undigestify-rmail-message’。它把各子邮件提取为单独的 Rmail 邮件，并插入到摘要之后。摘要邮件本身被标记为已删除。

35.17 阅读 Rot13 邮件

邮件列表中可能冒犯或惹恼某些读者的邮件，有时用一种称为 “rot13” 的简单编码来编码——这个名字来源于它把字母表旋转 13 个字母。这种编码不是为了保密，因为它根本不提供保密性；而是让那些希望避免看到邮件真实文本的人如愿。例如，一篇影评可能用 rot13 隐藏重要的剧情要点。

要查看使用 rot13 编码的缓冲区，请用命令 ‘M-x rot13-other-window’。它会在另一个窗口中显示当前缓冲区，并在显示文本时应用该编码。

如果你只对某个区域感兴趣，命令 ‘M-x rot13-region’ 可能更合适。它会就地加密/解密活动区域。若缓冲区是只读的，它会尝试在回显区中显示明文。若文本太长，回显区放不下，该命令会弹出一个临时缓冲区，显示加密/解密后的文本。

35.18 ‘movemail’ 程序

Rmail 使用 ‘movemail’ 程序把邮件从你的收件箱移到你的 Rmail 文件（参见“Rmail 文件与收件箱”）。首次加载时，Rmail 会尝试定位 ‘movemail’ 程序并确定其版本。‘movemail’ 程序有两个版本：GNU Mailutils 版本（参见《《mailutils》手册》的“movemail”），以及 Emacs 专用版本——除非配置 Emacs 时启用 ‘--with-mailutils’，

否则会构建并安装后者。两个 'movemail' 版本支持相同的命令行语法和相同的基本选项子集。不过 Mailutils 版本提供附加功能，并且更安全。

Emacs 版本的 'movemail' 可以从通常的 Unix 邮箱格式收取邮件。*警告*：虽然它也能使用 POP3 协议，但不推荐这样做，因为它不支持通过加密的 TLS 通道使用 POP3。

Mailutils 版本能处理多种邮箱格式，例如纯 Unix 邮箱、'maildir' 和 'MH' 邮箱等。它能用 POP3 或 IMAP4 协议访问远程邮箱，并可通过 TLS 加密通道从其中收取邮件。它还接受 URL 形式的邮箱参数。邮箱 URL 的详细说明见 参见《《mailutils》手册》的"Mailbox"。简言之，URL 的形式是：

```
PROTO://[USER[:PASSWORD]@]HOST-OR-FILE-NAME[:PORT]
```

其中方括号表示可选元素。

PROTO 指定要使用的"邮箱协议"（mailbox protocol）或"格式"。URL 其余元素的准确语义取决于 PROTO 的实际值（见下）。

USER 访问远程邮箱的用户名。

PASSWORD 访问远程邮箱的用户密码。

HOST-OR-FILE-NAME 远程邮箱的远程服务器主机名，或本地邮箱的文件名。

PORT 可选的端口号，如果不用该协议的默认端口。

PROTO 可以是下列之一：

'mbox' 通常的 Unix 邮箱格式。此时 USER、PASS 和 PORT 不使用，HOST-OR-FILE-NAME 表示邮箱文件的文件名，例如 'mbox:///var/spool/mail/smith'。

'mh' MH 格式的本地邮箱。USER、PASS 和 PORT 不使用。HOST-OR-FILE-NAME 表示 MH 文件夹的名称，例如 'mh:///Mail/inbox'。

'maildir' maildir 格式的本地邮箱。USER、PASS 和 PORT 不使用，HOST-OR-FILE-NAME 表示 'maildir' 邮箱的名称，例如 'maildir:///mail/inbox'。

'file' 任何邮箱格式的本地文件。其实际格式由 'movemail' 自动检测。

'pop' 'pops' 通过 POP3 协议访问的远程邮箱。详情参见"从远程邮箱收取邮件"。

'imap' 'imaps' 通过 IMAP4 协议访问的远程邮箱。详情参见"从远程邮箱收取邮件"。

另外，你也可以指定要使用的邮箱文件名。这等价于指定 'file' 协议：

```
/var/spool/mail/USER ≡ file:///var/spool/mail/USER
```

变量 'rmail-movemail-program' 控制使用哪个版本的 'movemail'。若它是字符串，则指定 'movemail' 可执行文件的绝对文件名。若它是 'nil'，Rmail 依次在 'rmail-movemail-search-path' 列出的目录、'exec-path'（参见"在 Emacs 中运行 Shell 命令"）以及 'exec-directory' 中查找 'movemail'。该变量应在启动 Rmail 之前定制；若在启动 Rmail 之后才定制，你需要把 'rmail-movemail-variant-in-use' 设为 'nil' 值，然后重启 Rmail。

35.19 从远程邮箱收取邮件

有些站点使用一种称为 POP3 的方法来访问用户的收件箱数据，而不是把数据存放在收件箱文件中。Mailutils ‘movemail’ 默认支持带 TLS 加密的 POP3。*警告*：虽然 ‘Emacs movemail’ 支持 POP3，但不推荐用它做这件事，因为它不支持加密连接——Mailutils 版本则支持。两个版本的 ‘movemail’ 都只适用于 POP3，而不适用于旧版本的 POP。

你可以用 POP3 “URL”（参见 “‘movemail’ 程序”）指定 POP3 收件箱。POP3 URL 的形式是 ‘pop://USERNAME@HOSTNAME:PORT’，其中 HOSTNAME 和 PORT 是远程邮件服务器的主机名（或 IP 地址）和端口号，USERNAME 是该服务器上的用户名。此外，你可以在邮箱 URL 中指定密码：‘pop://USERNAME:PASSWORD@HOSTNAME:PORT’。此时 PASSWORD 优先于 ‘rmail-remote-password’ 设置的值（见下）。若你有多个远程邮箱且密码各不相同，这尤其有用。若使用 Mailutils ‘movemail’ 且服务器支持加密连接，‘movemail’ 会尝试使用它；用 ‘pops:’ 代替 ‘pop:’ 可要求使用这种连接。

为向后兼容，Rmail 也支持另一种指定远程 POP3 邮箱的方式。以 ‘po:USERNAME:HOSTNAME:PORT’ 的形式指定收件箱名，等价于 ‘pop://USERNAME@HOSTNAME:PORT’。若你省略 :HOSTNAME 部分，则由 ‘MAILHOST’ 环境变量指定到哪台机器上寻找 POP3 服务器。

访问远程邮箱的另一种方法是 IMAP。该方法仅由 Mailutils ‘movemail’ 支持，使用 IMAP4 协议。要在收件箱列表中指定 IMAP 邮箱，请使用以下邮箱 URL：‘imap://USERNAME[:PASSWORD]@HOSTNAME:PORT’。PASSWORD 部分是可选的，如上所述。若服务器支持，‘movemail’ 会尝试使用加密连接——用 ‘imaps:’ 形式可要求使用加密连接。

访问远程邮箱可能需要密码。Rmail 使用以下算法获取密码：

1. 若邮箱 URL 中存在 PASSWORD（见上），则使用它。2. 若变量 ‘rmail-remote-password-required’ 为 ‘nil’，Rmail 认为不需要密码。3. 若变量 ‘rmail-remote-password’ 非 ‘nil’，则使用其值。4. 否则，Rmail 会向你询问要用的密码。

在某些邮件服务器上，用户名包含域名信息，这意味着其中可能含有 ‘@’ 字符。收件箱说明字符串用 ‘@’ 表示邮件服务器名的开始。这会给 movemail 造成混淆。若你的用户名含 ‘@’ 且你使用的是 Mailutils ‘movemail’，可以这样修正：把用户名中的 ‘@’ 替换为其 URL 编码 ‘%40’。

若你需要向 ‘movemail’ 传递额外的命令行标志，把你想用的标志组成列表设为变量 ‘rmail-movemail-flags’。不要用该变量传 ‘-p’ 标志来保留收件箱内容；请改用 ‘rmail-preserve-inbox’。

你所在站点安装的 ‘movemail’ 程序可能支持 Kerberos 认证。若它支持，则当 ‘rmail-remote-password’ 和 ‘rmail-remote-password-required’ 未设置时，你每次尝试收取 POP3 邮件都会默认使用它。

有些 POP3 服务器按相反顺序存储邮件。若你的服务器如此，而你更愿意按邮件收到的顺序阅读，可以给 ‘rmail-movemail-flags’ 加上 ‘-r’ 标志，让 ‘movemail’ 反转下载邮件的顺序。

Mailutils ‘movemail’ 支持 TLS 加密。若你想使用它，请给 ‘rmail-movemail-flags’ 加上 ‘--tls’ 标志。

35.20 从各种格式的本地邮箱收取邮件

若你的接收邮件以 Unix 邮箱之外的其他格式存放在本地机器上，你就需要 Mailutils ‘movemail’ 来收取它。‘movemail’ 各版本的详细说明参见“‘movemail’ 程序”。例如，要访问位于 ‘/var/spool/mail/in’、格式为 ‘maildir’ 的收件箱中的邮件，你应在 Rmail 收件箱列表中加入：

```
maildir:///var/spool/mail/in
```

36 用 Gnus 收发邮件与新闻

Gnus 是一个 Emacs 软件包，主要用来阅读和发布 Usenet 新闻。它也可以用来阅读和回复来自其他多种来源的消息——电子邮件、远程目录、摘要等等。这里我们介绍 Gnus 并描述若干基本功能。完整细节参见《Gnus 手册》。

36.1 Gnus 缓冲区

Gnus 使用多个缓冲区来显示信息并接收命令。最常用的三个 Gnus 缓冲区是“组缓冲区”（group buffer）、“摘要缓冲区”（summary buffer）和“文章缓冲区”（article buffer）。

“组缓冲区”包含一份文章来源的列表（例如新闻组和电子邮件收件箱），这些来源统称为“组”（group）。这是 Gnus 启动时显示的第一个缓冲区。它通常只显示你已订阅且含有未读文章的组。你可以从这个缓冲区中选择一个组来阅读。

“摘要缓冲区”列出单个组中的文章，每行一篇文章。默认情况下，它显示每篇文章的作者、主题和行号，不过这可以自定义；参见《Gnus 手册》的“Summary Buffer Format”。摘要在你于组缓冲区中选择某个组时创建，并在你退出该组时被删除。

从摘要缓冲区中，你可以选择一篇文章来查看。文章显示在“文章缓冲区”中。在正常的 Gnus 使用中，你只是查看这个缓冲区而不选中它——所有有用的 Gnus 命令都可以从摘要缓冲区调用。不过，如果你愿意，也可以选中文章缓冲区，并在其中执行 Gnus 命令。

36.2 Gnus 启动时

如果你的系统已经设置好用于阅读 Usenet 新闻，那么开始使用 Gnus 很简单——只需输入 ‘M-x gnus’。

启动时，Gnus 会读取你的“新闻初始化文件”：即主目录中名为 ‘.newsrc’ 的文件，它列出你的 Usenet 新闻组和订阅（此文件并非 Gnus 专有；许多其他新闻阅读器程序也使用它）。然后它尝试联系系统的默认新闻服务器，该服务器通常由环境变量 ‘NNTPSERVER’ 指定。

如果你的系统没有默认新闻服务器，或者你希望用 Gnus 阅读电子邮件，那么在调用 ‘M-x gnus’ 之前，需要告诉 Gnus 从哪里获取新闻和/或邮件。要做到这一点，请自定义变量 ‘gnus-select-method’ 和/或 ‘gnus-secondary-select-methods’。参见《Gnus 手册》的“Finding the News”。

Gnus 启动之后，会显示组缓冲区。默认情况下，组缓冲区只显示少量“已订阅组”。具有其他状态的组——“未订阅”、“已删除”或“僵尸”——会被隐藏。第一次启动 Gnus 时，你未订阅的任何组都会变成已删除组；此后新闻服务器上出现的任何新组都会变成僵尸组。

要继续，你必须在组缓冲区中选择一个组，以打开该组的摘要缓冲区；然后在摘要缓冲区中选择一篇文章，以便在单独的窗口中查看它的文章缓冲区。以下几节说明如何使用组缓冲区和摘要缓冲区来完成这些操作。

要退出 Gnus，请在组缓冲区中输入 ‘q’。这会自动把各组的状态记录到文件 ‘.newsrc’ 和 ‘.newsrc.eld’ 中，使它们在后续的 Gnus 会话中生效。

36.3 使用 Gnus 组缓冲区

以下命令在 Gnus 组缓冲区中可用：

‘<SPC>’ 切换到当前行所在组的摘要缓冲区（‘gnus-group-read-group’）。

‘l’ ‘A s’ 在组缓冲区中，只列出你已订阅且含有未读文章的组（‘gnus-group-list-groups’；这是默认的列表方式）。

‘L’ ‘A u’ 列出所有已订阅和未订阅的组，但不包括已删除组和僵尸组（‘gnus-group-list-all-groups’）。

‘A k’ 列出已删除的组（‘gnus-group-list-killed’）。

‘A z’ 列出僵尸组（‘gnus-group-list-zombies’）。

‘u’ 切换当前行上组的订阅状态（‘gnus-group-toggle-subscription-at-point’）。对已删除组或僵尸组调用此命令，会把它变成未订阅组。

‘C-k’ 删除当前行上的组（‘gnus-group-kill-group’）。已删除的组不会记录在 ‘.newsrc’ 文件中，也不会显示在 ‘l’ 或 ‘L’ 列表中。

‘<DEL>’ 把光标点移到上一个含有未读文章的组（‘gnus-group-prev-unread-group’）。

‘n’ 把光标点移到下一个未读组（‘gnus-group-next-unread-group’）。

‘p’ 把光标点移到上一个未读组（‘gnus-group-prev-unread-group’）。

‘q’ 更新你的 Gnus 设置，并退出 Gnus（‘gnus-group-exit’）。

36.4 使用 Gnus 摘要缓冲区

以下命令在 Gnus 摘要缓冲区中可用：

‘<SPC>’ 若未选中任何文章，则选中当前行上的文章并显示其文章缓冲区。否则，尝试在其窗口中滚动已选中的文章缓冲区；到达缓冲区末尾时，选中下一篇未读文章（‘gnus-summary-next-page’）。

因此，你可以通过反复输入 <SPC> 读完全部文章。

‘<DEL>’ 向后滚动文章正文（‘gnus-summary-prev-page’）。

‘n’ 选中下一篇未读文章（‘gnus-summary-next-unread-article’）。

‘p’ 选中上一篇未读文章（‘gnus-summary-prev-unread-article’）。

‘s’ 对选中的文章缓冲区进行增量搜索（‘gnus-summary-isearch-article’），就像切换到该缓冲区并输入 ‘C-s’ 一样（参见“增量搜索”）。

‘M-s M-s REGEXP <RET>’ 向前搜索含有与 REGEXP 匹配内容的文章（‘gnus-summary-search-article-forward’）。

‘M-r REGEXP <RET>’ 向后搜索含有与 REGEXP 匹配内容的文章（‘gnus-summary-search-article-backward’）。

‘q’ 退出摘要缓冲区并返回组缓冲区（‘gnus-summary-exit’）。

37 主机安全

Emacs 运行在 GNU/Linux 之类的操作系统之中，并依赖操作系统来检查文件访问等安全约束。Emacs 的默认设置是针对典型用途设计的；在安全性比通常更受关注、或更不受关注的环境中，这些设置可能需要做些调整。例如，文件局部变量可能有风险，你可以把变量 'enable-local-variables' 设为 ':safe'，或（更保守地）设为 'nil'；反过来，如果你的文件都可以信任，而这些变量的默认检查又令人厌烦，你可以把 'enable-local-variables' 设为 ':all'。参见"文件变量的安全性"。

用 'load-file' 或 'load-library' 加载一个 Emacs Lisp 代码文件时（参见"Emacs 的 Lisp 代码库"），可能会执行被加载文件中的部分 Lisp 代码，因此你只应加载源代码可信的 Lisp 文件。不过，某些 Emacs 特性在特定情况下即使没有你的明确命令或请求，也能执行 Lisp 代码。例如，Emacs 的即时语法检查器 Flymake（参见《《flymake》手册》）在启用之后，可能在你访问某个 Lisp 文件时，作为其语法检查工作的一部分自动执行该文件中的部分代码。类似地，在访问 Lisp 文件的缓冲区中，某些补全命令（参见"补全"）为了取得最佳结果，有时需要展开 Lisp 宏。在这些情况下，仅仅是访问一个 Lisp 文件并在其中做些编辑，就可能触发 Lisp 代码的执行。如果访问的文件来自不可信的来源，它就可能含有危险甚至恶意的代码，而 Emacs 会在上述情形中执行这些代码。

为防止这种情况，Emacs 禁用了 Flymake、补全以及某些其他特性对 Lisp 代码的执行，除非所访问的文件是"可信"的。哪些文件应被视为可信，需要你通过定制用户选项 'trusted-content' 来指定。

用户选项：trusted-content 该选项的值默认为 'nil'，意味着不信任任何文件。你可以把这个变量定制为一个列表，其中的元素是一个或多个可信文件与目录的名称。以斜杠 '/' 结尾的文件名被解释为目录，这意味着该目录下的所有文件及子目录也都可信。特殊值 ':all' 表示系统上的*所有*文件和目录都应被信任；*不建议这样做*，因为这会留下一个巨大的安全漏洞。

关于把 Emacs 作为更大规模应用程序的一部分使用时的安全考虑，更多信息参见《Emacs Lisp 参考手册》的"Security Considerations"。

38 网络安全

每当 Emacs 建立任何网络连接时，都会把已建立的连接交给“网络安全管理器”（Network Security Manager，NSM）。NSM 负责在你的掌控下实施网络安全。目前，这是通过使用传输层安全（Transport Layer Security，TLS）特性来实现的。

‘network-security-level’ 变量决定 NSM 实施的安全级别。若其值为 ‘low’，则不执行任何安全检查。不建议这样设置，因为这基本上意味着你的网络连接不可信。不过，该设置在有限的情况下可能有用，例如在测试网络问题时。

若该变量为 ‘medium’（这是默认值），则会执行若干检查。若 NSM 因此判定该网络连接可能不可信，它会让你注意到这一点，并询问你如何处理该网络连接。

你可以决定为未经验证的连接登记永久安全例外、登记临时例外，或完全拒绝该连接。

除基本的证书正确性检查外，还可进行若干 TLS 算法检查。一些过去被认为安全的加密技术已被证明是脆弱的，因此 Emacs（默认）会就其中某些问题向你发出警告。

协议网络检查由 ‘network-security-protocol-checks’ 变量控制。

它是一个关联列表，其中每个关联的第一个元素是检查的名称，第二个元素是应使用该检查的安全级别。

像 ‘(rc4 medium)’ 这样的元素将导致函数 ‘nsm-protocol-check--rc4’ 被这样调用：‘(nsm-protocol-check--rc4 host port status settings)’。若应当继续该连接，函数应返回非 ‘nil’；否则返回 ‘nil’。

下面是默认 ‘medium’ 级别下所做的检查列表。

- **无法验证 TLS 证书** 若连接是 TLS、SSL 或 STARTTLS 连接，NSM 会检查用于确立我们所连接的服务器身份的证书能否被验证。

无效证书常常令人担忧（可能有中间人在劫持你的网络连接并窃取你的密码），但继续建立连接也可能有正当理由。例如，服务器可能使用自签名证书，或者证书可能已经过期。是否可接受继续该连接，由你自行判断。

- **自签名证书已发生变化** 若你此前接受了某个自签名证书，但它现在变了，这可能意味着服务器刚刚更换了证书，但也可能意味着网络连接已被劫持。
- **之前加密的连接现在未加密** 若连接未加密，但它在之前的会话中是加密的，这可能意味着你和服务器之间存在一个代理，它剥离了 STARTTLS 通告，使连接保持未加密。这通常非常可疑。
- **发送密码时与未加密的服务通信** 连接 IMAP 或 POP3 服务器时，这些连接通常应加密，因为通过这类连接发送密码很常见。类似地，若你通过需要密码的 SMTP 发送邮件，通常也希望该连接加密。若连接未加密，NSM 会警告你。
- **Diffie-Hellman 素数位数过低** 进行公钥交换时，素数位数应足够高，以确保信道不会被第三方窃听。若该数值过低，Emacs 会警告你。（这是 ‘network-security-protocol-checks’ 中的 ‘diffie-hellman-prime-bits’ 检查）。
- **RC4 流密码** 人们认为 RC4 流密码质量较低，可能允许第三方窃听。（这是 ‘network-security-protocol-checks’ 中的 ‘rc4’ 检查）。

- **主机证书或中间证书中使用 SHA1** 人们认为，若中间证书使用 SHA1 哈希算法，第三方就能签发冒充该签发机构的证书。因此这些连接易受中间人攻击。（这些是 'network-security-protocol-checks' 中的 'signature-sha1' 和 'intermediate-sha1' 检查）。

- **SSL1、SSL2 和 SSL3** 人们认为早于 TLS1.0 的协议易受多种攻击；若你正在做的事情需要更高的安全性，可能应避免使用这些协议。（这是 'network-security-protocol-checks' 中的 'version' 检查）。

- **三重 DES（或 3DES）密码** 3DES 流密码最多提供 112 位的有效安全性，其一个重大安全漏洞已于 2016 年被披露（CVE-2016-2183）。从 2023 年末起，NIST 已在所有应用中弃用它。（这是 'network-security-protocol-checks' 中的 '3des-cipher' 检查）。

若 'network-security-level' 为 'high'，则除上述检查外，还将进行以下检查：

- **已验证的证书更换了公钥** 服务器偶尔会更换密钥，这通常无需担心。不过，若你担心自己的网络连接正被某些机构劫持——它们能接触到可被摆布的证书颁发机构，而这些机构会为第三方服务签发新证书——你可能想跟踪这些变化。

最后，若 'network-security-level' 为 'paranoid'，则在 NSM 第一次看到任何新证书时也会通知你。这样你就能检查 Emacs 建立的所有连接所用的全部证书。

以下附加变量可用于控制 NSM 运行的细节：

- '**nsm-settings-file**' 这是 NSM 存储连接相关详细信息的文件。默认值为 '~/.emacs.d/network-security.data'。

- '**nsm-save-host-names**' 默认情况下，非 'STARTTLS' 连接不会保存主机名。取而代之的是用主机/端口哈希来标识连接。这意味着人们无法随意读取设置文件来看用户连接过哪些服务器。若该变量为 't'，NSM 还会把主机名保存到 'nsm-settings-file' 中。

39 文档查看

DocView 模式是一种主模式，用于查看 DVI、PostScript（PS）、PDF、OpenDocument、Microsoft Office、EPUB、CBZ、FB2、XPS 和 OXPS 文档。它提供切片、缩放以及在文档内搜索等功能。它的工作方式是：借助 'gs'（GhostScript）或 'pdfdraw'/'mutool draw'（MuPDF）等外部工具把文档转换为一组图像，然后显示这些转换后的图像。

当你访问一个可以用 DocView 模式显示的文档文件时，Emacs 会自动使用该模式（1）。例外情况是：当你访问 PostScript 文件时，Emacs 会切换到 PS 模式——一种把 PostScript 文件当作文本编辑的主模式；不过它同时启用 DocView 次模式，因此你可以键入 'C-c C-c' 用 DocView 查看该文档。无论在 DocView 模式还是 DocView 次模式下，重复键入 'C-c C-c'（'doc-view-toggle-display'）都会在 DocView 与底层文件内容之间切换。

当你访问一个通常应由 DocView 模式处理的文件、但某些前提条件不满足时（例如你在终端框架中操作，或 Emacs 没有 PNG 支持），Emacs 会询问你是否要把文档内容当作纯文本查看。若回答是，Emacs 会在缓冲区中启用 Text 模式并激活 DocView 次模式。于是键入 'C-c C-c' 就能切换到这种后备模式，再键入一次 'C-c C-c' 则返回 DocView 模式。在 DocView 模式中键入 'C-c C-t'（'doc-view-open-text'），还可以把纯文本内容显示在一个单独的缓冲区中，其名字与原文档缓冲区相同并加上后缀 '/text'。

你可以用命令 'M-x doc-view-mode' 显式启用 DocView 模式。你可以用 'M-x doc-view-minor-mode' 切换 DocView 次模式。

DocView 模式启动时会显示一个欢迎屏幕，并开始逐页格式化该文件。第一页格式化完成后即会显示出来。

要杀死 DocView 缓冲区，键入 'k'（'doc-view-kill-proc-and-buffer'）。要把它埋藏起来，键入 'q'（'quit-window'）。

> 注：该文档类型所需的外部工具必须可用，并且 Emacs 必须在图形框架中运行且具有 PNG 图像支持。如果这些条件不满足，Emacs 会回退到另一种主模式。

39.1 DocView 导航

在 DocView 模式下，你可以用通常的 Emacs 移动键滚动当前页：'C-p'、'C-n'、'C-b'、'C-f' 以及方向键。

默认情况下，按行移动的键 'C-p' 和 'C-n' 分别会停在当前页的开头和结尾，不再继续滚动。不过，如果你把变量 'doc-view-continuous' 改为非 'nil' 值，那么当你已经位于当前页开头时，'C-p' 会显示上一页；当你位于当前页结尾时，'C-n' 会显示下一页。

你也可以键入 'n'、<PageDown>、<next> 或 'C-x]'（'doc-view-next-page'）来显示下一页。要显示上一页，键入 'p'、<PageUp>、<prior> 或 'C-x ['（'doc-view-previous-page'）。

<SPC>（'doc-view-scroll-up-or-next-page'）是逐页浏览文档的便捷方式：它在当前页内滚动，或者前进到下一页。<DEL> 则以类似方式向后移动（'doc-view-scroll-down-or-previous-page'）。

要跳到第一页，键入 'M-<'（'doc-view-first-page'）；要跳到最后一页，键入 'M->'（'doc-view-last-page'）。要按页码跳转，键入 'M-g M-g' 或 'M-g g'（'doc-view-goto-page'）。

你可以用‘+’（‘doc-view-enlarge’）和‘-’（‘doc-view-shrink’）放大或缩小文档。默认情况下，这些命令只是重新缩放已经渲染好的图像。如果你反而希望按新尺寸重新渲染图像，请把‘doc-view-scale-internally’设为‘nil’。要指定 DocView 的默认尺寸，可定制变量‘doc-view-resolution’。

当‘mutool’和‘djvused’程序可用时，DocView 可以分别用它们为 PDF 和 DjVu 文档生成大纲菜单。OpenOffice 和 LibreOffice 所使用的 Open Document Format 文件的大纲，则是在被转换为 PDF 之后同样用‘mutool’生成。这利用了‘imenu’设施（参见“Imenu”）。你可以用变量‘doc-view-imenu-format’和‘doc-view-imenu-flatten’定制该大纲的‘imenu’条目如何格式化与显示。‘djvused’程序的文件名可以通过修改‘doc-view-djvused-program’用户选项来定制。

你可以用‘m’（‘doc-view-page-to-register’）把当前页保存到寄存器（参见“寄存器”）。不过，这些寄存器不在缓冲区之间共享，只保留在 DocView 缓冲区本地。之后你可以用‘'’（‘doc-view-jump-to-register’）跳转到所保存的页面。

39.2 DocView 搜索

在 DocView 模式下，你可以用正则表达式搜索文件的文本（参见“正则表达式的语法”）。搜索界面借鉴了‘isearch’（参见“增量搜索”）。

要开始搜索，键入‘C-s’（‘doc-view-search’）或‘C-r’（‘doc-view-search-backward’）。这会用迷你缓冲区读取一个正则表达式，然后回显在文档中找到的匹配数目。你可以键入‘C-s’和‘C-r’在匹配之间前后移动。DocView 模式无法在页面图像内标出匹配位置；它改为（在鼠标位置）显示一个工具提示，列出当前页中所有匹配的行。要强制显示该工具提示，键入‘C-t’（‘doc-view-show-tooltip’）。

要开始一次新的搜索，请带前缀参数使用搜索命令；即用‘C-u C-s’向前搜索，或用‘C-u C-r’向后搜索。

39.3 DocView 切片

文档通常有为打印而留的宽页边。在屏幕上阅读文档时，这些页边很恼人，因为它们占用了屏幕空间，还可能导致不方便的滚动。

用 DocView 你可以通过选择要显示的页面“切片”（slice）来隐藏这些页边。切片是页面区域内的一个矩形；一旦你在 DocView 中指定了切片，它就会应用于你查看的任何页面。

要以数值方式指定切片，键入‘c s’（‘doc-view-set-slice’）；然后输入左上角的像素位置以及切片的宽度和高度。

指定切片还有一种更方便的图形化方式，就是‘c m’（‘doc-view-set-slice-using-mouse’），你可以在其中用鼠标选择切片。只需在你希望纳入切片的区域左上角按住鼠标左键，然后把鼠标指针移到右下角并松开按键。

最方便的方式是键入‘c b’（‘doc-view-set-slice-from-bounding-box’），利用文档中自动确定的 BoundingBox 信息设置最优切片。

要取消选定的切片，键入‘c r’（‘doc-view-reset-slice’）。随后 DocView 会显示整个页面，包括其全部页边。

39.4 DocView 转换

为提高效率，DocView 会缓存 ‘gs’ 生成的图像。缓存图像所在目录的名字由变量 ‘doc-view-cache-directory’ 给出。你可以键入 ‘M-x doc-view-clear-cache’ 清空缓存目录。

要强制重新转换当前正在查看的文档，键入 ‘r’ 或 ‘g’（‘revert-buffer’）。要杀死与当前缓冲区关联的转换器进程，键入 ‘K’（‘doc-view-kill-proc’）。命令 ‘k’（‘doc-view-kill-proc-and-buffer’）会杀死转换器进程和 DocView 缓冲区。

40 在 Emacs 中运行 Shell 命令

Emacs 提供了若干命令：把单条命令行传给 shell 子进程；在 Emacs 缓冲区中进行输入输出，以交互方式运行 shell；以及在终端模拟器窗口中运行 shell。

- ‘M-! CMD <RET>’ 运行 shell 命令 CMD 并显示其输出（‘shell-command’）。
- ‘M-| CMD <RET>’ 运行 shell 命令 CMD，以区域内容作为输入；可选地用输出替换该区域（‘shell-command-on-region’）。
- ‘M-& CMD <RET>’ 异步运行 shell 命令 CMD 并显示其输出（‘async-shell-command’）。
- ‘M-x shell’ 通过 Emacs 缓冲区进行输入输出，运行一个子 shell。随后你可以交互地输入命令。
- ‘M-x term’ 通过 Emacs 缓冲区进行输入输出，运行一个子 shell。随后你可以交互地输入命令。它提供完整的终端模拟。

无论何时为可执行程序指定相对文件名（无论是在上述某条命令的 CMD 参数中，还是在其他场合），Emacs 都会在变量 ‘exec-path’ 指定的目录中搜索该程序。此变量的值必须是目录列表；其默认值在 Emacs 启动时取决于环境变量 ‘PATH’（参见“通用变量”）。

‘M-x eshell’ 调用完全用 Emacs 实现的 shell。它有自己的手册加以说明。参见《Eshell 手册》

40.1 单条 Shell 命令

‘M-!’（‘shell-command’）用迷你缓冲区读取一行文本，并专门为该命令启动一个子 shell 来执行它。该命令的标准输入来自空设备。若 shell 命令产生任何输出，输出会显示在回显区（输出较短时），或者显示在 ‘"*Shell Command Output*"’（‘shell-command-buffer-name’）缓冲区中（输出较长时）。变量 ‘resize-mini-windows’ 和 ‘max-mini-window-height’（参见“在迷你缓冲区中编辑”）控制 Emacs 何时认为输出对回显区来说过长。请注意，下文将描述的定制 ‘shell-command-dont-erase-buffer’ 会影响回显区中显示的内容。

例如，解压名为 ‘foo.gz’ 的文件的一种方法是输入 ‘M-! gunzip foo.gz <RET>’。该 shell 命令通常会创建文件 ‘foo’，并且不产生终端输出。

给 ‘shell-command’ 一个数字参数，例如 ‘M-1 M-!’，会令它把终端输出插入当前缓冲区，而不是单独的缓冲区。默认情况下，它把光标点放在输出之前，并把标记设在输出之后（但 ‘shell-command-dont-erase-buffer’ 取非默认值时可能改变这一点，见下文）。例如，‘M-1 M-! gunzip < foo.gz <RET>’ 会把 ‘foo.gz’ 解压后的内容插入当前缓冲区。

只要指定的 shell 命令不以 ‘&’ 结尾，它就会“同步”运行，你必须等它退出才能继续使用 Emacs。要停止等待，输入 ‘C-g’ 退出；这会发送 ‘SIGINT’ 信号以终止该 shell 命令（这与 shell 中 ‘C-c’ 通常产生的信号相同）。随后 Emacs 会等到该命令真正终止。如果该 shell 命令没有停下（因为它忽略了 ‘SIGINT’ 信号），再输入一次 ‘C-g’；这会给该命令发送 ‘SIGKILL’ 信号，该信号无法被忽略。

以 ‘&’ 结尾的 shell 命令会“异步”执行，你可以在它运行时继续使用 Emacs。你也可以输入 ‘M-&’（‘async-shell-command’）来异步执行 shell 命令；这与带结尾 ‘&’ 调用 ‘M-!’ 完全相同，只是不需要写 ‘&’。默认情况下，异步 shell 命令的输出会进入 ‘"*Async Shell Command*"’（‘shell-command-buffer-name-async’）缓冲区。无论该缓冲区是否显示在窗口中，Emacs 都会在输出到达时把它插入该缓冲区。

如果你想同时运行多个异步 shell 命令，它们可能会争用同一个输出缓冲区。选项 ‘async-shell-command-buffer’ 规定了此时该如何处理；例如，是重命名原有的输出缓冲区，还是为新命令使用另一个缓冲区。更多可能性请参阅该变量的文档。

如果你希望异步 shell 命令的输出缓冲区仅在命令产生输出时才显示，请把 ‘async-shell-command-display-buffer’ 设为 ‘nil’。

选项 ‘async-shell-command-width’ 定义异步 shell 命令输出可用的显示列数。正整数告诉 shell 用该列数显示命令输出。默认值为 ‘nil’，表示使用 shell 所提供的相同列数。

要让上述命令在其提示中显示当前目录，请把变量 ‘shell-command-prompt-show-cwd’ 定制为非 ‘nil’ 值。

‘M-|’（‘shell-command-on-region’）与 ‘M-!’ 类似，但它把区域内容作为标准输入传给 shell 命令，而不是不给输入。带数字参数时，它会删除原区域并用 shell 命令的输出替换它。

例如，你可以配合 ‘gpg’ 程序使用 ‘M-|’ 来查看缓冲区中有哪些密钥。如果缓冲区中包含 GnuPG 密钥，输入 ‘C-x h M-| gpg <RET>’ 把整个缓冲区内容喂给 ‘gpg’。这会把密钥列表输出到名字为 ‘shell-command-buffer-name’ 之值的缓冲区中。

上述命令使用由变量 ‘shell-file-name’ 指定的 shell。其默认值在 Emacs 启动时由环境变量 ‘SHELL’ 决定。如果该文件名是相对路径，Emacs 会在 ‘exec-path’ 列出的目录中搜索（参见“在 Emacs 中运行 Shell 命令”）。

若默认目录是远程目录（参见“远程文件”），默认值为 ‘/bin/sh’。可以通过把 ‘shell-file-name’ 声明为连接局部的（参见“按连接的局部变量”）来改变它。

要为 ‘M-!’ 或 ‘M-|’ 指定编码系统，请在此前紧接着使用命令 ‘C-x <RET> c’。参见“用于进程间通信的编码系统”。

默认情况下，错误输出与常规输出混合在输出缓冲区中。但如果你把变量 ‘shell-command-default-error-buffer’ 的值改为字符串，错误输出就会插入到该名称的缓冲区中。

默认情况下，两次 shell 命令之间会清除输出缓冲区，除非输出进入当前缓冲区。如果把选项 ‘shell-command-dont-erase-buffer’ 的值改为 ‘erase’，那么输出缓冲区总是被清除。其他非 ‘nil’ 值会阻止清除输出缓冲区，并且——若输出缓冲区不是当前缓冲区——还会控制插入 shell 命令输出后把光标点放在哪里：

- ‘beg-last-out’ 把光标点放在上一条 shell 命令输出的开头。
- ‘end-last-out’ 把光标点放在上一条 shell 命令输出的末尾，即输出缓冲区的末尾。
- ‘save-point’ 把光标点的位置恢复到插入该 shell 命令输出之前的样子。

请注意，如果该选项非 ‘nil’，回显区中显示的输出可能不只来自最后一条命令，因为回显区只显示输出缓冲区的一部分。

若输出缓冲区不是当前缓冲区，shell 命令输出会追加到该缓冲区的末尾。

40.2 交互式子 shell

要以交互方式运行子 shell，输入 ‘M-x shell’。这会创建（或复用）名为 ‘*shell*’ 的缓冲区，并运行一个 shell 子进程，其输入来自该缓冲区，输出也送往该缓冲区。也就是说，子 shell 的任何终端输出都进入该缓冲区并推进

光标点，而子 shell 的任何终端输入都取自缓冲区中的文本。要给子 shell 输入，请到缓冲区末尾输入文本，并以 <RET> 结束。

默认情况下，以交互方式调用子 shell 时，‘*shell*’ 缓冲区会显示在一个新窗口中，除非当前窗口已经显示 ‘*shell*’ 缓冲区。此行为可通过 ‘display-buffer-alist’ 定制（参见“‘display-buffer’ 的工作方式”）。

子 shell 正在等待或运行命令时，你可以切换窗口或缓冲区，并在 Emacs 中进行其他编辑。只要 Emacs 有时间处理输出（例如在等待键盘输入时），它就会把子 shell 的输出插入 Shell 缓冲区。

在 Shell 缓冲区中，提示符以字体（face）‘comint-highlight-prompt’ 显示，提交的输入行以字体 ‘comint-highlight-input’ 显示。这样更容易区分输入行与 shell 输出。参见“文本字体”。

要创建多个子 shell，请带前缀参数调用 ‘M-x shell’（例如 ‘C-u M-x shell’）。随后该命令会读取缓冲区名，并在该缓冲区中创建（或复用）一个子 shell。你也可以用 ‘M-x rename-uniquely’ 重命名 ‘*shell*’ 缓冲区，然后用普通的 ‘M-x shell’ 创建新的 ‘*shell*’ 缓冲区。不同缓冲区中的子 shell 相互独立、并行运行。

Emacs 设法通过查看你输入的命令来跟踪当前目录，它会寻找 ‘cd’ 之类的命令。这种办法容易出错，因为改变当前目录的方法有很多，所以 Emacs 还会寻找特殊的 OSC（Operating System Commands，操作系统命令）转义码，这类转义码旨在以更可靠的方式传递此信息。你应当安排你的 shell 在每个提示符处打印合适的转义序列，例如使用以下命令：

```
printf "\e]7;file://%s%s\e\\" "$HOSTNAME" "$PWD"
```

然后告诉 Emacs 处理这些转义码：

```
(add-hook 'comint-output-filter-functions #'comint-osc-process-output)
```

要指定 ‘M-x shell’ 使用的 shell 文件名，请定制变量 ‘explicit-shell-file-name’。如果它为 ‘nil’（默认值），Emacs 会使用环境变量 ‘ESHELL’（若存在）。否则，它通常使用变量 ‘shell-file-name’（参见“单条 Shell 命令”）；但如果默认目录是远程的（参见“远程文件”），它会提示你输入 shell 文件名。关于如何高效地输入远程文件名的提示，参见“用于文件名的迷你缓冲区”。

如果文件 ‘~/.emacs_SHELLNAME’ 存在，Emacs 会把其内容作为输入发送给新 shell，其中 SHELLNAME 是加载该 shell 的文件名。例如，若你使用 bash，发送给它的文件是 ‘~/.emacs_bash’。如果找不到该文件，Emacs 会尝试 ‘~/.emacs.d/init_SHELLNAME.sh’。

通过把变量 ‘comint-terminfo-terminal’ 定制为值 ‘"dumb-emacs-ansi"’，你可以为许多命令启用彩色输出（参见“Shell 模式选项”）。

要为 shell 指定编码系统，可以在 ‘M-x shell’ 之前紧接着使用命令 ‘C-x <RET> c’。你也可以在 shell 缓冲区中输入 ‘C-x <RET> p’ 来更改正在运行的子 shell 的编码系统。参见“用于进程间通信的编码系统”。

Emacs 会把子 shell 中的环境变量 ‘INSIDE_EMACS’ 设为 ‘VERSION,comint’，其中 VERSION 是 Emacs 版本（例如 ‘28.1’）。程序可以检查该变量来判断自己是否运行在 Emacs 子 shell 中。

40.3 Shell 模式

Shell 缓冲区的主模式是 Shell 模式。它的许多专用命令绑定在 'C-c' 前缀上，并且与普通 shell 中常见的编辑和作业控制字符相似，只是你必须先输入 'C-c'。下面是 Shell 模式命令列表：

- '<RET>' 把当前行作为输入发送给子 shell（'comint-send-input'）。行首的任何 shell 提示符都会略去（参见"Shell 提示符"）。若光标点在缓冲区末尾，这就像在普通交互式 shell 中提交命令行。不过，你也可以在 shell 缓冲区的其他位置调用 <RET>，把当前行作为输入提交。
- '<TAB>' 补全 shell 缓冲区中光标点前的命令名或文件名（'completion-at-point'）。这使用通常的 Emacs 补全规则（参见"补全"），补全候选为文件名、环境变量名、shell 命令历史和历史引用（参见"Shell 历史引用"）。控制补全的选项见参见"Shell 模式选项"。
- 'M-?' 临时显示光标点前文件名的可能补全列表（'comint-dynamic-list-filename-completions'）。
- 'C-d' 删除一个字符或发送 EOF（'comint-delchar-or-maybe-eof'）。在 shell 缓冲区末尾输入时，它向子 shell 发送 EOF。在缓冲区中任何其他位置输入时，它照常删除一个字符。
- 'C-c C-a' 移到行首，但若有提示符则移到提示符之后（'comint-bol-or-process-mark'）。若连续两次重复此命令，第二次会移回进程标记，即尚未发送给子 shell 的输入的开头。（通常这是同一位置——本行提示符的末尾——但在 'C-c <SPC>' 之后，进程标记可能在前一行。）
- 'C-c <SPC>' 累积多行输入，然后一起发送（'comint-accumulate'）。此命令在光标点前插入一个换行，但不把前面的文本作为输入发送给子 shell——至少暂不发送。当你输入 <RET> 时，两行（新行之前的一行和新行之后的一行）会连同分隔它们的换行一起发送。
- 'C-c C-u' 删除缓冲区末尾待作为输入发送的所有文本（'comint-kill-input'）。若光标点不在缓冲区末尾，则只删除该文本中位于光标点之前的部分。
- 'C-c C-w' 删除光标点前的一个单词（'backward-kill-word'）。
- 'C-c C-c' 中断 shell 或其当前子作业（若有）（'comint-interrupt-subjob'）。此命令还会删除 shell 缓冲区中待发送而尚未发送的任何 shell 输入。
- 'C-c C-z' 停止 shell 或其当前子作业（若有）（'comint-stop-subjob'）。此命令还会删除 shell 缓冲区中待发送而尚未发送的任何 shell 输入。
- 'C-c C-\' 向 shell 或其当前子作业（若有）发送退出信号（'comint-quit-subjob'）。此命令还会删除 shell 缓冲区中待发送而尚未发送的任何 shell 输入。
- 'C-c C-o' 删除 shell 命令的最后一批输出（'comint-delete-output'）。当某条 shell 命令吐出大量碍事的输出时，这很有用。带前缀参数时，此命令把删除的文本存入 'kill-ring'（参见"删除环"），以便你之后在别处粘贴（参见"粘贴"）。
- 'C-c C-s' 把 shell 命令的最后一批输出写入文件（'comint-write-output'）。带前缀参数时改为追加到文件。输出末尾的任何提示符不会被写入。
- 'C-c C-r' 'C-M-l' 滚动显示，使最后一批输出的开头位于窗口顶部；同时把光标移到那里（'comint-show-output'）。
- 'C-c C-e' 滚动使缓冲区的最后一行位于窗口底部（'comint-show-maximum-output'）。
- 'C-c C-f' 向前移动越过一条 shell 命令，但不超出当前行（'shell-forward-command'）。变量 'shell-command-regexp' 规定如何识别命令的结尾。

- ‘C-c C-b’ 向后移动越过一条 shell 命令，但不超出当前行（‘shell-backward-command’）。
- ‘M-x dirs’ 向 shell 询问其工作目录，并更新 Shell 缓冲区的默认目录。参见“目录跟踪”。
- ‘M-x comint-send-invisible <RET> TEXT <RET>’ 在不回显的情况下读取 TEXT，然后把它作为输入发送给 shell。当 shell 命令运行的程序要求输入密码时，这很有用。

请注意，Emacs 默认不回显密码。如果你确实希望回显，请求值（参见“求值 Emacs Lisp 表达式”）以下 Lisp 表达式：

```
(remove-hook 'comint-output-filter-functions
             'comint-watch-for-password-prompt)
```

- ‘M-x comint-continue-subjob’ 继续 shell 进程。若你不小心挂起了 shell 进程，这很有用。（1）
- ‘M-x comint-strip-ctrl-m’ 从当前这组 shell 输出中丢弃所有 control-M 字符。使用此命令最方便的方式是让它在你从子 shell 获得输出时自动运行。要做到这点，请求值以下 Lisp 表达式：

```
(add-hook 'comint-output-filter-functions
          'comint-strip-ctrl-m)
```

- ‘M-x comint-truncate-buffer’ 此命令把 shell 缓冲区截断到由变量 ‘comint-buffer-maximum-size’ 指定的某个最大行数。以下做法可让你每次从子 shell 获得输出时都自动执行此操作：

```
(add-hook 'comint-output-filter-functions
          'comint-truncate-buffer)
```

默认情况下，Shell 模式处理常见的 ANSI 转义码（例如用于改变文本颜色的转义码）。如果你在初始化文件中加入以下内容，Emacs 还会选择性地支持某些扩展转义码，例如某些 OSC（Operating System Codes，操作系统码）：

```
(add-hook 'comint-output-filter-functions #'comint-osc-process-output)
```

启用后，例如 ‘ls --hyperlink’ 的输出会在 Shell 模式缓冲区中变成可点击的按钮。

Shell 模式是 Comint 模式的派生模式，后者是与交互式子进程通信的通用模式。Shell 模式的大多数特性其实来自 Comint 模式，从上文列出的命令名就能看出。Shell 模式的特殊特性包括目录跟踪功能以及少量用户命令。

使用 Comint 模式变体的其他 Emacs 特性包括 GUD（参见“在 Emacs 中运行调试器”）和 ‘M-x run-lisp’（参见“运行外部 Lisp”）。

你可以用 ‘M-x comint-run’ 在子进程中用未经修改的 Comint 模式（不带 Shell 模式的特殊化）执行你所选择的任何程序。要给程序传参数，请用 ‘C-u M-x comint-run’。

> 注：你不应挂起 shell 进程。挂起 shell 的子作业完全是另一回事——那是常规做法，但你必须用 shell 来继续该子作业；此命令做不到。

40.4 Shell 提示符

提示符是程序输出的文本，用来表明它已准备好接受新的用户输入。通常，Comint 模式（因而 Shell 模式）会根据子进程的输出来自动判断缓冲区中哪一部分是提示符。（具体来说，它假定收到的任何不以换行结尾的输出行都是提示符。）

Comint 模式把缓冲区分为两类“字段”（field）：输入字段（用户输入所在处）和输出字段（其他所有地方）。提示符属于输出字段的一部分。多数 Emacs 移动命令不会跨越字段边界，除非它们移动多行。例如，当光标点位于某条 shell 命令行的输入字段中时，‘C-a’ 会把光标点放到输入字段的开头，即提示符之后。在内部，字段是用 ‘field’ 文本属性实现的（参见《Emacs Lisp 参考手册》的“Text Properties”）。

如果你把变量 ‘comint-use-prompt-regexp’ 改为非 ‘nil’ 值，那么 Comint 模式会用正则表达式识别提示符（参见“正则表达式的语法”）。在 Shell 模式中，该正则表达式由变量 ‘shell-prompt-pattern’ 指定。‘comint-use-prompt-regexp’ 的默认值为 ‘nil’，因为这种识别提示符的方法不可靠，但在非常规情况下你可能想把它设为非 ‘nil’ 值。这种情况下，Emacs 不会把 Comint 缓冲区划分为字段，因此通用移动命令的行为与在没有特殊文本属性的缓冲区中一样。不过，你可以用段落移动命令方便地在缓冲区中导航（参见“段落”）；在 Shell 模式中，Emacs 使用 ‘shell-prompt-pattern’ 作为段落边界。

40.5 Shell 命令历史

Shell 缓冲区支持三种重复先前命令的方式。你可以使用类似迷你缓冲区历史的按键；它们的作用与在迷你缓冲区中大致相同，会插入先前命令的文本，而光标点始终位于缓冲区末尾。你可以在缓冲区中移动到原位置的先前输入处，然后重新提交它们，或把它们复制到末尾。你也可以使用 ‘!’ 风格的历史引用。

40.5.1 Shell 历史环

- ‘M-p’ ‘C-<UP>’ 取回下一条更早的旧 shell 命令（‘comint-previous-input’）。
- ‘C-x <UP>’ 用 shell 命令历史补全当前输入（‘comint-complete-input-ring’）。
- ‘M-n’ ‘C-<DOWN>’ 取回下一条更晚的旧 shell 命令（‘comint-next-input’）。
- ‘M-r’ 开始对旧 shell 命令进行增量正则表达式搜索（‘comint-history-isearch-backward-regexp’）。
- ‘C-c C-x’ 从历史中取回下一条后续命令（‘comint-get-next-from-history’）。
- ‘C-c .’ 从一条旧 shell 命令中取回一个参数（‘comint-input-previous-argument’）。
- ‘C-c C-l’ 在另一个窗口中显示缓冲区的 shell 命令历史（‘comint-dynamic-list-input-ring’）。

Shell 缓冲区保存先前输入的 shell 命令历史。要复用历史中的 shell 命令，可使用编辑命令 ‘M-p’、‘M-n’ 和 ‘M-r’。它们的工作方式与迷你缓冲区历史命令类似（参见“迷你缓冲区历史”），区别在于它们在 Shell 缓冲区而非迷你缓冲区中操作，而且 Shell 缓冲区中的 ‘M-r’ 会在 shell 命令历史中进行增量搜索。

‘M-p’ 取回一条更早的 shell 命令到 shell 缓冲区末尾。连续使用 ‘M-p’ 会依次取回更早的 shell 命令，每次都会替换掉已经作为潜在 shell 输入存在的文本。‘M-n’ 亦然，只是它从缓冲区中依次查找更近的 shell 命令。‘C-<UP>’ 的作用与 ‘M-p’ 相同，‘C-<DOWN>’ 与 ‘M-n’ 相同。

‘C-x <UP>’ 尝试根据旧 shell 命令的历史来补全（参见“补全”）你当前的输入。因此，例如，你可以先输入先前执行过的一条又长又复杂的 shell 命令的前几个词，然后调用此命令让 Emacs 填上其余部分。

历史搜索命令 'M-r' 开始对先前的 shell 命令进行增量正则表达式搜索。输入 'M-r' 后，开始输入想要的字符串或正则表达式；最后一条匹配的 shell 命令会显示在当前行。增量搜索命令会发挥其通常效果——例如，'C-s' 和 'C-r' 向前和向后搜索下一个匹配（参见"增量搜索"）。找到想要的输入后，输入 <RET> 结束搜索。这会把该输入放入命令行。在浏览历史列表之前你正在编写的任何部分输入，会在你走到历史环的开头或末尾时恢复。

命令 'C-c .'（'comint-insert-previous-argument'）从之前的命令中复制单个参数，类似 Bash 和 'zsh' 中的 '<ESC> .'。最简单的用法是复制上一条 shell 命令的最后一个参数。带前缀参数 N 时，改为复制第 N 个参数。重复执行 'C-c .' 会从更早的 shell 命令中复制，始终使用相同的 N 值（重复 'C-c .' 命令时不要给前缀参数）。

若把 'comint-insert-previous-argument-from-end' 设为非 'nil' 值，'C-c .' 将改为从末尾数起复制第 N 个参数；这模拟 'zsh' 中的 '<ESC> .'。

这些命令从一条特殊的历史列表中获取先前 shell 命令的文本，而不是从 shell 缓冲区本身获取。因此，编辑 shell 缓冲区，甚至删除其中很大一部分，都不会影响这些命令所访问的历史。

有些 shell 把命令历史保存在文件中，以便你引用先前 shell 会话中的命令。Emacs 会读取你所选 shell 的命令历史文件，以初始化自己的命令历史。历史文件名由 'shell-history-file-name' 中指定的字符串决定。若该用户选项为 't'，则不读取命令历史。若值为 'nil'，则从环境变量 'HISTFILE' 指定的文件读取命令历史；对 bash 从 '~/.bash_history' 读取，对 ksh 从 '~/.sh_history'，对 zsh 从 '~/.zsh_history"'，对其他 shell 从 '~/.history' 读取。

40.5.2 Shell 历史复制

'C-c C-p' 把光标点移到前一个提示符（'comint-previous-prompt'）。

'C-c C-n' 把光标点移到后一个提示符（'comint-next-prompt'）。

'C-c <RET>' 复制光标点处的输入命令，把副本插入缓冲区末尾（'comint-copy-old-input'）。当你把光标点移回先前的命令时，这很有用。复制命令后，你可以用 <RET> 把副本作为输入提交。如果愿意，你也可以在重新提交前编辑副本。若在输出行上使用此命令，则把该行复制到缓冲区末尾。

'mouse-2' 若 'comint-use-prompt-regexp' 为 'nil'（默认值），复制你点击的旧输入命令，把副本插入缓冲区末尾（'comint-insert-input'）。若 'comint-use-prompt-regexp' 非 'nil'，或点击处不在旧输入上，则像往常一样粘贴。

移到先前的输入再用 'C-c <RET>' 或 'mouse-2' 复制它，得到的结果——相同的缓冲区内容——与你使用 'M-p' 足够多次、从历史列表中取回该先前输入所得的结果相同。不过，'C-c <RET>' 复制的是缓冲区中的文本；如果你在输入发送后又在缓冲区中编辑了该文本，它可能与历史列表中的内容不同。

40.5.3 Shell 历史引用

包括 csh 和 bash 在内的多种 shell 支持以 '!' 和 '^' 开头的"历史引用"。Shell 模式能识别这些构造，并可替你完成历史替换。

若你插入一个历史引用并输入 <TAB>，它会在输入历史中搜索匹配的命令，必要时执行替换，并把结果放入缓冲区以取代该历史引用。例如，你可以用 '! m v <TAB>' 取回最近一条以 'mv' 开头的命令。你可以随意编辑该命令，然后按 <RET> 把它重新提交给 shell。

Shell 模式可以选择在你把历史引用发送给 shell 时在缓冲区中展开它们。要启用此功能，把变量 'comint-input-autoexpand' 设为 'input'。你也可以把 <SPC> 绑定到命令 'comint-magic-space' 来让 <SPC> 执行历史展开。参见"交互式更改键绑定"。

当历史引用跟在提示符之后时，Shell 模式能识别它们。Shell 模式如何识别提示符，参见"Shell 提示符"。

40.6 目录跟踪

Shell 模式跟踪发送给子 shell 的 'cd'、'pushd' 和 'popd' 命令，以使 Shell 缓冲区的默认目录（参见"文件名"）与 shell 的工作目录保持一致。它通过检查你发送的输入行来识别这些命令。

若你为这些命令使用了别名，可以把变量 'shell-pushd-regexp'、'shell-popd-regexp' 和 'shell-cd-regexp' 设为相应的正则表达式（参见"正则表达式的语法"），让 Emacs 也能识别它们。例如，若 'shell-pushd-regexp' 匹配某条 shell 命令行的开头，该行就被视为 'pushd' 命令。这些命令只在 shell 命令行的开头被识别。

若 Emacs 对子 shell 工作目录的变化判断有误，输入 'M-x dirs'。该命令向 shell 询问其工作目录，并相应地更新默认目录。它适用于支持最常见命令语法的 shell，但对不寻常的 shell 可能无效。

你也可以使用 Dirtrack 模式，这是一个缓冲区局部的次模式，实现了跟踪 shell 工作目录的另一种方法。要使用该方法，你的 shell 提示符必须始终包含工作目录，并且你必须提供一个正则表达式来识别提示符中哪一部分包含工作目录；详见变量 'dirtrack-list' 的文档。要使用 Dirtrack 模式，在 Shell 缓冲区中输入 'M-x dirtrack-mode'，或把 'dirtrack-mode' 加入 'shell-mode-hook'（参见"钩子"）。

40.7 Shell 书签

Shell 模式缓冲区可以加书签，本地和远程（参见"远程文件"）shell 缓冲区都受支持。参见"书签"。

打开（即"跳转"到）已加书签的 shell 时，会恢复其缓冲区名称、当前目录，并会按需使用你创建远程缓冲区时所用的 shell 命令建立远程连接。

可以自定义选项 'shell-bookmark-name-function' 以符合你的偏好。它默认是函数 'shell-bookmark-name-from-default-directory'，使用缓冲区 'default-directory' 的最后一部分。另一个可选函数 'shell-bookmark-name-from-buffer-name' 使用缓冲区名称，并去掉其 'rename-uniquely' 后缀方括号 "<>"。你可以把该选项绑定到你自己的函数。

打开远程 shell 书签时，你可以禁止建立远程连接。这在用 'desktop-load' 或通过其他会话管理包恢复会话时很有用，可以避免建立连接造成的时间延迟。你可以在未连接的远程缓冲区上用命令 'C-x C-v'（'find-alternate-file'）建立连接。要以交互方式禁止连接，请在调用打开/跳转书签菜单项之前，或在调用命令 'bookmark-jump' 之前，给出前缀参数。(1) (2)

注意：在创建即席多跳远程连接之前，请自定义以下一项或两项：'tramp-save-ad-hoc-proxies' 设为非 'nil'，以持久化代理路由；'tramp-show-ad-hoc-proxies' 设为非 'nil'，以确保连接完全限定。若你在多台主机上使用同一个持久化的书签文件，这很有帮助。

参见《Tramp 手册》的"Ad-hoc multi-hops"。

> 注：(1) 要以编程方式禁止连接，请参阅变量 'shell-bookmark-jump-non-essential' 的文档。

> 注：(2) 要正确处理多跳远程连接，请参阅函数 'shell-bookmark-jump' 的文档。

40.8 Shell 模式选项

若变量 'comint-scroll-to-bottom-on-input' 非 'nil'，插入和粘贴命令会在插入前把选中的窗口滚动到底部。默认值为 'nil'。

若 'comint-scroll-show-maximum-output' 非 'nil'，那么当光标点在末尾时，输出的到达会试图把最后一行文本滚动到窗口底行，以显示尽可能多的有用文本。（这模仿了大多数终端的滚动行为。）默认值为 't'。

通过设置 'comint-move-point-for-output'，你可以选择让光标点在输出到达时跳到缓冲区末尾——无论此前光标点在缓冲区何处。若值为 'this'，光标点在选中窗口中跳转。若值为 'all'，光标点在每个显示该 Comint 缓冲区的窗口中跳转。若值为 'other'，光标点在所有显示当前缓冲区的非选中窗口中跳转。默认值为 'nil'，表示光标点不跳到末尾。

若设置 'comint-prompt-read-only'，Comint 缓冲区中的提示符为只读。

变量 'comint-input-ignoredups' 控制是否把连续相同的输入存入输入历史。非 'nil' 值表示忽略与前一输入相同的输入。默认值为 'nil'，表示即使与前一输入相同，也存储每个输入。

有三个变量可自定义文件名补全。变量 'comint-completion-addsuffix' 控制补全是否插入空格或斜杠，以表明文件名或目录名已完全补全（非 'nil' 表示确实插入空格或斜杠）。'comint-completion-recexact' 若非 'nil'，则指示 <TAB> 在通常的 Emacs 补全算法连一个字符都无法补全时，选择最短的可能补全。'comint-completion-autolist' 若非 'nil'，表示只要补全不精确就列出所有可能的补全。

命令补全通常只考虑可执行文件。若把 'shell-completion-execonly' 设为 'nil'，它也会考虑不可执行文件。

变量 'shell-completion-fignore' 指定 Shell 模式补全中要忽略的文件名扩展名列表。默认设置为 'nil'，但有些用户偏好 '("~" "#" "%")'，以忽略以 '~'、'#' 或 '%' 结尾的文件名。其他相关的 Comint 模式改用变量 'comint-completion-fignore'。

shell 命令补全的一些实现细节也可以在 'shell-dynamic-complete-command' 函数的 lisp 文档中找到。

你可以配置 'pushd' 的行为。有若干变量控制：无参数时 'pushd' 是否像 'cd' 一样行为（'shell-pushd-tohome'）；带数值参数时是弹出而非轮转（'shell-pushd-dextract'）；以及是否只在目录尚未在栈上时才把它加入目录栈（'shell-pushd-dunique'）。当然，你选择的值应与底层 shell 相匹配。

Comint 模式把 'TERM' 环境变量设为安全默认值，但这会禁用一些有用的特性。例如，在使用 'TERM' 判断是否支持颜色的应用中，彩色输出会被禁用。若你系统上的 terminfo 数据库包含 '"dumb-emacs-ansi"' 终端类型的定义（所有较新版本的 terminfo 都如此），你可以通过把选项 'comint-terminfo-terminal' 自定义为 '"dumb-emacs-ansi"' 来启用高级终端特性，包括颜色。只要 'system-uses-terminfo' 非 'nil'，Emacs 就会把 'comint-terminfo-terminal' 用作 'TERM' 的值。

'comint-terminfo-terminal' 和 'system-uses-terminfo' 都可以声明为连接局部变量，以便调整这些选项以匹配远程系统的期望（参见"按连接的局部变量"）。

40.9 Emacs 终端模拟器

要在文本终端模拟器中运行子 shell，使用 'M-x term'。这会创建（或复用）名为 '*terminal*' 的缓冲区，并运行一个子 shell，其输入来自你的键盘，输出进入该缓冲区。

终端模拟器使用 Term 模式，它有两种输入模式。在"行模式"（line mode）中，Term 的行为基本与 Shell 模式相同（参见"Shell 模式"）。在"字符模式"（char mode）中，每个字符都作为终端输入直接发送给子 shell；唯一的例外是终端转义字符，默认是 'C-c'（参见"Term 模式"）。输入的回显由子 shell 负责；子 shell 的任何终端输出都进入缓冲区，并推进光标点。

有些程序（例如 Emacs 本身）需要详细控制终端屏幕的外观。它们通过发出特殊的控制码来实现。Term 模式能识别并处理 ANSI 标准的 VT100 风格转义序列，大多数现代终端（包括 'xterm'）都接受这种序列。（因此，你实际上可以在 Emacs 的 Term 窗口中运行 Emacs。）

'term' 字体（face）指定终端模拟器中文本的默认外观（默认与 'default' 字体相同）。当使用终端控制码改变文本外观时，在终端模拟器中以字体 'term-color-black'、'term-color-red'、'term-color-green'、'term-color-yellow'、'term-color-blue'、'term-color-magenta'、'term-color-cyan'、'term-color-white'、'term-color-underline' 和 'term-color-bold' 表示。参见"文本字体"。

你也可以用 Term 模式与连接到串口的设备通信。参见"串口终端"。

用于加载子 shell 的文件名，其确定方式与 Shell 模式相同。要创建多个终端模拟器，用 'M-x rename-uniquely' 把 '*terminal*' 缓冲区改名为别的名字，与 Shell 模式一样。

与 Shell 模式不同，Term 模式不通过检查你的输入来跟踪当前目录。但有些 shell 能告诉 Term 当前目录是什么。'bash' 1.15 及更高版本会自动这样做。

40.10 Term 模式

要在 Term 模式中在线模式和字符模式之间切换，使用以下命令：

'C-c C-j' 切换到行模式（'term-line-mode'）。若已处于行模式则不做任何事。

'C-c C-k' 切换到字符模式（'term-char-mode'）。若已处于字符模式则不做任何事。

以下命令仅在字符模式中可用：

'C-c C-c' 向子 shell 发送字面的 'C-c'（'term-interrupt-subjob'）。

'C-c CHAR' 这等价于普通 Emacs 中的 'C-x CHAR'。例如，'C-c o' 调用 'C-x o' 的全局绑定，通常即 'other-window'。

Term 模式有逐页显示特性。启用后，它使输出在每一屏结束时暂停：

'C-c C-q' 切换逐页显示特性（'term-pager-toggle'）。该命令在行模式和字符模式中都可用。启用该特性时，模式行显示单词 'page'，并且每当 Term 收到超过一屏的输出，它就暂停并在模式行中显示 'MORE'。输入 <SPC> 显示下一屏输出，或输入 '?' 查看其他选项。该界面类似于 'more' 程序。

40.11 远程主机 Shell

你可以从 Term 窗口登录远程计算机，使用的命令与在普通终端中相同（例如‘ssh’命令）。

向你索要密码的程序通常会禁止回显密码，因此密码不会出现在缓冲区中。若缓冲区处于字符模式，这会与使用真实终端时完全一样。若处于行模式，密码会暂时可见，但按回车时会被擦除。（这是自动发生的；没有特殊的密码处理。）

当你登录到另一台机器时，需要在远程登录命令的环境中设置‘TERM’环境变量，以指明你使用的终端类型。（若使用 bash，可在远程登录命令之前写出变量赋值，不用分隔逗号。）终端类型‘ansi’或‘vt100’在大多数系统上都能工作。

40.12 串口终端

若你有设备连接到计算机的串口，可以输入‘M-x serial-term’与其通信。该命令询问串口名称和速度，并切换到一个新的 Term 模式缓冲区。Emacs 通过该缓冲区与串行设备通信，就像在普通 Term 模式中与终端通信一样。

串口速度以比特每秒计量。最常见的速度是 9600 比特每秒。你可以通过点击模式行交互式地改变速度。

点击模式行中的‘8N1’还能进一步配置串口。默认情况下，串口配置为‘8N1’，意为每个字节由 8 个数据位、无校验位（No parity check bit）和 1 个停止位组成。

若速度或配置错误，你无法与设备通信，窗口中可能只会看到乱码输出。

41 把 Emacs 用作服务器

许多程序都可以调用你选定的编辑器来编辑某段特定的文本。例如，版本控制系统会调用编辑器来输入版本控制日志（参见“版本控制”），Unix 的 ‘mail’ 工具会调用编辑器来输入要发送的消息。按照惯例，你选定的编辑器由环境变量 ‘EDITOR’ 指定。若把 ‘EDITOR’ 设为 ‘emacs’，确实会调用 Emacs，但方式并不方便——它会启动一个新的 Emacs 进程。之所以不方便，是因为新的 Emacs 进程不会与任何已有的 Emacs 进程共享缓冲区、命令历史或其他种类的信息。

把 Emacs 设置为“编辑服务器”（edit server），让它“监听”外部的编辑请求并做出相应处理，即可解决这个问题。启动 Emacs 服务器有多种方式：

- 在已有的 Emacs 进程中运行命令 ‘server-start’：输入 ‘M-x server-start’，或者把表达式 ‘(server-start)’ 放进初始化文件（参见“Emacs 初始化文件”）。该已有 Emacs 进程就是服务器；退出 Emacs 时，服务器随 Emacs 进程一起终止。
- 用 ‘--daemon’ 系列命令行选项之一把 Emacs 作为“守护进程”运行。参见“初始选项”。以这种方式启动时，Emacs 会在初始化后调用 ‘server-start’，并且不打开初始框架。随后它便等待来自客户端的编辑请求。
- 带 ‘--alternate-editor=""’ 命令行选项运行命令 ‘emacsclient’。仅当尚无 Emacs 守护进程在运行时，才会启动一个 Emacs 守护进程。
- 如果你的操作系统用 ‘systemd’ 管理启动过程，就可以借助随附的“systemd unit 文件”，在登录时自动以守护进程模式启动 Emacs。要激活这一功能，执行 ‘systemctl --user enable emacs’。（如果 Emacs 安装在非标准位置，可能需要把 ‘emacs.service’ 文件复制到标准目录，例如 ‘~/.config/systemd/user/’。）
- 外部进程可以在指定套接字上出现连接事件时调用 Emacs 服务器，并把该套接字传给新的 Emacs 服务器进程。‘systemd’ 的套接字功能就是一例：‘systemd’ 服务创建一个套接字并在其上监听连接；当 ‘emacsclient’ 首次连接到该套接字时，‘systemd’ 可以启动 Emacs 服务器，并把套接字交给它，用以服务 ‘emacsclient’ 的连接。使用这一功能的配置可以是：

```
‘~/.config/systemd/user/emacs.socket’:
     [Socket]
     ListenStream=/path/to/.emacs.socket
     DirectoryMode=0700
```

```
[Install]
WantedBy=sockets.target
```

（上文所述的 ‘emacs.service’ 文件也必须安装。）

‘ListenStream’ 给出的路径就是 Emacs 监听 ‘emacsclient’ 连接的路径；该文件由你自行选择。

一旦启动了 Emacs 服务器，就可以用名为 ‘emacsclient’ 的 shell 命令连接到该 Emacs 进程，并让它访问某个文件。然后可以把环境变量 ‘EDITOR’ 设为 ‘emacsclient’，这样外部程序就会使用已有的 Emacs 进程来编辑。(1)

用变量 ‘server-name’ 给每个服务器取一个唯一的“服务器名”，就可以在同一台机器上运行多个 Emacs 服务器。例如，‘M-x set-variable <RET> server-name <RET> "foo" <RET>’ 把服务器名设为 ‘foo’。‘emacsclient’ 程序可

以用 '-s' 或 '-f' 选项按名字指定服务器（参见"'emacsclient' 选项"），具体用哪个取决于该服务器是否使用 TCP 套接字（参见"TCP Emacs 服务器"）。

如果你想运行多个 Emacs 守护进程（参见"初始选项"），可以像下面这样给每个守护进程指定各自的服务器名：

```
emacs --daemon=foo
```

在满足某些条件时，可以选择让 Emacs 服务器自动停止。为此，把选项 'server-stop-automatically' 设为下列值之一：

'empty' 该值使服务器在没有客户端、没有未保存的文件访问缓冲区、也不再有任何运行中的进程时停止。

'delete-frame' 该值表示，当最后一个客户端框架被关闭时，会询问你是否要保存每个未保存的文件访问缓冲区、是否可以停止每个未完成的进程；若都得到确认，服务器就停止。

'kill-terminal' 该值表示，当用 'C-x C-c'（'save-buffers-kill-terminal'）关闭最后一个客户端框架时，会询问你是否要保存每个未保存的文件访问缓冲区、是否可以停止每个未完成的进程；若都得到确认，服务器就停止。

如果你用唯一的服务器名定义了服务器，就可以从另一个 Emacs 实例连接到该服务器，并用函数 'server-eval-at' 在服务器上求值 Lisp 表达式。例如，'(server-eval-at "foo" '(+ 1 2))' 在 'foo' 服务器上求值表达式 '(+ 1 2)' 并返回 '3'。（如果没有该名字的服务器，就会发出错误信号。）目前这一功能主要对开发者有用。

如果你的操作系统的桌面环境兼容 freedesktop.org（大多数 GNU/Linux 及其他较新的类 Unix GUI 都是如此），就可以使用 'Emacs (Client)' 菜单项，通过 'emacsclient' 连接到 Emacs 服务器。若守护进程尚未运行，则会启动它。

> 注：有些程序使用不同的环境变量；例如，要让 TeX 使用 'emacsclient'，可把环境变量 'TEXEDIT' 设为 'emacsclient +%d %s'。

41.1 TCP Emacs 服务器

Emacs 服务器通常监听本地 Unix 域套接字上的连接。有些操作系统（如 MS-Windows）不支持本地套接字；在这种情况下，服务器改用 TCP 套接字。有时即使支持本地套接字，让服务器监听 TCP 套接字也很有用，例如你需要从远程机器联系 Emacs 服务器时。把 'server-use-tcp' 设为非 'nil' 值，Emacs 就会监听 TCP 套接字而不是本地套接字。如果你的操作系统不支持本地套接字，这就是默认行为。

如果 Emacs 服务器被设置为使用 TCP，它默认监听 localhost 接口上的随机端口。可以用变量 'server-host' 和 'server-port' 改到其他接口和/或固定端口。

TCP 套接字不受文件系统权限的约束。为了在一定程度上控制哪些用户可以通过 TCP 套接字与 Emacs 服务器通信，'emacsclient' 程序必须向服务器发送授权密钥。该密钥通常由 Emacs 服务器随机生成。这是推荐的操作方式。

如有需要，可以设置 'server-auth-key' 变量，把授权密钥设为静态值。该密钥必须由 64 个除空格以外的 ASCII 可打印字符组成（即 '!' 到 '~' 之间的字符，或十进制编码 33 到 126 的字符）。可以用 'M-x server-generate-key' 获得一个随机密钥。

启动 TCP Emacs 服务器时，Emacs 会创建一个“服务器文件”（server file），其中包含 ‘emacsclient’ 连接该服务器所用的 TCP 信息。变量 ‘server-auth-dir’ 指定存放服务器文件的默认目录；默认是 ‘~/.emacs.d/server/’。在没有带文件权限的本地套接字的情况下，该目录的权限决定了哪些用户可以让自己的 ‘emacsclient’ 进程与 Emacs 服务器通信。如果 ‘server-name’ 是绝对文件名，服务器文件就创建在该文件名指定的位置。

要让 ‘emacsclient’ 用指定的服务器文件通过 TCP 连接服务器，可使用 ‘-f’ 或 ‘--server-file’ 选项，或者设置环境变量 ‘EMACS_SERVER_FILE’（参见“‘emacsclient’ 选项”）。如果 ‘server-auth-dir’ 被设为非标准值，或者 ‘server-name’ 被设为绝对文件名，‘emacsclient’ 就需要服务器文件的绝对文件名，因为 ‘emacsclient’ 中硬编码了默认的 ‘server-auth-dir’，并以其作为解析相对文件名的目录。

41.2 调用 ‘emacsclient’

使用 ‘emacsclient’ 程序最简单的方式是运行 shell 命令 ‘emacsclient FILE’，其中 FILE 是文件名。该命令连接到 Emacs 服务器，并让那个 Emacs 进程在其已有的某个框架中访问 FILE——该框架可以是图形框架，也可以是文本终端中的框架（参见“框架与图形显示”）。随后你就可以选择那个框架开始编辑。

如果没有 Emacs 服务器，‘emacsclient’ 程序会带着错误消息终止（给 ‘emacsclient’ 使用 ‘--alternate-editor=""’ 选项可以避免这种情况，参见“‘emacsclient’ 选项”）。如果该 Emacs 进程没有已有框架——以守护进程方式启动时就会如此（参见“把 Emacs 用作服务器”）——那么 Emacs 会在你调用 ‘emacsclient’ 的那个终端上打开一个框架。

也可以用 ‘-c’ 选项强制 ‘emacsclient’ 在图形显示上打开新框架，或用 ‘-t’ 选项在文本终端上打开新框架。参见“‘emacsclient’ 选项”。如果用 ‘-c’ 通过 X 转发打开多个框架，可能希望禁用单键盘模式。参见“多个 X 显示”。

如果只在单个文本终端上运行，可以用以下两种方法之一在 ‘emacsclient’ 所在的 shell 与 Emacs 服务器之间切换：（i）把 Emacs 服务器和 ‘emacsclient’ 分别运行在不同的虚拟终端上，调用 ‘emacsclient’ 后切换到 Emacs 服务器所在的虚拟终端；或（ii）在 Emacs 服务器内部，用 Shell 模式（参见“交互式子 shell”）或 Term 模式（参见“Term 模式”）调用 ‘emacsclient’；这样 ‘emacsclient’ 只会阻塞 Emacs 下的子 shell，你仍可用 Emacs 编辑文件。

在 Emacs 服务器中编辑完 FILE 后，在其缓冲区里输入 ‘C-x #’（‘server-edit’）。这会保存文件，并向 ‘emacsclient’ 程序回送一条消息，告诉它退出。使用 ‘EDITOR’ 的程序通常会在做其他事情之前等待编辑器——这里是 ‘emacsclient’——退出。

如果反而想放弃这次编辑，可使用 ‘M-x server-edit-abort’ 命令。它向 ‘emacsclient’ 程序回送一条消息，告诉它以异常退出状态退出，并且不保存任何缓冲区。

也可以用多个文件名参数调用 ‘emacsclient’：‘emacsclient FILE1 FILE2 ...’ 让 Emacs 服务器访问 FILE1、FILE2 等等。Emacs 选中访问 FILE1 的缓冲区，并把其他缓冲区沉到缓冲区列表底部（参见“多缓冲区”）。所有指定的文件都处理完毕后（即你在每个服务器缓冲区中都输入过 ‘C-x #’ 后），‘emacsclient’ 程序就退出。

结束服务器缓冲区的编辑时也会删除该缓冲区，除非在服务器被要求创建它之前，该缓冲区就已存在于这个 Emacs 会话中。不过，如果把 ‘server-kill-new-buffers’ 设为 ‘nil’，就会改用另一条标准：仅当文件名匹配正则表达式 ‘server-temp-file-regexp’ 时，结束该服务器缓冲区才删除它。这样设置是为了区分某些临时文件。

每次 ‘C-x #’ 都会检查是否还有其他待处理的、编辑各种文件的外部请求，并选中下一个这样的文件。你愿意的话也可以手动切换到服务器缓冲区，不必通过 ‘C-x #’ 到达它。但 ‘C-x #’ 才是告诉 ‘emacsclient’ 你已编辑完毕的方式。

如果把变量 ‘server-window’ 的值设为一个窗口或一个框架，‘C-x #’ 就总是在该窗口或该框架中显示下一个服务器缓冲区。

‘emacsclient’ 连接时，服务器通常会输出一条消息，说明如何退出客户端框架。如果 ‘server-client-instructions’ 被设为 ‘nil’，则抑制这条消息。

41.3 ‘emacsclient’ 选项

可以给 ‘emacsclient’ 程序传递一些可选参数，例如：

```
emacsclient -c +12 FILE1 +4:3 FILE2
```

‘+LINE’ 或 ‘+LINE:COLUMN’ 参数为下一个文件参数指定行号，或行号与列号。它们的行为与 Emacs 本身的命令行参数相同。参见“动作参数”。

‘emacsclient’ 识别的其他可选参数列举如下：

‘-a COMMAND’ ‘--alternate-editor=COMMAND’ 指定当 ‘emacsclient’ 无法联系到 Emacs 时运行的 shell 命令。在脚本中运行 ‘emacsclient’ 时这很有用。该命令可以包含参数，参数可用 "like this" 这样的方式加引号。目前不支持对引号进行转义。

作为特例，如果 COMMAND 是空字符串，那么 ‘emacsclient’ 会以守护进程模式（即 ‘emacs --daemon’）启动 Emacs，然后再次尝试连接。

环境变量 ‘ALTERNATE_EDITOR’ 与 ‘-a’ 选项作用相同。如果两者都存在，后者优先。

‘-c’ ‘--create-frame’ 新建一个图形“客户端框架”，而不是使用已有的 Emacs 框架。‘C-x C-c’ 在客户端框架中的特殊行为见下文。如果 Emacs 无法创建新的图形框架（例如无法连接到 X 服务器），它会尝试创建文本终端客户端框架，就好像你改为提供了 ‘-t’ 选项一样。

在 MS-Windows 上，单个 Emacs 会话不能同时在图形终端和文本终端上显示框架，也不能在多个文本终端上显示框架。因此，如果 Emacs 服务器运行在文本终端上，‘-c’ 选项就和 ‘-t’ 选项一样，在服务器当前所在的文本终端中创建新框架。参见“如何在 MS-Windows 上启动 Emacs”。

如果在提供 ‘-c’ 选项时省略文件名参数，新框架默认显示 ‘*scratch*’ 缓冲区。可以用变量 ‘initial-buffer-choice’ 自定义这一行为（*note Entering Emacs::）。

‘-r’ ‘--reuse-frame’ 如果尚不存在图形客户端框架，就新建一个；否则使用已有的 Emacs 框架。

‘-F ALIST’ ‘--frame-parameters=ALIST’ 设置新建图形框架的参数（参见“框架参数”）。

‘-d DISPLAY’ ‘--display=DISPLAY’ 让 Emacs 在 X 显示 DISPLAY 上打开给定的文件（假设有多个可用的 X 显示）。

‘-e’ ‘--eval’ 让 Emacs 求值一些 Emacs Lisp 代码，而不是访问文件。给出该选项时，‘emacsclient’ 的参数被解释为要求值的表达式列表，而不是要访问的文件列表。

通过‘--eval’命令行选项传递复杂的 Lisp 表达式时，有时需要对 shell 的特殊字符做繁琐的转义。为避免这一点，可以把表达式中要传给 Lisp 函数的参数作为额外的独立参数传给‘emacsclient’，并在表达式中使用 SERVER-EVAL-ARGS-LEFT 访问这些参数。如果传给‘emacsclient’的是空字符串参数，它们也会被包含在内。请注意，无论代码是否成功，表达式都应从 SERVER-EVAL-ARGS-LEFT 中移除已处理过的参数，例如使用‘pop’，否则 Emacs 会把那些参数当作独立的 Lisp 表达式来求值。

‘-f SERVER-FILE’‘--server-file=SERVER-FILE’指定用于通过 TCP 连接 Emacs 服务器的服务器文件（参见“TCP Emacs 服务器”）。也可以设置环境变量‘EMACS_SERVER_FILE’指向该服务器文件。（命令行选项会覆盖环境变量。）

Emacs 服务器通常使用本地套接字监听连接，但也支持通过 TCP 连接。要连接到 TCP Emacs 服务器，‘emacsclient’需要读取一个“服务器文件”，其中包含该 Emacs 服务器的连接细节。该文件名由本选项指定，可以是相对于‘~/.emacs.d/server’的文件名，也可以是绝对文件名。参见“TCP Emacs 服务器”。

‘-n’‘--no-wait’让‘emacsclient’立即退出，而不是等到所有服务器缓冲区都结束。你可以在 Emacs 中随意花费时间编辑服务器缓冲区，在其中输入‘C-x #’时它们也不会被删除。

‘-w’‘--timeout=N’在放弃之前等待 Emacs 响应 N 秒。如果该时间内没有响应，‘emacsclient’会显示一条警告并退出。默认值是‘0’，表示一直等待。

‘--parent-id=ID’通过 XEmbed 协议，把‘emacsclient’框架作为客户端框架打开在 id 为 ID 的父 X 窗口中。目前该选项主要对开发者有用。

‘-q’‘--quiet’不让‘emacsclient’显示关于等待 Emacs 或连接远程服务器套接字的消息。

‘-u’‘--suppress-output’不让‘emacsclient’显示服务器返回的结果。这大多在与‘-e’配合使用、且求值是为了副作用而非结果时有用。

‘-s SERVER-NAME’‘--socket-name=SERVER-NAME’连接到名为 SERVER-NAME 的 Emacs 服务器。（该选项在 MS-Windows 上不受支持。）服务器名由 Emacs 服务器上的变量‘server-name’给出。如果省略该选项，‘emacsclient’连接默认套接字。如果你把 Emacs 服务器的‘server-name’设为绝对文件名，就把同一个绝对文件名作为 SERVER-NAME 提供给该选项，指示‘emacsclient’连接到那个服务器。如果你以守护进程方式启动 Emacs（参见“初始选项”）并为该守护进程启动的服务器指定了名字，就需要使用该选项。

也可以设置环境变量‘EMACS_SOCKET_NAME’指向服务器套接字。（命令行选项会覆盖环境变量。）

‘-t’‘--tty’‘-nw’‘--no-window-system’在当前文本终端上新建客户端框架，而不是使用已有的 Emacs 框架。其行为与上文所述的‘-c’选项相同，只是创建的是文本终端框架（参见“文本终端”）。

在 MS-Windows 上，如果 Emacs 服务器使用图形显示，‘-t’的行为与‘-c’相同；但如果 Emacs 服务器运行在文本终端上，它会在当前文本终端中创建新框架。

‘-T TRAMP-PREFIX’‘--tramp=TRAMP-PREFIX’设置添加到文件名前的前缀，以便 Emacs 使用 TRAMP（参见《Tramp 手册》）定位远程机器上的文件（参见“远程文件”）。这大多在从远程主机使用 Emacs 服务器时有用。通过 ssh 转发监听套接字，或 ssh 转发监听端口参见“TCP Emacs 服务器”并让 SERVER-FILE 在远程机器上可用，远程机器上的程序就可以把‘emacsclient’用作‘EDITOR’及类似环境变量的值；但文件不会交给远程机器上的 Emacs 服务器，而是在本地 Emacs 会话中用 TRAMP 访问。

设置环境变量‘EMACSCLIENT_TRAMP’与使用‘-T’选项作用相同。如果两者都指定了，以命令行选项为准。

例如，假设有两台主机‘local’和‘remote’。

```
local$ ssh -R "/home/%r/.emacs.socket":"${XDG_RUNTIME_DIR:-${TMPDIR:-/tmp}/emacs%i}${XDG_RUNTI
ME_DIR:+/emacs}/server" remote
remote$ export EMACS_SOCKET_NAME=$HOME/.emacs.socket
remote$ export EMACSCLIENT_TRAMP=/ssh:remote:
remote$ export EDITOR=emacsclient
remote$ $EDITOR /tmp/foo.txt #Should open in local emacs.
```

如果你使用的平台中‘emacsclient’不使用 Unix 域套接字（即 MS-Windows），或者你的 SSH 实现无法转发它们（例如 6.7 之前的 OpenSSH），可以改为转发一个 TCP 端口。在本例中，假设本地 Emacs 监听 TCP 端口 12345。再假设‘/home’位于共享文件系统上，因此服务器文件‘~/.emacs.d/server/server’在两台主机上都可读。

```
local$ ssh -R12345:localhost:12345 remote
remote$ export EMACS_SERVER_FILE=server
remote$ export EMACSCLIENT_TRAMP=/ssh:remote:
remote$ export EDITOR=emacsclient
remote$ $EDITOR /tmp/foo.txt #Should open in local emacs.
```

‘-V’‘--version’打印版本信息并退出。

‘-H’‘--help’打印用法信息并退出。

用‘-c’或‘-t’选项创建的新的图形框架或文本终端框架被视为“客户端框架”。你从客户端框架中新建的任何框架也被视为客户端框架。如果在客户端框架中输入‘C-x C-c’（‘save-buffers-kill-terminal’），该命令不会像通常那样结束 Emacs 会话（参见“退出 Emacs”），而是由 Emacs 删除该客户端框架；此外，如果该客户端框架有一个‘emacsclient’正等待重新获得控制权（即你没有提供‘-n’选项），Emacs 还会删除同一客户端的其他所有框架，并把该客户端的服务器缓冲区都标记为已结束，就好像你在这些缓冲区里都输入了‘C-x #’一样。如果删除客户端框架后恰好没有剩余框架，Emacs 会话就退出。

作为例外，当 Emacs 以守护进程方式启动时，所有框架都被视为客户端框架，‘C-x C-c’永远不会结束 Emacs。要结束守护进程会话，输入‘M-x kill-emacs’。

注意，‘-t’和‘-n’选项相互矛盾：‘-t’表示接管当前文本终端以创建新的客户端框架，而‘-n’表示不接管文本终端。如果同时提供这两个选项，Emacs 会在已有的框架中访问指定的文件，而不是新建客户端框架，从而使‘-t’的效果失效。

42 打印

Emacs 提供的命令可以打印整个缓冲区或其中一部分的硬拷贝。你可以直接调用下面详细介绍的打印命令，也可以使用菜单栏上的 'File' 菜单。

除了本节描述的命令，你还可以从 Dired（参见"对文件进行操作"）和日记（参见"显示日记"）打印硬拷贝。你也可以用命令 'M-x htmlfontify-buffer' 把 Emacs 缓冲区"打印"成 HTML：该命令把当前缓冲区转换成 HTML 文件，用基于 CSS 的标记替代 Emacs 字体（face）。此外，Org 模式允许你把 Org 文件打印成多种格式，例如 PDF（参见"Org 模式"）。

'M-x print-buffer' 打印当前缓冲区的硬拷贝，页眉包含文件名和页码。'M-x lpr-buffer' 打印当前缓冲区的硬拷贝，不带页眉。'M-x print-region' 与 'print-buffer' 类似，但只打印当前区域。'M-x lpr-region' 与 'lpr-buffer' 类似，但只打印当前区域。

在大多数操作系统上，上述硬拷贝命令通过调用 'lpr' 程序来提交打印文件。要更换打印机程序，请定制变量 'lpr-command'。要指定传给打印机程序的额外开关，请定制列表变量 'lpr-switches'。它的值应当是一个选项字符串列表，其中每个字符串都应以 '-' 开头（例如选项字符串 '"-w80"' 指定行宽为 80 列）。默认值是空列表 'nil'。

要指定使用的打印机，请设置变量 'printer-name'。默认值 'nil' 表示默认打印机。若你把它设成某个打印机名（字符串），该名字会以 '-P' 开关传给 'lpr'；如果你用的不是 'lpr'，就应当用 'lpr-printer-switch' 指定该开关。

变量 'lpr-headers-switches' 同样指定用于生成页眉的额外开关。变量 'lpr-add-switches' 控制是否向打印机程序提供 '-T' 和 '-J' 选项（适合 'lpr' 使用）：'nil' 表示不添加（如果你的打印机程序与 'lpr' 不兼容，就应当取这个值）。

42.1 PostScript 硬拷贝

这些命令把缓冲区内容转换成 PostScript，或直接打印，或留在另一个 Emacs 缓冲区中。

'M-x ps-print-buffer' 以 PostScript 形式打印当前缓冲区的硬拷贝。'M-x ps-print-region' 以 PostScript 形式打印当前区域的硬拷贝。'M-x ps-print-buffer-with-faces' 以 PostScript 形式打印当前缓冲区的硬拷贝，并借助 PostScript 特性显示文本所用的字体（face）。'M-x ps-print-region-with-faces' 以 PostScript 形式打印当前区域的硬拷贝，并显示文本所用的字体（face）。'M-x ps-spool-buffer' 为当前缓冲区的文本生成 PostScript 图像并排入队列。'M-x ps-spool-region' 为当前区域生成 PostScript 图像并排入队列。'M-x ps-spool-buffer-with-faces' 为当前缓冲区生成 PostScript 图像并排入队列，同时显示所用的字体（face）。'M-x ps-spool-region-with-faces' 为当前区域生成 PostScript 图像并排入队列，同时显示所用的字体（face）。'M-x ps-despool' 把排队中的 PostScript 送往打印机。'M-x handwrite' 把当前缓冲区生成/打印成手写体的 PostScript。

'ps-print-buffer' 和 'ps-print-region' 命令以 PostScript 形式打印缓冲区内容。一个命令打印整个缓冲区，另一个只打印区域。'ps-print-buffer-with-faces' 和 'ps-print-region-with-faces' 行为类似，但使用 PostScript 特性来显示缓冲区文本的字体（face，即字体与颜色）。

交互式地使用时，若你给出前缀参数（'C-u'），这些命令会提示用户输入文件名，并把 PostScript 图像保存到该文件中，而不是送往打印机。

名字中含有‘spool’而非‘print’的那些命令，会在 Emacs 缓冲区中生成 PostScript 输出，而不是送往打印机。

使用命令‘ps-despool’把排队中的图像送往打印机。该命令把‘-spool-’系列命令（见上面的命令）生成的 PostScript 送往打印机。带前缀参数（‘C-u’）时，它会提示输入文件名，并把排队中的 PostScript 图像保存到该文件，而不是送往打印机。

‘M-x handwrite’更为轻浮。它把当前缓冲区生成为连笔手写文档样式的 PostScript 呈现。可以在‘handwrite’组中定制它。该函数只支持 ISO 8859-1 字符。

42.2 PostScript 硬拷贝的变量

所有 PostScript 硬拷贝命令都用变量‘ps-lpr-command’和‘ps-lpr-switches’来指定如何打印输出。‘ps-lpr-command’指定要运行的命令名，‘ps-lpr-switches’指定要使用的命令行选项，‘ps-printer-name’指定打印机。如果你不亲自设置前两个变量，它们会从‘lpr-command’和‘lpr-switches’取初始值。若‘ps-printer-name’为‘nil’，则使用‘printer-name’。

变量‘ps-print-header’控制这些命令是否给每一页添加页眉行——把它设为‘nil’可关闭页眉。

如果你的打印机不支持彩色，就应当把‘ps-print-color-p’设为‘nil’来关闭颜色处理。默认情况下，若显示器支持彩色，Emacs 生成的硬拷贝会带有颜色信息；在黑白打印机上，颜色用深浅不同的灰色模拟。这可能会产生勉强可读甚至无法辨认的输出，即使你屏幕上的颜色只用到灰色深浅也是如此。

另一种办法是把‘ps-print-color-p’设为‘black-white’，以便颜色在黑白打印机上显示得更好。其做法是利用‘ps-black-white-faces’中的信息，通过可定制的灰色深浅列表来表达颜色，并以粗体和斜体字体属性作为补充。

默认情况下，PostScript 打印会忽略字体（face）的背景色，除非变量‘ps-use-face-background’非‘nil’。这是为了避免不必要地干扰斑马条纹以及背景图像/文本。

变量‘ps-paper-type’指定按哪种纸张尺寸排版；合法值包括‘a4’、‘a3’、‘a4small’、‘b4’、‘b5’、‘executive’、‘ledger’、‘legal’、‘letter’、‘letter-small’、‘statement’、‘tabloid’。默认值是‘letter’。你可以通过修改变量‘ps-page-dimensions-database’来定义更多纸张尺寸。

变量‘ps-landscape-mode’指定页面上的打印方向。默认值是‘nil’，表示纵向模式。任何非‘nil’值都表示横向模式。

变量‘ps-number-of-columns’指定栏数；它在横向和纵向模式下都生效。默认值是 1。

变量‘ps-font-family’指定打印普通文本所用的字体系列。合法值包括‘Courier’、‘Helvetica’、‘NewCenturySchlbk’、‘Palatino’和‘Times’。变量‘ps-font-size’指定普通文本的字体大小，默认是 8.5 点。‘ps-font-size’的值也可以是两个浮点数的 cons：一个用于横向模式，另一个用于纵向模式。

Emacs 支持的书写系统和字符比典型的 PostScript 打印机更多。因此，你缓冲区中的某些字符可能无法用打印机内置的字体打印。你可以用 GNU Intlfonts 包中的字体来补充打印机自带的字体，也可以让 Emacs 只使用 Intlfonts。变量‘ps-multibyte-buffer’控制这一点：默认值‘nil’适合打印 ASCII 和 Latin-1 字符；值‘non-latin-printer’用于内置了 ASCII、Latin-1、日文和韩文字形体的打印机。值‘bdf-font’安排对 *所有* 字符都使用 Intlfonts 包中的 BDF 字体。最后，值‘bdf-font-except-latin’指示打印机对 ASCII 和 Latin-1 字符使用内置字体，对其余字符使用 Intlfonts BDF 字体。

为了能使用 BDF 字体，Emacs 需要知道到哪里去找它们。变量 ‘bdf-directory-list’ 保存 Emacs 应当查找字体的目录列表；默认值只包含一个目录 ‘/usr/local/share/emacs/fonts/bdf’。

这些命令还有许多其他定制变量，它们在 Lisp 文件 ‘ps-print.el’ 和 ‘ps-mule.el’ 中定义并说明。

42.3 Printing 包

Emacs 打印硬拷贝的基本功能可以用 Printing 包来扩展。它提供了易于使用的界面，用来选择打印什么、在打印前预览 PostScript 文件，以及设置各种打印选项，例如页眉、横向或纵向模式、双面模式等等。在 GNU/Linux 或 Unix 系统上，Printing 包依赖 ‘gs’ 和 ‘gv’ 工具，它们是作为 GhostScript 程序的一部分发布的。在 MS-Windows 上，可以使用 Ghostscript 的 ‘gstools’ 移植版。

要使用 Printing 包，请把 ‘(require 'printing)’ 加入初始化文件（参见“Emacs 初始化文件”），随后加上 ‘(pr-update-menus)’。该函数把菜单栏中通常的打印命令替换为 ‘Printing’ 子菜单，其中包含各种打印选项。你也可以输入 ‘M-x pr-interface <RET>’；这会创建一个 ‘*Printing Interface*’ 缓冲区，类似定制缓冲区，你可在其中设置打印选项。选好打印什么以及怎样打印之后，用 ‘Print’ 按钮开始打印作业（在其上单击 ‘mouse-2’，或把光标点移到它上面再输入 <RET>）。关于各种选项的更多信息，请使用 ‘Interface Help’ 按钮。

43 文本排序

Emacs 提供若干命令用于对缓冲区中的文本排序。它们全都作用于区域的内容。这些命令把区域内的文本划分成许多“排序记录”（sort record），为每条记录确定一个“排序键”（sort key），然后按排序键所决定的顺序重排这些记录。记录按其键的字母顺序排列；若进行数值排序，则按数值顺序排列。在字母排序中，依照 ASCII 字符序列，‘A’ 到 ‘Z’ 的所有大写字母都排在小写字母 ‘a’ 之前（不过下文所述的 ‘sort-fold-case’ 可以改变这一点）。

各种排序命令的区别在于如何把文本划分成排序记录，以及用每条记录的哪一部分作为排序键。大多数命令把每一行作为一条独立的排序记录，但有些命令把段落或页作为排序记录。大多数排序命令用整条记录作为它自己的排序键，有些则只用记录的一部分作为排序键。

- ‘M-x sort-lines’：把区域分成若干行，按比较每一行的全部文本来排序。数值参数表示按降序排序。
- ‘M-x sort-paragraphs’：把区域分成若干段落，按比较每个段落的全部文本（前导空行除外）来排序。数值参数表示按降序排序。
- ‘M-x sort-pages’：把区域分成若干页，按比较每一页的全部文本（前导空行除外）来排序。数值参数表示按降序排序。
- ‘M-x sort-fields’：把区域分成若干行，按比较每一行中某个字段的内容来排序。字段以空白字符分隔，因此一行中第一段连续的非空白字符构成字段 1，第二段这样的字符构成字段 2，依此类推。

用数值参数指定按哪个字段排序：1 表示按字段 1 排序，依此类推；默认值为 1。负数表示从右往左数字段，因此 -1 表示按最后一个字段排序。若若干行在被排序的字段上内容相同，则它们保持其在原缓冲区中的相对顺序。

- ‘M-x sort-numeric-fields’：类似于 ‘M-x sort-fields’，但会把每一行中指定的字段转换为整数，然后比较这些数字。作为文本看待时 ‘10’ 排在 ‘2’ 之前，而作为数字看待时则排在 ‘2’ 之后。默认情况下，数字按 ‘sort-numeric-base’ 解释，但以 ‘0x’ 或 ‘0’ 开头的数字分别按十六进制和八进制解释。
- ‘M-x sort-columns’：类似于 ‘M-x sort-fields’，但每行中用于比较的文本取自固定的列范围。带前缀参数时按逆序排序。关于此命令的更多细节见下文。
- ‘M-x reverse-region’：反转区域中各行的顺序。这对于按字段降序排序很有用，因为那些排序命令没有实现此功能。

例如，若缓冲区包含以下内容：

```
On systems where clash detection (locking of files being edited) is
implemented, Emacs also checks the first time you modify a buffer
whether the file has changed on disk since it was last visited or
saved.  If it has, you are asked to confirm that you want to change
the buffer.
```

对整个缓冲区执行 ‘M-x sort-lines’，会得到：

```
On systems where clash detection (locking of files being edited) is
implemented, Emacs also checks the first time you modify a buffer
saved.  If it has, you are asked to confirm that you want to change
```

```
the buffer.
whether the file has changed on disk since it was last visited or
```

这里大写 'O' 排在所有小写字母之前。若改用 'C-u 2 M-x sort-fields'，则得到：

```
implemented, Emacs also checks the first time you modify a buffer
saved.  If it has, you are asked to confirm that you want to change
the buffer.
On systems where clash detection (locking of files being edited) is
whether the file has changed on disk since it was last visited or
```

这里的排序键分别是 'Emacs'、'If'、'buffer'、'systems' 和 'the'。

'M-x sort-columns' 需要更多说明。指定列的办法是把光标点放在其中一列上，把标记放在另一列上。由于这意味着你不能把光标点或标记放在待排序文本第一行的开头，该命令使用了非同寻常的"区域"定义：光标点所在的那一整行都算作区域的一部分，标记所在的那一整行以及两者之间的所有行同样如此。

例如，要按第 10 列至第 15 列中的信息对表格排序，你可以把标记放在表格第一行的第 10 列，把光标点放在表格最后一行的第 15 列，然后运行 'sort-columns'。等价地，也可以把标记放在第一行的第 15 列、把光标点放在最后一行的第 10 列，再运行它。

这可以看作是对由光标点和标记所指定的矩形进行排序，只不过矩形左右两侧每一行上的文本会随着矩形内的文本一起移动。参见"矩形"。

若 'sort-fold-case' 为非 'nil'，许多排序命令在比较时会忽略大小写差异。

44 编辑图片

要编辑由文本字符构成的图片（例如，程序中作为注释的、把寄存器划分成字段的示意图），使用命令‘M-x picture-mode’进入图片模式（Picture mode）。

在图片模式中，编辑基于文本的“四分之一平面”（quarter-plane）模型：按照该模型，文本字符散布在一片向右和向下无限延伸的区域上。该模型中不存在行尾的概念；你最多只能说出行上最后一个非空白字符位于何处。

当然，Emacs 实际上始终把文本视为字符序列，行也确实有结尾。但图片模式把最常用的命令替换为模拟四分之一平面模型的变体。它们通过插入空格或把制表符转换为空格来做到这一点。

Emacs 的大多数基本编辑命令被图片模式重新定义，做本质上相同的事，但以四分之一平面的方式做。此外，图片模式定义了以‘C-c’为前缀键的各种键，用于运行特殊的图片编辑命令。

其中有一个键‘C-c C-c’特别重要。图片常常是较大文件的一部分，而该文件通常以其他主模式编辑。图片模式会记录前一个主模式的名称，以便你之后可以用‘C-c C-c’命令（‘picture-mode-exit’）回到该模式。‘C-c C-c’还会删除行尾的空格，除非给定数值参数。

图片模式的特殊命令在其他模式下也能工作（前提是已加载‘picture’库），但除图片模式外没有绑定到键上。下面的描述提到移动“一列”等等，但所有图片模式命令处理数值参数的方式与其普通对应命令相同。

打开图片模式会运行钩子‘picture-mode-hook’。图片模式的更多扩展可在‘artist.el’中找到。

- 图片中的插入：控制自插入字符后光标移动的方向。
- 图片中的制表符：制表位和缩进的各种功能。
- 图片中的矩形：清除和叠加矩形。

44.1 图片模式中的基本编辑

大多数键在图片模式中的作用与平常相同，只是以四分之一平面的方式来做。例如，‘C-f’被重新绑定为运行‘picture-forward-column’，该命令把光标点向右移动一列，必要时插入空格，使得实际的行尾位置无关紧要。‘C-b’被重新绑定为运行‘picture-backward-column’，它总是把光标点向左移动一列，必要时把一个制表符转换为多个空格。‘C-n’和‘C-p’被重新绑定为运行‘picture-move-down’和‘picture-move-up’，它们可以按需插入空格或转换制表符，以确保光标点恰好停留在同一列。‘C-e’运行‘picture-end-of-line’，移动到行上最后一个非空白字符之后。‘C-a’运行‘picture-beginning-of-line’。（屏幕模型的选择不影响行首；该命令唯一额外做的事是把当前图片列更新为 0。）

文本的插入通过使用覆盖模式（Overwrite mode）来适应四分之一平面屏幕模型（参见“次模式”）。自插入字符逐列替换现有文本，而不是把现有文本向右推。`<RET>` 运行‘picture-newline’，它只是移动到下一行的行首，这样新文本将替换该行。

在图片模式中，通常删除或删除（kill）文本的命令改为擦除文本（用空格替换）。`<DEL>`（‘picture-backward-clear-column’）把前一个字符替换为空格而不是删除它；这会使光标点向后移动。‘C-d’（‘picture-clear-

column’）把下一个或下一组字符替换为空格，但不移动光标点。（若想清除字符为空格并向前越过它们，用 <SPC>。）‘C-k’（‘picture-clear-line’）真正删除行的内容，但不从缓冲区中删除换行符。

要做真正的插入，必须使用特殊命令。‘C-o’（‘picture-open-line’）在当前行之后创建空行；它从不拆分行。‘C-M-o’（‘split-line’）在图片模式中有意义，因此未作改动。‘C-j’（‘picture-duplicate-line’）在当前行下面插入另一行相同的内容。

要在图片模式中做真正的删除，用 ‘C-w’、‘C-c C-d’（它被定义为 ‘delete-char’，正如 ‘C-d’ 在其他模式中那样），或某个图片矩形命令（参见“图片模式矩形命令”）。

44.2 控制插入后的移动

由于图片模式中的自插入字符会覆盖文本并移动光标点，光标点应如何移动没有本质限制。通常光标点向右移动，但你可以为自插入字符之后的移动指定八个正交或对角方向中的任意一个。这对在缓冲区中画线很有用。

- ‘C-c <’、‘C-c <LEFT>’：插入后向左移动（‘picture-movement-left’）。
- ‘C-c >’、‘C-c <RIGHT>’：插入后向右移动（‘picture-movement-right’）。
- ‘C-c ^’、‘C-c <UP>’：插入后向上移动（‘picture-movement-up’）。
- ‘C-c .’、‘C-c <DOWN>’：插入后向下移动（‘picture-movement-down’）。
- ‘C-c `’、‘C-c <Home>’：插入后向左上（“西北”）移动（‘picture-movement-nw’）。
- ‘C-c '’、‘C-c <PageUp>’、‘C-c <prior>’：插入后向右上（“东北”）移动（‘picture-movement-ne’）。
- ‘C-c /’、‘C-c <End>’：插入后向左下（“西南”）移动（‘picture-movement-sw’）。
- ‘C-c \’、‘C-c <PageDown>’、‘C-c <next>’：插入后向右下（“东南”）移动（‘picture-movement-se’）。

有两个移动命令基于当前的图片插入方向移动。命令 ‘C-c C-f’（‘picture-motion’）沿当前的插入后移动方向移动，而 ‘C-c C-b’（‘picture-motion-reverse’）沿相反方向移动。

44.3 图片模式中的制表符

图片模式提供两种类似制表符的操作。用 ‘M-<TAB>’（‘picture-tab-search’）做基于上下文的制表。无参数时，它移动到上一个非空行中跟随空白之后的“有趣的”（interesting）字符下方的位置。这里的“下一个”指“出现在比光标点起始位置更靠右的水平位置上”。带前缀参数时，如 ‘C-u M-<TAB>’，该命令移动到当前行中下一个这样的有趣字符。‘M-<TAB>’ 不改变文本；它只移动光标点。“有趣”字符由变量 ‘picture-tab-chars’ 定义，该变量应定义一组字符。该变量的语法类似于正则表达式中 ‘[...]’ 内使用的语法——但不含 ‘[’ 和 ‘]’。其默认值为 ‘"!-~"’。

<TAB> 本身运行 ‘picture-tab’，它基于当前的制表位设置工作；它是 ‘tab-to-tab-stop’ 在图片模式中的等价命令。通常它只移动光标点，但带数值参数时它会清除其越过的文本。

基于上下文和基于制表位的两种制表方式由命令 ‘C-c <TAB>’（‘picture-set-tab-stops’）结合起来。该命令把制表位设置为 ‘M-<TAB>’ 在当前行中认为重要的位置。把该命令与 <TAB> 一起使用，可以达到基于上下文制表的效果。但在 ‘M-<TAB>’ 就足够的情况下，它更方便。

在图片中阻止使用真正的制表符可能很方便。例如，这可以防止 'C-x <TAB>' 弄乱图片。你可以把变量 'indent-tabs-mode' 设为 'nil' 来做到这一点。

44.4 图片模式矩形命令

图片模式定义了以契合四分之一平面模型的方式处理矩形文本块的命令。标准矩形命令也可能有用。参见"矩形"。

- 'C-c C-k'：用空格清除区域矩形（'picture-clear-rectangle'）。带前缀参数时，删除文本。
- 'C-c C-w R'：类似，但先把矩形内容保存到寄存器 R 中（'picture-clear-rectangle-to-register'）。参见"寄存器"。
- 'C-c C-y'：把最近删除的矩形以覆盖方式复制到缓冲区中，左上角位于光标点（'picture-yank-rectangle'）。带参数时改为插入。
- 'C-c C-x R'：类似，但使用寄存器 R 中的矩形（'picture-yank-rectangle-from-register'）。

图片矩形命令 'C-c C-k'（'picture-clear-rectangle'）和 'C-c C-w'（'picture-clear-rectangle-to-register'）与标准矩形命令的区别在于，它们通常清除矩形而不是删除它；这与 'C-d' 在图片模式中被改变的方式类似。

然而，删除矩形在图片模式中可能有用，所以这些命令在给定数值参数时会删除矩形。'C-c C-k' 无论带不带数值参数都会保存矩形供 'C-c C-y' 使用。

图片模式中粘贴矩形的命令与标准命令的区别在于它们覆盖而不是插入。这与图片模式插入其他文本不同于其他模式的方式相同。'C-c C-y'（'picture-yank-rectangle'）插入（通过覆盖）最近被删除的矩形，而 'C-c C-x'（'picture-yank-rectangle-from-register'）对指定寄存器中找到的矩形做同样的事。

45 编辑二进制文件

编辑二进制文件有一种特殊的主模式：Hexl 模式。要使用它，请用 ‘M-x hexl-find-file’ 而不是 ‘C-x C-f’ 来访问文件。该命令把文件内容转换成十六进制，并让你编辑转换后的文本。保存文件时，它会自动转换回二进制。

也可以用 ‘M-x hexl-mode’ 把已有的缓冲区转换成十六进制。如果你按常规方式访问一个文件，后来发现它是二进制文件，这个命令就很有用。

在 Hexl 模式中，插入文本总是覆盖原有内容。这样做是为了降低意外破坏文件中数据对齐的风险。普通文本字符会插入自身（即以自身覆盖）。还有一些命令可以按字符编码插入特殊字符。大多数光标移动键以及 ‘C-x C-s’，在 Hexl 模式中都绑定到效果相同的命令。下面是 Hexl 模式特有的其他重要命令列表：

‘C-M-d’ 按十进制输入的编码插入一个字节。

‘C-M-o’ 按八进制输入的编码插入一个字节。

‘C-M-x’ 按十六进制输入的编码插入一个字节。

‘C-M-a’ 移到 512 字节页的开头。

‘C-M-e’ 移到 512 字节页的末尾。

‘C-x [’ 移到 1k 字节页的开头。

‘C-x]’ 移到 1k 字节页的末尾。

‘M-g’ 移到以十六进制指定的地址。

‘M-j’ 移到以十进制指定的地址。

‘C-c C-c’ 离开 Hexl 模式，回到调用 ‘hexl-mode’ 之前该缓冲区所用的主模式。

其他 Hexl 命令可以插入由二进制字节组成的字符串（序列）、按 ‘short’ 或 ‘int’ 移动等；详情请键入 ‘C-h a hexl- <TAB>’。

Hexl 模式也可用于编辑文本文件。如果文本文件包含不寻常的字符或使用不寻常的编码（参见“编码系统”），这会很方便。为此，插入字节的 Hexl 命令也能插入 ASCII 和非 ASCII 字符，包括多字节字符。要用 Hexl 编辑文本文件，照常访问该文件，然后键入 ‘M-x hexl-mode <RET>’ 切换到 Hexl 模式。现在你可以通过键入来插入文本字符。不过，插入多字节字符需要特别小心，以免产生无效的多字节序列：你应当在光标点位于文件中某个多字节序列的第一个字节时开始键入这类字符。

46 保存 Emacs 会话

可以使用 desktop 库把 Emacs 的状态从一个会话保存到另一个会话。保存的 Emacs“桌面配置”（desktop configuration）包括缓冲区、它们的文件名、主模式、缓冲区位置、窗口和框架配置，以及一些重要的全局变量。

要启用此特性，可使用定制缓冲区（参见“简易自定义界面”）为以后的会话把 ‘desktop-save-mode’ 设为 ‘t’，或者在初始化文件中加入下面这一行（参见“Emacs 初始化文件”）：

```
(desktop-save-mode 1)
```

如果在初始化文件中开启 ‘desktop-save-mode’，那么 Emacs 启动时会在 ‘desktop-path’（默认先是 ‘user-emacs-directory’，然后是主目录）中查找已保存的桌面，并使用找到的第一个桌面。当 Emacs 在开启 ‘desktop-save-mode’ 的情况下运行时，只要桌面配置有任何改动，它默认就会自动保存桌面。变量 ‘desktop-auto-save-timeout’ 决定 Emacs 多久检查一次桌面是否被修改。退出 Emacs 时也会保存桌面。

如果不想让 Emacs 重新载入任何已保存的桌面配置，可在 Emacs 命令行上指定选项 ‘--no-desktop’。这会为当前会话关闭 ‘desktop-save-mode’。用 ‘--no-init-file’ 选项启动 Emacs 也会禁用桌面重载，因为它绕过了初始化文件，而 ‘desktop-save-mode’ 通常正是在初始化文件中开启的。

你可以在不同目录中保存各自独立的桌面配置；从某个存有已保存桌面配置的目录启动 Emacs 时就会恢复该配置，前提是你把 ‘desktop-path’ 定制为在其列出的其他目录之前加上 ‘.’（当前目录）。你可以键入 ‘M-x desktop-change-dir’ 保存当前桌面并重新载入另一个目录中保存的桌面。键入 ‘M-x desktop-revert’ 则回到先前重新载入的桌面。

Emacs 保存桌面的文件在会话运行期间会被锁定，以免被另一个 Emacs 会话无意中覆盖。该锁通常在 Emacs 退出时解除，但如果 Emacs 或系统崩溃，锁会保留下来，重新启动 Emacs 时默认会询问你是否使用被锁定的桌面文件。你可以把变量 ‘desktop-load-locked-desktop’ 定制为 ‘nil’（表示这种情况下绝不加载桌面）或 ‘t’（表示不加询问就加载桌面），从而避免该询问。还可以把该变量定制为特殊值 ‘check-pid’，表示如果锁定桌面的 Emacs 进程没有运行在本机上，就加载该文件。在锁定桌面的 Emacs 仍可能在另一台机器上运行的情形下，不应使用该值；例如用 NFS 或类似方式远程挂载主目录的多用户环境中就可能是这种情况。

当 Emacs 以守护进程模式（daemon mode）启动时，它无法向你提任何问题，因此若发现桌面文件被锁定，就不会加载它，除非 ‘desktop-load-locked-desktop’ 为 ‘t’。注意，由于其他一些原因，在守护进程模式下恢复桌面也有几分麻烦：例如守护进程无法使用 GUI 特性，所以框架位置、大小和装饰等参数无法恢复。为此，你可以把在守护进程模式下恢复桌面推迟到第一个客户端连接时，方法是在加入 ‘server-after-make-frame-hook’ 的钩子函数中调用 ‘desktop-read’（见下文）（参见《Emacs Lisp 参考手册》的“Creating Frames”）。

任何时候都可以用命令 ‘M-x desktop-save’ 强制立即保存当前桌面。如果你不想使用自动的桌面恢复、因而没有开启 ‘desktop-save-mode’，或者你对桌面做了重大改动、想确保即使 Emacs 或系统崩溃配置也不会丢失，这都很有用。如果当前 Emacs 会话尚未加载任何桌面，你可以用 ‘M-x desktop-read’ 恢复先前保存的桌面。

默认情况下，桌面会尝试保存和恢复框架与窗口配置。（1）要禁用或启用这一点，可分别把 ‘desktop-restore-frames’ 设为 ‘nil’ 或任意非 ‘nil’ 值。（关于可用来微调此行为的若干相关选项，见该变量的文档。）

当桌面恢复框架和窗口配置时，它使用记录的框架参数值，而忽略你在初始化文件中为这些参数所做的任何设置（参见“Emacs 初始化文件”）。这意味着恢复后的框架的字体和字体（face）等框架参数将来自桌面文件，也就是你退出上一个 Emacs 会话时保存的值；你在初始化文件中为这些参数所做的任何设置都会被忽略。要禁用这一点，可定制 ‘frameset-filter-alist’ 的值，过滤掉你不想恢复的框架参数；它们随后将按你在初始化文件中的定制来设置。

默认情况下不保存访问远程文件的缓冲区的信息。定制变量 ‘desktop-files-not-to-save’ 可以改变这一点。这种情况下，你也许还应考虑定制 ‘remote-file-name-access-timeout’，它表示过多少秒后停止恢复远程文件的缓冲区。这可以防止 Emacs 在恢复访问过远程文件的会话时被阻塞。

默认情况下，桌面中的所有缓冲区会一次性全部恢复。但如果桌面中的缓冲区很多，这可能很慢。你可以用变量 ‘desktop-restore-eager’ 指定立即恢复的缓冲区最大数目；其余缓冲区则在 Emacs 空闲时惰性恢复。

键入 ‘M-x desktop-clear’ 可清空 Emacs 桌面；例如，当你随后想通过调用 ‘M-x desktop-read’ 切换到另一个桌面时，这很有用。‘desktop-clear’ 命令会删除除内部缓冲区之外的所有缓冲区，并清除 ‘desktop-globals-to-clear’ 中列出的全局变量。如果想保留某些缓冲区，可定制变量 ‘desktop-clear-preserve-buffers’，其值是一个正则表达式列表，匹配不应删除的缓冲区名。

如果想从一个会话保存迷你缓冲区历史到另一个会话，请使用 ‘savehist’ 库。你也可以把选定的迷你缓冲区历史变量加入 ‘desktop-globals-to-save’ 的值，从而作为 ‘desktop-save-mode’ 的一部分保存它们。

> 注：Android 除外，在该平台上此选项默认为 ‘nil’，因为窗口管理器（参见“Android 窗口系统”）限制过于严格，不允许恢复框架配置。

47 递归编辑层级

“递归编辑”（recursive edit）是这样一种情形：你正处在另一个 Emacs 命令的执行过程中，却使用 Emacs 命令进行任意的编辑。例如，当你在‘query-replace’中键入‘C-r’时，就进入了一个递归编辑，在其中你可以修改当前缓冲区。退出递归编辑后，你会回到‘query-replace’。参见“查询替换”。

“退出”递归编辑意味着返回那个尚未完成的命令，让它继续执行。用于退出的命令是‘C-M-c’（‘exit-recursive-edit’）。

你也可以“中止”递归编辑。这类似于退出，但会立即放弃那个尚未完成的命令。使用命令‘C-]’（‘abort-recursive-edit’）即可做到。参见“退出与中止”。

当你处于递归编辑中时，模式行会在始终环绕主模式和次模式名称的括号外面显示一对方括号，以此提示你。每个窗口的模式行都以同样的方式显示这一点，因为处于递归编辑中是整个 Emacs 的状态，而不是某个特定窗口或缓冲区的状态。

递归编辑之中还可以再有递归编辑。例如，在‘query-replace’中键入‘C-r’之后，你可能会键入某个进入调试器的命令。这就在‘C-r’的递归编辑层级之内，为调试器开启了一个递归编辑层级。模式行会为当前正在进行的每一个递归编辑层级显示一对方括号。

退出内层的递归编辑（例如用调试器的‘c’命令）会恢复上一层中正在运行的命令。当那个命令结束时，你就可以用‘C-M-c’再退出一个递归编辑层级，依此类推。退出只作用于最内层。中止也只退出一层递归编辑；它会立即回到上一个递归编辑所处的命令层级。如有需要，你可以再中止下一层递归编辑。

另一种做法是，命令‘M-x top-level’会中止所有层级的递归编辑，立即返回顶层的命令读取循环。如果迷你缓冲区正处于活动状态，它还会退出迷你缓冲区。

在递归编辑中编辑的文本不必与你在顶层编辑的文本相同。这取决于递归编辑的用途。如果调用递归编辑的命令先选中了另一个缓冲区，那么你要递归编辑的就是那个缓冲区。无论如何，你都可以在递归编辑中按常规方式切换缓冲区（只要切换缓冲区的按键没有被重新绑定）。你大概可以在递归编辑中完成其余的全部编辑工作，包括访问文件等等。但这有时会带来意想不到的后果（例如栈溢出）。所以要记得，在不再需要时退出或中止递归编辑。

递归编辑通常会启用单键盘模式。参见“多个 X 显示”。

总的说来，我们尽量少用递归编辑层级。这是因为它们迫使你按特定顺序返回——从最内层向顶层。只要可能，我们就把不同的活动放在彼此独立的缓冲区中，好让你随意在它们之间切换。有些命令会切换到新的主模式，而该模式提供了一个切换回去的命令。这些做法让你能更灵活地按自己选定的顺序回到尚未完成的任务。

48 超链接与网页浏览

以下小节介绍处理 Emacs 缓冲区文本中出现的 URL 及其他类型链接的便利功能。

48.1 使用 EWW 浏览网页

“EWW”（Emacs Web Wowser）是 Emacs 的 Web 浏览器包。它允许在 Emacs 缓冲区中浏览 URL。命令 ‘M-x eww’ 可打开 URL 或在 Web 上搜索。也可以用命令 ‘M-x eww-open-file’ 打开文件。你可以把 EWW 用作 ‘browse-url’ 的 Web 浏览器，参见“跟随 URL”。完整细节见 参见 《《eww》手册》。

48.2 内嵌的 WebKit 部件

如果 Emacs 在编译时带上了相应的支持包，它就能在缓冲区中显示浏览器部件。命令 ‘M-x xwidget-webkit-browse-url’ 会询问要显示的 URL，并在浏览器部件中打开它。该 URL 默认取光标点处或其之前位置的 URL；但如果存在活动的区域（参见“标记与区域”），默认 URL 则改取该区域的内容，并从中删除所有空白字符。随后该命令会创建一个新的缓冲区，其中内嵌的浏览器显示指定的 URL。该缓冲区进入 Xwidget-WebKit 模式（类似于 Image 模式，参见“查看图像文件”），该模式提供了一键命令来滚动部件、改变其大小以及重新加载它。在该缓冲区中键入 ‘C-h b’ 可查看键绑定。

默认情况下，在 xwidget webkit 缓冲区中键入自插入字符不会做任何事，或者会触发某种特殊操作。要让这些字符和其他常用编辑键在被按下时插入自身，可以启用 ‘xwidget-webkit-edit-mode’，它把这些键重新定义为直接传给 WebKit xwidget。

你也可以在 xwidget webkit 缓冲区中键入 ‘e’ 来启用 ‘xwidget-webkit-edit-mode’。

‘xwidget-webkit-isearch-mode’ 是一个次模式，其行为类似于增量搜索（参见“增量搜索”），但作用对象是 WebKit 部件的内容而非当前缓冲区。它在 xwidget-webkit 缓冲区中绑定到 ‘C-s’ 和 ‘C-r’。当它由 ‘C-r’ 调用时，初始的搜索会在反方向上执行。

键入任何自插入字符都会使该字符被插入到当前的搜索查询中。键入 ‘C-s’ 会使 WebKit 部件显示下一个搜索结果，而键入 ‘C-r’ 则使其显示上一个结果。

要退出增量搜索，可以键入 ‘C-g’。

命令 ‘xwidget-webkit-browse-history’ 显示一个缓冲区，其中列出当前 WebKit 缓冲区此前加载过的页面，你可以按 ‘RET’ 导航到这些页面。

它绑定到 ‘H’。

WebKit 缓冲区中默认启用 JavaScript，这可能并不理想，因为网站常用它来跟踪你的在线活动。你可以把变量 ‘xwidget-webkit-disable-javascript’ 定制为非 ‘nil’ 值，从而在 WebKit 缓冲区中禁用 JavaScript。改动该设置后，必须杀死所有 WebKit 缓冲区，设置才会生效。

48.3 跟随 URL

‘M-x browse-url <RET> URL <RET>’ 在 Web 浏览器中加载一个 URL。

Browse-URL 包让你可以在 Emacs 内轻松跟随 URL。大多数 URL 通过调用 Web 浏览器来跟随；‘mailto:’ URL 则通过调用 Emacs 命令 ‘compose-mail’ 来跟随，以便向指定地址发送邮件（参见“发送邮件”）。

命令 ‘M-x browse-url’ 会询问一个 URL，并跟随它。如果光标点位于某个看似 URL 的内容附近，该 URL 会作为默认值提供。Browse-URL 包还提供其他命令，你可能愿意把它们绑定到按键上，例如 ‘browse-url-at-point’ 和 ‘browse-url-at-mouse’。

你可以通过 ‘browse-url’ 定制组中的各种选项来定制 Browse-URL 的行为。特别地，选项 ‘browse-url-mailto-function’ 让你定义如何跟随 ‘mailto:’ URL，而 ‘browse-url-browser-function’ 指定你的默认浏览器。

你可以定制 ‘browse-url-handlers’，从而定义某些 URL 用其他函数来浏览；它是一个 alist，把正则表达式或谓词与用于浏览匹配 URL 的函数配对。

要了解更多信息，可键入 ‘C-h P browse-url <RET>’ 查看该包的注释。

Emacs 还有一个次模式，对把 URL 当作文件来处理提供了一定的支持。‘url-handler-mode’ 是一个全局次模式，影响大多数处理文件名的 Emacs 命令和原语。打开该模式后，例如，你可以键入 ‘C-x C-f https://www.gnu.org/ RET’ 查看该网页的 HTML，随后还可以编辑它并将其保存到本地文件。

48.4 激活 URL

‘M-x goto-address-mode’ 在当前缓冲区中激活 URL 和电子邮件地址。

‘M-x global-goto-address-mode’ 在所有缓冲区中激活 ‘goto-address-mode’。

键入 ‘M-x goto-address-mode’，可以让 Emacs 在当前缓冲区中把 URL 特别标出。启用这个缓冲区局部的次模式后，它会找出缓冲区中的所有 URL，高亮它们，并把它们变成可点击的按钮。当光标点位于某个 URL 的文本上时，你可以键入 ‘C-c <RET>’（‘goto-address-at-point’）来跟随该 URL；也可以用 ‘mouse-2’ 点击，或快速点击 ‘mouse-1’（参见“用鼠标跟随引用”）。跟随 URL 是通过把 ‘browse-url’ 作为子程序调用来完成的（参见“跟随 URL”）。

把 ‘goto-address-mode’ 加入各模式钩子，以及用于显示收到消息的钩子（例如 Rmail 的 ‘rmail-show-message-hook’），可能很有用。Gnus 或 MH-E 不需要这样做，它们各自有类似的功能。

48.5 在光标点处查找文件和 URL

FFAP 包替换了某些用于查找文件的键绑定，例如 ‘C-x C-f’，代之以提供更合理默认值的命令。若给定前缀参数，这些命令的行为与普通命令相同。否则，它们从光标点周围的文本中取得默认文件名或 URL。如果缓冲区中找到的内容形式上是 URL 而不是文件名，这些命令就用 ‘browse-url’ 来查看它（参见“跟随 URL”）。

该功能对于跟随邮件或新闻缓冲区、‘README’ 文件、‘MANIFEST’ 文件等之中的引用很有用。要了解更多信息，可键入 ‘C-h P ffap <RET>’ 查看该包的注释。

要启用 FFAP，可键入 ‘M-x ffap-bindings’。这会建立下列键绑定，并为 Rmail、Gnus 和 VM 的文章缓冲区安装钩子以提供额外的 FFAP 功能。

- ‘C-x C-f FILENAME <RET>’ 查找 FILENAME，从光标点周围的文本猜测默认值（‘find-file-at-point’）。
- ‘C-x C-r FILENAME <RET>’ ‘ffap-read-only’，相当于 ‘find-file-read-only’。

- ‘C-x C-v FILENAME <RET>’ ‘ffap-alternate-file’，相当于 ‘find-alternate-file’。
- ‘C-x d DIRECTORY <RET>’ 在 DIRECTORY 上启动 Dired，默认取光标点处的目录（‘dired-at-point’）。
- ‘C-x C-d DIRECTORY <RET>’ ‘ffap-list-directory’，相当于 ‘list-directory’。
- ‘C-x 4 f FILENAME <RET>’ ‘ffap-other-window’，相当于 ‘find-file-other-window’。
- ‘C-x 4 r FILENAME <RET>’ ‘ffap-read-only-other-window’，相当于 ‘find-file-read-only-other-window’。
- ‘C-x 4 d DIRECTORY <RET>’ ‘ffap-dired-other-window’，类似 ‘dired-other-window’。
- ‘C-x 5 f FILENAME <RET>’ ‘ffap-other-frame’，相当于 ‘find-file-other-frame’。
- ‘C-x 5 r FILENAME <RET>’ ‘ffap-read-only-other-frame’，相当于 ‘find-file-read-only-other-frame’。
- ‘C-x 5 d DIRECTORY <RET>’ ‘ffap-dired-other-frame’，相当于 ‘dired-other-frame’。
- ‘C-x t C-f FILENAME <return>’ ‘ffap-other-tab’，相当于 ‘find-file-other-tab’。
- ‘C-x t C-r FILENAME <return>’ ‘ffap-read-only-other-tab’，相当于 ‘find-file-read-only-other-tab’。
- ‘M-x ffap-next’ 在缓冲区中搜索下一个文件名或 URL，然后查找该文件或 URL。
- ‘S-mouse-3’ ‘ffap-at-mouse’，根据鼠标点击位置周围的文本猜测文件并查找它。
- ‘C-S-mouse-3’ 显示当前缓冲区中提到的文件和 URL 的菜单，然后查找你所选中的那个（‘ffap-menu’）。

49 游戏与娱乐

‘animate’ 包能让文字跳舞（例如 ‘M-x animate-birthday-present’）。

‘M-x blackbox’、‘M-x mpuz’ 和 ‘M-x 5x5’ 都是益智游戏。‘blackbox’ 要求你通过断层扫描推断出盒子内物体的位置。‘mpuz’ 显示一道用字母代替数字的乘法谜题，其中的代码需要你来猜——要猜某个字母的取值，先输入该字母，再输入你认为它所代表的数字。‘5x5’ 的目标是填满所有方格。

‘M-x bubbles’ 是一个游戏，目标是用尽可能少的步数消除尽可能多的泡泡。

‘M-x decipher’ 帮助你破译一个用简单单表替换密码加密的缓冲区。

‘M-x dissociated-press’ 把当前 Emacs 缓冲区中的文本逐词或逐字符打乱，并把输出写入名为 ‘*Dissociation*’ 的缓冲区。正的前缀参数表示逐字符处理，并指定重叠字符的数目；负的前缀参数表示逐词处理，并指定重叠词的数目。分离新闻（Dissociated Press）产生的结果与马尔可夫链颇为相似，但它是一项独立的、原创的发明（ignoriginal）；它在随机跳跃之间会连续复制样本文本中的若干字符，而马尔可夫链则在每个词或字符之后随机跳跃。若你希望文档被人顺畅地阅读（userenced）、措辞得体（properbose），就不要把“分离词”（dissociwords）写进文档。

‘M-x dunnet’ 运行一个基于文本的冒险游戏。

如果你想多些亲身参与感，可以试试 ‘M-x gomoku’，它会和你下五子棋。

如果你有点无聊，可以试试 ‘M-x hanoi’。如果你相当无聊，就给它一个数字前缀参数。如果你非常非常无聊，就试试参数 9。然后坐下来观看。

‘M-x life’ 运行康威的生命游戏（Conway's Game of Life）元胞自动机。

‘M-x morse-region’ 把区域中的文本转换成摩尔斯电码；‘M-x unmorse-region’ 再把它转换回来。‘M-x nato-region’ 把区域中的文本转换成北约音标字母；‘M-x denato-region’ 再把它转换回来。

‘M-x pong’、‘M-x snake’ 和 ‘M-x tetris’ 是众所周知的 Pong、贪吃蛇和俄罗斯方块游戏的实现。

‘M-x solitaire’ 让你玩一种跳棋式的单人纸牌游戏，你要让棋子跳过其他棋子。

命令 ‘M-x zone’ 会在 Emacs 空闲时拿显示画面玩游戏。

“真正的程序员”会使用 ‘M-x butterfly’，它用蝴蝶来翻转驱动器盘片上的一个二进制位，参见 <https://xkcd.com/378>。

最后，如果你感到沮丧，可以试着向著名的心理治疗师 Eliza 倾诉你的问题。只需执行 ‘M-x doctor’，并在每次输入结束时按两次 <RET>。

50 Emacs Lisp 软件包

Emacs 的扩展方式是在“包”（package）中实现额外的功能，而包就是 Emacs Lisp 库。这些包可以由你自己编写，也可以由别人提供。若想安装这样的包，使之在你以后的 Emacs 会话中可用，就需要编译它，并把它放到 Emacs 查找 Lisp 库的某个目录中。关于这种手动安装方法的更多细节，参见“Emacs 的 Lisp 代码库”。许多包会在 Lisp 文件开头附近的大段注释中给出安装和使用说明；你可以按照这些说明安装该包，并精细调整对它的使用。

包也可以由“包归档”（package archive）提供，包归档是 Emacs Lisp 包的大型集合。每个包都是一个独立的 Emacs Lisp 程序，有时还包含其他组成部分，例如 Info 手册(1)。Emacs 提供了一套设施，让你可以轻松地从这类归档中下载并安装包。本章其余部分将介绍这套设施。

要列出可从包归档中安装的包，输入 ‘M-x list-packages <RET>’。它会打开一个名为 ‘*Packages*’ 的缓冲区，其中列出所有包。你可以通过这个缓冲区安装或卸载包。参见“包菜单缓冲区”。

命令 ‘C-h P’（‘describe-package’）会提示输入一个包的名称，并显示一个帮助缓冲区，描述该包的属性以及它实现的功能。

默认情况下，Emacs 从两个归档下载包：GNU ELPA（https://elpa.gnu.org/）和 NonGNU ELPA（https://elpa.nongnu.org/）。这两个归档由 Emacs 开发者维护，并由 GNU 工程托管。你也可以选择从第三方归档下载包。参见“包的安装”。

关于如何把 Emacs Lisp 程序变成可安装的包，参见《Emacs Lisp 参考手册》的“Packaging”。

> 注：若你的系统设置了环境变量 ‘INFOPATH’ 而末尾没有路径分隔符，那么它会遮蔽 Emacs 在启动时注册的手册，参见《《texinfo》手册》的“Other Info Directories”。

50.1 包菜单缓冲区

命令 ‘M-x list-packages’ 会打开“包菜单”。这是一个列出 Emacs 所知全部包的缓冲区，每行一个包，并带有以下信息：

- 包名（例如 ‘auctex’）。
- 包的版本号（例如 ‘11.86’）。
- 包的状态——通常为 ‘available’（可从包归档下载）、‘installed’ 或 ‘built-in’（Emacs 默认自带的）之一。参见“包状态”。
- 若启用了多个包归档，则该包来自哪个包归档。
- 包的简短描述。

‘list-packages’ 命令要访问网络，从包归档服务器获取可安装包的列表。若网络不可用，它就退回到最近一次获取的列表。

在包列表缓冲区中最常用的命令是‘x’命令。若光标点下的包尚未安装，该命令会安装它。若光标点下的包已经安装，该命令会删除它。

包菜单中可以使用以下命令：

- ‘h’打印一条简短消息，概述如何使用包菜单（‘package-menu-quick-help’）。
- ‘?’‘<RET>’显示当前行上包的帮助缓冲区（‘package-menu-describe-package’），与‘C-h P’命令显示的帮助窗口类似（参见“Emacs Lisp 软件包”）。
- ‘i’把当前行上的包标记为待安装（‘package-menu-mark-install’）。若该包的状态为‘available’，这会在行首添加一个字符‘I’；随后输入‘x’（见下文）就会下载并安装该包。
- ‘d’把当前行上的包标记为待删除（‘package-menu-mark-delete’）。若该包的状态为‘installed’，这会在行首添加一个字符‘D’；随后输入‘x’（见下文）就会删除该包。关于删除包具体涉及哪些内容，参见“包文件与目录布局”。
- ‘w’在浏览器中打开当前行上包的网站（‘package-browse-url’）。打开浏览器使用的是‘browse-url’。
- ‘~’把所有过时的包标记为待删除（‘package-menu-mark-obsolete-for-deletion’）。这会把所有状态为‘obsolete’的包标记为待删除。
- ‘u’‘<DEL>’去除之前由‘i’或‘d’命令加在当前行上的安装标记或删除标记（‘package-menu-mark-unmark’）。
- ‘U’把所有有更新版本可用的包标记为待升级（‘package-menu-mark-upgrades’）。这会在新的可用版本上放置安装标记，并在旧的已安装版本（状态标记为‘obsolete’）上放置删除标记。默认情况下，这不会标记那些有更新版本可用的内置包，但定制‘package-install-upgrade-built-in’可以改变这一点。参见“包的安装”。若你把‘package-install-upgrade-built-in’定制为非‘nil’值，务必逐一检查‘U’命令标记的所有内置包，以免更新你并不想覆盖的内置包。
- ‘x’下载并安装所有用‘i’标记的包及其依赖；同时删除所有用‘d’标记的包（‘package-menu-execute’）。这也会移除这些标记。若没有任何包被标记，该命令会安装光标点下的包（若它尚未安装），或者删除光标点下的包（若它已经安装）。
- ‘g’‘r’刷新包列表（‘revert-buffer’）。这会再次从包归档获取可安装包的列表，并重新显示包列表。
- ‘H’隐藏名称匹配某个正则表达式的包（‘package-menu-hide-package’）。这会提示输入一个正则表达式，然后隐藏名称匹配的那些包。该正则表达式的默认值只会隐藏名称位于光标点处的那个包，所以只需在提示处按 <RET> 就会隐藏当前的包。
- ‘(’切换旧版本包以及来自较低优先级归档的版本的可见性（‘package-menu-toggle-hiding’）。
- ‘/ a’按归档过滤包列表（‘package-menu-filter-by-archive’）。这会提示输入一个包归档（例如‘gnu’），然后只显示来自该归档的包。你可以用逗号分隔多个归档的名称来指定它们。
- ‘/ d’按描述过滤包列表（‘package-menu-filter-by-description’）。这会提示输入一个正则表达式，然后只显示描述匹配该正则表达式的包。

- ‘/ k’ 按关键字过滤包列表（‘package-menu-filter-by-keyword’）。这会提示输入一个关键字（例如‘games’），然后只显示带有该关键字的包。你可以用逗号分隔多个关键字来指定它们。
- ‘/ N’ 按名称或描述过滤包列表（‘package-menu-filter-by-name-or-description’）。这会提示输入一个正则表达式，然后只显示名称或描述匹配该正则表达式的包。
- ‘/ n’ 按名称过滤包列表（‘package-menu-filter-by-name’）。这会提示输入一个正则表达式，然后只显示名称匹配该正则表达式的包。
- ‘/ s’ 按状态过滤包列表（‘package-menu-filter-by-status’）。这会提示输入一个或多个状态（例如‘available’，参见“包状态”），然后只显示状态匹配的包。你可以用逗号分隔多个状态值来指定它们。
- ‘/ v’ 按版本过滤包列表（‘package-menu-filter-by-version’）。这会先提示输入比较符号‘<’、‘>’或‘=’之一以及一个版本字符串，然后显示版本相应低于、等于或高于你输入版本的包。
- ‘/ m’ 按非空标记过滤包列表（‘package-menu-filter-marked’）。这只显示已被标记为待安装或待删除的包。
- ‘/ u’ 过滤包列表，只显示有可用升级的包（‘package-menu-filter-upgradable’）。默认情况下，该过滤器会排除那些有更新版本可用的内置包，但定制‘package-install-upgrade-built-in’可以改变这一点。参见“包的安装”。
- ‘/ /’ 清除当前应用于包列表的过滤器（‘package-menu-filter-clear’）。

例如，你可以在列出某个包的那一行上输入‘i’，再输入‘x’，从而安装该包。

50.2 包状态

包可以具有以下状态之一：

- ‘available’：包尚未安装，但可以从包归档下载并安装。
- ‘avail-obso’：包可供安装，但同时还有更新的版本可用。默认情况下会隐藏具有该状态的包。
- ‘built-in’：包由 Emacs 默认自带。它无法通过包菜单删除，默认情况下也不会被考虑升级（但你可以通过定制‘package-install-upgrade-built-in’改变这一点，参见“包的安装”）。
- ‘dependency’：该包是为了满足另一个包的依赖而自动安装的。
- ‘disabled’：该包已通过变量‘package-load-list’禁用。
- ‘external’：该包既不是内置的，也不是来自‘package-user-dir’所指定的目录（参见“包文件与目录布局”）。外部包的处理方式与‘built-in’包大体相同，且无法删除。
- ‘held’：该包被保持（held），参见“包的安装”。
- ‘incompat’：由于某种原因无法安装该包，例如它依赖于无法安装的包。
- ‘installed’：包已安装。
- ‘new’：等同于‘available’，只是该包是在你上次调用‘M-x list-packages’之后才被加入包归档的。

- ‘obsolete’：该包是已安装的过时版本；除了这个版本之外，该包还安装了更新的版本。

50.3 包的安装

最方便的安装包的方式是使用包菜单（参见“包菜单缓冲区”），但你也可以使用命令 ‘M-x package-install’。它会提示输入一个状态为 ‘available’ 的包的名称，然后下载并安装它。类似地，若想升级某个包，可以使用 ‘M-x package-upgrade’ 命令；若想升级所有包，可以使用 ‘M-x package-upgrade-all’ 命令。默认情况下，这些命令会删除旧的安装，但你可以通过定制变量 ‘package-retention-policy’ 来调整这一行为。

注意，‘package-install’ 不会建议安装已经安装的包。类似地，‘package-upgrade’ 只会建议升级那些有更新版本可用的包。

默认情况下，‘package-install’ 不会考虑那些归档中有新版本可用的内置包。（若某个包包含在 Emacs 发行版中，它就是内置的。）特别是，当你在它的提示处输入时，它不会把内置包显示在补全候选列表中。但若你带前缀参数调用 ‘package-install’，它也会考虑可以升级的内置包。你可以通过定制变量 ‘package-install-upgrade-built-in’ 使这一行为成为默认行为：若其值为非 ‘nil’，则即使不带前缀参数调用，‘package-install’ 也会考虑内置包。注意，包菜单的命令（参见“包菜单缓冲区”）也受 ‘package-install-upgrade-built-in’ 影响。

与此相反，‘package-upgrade’ 和 ‘package-upgrade-all’ 从不考虑内置包。若想用这些命令升级某些内置包，你需要先把这些包逐个升级一次，方式可以是使用 ‘C-u M-x package-install <RET>’，也可以是把 ‘package-install-upgrade-built-in’ 定制为非 ‘nil’ 值，然后通过包菜单或 ‘package-install’ 把该包升级一次。

若你把 ‘package-install-upgrade-built-in’ 定制为非 ‘nil’ 值，那么在使用一次更新许多包的命令（例如 ‘package-upgrade-all’ 以及包菜单中的 ‘U’）时要格外小心：这些命令可能会用归档中的新版本覆盖你并不想替换的内置包。若你只想更新少量内置包，就不要使用这类批量命令。

一个包可能“要求”（require）安装某些其他包，因为它依赖这些包所提供的功能。当 Emacs 安装这样的包时，它也会自动下载并安装所有尚未安装的必需包。（若某个必需的包由于某种原因不可用，Emacs 会发出错误信号并停止安装。）包的需求列表显示在它的帮助缓冲区中。

若你在从包归档安装和升级包方面比较谨慎，可以配置 ‘package-review-policy’，让自己有机会在安装前审查这些包。把该用户选项设为 ‘t’ 之后，你就能审查所有包（包括依赖包），在此期间你可以浏览源代码、查看已下载包与先前安装之间的差异（diff），或者阅读变更日志。你也可以把 ‘package-review-policy’ 配置成有选择地信任或不信任特定的包或归档。

默认情况下，Emacs 从两个归档下载包：GNU ELPA（https://elpa.gnu.org/）和 NonGNU ELPA（https://elpa.nongnu.org/）。这两个归档由 Emacs 开发者维护，并由 GNU 工程托管。“GNU ELPA” 包含我们认为属于 GNU Emacs 一部分、但独立于 Emacs 核心分发的 GNU 包。“NonGNU ELPA” 包含版权尚未转让给自由软件基金会的第三方包。(1)

这一点由变量 ‘package-archives’ 控制，它的值是 Emacs 已知的包归档的列表。每个列表元素必须形如 ‘(ID . LOCATION)’，其中 ID 是包归档的名称，LOCATION 是包归档目录的 URL 或名称。若你想使用第三方包归档，可以修改这个列表——但这样做风险自负，并且只使用你认为可以信任的第三方！

-- 用户选项：package-archives 该变量的值是 Emacs 包管理器所识别的包归档的关联列表（alist）。

每个关联列表元素对应一个归档，应当形如‘(ID . LOCATION)’，其中 ID 是归档的名称（字符串），LOCATION 是它的“基位置”（base location，字符串）。

若基位置以‘http:’或‘https:’开头，则它被视为 HTTP(S) URL，包就通过 HTTP(S) 从该归档下载（默认的 GNU 归档就是这种情况）。

否则，基位置应当是目录名。在这种情况下，Emacs 通过普通的文件访问从该归档获取包。这类本地归档主要用于测试。

Emacs 可以在你访问某些类型的文件时建议安装包。当你访问的文件没有可用的特定模式时，可以使用‘package-autosuggest’命令查找并安装为该文件类型推荐的附加 ELPA 包。若你启用‘package-autosuggest-mode’，那么当缓冲区所访问的文件有推荐的附加包时，Emacs 会自动通知你。默认情况下，这类通知会在缓冲区窗口的模式行上显示一个按钮，建议你通过安装附加包来升级主模式，但你可以通过定制用户选项‘package-autosuggest-style’来控制 Emacs 呈现这些通知的方式。

包归档的维护者可以通过“签名”来提高你对其包的信任度。他们生成一对私钥/公钥，并用私钥为每个包创建“签名文件”。有了公钥，你就可以用签名文件来验证包的创建者，并确认包未被篡改。签名验证通过 EasyPG 接口使用 GnuPG 包（https://www.gnupg.org/）完成（参见《EasyPG 手册》）。有效的签名并不能铁定保证包没有恶意，所以你还是应当谨慎行事。包归档应当提供说明，告诉你如何获取其公钥。一种方式是从 <https://pgp.mit.edu/> 之类的服务器下载密钥。使用‘M-x package-import-keyring’把密钥导入 Emacs。Emacs 把包的密钥存放在变量‘package-gnupghome-dir’所指定的目录中，默认是‘package-user-dir’下的‘gnupg’子目录，这会使 Emacs 在验证签名时以‘--homedir’选项调用 GnuPG。若‘package-gnupghome-dir’为‘nil’，则省略 GnuPG 的‘--homedir’选项。GNU 包归档的公钥随 Emacs 一起分发，位于‘etc/package-keyring.gpg’。Emacs 会自动使用它。

若用户选项‘package-check-signature’为非‘nil’，则 Emacs 在你安装包时会尝试验证签名。若该选项的值为‘allow-unsigned’，并且找到了可用的 OpenPGP 配置，则会检查已签名的包，但你仍然可以安装未签名的包。若你使用某些不对其包签名的归档，可以把它们加入列表‘package-unsigned-archives’。（若该值为‘allow-unsigned’且未找到可用的 OpenPGP，则该选项会被当作其值为‘nil’来处理。）若值为‘t’，则至少要有一个签名有效；若值为‘all’，则所有签名都必须有效。

关于密码学密钥与签名的更多信息，参见《《gnupg》手册》。Emacs 自带一个 GNU Privacy Guard 的接口，参见《EasyPG 手册》。

若你启用了多个包归档，而其中一些归档提供同一个包的不同版本，你可能会发现选项‘package-pinned-packages’很有用。你可以把包/归档配对加入这个列表，以确保指定的包只从指定的归档下载。

当你启用了多个包归档时，另一个有用的选项是‘package-archive-priorities’。它指定每个归档的优先级（数字越大表示归档优先级越高）。默认情况下，归档的优先级为零，除非该选项的值另有指定。若同一个包可从更高优先级的归档获得，则来自较低优先级归档的该包不会显示在菜单中。（这一点由‘package-menu-hide-low-priority’的值控制。）

包一经下载、字节编译并安装，就会对当前 Emacs 会话可用。使包可用会把它的目录加入‘load-path’，并加载它的自动加载（autoloads）。包的自动加载的效果因包而异。多数包只是让一些新命令可用，另一些包则对 Emacs 会话有更广泛的影响。关于这类信息，请查阅该包的帮助缓冲区。

已安装的包会由 Emacs 在后续所有会话中自动设为可用。这发生在启动时，在处理初始化文件之前，但在处理早期初始化文件之后（参见“早期初始化文件”）。作为例外，若 Emacs 以 ‘-q’ 或 ‘--no-init-file’ 选项启动，则它在启动时不会使包可用（参见“初始选项”）。

要让 Emacs 在启动时不自动使包可用，请把变量 ‘package-enable-at-startup’ 改为 ‘nil’。你必须在早期初始化文件中做这件事，因为该变量在加载常规初始化文件之前就会被读取。因此，若你通过 Customize 定制这个变量，就应当把定制后的设置保存到早期初始化文件中。为此，在保存 ‘package-enable-at-startup’ 的定制值之前，先把变量 ‘custom-file’（参见“保存自定义”）的值设置或改为指向你的早期初始化文件。

若你安装了许多包，可以通过把用户选项 ‘package-quickstart’ 设为 ‘t’ 来改善启动时间。设置该选项会让 Emacs 预先计算许多东西，而不是在每次 Emacs 启动时重新计算。不过，若你这样做，那么当激活情况需要改变时（例如当你改变 ‘package-load-list’ 的值时），你必须手动运行命令 ‘package-quickstart-refresh’。

若你把 ‘package-enable-at-startup’ 设为 ‘nil’，你仍然可以在启动期间或启动之后使包可用。要在启动期间使已安装的包可用，请在你的初始化文件中调用函数 ‘package-activate-all’。要在启动之后使已安装的包可用，请调用命令 ‘M-: (package-activate-all) RET’。

若想更精细地控制启动时使哪些包可用，可以使用变量 ‘package-load-list’。它的值应当是一个列表。形如 ‘(NAME VERSION)’ 的列表元素告诉 Emacs 使名为 NAME 的包的 VERSION 版本可用。这里 VERSION 应当是一个版本字符串（对应包的某个特定版本），或是 ‘t’（表示使任何已安装的版本可用），或是 ‘nil’（表示不使用任何版本；这会禁用该包，使它无法被设为可用）。列表元素也可以是符号 ‘all’，表示使其他列表元素未指名的任何包的最新已安装版本可用。默认值就是 ‘(all)’。

例如，若你把 ‘package-load-list’ 设为 ‘((muse "3.20") all)’，那么 Emacs 只使 ‘muse’ 包的 3.20 版本可用，外加除 ‘muse’ 之外其他包的任何已安装版本。碰巧已安装的 ‘muse’ 的其他版本都会被忽略。‘muse’ 包会在包菜单中以 ‘held’ 状态列出。

Emacs 的字节码相当稳定，但字节码仍有可能过时，或者编译后的文件可能依赖于新版本 Emacs 中已经改变的宏。你可以使用命令 ‘M-x package-recompile’ 重新编译某个特定的包，或使用 ‘M-x package-recompile-all’ 重新编译所有包。（若你安装了许多包，后一个命令运行起来可能相当费时。）

> 注：关于版权转让的更多信息，参见 Why the FSF Gets Copyright Assignments from Contributors（https://www.gnu.org/licenses/why-assign.html）。

50.4 包文件与目录布局

每个包都从包归档以单个“包文件”（package file）的形式下载——它要么是一个 Emacs Lisp 源文件，要么是一个包含多个 Emacs Lisp 源文件及其他文件的 tar 文件。包文件由安装包的 Emacs 命令自动获取、处理并清理。通常你不需要直接处理它们，除非你在制作包（参见《Emacs Lisp 参考手册》的“Packaging”）。如果你确实需要直接从包文件安装包，请使用命令 ‘M-x package-install-file’。

安装之后，一个包的内容会被放在 ‘~/.emacs.d/elpa/’ 的一个子目录中（你可以通过定制变量 ‘package-user-dir’ 来改变该目录的名称）。包的子目录名为 ‘NAME-VERSION’，其中 NAME 是包名，VERSION 是它的版本字符串。

除了 'package-user-dir' 之外，Emacs 还会在 'package-directory-list' 所列的目录中查找已安装的包。这些目录供系统管理员在整个系统范围内提供 Emacs 包；Emacs 自身从不把包安装到那里。'package-directory-list' 中的包子目录布局与 'package-user-dir' 中相同。

删除一个包（参见"包菜单缓冲区"）意味着删除相应的包子目录。这只对安装在 'package-user-dir' 中的包有效；若要求删除命令作用于系统级包目录中的包，它会发出错误信号。

50.5 获取包源码

默认情况下，'package-install' 从包归档下载一个 Tarball 并安装其中的文件。如果你希望修改包的源码并与他人分享改动，这可能不够。在这种情况下，你可能更愿意直接获取上游源码并在其上工作。这通常更便于开发补丁和报告缺陷。

一种做法是使用 'package-vc-install'，直接从源码获取包的源代码。该命令还会自动确保所有文件都被字节编译并自动加载，就像常规包一样。以此方式安装的包的行为与其他任何包相同。你可以用 'package-upgrade' 或 'package-upgrade-all' 升级它们，再用 'package-delete' 删除它们。它们甚至会显示在常规的包列表中。如果你只想克隆包的源码，而不把它加入包列表，请使用 'package-vc-checkout'。

注意，目前内置包无法使用 'package-vc-install' 升级。

有了源码检出，你可能想针对当前的开发版本复现某个缺陷，或实现一个新特性以解一时之需。如果包的元数据说明了如何联系维护者，你可以使用命令 'package-report-bug' 通过电子邮件报告缺陷。该报告将包含你已自定义的所有用户选项。如果你做了希望与维护者分享的改动，请先提交你的改动，然后使用命令 'package-vc-prepare-patch' 分享它。参见"准备补丁"。

50.5.1 指定包源

要从源码安装包，Emacs 必须知道从哪里获取包的源代码（例如代码仓库），以及关于代码结构的基本信息（例如多文件包中的主文件）。"包规格"（package specification）描述这些属性。

当包归档支持时（参见《Emacs Lisp 参考手册》的"Package Archives"），Emacs 可以从该归档自动下载包的规格。如果传给 'package-vc-install' 的第一个参数是命名某个包的符号，那么 Emacs 将使用该归档为该包提供的规格。

```
;; Emacs will download BBDB's specification from GNU ELPA:
(package-vc-install 'bbdb)
```

'package-vc-install' 的第一个参数也可以是一个包规格。这使你能够从用户选项 'package-archives' 所列已知归档之外的位置安装源码包。包规格是形如 '(NAME . SPEC)' 的列表，其中 SPEC 应是使用下表中任一键的属性列表。

关于使用代码仓库和版本控制系统的基本术语的定义，参见《Emacs 手册》的"VCS Concepts"。

- ':url' 一个字符串，给出指定仓库的 URL，包的源代码将从该仓库获取。
- ':branch' 一个字符串，给出要安装的代码的修订版本。不要把它与包的版本号混淆。
- ':lisp-dir' 一个字符串，给出用于加载 Lisp 源码的目录相对于仓库的名称，默认为仓库的根目录。

- ‘:main-file’ 一个字符串，给出项目的主文件，包元数据即从该文件收集。若未给出，默认为在包名后追加“.el”。
- ‘:doc’ 一个字符串，给出用于构建 Info 文件的文档文件相对于仓库的名称。这可以是 Texinfo 文件或 Org 文件。
- ‘:make’ 一个字符串或字符串列表，给出仓库 Makefile 中定义的目标，这些目标应在构建 Info 文件之前运行。仅在 ‘package-vc-allow-build-commands’ 为非 ‘nil’ 时生效。
- ‘:shell-command’ 一个字符串，给出在构建 Info 文件之前要运行的 shell 命令。仅在 ‘package-vc-allow-build-commands’ 为非 ‘nil’ 时生效。
- ‘:vc-backend’ 一个符号，命名用于下载包仓库副本的 VC 后端（参见《Emacs 手册》的“Version Control Systems”）。若省略，Emacs 将尝试根据所提供的 URL 进行猜测；若仍无法确定，该过程将回退到 ‘package-vc-default-backend’ 的值。

```
;; Specifying information manually:
(package-vc-install
  '(bbdb :url "https://git.savannah.nongnu.org/git/bbdb.git"
         :lisp-dir "lisp"
         :doc "doc/bbdb.texi"))
```

51 自定义

本章介绍一些自定义 Emacs 行为的简单方法。

除了本章描述的方法之外，关于使用 X 资源来自定义 Emacs，参见“附录 D X 选项与资源”；关于录制和重放键盘宏，参见“键盘宏”。若要做更深远、更开放的更改，则涉及编写 Emacs Lisp 代码；参见《Emacs Lisp 参考手册》

- Variables:: 许多 Emacs 命令会检查 Emacs 变量来决定该做什么；通过设置变量，可以控制它们的工作方式。
- Key Bindings:: 键位图规定每个键运行哪条命令。通过修改键位图，你可以重新定义键。
- Init File:: 如何在初始化文件中编写常见的自定义。
- Authentication:: 保存持久的认证信息。
- Newcomers Theme:: 一个为新手启用预设的主题。

51.1 简易自定义界面

Emacs 有许多可以更改的“设置”（settings）。大多数设置是“可自定义变量”（customizable variables，参见“变量”），它们也被称为“用户选项”（user options）。可自定义变量的数量极其庞大，控制着 Emacs 行为的众多方面；本手册中记载的变量列于参见“Variable Index”。另一类设置是“字体”（face），它们决定文本的字体、颜色和其他属性（参见“文本字体”）。

要浏览和修改设置（包括变量和字体），请输入 ‘M-x customize’。这会创建一个“自定义缓冲区”（customization buffer），让你在按逻辑组织好的设置列表中浏览，编辑并设置它们的值，并将它们永久保存。

51.1.1 自定义组

自定义设置被组织成“自定义组”（customization groups）。这些组又被归入更大的组，一直上溯到一个名为 ‘Emacs’ 的总组。

‘M-x customize’ 创建的定制缓冲区会显示顶层的 ‘Emacs’ 组。它看上去（部分）如下：

```
For help using this buffer, see [Easy Customization] in the [Emacs manual].
```

```
_____________________________________ [ Search ]
```

```
Operate on all settings in this buffer:
[ Revert... ] [ Apply ] [ Apply and Save ]
```

```
Emacs group: Customization of the One True Editor.
      [State]: visible group members are all at standard values.
      See also [Manual].
```

```
[Editing]      Basic text editing facilities.
[Convenience]  Convenience features for faster editing.
```

```
...MORE SECOND-LEVEL GROUPS...
```

此缓冲区的主体部分显示 'Emacs' 自定义组，其中包含若干其他组（'Editing'、'Convenience' 等）。这里并不列出那些组的内容，每组只列一行说明文字。

组的"状态"（state）表明该组中的设置是否被编辑过、设置过或保存过。参见"修改变量"。

自定义缓冲区的大部分是只读的，但其中包含一些你可以编辑的"可编辑字段"（editable fields）。例如，自定义缓冲区的顶部有一个用于搜索设置的可编辑字段（参见"浏览与搜索设置"）。此外还有"按钮"（buttons）和"链接"（links），你可以用鼠标点击它们，或把光标点移到那里并键入 '<RET>' 来激活它们。例如 '[Editing]' 之类的组名就是链接；激活其中一个链接会打开该组的自定义缓冲区。

在自定义缓冲区中，键入 '<TAB>'（'widget-forward'）可以向前移动到下一个按钮或可编辑字段。'S-<TAB>'（'widget-backward'）则向后移动到前一个按钮或可编辑字段。

51.1.2 浏览与搜索设置

从 'M-x customize' 创建的顶层自定义缓冲区出发，你可以沿着链接进入 'Emacs' 自定义组的各个子组。这些子组中可能包含可供你自定义的设置；它们也可能再包含子组，涉及 Emacs 中更为专门的子系统。在自定义组的层次结构中浏览时，你应当能找到一些想要自定义的设置。

如果你只想自定义某个特定的设置或自定义组，可以直接用 'M-x customize-option'、'M-x customize-face' 或 'M-x customize-group' 命令前往。参见"自定义特定项目"。

如果你还不确定究竟要自定义哪些组或设置，可以利用每个自定义缓冲区顶部的可编辑搜索字段来搜索它们。在该字段中，你可以输入搜索词——一个或多个以空格分隔的词，或者一个正则表达式（参见"正则表达式的语法"）。然后在该字段中键入 '<RET>'，或激活它旁边的 'Search' 按钮，就会切换到包含与这些词匹配的组和设置的自定义缓冲区。但请注意，此功能只查找当前 Emacs 会话中已加载的组和设置。

如果你不希望自定义缓冲区显示搜索字段，请把变量 'custom-search-field' 改为 'nil'。

命令 'M-x customize-apropos' 与使用搜索字段类似，只是它通过迷你缓冲区读取搜索词。参见"自定义特定项目"。

'M-x customize-browse' 是浏览可用设置的另一种方式。该命令创建一个特殊的自定义缓冲区，它按结构化的布局只显示组和设置的名称。你可以在同一缓冲区中激活组名旁边的 '[+]' 按钮来展开该组的内容。组内容显示出来后，按钮变为 '[-]'；激活它会再次隐藏组内容。此缓冲区中的每个组或设置都有一个链接，标注为 '[Group]'、'[Option]' 或 '[Face]'。激活该链接会创建一个普通的自定义缓冲区，只显示那个组、选项或字体；这就是修改你用 'M-x customize-browse' 找到的设置的方法。

51.1.3 修改变量

下面是一个变量（即用户选项）在自定义缓冲区中的样子示例：

```
[Hide] Kill Ring Max: Integer (positive or zero): 120
   [State]: STANDARD.
   Maximum length of kill ring before oldest elements are thrown away.
```

第一行表明该变量名为‘kill-ring-max’，为便于查看而格式化为‘Kill Ring Max’，同时还显示它预期的类型：正整数或零。默认值是‘120’。你可以用‘C-c <TAB>’（‘Custom-goto-first-choice’）直接移动到该值处。标有‘[Hide]’的按钮若被激活，会隐藏该变量的值和状态；当值非常长时，这有助于避免自定义缓冲区变得杂乱（出于这个原因，值很长的变量可能一开始就是隐藏的）。如果你使用了‘[Hide]’按钮，它会变成‘[Show Value]’，激活它即可重新显示值和状态。在图形显示下，‘[Hide]’和‘[Show Value]’按钮分别被替换为指向下方和指向右方的图形三角形。

变量名之后的那一行表示该变量的“自定义状态”（customization state）：在本例中，‘STANDARD’表示你尚未修改变量，因此它的值是默认值。‘[State]’按钮给出一个用于自定义该变量的操作菜单。

自定义状态下面是该变量的文档。这与‘C-h v’命令所显示的文档相同（参见“查看与设置变量”）。如果文档超过一行，则可能只显示一行。这种情况下，该行末尾会有一个‘[More]’按钮；激活它即可看到完整的文档。

要为‘Kill Ring Max’输入新值，只需把光标点移到该值处并编辑它。例如，键入‘M-d’删除‘60’，再输入另一个数字。当你开始改动文本时，‘[State]’那一行会随之改变：

```
[State]: EDITED, shown value does not take effect until you
         set or save it.
```

编辑值并不会让它立即生效。要做到这一点，你必须通过激活‘[State]’按钮并选择‘Set for Current Session’来“设置”（set）该变量。此后变量的状态变为：

```
[State]: SET for current session only.
```

你不必担心指定的值无效；‘Set for Current Session’操作会检查有效性，不会装入不可接受的值。

在编辑某些类型的值时，例如文件名、目录名和 Emacs 命令名，你可以用‘C-M-i’（‘widget-complete’），或等效的‘M-<TAB>’或‘<ESC> <TAB>’进行补全。其行为与迷你缓冲区补全十分相似（参见“补全”）。

在可编辑的值字段上键入‘<RET>’会像‘<TAB>’一样把光标点向前移到下一个字段或按钮。因此，编辑完一个字段后，你可以键入‘<RET>’继续前往下一个按钮或字段。要在可编辑字段中插入换行，请使用‘C-o’或‘C-q C-j’。

对某些变量而言，合法的取值只有固定的一组，不允许你直接编辑其值。此时值的前面会出现一个‘[Value Menu]’按钮；激活该按钮会给出可选的取值。对于“开或关”的布尔值，该按钮显示为‘[Toggle]’，用来翻转该值。使用‘[Value Menu]’或‘[Toggle]’按钮之后，你必须再次设置该变量，才能使所选的值生效。

有些变量的值具有复杂的结构。例如，‘minibuffer-frame-alist’的值是一个关联列表。它在自定义缓冲区中的显示如下：

```
[Hide] Minibuffer Frame Alist:
[INS] [DEL] Parameter: width
            Value: 80
[INS] [DEL] Parameter: height
            Value: 2
```

```
[INS]
   [ State ]: STANDARD.
   Alist of parameters for the initial minibuffer frame. [Hide]
   [...more lines of documentation...]
```

本例中，列表里的每个关联由两项组成，一项标为 'Parameter'，一项标为 'Value'；两者都是可编辑字段。你可以用关联旁边的 '[DEL]' 按钮从列表中删除它。要添加关联，就在你希望插入的位置使用 '[INS]' 按钮；最后一个 '[INS]' 按钮会在列表末尾插入。

设置变量后，新值只在当前 Emacs 会话中生效。要为以后的会话"保存"（save）该值，请使用 '[State]' 按钮并选择 'Save for Future Sessions' 操作。参见"保存自定义"。

你也可以使用 '[State]' 按钮并选择 'Erase Customization' 操作，把变量恢复为标准值。实际上共有四种重置操作：

- 'Undo Edits'：如果你修改了变量但尚未设置它，此操作会把自定义缓冲区中的文本恢复成与实际值一致。
- 'Revert This Session's Customizations'：此操作把变量的值恢复为最近保存的值（若有）。否则恢复为标准值。它还会相应地更新文本。
- 'Erase Customization'：此操作把变量设置为其标准值。你保存过的任何值也会被消除。
- 'Set to Backup Value'：此操作把变量设置为本次会话中先前在自定义缓冲区里设置过的某个值。如果你自定义了某个变量，随后又重置了它（从而丢弃了自定义的值），你可以用此操作把丢弃的值找回来。

有时记录一条关于某次自定义的注释很有用。使用 '[State]' 菜单中的 'Add Comment' 项可以创建一个用于输入注释的字段。

自定义缓冲区靠近顶部的位置有两行按钮：

```
Operate on all settings in this buffer:
[Revert...] [Apply] [Apply and Save]
```

'[Revert...]' 按钮会弹出一个菜单，其中包含上面描述的前 3 种重置操作。'[Apply]' 按钮把设置应用于当前会话。'[Apply and Save]' 按钮应用设置并为以后的会话保存它们；如果 Emacs 启动时带有 '-q' 或 '-Q' 选项，此按钮不会出现（参见"初始选项"）。

命令 'C-c C-c'（'Custom-set'）等效于使用 '[Set for Current Session]' 按钮。命令 'C-x C-s'（'Custom-save'）相当于使用 '[Save for Future Sessions]' 按钮。同样，'C-c C-k'（'Custom-reset-standard'）相当于使用 '[Revert...]' 按钮。

'[Exit]' 按钮会切换出自定义缓冲区，并把该缓冲区埋到缓冲区列表的底部。若想让它改为杀死自定义缓冲区，请把变量 'custom-buffer-done-kill' 改为 't'。

51.1.4 保存自定义

在自定义缓冲区中，你可以通过从某个设置的 '[State]' 按钮中选择 'Save for Future Sessions'，来"保存"该自定义设置。'C-x C-s'（'Custom-save'）命令，或自定义缓冲区顶部的 '[Apply and Save]' 按钮，会保存缓冲区中所有适用的设置。

保存的做法是把代码写入一个文件，通常是你的初始化文件（参见"Emacs 初始化文件"）。以后的 Emacs 会话在启动时会自动读取该文件，从而重新建立这些自定义。

你也可以选择把自定义保存在初始化文件以外的地方。为此，必须在初始化文件中添加几行代码，把变量‘custom-file’设为所需文件的名字，并加载该文件。例如：

```
(setq custom-file "~/.config/emacs-custom.el")
(load custom-file)
```

你甚至可以针对不同的 Emacs 版本指定不同的自定义文件，如下所示：

```
(cond ((< emacs-major-version 28)
       ;; Emacs 27 customization.
       (setq custom-file "~/.config/custom-27.el"))
      ((and (= emacs-major-version 26)
            (< emacs-minor-version 3))
       ;; Emacs 26 customization, before version 26.3.
       (setq custom-file "~/.config/custom-26.el"))
      (t
       ;; Emacs version 28.1 or later.
       (setq custom-file "~/.config/emacs-custom.el")))

(load custom-file)
```

如果 Emacs 是以‘-q’或‘--no-init-file’选项启动的（参见“初始选项”），它不会允许你把自定义保存到初始化文件中。因为从这样的会话中保存自定义，会抹掉初始化文件中可能已有的其他所有自定义。

请注意，你没有选择为以后的会话保存的任何自定义，在你退出 Emacs 时都会丢失。如果你希望在退出时对有未保存的自定义给出提示，请把以下内容加入初始化文件：

```
(add-hook 'kill-emacs-query-functions
          'custom-prompt-customize-unsaved-options)
```

51.1.5 自定义字体

你可以自定义字体（face，参见“文本字体”），它们决定 Emacs 如何显示不同类型的文本。自定义组中可以同时包含变量和字体。

例如，在编程语言主模式中，源代码注释用‘font-lock-comment-face’显示（参见“Font Lock 模式”）。在自定义缓冲区中，点击‘[Show All Attributes]’链接后，该字体的显示如下：

```
[Hide] Font Lock Comment Face:[sample]
   [State] : STANDARD.
   Font Lock mode face used to highlight comments.
   [ ] Font Family: --
   [ ] Font Foundry: --
   [ ] Width: --
   [ ] Height: --
   [ ] Weight: --
   [ ] Slant: --
   [ ] Underline: --
   [ ] Overline: --
   [ ] Strike-through: --
   [ ] Box around text: --
   [ ] Inverse-video: --
```

```
[X] Foreground: Firebrick     [Choose]  (sample)
[ ] Background: --
[ ] Stipple: --
[ ] Inherit: --
[Hide Unused Attributes]
```

前三行显示该字体的名称、‘[State]’ 按钮和文档。其下是“字体属性”（face attributes）列表。每个属性前面都有一个复选框。填满的复选框 ‘[X]’ 表示该字体为此属性指定了值；空的复选框 ‘[ ]’ 表示该字体未为此属性指定任何特殊值。你可以激活复选框来指定或取消指定其属性。

字体不必指定每一个属性；事实上，大多数字体只指定少数几个属性。在上面的例子中，‘font-lock-comment-face’ 只指定了前景色。任何未指定的属性都取自名为 ‘default’ 的特殊字体，该字体的属性全都指定了。‘default’ 字体用于显示没有显式指定字体的任何文本；此外，它的背景色属性还充当框架的背景色。

属性列表末尾的 ‘[Hide Unused Attributes]’ 按钮会隐藏该字体未指定的属性。属性处于隐藏状态时，该按钮变为 ‘[Show All Attributes]’，激活它会显示完整的属性列表。为避免界面杂乱，自定义缓冲区一开始可能就隐藏了未指定的属性。

某个属性被指定后，你可以用通常的方式修改它的值。

前景色和背景色既可以用颜色名指定，也可以用 RGB 三元组指定（参见“字体的颜色”）。你也可以用 ‘[Choose]’ 按钮切换到一个颜色名列表；在该缓冲区中用 ‘<RET>’ 选择某个颜色，即可把该颜色名放入值字段。

设置、保存和重置字体的工作方式与变量的相应操作相同（参见“修改变量”）。

一个字体可以为不同类型的显示指定不同的外观。例如，字体可以让文本在彩色显示上为红色，而在单色显示上使用粗体。要为字体指定多种外观，请在激活 ‘[State]’ 后得到的菜单中选择 ‘For All Kinds of Displays’。

51.1.6 自定义特定项目

- ‘M-x customize-option <RET> OPTION <RET>’、‘M-x customize-variable <RET> OPTION <RET>’：只为某一个用户选项 OPTION 建立自定义缓冲区。
- ‘M-x customize-face <RET> FACE <RET>’：只为某一个字体 FACE 建立自定义缓冲区。
- ‘M-x customize-icon <RET> FACE <RET>’：只为某一个图标 ICON 建立自定义缓冲区。
- ‘M-x customize-group <RET> GROUP <RET>’：只为某一个组 GROUP 建立自定义缓冲区。
- ‘M-x customize-apropos <RET> REGEXP <RET>’：为所有与 REGEXP 匹配的设置和组建立自定义缓冲区。
- ‘M-x customize-changed <RET> VERSION <RET>’：为所有自 Emacs 版本 VERSION 以来其含义已改变（或在之后新增）的用户选项、字体和组建立自定义缓冲区。
- ‘M-x customize-saved’：建立一个自定义缓冲区，其中包含你已用自定义缓冲区保存的所有设置。
- ‘M-x customize-unsaved’：建立一个自定义缓冲区，其中包含你已设置但尚未保存的所有设置。

如果你想自定义某个特定的用户选项，键入 ‘M-x customize-option’。它会读取变量名，并只为那一个用户选项建立自定义缓冲区。在迷你缓冲区中输入变量名时可以使用补全，但只能补全已加载到 Emacs 中的变量名。

同样，你可以用 'M-x customize-face' 自定义某个特定的字体。你可以用 'M-x customize-group' 为特定的自定义组建立自定义缓冲区。

'M-x customize-apropos' 会提示输入搜索词——一个或多个以空格分隔的词，或一个正则表达式——并为所有名称匹配的*已加载*设置和组建立自定义缓冲区。这与使用自定义缓冲区顶部的搜索字段相似（参见"自定义组"）。

升级到新的 Emacs 版本时，你可能想考虑自定义新增的设置，以及含义或默认值已改变的设置。为此，请使用 'M-x customize-changed'，并在迷你缓冲区中指定先前的 Emacs 版本号。它会创建一个自定义缓冲区，显示自指定版本以来定义已改变的所有设置和组，必要时还会加载它们。

如果你修改了设置，随后又觉得这次修改是个错误，可以用两个命令重新查看你的修改。使用 'M-x customize-saved' 自定义你已保存的设置。使用 'M-x customize-unsaved' 自定义你已设置但未保存的设置。

51.1.7 自定义主题

"自定义主题"（Custom themes）是可以作为一个整体启用或禁用的一组设置的集合。你可以用自定义主题在各种设置集合之间轻松切换，并把这些集合从一台计算机转移到另一台。

自定义主题存储为一个 Emacs Lisp 源文件。如果自定义主题的名字是 NAME，则主题文件名为 'NAME-theme.el'。关于主题文件的格式以及如何制作主题，参见"创建自定义主题"。

键入 'M-x customize-themes' 可切换到一个名为 '*Custom Themes*' 的缓冲区，其中列出 Emacs 所知道的自定义主题。默认情况下，Emacs 在两个位置查找主题文件：变量 'custom-theme-directory' 指定的目录（默认是 '~/.emacs.d/'），以及你的 Emacs 安装目录中名为 'etc/themes' 的目录（参见变量 'data-directory'）。后者包含随 Emacs 分发的若干自定义主题，它们为适应各种配色方案而自定义 Emacs 的字体。（但请注意，自定义主题不必局限于这一用途；它们也可以用来自定义变量。）

如果你希望 Emacs 在其他目录中查找自定义主题，就把该目录加入列表变量 'custom-theme-load-path'。它的默认值是 '(custom-theme-directory t)'；在这里，符号 'custom-theme-directory' 具有"变量 'custom-theme-directory' 的值"这一特殊含义，而 't' 代表内置的主题目录 'etc/themes'。'*Custom Themes*' 缓冲区中列出的主题，就是那些在 'custom-theme-load-path' 所指定的目录中找到的主题。

在 '*Custom Themes*' 缓冲区中，你可以激活某个自定义主题旁边的复选框，以便在当前 Emacs 会话中启用或禁用该主题。启用某个自定义主题后，它的所有设置（变量和字体）都会在该 Emacs 会话中生效。若要把主题的选择应用于以后的 Emacs 会话，请键入 'C-x C-s'（'custom-theme-save'），或使用 '[Save Theme Settings]' 按钮。

第一次启用某个自定义主题时，Emacs 会显示主题文件的内容，并询问你是否确实要加载它。因为加载自定义主题可能执行任意 Lisp 代码，所以只有在你确知该主题安全时才应回答是；在这种情况下，Emacs 会提出记住该主题是安全的（做法是把主题文件的 SHA-256 哈希值保存到变量 'custom-safe-themes'；如果你想把所有主题都视为安全，把该值改为 't'）。随 Emacs 提供的主题（位于 'etc/themes' 目录）不受此检查约束，始终被视为安全。

设置或保存自定义主题实际上是通过定制变量 'custom-enabled-themes' 实现的。该变量的值是一个自定义主题名列表（作为 Lisp 符号，例如 'tango'）。除了使用 '*Custom Themes*' 缓冲区来设置 'custom-enabled-themes'，

你还可以用通常的定制界面来定制该变量，例如 'M-x customize-option'。注意，自定义主题本身不允许设置 'custom-enabled-themes'。

你通过定制缓冲区所做的任何定制都优先于主题设置。这让你可以轻松覆盖自己并不赞同的个别主题设置。如果两个不同主题的设置发生重叠，'custom-enabled-themes' 中靠前的主题优先。在定制缓冲区中，如果某个设置已被某个自定义主题从默认值改变，其 'State' 显示会显示 'THEMED' 而不是 'STANDARD'。

你可以在当前 Emacs 会话中输入 'M-x load-theme' 来启用某个特定的自定义主题。它会提示输入主题名，从主题文件加载该主题并启用它。如果某个主题文件此前已被加载过，你可以输入 'M-x enable-theme' 在不加载其文件的情况下启用该主题。要禁用某个自定义主题，输入 'M-x disable-theme'。有些主题提供一组用户选项（参见"新手主题"），这些选项可能随时间变化。如果你想保持当前的选项集合，可以把某个主题的用户选项复制到自己的配置中，可以使用 'copy-theme-options' 命令。

要查看某个自定义主题的说明，在 '*Custom Themes*' 缓冲区中该主题所在行上输入 '?'；或者在 Emacs 任意位置输入 'M-x describe-theme' 并输入主题名。

有些主题带有变体（最常见的是两个：浅色和深色）。你可以用 'M-x theme-choose-variant' 切换到另一个变体。如果当前启用的主题只有一个其他变体，就会直接选中它；如果有更多变体，该命令会提示你选择切换到哪一个。

注意，只有在只有一个主题处于启用状态时，'theme-choose-variant' 才有效。

51.1.8 创建自定义主题

你可以用类似于定制缓冲区的界面来定义自定义主题，方法是输入 'M-x customize-create-theme'。这会切换到一个名为 '*Custom Theme*' 的缓冲区。它还会提议向主题中插入一些常见的 Emacs 字体（face）（这很方便，因为自定义主题常用于定制字体）。如果你回答 no，该主题初始将不包含任何设置。

在 '*Custom Theme*' 缓冲区靠近顶部的地方，有可编辑的字段，用来输入主题的名称和描述。名称可以是除 'user' 之外的任何内容。描述就是在对该主题调用 'M-x describe-theme' 时显示的内容。其第一行应是一句简短的一句话概括；在 'M-x customize-themes' 生成的缓冲区中，这句话会显示在主题名旁边。

要向主题添加新设置，请使用 '[Insert Additional Face]' 或 '[Insert Additional Variable]' 按钮。每个按钮都用迷你缓冲区（带补全）读取一个字体或变量名，并为该字体或变量插入一条定制条目。你可以像在普通定制缓冲区中那样编辑变量值或字体属性。要从主题中移除某个字体或变量，取消勾选其名称旁边的复选框。

指定好自定义主题的字体和变量后，输入 'C-x C-s'（'custom-theme-write'）或使用缓冲区的 '[Save Theme]' 按钮。这会保存主题文件，文件名为 'NAME-theme.el'（其中 NAME 是主题名），保存在 'custom-theme-directory' 指定的目录中。

在 '*Custom Theme*' 缓冲区中，你可以通过激活 '[Visit Theme]' 按钮并指定主题名来查看和编辑已有的自定义主题。你还可以使用 '[Merge Theme]' 按钮把另一个主题的设置加入该缓冲区。你可以使用 '[Merge Theme]' 按钮并指定特殊主题 'user'，从而把自己的非主题设置导入到自定义主题中。

主题文件就是一个 Emacs Lisp 源文件，加载自定义主题实际上就是加载该 Lisp 文件。因此，你可以直接编辑主题文件，而不必使用 '*Custom Theme*' 缓冲区。详见参见《Emacs Lisp 参考手册》的"Custom Themes"。

51.2 变量

“变量”（variable）是带有值的 Lisp 符号。符号的名字也称为“变量名”。变量名可以包含任何能出现在文件名中的字符，但大多数变量名由用连字符分隔的普通单词组成。

变量的名字起到对其作用的简洁描述。大多数变量还有一个“文档字符串”（documentation string），描述该变量的用途、它应当具有何种值以及该值将如何被使用。你可以使用帮助命令 ‘C-h v’（‘describe-variable’）查看该文档。参见“查看与设置变量”。

Emacs 使用许多 Lisp 变量进行内部记录，但对非程序员用户最有意思的变量是那些供用户修改的变量——它们被称为“可定制变量”（customizable variables）或“用户选项”（user options）（参见“简易自定义界面”）。在下面几节中，我们将描述 Emacs 变量的其他方面，例如如何在 Customize 之外设置它们。

Emacs Lisp 允许任何变量（有少数例外）具有任何类型的值。然而，许多变量只有在被赋予某种类型的值时才有意义。例如，只有数字才是 ‘kill-ring-max’ 的有意义的值，该变量指定删除环的最大长度（参见“粘贴更早删除的内容”）；如果你给 ‘kill-ring-max’ 一个字符串值，诸如 ‘C-y’（‘yank’）这样的命令就会发出错误信号。另一方面，有些变量并不关心类型；例如，如果一个变量对 ‘nil’ 值有一种效果，而对非 ‘nil’ 值有另一种效果，那么任何不是符号 ‘nil’ 的值都会引发第二种效果，不论其类型如何（按惯例，我们通常用值 ‘t’——代表“真”的符号——来指定非 ‘nil’ 值）。如果你用定制缓冲区设置变量，就不必担心给它一个无效的类型：定制缓冲区通常只允许你输入有意义的值。有疑问时，用 ‘C-h v’（‘describe-variable’）查看变量的文档字符串，看看它期望何种类型的值（参见“查看与设置变量”）。

- 钩子：钩子变量让你指定 Emacs 的某些部分在特定场合运行的程序。
- 局部变量：变量在各缓冲区中的取值。
- 文件变量：文件如何指定变量值。
- 目录变量：如何按目录指定变量值。
- 连接变量：对默认目录为远程的缓冲区有效的变量。

51.2.1 查看与设置变量

‘C-h v VAR <RET>’ 显示变量 VAR 的值和文档（‘describe-variable’）。

‘M-x set-variable <RET> VAR <RET> VALUE <RET>’ 把变量 VAR 的值改为 VALUE。

要查看变量的值，使用 ‘C-h v’（‘describe-variable’）。它用迷你缓冲区（带补全）读取变量名，并显示该变量的值和文档。例如，

```
C-h v fill-column <RET>
```

显示的内容大致如下：

```
fill-column is a variable defined in ‘C source code’.
Its value is 70

Automatically becomes buffer-local when set.
This variable is safe as a file local variable if its value
```

```
satisfies the predicate ‘integerp’.
Probably introduced at or before Emacs version 18.

Documentation:
Column beyond which automatic line-wrapping should happen.
Interactively, you can set the buffer local value using C-x f.

You can customize this variable.
```

其中 ‘You can customize the variable’ 一行表明该变量是用户选项。‘C-h v’ 并不限于用户选项；它也允许查看不可定制的变量。

设置某个特定可定制变量最方便的方法是 ‘M-x set-variable’。它先用迷你缓冲区（带补全）读取变量名，然后再用迷你缓冲区读取新值的 Lisp 表达式（你可以用 ‘M-n’ 把旧值插入迷你缓冲区以便编辑）。例如，

```
M-x set-variable <RET> fill-column <RET> 75 <RET>
```

把 ‘fill-column’ 设为 75。

‘M-x set-variable’ 仅限于可定制变量，但你可以用这样的 Lisp 表达式设置任何变量：

```
(setq fill-column 75)
```

要执行这样的表达式，输入 ‘M-:’（‘eval-expression’）并在迷你缓冲区中输入该表达式（参见“求值 Emacs Lisp 表达式”）。或者，转到 ‘*scratch*’ 缓冲区，输入该表达式，然后输入 ‘C-j’（参见“Lisp 交互缓冲区”）。

设置变量与除特别说明外的所有定制 Emacs 的手段一样，只影响当前 Emacs 会话。要在将来的会话中改变该变量，唯一的办法是在初始化文件中写入相应内容（参见“Emacs 初始化文件”）。

如果你要设置的是可定制变量，又不想使用 Customize 界面，可以使用 ‘setopt’ 宏。例如：

```
M-: (setopt fill-column 75) <RET>
```

或者，如果你想在初始化文件中这样做：

```
(setopt fill-column 75)
```

这与 ‘setq’ 作用相同，但如果该变量带有专门的设置函数（setter），使用 ‘setopt’ 时会自动运行它们。你也可以对其他不可定制的变量使用 ‘setopt’，但这比使用 ‘setq’ 效率低。

还有一个缓冲区局部的 ‘setopt’ 版本，称为 ‘setopt-local’，你可以用它为可定制选项设置特定于缓冲区的值，例如在模式钩子中（参见“钩子”）。

它与 ‘setq-local’ 作用相同，但如果该变量带有专门的设置函数，使用 ‘setopt-local’ 时会自动运行它们。你也可以对其他不可定制的变量使用 ‘setopt-local’，但这比使用 ‘setq-local’ 效率低。

如果你只想在当前缓冲区中改变某个可定制变量的值，可以使用 ‘setopt-local’ 宏。例如：

```
M-: (setopt-local fill-column 75) <RET>
```

或者，如果你想在初始化文件中这样做，可在某个模式钩子内使用以下内容，这样该变量就会在该模式的缓冲区中被自动定制（参见“钩子”）：

```
(setopt-local fill-column 75)
```

51.2.2 钩子

“钩子”（hook）是定制 Emacs 的重要机制。钩子是一个 Lisp 变量，其中保存一个函数列表，这些函数会在某个定义明确的场合被调用。（这称为“运行钩子”。）列表中的各个函数称为该钩子的“钩子函数”。例如，钩子 ‘kill-emacs-hook’ 会在退出 Emacs 之前运行（参见“退出 Emacs”）。

大多数钩子是“普通钩子”（normal hooks）。这意味着 Emacs 运行钩子时会依次调用每个钩子函数，不传参数。我们努力使大多数钩子保持普通，以便你能以一致的方式使用它们。名字以 ‘-hook’ 结尾的每个变量都是普通钩子。

少数钩子是“异常钩子”（abnormal hooks）。它们的名字以 ‘-functions’ 结尾，而不是 ‘-hook’（一些旧代码也可能使用已废弃的后缀 ‘-hooks’）。这些钩子之所以异常，在于其函数的调用方式——它们可能被传入参数，或者其返回值会以某种方式被使用。例如，‘find-file-not-found-functions’ 是异常的，因为一旦某个钩子函数返回非 ‘nil’ 值，其余函数就完全不会被调用（参见“访问文件”）。每个异常钩子变量的文档都会说明其函数是如何被使用的。

你可以像设置其他 Lisp 变量一样用 ‘setq’ 设置钩子变量，但向钩子（无论普通还是异常）添加函数的推荐做法是使用 ‘add-hook’，如下例所示。详见参见《Emacs Lisp 参考手册》的“Hooks”。

大多数主模式在初始化的最后一步运行一个或多个“模式钩子”。模式钩子是定制各个模式行为的方便途径；它们总是普通的。例如，下面是设置一个钩子，在文本模式以及基于文本模式的其他模式中打开自动填充模式的方法：

```
(add-hook 'text-mode-hook 'auto-fill-mode)
```

它通过调用 ‘auto-fill-mode’ 起作用；当不提供参数时，该命令会启用这个次模式（参见“次模式”）。接下来，假设你不希望在 LaTeX 模式（它是基于文本模式的模式之一）中打开自动填充模式。你可以用下面额外的一行做到：

```
(add-hook 'latex-mode-hook (lambda () (auto-fill-mode -1)))
```

这里我们使用了特殊的宏 ‘lambda’ 来构造一个匿名函数（参见《Emacs Lisp 参考手册》的“Lambda Expressions”），它以 ‘-1’ 为参数调用 ‘auto-fill-mode’ 以禁用该次模式。由于 LaTeX 模式在运行 ‘text-mode-hook’ 之后运行 ‘latex-mode-hook’，结果就是自动填充模式被禁用。

下面是一个更复杂的例子，展示如何使用钩子定制 C 代码的缩进：

```
(setq my-c-style
  '((c-comment-only-line-offset . 4)
    (c-cleanup-list . (scope-operator
                       empty-defun-braces
                       defun-close-semi))))
```

```
(add-hook 'c-mode-common-hook
  (lambda () (c-add-style "my-style" my-c-style t)))
```

主模式钩子也适用于从原模式“派生”出的其他主模式（参见《Emacs Lisp 参考手册》的“Derived Modes”）。例如，HTML 模式派生自文本模式（参见“SGML 与 HTML 模式”）；启用 HTML 模式时，它会先运行 ‘text-mode-hook’，再运行 ‘html-mode-hook’。这提供了一种方便的方式，用单个钩子影响若干相关模式。特别是，如果你想对任何编程语言模式应用某个钩子函数，就把它加到 ‘prog-mode-hook’ 上；Prog 模式是一个主模式，它除了让其他主模式从它继承之外几乎不做别的事，正是为了这个目的。

最好把你的钩子函数设计成执行顺序无关紧要。任何对顺序的依赖都会招来麻烦。不过，顺序是可预测的：钩子函数按它们在钩子中出现的顺序执行。

如果你反复调用 ‘add-hook’ 来添加某个钩子函数的各种不同版本，请记住，你添加的所有版本都会一起留在该钩子变量中。你可以调用 ‘remove-hook’ 清除单个函数，或者执行 ‘(setq HOOK-VARIABLE nil)’ 移除全部。

如果钩子变量是缓冲区局部的，就会使用该缓冲区局部变量而不是全局变量。然而，如果缓冲区局部变量中含有元素 ‘t’，全局钩子变量也会被运行。

51.2.3 局部变量

‘M-x make-local-variable <RET> VAR <RET>’ 使变量 VAR 在当前缓冲区中具有局部值。

‘M-x kill-local-variable <RET> VAR <RET>’ 使变量 VAR 在当前缓冲区中使用其全局值。

‘M-x make-variable-buffer-local <RET> VAR <RET>’ 标记变量 VAR，使得设置它时会使其对当时为当前的缓冲区局部。

几乎任何变量都可以设为对某个特定 Emacs 缓冲区“局部”。这意味着它在那个缓冲区中的值独立于它在其他缓冲区中的值。少数变量在每个缓冲区中总是局部的。其他每个 Emacs 变量都有一个“全局”值，在所有未将该变量设为局部的缓冲区中生效。

‘M-x make-local-variable’ 读取变量名并使其对当前缓冲区局部。此后在该缓冲区中改变它的值不会影响其他缓冲区，而改变其全局值也不会影响该缓冲区。

‘M-x make-variable-buffer-local’ 标记一个变量，使它在被设置时自动变为局部。更准确地说，一旦变量以这种方式被标记，通常设置该变量的方式都会先自动执行 ‘make-local-variable’。我们称这样的变量为“按缓冲区”变量（per-buffer variables）。Emacs 中的许多变量通常是按缓冲区的；变量的文档字符串会告诉你何时如此。按缓冲区变量的全局值通常在任何缓冲区中都不生效，但它仍然有意义：它是每个新缓冲区中该变量的初始值。

主模式（参见“主模式”）总是在设置变量之前把它们变为缓冲区局部的。这就是为什么在一个缓冲区中改变主模式不会影响其他缓冲区。次模式也通过设置变量来工作——通常每个次模式都有一个控制变量，当模式启用时该变量为非 ‘nil’（参见“次模式”）。对许多次模式来说，控制变量是按缓冲区的，因此总是缓冲区局部的。否则，你可以像其他变量一样在特定缓冲区中把它设为局部。

少数变量不能是缓冲区局部的，因为它们总是对每个显示（display）局部（参见“多个 X 显示”）。如果你试图把这样的变量设为缓冲区局部，会得到一条错误消息。

‘M-x kill-local-variable’ 使指定变量不再对当前缓冲区局部。此后该变量的全局值在这个缓冲区中生效。设置主模式会清除该缓冲区的所有局部变量，只有少数被特别标记为“永久局部”（permanent locals）的变量除外。

要设置变量的全局值，无论该变量在当前缓冲区中是否有局部值，都可以使用 Lisp 构造‘setq-default’。这个构造的用法与‘setq’一样，但它设置的是变量的全局值而不是其局部值（如果有局部值的话）。当当前缓冲区确实有局部值时，新的全局值可能要等到你切换到另一个缓冲区才可见。下面是一个例子：

```
(setq-default fill-column 75)
```

对于已用‘make-variable-buffer-local’标记的变量，‘setq-default’是设置其全局值的唯一方法。

Lisp 程序可以用‘default-value’查看变量的默认值。该函数以一个符号为参数并返回其默认值。参数会被求值；通常你必须显式引用它。例如，下面是获得‘fill-column’默认值的方法：

```
(default-value 'fill-column)
```

51.2.4 文件中的局部变量

文件可以指定在用 Emacs 编辑该文件时要使用的局部变量值。访问该文件或设置主模式时会检查局部变量规定；它会自动使这些变量对缓冲区局部，并把它们设为文件中指定的值。

设置文件局部变量有两种方式：在第一行中，或者在文件末尾附近的局部变量列表中。

如果文件在第一行和局部变量列表中都有局部变量，Emacs 会先处理第一行中的*全部*内容，然后再处理局部变量列表中的*全部*内容。例外的是主模式的指定。无论它出现在哪里，Emacs 都会先应用它，因为大多数主模式会在初始化时清除所有局部变量。

如果为文件所在目录指定了目录局部变量（参见“按目录的局部变量”），文件局部变量会覆盖它们。

51.2.4.1 指定文件变量

下面是在第一行中指定局部变量的一种方式：

```
-*- mode: MODENAME; VAR: VALUE; ... -*-
```

你可以用这种方式指定任意数目的变量/值对，每对用冒号和分号分隔。特殊的变量/值对‘mode: MODENAME;’如果存在，就指定一个主模式（不带“-mode”后缀）。其中的 VALUE 按字面使用，不求值。

下面是一个首行的例子，它指定 Lisp 模式并设置两个具有数值的变量：

```
;; -*- mode: Lisp; fill-column: 75; comment-column: 50; -*-
```

对‘-*-’行做任何修改后，输入‘M-x normal-mode’重新解释它。参见“选择文件模式”。

除‘mode’之外，作为文件变量有特殊含义的其他关键字还有‘coding’、‘unibyte’和‘eval’。这些关键字在稍后描述。

在 shell 脚本中，第一行用于标识脚本解释器，因此你不能在那里放局部变量。为适应这一点，如果第一行指定了解释器，Emacs 会在*第二*行寻找局部变量。对于以魔法字符串“\"”开头、用于指定 troff 预处理器列表的 man 手册页也是如此（不过并非所有手册页都如此）。

你可以用‘M-x add-file-local-variable-prop-line’来代替手工添加条目。该命令提示输入一个变量和值，并以适当方式把它们加到第一行中。命令‘M-x delete-file-local-variable-prop-line’提示输入一个变量，并从该行删除它的

条目。如果存在任何目录局部变量（参见“按目录的局部变量”），命令 ‘M-x copy-dir-locals-to-file-locals-prop-line’ 会把它们复制到第一行。

你也可以使用文件末尾附近的“局部变量列表”（local variables list）来定义文件局部变量。

局部变量列表以包含字符串 ‘Local Variables:’ 的一行开始，以包含字符串 ‘End:’ 的一行结束。中间是变量名和值，每行一组，像这样：

```
/* Local Variables:  */
/* mode: c           */
/* comment-column: 0 */
/* End:              */
```

在这个例子中，每行以 *‘/’ 前缀开始，以 ‘/’* 后缀结束。Emacs 通过在列表第一行上找到环绕魔法字符串 ‘Local Variables:’ 的前缀和后缀来识别它们；然后它会自动把这些前缀和后缀从列表的其他行中去掉。使用前缀和/或后缀的通常原因，是为了把局部变量列表嵌入注释中，以免让该文件所面向的其他程序感到困惑。上面的例子针对 C 编程语言，其中注释以 *‘/’ 开始，以 ‘/’* 结束。

局部变量列表的起始位置距文件末尾不得超过 3000 个字符，并且如果文件分页，它必须位于最后一页。如果某些无关文本可能让 Emacs 看起来像局部变量列表，你可以在该文本之后插入一个换页字符（分页符，参见“页”）来取消它。

对局部变量做任何修改后，输入 ‘M-x normal-mode’ 重新读取它们。参见“选择文件模式”。

与 ‘-*-’ 行一样，局部变量列表中的变量按字面使用，不会先求值。如果你想把一个长字符串值分散到文件的多行中，可以使用反斜杠加换行（在 Lisp 字符串常量中会被忽略）；你应该在每一行都放上前缀和后缀，即使是在字符串内部开始或结束的行，因为处理列表时它们会被去掉。下面是一个例子：

```
# Local Variables:
# compile-command: "cc foo.c -Dfoo=bar -Dhack=whatever \
#   -Dmumble=blaah"
# End:
```

除了直接键入局部变量列表，你还可以使用命令 ‘M-x add-file-local-variable’。它会提示输入一个变量和值，并把它们加入列表，必要时还会添加 ‘Local Variables:’ 字符串以及起始和结束标记。命令 ‘M-x delete-file-local-variable’ 从列表中删除一个变量。‘M-x copy-dir-locals-to-file-locals’ 把目录局部变量复制到该列表（参见“按目录的局部变量”）。

局部变量列表中有一些名字具有特殊含义：

- ‘mode’ 启用指定的主模式。
- ‘eval’ 求值指定的 Lisp 表达式（该表达式的返回值被忽略）。
- ‘coding’ 指定本文件字符代码转换所用的编码系统。参见“编码系统”。
- ‘unibyte’ 表示以单字节模式加载或编译 Emacs Lisp 文件，前提是其值为 ‘t’。参见《Emacs Lisp 参考手册》的“Disabling Multibyte”

这四个关键字实际上并不是变量；在任何其他上下文中设置它们都没有特殊含义。

如果你要跨 Emacs 版本编辑同一个文件，而较新的 Emacs 版本引入了新模式来处理该文件，你可以使用多个 'mode' 条目：在新的 Emacs 中使用新模式（称为 'my-new-mode'），而在较旧的 Emacs 版本中回退到旧模式（称为 'my-old-mode'）。如果你在文件第一行启用模式，可以这样写：

```
-*- mode: my-old; mode: my-new -*-
```

Emacs 会使用它找到的最后一个已定义的模式，因此在较旧的 Emacs 版本中它会忽略 'my-new-mode'，而在定义了 'my-new-mode' 的 Emacs 版本中，它会忽略 'my-old-mode'。同样，在文件末尾的局部变量块中：

```
Local variables:
mode: my-old
mode: my-new
```

不要对次模式使用 'mode' 关键字。要在局部变量列表中启用或禁用某个次模式，请使用 'eval' 关键字，其 Lisp 表达式调用该模式的命令（参见"次模式"）。例如，下面的局部变量列表通过不带参数调用 'eldoc-mode' 来启用 ElDoc 模式（参见"编程语言文档查询"）（以参数 1 调用效果相同），并通过以参数 -1 调用 'font-lock-mode' 来禁用 Font Lock 模式（参见"Font Lock 模式"）。

```
;; Local Variables:
;; eval: (eldoc-mode)
;; eval: (font-lock-mode -1)
;; End:
```

不过请注意，用这种方式指定次模式往往是错误的。次模式代表个人的偏好，把你的偏好强加给可能编辑该文件的另一位用户未必合适。如果你希望根据具体情形自动启用或禁用某个次模式，通常更好的做法是在主模式钩子中完成（参见"钩子"）。

51.2.4.2 文件变量的安全性

文件局部变量可能很危险；当你访问别人的文件时，无法预知它的局部变量列表会对你的 Emacs 做些什么。'eval' 这个"变量"的不当取值，以及 'load-path' 等其他变量的不当取值，可能会执行你并不打算运行的 Lisp 代码。

因此，每当 Emacs 遇到它不知道是否安全的文件局部变量值时，它会显示该文件的整个局部变量列表，并在设置之前请求你确认。你可以键入 'y' 或 <SPC> 使该局部变量列表生效，或键入 'n' 忽略它。当 Emacs 以批处理模式运行时（参见"初始选项"），它无法真正询问你，因此假定答案为 'n'。

Emacs 通常会把某些变量/值对认定为安全。例如，给 'comment-column' 或 'fill-column' 指定任何整数值都是安全的。如果文件只指定了已知安全的变量/值对，Emacs 在设置它们之前不会请求确认。否则，你可以在确认提示处键入 '!'，让 Emacs 把该文件中的所有变量/值对记录为安全。此后 Emacs 在同一文件或其他文件中再次遇到这些变量/值对时，就会认为它们是安全的。

你也可以在确认提示处键入 'i'，让 Emacs 永久忽略该文件中的所有变量/值对——此后这些对将在此文件以及所有其他文件中被忽略。

当 Emacs 请求确认是否设置目录局部变量时（参见"按目录的局部变量"），在确认提示处键入 '+' 会设置所有变量，并把该目录加入 'safe-local-variable-directories'（见下文）列表，这会使 Emacs 今后认为该目录可以安

全地加载任何目录局部变量。‘+’ 应答只应用于你信任其内容的目录。

有些变量，如 ‘load-path’，被认为是特别“危险的”：几乎没有任何理由把它们指定为局部变量，而改动它们可能很危险。如果文件中只有危险的局部变量，Emacs 在确认提示处既不提供也不接受 ‘!’ 作为输入。如果文件中的局部变量有些是危险的、有些只是可能不安全，你可以在提示处键入 ‘!’。它会应用所有变量，但只把非危险的变量标记为今后安全。如果你确实想为危险变量记录安全值，请直接通过定制 ‘safe-local-variable-values’ 来完成（参见“简易自定义界面”）。同样，如果你想记录应当被永久忽略的危险变量的值，请定制 ‘ignored-local-variable-values’。

有时，始终信任某些目录中的目录变量、并在从这些目录加载局部变量时跳过确认提示是有帮助的，即使这些变量是危险的。变量 ‘safe-local-variable-directories’ 保存这类目录的列表。该列表中目录的名字必须是完整的绝对文件名。如果变量 ‘enable-remote-dir-locals’ 的值非 ‘nil’，该列表也可以包含远程目录（参见“远程文件”）。

变量 ‘enable-local-variables’ 让你改变 Emacs 处理局部变量的方式。其默认值为 ‘t’，即上面描述的行为。若为 ‘nil’，Emacs 会直接忽略所有文件局部变量。‘:safe’ 表示只使用安全的值，忽略其余的值。‘:all’ 指示 Emacs 设置所有文件局部变量，不论其值是否安全（我们建议不要长期使用）。其他任何值都表示对每个含局部变量的文件都向你询问，而不去判断这些值是否已知安全。

变量 ‘enable-local-eval’ 控制 Emacs 是否处理 ‘eval’ 变量。该变量值的三种可能是 ‘t’、‘nil’ 和其他任何值，与 ‘enable-local-variables’ 相同。默认值为 ‘maybe’，它既不是 ‘t’ 也不是 ‘nil’，所以通常 Emacs 确实会就处理 ‘eval’ 变量请求确认。

作为例外，如果某个 ‘eval’ 形式出现在变量 ‘safe-local-eval-forms’ 中，Emacs 绝不会为求值该形式而请求确认。

51.2.5 按目录的局部变量

有时，你可能希望为某个目录及其子目录中的所有文件定义同一组局部变量，例如大型软件项目的目录树。这可以通过“目录局部变量”（directory-local variables）来实现。文件局部变量会覆盖目录局部变量，因此如果某个目录中的某些文件需要专门的设置，你可以在目录变量中为该目录的大多数文件指定设置，然后在少数需要覆盖通用设置的文件中定义文件局部变量。

定义目录局部变量的通常做法是把一个名为 ‘.dir-locals.el’(1) 的文件放入某个目录。每当 Emacs 访问该目录或其任何子目录中的文件时，它都会应用 ‘.dir-locals.el’ 中指定的目录局部变量，就像这些变量是作为该文件的文件局部变量定义的一样（参见“文件中的局部变量”）。Emacs 从所访问文件所在的目录开始搜索 ‘.dir-locals.el’，并沿目录树向上移动。为避免变慢，对远程文件会跳过这一搜索。如有需要，可以把变量 ‘enable-remote-dir-locals’ 设为 ‘t’，将搜索扩展到远程文件。

你也可以使用 ‘.dir-locals-2.el’；如果在与 ‘.dir-locals.el’ 相同的目录中找到它，Emacs 会在加载 ‘.dir-locals.el’ 的同时加载它。当 ‘.dir-locals.el’ 处于共享仓库的版本控制之下、无法用于个人定制时，这很有用。

‘.dir-locals.el’ 文件应当包含一个特殊构造的列表，它把主模式名（符号）映射到关联列表（参见《Emacs Lisp 参考手册》的“Association Lists”）。每个关联列表项由一个变量名和当指定主模式启用时要赋给该变量的目录局部值组成。除了模式名，你也可以指定 ‘nil’，表示该关联列表适用于任何模式；或者指定一个子目录（字符串），此时该关联列表适用于该子目录中的所有文件。

下面是一个‘.dir-locals.el’文件的例子：

```
((nil . ((indent-tabs-mode . t)
         (fill-column . 80)
         (mode . auto-fill)))
 (c-mode . ((c-file-style . "BSD")
            (subdirs . nil)))
 ("src/imported"
  . ((nil . ((change-log-default-name
              . "ChangeLog.local"))))))
```

这会为该目录树中的任何文件设置变量‘indent-tabs-mode’和‘fill-column’，并为任何 C 源文件设置缩进风格。特殊的‘mode’元素指定要启用的次模式。因此‘(mode . auto-fill)’指定需要启用次模式‘auto-fill-mode’。特殊的‘subdirs’元素不是变量，而是一个特殊关键字，表示 C 模式的设置只应用于当前目录，不应用于任何子目录。最后，它为‘src/imported’子目录中的任何文件指定了不同的‘ChangeLog’文件名。

如果‘.dir-locals.el’文件使用不同的模式名或目录为同一个变量包含多个不同的值，这些值将按这样的顺序应用：更具体的模式的值优先于更通用的模式的值。在某个目录下指定的值优先级更高。例如：

```
((nil . ((fill-column . 40)))
 (c-mode . ((fill-column . 50)))
 (prog-mode . ((fill-column . 60)))
 ("narrow-files" . ((nil . ((fill-column . 20))))))
```

使用‘c-mode’的文件也会匹配‘prog-mode’，因为前者继承自后者。但在 C 文件中用于‘fill-column’的值将是‘50’，因为该模式名比‘prog-mode’更具体。使用其他继承自‘prog-mode’的模式的文件将使用‘60’。‘narrow-files’目录下的任何文件都将使用值‘20’，即使它们使用‘c-mode’，因为目录条目优先于模式条目。

你可以在‘.dir-locals.el’中指定变量‘mode’、‘eval’和‘unibyte’，它们的含义与在文件局部变量中相同。‘coding’不能作为目录局部变量指定。参见“文件中的局部变量”。

‘.dir-locals.el’中的特殊键‘auto-mode-alist’让你设置文件的主模式。它的工作方式很像变量‘auto-mode-alist’（参见“选择文件模式”）。例如，下面这样告诉 Emacs：该目录中的‘.def’源文件应处于 C 模式：

```
((auto-mode-alist . (("\\.def\\'" . c-mode))))
```

除了手工编辑‘.dir-locals.el’文件，你还可以使用命令‘M-x add-dir-local-variable’。它会提示输入一个模式或子目录，以及变量和值，并添加定义该目录局部变量的条目。‘M-x delete-dir-local-variable’删除一个条目。‘M-x copy-file-locals-to-dir-locals’把当前文件中的文件局部变量复制到‘.dir-locals.el’，如果‘.dir-locals-2.el’也存在，则复制到后者。

带前缀参数时，这三个命令都会提示输入你想要修改的文件。该文件不必存在，但你必须输入有效的文件名，即‘.dir-locals.el’或‘.dir-locals-2.el’。

还有一个命令可以弹出 Easy Customization 缓冲区（参见“简易自定义界面”）来编辑目录局部变量：‘customize-dirlocals’。

另一种指定目录局部变量的方法是：使用‘dir-locals-set-class-variables’函数在“目录类”（directory class）中定义一组变量/值对；然后使用‘dir-locals-set-directory-class’函数告诉 Emacs 哪些目录对应该类。这些函数调用通

常放在你的初始化文件中（参见“Emacs 初始化文件”）。当你由于某种原因无法把 ‘.dir-locals.el’ 放入某个目录时，这种方法很有用。例如，你可以这样把设置应用到不可写的目录：

```
(dir-locals-set-class-variables 'unwritable-directory
   '((nil . ((some-useful-setting . value)))))
```

```
(dir-locals-set-directory-class
   "/usr/include/" 'unwritable-directory)
```

如果某个变量同时指定了目录局部值和文件局部值，以文件局部值生效。不安全的目录局部变量与不安全的文件局部变量以相同方式处理（参见“文件变量的安全性”）。

目录局部变量在某些并不直接访问文件、但在某个目录内执行工作的缓冲区中也会生效，例如 Dired 缓冲区（参见“Dired 目录编辑器”）。

- EditorConfig 支持:: 通过 EditorConfig 设置目录局部变量

> 注：在 MS-DOS 上，由于 DOS 文件系统的限制，该文件的名字应为 ‘_dir-locals.el’。如果文件系统限制为 8+3 文件名，操作系统会把该文件名截断为 ‘_dir-loc.el’。

51.2.5.1 通过 EditorConfig 设置目录局部变量

EditorConfig 标准（<https://editorconfig.org/>）是 ‘.dir-locals.el’ 文件的替代方案，它只能控制很少的几个变量，但优点是编辑器中立，并非 Emacs 专用。这些设置存放在名为 ‘.editorconfig’ 的文件中，并作用于该目录及其子目录中的文件。

如果你希望 Emacs 遵循 ‘.editorconfig’ 文件中的设置，需要启用全局次模式 ‘editorconfig-mode’。通常只需这样即可：模式激活后，每当访问文件时，Emacs 都会在该文件所在目录及其父目录中查找 ‘.editorconfig’，就像它对 ‘.dir-locals.el’ 所做的那样。当同时找到 ‘.editorconfig’ 和 ‘.dir-locals.el’ 文件时，它们的设置会被合并；若发生冲突，来自目录层次中更近的文件的设置优先。如果两者距离相同，则 ‘.dir-locals.el’ 优先。在安全性方面，‘.editorconfig’ 的设置要接受与来自 ‘.dir-locals.el’ 及文件局部变量的设置相同的检查（并且也遵循 ‘enable-local-variables’）。参见“文件变量的安全性”。

EditorConfig 标准的 ‘indent_size’ 设置并不对应 Emacs 中的某个固定变量，而是需要根据主模式设置不同的变量。理想情况下所有主模式都应设置相应的 ‘editorconfig-indent-size-vars’，但如果你使用的主模式尚未支持 ‘indent_size’、因而该设置不生效，可以定制变量 ‘editorconfig-indentation-alist’，告诉 Emacs 在该主模式下需要设置哪些变量。

类似地，删除行尾空白有好几种不同做法。当使用 EditorConfig 的 ‘trim_trailing_whitespace’ 设置时，默认情况下 ‘editorconfig-mode’ 只会在你每次保存文件时调用 ‘delete-trailing-whitespace’（参见“无用的空白”）。如果你希望采用其他行为，可以把 ‘editorconfig-trim-whitespaces-mode’ 定制为你偏好的次模式，例如 ‘ws-butler-mode’。

51.2.6 按连接的局部变量

大多数变量反映的是本地机器上的情况。通常，当你在默认目录为远程的缓冲区中操作时，它们必须使用不同的值。想想调用 ‘shell’ 时的行为——在本地机器上，你可能使用 ‘/bin/bash’ 并依赖 termcap，而在远程机器

上，可能是‘/bin/ksh’和 terminfo。

这可以通过“连接局部变量”（connection-local variables）来实现。这类变量根据当前缓冲区的‘default-directory’值来声明。当缓冲区的‘default-directory’是远程的，并且存在与‘default-directory’匹配的连接局部变量时，就使用该变量的这个替代值。目录局部变量和文件局部变量会覆盖连接局部变量。不安全的连接局部变量与不安全的文件局部变量以相同方式处理（参见“文件变量的安全性”）。

连接局部变量在“profile”中声明为一组变量/值对，使用‘connection-local-set-profile-variables’函数。函数‘connection-local-set-profiles’为给定的条件（第一个参数）声明 profile，该条件相对于当前缓冲区的‘default-directory’标识一台远程机器：

```
(connection-local-set-profile-variables 'remote-terminfo
   '((system-uses-terminfo . t)
     (comint-terminfo-terminal . "dumb-emacs-ansi")))
```

```
(connection-local-set-profile-variables 'remote-ksh
   '((shell-file-name . "/bin/ksh")
     (shell-command-switch . "-c")))
```

```
(connection-local-set-profile-variables 'remote-bash
   '((shell-file-name . "/bin/bash")
     (shell-command-switch . "-c")))
```

```
(connection-local-set-profiles
   '(:application tramp :machine "remotemachine")
   'remote-terminfo 'remote-ksh)
```

这段代码声明了三个不同的 profile：‘remote-terminfo’、‘remote-ksh’和‘remote-bash’。profile‘remote-terminfo’和‘remote-ksh’应用于所有其远程‘default-directory’以字符串‘"remotemachine"’作为主机名匹配的缓冲区。

条件（criteria）是‘connection-local-set-profiles’的第一个参数，它规定 profile 如何匹配‘default-directory’。它是一个 plist，用于标识一个连接以及使用该连接的应用程序。属性名可以是‘:application’、‘:protocol’、‘:user’和‘:machine’。‘:application’的属性值是一个符号，其他所有属性值都是字符串。一般而言应把符号‘tramp’用作‘:application’的值。有些包使用不同的‘:application’（例如‘eshell’或‘vc-git’），它们会在自己的文档中说明。所有属性都是可选的。

其他属性用于检查‘default-directory’。属性‘:protocol’用于匹配远程‘default-directory’所使用的方法，属性‘:user’是远程用户名，属性‘:machine’是远程主机名。所有检查都通过‘string-equal’进行。‘nil’条件匹配所有默认目录为远程的缓冲区。

连接局部变量默认不会激活。使用连接局部变量的包必须为给定缓冲区激活它们，并说明它为哪个‘:application’使用这些变量。详见参见《Emacs Lisp 参考手册》的“Applying Connection Local Variables”。

在上述定义并激活 profile 之后，Tramp 到‘remotemachine’系统的任何连接都将使用

- ‘t’作为‘system-uses-terminfo’的连接特定值，
- ‘dumb-emacs-ansi’作为‘comint-terminfo-terminal’的连接特定值，
- ‘/bin/ksh’作为‘shell-file-name’的连接特定值，

- ‘-c’ 作为 ‘shell-command-switch’ 的连接特定值。

声明使用同一变量的不同 profile、并把这些 profile 设给可能同时匹配的多组条件时要小心。此时会使用哪个变量值是不确定的。

在缓冲区中设置连接局部变量、而该缓冲区之后又改变了主模式时，也要小心。因为改变主模式时会杀掉所有缓冲区局部变量，连接局部变量的值会丢失。你可以通过把相应变量的 ‘permanent-local’ 符号属性设为非 ‘nil’ 来防止这一点。

51.3 定制键绑定

本节介绍“键绑定”（key binding），它把键映射到命令，还介绍“键位图”（keymap），它记录键绑定。本节也解释了如何定制键绑定，这通过编辑你的初始化文件来完成（参见“在初始化文件中重绑定按键”）。

由于大多数模式定义了自己的键绑定，激活某个模式可能会覆盖你自定义的键绑定。少数键保留给用户自定义的绑定，模式不应使用它们，因此使用这些键的键绑定在这方面更安全。保留的键序列是由 ‘C-c’ 后跟一个字母（大写或小写均可）组成的序列，以及不带修饰键的功能键 <F5> 到 <F9>（参见“修饰键”）。

- 禁用命令:: 禁用命令意味着在执行前需要确认。这样做是为了保护初学者免受意外。

51.3.1 键位图

正如参见“按键与命令”中所述，每个 Emacs 命令都是一个 Lisp 函数，其定义提供了交互使用的方式。与每个 Lisp 函数一样，命令有一个函数名，通常由小写字母和连字符组成。

“键序列”（key sequence，简称“键”）指作为一个整体有意义的一系列“输入事件”。输入事件包括字符、功能键和鼠标按钮——即你能发送给计算机的所有输入。键序列从它的“绑定”中获得含义，绑定说明它运行哪个命令。

键序列与命令函数之间的绑定记录在称为“键位图”（keymap）的数据结构中。Emacs 有许多这样的键位图，各自用于特定的场合。

“全局”键位图是最重要的键位图，因为它始终有效。全局键位图为 Fundamental 模式定义键（参见“主模式”）；其中大多数定义对多数或所有主模式通用。每个主模式或次模式都可以有自己的键位图，覆盖某些键的全局定义。

例如，‘g’ 这样的自插入字符之所以自插入，是因为全局键位图把它绑定到命令 ‘self-insert-command’。‘C-a’ 等标准 Emacs 编辑字符也从全局键位图获得其标准含义。重新绑定键的命令，例如 ‘M-x keymap-global-set’，通过把新绑定存入全局键位图中的适当位置来工作（参见“交互式更改键绑定”）。要查看当前的键绑定，使用 ‘C-h b’ 命令。

大多数现代键盘既有字符键也有功能键。功能键与字符键一样发送输入事件，键位图可以为它们提供绑定。键序列可以混用功能键和字符。例如，如果你的键盘有 <Home> 功能键，Emacs 能识别像 ‘C-x <Home>’ 这样的键序列。你甚至可以把鼠标事件与键盘事件混用，例如 ‘S-down-mouse-1’。

在文本终端上，输入一个功能键实际上会向计算机发送一串字符；该序列的具体细节取决于功能键和终端类型。（序列常常以 ‘<ESC> [’ 开头。）如果 Emacs 正确理解你的终端类型，它就会自动把这类序列当作单个输入事件处理。

由 'C-c' 后跟一个字母（大写或小写；ASCII 或非 ASCII）组成的键序列保留给用户使用。Emacs 自身绝不会绑定这些键序列，Emacs 扩展也应避免绑定它们。换言之，用户可以绑定 'C-c a' 或 'C-c ç' 之类的键序列，并依赖它们永远不会被其他 Emacs 绑定遮蔽。

51.3.2 前缀键位图

在内部，Emacs 的每个键位图只记录单个事件。解释由多个事件组成的键序列会涉及一串键位图：第一个键位图为第一个事件给出定义，该定义是另一个键位图，用它来查找序列中的第二个事件，依此类推。因此，像 'C-x' 或 <ESC> 这样的前缀键有自己的键位图，其中保存紧跟该前缀之后那个事件的定义。

前缀键的定义通常就是用于查找后续事件的键位图。该定义也可以是一个 Lisp 符号，其函数定义是后面的键位图；效果相同，但这样能为前缀键提供一个命令名，可用作该前缀键用途的说明。因此，'C-x' 的绑定是符号 'Control-X-prefix'，其函数定义是用于 'C-x' 命令的键位图。'C-c'、'C-x'、'C-h' 和 <ESC> 作为前缀键的定义出现在全局键位图中，所以这些前缀键总是可用的。

除普通前缀键之外，还有一个虚构的"前缀键"代表菜单栏；关于菜单栏键绑定的特殊信息，参见《Emacs Lisp 参考手册》的"Menu Bar"。调用弹出菜单的鼠标按钮事件也是前缀键；详情参见《Emacs Lisp 参考手册》的"Menu Keymaps"。

有些前缀键位图保存在变量中，其名字如下：

- 'ctl-x-map' 是用于 'C-x' 之后字符的键位图的变量名。
- 'help-map' 用于 'C-h' 之后的字符。
- 'esc-map' 用于 <ESC> 之后的字符。因此，所有 Meta 字符实际上都由这个键位图定义。
- 'ctl-x-4-map' 用于 'C-x 4' 之后的字符。
- 'mode-specific-map' 用于 'C-c' 之后的字符。
- 'project-prefix-map' 用于 'C-x p' 之后的字符，供与项目相关的命令使用（参见"使用项目"）。

51.3.3 局部键位图

到目前为止，我们已解释了全局键位图的方方面面。主模式通过在"局部键位图"中提供自己的键绑定来定制 Emacs。例如，C 模式重新定义了 <TAB>，使其为 C 代码缩进当前行。次模式也可以有局部键位图；只要某个次模式生效，其键位图中的定义就会覆盖主模式的局部键位图和全局键位图。此外，缓冲区中的部分文本可以指定自己的键位图，其优先级高于所有其他键位图。

局部键位图可以把某个键重新定义为前缀键，方法是将它定义为一个前缀键位图。如果该键在全局也被定义为前缀，那么它的局部定义和全局定义（两个键位图）实际上会合并：两种定义都用于查找该前缀键之后的那个事件。例如，若某个局部键位图把 'C-c' 定义为前缀键位图，而该键位图又把 'C-z' 定义为命令，这就为 'C-c C-z' 提供了局部含义。这不会影响其他以 'C-c' 开头的序列；如果那些序列没有自己的局部绑定，它们的全局绑定仍然有效。

另一种理解方式是：Emacs 处理多事件键序列时，会逐个在若干键位图中查找整个键序列的绑定。它先检查已启用的次模式的键位图，然后检查主模式的键位图，最后检查全局键位图。这并非键查找的精确工作方式，但对于理解一般情形下的结果已经足够。

51.3.4 迷你缓冲区键位图

迷你缓冲区有自己的一套局部键位图；其中包含各种补全和退出命令。

- ‘minibuffer-local-map’ 用于普通输入（不做补全）。
- ‘minibuffer-local-ns-map’ 类似，区别在于 <SPC> 会像 <RET> 一样退出。
- ‘minibuffer-local-completion-map’ 用于宽松补全。
- ‘minibuffer-local-must-match-map’ 用于严格补全和谨慎补全。
- ‘minibuffer-local-filename-completion-map’ 与前两者类似，但专门用于文件名补全。它不绑定 <SPC>。

默认情况下，<TAB>、<SPC> 和 <?> 在 ‘minibuffer-local-completion-map’ 中执行补全。如果你经常对包含空格或问号字符元素的集合进行补全，那么把下面这段代码放进初始化文件来禁用这些键上的补全可能会很方便：

```
(keymap-set minibuffer-local-completion-map "SPC" 'self-insert-command)
(keymap-set minibuffer-local-completion-map "?" 'self-insert-command)
```

51.3.5 交互式更改键绑定

重新定义 Emacs 按键的方法是更改它在键位图中的条目。你可以更改全局键位图，这样更改会在所有主模式中生效（那些为同一按键提供了自己的覆盖性局部绑定的主模式除外）。你也可以更改局部键位图，这会影响使用同一主模式的所有缓冲区。

本节介绍如何为当前 Emacs 会话重绑定按键。关于如何让键重绑定影响以后的 Emacs 会话，参见“在初始化文件中重绑定按键”。

- ‘M-x keymap-global-set <RET> KEY CMD <RET>’ 全局地把 KEY 定义为运行 CMD。
- ‘M-x keymap-local-set <RET> KEY CMD <RET>’ 局部地（在当前生效的主模式中）把 KEY 定义为运行 CMD。
- ‘M-x keymap-global-unset <RET> KEY’ 在全局键位图中让 KEY 未定义。
- ‘M-x keymap-local-unset <RET> KEY’ 局部地（在当前生效的主模式中）让 KEY 未定义。

例如，下面把 ‘C-z’ 绑定到 ‘shell’ 命令（参见“交互式子 shell”），替换 ‘C-z’ 通常的全局定义：

```
M-x keymap-global-set <RET> C-z shell <RET>
```

‘keymap-global-set’ 命令在按键之后读取命令名。按下该键后，会出现类似下面的消息，以便你确认自己绑定的正是想要的键：

```
Set key C-z to command:
```

你可以用同样的方式重新定义功能键和鼠标事件；只需在需要指定要重绑定的键时按下该功能键或点击鼠标。

你也可以用同样的方式重绑定包含多个事件的键。Emacs 会持续读取要重绑定的键，直到它成为完整键（即不是前缀键）。因此，如果你为 KEY 输入 ‘C-f’，就到此结束；它会立即进入迷你缓冲区读取 CMD。但如果你

输入‘C-x’，由于它是前缀，它会再读一个字符；如果那是‘4’，又是一个前缀字符，它会再读一个字符，如此继续。例如，

```
M-x keymap-global-set <RET> C-x 4 $ spell-other-window <RET>
```

把‘C-x 4 $’重新定义为运行（虚构的）命令‘spell-other-window’。

你可以用‘keymap-global-unset’移除某个键的全局定义。这会让该键“未定义”；如果你输入它，Emacs 只会响铃。同样，‘keymap-local-unset’会在当前主模式的键位图中让某个键未定义，从而使全局定义（或全局未定义的状态）在该主模式中重新生效。

如果你重新定义（或取消定义）了某个键，之后又想撤回更改，取消定义该键并不能做到这一点——你需要用它的标准定义重新定义该键。要查找某个键标准定义的名字，请在一个全新的 Emacs 中进入 Fundamental 模式缓冲区，并使用‘C-h c’。本手册中关于键的文档也列出了它们的命令名。

如果你想防止自己误调用某个命令，那么禁用该命令比取消定义该键更好。真正想用的时候，被禁用的命令调用起来更省事。参见“禁用命令”。

51.3.6 在初始化文件中重绑定按键

如果你有一组希望一直使用的键绑定，可以在初始化文件中用 Lisp 代码指定它们。关于初始化文件的说明，参见“Emacs 初始化文件”。

用 Lisp 编写键绑定的推荐方式是使用‘keymap-global-set’或‘keymap-set’函数。例如，下面是在全局键位图中把‘C-z’绑定到‘shell’命令的方法（参见“交互式子 shell”）：

```
(keymap-global-set "C-z" 'shell)
```

‘keymap-global-set’的第一个参数描述键序列。它是一个由一系列字符组成的字符串，字符之间以空格分隔，每个字符对应一个键。带修饰符的键可以通过在前面加上修饰符来指定，例如‘C-’表示 Control，‘M-’表示 Meta。特殊键（如 <TAB> 和 <RET>）可以像‘<TAB>’和‘<RET>’那样写在尖括号中。

命令名前的单引号（上例中的‘shell’）把它标记为常量符号而不是变量。如果省略引号，Emacs 会把‘shell’当作变量求值。这很可能导致错误；那肯定不是你想要的。

下面还有一些示例，包括绑定功能键和鼠标事件：

```
(keymap-global-set "C-c y" 'clipboard-yank)
(keymap-global-set "C-M-q" 'query-replace)
(keymap-global-set "<f5>" 'flyspell-mode)
(keymap-global-set "C-<f5>" 'display-line-numbers-mode)
(keymap-global-set "C-<right>" 'forward-sentence)
(keymap-global-set "<mouse-2>" 'mouse-save-then-kill)
```

键序列也可以直接绑定到 Lisp 字符串，而不是绑定到命令。这类字符串使用与键序列相同的语法书写。例如，把‘C-c h’绑定到字符串‘hello’：

```
(keymap-global-set "C-c h" "h e l l o")
```

由于这样写有些繁琐，可以改用便捷函数 'key-description'：

```
(keymap-global-set "C-c h" (key-description "hello"))
```

非 ASCII 字符可以直接写在字符串中。例如，要绑定到 'olá'，使用：

```
(keymap-global-set "C-c h" (key-description "olá"))
```

然而要注意，语言和编码系统可能会给非 ASCII 字符的键绑定带来问题（参见"初始化文件中的非 ASCII 字符"）。直接用 Unicode 码点写出绑定可以避免这些问题（关于如何在 Emacs 内确定字符的码点，参见"国际字符集简介"）：

```
(keymap-global-set "C-c h" (key-description "ol\u00E1"))
```

或者，你也可以使用底层函数 'define-key' 和 'global-set-key'。例如，像上面的例子那样用这些底层函数把 'C-z' 绑定到 'shell' 命令，使用：

```
(global-set-key (kbd "C-z") 'shell)
```

指定键序列有多种方式，但最简单的是像上例那样使用函数 'kbd'。'kbd' 接受一个字符串参数，它是键序列的文本表示，并将其转换为适合 'global-set-key' 等底层函数的形式。关于用 Lisp 绑定键的更多细节，参见《Emacs Lisp 参考手册》的"Keymaps"。

如参见"局部键位图"所述，主模式和次模式可以定义局部键位图。这些键位图在模式于某个会话中首次加载时构建。函数 'keymap-set' 可用于在特定键位图中进行更改。要移除某个键绑定，使用 'keymap-unset'。

由于模式的键位图要等到该模式加载后才会构建，你必须延迟运行修改它们的代码，例如把它放到"模式钩子"上（参见"钩子"）。例如，Texinfo 模式会运行钩子 'texinfo-mode-hook'。下面演示如何利用该钩子为 'C-c n' 和 'C-c p' 添加局部绑定，并移除 Texinfo 模式中 'C-c C-x x' 的绑定：

```
(add-hook 'texinfo-mode-hook
          (lambda ()
            (keymap-set texinfo-mode-map "C-c p"
                        'backward-paragraph)
            (keymap-set texinfo-mode-map "C-c n"
                        'forward-paragraph)
            (keymap-set texinfo-mode-map "C-c C-x x" nil)))
```

51.3.7 修饰键

Emacs 中的默认键绑定经过设置，使得带修饰的字母字符不区分大小写。换言之，'C-A' 与 'C-a' 作用相同，'M-A' 与 'M-a' 作用相同。这仅涉及字母字符，不适用于其他键的加 Shift 版本；例如，'C-@' 与 'C-2' 并不相同。

带 <Control> 修饰的字母字符通常被视为不区分大小写：Emacs 总是把 'C-A' 当作 'C-a'，把 'C-B' 当作 'C-b'，等等。其原因在于历史：在非图形环境中，这些击键之间没有区别。不过，你可以在图形框架中绑定带 Shift 的 <Control> 字母击键：

```
(keymap-global-set "C-S-n" #'previous-line)
```

对于所有其他修饰符，你在定制 Emacs 时可以让带修饰的字母字符区分大小写（即使在非图形框架上）。例如，你可以让 'M-a' 和 'M-A' 运行不同的命令。

虽然常用的修饰键只有 <Control> 和 <Meta>，Emacs 还支持另外三个修饰键。它们称为 <Super>、<Hyper> 和 <Alt>。很少有终端提供使用这些修饰符的方式；大多数键盘上标着 <Alt> 的键通常发出 <Meta> 修饰符，而不是 <Alt>。Emacs 的标准键绑定不包含任何带 <Super> 和 <Hyper> 修饰符的字符，只有少数标准键绑定使用 <Alt>。不过，你可以定制 Emacs，为使用这些修饰符的键绑定赋予含义。相应的修饰位分别标记为 's-'、'H-' 和 'A-'。

即使你的键盘缺少这些额外的修饰键，你也可以用 'C-x @' 输入它们：'C-x @ h' 给下一个字符加上 Hyper 标志，'C-x @ s' 加上 Super 标志，'C-x @ a' 加上 Alt 标志。例如，'C-x @ h C-a' 是输入 'Hyper-Control-a' 的一种方式。（遗憾的是，无法通过对同一字符使用两次 'C-x @' 来添加两个修饰符，因为第一个会作用于 'C-x'。）类似地，你可以分别用 'C-x @ S'、'C-x @ c' 和 'C-x @ m' 输入 Shift、Control 和 Meta 修饰符，尽管很少需要这样做。

在图形终端上，你可以启用 Modifier Bar 模式，它允许通过点击工具栏按钮来模拟缺失的修饰键。参见"工具栏"。

51.3.8 重绑定功能键

键序列可以包含功能键，也可以包含普通字符。正如 Lisp 字符（实际上是整数）表示键盘字符一样，Lisp 符号表示功能键。如果功能键的标签是一个单词，那么该单词也是对应 Lisp 符号的名字。以下是常见功能键的惯用 Lisp 名字：

- 'left'、'up'、'right'、'down' 光标方向键。
- 'begin'、'end'、'home'、'next'、'prior' 其他光标重定位键。
- 'select'、'print'、'execute'、'backtab'、'insert'、'undo'、'redo'、'clearline'、'insertline'、'deleteline'、'insertchar'、'deletechar' 杂项功能键。
- 'f1'、'f2'、…… 'f35' 编号功能键（位于键盘顶部）。
- 'kp-add'、'kp-subtract'、'kp-multiply'、'kp-divide'、'kp-backtab'、'kp-space'、'kp-tab'、'kp-enter'、'kp-separator'、'kp-decimal'、'kp-equal'、'kp-prior'、'kp-next'、'kp-end'、'kp-home'、'kp-left'、'kp-up'、'kp-right'、'kp-down'、'kp-insert '、'kp-delete' 带名字或标点的小键盘键（位于常规键盘右侧）。
- 'kp-0'、'kp-1'、…… 'kp-9' 带数字的小键盘键。
- 'kp-f1'、'kp-f2'、'kp-f3'、'kp-f4' 小键盘 PF 键。

这些名字是惯用名，但某些系统（特别是在使用 X 时）可能使用不同的名字。要确定你的终端上某个功能键使用什么符号，输入 'C-h c' 后跟该键。

关于绑定功能键的示例，参见"在初始化文件中重绑定按键"。

许多键盘右侧有数字小键盘。小键盘中的数字键兼作光标移动键，由标着 'Num Lock' 的键切换。默认情况下，Emacs 会把这些键转换为标准键盘上对应的键。例如，当 'Num Lock' 打开时，数字小键盘上标着 '8' 的键

产生 'kp-8'，它被转换为 '8'；当 'Num Lock' 关闭时，同一个键产生 'kp-up'，它被转换为 <UP>。如果你重新绑定 '8' 或 <UP> 之类的键，等效的小键盘键也会受影响。然而，如果你直接重绑定某个 'kp-' 键，则不会影响其非小键盘的等效键。注意，带修饰符的键不会被转换：例如，如果你按住 <Meta> 键的同时按数字小键盘上的 '8' 键，产生的是 'M-<kp-8>'。

Emacs 提供了一种便捷的方法来绑定数字小键盘键，使用变量 'keypad-setup'、'keypad-numlock-setup'、'keypad-shifted-setup' 和 'keypad-numlock-shifted-setup'。这些可以在 'keyboard' 定制组中找到（参见"简易自定义界面"）。你可以把这些键重绑定为执行其他任务，例如发出数字前缀参数。

51.3.9 命名的 ASCII 控制字符

<TAB>、<RET>、<BS>、<LFD>、<ESC> 和 <DEL> 起初是某些 ASCII 控制字符的名字，它们使用得如此频繁，以至于有了自己的专用键。例如，<TAB> 曾是 'C-i' 的另一个名字。后来，用户发现在 Emacs 中区分这些键与用 <Ctrl> 键输入的相应控制字符很方便。因此，在大多数现代终端上，它们不再相同：<TAB> 与 'C-i' 不同。

如果键盘能区分，Emacs 就能区分这两类输入。它把特殊键当作名为 'tab'、'return'、'backspace'、'linefeed'、'escape' 和 'delete' 的功能键。仅在它们没有自己的绑定时，这些功能键才会自动转换为相应的 ASCII 字符。因此，用户和 Lisp 程序都无需关注这种区别，除非它们愿意。

如果你不想区分（例如）<TAB> 和 'C-i'，只给 ASCII 字符 <TAB>（八进制码 011）做一个绑定即可。如果你确实想区分，就为这个 ASCII 字符做一个绑定，再为功能键 'tab' 做一个绑定。

使用普通的 ASCII 终端时，无法区分 <TAB> 和 'C-i'（其他类似配对也是如此），因为终端在两种情况下发送相同的字符。

51.3.10 重绑定鼠标按钮

Emacs 也用 Lisp 符号表示鼠标按钮。Emacs 中普通的鼠标事件是"点击"事件；当你按下按钮并在不移动鼠标的情况下松开时，就发生这类事件。你也可以获得"拖动"事件，即按住按钮的同时移动鼠标。拖动事件在你最终松开按钮时发生。

基本点击事件的符号是 'mouse-1' 表示最左边的按钮，'mouse-2' 表示下一个，依此类推。下面演示如何重新定义第二个鼠标按钮来拆分当前窗口：

```
(keymap-global-set "<mouse-2>" 'split-window-below)
```

拖动事件的符号类似，但在 'mouse' 一词前加上前缀 'drag-'。例如，拖动第一个按钮产生 'drag-mouse-1' 事件。

你也可以为鼠标按钮按下时发生的事件定义绑定。这些事件以 'down-' 而不是 'drag-' 开头。只有当它们有键绑定时才会生成。当你收到按钮按下事件时，总会跟随一个相应的点击或拖动事件。

如果愿意，你可以区分单击、双击和三击。双击指在大致相同的位置点击鼠标按钮两次。第一次点击产生普通点击事件。第二次点击如果来得足够快，就改为产生双击事件。双击事件的事件类型以 'double-' 开头：例如，'double-mouse-3'。

这意味着你可以为同一位置的第二次点击赋予特殊含义，但它必须在假定收到第一次点击时已运行普通单击定义的前提下行事。

这限制了你用双击能做的事，但用户界面设计者说无论如何都应遵循这一约束。双击应该做与单击类似的事，只是做得更多。双击事件的命令应执行双击所需的那部分额外工作。

如果双击事件没有绑定，它就变成相应的单击事件。因此，如果你没有专门定义某个双击，它就会执行两次单击命令。

Emacs 还支持名字以 'triple-' 开头的三击事件。Emacs 不把四击区分为单独的事件类型；第三次之后的点击会产生额外的三击事件。不过，完整的点击次数记录在事件列表中，所以如果你懂 Emacs Lisp，可以在确实需要时加以区分（参见《Emacs Lisp 参考手册》的"Click Events"）。我们不建议为三次以上的点击赋予不同含义，但有时让后续点击循环通过同样的三种含义很有用，这样四次点击等价于一次，五次等价于两次，六次等价于三次。

Emacs 还会在拖动和按钮按下事件中记录多次按压。例如，当你按某个按钮两次，然后按住它移动鼠标，Emacs 会得到一个 'double-drag-' 事件。而在你第二次按下它的那一刻，Emacs 会得到一个 'double-down-' 事件（与所有按钮按下事件一样，如果它没有绑定就会被忽略）。

变量 'double-click-time' 指定两次点击之间可以流逝多长时间，仍能把它们归为多次点击。其值以毫秒为单位。如果值为 'nil'，则完全不检测双击。如果值为 't'，则没有时间限制。默认值是 500。

变量 'double-click-fuzz' 指定两次点击之间鼠标可以移动多远，仍能把它们归为多次点击。在有窗口的显示器上其值以像素为单位，在文本模式终端上以字符单元的 1/8 为单位；默认值是 3。

鼠标事件的符号也指示修饰键的状态，使用常见的前缀 'C-'、'M-'、'H-'、's-'、'A-' 和 'S-'。这些前缀总是位于 'double-' 或 'triple-' 之前，而后者总是位于 'drag-' 或 'down-' 之前。

框架包含一些不显示缓冲区文本的区域，例如模式行和滚动条。借助虚拟（dummy）前缀键，你可以判断某个鼠标按键是否来自屏幕的特殊区域。例如，在模式行中单击鼠标时，你会先得到前缀键 'mode-line'，然后才是普通的鼠标按键符号。因此，下面就是定义"在模式行中单击第一个按键时运行 'scroll-up-command'"的命令的方法：

```
(keymap-global-set "<mode-line> <mouse-1>" 'scroll-up-command)
```

下面是这些虚拟前缀键的完整列表及其含义：

- 'mode-line'：鼠标位于某窗口的模式行中。
- 'vertical-line'：鼠标位于分隔并排窗口的竖线上。（若你使用滚动条，滚动条会出现在这些竖线的位置。）
- 'vertical-scroll-bar'：鼠标位于垂直滚动条中。（这是 Emacs 目前支持的唯一一种滚动条。）
- 'menu-bar'：鼠标位于菜单栏中。
- 'tab-bar'：鼠标位于标签栏中。
- 'tab-line'：鼠标位于标签行中。
- 'header-line'：鼠标位于标题行中。

一个键序列中可以包含多个鼠标按键，但通常不这样做。

51.3.11 禁用命令

禁用（disabling）一条命令意味着以交互方式调用它时会请求用户确认。禁用命令的目的是防止用户误执行它；我们对那些可能让初学者感到困惑的命令这样做。

在 Emacs 中交互式地尝试调用被禁用的命令时，会显示一个窗口，其中包含该命令的名称、它的文档，以及关于接下来该怎么做的一些说明；然后 Emacs 询问输入，说明是按请求执行该命令、先启用它再执行，还是取消。若你决定启用该命令，还必须回答另一个问题——是永久这样做，还是仅在当前会话中这样做。（永久启用是通过自动编辑你的初始化文件来实现的。）你也可以输入 '!' 来启用*所有*命令，但仅限当前会话。

直接禁用命令的机制，是给该命令对应的 Lisp 符号加上非 'nil' 的 'disabled' 属性。下面是实现这一点的 Lisp 程序：

```
(put 'delete-region 'disabled t)
```

若 'disabled' 属性的值是一个字符串，该字符串会被包含在命令被使用时所显示的消息中：

```
(put 'delete-region 'disabled
     "It's better to use `kill-region' instead.\n")
```

作为比禁用命令更温和的替代方案，你可能希望在某条命令执行之前先被询问。例如，要在执行 'M->'（'end-of-buffer'）命令之前先被询问，可以在初始化文件中放入类似下面这样的内容：

```
(command-query
 'end-of-buffer
 "Do you really want to go to the end of the buffer?")
```

默认情况下，会以 'y'/'n' 提问的方式询问你，但若给第三个可选参数一个非 'nil' 值，就会改用 'yes'/'no' 来询问。

你可以直接编辑初始化文件来禁用某条命令，也可以使用命令 'M-x disable-command'，它会替你编辑初始化文件。同样，'M-x enable-command' 会编辑初始化文件以永久启用某条命令。参见"Emacs 初始化文件"。

若 Emacs 是以 '-q' 或 '--no-init-file' 选项启动的（参见"初始选项"），它就不会编辑你的初始化文件。这样做可能丢失信息，因为 Emacs 尚未读取你的初始化文件。

某条命令是否被禁用，与用哪个键调用它无关；用 'M-x' 调用时禁用同样生效。不过，禁用命令对从 Lisp 程序中把该命令作为函数调用没有影响。

51.4 Emacs 初始化文件

Emacs 启动时，通常尝试从"初始化文件"（initialization file，简称"init file"）加载一段 Lisp 程序。该文件若存在，就规定了如何为你初始化 Emacs，以及如何定制它的各种可选特性。你通过 'M-x customize' 所做的定制（参见"简易自定义界面"）默认也保存在初始化文件中（参见"保存自定义"）。

传统上，文件 '~/.emacs' 被用作初始化文件，不过 Emacs 也会查看 '~/.emacs.el'、'~/.emacs.d/init.el'、'~/.config/emacs/init.el' 或其他位置。参见"Emacs 如何查找你的初始化文件"。

你可能会觉得把所有 Emacs 配置放在一个目录中比较方便，这种情况下应使用 '~/.emacs.d/init.el'，或兼容 XDG 的 '~/.config/emacs/init.el'。

你可以用命令行开关 '-q' 阻止加载你的初始化文件，用 '-u'（或 '--user'）指定其他用户的初始化文件（参见"初始选项"）。

还可能存在一个"默认初始化文件"，即名为 'default.el' 的库，通过库的标准搜索路径找到。Emacs 发行版中不含这样的库；你的站点可以创建一个用于本地定制。若该库存在，则每次启动 Emacs 时都会加载它（用 '-q' 时除外）。但你的初始化文件（若有）会先加载；若它把 'inhibit-default-init' 设为非 'nil'，就不会加载 'default'。

你的站点也可能有一个"站点启动文件"；如果存在，它名为 'site-start.el'。与 'default.el' 一样，Emacs 通过 Lisp 库的标准搜索路径找到该文件。Emacs 在加载你的初始化文件之前加载这个库。要禁止加载该库，请使用选项 '--no-site-file'。参见"初始选项"。我们不建议用 'site-start.el' 来做某些用户可能不喜欢的改动。最好把它们放在 'default.el' 中，这样用户更容易覆盖它们。

你可以把 'default.el' 和 'site-start.el' 放在 Emacs 搜索 Lisp 库的任意目录中。变量 'load-path'（参见"Emacs 的 Lisp 代码库"）指定这些目录。许多站点把这些文件放在 Emacs 安装目录下名为 'site-lisp' 的子目录中，例如 '/usr/local/share/emacs/site-lisp'。

不建议对初始化文件做字节编译（参见《Emacs Lisp 参考手册》的"Byte Compilation"）。它通常不会使启动快多少，而且当你忘记重新编译该文件时常常导致问题。更好的办法是使用 Emacs 服务器来减少启动 Emacs 的次数（参见"把 Emacs 用作服务器"）。若你的初始化文件定义了许多函数，可考虑把它们移到一个单独的（经字节编译的）文件中，再在你的初始化文件中加载它。

如果你打算编写超出小型定制范围的真正 Emacs Lisp 程序，就应当阅读 'Emacs Lisp Reference Manual'。参见《Emacs Lisp 参考手册》。

51.4.1 初始化文件语法

初始化文件中包含一个或多个 Lisp 表达式。每个表达式由一个函数名后跟参数构成，整体用圆括号包围。例如，'(setq fill-column 60)' 调用函数 'setq'，把变量 'fill-column'（参见"填充文本"）设为 60。

你可以用 'setq' 设置任何 Lisp 变量，但对某些变量来说，在初始化文件中用 'setq' 并不会得到你可能想要的效果。有些变量在被 'setq' 设置时会自动变成缓冲区局部变量；在初始化文件中你要做的是设置默认值，应使用 'setq-default'。（下一节给出了这两种方法的示例。）

有些可定制的次模式变量在你用 Customize 设置它们时会做特殊处理以启用该模式，但普通的 'setq' 不会这样做；要在初始化文件中启用该模式，请调用对应的次模式命令。最后，少数可定制的用户选项是以复杂方式初始化的，这些必须通过 customize 界面（参见"自定义"）设置，或使用 'customize-set-variable'/'setopt'（参见"查看与设置变量"）设置。

'setq' 的第二个参数是表示变量新值的表达式。它可以是一个常量、一个变量或一个函数调用表达式。在初始化文件中，大多数时候使用常量。常量可以是：

- 数字：数字用十进制书写，可以带一个开头的负号。

- 字符串：Lisp 字符串语法与 C 字符串语法相同，只是多了少数特性。用双引号字符开始和结束字符串常量。

在字符串中，你可以原样包含换行符和特殊字符。但通常用反斜杠序列表示它们更清晰：‘\n’ 表示换行，‘\b’ 表示退格，‘\r’ 表示回车，‘\t’ 表示制表符，‘\f’ 表示换页（control-L），‘\e’ 表示 escape，‘\\’ 表示反斜杠，‘\"’ 表示双引号，而 ‘\OOO’ 表示八进制编码为 OOO 的字符。反斜杠和双引号是仅有的必须使用反斜杠序列的字符。

‘\C-’ 可用作控制字符的前缀，如 ‘\C-s’ 表示 ASCII 的 control-S；‘\M-’ 可用作 Meta 字符的前缀，如 ‘\M-a’ 表示 ‘<Meta>-A’，‘\M-\C-a’ 表示 ‘<Ctrl>-<Meta>-A’。

关于在初始化文件中包含非 ASCII 字符的信息，参见“初始化文件中的非 ASCII 字符”。

- 字符：Lisp 字符常量的语法是一个 ‘?’ 后跟一个字符或以 ‘\’ 开头的转义序列。示例：‘?x’、‘?\n’、‘?\"’、‘?\)’。注意在 Lisp 中字符串和字符不能互换；有些场合需要这一种，有些场合需要那一种。

关于把命令绑定到发送非 ASCII 字符的按键的信息，参见“初始化文件中的非 ASCII 字符”。

- 真：‘t’ 表示 “true”。
- 假：‘nil’ 表示 “false”。
- 其他 Lisp 对象：写一个单引号（‘'’），后跟你想要的那个 Lisp 对象。

关于 Emacs Lisp 语法的更多信息，参见《Emacs Lisp 参考手册》的“Introduction”。

51.4.2 初始化文件示例

下面是一些用 Lisp 表达式完成某些常见需求的示例：

- 把一个目录加入变量 ‘load-path’。之后你就可以把 Emacs 未随附的 Lisp 库放在该目录中，并用 ‘M-x load-library’ 加载它们。参见“Emacs 的 Lisp 代码库”。

```
(add-to-list 'load-path "/path/to/lisp/libraries")
```

- 让 C 模式下的 <TAB> 在光标点位于行中间时只插入一个制表符。

```
(setq c-tab-always-indent nil)
```

这里的变量通常取值 ‘t’ 表示 “true”，另一种取值是 ‘nil’ 表示 “false”。

- 让搜索默认区分大小写（在所有不覆盖此设置的缓冲区中）。

```
(setq-default case-fold-search nil)
```

这样设置的是默认值，它对所有没有该变量局部值的缓冲区都有效（*note Locals::）。用 ‘setq’ 设置 ‘case-fold-search’ 只影响当前缓冲区的局部值，这大概不是你在初始化文件中想做的。

- 指定你自己的邮件地址（若 Emacs 无法正确推断它）。

```
(setq user-mail-address "cheney@torture.gov")
```

各种 Emacs 包，例如 Message 模式，在需要知道你的邮件地址时会查询 ‘user-mail-address’。参见“邮件头字段”。

- 让文本模式成为新缓冲区的默认模式。

```
(setq-default major-mode 'text-mode)
```

注意这里用的是 ‘text-mode’，因为它是进入文本模式的命令。它前面的单引号使该符号成为常量；否则 ‘text-mode’ 会被当作变量名。

- 为 Latin-1 字符集设置默认值，它支持西欧的大多数语言。

```
(set-language-environment "Latin-1")
```

- 关闭行号模式（Line Number mode），这是一个全局次模式。

```
(line-number-mode 0)
```

- 在文本模式及相关模式中自动打开自动填充模式（参见“钩子”）。

```
(add-hook 'text-mode-hook 'auto-fill-mode)
```

- 更改使用剪贴板时所用的编码系统（参见“用于进程间通信的编码系统”）。

```
(setopt selection-coding-system 'utf-8)
```

- 加载名为 ‘foo’ 的已安装 Lisp 库（实际是标准 Emacs 目录中的文件 ‘foo.elc’ 或 ‘foo.el’）。

```
(load "foo")
```

当 ‘load’ 的参数是不以 ‘/’ 或 ‘~’ 开头的相对文件名时，‘load’ 会搜索 ‘load-path’ 中的目录（*note Lisp Libraries::）。

- 从你的主目录加载已编译的 Lisp 文件 ‘foo.elc’。

```
(load "~/foo.elc")
```

这里用的是完整文件名，因此不做搜索。

- 让 Emacs 通过加载名为 ‘mypackage’ 的 Lisp 库（即文件 ‘mypackage.elc’ 或 ‘mypackage.el’）来找到函数 ‘myfunction’ 的定义：

```
(autoload 'myfunction "mypackage" "Do what I say." t)
```

这里的字符串 “'Do what I say.'” 是该函数的文档字符串。你在 ‘autoload’ 定义中指定它，这样即使该包尚未加载，帮助命令也能用到它。最后一个参数 ‘t’ 表明该函数是交互式的；也就是说，可以通过输入 ‘M-x myfunction <RET>’ 或把它绑定到某个键来交互调用它。若该函数不是交互式的，省略 ‘t’ 或改用 ‘nil’。

- 把键 ‘C-x l’ 重新绑定为运行函数 ‘make-symbolic-link’（参见“在初始化文件中重绑定按键”）。

```
(keymap-global-set "C-x l" 'make-symbolic-link)
```

或者

```
(keymap-set global-map "C-x l" 'make-symbolic-link)
```

再次注意这里使用了单引号，以引用符号‘make-symbolic-link’，而不是它作为变量的值。

- 只在 Lisp 模式下做同样的事。

```
(keymap-set lisp-mode-map "C-x l" 'make-symbolic-link)
```

- 重新定义 Fundamental 模式下当前运行‘next-line’的所有键，使它们改为运行‘forward-line’。

```
(keymap-substitute global-map 'next-line 'forward-line)
```

- 使‘C-x C-v’未定义。

```
(keymap-global-unset "C-x C-v")
```

让某个键未定义的一个理由是，这样你可以把它变成前缀键。只要定义‘C-x C-v ANYTHING’，就会使‘C-x C-v’成为前缀键，但必须先解除‘C-x C-v’通常具有的非前缀定义。

- 让‘$’在文本模式下具有标点语法。注意这里对‘$’使用了字符常量。

```
(modify-syntax-entry ?\$ "." text-mode-syntax-table)
```

- 启用命令‘narrow-to-region’而不需确认。

```
(put 'narrow-to-region 'disabled nil)
```

- 根据各种平台和 Emacs 版本调整配置。

用户通常希望 Emacs 在所有系统上行为一致，因此同一个初始化文件适合所有平台。不过，有时你用来定制 Emacs 的某个函数在某些平台上或较旧的 Emacs 版本中不可用。要处理这种情况，可把定制放在一个检测该函数或设施是否可用的条件之中，像这样：

```
(if (fboundp 'blink-cursor-mode)
    (blink-cursor-mode 0))
```

```
(if (boundp 'coding-category-utf-8)
    (set-coding-priority '(coding-category-utf-8)))
```

你也可以干脆忽略在函数未定义时产生的错误。

```
(ignore-errors (set-face-background 'region "grey75"))
```

对不存在的变量执行‘setq’通常无害，因此那些不需要条件判断。

- 使用‘use-package’自动加载并配置某个包。

```
(use-package hi-lock
  :defer t
  :init (add-hook 'some-hook 'hi-lock-mode)
  :config (use-package my-hi-lock)
  :bind (("M-o l" . highlight-lines-matching-regexp)
         ("M-o r" . highlight-regexp)
         ("M-o w" . highlight-phrase)))
```

这会在首次使用‘hi-lock’的某些命令或变量时加载‘hi-lock’，把 3 个键绑定到它的命令，并在加载‘hi-lock’之后额外加载‘my-hi-lock’包（大概是对‘hi-lock’作进一步定制）。‘use-package’设施在其自己的手册中有完整文档，*note (use-package)Top::。

51.4.3 终端专用的初始化

每种终端类型都可以有一个 Lisp 库，当 Emacs 在该类终端上运行时加载它。对于名为 TERMTYPE 的终端类型，该库名为‘term/TERMTYPE’。（若‘term-file-aliases’关联列表中有形如‘(TERMTYPE . ALIAS)’的条目，Emacs 就用 ALIAS 代替 TERMTYPE。）该库照常通过搜索‘load-path’中的目录并尝试后缀‘.elc’和‘.el’找到。通常它出现在存放大多数 Emacs 库的目录下的‘term’子目录中。

终端专用库的通常用途，是使用‘input-decode-map’把该终端功能键所用的转义序列映射为更有意义的名称。关于如何做到这一点，参见文件‘term/lk201.el’中的示例。许多功能键会依据 Termcap 数据库中的信息自动映射；终端专用库只需映射 Termcap 未指定的那些功能键。

当终端类型包含连字符时，只有第一个连字符之前的那部分在选择库名时起作用。因此，终端类型‘aaa-48’和‘aaa-30-rv’都使用库‘term/aaa’。库中的代码可以用‘(getenv "TERM")’取得完整的终端类型名。

库名由变量‘term-file-prefix’的值与终端类型拼接而成。你的‘.emacs’文件可以通过把‘term-file-prefix’设为‘nil’来阻止加载终端专用库。

Emacs 在初始化结束时运行钩子‘tty-setup-hook’，此时你的‘.emacs’文件和任何终端专用库都已读入。如果你希望覆盖某个终端专用库的部分内容，或为没有对应库的终端定义初始化，就把钩子函数加入这个钩子。参见“钩子”。

51.4.4 Emacs 如何查找你的初始化文件

Emacs 通常在你主目录下的某个位置找到你的初始化文件(1)。参见“Emacs 初始化文件”。

Emacs 按如下文件名顺序查找你的初始化文件：‘~/.emacs.el’、‘~/.emacs’或‘~/.emacs.d/init.el’；你可以选择使用其中任意一个名称。（注意，只有直接位于你主目录中的那些位置，其基名才带前导点。）

Emacs 也可以在兼容 XDG 的位置查找‘init.el’，默认是目录‘~/.config/emacs’。这可以通过在环境中设置‘XDG_CONFIG_HOME’来覆盖，其值会替换默认 XDG 初始化文件名中的‘~/.config’。不过，若‘~/.emacs.d’、‘~/.emacs’和‘~/.emacs.el’存在，它们总是被优先使用，这意味着你必须删除或重命名它们，才能使用 XDG 位置。

另请注意，若 XDG 位置和‘~/.emacs.d’都不存在，Emacs 会创建‘~/.emacs.d’（因而在后续调用中使用它）。

Emacs 会把 'user-emacs-directory' 设为它决定使用的那个目录。该目录随后用于查找你其他的用户专用 Emacs 文件，如 'custom-file'（参见"保存自定义"）、保存的桌面（参见"保存 Emacs 会话"）等。它也用于计算 'native-comp-eln-load-path'（参见"Emacs 的 Lisp 代码库"）的值，Emacs 在那里查找本机编译的 Lisp 代码，也把由你的初始化文件及后续 Emacs 会话所加载的包的异步编译产生的新编译 Lisp 代码存放在那里。命令行选项 '--init-directory'（参见"初始选项"）会覆盖上述查找用户初始化文件过程中作为附带结果确定的 'user-emacs-directory' 值。

由于 'user-emacs-directory' 一经 Emacs 确定，就会被启动代码的其余部分用来定位其他文件和目录，我们不建议在你的初始化文件中更改它的值，因为那可能扰乱或破坏 Emacs 会话的正确启动。

尽管这与较旧的 Emacs 版本向后兼容，但现代 POSIX 平台更倾向于把你的初始化文件放在 '~/.config' 下，这样可以通过重命名该目录来排查可能由坏的初始化文件引起的问题，或归档一组初始化文件。为了帮助较旧的 Emacs 版本在其当前的默认位置找到配置文件，你可以执行下面的 Emacs Lisp 代码：

```
(make-symbolic-link ".config/emacs" "~/.emacs.d")
```

然而，若你从由 'su' 启动的 shell 中运行 Emacs，且环境中没有设置 'XDG_CONFIG_HOME'，Emacs 会试图查找你自己的初始化文件，而不是你当前假扮的那个用户的初始化文件。其用意是：即使你以超级用户身份运行，也应当得到你自己的编辑器定制。

更准确地说，Emacs 首先确定要使用哪个用户的初始化文件。它从环境变量 'LOGNAME' 和 'USER' 取得你的用户名；若两者都不存在，则使用有效用户 ID。若该用户名与真实用户 ID 匹配，Emacs 就使用 'HOME'；否则，它在系统的用户数据库中查找与该用户名对应的主目录。

为简洁起见，Emacs 文档的其余部分一般只用当前默认位置 '~/.emacs.d/init.el' 来表示初始化文件。

> 注：在 MS-Windows 上，并不存在一个被所有程序都视为用户"主目录"的单一目录。Emacs 使用相关目录之一作为你主目录的等价物；详情参见"MS-Windows 上的 HOME 与启动目录"。

51.4.5 初始化文件中的非 ASCII 字符

若你的初始化文件在字符串或键绑定中含有非 ASCII 字符（例如带重音的字母），语言和编码系统可能引起问题。

如果你想在初始化文件中使用非 ASCII 字符，应在初始化文件的第一行放一个 '*--coding: CODING-SYSTEM--*' 标签，并指定一个支持相关字符的编码系统。参见"识别编码系统"。这是因为在 Emacs 读取初始化文件中使用这类字符串的那些部分时，解码非 ASCII 文本的默认值可能尚未设置好，从而使 Emacs 可能错误地解码这些字符串。因此你还应避免加入以其他方式修改编码系统的 Emacs Lisp 代码，例如对 'set-language-environment' 的调用。

除直接使用非 ASCII 字符外，另一种做法是使用参见《Emacs Lisp 参考手册》的"General Escape Syntax"中描述的某种字符转义语法，因为这些语法允许仅用 ASCII 字符来指定所有 Unicode 码点。

要绑定非 ASCII 按键，必须使用向量（参见"在初始化文件中重绑定按键"）。不能使用字符串语法，因为其中的非 ASCII 字符会被解释为 meta 键。例如：

```
(global-set-key [?CHAR] 'some-function)
```

先输入 'C-q'，再输入要绑定的按键，即可插入 CHAR。

51.4.6 早期初始化文件

对 Emacs 的大多数自定义应放在普通初始化文件中。参见"Emacs 初始化文件"。不过，有时需要让自定义在 Emacs 启动期间、早于普通初始化文件被处理的时刻生效。这类自定义可以放在早期初始化文件中，即 '~/.config/emacs/early-init.el' 或 '~/.emacs.d/early-init.el'。该文件在包系统和 GUI 初始化之前加载，因此可以在其中自定义影响包初始化过程的变量，例如 'package-enable-at-startup'、'package-load-list' 和 'package-user-dir'。注意，像 'package-archives' 这样只影响新包安装、而不影响让已安装包可用的过程的变量，可以在常规初始化文件中自定义。参见"包的安装"。

我们不建议把可以留在普通初始化文件中的自定义搬进 'early-init.el'。这是因为早期初始化文件在 GUI 初始化之前就被读取，所以与 GUI 特性相关的自定义在 'early-init.el' 中无法可靠工作。相比之下，普通初始化文件在 GUI 初始化之后才被读取。如果确实必须在早期初始化文件中放置依赖 GUI 特性的自定义，就让它们通过 Emacs 启动时提供的钩子来运行，例如 'window-setup-hook' 或 'tty-setup-hook'。参见"钩子"。

有关早期初始化文件的更多信息，参见《Emacs Lisp 参考手册》的"Init File"。

51.4.7 用户 Lisp 目录

如果 'user-lisp-directory' 指定的目录（默认为 '~/.config/emacs/user-lisp/' 或 '~/.emacs.d/user-lisp/'）存在，那么 Emacs 启动时会准备好该目录中的 Lisp 文件，供本次会话使用。具体来说，Emacs 会做以下事情：

- 收集并激活 autoload cookie。这意味着你可以使用 'user-lisp-directory' 中文件里自动加载的命令和其他入口点，而无需在初始化文件中显式加载这些文件中的任何一个。参见《Emacs Lisp 参考手册》的"Autoload"。
- 对所有文件进行字节编译（参见《Emacs Lisp 参考手册》的"Byte Compilation"），并且如果你的 Emacs 构建支持，还会把它们编译为本机代码（参见《Emacs Lisp 参考手册》的"Native Compilation"）。这会加快这些文件中代码在被加载时以及之后执行时的速度。
- 调整 'load-path'，使所有文件都能以常规方式被加载和自动加载。参见《Emacs Lisp 参考手册》的"Library Search"。

用户 Lisp 目录在加载初始化文件之前就被处理（参见"Emacs 初始化文件"）。因此，下文讨论的对用户选项的任何自定义都必须在早期初始化文件中进行（参见"早期初始化文件"），才能生效。

默认情况下，Emacs 会考虑 'user-lisp-directory' 内的所有 Lisp 文件，包括子目录中的文件。你可以通过自定义 'user-lisp-ignored-directories' 来阻止 Emacs 进入特定的子目录。你可以把 'user-lisp-auto-scrape' 自定义为 'nil' 来完全禁用启动时的扫描。除了 Emacs 在启动时自动准备这些 Lisp 文件之外，你还可以用命令 'prepare-user-lisp' 手动触发准备过程。

尽管当用户 Lisp 目录中任何 Lisp 文件的内容发生变化时，Emacs 会自动处理重新编译，但还有其他原因可能导致需要重新编译，而 Emacs 无法自动检测到这些原因。一个能覆盖大多数（但不是全部）此类问题的好经验法则是：每次升级到新的 Emacs 发行版时都强制重新编译。启动新版本的 Emacs，输入 'C-u M-x prepare-user-lisp'，然后退出并再次重启 Emacs。

用户 Lisp 目录是比安装包更简单的机制（参见“Emacs Lisp 软件包”）。特别是，用户 Lisp 目录中的文件没有自动依赖解析或升级，也没有构建和注册包文档的设施。这意味着用户 Lisp 目录通常最适合你只为自己编写的代码。不过，也可以把用户 Lisp 目录用于第三方包或你维护的公开包。

51.5 保存持久认证信息

某些连接其他服务的 Emacs 包需要认证（参见“输入密码”），例如参见 *Note Gnus: (gnus)Top，或参见《Tramp 手册》。由于反复提供相同的用户名和密码可能很烦人，Emacs 通过 ‘auth-source’ 库提供保存这些信息的功能。

默认情况下，认证信息取自 ‘~/.authinfo’、‘~/.authinfo.gpg’ 或 ‘~/.netrc’ 文件。这些文件的语法类似于 ‘ftp’ 程序所用的 netrc 文件，如下所示：

```
machine MYMACHINE login MYLOGINNAME password MYPASSWORD port MYPORT
```

类似地，‘auth-source’ 库支持多种存储后端，目前包括经典的 netrc 后端、JSON 文件、Secret Service API，以及 pass——标准的 Unix 密码管理器。

所有这些可选方案都可以通过用户选项 ‘auth-sources’ 自定义，参见《《auth》手册》的“Help for users”。

当交互式输入的密码无法通过已配置的后端找到时，某些后端会提议将其持久保存。可以通过自定义用户选项 ‘auth-source-save-behavior’ 来改变这种行为。

51.6 新手主题

（如果你是点击启动画面上的链接来到这里的，可以输入 ‘q’ 返回。）

Emacs 的每个发行版都会带来新特性，但为了避免打乱用户现有的工作流程，这些特性默认未启用。然而其中许多特性旨在让 Emacs 对新用户更易上手。此外，新用户还没有任何可能被打乱的现有工作流程！因此，如果你是 Emacs 新手，可以启用 ‘newcomers-presets’ 主题（参见“自定义主题”），它会设置一组我们认为对熟悉其他程序惯例的新用户有用的用户选项。

‘newcomers-presets’ 主题所启用的功能会随 Emacs 发行版而变化。我们可能添加新功能，也可能移除我们认为已被取代的旧功能。因此，如果你习惯了新手预置（presets），应把它们复制到你自己的配置中，然后再次禁用该主题。你可以使用命令 ‘copy-theme-options’（参见“自定义主题”）来完成这件事。

52 退出与中止

```
‘C-g’ ‘C-<Break> (MS-DOS only)’ Quit: cancel running or partially typed command. ‘C-]’ Abort i
nnermost recursive editing level and cancel the command which invoked it (‘abort-recursive-edi
t’). ‘<ESC> <ESC> <ESC>’ Either quit or abort, whichever makes sense (‘keyboard-escape-quit’).
‘M-x top-level’ Abort all recursive editing levels that are currently executing. ‘C-/’ ‘C-x u’
‘C-_’ Cancel a previously made change in the buffer contents (‘undo’).
```

在命令执行完毕之前取消它有两种办法：用 ‘C-g’“退出”（quitting），以及用 ‘C-]’ 或 ‘M-x top-level’“中止”（aborting）。退出会取消已经输入了一部分的命令，或仍在运行的命令。中止则退出一个递归编辑层级，并取消调用该递归编辑的命令（参见“递归编辑层级”）。

用 ‘C-g’ 退出是舍弃输入了一半的命令、或舍弃你不想要的前缀参数的办法。此外，如果你正处于某条正在运行的命令中间，‘C-g’ 会以相对安全的方式停止该命令。例如，若你从一条耗时很长的删除（kill）命令中退出，你的文本要么全部仍留在缓冲区中，要么全部留在删除环中，也可能两者都有。如果区域处于活动状态，‘C-g’ 会停用标记，除非临时标记模式已关闭（参见“禁用临时标记模式”）。如果你正处于增量搜索中间，‘C-g’ 的行为比较特殊；要退出搜索，可能需要连按两次 ‘C-g’。详见参见“增量搜索”。

如果在迷你缓冲区中键入 ‘C-g’，这会退出打开该迷你缓冲区的那条命令并关闭该迷你缓冲区。如果该迷你缓冲区不是最近打开的那个（当 ‘minibuffer-follows-selected-frame’ 为 ‘nil’ 时可能如此，参见“使用迷你缓冲区”），那么 ‘C-g’ 还会先征求你的确认，然后关闭那些更晚打开的迷你缓冲区，并退出与之关联的命令。

在 MS-DOS 上，字符 ‘C-<Break>’ 像 ‘C-g’ 一样充当退出字符。原因是在 MS-DOS 上，命令运行期间、在与用户交互之间识别 ‘C-g’ 是不可行的；相比之下，随时识别 ‘C-<Break>’ 则是可行的。参见“MS-DOS 上的键盘使用”。

‘C-g’ 的工作原理是：在你键入 ‘C-g’ 的瞬间把变量 ‘quit-flag’ 置为 ‘t’；Emacs Lisp 会频繁检查该变量，一旦它非 ‘nil’ 就退出。只有在你于 Emacs 等待输入时键入 ‘C-g’，它才真正作为一条命令被执行；在这种情况下，它运行的命令是 ‘keyboard-quit’。

在文本终端上，如果在第一个 ‘C-g’ 被识别之前你又按了第二次 ‘C-g’，就会激活紧急逃生功能并返回到 shell。参见“紧急退出”。

有些情况下你无法退出。当 Emacs 正在等待操作系统完成某项操作时，退出是不可能的，除非在 Emacs 内部针对发生等待的那个特定系统调用做了特殊处理。对于用户可能想从中退出的那些系统调用，我们已经做了处理，但你仍可能遇到未处理的情形。有一种非常常见的情形——使用 NFS 等待文件输入或输出——Emacs 本身知道如何退出，但许多 NFS 实现就是不允许用户程序在 NFS 服务器无响应时停止等待。

用 ‘C-]’（‘abort-recursive-edit’）中止，用于退出一个递归编辑层级并取消调用它的那条命令。用 ‘C-g’ 退出做不到这一点，也不可能做到，因为 ‘C-g’ 是用来取消递归编辑层级内部输入了一部分的命令的。这两种操作都有用。例如，如果你处于递归编辑中并键入 ‘C-u 8’ 输入前缀参数，你可以用 ‘C-g’ 取消该参数并留在递归编辑中。

序列 ‘<ESC> <ESC> <ESC>’（‘keyboard-escape-quit’）既能退出也能中止。（我们这样定义是因为在许多 PC 程序中 <ESC> 表示“退出”。）它可以像 ‘C-g’ 那样取消前缀参数、清除选中的区域，或退出查询替换（Query Replace）。它可以像 ‘C-]’ 那样退出迷你缓冲区或递归编辑。它也可以退出把框架分割成多个窗口的状态，作

用如同‘C-x 1’。然而，有一件事它做不到，就是停止正在运行的命令。这是因为它是作为普通命令执行的，而 Emacs 要到准备接受下一条命令时才会注意到它。

命令‘M-x top-level’相当于执行足够多次‘C-]’，让你退出所处的所有层级的递归编辑；如果迷你缓冲区处于活动状态，它也会退出迷你缓冲区。‘C-]’一次退出一层，而‘M-x top-level’一次退出所有层。与‘C-g’不同，‘C-]’和‘M-x top-level’与所有其他命令一样，只在 Emacs 准备接受命令时才起作用。‘C-]’是一个普通按键，其含义仅仅来自它在键位图中的绑定。参见“递归编辑层级”。

严格说来，‘C-/’（‘undo’）并不是取消命令的办法，但你可以把它看作取消一条已经执行完毕的命令。关于撤销功能的更多信息，参见“撤销”。

53 故障处理

本节介绍如何识别并处理 Emacs 未按你的预期工作的情况，例如键盘代码错乱、显示乱码、内存耗尽，以及崩溃和挂起。

至于认为自己发现了 Emacs 中的 bug 时该怎么做，参见“报告 Bug”。

53.1 递归编辑层级

递归编辑层级是 Emacs 重要而有用的特性，但若不理解它，就可能把它当成故障。

若模式行中括住主模式和次模式名称的括号带有方括号 ‘[...]’，就说明你进入了递归编辑层级。若不是有意为之，或者不明白这意味着什么，直接退出该递归编辑层级即可。为此，输入 ‘M-x top-level’。参见“递归编辑层级”。

53.2 屏幕乱码

若文本终端上的文本看起来不对，首先要看缓冲区中的文本是否也错了。输入 ‘C-l’（‘recenter-top-bottom’）重绘整个屏幕。若此后屏幕显示正确，问题就完全出在上一次屏幕更新上。（否则参见下一节。）

显示更新方面的问题常常源于所用终端的 terminfo 条目不正确。Emacs 发行版中的 ‘etc/TERMS’ 文件给出了这类已知问题的修复方法。‘INSTALL’ 的某一节中包含针对这些问题的一般性建议。若你用的 terminfo 条目看来是正确的，那可能是该条目本身有 bug，或者 Emacs 中存在只在某些终端类型上才出现的 bug。

53.3 文本中出现乱码

若 ‘C-l’ 显示文本确实有误，先输入 ‘C-h l’（‘view-lossage’），看看为产生所见结果你输入了哪些命令。然后试着用 ‘C-x u’（‘undo’）逐步撤销改动，直到回到你认为正确的状态。

若缓冲区开头或末尾有大段文本似乎不见了，检查模式行中是否有 ‘Narrow’ 字样。若有，你没看到的文本多半仍然存在，只是暂时被禁止访问。要让它重新可访问，输入 ‘C-x n w’（‘widen’）。参见“窄化”。

53.4 内存耗尽

若出现错误消息 ‘Virtual memory exceeded’，用 ‘C-x s’（‘save-some-buffers’）保存已修改的缓冲区。这种保存方式所需的额外内存最少。Emacs 保有一份备用内存，在此错误发生时提供使用；它应足以让 ‘C-x s’ 完成工作。备用内存用尽后，模式行开头会出现 ‘!MEM FULL!’，表示再没有备用内存了。

保存好已修改的缓冲区后，你可以退出这个 Emacs 会话并另启一个，也可以用 ‘M-x kill-some-buffers’ 释放当前 Emacs 作业中的空间。若由此释放出足够空间，Emacs 会重新填满其备用内存，‘!MEM FULL!’ 会从模式行上消失。这意味着你可以安全地在同一 Emacs 会话中继续编辑。

内存耗尽时不要用 ‘M-x buffer-menu’ 来保存或删除缓冲区，因为 Buffer Menu 本身需要相当多的内存，备用内存可能不够用。

在 GNU/Linux 系统上，Emacs 通常不会收到内存耗尽的通知；相反，系统可能在其内存耗尽时杀掉 Emacs 进程。此特性称为“out-of-memory killer”，或“OOM killer”。此行为生效时，Emacs 无法及时察觉内存耗尽，也就无法按上述方式让你保存缓冲区。不过，可以关掉操作系统的这一行为，从而让 Emacs 在被杀掉之前有机会以更有用的方式处理内存耗尽。做法是：成为超级用户，编辑‘/etc/sysctl.conf’使其包含下面几行，然后在 shell 提示符下调用命令‘sysctl -p’：

```
vm.overcommit_memory=2
vm.overcommit_ratio=0
```

请注意，上述设置会影响系统上的所有进程，以及系统在内存紧张时的整体行为，而不只是 Emacs 进程。

53.5 Emacs 崩溃时

Emacs 本不该崩溃，但万一崩溃，它会在退出前生成一份“崩溃报告”（crash report）。崩溃报告打印到标准错误流。若 Emacs 是在 GNU 或 Unix 系统上从图形桌面启动的，标准错误流通常被重定向到‘~/.xsession-errors’之类的文件，因此你可以到那里去找崩溃报告。在 MS-Windows 上，崩溃报告除写入标准错误流外，还写入 Emacs 进程当前目录下名为‘emacs_backtrace.txt’的文件。

崩溃报告的格式取决于平台。在某些平台上，例如使用 GNU C Library 的平台，崩溃报告包含一段“回溯”（backtrace），描述崩溃前的执行状态，可用于帮助调试崩溃。下面是 GNU 系统上的一个例子：

```
Fatal error 11: Segmentation fault
Backtrace:
emacs[0x5094e4]
emacs[0x4ed3e6]
emacs[0x4ed504]
/lib64/libpthread.so.0[0x375220efe0]
/lib64/libpthread.so.0(read+0xe)[0x375220e08e]
emacs[0x509af6]
emacs[0x5acc26]
...
```

数字‘11’是对应于该崩溃的系统信号编号——在此例中是段错误（segmentation fault）。十六进制数是程序地址，可用调试工具把它们与源代码行关联起来。例如，GDB 命令‘list *0x509af6’会打印‘emacs[0x509af6]’这一项对应的源代码行。若你的系统有‘addr2line’工具，下面的 shell 命令会输出带源代码行号的回溯：

```
sed -n 's/.*\[\(.*\)]$/\1/p' BACKTRACE |
  addr2line -C -f -i -p -e BINDIR/EMACS-BINARY
```

在 MS-Windows 上，回溯看起来有些不同，例如：

```
Backtrace:
00007ff61166a12e
00007ff611538be1
00007ff611559601
00007ff6116ce84a
00007ff9b7977ff0
...
```

因此，不需要用 'sed' 过滤，而显示源代码行号的命令是

```
addr2line -C -f -i -p -e BINDIR/EMACS-BINARY < BACKTRACE
```

这里，BACKTRACE 是文本文件名，其中含有一份回溯的副本（在 MS-Windows 上即启动 Emacs 所在目录中的 'emacs_backtrace.txt'），BINDIR 是包含 Emacs 可执行文件的目录名，EMACS-BINARY 是 Emacs 可执行文件名，在 GNU 和 Unix 系统上通常是 'emacs'，在 MS-Windows 和 MS-DOS 上则是 'emacs.exe'。若你的 'addr2line' 版本太旧、没有 '-p' 选项，就省略该选项。

在支持核心文件的系统上，Emacs 崩溃时可以选择生成"核心转储"（core dump）。核心转储是一个文件，包含关于崩溃前程序状态的大量数据，通常通过将其载入 GDB 之类的调试器来检查。在许多平台上，核心转储默认被禁用，你必须运行 shell 命令 'ulimit -c unlimited'（例如在你的 shell 启动脚本中）来显式启用它。

53.6 崩溃后的恢复

若 Emacs 或计算机崩溃，你可以从自动保存文件中恢复崩溃时正在编辑的文件。为此，重新启动 Emacs，并输入命令 'M-x recover-session'。

该命令首先显示一个缓冲区，其中列出被中断的会话文件，每个都带有日期。你必须选择要从哪个会话恢复。通常想要的是最近的那一个。把光标点移到你选中的那个上，然后输入 'C-c C-c'。

接着，'recover-session' 会逐个考虑你在该会话期间编辑过的文件；对每个这样的文件，它询问是否恢复该文件。若你对某个文件回答 'y'，它会显示该文件及其自动保存文件的日期，然后再次询问是否恢复该文件。对第二个问题，你必须用 'yes' 确认。若你确认了，Emacs 会访问该文件，但文本取自自动保存文件。

'recover-session' 结束后，你选择恢复的文件已出现在 Emacs 缓冲区中。此后应当保存它们。只有保存——把它们存盘——才会更新文件本身。

作为最后的手段，若有内容的缓冲区未与任何文件关联，或者自动保存不够新、没能记录下重要改动，你可以用 GDB（GNU 调试器）配合 'etc/emacs-buffer.gdb' 脚本，从核心转储中取回它们——前提是保存了核心转储，且 Emacs 可执行文件没有被去掉调试符号。

拿到核心转储后，尽快把它改名成别的名字，例如 'core.emacs'，以免下一次崩溃把它覆盖掉。

要使用该脚本，用你的 Emacs 可执行文件名和核心转储文件名运行 'gdb'，例如 'gdb /usr/bin/emacs core.emacs'。在 '(gdb)' 提示符下载入恢复脚本：'source /usr/src/emacs/etc/emacs-buffer.gdb'。然后输入命令 'ybuffer-list' 查看有哪些缓冲区可用。对每个缓冲区，它列出一个缓冲区编号。要保存某个缓冲区，使用 'ysave-buffer'；你要指定缓冲区编号，以及把该缓冲区写入的文件名。你应当使用尚不存在的文件名；若该文件已存在，脚本不会备份其原有内容。

53.7 紧急退出

在文本终端上，若你在 Emacs 还来不及真正响应第一次 'C-g' 而退出之前再次输入 'C-g'，"紧急退出"（emergency escape）特性会立即挂起 Emacs。这样无论 GNU Emacs 卡得多死，你总能退出来。一切正常时，Emacs 识别并处理第一次 'C-g' 的速度很快，第二次不会触发紧急退出。然而，若某些问题妨碍 Emacs 正确处理第一次 'C-g'，第二次就会让你回到 shell。

紧急退出导致挂起之后，当你恢复 Emacs 时，它会报告恢复情况，并在回到原先所做的事情之前问一两个问题：

```
Emacs is resuming after an emergency escape.
Auto-save? (y or n)
Abort (and dump core)? (y or n)
```

对每个问题用 'y' 或 'n' 回答，后跟 <RET>。

对 'Auto-save?' 回答 'y'，会立即对所有启用了自动保存的已修改缓冲区进行自动保存。回答 'n' 则跳过。若 Emacs 处于无法安全自动保存的状态，则不问这个问题。

对 'Abort (and dump core)?' 回答 'y'，会使 Emacs 崩溃并转储核心。这是为了让高手查明 Emacs 起初为何无法退出。核心转储之后执行不再继续。

若你对这个问题回答 'n'，Emacs 会继续执行。运气好的话，Emacs 最终会执行所请求的退出。若不然，此后每次输入 'C-g' 都会再次调用紧急退出。

若 Emacs 并非真的卡住，只是慢，你可能无意中触发了双击 'C-g' 特性。那么只需恢复并对两个问题都回答 'n'，你就会回到先前的状态。你所请求的退出迟早会发生。

紧急退出仅对文本终端有效。在图形显示上，你可以用鼠标杀掉 Emacs 或切换到别的程序。

在 MS-DOS 上，你必须输入（两次）'C-<Break>' 才能触发紧急退出——但有些情况下它不会起作用，例如系统调用挂起，或者 Emacs 卡在 C 代码的紧凑循环中。

53.8 若 <DEL> 无法删除

每个键盘都有一个通常标为 <BACKSPACE> 的大键，一般用来擦除你输入的最后一个字符。在 Emacs 中，这个键应当等效于 <DEL>。

Emacs 在图形显示上启动时，会自动判断哪个键应当是 <DEL>。在某些不寻常的情况下，Emacs 从系统得到错误信息，结果 <BACKSPACE> 变成了向前删除而不是向后删除。

有些键盘还有 <Delete> 键，通常用于向前删除。若这个键在 Emacs 中向后删除，那也说明 Emacs 得到了错误信息——只是方向相反。

在文本终端上，若你发现 <BACKSPACE> 不是删除一个字符，而是像 'Control-h' 那样提示 Help 命令，那就说明该键实际发送的是 'BS' 字符。Emacs 本应把 <BS> 当作 <DEL> 处理，但它没有。

在所有这些情况下，立即可用的补救办法都一样：使用命令 'M-x normal-erase-is-backspace-mode'。它在 Emacs 支持的两种处理 <DEL> 的模式之间来回切换，因此若 Emacs 启动时处于错误的模式，这应当能切换到正确的模式。在文本终端上，若 <BS> 被当作 <DEL> 时你还想求助，请用 <F1> 代替 'C-h'；若 'C-?' 发送的是字符代码 127，它也可能有用。

要在每个 Emacs 会话中解决此问题，可把下面两行之一放入初始化文件（参见"Emacs 初始化文件"）。对于上面第一种情况，即 <BACKSPACE> 向前删除而不是向后删除，用下面这行使 <BACKSPACE> 充当 <DEL>：

```
(normal-erase-is-backspace-mode 0)
```

对于另外两种情况，用这一行：

```
(normal-erase-is-backspace-mode 1)
```

为每个 Emacs 会话解决此问题的另一种方式是定制变量 'normal-erase-is-backspace'：值 't' 指定 <BS> 或 <BACKSPACE> 为 <DEL> 的模式，'nil' 则指定另一种模式。参见"简易自定义界面"。

54 报告 Bug

如果你认为在 Emacs 中发现了 bug，请报告它。我们不能保证修复它，也不能保证总是同意它是 bug，但我们确实希望听到相关报告。你想看到添加的新功能同样如此。本节将帮助你判断自己发现的是否为 bug，若是，则帮助你构造一份有效的 bug 报告。

当你发现某件可能是 bug 的事情时，一般流程如下：

- 看看你发现的问题是否为已知问题，或已被报告和/或已修复的 bug。参见“阅读已有的 bug 报告和已知问题”，其中说明了如何查找已知的问题和 bug。
- 如果你不确定看到的行为是不是 bug，参见“何时算有 bug”，其中说明了我们认为哪些是 Emacs 中的明显 bug。
- 一旦你判定自己发现了 bug，参见“理解 bug 报告”，它帮助你以最有效的方式描述你所看到的现象，使我们复现并调查该问题的工作更容易。
- 接下来，参见“Checklist, where we describe in detail how to submit a bug report and what information to include in it”。其中详细描述了如何提交 bug 报告以及报告中应包含哪些信息。简而言之，你使用 Emacs 命令 ‘report-emacs-bug’ 通过电子邮件提交 bug 报告，该命令会协助你完成这件事。提交 bug 报告将启动调查并修复该 bug 的过程，期间你会收到讨论该 bug 的电子邮件副本，其中我们可能会请你提供更多信息、测试可能的修复等。
- 如果你认为自己可能发现了需要私下沟通的安全问题，请直接联系 GNU Emacs 维护者。维护者的联系方式见 Emacs 发行版中的 ‘admin/MAINTAINERS’ 文件。
- 最后，如果你想对 Emacs 提出具体的修改，无论是修复 bug、添加新功能还是改进文档，请参见“为 GNU Emacs 发送补丁”，了解提交此类修改的细节。
- 准则：Bug Criteria。你真的发现了 bug 吗?
- 理解 bug 报告：如何有效地报告 bug。
- 核对清单：撰写良好 bug 报告应遵循的步骤。
- 发送补丁：如何为 GNU Emacs 发送补丁。

54.1 阅读已有的 bug 报告和已知问题

报告 bug 之前，如有可能，请先检查我们是否已经知道它。确实，它可能已在较晚的 Emacs 发行版中修复，或在开发版中修复。以下是你可以了解已知问题的主要场所列表：

- ‘etc/PROBLEMS’ 文件；键入 ‘C-h C-p’ 可阅读它。该文件列出了在编译、安装和运行 Emacs 时遇到的特别著名的问题，尤其侧重于其他软件引起、在 Emacs 中不易解决的问题。你常能在其中找到绕过办法和解决方案的建议。
- <https://debbugs.gnu.org> 上的 GNU Bug Tracker。Emacs 的 bug 和问题在跟踪器中归档在 ‘emacs’ 包下。跟踪器记录每个 bug 的状态信息、最初的 bug 报告，以及报告者和参与讨论、修复该 bug 的 Emacs 开发者

的后续消息。你可以按主题、严重程度和其他条件搜索 bug。更复杂的搜索条件请使用 <https://debbugs.gnu.org/cgi/search.cgi>。

除把 bug 跟踪器当作网页浏览之外，你还可以在 Emacs 中使用 'debbugs' 包浏览它，该包可通过 Package Menu（*note Packages::）下载。该包提供命令 'M-x debbugs-gnu' 来列出 bug，以及 'M-x debbugs-gnu-search' 来搜索特定的 bug。由 Emacs 维护者添加的用户标签由 'M-x debbugs-gnu-usertags' 显示。

- 'bug-gnu-emacs' 邮件列表（也可通过新闻组 'gnu.emacs.bug' 访问）。你可以在 <https://lists.gnu.org/mailman/listinfo/bug-gnu-emacs> 阅读该列表的存档。该列表相当于发送到 bug 跟踪器的 Emacs bug 报告及后续消息的镜像。它还包含 bug 跟踪器引入之前（2008 年初）的旧 bug 报告。

如果愿意，你可以订阅该列表。但要注意，其目的是向 Emacs 维护者提供有关 bug 和功能请求的信息，因此报告可能包含相当大的数据量；旁观者不应为此抱怨。

- 'emacs-pretest-bug' 邮件列表。该列表已不再使用，主要只有历史意义。它曾一度用于 Emacs 开发版（即尚未发布的版本）中的 bug 报告。你可以在 <https://lists.gnu.org/r/emacs-pretest-bug/> 阅读 2003 年至 2007 年中的存档。如今，发往该列表的电子邮件会被重定向到 'bug-gnu-emacs'。

- 'emacs-devel' 邮件列表。有时人们会向该邮件列表报告 bug。然而这并不是该列表的主要用途，把 bug 报告发送到 bug 列表要好得多。在报告 bug 之前，你不必觉得有义务阅读该列表。

54.2 何时算有 bug

如果 Emacs 访问了无效的内存位置（又称"段错误"），或以操作系统错误消息退出，而该消息表明程序出了问题（而不是像"磁盘已满"之类），那它肯定是 bug。

如果 Emacs 的显示与缓冲区内容不符，那就是 bug。但你应检查缓冲区窄化（参见"窄化"）这类可能隐藏缓冲区部分内容或改变其显示方式的功能，确认不是它们导致的。

某条命令耗时过长可能是 bug，但你必须确认这确实是 Emacs 的过错。有些命令本来就要花很长时间。键入 'C-g'（在 MS-DOS 上是 'C-<Break>'），然后键入 'C-h l'，看看 Emacs 收到的输入是否是你本来打算键入的；如果输入是那种你_知道_本应很快处理完的，就报告 bug。如果你不知道某条命令是否本该耗时很长，请查阅手册或寻求帮助来弄清。

如果你熟悉的某条命令在其通常定义理应合理的情况下导致 Emacs 报错，那很可能是 bug。

如果某条命令做错了事，那就是 bug。但要确信你知道它本该做什么。如果你不熟悉该命令，它实际上可能工作正常。若拿不准，请阅读该命令的文档（参见"按命令或变量名获取帮助"）。

一条命令的预期定义可能并非最适合编辑使用的定义。这是非常重要的一类问题，但也是判断问题。而且，由于不了解某些已有功能，很容易得出这样的结论。在你按常规方式查阅了文档、确信自己理解了它，并且确实知道你想要的并不可用之前，最好不要抱怨这类问题。也不妨问问其他 Emacs 用户。如果在仔细阅读手册之后仍不确定该命令该做什么，请查看索引和术语表中任何可能不清楚的术语。

如果在仔细重读手册之后你仍然不明白该命令应该做什么，那就说明手册有 bug，你应该报告它。手册的任务就是让非 Emacs 专家——包括你——把一切弄清楚。报告文档 bug 与报告程序 bug 同样重要。

如果某个函数或变量的内置文档与手册不一致，那么二者之一必定有错；那就是 bug。

对于不属于 Emacs 的包的问题，最好先向包的开发者报告。

54.3 理解 bug 报告

当你认定存在 bug 时，报告它很重要，而用有用的方式报告它也很重要。最有用的是准确描述你键入了哪些命令——从运行 Emacs 的 shell 命令开始，直到问题发生——以及键入这些命令所产生的效果。

报告 bug 最重要的原则是报告_事实_。假设和口头描述不能替代详细的原始数据。报告事实很直接，但许多人费力提出种种解释，并报告解释而不是事实。如果解释基于对 Emacs 实现方式的猜测，它们可能没有用；同时，由于缺少事实，我们将得不到关于该 bug 的真正信息。如果你确实想_调试_这个问题，并报告不只是猜测的解释，那是有用的——但也请一并给出原始事实。

例如，假设你键入 'C-x C-f /glorp/baz.ugh <RET>'，访问一个（你知道）恰好相当大的文件，而 Emacs 显示 'I feel pretty today'。bug 报告需要提供所有这些信息。你不应假定问题出在文件大小上，然后说"我访问了一个大文件，Emacs 显示了 'I feel pretty today'"。这就是我们所说的"猜测解释"。问题也许是因为文件名里有个 'z'。若是这样，当我们收到你的报告后，会用某个大文件（很可能名字里没有 'z'）来试验，结果看不到任何问题。我们无法猜到自己应该尝试访问名字里带 'z' 的文件。

你甚至不应说"访问文件"而不说 'C-x C-f'。这是因为访问文件的方式不止一种，不能保证所有方式都能复现问题。同样，与其说"如果我在行上有三个字符"，不如说"在我键入 '<RET> A B C <RET> C-p' 之后"，如果那就是你输入文本的方式——也就是说，告诉我们那个在你这里产生问题的文本。

如有可能，请试着用 'emacs -Q' 调用 Emacs（这样 Emacs 启动时不加载任何初始定制；参见"初始选项"），并重复你触发该 bug 的步骤，快速复现该 bug。如果你能这样复现，就排除了你的个人定制中的 bug，并使该 bug 容易复现得多。那么你的 bug 报告应以说明你用 'emacs -Q' 启动了 Emacs 开头，接着是复现该 bug 的准确步骤序列。如有可能，请告知我们复现该 bug 所需的任何文件的准确内容。

有些 bug 无法从 'emacs -Q' 复现；有些则根本不容易复现。在这种情况下，你应报告你已有的东西——但和之前一样，请坚持只讲你第一次触发该 bug 时所作所为的原始事实。

如果你想报告多个问题，请为每个问题单独提交一份 bug 报告。

54.4 bug 报告核对清单

报告 bug 之前，请先看该问题是否已被报告（参见"阅读已有的 bug 报告和已知问题"）。

如有可能，请试用最新的 Emacs 发行版，看看问题是否已修复。更好的做法是试用最新的开发版。我们明白这对某些人并不容易，所以不要觉得在报告之前非做不可。

为 Emacs 撰写 bug 报告的最好方式是使用命令 'M-x report-emacs-bug'。它会建立一个邮件缓冲区（参见"发送邮件"），并自动插入_部分_必要信息。然而它无法提供所有必要信息；你仍应阅读并遵循下面的准则，以便在发出邮件之前手工填入其他关键信息。你可能觉得 'M-x report-emacs-bug' 插入的某些信息无关紧要，但除非你完全确定，最好保留它们，让开发者自己判断。

报告写完后，键入 'C-c C-c'，它将被发送给 <bug-gnu-emacs@gnu.org> 的 Emacs 维护者。如果你无法在 Emacs 内发送邮件，可以把报告文本复制到平常用的邮件客户端（如果你的系统支持，可以键入 'C-c M-i' 让 Emacs

替你完成），然后发送到该地址。或者你也可以直接向该地址发一封描述问题的电子邮件，其中包含下面提到的必要信息。

如果你想向 Emacs 提交代码（修复问题或实现新功能），最简单的做法是向 Emacs 问题跟踪器发送补丁。请使用 'M-x submit-emacs-patch' 命令，其用法与报告 bug 时大致相同；参见"为 GNU Emacs 发送补丁"。

无论如何，你的报告都会发送到 'bug-gnu-emacs' 邮件列表，并存入 <https://debbugs.gnu.org> 的 GNU Bug Tracker。请附上有效的回复电子邮件地址，以备我们需要就你的报告向你询问更多信息。提交内容要经过审核，因此你的报告真正出现在跟踪器上之前可能会有延迟。

报告 bug 并不需要了解 GNU Bug Tracker 如何工作，但如果你想知道，可以阅读跟踪器的在线文档（https://debbugs.gnu.org/Advanced.html），了解可用的各种功能。

发往 'bug-gnu-emacs' 邮件列表的所有邮件也会通过网关转发到 'gnu.emacs.bug' 新闻组。反之亦然，但我们请求你不要通过新闻组发布 bug 报告（或回复）。这会使我们在需要询问更多信息时难以联系到你，而且它与 bug 跟踪器的整合也不好。

如果你的数据超过 500,000 字节，请不要直接把它放进 bug 报告；而应表示可按要求发送，或把它放到网上并说明位置。大附件最好压缩后发送。

GNU Bug Tracker 会为你的报告分配一个 bug 编号；请在后续讨论中使用它，并在收件人列表中保留该 bug 地址，以便跟踪器记录该 bug 的讨论。bug 地址形如 'NNNNN@debbugs.gnu.org'，其中 NNNNN 是 bug 编号。

为使维护者能调查某个 bug，你的报告应包含以下所有内容：

- 描述你观察到、并认为不正确的行为。例如，"Emacs 进程收到致命信号"，或者，"产生的文本如下，我认为它是错的"。

当然，如果 bug 是 Emacs 收到致命信号，那谁也不会漏看。但如果 bug 是错误的文本，维护者可能注意不到哪里不对。何必把它留给运气呢?

即使你经历的问题是致命信号，也应明确说出来。假设出了某种怪事，比如你的源码副本不同步，或者你遇到了你系统上 C 库中的 bug。（这种事发生过！）你的副本可能崩溃，而这里的副本可能不会。如果你_说了_预期会崩溃，那么当这里的 Emacs 没有崩溃时，我们就知道该 bug 没有发生。如果你没说预期会崩溃，我们就不知道 bug 是否在发生——我们无法从观察中得出任何结论。

通常，对行为的描述以及复现问题的方法需要指明下面一个或多个方面：

- 复现该 bug 所需的任何文件的完整文本。

如果你能告诉我们一种不访问任何文件就能引起该问题的方法，请这么做。这会使调试容易得多。如果确实需要文件，请确保安排我们能看到它们的准确内容。例如，行尾是否有空格，或缓冲区最后一行之后是否有换行符，都可能有影响（本来不该有人关心最后一行是否终止，但你不妨去跟 bug 讲讲这个道理）。

- 复现该 bug 所需的准确命令。如有可能，请给出从以 '-Q' 选项启动的 Emacs 开始（*note Initial Options::）的完整配方。这会绕过你的个人定制。

精确记录对 Emacs 输入的一种方法是写一个 dribble 文件。要开始记录，请使用命令 'M-x open-dribble-file'。从那时起，Emacs 会把你的所有输入复制到指定的 dribble 文件，直到 Emacs 进程被杀掉。请注意，敏感信息

（例如密码）可能会最终被记录在 dribble 文件中。

- 如果 bug 是 Emacs 手册或 Emacs Lisp 参考手册未能描述 Emacs 的实际行为，或者文字令人困惑，请把手册中你认为有问题的文字抄进来。如果该节很短，只给节名就够。
- 如果 bug 的表现是 Emacs 的错误消息，那么重要的是报告错误消息的准确文本，以及显示 Emacs 中的 Lisp 程序如何到达该错误的回溯（backtrace）。

要准确获得错误消息文本，请把它从 '*Messages*' 缓冲区复制到 bug 报告中。要复制全部，不要只复制一部分。

- 检查你加载到 Lisp 世界中的任何程序（包括你的初始化文件）是否设置了可能影响 Emacs 运行的变量。还要看在不加载初始化文件的情况下（以 '-Q' 开关启动 Emacs 可阻止加载初始化文件），新启动的 Emacs 中是否出现该问题。如果那时问题_不_出现，你必须报告为了引起该问题而必须加载到 Lisp 世界中的任何程序的准确内容。
- 如果问题确实取决于某个初始化文件或不属于标准 Emacs 系统的其他 Lisp 程序，那么你应先向那些程序的维护者抱怨，以确认它不是那些程序中的 bug。待他们确认自己是以预期可用的方式使用 Emacs 之后，应由他们报告该 bug。
- 如果你想提到 GNU Emacs 源代码中的某处，请连同几行上下文一起展示那行代码。不要只给行号。

开发源码中的行号与你的源码中的行号并不一致。维护者要确定你版本中某个给定行号处是什么代码，需要额外的工作，而且我们无法确定。

- 对于文本模式终端上可能出现的显示 bug，请给出终端类型（环境变量 'TERM' 的值）、来自 '/etc/termcap' 的该终端的完整 termcap 条目（因为该文件在所有机器上并不相同），以及 Emacs 实际发送到终端的输出。

收集终端输出的方法是，在启动 Emacs 之后立即调用命令 'M-x open-termscript'；它会提示你输入一个文件名，并把此后直到 Emacs 进程被杀掉之前的所有终端输出记录到该文件。如果问题在 Emacs 启动时就发生，就把下面这个 Lisp 表达式

```
(open-termscript "~/termscript")
```

放进你的 Emacs 初始化文件，这样当 Emacs 第一次显示屏幕时 termscript 文件就已经打开。

请注意：在无法访问能引发该 bug 的那种终端的情况下，修复依赖终端的 bug 常常很困难，有时甚至不可能。

- Emacs 的版本号。没有它，我们就不知道在当前的 GNU Emacs 版本中查找该 bug 是否有意义。

'M-x report-emacs-bug' 会自动包含此信息，但如果你不用该命令写报告，可以键入 'M-x emacs-version <RET>' 获得版本号。如果该命令不起作用，你用的可能不是 GNU Emacs，那就得在别处报告该 bug。

- 你使用的机器类型，以及操作系统名称和版本号（同样由 'M-x report-emacs-bug' 自动包含）。'M-x emacs-version <RET>' 也提供此信息。请从 '*Messages*' 缓冲区复制其输出，以便完整而准确地获得它，或者用 'C-u M-x emacs-version <RET>' 把版本信息插入当前缓冲区。
- 构建 Emacs 时传给 'configure' 命令的命令行参数（由 'M-x report-emacs-bug' 自动包含）。

- 你对 Emacs 源代码所作全部修改的完整清单。（除非 bug 在未修改的 Emacs 中发生，否则我们可能没有时间调查它。但如果你作了修改却不告诉我们，那就是让我们白跑一趟。）

请准确说明这些改动。用英文描述是不够的——请发送针对它们的统一上下文差异（unified context diff）。

添加你自己的文件，或移植到另一台机器，都算对源代码的修改。

- 任何其他偏离安装 GNU Emacs 标准流程的细节。
- 如果涉及非 ASCII 文本或国际化，请给出启动 Emacs 时当前的 locale。这会由 'M-x report-emacs-bug' 自动包含；此外，在 GNU/Linux 和 Unix 系统上，或者如果你使用 Bash 之类 POSIX 风格的 shell，可以用下面的 shell 命令查看相关值：

```
echo LC_ALL=$LC_ALL LC_COLLATE=$LC_COLLATE LC_CTYPE=$LC_CTYPE \
  LC_MESSAGES=$LC_MESSAGES LC_TIME=$LC_TIME LANG=$LANG
```

如果你的系统有 'locale' 命令，也可以用它显示你的 locale 设置。

下面是一些在 bug 报告中不必要的东西：

- 对 bug 外围情形的描述——对于可复现的 bug，这并不必要。

遇到 bug 的人常常花大量时间研究：对输入文件作哪些改动会让 bug 消失，作哪些改动则不影响它。

这往往费时而且不太有用，因为我们找到 bug 的方式是在调试器下带断点运行一个例子，而不是从一系列例子中纯靠推理。你倒不如省下时间，不去寻找更多例子。更好的做法是立刻发出 bug 报告，回去继续编辑，再找另一个 bug 来报告。

当然，如果你能找到一个更简单的例子来_代替_原来的例子报告，那是很方便的。输出中的错误会更容易发现，在调试器下运行会花更少时间，等等。

然而，简化并非至关重要；如果你做不到，或者没有时间尝试，请用你最初的测试用例报告该 bug。

- 核心转储（core dump）文件。

调试核心转储可能有用，但这只能在你自己的机器上用你的 Emacs 可执行文件来做。因此，把核心转储文件发给 Emacs 维护者不会有帮助。最重要的是，不要把核心文件放进电子邮件 bug 报告！这么大的邮件会极其不便。

- Emacs 执行的系统调用跟踪（system-call trace）。

系统调用跟踪对某些特殊类型的调试非常有用，但在多数情况下它们提供的有用信息很少。因此奇怪的是，许多人似乎认为报告崩溃信息的_那个_方法就是发送系统调用跟踪。也许这是从调试没有源代码或调试符号的程序的经历中养成的习惯。

在大多数程序里，回溯（backtrace）通常比系统调用跟踪（system-call trace）有用得多。即使在 Emacs 中，简单的回溯一般也更有用，不过为了提供完整信息，你应当用变量值来补充回溯，并用 'pr' 以 Lisp 对象的形式把它们打印出来（见上文）。

- 该 bug 的补丁。

如果补丁写得好，那它是有用的。但不要因为以为补丁已经足够，就省略 bug 报告所需的其他信息，例如测试用例。我们可能会发现你的补丁有问题，从而决定用另一种方式修复；也可能完全看不懂它。而且，如果我们无法理解你想修复的是什么 bug，或者为什么你的补丁算是改进，我们就不能安装它。关于如何让补丁易于我们理解和安装的指导，参见“为 GNU Emacs 发送补丁”。

- 关于该 bug 是什么、或它取决于什么的猜测。

这类猜测通常是错的。即使是专家，如果不先用调试器查明事实，也无法把这类事情猜对。

如果你愿意调试 Emacs 并提供该 bug 的额外信息，这里有一些有用的建议：

- 如果该 bug 表现为错误消息，试着提供该错误的 Lisp 回溯。要生成该错误的回溯，在错误发生之前使用‘M-x toggle-debug-on-error’（也就是说，你必须先执行该命令，然后让 bug 发生）。这会使错误启动 Lisp 调试器，从而显示回溯。把调试器回溯的文本复制到 bug 报告中。（如果在触发错误之前先加载相关的 Lisp ‘*.el’ 源文件，回溯会更详细，所以如果你知道如何找到并加载这些文件，就那样做。）

要调试该错误，我们建议使用 Edebug。关于用 Edebug 包调试 Emacs Lisp 程序的信息，参见 (elisp)Edebug 节点。

只有当你懂得如何让该 bug 再次发生时，才能这样使用调试器。如果你无法让它再次发生，至少把完整的错误消息复制下来。

- 如果 Emacs 似乎陷入无限循环或非常漫长的操作，在变量 ‘debug-on-quit’ 非 ‘nil’ 时键入 ‘C-g’ 会启动 Lisp 调试器并显示回溯。这种回溯对调试这类长循环很有用，所以如果你能生成它，就把它复制到 bug 报告中。

如果你无法让 Emacs 响应 ‘C-g’（例如因为设置了 ‘inhibit-quit’），那么你可以试着从 Emacs 外部发送 ‘debug-on-event’ 指定的信号（默认为 SIGUSR2），使其进入调试器。

- 来自 GDB 之类 C 调试器的额外信息，可能使别人能够在自己没有的机器上找到问题所在。如果你不知道如何使用 GDB，请阅读 GDB 手册——它并不长，而且使用 GDB 很容易。你可以在大多数能找到 Emacs 发行版的地方找到 GDB 发行版，包括在线形式的 GDB 手册。要在 GDB 下运行 Emacs，你应切换到编译 Emacs 的 ‘src’ 子目录，然后键入 ‘gdb ./emacs’。让 ‘src’ 目录成为当前目录很重要，这样 GDB 才会读取该目录下的 ‘.gdbinit’ 文件。（你也可以在 GDB 内部键入 ‘source ./.gdbinit’ 让 GDB 读取该文件。）

然而，如果你想让收集到的额外信息显示出导致该 bug 的原因，收集时就需要动动脑筋。

例如，许多人只发送一个 C 层级的回溯，但那本身并不很有用。带参数的简单回溯往往不能传达 GNU Emacs 内部正在发生什么，因为回溯中列出的大多数参数都是指向 Lisp 对象的指针。这些指针的数值毫无意义；唯一重要的是它们所指向对象的内容（而大多数内容本身又是指针）。

要提供有用的信息，你需要以 Lisp 记法显示 Lisp 对象的值。对靠近栈底的若干个栈帧中每个作为 Lisp 对象的变量都这样做。查看源码以确定哪些变量是 Lisp 对象，因为调试器把它们当作整数看待。

要以 Lisp 语法显示变量的值，先打印它的值，然后使用用户定义的 GDB 命令 ‘pr’ 以 Lisp 语法打印该 Lisp 对象。（如果你必须使用其他调试器，就以该对象为参数调用函数 ‘debug_print’。）‘pr’ 命令由文件 ‘.gdbinit’ 定义，只有在你调试的是正在运行的进程时（而不是 core dump）它才有效。

要让 Lisp 错误使 Emacs 停下并返回 GDB，请在 'Fsignal' 处设置断点。

要获得正在运行的 Lisp 函数的回溯，请键入 GDB 命令 'xbacktrace'。

文件 '.gdbinit' 还定义了其他几个对检查 Lisp 对象的数据类型和内容有用的命令。它们的名字以 'x' 开头。这些命令比 'pr' 层次更低，用起来也没那么方便，但在 'pr' 失效时它们可能仍然有效，例如调试 core dump 时，或 Emacs 收到致命信号时。

关于调试 Emacs 的更详细建议和其他有用技巧，见 Emacs 发行版中的文件 'etc/DEBUG'。该文件还包含调查 Emacs 停止响应之类问题（许多人以为 Emacs 已经"挂起"，而实际上它可能正处于无限循环中）的说明。

要找到你所安装的 Emacs 中的文件 'etc/DEBUG'，请使用变量 'data-directory' 中保存的目录名。

54.5 为 GNU Emacs 发送补丁

如果你愿意为 GNU Emacs 编写 bug 修复或改进，那非常有帮助。发送改动时，请遵循以下准则，以便维护者能方便地使用它们。如果你不遵循这些准则，你的信息可能仍然有用，但使用它要花费额外的工夫。维护 GNU Emacs 即使在最好的情况下也是一项繁重的工作，除非你尽力帮忙，否则我们跟不上。

每个补丁都必须包含若干信息，我们才能恰当地评估它。下面分别说明。

当你备齐这些内容后，使用 'M-x submit-emacs-patch' 命令发送补丁。该命令会提示你输入补丁的 Subject（主题）和一个补丁文件。然后它会创建并显示一个 Message 模式缓冲区，把补丁文件作为附件，并让你进一步说明该补丁，以及按下面要求添加其他信息。完成后，键入 'C-c C-c' 通过电子邮件把补丁发送给开发者。它会被发到位于 <https://debbugs.gnu.org> 的 GNU Bug Tracker。跟踪系统会像处理 bug 报告那样给你的提交分配一个编号。开发者通常会有回应，也许会向你询问更多细节或其他补充信息，所以务必附上有效的回复邮箱地址。

以下是我们要求你在提交补丁时提供的内容：

- 说明你在修复什么问题，或者这些补丁会带来什么改进：
- 对于现有 bug 的修复，最好回复 'bug-gnu-emacs' 列表中相关的讨论，或位于 <https://debbugs.gnu.org> 的 GNU Bug Tracker 中相应的 bug 条目。解释你的改动为什么能修复该 bug。
- 对于新特性，请包含该特性的描述以及你的实现。
- 对于新 bug，请包含针对你认为已修复的问题的完整 bug 报告；*note Checklist for Bug Reports: Checklist。在安装之前，我们需要说服自己该改动是正确的。即使它确实正确，如果我们没有办法重现它试图修复的问题，也可能难以理解它。
- 在你的代码改动中包含所有适当的注释，以帮助将来阅读源码的人理解为什么需要这项改动。
- 不要把出于不同原因所作的改动混在一起。要_分别_发送它们。

如果你出于不同原因做了两处改动，我们可能不想把它们都安装。我们可能只想安装其中一个，或者把它们分别安装到不同版本的 Emacs 中。如果你把它们胡乱混在一组 diff 里发送，我们就得付出额外工作量把它们理清——弄清哪些部分服务于哪个目的。如果我们没有时间做这件事，可能不得不把你的补丁长期搁置。

如果你每写完一项改动就立即连同它自己的说明发送，那么两项改动就永远不会纠缠在一起，我们也能恰当地考虑每一项，而无需额外工作去理清它们。

- 每项改动一完成就发送。有时人们以为积攒许多改动一起发送是在帮我们。如上所述，这绝对是最糟糕的做法。

既然你应当分别发送每项改动，那不妨立刻发送。这样如果它很重要，我们就能够选择立即安装它。

- 补丁本身。可以用下列方式之一生成：

- 如果你在使用 Emacs 仓库，请确保你的副本是最新的（例如用‘git pull’）。你可以把改动提交到一个私有分支，然后用‘git format-patch master’基于 master 版本生成补丁。（这是首选方法，因为它让我们应用补丁的工作更容易。）或者你也可以让改动保持未提交状态，按下面的说明使用‘git diff’。

- 用‘diff -u’生成 diff。如果你有 GNU diff，在生成 C 代码的 diff 时使用‘diff -u -F'^[_a-zA-Z0-9$]\+ *('。它会显示每项改动所在函数的名称。

生成 diff 时，要避免旧版本与新版本之间的任何歧义。请把旧版本作为 diff 的第一个参数，新版本作为第二个参数。并且请给其中一个版本起个名字，用以表明它是旧版本还是你改动的新版本。

- 为你的改动编写提交日志条目。这既能省去我们撰写它们的额外工作，也有助于解释你的改动，使我们能够理解。

提交日志的目的是解释改动的理由、修改后的代码如何解决你的补丁试图修复的问题，并向人们展示到哪里去找改动的内容。因此你需要具体说明改了哪些函数以及为什么。关于我们对良好提交日志消息的风格与要求的细节，请参阅 Emacs 源码树中文件‘CONTRIBUTE’的“Commit messages”一节。

还请看看最近提交的提交日志条目，了解该放入什么样的信息，并学习我们使用的风格。注意，与某些其他项目不同，我们确实要求为文档（即 Texinfo 文件）编写提交日志。参见“变更日志”、参见《《standards》手册》的“Change Log Concepts”。

- 编写修复时，请记住我们不能安装会破坏其他系统的改动。请考虑你的改动在另一种类型的系统上编译和/或使用时会产生什么影响。

有时人们发来的修复_可能_是具有普遍意义的改进——但很难对此有把握。安装这类改动很难，因为我们必须非常仔细地研究它们。当然，如果你好好解释得出该改动正确这一结论的推理过程，会有助于说服我们。

最安全的改动是对仅用于特定机器或特定系统的文件或文件部分的改动。这些改动安全，是因为它们不会在其他机器或系统上制造新 bug。

请把补丁设计成明显可以安全安装的形式，以帮助我们应付工作量。

55 参与 Emacs 开发

Emacs 是一个协作项目，我们鼓励任何人贡献自己的力量。

为 Emacs 做贡献的方式有很多：

- 查找并报告缺陷；参见“报告 Bug”。
- 在 Emacs 用户邮件列表 <https://lists.gnu.org/mailman/listinfo/help-gnu-emacs> 上回答问题。
- 编写文档，可以写在 wiki（https://www.emacswiki.org/）上，也可以写进 Emacs 源代码仓库（参见“为 GNU Emacs 发送补丁”）。
- 检查已有的缺陷报告在更新版本的 Emacs 中是否已修复 <https://debbugs.gnu.org/cgi/pkgreport.cgi?which=pkg&data=emacs>。
- 修复已有的缺陷报告。
- 实现 Emacs 发行版中 ‘etc/TODO’ 文件里列出的某项功能，并提交补丁。
- 实现一项新功能，并提交补丁。
- 开发可与 Emacs 配合使用的包，并自行发布或发布到 GNU ELPA（<https://elpa.gnu.org/>）。
- 把 Emacs 移植到新平台，不过如今这并不常见。

如果你想参与改进 Emacs，请通过 <emacs-devel@gnu.org> 联系维护者。你可以询问有哪些建议的项目，也可以提出自己的想法。

如果你有功能需求，或对如何改进 Emacs 有建议，最好发送到 <bug-gnu-emacs@gnu.org>。请尽可能清楚地说明你希望看到什么改动，以及你认为它为什么会、以及如何能改进 Emacs。

如果你已经写好了某项改进，请告诉我们。如果你还没有开始动手，那么在开始之前联系 <emacs-devel@gnu.org> 会很有帮助；他们或许能建议一些方法，让你的扩展与 Emacs 的其他部分更好地配合。

实现功能时，请遵循 Emacs 编码规范；参见“编码规范”。此外，实质性的贡献需要将版权转让给 FSF；参见“版权转让”。

Emacs 的开发版本可以从仓库下载，那里有一组开发者积极地维护它。访问方式见 Emacs 项目页面 <https://savannah.gnu.org/projects/emacs/>。

重要的是，你的补丁要基于当前的工作版本编写。如果你从较旧的版本开始，你的补丁可能已经过时（维护者将很难应用它），或者 Emacs 中的改动可能已使你的补丁变得多余。下载仓库源代码之后，你应当阅读 ‘INSTALL.REPO’ 文件以了解构建说明（它们与普通构建在某种程度上有所不同）。

如果你想做出更广泛的贡献，请参阅 Emacs 源代码树中的 ‘CONTRIBUTE’ 文件，了解如何成为一名 Emacs 开发者。该文件作为每个已发布 Emacs 版本的源代码 tarball 的一部分分发，也可以在网上 Emacs 在线源代码仓库中找到（https://git.savannah.gnu.org/cgit/emacs.git/tree/CONTRIBUTE）。如果你按照 <https://savannah.gnu.org/projects/emacs/> 中的说明克隆了 Emacs 仓库，就会在源代码树的顶层目录中找到该文件。

关于 Emacs 的文档（以便理解如何实现你想要的改动），请参考：

- 参见《Emacs 手册》
- 参见《Emacs Lisp 参考手册》
- <https://www.gnu.org/software/emacs>
- <https://www.emacswiki.org/>
- 编码规范：GNU Emacs 的编码规范
- 版权转让：把版权转让给 FSF

55.1 编码规范

贡献的代码应当遵循 GNU 编码规范。该手册可在线获取，地址为 <https://www.gnu.org/prep/standards/>。它也可能以 Info 形式存在于你的本地系统中，参见《《standards》手册》。

如果代码不符合规范，我们就需要找人来修正它，然后才能使用。

Emacs 还有额外的风格与编码约定：

- 参见《Emacs Lisp 参考手册》的“Tips”
- 对于要纳入 Emacs 的 Lisp 代码，避免使用 ‘advice-add’ 或 ‘with-eval-after-load’。
- 删除所有源文件和文本文件中行尾的空白字符。
- 在 Lisp 代码中表示空格字符时，使用 ‘?\s’ 而非 ‘? ’。

55.2 版权转让

FSF（Free Software Foundation，自由软件基金会）是 GNU Emacs 的版权持有者。FSF 是一家非营利组织，其使命是在全世界促进计算机用户的自由，并捍卫所有自由软件用户的权利。一般信息请见网站 <https://www.fsf.org/>。

一般来说，对于向 GNU Emacs 以及存放在 GNU ELPA 中的包所做的非平凡贡献，我们要求把版权转让给 FSF。这样做的理由见 <https://www.gnu.org/licenses/why-assign.html>。

版权转让是一个简单的过程。许多国家的居民可以完全以电子方式完成。要开始办理，请按照 Emacs 发行版中 ‘etc/copyright-assign.txt’ 文件的说明操作。你若有任何疑问，我们可以在 <emacs-devel@gnu.org> 邮件列表上解答（或为你指出能解答的人）。

（请注意：关于某些 GNU 项目为何要求版权转让的一般性讨论，对 emacs-devel 而言属于跑题。请改到 gnu-misc-discuss 讨论。）

放弃版权声明也是一种可能，但我们更倾向于版权转让。请注意，与转让一样，放弃声明也需要你向 FSF 寄送签署好的文书（仅仅说“这属于公有领域”是不够的）。此外，放弃声明不能适用于将来的作品，每次你想提交新东西时都必须重复一次。

对于较小的改动（大致少于 15 行），我们可以接受而不要求版权转让。这是对你所有贡献累计计算的限额（例如，三个各自 5 行的补丁）。

56 获取 GNU Emacs 帮助

若你在安装、使用或修改 GNU Emacs 时需要帮助，有两种途径可以找到：

- 向邮件列表 <help-gnu-emacs@gnu.org> 发送消息，或在新闻组 'gnu.emacs.help' 上发布请求。（该邮件列表与新闻组互相连通，所以用哪一个都可以。）
- 在服务目录（https://www.fsf.org/resources/service/）中查找可能收费为你提供帮助的人。

附录 A GNU 通用公共许可证

第 3 版，2007 年 6 月 29 日

版权所有 © 2007 Free Software Foundation, Inc. <https://fsf.org/>

任何人都可以复制和分发本许可证文件的逐字副本，但不允许修改它。

前言

GNU 通用公共许可证是一种用于软件及其他类型作品的自由、copyleft 许可证。

大多数软件及其他实用作品的许可证旨在剥夺你共享和修改作品的自由。相比之下，GNU 通用公共许可证旨在保障你共享和修改程序所有版本的自由——确保它对所有用户始终是自由软件。我们自由软件基金会将 GNU 通用公共许可证用于我们的大多数软件；它也适用于作者以这种方式发布的其他任何作品。你也可以把它用于你的程序。

当我们谈到自由软件时，指的是自由，而不是价格。我们的通用公共许可证旨在确保你享有分发自由软件副本的自由（如果你愿意，也可以收费），确保你能收到源代码或在需要时能获得源代码，确保你能修改软件或将其中部分用于新的自由程序，并确保你知道自己可以做到这些事。

为了保护你的权利，我们需要防止他人否认你享有这些权利，或要求你放弃这些权利。因此，如果你分发软件的副本，或修改了软件，你就承担某些责任：尊重他人自由的责任。

例如，如果你分发此类程序的副本，无论免费还是收费，都必须把你自己获得的同样自由传递给接收者。你必须确保他们也能收到或能够获得源代码。而且你必须向他们展示这些条款，让他们知道自己的权利。

使用 GNU GPL 的开发者通过两个步骤保护你的权利：⑴ 主张软件的版权，⑵ 向你提供本许可证，授予你复制、分发和/或修改它的法律许可。

为保护开发者和作者，GPL 明确规定此自由软件不提供任何担保。为了用户和作者双方的利益，GPL 要求修改后的版本必须标明已被修改，以免它们遇到的问题被错误地归咎于先前版本的作者。

有些设备在设计上不允许用户安装或运行其内部软件的修改版本，尽管制造商自己可以这样做。这与保护用户修改软件自由的目标从根本上不相容。这种滥用的系统性模式出现在供个人使用的产品领域，而这恰恰是最不可接受的地方。因此，我们设计了本版 GPL 来禁止此类产品上的这种做法。如果此类问题在其他领域大量出现，我们随时准备在 GPL 的未来版本中把这一规定扩展到那些领域，以保护用户的自由。

最后，每个程序都不断受到软件专利的威胁。各国不应允许专利限制在通用计算机上开发和使用的软件，但在那些允许这样做的国家，我们希望避免一种特殊危险：针对自由程序的专利可能使其实际上变成专有软件。为防止这种情况，GPL 确保专利不能被用来使程序变得不自由。

下面是复制、分发和修改的确切条款与条件。

条款与条件

0. 定义

“本许可证”指 GNU 通用公共许可证第 3 版。

“版权”也指适用于其他类型作品的类似版权的法律，例如半导体掩模。

“本程序”指在本许可证下获得许可的任何可受版权保护的作品。每个被许可人都称为“你”。“被许可人”和“接收者”可以是个人或组织。

“修改”作品指以需要版权许可的方式复制或改编作品的全部或部分，但制作完全相同的副本除外。由此产生的作品称为先前作品的“修改版本”，或称为“基于”先前作品的作品。

“受保护作品”指未修改的本程序或基于本程序的作品。

“传播”作品指对作品做出未经许可即会使你依照适用的版权法承担直接或间接侵权责任的任何事情，但在计算机上运行它或修改私人副本除外。传播包括复制、分发（无论是否修改）、向公众提供，在某些国家还包括其他活动。

“传递”作品指使其他方能够制作或接收副本的任何一种传播。仅仅通过计算机网络与用户交互而不传输副本，不构成传递。

交互式用户界面显示“适当的法律声明”，是指它包含一个方便且显著可见的功能，该功能 (1) 显示适当的版权声明，并且 (2) 告知用户该作品不提供担保（除非有担保提供），告知被许可人可以依照本许可证传递该作品，以及告知如何查看本许可证的副本。如果界面呈现用户命令或选项的列表（例如菜单），则列表中一个显著的项目即满足此标准。

1. 源代码

作品的“源代码”指为了修改该作品而首选的形式。“目标代码”指作品的任何非源代码形式。

“标准接口”指这样的接口：它或者是由公认的标准机构定义的正式标准，或者，对于为某种特定编程语言规定的接口而言，是在使用该语言的开发者中广泛使用的接口。

可执行作品的“系统库”包括除该作品整体之外的任何东西，只要它 (a) 包含在打包某个主要组件的通常形式中，但并不属于该主要组件，且 (b) 仅用于使该作品能与该主要组件一起使用，或用于实现某个标准接口，而该接口有以源代码形式向公众提供的实现。“主要组件”在此语境中指可执行作品所运行的具体操作系统（如果有）的主要基本组件（内核、窗口系统等），或用于产生该作品的编译器，或用于运行该作品的目标代码解释器。

对应源代码不必包含用户可以从对应源代码的其他部分自动重新生成的任何东西。

以源代码形式存在的作品的对应源代码就是该作品本身。

2. 基本许可

本许可证授予的所有权利均在本程序的版权期限内授予，并且在满足所述条件的前提下不可撤销。本许可证明确确认你运行未修改的本程序的不受限许可。运行受保护作品所得输出，只有在该输出就其内容而言构成受保护作品时，才受本许可证约束。本许可证承认你依版权法享有的合理使用或其他同等权利。

只要你的许可在其他方面仍然有效，你就可以制造、运行和传播你不传递的受保护作品，而不受任何条件限制。你可以仅为让他人专为你做修改，或为你提供运行这些作品的设施，而将受保护作品传递给他人，前提是你在传递你并不掌控版权的所有材料时遵守本许可证的条款。为你制作或运行受保护作品的人必须完全代表你、在你的指导和控制之下进行，且其条件禁止他们在与你的关系之外制作你享有版权的材料的任何副本。

在任何其他情形下传递，仅在下述条件允许时才被许可。不允许再许可（sublicensing）；第 10 节使之没有必要。

3. 保护用户的合法权利免受反规避法律的侵害

在履行 1996 年 12 月 20 日通过的 WIPO 版权条约第 11 条义务的任何适用法律下，或在禁止或限制规避此类措施的类似法律下，任何受保护作品均不得被视为有效技术措施的一部分。

当你传递受保护作品时，你放弃任何禁止规避技术措施的法律权力，只要该规避是通过就受保护作品行使本许可证下的权利而实现的；并且你否认有任何意图，即把限制作品的运行或修改作为对作品用户强制执行你或第三方禁止规避技术措施的法律权利的手段。

4. 传递逐字副本

你可以用任何介质传递你收到的本程序源代码的逐字副本，前提是你在每份副本上显著而恰当地发布适当的版权声明；完整保留所有声明本许可证以及依照第 7 节添加的任何非许可性条款适用于该代码的告示；完整保留所有关于不提供任何担保的告示；并将本许可证的副本连同本程序一起交给所有接收者。

你可以为你传递的每份副本收取任何价格或不收费，也可以收费提供支持或担保保护。

5. 传递修改过的源代码版本

你可以依照第 4 节的条款，以源代码形式传递基于本程序的作品，或传递从本程序产生该作品所做的修改，前提是你也满足下列全部条件：

a. 该作品必须带有显著告示，说明你修改了它，并给出相关日期。

b. 该作品必须带有显著告示，说明它是在本许可证以及依照第 7 节添加的任何条件下发布的。此要求修改了第 4 节中“完整保留所有告示”的要求。

c. 你必须就整部作品整体，向任何获得副本的人以本许可证授予许可。因此，本许可证连同任何适用的第 7 节附加条款，将适用于整部作品及其所有部分，无论它们如何打包。本许可证不允许以任何其他方式为该作品授予许可，但如果你另行获得了此类许可，本许可证并不使其无效。

d. 如果该作品有交互式用户界面，则每个界面都必须显示适当的法律声明；但是，如果本程序有交互式界面不显示适当的法律声明，你的作品也不必使它们显示。

受保护作品与其他独立作品（这些作品本质上不是受保护作品的延伸，也未与之组合成更大的程序）在某个存储或分发介质的卷中或卷上汇编在一起，如果该汇编及其产生的版权未被用来把汇编用户的访问或法律权利限制到超出各单独作品所允许的范围，则称为“聚合体”。将受保护作品纳入聚合体，不会使本许可证适用于聚合体的其他部分。

6. 传递非源代码形式

你可以依照第 4 节和第 5 节的条款，以目标代码形式传递受保护作品，前提是你也依照本许可证的条款，以下列方式之一传递机器可读的对应源代码：

a. 在有形产品（包括有形分发介质）中或上传递目标代码，并附上固定在通常用于软件交换的耐用有形介质上的对应源代码。

b. 在有形产品（包括有形分发介质）中或上传递目标代码，并附上一份书面要约，其有效期至少三年，且只要你为该产品型号提供备件或客户支持就一直有效：向任何拥有该目标代码的人提供下列之一：⑴ 对产品中受

本许可证约束的所有软件，在通常用于软件交换的耐用有形介质上提供对应源代码副本，价格不超过你实际执行该源代码传递的合理成本；或 ⑵ 从网络服务器免费复制对应源代码的访问权。

c. 传递目标代码的单个副本，并附上提供对应源代码的书面要约副本。仅允许偶尔且非商业地采用这一替代方式，并且仅当你依照第 6b 小节连同该要约一起收到该目标代码时才可如此。

d. 通过从指定地点提供访问（免费或收费）的方式传递目标代码，并以同样方式通过同一地点免费提供对对应源代码的同等访问。你不必要求接收者随目标代码一起复制对应源代码。如果复制目标代码的地点是网络服务器，对应源代码可以位于另一台支持同等复制设施的服务器（由你或第三方运营）上，只要你紧邻目标代码处保留清晰的指引，说明在哪里可以找到对应源代码。无论由哪台服务器托管对应源代码，你仍有义务确保它在满足这些要求所需的期间内可用。

e. 使用点对点传输传递目标代码，前提是你告知其他节点，依照第 6d 小节，该作品的目标代码和对应源代码在何处免费向公众提供。

目标代码中可分离的部分，如果其源代码作为系统库被排除在对应源代码之外，则无需在传递该目标代码作品时包含在内。

“用户产品”指下列之一：⑴“消费产品”，即通常用于个人、家庭或居家用途的任何有形个人财产；或 ⑵ 任何为装入住宅而设计或销售的东西。在判定某产品是否为消费产品时，存疑的情形应作出有利于覆盖的裁决。对于特定用户收到的特定产品，“通常使用”指该类产品的典型或常见使用，而不论该特定用户的状态，也不论该特定用户实际使用或期望、被期望使用该产品的方式。某产品是消费产品，无论该产品是否有大量商业、工业或非消费用途，除非此类用途构成该产品唯一重要的使用方式。

用户产品的“安装信息”指为了从对应源代码的修改版本，在该用户产品中安装和执行受保护作品的修改版本所需的方法、程序、授权密钥或其他信息。该信息必须足以确保修改后的目标代码的持续运行在任何情况下都不会仅仅因为进行了修改而被阻止或干扰。

如果你在本节下，在用户产品中、随用户产品一起或专门为用户产品使用而传递目标代码作品，并且该传递属于某项交易的一部分，而该交易将用户产品的占有权和使用权永久或定期转移给接收者（无论该交易如何定性），则依照本节传递的对应源代码必须附带安装信息。但如果你和任何第三方都不保留在用户产品上安装修改后的目标代码的能力（例如作品已安装到 ROM 中），则此要求不适用。

提供安装信息的要求，不包括为接收者已修改或安装的作品，或为其上已修改或安装该作品的用户产品，继续提供支持服务、担保或更新的要求。当修改本身实质性且不利地影响网络的运行，或违反跨网络通信的规则和协议时，可以拒绝网络访问。

依照本节传递的对应源代码和提供的安装信息，必须采用已公开文档化的格式（并且有以源代码形式向公众提供的实现），且解包、读取或复制时无需任何特殊密码或密钥。

7. 附加条款

“附加许可”是通过对本许可证的一项或多项条件作出例外来补充本许可证条款的条款。适用于整个本程序的附加许可，在其依适用法律有效的范围内，应视为已包含在本许可证中。如果附加许可仅适用于本程序的一部分，则该部分可以单独在那些许可下使用，但整个本程序仍受本许可证管辖，而不考虑附加许可。

当你传递受保护作品的副本时，你可以自行选择从该副本或其任何部分中移除任何附加许可。（附加许可可能规定，在你修改作品时的某些情形下必须移除它们。）对于你添加到受保护作品中、你拥有或能够给予适当版

权许可的材料，你可以为其设置附加许可。

尽管本许可证有任何其他规定，对于你添加到受保护作品中的材料，你可以（如果该材料的版权持有者授权）以如下条款补充本许可证的条款：

a. 以不同于本许可证第 15 节和第 16 节条款的方式放弃担保或限制责任；或

b. 要求在该材料中，或在包含该材料的作品所显示的适当法律声明中，保留指定的合理法律告示或作者署名；或

c. 禁止虚假表述该材料的来源，或要求此类材料的修改版本以合理方式标明与原始版本不同；或

d. 限制为宣传目的使用该材料的许可方或作者的名字；或

e. 拒绝授予商标法下使用某些商号、商标或服务标记的权利；或

f. 要求任何传递该材料（或其修改版本）并对接收者承担合同责任假设的人，就这些合同责任假设直接加诸该材料许可方和作者的任何责任，对该许可方和作者作出赔偿。

所有其他非许可性附加条款均视为第 10 节意义上的“进一步限制”。如果你收到的本程序或其任何部分包含告示，说明它受本许可证连同某项进一步限制的条款管辖，你可以移除该条款。如果某许可证文件包含进一步限制，但允许依照本许可证重新许可或传递，你可以把受该许可证文件条款管辖的材料添加到受保护作品中，前提是该进一步限制在此类重新许可或传递之后不再存续。

如果你依照本节向受保护作品添加条款，你必须在相关源文件中放置一份声明，说明适用于那些文件的附加条款，或者放置一份告示，指明在哪里可以找到适用的条款。

附加条款无论是许可性的还是非许可性的，都可以以单独书面许可证的形式陈述，或作为例外陈述；无论哪种方式，上述要求都适用。

8. 终止

除本许可证明确规定外，你不得传播或修改受保护作品。任何以其他方式传播或修改它的企图均属无效，并将自动终止你在本许可证下的权利（包括依照第 11 节第三段授予的任何专利许可）。

但是，如果你停止一切违反本许可证的行为，则你从某一特定版权持有者处获得的许可即予恢复：(a) 临时恢复，除非该版权持有者明确且最终终止你的许可，在此之前一直有效；(b) 永久恢复，如果该版权持有者未在你停止违约后 60 天内以某种合理方式通知你该违约行为。

此外，如果该版权持有者以某种合理方式通知你该违约行为，而这是你第一次收到该版权持有者关于违反本许可证（就任何作品而言）的通知，并且你在收到通知后 30 天内纠正了该违约行为，则你从该特定版权持有者处获得的许可即永久恢复。

依照本节终止你的权利，并不终止从你处依照本许可证获得副本或权利的第三方的许可。如果你的权利已被终止且未被永久恢复，则你没有资格依照第 10 节就同一材料获得新的许可。

9. 持有副本无需接受本许可证

为了接收或运行本程序的副本，你无需接受本许可证。仅仅由于使用点对点传输接收副本而发生的受保护作品的附带传播，同样不需要接受。但是，除本许可证外，没有任何东西授予你传播或修改任何受保护作品的许

可。如果你不接受本许可证，这些行为即构成侵犯版权。因此，通过修改或传播受保护作品，你表示接受本许可证并如此行事。

10. 自动许可下游接收者

每次你传递受保护作品时，接收者都自动从原始许可方获得许可，以依照本许可证运行、修改和传播该作品。你没有责任强制第三方遵守本许可证。

“实体交易”（entity transaction）指转移某组织的控制权、或转移某组织实质上全部资产的交易，也指拆分某组织或合并组织。若某受保护作品的传播源于一笔实体交易，则该交易的每一方在收到该作品的一份副本时，也就同时获得其利益前手依据上一段所拥有或所能给予的、针对该作品的各种许可；此外，若该利益前手持有该作品的对应源代码，或能以合理努力取得它，则该方还有权从该前手处取得该作品的对应源代码。

你不得对本许可所授予或确认的权利的行使施加任何进一步限制。例如，你不得对行使本许可所授予的权利收取许可费、版税或其他费用；也不得提起诉讼（包括诉讼中的交叉请求或反诉），主张制造、使用、销售、许诺销售或进口本程序或其任何部分侵犯了任何专利权利要求。

1. 专利。

“贡献者”（contributor）指依据本许可授权使用本程序、或使用本程序所基于的作品的版权持有人。如此获得许可的作品称为该贡献者的“贡献者版本”（contributor version）。

贡献者的“关键专利权利要求”（essential patent claims）指该贡献者拥有或控制的所有专利权利要求，无论是已经取得还是今后取得，只要以本许可所允许的某种方式制造、使用或销售其贡献者版本就会侵犯这些权利要求；但不包括仅因对贡献者版本作进一步修改才会被侵犯的权利要求。就本定义而言，“控制”包括以符合本许可要求的方式授予专利再许可的权利。

每位贡献者就其关键专利权利要求，向你授予一项非排他、全球范围、免版税的专利许可，以便制造、使用、销售、许诺销售、进口以及以其他方式运行、修改和传播其贡献者版本的内容。

在以下三段中，“专利许可”指任何明示的协议或承诺，不论其名目如何，其内容是不实施某项专利（例如明示允许实施某项专利，或承诺不就专利侵权提起诉讼）。向某方“授予”这样的专利许可，指作出这样的协议或承诺：不对该方实施某项专利。

若你在传递某受保护作品时明知而依赖某项专利许可，而该作品的对应源代码并未通过公开可用的网络服务器或其他易于访问的方式，以免费且符合本许可条款的条件供任何人复制，那么你必须做到下列之一：⑴ 使该对应源代码以这种方式可用；⑵ 作出安排，使自己丧失就该特定作品享有的专利许可利益；⑶ 以符合本许可要求的方式作出安排，将专利许可延伸至下游接收者。“明知而依赖”指你实际知晓：若非该专利许可，你在某国传递该受保护作品、或你的接收者在某国使用该受保护作品，就会侵犯该国一项或多项可识别的专利，而你有理由相信这些专利是有效的。

若依据某单笔交易或安排、或与之相关，你传递了某受保护作品，或通过促成传递而传播了某受保护作品，并向接收该受保护作品的某些方授予专利许可，授权他们使用、传播、修改或传递该受保护作品的特定副本，则你所授予的专利许可自动延伸至该受保护作品以及基于该作品的作品的所有接收者。

若某专利许可未将本许可明确授予的一项或多项权利纳入其覆盖范围、禁止行使这些权利，或以不行使这些权利为条件，则该许可即为“歧视性”许可。若你是与某第三方（其业务为分发软件）达成的安排的当事方，而根据该安排你依自己传递该作品的活动规模向该第三方付款，并且根据该安排该第三方将歧视性专利许可授予任

何将从你处接收该受保护作品的当事方——(a) 与该受保护作品的副本（你所传递的副本或由这些副本制作的副本）相关，或 (b) 主要针对并涉及包含该受保护作品的特定产品或汇编——则你不得传递该受保护作品，除非你订立该安排、或该专利许可获得授予的时间早于 2007 年 3 月 28 日。

本许可中的任何内容均不得解释为排除或限制你在适用专利法下本来可能享有的任何默示许可或其他侵权抗辩。

1. 不得放弃他人的自由。

若对你施加的条件（无论是法院命令、协议还是其他方式）与本许可的条件相抵触，这些条件并不能免除你遵守本许可条件的义务。若你传递某受保护作品时无法同时满足本许可下的义务和任何其他相关义务，则其后果是你完全不得传递该作品。例如，若你同意的条款要求你向从你处接收本程序的人就后续传递收取版税，那么同时满足这些条款与本许可的唯一办法就是完全不传递本程序。

1. 与 GNU Affero 通用公共许可证一起使用。

尽管本许可能有任何其他规定，你仍获准将任何受保护作品与依据 GNU Affero 通用公共许可证第 3 版许可的作品链接或结合为单一结合作品，并传递由此产生的作品。本许可的条款将继续适用于其中属于受保护作品的那一部分，但 GNU Affero 通用公共许可证第 13 节关于通过网络进行交互的特殊要求将适用于该结合作品本身。

1. 本许可的修订版本。

自由软件基金会可能不时发布 GNU 通用公共许可证的修订版和/或新版本。此类新版本在精神上与本版本相似，但在细节上可能有所不同，以应对新问题或新关切。

每个版本都有一个可资区分的版本号。若本程序指明某一编号版本的 GNU 通用公共许可证“或任何更高版本”适用于它，你可以选择遵循该编号版本的条款和条件，或遵循自由软件基金会发布的任何更高版本的条款和条件。若本程序未指明 GNU 通用公共许可证的版本号，则你可以选择自由软件基金会曾发布的任何版本。

若本程序指明某代理机构可决定今后 GNU 通用公共许可证的哪些版本可以使用，则该代理机构公开声明接受某版本，即永久授权你为本程序选择该版本。

后续许可版本可能赋予你额外或不同的许可。但是，你选择遵循后续版本，并不会因此对任何作者或版权持有人施加额外义务。

1. 担保免责声明。

在适用法律允许的范围内，本程序不提供任何担保。除另有书面声明外，版权持有人和/或其他方按“原样”提供本程序，不提供任何形式的明示或默示担保，包括但不限于对适销性和适于特定用途的默示担保。本程序的质量和性能的全部风险由你承担。若证明本程序有缺陷，所有必要的服务、修复或纠正费用由你承担。

1. 责任限制。

除适用法律要求或经书面同意外，任何版权持有人、或任何按上述许可修改和/或传递本程序的其他方，在任何情况下均不就你的损害承担责任，包括因使用或无法使用本程序而产生的任何一般、特殊、偶然或后果性损害（包括但不限于数据丢失或数据变得不准确、你或第三方蒙受的损失，或本程序无法与任何其他程序协同运行），即使该持有人或其他方已被告知发生此类损害的可能性。

1. 第 15 节和第 16 节的解释。

若上文提供的担保免责声明和责任限制按其条款无法在当地产生法律效力，审查法院应适用与该程序相关的一切民事责任上最接近于绝对放弃的当地法律，除非有担保或责任承担随本程序的副本一并提供并收取费用。

条款和条件结束

如何将这些条款用于你的新程序

若你开发新程序，并希望它能为公众所最大程度地使用，那么达到这一点的最好办法是使它成为自由软件，让每个人都能依据这些条款再分发和修改它。

为此，请将下列声明附在程序中。最安全的做法是将它们附在每个源文件的开头，以最有效地声明排除担保；每个文件至少应包含“版权”行，以及说明完整声明所在位置的指引。

用一行给出程序的名称以及对其功能的简要说明。

Copyright (C) YEAR NAME OF AUTHOR

本程序是自由软件：你可以依据自由软件基金会发布的 GNU 通用公共许可证的条款（第 3 版，或你选择的任何更高版本）再分发和/或修改它。

本程序的分发是希望它能有用，但不提供任何担保；甚至不提供适销性或适于特定用途的默示担保。详见 GNU 通用公共许可证。

你应当已随本程序收到 GNU 通用公共许可证的副本。如果没有，请参见 <https://www.gnu.org/licenses/>。

还要加入如何通过电子邮件和纸质邮件与你联系的信息。

若程序具有终端交互功能，请在它以交互方式启动时输出类似下面的简短声明：

PROGRAM Copyright (C) YEAR NAME OF AUTHOR 本程序绝对不提供任何担保；详情请键入 ‘show w’。 这是自由软件，欢迎你在某些条件下再分发；详情请键入 ‘show c’。

假想的命令 ‘show w’ 和 ‘show c’ 应显示通用公共许可证的相应部分。当然，你的程序的命令可能有所不同；对于 GUI 界面，你会使用“关于”框。

若有必要，你还应让你的雇主（若你的工作是程序员）或学校为该程序签署一份“版权免责声明”。有关此事的更多信息，以及如何应用和遵循 GNU GPL，请参见 <https://www.gnu.org/licenses/>。

GNU 通用公共许可证不允许将你的程序并入专有程序。若你的程序是子例程库，你可能会认为允许专有应用程序链接该库更有用。若这正是你想要的，请改用 GNU 宽通用公共许可证（GNU Lesser General Public License）而不是本许可。但首先请阅读 <https://www.gnu.org/licenses/why-not-lgpl.html>。

附录 B GNU 自由文档许可证

版本 1.3，2008 年 11 月 3 日

版权所有 © 2000、2001、2002、2007、2008 Free Software Foundation, Inc. <https://fsf.org/>

任何人都可以复制和分发本许可证文档的逐字副本，但不允许修改。

0. 序言

本许可证的目的是使一本手册、教科书或其他功能性的、有用的文档在自由的意义上成为“自由的”：确保每个人都有有效的自由去复制和再分发它，无论是否修改，也无论是商业性还是非商业性。其次，本许可证为作者和出版者保留了一种为其工作获得赞誉的方式，同时不必为他人所作的修改负责。

本许可证是一种“copyleft”（著佐权），这意味着文档的衍生作品本身也必须以同样的意义保持自由。它与 GNU 通用公共许可证互补，后者是为自由软件设计的 copyleft 许可证。

我们设计本许可证是为了将其用于自由软件的手册，因为自由软件需要自由的文档：自由程序应当附带提供与软件本身相同自由的手册。但本许可证并不限于软件手册；它可用于任何文本作品，不论主题为何，也不论是否以印刷书籍形式出版。我们主要推荐将本许可证用于以教学或参考为目的的作品。

1. 适用范围与定义

本许可证适用于任何介质中、包含版权持有者所放置的声明——声明该作品可依本许可证条款分发——的任何手册或其他作品。此种声明授予一项全球范围、免版税、无期限限制的许可，允许按本文所述条件使用该作品。下文中的“文档”（Document）指任何此类手册或作品。公众中的任何成员都是被许可人，并被称作“你”。如果你以版权法要求获得许可的方式复制、修改或分发该作品，即表示你接受本许可证。

文档的“修改版本”（Modified Version）指包含文档或其一部分的任何作品，无论是逐字复制，还是经过修改和/或翻译成另一种语言。

“次要章节”（Secondary Section）是文档中一个具名的附录或前言部分，它专门讨论文档的出版者或作者与文档整体主题（或相关事项）之间的关系，并且不包含任何可直接归入该整体主题的内容。（因此，如果文档有一部分是数学教科书，次要章节不得讲解任何数学内容。）这种关系可以是与该主题或相关事项的历史关联，也可以是关于它们的法律、商业、哲学、伦理或政治立场。

“不变章节”（Invariant Sections）是某些次要章节，其标题在声明文档依本许可证发布的通告中被指定为不变章节的标题。如果某章节不符合上述次要章节的定义，则不允许将其指定为不变章节。文档可以包含零个不变章节。如果文档未指明任何不变章节，则不存在不变章节。

“封面文字”（Cover Texts）是某些短文，在声明文档依本许可证发布的通告中被列为前封面文字或后封面文字。前封面文字最多 5 个词，后封面文字最多 25 个词。

文档的“透明”（Transparent）副本指一种机器可读副本，以某种其规范可为公众获得的格式表示，适合用通用文本编辑器直接修订（对于由像素构成的图像，则用通用绘图程序；对于图形，则用某种广泛可用的绘图编辑器），并且适合作为文本格式化程序的输入，或适合自动转换为多种适合作为文本格式化程序输入的格式。以其他方面透明的文件格式制作、但其标记的有无被安排成阻挠或阻碍读者后续修改的副本不是透明的。如果图像格式被用于任何大量文本，则该格式不是透明的。不是“透明”的副本称为“不透明”（Opaque）副本。

适合透明副本的格式的例子包括不含标记的纯 ASCII、Texinfo 输入格式、LaTeX 输入格式、使用公开可用 DTD 的 SGML 或 XML，以及为便于人工修改而设计的符合标准的简单 HTML、PostScript 或 PDF。透明图像格式的例子包括 PNG、XCF 和 JPG。不透明格式包括只能由专有字处理程序读取和编辑的专有格式、其 DTD 和/或处理工具并非普遍可用的 SGML 或 XML，以及某些字处理程序仅为输出目的而生成的机器生成的 HTML、PostScript 或 PDF。

“标题页”（Title Page）对于印刷书籍而言，指书名页本身，加上为使本许可证要求在标题页上出现的材料清晰可读而必需的后续各页。对于没有书名义页的格式的作品，“标题页”指作品标题最显著出现处附近的文字，位于正文开始之前。

“出版者”（publisher）指任何向公众分发文档副本的个人或实体。

某章节“题为 XYZ”（Entitled XYZ）指文档中一个具名的子单元，其标题要么正好是 XYZ，要么在译成另一种语言的文字之后以括号形式包含 XYZ。（此处 XYZ 代表下文提到的某个特定章节名，例如“Acknowledgements”、“Dedications”、“Endorsements”或“History”。）当你修改文档时，“保留其标题”（Preserve the Title）指该章节按此定义仍为“题为 XYZ”的章节。

文档可以在声明本许可证适用于该文档的通告旁边包含免责声明。这些免责声明被视为通过引用纳入本许可证，但仅限于免除担保的方面：这些免责声明可能具有的任何其他含义均属无效，对本许可证的含义没有影响。

2. 逐字复制

你可以任何介质、商业性或非商业性地复制和分发文档，条件是：在所有副本中重现本许可证、版权声明以及声明本许可证适用于该文档的许可通告，并且除本许可证的条件外不添加任何其他条件。你不得使用技术措施阻碍或控制你所制作或分发的副本的阅读或进一步复制。但是，你可以接受以副本为交换的报酬。如果你分发的副本数量足够多，还必须遵守第 3 节的条件。

你也可以在上述相同条件下出借副本，并可以公开展示副本。

3. 大量复制

如果你出版文档的印刷副本（或在通常带有印刷封面的介质中的副本），数量超过 100，且文档的许可通告要求包含封面文字，你必须将这些副本包在封面上，清晰、可读地载有所有这些封面文字：前封面文字印在前封面上，后封面文字印在后封面上。两个封面还必须清晰、可读地标明你是这些副本的出版者。前封面必须呈现完整标题，标题中所有词语同等醒目、可见。你还可以在封面上额外添加其他材料。仅对封面作改动进行复制，只要保留了文档的标题并满足这些条件，在其他方面可以视为逐字复制。

如果任一封面所需的文字太多，无法清晰容纳，你应把列在最前面的那些文字（尽可能合理地容纳多少就放多少）放在实际封面上，其余文字续到相邻页面。

如果你出版或分发文档的不透明副本，数量超过 100，你必须要么为每一份不透明副本附上机器可读的透明副本，要么在每一份不透明副本中或随其说明一个计算机网络位置，普通使用网络的公众可从该位置使用公共标准网络协议下载该文档的完整透明副本，且不含附加材料。如果你选择后一种方式，你必须在开始大量分发不透明副本时采取合理审慎的步骤，确保该透明副本在所述位置保持如此可访问，直至你（直接或通过你的代理人或零售商）向公众分发该版本的某份不透明副本的最后一次之后至少一年。

我们请求（但不要求）你在再分发任何大量副本之前尽早联系文档的作者，以便他们有机会向你提供文档的更新版本。

4. 修改

你可以在上述第 2 节和第 3 节的条件下复制和分发文档的修改版本，条件是你恰以本许可证发布该修改版本，由修改版本充当文档的角色，从而将修改版本的分发和修改许可授予任何持有其副本的人。此外，在修改版本中你必须做到以下几点：

A. 在标题页（以及封面，如果有）上使用一个既不同于文档标题、也不同于先前各版本标题的标题（如果有先前版本，应将其列在文档的历史章节中）。如果该先前版本的原出版者允许，你可以使用与先前版本相同的标题。

B. 在标题页上作为作者列出一个或多个对修改版本中修改内容之著作权负责的个人或实体，并至少列出文档的五位主要作者（如果主要作者少于五位，则列出全部主要作者），除非他们免除你的这项要求。

C. 在标题页上声明修改版本出版者的名称，作为出版者。

D. 保留文档的所有版权声明。

E. 在其他版权声明旁边为你的修改添加适当的版权声明。

F. 在版权声明之后立即加入一份许可通告，按下文附录中所示的形式，授予公众依本许可证条款使用修改版本的许可。

G. 在该许可通告中保留文档许可通告中所给出的不变章节和所需封面文字的完整清单。

H. 附上本许可证的一份未经改动的副本。

I. 保留题为“History”的章节，保留其标题，并向其中添加一项，至少说明标题页上给出的修改版本的标题、年份、新作者和出版者。如果文档中没有题为“History”的章节，则创建一个，说明其标题页上给出的文档的标题、年份、作者和出版者，然后按上一句所述添加一项描述修改版本的内容。

J. 保留文档中给出的用于公众访问文档透明副本的网络位置（如果有），同样保留文档中给出的其所依据的先前版本的网络位置。这些可以放在“History”章节中。对于在文档本身之前至少四年出版的作品，你可以省略网络位置；或者，如果其所指版本的原出版者允许，也可以省略。

K. 对于任何题为“Acknowledgements”或“Dedications”的章节，保留该章节的标题，并在该章节中保留其中所载每一项贡献者致谢和/或献词的全部实质内容和语气。

L. 保留文档的所有不变章节，其文本和标题均不作改动。章节编号或等效标记不被视为章节标题的一部分。

M. 删除任何题为“Endorsements”的章节。此类章节不得包含在修改版本中。

N. 不得将任何现有章节改题为“Endorsements”，也不得使其标题与任何不变章节冲突。

O. 保留任何免责声明。

如果修改版本包含新的前言章节或附录，它们符合次要章节的定义，且不包含从文档复制的材料，你可以自行选择将其中部分或全部章节指定为不变的。为此，将其标题加入修改版本许可通告中的不变章节清单。这些标题必须区别于任何其他章节标题。

你可以添加一个题为“Endorsements”的章节，只要它仅包含各方对你的修改版本的认可——例如同行评议的声明，或该文本已被某组织批准为某标准的权威定义。

你可以在修改版本的封面文字清单末尾添加一段最多五个词的文字作为前封面文字，以及一段最多 25 个词的文字作为后封面文字。任何单个实体（或通过其所作安排）只能添加一段前封面文字和一段后封面文字。如果文档已包含针对同一封面的封面文字，且该文字先前由你添加或由你代其行事的同一实体安排添加，你不得再添加另一段；但经添加旧文字的原出版者明确许可，你可以替换旧文字。

文档的作者和出版者不因本许可证而准许将其姓名用于任何修改版本的宣传，或用于主张或暗示对任何修改版本的认可。

5. 合并文档

你可以将文档与依本许可证发布的其他文档合并，条件按上文第 4 节对修改版本的规定，前提是你在合并作品中包含所有原始文档的全部不变章节（不作修改），并在其许可通告中将它们全部列为你的合并作品的不变章节，且保留其全部免责声明。

合并作品只需包含一份本许可证，多个相同的不变章节可以用单份副本替代。如果有多个名称相同但内容不同的不变章节，请在每个此类章节标题末尾以括号形式加上该章节原作者或出版者的名称（如已知），否则加上一个唯一编号，使其标题唯一。对合并作品许可通告中不变章节清单里的章节标题作同样的调整。

在合并中，你必须将各原始文档中任何题为“History”的章节合并为一个题为“History”的章节；同样，合并任何题为“Acknowledgements”的章节，以及任何题为“Dedications”的章节。你必须删除所有题为“Endorsements”的章节。

6. 文档汇编

你可以制作一个由文档和依本许可证发布的其他文档组成的汇编，并用汇编中包含的单份本许可证副本替换各文档中的单独副本，条件是在所有其他方面对每份文档的逐字复制遵守本许可证的规则。

你可以从这样的汇编中取出单份文档，并依本许可证单独分发，条件是你在取出的文档中插入一份本许可证副本，并在关于该文档逐字复制的所有其他方面遵守本许可证。

7. 与独立作品聚合

文档或其衍生作品与其他独立、单独的文档或作品在某一存储或分发介质卷中的汇集，如果因该汇集而产生的版权未被用于将汇编用户的法律权利限制到超出各单个作品所允许的范围，则称为“聚合体”（aggregate）。当文档被包含在聚合体中时，本许可证不适用于聚合体中本身并非文档衍生作品的其他作品。

如果第 3 节的封面文字要求适用于文档的这些副本，那么当文档不足整个聚合体的一半时，文档的封面文字可以置于将文档框在聚合体之中的封面上；如果文档为电子形式，则置于等效的电子封面位置。否则，这些文字必须出现在框住整个聚合体的印刷封面上。

8. 翻译

翻译被视为一种修改，因此你可以按第 4 节的条款分发文档的译文。用译文替换不变章节需要其版权持有者的特别许可，但你可以在这些不变章节的原文之外另外包含部分或全部不变章节的译文。你可以包含本许可证的译文、文档中所有许可通告的译文以及任何免责声明的译文，条件是你同时包含本许可证的英文原版以及这些通告和免责声明的原版。如果本许可证或某项通告、免责声明的译文与原版之间存在分歧，以原版为准。

如果文档中某章节题为“Acknowledgements”、“Dedications”或“History”，（第 4 节）保留其标题（第 1 节）的要求通常需要更改实际标题。

9. 终止

除本许可证明确规定外，你不得复制、修改、再许可或分发文档。任何以其他方式复制、修改、再许可或分发它的尝试均属无效，并自动终止你依本许可证享有的权利。

但是，如果你停止一切违反本许可证的行为，那么来自特定版权持有者的许可即被恢复：(a) 暂时恢复，除非且直到该版权持有者明确并最终终止你的许可；(b) 永久恢复，如果该版权持有者未能在你停止违反后 60 天内以某种合理方式通知你该违反行为。

此外，如果特定版权持有者以某种合理方式通知你该违反行为，这是你第一次收到该版权持有者（就任何作品）发出的违反本许可证的通知，并且你在收到通知后 30 天内纠正了该违反行为，那么来自该版权持有者的许可即被永久恢复。

依本节终止你的权利并不终止那些依本许可证从你处获得副本或权利的当事人的许可。如果你的权利已被终止且未永久恢复，收到相同材料的部分或全部副本并不授予你使用它的任何权利。

10. 本许可证的未来修订

自由软件基金会可能不时发布 GNU 自由文档许可证的新的修订版本。此类新版本在精神上与当前版本相似，但可能在细节上有所不同，以应对新的问题或关切。参见 <https://www.gnu.org/licenses/>。

本许可证的每个版本都有一个可区分的版本号。如果文档指明某一特定编号版本的“本许可证或任何更高版本”适用于它，你可以选择遵循该指定版本的条款和条件，或遵循自由软件基金会已发布（非草案）的任何更高版本的条款和条件。如果文档未指明本许可证的版本号，你可以选择自由软件基金会曾经发布（非草案）的任何版本。如果文档指明可由某代理人决定使用本许可证的哪个未来版本，该代理人公开声明接受某版本即永久授权你为文档选择该版本。

11. 重新许可

“大规模多作者协作站点”（“Massive Multiauthor Collaboration Site”，简称“MMC 站点”）指任何发布可享版权的作品、同时为任何人提供醒目的编辑这些作品之设施的万维网服务器。任何人都可以编辑的公共 wiki 就是此类服务器的一个例子。站点中所包含的“大规模多作者协作”（“Massive Multiauthor Collaboration”，简称“MMC”）指如此在该 MMC 站点上发布的任何一组可享版权的作品。

“CC-BY-SA”指 Creative Commons Corporation（一家主要营业地位于加利福尼亚州旧金山的非营利性公司）发布的 Creative Commons Attribution-Share Alike 3.0 许可证，以及该组织发布的该许可证的未来 copyleft 版本。

“并入”（Incorporate）指将文档整体或部分作为另一文档的一部分发布或重新发布。

如果某 MMC 依本许可证获得许可，并且所有最初在本 MMC 之外某处依本许可证发布、随后整体或部分并入该 MMC 的作品 (1) 没有封面文字或不变章节，且 (2) 是在 2008 年 11 月 1 日之前如此并入的，则该 MMC“有资格重新许可”。

MMC 站点的运营者可以在 2009 年 8 月 1 日之前的任何时间，在同一站点上以 CC-BY-SA 重新发布该站点中所包含的 MMC，前提是该 MMC 有资格重新许可。

附录：如何为你的文档使用本许可证

要在你撰写的文档中使用本许可证，请在文档中包含一份本许可证副本，并将以下版权和许可通告放在紧接标题页之后：

Copyright (C) YEAR YOUR NAME. 允许任何人在自由软件基金会发布的 GNU 自由文档许可证 1.3 版或任何更高版本的条款下复制、分发和/或修改本文件；无不变章节、无前封面文字、无后封面文字。本许可证的副本包含在题为“GNU Free Documentation License”的章节中。

如果你有不变章节、前封面文字和后封面文字，请将“无不变章节、无前封面文字、无后封面文字”这一行替换为：

其中不变章节为 LIST THEIR TITLES，前封面文字为 LIST，后封面文字为 LIST。

如果你有不变章节但没有封面文字，或三者有其他组合，请合并这两种替代形式以适应当前情况。

如果你的文档包含非平凡的程序代码示例，我们建议你同时以你所选的一种自由软件许可证（例如 GNU 通用公共许可证）发布这些示例，以允许它们在自由软件中使用。

附录 C 启动 Emacs 的命令行参数

Emacs 支持命令行参数，以便在调用 Emacs 时请求各种操作。这些参数用于与其他编辑器兼容，也用于一些高级活动。我们不建议在普通编辑中使用它们（若要从命令行访问已有的 Emacs 作业，参见“把 Emacs 用作服务器”）。

以‘-’开头的参数是“选项”，‘+LINENUM’也是。其他所有参数都指定要访问的文件。Emacs 在启动时访问这些指定的文件。命令行中最后指定的文件成为当前缓冲区；其他文件也会在其他缓冲区中被访问。与大多数程序一样，特殊参数‘--’表示其后所有参数都是文件名而非选项，即使它们以‘-’开头。

Emacs 命令选项可以指定许多内容，例如 Emacs 所用 X 窗口的大小和位置、它的颜色等等。少数选项支持高级用法，例如在批处理模式下对文件运行 Lisp 函数。本章各节按用途分类介绍可用的选项。

选项有两种写法：以单个‘-’开头的短形式，以及以‘--’开头的长形式。例如，‘-d’是短形式，‘--display’是对应的长形式。

以‘--’开头的长形式更易记住，但键入起来更长。不过，你不必拼出整个选项名；任何无歧义的缩写都可以。当长选项需要参数时，可以用空格或等号分隔选项名与参数。因此，对于选项‘--display’，你可以写成‘--display sugar-bombs:0.0’，也可以写成‘--display=sugar-bombs:0.0’。我们推荐使用等号，因为它使关系更清楚，下面的表格也一律显示等号。

大多数选项指定如何初始化 Emacs，或为 Emacs 会话设置参数。我们称它们为“初始选项”。少数选项指定要做的事情，例如加载库或调用 Lisp 函数。这些称为“动作选项”。它们与文件名一起称为“动作参数”。动作参数以字符串列表的形式存储在变量‘command-line-args’中。（实际上，Emacs 启动时，‘command-line-args’包含从命令行传入的所有参数；在初始化期间，初始参数被处理时会从该列表中移除，只留下动作参数。）

- 动作参数：访问文件、加载库和调用函数的参数。
- 初始选项：启动 Emacs 时生效的参数。
- 命令示例：使用命令行参数的示例。
- 环境变量：Emacs 使用的环境变量。
- X 显示：更改默认显示并使用远程登录。
- X 字体：在 X 下为文本选择字体。
- X 颜色：选择显示颜色。
- X 窗口大小：X 下的启动窗口大小。
- X 边框：X 下的内部和外部边框。
- X 标题：指定初始框架的标题。
- X 图标：在 X 下选择使用何种图标。
- X 杂项：其他显示选项。

C.1 动作参数

下面是动作参数表：

‘FILE’ ‘--file=FILE’ ‘--find-file=FILE’ ‘--visit=FILE’ 访问指定的 FILE。参见“访问文件”。

当 Emacs 启动时，它在一个窗口中显示启动缓冲区，并在另一个窗口中显示访问 FILE 的缓冲区（参见“多窗口”）。如果你提供多个文件参数，所显示的文件是命令行中最后指定的那个；其他文件也会被访问，但它们的缓冲区不会显示。

如果启动缓冲区被禁用（参见“启动 Emacs”），那么用一个文件参数启动 Emacs 时，会在单个窗口中显示访问 FILE 的缓冲区。用两个文件参数时，Emacs 在两个不同的窗口中显示这两个文件。用多于两个文件参数时，Emacs 在一个窗口中显示最后指定的文件，另用一个窗口显示列出所有其他文件的缓冲区菜单（Buffer Menu）（参见“对多个缓冲区操作”）。要禁止为此使用缓冲区菜单，请把变量 ‘inhibit-startup-buffer-menu’ 改为 ‘t’。

‘+LINENUM FILE’ 访问指定的 FILE，然后转到其中的行号 LINENUM。

‘+LINENUM:COLUMNNUM FILE’ 访问指定的 FILE，然后转到行号 LINENUM，并把光标点放在列号 COLUMNNUM 处。

‘-l FILE’ ‘--load=FILE’ 用函数 ‘load’ 加载名为 FILE 的 Lisp 库。如果 FILE 不是绝对文件名，Emacs 先在当前目录中查找，然后在 ‘load-path’ 所列的目录中查找（参见“Emacs 的 Lisp 代码库”）。

警告：如果之前的命令行参数访问过文件，当前目录就是最后访问的那个文件所在的目录。

‘-L DIR’ ‘--directory=DIR’ 把目录 DIR 前加到变量 ‘load-path’。如果指定多个 ‘-L’ 选项，Emacs 保持它们的相对顺序；也就是说，使用 ‘-L /foo -L /bar’ 得到的 ‘load-path’ 形如 ‘("/foo" "/bar" ...)’。如果 DIR 以 ‘:’ 开头，Emacs 会去掉这个 ‘:’，并把其余部分追加（而不是前加）到 ‘load-path’。（在 MS Windows 上，改用 ‘;’ 代替 ‘:’；也就是说，使用 ‘path-separator’ 的值。）

注意，使用此选项对 ‘load-path’ 的改动*不会*影响 Emacs 查找站点启动文件以及由用户的 early-init 和 init 文件所加载的 Lisp 包时所用的目录（参见“Emacs 初始化文件”），因为 ‘load-path’ 是在这些文件被加载和解释*之后*才由 ‘-L’ 选项更改的。

‘-f FUNCTION’ ‘--funcall=FUNCTION’ 调用 Lisp 函数 FUNCTION。如果它是交互式函数（命令），它会交互地读取参数，就如同你用键序列调用同一函数一样。否则，它以无参数的方式调用该函数。

‘--eval=EXPRESSION’ ‘--execute=EXPRESSION’ 求值 Lisp 表达式 EXPRESSION。

‘--insert=FILE’ 把 FILE 的内容插入到处理此命令行参数时当前的缓冲区中。通常这是 ‘*scratch*’ 缓冲区（参见“Lisp 交互缓冲区”），但如果命令行中较早的参数访问了文件或切换了缓冲区，那可能是另一个缓冲区。此命令行参数的效果类似于 ‘M-x insert-file’ 所做的事（参见“其他文件操作”）。

‘--kill’ 不请求确认即退出 Emacs。

‘--help’ 打印一条用法信息，列出所有可用选项，然后成功退出。

‘--version’ 打印 Emacs 版本，然后成功退出。

‘--fingerprint’ 打印 Emacs 的“fingerprint”（指纹），它用于唯一标识 Emacs 的编译版本。

C.2 初始选项

初始选项为 Emacs 会话指定参数。本节介绍比较通用的初始选项；一些专门与 X Window System 相关的选项出现在后面几节中。

某些初始选项会影响初始化文件的加载。通常，Emacs 先加载 'site-start.el'（如果存在），然后加载你自己的初始化文件（如果存在），最后加载默认初始化文件 'default.el'（如果存在）（参见"Emacs 初始化文件"）。某些选项会阻止加载其中一些文件，或用其他文件替换它们。

'-chdir DIRECTORY' '--chdir=DIRECTORY' 在做其他任何事情之前切换到 DIRECTORY。这主要供 X 的会话管理使用，以便 Emacs 在它停止时所在的同一目录中启动。这使桌面的保存与恢复更容易。

'-t DEVICE' '--terminal=DEVICE' 用 DEVICE 作为终端输入和输出的设备。此选项隐含 '--no-window-system'。

'-d DISPLAY' '--display=DISPLAY' 使用 X Window System，并使用名为 DISPLAY 的显示来打开初始 Emacs 框架。更多细节参见"指定显示名称"。

'-nw' '--no-window-system' 不直接与窗口系统通信，忽略 'DISPLAY' 环境变量（即使它已设置）。这意味着 Emacs 使用启动它的那个终端进行所有显示和输入。

'-batch' '--batch' 以"批处理模式"运行 Emacs。批处理模式用于从 shell 脚本、makefile 等运行用 Emacs Lisp 编写的程序。要调用 Lisp 程序，请把 '-batch' 选项与 '-l'、'-f' 或 '--eval' 中的一个或多个结合使用（参见"动作参数"）。示例见参见"命令参数示例"。

在批处理模式下，Emacs 不显示正在编辑的文本，而像 'C-z' 和 'C-c' 这样的标准终端中断字符具有其通常效果：对 'C-c' 而言，其效果要么是退出 Emacs，要么是发出 'quit' 信号，取决于变量 'kill-emacs-on-sigint'。

通常会在回显区打印消息的 Emacs 函数，将改为打印到标准输出流（'stdout'）或标准错误流（'stderr'）。（确切地说，像 'prin1'、'princ' 和 'print' 这样的函数打印到 'stdout'，而 'message' 和 'error' 打印到 'stderr'。）通常从迷你缓冲区读取键盘输入的函数，则改为从终端的标准输入流（'stdin'）获取输入。

'--batch' 隐含 '-q'（不加载初始化文件），但仍会加载 'site-start.el'。它还会使 Emacs 在处理完所有命令选项后退出。此外，它禁用自动保存，除非某些缓冲区明确要求自动保存；保存文件时，除非另有要求，它会省略 'fsync' 系统调用。

运行 '--batch' Emacs 时发生的错误会导致打印 Emacs Lisp 回溯。要禁用此行为，请把 'backtrace-on-error-noninteractive' 设为 'nil'。

'--script FILE' 以批处理模式运行 Emacs（类似 '--batch'），然后读取并执行 FILE 中的 Lisp 代码。

此选项的通常用法是在运行 Emacs 的可执行脚本文件中。这类文件可以在第一行以如下文本开头：

```
#!/usr/bin/emacs --script
```

这将以 '--script' 调用 Emacs，并把脚本文件的名称作为 FILE 提供。Emacs Lisp 随后会把第一行中的 '#!' 视为注释定界符。

'-x' 此选项只能用在可执行脚本文件中，应像这样调用：

```
#!/usr/bin/emacs -x
```

这类似‘--script’，但会抑制加载初始化文件（类似‘--quick’），且不能用在普通命令行上（因为它没有指定要加载的脚本）。此外，当它到达脚本末尾时，它会退出 Emacs，并把最后一个表单的值作为脚本的退出值（如果最后一个值是数值）。否则，它总是以零值退出。注意，在这种情况下，Emacs 读取 Lisp 代码时会忽略任何文件局部变量（参见“指定文件变量”），无论是第一行中的，还是文件末尾附近的局部变量节中的。

‘--no-build-details’从 Emacs 可执行文件中省略系统名称和构建时间等细节，使构建更具确定性。此开关不用于常规（或交互式）使用，因为它会使‘system-name’之类的命令返回‘nil’。

‘-q’‘--no-init-file’不加载任何初始化文件（参见“Emacs 初始化文件”）。用此选项调用 Emacs 时，Customize 功能不允许保存选项（参见“简易自定义界面”）。此选项不会阻止加载‘site-start.el’。

‘--no-site-file’‘-nsl’不加载‘site-start.el’（参见“Emacs 初始化文件”）。‘-Q’选项也会这样做，但‘-q’等其他选项则不会。

‘--no-site-lisp’不把‘site-lisp’目录包含在‘load-path’中（参见“Emacs 初始化文件”）。‘-Q’选项也会这样做。

‘--init-directory’指定查找 Emacs 初始化文件时使用的目录。注意，这仅仅覆盖‘user-emacs-directory’的值——该目录通常是 Emacs 在查找你的初始化文件时顺带确定的（参见“Emacs 如何查找你的初始化文件”）——但不会改变对‘~/.emacs’初始化文件的查找。特别地，如果此选项所指定的目录中没有‘init.el’文件，Emacs 会找到并使用它在没有此选项时本会使用的‘init.el’（但会在本选项指定的目录中查找你的其他每用户 Emacs 文件）。如果你希望强制 Emacs 使用此目录中的‘init.el’文件，请在用此选项启动 Emacs 之前确保该文件存在。

‘--no-splash’不显示启动屏幕。你也可以在初始化文件中把变量‘inhibit-startup-screen’设为非‘nil’值来达到这一效果（参见“启动 Emacs”）。

‘--no-x-resources’不加载 X 资源。你也可以在初始化文件中把变量‘inhibit-x-resources’设为‘t’来达到这一效果（参见“X 资源”）。

‘-Q’‘--quick’以最少的自定义设置启动 Emacs。这类似于同时使用‘-q’、‘--no-site-file’、‘--no-site-lisp’、‘--no-x-resources’和‘--no-splash’。

‘-daemon’‘--daemon[=NAME]’‘--bg-daemon[=NAME]’‘--fg-daemon[=NAME]’把 Emacs 作为守护进程启动：Emacs 启动之后，会在不打开任何框架的情况下启动 Emacs 服务器。之后你就可以用‘emacsclient’命令连接 Emacs 进行编辑。（可选地，你可以为服务器指定显式的 NAME；如果指定了，在调用‘emacsclient’时需要通过其‘--socket-name’选项指定相同的 NAME，参见“‘emacsclient’选项”。）关于把 Emacs 用作守护进程的信息，参见“把 Emacs 用作服务器”。“后台”守护进程会与终端断开连接并在后台运行（‘--daemon’是‘--bg-daemon’的别名）。

‘--no-desktop’不重新加载任何已保存的桌面。参见“保存 Emacs 会话”。

‘-u USER’‘--user=USER’加载 USER 的初始化文件，而不是你自己的(1)。

> 注：此选项对 MS-Windows 无效。

‘--debug-init’为初始化文件中的错误启用 Emacs Lisp 调试器。参见《Emacs Lisp 参考手册》的“Error Debugging”。

‘--module-assertions’在处理动态可加载模块时启用开销很大的正确性检查。这面向希望验证其模块符合模块 API 要求的模块作者。此选项使 Emacs 在触发模块相关断言时中止。参见《Emacs Lisp 参考手册》的“Writing

Dynamic Modules”。

‘--dump-file=FILE’ 从指定的 FILE 加载已转储的 Emacs 状态。默认情况下，已安装的 Emacs 会在 Emacs 安装放置与体系结构相关文件的目录中查找名为 ‘EMACS.pdmp’ 的转储状态文件；变量 ‘exec-directory’ 保存该目录的名称。EMACS 是 Emacs 可执行文件的名称，通常就是 ‘emacs’。（当你从未安装的构建目录 ‘src’ 中调用 Emacs 时，它会在可执行文件所在目录中查找转储文件。）如果你重命名了转储文件或把它移到了别处，可以用此选项告诉 Emacs 到哪里去找该文件。

C.3 命令参数示例

下面是一个带参数和选项使用 Emacs 的示例。它假设你有一个名为 ‘hack-c.el’ 的 Lisp 程序文件，该文件加载后会对当前缓冲区（预期是一个 C 程序）执行一些有用的操作。

```
emacs --batch foo.c -l hack-c -f save-buffer >& log
```

这条命令的意思是：访问 ‘foo.c’，加载 ‘hack-c.el’（它会对所访问的文件做些修改），保存 ‘foo.c’（注意 ‘save-buffer’ 正是 ‘C-x C-s’ 所绑定的函数），然后退出回到 shell（因为 ‘--batch’）。‘--batch’ 还保证把输出重定向到 ‘log’ 不会出问题，因为 Emacs 不会假定自己有一个可供使用的显示终端。

C.4 环境变量

“环境”（environment）是操作系统的一项特性；它由一组带名称和值的变量组成。每个变量称为一个“环境变量”；环境变量名区分大小写，习惯上只使用大写字母。其值都是文本字符串。

环境之所以有用，是因为子进程会自动从父进程继承环境。这意味着你可以在登录 shell 中设置一个环境变量，而你运行的所有程序（包括 Emacs）都会自动看到它。Emacs 的子进程（例如 shell、编译器和版本控制程序）也会从 Emacs 继承环境。

在 Emacs 内部，命令 ‘M-x getenv’ 读取环境变量的名称，并在回显区打印它的值。‘M-x setenv’ 在 Emacs 环境中设置变量，‘C-u M-x setenv’ 删除变量。（在值中可以用 ‘$’ 进行环境变量替换，就像在文件名中一样；参见 “File Names with $”。）变量 ‘initial-environment’ 保存 Emacs 继承的初始环境。

在 Emacs 之外设置环境变量的方法取决于操作系统，尤其是你使用的 shell。例如，下面是用 Bash 把环境变量 ‘ORGANIZATION’ 设为 ‘not very much’ 的方法：

```
export ORGANIZATION="not very much"
```

下面是在 csh 或 tcsh 中做同样事情的方法：

```
setenv ORGANIZATION "not very much"
```

当 Emacs 使用 X Window System 时，各种控制 X 的环境变量对 Emacs 也同样有效。更多信息请参阅 X 的文档。

C.4.1 通用变量

下面按字母顺序列出在 Emacs 中有特殊含义的环境变量。这些变量大多也被其他一些程序使用。Emacs 不要求设置其中任何一个环境变量，但如果设置了，它会使用它们的值。

‘CDPATH’ 当你在 ‘cd’ 命令中指定相对目录时，该命令用它来搜索你指定的目录。

‘COLORTERM’ 如果此变量被设为 ‘truecolor’，它告诉 Emacs 即使在未安装 terminfo 数据库时也在文本模式显示中使用 24 位真彩色。Emacs 将使用内置命令通过 RGB 值请求真彩色，而不是依赖缺失的 terminfo 信息。

‘DBUS_SESSION_BUS_ADDRESS’ 当 Emacs 在编译时包含 D-Bus 支持时，此变量供 D-Bus 使用。通常无需更改它。把它设为一个虚拟地址（例如 ‘unix:path=/dev/null’），会抑制与 D-Bus 会话总线的连接，也会抑制在 D-Bus 会话总线尚未运行时自动启动它。

‘EMACSDATA’ 存放 Emacs 自带的、与体系结构无关的文件的目录。它用于初始化变量 ‘data-directory’。

‘EMACSDOC’ 文档字符串文件所在的目录，用于初始化 Lisp 变量 ‘doc-directory’。

‘EMACSLOADPATH’ 用于搜索 Emacs Lisp 文件的、以冒号分隔的目录列表(1)。如果设置，它会替换 ‘load-path’ 变量通常的初始值（参见“Emacs 的 Lisp 代码库”）。空元素代表 ‘load-path’ 的默认值；例如，使用 ‘EMACSLOADPATH="/tmp:"’ 会把 ‘/tmp’ 加到默认 ‘load-path’ 的前面。要在列表中间指定空元素，请连用 2 个冒号，如 ‘EMACSLOADPATH="/tmp::/foo"’。

‘EMACSPATH’ 用于搜索可执行文件的、以冒号分隔的目录列表。如果设置，Emacs 在初始化变量 ‘exec-path’ 时会把它与 ‘PATH’（见下文）一并使用（参见“在 Emacs 中运行 Shell 命令”）。

‘EMAIL’ 你的电子邮件地址；用于初始化 Lisp 变量 ‘user-mail-address’，Emacs 邮件接口会把它放进外发消息的 ‘From’ 邮件头（参见“邮件头字段”）。

‘ESHELL’ 供 shell-mode 使用，以覆盖 ‘SHELL’ 环境变量（参见“交互式子 shell”）。

‘HISTFILE’ 两次登录之间保存 shell 命令的文件名。如果你使用 Bash，此变量默认为 ‘~/.bash_history’；如果使用 ksh，默认为 ‘~/.sh_history’；否则默认为 ‘~/.history’。

‘HOME’ 你的文件在目录树中的位置；用于展开以波浪号（‘~’）开头的文件名。如果设置，它应被设为绝对文件名。（如果设为相对文件名，Emacs 会相对于启动 Emacs 时所在的目录来解释它，但我们不推荐使用这一特性。）如果未设置，‘HOME’ 通常默认为由 ‘LOGNAME’、‘USER’ 或你的用户 ID 所指定的用户的主目录，如果全都不行则默认为 ‘/’。在 MS-DOS 上，它默认为启动 Emacs 时所在的目录，如果末尾有 ‘/bin’ 就把它去掉。在 Windows 上，‘HOME’ 的默认值是用户配置文件目录下的 ‘Application Data’ 子目录（通常为 ‘C:/Documents and Settings/USERNAME/Application Data’，其中 USERNAME 是你的用户名），不过为了向后兼容，如果在那里找到 ‘.emacs’ 文件，就会改用 ‘C:/’。

‘HOSTNAME’ Emacs 所运行机器的名称。

‘INFOPATH’ 用于搜索 Info 文件的、以冒号分隔的目录列表。

‘LC_ALL’ ‘LC_COLLATE’ ‘LC_CTYPE’ ‘LC_MESSAGES’ ‘LC_MONETARY’ ‘LC_NUMERIC’ ‘LC_TIME’ ‘LANG’ 用户首选的语言环境（locale）。语言环境有六个类别，分别由环境变量 ‘LC_COLLATE’（排序）、‘LC_CTYPE’（字符编码）、‘LC_MESSAGES’（系统消息）、‘LC_MONETARY’（货币格式）、‘LC_NUMERIC’（数字）和 ‘LC_TIME’（日期和时间）指定。如果其中某个变量未设置，该类别就默认

为 'LANG' 环境变量的值；如果 'LANG' 也未设置，则默认为 'C' 语言环境。但如果指定了 'LC_ALL'，它会覆盖其他所有 locale 环境变量的设置。

在 MS-Windows 和 macOS 上，如果环境中尚未设置 'LANG'，Emacs 会根据系统范围的默认值来设置它。在某些版本的 MS-Windows 上，你可以在"区域设置"控制面板中设置它；在 macOS 上，则可以在"语言与地区"系统偏好设置中设置。

在 Android 上运行 GUI 会话时，'LANG' 被设为固定值，但语言和语言环境设置是从系统的"语言与输入"（Languages & Input）偏好设置中推导出来的。参见"在 Android 下运行 Emacs"。

'LC_CTYPE' 类别的值会与 'locale-language-names'、'locale-charset-language-names' 和 'locale-preferred-coding-systems' 中的条目进行匹配，以选择默认的语言环境和编码系统。参见"语言环境"。

'LOGNAME' 用户的登录名。另见 'USER'。

'MAIL' 你的系统收件箱的名称。

'MH' mh 系统的设置文件名。参见《《mh-e》手册》。

'NAME' 你在现实世界中的名字。用于初始化变量 'user-full-name'（参见"邮件头字段"）。

'NNTPSERVER' 新闻服务器的名称。供 mh 和 Gnus 包使用。

'ORGANIZATION' 你所属组织的名称。用于设置从 Gnus 包发出的帖子中的 'Organization:' 邮件头。

'PATH' 包含可执行文件的、以冒号分隔的目录列表。用于初始化变量 'exec-path'（参见"在 Emacs 中运行 Shell 命令"）。如果未设置或为空，则使用与具体实现相关的默认值。

'PWD' 如果设置，它应是启动 Emacs 时的默认目录。

'REPLYTO' 如果设置，它为变量 'mail-default-reply-to' 指定初始值（参见"邮件头字段"）。

'SAVEDIR' 默认保存新闻文章的目录名。供 Gnus 包使用。

'SHELL' 用于解析和执行从 Emacs 内部运行的程序的解释器名称。用于初始化变量 'shell-file-name'（参见"单条 Shell 命令"）。

'SMTPSERVER' 外发邮件服务器的名称。用于初始化变量 'smtpmail-smtp-server'（参见"发送邮件"）。

'TERM' Emacs 正在使用的终端类型。除非 Emacs 以批处理模式运行，否则必须设置此变量。在 MS-DOS 上，它默认为 'internal'，指定一种处理机器自身显示的内置终端仿真。

'TERMCAP' termcap 库文件的名称，该文件描述如何对 'TERM' 指定的终端进行编程。它默认为 '/etc/termcap'。

'TMPDIR' 'TMP' 'TEMP' 这些环境变量用于初始化变量 'temporary-file-directory'，该变量指定放置临时文件的目录（参见"备份文件"）。Emacs 首先尝试使用 'TMPDIR'。如果它未设置，Emacs 通常回退到 '/tmp'，但在 MS-Windows 和 MS-DOS 上则回退到 'TMP'，再回退到 'TEMP'，最后回退到 'c:/temp'。

'TZ' 此变量指定默认时区，可能还指定夏令时信息。参见《Emacs Lisp 参考手册》的"Time Zone Rules"。在 MS-DOS 上，如果 Emacs 启动时环境中未设置 'TZ'，Emacs 会根据 DOS 返回的国家代码定义一个合适的默认值。在 MS-Windows 上，Emacs 完全不使用 'TZ'。

‘USER’ 用户的登录名。另见 ‘LOGNAME’。在 MS-DOS 上，它默认为 ‘root’。

‘VERSION_CONTROL’ 用于初始化 ‘version-control’ 变量（参见“单个备份与编号备份”）。

> 注：此处及下文中，凡说“以冒号分隔的目录列表”时，指的都是 Unix 和 GNU/Linux 系统。在 MS-DOS 和 MS-Windows 上，目录之间改用分号分隔，因为 DOS/Windows 文件名在盘符之后可能含有冒号。

C.4.2 其他变量

这些变量仅用于特定的系统配置：

‘COMSPEC’ 在 MS-DOS 和 MS-Windows 上，指定调用批处理文件和 shell 内部命令时所使用的命令解释器名称。在 MS-DOS 上，它还被用来为 ‘SHELL’ 环境变量提供默认值。

‘NAME’ 在 MS-DOS 上，该变量默认取 ‘USER’ 变量的值。

‘EMACSTEST’ 在 MS-DOS 上，指定一个文件，用于记录内部终端模拟器的运行情况。这一功能在提交错误报告时很有用。

‘EMACSCOLORS’ 在 MS-DOS 上，指定屏幕颜色。以这种方式设置颜色很有用，因为否则 Emacs 启动时会短暂显示默认颜色。

该变量的值应为两个字符的编码，分别表示默认字体的前景色（第一个字符）和背景色（第二个字符）。每个字符应是在标准 PC 文本模式显示器上所需颜色的十六进制代码。例如，要得到浅灰色背景上的蓝色文字，可指定 ‘EMACSCOLORS=17’，因为 1 是蓝色的代码，7 是浅灰色的代码。

PC 显示器通常只支持八种背景色。不过，Emacs 会把 DOS 显示切换到一种模式，使全部 16 种颜色都能用作背景色，因此背景色的四个比特实际上都会用到。

‘PRELOAD_WINSOCK’ 在 MS-Windows 上，若设置该变量，Emacs 会在启动时加载并初始化网络库，而不是等到第一次需要时才做。

‘WAYLAND_DISPLAY’ Pgtk 版 Emacs（以 ‘--with-pgtk’ 构建）可以原生运行在 Wayland 上。‘WAYLAND_DISPLAY’ 指定与合成器的连接。

‘emacs_dir’ 在 MS-Windows 上，‘emacs_dir’ 是一个特殊的环境变量，表示 Emacs 所安装目录的完整路径。若 Emacs 安装在标准目录结构中，它会自动计算出该值。除非你的安装位置非标准，否则自己设置该变量没多大用处，因为它与其他环境变量不同，会在启动时被 Emacs 覆盖。在设置诸如 ‘EMACSLOADPATH’ 之类的其他环境变量时，你可能会发现使用 ‘emacs_dir’ 比把绝对路径硬编码进去更方便。这样可以让多个 Emacs 版本共用同一套环境变量设置，也可以在移动 Emacs 安装目录时不必改动任何环境或注册表设置。

C.4.3 MS-Windows 系统注册表

在 MS-Windows 上，环境变量 ‘emacs_dir’、‘EMACSLOADPATH’、‘EMACSDATA’、‘EMACSPATH’、‘EMACSDOC’、‘SHELL’、‘TERM’、‘HOME’、‘LANG’ 和 ‘PRELOAD_WINSOCK’ 也可以在系统注册表的 ‘HKEY_CURRENT_USER’ 或 ‘HKEY_LOCAL_MACHINE’ 分支中、‘/Software/GNU/Emacs’ 键之下设置。Emacs 启动时除检查环境变量外，还会到系统注册表中检查这些变量。

为确定这些变量的值，Emacs 依次执行以下步骤。首先检查环境变量。若在那里找不到该变量，Emacs 会在‘/Software/GNU/Emacs’之下查找以该变量为名的注册表键：先在注册表的‘HKEY_CURRENT_USER’分支中查找，若没有，再到‘HKEY_LOCAL_MACHINE’分支中查找。最后，若 Emacs 仍无法确定这些值，就使用编译时内置的默认值。

注意，注册表设置具有系统范围的全局影响：它们会影响在该系统上运行的所有 Emacs 会话。因此，如果你运行不同的 Emacs 版本，或同时使用已安装和未安装的 Emacs 可执行文件，或构建较新版本的 Emacs，注册表中的设置就会让它们全都使用相同的目录，这大概不是你想要的。出于这个原因，*我们不建议在注册表中设置这些变量*。若你在注册表中已有这类设置，我们建议将其删除。

若你运行 Emacs 的 MS-Windows 安装程序‘addpm.exe’，它会更新‘emacs_dir’、‘EMACSLOADPATH’、‘EMACSDATA’、‘EMACSPATH’、‘EMACSDOC’、‘SHELL’和‘TERM’变量已有的注册表设置，使其值适合与该‘addpm.exe’一同提供的已安装 Emacs 版本。注意，‘addpm.exe’*不会*创建任何原本不存在的注册表设置，它只更新已有的设置——这些设置多半继承自旧的 Emacs 安装——使其与新安装的 Emacs 版本兼容。安装较新版本的 Emacs 时已不再需要运行‘addpm.exe’，因此我们只建议在从旧版本升级、又因故无法从注册表中删除这些设置时才这样做。

除上述环境变量外，你还可以向‘/Software/GNU/Emacs’注册表键添加设置来指定 X 资源（参见“附录 D X 选项与资源”）。你在‘.Xdefaults’文件中能指定的大多数设置，都可以从该注册表键设置。

C.5 指定显示名称

环境变量‘DISPLAY’告诉包括 Emacs 在内的所有 X 客户端把窗口显示在何处。通常情况下，当你在本地启动 X 服务器并运行作业时，它的值会自动设定。你也可以自己指定显示；这样做的一个理由是，你想登录到另一台系统并在那里运行 Emacs，而让窗口显示在本地终端上。

‘DISPLAY’的语法为‘HOST:DISPLAY.SCREEN’，其中 HOST 是 X Window System 服务器主机的主机名，DISPLAY 是任意分配的编号，用于把你的服务器（X 终端）与同一机器上的其他服务器区分开，SCREEN 则是一个字段，使一个 X 服务器能够控制多个终端屏幕。句点和 SCREEN 字段都是可选的。若包含 SCREEN，它通常为零。

例如，若你的主机名为‘glasperle’，且你的服务器是配置中列出的第一个（或许也是唯一的）服务器，那么你的‘DISPLAY’就是‘glasperle:0.0’。

运行 Emacs 时，你可以显式指定显示名称，方法可以是修改‘DISPLAY’变量，也可以是使用选项‘-d DISPLAY’或‘--display=DISPLAY’。下面是一个例子：

```
emacs --display=glasperle:0 &
```

你可以用‘-nw’选项禁止使用 X 窗口系统。此时 Emacs 使用其控制终端进行显示。参见“初始选项”。

有时，安全设置会阻止远程系统上的程序在你的本地系统上显示。此时，尝试运行 Emacs 会产生类似下面的消息：

```
Xlib:  connection to "glasperle:0.0" refused by server
```

你也许能通过在本地系统上使用‘xhost’命令授予远程机器访问权限，来解决这个问题。

C.6 字体指定选项

你可以使用命令行选项‘-fn FONT’（或‘--font’，它是‘-fn’的别名）来指定默认字体：

‘-fn FONT’‘--font=FONT’将 FONT 用作默认字体。

在命令行上把字体名传给 Emacs 时，若其中含有 shell 会特殊处理的字符（例如空格），你可能需要给它加引号。例如：

```
emacs -fn "DejaVu Sans Mono-12"
```

关于字体名称的细节以及指定默认字体的其他方法，参见“字体”。

C.7 窗口颜色选项

你可以使用以下命令行选项指定 Emacs 显示各部分所用的颜色。颜色可以用颜色名称或 RGB 三元组来指定（参见“字体的颜色”）。

‘-fg COLOR’‘--foreground-color=COLOR’指定前景色，覆盖‘default’字体所指定的颜色（参见“文本字体”）。

‘-bg COLOR’‘--background-color=COLOR’指定背景色，覆盖‘default’字体所指定的颜色。

‘-bd COLOR’‘--border-color=COLOR’指定 X 窗口边框的颜色。若 Emacs 以 GTK+ 支持编译，该选项无效。

‘-cr COLOR’‘--cursor-color=COLOR’指定 Emacs 光标的颜色，该光标用于指示光标点的位置。

‘-ms COLOR’‘--mouse-color=COLOR’指定鼠标位于 Emacs 窗口中时鼠标光标的颜色。

‘-r’‘-rv’‘--reverse-video’反显：交换前景色和背景色。

‘--color=MODE’设置 Emacs 在文本终端上运行时的“颜色支持模式”。该选项会覆盖字符终端在其‘termcap’或‘terminfo’数据库中所声明的支持颜色数。参数 MODE 可以是下列之一：

‘never’‘no’即使终端的能力表明支持颜色，也不使用颜色。

‘default’‘auto’与完全不使用‘--color’时相同：Emacs 在启动时检测终端是否支持颜色，若支持，则开启彩色显示。

‘always’‘yes’‘ansi8’无条件开启颜色支持，并使用 ANSI 转义序列为 8 种标准颜色指定的颜色命令。

‘8bit’若可用，开启对 8 位（256 色）显示的支持。目前该选项仅在 MS-Windows（10 及更高版本）上有效。在其他系统上，会自动启用最大程度的颜色支持。

‘24bit’若可用，开启对 24 位（真彩色）显示的支持。目前该选项仅在 MS-Windows（10 及更高版本）上有效。在其他系统上，若支持 24 位颜色，会自动启用。

‘NUM’使用适用于 NUM 种颜色的颜色模式。若 NUM 为 −1，关闭颜色支持（等价于‘never’）；若为 0，使用该终端的默认颜色支持（等价于‘auto’）；否则使用适用于 NUM 种颜色的相应标准模式。取决于终端的能力，Emacs 或许能为 8、16、88、256 或 16777216 等 NUM 值开启相应的颜色模式。若没有任何模式支持 NUM 种颜色，Emacs 就视 NUM 为 0，即使用终端的默认颜色支持模式。

该选项在 MS-DOS 上无效，在 Windows 10 之前的 MS-Windows 上同样无效。

若省略 MODE，则默认为 ANSI8。

颜色模式可以在 Emacs 会话运行期间动态更改：当前模式可通过 'tty-color-mode' 框架参数获取，也可通过修改该框架参数来更改。(1) 这意味着你也可以改用 'default-frame-alist' 而不通过命令行选项来指定初始值。

例如，要使用珊瑚色的鼠标光标和石板蓝色的文本光标，请输入：

```
emacs -ms coral -cr 'slate blue' &
```

你可以通过 '-rv' 选项或 X 资源 'reverseVideo' 反转前景色和背景色。

'-fg'、'-bg' 和 '-rv' 选项在文本终端和图形显示上都能起作用。

> 注：这在 MS-Windows 和 MS-DOS 文本模式终端上不起作用。

C.8 窗口大小和位置选项

下面列出用于指定初始 Emacs 框架大小和位置的命令行选项：

'-g WIDTHxHEIGHT[{+-}XOFFSET{+-}YOFFSET]]' '--geometry=WIDTHxHEIGHT[{+-}XOFFSET{+-}YOFFSET]]' 指定尺寸 WIDTH 和 HEIGHT（以字符列数和行数计）以及位置 XOFFSET 和 YOFFSET（以像素计）。WIDTH 和 HEIGHT 参数适用于所有框架，而 XOFFSET 和 YOFFSET 只适用于初始框架。

'-fs' '--fullscreen' 指定宽度和高度取屏幕的宽度和高度。通常不显示窗口管理器装饰。（启动 Emacs 后，你可以用 <F11>、'toggle-frame-fullscreen' 切换该状态。）

'-mm' '--maximized' 指定 Emacs 框架应最大化。这通常意味着框架带有窗口管理器装饰。（启动 Emacs 后，你可以用 'M-F10'、'toggle-frame-maximized' 切换该状态。）

'-fh' '--fullheight' 指定高度取屏幕的高度。

'-fw' '--fullwidth' 指定宽度取屏幕的宽度。

在 '--geometry' 选项中，'{+-}' 表示正号或负号。XOFFSET 前的正号表示距屏幕左边的距离；负号表示从右边算起。YOFFSET 前的正号表示距屏幕顶部的距离，负号则表示距底部的距离。XOFFSET 和 YOFFSET 的取值本身可以为正或为负，但这不改变它们的含义，只改变方向。

Emacs 解释几何参数时使用与 'xterm' 相同的单位。WIDTH 和 HEIGHT 以字符计，因此大字体产生的框架比小字体大。（若指定比例字体，Emacs 以其最大边界宽度作为宽度单位。）XOFFSET 和 YOFFSET 以像素计。

你不必指定几何规格中的所有字段。若同时省略 XOFFSET 和 YOFFSET，则由窗口管理器决定把 Emacs 框架放在何处，可能让你用鼠标放置。例如，'164x55' 指定一个 164 列宽、55 行高的窗口；164 列宽足以并排放置两个常规宽度的窗口。

框架的默认宽度是 80 个字符，默认高度在 35 到 40 行之间，具体取决于操作系统和窗口管理器。你可以省略宽度或高度，或两者都省略。若几何规格以整数开头，Emacs 将其解释为宽度。若以 'x' 后跟一个整数开头，

Emacs 将其解释为高度。因此，‘81’只指定宽度；‘x45’只指定高度。

若几何规格以‘+’或‘-’开头，那就引入一个偏移量，这意味着两个尺寸都被省略。因此，‘-3’只指定 XOFFSET。（若只给出一个偏移量，它总是 XOFFSET。）‘+3-3’同时指定 XOFFSET 和 YOFFSET，把框架放在屏幕左下角附近。

你可以在 X 资源文件中为任一字段或全部字段指定默认值（参见“X 资源”），然后用‘--geometry’选项覆盖选定的字段。

由于模式行和回显区占据框架的最后 2 行，初始文本窗口的高度比几何规格中指定的高度少 2。在非 X toolkit 版本的 Emacs 中，菜单栏还会占用指定行数中的一行。但在 X toolkit 版本中，菜单栏是额外的，不计入指定的高度。工具栏若有，也是额外的。

启用或禁用菜单栏或工具栏会改变可用于普通文本的空间大小。因此，若 Emacs 启动时带有工具栏（这是默认情况），并在假定存在工具栏的前提下处理几何规格，而随后你的初始化文件禁用了工具栏，你最终得到的框架几何尺寸就会与你要求的不同。要在没有工具栏的情况下得到预期尺寸，可用 X 资源指定“不要工具栏”（参见“Emacs 的 X 资源表”）；这样 Emacs 处理指定几何规格时就已经知道没有工具栏。

使用‘--fullscreen’、‘--maximized’、‘--fullwidth’或‘--fullheight’之一时，有些窗口管理器要求你把变量‘frame-resize-pixelwise’设为非‘nil’值，框架才会真正显示为最大化或全屏。

有些窗口管理器带有选项，能使它们既忽略程序指定的位置，也忽略用户指定的位置。若设置了这些选项，Emacs 就无法正确放置窗口。

C.9 内部边框和外部边框

Emacs 框架有内部边框和外部边框。内部边框是框架文本区域周围一圈额外背景色的边条，由 Emacs 自己绘制。外部边框由 X 绘制，位于框架的工具栏和菜单栏之外。此外还有一个由窗口管理器绘制的“外侧边框”（external border）。外侧边框的尺寸无法从 Emacs 内部设置。

‘-ib WIDTH’‘--internal-border=WIDTH’指定内部边框（围绕框架的文本区域）的宽度为 WIDTH 像素。

‘-bw WIDTH’‘--border-width=WIDTH’指定外部边框的宽度为 WIDTH 像素。

指定框架尺寸时不包括边框。框架的位置从外侧边框的外缘量起。

用‘-ib N’选项指定 N 像素宽的内部边框，默认值为 1。用‘-bw N’指定外部边框的宽度（不过窗口管理器可能不理会你的指定）。外部边框的默认宽度是 2。

C.10 框架标题

每个 Emacs 框架总有一个标题，它在窗口装饰和图标中作为框架的名称显示。默认标题的形式为‘INVOCATION-NAME@MACHINE’（若只有一个框架），或显示所选窗口的缓冲区名（若有多个框架）。

你可以用命令行选项为初始 Emacs 框架指定非默认的标题：

‘-T TITLE’‘--title=TITLE’将 TITLE 指定为初始 Emacs 框架的标题。

‘--name’选项（参见“X 资源”）也指定初始 Emacs 框架的标题。

C.11 图标

‘-iconic’ ‘--iconic’ 以图标化状态启动 Emacs。

‘-nbi’ ‘--no-bitmap-icon’ 禁用 Emacs 图标。

大多数窗口管理器允许你把 Emacs 框架图标化（或“最小化”），使其从视野中隐藏。有些窗口管理器用很小的图标代替图标化的窗口，另一些则把它们完全从视野中移走。‘-iconic’ 选项让 Emacs 一开始就以图标化状态运行，而不是立即显示框架。在你取消图标化（或“取消最小化”）之前，文本框架不会出现。

默认情况下，Emacs 使用包含 Emacs 徽标的图标。在 Gnome 等桌面环境中，该图标还会在其他场合显示，例如切换到某个 Emacs 框架时。‘-nbi’ 或 ‘--no-bitmap-icon’ 选项让 Emacs 由窗口管理器选择使用何种图标——通常只是一个包含框架标题的小矩形。

C.12 其他显示选项

‘--parent-id ID’ 通过 XEmbed 协议把 Emacs 作为客户端 X 窗口打开，以 ID 作为父 X 窗口的 id。目前该选项主要对开发者有用。

‘-vb’ ‘--vertical-scroll-bars’ 启用垂直滚动条。

‘-lsp PIXELS’ ‘--line-spacing=PIXELS’ 指定行与行之间额外空出的空间为 PIXELS 像素。

‘-nbc’ ‘--no-blinking-cursor’ 在图形显示上禁用闪烁光标。

‘-D’ ‘--basic-display’ 禁用菜单栏、工具栏、滚动条和工具提示，并关闭 font-lock-mode 和闪烁光标。这对制作能简化显示问题调试的测试用例很有用。

‘--xrm’ 选项（参见“X 资源”）指定额外的 X 资源值。

附录 D X 选项与资源

与使用 X 的程序通常的做法一样，你可以用 X 资源来自定义 Emacs 行为中与 X 相关的某些方面。

当 Emacs 编译时带有 GTK+ 支持，菜单栏、滚动条和对话框等各种图形部件的外观由 GTK+ 资源决定，我们也会介绍这些资源。当 Emacs 构建时不带 GTK+ 支持，这些部件的外观由额外的 X 资源决定。

在 MS-Windows 上，你可以用系统注册表来自定义其中一些相同的方面（参见“MS-Windows 系统注册表”）。

D.1 X 资源

在 X 窗口系统下运行的程序把它们的用户选项按类和资源的层次组织起来。你可以在“X 资源文件”中为这些选项指定默认值，该文件通常名为 ‘~/.Xdefaults’ 或 ‘~/.Xresources’。对此文件的修改不会立即生效，因为 X 服务器保存着自己的资源列表；要更新它，请使用 ‘xrdb’ 命令——例如 ‘xrdb ~/.Xdefaults’。

通过 X 资源指定的设置通常覆盖 Emacs 初始化文件中的等效设置（参见“Emacs 初始化文件”），尤其是初始框架的参数（参见“框架参数”）。

（MS-Windows 系统不支持 X 资源文件；在这类系统上，Emacs 在 Windows 注册表中查找 X 资源，先查找 ‘HKEY_CURRENT_USER\SOFTWARE\GNU\Emacs’ 键，它只影响当前用户，并覆盖系统范围的设置；然后查找 ‘HKEY_LOCAL_MACHINE\SOFTWARE\GNU\Emacs’ 键，它影响系统的所有用户。菜单和滚动条在 MS-Windows 上是原生部件，因此只能通过“显示”控制面板中的系统范围设置来自定义。你也可以用下面将要说明的 ‘-xrm’ 命令行选项设置资源。）

X 资源文件中的每一行为一个选项或一组相关选项指定值。各行在文件中出现的顺序无关紧要。每条资源规格由一个“程序名”和一个“资源名”组成。这两个名称都区分大小写。下面是一个例子：

```
emacs.cursorColor: dark green
```

程序名是资源所适用的可执行文件的名称。对 Emacs 而言，这通常是 ‘emacs’。若要指定适用于所有 Emacs 实例（无论 Emacs 可执行文件叫什么名字）的定义，请使用 ‘Emacs’。

资源名是某项程序设置的名称。例如，Emacs 识别控制文本光标颜色的 ‘cursorColor’ 资源。

资源被分组到有名称的类中。例如，‘Foreground’ 类包含 ‘cursorColor’、‘foreground’ 和 ‘pointerColor’ 资源（参见“Emacs 的 X 资源表”）。你可以用类名代替资源名，为该类中所有资源指定默认值，如下所示：

```
emacs.Foreground: dark green
```

如果你把变量 ‘inhibit-x-resources’ 设为非 ‘nil’ 值，Emacs 就完全不处理 X 资源。如果你用 ‘-Q’（或 ‘--quick’）命令行选项启动 Emacs，‘inhibit-x-resources’ 会自动设为 ‘t’（参见“初始选项”）。

此外，你可以用以下命令行选项来覆盖 X 资源文件：

‘-name NAME’ ‘--name=NAME’ 此选项把初始 Emacs 框架的程序名设为 NAME。它还把初始框架的标题设为 NAME。此选项不影响后续的框架。

如果你不指定此选项，默认使用 Emacs 可执行文件的名称作为程序名。

为保持一致，‘-name’ 也指定用于其他不属于任何特定框架的资源值的名称。

命名 Emacs 实例的那些资源也属于一个名为 ‘Emacs’ 的类。如果你写 ‘Emacs’ 而不是 ‘emacs’，该资源就适用于所有 Emacs 作业中的所有框架，不论框架标题如何，也不论可执行文件的名称是什么。

‘-xrm RESOURCE-VALUE’ ‘--xrm=RESOURCE-VALUE’ 此选项为当前的 Emacs 作业指定 X 资源值。

RESOURCE-VALUE 应采用与在 X 资源文件内部使用相同的格式。可以给出多个 ‘-xrm’ 选项以包含多条资源规格。你也可以把 ‘#include "FILENAME"’ 用作 RESOURCE-VALUE，以包含一个满是资源规格的文件。用 ‘-xrm’ 指定的资源值优先于所有其他资源规格。

D.2 Emacs 的 X 资源表

下表列出 Emacs 识别的 X 资源名。注意，有些资源在用各种 X 工具包（GTK+、Lucid 等）编译的 Emacs 中没有效果——下面会指出属于这种情况的地方。

‘alpha’（类 ‘Alpha’）设置 ‘alpha’ 框架参数，决定框架的透明性（参见《Emacs Lisp 参考手册》的“Frame Parameters”）。

‘alphaBackground’（类 ‘AlphaBackground’）设置 ‘alpha-background’ 框架参数，决定背景透明性（参见《Emacs Lisp 参考手册》的“Frame Parameters”）。

‘background’（类 ‘Background’）背景颜色（参见“字体的颜色”）。

‘bitmapIcon’（类 ‘BitmapIcon’）若为 ‘on’，告诉窗口管理器显示 Emacs 图标；若为 ‘off’，则不显示。关于该图标的说明，参见“图标”。

‘borderColor’（类 ‘BorderColor’）框架外部边框的颜色。如果 Emacs 编译时带有 GTK+ 支持，此资源没有效果。

‘borderWidth’（类 ‘BorderWidth’）框架外部边框的宽度，单位为像素。如果 Emacs 编译时带有 GTK+ 支持，此资源没有效果。

‘cursorBlink’（类 ‘CursorBlink’）如果启动时此资源的值为 ‘off’ 或 ‘false’ 或 ‘0’，Emacs 会禁用闪烁光标模式（参见“光标的显示”）。

‘cursorColor’（类 ‘Foreground’）文本光标的颜色。如果在 Emacs 启动时指定了此资源，Emacs 会把它的值设为 ‘cursor’ 字体（face）的背景色（参见“文本字体”）。

‘font’（类 ‘Font’）‘default’ 字体的字体名（参见“字体”）。你也可以指定字体集名（参见“字体集”）。

‘fontBackend’（类 ‘FontBackend’）用于绘制字体的后端列表，以逗号分隔，按优先级排列。例如，值 ‘x,xft’ 告诉 Emacs 用 X 核心字体驱动绘制字体，若失败则回退到 Xft 字体驱动。通常你应当不设置此资源，此时 Emacs 会尝试使用所有可用的字体后端。

‘foreground’（类 ‘Foreground’）文本的默认前景色。

‘fullscreen’（类 ‘Fullscreen’）期望的全屏尺寸。值可以是 ‘fullboth’、‘maximized’、‘fullwidth’ 或 ‘fullheight’ 之一，它们分别对应命令行选项 ‘-fs’、‘-mm’、‘-fw’ 和 ‘-fh’（参见“窗口大小和位置选项”）。注意，这只适用于初始框架。

‘geometry’（类‘Geometry’）窗口大小和位置。值应是大小和位置的规格，形式与‘-g’或‘--geometry’命令行选项相同（参见“窗口大小和位置选项”）。

大小适用于 Emacs 会话中的所有框架，但位置只适用于初始 Emacs 框架（或者，若资源是为特定框架名指定的，则只适用于该框架）。

注意不要把此资源写成‘emacs*geometry’，因为那可能既影响主 Emacs 框架，也影响各个菜单。

‘horizontalScrollBars’（类‘ScrollBars’）如果此资源的值为‘off’或‘false’或‘0’，Emacs 在启动时禁用水平滚动条模式（参见“滚动条”）。

‘iconName’（类‘Title’）在图标中显示的名称。

‘internalBorder’（类‘BorderWidth’）框架内部边框的宽度，单位为像素。

‘lineSpacing’（类‘LineSpacing’）行间的额外间距，单位为像素。

‘menuBackground’（类‘Background’）非工具包版本 Emacs 中菜单的背景色。（对于工具包版本，参见“Lucid 菜单和对话框 X 资源”，另见参见“Motif 菜单 X 资源”，以及参见“GTK+ 资源”。）

‘menuBar’（类‘MenuBar’）如果此资源的值为‘off’或‘false’或‘0’，Emacs 在启动时禁用菜单栏模式（参见“菜单栏”）。

‘minibuffer’（类‘Minibuffer’）若为‘none’，Emacs 不会在此框架中创建迷你缓冲区；而是改用单独的迷你缓冲区框架。

‘paneFont’（类‘Font’）非工具包版本 Emacs 中菜单面板标题的字体名。（对于工具包版本，参见“Lucid 菜单和对话框 X 资源”，另见参见“Motif 菜单 X 资源”，以及参见“GTK+ 资源”。）

‘paneForeground’（类‘Foreground’）非工具包版本 Emacs 中菜单面板标题的前景色。（对于工具包版本，参见“Lucid 菜单和对话框 X 资源”，另见参见“Motif 菜单 X 资源”，以及参见“GTK+ 资源”。）

‘pointerColor’（类‘Foreground’）鼠标光标的颜色。在许多图形桌面环境中此资源没有效果，因为这些环境不允许 Emacs 以这种方式改变鼠标光标。

‘privateColormap’（类‘PrivateColormap’）若为‘on’，则在默认 visual 的类为 PseudoColor 且 Emacs 正使用它的情况下，使用私有颜色映射。

‘reverseVideo’（类‘ReverseVideo’）若为‘on’，交换前景色和背景色的默认值；若为‘off’，则按指定的颜色使用。

‘screenGamma’（类‘ScreenGamma’）颜色的 gamma 校正，等同于框架参数‘screen-gamma’。

‘scrollBar’（类‘ScrollBar’）如果此资源的值为‘off’或‘false’或‘0’，Emacs 在启动时禁用滚动条模式（参见“滚动条”）。

‘scrollBarWidth’（类‘ScrollBarWidth’）滚动条的宽度，单位为像素，等同于框架参数‘scroll-bar-width’。如果 Emacs 编译时带有 GTK+ 支持，请不要设置此资源。

‘selectionFont’（类‘SelectionFont’）非工具包版本 Emacs 中弹出菜单项的字体名。（对于工具包版本，参见“Lucid 菜单和对话框 X 资源”，另见参见“Motif 菜单 X 资源”，以及参见“GTK+ 资源”。）

‘selectionForeground’（类‘SelectionForeground’）非工具包版本 Emacs 中弹出菜单项的前景色。（对于工具包版本，参见“Lucid 菜单和对话框 X 资源”，另见参见“Motif 菜单 X 资源”，以及参见“GTK+ 资源”。）

‘selectionTimeout’（类‘SelectionTimeout’）等待选择应答的毫秒数。如果选择的所有者没有在此时间内应答，我们就放弃。值为 0 表示需要等多久就等多久。

‘synchronous’（类‘Synchronous’）若为‘on’，以同步模式运行 Emacs。同步模式对调试 X 问题很有用。

‘title’（类‘Title’）在初始 Emacs 框架的标题栏中显示的名称。

‘toolBar’（类‘ToolBar’）如果此资源的值为‘off’或‘false’或‘0’，Emacs 在启动时禁用工具栏模式（参见“工具栏”）。

‘tabBar’（类‘TabBar’）如果此资源的值为‘on’或‘yes’或‘1’，Emacs 在启动时启用标签栏模式（参见“标签栏”）。

‘useXIM’（类‘UseXIM’）若为‘false’或‘off’，禁用 X 输入法（XIM）。只有你的 Emacs 构建时带有 XIM 支持，此项才相关。在缓慢的 X 客户端/服务器链路上，关闭 XIM 可能有用。

‘inputStyle’（类‘InputStyle’）此资源控制 X 输入法生成的预览文本如何显示。其值可以是下列之一：

‘callback’在当前缓冲区中显示预览文本的内容。

‘offthespot’在 Emacs 提供的显示区域中一个单独的区域里显示预览文本。

‘overthespot’在当前窗口中光标点所在位置的弹出窗口里显示预览文本。

‘none’让输入法自行决定如何显示。这通常等同于‘overthespot’，但可能对更多输入法有效。

‘native’使用工具包处理输入法。目前只在 GTK 上实现。

‘root’使用显示设备上由输入法指定的某个位置来显示预览文本。

‘synchronizeResize’（类‘SynchronizeResize’）若为‘off’或‘false’，Emacs 在响应框架被调整大小而完成屏幕重绘时，不会试图通知窗口管理器。否则，窗口管理器会推迟绘制刚调整过大小的框架，直到其内容更新，从而避免显示框架中尚未绘制的空白区域。若设为‘extended’，将启用另一种框架同步协议，某些不支持 Emacs 默认所用协议的合成窗口管理器可能支持它；当使用兼容的合成窗口管理器时，这会让 Emacs 把显示与显示器刷新率同步。

‘verticalScrollBars’（类‘ScrollBars’）若为‘left’，框架的滚动条在左侧；若为‘right’，在右侧；若为‘off’，则没有滚动条（参见“滚动条”）。

‘visualClass’（类‘VisualClass’）X 彩色显示的“visual 类”。若指定，值应以‘TrueColor’、‘PseudoColor’、‘DirectColor’、‘StaticColor’、‘GrayScale’和‘StaticGray’之一开头，后接‘-DEPTH’，其中 DEPTH 是颜色平面的数目。

你也可以用 X 资源来自定义各个 Emacs 字体（face）（参见“文本字体”）。例如，设置资源‘FACE.attributeForeground’等同于自定义字体 FACE 的‘foreground’属性。不过，我们建议在 Emacs 内部自定义字体，而不要使用 X 资源。参见“自定义字体”。

D.3 Lucid 菜单和对话框 X 资源

如果 Emacs 编译时带有使用 Lucid 部件的 X 工具包支持，你可以用 X 资源来自定义菜单栏（参见“菜单栏”）、弹出菜单和对话框（参见“使用对话框”）的外观。菜单栏的资源属于 ‘pane.menubar’ 类（与往常一样，其前缀要么是 Emacs 可执行文件的名称，要么是适用于所有 Emacs 实例的 ‘Emacs’）。弹出菜单的资源属于 ‘menu*’ 类。对话框的资源属于 ‘dialog*’ 类。

例如，要用 ‘Courier-12’ 字体（参见“字体”）显示菜单栏条目，请这样写：

```
Emacs.pane.menubar.font: Courier-12
```

Lucid 部件可以显示你所处区域设置中的多语言文本。要启用此功能，请指定 ‘fontSet’ 资源而不是 ‘font’ 资源。参见“字体集”。如果同时指定了 ‘font’ 和 ‘fontSet’ 资源，则使用 ‘fontSet’ 资源。

下面是菜单栏、弹出菜单和对话框的资源列表：

- ‘font’ 菜单项文本的字体。
- ‘fontSet’ 菜单项文本的字体集。
- ‘background’ 背景色。
- ‘buttonForeground’ 选中项的前景色。
- ‘foreground’ 前景色。
- ‘disabledForeground’ 禁用的菜单项的前景色。
- ‘highlightForeground’ 被鼠标或键盘导航高亮的菜单项的前景色。
- ‘highlightBackground’ 被鼠标或键盘导航高亮的菜单项的背景色。
- ‘horizontalSpacing’ 项之间的水平间距，单位为像素。默认值为 3。
- ‘verticalSpacing’ 项之间的垂直间距，单位为像素。默认值为 2。
- ‘arrowSpacing’ 箭头（表示子菜单）与相关文本之间的水平间距。默认值为 10。
- ‘shadowThickness’ 3D 按钮、箭头和其他图形元素的阴影线粗细。默认值为 1。
- ‘borderThickness’ 菜单栏和弹出菜单外部边框的粗细。默认值为 1。
- ‘cursor’ 在菜单栏和弹出菜单中使用的光标名称。默认值为 ‘"right_ptr"’。
- ‘margin’ 菜单栏的边距，以字符为单位。默认值为 1。

D.4 Motif 菜单 X 资源

如果 Emacs 编译时带有使用 Motif 或 LessTif 部件的 X 工具包支持，你可以用 X 资源来自定义菜单栏（参见“菜单栏”）、弹出菜单和对话框（参见“使用对话框”）的外观。不过，这些资源的组织方式与 Lucid 部件不同。

菜单栏的资源名属于 ‘pane.menubar’ 类，而且必须按以下形式指定：

```
Emacs.pane.menubar.SUBWIDGET.RESOURCE: VALUE
```

对于弹出菜单，资源属于 'menu*' 类，而不是 'pane.menubar'。对于对话框，资源属于 'dialog'。在每种情况下，每个单独的菜单字符串都是一个子部件；子部件的名称与菜单项字符串相同。例如，菜单栏中的 'File' 菜单是一个名为 'emacs.pane.menubar.File' 的子部件。

通常，你会希望为整个菜单栏指定相同的资源。为此，用 '*' 代替具体的子部件名。例如，要为所有菜单栏项（包括子菜单）指定字体 '8x16'，请这样写：

```
Emacs.pane.menubar.*.fontList: 8x16
```

子菜单中的每一项也都有自己的 X 资源名称；例如，'File' 子菜单有一个名为 'Save (current buffer)' 的项。子菜单项的资源规格如下所示：

```
Emacs.pane.menubar.popup_*.MENU.ITEM.RESOURCE: VALUE
```

例如，下面说明如何为 'Save (current buffer)' 项指定字体：

```
Emacs.pane.menubar.popup_*.File.Save (current buffer).fontList: 8x16
```

对于第二级子菜单中的项，例如 'Tools' 下 'Spell Checking' 下的 'Complete Word'，资源符合以下模板：

```
Emacs.pane.menubar.popup_*.popup_*.MENU.RESOURCE: VALUE
```

例如，

```
Emacs.pane.menubar.popup_*.popup_*.Spell Checking.Complete Word: VALUE
```

（这应当写成一行长行。）

如果你希望子菜单项的外观与菜单栏本身不同，必须首先为所有项指定资源，然后单独覆盖子菜单的值。下面是一个例子：

```
Emacs.pane.menubar.*.fontList: 9x18
Emacs.pane.menubar.popup_*.fontList: 8x16
```

要为 LessTif 的文件选择框指定资源，请使用 'fsb*'，如下所示：

```
Emacs.fsb*.fontList: 8x16
```

下面是 LessTif 菜单栏和弹出菜单的资源列表：

- 'armColor' 处于激活状态的按钮中显示的颜色。
- 'fontList' 要使用的字体。
- 'marginBottom' 'marginHeight' 'marginLeft' 'marginRight' 'marginTop' 'marginWidth' 项周围、边框内留出的空间量。
- 'borderWidth' 菜单项四周边框的宽度。
- 'shadowThickness' 边框阴影的宽度。
- 'bottomShadowColor' 底部和右侧边框阴影的颜色。

- ‘topShadowColor’ 顶部和左侧边框阴影的颜色。

D.5 GTK+ 资源

如果 Emacs 编译时带有 GTK+ 工具包支持，自定义其 GTK+ 部件（例如菜单、对话框、工具栏和滚动条）最简单的方法是选择合适的 GTK+ 主题，例如使用 GNOME 主题选择器。

在 GTK+ 版本 2 中，你还可以用“GTK+ 资源”来自定义 Emacs 所用 GTK+ 部件的外观。这些资源在文件 ‘~/.emacs.d/gtkrc’（用于 Emacs 专用的 GTK+ 资源）或 ‘~/.gtkrc-2.0’（用于通用的 GTK+ 资源）中指定。我们建议使用 ‘~/.emacs.d/gtkrc’，因为在 GNOME 下运行 GConf 时 GTK+ 似乎会忽略 ‘~/.gtkrc-2.0’。不过要注意，某些 GTK+ 主题可能会覆盖 ‘~/.emacs.d/gtkrc’ 中的自定义设置；对此我们无能为力。GTK+ 资源不影响 Emacs 中与 GTK+ 部件无关的方面，例如主 Emacs 窗口中的字体和颜色；那些由普通的 X 资源控制（参见“X 资源”）。

以下各节说明如何为 Emacs 自定义 GTK+ 资源。有关 GTK+ 资源的细节，参见 GTK+ API 文档 <https://developer-old.gnome.org/gtk2/stable/gtk2-Resource-Files.html>。

在 GTK+ 版本 3 中，GTK+ 资源已被一套完全不同的系统取代。GTK+ 部件的外观现在由类似 CSS 的样式文件决定：GTK+ 安装目录中的 ‘gtk-3.0/gtk.css’，以及用于本地样式设置的 ‘~/.themes/THEME/gtk-3.0/gtk.css’（其中 THEME 是当前 GTK+ 主题的名称）。因此，本节对 GTK+ 资源的描述不适用于 GTK+ 3。有关 GTK+ 3 样式系统的细节，参见 <https://developer-old.gnome.org/gtk3/3.0/GtkCssProvider.html>。

D.5.1 GTK+ 资源基础

在 GTK+ 2 资源文件（通常是 ‘~/.emacs.d/gtkrc’）中，最简单的一种资源设置就是给变量赋值。例如，把下面这一行放入资源文件，会把所有 GTK+ 部件上的字体改为 ‘courier-12’：

```
gtk-font-name = "courier 12"
```

注意，在这种情况下，字体名必须按 GTK 字体模式（也称为“Pango 字体名”）给出，而不是 Fontconfig 风格的字体名或 XLFD。参见“字体”。

要自定义部件，你首先要定义一个“样式”，然后把该样式应用到部件上。下面是一个设置菜单字体的例子（‘#’ 字符表示注释）：

```
# Define the style ‘my_style’.
style "my_style"
{
  font_name = "helvetica bold 14"
}
```

```
# Specify that widget type ‘*emacs-menuitem*’ uses ‘my_style’.
widget "*emacs-menuitem*" style "my_style"
```

此例中的部件名包含通配符，因此该样式应用于所有匹配 ‘*emacs-menuitem*’ 的部件。部件的命名依据其包含关系，从外部部件到内部部件。下面是另一个例子，把 ‘my_style’ 专门应用于 Emacs 菜单栏：

```
widget "Emacs.pane.menubar.*" style "my_style"
```

下面是一个更复杂的例子，展示如何更改滚动条的各个部分：

```
style "scroll"
{
  fg[NORMAL] = "red"     # Arrow color.
  bg[NORMAL] = "yellow"  # Thumb and background around arrow.
  bg[ACTIVE] = "blue"    # Trough color.
  bg[PRELIGHT] = "white" # Thumb color when the mouse is over it.
}
```

```
widget "*verticalScrollBar*" style "scroll"
```

D.5.2 GTK+ 部件名称

GTK+ 部件由“部件名”和“部件类”来指定。部件名指向一个特定的部件（例如‘emacs-menuitem’），而部件类指向一组相似的部件（例如‘GtkMenuItem’）。部件总是有一个类，但不必有名称。

“绝对名称”（absolute name）是由控件名或控件类名构成的序列，对应嵌入在其他控件之内的控件层次结构。例如，若一个名为‘top’的‘GtkWindow’包含名为‘box’的‘GtkVBox’，后者又包含名为‘menubar’的‘GtkMenuBar’，则菜单栏控件的绝对类名为‘GtkWindow.GtkVBox.GtkMenuBar’，其绝对控件名为‘top.box.menubar’。

GTK+ 资源文件可以包含两类用于指定控件外观的命令：

- ‘widget’：依据类名或仅依据类为控件指定样式。
- ‘widget_class’：依据类名为控件指定样式。

使用‘widget’命令的示例见上一小节；‘widget_class’命令的用法与之类似。注意，控件名/类名与样式都必须用双引号括起，而且这些命令必须位于 GTK+ 资源文件的顶层。

如前所述，可以用 shell 通配符语法指定控件名或类名：‘*’匹配零个或多个字符，‘?’匹配一个字符。下例把某个样式赋予所有控件：

```
widget "*" style "my_style"
```

D.5.3 Emacs 中的 GTK+ 控件名

Emacs 框架所使用的 GTK+ 控件列举如下：

- ‘Emacs’（类‘GtkWindow’）
- ‘pane’（类‘GtkVBox’）
- ‘menubar’（类‘GtkMenuBar’）
- [菜单项控件]
- [未命名控件]（类‘GtkHandleBox’）
- ‘emacs-toolbar’（类‘GtkToolbar’）

- [工具栏项控件]
- ‘emacs’（类‘GtkFixed’）
- ‘verticalScrollBar’（类‘GtkVScrollbar’）

Emacs 窗口的内容绘制在‘emacs’控件中。注意，即使存在多个 Emacs 窗口，每个滚动条控件的名称都是‘verticalScrollBar’。

例如，下面是设置菜单栏样式的两种不同方式：

```
widget "Emacs.pane.menubar.*" style "my_style"
widget_class "GtkWindow.GtkVBox.GtkMenuBar.*" style "my_style"
```

对于 GTK+ 对话框，Emacs 使用名为‘emacs-dialog’、类为‘GtkDialog’的控件。对于文件选择，Emacs 使用名为‘emacs-filedialog’、类为‘GtkFileSelection’的控件。

由于弹出菜单和对话框的控件是独立窗口，并不包含在‘Emacs’控件之中，它们的 GTK+ 绝对名称不以‘Emacs’开头。要定制这些控件，可像下面这样使用通配符：

```
widget "*emacs-dialog*" style "my_dialog_style"
widget "*emacs-filedialog* style "my_file_style"
widget "*emacs-menuitem* style "my_menu_style"
```

若想把某个样式应用于 Emacs 中的所有菜单，可以这样写：

```
widget_class "*Menu*" style "my_menu_style"
```

D.5.4 GTK+ 样式

下面是两个 GTK+ 样式声明的示例：

```
pixmap_path "/usr/share/pixmaps:/usr/include/X11/pixmaps"
```

```
style "default"
{
  font_name = "helvetica 12"

bg[NORMAL] = { 0.83, 0.80, 0.73 }
bg[SELECTED] = { 0.0, 0.55, 0.55 }
bg[INSENSITIVE] = { 0.77, 0.77, 0.66 }
bg[ACTIVE] = { 0.0, 0.55, 0.55 }
bg[PRELIGHT] = { 0.0, 0.55, 0.55 }

fg[NORMAL] = "black"
fg[SELECTED] = { 0.9, 0.9, 0.9 }
fg[ACTIVE] = "black"
fg[PRELIGHT] = { 0.9, 0.9, 0.9 }

base[INSENSITIVE] = "#777766"
text[INSENSITIVE] = { 0.60, 0.65, 0.57 }

bg_pixmap[NORMAL] = "background.xpm"
```

```
bg_pixmap[INSENSITIVE] = "background.xpm"
bg_pixmap[ACTIVE] = "background.xpm"
bg_pixmap[PRELIGHT] = "<none>"

}

style "ruler" = "default"
{
  font_name = "helvetica 8"
}
```

‘ruler’ 样式继承自 ‘default’。这样就能在已有样式的基础上继续构建。字体和颜色的语法见下文。

如该示例所示，可以依据控件的“状态”（state）为前景色和背景色分别指定多个值。可能的状态有：

‘NORMAL’：控件的默认状态。

‘ACTIVE’：控件已准备好执行某种操作时所处的状态。它也用于滚动条的槽（trough），即 ‘bg[ACTIVE] = "red"’ 会把滚动条的槽设为红色。已处于待发状态（已按下但尚未释放）的按钮处于此状态。

‘PRELIGHT’：控件可被操作、且鼠标指针位于其上方时所处的状态——例如鼠标位于滚动条的滑块上或某个菜单项上时。当鼠标位于未按下的按钮上时，该按钮处于此状态。

‘SELECTED’：用户已选中的数据所处的状态。它可以是选中的文本，也可以是列表中选中的项。Emacs 不使用此状态。

‘INSENSITIVE’：控件可见但无法按常规方式操作时所处的状态——例如无法按下的按钮和被禁用的菜单项。若要以黄色显示被禁用的菜单项，可使用 ‘fg[INSENSITIVE] = "yellow"’。

样式声明中可以包含以下内容：

‘bg[STATE] = COLOR’：指定控件的背景色。注意，可编辑文本不使用 ‘bg’，而使用 ‘base’。

‘base[STATE] = COLOR’：指定可编辑文本的背景色。在 Emacs 中，该颜色用作文件对话框中文本字段的背景。

‘bg_pixmap[STATE] = "PIXMAP"’：指定图像背景（而非背景色）。PIXMAP 应为图像文件名。GTK+ 可以使用多种图像文件格式，包括 XPM、XBM、GIF、JPEG 和 PNG。若希望某个控件使用与其父控件相同的图像，可使用 ‘<parent>’。若不需要任何图像，可使用 ‘<none>’。‘<none>’ 是取消从父样式继承的背景图像的办法。

不能用绝对文件名指定该文件。GTK+ 会在 ‘pixmap_path’ 所指定的目录中查找像素图文件。‘pixmap_path’ 是用双引号括起的、以冒号分隔的目录列表，需在 ‘gtkrc’ 文件的顶层指定（即不要放在样式定义内部；参见上面的示例）：

```
pixmap_path "/usr/share/pixmaps:/usr/include/X11/pixmaps"
```

‘fg[STATE] = COLOR’：指定控件使用的前景色。它是菜单和按钮中文本的颜色，也是滚动条中箭头的颜色。对于可编辑文本，应使用 ‘text’。

‘text[STATE] = COLOR’：这是可编辑文本的颜色。在 Emacs 中，该颜色用于文件对话框中的文本字段。

‘font_name = "FONT"’：指定控件中文本的字体。FONT 是 GTK 风格（或 Pango）的字体名，如 ‘Sans Italic 10’。参见“字体”。这些名称不区分大小写。

指定颜色有三种方式：颜色名、RGB 三元组，或 GTK 风格的 RGB 三元组。颜色名与 RGB 三元组的说明参见“字体的颜色”。颜色名需用双引号括起，例如 ‘"red"’。RGB 三元组不带双引号，例如 ‘#ff0000’。GTK 风格的 RGB 三元组形式为 ‘{ R, G, B }’，其中 R、G、B 为 0-65535 范围内的整数，或 0.0-1.0 范围内的浮点数。

附录 E Emacs 30 反新闻

对于那些倒着时间生活的用户，这里提供关于降级到 Emacs 30.2 版的信息。我们希望您会喜欢由于缺少 Emacs 31.1 的许多特性而带来的更大简洁性。

• 古老的 'unexec' dumper 又回来了！它仍然被弃用，但我们计划在 Emacs 26 中取消对它的弃用，以便为 'glibc' 开发者恢复各种钩子做好准备，这些钩子是 Emacs 使用 'unexec' 可靠构建所必需的。

• 'ctags' 再次成为独立的程序。我们认为，必须记住 'etags' 的 '--ctags' 选项对用户来说不够友好，于是回到了久经考验的 Unix 哲学："程序只做一件事，并把它做好"。

• 我们去掉了那个恼人的特性：启动时扫描 Emacs 配置目录下 'user-lisp' 子目录中的文件和目录。它拖慢启动速度，尤其是当您回到过去、计算机和硬盘变得越来越慢的时候。

• 作为我们不断努力让 Emacs 更简单的一部分，我们移除了 'margin' 字体（face），因此显示边距中显示的文本现在将使用与缓冲区其余文本相同的 'default' 字体。我们的长期目标是彻底移除字体，使 Emacs 只剩下两种颜色，因此我们会逐步移除越来越多的字体。

• 我们摆脱的又一个复杂之处是纯文本终端上的子框架。GUI 框架上仍然支持它们，但改进要一步一步来。

• 花哨的补全装饰，例如预先显示补全候选、在您输入时更新 '*Completions*' 缓冲区、通过 'completions-format' 控制 '*Completions*' 中的光标点移动等等，将不再让您困惑，因为我们已经把这些东西全都删除了。除了输入 <TAB> 产生的结果之外，任何人都不应需要补全特性！

• 修改框架窗口布局的命令没有了。您再也不会被引诱去转置或旋转框架上的窗口。再一次，只有 Emacs 决定如何排列和重新排列窗口。

出于同样的原因，我们干掉了 'split-frame' 和 'merge-frames' 命令。'C-x 2' 和 'C-x 0' 对每个人都应该足够了！

• 我们移除了不少由 Unicode 标准 17.0 版及相关特性造成的复杂之处。这包括不少在过去永远用不到的字符、叙利亚语和提非纳语的语言环境，以及若干无人会怀念、而且不久之后反正也会被弃用的输入法。随着内存变得越来越宝贵，我们也必须让 Emacs 随之变得更精简、更犀利。

• 用户选项 'treesit-enabled-modes' 不再是 Emacs 的一部分。请自定义 'major-mode-remap-alist' 来启用基于 Tree-sitter 的模式以取代传统模式；这要简单得多，也更容易记住和管理。

• Sendmail 和 Rmail 不再尝试检测可疑的电子邮件地址并向您发出警告。当您回到过去，人们会变得越来越友善，而那些不断寻找机会给您造成损害的恶毒灵魂，则越来越成为来自遥远未来的模糊记忆。最终，我们将再次只拥有那个小小的 Arpanet，那里人人都是您的朋友，那时谁还需要这些偏执的特性呢?

• 您不能再编辑 Grep 的结果并让编辑反映到原始文件中。我们移除了这一功能，因为我们认为人们应当直接编辑原始文件，并充分享受周围上下文的便利。

• 从 VC 中删除了处理具有多个工作树的 VCS 仓库的命令。我们得出结论：单个项目中有多个工作树是设计臃肿的明显迹象，我们拒绝让这种臃肿软件进入 Emacs。所以它没了——早该如此！

许多其他 VC 特性，我们认为它们过于花哨且不够有用，也出于同样的原因被移除了。

• 要在从 ELPA 安装 Emacs Lisp 包之前审查它，您现在需要真正阅读它的代码和文档，因为用户选项 'package-review-policy' 以及相关的显示与前一版本差异的功能都已消失。"用源码吧，卢克！"

我们还删除了 'package-autosuggest' 和 'package-autosuggest-mode'，因为我们愿意相信用户更懂行。

• 为了迎接上一个 Emacs 版本中 'customize-dirlocals' 命令的移除，我们干掉了 'Custom-dirlocals-mode'。

• 我们移除了若干我们认为对 Emacs 的过去而言不必要的包和模式。这包括 'icalendar-mode' 和 'lua-mode' 模式，以及 'timeout' 和 'system-taskbar-mode' 库。出于同样的原因，'system-sleep' 包也没有了。

• 为了跟上计算机内存容量和磁盘空间不断减少的趋势，Emacs 30.2 中删除了许多其他函数和文件。

附录 F Emacs 与 macOS / GNUstep

本节介绍在 GNU/Linux 或其他操作系统上使用以 GNUstep 库构建的 Emacs、以及在 macOS 上使用支持原生窗口系统的 Emacs 时的特殊之处。在 macOS 上，Emacs 可以不带窗口系统支持构建，也可以带 X11 或 Cocoa 接口构建；本节只适用于 Cocoa 构建。它不支持 macOS 10.6 之前的版本。

GNUstep 是自由软件；macOS 不是。由于它是非自由的操作系统，macOS 剥夺了用户本应享有的自由——每位计算机用户都值得拥有这种自由。这是一种不公。为了你的自由，我们敦促你改用自由的操作系统。

我们在专有操作系统上支持 GNU Emacs，是希望这丝自由的滋味能激励用户逃离这些系统。

由于各种历史和技术原因，Emacs 内部使用 'Nextstep' 一词，而非 "Cocoa" 或 "macOS"；例如，本节所述的大多数命令和变量都以 'ns-' 开头，它是 'Nextstep' 的缩写。NeXTstep 是 NeXT Inc. 在 1980 年代发布的一种应用程序接口，Cocoa 是它的直系后代。除 Cocoa 之外，还有另一种 NeXTstep 风格的系统：GNUstep，它是自由软件。截至本文写作时，Emacs 的 GNUstep 支持仍处于 alpha 状态（参见"GNUstep 支持"），但我们希望将来能改进它。

F.1 macOS 与 GNUstep 下使用 Emacs 的基本方式

默认情况下，<Alt> 键和 <Option> 键等同于 <Meta>。Mac 的 <Cmd> 键等同于 <Super>，Emacs 提供了一组使用该修饰键的键绑定，以模仿其他 Mac / GNUstep 应用程序（参见"macOS / GNUstep 下的窗口系统事件"）。你可以用惯常的方式更改这些绑定（参见"定制键绑定"）。修饰键本身也可以自定义；参见"Mac / GNUstep 自定义"。

'S-mouse-1' 将区域调整到点击位置，就像 'mouse-3'（'mouse-save-then-kill'）一样；它不会像 'S-mouse-1' 通常那样弹出用于更改默认字体（face）的菜单（参见"文本缩放"）。这一改动使 Emacs 的行为更像其他 Mac / GNUstep 应用程序。

当你通过菜单打开或保存文件，或使用 'Cmd-o' 和 'Cmd-S' 绑定打开或保存文件时，Emacs 使用图形文件对话框读取文件名。然而，如果你使用常规的 Emacs 键序列（如 'C-x C-f'），Emacs 则使用迷你缓冲区读取文件名。

在 GNUstep 的 X 窗口环境中，要把文本传送到 X 的主选择区，需要使用 'Cmd-c'，而不是 'C-w' 或 'M-w' 命令中的某一个；否则，Emacs 将使用剪贴板选择区。同样，'Cmd-y'（而非 'C-y'）从 X 主选择区粘贴，而不是从删除环或剪贴板粘贴。

F.1.1 获取环境变量

许多可能在 Emacs 下运行的程序（如 latex 或 man）依赖于环境变量的设置。若 Emacs 从 shell 启动，它会自动继承这些环境变量，其子进程也会从它那里继承。但若 Emacs 从 Finder 启动，它就不是任何 shell 的后代，因此其环境变量尚未设置，这常常导致它启动的子进程的行为与从 shell 启动时不同。

对于 PATH 和 MANPATH 变量，在 macOS 上推荐使用系统级的方法设置 PATH，即使用 '/etc/paths' 文件和 '/etc/paths.d' 目录。

F.2 Mac / GNUstep 自定义

有几个定制选项是 Nextstep 移植版特有的。例如，它们会影响修饰键和全屏行为等。要查看所有这类选项，请使用 ‘M-x customize-group <RET> ns <RET>’。

F.2.1 修饰键

以下变量控制实际修饰键的行为：

- ‘ns-alternate-modifier’、‘ns-right-alternate-modifier’：左右 <Option> 或 <Alt> 键。
- ‘ns-command-modifier’、‘ns-right-command-modifier’：左右 <Command> 键。
- ‘ns-control-modifier’、‘ns-right-control-modifier’：左右 <Control> 键。
- ‘ns-function-modifier’：<Function>（fn）键。

每个变量的值要么是一个符号，描述该键在一切用途下的含义；要么是形如 ‘(:ordinary SYMBOL :function SYMBOL :mouse SYMBOL)’ 的列表，分别描述该修饰键与普通键、功能键（不产生字符的键，如方向键）以及鼠标点击一起使用时的含义。

若 SYMBOL 是 ‘control’、‘meta’、‘alt’、‘super’ 或 ‘hyper’ 之一，则它描述该键所代表的 Emacs 修饰键。若 SYMBOL 为 ‘none’，Emacs 不使用该键，该键保留其标准行为。例如，macOS 中的 <Option> 键此时用于组合附加字符。

右手键对应的变量（如 ‘ns-right-alternate-modifier’）也可以设为 ‘left’，表示采用与相应左手键相同的行为。

F.2.2 框架变量

- ‘ns-use-proxy-icon’：该变量指定是否在标题栏中显示代理图标。代理图标可用于将当前缓冲区关联的文件拖到其他应用程序、打印机、桌面等，方式与在 Finder 中相同。你可能需要禁用 ‘tool-bar-mode’ 才能看到代理图标。
- ‘ns-confirm-quit’：该变量指定退出时是否显示图形确认对话框。
- ‘ns-auto-hide-menu-bar’：该变量指定当选中某个 Emacs 框架时是否隐藏 macOS 菜单栏。若值为非 ‘nil’，则除非鼠标指针移到屏幕顶部附近，否则不显示菜单栏。
- ‘ns-use-native-fullscreen’：该变量控制使用原生还是非原生全屏。原生全屏仅在 macOS 10.7 及以上版本可用。

F.2.3 macOS 触控板/鼠标滚轮变量

这些变量仅适用于 macOS 10.7（Lion）及以上版本。

- ‘ns-use-mwheel-acceleration’：该变量控制 Emacs 是否忽略系统的鼠标滚轮加速。当值为 ‘nil’ 时，鼠标滚轮的每次“咔哒”恰好对应一个鼠标滚轮事件。当值为非 ‘nil’（默认值）时，每次“咔哒”可能对应多个鼠标滚轮事件，具体取决于用户的输入。
- ‘ns-use-mwheel-momentum’：该变量控制 Emacs 在使用触控板滚动时是否忽略系统的“惯性”。当值为非 ‘nil’（默认值）时，快速滚动可能导致用户手指离开触控板后缓冲区仍继续滚动一小会儿。
- ‘ns-mwheel-line-height’：该变量控制使用触控板滚动的灵敏度。Apple 触控板按像素而非按行滚动，因此 Emacs 会将系统的像素值转换为行数。当设为一个数字时，该变量设定 Emacs 将多少像素视为一行。当值

为‘nil’或非数字时，使用默认的行高。

设置较小的数字会使触控板更灵敏，较大的数字则使触控板较不灵敏。

F.3 macOS / GNUstep 下的窗口系统事件

Nextstep 应用程序会收到若干在 X 中没有对应物的特殊事件。这些事件以特别定义的按键事件形式发送，它们不对应任何击键序列。在 Emacs 中，这些按键事件可以像普通击键一样绑定到函数。下面是这些事件的列表。

- `<ns-open-file>`：当另一个 Nextstep 应用程序请求 Emacs 打开文件时发生此事件。典型情形是用户在 Finder 应用程序中双击某个文件。默认情况下，Emacs 通过打开一个新框架并在该框架中访问文件来响应此事件（‘ns-find-file’）。作为例外，若选中的缓冲区是‘*scratch*’缓冲区，Emacs 则在选中的框架中访问该文件。

你可以通过更改变量‘ns-pop-up-frames’来改变 Emacs 对‘ns-open-file’事件的响应方式。其默认值‘fresh’就是我们刚才描述的行为。值为‘t’表示总是在新框架中访问文件。值为‘nil’表示总是在选中的框架中访问文件。

- `<ns-open-temp-file>`：当另一个应用程序请求 Emacs 打开临时文件时发生此事件。默认情况下，处理方式只是生成一个‘ns-open-file’事件，其结果如上所述。
- `<ns-open-file-line>`：有些应用程序（如 ProjectBuilder 和 gdb）不仅请求特定文件，还请求文件中的特定行或若干行。Emacs 通过访问该文件并高亮所请求的行来处理此事件（‘ns-open-file-select-line’）。
- `<ns-power-off>`：当用户注销而 Emacs 仍在运行，或从应用程序菜单中选择“退出 Emacs”时发生此事件。默认行为是保存所有访问文件的缓冲区。
- `<ns-show-prefs>`：当用户从应用程序菜单中选择“偏好设置”时发生此事件。默认情况下，它绑定到命令‘customize’。

Emacs 还允许用户使用 Nextstep 服务，方法是一组名称以‘ns-service-’开头、以服务名结尾的命令。输入‘M-x ns-service- <TAB>’可查看这些命令的列表。这些函数要么作用于标记的文本（用结果替换它），要么接受一个字符串参数并以字符串形式返回结果。你也可以使用 Lisp 函数‘ns-perform-service’把任意字符串传给任意服务并取回结果。注意，你可能需要重启 Emacs 才能访问新近可用的服务。

F.4 GNUstep 支持

Emacs 可以在 GNUstep 下构建并运行，但仍有问题有待解决。有兴趣的开发者请联系 <emacs-devel@gnu.org>。

附录 G Emacs 与 Haiku

Haiku 是一种类 Unix 操作系统，最初是作为 BeOS 操作系统的重新实现而出现的。

本附录介绍使用以 Application Kit（Haiku 原生的窗口系统）构建的 Emacs 时的特殊之处。此处说明的这些特性不适用于在 Haiku 上以不支持窗口的方式构建、或配置为使用 X11 的 Emacs。

G.1 Haiku 安装与启动

在 Haiku 上安装 Emacs 时，会有两个可执行文件被复制到可执行文件目录中，二者除了一些用于识别的文件系统元数据之外完全相同。第一个是普通的 Emacs 可执行文件 'emacs'，第二个则是 'Emacs'，它带有一个图标和一个应用程序"签名"（signature），后者帮助系统将文件类型和打开的框架都归属到它，从而使它能接受文件类型关联，并因此可以直接从 Tracker 打开文件。

'Emacs' 中设置了若干文件属性，促使系统在任何时刻只允许运行一个副本。这一不变式在建立显示连接时得到校验，其强制执行方式是：当显示连接已经存在时，终止任何仍试图创建显示连接的 Emacs 进程。

出于这一原因及其他原因，'Emacs' 适合用于启动 Emacs 的 GUI 会话，而 'emacs' 则应用于其他类型的 Emacs 会话。

在 Haiku 下，Emacs 无法接收 'Hyper' 之类特殊的修饰键，也无法接收由系统 Super 键映射产生的带重音字符。

默认情况下，按下操作系统定义的 Option 键时报告的是 <Super> 修饰键。类似地，<Meta> 修饰键被分配给 Command 键，而 <Control> 自然对应系统的 Control 键，<Shift> 对应系统的 Shift 键。在标准 PC 键盘上，Haiku 应会把这些键映射到 GNU 系统用户所熟悉的位置，但这可能需要对你的系统配置作一些调整才能生效。

你可以通过下列变量自定义系统已知的修饰键与 Emacs 已知的修饰键之间的关系。

- 'haiku-meta-keysym'：将被 Emacs 视为 Meta 键的系统修饰键。默认为 'command'。
- 'haiku-control-keysym'：将被 Emacs 视为 Control 键的系统修饰键。默认为 'control'。
- 'haiku-super-keysym'：将被 Emacs 视为 Super 键的系统修饰键。默认为 'option'。
- 'haiku-shift-keysym'：将被 Emacs 视为 Shift 键的系统修饰键。默认为 'shift'。

每个变量的取值可以是符号 'command'、'control'、'option'、'shift' 或 'nil' 之一。'nil' 或任何其他取值都会导致改用默认值。

在 Haiku 上，Emacs 默认使用系统工具提示机制。这样生成的工具提示有时响应更快，但无法显示文本属性或字体（face）。若需要这些特性，请将变量 'use-system-tooltips' 定制为 'nil'，此时 Emacs 将改用自己实现的工具提示。

与 X 窗口系统不同，Haiku 不提供系统级的资源数据库。由于许多重要选项都是通过 X 资源指定的（参见"附录 D X 选项与资源"），因此提供了一种模拟方式：启动时，Emacs 会加载用户配置目录（通常为 '/boot/home/config/settings'）中名为 'GNU Emacs' 的文件，该文件应当是一个扁平化的系统消息，其中的键和值都是字符串，分别对应属性及其值。这样的文件可以用 'xmlbmessage' 工具创建。

如果变量 'haiku-debug-on-fatal-error' 的值非 'nil'，则 Emacs 在收到致命信号时会启动系统调试器。其默认值为 't'。如果无法在你的系统上使用 GDB，请在报告 bug 时附上系统调试器生成的报告。

G.2 Haiku 下的字体后端与选区

在以 Haiku 窗口支持构建时，Emacs 支持若干不同的字体后端，但所支持的子集取决于配置 Emacs 时存在并启用的依赖项列表。你可以在调用 Emacs 的命令行上提供 '-xrm Emacs.fontBackend:BACKEND' 来指定要使用哪些字体后端，其中 BACKEND 是下面列出的后端之一；也可以通过修改 'font-backend' 框架参数来按框架指定。

其中两个后端 'ftcr' 和 'ftcrhb' 与 X 窗口系统上的对应后端完全相同。此外还有一个名为 'haiku' 的 Haiku 专属后端，它使用 App Server 绘制字体，但目前无法显示彩色字体或 Emoji。

附录 H Emacs 与 Android

Android 是由开放手机联盟（Open Handset Alliance）开发的移动操作系统。本节介绍在运行 Android 2.2 或更高版本的 Android 设备上使用 Emacs 的特殊之处。

Android 设备通常依靠触摸屏或数字化输入设备以及虚拟键盘进行用户输入。关于在此类设备上使用 Emacs 的更多信息，参见“触摸屏输入与虚拟键盘”。

H.1 Android 历史

Android 是由开放手机联盟开发的移动设备操作系统，该联盟由一批有志于开发能运行同一套软件的手机的公司组成。据说它是自由软件。

与昔日的 X 联盟一样，开放手机联盟认为“开放性”（即定期发布 Android 源代码）不过是提升 Android 平台人气的一种工具。计算机公司通常生产专有软件。开放手机联盟中的公司也不例外——安装在设备上的大多数 Android 版本都是专有的，因为它们包含专有组件，而这些组件往往连用户自己都无法替换。

Android 的设计并不尊重用户的自由。几乎所有 Android 版本（包括一些号称是自由软件的版本）都包含对数字限制管理（Digital Restrictions Management）的支持，这种技术旨在限制用户在自己设备之间复制媒体的能力。大多数 Android 设备还附带了运行系统所必需的专有 Google 应用程序，以及许多其他 Android 应用程序。

因此，从实用角度出发，必须把 Android 视为专有软件。这是一种不公正。如果你使用 Android，我们敦促你改用自由操作系统，哪怕只是为了你自己的自由。

我们在专有操作系统上支持 GNU Emacs，因为我们希望这自由的滋味能激励用户逃离这些系统。

H.2 在 Android 上启动 Emacs

Emacs 不会在 Android 设备上从源代码编译安装，也不会通过包管理器安装。相反，Emacs 是在另一个操作系统上为 Android 编译的，编译得到的二进制文件被打包成一个归档文件，然后传输到系统上并安装。

安装之后，系统会在桌面（又称“主屏幕”）上放置一个应用图标。点击该应用图标后，Emacs 就会启动。

启动过程中，Emacs 会在系统日志缓冲区中显示消息；要在启动期间读取该缓冲区，需要在另一台计算机上安装 Android 调试桥（‘adb’）工具。

在 Android 系统上启用“USB 调试”功能，并通过 USB 将其连接到另一台安装了 ‘adb’ 工具的计算机之后，在那台计算机上运行以下命令即可查看日志：

```
$ adb logcat | grep -E "(android_run_debug_thread|[Ee]macs)"
```

假设 ‘adb’ 工具安装在 GNU/Linux 或 Unix 系统上，请按以下步骤连接你的设备。

1. 在设备上启用“开发者选项”：进入系统设置应用中的“关于”页面，连续点击“版本号”或“内核版本”项五到七次。

1. 打开“开发者选项”设置页面，它应该位于设置应用的“系统”页面下。

1. 打开“USB 调试”开关。

1. 将 USB 线的一端连接到设备，另一端连接到计算机的 USB 端口。

1. 在计算机上运行命令 ‘adb shell’。由于你尚未授予计算机访问所连接设备的权限，该命令会失败或挂起。

1. 在设备上确认弹出的对话框，它询问是否允许来自计算机的访问。

视所安装的 Android 与 ‘adb’ 版本而定，建立连接可能还有其他方式。更多细节参见 <https://developer.android.com/studio/command-line/adb> 上的官方文档。

Emacs 启动之后，只要以异步 shell 命令运行 ‘logcat’（参见“在 Emacs 中运行 Shell 命令”），就会显示日志缓冲区。

由于没有其他办法从另一个 Android 程序启动 ‘emacsclient’ 程序（参见“把 Emacs 用作服务器”），Emacs 提供了一个围绕 ‘emacsclient’ 程序的包装程序，该包装程序在系统中注册为可以打开任何文件的应用程序。

当该包装程序被选为打开文件的程序时，它会带上选项 ‘--reuse-frame’、‘--timeout=10’、‘--no-wait’ 以及所打开文件的名称来调用 ‘emacsclient’。随后，若调用成功，焦点会转移到任何已打开的 Emacs 框架。

不过，如果在打开包装程序时 Emacs 并未运行，它会启动 Emacs，并把要打开的文件作为参数交给它。请注意，如果这样启动的 Emacs 又未能启动 Emacs 服务器，那么后续再用包装程序打开文件就会失败。

有些文件是以“内容标识符”（content identifier）的形式交给 Emacs 的，系统独立于常规文件系统 API 提供对这些内容的访问。Emacs 使用名为 ‘/content/by-authority’ 和 ‘/content/by-authority-named’ 的伪目录来访问这些文件。不要对这些目录的内容做任何假设，也不要自己尝试打开其中的文件。

Android 4.3 及更早版本不提供此功能，在那些系统上，这类文件会先被复制到临时目录再打开。

除了打开普通文本文件，Emacs 还将其 ‘emacsclient’ 包装程序注册为能够打开 “org-protocol” 链接的程序（参见《Org 手册》的“Protocols”）。

此外，该包装程序还被注册为能够向 ‘mailto’ URI 发送邮件的程序；当它被调用来打开此类 URL 时，它会以该 URI 为第一个参数调用函数 ‘message-mailto’。此功能在 Emacs 服务器尚未运行时不起作用。

H.3 Emacs 在 Android 上能访问哪些文件

Emacs 在 Android 系统上暴露了一个特殊的目录：该目录名为 ‘/assets’，其中包含通常安装在 GNU 和 Unix 系统 ‘/usr/share/emacs’ 目录下的 ‘etc’、‘lisp’ 和 ‘info’ 目录。在 Android 系统上，‘ls’ 的 Lisp 仿真（参见“在 MS-Windows 上模拟 ‘ls’”）也默认启用，因为系统自带的 ‘ls’ 二进制文件因厂商而异，通常不支持 Emacs 所需的所有特性。某些 Android 系统分发的一份 ‘ls’ 甚至已知不支持 ‘-l’ 选项。

这个目录之所以存在，是因为 Android 在解包应用包时并不把其内容解压到文件系统中，而是要求 Emacs 之类的程序使用一种特殊的“资源管理器”（asset manager）接口访问其内容。以下是这种实现方式带来的特殊之处：

- 子进程（如 ‘ls’）无法从 ‘/assets’ 目录运行；如果你在 ‘current-directory’ 设为 ‘/assets’、‘/content/storage’ 或其子目录的情况下运行子进程，它反而会从主目录运行。

- ‘/assets’ 或 ‘/content’ 目录内没有 ‘.’ 和 ‘..’ 目录。

- ‘/assets’ 目录中的文件始终是只读的，而且每打开一次都可能被多次读入内存。

除 ‘/assets’ 目录之外，Android 程序通常还可以访问另外四个目录。它们是：

- “应用数据”（app data）目录。它同时充当 Emacs 的主目录，并且始终可读写访问。
- “应用库”（app library）目录。启动时它会被自动追加到 ‘exec-path’，并被设为 ‘exec-directory’，其中包含与 Emacs 本身放在一起的实用工具可执行文件。
- “外部存储”（external storage）目录。当用户通过系统设置授予 Emacs “文件和媒体”（Files and Media）权限后，Emacs 即可访问它。
- Android 5.0 及更高版本中由“文档提供者”（document provider）提供的目录。这些目录独立于 Unix 虚拟文件系统而存在，其中包含外部程序提供的文件（参见“在 Android 上访问其他程序的文件”。）

尽管普通安装的 Android 的（通常是只读的）根目录中并没有名为 ‘content’ 或 ‘assets’ 的文件，但如果所使用的 Android 安装经过定制，你可能想访问这些名字的真实文件。这些文件会与前述特殊目录冲突，但仍可以通过相对于根目录的“父”目录写出它们的名字来访问，如下所示：‘/../content’、‘/../assets’。

外部存储目录位于 ‘/sdcard’。其他目录不在任何固定位置（但见下文），尽管应用数据目录通常符号链接到 ‘/data/data/org.gnu.emacs/files’。

旧版本的 Android 习惯把应用库目录放在应用数据目录的父目录下，名为 ‘lib’。如今，该目录常被放在 ‘/data/app’ 下某个随机生成的名字的目录中。

为了方便那些与 Emacs 共用同一用户 ID 的应用程序中运行的脚本（它们无法访问 ‘exec-directory’ 变量），启动时会付出相当大的努力，把应用库目录符号链接到它在应用数据目录父目录中的传统位置。

如果重新安装了 Emacs，应用库目录的位置随之改变，那么下次系统启动 Emacs 时，该符号链接也会更新为指向新位置。

在运行非常古老（2.6.29）Linux 内核版本的 Android 设备上，Emacs 需要在临时文件目录中创建名字以 ‘temp~unlinked’ 开头的文件，才能读取资源文件。请不要自己创建这种名字的文件，否则它们可能被覆盖或删除。

在 Android 11 及更高版本上，Android 系统限制应用使用 ‘open’、‘readdir’ 等与文件相关的系统调用来访问 ‘/sdcard’ 目录中的文件。

这项限制被称为“分区存储”（Scoped Storage），据说能让系统更安全。不幸的是，这也意味着即使 Emacs 拥有必要的权限，也无法访问那些目录中的文件。所幸开放手机联盟版本的 Android 允许按程序禁用此限制；系统设置面板中对应的选项是：

```
System -> Apps -> Special App Access -> All files access -> Emacs
```

在你按需禁用或启用该设置并授予 Emacs “文件和媒体”权限之后，它就能像往常一样访问 ‘/sdcard’ 下的文件。某些专有版本的 Android 上没有这些设置。

H.4 在 Android 上访问其他程序的文件

Android 5.0 引入了一种新的程序类型，即“文档提供者”：这类程序是一些小型服务，它们独立于资源管理器和 Unix 文件系统提供对自身文件的访问。Emacs 支持访问它们提供的文件和目录，把这些文件放在目录 ‘/content/storage’ 中。

在 Emacs 被授予访问某个此类目录的权限之前，它必须先请求访问权。这通过运行命令 ‘android-request-directory-access’（参见“按名称执行命令”）来完成，该命令会显示一个文件选择对话框。

如果从该对话框中选择了一个目录，其内容随后会在名为 ‘/content/storage/AUTHORITY/ID’ 的新目录中可用，其中 AUTHORITY 是文档提供者的名称，ID 是文档提供者分配给该目录的唯一标识符。

当不再需要这样的目录时，可以把它的名字提供给命令 ‘android-relinquish-directory-access’ 来删除它。

在那些目录内创建子进程时，会遇到与 ‘/assets’ 目录相同的限制（参见“Emacs 在 Android 上能访问哪些文件”），因为这些目录并不存在于 Unix 文件系统之中。此外，尽管 Emacs 通常可以在这些目录中写入和创建文件，但它无法创建符号链接或硬链接。

由于文档提供者可以执行昂贵的网络操作来获取文件内容，因此在其中一个目录内进行文件访问操作可能需要相当长的时间。

H.5 在 Android 下运行 Emacs

从用户的角度看，Android 基本上是一个单用户操作系统；然而从应用程序和 Emacs 的角度看，该系统承载着数量惊人的用户。

每个应用程序都在自己的用户下运行，其主目录被设为它的应用数据目录（参见“Emacs 在 Android 上能访问哪些文件”。）(1)

每个应用程序还被禁止访问许多系统目录以及其他应用程序的应用数据目录。

Emacs 发行版还包含若干二进制文件。虽然它们是可执行文件，却被作为库打包在库目录中，否则系统在安装 Emacs 时不会解包它们。这意味着在子进程中启动这些程序之一时，Lisp 代码必须在命令行上指定 ‘libemacsclient.so’ 而不是 ‘emacsclient’；要确定应使用哪些名字，可查阅变量 ‘etags-program-name’、‘hexl-program-name’、‘emacsclient-program-name’、‘movemail-program-name’、‘ebrowse-program-name’ 和 ‘rcs2log-program-name’ 的值。参见《Emacs Lisp 参考手册》的“Subprocess Creation”。

包含 Emacs 启动文件的 ‘/assets’ 目录本意是对并非由 ‘zygote’（负责启动应用程序的系统服务）直接创建的进程不可访问。由于所需的 Lisp 位于 ‘/assets’ 目录中，因此可以推断 Emacs 无法把自身作为子进程启动。Emacs 附带了一个名为 ‘libandroid-emacs.so’ 的特殊二进制文件，它被安装到库目录中，会尽力启动 Emacs 以便以批处理模式运行 Lisp。它采用的方法是在参考 Android 源代码的基础上设计的，并未得到 Android 兼容性定义文档的认可，因此效果可能因设备而异。

即使事先知道 ‘libandroid-emacs.so’ 命令的位置，要从某个现有 Emacs 会话的子进程之外启动 Emacs 也需要特殊准备，因为必须让它了解资源以及已安装应用包中或从中解出的共享库的位置。操作系统命令 ‘pm path org.gnu.emacs’ 会打印应用包的位置，但该命令必须以特殊方式调用，才能满足系统对用户应用程序创建的伪终端设备与包管理器等系统服务之间通信的限制，也就是说，要把标准 IO 流重定向到真实文件或管道。这个

值一旦确定，就必须在环境变量 'EMACS_CLASS_PATH' 中指定，这样下面的示例 shell 脚本就可以作为 'emacs' 安装到任何可访问的位置：

```
#!/system/bin/sh
```

```
package_name=`pm path org.gnu.emacs 2>/dev/null </dev/null \
               | sed 's/^package://'`
emacs=
EMACS_CLASS_PATH=$package_name
```

```
for libdir in `dirname $package_name`/lib/*; do
  ld_path=${ld_path:+${ld_path}:}$libdir
  test -x "$libdir"/libandroid-emacs.so \
    && emacs="$libdir"/libandroid-emacs.so
done
```

```
export EMACS_CLASS_PATH
test -x "$emacs" || exit 1
exec $emacs "$@"
```

Android 10 及更高版本还禁止 Emacs 本身在应用数据目录内运行可执行文件，表面上是出于安全考虑。在这些系统上，Emacs 通常会应用一种变通方法；然而这种方法要求通过另一个子进程来运行所有子进程，该子进程实现了一个可执行文件加载器，并对它的所有子进程应用进程跟踪，这可能因各种原因引发问题。在这种情况下，可以把变量 'android-use-exec-loader' 改为 'nil' 来禁用该变通方法。

当该变通方法生效时，通过 'process-id' 函数取得的进程 ID 将是可执行文件加载器进程的 ID；它的子进程属于与加载器相同的进程组。因此，'interrupt-process' 及其他相关函数仍能正常工作，但把 'process-id' 返回的进程 ID 用于其他用途则不行。

进程跟踪实现机制的一个后果是，下级 shell（参见"交互式子 shell"）中的作业控制功能将无法停止进程，向 Emacs 创建的子进程发送 'SIGSTOP' 信号也不会生效。

此外，当 Emacs 本身处于后台时，Android 12 还会终止正在消耗 CPU 的子进程。系统每隔五分钟判断一次哪些进程消耗了过多 CPU，并终止消耗 CPU 时间最多的那个进程。

Android 12.1 和 Android 13 提供了一个禁用此行为的选项；要使用它，请启用"USB 调试"（参见"在 Android 上启动 Emacs"），把 Android 系统连接到另一台计算机，然后运行：

```
$ adb shell "settings put global settings_enable_monitor_phantom_procs false"
```

适用于操作系统的"语言和输入法"（Languages & Input）偏好设置并不会影响为程序设置的 C locale，但 Emacs 会在启动时考虑它们：它会根据所选语言和地区变体生成一个 locale 名称，并据此选择一个语言环境（参见"语言环境"），这不会覆盖 'LANG' 或其他与 locale 相关的环境变量。以这种方式设置的语言环境的编码系统无一例外都是 'utf-8-unix'。

相反，在 Android 5.0 或更新版本上启动 Emacs 时，'LANG' 环境变量（参见"通用变量"）会被设为 'en_US.utf8'，这会促使链接 Android C 库的子进程合理地打印输出。更早的 Android 版本根本不实现 locale，因此该变量被设为 'C'。

应用程序进程被系统视为可随意丢弃的实体。当所有 Emacs 框架都转入后台时，为了节省系统资源，Emacs 可能随时被系统终止。

在 Android 7.1 及更早版本上，Emacs 把自己指定为“后台服务”，这会促使系统避免终止 Emacs，除非内存紧张。

Android 8.0 取消了后台服务获得这种特殊待遇的能力。不过 Emacs 采用了一种变通方法：系统认为创建常驻通知的应用程序正在执行活动工作，因而会避免终止这类应用程序。于是，在这些系统上，Emacs 只要在运行就会显示一条常驻通知。

在 Android 13 之前，Emacs 不需要显示通知的权限。在 Android 13 或更高版本下，除非用户授予 Emacs 该权限，否则通知会被隐藏。尽管如此，仅仅尝试显示通知就足以避免被突然终止；通知是否显示并不影响 Emacs 在后台执行的能力，而且禁用通知也不会有任何不良后果。

然而，并不能保证系统不会终止 Emacs。虽然开放手机联盟的 Android 示例实现行为正确，但许多厂商在其专有版本的 Android 中对后台程序执行施加了额外限制。这类麻烦厂商的列表以及有时可用的变通方法见 <https://dontkillmyapp.com/>。

Android 还定义了一套权限系统，用来决定 Emacs 可以访问哪些系统服务。程序必须声明自己需要哪些权限；之后会发生什么取决于所使用的 Android 版本：

- 在或新或旧的 Android 版本（如 Android 6.0 及更高版本）下，视所安装的 Android 版本中各个权限是否存在，Emacs 在安装时只会获得以下权限：

- ‘android.permission.ACCESS_ADSERVICES_AD_ID’
- ‘android.permission.ACCESS_ADSERVICES_ATTRIBUTION’
- ‘android.permission.ACCESS_ADSERVICES_CUSTOM_AUDIENCE’
- ‘android.permission.ACCESS_ADSERVICES_TOPICS’
- ‘android.permission.ACCESS_LOCATION_EXTRA_COMMANDS’
- ‘android.permission.ACCESS_NETWORK_STATE’
- ‘android.permission.ACCESS_NOTIFICATION_POLICY’
- ‘android.permission.ACCESS_WIFI_STATE’
- ‘android.permission.AUTHENTICATE_ACCOUNTS’
- ‘android.permission.BLUETOOTH’
- ‘android.permission.BLUETOOTH_ADMIN’
- ‘android.permission.BROADCAST_STICKY’
- ‘android.permission.CALL_COMPANION_APP’
- ‘android.permission.CHANGE_NETWORK_STATE’
- ‘android.permission.CHANGE_WIFI_MULTICAST_STATE’
- ‘android.permission.CHANGE_WIFI_STATE’
- ‘android.permission.CREDENTIAL_MANAGER_QUERY_CANDIDATE_CREDENTIALS’
- ‘android.permission.CREDENTIAL_MANAGER_SET_ALLOWED_PROVIDERS’

- ‘android.permission.CREDENTIAL_MANAGER_SET_ORIGIN’
- ‘android.permission.DELIVER_COMPANION_MESSAGES’
- ‘android.permission.DETECT_SCREEN_CAPTURE’
- ‘android.permission.DISABLE_KEYGUARD’
- ‘android.permission.ENFORCE_UPDATE_OWNERSHIP’
- ‘android.permission.EXPAND_STATUS_BAR’
- ‘android.permission.FLASHLIGHT’
- ‘android.permission.FOREGROUND_SERVICE’
- ‘android.permission.FOREGROUND_SERVICE_CAMERA’
- ‘android.permission.FOREGROUND_SERVICE_CONNECTED_DEVICE’
- ‘android.permission.FOREGROUND_SERVICE_DATA_SYNC’
- ‘android.permission.FOREGROUND_SERVICE_FILE_MANAGEMENT’
- ‘android.permission.FOREGROUND_SERVICE_HEALTH’
- ‘android.permission.FOREGROUND_SERVICE_LOCATION’
- ‘android.permission.FOREGROUND_SERVICE_MEDIA_PLAYBACK’
- ‘android.permission.FOREGROUND_SERVICE_MEDIA_PROJECTION’
- ‘android.permission.FOREGROUND_SERVICE_MICROPHONE’
- ‘android.permission.FOREGROUND_SERVICE_PHONE_CALL’
- ‘android.permission.FOREGROUND_SERVICE_REMOTE_MESSAGING’
- ‘android.permission.FOREGROUND_SERVICE_SPECIAL_USE’
- ‘android.permission.FOREGROUND_SERVICE_SYSTEM_EXEMPTED’
- ‘android.permission.GET_PACKAGE_SIZE’
- ‘android.permission.GET_TASKS’
- ‘android.permission.HIDE_OVERLAY_WINDOWS’
- ‘android.permission.HIGH_SAMPLING_RATE_SENSORS’
- ‘android.permission.INTERNET’
- ‘android.permission.KILL_BACKGROUND_PROCESSES’
- ‘android.permission.MANAGE_ACCOUNTS’
- ‘android.permission.MANAGE_OWN_CALLS’
- ‘android.permission.MODIFY_AUDIO_SETTINGS’
- ‘android.permission.NFC’
- ‘android.permission.NFC_PREFERRED_PAYMENT_INFO’
- ‘android.permission.NFC_TRANSACTION_EVENT’
- ‘android.permission.PERSISTENT_ACTIVITY’
- ‘android.permission.QUERY_ALL_PACKAGES’

- ‘android.permission.READ_BASIC_PHONE_STATE’
- ‘android.permission.READ_INSTALL_SESSIONS’
- ‘android.permission.READ_NEARBY_STREAMING_POLICY’
- ‘android.permission.READ_PROFILE’
- ‘android.permission.READ_SOCIAL_STREAM’
- ‘android.permission.READ_SYNC_SETTINGS’
- ‘android.permission.READ_SYNC_STATS’
- ‘android.permission.READ_USER_DICTIONARY’
- ‘android.permission.RECEIVE_BOOT_COMPLETED’
- ‘android.permission.REORDER_TASKS’
- ‘android.permission.REQUEST_COMPANION_PROFILE_GLASSES’
- ‘android.permission.REQUEST_COMPANION_PROFILE_WATCH’
- ‘android.permission.REQUEST_COMPANION_RUN_IN_BACKGROUND’
- ‘android.permission.REQUEST_COMPANION_START_FOREGROUND_SERVICES_FROM_BACKGROUND’
- ‘android.permission.REQUEST_COMPANION_USE_DATA_IN_BACKGROUND’
- ‘android.permission.REQUEST_DELETE_PACKAGES’
- ‘android.permission.REQUEST_IGNORE_BATTERY_OPTIMIZATIONS’
- ‘android.permission.REQUEST_OBSERVE_COMPANION_DEVICE_PRESENCE’
- ‘android.permission.REQUEST_PASSWORD_COMPLEXITY’
- ‘android.permission.RESTART_PACKAGES’
- ‘android.permission.RUN_USER_INITIATED_JOBS’
- ‘android.permission.SET_WALLPAPER’
- ‘android.permission.SET_WALLPAPER_HINTS’
- ‘android.permission.SUBSCRIBED_FEEDS_READ’
- ‘android.permission.SUBSCRIBED_FEEDS_WRITE’
- ‘android.permission.TRANSMIT_IR’
- ‘android.permission.UPDATE_PACKAGES_WITHOUT_USER_ACTION’
- ‘android.permission.USE_BIOMETRIC’
- ‘android.permission.USE_CREDENTIALS’
- ‘android.permission.USE_EXACT_ALARM’
- ‘android.permission.USE_FINGERPRINT’
- ‘android.permission.USE_FULL_SCREEN_INTENT’
- ‘android.permission.VIBRATE’
- ‘android.permission.WAKE_LOCK’
- ‘android.permission.WRITE_PROFILE’

- ‘android.permission.WRITE_SMS’
- ‘android.permission.WRITE_SOCIAL_STREAM’
- ‘android.permission.WRITE_SYNC_SETTINGS’
- ‘android.permission.WRITE_USER_DICTIONARY’

其他权限必须由用户在系统设置应用程序中授予。更多细节请咨询你的设备制造商，因为具体做法因设备而异。

- 在 Android 5.1 及更早版本上，Emacs 安装后会自动获得其请求的下列权限：

- ‘android.permission.ACCESS_COARSE_LOCATION’
- ‘android.permission.ACCESS_FINE_LOCATION’
- ‘android.permission.BODY_SENSORS’
- ‘android.permission.CALL_PHONE’
- ‘android.permission.CAMERA’
- ‘android.permission.CAPTURE_CONSENTLESS_BUGREPORT_ON_USERDEBUG_BUILD’
- ‘android.permission.GET_ACCOUNTS’
- ‘android.permission.POST_NOTIFICATIONS’
- ‘android.permission.PROCESS_OUTGOING_CALLS’
- ‘android.permission.READ_CALENDAR’
- ‘android.permission.READ_CALL_LOG’
- ‘android.permission.READ_CELL_BROADCASTS’
- ‘android.permission.READ_CONTACTS’
- ‘android.permission.READ_EXTERNAL_STORAGE’
- ‘android.permission.READ_PHONE_NUMBERS’
- ‘android.permission.READ_PHONE_STATE’
- ‘android.permission.READ_SMS’
- ‘android.permission.RECEIVE_MMS’
- ‘android.permission.RECEIVE_SMS’
- ‘android.permission.RECEIVE_WAP_PUSH’
- ‘android.permission.RECORD_AUDIO’
- ‘android.permission.REQUEST_INSTALL_PACKAGES’
- ‘android.permission.SEND_SMS’
- ‘android.permission.SMS_FINANCIAL_TRANSACTIONS’
- ‘android.permission.SYSTEM_ALERT_WINDOW’
- ‘android.permission.WRITE_CALENDAR’
- ‘android.permission.WRITE_CALL_LOG’
- ‘android.permission.WRITE_CONTACTS’

- ‘android.permission.WRITE_EXTERNAL_STORAGE’
- ‘android.permission.WRITE_SETTINGS’
- ‘android.permission.ACCESS_LOCATION_EXTRA_COMMANDS’
- ‘android.permission.ACCESS_NETWORK_STATE’
- ‘android.permission.ACCESS_WIFI_STATE’
- ‘android.permission.BLUETOOTH’
- ‘android.permission.BLUETOOTH_ADMIN’
- ‘android.permission.BROADCAST_STICKY’
- ‘android.permission.CHANGE_NETWORK_STATE’
- ‘android.permission.CHANGE_WIFI_MULTICAST_STATE’
- ‘android.permission.CHANGE_WIFI_STATE’
- ‘android.permission.DISABLE_KEYGUARD’
- ‘android.permission.EXPAND_STATUS_BAR’
- ‘android.permission.FLASHLIGHT’
- ‘android.permission.GET_PACKAGE_SIZE’
- ‘android.permission.GET_TASKS’
- ‘android.permission.INTERNET’
- ‘android.permission.KILL_BACKGROUND_PROCESSES’
- ‘android.permission.MODIFY_AUDIO_SETTINGS’
- ‘android.permission.NFC’
- ‘android.permission.PERSISTENT_ACTIVITY’
- ‘android.permission.QUERY_ALL_PACKAGES’
- ‘android.permission.READ_BASIC_PHONE_STATE’
- ‘android.permission.READ_SYNC_SETTINGS’
- ‘android.permission.READ_SYNC_STATS’
- ‘android.permission.READ_USER_DICTIONARY’
- ‘android.permission.RECEIVE_BOOT_COMPLETED’
- ‘android.permission.REORDER_TASKS’
- ‘android.permission.REQUEST_DELETE_PACKAGES’
- ‘android.permission.REQUEST_IGNORE_BATTERY_OPTIMIZATIONS’
- ‘android.permission.REQUEST_OBSERVE_COMPANION_DEVICE_PRESENCE’
- ‘android.permission.RESTART_PACKAGES’
- ‘android.permission.SET_WALLPAPER’
- ‘android.permission.SET_WALLPAPER_HINTS’
- ‘android.permission.TRANSMIT_IR’

- ‘android.permission.VIBRATE’
- ‘android.permission.WAKE_LOCK’
- ‘android.permission.WRITE_SYNC_SETTINGS’
- ‘android.permission.WRITE_USER_DICTIONARY’

尽管这些权限大多不为 Emacs 本身所用，Emacs 仍声明了它们，因为它们可能对其他程序有用；例如，访问联系人的权限对 EUDC 可能就有用。

> 注：除非指定了“共享用户 ID”，且安装了使用同一“包签名密钥”签名的其他应用程序；在这种情况下，Emacs 以同一用户身份运行，并可以访问与上述每个应用程序相同的文件。

H.6 Android 窗口系统

Android 的窗口系统与众不同：所有窗口都向应用程序报告为最大化或全屏，而且在一般情况下，同一时间只能显示一个窗口。在较大的设备上，系统允许同时在屏幕上平铺最多四个窗口；不过在模拟器中，或者在配置为“桌面”系统的安装中，它会像其他桌面窗口管理器那样自由堆叠可调整大小的窗口。

窗口（在系统术语中称为 activity）创建后并不会永远存在，因为系统为了节省内存可能会暂停不可见的窗口，前提是假定程序会把自身内容保存到磁盘，待用户从任务切换器中选中这些窗口时再恢复。此外，启动 Emacs 时操作系统还会创建一个受到特殊待遇的窗口，Emacs 应当采用它。

Emacs 处理窗口管理时的一般目标是：尽量缩小暴露给 Lisp 的框架行为与普通窗口系统（如 X Windows）上框架行为之间的差异；这一目标实际达成的程度，取决于所安装的 Android 版本中窗口管理设施的可用性，以及操作系统对非活动窗口的策略。当不得不对此类策略做出让步时，Emacs 宁可销毁框架，也不保留没有 activity 可用于显示它们的框架——除非该框架是初始框架，因而显示在启动时创建的 activity 中，只要 Emacs 仍在运行，这个 activity 就可以被打开和识别。

Android 5.0 及更高版本支持精确的窗口管理实现：框架与其显示所在的 activity 之间是一一对应的，因此在任务切换器中删除 activity 会直接影响相应的框架，反之亦然。只有两个例外：

- 系统因无活动而暂停某个仍留在任务切换器中的 activity 之后，如果在该 activity 仍处于非活动状态时把它从任务切换器中移除，则不会删除其中的框架，因为在这种情况下 Emacs 不会收到其 activity 被删除的通知。该框架会在下一次促发检查存活窗口列表的窗口管理操作时被删除。同样，显示某个框架的非活动 activity 也不会随其框架一起被立即删除，但如果从窗口列表中选中它，或者在另一次检查窗口列表时，它就会被删除。

- 除初始框架外，任何框架在后台闲置 4 到 6 小时后都可能被删除——如果系统在“清理”任务切换器中多余的（想必也是不需要的）任务时将其移除的话。初始框架不受此待遇，因为它可以通过任务切换器以外的方式重新打开；但由于这种机制造成的删除与用户从任务切换器中移除 activity 的正当操作无法区分，所以在距最后一次活动满 4 小时之后，初始框架也会忽略用户的这类移除操作。

Android 4.4 及更早版本提供的接口要差得多，不足以完整实现窗口管理。在此类系统上，Emacs 改用一种相当原始的机制：除初始框架外，所有框架在其 activity 被暂停时都被删除；同一时间只有一个 activity（不计启动时创建的 activity）可见；未附着的框架显示在第一个可用的空闲 activity 中。

Emacs 在 Android 上只支持 GUI 功能的一个有限子集；限制如下：

- 不支持滚动条，因为它们在 Android 设备上几乎无用。
- ‘alpha’、‘alpha-background’、‘z-group’、‘override-redirect’、‘mouse-color’、‘title’、‘wait-for-wm’、‘sticky’ 和 ‘undecorated’ 等框架参数（参见《Emacs Lisp 参考手册》的“Frame Parameters”）不受支持。
- 在 Android 4.0 及更早版本上，顶层框架的 ‘fullscreen’ 框架参数始终为 ‘maximized’；在更高版本的 Android 上，它也可以是 ‘fullscreen’。

在 Android 上，Emacs 并未实现 X Window System 下支持的所有与选区（selection）相关的功能。例如，只支持 ‘CLIPBOARD’ 和 ‘PRIMARY’ 选区（参见“图形显示环境中的“剪切与粘贴”操作”），而且 Emacs 只能把选区设为纯文本。

此外，Android 系统本身对 Emacs 可以访问哪些选区数据施加了一些限制：

- 在 Android 2.3 及更早版本上，函数 ‘gui-selection-owner-p’ 对剪贴板选区始终返回 ‘nil’。
- 在 Android 3.0 到 Android 9.0 之间，Emacs 可以随时访问剪贴板，并且 ‘gui-selection-owner-p’ 始终返回准确的结果。
- 在 Android 10.0 及更高版本下，只有当 Emacs 的某个框架拥有输入焦点时，它才能访问剪贴板数据；并且 ‘gui-selection-owner-p’ 对剪贴板选区始终返回 ‘nil’。

由于 Android 系统本身没有主选区（primary selection）的概念，Emacs 转而提供了一种模拟。这意味着无法通过剪切粘贴把主选区的内容传送给其他应用程序。

音量键通常由 Emacs 保留，用于在没有物理键盘时提供退出 Emacs 的能力（参见“配合虚拟键盘使用 Emacs”）。不过，如果你希望它们改为调节音量，可以把变量 ‘android-pass-multimedia-buttons-to-system’ 设为非 ‘nil’ 值；注意，那样一来你就不能再使用音量键退出 Emacs 了，而且通常更容易的做法是：在按住音量键的同时，拉下通知栏或调出其他界面，使 Emacs 暂时失去键盘焦点。

在 Android 6.0 及更高版本上，当 Emacs 没有输入焦点时无法显示对话框（参见“使用对话框”）。如果这对你很重要，可以通过授予 Emacs 在其他程序之上显示的权限来恢复此能力。在大多数系统上，可以从以下“设置”菜单完成：

```
System -> Apps -> Emacs -> More -> Display over other apps
```

物理修饰键与键事件中报告的 Emacs 修饰键（参见“修饰键”）之间存在直接的对应关系，只有一个例外：如果按下键盘上的 <Alt>，Emacs 会改为报告 <Meta> 修饰键，反之亦然。这种不规则之处是因为大多数键盘没有专门的 <Meta> 键，而 <Alt> 修饰键在 Emacs 中很少使用。

请记住，Android 对 <Super> 修饰键使用了不同的名称：在 Android 键盘上以及“设置”的键位图菜单中，它被称为 <SYM>。

Android 输入法惯于在事件过滤过程中悄然丢弃含 ‘C-SPC’ 的键序列，令用户恼火，尽管它们对这类键序列通常并没有真正的用途。默认情况下，Emacs 会在这些键序列被输入法过滤之前截获它们。

如果发现这样做并不需要（例如输入法把 'C-SPC' 当作切换语言的快捷键），可以把变量 'android-intercept-control-space' 设为 'nil' 来禁用它。

Android 系统中安装的键盘响铃表现为一个振动元件：每当响铃时，它会振动若干毫秒。可以通过把变量 'android-keyboard-bell-duration' 改为 '10' 到 '1000' 之间的任意值来自定义振动时长。

在 Android 上无法自动检测显示器与颜色相关的特性，因此，如果希望 Emacs 呈现字体（face）和图像的方式与灰度或单色显示器的真实视觉属性一致，就应把变量 'android-display-planes' 配置为合适的值：前一类显示器设为 '8'，后一类设为 '1'，这会分别强制所有颜色以 256 级灰度或单色渲染。由于此变量在建立显示连接时就会被处理，除非在 'early-init.el' 中进行自定义（参见"早期初始化文件"），否则自定义不会生效。

此变量的值不影响字体驱动中的抗锯齿，因为单色显示器仍然期望 Emacs 提供抗锯齿文本，这些文本经显示驱动处理为位图数据后才交给它们。

H.7 Android 下的字体后端与选区

Emacs 在 Android 下支持两种字体后端：它们分别名为 'sfnt-android' 和 'android'。

启动时，Emacs 会枚举目录 '/system/fonts' 和 '/product/fonts' 中的所有 TrueType 格式字体，以及 Emacs 主目录内的 'fonts' 目录（"用户字体目录"）。Emacs 假定总会有名为 "Droid Sans Mono" 的字体，并默认使用该字体。这些字体随后由 'sfnt-android' 字体驱动显示。

此字体驱动目前尚不支持 OpenType 或彩色字体；因此，Emacs 只能使用任一给定系统上所安装字体的一部分。如果你有兴趣解除这一限制，请联系 <emacs-devel@gnu.org>。

如果 'sfnt-android' 字体驱动完全找不到任何字体，Emacs 会退回到 'android' 字体驱动。这是一个质量很差的字体驱动，原因在于 Android 平台提供的字体度量存在局限和不准确之处。在这种情况下，Emacs 使用系统中配置的 "Monospace" 字体；它应当始终是 Droid Sans Mono。

与 X 系统一样，Emacs 在 Android 下支持可变形字体（distortable fonts）。这类字体（也称 "TrueType GX 字体"、"可变字体"和"多主字体"）用单个字体文件提供多种不同样式（"Bold"、"Italic" 等）。

当发现用户安装的可变形字体时，此前已发现的字体所提供的每个样式都将不再使用。此外，只要新的可变形字体提供了此前安装的字体所提供样式的超集，任何先前安装的同族名字体也会被忽略。当发现常规字体时，任何具有相同样式和族的先前常规字体都会被移除；同族的可变形字体将不再用于提供该样式。

Emacs 通常假定存在名为 'Monospace'、'Monospace Serif'、'Sans Serif' 和 'DejaVu Serif' 的字体族。由于 Android 不提供任何使用这些名称的字体，Emacs 会把对它们的请求改为请求 Android 随附的一组对应字体族之一。

若要改变被替换的字体族集合，或改变用于替换它们的字体族，请修改变量 'sfnt-default-family-alist'，然后重启 Emacs。请记住，这通常并无必要；对默认字体或 'variable-pitch' 字体（face）的自定义，最好通过修改它们的定义来完成（参见"自定义字体"）。

H.8 Android 启动问题排查

由于 Android 没有命令行，启动 Emacs 时通常无法指定命令行参数。如果你在 Emacs 初始化文件中犯错，导致 Emacs 完全无法启动，这会非常麻烦，因为系统通常禁止其他程序访问 Emacs 的主目录。参见“初始选项”。

不过，可以通过一个特殊的首选项界面，以相当于 '--quick' 或 '--debug-init' 选项的方式启动 Emacs。在 Android 7.0 及更高版本上，可以从系统设置程序中的 Emacs“应用信息”页面进入该界面；在更旧的系统上，它会作为桌面上一个单独的图标显示，标签为 “Emacs options”。

更多细节请咨询你的设备制造商，因为具体做法因设备而异。

某一份 Emacs 在设备上首次启动时，会花一些时间加载通常随 Emacs 提供的预加载 Lisp 文件。这会在 files 目录中生成一个“转储文件”（dump file）（参见“初始选项”），其中含有该份 Emacs 独有的标识符。

同一份 Emacs 下次启动时，只需加载该转储文件中包含的数据，从而大大缩短启动时间。

如果因某种意外情况导致转储文件损坏，Emacs 可能会崩溃。若发生这种情况，可以通过上述首选项界面删除 Emacs files 目录中保存的转储文件。

在 Android 4.4 及更高版本上，Emacs 还支持另一种抢救损坏的 Emacs 安装的方法：Emacs 导出一个“文档提供程序”（documents provider），它可访问 Emacs 主目录的内容，任何文件管理器程序都能访问它。

如果你能弄清如何在设备自带的文件管理器中打开该文档提供程序，就可以改为在那里重命名、删除或编辑你的初始化文件或转储文件。

H.9 在 Android 上安装额外软件

Android 的默认安装只附带极其有限的 Unix 风格命令行工具。有若干项目可以扩充这一选择，提供的选项从改进版 Unix 命令行工具复刻，到提供大量自由 GNU 与 Unix 软件的软件包仓库。

Busybox（https://busybox.net）以单个静态链接的 Linux 二进制文件提供 Unix 工具以及某些流行 GNU 程序（如 'wget'）的有限复刻，该二进制文件可在 Android 下运行。

Termux（https://termux.dev）提供基于 Debian 项目 'dpkg' 系统的包管理器，以及一组包含大量 Unix 系统自由软件的软件包仓库，其中包括编译器、调试器，以及 C、C++、Java、Python 和 Common Lisp 等语言的运行时。这些包通常是在专门构建的终端模拟器应用程序中安装的；但如果 Emacs 使用相同的应用签名密钥构建，并且其“共享用户 ID”被设为与终端模拟器程序相同的包名，那么 Emacs 也能访问这些包。Emacs 发行版中的文件 'java/INSTALL' 演示了如何以这种方式构建 Emacs。

termux-packages（https://github.com/termux/termux-packages）提供 Termux 用于生成其软件包仓库的包定义，也可以独立编译这些定义，以便安装到 Emacs 的主目录中。

除上述项目外，大多数基于 Linux 内核的系统的静态链接二进制文件也能在 Android 上运行。

Emacs 可以配置为借助 'librsvg' 库支持查看 SVG 图像文件。SVG 文件中可能引用文件系统上的其他图像，而 'librsvg' 默认无法检测这些图像的格式；除非在主目录下的目录 '.local/share/mime' 中安装了符合 XDG 规范的“MIME 数据库”，否则它们会被渲染成空白方块。

由于生成此类数据库必须要有 XDG 的 'shared-mime-info' 工具，建议先在 GNU/Linux 或 Unix 系统上的临时目录中生成该数据库，然后再传输到目标 Android 设备。安装最新版本的 'shared-mime-info' 后，创建一个用于生成 MIME 数据库的临时目录，把默认的 'freedesktop.org.in' MIME 目录复制到名为 'packages' 的目录中，然后执行 'update-mime-info' 生成数据库：

```
$ mkdir -p my-mime-database/packages
$ cp /usr/share/mime/packages/freedesktop.org.xml my-mime-database/packages
$ update-mime-info my-mime-database
```

这可能会打印一系列提示，说明所指定的 MIME 数据库不在宿主系统的搜索路径中，这些提示无关紧要。接着把 MIME 数据库的内容传输到目标系统，例如传到 '/sdcard/Download/my-mime-database.tar.gz'：

```
$ cd; mkdir -p .local/share
$ tar xfz /sdcard/Download/my-mime-database.tar.gz
$ mv my-mime-database .local/share/mime
$ rm /sdcard/Download/my-mime-database.tar.gz
```

如果你的 Emacs 会话已经尝试过显示带嵌入内容的 SVG 图像，则必须重启 Emacs，新的 MIME 数据库才会生效。

附录 I Emacs 与 Microsoft Windows/MS-DOS

本节介绍在 Microsoft Windows 上使用 Emacs 的一些特殊之处。其中有些特殊之处也与 Microsoft 较旧的 MS-DOS 操作系统有关。不过，仅与 MS-DOS 有关的 Emacs 特性在单独一节中介绍（参见“Emacs 与 MS-DOS”）。

MS-Windows 是一个非自由操作系统；这意味着它剥夺了用户理应享有的自由。这是一种不公正。为了你的自由，我们敦促你改用自由操作系统。

我们之所以在专有操作系统上支持 GNU Emacs，是希望这种自由的体验能激励用户摆脱它们。

Emacs 在 MS-Windows 上的行为与本手册其余部分所记载的相当接近，包括对长文件名、多框架、滚动条、鼠标菜单和子进程的支持。不过，还有一些特殊的考虑，在此加以说明。

- Windows HOME：Emacs 在哪里查找你的 ‘.emacs’，以及从哪里启动。
- Windows Keyboard：Windows 特有的键盘特性。
- Windows Mouse：Windows 特有的鼠标特性。
- Windows Processes：在 Windows 上运行子进程。
- Windows Printing：如何在 MS-Windows 上指定打印机。
- Windows Fonts：在 MS-Windows 上指定字体。
- Windows Misc：Windows 的杂项特性。
- MS-DOS：在 MS-DOS 上使用 Emacs。

I.1 如何在 MS-Windows 上启动 Emacs

在 MS-Windows 上启动 Emacs 有几种方式：

1. 通过桌面快捷方式图标：双击该图标，或单击一次再按 <RET>。桌面快捷方式应在其“Target”（位于快捷方式的“Properties”中）中指定 ‘runemacs.exe’ 的完整绝对文件名，而不要指定 ‘emacs.exe’。这是因为，如果快捷方式的目标是 ‘emacs.exe’（就 Windows 而言它是一个控制台程序），就会创建一个控制台窗口，而 ‘runemacs.exe’ 会隐藏该窗口。若使用这种方式，Emacs 会在快捷方式所指定的目录中启动。要控制该目录，请右键单击快捷方式，选择“Properties”，然后在“Shortcut”选项卡中按你的喜好修改“Start in”字段。

1. 通过任务栏快捷方式图标：单击一次鼠标左键。自 Vista 起的 Windows 版本允许你通过“固定”任务栏中正在运行程序的图标来创建此类快捷方式。你可以对 Emacs 这样做，但之后必须把固定后的快捷方式属性改为运行 ‘runemacs.exe’，而不是 ‘emacs.exe’。你也可以在开始菜单中右键单击 Emacs 图标，然后选择 ‘Pin to taskbar’，把 Emacs 固定到任务栏。同样，务必把要运行的程序指定为 ‘runemacs.exe’。你可以通过设置快捷方式属性中的“Start in”字段来控制 Emacs 的启动位置。

1. 通过命令提示符窗口：在提示符下输入 ‘emacs <RET>’。在你这样做的那个命令提示符窗口中，直到 Emacs 退出之前都无法再调用其他命令。此时，Emacs 会在 Windows shell 的当前目录中启动。

1. 通过命令提示符窗口：在提示符下输入 ‘runemacs <RET>’。你这样做过的那个命令提示符窗口会立即可以用于调用其他命令。此时，Emacs 会在 Windows shell 的当前目录中启动。

1. 通过 Windows 的 'Run' 对话框（通常单击 'Start' 按钮即可打开）。在对话框中输入 'runemacs <RET>'，Emacs 会在 Windows 中相当于你的用户 'HOME' 目录的父目录中启动，参见"MS-Windows 上的 HOME 与启动目录"。

1. 通过 'emacsclient.exe' 或 'emacsclientw.exe'，它们让你可以从其他程序调用 Emacs，并复用一个正在运行的 Emacs 进程来完成其他程序所需的编辑任务。参见"把 Emacs 用作服务器"。'emacsclient.exe' 与 'emacsclientw.exe' 的区别在于，前者是控制台程序，而后者是 Windows GUI 程序。这两个程序都会等待 Emacs 发出编辑任务已完成的信号，然后才退出并把控制权交还给调用它们的程序。具体用哪一个取决于需要编辑服务的程序的期望。如果该程序本身是控制台（文本模式）程序，应使用 'emacsclient.exe'，这样它的消息和提示会与调用程序出现在同一个命令窗口中。相反，如果调用程序是 GUI 程序，最好使用 'emacsclientw.exe'，因为从 GUI 程序调用 'emacsclient.exe' 会弹出一个命令窗口。一个典型情形是，在 Windows 资源管理器中右键单击文件并从弹出菜单中选择"Open With"，此时你会希望使用 'emacsclientw.exe'。如果调用 'emacsclient' 时 Emacs 可能没有运行（或没有作为服务器运行），请使用 '--alternate-editor=' 或 '-a' 选项——这样总能得到一个编辑器。通过 'emacsclient' 调用时，Emacs 会在调用 'emacsclient' 的那个程序的当前目录中启动。

注意，由于 MS-Windows 的限制，Emacs 无法在同一会话中同时拥有 GUI 框架和文本模式框架。它也无法在多于一个"命令提示符"窗口中打开文本模式框架，因为每个 Windows 程序在任何时刻只能有一个控制台。由于这些原因，如果你带 '-c' 选项调用 'emacsclient'，而 Emacs 服务器运行在文本模式会话中，Emacs 就总会在启动它的同一个"命令提示符"窗口中创建新的文本模式框架；只有当服务器运行在 GUI 会话中时才会创建 GUI 框架。同样，如果你带 '-t' 选项调用 'emacsclient'，则服务器运行在 GUI 会话中时 Emacs 会创建 GUI 框架，而会话以文本模式运行在"命令提示符"窗口中时会创建文本模式框架。参见"'emacsclient' 选项"。

I.2 文本文件与二进制文件

GNU Emacs 使用换行符分隔文本行。这是 GNU、Unix 及其他符合 POSIX 的系统所采用的约定。

相比之下，MS-DOS 和 MS-Windows 通常用回车后跟换行这两个字符组成的序列来分隔文本行。（换行符与 newline 是同一个字符。）因此，要用 Emacs 方便地编辑典型文件，就需要转换这些行尾（end-of-line，EOL）序列。Emacs 通常正是这样做的：读取文件时把回车加换行转换成换行，写文件时把换行转换成回车加换行。处理国际字符编码转换的同一机制也完成这种转换（参见"编码系统"）。

大多数文件都经过这种特殊格式转换，其一个后果是：Emacs 报告的字符位置（参见"光标位置信息"）与操作系统所知的文件大小信息并不一致。

此外，如果 Emacs 从文件内容判断出它使用换行而非回车加换行作为行分隔符，那么读写该文件时就不做 EOL 转换。因此，你可以在 MS-DOS 上无需特别处理就能读取并编辑来自 GNU 和 Unix 系统的文件，而且编辑后它们仍保持 Unix 风格的行尾约定。

模式行会指出当前缓冲区是否使用了行尾转换。如果该缓冲区使用了 MS-DOS 行尾转换，MS-Windows 版的 Emacs 会在模式行开头附近的编码系统助记符后显示反斜杠 '\'。如果没有执行 EOL 转换，则显示字符串 '(Unix)' 代替反斜杠，提醒你该文件的 EOL 格式不是通常的回车加换行。

要访问文件并指定它使用 DOS 风格还是 Unix 风格的行尾，请指定一个编码系统（参见"为文件文本指定编码系统"）。例如，'C-x <RET> c unix <RET> C-x C-f foobar.txt' 会在不转换 EOL 的情况下访问文件 'foobar.txt'；

如果某些行以回车加换行对结尾，Emacs 会在该行末尾显示 '^M'。同样，你可以用 'C-x <RET> f' 命令让 Emacs 以指定的 EOL 格式保存缓冲区。例如，要以 Unix EOL 格式保存缓冲区，输入 'C-x <RET> f unix <RET> C-x C-s'。如果你访问文件时使用 DOS EOL 转换，随后又以 Unix EOL 格式保存它，那实际上就把该文件转换成了 Unix EOL 风格，如同 'dos2unix' 程序一样。

当你使用 NFS、Samba 或其他类似方法访问位于使用 GNU 或 Unix 系统的计算机上的文件系统时，Emacs 不应对这些文件系统中的任何文件执行行尾转换——即使你新建文件时也不应如此。为此，可调用函数 'w32-add-untranslated-filesystem' 把这些文件系统指定为"不转换"的文件系统。它接受一个参数：文件系统名，包括驱动器号和可选的目录。例如，

```
(w32-add-untranslated-filesystem "Z:")
```

将驱动器 Z 指定为不转换的文件系统，而

```
(w32-add-untranslated-filesystem "Z:\\foo")
```

将驱动器 Z 上的目录 '\foo' 指定为不转换的文件系统。

最常见的是在 '.emacs' 或 'init.el' 初始化文件中，或在 'site-start.el' 中使用 'w32-add-untranslated-filesystem'，这样你站点上的所有用户都能从中受益。

要抵消 'w32-add-untranslated-filesystem' 的效果，请使用函数 'w32-remove-untranslated-filesystem'。该函数接受一个参数，应当是一个与之前传给 'w32-add-untranslated-filesystem' 的字符串相同的字符串。

把文件系统指定为不转换只影响字符集转换，不影响行尾转换。本质上，它让 Emacs 默认以 Unix 风格约定（在行尾使用换行）创建新文件。参见"编码系统"。

I.3 MS-Windows 上的文件名

MS-Windows 和 MS-DOS 通常用反斜杠 '\' 分隔文件名中的名称单元，而不用其他系统上的斜杠。MS-DOS/MS-Windows 上的 Emacs 允许使用斜杠或反斜杠，并且能识别文件名中的驱动器号。

在 MS-DOS/MS-Windows 上，文件名不区分大小写，因此 Emacs 在补全时默认忽略文件名中的字母大小写。为此，在 MS-DOS/MS-Windows 上 'read-file-name-completion-ignore-case' 的默认值不是 'nil'。参见"补全选项"。

变量 'w32-get-true-file-attributes' 控制 Emacs 是否应发出额外的系统调用，以便在 'file-attributes' 和 'directory-files-and-attributes' 之类的原语中更准确地确定文件属性。这些额外调用是正确报告管道等特殊文件的所有者、链接数和文件类型所必需的。没有这些系统调用，文件所有者会被归于当前用户，链接数总被报告为 1，特殊文件则被报告为普通文件。

如果该变量的值为 'local'（默认值），Emacs 只对本地固定驱动器上的文件发出这些额外系统调用。任何其他非 'nil' 值意味着连可移动卷和远程卷也这样做，这可能会拖慢 Dired 及其他相关特性。值为 'nil' 意味着从不发出这些系统调用。非 'nil' 值在支持硬链接和文件安全的 NTFS 卷上比在 FAT、FAT32 和 exFAT 卷上更有用。

与 Unix 不同，MS-Windows 文件系统限制文件名中可用的字符。以下字符不允许使用：

- Shell 重定向符号 '<'、'>' 和 '|'。

- 冒号 ':'（驱动器号之后的除外）。
- 正斜杠 '/' 和反斜杠 '\'（作为目录分隔符时除外）。
- 通配符 '*' 和 '?'。
- 码位为十进制 1 到 31 的控制字符。尤其是文件名中不允许换行。
- 码位为零的空字符（这一限制在 Unix 文件系统中同样存在）。

此外，引用任何名称与 DOS 字符设备（如 'NUL'、'LPT1'、'PRN' 或 'CON'）相匹配的文件，无论是否带文件名扩展名，在任何目录中都会解析到这些字符设备。因此，只有在你想要使用相应字符设备时才使用这类文件名。

I.4 在 MS-Windows 上模拟 'ls'

Dired 通常使用外部程序 'ls' 生成显示在 Dired 缓冲区中的目录列表（参见"Dired 目录编辑器"）。然而，MS-Windows 和 MS-DOS 系统并不自带这样的程序，尽管有若干 GNU 'ls' 的移植版本可用。因此，这些系统上的 Emacs 用 Lisp 通过 'ls-lisp.el' 包来模拟 'ls'。虽然 'ls-lisp.el' 提供了相当完整的 'ls' 模拟，但该模拟有一些特有的选项和特性；本节将介绍它们。

'ls' 模拟支持许多 'ls' 开关，但并非全部。以下是它确实支持的开关列表：'-A'、'-a'、'-B'、'-C'、'-c'、'-G'、'-g'、'-h'、'-i'、'-n'、'-R'、'-r'、'-S'、'-s'、'-t'、'-U'、'-u'、'-v' 和 '-X'。'-F' 开关得到部分支持（它会追加分类文件的字符，但不会阻止跟随符号链接）。

在 MS-Windows 和 MS-DOS 上，构建 Emacs 时已预加载 'ls-lisp.el'，因此这些平台上总是使用 'ls' 的 Lisp 模拟。如果你有移植版的 'ls'，把 'ls-lisp-use-insert-directory-program' 设为非 'nil' 值即可改为使用由变量 'insert-directory-program' 指定的外部程序。

'ls-lisp.el' 对文件排序的顺序取决于下面介绍的几个可定制选项。

默认排序顺序遵循由系统区域设置派生的区域特定规则。把 'ls-lisp-use-string-collate' 定制为 'nil' 值可以使排序顺序与区域设置无关。

在 GNU 和 Unix 系统上，当区域设置的编码为 UTF-8 时，排序顺序遵循 Unicode 排序算法（UCA）。要在 MS-Windows 上获得类似效果，'ls-lisp-UCA-like-collation' 变量应取非 'nil' 值（这是默认值）。所得的排序顺序会忽略标点、符号字符和空白字符，因此 '.foobar'、'foobar' 和 'foo bar' 会排在一起，而不是相隔很远。

默认情况下，'ls-lisp.el' 对它生成的目录列表使用区分大小写的排序顺序；这样列表看起来与其他平台相同。如果你希望文件按不区分大小写的顺序排序，请把变量 'ls-lisp-ignore-case' 设为非 'nil' 值。

默认情况下，文件和子目录一起排序，以模拟 'ls' 的行为。然而，原生 MS-Windows/MS-DOS 文件管理器把目录列在文件之前；如果你想要这种行为，请把选项 'ls-lisp-dirs-first' 定制为非 'nil' 值。

变量 'ls-lisp-verbosity' 控制 'ls-lisp.el' 显示哪些文件属性。其值应为 'nil' 或一个包含符号 'links'、'uid' 和 'gid' 中一个或多个的列表。'links' 表示显示与该文件数据相关联（又称"指向"该文件）的不同文件名的计数；这只在 NTFS 卷上有用。'uid' 表示显示拥有该文件的用户的数字标识符。'gid' 表示显示文件所有者所属组的数字标识符。默认值是 '(links uid gid)' 即显示全部 3 个可选属性。值 'nil' 表示不显示这些属性中的任何一个。

变量 'ls-lisp-emulation' 通过为上述 3 个选项（'ls-lisp-ignore-case'、'ls-lisp-dirs-first' 和 'ls-lisp-verbosity'）设定默认值来控制 'ls' 模拟的风格。该选项的值可以是下列符号之一：

- 'GNU'、'nil'：模拟 GNU 系统；这是默认值。这会把 'ls-lisp-ignore-case' 和 'ls-lisp-dirs-first' 设为 'nil'，把 'ls-lisp-verbosity' 设为 '(links uid gid modes)'。
- 'UNIX'：模拟 Unix 系统。与 'GNU' 类似，但把 'ls-lisp-verbosity' 设为 '(links uid modes)'。
- 'MacOS'：模拟 macOS。把 'ls-lisp-ignore-case' 设为 't'，把 'ls-lisp-dirs-first' 和 'ls-lisp-verbosity' 设为 'nil'。
- 'MS-Windows'：模拟 MS-Windows。把 'ls-lisp-ignore-case' 和 'ls-lisp-dirs-first' 设为 't'，在 Windows 9X 上把 'ls-lisp-verbosity' 设为 'nil'，在现代版本的 Windows 上设为 't'。注意，即使在 Windows 上，默认的模拟也不是 'MS-Windows'，因为那些平台上许多 Emacs 用户更喜欢 GNU 默认值。

'ls-lisp-emulation' 的任何其他值都等同于 'GNU'。定制该选项会调用函数 'ls-lisp-set-options' 以按需更新这 3 个依赖选项。如果在 'ls-lisp.el' 加载后（注意它在 MS-Windows 和 MS-DOS 上是预加载的）不通过 customize 而直接改变该变量的值，你可以手动调用该函数以获得相同结果。

变量 'ls-lisp-support-shell-wildcards' 控制如何支持文件名模式：若它非 'nil'（默认值），模式被当作 shell 风格的通配符；否则被当作 Emacs 正则表达式。

变量 'ls-lisp-format-time-list' 定义如何格式化文件的日期和时间。该变量的值被忽略，除非 Emacs 无法确定当前区域设置。（不过，如果 'ls-lisp-use-localized-time-format' 的值为非 'nil'，即使当前区域设置可用，Emacs 也遵循 'ls-lisp-format-time-list'；见下文。）

'ls-lisp-format-time-list' 的值是两个字符串组成的列表。如果文件的修改时间在当前年份之内，就使用第一个字符串；对于较旧的文件则使用第二个字符串。在这两个字符串中，你都可以用 '%' 序列来替换时间的各部分。例如：("%b %e %H:%M" "%b %e %Y")

注意，替换这些 '%' 序列的字符串取决于当前区域设置。关于格式时间说明符的更多信息，参见《Emacs Lisp 参考手册》的"Time Parsing"。

通常，Emacs 以传统格式或 ISO 风格格式格式化文件时间戳。不过，如果变量 'ls-lisp-use-localized-time-format' 的值为非 'nil'，Emacs 就按 'ls-lisp-format-time-list' 指定的方式格式化文件时间戳。'ls-lisp-format-time-list' 中的 '%' 序列会产生依赖区域设置的月份和日期名称，这可能导致 Dired 显示中的列错位。'ls-lisp-use-localized-time-format' 的默认值是 'nil'。

I.5 MS-Windows 上的 HOME 与启动目录

Windows 中相当于 'HOME' 的是"用户特定的应用程序数据目录"。实际位置取决于 Windows 版本；典型值是 Windows 2000 至 XP 上的 'C:\Documents and Settings\USERNAME\Application Data'，Windows Vista 及更高版本上的 'C:\Users\USERNAME\AppData\Roaming'，以及 Windows 9X/ME 上的 'C:\WINDOWS\Application Data' 或 'C:\WINDOWS\Profiles\USERNAME\Application Data'。如果该目录不存在或无法访问，Emacs 会退回到把 'C:\' 作为 'HOME' 的默认值。

你可以通过显式设置环境变量 'HOME' 指向系统上的任意目录，来覆盖 'HOME' 的这个默认值。'HOME' 既可以在命令 shell 提示符下设置，也可以在"我的电脑"的 'Properties' 对话框中设置。'HOME' 还可以在系统注册表中设置，参见"MS-Windows 系统注册表"。

为了与旧版 Emacs(1) 兼容，如果 'C:\'（驱动器 'C:' 的根目录）下存在名为 '.emacs' 的文件，而 'HOME' 既未在环境中设置也未在注册表中设置，Emacs 就会把 'C:\' 当作默认的 'HOME' 位置，而不会去应用程序数据目录中查找，即使该目录存在。注意，只在 'C:\' 中查找 '.emacs'；不会查找旧名称 '_emacs'（见下文）。用 'C:\.emacs' 来定义 'HOME' 的这种用法已被废弃；Emacs 会在启动时显示有关它的警告。

无论最终位置是哪里，Emacs 都会把 'HOME' 环境变量的内部值设为指向该位置，并将该位置用于它通常在你的主目录中查找或创建的其他文件和目录。

你随时可以输入 'C-x d ~/ <RET>' 来查明 Emacs 认为你的主目录在哪里。这应会列出主目录中的文件，并在第一行显示其完整名称。同样，要访问你的初始化文件，输入 'C-x C-f ~/.emacs <RET>'（假设文件名是 '.emacs'）。

你的初始化文件可以使用 参见"Emacs 初始化文件" 中提到的任何名称。

因为 MS-DOS 不允许以点开头的文件名，而较旧的 Windows 系统也难以创建这类名称的文件，所以 Emacs 的 Windows 移植版支持初始化文件名 '_emacs'——前提是主目录中存在该文件而 '.emacs' 不存在。这个名称被视为过时，因此使用它时 Emacs 会显示警告。

> 注：旧版 Emacs 不检查应用程序数据目录。

I.6 MS-Windows 上的键盘用法

本节介绍 Emacs 中与键盘输入相关的 Windows 特有特性。

许多在 MS-Windows 程序中有传统用途的键组合（称为"键盘快捷键"）与传统 Emacs 键绑定冲突。（这些 Emacs 键绑定早在 Microsoft 成立之前多年就已确立。）冲突的例子包括 'C-c'、'C-x'、'C-z' 和 'C-a'。你可以启用 CUA 模式（参见"CUA 绑定"）来重新定义其中一些键，使其含义更接近 MS-Windows 的含义。另一个能让 Emacs 表现得像其他 Windows 应用程序的可选特性是 Delete Selection 模式（参见"对区域进行操作"）。

默认情况下，标有 <Alt> 的键被映射为 <Meta> 键。如果你希望它产生 'Alt' 修饰符，请把变量 'w32-alt-is-meta' 设为 'nil' 值。

MS-Windows 保留某些键组合供自己使用，例如 '<Alt>-<TAB>' 以及若干 Windows 键组合。这些键组合在 Emacs 看到它们之前就被系统拦截了。此外，在 Windows 10 上，所有 Windows 键组合都被系统保留，以至于它们永远不会传递给应用程序，即使系统当前并未在该特定组合上定义热键。你可以使用 'w32-register-hot-key' 函数让某个键序列由 Emacs 看到，而不是被 Windows 抢走。注册为热键后，该键组合在被 Windows 处理之前就从系统输入队列中取出，从而有效地覆盖该键序列对 Windows 的特殊含义。这种覆盖只在 Emacs 处于活动状态时有效；当其他应用程序位于前台时，这些键表现正常。

'w32-register-hot-key' 的参数必须是带单个修饰符的单个键，采用 'define-key' 可接受的向量形式。control 和 shift 修饰符对该参数没有影响。如果 'w32-alt-is-meta' 为 't'（默认值），meta 修饰符被解释为 <Alt> 键；而 super 和 hyper 修饰符按照 'w32-lwindow-modifier' 和 'w32-rwindow-modifier' 的绑定来解释。此外，带尾部连字符但不带键的修饰符表示：该修饰符上所有 Windows 定义的热键都要被覆盖，让给 Emacs。

例如，'(w32-register-hot-key [M-tab])' 让你可以在 Emacs 中正常使用 'M-<TAB>'；比如在顶层补全光标点处的单词或符号，或在增量搜索中补全当前搜索串（对照之前搜索过的字符串）。在 'w32-lwindow-modifier' 绑定

为‘super’的情况下，‘(w32-register-hot-key [s-])’会禁用所有 Windows 自带的基于 Windows 键的快捷键。(1)

注意，‘w32-register-hot-key’是在函数调用时检查‘w32-[lr]window-modifier’的值的。因此，你可以先把‘w32-lwindow-modifier’设为‘super’，再调用‘(w32-register-hot-key [s-r])’，最后把‘w32-rwindow-modifier’也设为‘super’。结果就是：左 Windows 键加‘R’会调用你在 Emacs 中为该组合绑定的函数，而右 Windows 键加‘R’会打开 Windows 的“运行”对话框。

热键注册总是同时包含给定热键的所有 shift 和 control 修饰键组合；也就是说，把‘s-a’注册为热键，也就同时得到‘S-s-a’、‘C-s-a’和‘C-S-s-a’。

在 Windows 98 和 ME 上，热键注册受到更多限制。所需的热键必须始终完整指定，而‘w32-phantom-key-code’可以被定制，以达到预期效果。

函数‘w32-unregister-hot-key’对其参数键序列撤销‘w32-register-hot-key’的效果。

默认情况下，<CapsLock> 键只影响普通字符键（它把小写字符转换为对应的大写形式）。但是，如果把变量‘w32-capslock-is-shiftlock’设为非‘nil’值，<CapsLock> 键也会影响非字符键，就像输入非字符键时按下了 <SHIFT> 键一样。

如果把变量‘w32-enable-caps-lock’设为‘nil’值，<CapsLock> 键会产生符号‘capslock’，而不是所键入按键的 shift 版本。默认值是‘t’。

类似地，如果‘w32-enable-num-lock’为‘nil’，<NumLock> 键会产生符号‘kp-numlock’。默认值是‘t’，这使 <NumLock> 按预期工作：切换数字小键盘上各键的含义。

变量‘w32-apps-modifier’控制 <Apps> 键（通常位于右 <Alt> 和右 <Ctrl> 键之间）的作用。其值可以是符号‘hyper’、‘super’、‘meta’、‘alt’、‘control’或‘shift’之一，以表示相应的修饰键；也可以是‘nil’，表示该键表现为‘apps’键。默认值是‘nil’。

变量‘w32-lwindow-modifier’决定左 Windows 键（通常标有 <start> 和 Windows 徽标）的作用。如果其值为‘nil’（默认值），该键会产生符号‘lwindow’。把它设为符号‘hyper’、‘super’、‘meta’、‘alt’、‘control’或‘shift’之一，就会产生相应的修饰键。类似的变量‘w32-rwindow-modifier’控制右 Windows 键的作用，而‘w32-scroll-lock-modifier’对 <ScrLock> 键做同样的事。如果这些变量被设为‘nil’，右 Windows 键产生符号‘rwindow’，<ScrLock> 产生符号‘scroll’。如果你希望 <ScrLock> 产生与其他应用程序相同的效果，即切换键盘上的 Scroll Lock LED 指示，就把‘w32-scroll-lock-modifier’设为‘t’或上述修饰键符号之外的任何非‘nil’值。

编译为原生 Windows 应用程序的 Emacs 通常会关闭 Windows 的“轻敲 <Alt> 键即调用 Windows 菜单”这一功能。原因是 <Alt> 在 Emacs 中充当 <Meta>。使用 Emacs 时，用户常常临时按下 <Meta> 键后又改变主意；如果这会产生弹出 Windows 菜单的效果，就会改变后续命令的含义。许多用户觉得这很令人沮丧。

把‘w32-pass-alt-to-system’设为非‘nil’值，即可重新启用 Windows 对轻敲 <Alt> 键的默认处理。

变量‘w32-pass-lwindow-to-system’和‘w32-pass-rwindow-to-system’决定相应的键是传递给 Windows 还是被 Emacs 吞掉。如果值为‘nil’，相应的键会被 Emacs 静默吞掉，否则就传递给 Windows。这两个变量的默认值都是‘t’。把每个键传递给 Windows 会产生其正常效果：例如‘<Lwindow>’打开‘开始’菜单，等等。

变量‘w32-recognize-altgr’控制 <AltGr> 键（如果你的键盘上有的话），或与之等价的右 <Alt> 与左 <Ctrl> 键组合，是否被识别为 <AltGr> 键。默认值是‘t’，意味着这些键产生‘AltGr’；把它设为‘nil’会使 <AltGr> 或等

价的键组合被解释为 <Ctrl> 和 <Meta> 修饰键的组合。

某些版本的 MS-Windows（通常是东亚本地化版本）会启用输入法管理器（Input Method Manager，IMM），它允许应用程序与输入法编辑器（Input Method Editor，IME，Windows 原生的输入法服务）通信。Emacs 在可用时使用 IME，让用户能够输入东亚的非 ASCII 字符，类似于 Emacs 内置的输入法（参见“输入法”）。然而，在某些情况下，如果 IME 把你输入的简单 ASCII 键解释为指定非 ASCII 字符的键序列的一部分，它就会造成妨碍。使用‘w32-set-ime-open-status’函数可以暂时关闭 IME，然后再重新打开。

> 注：有一个已知的例外：用于锁定工作站的‘<Windows>-L’组合由系统在更底层处理。因此，‘w32-register-hot-key’无法覆盖这个键组合——它总是会锁定计算机。

I.7 MS-Windows 上的鼠标使用

本节介绍与鼠标相关的 Windows 特有变量。

变量‘w32-mouse-button-tolerance’指定在双键鼠标上伪造中键按下所用的时间间隔（以毫秒为单位）。如果在此时间间隔内同时按下两个鼠标键，Emacs 会产生一个中键单击事件，而不是其中一个键的双击。

如果变量‘w32-pass-extra-mouse-buttons-to-system’为非‘nil’，Emacs 会把第四和第五个鼠标键传递给 Windows。

变量‘w32-swap-mouse-buttons’控制 3 个鼠标键中哪一个产生‘mouse-2’事件。当它为‘nil’（默认值）时，中间键产生‘mouse-2’事件，右键产生‘mouse-3’事件。如果此变量为非‘nil’，这两个鼠标键的角色互换。

I.8 Windows 9X/ME 和 Windows NT/2K/XP/Vista/7/8/10 上的子进程

编译为原生 Windows 应用程序的 Emacs（相对于 DOS 版本）包含对异步子进程的完整支持。在 Windows 版本中，只要只运行 32 位或 64 位 Windows 应用程序，同步和异步子进程在所有版本的 MS-Windows 上都能正常工作。然而，当你在子进程中运行 DOS 应用程序时，可能会遇到问题，或者根本无法运行该程序；而如果在两个子进程中同时运行两个 DOS 应用程序，你可能不得不重启系统。

由于 Windows 9X 上的标准命令解释器（以及大多数命令行工具）都是 DOS 应用程序，使用该系统时这些问题很严重。但我们对此无能为力；只有 Microsoft 能解决它们。

如果只运行一个 DOS 应用程序子进程，只要它“行为良好”，不直接访问屏幕或做其他不寻常的操作，该子进程应当能按预期工作。如果你有 CPU 监视器程序，即使 DOS 应用程序空闲，你的机器也会显得 100% 忙，但这只是 CPU 监视器测量处理器负载的方式造成的假象。

在另一个子进程中启动其他 DOS 应用程序之前，必须先终止当前的 DOS 应用程序。Emacs 无法中断或终止 DOS 子进程。终止这种子进程的唯一办法是给它一个命令，让它的程序退出。

如果你试图在两个独立子进程中同时运行两个 DOS 应用程序，后启动的那个会被挂起，直到第一个结束，即使其中任一个或两个都是异步的。

如果你能切换到第一个子进程并让它退出，第二个子进程就应能正常继续。然而，如果第二个子进程是同步的，Emacs 本身会一直挂起，直到第一个子进程结束。如果它没有用户输入就不会结束，那么在 Windows 9X

上运行时你就只能重启。如果在 Windows NT 及更高版本上运行，你可以改用进程查看器程序杀掉相应的 NTVDM 实例（这会终止两个 DOS 子进程）。

如果在这种情况下必须重启 Windows 9X，不要使用‘开始’菜单上的‘关机’命令；那通常会使系统挂起。而应输入 ‘<Ctrl>-<Alt>-<DEL>’，然后选择‘关机’。这通常有效，尽管可能要花几分钟才能完成。

变量 ‘w32-quote-process-args’ 控制 Emacs 如何引用进程参数。非 ‘nil’ 表示用 ‘"’ 字符来引用。如果该值是一个字符，Emacs 就用该字符转义出现的任何引号字符；否则它会根据程序的类型选择合适的转义字符。

变量 ‘w32-pipe-buffer-size’ 控制 Emacs 为与子进程通信而创建管道时向系统请求的缓冲区大小。默认值为零，表示让操作系统选择大小。任何有效的正值都会请求该字节大小的缓冲区。对于在缓冲管道 I/O 方面表现异常的程序，可以用它来调整与子进程的通信。

如果需要把 MS-DOS 程序作为 Emacs 子进程调用，你可能会发现从这类程序读取数据的速率很低。在这些情况下，把变量 ‘w32-pipe-read-delay’ 设为非零值可以提高吞吐量；我们建议这种情况下使用值 50。默认值为零。

函数 ‘w32-shell-execute’ 在编写定制命令时很有用，这类命令运行那些已注册为对特定类型的文档或文件处理某种标准 Windows 操作的 MS-Windows 应用程序。该函数是 Windows ‘ShellExecute’ API 的封装。更多细节参见 MS-Windows API 文档。

I.9 打印与 MS-Windows

如果没有 POSIX 风格的 ‘lpr’ 程序，打印命令（如 ‘lpr-buffer’（参见“打印”）和 ‘ps-print-buffer’（参见“PostScript 硬拷贝”））在 MS-DOS 和 MS-Windows 上通过把输出发送到某个打印机端口来工作。所有系统上的打印都由同样的 Emacs 变量控制，但在某些情况下，这些变量在 MS-DOS 和 MS-Windows 上的默认值不同。

MS Windows 上的 Emacs 会尝试自动确定你的默认打印机（使用函数 ‘default-printer-name’）。但在少数情况下这可能失败，或者你可能希望在 Emacs 内使用另一台打印机。本节余下部分说明如何告诉 Emacs 使用哪台打印机。

如果要使用本地打印机，就把 Lisp 变量 ‘lpr-command’ 设为 ‘""’（它在 Windows 上的默认值），并把 ‘printer-name’ 设为打印机端口的名称——例如 ‘"PRN"’（常见的本地打印机端口），或 ‘"LPT2"’，串行打印机则用 ‘"COM1"’。也可以把 ‘printer-name’ 设为文件名，这种情况下“打印”输出实际上会追加到该文件。如果把 ‘printer-name’ 设为 ‘"NUL"’，打印输出会被静默丢弃（送到系统空设备）。

也可以使用另一台机器共享的打印机，方法是把 ‘printer-name’ 设为该打印机的 UNC 共享名——例如 ‘"//joes_pc/hp4si"’。（这里用正斜杠还是反斜杠都无所谓。）要查明共享打印机的名称，可在命令提示符下运行 ‘net view’ 命令获得服务器列表，并用 ‘net view SERVER-NAME’ 查看该服务器共享的打印机（和目录）的名称。或者，单击桌面上的‘网上邻居’图标，查找通过网络共享打印机的机器。

如果打印机没有出现在 ‘net view’ 的输出中，或者把 ‘printer-name’ 设为 UNC 共享名后无法在该打印机上打印出硬拷贝，你可以用 ‘net use’ 命令把本地打印端口（如 ‘"LPT2"’）连接到网络打印机。例如，输入 ‘net use LPT2: \\joes_pc\hp4si’(1) 会让 Windows“捕获” ‘LPT2’ 端口，并把打印内容重定向到连接在机器 ‘joes_pc’ 上的打印机。执行该命令后，把 ‘printer-name’ 设为 ‘"LPT2"’ 应能在网络打印机上打印出硬拷贝。

对于某些类型的 Windows 网络软件，你可以不用 'net use'，而是通过'控制面板->打印机'小程序让 Windows 捕获特定打印机端口（如 '"LPT2"'）并将其重定向到网络打印机。

如果把 'printer-name' 设为文件名，最好使用绝对文件名。Emacs 会根据当前缓冲区的默认目录改变工作目录，因此如果 'printer-name' 中的文件名是相对路径，你最终会得到好几个这样的文件，每个都在进行打印的那个缓冲区所在目录中。

如果 'printer-name' 的值正确，但打印后打印机没有输出硬拷贝，可能是你的打印机不支持打印纯文本（一些廉价打印机会省略此功能）。这种情况下，可以试试下面介绍的 PostScript 打印命令。

命令 'print-buffer' 和 'print-region' 会调用 'pr' 程序，或使用 'lpr' 程序的特殊开关，在每个打印页上生成页眉。MS-DOS 和 MS-Windows 通常没有这些程序，因此默认情况下，变量 'lpr-headers-switches' 的设置使打印页眉的请求被静默忽略。于是 'print-buffer' 和 'print-region' 产生的输出分别与 'lpr-buffer' 和 'lpr-region' 相同。如果你确实有合适的 'pr' 程序（例如来自 GNU Coreutils），就把 'lpr-headers-switches' 设为 'nil'；Emacs 随后会调用 'pr' 生成页眉，并按 'printer-name' 指定的方式打印结果输出。

最后，如果你确实有类似 'lpr' 的程序，可以把变量 'lpr-command' 设为 '"lpr"'。这样 Emacs 就会像在其他系统上一样使用 'lpr' 打印。（如果该程序的名字不是 'lpr'，把 'lpr-command' 设为适当的值。）当 'lpr-command' 不是 '""' 时，变量 'lpr-switches' 具有其标准含义。如果变量 'printer-name' 的值是字符串，它会被用作 'lpr' 的 '-P' 选项的值，与 Unix 上相同。

另一组并行的变量 'ps-lpr-command'、'ps-lpr-switches' 和 'ps-printer-name'（参见"PostScript 硬拷贝的变量"）定义了应如何打印 PostScript 文件。这些变量的用法与上面介绍的非 PostScript 打印对应变量相同。因此，'ps-printer-name' 的值被用作 PostScript 输出送往的设备（或文件）名，正如 'printer-name' 用于非 PostScript 打印那样。（之所以有两组不同的变量，是因为你可能有两台打印机接在两个不同的端口上，而其中只有一台是 PostScript 打印机。）

变量 'ps-lpr-command' 的默认值是 '""'，这会使 PostScript 输出被送到 'ps-printer-name' 指定的打印机端口；但 'ps-lpr-command' 也可以设为一个能接受 PostScript 文件的程序名。因此，如果你有一台非 PostScript 打印机，可以把该变量设为 PostScript 解释器程序（如 Ghostscript）的名字。需要传给解释器程序的开关用 'ps-lpr-switches' 指定。（如果 'ps-printer-name' 的值是字符串，它将被加入开关列表，作为 '-P' 选项的值。这大概只在使用 'lpr' 时有用，所以使用解释器时通常会把 'ps-printer-name' 设为字符串以外的值，以便它被忽略。）

例如，要用 Ghostscript 在系统的默认打印机上打印，请把下面内容放进你的 '.emacs' 文件：

```
(setq ps-printer-name t)
(setq ps-lpr-command "D:/gs6.01/bin/gswin32c.exe")
(setq ps-lpr-switches '("-q" "-dNOPAUSE" "-dBATCH"
                        "-sDEVICE=mswinpr2"
                        "-sPAPERSIZE=a4"))
```

（这假定 Ghostscript 安装在 'D:/gs6.01' 目录中。）

> 注：'net use' 命令要求 UNC 共享名使用 Windows 风格的反斜杠，而 'printer-name' 的值可以用正斜杠或反斜杠设置。

I.10 在 MS-Windows 上指定字体

字体由名称、大小和可选属性来指定。指定字体的格式来自现代自由桌面所用的 fontconfig 库：

```
[Family[-PointSize]][:Option1=Value1[:Option2=Value2[...]]]
```

出于向后兼容，也支持旧的基于 XLFD 的格式。

MS-Windows 上的 Emacs 支持若干字体后端。目前可用的后端有 'gdi'、'uniscribe' 和 'harfbuzz'。'gdi' 字体后端在所有版本的 Windows 上都可用，支持 Windows 原生支持的所有字体。'uniscribe' 字体后端在 Windows 2000 及更高版本上可用，支持 TrueType 和 OpenType 字体。'harfbuzz' 字体后端在 Emacs 编译时带有 HarfBuzz 支持、且你的系统上安装了 HarfBuzz DLL 时可用；与 'uniscribe' 一样，该后端只支持 TrueType 和 OpenType 字体。某些需要复杂布局的语言只能由 Uniscribe 或 HarfBuzz 后端正确支持。默认情况下，每个框架启用两个后端：'gdi' 以及 'harfbuzz' 或 'uniscribe' 之一（取决于哪个可用；如果两者都可用，默认只启用 'harfbuzz'）。当 Emacs 查找合适的字体时，'harfbuzz' 和 'uniscribe' 后端优先于 'gdi'。要覆盖这一点，即使 Uniscribe 可用也使用 GDI 后端，可以用 '-xrm Emacs.fontBackend:gdi' 命令行参数调用 Emacs，或者在注册表中 'HKEY_CURRENT_USER\SOFTWARE\GNU\Emacs' 或 'HKEY_LOCAL_MACHINE\SOFTWARE\GNU\Emacs' 键下添加值为 'gdi' 的 'Emacs.fontBackend' 资源（参见 "X 资源"）。类似地，要即使 HarfBuzz 可用也使用 Uniscribe 后端，就在调用 Emacs 的命令行上使用 '-xrm Emacs.fontBackend:uniscribe'。你也可以通过 'font-backend' 框架参数请求全部 3 个后端，但要注意，这种情况下，对系统上没有可用字体的字符进行字体查找会花更长时间。

另外，你也可以通过 'font-backend' 框架参数为某个框架指定字体后端，方法是使用 'modify-frame-parameters'（参见《Emacs Lisp 参考手册》的"Parameter Access"）。你也可以通过 'default-frame-alist' 和 'initial-frame-alist' 为所有框架请求特定的字体后端（参见"框架参数"）。注意 'font-backend' 参数的值应当是一个符号列表，如 '(uniscribe)' 或 '(harfbuzz uniscribe gdi)'。

MS-Windows 上支持的可选字体属性有：

'weight' 指定字体的粗细。特殊值 'light'、'medium'、'demibold'、'bold' 和 'black' 可以不写 'weight=' 直接指定（例如 'Courier New-12:bold'）。否则，粗细应为 100 到 900 之间的数值，或 'font-weight-table' 中某个有名称的粗细值。若未指定，则假定为常规字体。

'slant' 指定字体是否为斜体。特殊值 'roman'、'italic' 和 'oblique' 可以不写 'slant=' 直接指定（例如 'Courier New-12:italic'）。否则，倾斜度应为数值，或 'font-slant-table' 中某个有名称的倾斜值。在 Windows 上，大于 150 的倾斜度被视为斜体，低于 150 的则视为正体。

'family' 指定字体系列，但通常会在字体名称开头指定。

'pixelsize' 以像素为单位指定字体大小。可以用它替代在系列名之后指定的磅值大小。

'adstyle' 指定字体的附加样式信息。在 MS-Windows 上，可识别 'mono'、'sans'、'serif'、'script' 和 'decorative' 这些值。在字体系列未指定时把它们用作后备最为有用。

'registry' 指定字体预期覆盖的字符集注册表。大多数 TrueType 和 OpenType 字体是覆盖多个国家字符集的 Unicode 字体，但你可以在这里使用 'w32-charset-info-alist' 中的某个特定注册表，把字体选择范围缩小到支持特定字符集的那些字体。

‘spacing’ 指定字体的间距方式。‘p’ 间距表示比例字体，‘m’ 或 ‘c’ 表示等宽字体。

‘foundry’ 在 Windows 上不使用，但为了提供信息、并防止那些期望它被设置的代码出问题，它在内部会被设为 ‘raster’（位图字体）、‘outline’（可缩放字体），或者如果无法确定类型是这两者之一，则设为 ‘unknown’。

‘script’ 指定字体应支持的 Unicode 子范围。

Emacs 已知的所有文字系统（script，通常指最新 Unicode 标准定义的所有文字系统）在 MS-Windows 上都能识别。然而，‘GDI’ 字体只支持已知文字系统的一个子集：‘greek’、‘hangul’、‘kana’、‘kanbun’、‘bopomofo’、‘tibetan’、‘yi’、‘mongolian’、‘hebrew’、‘arabic’ 和 ‘thai’。

‘antialias’ 指定抗锯齿方法。值 ‘none’ 表示不使用抗锯齿，‘standard’ 表示使用标准抗锯齿，‘subpixel’ 表示使用次像素抗锯齿（在 Windows 上称为“Cleartype”），‘natural’ 表示使用调整了字母间距的次像素抗锯齿。若未指定，字体会使用系统默认的抗锯齿方式。

MS-Windows 上 Emacs 用于查找适合显示给定非 ASCII 字符的字体的方法，对某些罕见文字系统（特别是 Unicode 相对较近才添加的那些）可能会失败，即使你的系统上安装了支持这些文字系统的字体。这是因为在 MS-Windows 上 Emacs 用来查找字体的信息中，这些文字系统没有定义 Unicode 子范围位（Unicode Subrange Bits，USB）。你可以用 ‘w32-find-non-USB-fonts’ 函数克服这些问题。它需要在 Emacs 会话开始时运行一次，安装了新字体后还需再次运行。可以把下面一行加入初始化文件，让该函数在每次启动 Emacs 时运行：

```
(w32-find-non-USB-fonts)
```

另外，你也可以随时通过 ‘M-:’ 手动运行该函数（参见“求值 Emacs Lisp 表达式”）。在安装了许多字体的系统上，运行 ‘w32-find-non-USB-fonts’ 可能要花几秒钟；如果你认为启动时运行它太慢，而且你很少安装新字体，可以通过 ‘M-:’ 运行该函数一次，然后把它返回的非 ‘nil’ 值赋给初始化文件中的变量 ‘w32-non-USB-fonts’。（如果该函数返回 ‘nil’，说明你安装的字体都不能显示需要此机制的文字系统的字符。）

变量 ‘w32-use-w32-font-dialog’ 控制通过 ‘S-mouse-1’（‘mouse-appearance-menu’）选择字体的方式。如果值为 ‘t’（默认值），Emacs 使用标准的 Windows 字体选择对话框。如果值为 ‘nil’，Emacs 改为弹出一个包含一组固定字体的菜单。菜单中出现的字体由 ‘w32-fixed-font-alist’ 决定。

I.11 其他 Windows 特有功能

本节介绍其他章节容纳不下的 Windows 特有功能。

变量 ‘w32-use-visible-system-caret’ 是一个标志，决定是否让系统光标（caret）可见。未使用屏幕阅读器软件时默认值为 ‘nil’，表示 Emacs 自己绘制光标来指示光标点位置。非 ‘nil’ 值表示 Emacs 用系统光标指示光标点位置；这便于使用屏幕阅读器软件，也是在 Emacs 运行时检测到此类软件时的默认值。当该变量为非 ‘nil’ 时，其他影响光标显示的变量都不起作用。

变量 ‘w32-grab-focus-on-raise’ 若设为非 ‘nil’ 值，则框架被提升（raise）时会获取焦点。默认值为 ‘t’，这与 Windows 默认的“点击获取焦点”策略相符。

在 Windows 10（版本 1809 及更高）和 Windows 11 上，Emacs 的标题栏和滚动条默认跟随系统的浅色或深色模式，与资源管理器、命令提示符等其他程序类似。要更改颜色模式，请从 ‘Windows Settings’ 中选择

'Personalization'，然后选择 'Colors->Choose your color'（或 'Choose your default app mode'、'Choose your mode'）；之后重启 Emacs。在 Windows 11 上，可以为 Windows 和应用程序分别选择默认模式。

如果不想让 Emacs 跟随系统的深色模式设置，可把变量 'w32-follow-system-dark-mode' 自定义为 'nil' 值；这样无论系统范围的设置如何，Emacs 都会使用默认的浅色模式。更改该变量的值只影响更改之后创建的 Emacs 框架，因此应在初始化文件中设置它的值（参见"Emacs 初始化文件"），可以直接设置，也可以通过 'M-x customize-variable' 设置，后者可保存自定义的值，参见"保存自定义"。

Windows 控制台的显示功能实现与其他终端模拟器的实现不同，因为历史上 Windows 要求使用一套独特的 API。那套 API 把 Emacs 在 Windows 控制台上的显示限制为 16 种基本颜色。随着 Windows Terminal 的引入，Microsoft 实现了对仿照 VT100 和 Xterm 的 ANSI 控制序列的支持，以及对 24 位 RGB 颜色显示的支持。

函数 'w32-use-virtual-terminal' 和 'w32-use-virtual-terminal-p' 可分别用于设置和查看一个内部变量，该变量决定显示是使用这套较新的机制还是较旧的机制。启动时，该内部变量会根据终端的能力自动设置。默认使用 24 位 RGB 颜色，但也可以使用其他（8、16、256）颜色空间，办法是传入 '--color' 命令行参数，或设置 'tty-color-mode' 框架参数的值。

'(w32-use-virtual-terminal-p)' 当且仅当内部变量为非零数值时求值为 't'，否则求值为 'nil'。若它求值为 't'，则颜色使用 ANSI 转义序列，否则使用较旧的机制。内部变量可通过求值 '(w32-use-virtual-terminal x)' 来设置，其中 'x' 为 't' 或 'nil'：若 'x' 为 't' 且终端支持该特性，则启用该特性；否则禁用该特性。

I.12 Emacs 与 MS-DOS

本节简要介绍在 MS-DOS 上使用 Emacs 的特殊之处。MS-DOS 与 Microsoft 当前操作系统 Windows 共有的特殊之处参见"附录 I Emacs 与 Microsoft Windows/MS-DOS"。

如果为 MS-DOS 构建 Emacs，得到的可执行文件也能作为 DOS 应用程序在 Windows 3.X、Windows NT、Windows 9X/ME 或 Windows 2000/XP 上运行；若使用为 MS-DOS 构建的 Emacs，本章全部内容适用于上述所有系统。

关于 Emacs 在 MS-DOS（及 Windows）下对文本文件的特殊处理，参见"文本文件与二进制文件"。

I.12.1 MS-DOS 上的键盘使用

在 Emacs 中称为 <DEL> 的键（因为大多数工作站上如此标记）在 PC 上称为 <BS>（退格）。这就是 PC 专用的终端初始化把 <BS> 键重映射为 <DEL> 的原因；出于同样原因，<Delete> 键被重映射为 'C-d'。

为 MS-DOS 构建的 Emacs 把 'C-<Break>' 识别为退出字符，就像 'C-g' 一样。这是因为 Emacs 在准备好接受更多输入之前无法检测到你输入了 'C-g'。因此，你不能用 'C-g' 停止正在运行的命令（参见"退出与中止"）。相反，'C-<Break>' 一按下就会被检测到（如同其他系统上的 'C-g'），因此可用于停止正在运行的命令以及紧急退出（参见"紧急退出"）。

PC 键盘映射把左侧 <Alt> 键用作 <Meta> 键。模拟 <SUPER> 和 <Hyper> 键有两种选择：分别把变量 'dos-hyper-key' 和 'dos-super-key' 设为 1 或 2，以选用右侧 <Ctrl> 键或右侧 <Alt> 键。若 'dos-super-key' 和 'dos-hyper-key' 都不为 1，则默认情况下右侧 <Alt> 键也映射为 <Meta> 键。然而，若安装了 MS-DOS 国际化键盘支持程序 'KEYB.COM'，Emacs 就不会把右侧 <Alt> 映射为 <Meta>，因为在非美式键盘布局中它用于输入 '~' 和 '@' 等字符；此时只能把左侧 <Alt> 用作 <Meta> 键。

变量 'dos-keypad-mode' 是一个标志变量，控制数字小键盘上的按键返回哪些键码。还可以把小键盘上的 <ENTER> 键定义为作用同 'C-j'，方法是将下面这行放入 '_emacs' 文件：

```
;; Make the <ENTER> key from the numeric keypad act as 'C-j'.
(keymap-set function-key-map "<kp-enter>" "C-j")
```

I.12.2 MS-DOS 上的鼠标使用

MS-DOS 上的 Emacs 支持鼠标（仅在默认终端上）。鼠标命令按文档所述工作，包括那些使用菜单和菜单栏的命令（参见"菜单栏"）。滚动条在 MS-DOS 版 Emacs 中不起作用。PC 鼠标通常只有两个按钮；它们分别对应 'mouse-1' 和 'mouse-2'，但同时按下两者会产生 'mouse-3' 的效果。若鼠标确实有 3 个按钮，Emacs 启动时会检测到，三个按钮都正常工作，如同在 X 上。

当鼠标指针移过菜单项时，菜单栏菜单和弹出菜单的帮助字符串显示在回显区。鼠标敏感文本的高亮（参见"用鼠标跟随引用"）也受支持。

某些版本的鼠标驱动程序不能正确报告鼠标按钮数。例如，带滚轮的鼠标报告自己有 3 个按钮，但只有其中 2 个传给 Emacs；滚轮（用作中间按钮）上的点击不会被传递。在这些情况下，可以用 'M-x msdos-set-mouse-buttons' 命令告诉 Emacs 应期望多少个鼠标按钮。可以把下面这段加入 '_emacs' 初始化文件，使该设置永久生效：

```
;; Treat the mouse like a 2-button mouse.
(msdos-set-mouse-buttons 2)
```

为 MS-DOS 构建的 Emacs 在 Windows 上运行时支持剪贴板操作。把文本放入删除环或从环中粘贴文本的命令会先检查 Windows 剪贴板，正如 Emacs 在 X Window System 上所做的那样（参见"鼠标编辑命令"）。在 Windows 上，MS-DOS 版 Emacs 只支持主选择（primary selection）和剪切缓冲区（cut buffer）；次选择（secondary selection）总是显示为空。

由于 Windows 实现剪贴板访问的方式，能放入剪贴板的文本长度受 Emacs 可用的 DOS 空闲内存量限制。通常最多可将 620KB 文本放入剪贴板，但该限制取决于系统配置，若把 Emacs 作为另一个程序的子进程运行，限制会更低。若删除的文本放不下，Emacs 会输出一条消息说明这一点，并且不把该文本放入剪贴板。

空字符也无法放入 Windows 剪贴板。若删除的文本包含空字符，Emacs 不会把这样的文本放入剪贴板，并在回显区显示相应消息。

变量 'dos-display-scancodes' 为非 'nil' 时，指示 Emacs 显示每次击键的 ASCII 值和键盘扫描码；该特性作为 'view-lossage' 命令的补充，用于调试。

I.12.3 MS-DOS 上的显示

MS-DOS 上的显示不能使用粗体、斜体等字体变体，但支持多种字体（face），每种字体可指定前景色和背景色。因此，通过为相关字体定义不同的颜色，就能获得使用字体的 Emacs 包（如 'font-lock'、Enriched Text 模式等）的全部功能。使用 'list-colors-display' 命令（参见"字体的颜色"）和 'list-faces-display' 命令（参见"文本字体"）查看有哪些颜色和字体可用以及它们的外观。

关于 Emacs 如何显示 DOS 显示硬件内置字体不支持的图形符号和字符，参见本章后面的参见“MS-DOS 上的国际化支持”。

Emacs 启动时把光标形状改为实心方块。这是为了与其他系统兼容，在其他系统中方块光标是 Emacs 的默认形状。可以通过在变量 ‘default-frame-alist’ 中指定 ‘cursor-type’ 参数，把这种默认形状改为条状（参见“创建框架”）。MS-DOS 终端不支持竖条光标，因此条状光标是水平的，而框架参数中若指定了 ‘WIDTH’ 参数，它实际决定的是光标高度。因此，‘bar’ 和 ‘hbar’ 两种光标类型在 MS-DOS 上效果相同。作为扩展，条状光标规格还可以包含光标的起始扫描行以及其宽度，如下所示：

```
'(cursor-type bar WIDTH . START)
```

此外，若 WIDTH 参数为负，光标条从字符单元的顶部开始。

MS-DOS 终端一次只能显示一个框架。Emacs 的框架功能在 MS-DOS 上的工作方式与在文本终端上大体相同（参见“框架与图形显示”）。在 MS-Windows 上从 DOS 窗口运行 Emacs 时，可以让可见框架小于整个屏幕，但 Emacs 仍然一次只能显示一个框架。

命令 ‘dos-mode4350’ 根据硬件把显示切换为 43 行或 50 行；命令 ‘dos-mode25’ 切换为默认的 80x25 屏幕尺寸。

默认情况下，Emacs 只知道如何设置 80 列、25、28、35、40、43 或 50 行的屏幕尺寸。然而，若你的显示适配器有可把显示切换到其他尺寸的特殊视频模式，也可以让 Emacs 支持它们。当你要求 Emacs 把框架切换为 N 行 M 列的尺寸时，它会检查是否存在名为 ‘screen-dimensions-NxM’ 的变量，若存在，就用其值（必须是整数）作为要切换到的视频模式。（Emacs 通过以 ‘AL’ 寄存器中的 ‘screen-dimensions-NxM’ 值调用 BIOS 的 ‘Set Video Mode’ 函数来切换到该视频模式。）例如，假设你的适配器在进入视频模式 85 时切换到 66x80 尺寸。那么可以把下面内容放入 ‘_emacs’ 文件，使 Emacs 支持该屏幕尺寸：

```
(setq screen-dimensions-66x80 85)
```

由于 MS-DOS 上的 Emacs 只能把框架尺寸设为特定的受支持尺寸，它无法满足所有可能的框架缩放请求。请求不支持的尺寸时，Emacs 会选择超过指定尺寸的下一个更大的受支持尺寸。例如，若请求 36x80 的框架，得到的会是 40x80。

变量 ‘screen-dimensions-NxM’ 仅在完全匹配指定尺寸时才使用；搜索下一个更大的受支持尺寸时会忽略它们。在上例中，即使你的 VGA 支持 38x80 尺寸并且你定义了取合适值的变量 ‘screen-dimensions-38x80’，请求 36x80 框架时仍会得到 40x80 屏幕。若在这种情况下想得到 38x80 尺寸，可以把名为 ‘screen-dimensions-36x80’ 的变量设为与 ‘screen-dimensions-38x80’ 相同的视频模式值。

在 MS-DOS 上更改框架尺寸会使所有其他框架也变为新尺寸。

I.12.4 MS-DOS 上的文件名

在 MS-DOS 上，文件名不区分大小写，且限制为八个字符，另外可选一个点和三个字符。Emacs 对这些限制了解得足够多，能处理原本面向其他操作系统的文件名。例如，文件名开头的点 ‘.’ 在 MS-DOS 中无效，因此 Emacs 会透明地把它转换为下划线 ‘_’；因此在 MS-DOS 上默认初始化文件（参见“Emacs 初始化文件”）名为 ‘_emacs’。点之前或之后多余的字符通常被 MS-DOS 本身忽略；因此若你访问文件

‘LongFileName.EvenLongerExtension’，会静默地得到‘longfile.eve’，但 Emacs 仍会在模式行上显示长文件名。除此之外，指定在 MS-DOS 下有效的文件名由你负责；上述透明转换只对 Emacs 内部构造的文件名有效。

上述对 MS-DOS 文件名的限制使得构造备份文件名（参见“单个备份与编号备份”）几乎不可能不丢失原文件名的某些字符。例如，即使只做单份备份，‘docs.txt’ 的备份名也是‘docs.tx~’。

若在 Windows 9X、Windows ME 或 Windows 2000/XP 下把 Emacs 作为 DOS 应用程序运行，可以开启对长文件名的支持。开启后，Emacs 不会截断文件名或把它们转换为小写；而是原样使用你指定的文件名。要启用长文件名支持，请在启动 Emacs 之前把环境变量‘LFN’设为‘y’。遗憾的是，Windows NT 不允许 DOS 程序访问长文件名，因此为 MS-DOS 构建的 Emacs 只能看到其 8+3 短别名。

MS-DOS 没有主目录的概念，因此 MS-DOS 上的 Emacs 假装其安装目录就是‘HOME’环境变量的值。也就是说，若 Emacs 可执行文件‘emacs.exe’位于目录‘c:/utils/emacs/bin’，Emacs 就当作‘HOME’被设为‘c:/utils/emacs’。特别是，Emacs 在该位置查找初始化文件‘_emacs’。明白了这一点，就可以在文件名中用‘~’作为主目录的别名，如同在 GNU 或 Unix 上那样。也可以在启动 Emacs 之前设置环境中的‘HOME’变量；其值将覆盖上述默认行为。

MS-DOS 上的 Emacs 对名称‘/dev’做特殊处理，因为 DJGPP 的模拟器库有一个特性，假装 I/O 设备在该目录下有名。我们建议避免在任何磁盘上使用名为‘/dev’的实际目录。

I.12.5 打印与 MS-DOS

在没有 POSIX 风格的‘lpr’程序时，‘lpr-buffer’（参见“打印”）和‘ps-print-buffer’（参见“PostScript 硬拷贝”）等打印命令可以通过把输出发送到某个打印机端口来在 MS-DOS 上工作。所有系统上控制打印的 Emacs 变量相同，但在 MS-DOS 上某些变量的默认值不同。

关于设置打印到联网打印机的细节，参见“打印与 MS-Windows”。

有些打印机期望非 ASCII 文本采用 DOS 代码页编码，即使它们连接的是对同一区域设置使用不同编码的 Windows 机器。例如，在 Latin-1 区域设置中，DOS 使用代码页 850，而 Windows 使用代码页 1252。参见“MS-DOS 上的国际化支持”。从 Windows 向这类打印机打印时，可以在‘M-x lpr-buffer’之前使用‘C-x <RET> c’（‘universal-coding-system-argument’）命令；Emacs 随后会把文本转换为你指定的 DOS 代码页。例如，‘C-x <RET> c cp850-dos <RET> M-x lpr-region <RET>’会把区域转换为代码页 850 编码后打印。

为向后兼容，在 MS-DOS 上，‘dos-printer’（‘dos-ps-printer’）若有值，则覆盖‘printer-name’（‘ps-printer-name’）的值。

I.12.6 MS-DOS 上的国际化支持

MS-DOS 上的 Emacs 支持的国际化字符集与 GNU、Unix 及其他平台相同（参见“国际字符集支持”），包括在不同字符集之间转换的编码系统。然而，由于 MS-DOS/MS-Windows 与其他系统之间的不兼容性，这一支持有几个 DOS 特有的方面需要注意。本节介绍这些方面。

下面的描述主要针对 Emacs 的 MS-DOS 移植版，尤其是涉及对 Emacs 用户的实际影响之处。

- ‘M-x dos-codepage-setup’按当前 DOS 代码页适当地设置 Emacs 的显示与编码系统。

MS-DOS 设计为在任一时刻支持一种 256 字符的字符集，但提供多种字符集供选择。这些备选字符集称为“DOS 代码页”（DOS codepage）。每个代码页都包含全部 128 个 ASCII 字符，但其余 128 个字符（编码 128 到 255）因代码页而异。每个 DOS 代码页用三位数字标识，如 850、862 等。

与可以同时使用多种字体的 X 不同，MS-DOS 通常不允许在单个会话中使用多个代码页。MS-DOS 设计为在系统启动时加载单个代码页，并要求重启才能更改它(1)。在其他系统（如 MS-Windows）上运行 DOS 可执行文件时，也有大致相同的限制。

为了在 MS-DOS 上进行多字节操作，Emacs 需要知道所选 DOS 代码页能显示哪些字符。因此它在启动后不久查询系统以获取所选代码页编号，并把该编号存入变量 ‘dos-codepage’。有些系统对当前代码页返回默认值 437，即使实际代码页不同。（这通常发生在使用显示硬件内置代码页的时候。）可以通过在初始化文件中设置变量 ‘dos-codepage’，为 Emacs 指定要使用的其他代码页。

多字节 Emacs 只支持某些 DOS 代码页：能显示远东文字（如日文代码页 932）的代码页，以及编码单个 ISO 8859 字符集的代码页。

远东代码页可以直接显示这些国家对应的某个 MULE 字符集，因此 Emacs 只需设置使用该代码页支持的相应终端编码系统。本节其余部分描述的特殊特性大多与编码 ISO 8859 字符集的代码页有关。

对于与某个 ISO 字符集对应的代码页，Emacs 根据代码页编号得知该字符集。Emacs 会自动创建一个编码系统，以支持读写使用当前代码页的文件，并默认使用该编码系统。该编码系统的名称是 ‘cpNNN’，其中 NNN 是代码页编号(2)。

所有 ‘cpNNN’ 编码系统都用字母 ‘D’（代表“DOS”）作为模式行助记符。由于终端编码系统和文件 I/O 的默认编码系统在启动时都设为适当的 ‘cpNNN’ 编码系统，MS-DOS 上的模式行以 ‘-DD\-’ 开头是正常的。参见“模式行”。远东 DOS 终端不使用 ‘cpNNN’ 编码系统，因此它们初始的模式行看起来与 Emacs 默认值相同。

由于代码页编号也指示你所使用的文字（script），Emacs 会自动运行 ‘set-language-environment’，为该文字选择语言环境（参见“语言环境”）。

若缓冲区含有属于某个其他 ISO 8859 字符集（而非所选 DOS 代码页支持的那个）的字符，Emacs 会用一串 ASCII 字符来显示它。例如，若当前代码页没有字母 ‘ò’（带重音符的小写 ‘o’）的字形，它就会显示为 ‘{`o}’，其中花括号用来直观表明这是一个字符。（对某些非拉丁字符，如希腊或希伯来字母表中的字符，这可能看起来有些别扭，但懂该语言的人仍能读懂。）即使该字符在屏幕上可能占据几列，它实际上仍只是一个字符，所有 Emacs 命令都把它当作一个字符。

MS-Windows 提供自己的代码页，同一区域设置下的这些代码页与 DOS 代码页不同。例如，DOS 代码页 850 与 Windows 代码页 1252 支持相同的字符集；DOS 代码页 855 与 Windows 代码页 1251 支持相同的字符集，等等。以 ‘-nw’ 选项调用时，MS-Windows 版 Emacs 使用当前代码页进行显示。

> 注：(1) 通常情况下，某个特定代码页被烧录进显示内存，而其他代码页可通过修改系统配置文件（如 ‘CONFIG.SYS’）并重启来安装。虽然有第三方软件可以在不重启的情况下更改代码页，但这里描述的是标准 MS-DOS 系统的行为。

> 注：(2) 用于 ISO 8859 的标准 Emacs 编码系统并不完全适用于此目的，因为 DOS 代码页通常与标准 ISO 字符编码不一致。例如，字母 ‘ç’（带软音符的 ‘c’）在标准 Latin-1 字符集中编码为 231，而相应的 DOS 代

> 码页 850 用 135 表示该字形。

I.12.7 MS-DOS 上的子进程

由于 MS-DOS 是单进程的“操作系统”，异步子进程不可用。特别是，Shell 模式及其变体不能工作。大多数使用异步子进程的 Emacs 功能在 MS-DOS 上也不能工作，包括 Shell 模式和 GUD。拿不准时，试一试便知；不能工作的命令会输出错误消息，说明不支持异步进程。

在 Emacs 中用 ‘M-x compile’ 编译、用 ‘M-x grep’ 搜索文件、用 ‘M-x diff’ 显示文件间差异，这些都能工作，办法是同步运行下级进程。这意味着在下级进程结束之前你无法继续做任何编辑。

拼写检查也可以工作，办法是对 ‘ispell’ 程序的同步调用提供特殊支持。这比其他平台上的异步调用慢。

Shell 模式在 MS-DOS 上不能工作，代替它可以使用 ‘M-x eshell’ 命令。该命令调用 Eshell 包，这个包完全用 Emacs Lisp 实现了一个类 POSIX 的 shell。

相反，编译为原生 Windows 应用程序的 Emacs 确实支持异步子进程。参见“Windows 9X/ME 和 Windows NT/2K/XP/Vista/7/8/10 上的子进程”。

‘lpr-buffer’（参见“打印”）和 ‘ps-print-buffer’（参见“PostScript 硬拷贝”）等打印命令在 MS-DOS 上通过把输出发送到某个打印机端口来工作。参见“打印与 MS-DOS”。

在 MS-DOS 上同步运行子进程时，要确保程序会终止，且不试图读取键盘输入。若程序不自行终止，你将无法终止它，因为 MS-DOS 没有终止进程的通用方法。在这些情况下，按 ‘C-c’ 或 ‘C-<Break>’ 有时会有帮助。

MS-DOS 不支持访问其他机器上的文件。其他面向网络的命令，例如发送邮件、浏览 Web、远程登录等，也都不能使用，除非通过某个网络重定向器把网络访问内建到 MS-DOS 中。

MS-DOS 上的 Dired 使用 ‘ls-lisp’ 包（参见“在 MS-Windows 上模拟 ‘ls’”）。因此，MS-DOS 上的 Dired 只支持 ‘dired-listing-switches’ 变量中可以指定的部分选项。可用的选项是 ‘-A’、‘-a’、‘-c’、‘-i’、‘-r’、‘-S’、‘-s’、‘-t’ 和 ‘-u’。

GNU 宣言

下面这份《GNU 宣言》是 Richard Stallman 在 GNU 工程初创时写下的，用以请求人们参与和支持。最初几年里，它为了反映各项进展而做过些微修订，但现在看来最好保持原样，因为大多数人看到的就是这个版本。

自那时以来，我们了解到一些常见的误解，换一种措辞本可有助于避免它们。1993 年增补的脚注有助于澄清这些要点。

关于可获得的 GNU 软件的最新信息，请访问我们的网站 <https://www.gnu.org>。关于软件任务及其他贡献方式，参见 <https://www.gnu.org/help>。

GNU 是什么？Gnu's Not Unix!

GNU 是 Gnu's Not Unix 的缩写，是我正在编写的完整 Unix 兼容软件系统的名字，为的是能把它免费赠送给所有能使用它的人。⑴ 另有几位志愿者在帮助我。我们非常需要时间、金钱、程序和设备方面的贡献。

到目前为止，我们已有带 Lisp（用于编写编辑器命令）的 Emacs 文本编辑器、源代码级调试器、与 yacc 兼容的语法分析器生成器、链接器，以及约 35 个实用程序。shell（命令解释器）已接近完成。一个新的可移植优化 C 编译器已能编译自身，并可能于今年发布。初始内核已经存在，但要仿真 Unix 还需要多得多的特性。等内核和编译器完成后，就有可能发行适合程序开发的 GNU 系统。我们将使用 TeX 作为文本格式化程序，不过也在开发一个 nroff。我们还将使用免费、可移植的 X 窗口系统。此后我们会加入可移植的 Common Lisp、一个 Empire 游戏、一个电子表格，以及数百个其他东西，还有在线文档。我们希望最终提供通常随 Unix 系统而来的一切有用之物，而且更多。

GNU 将能运行 Unix 程序，但不会与 Unix 完全相同。我们将根据自己的其他操作系统使用经验，做出一切便利的改进。具体地说，我们计划采用更长的文件名、文件版本号、防崩溃的文件系统、或许还有文件名补全、与终端无关的显示支持，并且也许最终实现一个基于 Lisp 的窗口系统，让若干个 Lisp 程序和普通的 Unix 程序共用一块屏幕。C 和 Lisp 都将作为系统编程语言提供。我们将尝试支持 UUCP、MIT Chaosnet 和 Internet 协议用于通信。

GNU 最初面向具有虚拟内存的 68000/16000 级机器，因为让它在这些机器上运行最容易。为让它在更小的机器上运行而付出的额外努力，将留给想在那些机器上使用它的人。

为避免可怕的混淆，当指本工程时，请把“GNU”一词中的“G”发出来。

我为什么必须编写 GNU

我认为，黄金法则要求：如果我喜欢某个程序，就必须与同样喜欢它的其他人分享。软件销售商想分化用户并征服他们，让每个用户同意不与他人分享。我拒绝以这种方式破坏与其他用户的团结。我无法凭良心签署保密协议或软件许可协议。多年来我在人工智能实验室工作，以抵制这类倾向和其他不友善的做法，但最终他们走得太远：我无法留在一个别人违背我的意愿替我干这种事的机构里。

为了能继续有尊严地使用计算机，我决定汇集足够多的自由软件，使我在没有任何非自由软件的情况下也能过得下去。我已从 AI 实验室辞职，使 MIT 没有任何法律借口阻止我把 GNU 送出去。

GNU 为何与 Unix 兼容

Unix 不是我的理想系统，但并不太糟。Unix 的基本特性看来是好的，我认为我可以在不破坏它们的前提下填补 Unix 所缺的东西。而且，一个与 Unix 兼容的系统将便于许多其他人采用。

GNU 将如何发布

GNU 不属于公有领域。每个人都将被允许修改和再分发 GNU，但任何分发者都不得限制其进一步再分发。也就是说，不允许专有的修改。我要确保 GNU 的所有版本都保持自由。

为何许多其他程序员愿意帮忙

我发现许多其他程序员对 GNU 感到兴奋并愿意帮忙。

许多程序员对系统软件商业化感到不满。商业化也许能让他们赚更多的钱，却要求他们觉得自己与其他程序员处于冲突之中，而不是同志关系。程序员之间友谊的基本行为是共享程序；如今通常采用的营销安排实质上禁止程序员把别人当朋友。软件的购买者必须在友谊与守法之间做选择。自然，许多人认定友谊更重要。但那些信奉法律的人往往对两种选择都感到不安。他们变得愤世嫉俗，认为编程只是一种赚钱的手段。

通过编写和使用 GNU 而非专有程序，我们可以对每个人都友善，同时又遵守法律。此外，GNU 还是一个激励他人的榜样、一面召唤他人加入共享的旗帜。这能给我们一种和谐感，而使用非自由软件时这是不可能的。对于与我交谈过的约半数程序员来说，这是一种金钱无法替代的重要幸福。

你可以怎样贡献

我正在请求计算机制造商捐献机器和资金。我正在请求个人捐献程序和工作。

如果你捐献机器，可以预期的一个结果是 GNU 会早日运行在这些机器上。机器应是完整、可用的系统，获准在住宅区使用，且不需要复杂的冷却或供电条件。

我发现非常多的程序员渴望为 GNU 贡献部分时间的工作。对大多数项目来说，这种业余的分布式工作很难协调；独立编写的各部分无法协同工作。但对于替换 Unix 这一特定任务，这个问题并不存在。一个完整的 Unix 系统包含数百个实用程序，每个都单独编写文档。大多数接口规范都由 Unix 兼容性固定下来。如果每个贡献者都能为单个 Unix 实用程序写一个兼容的替代品，并让它在 Unix 系统上正确替代原有的程序，那么这些实用程序组合起来就能正常工作。即使让墨菲定律制造出几个意外问题，装配这些组件也将是一项可行的任务。（内核将需要更密切的沟通，并由一个紧密的小组来做。）

如果我得到资金捐献，也许能雇用几个全职或兼职的人。按程序员的标准，薪水不会高，但我在寻找的是那些认为建立社群精神与赚钱同样重要的人。我把这看作一种方式：让专心致志的人免受另谋生计之累，从而把全部精力投入到 GNU 工作中。

为何所有计算机用户都将受益

一旦 GNU 写成了，每个人都能像得到空气一样免费获得良好的系统软件。⑵

这意味着远不只是替每个人省下 Unix 许可证的费用。它意味着将避免系统编程工作中大量浪费的重复劳动。这些精力可以转而用于推进技术水平。

完整的系统源代码将对所有人可用。结果，需要改动系统的用户总能自己动手修改，或者雇任何找得到的程序员或公司来为他修改。用户将不再任由某个掌握源代码、居于唯一可修改地位的编程者或公司摆布。

学校将能通过鼓励所有学生学习和改进系统代码，提供远更有教育意义的环境。哈佛的计算机实验室曾有一项政策：如果某个程序的源代码没有公开展示，就不能安装在该系统上；他们甚至真的拒绝安装某些程序来坚持这一政策。我深受启发。

最后，考虑谁拥有系统软件、谁有权用它做什么事情，这种管理开销将被免除。

让人们为使用程序付费的各种安排，包括复制件的许可，总要借助繁琐的机制来确定一个人必须为多少（也就是哪些程序）付费，从而给社会带来巨大成本。而只有警察国家才能强迫人人遵守它们。设想一个必须花极大代价制造空气的空间站：按每升空气向每个呼吸者收费也许是公平的，但昼夜戴着计量面罩，即使人人都付得起空气账单，也是无法忍受的。而到处安装电视摄像机，看你是否摘下面罩，更是骇人听闻。更好的做法是用人头税来维持空气工厂，然后把面罩扔掉。

复制一个程序的全部或部分，对程序员来说就像呼吸一样自然，也一样富有成效。它理应同样自由。

对 GNU 目标的一些容易反驳的异议

“如果它是免费的，就没人会用，因为这意味着他们不能依赖任何支持。”

“你必须对程序收费，才能支付提供支持的费用。”

如果人们宁愿为 GNU 加服务付费，而不愿免费获得不带服务的 GNU，那么一家只向已免费获得 GNU 的人提供服务的公司应当是有利可图的。(3)

我们必须区分以真正编程工作形式提供的支持和单纯的扶助。前者是没法指望软件销售商提供的。如果你的问题没有足够多的人共同遇到，销售商会叫你别烦他。

如果你的业务需要能够依赖支持，唯一的办法是拥有全部必要的源代码和工具。这样你就可以雇任何找得到的人来解决你的问题；你不必受任何个人摆布。在 Unix 下，源代码的价格使大多数企业无法考虑这一点。在 GNU 下这将是容易的。仍然可能出现找不到合适能人的情况，但这个问题不能归咎于分发方式。GNU 并不消除世界上所有的问题，只消除其中一部分。

同时，对计算机一无所知的用户需要扶助：替他们做那些他们自己很容易做但不知道怎么做的事情。

这类服务可以由只提供扶助和维修服务的公司来提供。如果用户确实宁愿花钱买带服务的产品，他们也会愿意在免费得到产品之后购买服务。服务公司将在质量和价格上竞争；用户不会被绑定于任何特定一家。同时，我们这些不需要服务的人应当能够不付服务费而使用该程序。

“没有广告，你无法触达许多人，而你必须对程序收费来支持广告。”

“给一个人们可以免费得到的程序做广告是没有用的。”

有各种形式的免费或极廉价宣传，可以用来让众多计算机用户了解 GNU 之类的东西。但也许确实可以通过广告触达更多微型计算机用户。如果真是这样，一家收费提供复制和邮寄 GNU 服务的公司，应当成功到足以支付其广告费用并且还有盈余。这样，只有从广告中受益的用户才为此付费。

另一方面，如果许多人从朋友那里得到 GNU，而这类公司并不成功，这就表明广告对传播 GNU 并非真正必要。为什么自由市场的拥护者不愿让自由市场来决定这件事呢？⑷

“我的公司需要专有操作系统来获得竞争优势。”

GNU 将把操作系统软件移出竞争领域。你无法在这一领域获得优势，但你的竞争者也无法胜过你。你和他们将在其他领域竞争，而在这个领域彼此受益。如果你的业务是销售操作系统，你不会喜欢 GNU，那你就只好认了。如果你的业务是别的，GNU 可以使你不被推入销售操作系统的昂贵生意。

我希望 GNU 的开发能由许多厂商和用户的捐赠来支持，从而降低每个人的负担。⑸

“程序员难道不该因其创造力而得到报酬吗？”

如果有什么东西值得奖赏，那就是社会贡献。创造力可以是一种社会贡献，但仅当社会能自由使用其成果时才如此。如果程序员因创造新颖程序而理应得到奖赏，那么同理，如果他们限制这些程序的使用，就理应受到惩罚。

“程序员难道不应该因其创造力而要求报酬吗？”

为自己工作索要报酬，或设法使收入最大化，这本身没有错，只要不使用破坏性的手段。但如今软件领域惯用的手段都是以破坏为基础的。

通过限制用户对程序的使用来从他们身上榨取金钱是有破坏性的，因为这些限制减少了程序可被使用的数量和方式。这减少了人类从该程序中获得的财富。当限制是刻意选择的时候，有害后果就是蓄意的破坏。

好公民不用这种破坏性手段致富的原因是：如果人人都这样做，我们都会因相互破坏而变得更穷。这是康德式的伦理；或者说，黄金法则。既然我不喜欢人人囤积信息所导致的后果，我就必须认为这样做是错的。具体地说，希望因自己的创造力得到报偿，并不能成为剥夺整个世界全部或部分创造力的理由。

“程序员难道不会挨饿吗？”

我可以说，没有人被迫当程序员。我们大多数人靠站在街上做鬼脸是挣不到钱的。但我们并不因此注定要一辈子站在街上做鬼脸、挨饿。我们做别的事。

但那是错误的回答，因为它接受了提问者隐含的假设：没有软件所有权，程序员就一分钱也拿不到。据说这是全有或全无的事。

程序员不会挨饿的真正原因是，他们仍然可以因编程而得到报酬；只是不像现在那么多。

限制复制并不是软件业的唯一基础。它是最常见的基础，因为它带来最多的钱。如果它被禁止，或被顾客拒绝，软件业就会转向其他目前较少使用的组织基础。任何行业总有无数种组织方式。

在新基础上，编程大概不会像现在这样有利可图。但这不是反对变革的论据。售货员挣他们现在这样的薪水，并不被认为是不公正。如果程序员挣同样的钱，那也不算不公正。（实际上他们仍会挣得比这多得多。）

“人们难道没有权利控制自己的创造力如何被使用吗？”

“控制自己思想的使用”实际上构成对他人生活的控制；而且它通常被用来使他人的生活更艰难。

仔细研究过知识产权问题⑹的人（例如律师）说，并不存在固有的知识产权。政府承认的那些所谓知识产权，是通过具体的立法行为为具体目的创造出来的。

例如，专利制度的建立是为了鼓励发明人公开其发明的细节。其目的是帮助社会，而不是帮助发明人。当时专利 17 年的有效期与技术进步的速度相比是短暂的。由于专利只是厂商之间的问题，而对厂商来说许可协议的成本和精力与建立生产相比很小，所以专利往往为害不大。它们不会妨碍大多数使用专利产品的个人。

古代并不存在版权的观念，那时作者常常在非虚构作品中大段抄袭其他作者。这种做法很有用，而且这是许多作者的作品能部分留存下来的唯一途径。版权制度是明确为鼓励写作而创设的。在它被发明出来的领域——书籍，只有靠印刷机才能经济地复制——它为害甚小，也未妨碍大多数读书的个人。

所有知识产权都只是社会授予的许可，因为人们认为（无论对错）授予它们会使整个社会受益。但在任何具体情况下，我们都必须问：授予这种许可真的让我们处境更好吗？我们是在许可一个人做什么样的行为？

当今程序的情况与一百年前的书籍大不相同。最容易的复制程序方式是从一个邻居到另一个邻居；程序同时具有性质不同的源代码和目标代码；程序是被使用而不是被阅读和欣赏的——这些事实合在一起造成了一种局面：执行版权的人正在物质上和精神上损害整个社会；在这种局面下，一个人不应这样做，无论法律是否允许他。

“竞争使事情做得更好。”

竞争的范式是一场赛跑：通过奖励胜者，我们鼓励每个人跑得更快。当资本主义真的这样运作时，它干得不错；但它的辩护者错误地假定它总是这样运作。如果赛跑者忘了为什么设奖，而一心要赢，不择手段，他们可能找到别的策略——比如攻击其他赛跑者。如果赛跑者打起架来，他们都会很晚才跑完。

专有且保密的软件在道德上相当于打架的赛跑者。遗憾的是，我们唯一的裁判似乎并不反对打架；他只是规范打架（“每跑十码，你可以开一枪”）。他真应该把他们拉开，并且惩罚哪怕想打架的赛跑者。

“没有金钱激励，难道人人都会停止编程吗？”

实际上，许多人会在完全没有金钱激励的情况下编程。编程对某些人有不可抗拒的吸引力，通常是最擅长编程的人。职业音乐家也不缺——尽管靠音乐谋生毫无希望，他们仍坚持做下去。

但这个问题虽然常被提出，其实并不切合实际情况。程序员的报酬不会消失，只会变少。所以正确的问题是：在报酬减少的情况下，还会有人编程吗？我的经验表明会的。

十多年来，世界上许多最优秀的程序员在人工智能实验室工作，拿的钱远少于他们在别处能拿到的。他们得到许多种非金钱的报偿：例如名声和赏识。而创造本身也是乐趣，本身就是一种报偿。

后来，当有人给他们机会去做同样有趣的工作并拿很多钱时，他们大多离开了。

事实表明，人们会出于财富之外的原因编程；但如果也有机会赚很多钱，他们就会期待并要求它。低薪组织在与高薪组织竞争时表现不佳，但如果高薪组织被禁止，它们就不一定表现不佳。

“我们迫切需要程序员。如果他们要求我们停止帮助邻居，我们就必须服从。”

你永远不会绝望到必须服从这类要求的地步。记住：防御可以花上百万，进贡却一分不给！

“程序员总得有办法谋生。”

短期来看，这是真的。然而，程序员有许多办法可以在不卖程序使用权的情况下谋生。这种方式如今盛行，是因为它给程序员和商人带来最多的钱，而不是因为它是唯一的谋生之道。如果你想找别的办法，很容易找到。下面是一些例子。

推出新计算机的制造商会为把操作系统移植到新硬件上付费。

教学、扶助和维护服务的销售也可以雇用程序员。

有新想法的人可以把程序作为免费软件（freeware）⑺分发，请求满意的用户捐赠，或出售扶助服务。我遇到过已经这样成功运作的人。

有相关需求的用户可以组成用户组，并缴纳会费。用户组可以与编程公司签约，编写组内成员想要使用的程序。

各种开发都可以用“软件税”来资助：

假设每个买计算机的人都必须按价格的 x% 缴纳软件税。政府把这笔钱交给像 NSF 这样的机构，用于软件开发。

但如果计算机购买者本人向软件开发捐款，他可以凭此抵税。他可以捐给自己选择的项目——通常是希望项目完成后能使用其结果的项目。捐款额中不超过其应缴税款总额的部分，都可以用来抵税。

总税率可以由纳税人投票决定，投票权重按他们将被课税的金额计算。

后果是：

- 使用计算机的群体支持软件开发。
- 该群体决定需要何种程度的支持。
- 关心自己的份额用在哪些项目上的用户，可以自行选择。

从长远看，让程序自由是迈向“后匮乏”世界的一步。在那个世界里，没有人仅仅为了谋生就必须拼命工作。人们只要每周花上必要的十个小时去做立法、家庭咨询、机器人维修和小行星勘探这类必做之事，就可以自由地把精力投入编程这类有趣的活动。到那时，人们不必靠编程谋生。

我们已经大大减少了整个社会为实际生产力所必须付出的工作量，但其中只有一小部分转化成了劳动者的闲暇，因为许多非生产性活动是生产性活动所必需的伴随物。造成这一情况的主要原因是官僚主义，以及为对抗竞争而进行的同构角力。自由软件将大大减少软件生产领域的这类损耗。为了把生产率上的技术进步转化为我们更少的工作，我们必须这样做。

> 注：⑴ 这里的措辞不够严谨。本意是：使用 GNU 系统无需为*许可*付费。但这些话没有说清这一点，人们常把它们理解为：GNU 的副本应当总是以极低价格或免费分发。这从来不是本意；下文宣言中还提到，公司可以通过提供分发服务来获利。后来我学会了仔细区分“自由”意义上（freedom）的“free”和“价格”意义上（price）的“free”。自由软件是用户可以自由分发和修改的软件。有些用户可能免费获得副本，另一些用户则付费获得副本——如果这些钱有助于支持软件改进，那就更好。重要的是，每个持有副本的人都有与他人合作使用它的自由。

> 注：⑵ 这又是我未能仔细区分 “free” 两种不同含义的一处。这一说法本身并不错——你可以从朋友那里或通过网络免费获得 GNU 软件的副本。但它确实暗示了错误的观念。

> 注：⑶ 如今已有多家这样的公司存在。

> 注：⑷ 自由软件基金会的大部分资金来自分发服务，尽管它是慈善机构而非公司。如果*没有人*选择向 FSF 订购副本来获取，它就无法开展工作。但这并不意味着专有性限制就是合理的，可以强迫每个用户付费。只要全部用户中的一小部分向 FSF 订购副本，就足以维持 FSF 的运转。因此我们请求用户选择以这种方式支持我们。你尽了自己的一份力吗?

> 注：⑸ 最近一群计算机公司共同出资，支持维护 GNU C 编译器。

> 注：⑹ 在八十年代，我还没有意识到谈论“知识产权”的“问题”有多么令人困惑。这个词显然带有偏见；更微妙的是，它把各种互不相同、提出非常不同问题的法律混在一起。如今我力劝人们彻底拒用“知识产权”一词，以免他人以为那些法律构成了一个连贯的问题。要把问题说清楚，办法是分别讨论专利、版权和商标。关于这个词如何散布混乱与偏见，更多解释参见 <https://www.gnu.org/philosophy/not-ipr.xhtml>。

> 注：⑺ 后来我们发现有必要区分“自由软件”（free software）与“免费软件”（freeware）。“freeware”一词指你可以自由再分发的软件，但通常你不能自由研究和修改其源代码，所以它大多不是自由软件。更多解释参见 <https://www.gnu.org/philosophy/words-to-avoid.html>。

术语表

Abbrev 缩写（abbrev）是一种文本字符串，当它出现在缓冲区中时，会展开成另一个不同的文本字符串。例如，你可以把几个字母定义为某个你希望频繁插入的长短语的缩写。参见“缩写”。

Aborting 中止（aborting）指退出一次递归编辑（recursive edit，见该条）。命令 ‘C-]’ 和 ‘M-x top-level’ 用于此目的。参见“退出与中止”。

Active Region 在文本中某个位置设置标记（mark，见该条）的同时也会激活它。当标记处于活动状态时，我们称该区域为活动区域（active region）。参见“标记与区域”。

<Alt> Alt 是键盘输入字符可能具有的一个修饰位（modifier bit）的名称。要使某个字符带上 Alt，请在按住 <Alt> 键的同时键入它。这类字符的名称以 ‘<Alt>-’ 开头（通常简写为 ‘A-’）。（请注意，许多终端的 <Alt> 键实际上是 <Meta> 键。）参见“输入类型”。

Argument 参见“Glossary---Numeric Argument”。

ASCII character ASCII 字符要么是 ASCII 控制字符，要么是 ASCII 打印字符。参见“输入类型”。

ASCII control character ASCII 控制字符是大写字母的 Control 版本，或者是字符 ‘@[\]^_?’ 中某一个的 Control 版本。

ASCII printing character ASCII 字母、数字、空格，以及下列标点字符：‘!@#$%^&*()_-+=|\~`{}[]:;"'<>,.?/’。

Auto Fill Mode 自动填充模式（Auto Fill mode）是一种次模式（minor mode，见该条），在你插入文本时，文本会自动断成不超过给定最大宽度的行。参见“填充文本”。

Auto Saving 自动保存（auto saving）是指定期把 Emacs 缓冲区的内容保存到一个以特殊方式命名的文件中，这样即使缓冲区因系统错误或用户失误而丢失，信息也能得以保留。参见“自动保存：灾难防护”。

Autoloading Emacs 可以在 Lisp 程序请求某个 Lisp 库中的函数时自动加载该库。这称为“autoloading”（自动加载）。参见“Emacs 的 Lisp 代码库”。

Backtrace 回溯（backtrace）是对一系列函数调用的踪迹，显示程序如何到达某一位置。它主要用于查找和修正程序缺陷（bug，见该条）。当 Emacs 发出错误信号时，或当你键入 ‘C-g’ 时（参见“Glossary---Quitting”），Emacs 可以显示回溯。参见“bug 报告核对清单”。

Backup File 备份文件（backup file）记录某个文件在当前编辑会话之前的内容。Emacs 会自动创建备份文件，以帮助你找回或撤销事后后悔的修改。参见“备份文件”。

Balancing Parentheses Emacs 可以手动或自动地平衡圆括号（或其他配对分隔符）。手动平衡通过移动越过成对括号组的命令来完成（参见“在括号结构中移动”）。自动平衡通过闪烁或高亮显示与你刚插入的分隔符配对的那个分隔符，或替你插入配对的分隔符来实现（参见“匹配括号”）。

Balanced Expressions 平衡表达式（balanced expression）是语法上可识别的表达式，例如符号（symbol，见该条）、数字、字符串常量、块，或 C 中的括号表达式。参见“平衡括号表达式”。

Balloon Help 参见“Glossary---Tooltips”。

Base Buffer 基础缓冲区（base buffer）是其文本被某个间接缓冲区（indirect buffer，见该条）共享的缓冲区。

Bidirectional Text 有些人类语言（例如英语）从左向右书写，另一些（例如阿拉伯语）则从右向左书写。Emacs 支持这两种形式以及它们的任意混合——这就是“双向文本”（bidirectional text）。参见“双向编辑”。

Bind 绑定（bind）某个键序列，就是为它指定一个绑定（binding，见该条）。参见“交互式更改键绑定”。

Binding 键序列在 Emacs 中获得含义的方式是拥有一个绑定（binding），绑定即一条命令（command，见该条）——一个在你键入该序列时运行的 Lisp 函数。参见“按键与命令”。自定义常常涉及把某个字符重新绑定到另一个命令函数。所有键序列的绑定都记录在键位图（keymap，见该条）中。参见“键位图”。

Blank Lines 空行（blank line）是只含空白字符的行。Emacs 有若干命令用于操作缓冲区中的空行。参见“空行”。

Bookmark 书签（bookmark）与寄存器（register，见该条）类似，都记录缓冲区中的位置，供你以后返回。与寄存器不同，书签在 Emacs 会话之间保持存在。参见“书签”。

Border 边框（border）是沿框架边缘的一条狭窄空间，仅用于间隔，不用于显示任何内容。Emacs 框架有一个普通的外部边框，它位于包括菜单栏在内的所有元素之外；此外还有一个内部边框，环绕文本窗口及其滚动条和边缘，并把它们与菜单栏和工具栏隔开。你可以用选项和资源来自定义这两种边框（参见“内部边框和外部边框”）。边框不同于边缘（fringe，见该条）。

Buffer 缓冲区（buffer）是基本的编辑单位；一个缓冲区对应一段正在编辑的文本。你通常有多个缓冲区，但任何时候你只编辑其中一个，即当前缓冲区，不过在使用多个窗口或框架（见该条）时可能有多个缓冲区可见。大多数缓冲区都在访问（visiting，见该条）某个文件。参见“多缓冲区”。

Buffer Selection History Emacs 维护一个缓冲区选择历史（buffer selection history），记录每个 Emacs 缓冲区最近被选中的先后顺序。它用于决定该选择哪个缓冲区。参见“多缓冲区”。

Bug 缺陷（bug）是程序不正确或不合理的行为，或是不准确、令人困惑的文档。Emacs 开发者非常重视针对 Emacs 代码及其文档的缺陷报告，并请你报告你发现的任何缺陷。参见“报告 Bug”。

Button Down Event 按下鼠标按钮事件（button down event）是按下鼠标按钮时立即产生的那类输入事件（input event，见该条）。参见“重绑定鼠标按钮”。

By Default 参见“Glossary---Default”。

Byte Compilation 参见“Glossary---Compilation”。

cf. c.f. 是拉丁语“confer”的缩写，意为“compare with”或“compare to”（与……比较）。第二种写法“c.f.”是流传很广的拼写错误。

‘C-’ 字符名中的 ‘C-’ 是 Control 的缩写。参见“输入类型”。

‘C-M-’ 字符名中的 ‘C-M-’ 是 Control-Meta 的缩写。如果你的终端没有真正的 <Meta> 键，那么键入 Control-Meta 字符的方法是先键入 <ESC>，再键入相应的 Control 字符。参见“输入类型”。

Case Conversion 大小写转换（case conversion）指把文本从大写变为小写，或反之。参见“大小写转换命令”。

Case Folding 大小写折叠（case folding）指忽略同一字母不同大小写形式（大写、小写和词首大写）之间的差异。在文本搜索中，Emacs 默认执行大小写折叠。参见“搜索时的宽松匹配”。

Character 字符（character）构成 Emacs 缓冲区的内容。此外，键序列（key sequence，见该条）通常也由字符组成（不过它们也可能包含其他输入事件）。参见“输入类型”。

Character Folding 字符折叠（character folding）指忽略外观相似字符之间的差异，例如 ‘a’、‘ă’ 和 ‘á’ 之间的差异。在文本搜索中，Emacs 默认执行字符折叠。参见“搜索时的宽松匹配”。

Character Set Emacs 支持若干字符集（character set），每个字符集代表某种特定的字母表或文字系统。参见“国际字符集支持”。

Character Terminal 参见“Glossary---Text Terminal”。

Click Event 单击事件（click event）是按下鼠标按钮并在不移动鼠标的情况下松开时产生的那类输入事件（input event，见该条）。参见“重绑定鼠标按钮”。

Client 参见“Glossary---Server”。

Clipboard 剪贴板（clipboard）是窗口系统提供的、用于在应用程序之间传送文本的缓冲区。在 X Window System 上，剪贴板是在主选择（primary selection，见该条）之外另行提供的；在 MS-Windows 和 Mac 上，使用的是剪贴板，*而不是*主选择。参见“使用剪贴板”。

Coding System 编码系统（coding system）是在文件或信息流中编码文本字符的方式。Emacs 在读写文本时能够与各种编码系统相互转换。参见“编码系统”。

Command 命令（command）是专门定义的一种 Lisp 函数，能够充当 Emacs 中的键绑定，或能按其名称被调用（参见“Glossary---Command Name”）。（“命令”的另一个说法是“交互式函数”——两者可互换使用。）当你键入某个键序列（key sequence，见该条）时，它的绑定（binding，见该条）会在相应的键位图（keymap，见该条）中查找，以找出要运行的命令。参见“按键与命令”。

Command History 参见“Glossary---Minibuffer History”。

Command Name 命令名（command name）是作为命令的 Lisp 符号（symbol，见该条）的名称（参见“按键与命令”）。你可以用 ‘M-x’ 按名称调用任何命令（参见“按名称执行命令”）。

Comment 注释（comment）是程序中仅供阅读程序的人查看的文本，它带有特殊标记，因此程序被加载或编译时会被忽略。Emacs 提供专门的命令来创建、对齐和删除注释。参见“操作注释”。

Common Lisp Common Lisp 是 Lisp（见该条）的一种方言，比 Emacs Lisp 大得多、也强大得多。Emacs 在 CL 包中提供了 Common Lisp 的一个子集。参见 《《cl》手册》。

Compilation 编译（compilation）是从源代码生成可执行程序的过程。Emacs 有命令可用于编译 Emacs Lisp 代码文件（参见《Emacs Lisp 参考手册》的“Byte Compilation”）以及 C 和其他语言的程序（参见“在 Emacs 中运行编译”）。经字节编译的 Emacs Lisp 代码加载和运行得更快。

Complete Key 完整键（complete key）是完整指定 Emacs 要执行的一个动作的键序列。例如，‘X’、‘C-f’ 和 ‘C-x m’ 都是完整键。完整键的含义来自它们被绑定（参见“Glossary---Bind”）到命令（command，见该条）。因此，‘X’ 按惯例绑定到一个在缓冲区中插入 ‘X’ 的命令；‘C-x m’ 按惯例绑定到一个开始撰写邮件的命令。参见“按键”。

Completion 补全（completion）指 Emacs 把某个名称的缩写自动展开成完整名称的操作。当可能的有效输入集合已知时，补全用于迷你缓冲区（minibuffer，见该条）中的参数，例如命令名、缓冲区名和文件名。补全通

常在键入 <TAB>、<SPC> 或 <RET> 时发生。参见“补全”。

Continuation Line 当一行文本长于窗口宽度时，显示时它通常会占用多于一行的屏幕行（但参见“Glossary---Truncation”）。我们说该文本行被续行，它所占用的第一行之后的屏幕行都称为续行（continuation line）。参见“续行”。与此相关的 Emacs 功能是填充（filling，见该条）。

Control Character 控制字符（control character）是按住 <Ctrl> 键键入的字符。有些控制字符也有自己的键，因此你无需使用 <Ctrl> 就能键入它们。例如，<RET>、<TAB>、<ESC> 和 <DEL> 都是控制字符。参见“输入类型”。

Copyleft 版权开放（copyleft）是一种声明，它从法律上允许公众重新分发和修改某个程序或其他作品，但要求修改后的版本也带有类似的许可。版权通常被用来使使用者彼此隔绝、无能为力；而借助 copyleft，我们把它反过来，让使用者拥有权力，并鼓励他们合作。

GNU 工程所使用的具体 copyleft 形式称为 GNU 通用公共许可证（GNU General Public License）。参见“附录 A GNU 通用公共许可证”。

<Ctrl> <Ctrl> 键或 control 键就是为输入控制字符（control character，见该条）而按住的键。参见“Glossary---C-”。

Current Buffer Emacs 中的当前缓冲区（current buffer）是大多数编辑命令所作用的那个 Emacs 缓冲区。你可以选择任何 Emacs 缓冲区作为当前缓冲区。参见“多缓冲区”。

Current Line 当前行（current line）是光标点所在的行（参见“光标点”）。

Current Paragraph 当前段落（current paragraph）是光标点所在的段落。如果光标点处于两个段落之间，则当前段落是光标点之后的那个段落。参见“段落”。

Current Defun 当前 defun 是光标点所在的 defun（见该条）。如果光标点处于两个 defun 之间，则当前 defun 是光标点之后的那个。参见“顶层定义（defun）”。

Cursor 光标（cursor）是屏幕上指示插入和删除发生位置（称为光标点，point，见该条）的矩形。光标位于光标点之后那个字符之上或其下方。人们常说的“光标”，严格说来往往指的是“光标点”。参见“光标点”。

Customization 自定义（customization）是对 Emacs 的工作方式做小幅修改，以反映你的偏好或需要。它通常通过设置变量（参见“变量”）或字体（face，参见“自定义字体”）来完成，或者通过重新绑定键序列（参见“键位图”）来完成。

Cut and Paste 参见“Glossary---Killing”和参见“Glossary---Yanking”。

Daemon 守护进程（daemon）是在后台运行的系统级进程的标准术语。守护进程通常在系统首次启动时启动。当 Emacs 以 daemon 模式运行时，它不打开显示。你用 ‘emacsclient’ 程序连接到它。参见“把 Emacs 用作服务器”。

Default Argument 参数的默认值（default）是在你没有指定参数时所假定的值。当使用迷你缓冲区读取参数时，如果你直接键入 <RET>，就使用默认参数。参见“迷你缓冲区”。

Default 默认值（default）是在你没有明确指定要用的值时，为某一目的而使用的值。

Default Directory 当你指定的文件名不以‘/’或‘~’开头时，它相对于当前缓冲区的默认目录（default directory）来解释。（在 MS 系统上，以驱动器字母‘X:’开头的文件名被视为绝对路径，而非相对路径。）参见“用于文件名的迷你缓冲区”。

Defun defun 是程序顶层的主要定义。“defun”这个名称来自 Lisp，在那里大多数此类定义都使用‘defun’构造。参见“顶层定义（defun）”。

<DEL> <DEL> 是运行“删除光标前一个字符”命令的字符。它通常是 <Delete> 键或 <BACKSPACE> 键，哪个好按就用哪个。参见“删除文本”。

Deletion 删除（deletion）指擦除文本而不把它复制到删除环（kill ring，见该条）中。与之相对的是删除（killing，见该条）。

Deletion of Files 删除文件（deleting a file）指把它从文件系统中擦除。（注意，有些系统使用回收站或垃圾箱的概念，让你可以恢复已删除的文件。）参见“其他文件操作”。

Deletion of Messages 删除邮件（deleting a message，在 Rmail 和其他邮件客户端中）指把它标记为要从邮件文件中清除。在你清除（expunge，见该条）Rmail 文件之前，你仍然可以恢复已删除的邮件。参见“删除消息”。

Deletion of Windows 删除窗口（deleting a window）指把它从屏幕上消除。其他窗口会扩大以占用其空间。窗口中的文本不会丢失，如果你愿意，可以创建一个与旧窗口尺寸相同的新窗口。参见“多窗口”。

Directory 目录（directory）是文件系统中命名的集合，你可以在其中放置单个文件或子目录。它们有时被称为“文件夹”（folder）。参见“文件目录”。

Directory Local Variable 目录局部变量（directory local variable）是作用于某个目录内所有文件的局部变量（local variable，见该条）。参见“按目录的局部变量”。

Directory Name 在 GNU 和其他类 Unix 系统上，目录名（directory name）是以‘/’结尾的字符串。例如，‘/no-such-dir/’是目录名，而‘/tmp’不是，即使‘/tmp’所命名的文件恰好是一个目录。在 MS-Windows 上，这种关系更为复杂。参见《Emacs Lisp 参考手册》的“Directory Names”。

Dired Dired 是 Emacs 的一项功能，它显示文件目录的内容，并允许你“编辑目录”，对目录中的文件执行操作。参见“Dired 目录编辑器”。

Disabled Command 被禁用的命令（disabled command）是未经特别确认就不能运行的命令。禁用命令的常见原因是它会让初学者困惑。参见“禁用命令”。

Down Event 按下事件（down event）是“按下鼠标按钮事件”（button down event，见该条）的简称。

Drag Event 拖动事件（drag event）是按下鼠标按钮、移动鼠标、然后松开按钮时产生的那类输入事件（input event，见该条）。参见“重绑定鼠标按钮”。

Dribble File 记录文件（dribble file）是 Emacs 把你键入的所有字符写入其中的文件。记录文件可用于为调试 Emacs 缺陷留下记录。除非你要求，Emacs 不会创建记录文件。参见“报告 Bug”。

e.g. 是拉丁语“exempli gratia”的缩写，意为“例如”。

Echo Area 回显区（echo area）是屏幕的最下面一行，用于回显命令的参数、提问以及显示简短消息（包括错误消息）。这些消息存储在缓冲区‘*Messages*’中，以便你以后查看。参见“回显区”。

Echoing 回显（echoing）指通过显示输入事件（在回显区中）来确认收到它们。Emacs 从不回显单字符键序列；较长的键序列只有在你键入过程中停顿时才回显。

Electric 如果某个字符通常是自插入的（self-inserting，见该条），而当前主模式（major mode，见该条）重新定义它，让它同时做别的事情，我们就说这个字符是电动的（electric）。例如，某些编程语言主模式把特定分隔符字符定义为：除自插入外还重排该行的缩进，或插入一个或多个换行符。

End Of Line 行尾（end of line）是表示文本行结束的字符或字符序列。在 GNU 和 Unix 系统上，这是换行符（newline，见该条），但其他系统有其他约定。参见“编码系统”。Emacs 能识别文件中的多种行尾约定，并在它们之间转换。

Environment Variable 环境变量（environment variable）是操作系统保存的一组变量之一，每个变量都有一个名称和一个值。Emacs 可以访问其父 shell 设置的环境变量，也可以设置它传给所调用程序的环境中的变量。参见“环境变量”。

EOL 参见“Glossary---End Of Line”。

Error 当 Emacs 命令在当前情形下无法执行时，就发生错误（error）。发生错误时，命令的执行停止（除非命令被编写为另行处理），并且 Emacs 通过显示错误消息（error message，见该条）来报告错误。

Error Message 错误消息（error message）是 Emacs 在你要求它做不可能的事情时（例如光标点在缓冲区末尾时向前删除文本），或命令以某种方式发生故障时显示的输出。这类消息出现在回显区中，并伴随一声鸣响。

<ESC> <ESC> 是在缺少 <Meta> 键的键盘上用于键入 Meta 字符的前缀字符。与 <Meta> 键不同（<Meta> 键像 <SHIFT> 键一样，在键入另一个字符时被按住），<ESC> 键要像按字母键那样按一下，它作用于你键入的下一个字符。

etc. 是拉丁语“et cetera”的缩写，意为“等等”。

Expression 参见“Glossary---Balanced Expression”。

Expunging 清除（expunging）Rmail 缓冲区、Gnus 新闻组或 Dired 缓冲区，是一种真正丢弃你先前标记为删除的邮件或文件的操作。

Face 字体（face）是显示字符的一种样式。它规定字体族和字号、前景色和背景色、下划线和删除线、背景点画等属性。Emacs 提供功能把特定字体与缓冲区文本的某些部分关联起来，以便按字体属性所规定的方式显示该文本。参见“文本字体”。

File Local Variable 文件局部变量（file local variable）是在某个给定文件中指定的局部变量（local variable，见该条）。参见“文件中的局部变量”和参见“Glossary---Directory Local Variable”。

File Locking Emacs 使用文件锁（file locking）来察觉两个不同用户何时开始同时编辑同一个文件。参见“防止同时编辑”。

File Name 文件名（file name）是指向某个文件的名称。文件名可以是相对的或绝对的；相对文件名的含义取决于当前目录，而绝对文件名无论当前目录是哪一个都指向同一个文件。在 GNU 和 Unix 系统上，绝对文件名以斜杠（根目录）开头，或以‘~/’或‘~USER/’（主目录）开头。在 MS-Windows/MS-DOS 上，绝对文件名也可以以驱动器字母加冒号开头，例如‘D:’。

有些人用“pathname”一词表示文件名，但我们不这样用；我们只在“搜索路径”（search path，见该条）这一说法中使用“path”一词。

File-Name Component 文件名组件（file-name component）直接命名某个特定目录内的一个文件。在 GNU 和 Unix 系统上，文件名是由斜杠分隔的一系列文件名组件。例如，‘foo/bar’ 是包含两个组件 ‘foo’ 和 ‘bar’ 的文件名；它指当前目录下名为 ‘foo’ 的目录中名为 ‘bar’ 的文件。MS-DOS/MS-Windows 的文件名也可以用反斜杠分隔组件，如 ‘foo\bar’。

Fill Prefix 填充前缀（fill prefix）是填充时应当在每行行首出现的字符串。它不被视为待填充文本的一部分。参见“填充文本”。

Filling 填充（filling）文本指调整换行的位置，使文本在相邻行之间移动，从而让所有行长度大致相同。参见“填充文本”。其他一些编辑器把此功能称为“line wrapping”。

Font Lock Font Lock 是一种模式，它根据语法用不同的字体高亮显示缓冲区文本的某些部分。其他一些编辑器称之为“语法高亮”（syntax highlighting）。例如，所有注释（comment，见该条）都可能被着色为红色。参见“Font Lock 模式”。

Fontset 字体集（fontset）是命名的字体集合。字体集规格列出各字符集以及显示每个字符集所用的字体。通过指定字体集的名称，字体集使得一次更改若干字体变得容易，而不必分别更改每个字体。参见“字体集”。

Formfeed Character 参见“Glossary---Page”。

Frame 框架（frame）是 Emacs 窗口的矩形组合。Emacs 启动时有一个框架，但你可以创建更多。你可以把每个框架细分为 Emacs 窗口（window，见该条）。当你使用窗口系统（window system，见该条）时，同一时间可以有多个框架可见。参见“框架与图形显示”。其他一些编辑器用“窗口”一词表示这东西，但在 Emacs 中，窗口指的是别的东西。

Free Software 自由软件是赋予你共享、学习和修改自由的软件。Emacs 是自由软件，是 GNU 工程（q.v.）的一部分，按照一种称为 GNU 通用公共许可证的 copyleft（q.v.）许可证分发。参见“附录 A GNU 通用公共许可证”。

Free Software Foundation 自由软件基金会（Free Software Foundation，FSF）是一个慈善基金会，致力于推动自由软件（q.v.）的发展。更多信息请见 FSF 网站（https://fsf.org/）。

Fringe 在图形显示（q.v.）上，文本区与窗口边框之间有一条狭窄的区域，它是框架（q.v.）的一部分。这些“边缘”（fringes）用于显示提供缓冲区文本相关信息的符号（参见“窗口边缘”）。Emacs 使用一种名为 ‘fringe’ 的特殊字体（face）来显示边缘。参见“文本字体”

FSF 参见“Glossary---Free Software Foundation”。

FTP FTP 是 File Transfer Protocol（文件传输协议）的缩写。它是获取远程文件（q.v.）的一种标准方法。

Function Key 功能键是键盘上发送输入但不对应任何字符的键。参见“重绑定功能键”。

Global 全局（global）意为“与当前环境无关；在整个 Emacs 中生效”。它与局部（local，q.v.）相对。“全局”一词的具体用法见下文各条。

Global Abbrev 缩写的全局定义在所有没有针对同一缩写的局部（q.v.）定义的主模式中生效。参见“缩写”。

Global Keymap 全局键位图（q.v.）包含在所有地方都生效的键绑定，除非它们被某个主模式的局部键位图（q.v.）中的局部键绑定覆盖。参见“键位图”。

Global Mark Ring 全局标记环记录你最近在其中设置过标记（q.v.）的一系列缓冲区。在很多情况下，你可以借助它回溯你编辑过的缓冲区，或你在其中找到过标签（tag）的缓冲区（参见“Glossary---Tags Table”）。参见“全局标记环”。

Global Substitution 全局替换是指在大量文本中把某个字符串的每一处出现都替换为另一个字符串。参见“替换命令”。

Global Variable 变量的全局值（q.v.）在所有没有自己的局部（q.v.）值的缓冲区中生效。参见“变量”。

GNU GNU 是“GNU's Not Unix”的递归缩写，指一个与 Unix 兼容的操作系统，它是自由软件（q.v.）。参见“GNU 宣言”。GNU 通常与 Linux 内核一起使用，因为 Linux 比 GNU 内核工作得更好。更多信息请见 GNU 网站（https://www.gnu.org/）。

Graphic Character 图形字符是那些被赋予图形形象而不只是名称的字符。除 Control（q.v.）字符之外的所有非 Meta（q.v.）字符都是图形字符。它们包括字母、数字、标点和空格；不包括 <RET> 或 <ESC>。在 Emacs 中，输入图形字符会插入该字符（在普通编辑模式下）。参见“插入文本”。

Graphical Display 图形显示是能够显示图像和多种字体的显示设备。通常它还具有窗口系统（q.v.）。

Highlighting 高亮文本是指以不同的前景色和/或背景色显示它，使其在缓冲区其余文本中显得突出。

Emacs 以多种方式使用高亮。每当区域处于活动状态时，它都会高亮该区域（*note Mark::）。增量搜索也会高亮匹配项（*note Incremental Search::）。参见“Glossary---Font Lock”。

Hardcopy 硬拷贝指打印输出。Emacs 提供多种打印 Emacs 缓冲区内容的命令。参见“打印”。

<HELP> <HELP> 是 ‘C-h’ 或 <F1> 的 Emacs 名称。你可以随时输入 <HELP> 询问有哪些选项，或询问某个命令做什么。参见“帮助”。

Help Echo 帮助回显（help echo）是当鼠标指针位于需要一些说明的显示部分时显示在回显区（q.v.）的短消息。Emacs 会为菜单项、模式行的某些部分、工具栏按钮等显示帮助回显。在图形显示上，这些消息可以显示为工具提示（tooltip，q.v.）。参见“工具提示”。

Home Directory 主目录包含你的个人文件。在多用户的 GNU 或 Unix 系统上，每个用户都有自己的主目录。当你开始新的登录会话时，主目录就是默认的起始目录。主目录的标准简写是 ‘~’。类似地，‘~USER’ 表示其他某个用户的主目录。

Hook 钩子是一个函数列表，其中的函数在特定场合被调用，例如把缓冲区保存到文件、激活主模式等。通过定制各种钩子，你可以在不修改任何代码的情况下改变 Emacs 的行为。参见“钩子”。

Hyper Hyper 是一种修饰位的名称，键盘输入字符可以带有该修饰位。要输入 Hyper 字符，在按住 <Hyper> 键的同时输入字符。这类字符的名称以 ‘Hyper-’ 开头（通常简写为 ‘H-’）。参见“修饰键”。

i.e. 拉丁语 “id est” 的缩写，意为“即”。

Iff “Iff” 意为“当且仅当”。这一术语来自数学。在文档中应尽量避免使用它，因为许多人并不熟悉，还会把它误认为拼写错误。

Inbox 收件箱（inbox）是操作系统投递邮件的文件。Rmail 把邮件从收件箱转移到 Rmail 文件中，邮件在那里被永久保存，直到被显式删除。参见“Rmail 文件与收件箱”。

Incremental Search Emacs 提供增量搜索功能：你一输入第一个字符，Emacs 就开始搜索字符串；随着你输入更多字符，它会逐步细化搜索。参见“增量搜索”。

Indentation 缩进指行首的空白。大多数编程语言都有用缩进来凸显程序结构的惯例，Emacs 有专门的命令来调整缩进。参见“缩进”。

Indirect Buffer 间接缓冲区是与另一个缓冲区共享文本的缓冲区，后者称为它的基础缓冲区（base buffer，q.v.）。参见“间接缓冲区”。

Info Info 是 GNU 工程用于编写文档的超文本格式。

Input Event 输入事件在 Emacs 内部表示用户在终端上采取的一次动作。输入事件包括输入字符、输入功能键、按下或释放鼠标按钮，以及在 Emacs 框架之间切换。参见“输入类型”。

Input Method 输入法是一种通过输入 ASCII（q.v.）字符序列来输入非 ASCII 文本字符的系统。参见“输入法”。

Insertion 插入是指向缓冲区中添加文本，可以来自键盘，也可以来自 Emacs 中的其他位置。

Interactive Function “命令”（q.v.）的另一种说法。

Interactive Invocation 函数可以从 Lisp 代码中调用，也可以作为用户级命令调用（通过 ‘M-x’、某个键绑定或某个菜单）。在后一种情况下，称该函数被“交互式调用”。

Interlocking 参见“Glossary---File Locking”。

Isearch 参见“Glossary---Incremental Search”。

Justification 两端对齐（justification）是指在文本行内添加额外的空格，以调整文本边缘的位置。参见“显式填充命令”。

Key Binding 参见“Glossary---Binding”。

Keyboard Macro 键盘宏是一种从既有命令序列定义新 Emacs 命令的方法，无需编写 Lisp 程序。你可以用宏记录一串命令，然后任意多次回放。参见“键盘宏”。

Keyboard Shortcut 键盘快捷键是调用命令的键序列（q.v.）。有些程序所说的“分配键盘快捷键”，在 Emacs 中称为“绑定键序列”。参见“Glossary---Binding”。

Key Sequence 键序列（简称键）是作为一个整体有意义的一系列输入事件（q.v.）。若该键序列足以指明一个动作，它就是完整键（q.v.）；若不足，它就是前缀键（q.v.）。参见“按键”。

Keymap 键位图是记录键序列到其所运行命令的绑定（q.v.）的数据结构。例如，全局键位图把字符 ‘C-n’ 绑定到命令函数 ‘next-line’。参见“键位图”。

Keyboard Translation Table 键盘转换表是一个数组，它把来自终端的字符编码转换为构成键序列的字符编码。

Kill Ring 删除环保存你最近删除（kill，参见“Glossary---Killing”）的所有文本。你可以重新插入环中仍保留的任何已删除文本；这称为粘贴（yank，q.v.）。参见“粘贴”。

Killing 删除（killing）是指擦除文本并将其保存在删除环上，以便以后粘贴（q.v.）。有些别的系统称之为“剪切”。大多数擦除文本的 Emacs 命令执行的是删除（killing），而不是抹除（deletion，q.v.）。参见“删除与移动文本”。

Killing a Job 终止作业（例如一次 Emacs 调用）是指使它不复存在。其中任何未保存到文件的数据都会丢失。参见“退出 Emacs”。

Language Environment 你选择语言环境，就为输入法（q.v.）和编码系统（q.v.）指定了默认值。参见“语言环境”。如果你编辑非 ASCII 文本，这些默认值是相关的（参见“国际字符集支持”）。

Line Wrapping 参见“Glossary---Filling”。

Lisp Lisp 是一种编程语言。Emacs 的大部分是用一种称为 Emacs Lisp 的 Lisp 方言编写的，该方言扩展了若干特殊特性，使其特别适合文本编辑任务。

List 列表大致是指以左括号开头并以匹配的右括号结尾的文本字符串。在 C 模式和其他非 Lisp 模式中，由适合该语言的其他匹配定界符（例如花括号）包围的分组也被视为列表。Emacs 有许多用于对列表进行各种操作的命令。参见“在括号结构中移动”。

Local 局部（local）意为“只在特定上下文中生效”；相关的上下文类型是某次函数执行、某个缓冲区或某个主模式。它与“全局”（global，q.v.）相对。Emacs 术语中“局部”的具体用法见下文各条。

Local Abbrev 缩写的局部定义只在选定某个特定主模式时才生效。在该主模式中，它覆盖同一缩写的任何全局定义。参见“缩写”。

Local Keymap 局部键位图用于某个特定主模式；当前局部键位图中的键绑定（q.v.）覆盖相同键序列的全局绑定。参见“键位图”。

Local Variable 变量的局部值（q.v.）只作用于一个缓冲区。参见“局部变量”。

‘M-’ ‘M-’ 出现在字符名中时是 <Meta> 的缩写，<Meta> 是可以与任何字符组合的修饰键之一。参见“输入类型”

‘M-C-’ ‘M-C-’ 出现在字符名中时是 Control-Meta 的缩写；它与 ‘C-M-’（q.v.）含义相同。

‘M-x’ ‘M-x’ 是用于按名称调用 Emacs 命令的键序列。凡是没有绑定到键序列的命令，都用这种方式运行。参见“按名称执行命令”

Mail 邮件指通过计算机系统从一个用户发送给另一个用户、由收件人方便时阅读的消息。Emacs 有撰写和发送邮件以及阅读和编辑已收邮件的命令。参见“发送邮件”。用 Emacs 阅读邮件的一种方式见 参见“用 Rmail 读信”。

Mail Composition Method 邮件撰写方法是一种可在 Emacs 内运行、用于编辑和发送邮件的程序。Emacs 允许你从几种备选的邮件撰写方法中选择。参见“邮件撰写方法”。

Major Mode Emacs 的主模式是一组互斥的选项，每个主模式都把 Emacs 配置为编辑某一类文本。理想情况下，每种编程语言都有自己的主模式。参见“主模式”。

Margin 窗口可用部分（包括边缘）与窗口外缘之间的空白。

Mark 标记指向文本中的一个位置。它指明区域（q.v.）的一端，光标点则是另一端。许多命令对从光标点到标记之间的全部文本进行操作。每个缓冲区都有自己的标记。参见“标记与区域”。

Mark Ring 标记环用于保存标记最近访问过的若干位置，以便你想回到它们时可以回去。每个缓冲区都有自己的标记环；此外还有一个全局标记环（q.v.）。参见“标记环”。

Menu Bar 菜单栏是 Emacs 框架顶部的一行。它包含一些词，你可以用鼠标点击它们以弹出菜单，也可以用键盘界面来浏览它。参见“菜单栏”。

Message 参见“Glossary---Mail”。

Meta Meta 是一种修饰位的名称，你可以在命令字符中使用它。要输入 meta 字符，在输入字符的同时按住 <Meta> 键。我们用 ‘Meta-’ 开头的名称指称这类字符（通常简写为 ‘M-’）。例如，‘M-<’ 的输入方法是按住 <Meta> 的同时输入 ‘<’（在大多数终端上，输入 ‘<’ 本身要按住 <SHIFT> 并输入 ‘,’）。参见“输入类型”

在有些终端上，<Meta> 键实际标为 <Alt> 或 <Edit>。

Meta Character Meta 字符是其字符编码包含 Meta 位的字符。

Minibuffer 迷你缓冲区是在需要时出现在回显区（q.v.）内部的窗口，用于读取命令的参数。参见“迷你缓冲区”。

Minibuffer History 迷你缓冲区历史记录你过去为迷你缓冲区参数指定的文本，以便你方便地再次使用同样的文本。参见“迷你缓冲区历史”。

Minor Mode 次模式是 Emacs 的可选特性，可以独立于所有其他特性打开或关闭。每个次模式都有一个用于打开或关闭它的命令。有些次模式是全局的（q.v.），有些是局部的（q.v.）。参见“次模式”。

Minor Mode Keymap 次模式键位图是属于某个次模式的键位图，在该模式启用时生效。次模式键位图优先于缓冲区的局部键位图，正如局部键位图优先于全局键位图一样。参见“键位图”。

Mode Line 模式行是每个窗口（q.v.）底部的一行，给出该窗口所显示缓冲区的状态信息。参见“模式行”。

Modified Buffer 如果缓冲区的文本自上次保存以来已被改动（若从未保存过，则自创建以来已被改动），该缓冲区（q.v.）就是已修改的。参见“保存文件”。

Moving Text 移动文本是指把文本从一处擦除并插入到另一处。移动文本的常用方法是先删除（q.v.）它，再粘贴（q.v.）它。参见“删除与移动文本”。

MULE 在 Emacs 23 之前，MULE 是一个软件包的名字，它通过增加对多种字符集（q.v.）的支持，为 Emacs 提供“多语言增强”（MULtilingual Enhancement）。MULE 后来被并入 Emacs，而在 Emacs 23 获得内部 Unicode 支持后，其大部分内容被取代。

Emacs 中涉及字符集支持的某些部分仍使用 MULE 这个名称。参见“国际字符集支持”。

Multibyte Character 多字节字符是在缓冲区中占用多个字节的字符。Emacs 使用多字节字符表示非 ASCII 文本，因为非 ASCII 字符的数量远多于 256 个。参见“国际字符集简介”

Named Mark 命名标记是一种寄存器（q.v.），它扮演记录文本中某个位置的角色，以便你把光标点移到该位置。参见“寄存器”。

Narrowing 窄化是指创建一种限制（restriction，q.v.），把当前缓冲区中的编辑限定在文本的某一部分。在边界重新放宽之前，该部分之外的文本无法编辑（或查看），但它仍然存在，保存文件时会全部保存。参见“窄化”。

Newline 缓冲区中的 Control-J 字符终止文本行，因此也称为换行符（newline）。参见“Glossary---End Of Line”。

‘nil’ ‘nil’ 是一个通常解释为逻辑“假”的值。它的对立面是 ‘t’，解释为“真”。

Numeric Argument 数字参数是在命令之前指定的数字，用于改变命令的效果。数字参数常常充当重复次数。参见“数字参数”。

Overwrite Mode 覆盖模式是一种次模式。启用后，普通文本字符会替换光标点之后的现有文本，而不是把它推到一边。参见“次模式”。

Package 包是你在 Emacs 内部下载并自动安装的 Lisp 代码集合。包提供了一种添加新特性的方便方式。参见“Emacs Lisp 软件包”。

Page 页是一种文本单位，由位于行首的换页字符（ASCII 的 control-L，编码 014）分隔。Emacs 提供了一些用于跨页移动和对页进行操作的命令。参见“页”。

Paragraph 段落是人类语言文本中等大小的单位。Emacs 有专门的命令用于跨段落移动和对段落进行操作。参见“段落”。

Parsing 我们说某些 Emacs 命令会解析正在编辑的文本中的单词或表达式。实际上，它们所能做的只是找到单词或表达式的另一端。

Point 光标点是缓冲区中插入和删除发生的位置。光标点被认为位于两个字符之间，而不是在某个字符上。终端的光标（cursor，q.v.）指示光标点的位置。参见“光标点”。

Prefix Argument 参见“Glossary---Numeric Argument”。

Prefix Key 前缀键是其唯一功能为引出一组更长键序列的键序列（q.v.）。‘C-x’ 就是前缀键的一个例子；因此，任何以 ‘C-x’ 开头的两字符序列都是合法的键序列。参见“按键”。

Primary Selection 主选择区是一个特定的 X 选择区（selection，q.v.）；它是多数 X 应用程序用于与其他应用程序之间传输文本的选择区。

标记或选区文本的 Emacs 命令会设置主选择区；在适当的时候点击鼠标会插入主选择区中的文本。参见“Shift 选择”。

Prompt 提示是用于向你索取输入的文本。显示提示称为 prompting。Emacs 的提示总是出现在回显区（q.v.）。一种提示发生在使用迷你缓冲区读取参数时（参见“迷你缓冲区”）；当你在输入多字符键序列的中途停顿时所发生的回显也是一种提示（参见“回显区”）。

q.v. 拉丁语 “quod vide” 的缩写，意为“参见该条”。

Query-Replace 查询替换是 Emacs 提供的一种交互式字符串替换功能。参见“查询替换”。

Quitting 退出（quitting）是指用 ‘C-g’（在 MS-DOS 上为 ‘C-<Break>’）取消只输入了一部分的命令或正在运行的命令。参见“退出与中止”。

Quoting 引用是指剥夺字符通常具有的特殊意义。Emacs 中最常见的引用方式是 'C-q'。什么构成特殊意义取决于上下文和惯例。例如，普通字符作为 Emacs 命令时会插入自身；所以在该上下文中，特殊字符是指通常不会插入自身的任何字符（例如 <DEL>），引用它会使其像不特殊一样插入自身。并非所有上下文都允许引用。参见"插入文本"

Quoting File Names 引用文件名会关闭 '$'、'~' 和 ':' 等构造的特殊意义。参见"引用的文件名"。

Read-Only Buffer 只读缓冲区的文本不允许你更改。通常，当缓冲区包含对 Emacs 有特殊意义的文本时（例如 Dired 缓冲区），Emacs 会把它设为只读。访问写保护的文件也会产生只读缓冲区。参见"多缓冲区"。

Rectangle 矩形由若干行上某个列范围内的文本组成。通常，你把光标点放在一个角上，把标记放在对角上的另一个角，以此指定矩形。参见"矩形"。

Recursive Editing Level 递归编辑层级是这样一种状态：命令执行的一部分涉及请你编辑某些文本。该文本可能与命令所作用的文本相同，也可能不同。模式行（q.v.）用方括号（'[' 和 ']'）指示递归编辑层级。参见"递归编辑层级"。

Redisplay 重显示是修正屏幕上的图像以对应于正在编辑的文本中所发生改动的过程。参见"屏幕布局"

Regexp 参见"Glossary---Regular Expression"。

Region 区域是光标点（q.v.）与标记（q.v.）之间的文本。许多命令对区域中的文本进行操作。参见"标记与区域"

Register 寄存器是一些有名字的槽位，可以在其中保存文本、缓冲区位置或矩形以供以后使用。参见"寄存器"。Emacs 的一个相关特性是书签（bookmark，q.v.）。

Regular Expression 正则表达式是一种能匹配各种文本字符串的模式；例如，'a[0-9]+' 匹配 'a' 后跟一个或多个数字。参见"正则表达式的语法"。

Remote File 远程文件是存储在你自己的系统之外的其他系统上的文件。只要能从你的机器通过网络访问其他计算机，并且（显然）你有受支持的方法访问那些文件，Emacs 就能访问这些计算机上的文件。参见"远程文件"。

Repeat Count 参见"Glossary---Numeric Argument"。

Replacement 参见"Glossary---Global Substitution"。

Restriction 缓冲区的限制是指在缓冲区开头或结尾临时无法访问的那部分文本。给缓冲区设置非零的限制称为窄化（narrowing，q.v.）；移除限制称为解除窄化（widening，q.v.）。参见"窄化"。

<RET> <RET> 是一个字符，在 Emacs 中它运行向文本中插入换行的命令。它也用于终止在迷你缓冲区（见该词条）中读取的大多数参数。参见"输入类型"

Reverting（回退） 回退指回到原始状态。例如，Emacs 允许你从磁盘重新读取缓冲区对应的文件，从而回退该缓冲区。参见"还原缓冲区"。

Saving（保存） 保存缓冲区意味着把它的文本复制到该缓冲区所访问（见该词条）的文件中。你的 Emacs 编辑实际上就是这样改变文件中的文本的。参见"保存文件"。

Scroll Bar（滚动条） 滚动条是出现在窗口一侧的高而窄的空心框。你可以用滚动条中的鼠标命令滚动窗口。滚动条特性只在窗口系统下受支持。参见“滚动条”。

Scrolling（滚动） 滚动指移动 Emacs 窗口中的文本，以便看到缓冲区的另一部分。参见“滚动”。

Searching（搜索） 搜索指把光标点移到指定字符串的下一个出现处，或指定正则表达式的下一个匹配处。参见“搜索与替换”。

Search Path（搜索路径） 搜索路径是一个目录列表，用于为某些目的查找文件。例如，变量 ‘load-path’ 保存用于查找 Lisp 库文件的搜索路径。参见“Emacs 的 Lisp 代码库”。

Secondary Selection（次级选择） 次级选择是 X 的一种特定选择（见该词条）；某些 X 应用程序可用它在各应用程序之间传送文本。Emacs 有专门的鼠标命令，利用次级选择传送文本。参见“辅助选区”。

Selected Frame（选中的框架） 选中的框架是你当前输入所作用的框架。参见“框架与图形显示”。

Selected Window（选中的窗口） 选中的窗口是你当前输入所作用的窗口。参见“Emacs 窗口的概念”。

Selecting a Buffer（选择缓冲区） 选择缓冲区意味着使它成为当前（见该词条）缓冲区。参见“创建与选择缓冲区”。

Selection（选择） 窗口系统允许应用程序指定取值为文本的选择。程序也可以读取其他程序设置的选择。这是在窗口应用程序之间传送文本的主要方式。Emacs 有处理主选择（见该词条）、次级选择（见该词条）以及剪贴板（见该词条）的命令。

Self-Documentation（自文档） 自文档是 Emacs 的一项特性：它能告诉你任何命令做什么，或给出与你指定主题相关的所有命令的列表。你用帮助字符 ‘C-h’ 请求自文档。参见“帮助”。

Self-Inserting Character（自插入字符） 若键入某个字符会在缓冲区中插入该字符，则该字符是自插入的。在 Emacs 中，普通打印字符和空白字符都是自插入的，某些特殊主模式除外。

Sentences（句子） Emacs 有按句子移动或按句子删除的命令。参见“句子”。

Server（服务器） 在 Emacs 内，你可以启动一个“服务器”进程，它监听来自“客户端”的连接。相比启动多个 Emacs 实例，这提供了更快的替代方案。参见“把 Emacs 用作服务器”，以及参见“Glossary---Daemon”。

Sexp sexp（“s-expression”的缩写）是 Lisp 文本形式的基本句法单位：要么是列表，要么是 Lisp 原子。sexp 也是 Lisp 语言的平衡表达式（见该词条）；这就是编辑平衡表达式的命令名称中含有 ‘sexp’ 的原因。参见“平衡括号表达式”

Simultaneous Editing（同时编辑） 同时编辑指两个用户同时修改同一个文件。若未被检测到，同时编辑可能导致某个用户丢失自己的工作。Emacs 会检测所有同时编辑的情形，并警告其中一个用户去调查。参见“防止同时编辑”

<SPC> <SPC> 是空格字符，按空格键即可输入。

Speedbar（速度条） 速度条是一个特殊的高框架，提供对 Emacs 缓冲区、这些缓冲区中的函数、Info 节点以及 Emacs 中其他有趣文本部分的快速访问。参见“Speedbar 框架”。

Spell Checking（拼写检查） 拼写检查指检查文本中每个单词书写形式的正确性。Emacs 可以使用各种外部拼写检查程序，通过方便的用户界面检查缓冲区各部分的拼写。参见“拼写检查与纠正”。

String（字符串） 字符串是一种 Lisp 数据对象，包含一个字符序列。许多 Emacs 变量都打算以字符串为值。字符串的 Lisp 语法由字符串中的字符以及前后各一个 '"' 构成。作为字符串一部分的 '"' 必须写成 '\"'，作为字符串一部分的 '\' 必须写成 '\\'。所有其他字符（包括换行）只需写在字符串内即可；不过也允许像 C 中那样的反斜杠序列，例如用 '\n' 表示换行，或使用八进制字符码的 '\241'。

String Substitution（字符串替换） 参见"Glossary---Global Substitution"。

Symbol（符号） Emacs Lisp 中的符号是带名称的对象。该对象可以是变量（见该词条）、函数或命令（见该词条），也可以是字体（face，见该词条）。符号的名称用作该符号的打印表示。参见《Emacs Lisp 参考手册》的"Symbol Type"

Syntax Highlighting（语法高亮） 参见"Glossary---Font Lock"。

Syntax Table（语法表） 语法表告诉 Emacs 哪些字符是单词的一部分，哪些字符像圆括号那样彼此配对，等等。参见《Emacs Lisp 参考手册》的"Syntax Tables"

Super Super 是键盘输入字符可能具有的一种修饰位名。要使字符成为 Super 字符，按住 <SUPER> 键的同时输入它。这类字符的名字以 'Super-' 开头（通常简写为 's-'）。参见"修饰键"。

Suspending（挂起） 挂起 Emacs 指暂时停止它，并把控制权交还其父进程（通常是 shell）。与杀死作业（见该词条）不同，你之后可以恢复被挂起的 Emacs 作业，而不会丢失缓冲区、未保存的编辑、撤销历史等。参见"退出 Emacs"。

<TAB> <TAB> 是制表符。在 Emacs 中它通常用于缩进或补全。

Tab Bar（标签栏） 标签栏是 Emacs 框架顶部的一行标签。点击其中一个标签会切换到具名的持久窗口配置。参见"标签栏"。

Tab Line（标签行） 标签行是 Emacs 窗口顶部的一行标签。点击其中一个标签会切换窗口的缓冲区。参见"窗口标签行"。

Tag（标签） 标签是程序源码中的标识符。参见"查找标识符引用"。

Tags Table（标签表） 标签表是一个文件，充当标识符的索引：一个或多个其他文件中函数、宏、数据结构等的定义。参见"标签表"。

Termscript File（termscript 文件） termscript 文件记录 Emacs 发送给终端的所有字符。它用于追查 Emacs 重显示中的 bug。除非你要求，Emacs 不会生成 termscript 文件。参见"报告 Bug"。

Text（文本） "Text"有两种含义（*note Text::）：

- 由字符序列构成的数据，与二进制数、可执行程序等相对。Emacs 缓冲区的基本内容（除文本属性，见该词条之外）在这种意义上总是文本。
- 由书面的人类语言构成的数据（与程序相对），或遵循人类语言文体惯例的数据。

Text Terminal（文本终端） 文本终端（或称字符终端）是只能以字符为单位显示文本的显示器。此类终端无法控制它所显示的单个像素。Emacs 在文本终端上支持一部分显示特性。

Text Properties（文本属性） 文本属性是为缓冲区中特定字符记录的注解。缓冲区中的图像记录为文本属性；它们还指定格式信息。参见"编辑格式信息"。

Theme（主题） 主题是一组定制（见该词条），赋予 Emacs 特定的外观或行为。例如，你可以为喜爱的一组字体（face，见该词条）使用某个主题。

Tool Bar（工具栏） 工具栏是 Emacs 框架顶部的一行（有时多行）带图标的按钮。点击其中某个按钮会执行一个命令。你可以把它看作菜单栏（见该词条）的图形近亲。参见“工具栏”。还有一种窗口工具栏，行为类似，但位于 Emacs 窗口顶部。参见“窗口工具栏”。

Tooltips（工具提示） 工具提示是显示帮助回显（见该词条）文本的小窗口，它解释显示内容的各部分、列出可通过鼠标点击获得的有用选项等。参见“工具提示”。

Top Level（顶层） 顶层是 Emacs 的正常状态，此时你在编辑所访问文件的文本。只要你不在递归编辑层（见该词条）或迷你缓冲区（见该词条）中，也不在命令执行中途，就处于顶层。你可以通过中止（见该词条）和退出（见该词条）回到顶层。参见“退出与中止”。

Transient Mark Mode（瞬态标记模式） 标记（见该词条）和区域（见该词条）的默认行为——设置标记即激活它并高亮区域——称为瞬态标记模式。它默认启用。参见“禁用临时标记模式”。

Transposition（调换） 调换两段文本意味着把各自放到对方原先占据的位置。Emacs 有调换两个相邻的字符、单词、平衡表达式（见该词条）或行的命令（参见“调换文本”）。

Trash Can（回收站） 参见“Glossary---Deletion of Files”。

Truncation（截断） 在显示中截断文本行意味着省略一行中放不进显示该行的窗口右边界之内的任何文本。*note Truncation: Continuation Lines，以及参见“Glossary---Continuation Line”。

TTY 参见“Glossary---Text Terminal”。

Undoing（撤销） 撤销指使先前的编辑反向进行，恢复编辑会话中较早存在的文本。参见“撤销”。

Unix Unix 是一类历史悠久的多用户计算机操作系统。今天有多种实现。GNU 工程（见该词条）旨在开发一个完整的、作为自由软件（见该词条）的类 Unix 操作系统。

User Option（用户选项） 用户选项是为让你能通过把它设为新值来定制 Emacs 而存在的字体（face，见该词条）或变量（见该词条）。参见“简易自定义界面”。

Variable（变量） 变量是 Lisp 中可以存储任意值的对象。Emacs 把一些变量用于内部目的，另一些（称为“用户选项”，见该词条）则只是让你设置其值以控制 Emacs 的行为。你可能感兴趣的 Emacs 所用变量列在本手册的变量索引中（参见“Variable Index”）。关于变量的信息，见参见“变量”。

Version Control（版本控制） 版本控制系统跟踪源文件的多个版本。它们提供了比保留备份文件（见该词条）更强大的替代方案。参见“版本控制”。

Visiting（访问） 访问文件指把它的内容加载到缓冲区（见该词条）中，以便编辑。参见“访问文件”。

Whitespace（空白） 空白是任意一串连续的格式字符（空格、制表符、换行、退格等）。

Widening（解除窄化） 解除窄化是去除对当前缓冲区的任何限制（见该词条）；它是窄化（见该词条）的反面。参见“窄化”。

Window（窗口） Emacs 把框架（见该词条）划分为一个或多个窗口，其中每个窗口随时可以显示一个缓冲区（见该词条）的内容。关于 Emacs 如何使用屏幕的基本信息，见参见“屏幕布局”。关于控制窗口使用的命令，

见参见“多窗口”。其他一些编辑器用“window”一词指我们在 Emacs 中称为“框架”的东西。

Window System（窗口系统） 窗口系统是在图形显示器（见该词条）上运行的软件，它细分屏幕，使多个应用程序可以同时拥有各自的窗口。所有现代操作系统都包含窗口系统。

Word Abbrev（单词缩写） 参见“Glossary---Abbrev”。

Word Search（单词搜索） 单词搜索是搜索一个单词序列，把单词之间的标点视为无关紧要。参见“单词搜索”。

Yanking（粘贴） 粘贴指重新插入先前删除（见该词条）的文本。它可用于撤销误删，也可用于复制或移动文本。其他一些系统称之为“pasting”。参见“粘贴”。

致谢

许多人贡献了自由软件基金会所发布的 GNU Emacs 发行版中包含的代码。为表达对他们公共精神的感谢，我们在此按字母顺序列出那些编写了相当大部分代码的人。还有多得无法一一列举的人报告并修复了缺陷，并为 Emacs 的许多部分添加了特性。我们同样感谢他们的慷慨。

本列表意在提到每个主要包或特性的贡献者；如果你知道有被我们遗漏的人，请提交缺陷报告。更全面的信息可在 'ChangeLog' 文件中找到，其摘要见发行版中的 'etc/AUTHORS' 文件。

- Per Abrahamsen 编写了定制设施，以及 'double.el'（用于输入键盘上通常没有的重音字符）；'xt-mouse.el'（允许通过 Xterm 使用鼠标命令）；'gnus-cus.el'（实现 Gnus 的定制命令）；'gnus-cite.el'（针对新闻文章的引文解析设施）；'gnus-score.el'（Gnus 的评分）；'cpp.el'（根据预处理器条件隐藏或高亮 C 程序的某些部分）；以及 widget 库文件 'wid-browse.el'、'wid-edit.el'、'widget.el'。他还参与编写了 'gnus-soup.el'。

- Tomas Abrahamsson 编写了 'artist.el'，一个用鼠标或键盘按键制作 ASCII 艺术的包。

- Jay K. Adams 编写了 'jka-compr.el' 和 'jka-cmpr-hook.el'，为压缩文件提供自动解压与重新压缩。

- Michael Albinus 编写了 'dbus.el'（实现 D-Bus 消息总线协议的包）；'zeroconf.el'（浏览 Avahi 服务的模式）；'secrets.el'（访问钥匙环守护进程以存储机密数据的接口）；以及 'filenotify.el' 和相关的底层接口例程（用于监视文件状态的变化）。他与 Kai Großjohann 编写了 Tramp 包，该包使用 ssh、ftp 及其他网络协议提供透明的远程文件编辑。他与 Daniel Pittman 编写了 'tramp-cache.el'。

- Ralf Angeli 编写了 'scroll-lock.el'，一个次模式，它在缓冲区中上下移动时通过滚动窗口使光标点在垂直方向上保持固定。

- Aurélien Aptel 为 Emacs 添加了动态模块支持。Philipp Stephani 等人也参与了动态模块代码。

- Joe Arceneaux 编写了最初的文本属性实现，并实现了对 X11 的支持。

- Emil Åstŕom、Milan Zamaza 和 Stefan Bruda 编写了 'prolog.el'，一个编辑 Prolog（以及 Mercury）代码的模式。

- Miles Bader 编写了 'image-file.el'（访问图像文件的支撑代码）；'minibuf-eldef.el'（在适当时隐藏迷你缓冲区默认值的次模式）；'rfn-eshadow.el'（对 'read-file-name' 输入的遮蔽）；'mb-depth.el'（显示迷你缓冲区深度）；'button.el'（实现可点击按钮的库）；'face-remap.el'（更改各个缓冲区中默认字体（face）的包）；以及用于宏展开的 'macroexp.el'。他还参与了词法绑定代码早期版本的工作。

- David Bakhash 编写了 'strokes.el'，一个通过按特定图案移动鼠标来控制 Emacs 的模式。

- Juanma Barranquero 编写了 'emacs-lock.el'（基于 Tom Wurgler 的原始版本），它使在有价值缓冲区未保存时退出变得更加困难；以及 'frameset.el'，用于保存和恢复框架/窗口设置。他还在其他领域做了许多贡献，包括 MS Windows 支持。

- Eli Barzilay 编写了 'calculator.el'，一个用于 Emacs 的桌面计算器。

- Steven L. Baur 编写了 'footnote.el'，让你可以在电子邮件消息中包含脚注；以及 'gnus-audio.el' 和 'earcon.el'，为 Gnus 提供音效。他还编写了 'gnus-setup.el'。
- Alexander L. Belikoff、Sergey Berezin、Sacha Chua、David Edmondson、Noah Friedman、Andreas Fuchs、Mario Lang、Ben Mesander、Lawrence Mitchell、Gergely Nagy、Michael Olson、Per Persson、Jorgen Schäfer、Alex Schroeder 和 Tom Tromey 编写了 ERC，一个高级的 Internet Relay Chat 客户端（更多信息见 ERC 发行版中的 'CREDITS' 文件）。
- Scott Bender、Michael Brouwer、Christophe de Dinechin、Carl Edman、Christian Limpach 和 Adrian Robert 开发并维护了 Emacs 的 NeXTstep 移植。
- Stephen Berman 编写了 'todo-mode.el'（基于 Oliver Seidel 的原始版本），一个维护 'TODO' 列表文件的包。他还与 Stefan Monnier 一起编写了 'visual-wrap.el'。
- Anna M. Bigatti 编写了 'cal-html.el'，它生成 HTML 日历。
- Ray Blaak 和 Simon South 编写了 'opascal.el'，一个编辑 Object Pascal 源代码的模式。
- Martin Blais、Stefan Merten 和 David Goodger 编写了 'rst.el'，一个编辑 reStructuredText 文档的模式。
- Jim Blandy 编写了 Emacs 19 的输入系统，使其配置和构建过程符合 GNU 编码标准，并为框架支持和多字体（face）支持做出贡献。Jim 还编写了 'tvi970.el'（对 TeleVideo 970 终端的终端支持）；并参与编写了 'wyse50.el'（见该条）。
- Per Bothner 编写了 'term.el'，一个位于 Emacs 缓冲区中的终端模拟器。
- Terrence M. Brannon 编写了 'landmark.el'，一个学习地标的神经网络机器人。
- Frank Bresz 编写了 'diff.el'，一个显示 'diff' 输出的程序。
- Peter Breton 实现了 'dirtrack.el'（一个跟踪 shell 缓冲区中目录变化的库）；'filecache.el'（记录你的文件位于哪些目录）；'locate.el'（到 'locate' 命令的接口）；'find-lisp.el'（对 'find' 程序的 Emacs Lisp 模拟）；'net-utils.el'；以及通用模式（generic mode）特性。
- Emmanuel Briot 编写了 'xml.el'（一个用于 Emacs 的 XML 解析器）；以及 'ada-prj.el'（编辑 Ada 模式的项目文件），还合著了 'ada-mode.el' 和 'ada-xref.el'。
- Kevin Broadey 编写了 'foldout.el'，为 Emacs 的大纲模式提供折叠扩展。
- David M. Brown 编写了 'array.el'，用于编辑数组及其他表格数据。
- Włodek Bzyl 和 Ryszard Kubiak 编写了 'ogonek.el'，一个更改波兰字符编码的包。
- Bill Carpenter 提供了 'feedmail.el'，一个整理待发邮件消息并通过各种流行邮件程序发送它们的包。
- Per Cederqvist 和 Inge Wallin 编写了 'ewoc.el'，一个用于操作对象集合的 Emacs widget。Per Cederqvist、Inge Wallin 和 Thomas Bellman 编写了 'avl-tree.el'，用于平衡二叉树。

- Hans Chalupsky 编写了 'advice.el'（Emacs Lisp 函数的重载机制）；以及 'trace.el'，一个针对 Emacs Lisp 的跟踪设施。
- Chris Chase、Carsten Dominik 和 J. D. Smith 编写了 IDLWAVE 模式，用于编辑 IDL 和 WAVE CL。
- Bob Chassell 编写了 'texnfo-upd.el'、'texinfo.el' 和 'makeinfo.el'，它们是处理 Texinfo 文件的模式与实用工具；以及 'page-ext.el'，扩展的分页处理命令。他还编写了 Emacs Lisp 入门。参见《《eintr》手册》。
- Jihyun Cho 编写了 'hanja-util.el' 和 'hangul.el'，韩国汉字（Hanja）的实用工具。
- Andrew Choi 和 Yamamoto Mitsuharu 编写了 Carbon 支持，在 Emacs 23 之前用于 macOS。Yamamoto Mitsuharu 在较新的 Nextstep 移植中继续为 macOS 支持做贡献；并改进了对多显示器显示的支持。
- Chong Yidong 是 Emacs 23 至 24.3 期间的 Emacs 共同维护者。他对 Emacs 显示引擎做了许多改进。他还编写了 'tabulated-list.el'（一个用于数据列表的通用主模式）；并改进了对主题和包的支持。
- James Clark 编写了 SGML 模式，一个编辑 SGML 文档的模式；以及 nXML 模式，一个编辑 XML 文档的模式。他还对 Emacs 的转储（dumping）过程有所贡献。
- Mike Clarkson 编写了 'edt.el'，一个对 DEC 的 EDT 编辑器的模拟。
- Glynn Clements 提供了 'gamegrid.el' 以及两个使用它的游戏：Snake 和 Tetris。
- Andrew Cohen 编写了 'spam-wash.el'，用于在消息被分析是否为垃圾邮件之前对其进行解码和清理。
- Daniel Colascione 编写了 'pdumper.c' 及其他位置的可移植转储代码。他还为基于 X 的 GUI 框架实现了双缓冲，以及最初把共享对象模块加载进 Emacs 的支持。
- Theresa O'Connor 编写了 'json.el'，一个解析和生成 JSON 的文件。
- Andrea Corallo 从 29.3 起担任 Emacs 的（共同）维护者。他编写了 'comp.c' 和 'comp.el' 中的原生编译支持，用于借助 'libgccjit' 把 Emacs Lisp 编译为本机代码。
- Georges Brun-Cottan 和 Stefan Monnier 编写了 'easy-mmode.el'，一个便于定义主模式和次模式的包。
- Andrew Csillag 编写了 M4 模式（'m4-mode.el'）。
- Doug Cutting 和 Jamie Zawinski 编写了 'disass.el'，一个针对编译后的 Emacs Lisp 代码的反汇编器。
- Mathias Dahl 编写了 'image-dired.el'，一个以缩略图形式查看图像文件的包。
- Julien Danjou 编写了桌面通知的实现（'notifications.el'，以及针对 ERC 和 Gnus 的相关包）；以及 'color.el'，一个通用颜色操作库。他还对 Gnus 做出了各种贡献。
- Vivek Dasmohapatra 编写了 'htmlfontify.el'，用于把缓冲区或源代码树转换为 HTML。
- Matthieu Devin 编写了 'delsel.el'，一个让新键入的文本替换当前选区的包。
- Eric Ding 编写了 'goto-addr.el'。
- Jan Djärv 添加了对 GTK+ 工具包以及 X 拖放的支持。他还编写了 'dynamic-setting.el'。

- Carsten Dominik 编写了 RefTeX，一个在 LaTeX 文档中设置标签和交叉引用的包；并参与编写了 IDLWAVE 模式（见该条）。他是 Org 模式的原创作者，该模式用于维护笔记、待办列表和项目规划。Bastien Guerry 随后接任维护。Benjamin Andresen、Thomas Baumann、Joel Boehland、Jan Böcker、Lennart Borgman、Baoqiu Cui、Dan Davison、Christian Egli、Eric S. Fraga、Daniel German、Chris Gray、Konrad Hinsen、Tassilo Horn、Philip Jackson、Martyn Jago、Thorsten Jolitz、Jambunathan K、Tokuya Kameshima、Sergey Litvinov、David Maus、Ross Patterson、Juan Pechiar、Sebastian Rose、Eric Schulte、Paul Sexton、Ulf Stegemann、Andy Stewart、Christopher Suckling、David O'Toole、John Wiegley、Zhang Weize、Piotr Zieliński 等人也编写了 Org 模式的各种组件。更多信息参见《Org 手册》的“History and Acknowledgments”。
- Scott Draves 编写了 ‘tq.el’，用于维护 Emacs 与其子进程之间事务队列的辅助函数。
- Benjamin Drieu 编写了 ‘pong.el’，经典 pong 游戏的一个实现。
- Viktor Dukhovni 编写了在 SunOS 版本 4 下转储的支持。
- John Eaton 和 Kurt Hornik 编写了 Octave 模式。
- Rolf Ebert、Markus Heritsch 和 Emmanuel Briot 编写了 Ada 模式。
- Paul Eggert 集成了 Gnulib 可移植性库，并对 C 代码做了许多其他可移植性修复；此外还对 VC 和日历有所贡献。
- Stephen Eglen 编写了 ‘mspools.el’，它告诉你哪些 Procmail 文件夹中有邮件在等待。
- Torbjörn Einarsson 编写了 ‘f90.el’，一个针对 Fortran 90 文件的模式。
- Helmut Eller 编写了 ‘peg.el’，一个用于 Emacs 的解析表达式语法（PEG）包。
- Tsugutomo Enami 参与编写了国际字符集的支持。
- David Engster 编写了 ‘mairix.el’ 和 ‘nnmairix.el’，一个到 Mairix 索引工具的接口。
- Hans Henrik Eriksen 编写了 ‘simula.el’，一个编辑 SIMULA 87 代码的模式。
- Michael Ernst 编写了 ‘reposition.el’，一个把函数的源代码及其前面注释重新居中显示在屏幕上的命令。
- Ata Etemadi 编写了 ‘cdl.el’，处理 Common Data Language 源代码的函数。
- Frederick Farnbach 实现了 ‘morse.el’，它把文本转换为摩尔斯电码。
- Oscar Figueiredo 编写了 EUDC，即 Emacs 统一目录客户端（Emacs Unified Directory Client），它是通过 LDAP、CCSO PH/QI 或 BBDB 访问目录服务器的接口；以及 ‘ldap.el’，LDAP 客户端接口。
- Jared Finder 在 ‘window-tool-bar.el’ 中实现了窗口工具栏，并增强了 ‘xt-mouse.el’ 中的 xterm 鼠标支持。
- Fred Fish 编写了转储 COFF 可执行文件的支持。
- Karl Fogel 编写了 ‘bookmark.el’（实现有名字的位置占位符）；‘mail-hist.el’（针对外发邮件消息的历史机制）；以及 ‘saveplace.el’，用于在编辑会话之间保留光标点在文件中的位置。

- Gary Foster 编写了‘scroll-all.el’，一个同时滚动多个缓冲区的模式。
- Romain Francoise 贡献了 ACL（访问控制列表）支持，用于在备份和复制时保留扩展文件属性。
- Noah Friedman 编写了‘rlogin.el’（到 Rlogin 的接口）、‘type-break.el’（提醒你定期停止打字休息一下）以及‘eldoc-mode’，一个显示光标点附近 Lisp 函数已定义参数或文档字符串的模式。
- Shigeru Fukaya 编写了字节编译器的测试套件。
- Keith Gabryelski 编写了‘hexl.el’，一个编辑二进制文件的模式。
- Kevin Gallagher 重写并增强了 EDT 模拟，并编写了‘flow-ctrl.el’，一个应对无法抑制的 XON/XOFF 流控的包。
- Fabián E. Gallina 重写了‘python.el’，即 Emacs 24.3 起使用的 Python 编程语言主模式。
- Kevin Gallo 为 Windows NT 添加了多框架支持，并编写了‘w32-win.el’，MS-Windows 窗口系统的支撑函数。
- Juan León Lahoz García 编写了‘wdired.el’，一个通过直接编辑 Dired 缓冲区来执行文件操作的包。
- Howard Gayle 编写了显示表和大小写表（case table）的大部分 C 和 Lisp 代码。他还编写了‘rot13.el’（一个显示以凯撒密码编码的缓冲区的纯文本形式的命令）；‘vt100-led.el’（一个控制 VT100 兼容终端上 LED 的包）；以及 ISO-8859 欧洲字符集支持的大部分内容（其中包括‘iso-ascii.el’、‘iso-insert.el’、‘iso-swed.el’、‘iso-syntax.el’、‘iso-transl.el’和‘swedish.el’）。
- Stephen Gildea 制作了 Emacs 快速参考卡，并为‘time-stamp.el’（一个维护文件中最后修改时间戳的包）做出了许多贡献。
- Julien Gilles 编写了‘gnus-ml.el’，一个针对 Gnus 的邮件列表次模式。
- David Gillespie 编写了 Common Lisp 兼容包；‘Calc’，一个高级计算器与数学工具（此后由 Jay Belanger 维护和开发）；‘complete.el’，一个部分补全机制；以及‘edmacro.el’，一个编辑键盘宏的包。
- Bob Glickstein 编写了‘sregex.el’，一个用类似 Lisp 的语法编写正则表达式的设施。
- Boris Goldowsky 编写了‘avoid.el’（一个让鼠标光标避开文本光标的包）；‘shadowfile.el’（一个在多处保留文件相同副本的包）；‘format.el’（一个以各种格式读写文件的包）；‘enriched.el’（一个把文本属性保存在文件中的包）；‘facemenu.el’（一个指定字体（face）的包）；以及‘descr-text.el’，描述文本和字符属性。
- Michelangelo Grigni 编写了‘ffap.el’，它从缓冲区中取出文件名来访问文件。
- Odd Gripenstam 编写了‘dcl-mode.el’，用于编辑 DCL 命令文件。
- Michael Gschwind 编写了‘iso-cvt.el’，一个在 ISO 8859-1 字符集与 TeX 和网络传统所使用的非 ASCII 字符表示法之间转换的包。
- Bastien Guerry 编写了‘gnus-bookmark.el’，为 Gnus 提供书签支持；并帮助维护 Org 模式（见该条）。
- Henry Guillaume 编写了‘find-file.el’，一个访问与当前所访问文件相关的文件的包。

- Dmitry Gutov 编写了‘etags-regen.el’，一个自动（重新）生成‘TAGS’文件的包。
- Doug Gwyn 编写了可移植的‘alloca’实现。
- Ken'ichi Handa 实现了国际字符集支持的大部分内容，并编写了 Emacs 23 字体处理代码的大部分。他还编写了‘composite.el’（提供一个在显示字符时自动组合字符的次模式）；‘isearch-x.el’（一个搜索非 ASCII 文本的设施）；以及‘ps-bdf.el’，一个在 PostScript 打印机上打印非 ASCII 文本的 BDF 字体支持。他与 Naoto Takahashi 一起编写了‘quail.el’，一个从 ASCII 键盘键入非 ASCII 文本的输入设施。
- Jesper Harder 编写了‘yenc.el’，用于解码 yenc 编码的消息。
- Alexandru Harsanyi 编写了一个访问 SOAP Web 服务的库。
- K. Shane Hartman 编写了‘chistory.el’和‘echistory.el’（浏览命令历史列表的包）；‘electric.el’和‘helper.el’（提供另一种命令循环及相应的帮助设施）；‘emacsbug.el’（一个报告 Emacs 缺陷的包）；‘picture.el’（一个编辑 ASCII 图片的模式）；以及‘view.el’，一个不编辑而只浏览文件和缓冲区的包。
- John Heidemann 编写了‘mouse-copy.el’和‘mouse-drag.el’，它们提供基于鼠标的替代编辑和滚动特性。
- Jon K Hellan 编写了‘utf7.el’，对 Unicode 的邮件安全转换格式的支持。
- Karl Heuer 编写了最初的 blessmail 脚本，实现了‘intangible’文本属性，并重新安排了‘Lisp_Object’类型的结构以容纳更多数据位。
- Manabu Higashida 把 Emacs 移植到了 MS-DOS。
- Anders Holst 编写了‘hippie-exp.el’，一个多用途的补全与扩展包。
- Tassilo Horn 编写了 DocView 模式，用于查看 PDF、PostScript 和 DVI 文档。
- Khaled Hosny、Yamamoto Mitsuharu 和 Eli Zaretskii 为 Emacs 实现了使用 HarfBuzz 的文本整形。
- Joakim Hove 编写了‘html2text.el’，一个 HTML 到纯文本的转换器。
- Denis Howe 编写了‘browse-url.el’，一个调用 WWW 浏览器显示 URL 的包。
- Lars Magne Ingebrigtsen 是 Emacs 27.2 至 29.1 期间的 Emacs（共同）维护者。他对 Gnus 新闻阅读器做了重大重新设计，并编写了它的许多部分。其中若干部分现在是 Emacs 的通用组件，包括：用于域名服务查询的‘dns.el’；用于格式化任意格式字符串的‘format-spec.el’；用于解析‘.netrc’文件的‘netrc.el’；以及用于通用日期和时间处理的‘time-date.el’。他还编写了‘network-stream.el’（用于打开网络进程）；‘url-queue.el’（用于控制 URL 的并行下载）；并实现了 libxml2 支持。他还编写了‘eww.el’，一个 Emacs Lisp Web 浏览器；并实现了原生 zlib 解压。Gnus 的组件也曾由以下人员编写：Nagy Andras、David Blacka、Scott Byer、Ludovic Courtès、Julien Danjou、Kevin Greiner、Kai Großjohann、Joe Hildebrand、Paul Jarc、Simon Josefsson、Sascha Lüdecke、David Moore、Jim Radford、Benjamin Rutt、Raymond Scholz、Thomas Steffen、Reiner Steib、Jan Tatarik、Didier Verna、Ilja Weis、Katsumi Yamaoka、Teodor Zlatanov 等人（参见《Gnus 手册》的“Contributors”）。

- Andrew Innes 对 MS-Windows 支持做出了大量贡献。
- Seiichiro Inoue 改进了 Emacs 的 XIM 支持。
- Philip Jackson 编写了 'find-cmd.el'，用于构建 'find' 命令行。
- Ulf Jasper 编写了 'icalendar.el'（一个在 Emacs 日记条目与 iCalendar 格式之间转换的包）；'newsticker.el'（一个基于 RSS 和 Atom 的新闻阅读器）；以及 'bubbles.el'，一个益智游戏。
- Kyle Jones 编写了 'life.el'，一个玩 Conway 生命游戏的包。
- Terry Jones 编写了 'shadow.el'，一个在某个 Lisp 文件遮蔽另一个文件时查找潜在 load-path 问题的包。
- Simon Josefsson 编写了 'dns-mode.el'（一个编辑域名系统主文件的编辑模式）；'dig.el'（一个域名系统接口）；'flow-fill.el'（一个解释消息中 RFC2646 格式文本的包）；'fringe.el'（一个定制边缘（fringe）的包）；'imap.el'（一个与 IMAP 服务器交谈的 Emacs Lisp 库）；'password-cache.el'（一个密码读取器）；'nnimap.el'（Gnus 的 IMAP 后端）；用于 URL 库的 'url-imap.el'；'rfc2104.el'（一个散列消息认证设施）；Gnus 的 S/MIME 和 Sieve 组件；以及用于传输层安全协议的 'tls.el' 和 'starttls.el'。
- Arne Jørgensen 编写了 'latexenc.el'，一个自动猜测 LaTeX 文件中正确编码系统的包。
- Rahul Martim Juliato 和 Stéphane Marks 编写了 'markdown-ts-mode.el' 和 'markdown-ts-mode-x.el'，一个编辑 Markdown 文件的主模式。
- Alexandre Julliard 编写了 'vc-git.el'，对 Git 版本控制系统的支持。
- Tomoji Kagatani 实现了 'smtpmail.el'，用于通过 SMTP 发送邮件。
- Stefan Kangas 从 29.2 起担任 Emacs 的（共同）维护者。
- Ivan Kanis 编写了 'vc-hg.el'，对 Mercurial 版本控制系统的支持。
- Henry Kautz 编写了 'bib-mode.el'（一个维护与 'refer'（'troff' 版本）和 'lookbib' 兼容的参考文献数据库的模式），以及 'refbib.el'，一个把这些数据库转换为 LaTeX 文本格式化包所用格式的包。
- Taichi Kawabata 添加了对天城文（Devanagari）和印度语言的支持，并编写了 'ucs-normalize.el' 用于 Unicode 规范化。
- Taro Kawagishi 用 Lisp 实现了 MD4 消息摘要算法；并编写了 'ntlm.el' 和 'sasl-ntlm.el'，用于支持 NT LanManager 认证。
- Howard Kaye 编写了 'sort.el'，用于对 Emacs 缓冲区中的文本排序的命令。
- Michael Kifer 编写了 'ediff'，一个到 'diff'、'patch' 和 'merge' 程序的交互式接口；以及 Viper，一个 VI 编辑器的模拟器。
- Richard King 编写了 'userlock.el' 和 'filelock.c' 的第一个版本，它们为多个用户编辑同一文件提供简单支持。他还编写了 'uniquify.el' 的初始版本，一个通过在缓冲区名中加入文件名的一部分来使缓冲区名唯一的设施。

- Wilhelm H Kirschbaum 编写了 'elixir-ts-mode.el' 和 'heex-ts-mode.el'，这是用于编辑 Elixir 和 Heex 文件的 Emacs 主模式。
- Peter Kleiweg 编写了 'ps-mode.el'，该模式用于编辑 PostScript 文件，并可在 Emacs 中交互式运行 PostScript 解释器。
- Karel Klíč 贡献了对 SELinux 的支持，用于在备份和复制时保留文件的安全增强 Linux（Security-Enhanced Linux）上下文。
- Shuhei Kobayashi 编写了 'hex-util.el'，用于处理十六进制字符串；并提供了对 HMAC（Keyed-Hashing for Message Authentication，带密钥的散列消息认证）的支持。
- Pavel Kobyakov 编写了 'flymake.el'，这是一个即时进行语法检查的次模式。
- David M. Koppelman 编写了 'hi-lock.el'，这是一个对缓冲区文本的某些部分进行交互式自动高亮的次模式。
- Koseki Yoshinori 编写了 'iimage.el'，这是一个显示内嵌图片的次模式。
- Robert Krawitz 编写了最初的 'xmenu.c'，它是 Emacs 弹出菜单支持的一部分。
- Sebastian Kremer 编写了 'dired-mode'，Lawrence R. Dodd 也做出了贡献。他还编写了 'ls-lisp.el'，这是在未把 'ls' 作为标准程序的平台上对 'ls' 命令的 Lisp 模拟。
- David Kågedal 编写了 'tempo.el'，为方便地插入样板文本和其他常见构造提供支持。
- Igor Kuzmin 编写了 'cconv.el'，为具有静态作用域的 Emacs Lisp 提供闭包转换。
- Daniel LaLiberte 编写了 'edebug.el'，这是 Emacs Lisp 的源码级调试器；'cl-specs.el'，用于帮助 'edebug' 调试以 David Gillespie 的 Common Lisp 支持所写代码的规范；以及 'isearch.el'，Emacs 的增量搜索次模式。他还参与编写了 'hideif.el'（见该条）。
- Karl Landstrom 和 Daniel Colascione 编写了 'js.el'，这是一个用于编辑 JavaScript 的模式。
- Vinicius Jose Latorre 编写了 Emacs 的打印功能，以及与 Jim Thompson、Jacques Duthen 和 Kenichi Handa 合写的 'ps-print'——一个把 Emacs 缓冲区漂亮地打印到 PostScript 打印机的软件包；'delim-col.el'，一个把文本排成列的软件包；'ebnf2ps.el'，一个把 EBNF 文法转换为可打印到 PostScript 打印机的语法图的软件包；以及 'whitespace.el'，一个检测并清理文件中多余空白的软件包（基于 Rajesh Vaidheeswarran 的早期版本）。
- Richard Lawrence 编写了 'diary-icalendar.el'，这是一个处理 iCalendar（RFC 5545）数据的库。
- Frederic Lepied 编写了 'expand.el'，它利用缩写机制插入编程构造。
- Peter Liljenberg 编写了 'elint.el'，这是一个针对 Emacs Lisp 程序的 Lint 风格代码检查器。
- Lars Lindberg 编写了 'msb.el'，它为缓冲区选择提供了更灵活的菜单；参与编写了 'imenu.el'（见该条）；并重写了最初由 Don Morrison 编写的 'dabbrev.el'。

- Anders Lindgren 编写了‘autorevert.el’，一个自动还原 Emacs 所访问但在磁盘上已被修改的文件的软件包；‘cwarn.el’，一个高亮可疑的 C 和 C++ 构造的软件包；以及‘follow.el’，一个使显示同一缓冲区的窗口保持同步的次模式。
- Thomas Link 编写了‘filesets.el’，这是一个处理文件集合的软件包。
- Juri Linkov 编写了‘misearch.el’，把 isearch 扩展为多缓冲区搜索；‘files-x.el’中处理文件和目录局部变量的代码；以及‘info-finder’特性，它创建以软件包关键字为索引的虚拟 Info 手册。他还实现了标签栏（Tab Bar）和窗口标签行，并为 I-search 增加了大量增强和改进。
- Leo Liu 编写了‘pcmpl-x.el’，为各种外部工具提供补全；并在 Emacs 24.4 中改造了对 Octave 的支持。
- Károly Lőrentey 编写了多终端代码，使 Emacs 能够同时运行在图形终端和文本终端上。
- Martin Lorentzon 编写了‘vc-annotate.el’，即对版本控制注释的支持。
- Dave Love 编写了处理 Unicode 支持和 Latin-N 统一的大量代码。他增加了对许多编码系统的支持，包括各种 UTF-7 和 UTF-16 编码系统。他还编写了‘autoarg-mode’，一个用数字键提供前缀参数的全局次模式；‘autoarg-kp-mode’，它把小键盘数字键重新定义为数字参数；‘autoconf.el’，一个编辑 Autoconf 文件的模式；‘cfengine.el’，一个编辑 Cfengine 文件的模式；‘elide-head.el’，一个从文件头部省略样板文本的软件包；‘hl-line.el’，一个高亮当前窗口中光标点所在行的次模式；‘cap-words.el’，一个在‘CapitalizedWordIdentifiers’中移动的次模式；‘latin1-disp.el’，一个通过设置适当的显示表，使你能在 Latin-1 终端上显示 ISO 8859 字符的软件包；Emacs 24.3 之前使用的‘python.el’版本；‘smiley.el’，显示笑脸的设施；‘sym-comp.el’，一个执行依赖于模式的符号补全的库；用于对代码执行计时的‘benchmark.el’；以及‘tool-bar.el’，一个控制 Emacs 工具栏显示的次模式。他与 Riccardo Murri 合写了‘vc-bzr.el’，即对 Bazaar 版本控制系统的支持。
- Eric Ludlam 编写了 Speedbar 软件包；‘checkdoc.el’，用于检查 Emacs Lisp 程序中的文档字符串；‘dframe.el’，提供专用框架支持模式；‘ezimage.el’，一种在文本上放置图像的通用方法；‘chart.el’，用于绘制条形图等；以及 EIEIO（Enhanced Implementation of Emacs Interpreted Objects，Emacs 解释对象增强实现）软件包。他还是 CEDET（Collection of Emacs Development Environment Tools，Emacs 开发环境工具集）软件包的主要作者。部分内容也由 Jan Moringen、David Ponce 和 Joakim Verona 编写。
- Roland McGrath 编写了‘compile.el’（后来由 Daniel Pfeiffer 更新），一个在缓冲区中运行编译、随后访问错误消息中所报告位置的软件包；‘etags.el’，一个跳转到函数定义并在‘TAGS’文件中提到的所有文件里搜索或替换的软件包；与 Sebastian Kremer 合写了‘find-dired.el’，用于对‘find’程序的输出使用‘dired’命令；‘grep.el’，用于运行‘grep’命令；‘map-ynp.el’，一个通用的布尔问题询问器；‘autoload.el’，提供自动加载文件的半自动维护。
- Alan Mackenzie 编写了 CC Mode 中集成的 AWK 支持，并从 Emacs 22 起维护 CC Mode。
- Michael McNamara 和 Wilson Snyder 编写了 Verilog 模式。
- Christopher J. Madsen 编写了‘decipher.el’，这是一个破解简单替换密码的软件包。

- Neil M. Mager 编写了 'appt.el'，其中的函数用于提醒用户其约会。它会从 'calendar' 软件包所使用的日记文件中找出记录的约会。
- Ken Manheimer 编写了 'allout.el'，一个操作和格式化大纲的模式，以及 'icomplete.el'，它在迷你缓冲区中提供增量补全反馈。
- Bill Mann 编写了 'perl-mode.el'，这是一个编辑 Perl 代码的模式。
- Brian Marick 和 Daniel LaLiberte 编写了 'hideif.el'，即对隐藏 C '#ifdef' 子句中选定代码的支持。
- Stéphane Marks 编写了 'system-sleep.el' 和 'system-taskbar.el'，这些库用于与操作系统的睡眠/唤醒和任务栏设施交互。
- Simon Marshall 编写了 'regexp-opt.el'，它从字符串列表生成正则表达式；以及 fast-lock 和 lazy-lock 这两个字体锁定（font-lock）支持模式。他还扩展了最初由 Olin Shivers 编写的 'comint.el' 和 'shell.el'。
- Bengt Martensson、Dirk Herrmann、Marc Shapiro、Mike Newton、Aaron Larson 和 Stefan Schoef 编写了 'bibtex.el'，这是一个编辑 BibTeX 文献数据库文件的模式。
- Charlie Martin 编写了 'autoinsert.el'，它提供根据模式自动向新文件插入文本的功能。
- Yukihiro Matsumoto 和 Nobuyoshi Nakada 编写了 Ruby 模式。
- Tomohiro Matsuyama 编写了原生 Elisp 性能分析器。
- Thomas May 编写了 'blackbox.el'，即传统黑箱游戏的一个版本。
- David Megginson 编写了 'derived.el'，它允许通过继承现有主模式的键绑定和命令来定义新的主模式。
- Jimmy Aguilar Mena 编写了支持 ':extend' 字体（face）属性的代码，还实现了可选的 'display-fill-column-indicator' 特性。
- Will Mengarini 编写了 'repeat.el'，这是一个带参数重复前一命令的命令。
- Richard Mlynarik 编写了 'cl-indent.el'，一个缩进 Common Lisp 代码的软件包；'ebuff-menu.el'，一个缓冲区列表的电击式浏览器；'ehelp.el'，浏览帮助屏幕的键绑定；以及 'rfc822.el'，一个解析邮件消息和新闻文章中所用格式（Internet RFC 822 及其后续标准）电子邮件地址的解析器。
- Bret Mogilefsky、Christian Vogler、Juergen Hoetzel 和 Reuben Thomas 编写了 'lua-mode.el'，这是一个用于编辑 Lua 程序的 Emacs 模式。
- Gerd Möllmann 从 Emacs 21 开发之初直到 21.1 发布期间担任 Emacs 维护者。他编写了从 Emacs 21 起使用的新显示引擎，以及异步计时器设施。他还编写了 'ebrowse'，即 C++ 浏览器；'jit-lock.el'，即即时（Just-In-Time）字体锁定支持模式；'tooltip.el'，一个显示工具提示的软件包；'authors.el'，一个维护 'AUTHORS' 文件的软件包；以及 'rx.el'，一个正则表达式构造器。
- Stefan Monnier 从 Emacs 23 起直到 25.1 开发后期担任 Emacs（联合）维护者。他给 VC 增加了对 Arch 和 Subversion 的支持，重写了 Emacs 服务器的大部分内容以使用内建网络原语，并为 Emacs 23 重写了缩写和迷你缓冲区补全代码。他还编写了 'PCL-CVS'，一个 CVS 版本控制系统的目录级前端；'reveal.el'，一个自

动显示不可见文本的次模式；‘smerge-mode.el’，一个解决‘diff3’冲突的次模式；‘diff-mode.el’，一个查看和编辑上下文差异的模式；用于层叠样式表的‘css-mode.el’；用于 BibTeX 样式文件的‘bibtex-style.el’；‘mpc.el’，一个 Music Player Daemon（MPD）客户端；‘smie.el’，一个通用的缩进引擎；以及‘pcase.el’，实现了 ML 风格的模式匹配。在 Emacs 24 中，他整合了词法绑定代码，清理了 CL 命名空间（使得在运行时使用 CL 函数成为可接受的），为核心 Emacs Lisp 增加了广义变量，并实现了一种新的轻量级建议（advice）机制。他还编写了‘track-changes.el’，这个库在缓冲区中累积变更，直到某个客户端 Lisp 程序想要消费这些变更。

- Morioka Tomohiko 编写了若干软件包，用于 Gnus 及其他地方的 MIME 支持。
- John Muhl 编写了‘lua-ts-mode.el’，这是一个用于编辑 Lua 程序的 Emacs 模式。
- Sen Nagata 编写了‘crm.el’，一个通过补全读取多个字符串的软件包，以及‘rfc2368.el’，即对‘mailto:’URL 的支持。
- Erik Naggum 编写了时间转换函数。他还编写了‘disp-table.el’，用于处理显示表；‘mailheader.el’，用于解析电子邮件头部；以及‘parse-time.el’，用于解析时间字符串。
- Takahashi Naoto 参与编写了‘quail.el’（见该条），并编写了‘robin.el’，这是另一种输入法。
- Thomas Neumann 和 Eric S. Raymond 编写了‘make-mode.el’，这是一个编辑 makefile 的模式。
- Thien-Thi Nguyen 编写了‘xpm’、‘gnugo’和‘ascii-art-to-unicode’软件包。他还对许多其他软件包做出了大量贡献，例如‘vc.el’。
- Thien-Thi Nguyen 和 Dan Nicolaescu 编写了‘hideshow.el’，这是一个有选择地显示文本块的次模式。
- Dan Nicolaescu 增加了把 Emacs 作为守护进程运行的支持。他还编写了‘romanian.el’，即编辑罗马尼亚语文本的支持；‘iris-ansi.el’，即在 SGI 的‘xwsh’和‘winterm’终端模拟器上运行 Emacs 的支持；以及‘vc-dir.el’，用于显示受版本控制目录的状态。
- Hrvoje Nikšić 编写了‘savehist.el’，用于在 Emacs 会话之间保存迷你缓冲区历史。
- Jeff Norden 编写了‘kermit.el’，一个帮助 Kermit 拨号通信程序在 Emacs shell 缓冲区中舒适运行的软件包。
- Andrew Norman 编写了‘ange-ftp.el’，提供透明的 FTP 支持。
- Kentaro Ohkouchi 创建了从 Emacs 23 起使用的 Emacs 图标。
- Christian Ohler 编写了‘ert.el’，这是一个自动化回归测试的库。
- Alexandre Oliva 编写了‘gnus-mlspl.el’，一种基于组参数的邮件拆分机制。
- Takaaki Ota 编写了‘table.el’，这是一个创建和编辑嵌入式文本表格的软件包。
- Pieter E. J. Pareit 编写了‘mixal-mode.el’，这是一个 MIX 汇编语言的编辑模式。
- David Pearson 编写了‘quickurl.el’，一种根据光标点处文本把 URL 插入当前缓冲区的简单方法；‘5x5.el’，一个填满方格盘上所有方格的游戏。

- Jeff Peck 编写了 'sun.el'，即 sunterm 按键的键绑定。
- Damon Anton Permezel 编写了 'hanoi.el'，即汉诺塔谜题的动画演示。
- William M. Perry 编写了 'mailcap.el'（与 Lars Magne Ingebrigtsen 合写），一个 MIME 媒体类型配置设施；'mwheel.el'，一个支持鼠标滚轮的软件包；与 Dave Love 合写了 'socks.el'，一个 Socks v5 客户端；并开发了 URL 软件包。
- Per Persson 编写了 'gnus-vm.el'，即 Gnus 的 VM 接口。
- Jens Petersen 编写了 'find-func.el'，它使查找 Emacs Lisp 函数或变量的源代码变得容易。
- Nicolas Petton 编写了 'map.el'，一个提供可作用于关联列表、哈希表和数组的映射操作函数的库；'seq.el'，一个提供高级序列操作函数和宏的库；以及 'thunk.el'，一个提供延迟求值表达式的函数和宏的库。他还创建了 Emacs 25 中的新图标。
- Daniel Pfeiffer 编写了 'conf-mode.el'，一个编辑配置文件的模式；'copyright.el'，一个更新文件中版权声明的软件包；'executable.el'，一个执行解释器脚本的软件包；'sh-script.el'，一个编辑 shell 脚本的模式；'skeleton.el'，实现一种用于编写语句骨架的简洁语言；以及 'two-column.el'，一个同时进行双栏编辑的次模式。

Daniel 还重写了 'apropos.el'（最初由 Joe Wells 编写），用于查找匹配某个正则表达式的命令、函数和变量；并与 Jim Blandy 合写了 'wyse50.el'，即对 Wyse 50 终端的支持。他还参与编写了 'compile.el'（见该条）和 'ada-stmt.el'。

- Richard L. Pieri 编写了 'pop3.el'，这是 Emacs 的邮局协议（RFC 1460）接口。
- Fred Pierresteguy 和 Paul Reilly 使 Emacs 能与 X Toolkit 部件协同工作。
- François Pinard、Greg McGary 和 Bruno Haible 编写了 'po.el'，即对 PO 翻译文件的支持。
- Christian Plaunt 编写了 'soundex.el'，即 Soundex 算法的一种实现，用于按发音比较英语单词。
- David Ponce 编写了 'recentf.el'，一个把最近访问文件菜单放入 Emacs 菜单栏的软件包；'ruler-mode.el'，一个在标题行中显示标尺的次模式；以及 'tree-widget.el'，一个显示层次数据结构的软件包。
- Francesco A. Potortì 编写了 'cmacexp.el'，提供一个对文件某区域运行 C 预处理器并显示结果的命令。他还扩展并重新设计了 'etags' 程序。
- Michael D. Prange 和 Steven A. Wood 编写了 'fortran.el'，这是一个编辑 Fortran 代码的模式。
- Vincenzo Pupillo 编写了 'mhtml-ts-mode.el'，一个用于编辑 HTML 文件的主模式，以及 'php-ts-mode.el'，一个用于编辑 PHP 程序的主模式。
- Ashwin Ram 编写了 'refer.el'，其中的命令可按关键字在文献文件中查找参考文献。
- Eric S. Raymond 与 Paul Eggert 合写了 'vc.el'，即 RCS 和 SCCS 源代码版本控制系统的接口；'gud.el'，一个在 Emacs 中运行 GDB、SDB 等源码级调试器的软件包；'asm-mode.el'，一个编辑汇编语言代码的模式；'AT386.el'，IBM AT 键盘的终端支持软件包；'cookie1.el'，对 'spook.el' 之类幸运饼干程序的支

持；‘finder.el’，一个按关键字和主题查找 Emacs Lisp 软件包的软件包；‘keyswap.el’，交换 <BS> 和 <DEL> 键的代码；‘loadhist.el’，加载和卸载 Emacs 特性的函数；‘lisp-mnt.el’，处理 Emacs Lisp 库文件中特殊头部的函数；以及设置和使用 ‘load-history’ Lisp 变量的代码，该变量记录加载到 Emacs 中的每个 Lisp 函数来自哪个源文件。

- Edward M. Reingold 编写了日历和日记支持，Stewart Clamen（‘cal-mayan.el’）、Nachum Dershowitz（‘cal-hebrew.el’）、Paul Eggert（‘cal-dst.el’）、Steve Fisk（‘cal-tex.el’）、Michael Kifer（‘cal-x.el’）、Lara Rios（‘cal-menu.el’）和 Denis B. Roegel（‘solar.el’）也做出了贡献。Andy Oram 为其文档做出了贡献。Reingold 还对 ‘tex-mode.el’ 做出了贡献，这是一个编辑 TeX 文件的模式，William F. Schelter、Dick King、Stephen Gildea、Michael Prange 和 Jacob Gore 同样如此。
- David Reitter 编写了 ‘mailclient.el’，它可以通过系统指定的邮件客户端发送邮件。
- Alex Rezinsky 编写了 ‘which-func.el’，这是一个在模式行中显示当前函数名称的模式。
- Rob Riepel 编写了 ‘vt-control.el’，为 DEC VT 系列终端提供了一些控制功能。
- Nick Roberts 编写了 ‘t-mouse.el’，用于文本终端中的鼠标支持；以及 ‘gdb-ui.el’，即 GDB 的图形用户界面。他与 Dmitry Dzhus 合写了 ‘gdb-mi.el’，它是 ‘gdb-ui.el’ 的后继者。
- Danny Roozendaal 实现了 ‘handwrite.el’，它把文本转换成“手写体”。
- Markus Rost 编写了 ‘cus-test.el’，这是一个用于 customize 的测试框架。
- Guillermo J. Rozas 编写了 ‘scheme.el’，这是一个编辑 Scheme 和 DSSSL 代码的模式。
- Martin Rudalics 在 Emacs 24 中实现了改进的 display-buffer 处理；并实现了按像素调整窗口和框架大小。
- Ivar Rummelhoff 编写了 ‘winner.el’，它记录最近的窗口配置，以便你能回到这些配置。
- Jason Rumney 把 Emacs 21 的显示引擎移植到了 MS-Windows，并为 Emacs 的 MS-Windows 移植版做出了大量贡献。
- Wolfgang Rupprecht 编写了 Emacs 19 的浮点支持（包括 ‘float-sup.el’ 和 ‘floatfns.c’）。
- Kevin Ryde 编写了 ‘info-xref.el’，这是一个检查 Info 文件中引用的库。
- Phil Sainty 编写了 ‘so-long.el’，这是一组便于编辑包含超长行的文件的功能。
- James B. Salem 和 Brewster Kahle 编写了 ‘completion.el’，提供动态单词补全。
- Holger Schauer 编写了 ‘fortune.el’，一个在消息签名中使用 fortune 的软件包。
- William Schelter 编写了 ‘telnet.el’，即在 Emacs 内支持 ‘telnet’ 会话。
- Ralph Schleicher 编写了 ‘battery.el’，一个显示笔记本电脑电池状态的软件包，以及 ‘info-look.el’，一个为缓冲区中的符号查找 Info 文档的软件包。
- Michael Schmidt 和 Tom Perrine 编写了 ‘modula2.el’，这是一个编辑 Modula-2 代码的模式，基于 Mick Jordan 和 Peter Robinson 的工作。

- Ronald S. Schnell 编写了‘dunnet.el’，一个文字冒险游戏。
- Philippe Schnoebelen 编写了‘gomoku.el’，一个与 Emacs 对弈的五子棋游戏；以及‘mpuz.el’，一个乘法谜题。
- Jan Schormann 编写了‘solitaire.el’，即纸牌接龙游戏的一种实现。
- Alex Schroeder 编写了‘ansi-color.el’，一个把 ANSI 颜色转义序列转换为 Emacs 字体（face）的软件包；‘sql.el’，一个在 Emacs 缓冲区中交互式运行 SQL 解释器的软件包；‘cus-theme.el’，自定义主题的接口；‘master.el’，一个使某个缓冲区成为另一个缓冲区“主”缓冲区的软件包；以及‘spam-stat.el’，用于统计检测垃圾邮件。他还编写了 IRC 客户端 ERC（见该条）的部分内容。
- Randal Schwartz 编写了‘pp.el’，这是一个 Lisp 对象的漂亮打印器。
- Daniel Semyonov 编写了‘nnatom.el’，即 Gnus 的 Atom 后端，以及‘nnfeed.el’，即 Gnus 的通用 feed 后端。
- Manuel Serrano 编写了 Flyspell 软件包，它在键入时进行拼写检查。
- Hovav Shacham 编写了‘windmove.el’，这是一组根据窗口在框架上的几何位置选择窗口的命令。
- Stanislav Shalunov 编写了‘uce.el’，用于回复不请自来的商业电子邮件。
- Richard Sharman 编写了‘hilit-chg.el’，它用颜色显示最近的编辑改动。
- Olin Shivers 编写了‘comint.el’，一个供运行面向交互式命令行的子进程的模式使用的库，以及‘shell.el’，用于运行下级 shell（两者后来都由 Simon Marshall 扩展）；‘cmuscheme.el’，用于运行下级 Scheme 进程；‘inf-lisp.el’，用于运行下级 Lisp 进程。
- Espen Skoglund 编写了‘pascal.el’，这是一个编辑 Pascal 代码的模式。
- Rick Sladkey 编写了‘backquote.el’，这是一个创建大部分为常量数据的 Lisp 宏。
- Lynn Slater 编写了‘help-macro.el’，这是一个为键绑定编写交互式帮助的宏。
- Chris Smith 编写了‘icon.el’，这是一个编辑 Icon 代码的模式。
- David Smith 编写了‘ielm.el’，这是一个以子进程方式与 Emacs Lisp 解释器交互的模式。
- Paul D. Smith 编写了‘snmp-mode.el’。
- William Sommerfeld 编写了‘scribe.el’，一个编辑 Scribe 文件的模式，以及‘server.el’，一个允许程序把文件发送给正在运行的 Emacs 作业进行编辑的软件包。
- Andre Spiegel 为 Emacs 版本控制软件包做出了许多贡献，特别是使它支持多个后端。
- Michael Staats 编写了‘pc-select.el’，它重新绑定用于选择区域的键，以符合许多其他系统的习惯。
- Richard Stallman 发明了 Emacs。他是 GNU Emacs 的原始作者，并在若干不连续的时期担任过 Emacs 维护者。除 Emacs 核心代码的很大一部分外，他还编写了‘easymenu.el’，一个定义 Emacs 菜单的设施；‘image-

mode.el’，即访问图像文件的支持；‘menu-bar.el’，Emacs 菜单栏支持代码；‘paren.el’，一个使匹配的括号以颜色突出的软件包；还参与编写了 CC 模式的部分内容。

- Sam Steingold 编写了‘midnight.el’，这是一个每逢午夜运行一次命令的软件包。
- Ake Stenhoff 和 Lars Lindberg 编写了‘imenu.el’，这是一个用于浏览由缓冲区内容生成的索引的框架。
- Peter Stephenson 编写了‘vcursor.el’，它实现了一个虚拟光标，你可以用键盘移动它，并用它来复制文本。
- Ken Stevens 编写了‘ispell.el’，这是一个拼写检查器接口。
- Kim F. Storm 对 Emacs 的显示引擎、进程支持和网络支持做了许多改进。他还编写了‘bindat.el’（一个用于编码和解码二进制数据的软件包）、CUA 模式（它让 Emacs 可以模拟标准的 CUA 键绑定）、‘ido.el’（一个用于快速选择缓冲区和文件的软件包）、用于简化小键盘键绑定的‘keypad.el’，以及键盘宏设施‘kmacro.el’。
- Martin Stjernholm 参与编写了 CC Mode，这是用于编辑 C、C++、Objective-C、Java、Pike、CORBA IDL 和 AWK 代码的主模式。
- Steve Strassmann 并没有编写‘spook.el’；就算他写了，他也真的没打算让你以无政府主义的方式使用它。
- Olaf Sylvester 编写了‘bs.el’，这是一个用于操作 Emacs 缓冲区的软件包。
- Tibor Šimko 和 Milan Zamazal 编写了‘slovak.el’，用于支持编辑斯洛伐克语文本。
- João Távora 为‘flymake.el’做了许多改进，这是一个即时语法检查软件包。他还编写了‘eglot.el’，这是一个语言服务器协议（LSP）客户端，在 Emacs 29 中加入。
- Luc Teirlinck 编写了‘help-at-pt.el’，通过键盘提供局部帮助。
- Jean-Philippe Theberge 编写了‘thumbs.el’，这是一个以缩略图方式查看图像文件的软件包。
- Spencer Thomas 编写了最初的‘dabbrev.el’，它提供的命令以光标点附近其他以该词为前缀的词为基础，补全光标点前的部分单词。他还编写了最初的转储（dumping）支持。
- Theodor Thornhill 编写了‘html-ts-mode.el’，这是基于 tree-sitter 库编辑 HTML 文件的主模式。
- Toru Tomabechi 为藏文支持做出了贡献。
- Markus Triska 编写了‘linum.el’，这是一个在左侧边缘显示行号的次模式。
- Tom Tromey 和 Chris Lindblad 编写了‘tcl.el’，这是一个用于编辑 Tcl/Tk 源文件并把 Tcl 解释器作为 Emacs 子进程运行的模式。Tom Tromey 还编写了‘bug-reference.el’（提供指向缺陷报告的可点击链接）以及 Emacs 软件包系统的第一版。
- Eli Tziperman 编写了‘rmail-spam-filter.el’，这是一个用于 RMAIL 的垃圾邮件过滤器。
- Daiki Ueno 编写了‘starttls.el’（对传输层安全协议的支持）、‘sasl-cram.el’与‘sasl-digest.el’（与 Kenichi Okada 合作）以及‘sasl.el’（对简单认证与安全层（SASL）的支持）、用于安全存储属性列表的‘plstore.el’，以及 EasyPG（及其前身 PGG）软件包，用于支持 GnuPG 和 PGP。

- Masanobu Umeda 编写了 GNUS，这是一个功能丰富的 Usenet 新闻阅读器，也是当前 Gnus 软件包的前身。他还编写了 'rmailsort.el'（一个用于对 RMAIL 文件夹中的邮件排序的软件包）、'metamail.el'（Metamail 程序的接口）、'gnus-kill.el'（Gnus 的 Kill File 模式）、'gnus-mh.el'（Gnus 的 mh-e 接口）、'gnus-msg.el'（Gnus 的邮件与发帖接口），以及 'timezone.el'（提供处理时区的函数）。
- Neil W. Van Dyke 编写了 'webjump.el'，这是一个 Web 热表软件包。
- Didier Verna 编写了 'rect.el'，这是一个对文本矩形区域进行操作的函数集。他还为 Gnus 做出了贡献（见前文）。
- Joakim Verona 实现了 ImageMagick 支持。
- Ulrik Vieth 实现了 'meta-mode.el'，用于编辑 MetaFont 代码。
- Geoffrey Voelker 编写了 Windows NT 支持。他还编写了 'dos-w32.el'（MS-DOS 与 MS-Windows 版 Emacs 共用的函数）和 'w32-fns.el'（MS-Windows 专用的支持函数）。
- Johan Vromans 编写了 'forms.el' 及其相关文件，这是一个填写表单的模式。他还编写了 'iso-acc.el'，这是一个提供电气重音键的次模式。
- Colin Walters 编写了 Ibuffer，一个增强的缓冲区菜单。
- Barry Warsaw 编写了 'cc-mode.el'（这是一个编辑 C、C++ 和 Java 代码的模式，基于 Dave Detlefs、Stewart Clamen 和 Richard Stallman 早先的工作）、'elp.el'（Emacs Lisp 程序的性能分析器）、'man.el'（阅读 Unix 手册页的模式）、'regi.el'（在 Lisp 程序中提供类似 AWK 的功能）、'reporter.el'（为 Lisp 软件包提供可定制的缺陷报告功能），以及 'supercite.el'（一个引用邮件消息和新闻文章片段的次模式）。
- Christoph Wedler 编写了 'antlr-mode.el'，这是一个用于 ANTLR 语法文件的主模式。
- Morten Welinder 帮助将 Emacs 移植到 MS-DOS，并把字体（face）支持引入了 MS-DOS 版的 Emacs。他还编写了 'desktop.el'（在会话之间保存 Emacs 部分状态的设施）、'timer.el'（在指定时间、指定频率或 Emacs 空闲时运行命令的 Emacs 设施及其 C 层支持代码）、'pc-win.el'（MS-DOS 的"窗口系统"支持）、'internal.el'（MS-DOS 版 Emacs 的"内部终端"模拟器）、'arc-mode.el'（编辑压缩归档文件的模式）、's-region.el'（使用 shift 键和移动命令设置区域的命令），以及 'dos-fns.el'（在 MS-DOS 下使用的函数）。
- Joe Wells 编写了 'apropos.el' 的最初版本（见前文）、'resume.el'（用于在恢复被挂起的 Emacs 作业后处理命令行参数），以及 'mail-extr.el'（一个从邮件头中提取姓名和地址的软件包，Jamie Zawinski 亦有贡献）。
- Rodney Whitby 和 Reto Zimmermann 编写了 'vhdl-mode.el'，这是一个用于编辑 VHDL 源代码的主模式。
- Sean Whitton 从 Emacs 31.1 起担任 Emacs 的（联合）维护者，并为 VC 编写了许多扩展和新功能。
- John Wiegley 从 Emacs 25 到 29.1 担任 Emacs 的（联合）维护者。他编写了 'align.el'（一组按照基于正则表达式的规则对齐文本的命令）、用于快速切换缓冲区的 'isearchb.el'、'timeclock.el'（一个记录项目上所花时间的软件包）、巴哈伊历法支持、'pcomplete.el'（一个可编程的补全设施）、'remember.el'（一个记下待办

事项的模式）、‘eudcb-mab.el’（Emacs 统一目录客户端的地址簿后端），以及‘eshell’（一个完全用 Emacs Lisp 实现的命令 shell）。他还为 Org 模式做出了贡献（见前文）。

- Mike Williams 编写了‘thingatpt.el’，这是一个用于查找光标点处的“事物”（词、行、S 表达式）的函数库。
- Roland Winkler 编写了‘proced.el’，这是一个系统进程编辑器。
- Bill Wohler 编写了 MH-E，即 Emacs 对 MH 邮件系统的接口，其中利用了 James R. Larus 早先的工作。Satyaki Das、Peter S. Galbraith、Stephen Gildea 和 Jeffrey C. Honig 也编写了 MH-E 的各个组件。
- Dale R. Worley 编写了‘emerge.el’，这是一个以交互方式合并文件的两个版本的软件包。
- Francis J. Wright 编写了‘woman.el’，这是一个无需‘man’命令即可浏览手册页的软件包。
- Po Lu 将 Emacs 移植到 Android，并编写了‘pixel-scroll-precision-mode’。他还为 Emacs 添加了 XInput2 支持，在‘touch-screen.el’中实现了触摸板命令，并扩展了拖放支持。
- Masatake Yamato 编写了‘ld-script.el’（一个用于 GNU 链接器脚本的编辑模式），并在 CC 模式中贡献了子词处理和风格推测功能。
- Eshel Yaron 编写了‘elisp-scope.el’，这是对 Emacs Lisp 符号进行语义分析的一种实现。
- Jonathan Yavner 编写了‘testcover.el’（一个跟踪 Emacs Lisp 代码测试状态的软件包）、用于判断某个 Lisp 形式是否安全的‘unsafep.el’，以及 SES 电子表格软件包。
- Ryan Yeske 编写了‘rcirc.el’，这是一个简单的 Internet Relay Chat 客户端。
- Ilya Zakharevich 和 Bob Olson 编写了‘cperl-mode.el’，这是一个用于编辑 Perl 代码的主模式。Ilya Zakharevich 还编写了‘tmm.el’，这是一个在文本模式终端上访问 Emacs 菜单栏的模式。
- Milan Zamazal 编写了‘czech.el’（对编辑捷克语文本的支持）、‘glasses.el’（一个便于阅读使用难辨标识符名的源代码的软件包），以及‘tildify.el’（向文本、TeX 和 SGML/HTML 文件添加硬空格的命令）。
- Victor Zandy 编写了‘zone.el’，这是一个为喜欢在 Emacs 前发呆走神的人准备的软件包。
- Eli Zaretskii 从 Emacs 25 起担任 Emacs 的（联合）维护者。他让许多标准的 Emacs 功能能在 MS-DOS 和 Microsoft Windows 上工作。他还编写了‘tty-colors.el’（实现 X 颜色到 tty 颜色的透明映射）和‘rxvt.el’。他实现了对双向文本、文本模式终端菜单以及内置行号显示的支持。
- Jamie Zawinski 编写了字体（face）和 X 选择方面的大量支持代码。他与 Hallvard Furuseth 一起编写了从 Emacs 19 起使用的优化字节编译器。他还编写了‘mailabbrev.el’（一个提供邮件别名自动展开的软件包）和‘tar-mode.el’（为 tar 文件提供简单的查看和编辑命令）。
- Andrew Zhilin 设计了 Emacs 22 的图标。
- Shenghuo Zhu 编写了‘binhex.el’（一个读写 binhex 文件的软件包）、‘mm-partial.el’（对 MIME 消息的 message/partial 支持）、‘rfc1843.el’（一个 HZ 解码软件包）、‘uudecode.el’（一个用于 uuencode 数据的 Emacs Lisp 解码器），以及‘webmail.el’（一个 Web 邮件接口）。他还编写了 Gnus 的另外几个组件。

- Ian T. Zimmerman 编写了‘gametree.el’。
- Reto Zimmermann 编写了‘vera-mode.el’。
- Neal Ziring 和 Felix S. T. Wu 编写了‘vi.el’，这是一个 VI 文本编辑器的模拟。
- Ted Zlatanov（除了对 Gnus 新闻阅读器的贡献之外）编写了 GnuTLS 库的接口（用于安全的网络连接），以及 URL 库的 futures 设施。
- Detlev Zundel 编写了‘re-builder.el’，这是一个带有可视化反馈的正则表达式构建软件包。

附录 J 常用配置速查

本附录汇总手册各章中与日常配置最相关的设置，全部取自原手册正文，可直接放入初始化文件。

初始化文件

Emacs 依次查找以下位置（见第 51 章）：

- `~/.emacs`
- `~/.emacs.d/init.el`
- `$XDG_CONFIG_HOME/emacs/init.el`（未设置时为 `~/.config/emacs/init.el`）

`early-init.el` 与 `init.el` 同目录，在图形界面初始化之前加载，适合设置字体、框架参数和包管理器。

最小可用模板

```
;;; 编码与界面
(prefer-coding-system 'utf-8)
(set-language-environment "UTF-8")
(setq inhibit-startup-screen t)
(tool-bar-mode -1)
(menu-bar-mode -1)
(scroll-bar-mode -1)
(column-number-mode 1)
(show-paren-mode 1)
(global-display-line-numbers-mode 1)   ; 如不需要行号可删除

;;; 编辑习惯
(setq-default indent-tabs-mode nil)    ; 用空格缩进
(setq-default tab-width 4)
(setq-default fill-column 80)
(delete-selection-mode 1)              ; 输入时替换选区

;;; 备份与自动保存
(setq backup-directory-alist
      `(("." . ,(expand-file-name "backups" user-emacs-directory))))
(setq auto-save-file-name-transforms
      `((".*" ,(expand-file-name "auto-save/" user-emacs-directory) t)))
(make-directory (expand-file-name "auto-save/" user-emacs-directory) t)

;;; 会话与历史
(savehist-mode 1)
(save-place-mode 1)
(recentf-mode 1)
(setq recentf-max-saved-items 200)
(global-auto-revert-mode 1)            ; 文件在外部被改动时自动重读

;;; 包管理（见第 50 章）
(require 'package)
(setq package-archives
      '(("gnu"    . "https://elpa.gnu.org/packages/")
```

```
        ("nongnu" . "https://elpa.nongnu.org/nongnu/")
        ("melpa"  . "https://melpa.org/packages/")))
(package-initialize)
;; M-x package-refresh-contents 后 M-x package-install 安装软件包
```

常用按键绑定

```
;; 全局绑定（第 51 章、第 3 章）
(global-set-key (kbd "C-c f") #'find-file)
(global-set-key (kbd "C-c g") #'magit-status)     ; 需先安装 magit

;; 只在某个键位图里绑定
(define-key global-map (kbd "C-c b") #'switch-to-buffer)

;; 为某个模式绑定（当地键位图）
(with-eval-after-load 'org
  (define-key org-mode-map (kbd "C-c l") #'org-store-link))
```

钩子

```
;; 进入某类模式时执行（第 25 章）
(add-hook 'prog-mode-hook #'display-line-numbers-mode)
(add-hook 'prog-mode-hook (lambda () (setq show-trailing-whitespace t)))

;; 打开文件前处理
(add-hook 'find-file-hook #'recentf-track-opened-file)
```

钩子函数名通常以 `-hook` 结尾；`-mode-hook` 在模式启用时运行。用 `C-h v 钩子名` 查看说明。

常用变量速查

变量	作用	设置示例
`inhibit-startup-screen`	不显示启动画面	`(setq inhibit-startup-screen t)`
`indent-tabs-mode`	缩进是否用 Tab	`(setq-default indent-tabs-mode nil)`
`tab-width`	Tab 显示宽度	`(setq-default tab-width 4)`
`fill-column`	自动换行列宽	`(setq-default fill-column 80)`
`make-backup-files`	是否生成备份文件	`(setq make-backup-files nil)`
`backup-directory-alist`	备份文件存放目录	见上例
`auto-save-default`	是否自动保存	`(setq auto-save-default nil)`
`create-lockfiles`	是否生成锁文件	`(setq create-lockfiles nil)`
`require-final-newline`	保存时补末尾换行	`(setq require-final-newline t)`
`sentence-end-double-space`	句子间是否要求两个空格	`(setq sentence-end-double-space nil)`

变量	作用	设置示例
`scroll-conservatively`	滚动时保留上下文	`(setq scroll-conservatively 101)`
`show-trailing-whitespace`	显示行尾空格	`(setq show-trailing-whitespace t)`
`default-input-method`	默认输入法	`(setq default-input-method "chinese-py")`
`user-full-name` / `user-mail-address`	姓名与邮箱	`(setq user-full-name "…")`

主题与字体

```
;; 字体（第 23 章）
(set-face-attribute 'default nil :family "SF Mono" :height 140)

;; 关闭界面元素
(blink-cursor-mode -1)

;; 主题（内置 modus 系列）
(load-theme 'modus-vivendi t)
```

启动与调试

```
emacs -Q                  # 不加载任何初始化文件，排查配置问题
emacs --debug-init        # 初始化文件出错时进入调试器
emacs -nw                 # 在终端中运行
```

- `M-x profiler-start`、操作、`M-x profiler-report`：分析启动或操作耗时。
- 出问题时的排查顺序见第 53 章《故障处理》。